PLACEMENT EDITION · 2027

Network Engineer Placements 2027

From first principles to the offer: networks, routing, systems, code and interviews for Cisco, Qualcomm, Google, Microsoft, Amazon, Juniper, Arista, Nokia and Broadcom.

Working edition · built 29 October 2026 · information correct as of this date

CONTENTS

Contents

0 How to Use This Book
1 Data Structures and Algorithms
2 Programming: C, Python, C++
3 Computer Networks Fundamentals
4 Routing and Switching, CCNA to CCNP ENCOR
5 Data Centre and Cloud Networking
6 Wireless and Cellular Networks
7 Operating Systems and Socket Programming
8 Linux and Network Tools
9 Network Security
10 Network Automation and Programmability
11 Troubleshooting Methodology
12 Supporting Subjects
II Cheat Codes

PART 0 · THE PLANS

0

How to Use This Book

A single path from first principles to the offer, with one home for every idea.

0.1 What this book is —

0.2 Tracks and tags —

0.3 The 8-week plan —

0.1 What this book is

This book prepares a beginner for network-engineer and networking-software roles at Cisco, Qualcomm, Google, Microsoft, Amazon, Juniper, Arista, Nokia and Broadcom. It is designed to be the only resource one needs. Every concept is explained from first principles, every interview question is solved in full, and every lab prints the output one should expect, so that a lab can also be followed on paper.

Nothing is printed twice. Each concept, figure and solved question has exactly one home, and every other mention is a cross-reference that prints its page number. Thus, the book stays comprehensive without growing long, and a reader who meets a term out of order can always find its single, complete treatment.

> BIG IDEA
>
> Read the Must sections in order, solve the Problem Set at the end of each pillar, and revise from the two-page Revision Spread; the rest of the book is depth you add when time allows.

Each pillar follows the same order: (i) teaching sections, each opening with its Big Idea; (ii) a Problem Set, in which every reported interview question for the pillar is solved; and (iii) a Revision Spread, whose boxed top half is the version to re-read in the fifteen minutes before an interview.

0.2 Tracks, tags and the six elements

Track — the family of roles one is preparing for. Most students prepare for two.

Roles fall into three tracks, which differ in what the interviews emphasise. The **Software** track covers networking software engineers at Cisco, Juniper, Arista, Nokia and Broadcom, and software engineers on networking teams at Google and Microsoft. The **Infrastructure** track covers network engineers, Amazon network development engineers and Cisco consulting engineers. The **Wireless** track covers WLAN and modem software roles, principally at Qualcomm.

Pillar	Software	Infrastructure	Wireless
1 DSA & coding	●●●	●●	●●
2 C, Python, C++	●●●	●●	●●●
3 Computer Networks	●●●	●●●	●●●
4 Routing & Switching	●	●●●	●
5 Data Centre & Cloud	●●	●●●	●
6 Wireless & Cellular	—	●	●●●
7 OS & Sockets	●●●	●	●●●
8 Linux & Tools	●●	●●●	●●
9 Security	●	●●	●
10 Automation	●●	●●	—
11 Troubleshooting	●	●●●	●
12 Supporting subjects	●●●	●●●	●●●

Every section heading carries a tag such as `MUST · ■□□`. The word states priority: **Must** sections form the 8-week plan, **Should** sections are added by the 3-month plan, and **Optional** sections complete the 6-month plan. The squares state difficulty, from foundation (■□□) to the level of the hardest interviews (■■■). Note that the tags guide the order of reading; they do not change the content, which is written to the same standard throughout.

The text uses only six recurring elements. A **Big Idea** states what a section exists to teach. A **Worked Example** solves a problem step by step. A **Trap** names a common misconception and why it is wrong. An **Interview Lens** states how the topic is examined and which companies report asking it. A **Lab** gives commands and the output to expect. **Check Yourself** questions close a section, with answers in the footnotes. Definitions, analogies and mnemonics sit in the outer margin.

0.3 The 8-week plan

The plan assumes about 4 hours on weekdays and 6 hours on Saturdays. Each weekday has the same shape: 60 minutes of coding practice, 120 minutes of the day's Must sections, 40 minutes on a lab or a worked example redone without the text, and 20 minutes of recall. Sunday holds the weekly checkpoint and a buffer, because a plan with no slack fails at the first missed day.

Week	Main topic	Daily coding strand	Sunday
1	Layers, Ethernet, ARP, subnetting; C pointers and structs	Complexity, arrays, strings, hashing, bits	Checkpoint 1
2	TCP and UDP, congestion control, DNS, DHCP, "type google.com"; C traps	Linked lists, stacks, queues, sliding window	Checkpoint 2
3	VLANs, STP, the routing table, OSPF, BGP, ACLs and NAT	Trees, tries, heaps, BFS and DFS	Checkpoint 3, mock OA
4	Processes, threads, synchronisation, memory; sockets and epoll; Linux tools	Shortest paths, union-find, backtracking	Checkpoint 4
5	Spine-leaf, ECMP, VXLAN, cloud networks; TLS, IPsec, firewalls	Dynamic programming, greedy	Checkpoint 5
6	Track variant: troubleshooting and automation, or design coding, or Wi-Fi	LRU cache, rate limiter	Checkpoint 6, mock OA
7	Troubleshooting rounds; aptitude; OOP and DBMS	Timed mixed sets	Checkpoint 7, mock OA
8	Company playbook; behavioural stories; Revision Spreads	The top-30 list	Checkpoint 8, mock OA

> **TRAP**
>
> *Claim:* one should finish all the theory before starting coding practice. *Why it is wrong:* coding fluency is built by short, repeated practice spread over weeks, and it cannot be compressed

> into the final fortnight. *Correct statement:* code every day from the first day, in parallel with the networking chapters.

PART I · PILLAR 1

1

Data Structures and Algorithms

Coding rounds for network engineers: from a blank screen to a longest-prefix-match table, an LRU cache and a token bucket.

1.0 How network-company coding rounds work	MUST
1.1 Complexity and the problem-solving routine	MUST
1.2 Arrays: two pointers, windows, prefix sums	MUST
1.3 Hashing: maps and sets	MUST
1.4 Strings and parsing network data	MUST
1.5 Bit manipulation and subnet code	MUST
1.6 Linked lists in C	MUST
1.7 Stacks, queues, ring buffers, deques	MUST
1.8 Trees and binary search trees	MUST
1.9 Tries and longest-prefix match	MUST
1.10 Heaps and priority queues	SHOULD
1.11 Graphs: networks are graphs	MUST
1.12 Recursion and backtracking	SHOULD
1.13 Sorting, binary search, intervals	MUST
1.14 Dynamic programming essentials	SHOULD
1.15 Design-style coding: caches and rate limiters	MUST
1.16 Finite-state machines	SHOULD
1.17 Testing your own code	MUST

1.0 How network-company coding rounds work MUST · ■□□

Candidate reports describe the coding rounds of networking companies as narrower than those of general software firms. The problems are mostly easy or medium, they favour linked lists, bits, strings and trees over contest techniques, and several companies expect the solution in C. Hence, breadth and clean code earn more than obscure algorithms. The table summarises what candidates report; it is indicative, not a guarantee, and every drive should be checked against its own invitation.

Company	Reported shape of the coding part	Topics most often reported
Cisco	online test, about 90 min: 1–2 coding problems and 30–40 MCQs; C, Python or Java	graphs (snake and ladder), strings, longest substring, bits, pointer output
Juniper	online test, about 90 min: 1–3 coding problems and about 35 MCQs	merge sorted arrays, multiply without `*`, list cycles, bits, state machines
Qualcomm	written test then 2–3 technical rounds	bit counting and flipping, struct padding, `volatile`
Arista	online test in C or C++, then a shared-editor onsite	linked lists from scratch, BST operations, program memory layout
Google	phone screen with 1–2 problems	tries and prefix match on IPs, log parsing, frequency sorting
Microsoft	standard software loop	LRU cache with TTL and thread safety, rate limiters
Amazon (network development)	online test of one easy and one medium problem, plus Python in the loop	strings, trees, graphs, LRU variants
Nokia	aptitude test and two simple programs	list loops and intersection, depth-first search with a stack
Broadcom	phone and onsite rounds	bits, list reversal, stacks, queues, state machines

Three consequences follow. First, a test with partial marks per test case rewards a correct brute force submitted early; one should read all problems, bank the easiest, then return. Secondly, an onsite round is judged on the visible process as much as on the code, which is the reason for the routine of §1.1 (p. 9). Thirdly, the language is a decision to be made before the clock starts.

If …	Write in …	Because
the role mentions embedded, data plane, drivers or the company is Qualcomm, Broadcom or Arista	C	pointer questions are the point; the STL is unavailable or disapproved
the role is network development or automation	Python	brevity; built-in dictionaries, `heapq`, `deque`
the test allows it and you know the STL	C++	brevity close to Python, with no hidden costs

This chapter uses C++ for general algorithms and C for linked lists, bit tricks and networking problems, as the interviews do. One language is used for each solution; where Python differs materially, one sentence says so.

WORKED EXAMPLE

A 90-minute test with about 40 multiple-choice questions and two coding problems. The format is typical of the reports for Cisco and Juniper; the aim is to spend the time where marks are cheapest.

1. Minutes 0–3: read every question title and the marking scheme. Note whether wrong answers are penalised and whether sections lock after submission.
2. Minutes 3–33: answer the multiple-choice questions in two passes. In the first, answer anything that takes under 45 seconds and flag the rest; in the second, return to the flagged ones in order of confidence. Never leave a blank if there is no penalty.
3. Minutes 33–38: read both coding problems and their constraints, and choose the easier one first.
4. Minutes 38–63: solve the first problem. Submit a correct brute force early, because most platforms award marks per passed test case, and then optimise.
5. Minutes 63–83: solve the second problem in the same way.
6. Minutes 83–90: re-run the edge cases of §1.17 (p. 67) on both, then submit.

Block	Minutes	Rule
read and plan	3	titles, marking, negative marking
multiple choice	30	45 seconds each, two passes
coding problem 1	25	brute force first
coding problem 2	20	partial credit counts
buffer	12	edge cases, resubmission

Result: a plan that banks the certain marks first. Rehearse it on a timer before the real test.

INTERVIEW LENS

CISCO · JUNIPER · ARISTA

Interviewers ask for a complete, compilable function, not pseudocode. A strong answer states the signature and the edge cases first, writes the code in small named steps, and ends by tracing one example aloud. Expect “now do it without extra memory” or “now in C” as the follow-up.

1.1 Complexity and the problem-solving routine MUST · ■□□

BIG IDEA

The input size in the problem statement tells us which algorithms can finish in time, because a judge performs about 10^8 simple operations per second.

We count growth, not seconds. An algorithm taking $c \cdot n$ steps and one taking $c' \cdot n + 50$ steps are both O(n), since for large n the constants do not change the shape of the curve. The three loops below show the three cases that cover most code.

Big-O — an upper bound on how the work grows with the input size n, ignoring constants and lower-order terms.

```
for (i = 0; i < n; i++) sum += a[i];  /* O(n): one pass          */
for (i = 0; i < n; i++)
    for (j = i + 1; j < n; j++) pairs++;  /* O(n^2): n(n-1)/2 steps  */
while (n) n >>= 1;  /* O(log n): halves n      */
```

Consider a switch. Flooding a frame to every port costs O(p) for p ports, whereas finding the egress port in the MAC table costs O(1) on average because the table is hashed. Thus, the data structure, not the hardware speed, sets the cost per frame.

Reading the constraints. A test limits n, and the limit selects the algorithm. At $n = 10^5$ an O(n^2) method needs 10^{10} steps and fails, whereas O(n log n) needs about 1.7×10^6 and passes. The full table from $n \le 12$ up to $n \le 10^9$ is in the Revision Spread (p. 101); the two rules to remember are that 10^5 allows O(n log n) and 10^3–10^4 allows O(n^2). Python is slower by a factor of 10 to 50, so one steps down a row.

Space and the call stack. A recursive call stores a frame holding its arguments, locals and return address. Hence, a recursion of depth d uses O(d) memory even when it allocates nothing, and a depth of 10^6 overflows a typical 8 MB stack. Space counts the extra memory only; the input and the required output are excluded, so “O(1) extra” permits a few variables.

Amortised cost. A dynamic array doubles its capacity when full. A push is normally one write, but a push into a full array copies every element. Sixteen pushes starting from capacity one copy 1 + 2 + 4 + 8 = 15 elements in total, fewer than the number of pushes. In general the copies sum to less than 2n, so the cost per push averages to O(1); we call this an *amortised* bound. It holds for the whole sequence, not for each push.

Push number	1	2	3	5	9	17
Capacity after	1	2	4	8	16	32
Elements copied	0	1	2	4	8	16

Counting operations in practice

The table gives the number of steps for $n = 10^5$, and the time they take at 10^8 steps per second. It shows why the ladder of growth rates, and not the constant factors, decides whether a solution passes.

Growth	Steps for $n = 10^5$	Time at 10^8 steps/s	Typical source
O(1)	1	instant	array index, hash lookup
O(log n)	17	instant	binary search, heap operation
O(n)	10^5	1 ms	one pass
O(n log n)	1.7×10^6	17 ms	sorting
O(n^2)	10^{10}	100 s	all pairs
O(2^n)	more than the atoms in the universe	never	all subsets

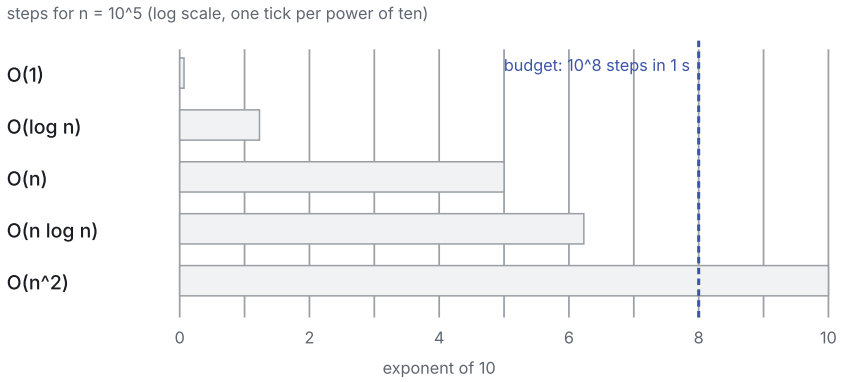


Figure 1.1 Growth rates at n = 10^5. The accent line is the budget of about 10^8 simple steps per second; quadratic work lies two decades beyond it.

Recursion. To find the cost of a recursive function, write the recurrence and solve it. A function that makes two calls on halves and does a linear amount of work, `T(n) = 2T(n/2) + O(n)`, as in merge sort, costs O(*n* log *n*): there are log *n* levels and each level does O(*n*) work in total. A function that halves and does constant work, `T(n) = T(n/2) + O(1)`, as in binary search, costs O(log *n*). A function that makes two calls on `n - 1`, `T(n) = 2T(n-1) + O(1)`, costs $O(2^n)$, which is why the naive Fibonacci function is unusable beyond $n \approx 40$. Remembering the results of earlier calls, termed *memoisation*, removes the repeated work and reduces it to O(*n*):

```
long long fib(int n, vector<long long>& memo) {   // memo sized n + 1, zeros
    if (n < 2) return n;
    if (memo[n]) return memo[n];                    // already computed
    return memo[n] = fib(n - 1, memo) + fib(n - 2, memo);
}
```

The same idea, applied to a table filled in order, is dynamic programming (§1.14 (p. 58)). Further, the *best*, *average* and *worst* cases may differ: a hash lookup is O(1) on average but O(*n*) at worst, and quicksort is O(*n* log *n*) on average but $O(n^2)$ at worst. In an interview one states which case is meant.

The six-step routine

A visible process earns marks even when the code is unfinished. The steps are: (i) *understand*, by restating the problem and asking about size, duplicates, negatives and empty input; (ii) *examples*, by working one by hand; (iii) *brute force*, stated with its complexity; (iv) *optimise*, by finding the repeated work; (v) *code*; and (vi) *test*, by tracing the example and the edge cases of §1.17 (p. 67).

> WORKED EXAMPLE
>
> **Two Sum, through all six steps.** Given an array of integers and a target, return the indices of two elements that add to the target. Exactly one answer exists and an element may not be used twice.
>
> 1. *Understand.* Ask whether the array is sorted (no), whether it may hold negatives (yes), and what to return when no pair exists (an empty result).
> 2. *Example.* `[2, 7, 11, 15]`, target 9 gives indices `{0, 1}`.
> 3. *Brute force.* Test every pair: $O(n^2)$ time, O(1) space.
> 4. *Optimise.* The inner loop searches for `target - a[i]`. A hash map from value to index answers that search in O(1), so one pass suffices.

5. *Code.* Below.
6. *Test.* Trace the example, then `[3, 3]` with target 6 (a duplicate value) and a single element.

```
vector<int> twoSum(const vector<int>& a, int target) {
    unordered_map<int, int> seen;                // value -> index
    for (int i = 0; i < (int)a.size(); ++i) {
        auto it = seen.find(target - a[i]);
        if (it != seen.end()) return {it->second, i};
        seen[a[i]] = i;  // insert after the lookup
    }
    return {};
}
```

i	a[i]	need	in map?	map after
0	2	7	no	{2→0}
1	7	2	yes, index 0	return {0, 1}

Result: O(*n*) time and O(*n*) space. Inserting after the lookup is what prevents an element pairing with itself.

CHECK YOURSELF

1. What is the complexity of `for (i = 1; i < n; i *= 2) work();`? [1]
2. A function recurses to depth *n* and allocates nothing. What is its space complexity? [2]
3. Why is appending to a dynamic array O(1) amortised although one push can copy *n* elements? [3]

1.2 Arrays: two pointers, windows and prefix sums MUST · ■□□

BIG IDEA

Most array problems become linear when we maintain two indices or one running total, because each element is then inspected a constant number of times.

An array stores its elements contiguously, so the address of `a[i]` is `base + i * sizeof(T)` and indexing costs O(1). For the same reason, inserting or deleting in the middle costs O(*n*), since the later elements must move. A two-dimensional array in C is stored row by row, so `m[r][c]` sits at offset `r * cols + c`; scanning along a row is therefore faster than scanning down a column, because it follows memory order.

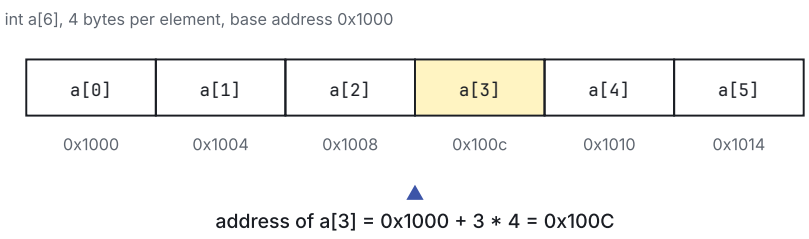


Figure 1.2 An array is one contiguous block. The address of a[i] is base + i times the element size, so indexing is one multiplication and one addition.

Two pointers, opposite ends

Idea. On a sorted array, one pointer at each end moves inward, and each step discards an element that cannot be part of any answer. *Recognise:* “sorted”, “pair with sum”, “palindrome”, “container”. *Complexity:* O(*n*) time, O(1) space.

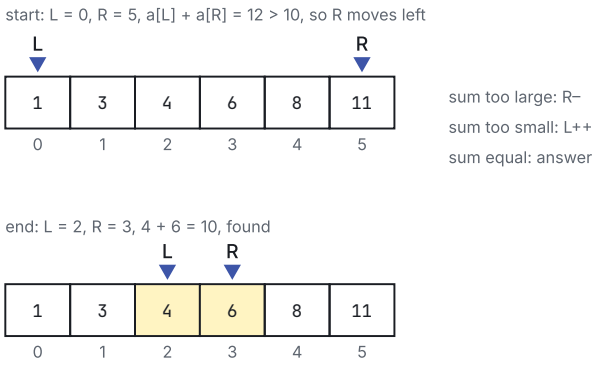


Figure 1.3 Opposite-end pointers on a sorted array with target 10. Each step discards one element that cannot belong to any answer, so at most n steps occur.

```
pair<int,int> pairSum(const vector<int>& a, long long t) {   // a sorted
    int l = 0, r = (int)a.size() - 1;
    while (l < r) {
        long long s = (long long)a[l] + a[r];
        if (s == t) return {l, r};
        if (s < t) ++l; else --r;  // too small: need a larger left value
    }
    return {-1, -1};
}
```

The argument for correctness is that if `a[l] + a[r]` exceeds *t*, then `a[r]` plus any element at or right of `l` also exceeds *t*, so `r` can move without losing an answer. The same argument for `s < t` justifies moving `l`. For `[1, 3, 4, 6, 8, 11]` and *t* = 10 the pointers visit (0,5) → (0,4) → (1,4) → (1,3) → (2,3), and the last pair sums to 10.

Two pointers, read and write

Idea. A fast index `r` reads every element; a slow index `w` marks the end of the answer built so far. *Recognise:* “in place”, “remove duplicates”, “move zeroes”, “partition”. *Complexity:* O(*n*) time, O(1) space.

```
int dedupe(vector<int>& a) {  // a sorted; returns the new length
    int w = 0;                          // a[0..w) holds the answer so far
    for (int r = 0; r < (int)a.size(); ++r)
        if (w == 0 || a[r] != a[w - 1]) a[w++] = a[r];
    return w;
}
```

r	a[r]	compared with	action	answer so far
0	1	none	write	1
1	1	a[w-1] = 1	skip	1

2	2	1	write	1 2
3	2	2	skip	1 2
4	3	2	write	1 2 3

> **TRAP**
>
> *Claim:* compare `a[r]` with `a[r-1]` to detect a duplicate. *Why it is wrong:* once the writer has overwritten earlier cells, `a[r-1]` may no longer be the previous kept value. *Correct statement:* compare with `a[w-1]`, the last element of the answer, and never write before the cell has been read.

Three-way partition (the Dutch national flag)

Idea. To sort an array that holds only three distinct values in one pass, keep three regions: values equal to the first class at the left, the second class in the middle, and the third class at the right, with an unexamined region between the middle and the right. The same partition step sorts packets into three priority classes in place, and it is the core of quicksort for inputs with many equal keys. *Recognise:* "sort an array of 0, 1 and 2", "partition around a pivot", "segregate". *Complexity:* O(*n*) time, O(1) space, one pass.

```
void sortColors(vector<int>& a) {        // values 0, 1, 2 only
    int lo = 0, mid = 0, hi = (int)a.size() - 1;
    while (mid <= hi) {                  // [mid, hi] is unexamined
        if (a[mid] == 0)      swap(a[lo++], a[mid++]);
        else if (a[mid] == 1) ++mid;
        else                  swap(a[mid], a[hi--]);   // do not advance mid
    }
}
```

The invariant is: `a[0..lo)` holds zeros, `a[lo..mid)` holds ones, `a[hi+1..n)` holds twos. A value swapped in from the right has not been examined, so `mid` stays; a value swapped in from the left is a one and is already in its region.

lo	mid	hi	a[mid]	action	array after
0	0	5	2	swap with a[hi]	0 0 2 1 1 2
0	0	4	0	lo++, mid++	0 0 2 1 1 2
1	1	4	0	lo++, mid++	0 0 2 1 1 2
2	2	4	2	swap with a[hi]	0 0 1 1 2 2
2	2	3	1	mid++	0 0 1 1 2 2
2	3	3	1	mid++, loop ends	0 0 1 1 2 2

Starting array `2 0 2 1 1 0`; result `0 0 1 1 2 2`. The classic bug is advancing `mid` after swapping with `hi`, which skips an unexamined value.

Sliding window, fixed size

Idea. Instead of re-summing *k* elements for each position, add the entering element and subtract the leaving one. *Recognise:* "subarray of size *k*", "every window of length *k*". *Complexity:* O(*n*) time, O(1) space.

```cpp
long long maxSumK(const vector<int>& a, int k) {  // requires 1 <= k <= n
    long long cur = 0, best = LLONG_MIN;
    for (int i = 0; i < (int)a.size(); ++i) {
        cur += a[i];
        if (i >= k) cur -= a[i - k];                // element leaving
        if (i >= k - 1) best = max(best, cur);      // window is full
    }
    return best;
}
```

The classic bug is the off-by-one at eviction: the element leaving when index `i` enters is `a[i-k]`, not `a[i-k+1]`. For `[2, 1, 5, 1, 3, 2]` with $k = 3$ the window sums are 8, 7, 9, 6, so the answer is 9.

Sliding window, variable size

Idea. The right edge advances every step; the left edge advances only while the window violates a condition. Each index enters and leaves once, so the work is O(n) although two loops are nested. *Recognise:* "longest or shortest contiguous subarray or substring such that …". *Complexity:* O(n) time, O(*alphabet*) space.

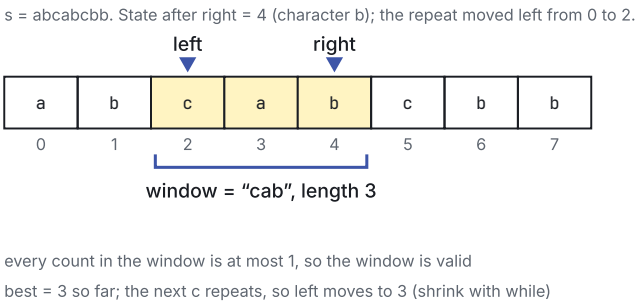


Figure 1.4 Variable sliding window. The right edge always advances; the left edge advances only to restore the invariant.

```cpp
int lengthOfLongest(const string& s) {  // longest run with no repeats
    int cnt[256] = {0}, best = 0, left = 0;
    for (int right = 0; right < (int)s.size(); ++right) {
        ++cnt[(unsigned char)s[right]];
        while (cnt[(unsigned char)s[right]] > 1)  // shrink: while, not if
            --cnt[(unsigned char)s[left++]];
        best = max(best, right - left + 1);
    }
    return best;
}
```

right	char	window after shrinking	length	best
0	a	a	1	1
1	b	ab	2	2
2	c	abc	3	3
3	a	bca	3	3
4	b	cab	3	3
5	c	abc	3	3

6	b	cb	2	3
7	b	b	1	3

At `right = 6` the window `abc` meets a second `b`; the loop removes `a`, then `b`, leaving `cb`. A single `if` would remove only `a` and leave the invalid window `bcb`. This is the standard bug, and it is the reason the template uses `while`.

Prefix sums and the hash map

Idea. Let `pre[i]` be the sum of the first *i* elements. The sum of `a[j..i)` is `pre[i] - pre[j]`, so "a subarray with sum *k* ends here" means "an earlier prefix equals `pre[i] - k`". A map from prefix value to occurrence count answers this in O(1). *Recognise:* "subarray sum equals *k*", negatives present (which defeats a sliding window), "divisible by *k*". *Complexity:* O(*n*) time and space.

```
int subarraySum(const vector<int>& a, int k) {
    unordered_map<long long, int> seen{{0, 1}};  // the empty prefix, seeded
    long long pre = 0; int count = 0;
    for (int x : a) {
        pre += x;
        auto it = seen.find(pre - k);
        if (it != seen.end()) count += it->second;
        ++seen[pre];
    }
    return count;
}
```

For `[1, 2, 3, -2, 2]` and *k* = 3, the prefixes are 1, 3, 6, 4, 6. At prefix 3 we find the seed 0 (subarray `[1,2]`); at 6 we find 3 (`[3]`); at 4 we find 1 (`[2,3,-2]`); at the second 6 we find 3 again (`[3,-2,2]`). The count is 4. Omitting the seed `{0, 1}` loses every subarray that starts at index 0, which is the classic bug.

> CHECK YOURSELF
>
> 1. In `maxSumK`, why is `best` initialised to `LLONG_MIN` rather than 0? [4]
> 2. Why does a sliding window fail on `[1, -1, 5]` with the condition "sum at most 5"? [5]

1.3 Hashing: maps and sets MUST · ■□□

> BIG IDEA
>
> A hash table turns "have I seen this?" from a scan into a single computed lookup, which is why so many O(n^2) solutions collapse to O(n).

1. O(log *n*), because *i* doubles each time.↩
2. O(*n*), from the call stack.↩
3. Copies occur only at doublings and total fewer than 2*n* over *n* pushes.↩
4. Every window sum may be negative, and 0 would then be returned wrongly.↩
5. The window sum is not monotone in its length when negatives exist, so shrinking cannot restore validity reliably; prefix sums are used instead.↩

Hash function — a function mapping a key to an integer; equal keys must give equal values, and different keys should rarely collide.

A hash table stores each key in a bucket chosen by `hash(key) mod m`. Two keys sharing a bucket collide, and the usual remedy, *chaining*, keeps a short list in each bucket. With a good hash and a load factor (entries per bucket) kept below about one, a lookup inspects a constant expected number of entries. Thus, insert, find and erase are O(1) on average, but O(n) in the worst case when many keys collide. The ordered `map` in C++ is a balanced tree instead: O(log n) per operation, but it keeps keys sorted.

Need	Structure	Cost
membership, no order	`unordered_set`	O(1) average
key → value, no order	`unordered_map`	O(1) average
sorted iteration, nearest key	`set`, `map`	O(log n)
counts of small integers or letters	array indexed by value	O(1), smallest constant

A switch's MAC table and a router's flow table are the networking forms of this structure: the key is the destination MAC or the 5-tuple, and the value is the egress port or the action. Further, when values are bounded by a small range, a plain array replaces the hash and is faster.

Recognise: "have we seen", "count occurrences", "pair that …", "group by", "first non-repeating", "duplicate". *Typical templates:* a set for membership, a map for counts, and a sorted or canonical form of an item as the key (for anagrams, the sorted letters).

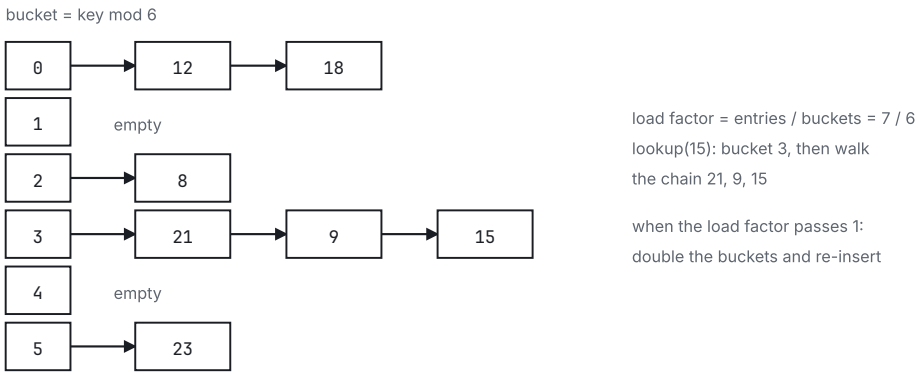


Figure 1.5 A hash table with chaining. Keys that collide share a bucket's list; the expected chain length equals the load factor.

Two collision strategies exist. *Chaining* keeps a list per bucket, as drawn; deletion is easy and the table degrades gracefully when full. *Open addressing* stores entries in the array itself and probes to the next free cell (linear probing) after a collision; it is more compact and faster to scan, but it needs a load factor well below one, and deletion needs a tombstone marker. When the load factor passes a threshold the table doubles its bucket count and re-inserts every entry, an O(n) step that is amortised over the insertions exactly as for the dynamic array of §1.1 (p. 9).

WORKED EXAMPLE

Longest consecutive sequence. Given an unsorted array, find the length of the longest run of consecutive integers (for example 1, 2, 3, 4), in O(n) time.

1. Sorting gives O(n log n); the target forbids it.
2. Put all values in a set. A value x starts a run exactly when $x - 1$ is absent.
3. For each start, extend with `x+1, x+2, …` while present. Each element is visited once as a start-check and at most once inside a run, so the total is O(n).

```cpp
int longestConsecutive(const vector<int>& a) {
    unordered_set<int> s(a.begin(), a.end());
    int best = 0;
    for (int x : s) {
        if (s.count(x - 1)) continue;          // not the start of a run
        int y = x;
        while (y < INT_MAX && s.count(y + 1)) ++y;
        best = max(best, y - x + 1);               // run length
    }
    return best;
}
```

x	x − 1 present?	run extended	length
100	no	100	1
4	yes	skipped	–
200	no	200	1
1	no	1, 2, 3, 4	4

Result for `[100, 4, 200, 1, 3, 2]`: 4. Time O(*n*), space O(*n*).

TRAP

Claim: `m[key]` is a harmless way to test whether a key exists. *Why it is wrong:* in C++ the subscript operator inserts a default value when the key is absent, which changes the map and its size during the test. *Correct statement:* use `find` or `count` to test, and `[]` only when inserting or incrementing is intended.

Hashing network keys: a flow table

A router or firewall keeps per-flow state in a hash table whose key is the 5-tuple (source address, destination address, source port, destination port, protocol). Two points make this a favourite question. First, one must hash the *fields*, not the bytes of the struct, because a C struct may contain padding bytes whose values are unspecified; two equal keys could then hash differently. Secondly, equality must compare the same fields.

```c
#include <stdint.h>
#include <stdbool.h>

typedef struct {
    uint32_t src, dst;
    uint16_t sport, dport;
    uint8_t  proto;
} FlowKey;

static uint32_t fnv_byte(uint32_t h, uint8_t b) {
    return (h ^ b) * 16777619u;
}

uint32_t flow_hash(const FlowKey *k) {  /* FNV-1a over the fields */
    uint32_t h = 2166136261u;
    uint32_t w[3] = { k->src, k->dst, (uint32_t)k->sport << 16 | k->dport };
    for (int i = 0; i < 3; i++)
        for (int b = 0; b < 32; b += 8)
            h = fnv_byte(h, (uint8_t)(w[i] >> b));
    return fnv_byte(h, k->proto);
}
bool flow_eq(const FlowKey *a, const FlowKey *b) {
    return a->src == b->src && a->dst == b->dst && a->sport == b->sport
        && a->dport == b->dport && a->proto == b->proto;
}
```

The bucket is `flow_hash(k) % nbuckets`. *Follow-ups.* To make the two directions of a connection land in the same entry, order the two (address, port) pairs before hashing. Equal-cost multipath forwarding applies the same idea in hardware: a hash of the 5-tuple selects the path, so all packets of a flow take one path and arrive in order, while different flows spread over the paths.

> CHECK YOURSELF
>
> 1. Which is faster for counting lowercase letters: `unordered_map<char,int>` or `int cnt[26]`? [1]
> 2. What is the worst-case lookup in a chained hash table, and when does it occur? [2]

1.4 Strings and parsing network data MUST · ■■□

> BIG IDEA
>
> Parsing is validation first and conversion second, because network input is untrusted and every malformed case an interviewer can imagine is a test case.

A C string is an array of `char` ended by a zero byte, so `strlen` must walk the whole string and costs O(n). Hence a loop written `for (i = 0; i < strlen(s); i++)` is O(n^2), because the length is recomputed on every iteration. A C++ `string` stores its length and answers `size()` in O(1). A string literal in C is read-only, so writing to `char *p = "abc"` is undefined behaviour; copy it into an array first.

> TRAP
>
> *Claim:* `strncpy(dst, src, n)` is a safe `strcpy`. *Why it is wrong:* when `src` has `n` or more characters, `strncpy` copies exactly `n` characters and writes no terminating zero. *Correct statement:* always set `dst[n-1] = '\0'` after `strncpy`, or use `snprintf(dst, n, "%s", src)`, which always terminates.

Recognise: “valid”, “parse”, “split”, “tokens”, dotted or colon-separated text, “log lines”. *Method:* scan once with an index; at each field check its character class, its length and its numeric range, and then check that nothing remains. Do not use `atoi` or `strtok` for validation: `atoi` accepts leading spaces, signs and trailing garbage and cannot report an error, and `strtok` modifies its input and keeps hidden state.

The function below parses a dotted-quad IPv4 address into a host-order 32-bit integer. It rejects an empty octet, a non-digit, a value above 255, a leading zero (“01”) and trailing characters.

```
#include <stdint.h>
#include <stdbool.h>

bool parse_ipv4(const char *s, uint32_t *out) {
    uint32_t addr = 0;
    for (int part = 0; part < 4; part++) {
        if (*s < '0' || *s > '9') return false;  /* empty octet or junk */
        const char *start = s;
        int val = 0, digits = 0;
        while (*s >= '0' && *s <= '9') {
            val = val * 10 + (*s - '0');
            if (++digits > 3 || val > 255) return false;
            s++;
        }
        if (digits > 1 && *start == '0') return false;  /* leading zero */
        addr = (addr << 8) | (uint32_t)val;
        if (part < 3) {
            if (*s != '.') return false;
            s++;
        }
    }
    if (*s != '\0') return false;  /* trailing junk */
    *out = addr;
    return true;
}
```

Input	Result	Reason
`192.168.1.1`	true, 0xC0A80101	four valid octets
`01.2.3.4`	false	leading zero in first octet
`1.2.3`	false	after the third octet the next character is the terminator, not a dot
`1.2.3.4.5`	false	trailing characters remain after the fourth octet
`1..2.3`	false	second octet is empty
`256.1.1.1`	false	value exceeds 255

The reverse conversion extracts each byte with a shift and a mask, using `%u` with an explicit cast because `uint32_t` is not guaranteed to be `unsigned int`:

```
void format_ipv4(uint32_t a, char buf[16]) {  /* 15 characters + NUL */
    snprintf(buf, 16, "%u.%u.%u.%u",
             (unsigned)(a >> 24), (unsigned)((a >> 16) & 255),
             (unsigned)((a >> 8) & 255), (unsigned)(a & 255));
}
```

A MAC address follows the same pattern with hexadecimal digits and colons. Note how the second digit is read only after the first has been validated, so the code never reads past the terminator.

```
static int hexval(char c) {
    if (c >= '0' && c <= '9') return c - '0';
    if (c >= 'a' && c <= 'f') return c - 'a' + 10;
    if (c >= 'A' && c <= 'F') return c - 'A' + 10;
    return -1;
}
bool parse_mac(const char *s, uint8_t mac[6]) {
    for (int i = 0; i < 6; i++) {
        int hi = hexval(s[0]), lo = hi < 0 ? -1 : hexval(s[1]);
        if (hi < 0 || lo < 0) return false;
        mac[i] = (uint8_t)(hi << 4 | lo);
        s += 2;
        if (i < 5) { if (*s != ':') return false; s++; }
    }
    return *s == '\0';
}
```

An IPv6 address adds one difficulty, the *zero compression* `::`, which stands for one or more groups of zeros and may appear once. The method is to collect the groups before and after the `::` in two arrays, and to place the second array at the end of the 16 bytes, leaving the zeros in between.

```
bool parse_ipv6(const char *s, uint8_t out[16]) {
    uint16_t g[2][8];                    /* groups before and after "::" */
    int n[2] = {0, 0}, side = 0;
    if (s[0] == ':') {                   /* a leading colon must be "::" */
        if (s[1] != ':') return false;
        side = 1; s += 2;
    }
    while (*s) {
        unsigned v = 0; int digits = 0;
        while (isxdigit((unsigned char)*s)) {
            v = v * 16 + (unsigned)hexval(*s++);
            if (++digits > 4) return false;
        }
        if (digits == 0 || n[0] + n[1] >= 8) return false;
        g[side][n[side]++] = (uint16_t)v;
        if (!*s) break;
        if (*s++ != ':') return false;
        if (*s == ':') {                 /* "::" */
            if (side == 1) return false;  /* only one allowed */
            side = 1; s++;
        } else if (!*s) return false;     /* trailing single colon */
    }
    int total = n[0] + n[1];
    if (side == 0 ? total != 8 : total > 7) return false;
    memset(out, 0, 16);
    for (int i = 0; i < n[0]; i++) {
        out[2*i] = g[0][i] >> 8;  out[2*i + 1] = g[0][i] & 0xFF;
    }
    for (int i = 0; i < n[1]; i++) {
        int pos = 8 - n[1] + i;          /* right-aligned */
        out[2*pos] = g[1][i] >> 8;  out[2*pos + 1] = g[1][i] & 0xFF;
    }
    return true;
}
```

Without a `::` exactly eight groups are required; with one, at most seven, because `::` must replace at least one group. For `2001:db8::1`, the groups before the gap are `2001` and `0db8`, the group after it is `1`, and the 16 bytes are `20 01 0d b8` followed by ten zero bytes and `00 01`. The forms with an embedded IPv4 suffix (`::ffff:192.0.2.1`) are omitted here; they add a final step that calls `parse_ipv4`.

A CIDR string `a.b.c.d/p` is handled by parsing the address up to the slash, then parsing *p* as a decimal number in 0–32 with the same no-leading-zero rule. Validation of IPv6 groups and the reporting problems built on these parsers are in the Problem Set (Problem Set (p. 71)).

Substring search

Idea. To find a pattern of length *m* in a text of length *n*, the naive method compares at every start position and costs O(*nm*) in the worst case (`aaaa…ab` against `aaab`). The Knuth–Morris–Pratt (KMP) method never moves backwards in the text. It precomputes, for each prefix of the pattern, the length of the longest proper prefix that is also a suffix; after a mismatch, that length says how much of the pattern is still known to match. *Recognise:* “find the pattern”, “repeated substring”, “does *B* occur in *A*”. *Complexity:* O(*n* + *m*) time, O(*m*) space.

```
vector<int> prefixFunction(const string& p) {   // pi[i]: border of p[0..i]
    vector<int> pi(p.size(), 0);
    for (size_t i = 1, k = 0; i < p.size(); ++i) {
        while (k && p[i] != p[k]) k = pi[k - 1];   // shorter border
        if (p[i] == p[k]) ++k;
        pi[i] = k;
    }
    return pi;
}
int findFirst(const string& s, const string& p) {  // first index or -1
    if (p.empty()) return 0;
    vector<int> pi = prefixFunction(p);
    for (size_t i = 0, k = 0; i < s.size(); ++i) {
        while (k && s[i] != p[k]) k = pi[k - 1];
        if (s[i] == p[k]) ++k;
        if (k == p.size()) return (int)(i - k + 1);
    }
    return -1;
}
```

For the pattern `ababc`, the table is `pi = [0, 0, 1, 2, 0]`: for example `abab` has the border `ab`, so `pi[3] = 2`. Searching `abababc`, the match length `k` runs 1, 2, 3, 4 over `abab`, then the text character `a` meets pattern character `c` and `k` falls back from 4 to `pi[3] = 2` instead of restarting at zero; the match then continues 3, 4, 5 and ends at index 6, so the answer is 6 − 5 + 1 = 2. The pointer `i` never moves back, which is the source of the linear bound. A common alternative is the rolling hash of Rabin and Karp: hash each window of length *m* in O(1) by removing the leading character and adding the trailing one, and verify only on a hash match; it runs in O(*n* + *m*) on average and extends to many patterns at once.

> LAB
>
> `LAB 1.4 · 10 MIN` Check the parser against the C library, which is the reference implementation. Put `hexval` and `parse_ipv6` of this section in `v6.c`, with the driver below, and compile.

```c
#include <arpa/inet.h>
#include <ctype.h>
#include <stdio.h>
#include <string.h>
#include <stdint.h>
#include <stdbool.h>
/* ... hexval() and parse_ipv6() from this section ... */
int main(void) {
    const char *t[] = { "::", "::1", "1::", "2001:db8::ff00:42:8329",
        "1:2:3:4:5:6:7:8", "1:2:3:4:5:6:7::", "1::2::3", ":1:2:3:4:5:6:7",
        "12345::", "1:2:3:4:5:6:7:8:9", "1:2:3:4:5:6:7:8:", "g::1",
        "1:::2" };
    int bad = 0;
    for (size_t i = 0; i < sizeof t / sizeof *t; i++) {
        uint8_t mine[16], ref[16];
        int a = parse_ipv6(t[i], mine);
        int b = inet_pton(AF_INET6, t[i], ref) == 1;
        int same = (a == b) && (!a || memcmp(mine, ref, 16) == 0);
        printf("%-26s mine=%d lib=%d %s\n", t[i], a, b,
               same ? "ok" : "MISMATCH");
        bad += !same;
    }
    printf("%d mismatches\n", bad);
    return bad != 0;
}
```

```
$ gcc -Wall -o v6 v6.c && ./v6
::                         mine=1 lib=1 ok
::1                        mine=1 lib=1 ok
1::                        mine=1 lib=1 ok
2001:db8::ff00:42:8329     mine=1 lib=1 ok
1:2:3:4:5:6:7:8            mine=1 lib=1 ok
1:2:3:4:5:6:7::            mine=1 lib=1 ok
1::2::3                    mine=0 lib=0 ok
:1:2:3:4:5:6:7             mine=0 lib=0 ok
12345::                    mine=0 lib=0 ok
1:2:3:4:5:6:7:8:9          mine=0 lib=0 ok
1:2:3:4:5:6:7:8:           mine=0 lib=0 ok
g::1                       mine=0 lib=0 ok
1:::2                      mine=0 lib=0 ok
0 mismatches
```

What you should notice: the seven malformed strings (two `::`, a leading single colon, five-digit group, nine groups, trailing colon, non-hex letter, three colons) are all rejected by both, and the valid ones give identical bytes. Add your own cases; every mismatch is a bug in one of the two, and the library is usually right.

CHECK YOURSELF

1. Why is `01.2.3.4` rejected even though `atoi("01")` is 1? [3]
2. What goes wrong with `char buf[15]` in `format_ipv4`? [4]

1. The array, since it has no hashing or allocation; both are O(*n*) overall.↩

1.5 Bit manipulation and subnet code MUST · ■■□

BIG IDEA

An integer is a row of switches, and a mask with AND, OR, XOR and shifts reads or writes any group of them in one step, which is why addresses, flags and registers are all handled this way.

The theory of subnetting, masks and CIDR is taught in the networking chapter (Pillar 3, §3.4); this section supplies only the bit code that implements it. The operators are `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), `<<` and `>>`. We use unsigned types throughout, because left-shifting a negative signed value or shifting a `1` into the sign bit is undefined behaviour in C, and `>>` on a signed negative value is implementation-defined.

TRAP

Claim: `if (x & 1 == 0)` tests whether *x* is even. *Why it is wrong:* `==` binds tighter than `&`, so the expression means `x & (1 == 0)`, that is `x & 0`, which is always zero. *Correct statement:* write `(x & 1) == 0`, and parenthesise every bitwise sub-expression.

Task	Expression	Why it works
set bit *n*	`x | (1u << n)`	OR with a one forces the bit to 1
clear bit *n*	`x & ~(1u << n)`	AND with a zero forces the bit to 0
toggle bit *n*	`x ^ (1u << n)`	XOR with a one flips the bit
test bit *n*	`(x >> n) & 1u`	brings the bit to position 0
clear lowest set bit	`x & (x - 1)`	subtracting 1 flips the lowest 1 and the zeros below it
isolate lowest set bit	`x & -x`	two's complement negation keeps only that bit
power of two (x > 0)	`x && !(x & (x - 1))`	exactly one bit is set
count set bits	loop `x &= x - 1`	each pass removes one bit; runs popcount times
round up to multiple of *a* (power of two)	`(x + a - 1) & ~(a - 1)`	adds slack, then clears the low bits

As macros for C, with arguments in parentheses and `1u` to avoid signed overflow, together with a population count and a byte swap (the conversion between big-endian and little-endian forms):

2. O(*n*), when every key lands in one bucket, for example with a poor hash or adversarial input.↩
3. Leading zeros are ambiguous (some parsers read them as octal), so strict validators reject them.↩
4. The longest address `255.255.255.255` has 15 characters, and the terminating zero needs a sixteenth byte.↩

```c
#define SET_BIT(x, n)    ((x) |=  (1u << (n)))
#define CLEAR_BIT(x, n)  ((x) &= ~(1u << (n)))
#define TOGGLE_BIT(x, n) ((x) ^=  (1u << (n)))
#define TEST_BIT(x, n)   (((x) >> (n)) & 1u)

static unsigned popcount(uint32_t x) {
    unsigned c = 0;
    while (x) { x &= x - 1; c++; }
    return c;
}
static uint32_t swap32(uint32_t x) {
    return (x >> 24) | ((x >> 8) & 0x0000FF00u) |
           ((x << 8) & 0x00FF0000u) | (x << 24);
}
```

Subnet arithmetic as bit operations

A prefix length p means that the top p bits are the network part. The mask has p ones followed by $32 - p$ zeros.

```c
static uint32_t mask_of(unsigned p) {            /* p in 0..32 */
    return p ? 0xFFFFFFFFu << (32 - p) : 0u;  /* shift by 32 would be UB */
}
bool same_subnet(uint32_t a, uint32_t b, unsigned p) {
    uint32_t m = mask_of(p);
    return (a & m) == (b & m);
}
void describe(uint32_t ip, unsigned p,
              uint32_t *net, uint32_t *bc, uint64_t *hosts) {
    uint32_t m = mask_of(p);  /* p <= 30 for a host range */
    *net = ip & m;
    *bc = *net | ~m;
    *hosts = (1ull << (32 - p)) - 2;  /* usable: net + 1 .. bc - 1 */
}
```

> **TRAP**
>
> *Claim:* `0xFFFFFFFFu << (32 - p)` is correct for every prefix length. *Why it is wrong:* for $p = 0$ the shift count is 32, which equals the width of the type, and the C standard leaves that undefined (x86 silently reduces the count modulo 32 and returns the unshifted value). *Correct statement:* special-case $p = 0$ as in `mask_of`.

Figure 1.6 The same-subnet test in bits. AND with the mask keeps the 22 network bits and clears the 10 host bits.

> **WORKED EXAMPLE**
>
> **Describe 10.20.30.40/22 with bit operations only.**
>
> 1. The mask has 22 ones: `0xFFFFFC00`, which is 255.255.252.0.

2. The network address is `ip & mask`. The third octet 30 = `00011110` keeps its top six bits, giving `00011100` = 28, and the fourth octet is cleared. Network: 10.20.28.0.
3. The broadcast address is `net | ~mask`, which sets the ten host bits: third octet 28 + 3 = 31, fourth octet 255. Broadcast: 10.20.31.255.
4. Usable hosts run from 10.20.28.1 to 10.20.31.254, which is $2^{10} - 2 = 1022$ addresses.
5. Test 10.20.29.7 against it: its third octet 29 = `00011101` also clears to 28, so it lies in the subnet.

Result: network 10.20.28.0, broadcast 10.20.31.255, 1022 hosts.

Complexity. Every operation here is O(1); `popcount` by clearing the lowest bit is O(*number of set bits*). GCC and Clang offer `__builtin_popcount` and `__builtin_clz`, but interviewers usually want the loop.

Packing and unpacking header fields

Protocol headers pack several fields into one byte or word, and reading them is a matter of shifts and masks. The layout of the IPv4 header is taught with the other headers (Pillar 3); the code below extracts the fields of its first eight bytes: the version and header length share byte 0, the traffic class shares byte 1, the total length fills bytes 2–3, and bytes 6–7 hold three flag bits above a 13-bit fragment offset counted in units of eight bytes.

```
typedef struct {
    unsigned version, hdr_bytes, dscp, ecn, total_len, df, mf, frag_bytes;
} Ip4Info;

void ip4_fields(const uint8_t *h, Ip4Info *o) {
    unsigned ff = (unsigned)h[6] << 8 | h[7];           /* flags + offset */
    o->version   = h[0] >> 4;
    o->hdr_bytes = (h[0] & 0x0Fu) * 4;  /* IHL counts words */
    o->dscp      = h[1] >> 2;
    o->ecn       = h[1] & 0x03u;
    o->total_len = (unsigned)h[2] << 8 | h[3];
    o->df        = (ff >> 14) & 1u;
    o->mf        = (ff >> 13) & 1u;
    o->frag_bytes = (ff & 0x1FFFu) * 8;
}
```

Byte	0	1	2	3	6	7
Value	0x45	0x00	0x00	0x73	0x40	0x00
Field	version 4, IHL 5	DSCP 0, ECN 0	total length 0x0073 = 115		flags 010 (DF set)	offset 0

For the bytes shown, the header is 5 × 4 = 20 bytes long, the packet is 115 bytes, and the *don't fragment* flag is set. Reading multi-byte fields byte by byte, as above, avoids both alignment faults and byte-order mistakes, because it does not depend on the host's endianness.

Bitmasks as sets

An integer of n bits can represent a subset of n items, so that the union is `|`, the intersection is `&`, membership is a bit test, and all subsets are the integers 0 to $2^n - 1$. This is practical for $n \le 20$ (about 10^6 subsets). *Recognise:* "all subsets", "choose a subset such that", small n.

```cpp
int countSubsetsWithSum(const vector<int>& a, int target) {   // n <= 20
    int n = a.size(), count = 0;
    for (unsigned m = 0; m < (1u << n); ++m) {
        long long sum = 0;
        for (int i = 0; i < n; ++i)
            if (m >> i & 1u) sum += a[i];
        if (sum == target) ++count;
    }
    return count;
}
```

The empty subset is included (it has sum zero). The cost is $O(2^n \cdot n)$; iterating only over the set bits with `x &= x - 1` and `ctz` removes the inner scan.

> INTERVIEW LENS
>
> QUALCOMM · BROADCOM · JUNIPER · CISCO
>
> Bit questions are asked in C and are examined for care with types: unsigned operands, `1u`, parentheses and the shift-by-width case. A strong answer states the idiom, explains why it works in one sentence, and mentions the undefined-behaviour pitfall unprompted. Typical follow-up: “do it without a loop” or “do it for a 64-bit value”.

> CHECK YOURSELF
>
> 1. What does `x & (x - 1)` return for $x = 40$ (101000)? [1]
> 2. How many usable hosts has a /27? [2]

1.6 Linked lists in C MUST · ■■□

> BIG IDEA
>
> A linked list is a chain of nodes joined by pointers, and every list problem is a question about which pointer to rewrite, and in which order, so that no node is lost.

A singly linked list node holds a value and the address of the next node; the last node holds `NULL`. Access by position costs $O(n)$, but inserting or deleting after a known node costs O(1) because only pointers change. Routers keep many such lists (the chains of a hash table, a queue of packet buffers), and interviews at Arista, Juniper, Nokia and Broadcom ask for them in C from a blank page.

```cpp
typedef struct Node { int val; struct Node *next; } Node;

Node *push_front(Node *head, int v) {
    Node *n = malloc(sizeof *n);
    if (!n) return head;  /* allocation failed: list unchanged */
    n->val = v; n->next = head;
    return n;                          /* the new head */
}
```

The pointer-to-pointer idiom

To delete a node we must re-point the link that leads to it. For a middle node that link is the previous node’s `next`; for the first node it is the `head` variable itself, which is why a naive version needs a special case for the head. If we hold a pointer to the link (a `Node **`), both cases are the same.

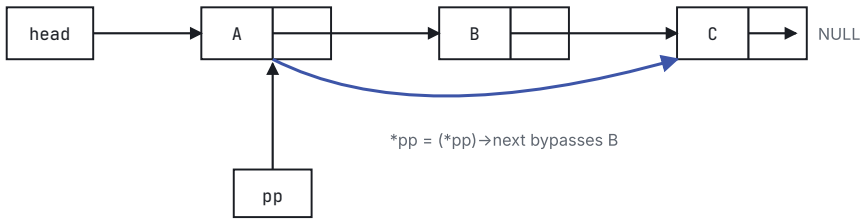


Figure 1.7 The pointer-to-pointer idiom. pp holds the address of the link that points at the current node (head, or the next field of the predecessor), so the head needs no special case.

```
Node *delete_value(Node *head, int key) {          /* first match only */
    Node **pp = &head;
    while (*pp && (*pp)->val != key) pp = &(*pp)->next;
    if (*pp) { Node *dead = *pp; *pp = dead->next; free(dead); }
    return head;
}
Node *delete_all(Node *head, int key) {            /* every match */
    Node **pp = &head;
    while (*pp) {
        if ((*pp)->val == key) {
            Node *dead = *pp; *pp = dead->next; free(dead);
        } else pp = &(*pp)->next;  /* advance only if nothing was removed */
    }
    return head;
}
```

For `3 → 5 → 3 → 7` and key 3, `delete_all` removes the head (the link `head` now points at the 5), skips the 5, removes the second 3 (the link `5.next` now points at the 7), and returns `5 → 7`. The alternative to `Node **` is a dummy node placed before the head; both remove the special case.

> **TRAP**
>
> *Claim:* after `free(dead)` the loop may continue with `dead->next`. *Why it is wrong:* the memory has been returned and reading it is undefined behaviour. *Correct statement:* read `dead->next` into a local (or into `*pp`) before calling `free`.

Reversal

Three pointers walk the list: `prev` (the already reversed part), `head` (the current node) and `next`, saved before the current `next` field is overwritten. The classic bug is to overwrite first and so lose the rest of the list.

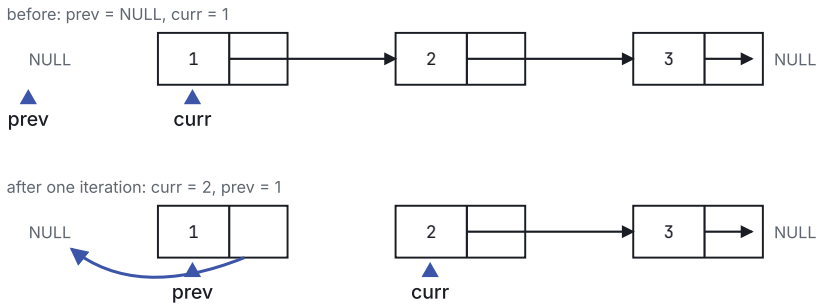


Figure 1.8 One step of in-place reversal. The accent arrow is the single pointer rewritten; next was saved first so that the rest of the list is not lost.

```
Node *reverse(Node *head) {
    Node *prev = NULL;
    while (head) {
        Node *next = head->next;       /* save */
        head->next = prev;             /* rewire */
        prev = head;                   /* advance both */
        head = next;
    }
    return prev;
}
```

Step	head (current)	next saved	After rewire	prev
1	1	2	1 → NULL	1
2	2	3	2 → 1	2
3	3	NULL	3 → 2	3

The loop ends with `head == NULL` and `prev` pointing at the old tail, which is the new head. Time O(n), space O(1).

Fast and slow pointers

Two pointers moving at speeds one and two give the middle node and detect cycles. For the middle, when `fast` reaches the end `slow` is halfway. For a cycle, `fast` closes on `slow` by one node per step, so inside a cycle they must meet.

```
Node *middle(Node *head) {  /* second middle when the length is even */
    Node *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next; fast = fast->next->next;
    }
    return slow;
}
Node *cycle_start(Node *head) {           /* NULL if there is no cycle */
    Node *slow = head, *fast = head;
    while (fast && fast->next) {  /* test fast before fast->next */
        slow = slow->next; fast = fast->next->next;
        if (slow == fast) {
            for (slow = head; slow != fast; ) {
                slow = slow->next; fast = fast->next;
            }
            return slow;
        }
    }
    return NULL;
}
```

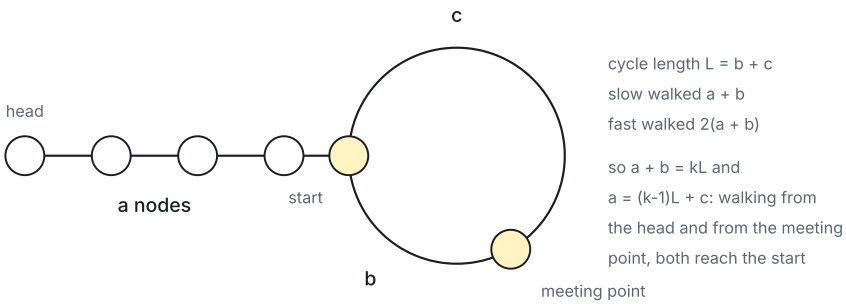


Figure 1.9 The geometry behind Floyd's cycle test. The tail has a nodes; the meeting point lies b steps round the cycle from its start and c steps short of completing it.

Why the second phase finds the start. Let a be the number of nodes from the head to the cycle start, b the distance from the start to the meeting point along the cycle, and c the remainder of the cycle, so the cycle length is $L = b + c$. When they meet, `slow` has taken $a + b$ steps and `fast` twice as many, and the difference is a whole number k of laps: $2(a + b) - (a + b) = kL$. Hence $a = kL - b = (k - 1)L + c$. A pointer from the head and one from the meeting point, both moving one step at a time, therefore reach the cycle start together, because the second one needs c steps to reach it and then some whole laps.

> **TRAP**
>
> *Claim:* `while (fast->next && fast)` is an equivalent loop condition. *Why it is wrong:* the first operand dereferences `fast` before it is known to be non-null, so an empty list crashes. *Correct statement:* test `fast` first, then `fast->next`, and only then use `fast->next->next`.

Merging with a dummy head

A temporary node on the stack gives the output list a fixed first node, so that the first append needs no special case.

```c
Node *merge(Node *a, Node *b) {            /* both sorted ascending */
    Node dummy = { 0, NULL }, *tail = &dummy;
    while (a && b) {
        if (a->val <= b->val) { tail->next = a; a = a->next; }
        else                  { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;  /* append the non-empty remainder */
    return dummy.next;  /* the pointer, not the stack object */
}
```

Returning `dummy.next` is legal: it points to nodes that were allocated elsewhere, and only the local `dummy` itself vanishes on return. Returning `&dummy`, or a pointer to any local array, would be the classic dangling-pointer bug.

To trace `merge` on `1 → 4 → 6` and `2 → 3 → 7 → 8`, the pointer `tail` appends the smaller head each time:

a	b	taken	output so far
1	2	1	1
4	2	2	1 2
4	3	3	1 2 3
4	7	4	1 2 3 4
6	7	6	1 2 3 4 6
empty	7	remainder of b	1 2 3 4 6 7 8

The first line of every list problem is the choice between a list and an array, which depends on what the program does most.

Operation	Array	Singly linked list
index *i*	O(1)	O(*i*)
insert or delete at the front	O(*n*)	O(1)
insert or delete after a known node	O(*n*)	O(1)
search	O(*n*), O(log *n*) if sorted	O(*n*)
memory per element	the value only	the value and a pointer, plus allocator overhead
cache behaviour	sequential, fast	scattered, slow

Doubly linked lists

A doubly linked list adds a `prev` field, which makes deleting a *given* node O(1) with no search for its predecessor. Using a circular list with a sentinel node removes every `NULL` check:

```c
typedef struct DNode { int val; struct DNode *prev, *next; } DNode;

void dl_init(DNode *s) { s->prev = s->next = s; }  /* empty list */
void dl_insert_after(DNode *p, DNode *n) {
    n->prev = p; n->next = p->next;
    p->next->prev = n; p->next = n;
}
void dl_unlink(DNode *n) {
    n->prev->next = n->next; n->next->prev = n->prev;
}
```

The order of the four writes in `dl_insert_after` matters: `n` is wired to its neighbours first, and the neighbours are changed last. This structure is the core of the LRU cache in §1.15 (p. 60) and of the intrusive lists inside operating-system kernels.

Operating-system kernels and router software often *embed* the list node inside the object it links, so that one allocation serves the object and its place on several lists. The macro `container_of` then recovers the object from a pointer to the embedded node, by subtracting the offset of the member.

```c
#include <stddef.h>
#define container_of(ptr, type, member) \
    ((type *)((char *)(ptr) - offsetof(type, member)))

struct pkt { int len; DNode link; };        /* DNode from above */

int pkt_len(DNode *n) { return container_of(n, struct pkt, link)->len; }
```

The list code is written once for `DNode` and works for every object type that embeds one; this is the design behind the Linux `list_head`.

> **INTERVIEW LENS**
>
> ARISTA · JUNIPER · NOKIA · BROADCOM
>
> The question is usually phrased as “write it in C on the shared editor”: delete a node (head included), delete all nodes with a value, reverse, merge, find the loop. A strong answer draws the boxes first, names the three pointers, and checks the empty list, the one-node list and the head. Typical follow-ups are “what if the node is the tail” and “reverse it recursively”.

> **CHECK YOURSELF**
>
> 1. In `delete_value`, what does `pp` point to when the list is empty? [3]
> 2. Why does `reverse` need the variable `next`? [4]

1. 32 (100000): the lowest set bit is cleared.↩
2. $2^5 - 2 = 30$.↩
3. To `head` itself, which is `NULL`, so the loop does not run and nothing is freed.↩
4. Without it, writing `head->next = prev` destroys the only refere-

1.7 Stacks, queues, ring buffers and deques MUST · ■■□

nce to the rest of the list.↩

> BIG IDEA
>
> A stack remembers the most recent unfinished thing and a queue remembers the oldest, and a ring buffer gives a queue of fixed size with no allocation, which is exactly what a network driver needs.

A *stack* is last in, first out: `push`, `pop` and `top` are O(1) on an array with a top index. *Recognise:* nested structure, “undo”, “matching”, “most recent”, expression evaluation. A *queue* is first in, first out and is the engine of breadth-first search and of packet queues. A *deque* allows insertion and removal at both ends in O(1); `std::deque` provides it, and Python’s `collections.deque` does the same.

Matching brackets is the model stack problem. Each opening bracket is pushed; each closing bracket must match the top.

```
bool balanced(const char *s) {
    size_t top = 0;
    char *st = malloc(strlen(s) + 1);
    if (!st) return false;
    bool ok = true;
    for (; *s && ok; s++) {
        switch (*s) {
        case '(': case '[': case '{': st[top++] = *s; break;
        case ')': ok = top && st[--top] == '('; break;
        case ']': ok = top && st[--top] == '['; break;
        case '}': ok = top && st[--top] == '{'; break;
        }
    }
    free(st);
    return ok && top == 0;  /* leftovers mean unclosed brackets */
}
```

Stacks and queues in C

An array with a top index is the simplest stack. Every operation checks for overflow and underflow and reports it, because the caller, not the language, owns the error. A linked queue keeps pointers to both ends so that `enqueue` at the tail is O(1); with a single pointer it would be O(*n*).

```c
typedef struct { int a[64]; int top; } Stack;  /* top = element count */
bool st_push(Stack *s, int v) {
    if (s->top == 64) return false;
    s->a[s->top++] = v;
    return true;
}
bool st_pop(Stack *s, int *v) {
    if (s->top == 0) return false;
    *v = s->a[--s->top];
    return true;
}

typedef struct { Node *head, *tail; } Queue;      /* Node: list section */
bool enqueue(Queue *q, int v) {
    Node *n = malloc(sizeof *n);
    if (!n) return false;
    n->val = v; n->next = NULL;
    if (q->tail) q->tail->next = n;
    else q->head = n;                        /* empty queue: head too */
    q->tail = n;
    return true;
}
bool dequeue(Queue *q, int *v) {
    Node *n = q->head;
    if (!n) return false;
    *v = n->val;
    q->head = n->next;
    if (!q->head) q->tail = NULL;  /* queue became empty */
    free(n);
    return true;
}
```

The classic queue bug is leaving `tail` pointing at the freed last node after the queue empties, which the second-to-last line above prevents.

The ring buffer

Packet drivers hand frames from an interrupt handler to a consumer through a fixed array used circularly. The producer writes at `tail` and the consumer reads at `head`; both wrap with a modulus. The difficulty is that `head == tail` describes both an empty and a full buffer. Three remedies exist: keep a `count`, waste one slot, or let the indices run freely and compare their difference.

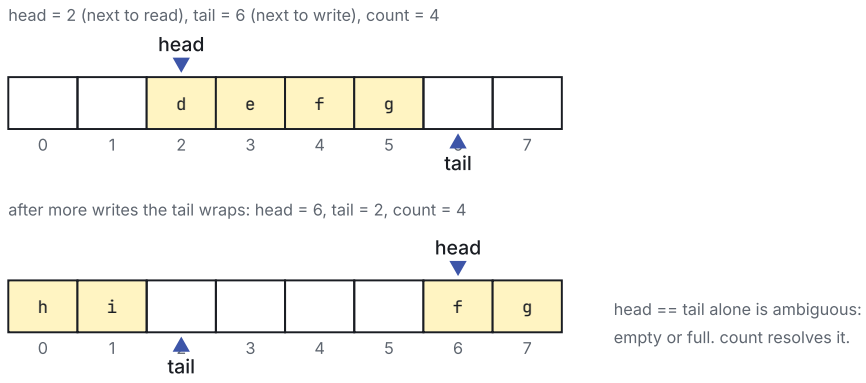


Figure 1.10 A circular buffer of capacity eight, before and after the write index wraps. A separate count removes the full-versus-empty ambiguity.

```c
#define N 8
typedef struct { uint8_t buf[N]; size_t head, tail, count; } Ring;

bool ring_push(Ring *r, uint8_t b) {
    if (r->count == N) return false;              /* full */
    r->buf[r->tail] = b;
    r->tail = (r->tail + 1) % N;
    r->count++;
    return true;
}
bool ring_pop(Ring *r, uint8_t *out) {
    if (r->count == 0) return false;              /* empty */
    *out = r->buf[r->head];
    r->head = (r->head + 1) % N;
    r->count--;
    return true;
}
```

When one thread produces and another consumes, the `count` variable is written by both and needs a lock. The lock-free alternative keeps `head` and `tail` as free-running unsigned counters and uses a power-of-two capacity: the slot is `idx & (N - 1)`, the buffer is empty when `head == tail` and full when `tail - head == N`. The subtraction is correct even after the counters wrap, because unsigned arithmetic is modular. The producer writes only `tail` and the consumer only `head`, so no lock is needed, provided each side publishes its index with the memory ordering that the concurrency chapters describe.

The monotonic stack

Idea. To find, for each element, the next greater element, keep a stack of indices whose values are decreasing. A new value pops every smaller entry, and each popped entry has found its answer. Every index is pushed and popped once, so the cost is O(*n*). *Recognise:* “next greater or smaller”, “days until”, “span”, “histogram”.

```cpp
vector<int> dailyTemps(const vector<int>& t) {
    vector<int> ans(t.size(), 0), st;   // st: indices, temps decreasing
    for (int i = 0; i < (int)t.size(); ++i) {
        while (!st.empty() && t[st.back()] < t[i]) {
            ans[st.back()] = i - st.back();   // wait = index distance
            st.pop_back();
        }
        st.push_back(i);
    }
    return ans;  // never-warmer days stay 0
}
```

i	t[i]	indices popped (answers set)	stack after
0	73	none	0
1	74	0 → 1	1
2	75	1 → 1	2
3	71	none	2, 3
4	69	none	2, 3, 4
5	72	4 → 1, 3 → 2	2, 5

6	76	5 → 1, 2 → 4	6
7	73	none	6, 7

Result for `[73, 74, 75, 71, 69, 72, 76, 73]`: `[1, 1, 4, 2, 1, 1, 0, 0]`. The classic bug is to store values instead of indices, which makes the distance impossible to compute.

The sliding-window maximum uses the same idea with a deque; it is solved in the Problem Set.

> CHECK YOURSELF
>
> 1. In the lock-free ring, why does `tail - head == N` identify a full buffer after both counters have wrapped past 2^{32}? [1]
> 2. What is the time complexity of `dailyTemps`, given the inner `while`? [2]

1.8 Trees and binary search trees MUST · ■■□

> BIG IDEA
>
> A binary search tree keeps its keys ordered so that each comparison discards a whole subtree, giving search in time proportional to the height, which is logarithmic only while the tree stays balanced.

A *binary tree* is a node with at most two children; a *binary search tree* (BST) adds the rule that every key in the left subtree is smaller than the node and every key in the right subtree is larger. The *height* h is the number of edges on the longest root-to-leaf path. Search, insert and delete each follow one path, so they cost O(h). For n random keys h is about 2 log n, but inserting sorted keys produces a chain with $h = n$, and every operation degrades to O(n). Self-balancing trees (AVL, red-black) restore O(log n); C++ `std::map` and `std::set` are red-black trees.

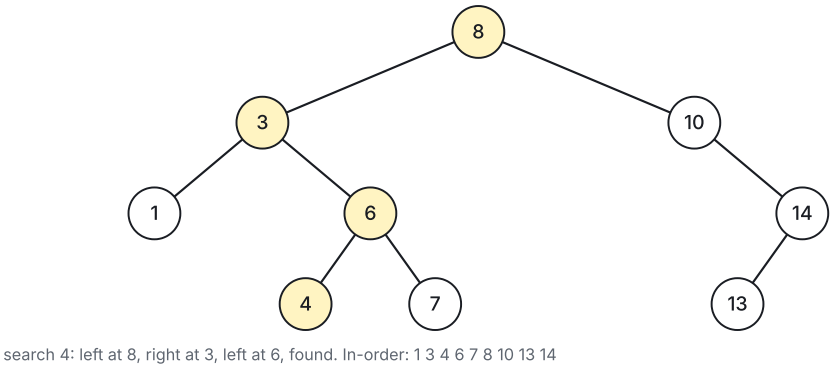


Figure 1.11 A binary search tree. Everything left of a node is smaller and everything right is larger; the highlighted path is the search for 4.

Traversals

Depth-first traversals differ only in when the node is visited: *pre-order* (node, left, right), *in-order* (left, node, right) and *post-order* (left, right, node). The in-order traversal of a BST yields the keys in sorted order, which is the fact behind “k-th smallest” and “validate BST”. Breadth-first traversal visits level by level with a queue. *Recognise:* “depth”, “path”, “subtree” (DFS, usually post-order, returning information to the parent); “level”, “right side view”, “shortest” (BFS); “sorted”, “k-th”, “validate” (in-order).

```cpp
struct TreeNode { int val; TreeNode *left = nullptr, *right = nullptr; };

int height(TreeNode* t) {                  // nodes on the longest path
    return t ? 1 + max(height(t->left), height(t->right)) : 0;
}

vector<int> inorderIter(TreeNode* root) {  // no recursion: explicit stack
    vector<int> out; vector<TreeNode*> st; TreeNode* cur = root;
    while (cur || !st.empty()) {
        while (cur) { st.push_back(cur); cur = cur->left; }  // far left
        cur = st.back(); st.pop_back();
        out.push_back(cur->val);
        cur = cur->right;
    }
    return out;
}

vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> res; if (!root) return res;
    queue<TreeNode*> q; q.push(root);
    while (!q.empty()) {
        int sz = q.size(); res.emplace_back();  // sz = nodes on this level
        while (sz--) {
            TreeNode* t = q.front(); q.pop();
            res.back().push_back(t->val);
            if (t->left) q.push(t->left);
            if (t->right) q.push(t->right);
        }
    }
    return res;
}
```

All three run in O(n) time. Their space is O(h) for the explicit stack and O(*width*) for the queue. A skewed tree of 10^6 nodes also overflows the call stack of the recursive `height`, which is the reason for asking in an interview whether the tree may be skewed.

BST operations in C

Insert and delete return the root of the (possibly changed) subtree, and the caller stores the result back into its link. Forgetting to return or store it is the standard delete bug.

```c
typedef struct T { int key; struct T *l, *r; } T;

T *bst_insert(T *t, int k) {
    if (!t) { t = calloc(1, sizeof *t); if (t) t->key = k; return t; }
    if (k < t->key)      t->l = bst_insert(t->l, k);
    else if (k > t->key) t->r = bst_insert(t->r, k);
    return t;  /* duplicates are ignored */
}
const T *bst_search(const T *t, int k) {
    while (t && t->key != k) t = k < t->key ? t->l : t->r;
    return t;
}
static T *min_node(T *t) { while (t->l) t = t->l; return t; }

T *bst_delete(T *t, int k) {
    if (!t) return NULL;
    if (k < t->key)      t->l = bst_delete(t->l, k);
    else if (k > t->key) t->r = bst_delete(t->r, k);
    else {
        if (!t->l || !t->r) {  /* zero or one child */
            T *child = t->l ? t->l : t->r;
            free(t);
            return child;  /* parent adopts the child */
        }
        T *s = min_node(t->r);  /* in-order successor */
        t->key = s->key;
        t->r = bst_delete(t->r, s->key);
    }
    return t;
}
```

Deletion has three cases: a leaf is removed; a node with one child is replaced by that child; a node with two children takes the key of its in-order successor (the smallest key of the right subtree), and the successor, which has no left child, is then deleted by one of the first two cases.

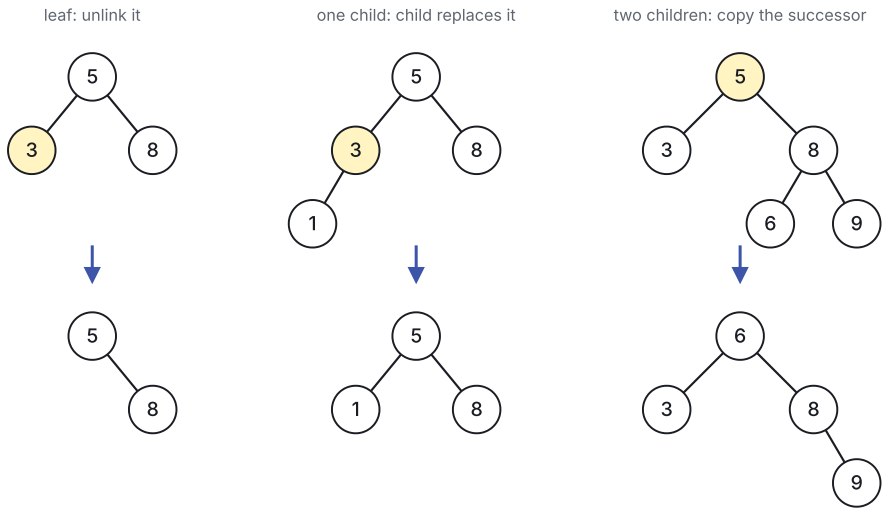


Figure 1.12 The three cases of BST deletion, with the deleted node shaded. In the third case the successor 6 is the left-most node of the right subtree; its key moves up and its old position is removed.

WORKED EXAMPLE

Delete 3 from the tree of Figure 1.11.

1. Search: 3 < 8, so go left and find the node 3.

2. It has two children (1 and 6), so find the successor: the left-most node of the right subtree, `6 → 4`. The successor is 4.
3. Copy 4 into the node, so the node now holds 4.
4. Delete 4 from the right subtree. The node 4 is a leaf, so it is removed and the left link of 6 becomes `NULL`.
5. The in-order sequence is now 1 4 6 7 8 10 13 14: still sorted, with 3 removed.

Result: the node 3 now holds 4, and the old leaf 4 is gone.

Validating a BST

Checking each node only against its parent is wrong, since a node deep in the right subtree may still be smaller than an ancestor. We pass down the open interval that each key must lie in.

```
bool valid(TreeNode* t, long long lo, long long hi) {  // lo < key < hi
    return !t || (lo < t->val && t->val < hi &&
                  valid(t->left, lo, t->val) &&
                  valid(t->right, t->val, hi));
}
// call: valid(root, LLONG_MIN, LLONG_MAX)
```

TRAP

Claim: a tree is a BST if every node is greater than its left child and smaller than its right child. *Why it is wrong:* the tree 5 with left child 1 and right child 6, where 6 has left child 4, satisfies the local rule but 4 sits in the right subtree of 5. *Correct statement:* every key must lie between the bounds inherited from all its ancestors, or the in-order sequence must be strictly increasing.

Keeping a BST balanced

An *AVL tree* stores in each node the height of its subtree and restores the rule that the heights of a node's two subtrees differ by at most one, after every insert or delete, using *rotations*. A rotation rearranges three pointers, lowers one side by a level and raises the other, and preserves the in-order sequence, so the search property survives.

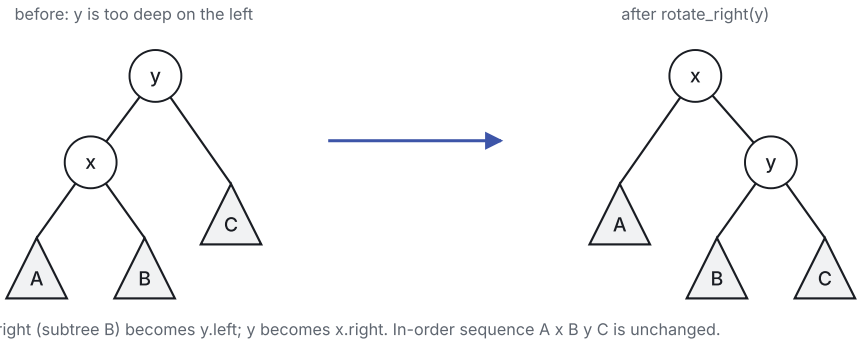


Figure 1.13 A right rotation. Three pointer writes lower the left side by one level and raise the right, and the search order is preserved.

```
T *rotate_right(T *y) {          /* y->l becomes the new subtree root */
    T *x = y->l;
    y->l = x->r;                 /* subtree B moves across */
    x->r = y;
    return x;                    /* caller stores x where y was */
}
T *rotate_left(T *x) {           /* mirror image */
    T *y = x->r;
    x->r = y->l;
    y->l = x;
    return y;
}
```

After an insertion, the first unbalanced node on the path back to the root is repaired by one of four cases, chosen by where the new key went.

Imbalance	Where the key went	Repair
left-left	into the left subtree of the left child	rotate right at the node
right-right	into the right subtree of the right child	rotate left at the node
left-right	into the right subtree of the left child	rotate left at the child, then right at the node
right-left	into the left subtree of the right child	rotate right at the child, then left at the node

Inserting 1, 2, 3 in order into an AVL tree leaves $1 \to 2 \to 3$ as a chain after the third key; this is the right-right case, and one left rotation at 1 gives the balanced tree with 2 at the root. Height is then at most about $1.44 \log_2 n$, so every operation is O(log n). An interview rarely asks for the full AVL code; it asks why a plain BST degrades and what a rotation is, and expects `std::map` as the practical answer.

Structure	Ordering rule	Find	Typical use
binary search tree	left < node < right	O(h)	ordered set, range queries
heap	parent ≤ children	O(n)	the minimum, the next event
trie	path spells the key	O(L)	prefixes, longest-prefix match
hash table	none	O(1) average	membership, counts

> CHECK YOURSELF
>
> 1. What is the height of the BST built by inserting 1, 2, 3, 4, 5 in order? [3]
> 2. Which traversal prints a BST in sorted order? [4]

1. Unsigned subtraction is modulo 2^{32}, so the difference is the true occup-

1.9 Tries and longest-prefix match MUST · ■■■

BIG IDEA

A router forwards a packet along the most specific route that contains its destination, and a trie finds that route by reading the address one bit at a time, in a number of steps bounded by the address length and independent of the number of routes.

A *trie* (prefix tree) stores a set of strings along the edges of a tree: the path from the root spells the string, and a flag on a node marks that a stored string ends there. Lookup costs O(L) for a key of length L, regardless of how many keys are stored. *Recognise:* "words with this prefix", "autocomplete", "longest common prefix", "maximum XOR", and above all "longest prefix match" on IP addresses.

The routing problem. A forwarding table maps *prefixes* to next hops. A prefix `10.1.0.0/16` matches every address whose first 16 bits equal those of 10.1.0.0. Prefixes overlap: an address in 10.1.2.0/24 also lies in 10.1.0.0/16 and in 10.0.0.0/8. The forwarding rule is that the matching prefix with the greatest length wins, termed *longest-prefix match* (LPM), because the longer prefix is the more specific route. (Why routers need this, and how routes are learned, is taught in the routing chapter; here we implement the lookup.)

Binary trie. We treat a 32-bit address as a string over {0, 1}, with one level per bit. Inserting a prefix of length p follows (and creates) p edges and stores the next hop at the final node. A lookup follows the address bits from the root and records the next hop of every marked node it passes; when the next edge is missing, the last recorded hop is the longest match.

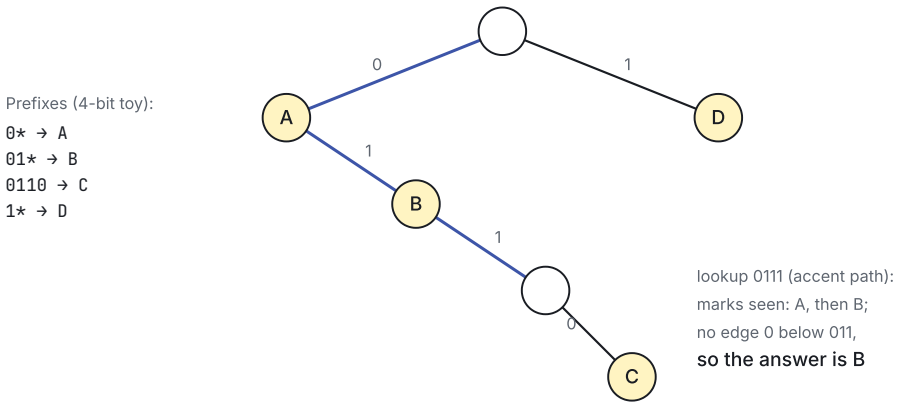


Figure 1.14 A binary trie for four toy prefixes. A shaded node carries a next hop; a lookup walks the address bits and remembers the deepest shaded node it passed.

ancy as long as it never exceeds N.↩

2. O(n): each index is pushed once and popped at most once.↩
3. 4 edges (5 nodes): each key becomes the right child of the previous one, so operations cost O(n).↩
4. In-order (left, node, right).↩

```c
#include <stdint.h>
#include <stdlib.h>
#include <stdbool.h>

typedef struct Node {
    struct Node *c[2];
    int hop;                        /* hop < 0: no route ends here */
} Node;

static Node *new_node(void) {
    Node *n = malloc(sizeof *n);
    if (n) { n->c[0] = n->c[1] = NULL; n->hop = -1; }
    return n;
}
bool trie_insert(Node *root, uint32_t prefix, unsigned len, int hop) {
    /* len <= 32 */
    Node *t = root;
    for (unsigned i = 0; i < len; i++) {
        unsigned bit = (prefix >> (31 - i)) & 1u;
        if (!t->c[bit] && !(t->c[bit] = new_node())) return false;
        t = t->c[bit];
    }
    t->hop = hop;
    return true;
}
int trie_lookup(const Node *root, uint32_t addr) {
    int best = root->hop;  /* the default route /0, if any */
    const Node *t = root;
    for (unsigned i = 0; i < 32 && t; i++) {
        t = t->c[(addr >> (31 - i)) & 1u];
        if (t && t->hop >= 0) best = t->hop;   /* longer match found */
    }
    return best;  /* -1: no matching prefix */
}
```

Complexity. Insert O(p), lookup O(32), independent of the table size n. The memory is at most 32 nodes per prefix in the worst case, and typically far fewer because prefixes share their upper bits. A linear scan of all prefixes costs O(n) per lookup, which is the baseline this structure replaces.

Multibit tries. Reading eight bits per level (a *stride* of 8) needs only four steps for IPv4, at the cost of nodes with 256 children. Figure 1.15 draws such a trie for four prefixes. Real routers go further: they use compressed tries or TCAM hardware (the device internals are treated in Pillar 5, §5.2.8). The software alternative to a trie is one hash table per prefix length: probe the /32 table, then /31, and so on, down to /0, and keep the first hit; this costs up to 32 hash lookups but needs no pointer chasing.

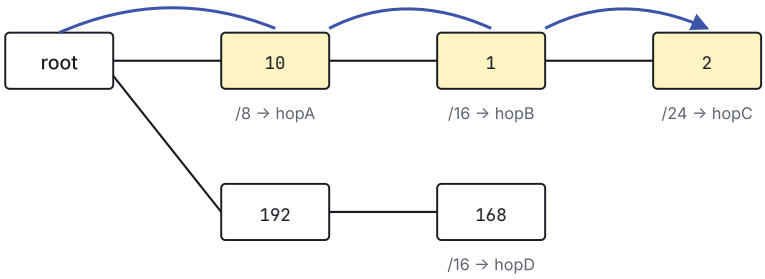


Figure 1.15 A stride-8 trie (one level per octet) for 10.0.0.0/8, 10.1.0.0/16, 10.1.2.0/24 and 192.168.0.0/16. The accent walk is the lookup of 10.1.2.77.

WORKED EXAMPLE

Lookup in a table of four prefixes and a default route. Routes: 10.0.0.0/8 → A, 10.1.0.0/16 → B, 10.1.2.0/24 → C, 192.168.0.0/16 → D, 0.0.0.0/0 → E.

1. Insert each prefix: the /8 path has 8 edges and marks hop A; the /16 path shares those 8 edges and extends by 8 more; the /24 extends by 8 more again.
2. Look up 10.1.2.77. The first 8 bits (10) reach the node marked A, the next 8 (1) reach the node marked B, the next 8 (2) reach the node marked C, and no further route exists. The last mark seen is C.
3. Look up 10.1.9.9. The walk reaches B after 16 bits; the third octet 9 = `00001001` leaves the path of 10.1.2.0/24 (third octet `00000010`) at its fifth bit. The last mark seen is B.
4. Look up 10.200.0.1 → A (leaves the 10.1 path at the ninth bit). Look up 172.16.0.1: the walk follows the first bit (1) shared with 192.168.0.0/16 and leaves that path at the second bit, having passed no mark, so only the default route E matches.

Address	Deepest mark on the walk	Next hop
10.1.2.77	/24	C
10.1.9.9	/16	B
10.200.0.1	/8	A
172.16.0.1	/0	E

Result: lookups touch at most 32 nodes whatever the table size; the answer is the last mark seen.

TRAP

Claim: stop the lookup at the first node that carries a next hop. *Why it is wrong:* a shorter prefix such as 10.0.0.0/8 is met before the longer 10.1.2.0/24, so stopping early returns the least specific route. *Correct statement:* keep walking while edges exist and remember the deepest marked node; return it when the walk ends.

Deleting a prefix clears the hop at its node and removes any chain of childless, unmarked nodes below the last useful ancestor. For text keys the same code applies with 26 children per node and

an `end` flag in place of `hop`; “implement a trie” and “maximum XOR of two numbers” (a binary trie of the numbers’ bits, queried greedily for the opposite bit) are solved in the Problem Set.

Choosing a lookup structure. The binary trie is the baseline; every other method trades memory, update cost or hardware for speed. A *path-compressed* trie (a Patricia or radix tree) merges chains of single-child nodes into one edge labelled with the skipped bits, so the number of nodes is proportional to the number of prefixes instead of the number of bits.

Method	Lookup	Memory	Updates	Where used
linear scan of prefixes	O(*n*)	O(*n*)	trivial	tiny tables, tests
binary trie	O(32) steps	up to 32 nodes per prefix	easy	teaching, software routers
path-compressed trie	O(32) steps, fewer nodes	O(*n*)	moderate	Linux and BSD routing tables
multibit trie (stride 8)	4 steps	large nodes, 256 pointers	prefix expansion	high-speed software
hash per prefix length	up to 33 probes	O(*n*)	easy	simple fast paths
TCAM	one clock cycle	expensive, power-hungry	slow, ordered by length	hardware line cards

> INTERVIEW LENS
>
> GOOGLE · CISCO · JUNIPER · ARISTA
>
> The reported form is: “given a list of IP prefixes and a target address, find the longest matching prefix; which data structure, and what is the lookup complexity?” A strong answer names the binary trie, states O(32) per lookup, explains why a hash set of prefixes needs up to 32 probes, and mentions that hardware uses TCAM. Follow-ups: deletion, memory use, updating the table while lookups continue, IPv6 (128 levels, hence compression).

> LAB
>
> `LAB 1.9 · 30 MIN` Build the trie above into a small lookup engine and measure it against a linear scan. Append this `main` to the code of this section (with `<stdio.h>` and `<time.h>`), save the result as `minifib.c`, compile with optimisation, and run.

```c
int main(void) {
    enum { N = 20000, Q = 1000000, L = 2000 };
    static uint32_t pre[N], qa[L]; static unsigned len[N];
    Node *root = new_node();
    srand(1);
    for (int i = 0; i < N; i++) {
        len[i] = 8 + rand() % 17;                          /* /8 .. /24 */
        pre[i] = ((uint32_t)rand() << 16) ^ (uint32_t)rand();
        pre[i] &= ~0u << (32 - len[i]);
        trie_insert(root, pre[i], len[i], i);
    }
    for (int i = 0; i < L; i++)
        qa[i] = ((uint32_t)rand() << 16) ^ (uint32_t)rand();

    clock_t t0 = clock(); long agree = 0;
    for (int q = 0; q < L; q++) {  /* linear scan */
        int best = -1; unsigned bl = 0;
        for (int i = 0; i < N; i++)
            if (len[i] >= bl &&
                (qa[q] >> (32 - len[i])) == (pre[i] >> (32 - len[i])))
                best = i, bl = len[i];
        agree += (best == trie_lookup(root, qa[q]));
    }
    double ls = (double)(clock() - t0) / CLOCKS_PER_SEC;
    printf("linear: %d lookups in %.3f s (%.2f M/s), trie agrees on %ld\n",
           L, ls, L / ls / 1e6, agree);

    t0 = clock(); long sum = 0;
    for (int q = 0; q < Q; q++)
        sum += trie_lookup(root,
                   ((uint32_t)rand() << 16) ^ (uint32_t)rand());
    double ts = (double)(clock() - t0) / CLOCKS_PER_SEC;
    printf("trie  : %d lookups in %.3f s (%.2f M/s)  check %ld\n",
           Q, ts, Q / ts / 1e6, sum);
    return 0;
}
```

```
$ gcc -O2 -Wall -o minifib minifib.c && ./minifib
linear: 2000 lookups in 0.105 s (0.02 M/s), trie agrees on 2000
trie  : 1000000 lookups in 0.133 s (7.52 M/s)  check 11280283566
```

What you should notice: the trie answers about 300 times faster than the scan, and the two agree on every query, which is the correctness check. The scan examines all N prefixes per lookup; the trie walks at most 32 nodes. Change `N` to 200000 and rerun: the scan becomes ten times slower, as its cost is proportional to N, while the trie slows only from roughly 7.5 to 1.6 million lookups per second on the machine used here, because a larger trie no longer fits in the processor cache and each step of the walk becomes a slow dependent memory load. The figures depend on the machine; the ratio and the trend are what to look for.

CHECK YOURSELF

1. How many nodes does a binary trie contain after inserting only 10.0.0.0/8? [1]
2. Why is lookup time independent of the number of prefixes? [2]

1.10 Heaps and priority queues SHOULD · ■■□

BIG IDEA

> A heap keeps the smallest (or largest) element at the front of a partially ordered array, so that the extreme can be read at once and replaced in logarithmic time, which suits "the best k so far".

A *binary min-heap* is a complete binary tree in which every parent is no larger than its children. Because the tree is complete, it is stored in an array with no pointers: the children of index *i* are `2i+1` and `2i+2`, and its parent is `(i-1)/2`.

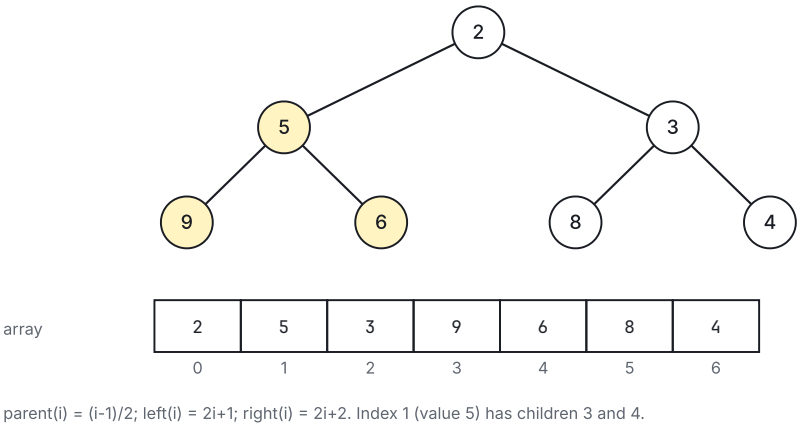


Figure 1.16 A min-heap as a tree and as the array that stores it. No pointers are needed because the shape is always a complete tree.

`push` appends at the end and swaps upward while smaller than the parent (sift-up); `pop` moves the last element to the root and swaps downward with the smaller child (sift-down). Each follows one root-to-leaf path, so both cost O(log *n*), and reading the minimum costs O(1). Building a heap from *n* elements by sifting down from the middle takes O(*n*), not O(*n* log *n*), because most nodes lie near the leaves.

Recognise: "k largest or smallest", "k most frequent", "merge k sorted", "median of a stream", "next event by time", "reorder packets by sequence number". In C++ `priority_queue<int>` is a max-heap; for a min-heap pass `greater<int>`. Python's `heapq` is a min-heap; negate values for a max-heap.

```
vector<int> topK(const vector<int>& a, int k) {  // the k largest, ascending
    priority_queue<int, vector<int>, greater<int>> h;  // min-heap of k
    for (int x : a) {
        h.push(x);
        if ((int)h.size() > k) h.pop();  // discard the smallest
    }
    vector<int> out;
    while (!h.empty()) { out.push_back(h.top()); h.pop(); }
    return out;
}
```

The heap holds the *k* best seen so far; its root is the weakest of them, which is the one to evict. For `[3, 1, 5, 12, 2, 11]` and *k* = 3 the heap evolves as {3}, {1,3}, {1,3,5}, then 12 arrives and 1 is evicted giving {3,5,12}, then 2 arrives and is itself evicted, then 11 evicts 3, giving {5,11,12}. The cost is O(*n* log *k*) time and O(*k*) space. For the median of a stream, a max-heap holds the lower half and a min-heap the upper half, rebalanced so that their sizes differ by at most one; the median is at one of the two roots (Problem Set).

The library hides the two moves; in C they are written by hand on a plain array, and an interviewer may ask for them.

```c
static void swap_int(int *a, int *b) { int t = *a; *a = *b; *b = t; }

static void sift_up(int *a, size_t i) {
    while (i && a[(i - 1) / 2] > a[i]) {
        swap_int(&a[(i - 1) / 2], &a[i]);
        i = (i - 1) / 2;
    }
}
static void sift_down(int *a, size_t n, size_t i) {
    for (;;) {
        size_t l = 2 * i + 1, r = l + 1, m = i;
        if (l < n && a[l] < a[m]) m = l;
        if (r < n && a[r] < a[m]) m = r;
        if (m == i) return;
        swap_int(&a[i], &a[m]);
        i = m;
    }
}
void heap_push(int *a, size_t *n, int v) {
    a[(*n)++] = v;
    sift_up(a, *n - 1);
}
int  heap_pop(int *a, size_t *n) {               /* requires *n > 0 */
    int top = a[0];
    a[0] = a[--*n];
    sift_down(a, *n, 0);
    return top;
}
```

Pushing 5, 3, 8, 1 gives `[5]`, `[3, 5]`, `[3, 5, 8]`, and then 1 is appended at index 3 and moves up past its parent 5 and past 3, leaving `[1, 3, 8, 5]`. A pop returns 1, moves the last element 5 to the root, and sifts it down: the smaller child is 3, so the heap becomes `[3, 5, 8]`. Repeatedly popping a heap built from an array sorts it in O(*n* log *n*) time and O(1) extra space; this is heapsort.

> **TRAP**
>
> *Claim:* use a max-heap of size *k* to keep the *k* largest values. *Why it is wrong:* the root of a max-heap is the largest element, but the element to evict is the smallest of the kept ones. *Correct statement:* keep the *k* largest in a min-heap, and the *k* smallest in a max-heap.

1.11 Graphs: networks are graphs MUST · ■■■

> **BIG IDEA**
>
> A network is a graph of nodes and links, so routing, loop detection and reachability become traversal, shortest-path and connectivity problems, each solved by one of five short templates.

A *graph* has *V* vertices and *E* edges; an edge may be directed, may carry a weight (a link cost or delay), and the graph may be disconnected. The usual representation is an *adjacency list*: for each vertex, the list of its neighbours (with weights). It takes O(*V* + *E*) space, which suits the sparse graphs of networks. An *adjacency matrix* takes O(V^2) space and answers “is there an edge” in O(1); it suits dense graphs and small *V*. The choice of algorithm follows the question.

1. Nine: the root and one node per bit of the prefix, eight in all.↩
2. The walk consumes one address bit per step and stops after at most 32 steps, whatever else the trie holds.↩

Question	Algorithm	Time
reachable? components? loop present?	DFS or BFS	O(V + E)
fewest hops (unweighted shortest path)	BFS	O(V + E)
cheapest path, non-negative weights	Dijkstra	O((V + E) log V)
negative weights, or at most k edges	Bellman–Ford	O(VE)
all pairs, $V \le 400$	Floyd–Warshall	O(V^3)
order tasks with dependencies	topological sort (Kahn)	O(V + E)
dynamic connectivity, "does this edge close a loop"	union–find	about O(α(n)) per operation

Graphs in C. Without `vector`, the compact form is a set of arrays: `head[u]` holds the index of the first outgoing edge of u, each edge stores its target, weight and the index of the next edge of the same vertex, and a new edge is pushed on the front of its list. Visiting the neighbours of u is then a walk along that chain.

```
#define MAXV 1000
#define MAXE 10000
static int head[MAXV], nxt[MAXE], to[MAXE], wt[MAXE], ecnt;

void g_init(int n) { for (int i = 0; i < n; i++) head[i] = -1; ecnt = 0; }
void add_edge(int u, int v, int w) {     /* directed edge */
    to[ecnt] = v; wt[ecnt] = w; nxt[ecnt] = head[u];
    head[u] = ecnt++;
}
/* for (int e = head[u]; e != -1; e = nxt[e]) { use to[e], wt[e]; } */
```

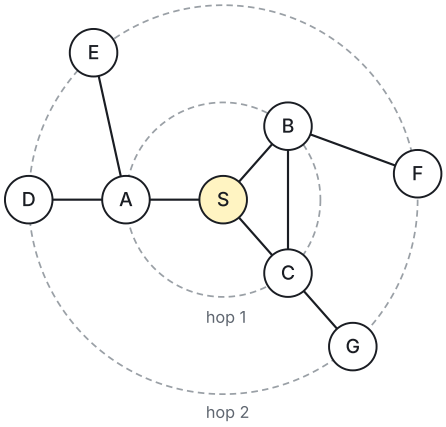
Modelling. Many problems are graph problems in disguise, and the skill is to name the vertices and edges. In snake and ladder the vertices are the board cells and the edges are the six throws; in a word ladder the vertices are words and an edge joins words that differ in one letter; in a maze the vertices are the free cells. When every edge costs one, BFS is the answer; when edges cost zero or one, a deque-based variant (0-1 BFS) pushes zero-cost edges at the front.

Breadth-first and depth-first search

BFS explores in rings of increasing hop count, using a queue, so the first time it reaches a vertex is by a shortest path. Marking a vertex visited when it is *pushed* (not when it is popped) keeps duplicates out of the queue. DFS follows one path as deep as it goes, using a stack, and is the tool for connectivity, cycles and ordering. The iterative form avoids the recursion limit (about 1000 in Python, and the stack size in C).

```cpp
vector<int> bfs(const vector<vector<int>>& g, int src) {
    // hop distance from src, -1 if unreachable
    vector<int> dist(g.size(), -1);
    queue<int> q; dist[src] = 0; q.push(src);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u])
            if (dist[v] < 0) {                      // mark on push
                dist[v] = dist[u] + 1; q.push(v);
            }
    }
    return dist;
}

vector<int> dfsOrder(const vector<vector<int>>& g, int s) {   // iterative
    vector<char> seen(g.size(), 0); vector<int> st{s}, order;
    while (!st.empty()) {
        int u = st.back(); st.pop_back();
        if (seen[u]) continue;
        seen[u] = 1; order.push_back(u);
        for (int i = (int)g[u].size() - 1; i >= 0; --i)  // reverse order
            if (!seen[g[u][i]]) st.push_back(g[u][i]);
    }
    return order;
}
```



queue order: S | A B C | D E F G. A vertex joins a ring when first reached.

Figure 1.17 Breadth-first search explores in rings of equal hop count, so the first time a vertex is reached is by a shortest path.

> **TRAP**
>
> *Claim:* a traversal needs no visited set if the graph "looks like a tree". *Why it is wrong:* a single cycle, or a second path to the same vertex, makes the traversal revisit vertices, and on a cycle it never ends. This is a broadcast storm in code form: a frame flooded around a switched loop is never removed. *Correct statement:* mark each vertex when first reached and skip marked vertices.

A grid is a graph whose vertices are cells and whose neighbours are the four adjacent cells; "number of islands" is the count of DFS starts, and "rotting oranges" and "distance to the nearest gate" are BFS started from all sources at once (multi-source BFS). These are in the Problem Set.

Dijkstra's algorithm

Idea. Keep a tentative distance to each vertex. Repeatedly settle the unsettled vertex with the smallest tentative distance, and relax its edges: if going through it improves a neighbour's distance, update it. Because every weight is non-negative, no later path can reach a settled vertex more cheaply, so a settled distance is final. A min-heap supplies the next vertex. Entries are never updated in place; an improved vertex is pushed again, and the older, larger entry is discarded when it surfaces. *Recognise:* "cheapest", "minimum time", "network delay", weights present and non-negative.

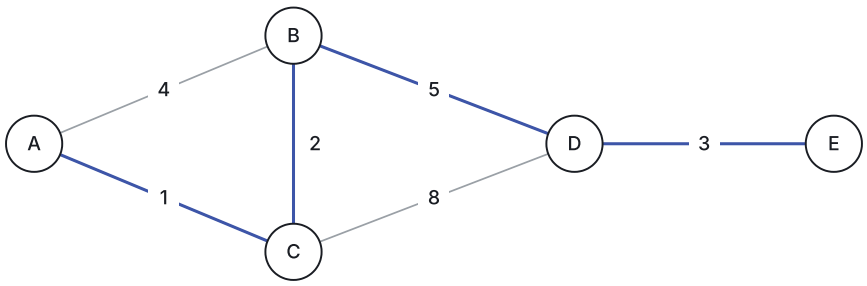


Figure 1.18 A weighted undirected graph (link costs). The accent path is the shortest route from A to E.

```
using Graph = vector<vector<pair<int, int>>>;  // g[u] = {(v, weight), ...}

vector<long long> dijkstra(const Graph& g, int src) {
    const long long INF = LLONG_MAX / 4;
    vector<long long> dist(g.size(), INF);
    using P = pair<long long, int>;
    priority_queue<P, vector<P>, greater<>> pq;
    dist[src] = 0; pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;  // stale entry
        for (auto [v, w] : g[u])
            if (d + w < dist[v]) { dist[v] = d + w; pq.push({dist[v], v}); }
    }
    return dist;
}
```

WORKED EXAMPLE

Shortest paths from A in Figure 1.18. Edges: A–B 4, A–C 1, C–B 2, B–D 5, C–D 8, D–E 3.

Pop (d, u)	Action	dist after (A B C D E)	Heap after
(0, A)	relax B = 4, C = 1	0 4 1 ∞ ∞	(1,C) (4,B)
(1, C)	B improves to 3; D = 9	0 3 1 9 ∞	(3,B) (4,B) (9,D)
(3, B)	D improves to 8	0 3 1 8 ∞	(4,B) (8,D) (9,D)
(4, B)	stale: 4 > 3, skipped	unchanged	(8,D) (9,D)
(8, D)	E = 11	0 3 1 8 11	(9,D) (11,E)
(9, D)	stale: 9 > 8, skipped	unchanged	(11,E)

(11, E)	no edges to improve	final	empty

Result: distances 0, 3, 1, 8, 11; the route to E is A–C–B–D–E. Without the stale check, the entries (4,B) and (9,D) would be expanded again, repeating work.

Complexity. Each edge causes at most one push, so the heap holds at most E entries and the total cost is $O((V + E) \log V)$. The algorithm is wrong when an edge weight is negative, because a settled vertex could later be improved.

TRAP

Claim: the line `if (d > dist[u]) continue;` is an optimisation that can be left out. *Why it is wrong:* the result stays correct, but each stale entry re-expands its vertex, and on dense graphs the running time grows from $O(E \log V)$ to $O(VE)$. *Correct statement:* always skip stale entries.

Bellman–Ford

Bellman–Ford relaxes every edge, $V - 1$ times. After round r, `dist[v]` is the cost of the best path using at most r edges, so stopping after k rounds answers "cheapest path with at most k edges", the hop-limited form that appears in "cheapest flights within k stops". Copying the array each round ensures that round r uses only distances from round $r - 1$. A further round that still improves some distance proves a negative cycle.

```
struct Edge { int u, v, w; };
vector<long long> bellmanFord(int n, const vector<Edge>& es,
                              int src, int rounds) {
    const long long INF = LLONG_MAX / 4;
    vector<long long> dist(n, INF); dist[src] = 0;
    for (int r = 0; r < rounds; ++r) {  // rounds <= n - 1
        vector<long long> nxt = dist;  // use last round's values only
        for (const Edge& e : es)
            if (dist[e.u] < INF && dist[e.u] + e.w < nxt[e.v])
                nxt[e.v] = dist[e.u] + e.w;
        dist = nxt;
    }
    return dist;
}
```

The table traces the synchronous rounds on a four-router chain with costs A–B 1, B–C 1, C–D 1 and a direct A–D link of cost 10. After round r, `dist[v]` is the cheapest cost using at most r links.

Round	dist[A]	dist[B]	dist[C]	dist[D]	What improved
0	0	∞	∞	∞	start
1	0	1	∞	10	B through A–B; D through the direct link
2	0	1	2	10	C through B
3	0	1	2	3	D through C replaces the direct link

The direct link is cheaper in hops but dearer in cost, which is the reason a hop-limited query (at most one link) answers 10 while the unrestricted answer is 3.

All pairs. Floyd–Warshall computes the cheapest path between every pair by allowing, one vertex at a time, that vertex as an intermediate stop. The loop over the intermediate vertex k must be

outermost, because `d[i][k]` and `d[k][j]` must already describe paths that use only the vertices before *k*.

```
void floyd(vector<vector<long long>>& d) {   // d[i][j]: cost or INF
    int n = d.size();
    const long long INF = LLONG_MAX / 4;
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (d[i][k] < INF && d[k][j] < INF)
                    d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
```

It runs in $O(V^3)$ time and $O(V^2)$ space, which limits it to about $V \le 400$.

Dijkstra and Bellman–Ford are also the two shortest-path methods of routing protocols. Link-state protocols such as OSPF and IS-IS give every router the whole graph, and each router runs Dijkstra locally; distance-vector protocols such as RIP run Bellman–Ford in a distributed way, each router relaxing the costs its neighbours advertise. The protocols are taught in the routing chapter; the comparison is a common interview bridge question, and this paragraph is its core.

Topological order and union–find

A *topological order* lists the vertices of a directed acyclic graph so that every edge goes forward; it is the order in which dependent tasks can run. Kahn's algorithm repeatedly removes a vertex of in-degree zero. If fewer than *V* vertices are output, a cycle exists.

```
vector<int> topo(int n, const vector<vector<int>>& g) {
    vector<int> indeg(n, 0), order;
    for (const auto& adj : g) for (int v : adj) ++indeg[v];
    queue<int> q;
    for (int i = 0; i < n; ++i) if (!indeg[i]) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop(); order.push_back(u);
        for (int v : g[u]) if (--indeg[v] == 0) q.push(v);
    }
    return order;  // size < n: a cycle exists
}
```

Union–find (disjoint-set union, DSU) maintains a partition of vertices into components under "add edge". `find` returns a component's representative, with path compression; `unite` merges two components, attaching the smaller to the larger. An edge whose endpoints already share a representative closes a loop, which is how a network with redundant links, or a spanning-tree protocol, is reasoned about.

```
struct DSU {
    vector<int> p, sz;
    explicit DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;  // already connected: edge closes a loop
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b];
        return true;
    }
};
```

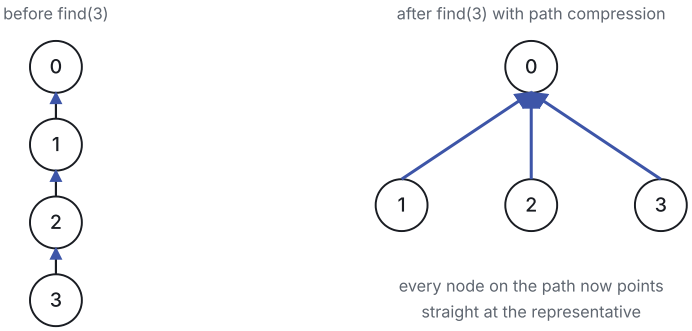


Figure 1.19 Path compression in union-find. Arrows are parent pointers; after one find, later finds on the same nodes take a single step.

With both optimisations each operation is O(α(*n*)), where α is the inverse Ackermann function, below 5 for any input that fits in memory. Kruskal's minimum spanning tree sorts the edges by weight and adds each edge whose `unite` returns true. Floyd–Warshall computes all pairs with three nested loops, `for k, for i, for j: d[i][j] = min(d[i][j], d[i][k] + d[k][j])`, and the loop over *k* must be outermost.

Minimum spanning tree. A *spanning tree* connects all vertices with $V - 1$ edges and no loop; a minimum one has the least total weight. Kruskal's algorithm sorts the edges and adds each edge whose endpoints are still in different components, which union–find decides.

```
long long kruskal(int n, vector<Edge> es) {   // Edge: see Bellman-Ford
    sort(es.begin(), es.end(),
         [](const Edge& a, const Edge& b) { return a.w < b.w; });
    DSU d(n);
    long long total = 0;
    for (const Edge& e : es)
        if (d.unite(e.u, e.v)) total += e.w;  // joins two components: keep
    return total;  // add a check for connectivity if needed
}
```

For the edges A–B 1, B–C 1, C–D 1, A–D 10 the sorted order is 1, 1, 1, 10; the three links of cost 1 are kept, and A–D is rejected because A and D are already joined, giving a tree of cost 3. The same loop-avoidance idea, applied to a switched network, is what a spanning-tree protocol does with its root election and blocked ports; those protocols are taught in the switching chapter.

> INTERVIEW LENS
>
> CISCO · GOOGLE · AMAZON · NOKIA
>
> Reported problems include the minimum dice throws of a snake-and-ladder board (BFS on 100 cells), depth-first search without recursion, the time for a signal to reach all routers (Dijkstra), links whose failure partitions the network (bridges), and the first link that creates a loop (union–find). A strong answer names the graph (what is a vertex, what is an edge), picks the algorithm from the table above, and states its complexity. If asked "why not union–find, why BFS", answering with the trade-off, not defending one tool, is what is scored.

> CHECK YOURSELF
>
> 1. Why must BFS mark a vertex when it is pushed rather than popped? [1]
> 2. What does Dijkstra do wrongly on a graph with a negative edge? [2]
> 3. In union–find, what does `unite` returning false mean? [3]

1.12 Recursion and backtracking SHOULD · ■■□

BIG IDEA

Backtracking builds an answer one choice at a time and undoes each choice after exploring it, so a single recursive function enumerates every candidate while the constraints prune the hopeless ones early.

A recursive function needs a *base case* that stops the recursion and a *step* that moves towards it. Backtracking adds a *path* that is extended before the recursive call and restored after it: choose, explore, un-choose. *Recognise:* "all combinations, permutations or subsets", "generate all", "place so that no two conflict", and small limits ($n \le 12$ for permutations, $n \le 20$ for subsets). The cost is the size of the search tree: 2^n for subsets, $n!$ for permutations.

```
void subsets(const vector<int>& a, int i, vector<int>& path,
             vector<vector<int>>& out) {
    if (i == (int)a.size()) { out.push_back(path); return; }   // copy
    subsets(a, i + 1, path, out);  // skip a[i]
    path.push_back(a[i]);                                       // choose
    subsets(a, i + 1, path, out);  // explore
    path.pop_back();  // un-choose
}
```

The two classic bugs are forgetting the un-choose step, which leaks choices into sibling branches, and (in Python) appending `path` itself rather than a copy `path[:]`, so that every stored answer changes as the path changes.

WORKED EXAMPLE

Restore IP addresses. Given a string of digits such as `25525511135`, return every way to insert three dots so that the result is a valid IPv4 address (four parts, each 0–255, no leading zeros).

1. The search has four levels. At each level we choose the length of the next part, 1 to 3 digits.
2. A part is rejected if it has a leading zero and length above one, or exceeds 255.
3. After four parts, the answer is valid only if all digits have been consumed.
4. Pruning by these rules leaves very few branches: at most $3^4 = 81$ leaves are ever reached.

```
void restore(const string& s, int pos, int parts, string& cur,
             vector<string>& out) {
    if (parts == 4) {
        if (pos == (int)s.size())                    // all digits used
            out.push_back(cur.substr(0, cur.size() - 1));   // drop last '.'
        return;
    }
    size_t keep = cur.size();
    for (int len = 1; len <= 3 && pos + len <= (int)s.size(); ++len) {
        string seg = s.substr(pos, len);
        if ((len > 1 && seg[0] == '0') || stoi(seg) > 255) continue;
        cur += seg + '.';  // choose
        restore(s, pos + len, parts + 1, cur, out);  // explore
        cur.resize(keep);  // un-choose
    }
}
```

Part lengths tried	Parts	Outcome
3, 3, 2, 3	255.255.11.135	all 11 digits used: accepted
3, 3, 3, 2	255.255.111.35	all 11 digits used: accepted
3, 3, 1, ?	255.255.1.?	the remaining 1135 has four digits, no length 1–3 uses them all: dead end
2, 2, 2, 3	25.52.55.111	four parts built but two digits are left over: rejected
2, 5, …	25.5... then 525	a part such as 511 exceeds 255 and is skipped

Result for `25525511135`: `255.255.11.135` and `255.255.111.35`. Call as `restore(s, 0, 0, cur, out)` with an empty `cur`.

Pruning in practice. The power of backtracking comes from rejecting a partial answer at once. In the *N*-queens problem we place one queen per row and must avoid shared columns and diagonals. A bitmask per constraint makes the test a single AND: `cols` marks the occupied columns, and the two diagonal masks shift by one place at each new row.

```
int nQueens(int n) {
    int count = 0;
    function<void(int,int,int,int)> go =
        [&](int row, int cols, int d1, int d2) {
        if (row == n) { ++count; return; }
        int free = ((1 << n) - 1) & ~(cols | d1 | d2);   // safe columns
        while (free) {
            int bit = free & -free;  // lowest safe column
            free -= bit;
            go(row + 1, cols | bit, (d1 | bit) << 1, (d2 | bit) >> 1);
        }
    };
    go(0, 0, 0, 0);
    return count;
}
```

The counts are 2 for $n = 4$ and 92 for $n = 8$. Without pruning the search would examine $8^8 \approx 1.7 \times 10^7$ placements; with it, only a few thousand partial boards are visited. The same shape (choose a candidate from a mask of the allowed ones, recurse, undo) solves Sudoku and word-search problems.

TRAP

Claim: Python's recursion handles a depth-first search of any size. *Why it is wrong:* CPython allows about 1000 nested calls by default, and a path graph of 10^5 vertices raises `RecursionError`; C overflows its stack in the same way. *Correct statement:* use an explicit stack for deep graphs, or raise the limit only when the depth is known and modest.

1.13 Sorting, binary search and intervals MUST · ■■□

BIG IDEA

Sorting costs O(n log n) once and then makes many questions cheap, because order lets us discard half the data at each comparison.

Comparison sorts cannot beat O(n log n); the table of algorithms is in the Revision Spread (the Revision Spread (p. 101)). In C, `qsort` takes a comparator that must return negative, zero or positive, and it must never be written as `return a - b`, which overflows for large values.

```
int cmp_int(const void *pa, const void *pb) {
    int a = *(const int *)pa, b = *(const int *)pb;
    return (a > b) - (a < b);  /* -1, 0 or 1; no overflow */
}
/* qsort(arr, n, sizeof arr[0], cmp_int); */
```

Merge sort, partition and quickselect

Two sorting ideas recur in interviews. *Merge sort* splits the array, sorts both halves and merges them, with the recurrence `T(n) = 2T(n/2) + O(n)` that gives O(n log n) in every case; it is stable and needs O(n) extra space, and its merge step is exactly the merge of two sorted lists. *Partitioning* rearranges an array around a pivot so that smaller values precede it and larger values follow; it is the step of quicksort, and it leads to *quickselect*, which finds the k-th smallest value without sorting everything.

```
void mergeSort(vector<int>& a, int l, int r,
               vector<int>& tmp) {                    // sorts [l, r)
    if (r - l < 2) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m, tmp);
    mergeSort(a, m, r, tmp);
    merge(a.begin() + l, a.begin() + m, a.begin() + m, a.begin() + r,
          tmp.begin() + l);
    copy(tmp.begin() + l, tmp.begin() + r, a.begin() + l);
}

int partition(vector<int>& a, int lo, int hi) {   // [lo, hi]; pivot index
    swap(a[hi], a[lo + rand() % (hi - lo + 1)]);   // random pivot
    int p = a[hi], i = lo;
    for (int j = lo; j < hi; ++j)
        if (a[j] < p) swap(a[i++], a[j]);
    swap(a[i], a[hi]);
    return i;
}
int quickselect(vector<int> a, int k) {  // k-th smallest, k from 0
    int lo = 0, hi = (int)a.size() - 1;
    while (true) {
        int p = partition(a, lo, hi);
        if (p == k) return a[p];
        if (p < k) lo = p + 1; else hi = p - 1;   // side holding k
    }
}
```

For `[7, 2, 9, 4, 1]` with $k = 1$ (the second smallest), a partition around the pivot 4 produces `[2, 1, 4, 7, 9]` with the pivot at index 2; since $2 > 1$ the search continues on the left part `[2, 1]`, whose partition places 1 at index 0 and 2 at index 1. Index 0 is below k, so the search narrows to index 1, which holds 2, and the answer is 2. Each partition discards a constant fraction of the data on average, so the work is $n + n/2 + n/4 + \ldots = O(n)$ on average, and $O(n^2)$ only for an unlucky sequence of pivots. When the keys are small integers, *counting sort* beats all of these: count the occurrences of each value in O($n + k$) and write the values out in order.

Binary search

Idea. In a sorted range, compare with the middle element and discard the half that cannot contain the target. We use one canonical form with a half-open range `[lo, hi)`, which finds the first index whose value is not less than *x*, termed the *lower bound*. Every variant (first occurrence, insertion point, first true of a monotone predicate) is this loop.

```
int lowerBound(const vector<int>& a, int x) {   // first a[i] >= x, or n
    int lo = 0, hi = (int)a.size();
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;  // not (lo + hi) / 2: avoids overflow
        if (a[mid] < x) lo = mid + 1; else hi = mid;
    }
    return lo;
}
```

The two classic bugs are the midpoint overflow in `(lo + hi) / 2` for ranges beyond 2^{30}, and an update such as `lo = mid`, which loops forever when `hi = lo + 1`. The loop runs $\lceil \log_2 n \rceil$ times.

Binary search on the answer. When the answer is a number and a yes/no test “is *x* enough?” is monotone (false up to a threshold, true after it), the same loop searches the range of possible answers. *Recognise:* “minimise the maximum”, “smallest capacity such that …”, “at most *D* days”.

WORKED EXAMPLE

Ship packages within D days. Packages of weights `1..10` must be shipped in order; a ship carries at most its capacity per day. Find the smallest capacity that ships everything in 5 days.

1. The capacity is at least the heaviest package (10) and at most the total (55).
2. For a trial capacity, load greedily and count days: a larger capacity never needs more days, so the test is monotone.
3. Search the range for the first capacity needing at most 5 days.

```
int shipWithinDays(const vector<int>& w, int days) {
    int lo = *max_element(w.begin(), w.end());
    int hi = accumulate(w.begin(), w.end(), 0);  // answer lies in [lo, hi]
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2, need = 1, load = 0;
        for (int x : w) {
            if (load + x > mid) { ++need; load = 0; }  // start the next day
            load += x;
        }
        if (need <= days) hi = mid; else lo = mid + 1;
    }
    return lo;
}
```

lo	hi	mid	days needed	action
10	55	32	2	hi = 32
10	32	21	3	hi = 21
10	21	15	5	hi = 15
10	15	12	6	lo = 13
13	15	14	6	lo = 15

> **Result: 15. Time O(*n* log(sum)).**

Intervals

Idea. Sorting intervals by start brings overlapping ones next to each other, so one pass can merge them: an interval overlaps the last merged interval when its start is at most that interval's end. *Recognise:* "merge", "overlapping", "meeting rooms", "insert interval", IP address ranges (as pairs of 32-bit integers).

```
vector<array<long long, 2>> mergeIntervals(vector<array<long long, 2>> v) {
    sort(v.begin(), v.end());  // by start, then end
    vector<array<long long, 2>> out;
    for (auto& iv : v) {
        if (!out.empty() && iv[0] <= out.back()[1])   // overlaps the last
            out.back()[1] = max(out.back()[1], iv[1]);
        else
            out.push_back(iv);
    }
    return out;
}
```

For `[1,3], [2,6], [8,10], [15,18]`: `[2,6]` starts at $2 \le 3$ and extends the first interval to `[1,6]`; `[8,10]` starts after 6 and is kept; `[15,18]` is kept. Whether `[1,2]` and `[3,4]` should merge (as integer ranges they are adjacent) or `[1,2]` and `[2,3]` should (they touch) is a question to ask; the code above merges touching intervals but not adjacent ones, and `iv[0] <= out.back()[1] + 1` merges adjacent integer ranges as well. Time O(*n* log *n*) for the sort.

> CHECK YOURSELF
>
> 1. Why does `lo = mid` in a half-open search risk an infinite loop? [4]
> 2. After sorting by start, why is it enough to compare with only the last merged interval? [5]

1.14 Dynamic programming essentials SHOULD · ■■■

> BIG IDEA
>
> Dynamic programming replaces an exponential recursion by a table, because the same subproblems recur and each need be solved only once.

Recognise: "number of ways", "minimum or maximum cost", "can we reach", and a choice at each step whose consequences depend only on a small *state*. The routine has five steps: (i) define the state (what `dp[i]` means in words); (ii) write the recurrence relating it to smaller states; (iii) set the base cases; (iv) choose an evaluation order in which dependencies come first; (v) read the answer from the right cell. Space often reduces to the last row or two.

Family	State	Recurrence (sketch)
linear	`dp[i]` best for prefix *i*	`dp[i] = f(dp[i-1], dp[i-2])` (stairs, house robber)
knapsack / coins	`dp[c]` for capacity or amount *c*	`dp[c] = min(dp[c - coin] + 1)`
subsequence	`dp[i][j]` for prefixes of two strings	match: `dp[i-1][j-1] + 1`; else max or min of neighbours (LCS, edit distance)

1. Otherwise a vertex reachable by two paths can be queued twice before either copy is popped, doing the work again.↩
2. It may settle a vertex with a distance that a later negative edge would have lowered, and it never revisits settled vertices.↩
3. The two endpoints were already connected, so the edge would close a loop.↩
4. With `hi = lo + 1`, `mid` equals `lo`, so the range never shrinks.↩
5. Every earlier merged interval ends before the last one starts, so a later interval cannot reach them.↩

grid	`dp[r][c]`	from the cell above and the cell to the left
increasing subsequence	`dp[i]` ending at *i*	`1 + max dp[j]` over smaller `a[j]`, j < i

WORKED EXAMPLE

Coin change. With coins `{1, 3, 4}`, find the fewest coins that sum to 6, or −1 if impossible.

1. State: `dp[a]` is the fewest coins for amount *a*.
2. Recurrence: `dp[a] = 1 + min over coins c ≤ a of dp[a - c]`.
3. Base: `dp[0] = 0`; unreachable amounts hold a large sentinel.
4. Fill *a* = 1 to 6 in increasing order, since `dp[a]` needs smaller amounts.

```
int coinChange(const vector<int>& coins, int amount) {
    const int INF = 1e9;
    vector<int> dp(amount + 1, INF); dp[0] = 0;
    for (int a = 1; a <= amount; ++a)
        for (int c : coins)
            if (c <= a && dp[a - c] + 1 < dp[a]) dp[a] = dp[a - c] + 1;
    return dp[amount] >= INF ? -1 : dp[amount];
}
```

a	0	1	2	3	4	5	6
dp[a]	0	1	2	1	1	2	2

Result: 2 (3 + 3). A greedy choice of the largest coin gives 4 + 1 + 1 = 3 coins, which shows why greedy is unsafe here. Time O(amount × coins), space O(amount).

Top-down and bottom-up. The recurrence can be evaluated in two ways: *top-down*, as the recursive function with a memo table that stores each result on first computation (the `fib` of §1.1 (p. 9)); or *bottom-up*, by filling a table in an order that respects the dependencies, as `coinChange` does. The results are the same; bottom-up avoids recursion depth and allows the space to be trimmed, while top-down is quicker to write when the order of evaluation is awkward.

The 0/1 knapsack. Given items with weights `w[i]` and values `v[i]` and a capacity *W*, choose items, each at most once, to maximise the total value. Let `dp[c]` be the best value achievable within capacity *c*. Processing the items one at a time, `dp[c] = max(dp[c], dp[c - w] + v)`. The loop over *c* must run *downwards*: it then reads `dp[c - w]` from before the current item was considered, so each item is used once. A loop running upwards would read a value that already includes the item, which is the unbounded knapsack and the coin-change case.

```
int knapsack(const vector<int>& w, const vector<int>& v, int W) {
    vector<int> dp(W + 1, 0);
    for (size_t i = 0; i < w.size(); ++i)
        for (int c = W; c >= w[i]; --c)  // downwards: each item once
            dp[c] = max(dp[c], dp[c - w[i]] + v[i]);
    return dp[W];
}
```

After item (w, v)	dp[0..7]
none	0 0 0 0 0 0 0 0
(1, 1)	0 1 1 1 1 1 1 1
(3, 4)	0 1 1 4 5 5 5 5

(4, 5)	0 1 1 4 5 6 6 9
(5, 7)	0 1 1 4 5 7 8 9

For capacity 7 the best value is 9, from the items of weight 3 and 4. Time O(*nW*), space O(*W*).

Two strings. For the longest common subsequence of strings *s* and *t*, let `d[i][j]` be the answer for their first *i* and *j* characters. Matching last characters extend the diagonal answer by one; otherwise the answer is the larger of dropping a character from either string.

```
int lcs(const string& s, const string& t) {
    int n = s.size(), m = t.size();
    vector<vector<int>> d(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; ++i)
        for (int j = 1; j <= m; ++j)
            d[i][j] = s[i-1] == t[j-1] ? d[i-1][j-1] + 1
                                       : max(d[i-1][j], d[i][j-1]);
    return d[n][m];
}
```

	(empty)	A	C	B	D
(empty)	0	0	0	0	0
A	0	1	1	1	1
B	0	1	1	2	2
C	0	1	2	2	2
D	0	1	2	2	3

For `ABCD` and `ACBD` the table ends in 3 (for example `ABD`). Time and space O(*nm*). The edit-distance table of the Problem Set has the same shape.

When counting *combinations* (order ignored), the loop over coins is the outer loop; when counting *sequences* (order matters), the loop over amounts is the outer loop. Swapping them changes the answer, and this is the standard knapsack bug. For two-string problems, edit distance and longest common subsequence are solved in the Problem Set with a two-dimensional table.

1.15 Design-style coding: caches and rate limiters MUST · ■■■

> BIG IDEA
>
> Design-style questions combine two structures so that each covers the other's weakness: a hash map gives O(1) lookup, a doubly linked list gives O(1) reordering, and a counter driven by a clock gives rate control without storing history.

LRU cache

A *least-recently-used* (LRU) cache holds at most *c* entries and, when full, evicts the entry that was used longest ago. Router software uses the idea for ARP and route caches. The interface is `get(key)`, which returns the value or −1 and marks the entry as most recent, and `put(key, value)`, which inserts or updates and evicts if over capacity. Both must be O(1).

No single structure does both jobs: a hash map finds an entry in O(1) but keeps no order, and a list keeps order but finds in O(*n*). Combining them works because the map stores, for each key,

the *address of its list node*; and a doubly linked list can unlink a node it has the address of in O(1), since the node knows both neighbours.

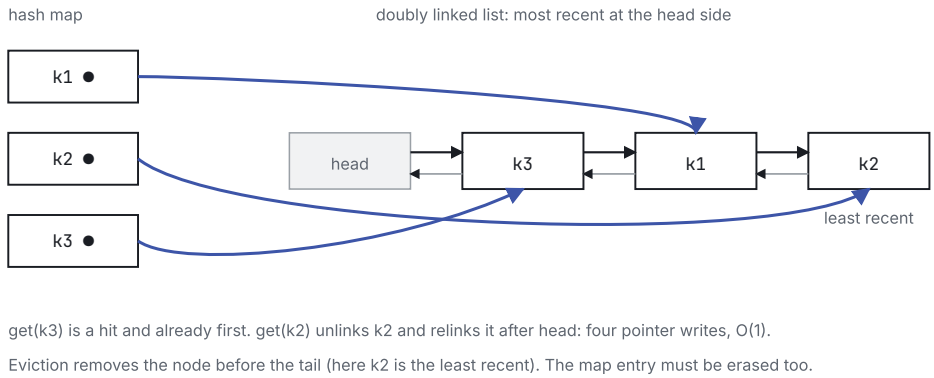


Figure 1.20 LRU cache: the map finds a node in O(1); the list orders nodes by recency and lets us unlink one in O(1) because the node holds both neighbours.

```
class LRU {
    using Item = pair<int, int>;                          // (key, value)
    size_t cap;
    list<Item> items;  // front = most recent
    unordered_map<int, list<Item>::iterator> pos;         // key -> node
public:
    explicit LRU(size_t c) : cap(c) {}
    int get(int k) {
        auto it = pos.find(k);
        if (it == pos.end()) return -1;
        items.splice(items.begin(), items, it->second);   // to front, O(1)
        return it->second->second;
    }
    void put(int k, int v) {
        if (cap == 0) return;
        auto it = pos.find(k);
        if (it != pos.end()) {  // update and refresh
            it->second->second = v;
            items.splice(items.begin(), items, it->second);
            return;
        }
        if (items.size() == cap) {  // evict the least recent
            pos.erase(items.back().first);  // erase from the map as well
            items.pop_back();
        }
        items.emplace_front(k, v);
        pos[k] = items.begin();
    }
};
```

WORKED EXAMPLE

Trace of a cache of capacity 2.

Operation	List after (front = most recent)	Result
put(1,1)	[1]	
put(2,2)	[2, 1]	
get(1)	[1, 2]	1

put(3,3)	[3, 1] (key 2 evicted)	
get(2)	[3, 1]	−1
put(4,4)	[4, 3] (key 1 evicted)	
get(1)	[4, 3]	−1
get(3)	[3, 4]	3
get(4)	[4, 3]	4

Result: every operation is O(1) on average: one hash lookup and at most four pointer writes.

TRAP

Claim: on eviction it is enough to remove the least recent node from the list. *Why it is wrong:* the map still holds a pointer to the freed node, so a later `get` of that key reads freed memory or reports a stale hit. *Correct statement:* every list change must be mirrored in the map: erase the key on eviction, and re-point it on insertion.

Variants. With a time-to-live (an ARP entry expires after a fixed time), store an expiry time in each node and treat an expired entry as a miss on `get`; entries expire lazily, or a sweep removes them from the tail end. For thread safety, a single mutex around `get` and `put` is correct because even `get` modifies the list; sharding the cache by key hash across several mutexes reduces contention. LFU (least frequently used) needs a second ordering and is solved in the Problem Set.

Token-bucket rate limiter

A *token bucket* limits the average rate to *r* tokens per second while allowing bursts of at most *B* tokens. The bucket holds up to *B* tokens, and tokens arrive at *r* per second. A packet of size *s* may pass if the bucket holds at least *s* tokens, which it then removes; otherwise the packet is dropped (policing) or delayed (shaping). In router terms, *r* is the committed information rate and *B* the committed burst size.

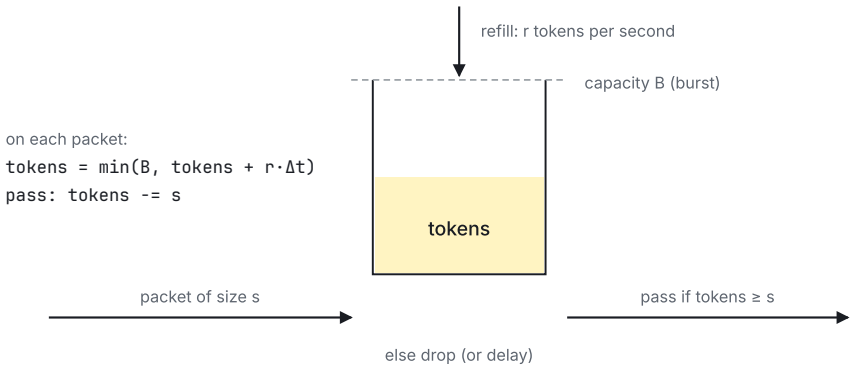


Figure 1.21 A token bucket. The long-run rate is r and the largest burst is B, because the level can never exceed the capacity.

We do not run a timer. Instead, on each packet we compute how many tokens would have arrived since the last update, add them, and cap the total at *B*. The state is only the current level and the last timestamp. Two details matter. First, time must come from a monotonic clock, because wall-clock time can jump backwards. Secondly, floating-point accumulation drifts, so we use integers: the level is stored in *micro-tokens* (tokens × 10^6), and with a clock in microseconds the refill over Δt µs is exactly `r × Δt` micro-tokens.

```c
#include <stdint.h>
#include <stdbool.h>

typedef struct {
    uint64_t rate;       /* tokens (bytes) per second           */
    uint64_t burst;      /* capacity B in tokens                */
    uint64_t level;      /* current level, in micro-tokens      */
    uint64_t last_us;    /* timestamp of the last update (µs)   */
} Bucket;

void tb_init(Bucket *b, uint64_t rate, uint64_t burst, uint64_t now_us) {
    b->rate = rate; b->burst = burst;
    b->level = burst * 1000000ull;                /* start full */
    b->last_us = now_us;
}
bool tb_allow(Bucket *b, uint64_t size, uint64_t now_us) {
    uint64_t cap = b->burst * 1000000ull;
    uint64_t dt = now_us - b->last_us;  /* monotonic: never negative */
    b->last_us = now_us;
    if (b->rate && dt > cap / b->rate)  /* long idle: bucket is full, and */
        b->level = cap;  /* r*dt could overflow otherwise   */
    else {
        b->level += b->rate * dt;
        if (b->level > cap) b->level = cap;
    }
    uint64_t need = size * 1000000ull;
    if (b->level < need) return false;  /* not enough tokens: drop */
    b->level -= need;
    return true;
}
```

WORKED EXAMPLE

Rate 1000 bytes per second, burst 1500 bytes. The bucket starts full (1500 tokens).

Time (s)	Packet (bytes)	Tokens before	Decision	Tokens after
0.0	1500	1500	pass	0
0.5	600	500	drop (500 < 600)	500
0.7	600	700	pass	100
2.7	1500	1500 (capped; 2100 earned)	pass	0

Result: the long-run rate is 1000 bytes per second and no burst exceeds 1500 bytes. Note that the dropped packet still advances the timestamp, so its interval is not credited twice.

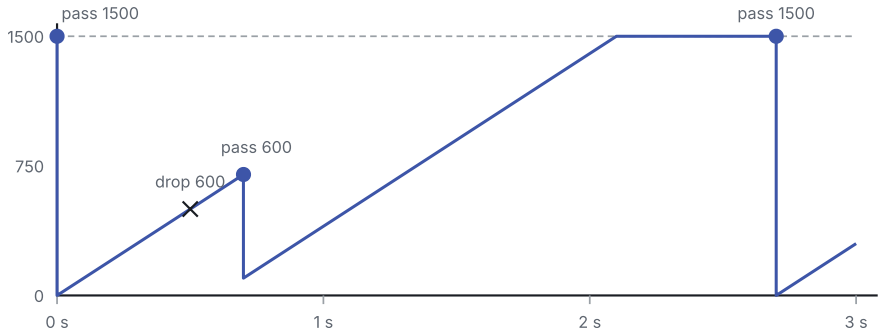


Figure 1.22 The level of the bucket in the worked example. The level rises at the refill rate, is cut off at the cap, and falls by the packet size when a packet passes; a dropped packet leaves it unchanged.

Scheme	Memory	Behaviour at the boundary	Use
fixed window counter	one counter	up to 2× the limit across a window edge	simplest; coarse
sliding window log	one timestamp per request	exact	strict limits, small limits
token bucket	two numbers	bursts up to *B*, exact average	traffic policing, API limits
leaky bucket	queue and a drain rate	smooth output, no bursts	traffic shaping

A token bucket with equal-sized packets can be run with a single stored time instead of a level. The *generic cell rate algorithm* keeps the theoretical arrival time `tat` of the next conforming packet: a packet is accepted if it is not earlier than `tat` by more than the burst allowance `burst_us`, and each accepted packet pushes `tat` forward by one packet interval.

```
bool gcra_allow(uint64_t *tat, uint64_t now,
                uint64_t interval, uint64_t burst) {
    uint64_t t = *tat > now ? *tat : now;      /* never in the past */
    if (t > now + burst) return false;         /* too early */
    *tat = t + interval;
    return true;
}
```

With `interval` = 1000 µs (1000 packets per second) and `burst` = 2000 µs, four packets arriving at time 0 give `tat` = 1000, 2000, 3000 and then a refusal: the first packet is on time, and the burst allowance admits two more, so three back-to-back packets pass, which is 1 + `burst` / `interval`. One 64-bit variable replaces the level and the timestamp, and no multiplication by the packet size is needed; for variable-size packets the interval becomes size / rate.

A fixed-window counter of five requests per ten seconds accepts five requests at second 9 and five more at second 11, ten in two seconds. A sliding log and a token bucket reject the excess; both are asked as follow-ups in the Problem Set.

> LAB
>
> `LAB 1.15 · 10 MIN` Test the token bucket on the trace of the worked example. Save the `Bucket` code of this section together with this driver as `tb.c`, and compile with the sanitisers

on, which catch overflow and invalid shifts.

```
#include <stdio.h>
int main(void) {
    Bucket b;
    tb_init(&b, 1000, 1500, 0);  /* 1000 bytes/s, burst 1500 bytes */
    struct { uint64_t t_us, size; } ev[] = {
        {0, 1500}, {500000, 600}, {700000, 600},
        {2700000, 1500}, {2700000, 1} };
    for (int i = 0; i < 5; i++)
        printf("t=%.1fs size=%4llu -> %s\n", ev[i].t_us / 1e6,
               (unsigned long long)ev[i].size,
               tb_allow(&b, ev[i].size, ev[i].t_us) ? "pass" : "drop");
    return 0;
}
```

```
$ gcc -fsanitize=address,undefined -g -o tb tb.c && ./tb
t=0.0s size=1500 -> pass
t=0.5s size= 600 -> drop
t=0.7s size= 600 -> pass
t=2.7s size=1500 -> pass
t=2.7s size=   1 -> drop
```

What you should notice: the second packet is dropped although the bucket held 500 tokens, and the third passes 0.2 s later; the fourth empties the bucket and the fifth, in the same instant, is dropped. Change the burst to 1000 and rerun: the first 1500-byte packet is dropped at once, because no packet larger than the bucket can ever pass.

INTERVIEW LENS

MICROSOFT · AMAZON · CISCO · JUNIPER

For the LRU cache the expected sequence is: hash map plus doubly linked list, the O(1) argument, code, then TTL and thread safety. For the rate limiter, interviewers listen for “token bucket”, the two parameters, refill-on-demand rather than a timer thread, the monotonic clock and the integer arithmetic. A strong answer also names the concurrency model and what happens at the boundary.

1.16 Finite-state machines SHOULD · ■■□

BIG IDEA

A finite-state machine replaces tangled conditionals by a table that gives the next state for every state and event, so the protocol logic can be read, checked and extended as data.

A *finite-state machine* (FSM) has a set of states, a set of events (or input symbols), a start state and a transition function from (state, event) to the next state; an event with no entry is illegal. Protocols such as TCP, link-state adjacency formation and DHCP are specified this way, and Juniper and Broadcom interviews ask for one in C. *Recognise:* “validate a sequence of events”, “parse a protocol”, “valid number” (a number grammar is an FSM over characters).

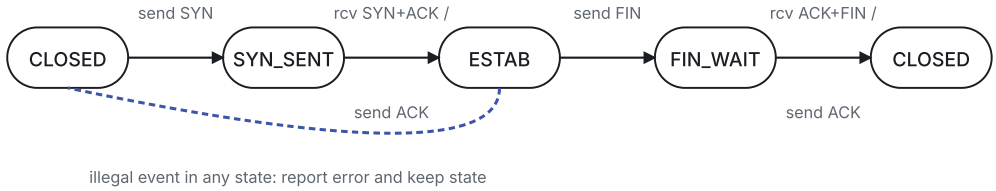


Figure 1.23 A reduced connection state machine (handshake and teardown only). A table indexed by state and event replaces nested conditionals.

A two-dimensional array indexed by state and event is the compact form. Entries of −1 mark illegal transitions; the driver loop reads the table and stops at the first illegal event.

```
#include <stddef.h>
#include <stdint.h>

typedef enum { CLOSED, SYN_SENT, ESTAB, FIN_WAIT, NSTATES } State;
typedef enum { E_SEND_SYN, E_RCV_SYNACK, E_SEND_FIN, E_RCV_FINACK,
               NEVENTS } Event;

static const int8_t next_state[NSTATES][NEVENTS] = {
    /* CLOSED   */ { SYN_SENT, -1,    -1,       -1     },
    /* SYN_SENT */ { -1,       ESTAB, -1,       -1     },
    /* ESTAB    */ { -1,       -1,    FIN_WAIT, -1     },
    /* FIN_WAIT */ { -1,       -1,    -1,       CLOSED },
};

/* Final state, or -1 if event *bad is illegal. */
int run_fsm(const Event *ev, size_t n, size_t *bad) {
    State s = CLOSED;
    for (size_t i = 0; i < n; i++) {
        int t = (ev[i] < NEVENTS) ? next_state[s][ev[i]] : -1;
        if (t < 0) { *bad = i; return -1; }
        s = (State)t;
    }
    return s;
}
```

For the events (send SYN, receive SYN+ACK, send FIN) the machine moves CLOSED → SYN_SENT → ESTAB → FIN_WAIT and returns FIN_WAIT. For (send SYN, send FIN) the second event is illegal in SYN_SENT, so the function returns −1 with `*bad = 1`. When actions must accompany transitions, the table cells hold function pointers instead of states, which is how production protocol code is written. Table size is *states* × *events*, so the transition lookup is O(1).

A byte-stream parser. An FSM is also the natural form of a parser that is fed one byte at a time, as a serial driver is. The framing used on many point-to-point links marks a frame boundary with the flag byte `0x7E`; a data byte equal to the flag or to the escape byte `0x7D` is sent as `0x7D` followed by the byte XOR `0x20`, so the flag never occurs inside a frame. Three states suffice: idle (hunting for a flag), body, and escaped (the next byte is stuffed).

```c
typedef struct {
    enum { IDLE, BODY, ESC } st;
    uint8_t buf[256];
    size_t len;
} Parser;

/* Feed one byte; returns 1 when a frame is in p->buf[0..p->len). */
int feed(Parser *p, uint8_t b) {
    switch (p->st) {
    case IDLE:
        if (b == 0x7E) { p->st = BODY; p->len = 0; }
        return 0;
    case BODY:
        if (b == 0x7E) { p->st = IDLE; return p->len > 0; }   /* end flag */
        if (b == 0x7D) { p->st = ESC; return 0; }
        break;
    case ESC:
        b ^= 0x20;
        p->st = BODY;
        break;
    }
    if (p->len == sizeof p->buf) p->st = IDLE;   /* overflow: resync */
    else p->buf[p->len++] = b;
    return 0;
}
```

For the stream `7E 01 7D 5E 02 7E` the parser enters BODY at the first flag, stores `01`, sees the escape `7D`, converts the next byte `5E` to `5E ^ 20 = 7E` and stores it, stores `02`, and on the closing flag returns 1 with the frame `01 7E 02`. Two flags in a row produce an empty frame, which `p->len > 0` filters out, and a missing flag costs one frame but not the following ones because the parser resynchronises at the next flag. The state variable, not a pile of flags, is what makes each of these cases a single line.

> CHECK YOURSELF
>
> 1. Why is a table preferable to nested `switch` statements for a protocol with 11 states? [1]
> 2. How does an FSM relate to “valid number”? [2]

1.17 Testing your own code and interview performance

MUST · ■□□

> BIG IDEA
>
> Code that has been traced by hand on one normal case and three edge cases is usually correct, and showing that trace is itself what the interviewer is assessing.

Before declaring a solution finished, one performs four checks in order. (i) *Dry-run*: execute the code on the worked example, line by line, keeping a small table of variable values, exactly as this chapter’s tables do. (ii) *Edge cases*: run the list in the Revision Spread (the Revision Spread (p. 101)): empty input, one element, duplicates, extremes of the value range, and for networking input the “/0”, “/32”, leading-zero and 256 cases. (iii) *Complexity*: state time and space, and justify them by pointing to the loop or the recursion. (iv) *Cross-check*: if time remains, compare against the brute force on small inputs.

In C, a tiny harness costs a minute and catches most mistakes:

```
#include <stdio.h>
static int fails;
#define CHECK(c) do { if (!(c)) { fails++; \
    printf("FAIL line %d: %s\n", __LINE__, #c); } } while (0)

int main(void) {
    uint32_t a;
    CHECK(parse_ipv4("10.0.0.1", &a) && a == 0x0A000001u);
    CHECK(!parse_ipv4("01.0.0.1", &a));
    CHECK(mask_of(0) == 0 && mask_of(32) == 0xFFFFFFFFu);
    printf(fails ? "%d failures\n" : "all passed\n", fails);
    return fails != 0;
}
```

Symptom	Likely cause	Where to look
crash on empty input	missing null or length guard	first lines of the function
off by one in the last element	< against <=, or a window not evicted	loop bounds, eviction index
works on small input, times out on large	hidden O(*n*) inside a loop (`strlen`, list `size`, `erase` at front)	the loop body
wrong answer only with duplicates	strict versus non-strict comparison	comparisons and the map update
garbage or a crash after `free`	use after free, or a stale pointer in a second structure	the deletion path
sign or overflow surprises	`int` where `long long` or unsigned is needed	sums, shifts, products

Communication. A round is scored on four things: the correctness of the code, the choice of algorithm, the testing, and the communication. The following habits cost nothing. State the brute force first and its cost, then the improvement and why it is valid. Name the invariant of each loop in a comment. When stuck, say what repeated work you are trying to remove. When given a hint, adopt it openly. Never code silently for more than two minutes, and never claim a complexity without pointing at the loop that produces it.

Minute (45-min round)	Step
0–5	restate, ask about size, duplicates, negatives, empty input; work one example
5–15	brute force, optimisation, and agreement on the plan
15–32	write code in small named pieces, narrating intent
32–38	dry-run and edge cases
38–45	complexity, trade-offs and follow-ups

WORKED EXAMPLE

What a good ten minutes sounds like. The question is “delete a node with a given value from a singly linked list, in C”. A concise version of the exchange follows; the candidate’s statements are the point, not the interviewer’s.

1. *Clarify.* “Is the list singly linked with a head pointer? May the value occur more than once? Should I delete the first occurrence or all? I will assume the first, and return the new

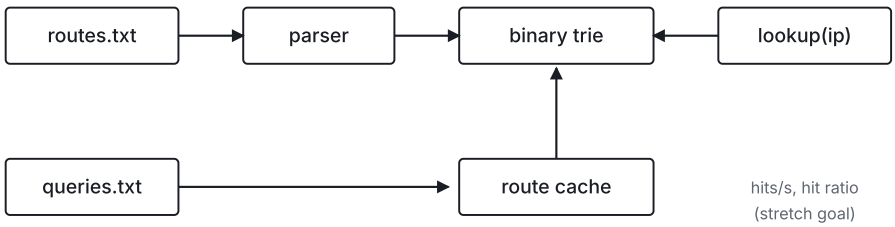
head."

2. *Example.* "For 3 → 5 → 7 and key 5 the result is 3 → 7. For key 3 the head changes, and for key 9 nothing changes."
3. *Brute force.* "Walk with a trailing pointer to the previous node; special-case the head. That is O(*n*) time and O(1) space, which is optimal, since every node may have to be examined."
4. *Improve.* "The head special case is easy to get wrong, so I will hold the address of the link to rewrite instead, a `Node **`."
5. *Code.* The five-line function of §1.6 (p. 27), narrated: "`pp` starts at the head variable; I advance it to the link whose target has the key; then I rewrite that link and free the node after saving it."
6. *Test.* "Empty list: `*pp` is null, nothing happens. One node, key present: the head variable becomes null. Key at the tail: the previous node's link becomes null. Key absent: unchanged."
7. *Evaluate.* "O(*n*) time, O(1) space. If we must delete all occurrences, I do not advance `pp` after a deletion."

Result: every step is stated aloud, the edge cases are named before the interviewer asks, and the follow-up is anticipated.

LAB

`LAB 1.17 · 2–3 DAYS` **Pillar project: a mini-FIB.** Build a longest-prefix-match lookup engine and measure it. The project joins the pieces of this pillar and gives a concrete talking point for an interview: "I built a forwarding table lookup and compared four methods."



stages 1-5: parser with tests, linear scan, trie, per-length hash, benchmark table; stage 6: LRU route cache

Figure 1.24 Architecture of the mini-FIB project. Queries are answered through the cache first, then the trie; the benchmark compares all lookup methods on the same inputs.

1. *Parser.* Read lines `prefix/len next_hop` with the validating parsers of §1.4 (p. 19); reject malformed lines with a line number. Write unit tests with the `CHECK` harness above, including /0, /32 and host bits set beyond the prefix length.
2. *Linear scan.* The baseline: for each query, test every prefix with the mask and keep the longest. It is the oracle for the later steps.
3. *Binary trie.* The code of §1.9 (p. 41); compare every answer with the oracle on a million random queries.
4. *Per-length hash.* The 33 hash tables of §1.9 (p. 41)'s alternative, probing from /32 downwards.
5. *Benchmark.* Generate a table of 10^4 to 10^6 prefixes with a realistic length mix (many /24, fewer /16 and /8) and report lookups per second for each method in a table.

6. *Stretch.* Put an LRU cache of recent destinations in front of the trie (§1.15 (p. 60)) and report the hit ratio; add prefix deletion with pruning.

What you should notice: the oracle catches every off-by-one in the masks, which is why it is written first; and the ranking of the methods depends on the table, because the hash method is fast when few prefix lengths occur and the trie when many do. Check the program with `valgrind --leak-check=full` or the sanitisers before showing it.

CHECK YOURSELF

1. What are the first two edge cases to try on a linked-list function? [3]
2. Why state the brute force before the optimised method? [4]

1. The table is data, so the full set of transitions can be reviewed at a glance and tested exhaustively; nested switches hide missing cases.↩
2. States record how much of a number has been read (sign, digits, point, exponent), and each character class is an event.↩
3. The empty list and the one-node list, followed by operating on the head and the tail.↩
4. It shows the problem is understood, gives a correct fallback, and makes the source of the improvement explicit.↩

Problem Set

Every question of this pillar is solved once, here. Difficulty is ■□□ (foundation), ■■□ (typical test) or ■■■ (hardest interviews). The tag **S** means that candidates report this specific question; **T** means that the topic is reported and the wording is ours. Round names are OA (online assessment), TECH-1 and TECH-2 (technical interviews) and HM (hiring manager). Code is C++ for general algorithms and C for networking and systems problems, and each solution is a complete function. The first table lists the problems whose full solutions are already in the teaching sections, so that nothing is solved twice.

Q	Problem	Tags	Solved in
Q1.1	Two Sum: indices of two values adding to a target	CISCO · AMAZON · OA · ■□□ · S	§1.1 (p. 9)
Q1.2	Longest substring without repeating characters	CISCO · OA · ■■□ · S	§1.2 (p. 12)
Q1.3	Number of subarrays with sum *k* (negatives allowed)	GOOGLE · MICROSOFT · TECH-1 · ■■□ · T	§1.2 (p. 12)
Q1.4	Longest consecutive sequence in O(*n*)	GOOGLE · TECH-1 · ■■□ · T	§1.3 (p. 16)
Q1.5	Validate a dotted-quad IPv4 address; parse a MAC address	CISCO · AMAZON · OA · ■■□ · T	§1.4 (p. 19)
Q1.6	Are two IPs in the same subnet (mask or prefix)?	CISCO · AMAZON · OA · ■□□ · T	§1.5 (p. 24)
Q1.7	Network, broadcast, first, last and host count of 10.20.30.40/22	CISCO · TECH-1 · ■■□ · T	§1.5 (p. 24)
Q1.8	Set, clear and toggle the *k*-th bit as macros	QUALCOMM · TECH-1 · ■□□ · S	§1.5 (p. 24)
Q1.9	Delete a node by value (head included); delete all matches	ARISTA · OA · ■■□ · S	§1.6 (p. 27)
Q1.10	Reverse a singly linked list	BROADCOM · ARISTA · TECH-1 · ■□□ · S	§1.6 (p. 27)
Q1.11	Merge two sorted linked lists	ARISTA · TECH-1 · ■□□ · S	§1.6 (p. 27)
Q1.12	Detect a cycle; find where it starts; prove it	JUNIPER · NOKIA · TECH-1 · ■■□ · S	§1.6 (p. 27)
Q1.13	Middle node of a linked list	NOKIA · TECH-1 · ■□□ · T	§1.6 (p. 27)
Q1.14	Circular buffer for a producer and a consumer: full versus empty	BROADCOM · QUALCOMM · ARISTA · TECH-1 · ■■□ · T	§1.7 (p. 33)
Q1.15	Balanced brackets	NOKIA · OA · ■□□ · S	§1.7 (p. 33)
Q1.16	Days until a warmer temperature (next greater element)	MICROSOFT · TECH-1 · ■■□ · T	§1.7 (p. 33)
Q1.17	BST insert, search, delete; the effect of sorted input	ARISTA · TECH-1 · ■■□ · S	§1.8 (p. 36)
Q1.18	Validate a BST; level-order traversal	ARISTA · GOOGLE · TECH-1 · ■■□ · S	§1.8 (p. 36)
Q1.19	Longest-prefix match of an IP against a list of prefixes	GOOGLE · TECH-1 · ■■■ · S	§1.9 (p. 41)
Q1.20	Shortest paths in a weighted network; link-state versus distance-vector	CISCO · GOOGLE · TECH-1 · ■■□ · T	§1.11 (p. 47)
Q1.21	Depth-first search without recursion	NOKIA · OA · ■■□ · S	§1.11 (p. 47)

Q1.22	Restore IP addresses from a digit string	AMAZON · MICROSOFT · TECH-1 · ■■□ · T	§1.12 (p. 54)
Q1.23	Binary search: lower bound; minimum capacity to ship in *D* days	GOOGLE · MICROSOFT · TECH-1 · ■■□ · T	§1.13 (p. 55)
Q1.24	Merge overlapping intervals	GOOGLE · AMAZON · TECH-1 · ■■□ · T	§1.13 (p. 55)
Q1.25	Coin change: fewest coins for an amount	MICROSOFT · GOOGLE · TECH-1 · ■■□ · T	§1.14 (p. 58)
Q1.26	LRU cache with O(1) `get` and `put`	MICROSOFT · AMAZON · TECH-1 · ■■□ · S	§1.15 (p. 60)
Q1.27	Token-bucket rate limiter and its router parameters	CISCO · JUNIPER · ARISTA · MICROSOFT · TECH-1 · ■■□ · T	§1.15 (p. 60)
Q1.28	Validate a sequence of TCP-style events against a state machine	JUNIPER · BROADCOM · TECH-1 · ■■□ · S	§1.16 (p. 65)

Arrays, hashing, strings and bits

Q1.29 Merge two sorted arrays in place

JUNIPER · OA · ■□□ · S

Array `a` holds `m` sorted values followed by `n` unused cells; array `b` holds `n` sorted values. Merge `b` into `a` with no extra array.

30-second answer: fill `a` from the back, taking the larger of the two current tails, so no unread value is overwritten.

Approach. Writing from the front would overwrite elements of `a` not yet read; writing from the back cannot, because the free cells are at the back. When `a` runs out, the rest of `b` is copied; when `b` runs out, the rest of `a` is already in place.

```
void mergeInto(vector<int>& a, int m,
               vector<int>& b, int n) {
  int i = m - 1, j = n - 1, k = m + n - 1;
  while (j >= 0)
    a[k--] = (i >= 0 && a[i] > b[j])
             ? a[i--] : b[j--];
}
```

Complexity. O($m + n$) time, O(1) space. **Follow-ups.** Stability (use `>` so equal values keep the order of `a` first). **Mistakes.** Looping while `i >= 0 && j >= 0` and forgetting to copy the remainder of `b`.

Q1.30 Rotate an array right by one, then by k

JUNIPER · TECH-1 · ■□□ · S

Rotate an array right by `k` positions in O(1) extra space.

30-second answer: reverse the whole array, then reverse the first `k` and the last `n-k` elements.

Approach. For one step, save the last element, shift the rest right, and store it at the front: O(n). For k steps, rotating one at a time costs O(nk). Three reversals do it in O(n): reversing everything puts the last k elements first but backwards, and reversing each part restores their order. Reduce k modulo n first.

```
void rotateRight(vector<int>& a, int k) {
  int n = a.size();
  if (!n) return;
  k %= n;
  reverse(a.begin(), a.end());
  reverse(a.begin(), a.begin() + k);
  reverse(a.begin() + k, a.end());
}
```

[1,2,3,4,5], $k = 2$: reversed 5,4,3,2,1; first two reversed 4,5,3,2,1; rest reversed 4,5,1,2,3, which is the right rotation. **Complexity.** O(n) time, O(1) space. **Mistakes.** Forgetting `k %= n` (and `n == 0`).

Q1.31 Move zeroes and remove an element in place

JUNIPER · AMAZON · OA · ■□□ · T

Move all zeroes of an array to the end, keeping the order of the other elements.

30-second answer: read/write pointers: copy each non-zero forward, then fill the tail with zeroes.

```
void moveZeroes(vector<int>& a) {
  size_t w = 0;
  for (size_t r = 0; r < a.size(); ++r)
    if (a[r]) a[w++] = a[r];
  while (w < a.size()) a[w++] = 0;
}
```

Removing a given value is the same loop with the test `a[r] != val`, returning `w`. **Complexity.** O(*n*) time, O(1) space, one write per kept element. **Mistakes.** Starting `w` at 1, or swapping elements needlessly (correct, but it does more writes).

Q1.32 Three Sum

GOOGLE · MICROSOFT · TECH-1 · ■■□ · T

Return all unique triples of an array whose sum is zero.

30-second answer: sort, fix the first value, and run opposite-end two pointers on the rest, skipping duplicates.

Approach. Brute force is O(n^3). After sorting, for each `i` the pair sum `-a[i]` is searched with two pointers in O(*n*), giving O(n^2). Uniqueness comes from skipping equal values at `i`, and, after a hit, at both pointers.

```
vector<vector<int>> threeSum(vector<int> a) {
  sort(a.begin(), a.end());
  vector<vector<int>> r;
  int n = a.size();
  for (int i = 0; i + 2 < n; ++i) {
    if (i && a[i] == a[i-1]) continue;
    int l = i + 1, h = n - 1;
    while (l < h) {
      long long s = (long long)a[i]
                  + a[l] + a[h];
      if (s < 0) ++l;
      else if (s > 0) --h;
      else {
        r.push_back({a[i], a[l], a[h]});
        ++l; --h;
        while (l < h && a[l] == a[l-1]) ++l;
      }
    }
  }
  return r;
}
```

Complexity. O(n^2) time, O(1) extra. **Mistakes.** Not skipping duplicates (repeated triples); using `int` for the sum.

Q1.33 Container with most water

GOOGLE · AMAZON · TECH-1 · ■■□ · T

Given heights `h[i]`, choose two lines maximising `min(h[i], h[j]) * (j - i)`.

30-second answer: start with the widest pair and always move the pointer at the shorter line, because only a taller line can improve the area.

Approach. Moving the taller pointer can only reduce the width and cannot raise the minimum height, so it never helps; moving the shorter one might.

```
int maxArea(const vector<int>& h) {
  int l = 0, r = h.size() - 1, best = 0;
  while (l < r) {
    best = max(best,
               min(h[l], h[r]) * (r - l));
    if (h[l] < h[r]) ++l; else --r;
  }
  return best;
}
```

Complexity. O(*n*) time, O(1) space.

Q1.34 Product of array except self

AMAZON · MICROSOFT · TECH-1 · ■■□ · T

*Return an array whose `i`-th element is the product of all others, without division and in O(*n*).*

30-second answer: the answer at `i` is (product of the elements before `i`) times (product of the elements after `i`); build the first in a forward pass and multiply in the second in a backward pass.

```
vector<int> productExceptSelf(
    const vector<int>& a) {
  int n = a.size();
  vector<int> r(n, 1);
  for (int i = 1; i < n; ++i)
    r[i] = r[i-1] * a[i-1];
  for (int i = n - 1, s = 1; i >= 0; --i) {
    r[i] *= s;
    s *= a[i];
  }
  return r;
}
```

Complexity. O(*n*) time, O(1) extra beyond the output. **Mistakes.** Using division fails when a value is zero. **Follow-ups.** The same prefix-and-suffix idea gives “trapping rain water”.

Q1.35 Maximum subarray (Kadane)

MICROSOFT · GOOGLE · OA · ■■□ · T

Find the largest sum of a non-empty contiguous subarray.

30-second answer: at each element, either extend the best subarray ending before it or start afresh; keep the maximum seen.

```
int maxSubarray(const vector<int>& a) {
  int cur = a[0], best = a[0];
  for (size_t i = 1; i < a.size(); ++i) {
    cur = max(a[i], cur + a[i]);
    best = max(best, cur);
  }
  return best;
}
```

For `[-2,1,-3,4,-1,2,1,-5,4]`: `cur` runs −2, 1, −2, 4, 3, 5, 6, 1, 5 and `best` is 6 (the subarray `4,-1,2,1`). **Complexity.** O(*n*) time, O(1) space. **Mistakes.** Initialising `best` to 0, which is wrong when every value is negative.

Q1.36 Group anagrams

AMAZON · GOOGLE · OA · ■■□ · T

Group the strings of a list that are anagrams of each other.

30-second answer: two strings are anagrams exactly when their sorted letters agree, so use the sorted string as the map key.

```
vector<vector<string>> groupAnagrams(
    const vector<string>& v) {
  unordered_map<string, vector<string>> m;
  for (auto& s : v) {
    string k = s;
    sort(k.begin(), k.end());
    m[k].push_back(s);
  }
  vector<vector<string>> r;
  for (auto& p : m) r.push_back(p.second);
  return r;
}
```

Complexity. O($N\,L \log L$) for N strings of length at most L. **Follow-ups.** For lowercase letters only, a 26-count signature as the key gives O(NL).

Q1.37 Extract IPs from a log and sort by frequency

GOOGLE · TECH-1 · ■□□ · S

Read a log, find every IPv4 address, and print the addresses ordered by how often they occur (most frequent first, ties by address).

30-second answer: split the input into tokens, keep those that parse as IPv4, count them in a hash map, then sort the (address, count) pairs.

Approach. Validation reuses `parse_ipv4` of §1.4 (p. 19). A single pass counts; the sort costs O(d log d) for d distinct addresses. A heap of size k replaces the sort when only the top k are wanted.

```
extern "C" bool parse_ipv4(const char*,
                           uint32_t*);
vector<pair<string,int>>
topIPs(istream& in) {
  unordered_map<string,int> cnt;
  string t; uint32_t x;
  while (in >> t)
    if (parse_ipv4(t.c_str(), &x)) ++cnt[t];
  vector<pair<string,int>> v(cnt.begin(),
                             cnt.end());
  sort(v.begin(), v.end(),
       [](auto& a, auto& b) {
    return a.second != b.second
         ? a.second > b.second
         : a.first < b.first; });
  return v;
}
```

Complexity. O($N + d \log d$) time, O(d) space. **Mistakes.** Counting `10.0.0.1:80` or `10.0.0.1,` as a different token; real logs need a scan for the dotted pattern rather than whitespace tokens.

Q1.38 Design a hash map in C

ARISTA · JUNIPER · TECH-1 · ■■□ · T

Implement `put`, `get` *and* `remove` *for integer keys without a library.*

30-second answer: an array of bucket heads, each a linked list; the bucket is `key % size`, and collisions share a list.

```
#define B 1009
typedef struct E { int k, v;
                   struct E *next; } E;
static E *tab[B];

static E **slot(int k) {  /* link */
  E **pp = &tab[(unsigned)k % B];
  while (*pp && (*pp)->k != k)
    pp = &(*pp)->next;
  return pp;
}
void put(int k, int v) {
  E **pp = slot(k);
  if (*pp) { (*pp)->v = v; return; }
  E *e = malloc(sizeof *e);
  if (!e) return;
  e->k = k; e->v = v; e->next = NULL;
  *pp = e;
}
int get(int k, int *out) {
  E *e = *slot(k);
  if (!e) return 0;
  *out = e->v; return 1;
}
void remove_key(int k) {
  E **pp = slot(k);
  if (*pp) {
    E *d = *pp; *pp = d->next; free(d);
  }
}
```

Complexity. O(1) average, O(*n*) worst. **Follow-ups.** Resize when the load factor exceeds one; a prime size spreads keys evenly for simple moduli. The pointer-to-pointer idiom of §1.6 (p. 27) removes the head special case.

Q1.39 Classify a string as IPv4, IPv6 or neither

CISCO · AMAZON · MICROSOFT · TECH-1 · ■■□ · T

Return which kind of address a string is. IPv6 here means eight groups of one to four hexadecimal digits separated by colons (no `::` *compression).*

30-second answer: choose the parser by the separator, then validate group by group, rejecting empty groups, too many digits and trailing text.

Approach. IPv4 uses `parse_ipv4` of §1.4 (p. 19). IPv6 reads, for each of eight groups, one to four hex digits and then a colon (not after the last group), and requires the end of the string.

```
#include <ctype.h>
bool valid_ipv6(const char *s) {
  for (int g = 0; g < 8; g++) {
    int n = 0;
    while (isxdigit((unsigned char)*s)) {
      s++; n++;
    }
    if (n < 1 || n > 4) return false;
    if (g < 7 && *s++ != ':') return false;
  }
  return *s == '\0';
}
```

Classification: contains `.` and no `:` → test IPv4; contains `:` → test IPv6; otherwise neither. **Complexity.** O(*n*). **Mistakes.** Accepting `01.1.1.1` or `1.1.1.1.`; letters beyond `f`; groups of five digits. Real parsers such as `inet_pton` also accept `::`, which needs a count of groups on each side.

Q1.40 String to integer (atoi) with clamping

QUALCOMM · BROADCOM · OA · ■■□ · T

Skip leading spaces, read an optional sign and digits, stop at the first other character, and clamp to the `int` range.

30-second answer: accumulate in a 64-bit value, stop growing once it passes the limit, apply the sign and clamp.

```
#include <limits.h>
int my_atoi(const char *s) {
  while (*s == ' ') s++;
  int neg = 0;
  if (*s == '+' || *s == '-')
    neg = (*s++ == '-');
  long long v = 0;
  while (*s >= '0' && *s <= '9') {
    v = v * 10 + (*s++ - '0');
    if (v > 2147483648LL) break;
  }
  if (neg) v = -v;
  if (v > INT_MAX) return INT_MAX;
  if (v < INT_MIN) return INT_MIN;
  return (int)v;
}
```

Complexity. O(*n*). **Mistakes.** Overflowing the accumulator before checking; treating `-` followed by a space as a number; mishandling `"-2147483648"`, which is exactly `INT_MIN`.

Q1.41 Compare version numbers

CISCO · AMAZON · OA · ■■□ · T

Compare `"1.01"` with `"1.001"`: dot-separated revisions are compared numerically from the left; missing revisions count as zero.

30-second answer: walk both strings at once, parse one number from each up to the next dot, and compare.

```
int compareVersion(const string& a,
                   const string& b) {
  size_t i = 0, j = 0;
  while (i < a.size() || j < b.size()) {
    long x = 0, y = 0;
    while (i < a.size() && a[i] != '.')
      x = x * 10 + (a[i++] - '0');
    while (j < b.size() && b[j] != '.')
      y = y * 10 + (b[j++] - '0');
    if (x != y) return x < y ? -1 : 1;
    ++i; ++j;
  }
  return 0;
}
```

Complexity. O(*n* + *m*) time, O(1) space. **Mistakes.** Comparing revisions as strings (so `"9"` > `"10"`); splitting into arrays when a scan suffices.

Q1.42 Expressive words (stretchy strings)

CISCO · OA · ■■□ · S

A word is stretchy to `S` if repeated letter groups in the word can be extended to length at least three to give `S`. Count the stretchy words in a list.

30-second answer: compare run-length groups: each pair of groups has the same letter, and the group in `S` either equals the word's group or is longer and has length at least three.

```
bool stretchy(const string& s,
              const string& w) {
  size_t i = 0, j = 0;
  while (i < s.size() && j < w.size()) {
    if (s[i] != w[j]) return false;
    size_t i2 = i, j2 = j;
    while (i2 < s.size() && s[i2] == s[i])
      ++i2;
    while (j2 < w.size() && w[j2] == w[j])
      ++j2;
    size_t a = i2 - i, b = j2 - j;
    if (a < b || (a != b && a < 3))
      return false;
    i = i2; j = j2;
  }
  return i == s.size() && j == w.size();
}
```

For `S = "heeellooo"`, `"hello"` is stretchy (e: 3 ≥ 3, l: 2 = 2, o: 3 ≥ 3) but `"hi"` is not. **Complexity.** O(|`S`| + |word|) per word. **Mistakes.** Allowing a group of size two to stretch (it must already match or reach three).

Q1.43 Longest common prefix of strings

GOOGLE · AMAZON · OA · ■□□ · T

Return the longest string that begins every string of a list.

30-second answer: compare column by column against the first string and stop at the first mismatch.

```
string lcp(const vector<string>& v) {
  if (v.empty()) return "";
  for (size_t i = 0; i < v[0].size(); ++i)
    for (auto& s : v)
      if (i == s.size() || s[i] != v[0][i])
        return v[0].substr(0, i);
  return v[0];
}
```

Complexity. O(total characters). **Follow-ups.** For two IPv4 addresses the common prefix in *bits* is `clz(a ^ b)` (with 32 when equal), the shape of the CIDR block that covers both. A trie handles many strings with repeated queries.

Q1.44 Count the bits to flip to turn A into B

QUALCOMM · TECH-1 · ■□□ · S

How many bit positions differ between two integers?

30-second answer: XOR marks the differing positions; count the set bits of the result.

```
unsigned flips(uint32_t a, uint32_t b) {
  uint32_t x = a ^ b;
  unsigned c = 0;
  while (x) { x &= x - 1; c++; }
  return c;
}
```

`flips(10, 20)`: 01010 ^ 10100 = 11110, four set bits, so four flips. **Complexity.** O(number of differing bits). **Follow-ups.** The Hamming distance of two addresses; `__builtin_popcount` as the one-line form.

Q1.45 Multiply without the * operator

JUNIPER · TECH-1 · ■■□ · S

Multiply two integers using only shifts, additions and tests.

30-second answer: shift-and-add: for each set bit of the multiplier, add the shifted multiplicand.

```
unsigned mul(unsigned a, unsigned b) {
  unsigned r = 0;
  while (b) {
    if (b & 1) r += a;
    a <<= 1;
    b >>= 1;
  }
  return r;
}
```

`mul(6, 5)`: b = 101 adds 6 (b bit 0) and 6 << 2 = 24 (bit 2): 30. **Complexity.** O(bit-width) = O(32). **Follow-ups.** For signed operands, multiply the absolute values and apply the sign (the sign is the XOR of the signs). Also ask: what if the result overflows?

Q1.46 Power of two, and counting bits for 0 to n

QUALCOMM · BROADCOM · OA · ■□□ · T

Test whether x* is a power of two in one expression. Then return the popcount of every number from 0 to *n.**

30-second answer: a power of two has exactly one set bit, so `x && !(x & (x-1))`; for the table, `bits[i] = bits[i >> 1] + (i & 1)`.

```
bool isPow2(unsigned x) {
  return x && !(x & (x - 1));
}
vector<int> countBits(int n) {
  vector<int> b(n + 1, 0);
  for (int i = 1; i <= n; ++i)
    b[i] = b[i >> 1] + (i & 1);
  return b;
}
```

Complexity. O(1) and O(*n*). **Mistakes.** Omitting `x &&`, which wrongly accepts zero.

Q1.47 Reverse the bits of a 32-bit integer

QUALCOMM · BROADCOM · TECH-1 · ■■□ · T

Reverse the bit order of a `uint32_t`. *Make it fast for a million calls.*

30-second answer: a loop over 32 bits works; for many calls, precompute a 256-entry byte-reversal table and swap the four bytes.

```
static uint8_t rev8[256];
void init_rev(void) {
  for (int i = 0; i < 256; i++)
    rev8[i] = (rev8[i >> 1] >> 1)
              | ((i & 1) << 7);
}
uint32_t rev32(uint32_t x) {
  return (uint32_t)rev8[x & 255] << 24
       | (uint32_t)rev8[(x >> 8) & 255] << 16
       | (uint32_t)rev8[(x >> 16) & 255] << 8
       | rev8[x >> 24];
}
```

The table recurrence reverses `i` by reversing `i >> 1`, moving it down one place and putting the low bit of `i` on top. **Complexity.** O(1) per call after an O(256) set-up. **Follow-ups.** The mask-and-swap ladder (swap halves, then pairs, then nibbles, bytes) needs no table.

Q1.48 Single number (every other value appears twice, or thrice)

GOOGLE · QUALCOMM · OA · ■■□ · T

In an array where every value appears exactly twice except one, find the odd one out. Then solve it when the others appear three times.

30-second answer: twice: XOR everything, since equal values cancel. Thrice: keep, per bit, a count modulo 3 using two accumulators.

```
int single2(const vector<int>& a) {
  int x = 0;
  for (int v : a) x ^= v;
  return x;
}
int single3(const vector<int>& a) {
  int ones = 0, twos = 0;
  for (int v : a) {
    ones = (ones ^ v) & ~twos;
    twos = (twos ^ v) & ~ones;
  }
  return ones;
}
```

In `single3`, `ones` holds the bits seen once (mod 3) and `twos` those seen twice; a third occurrence clears both. **Complexity.** O(*n*) time, O(1) space.

Q1.49 Bitwise AND of a range

JUNIPER · GOOGLE · TECH-1 · ■■□ · T

Return the AND of all integers in [`m` , `n`].

30-second answer: the answer is the common binary prefix of `m` and `n`, followed by zeros; shift both right until they are equal, then shift back.

```
int rangeAnd(int m, int n) {
  int s = 0;
  while (m != n) { m >>= 1; n >>= 1; ++s; }
  return m << s;
}
```

Any bit position that differs between `m` and `n` takes both values somewhere in the range, so it ANDs to zero, along with every lower bit. This is the same idea as the common prefix of two IP addresses. **Complexity.** O(32).

Q1.50 Add without +; divide without /

JUNIPER · QUALCOMM · TECH-1 · ■■□ · T

Add two integers with bit operations only. Then divide two integers without `*`, `/` or `%`.

30-second answer: addition is XOR (sum without carries) plus AND shifted left (carries), repeated until no carry remains; division is shift-and-subtract from the highest bit down.

```
int add(int a, int b) {
  unsigned x = a, y = b;
  while (y) {
    unsigned c = (x & y) << 1;
    x ^= y;
    y = c;
  }
  return (int)x;
}
```

```
int divide(int a, int b) {
  if (a == INT_MIN && b == -1)
    return INT_MAX;
  bool neg = (a < 0) != (b < 0);
  unsigned long long x = llabs((long long)a),
                     y = llabs((long long)b),
                     q = 0;
  for (int i = 31; i >= 0; --i)
    if ((x >> i) >= y) {
      x -= y << i;
      q |= 1ull << i;
    }
  return (int)(neg ? -(long long)q
                   : (long long)q);
}
```

Complexity. O(32) each. **Mistakes.** Negating `INT_MIN` in 32-bit arithmetic (undefined); not handling `INT_MIN / -1`, whose true result does not fit.

Q1.51 Convert an IP range to the fewest CIDR blocks

CISCO · GOOGLE · TECH-2 · ■■■ · T

Given a start address and a count `n`, cover exactly those `n` consecutive addresses with the fewest CIDR blocks.

30-second answer: repeatedly take the largest block that is aligned at the current address and no bigger than what remains.

Approach. A block of size 2^k must start at a multiple of 2^k, so the largest allowed size at address `a` is its lowest set bit, `a & -a` (the whole space when `a` is zero). Shrink the size by halves until it fits in `n`, emit the block with prefix length $32 - k$, and advance.

```
vector<pair<uint32_t,int>> rangeToCidr(
    uint32_t a, uint64_t n) {
  vector<pair<uint32_t,int>> out;
  while (n > 0) {
    uint64_t size =
        a ? (a & -a) : (1ull << 32);
    while (size > n) size >>= 1;
    int len = 32 - __builtin_ctzll(size);
    out.push_back({a, len});
    a += size;
    n -= size;
  }
  return out;
}
```

For 255.0.0.7 and $n = 10$: block 255.0.0.7/32 (alignment 1), then 255.0.0.8/29 (alignment 8, $8 \le 9$), then 255.0.0.16/32 (alignment 16 shrunk to 1). **Complexity.** O(32) blocks at most, each O(32). **Mistakes.** Choosing the largest power of two below `n` without the alignment test, which yields blocks that are not valid prefixes.

Q1.52 The Internet checksum

CISCO · JUNIPER · BROADCOM · TECH-1 · ■■□ · T

Compute the 16-bit one's-complement checksum of a byte buffer, as used by IPv4, ICMP, UDP and TCP.

30-second answer: add the data as 16-bit big-endian words, fold the carries back into the low 16 bits, and complement the result.

```
uint16_t checksum(const uint8_t *p,
                  size_t n) {
  uint32_t sum = 0;
  for (; n > 1; p += 2, n -= 2)
    sum += (uint32_t)p[0] << 8 | p[1];
  if (n) sum += (uint32_t)p[0] << 8;
  while (sum >> 16)
    sum = (sum & 0xFFFF) + (sum >> 16);
  return (uint16_t)~sum;
}
```

For the bytes `45 00 00 73`, the words are 0x4500 and 0x0073, the sum is 0x4573 and the checksum is 0xBA8C. A receiver that sums the data including the checksum obtains 0xFFFF, which is the test. **Complexity.** O(n). **Mistakes.** Forgetting the odd trailing byte (padded with a zero low byte); folding the carry only once; reading the words in host byte order on a little-endian machine.

Q1.53 Allocate VLAN identifiers with a bitmap

ARISTA · BROADCOM · TECH-1 · ■■□ · T

Manage the 4096 VLAN IDs: allocate the lowest free one, free one, in constant time per word.

30-second answer: a bitmap of 64 words; find a word that is not all ones, take its lowest zero bit with a count-trailing-zeros instruction.

```
#define NW (4096 / 64)
static uint64_t used[NW];
int alloc_vlan(void) {
  for (int w = 0; w < NW; w++)
    if (~used[w]) {
      int b = __builtin_ctzll(~used[w]);
      used[w] |= 1ull << b;
      return w * 64 + b;
    }
  return -1;
}
void free_vlan(int id) {
  used[id / 64] &= ~(1ull << (id % 64));
}
```

Complexity. O(64) per allocation, O(1) per free. **Follow-ups.** Reserve IDs 0 and 4095 by presetting their bits; a two-level summary bitmap makes allocation O(1). **Mistakes.** Shifting a 32-bit `1` (use `1ull`).

Linked lists, stacks and queues

Q1.54 Intersection of two linked lists

NOKIA · TECH-1 · ■□□ · S

Two singly linked lists may merge into a common tail. Return the first shared node, or `NULL`*, in O(1) extra space.*

30-second answer: let each pointer walk its own list and then the other; both cover the same total length, so they meet at the intersection, or both reach `NULL` **together.**

```
Node *intersect(Node *a, Node *b) {
  Node *p = a, *q = b;
  while (p != q) {
    p = p ? p->next : b;
    q = q ? q->next : a;
  }
  return p;
}
```

If the lists have x and y private nodes and z shared ones, each pointer travels $x + y + z$ nodes before the meeting point, so the shared tail aligns. **Complexity.** O($m + n$) time, O(1) space. **Mistakes.** Comparing values instead of node addresses.

Q1.55 Remove duplicates from a sorted list, linear and circular

ARISTA · TECH-1 · ■■□ · S

Remove repeated values from a sorted singly linked list. Then do it for a sorted circular list.

30-second answer: compare each node with its successor and unlink the successor while they are equal; for the circular list, stop when the walk returns to the head and finally compare the last node with the head.

```
Node *dedup(Node *h) {
  for (Node *p = h; p && p->next; ) {
    if (p->val == p->next->val) {
      Node *d = p->next;
      p->next = d->next;
      free(d);
    } else p = p->next;
  }
  return h;
}
void dedup_circ(Node *h) {
  if (!h) return;
  Node *p = h;
  while (p->next != h) {
    if (p->val == p->next->val) {
      Node *d = p->next;
      p->next = d->next;
      free(d);
    } else p = p->next;
  }
  if (p != h && p->val == h->val) {
    Node *q = h;
    while (q->next != p) q = q->next;
    q->next = h;
    free(p);
  }
}
```

In the circular version a duplicate may wrap around (the tail equals the head); the head is kept and the tail removed. **Complexity.** O(n). **Mistakes.** Advancing `p` after a deletion (it would skip a third copy); looping `while (p)` on a circular list, which never ends.

Q1.56 Delete a node given only a pointer to it

ARISTA · TECH-1 · ■□□ · S

You are given a pointer to a node that is not the tail of a singly linked list, and no head. Delete it.

30-second answer: copy the successor's value into the node and unlink the successor.

```
void delete_node(Node *n) {
  Node *d = n->next;
  n->val = d->val;
  n->next = d->next;
  free(d);
}
```

Complexity. O(1). **Follow-ups.** The tail cannot be deleted this way, since it has no successor to copy; any other pointer held to the successor is now dangling. **Mistakes.** Trying to unlink `n` itself without access to its predecessor.

Q1.57 Remove the n-th node from the end

AMAZON · MICROSOFT · TECH-1 · ■■□ · T

Delete the n*-th node from the end of a singly linked list in one pass.*

30-second answer: advance one pointer *n* nodes ahead, then move both until the lead pointer reaches the last node; the trailing pointer is then the predecessor of the node to delete.

```
Node *remove_nth(Node *head, int n) {
  Node dummy = {0, head};
  Node *fast = &dummy, *slow = &dummy;
  while (n-- > 0 && fast) fast = fast->next;
  if (!fast) return head;     /* n too big */
  while (fast->next) {
    fast = fast->next;
    slow = slow->next;
  }
  Node *d = slow->next;
  slow->next = d->next;
  free(d);
  return dummy.next;
}
```

The dummy node lets the head itself be removed (*n* equal to the length). **Complexity.** O(*L*) time, O(1) space.

Q1.58 Palindrome linked list

AMAZON · TECH-1 · ■■□ · T

Decide whether the values of a singly linked list read the same in both directions, in O(1) extra space.

30-second answer: find the middle with fast and slow pointers, reverse the second half in place, compare it with the first half, and reverse it back.

```
bool is_pal(Node *h) {
  Node *slow = h, *fast = h;
  while (fast && fast->next) {
    slow = slow->next;
    fast = fast->next->next;
  }
  Node *r = reverse(slow), *save = r;
  bool ok = true;
  for (Node *p = h; ok && r;
       p = p->next, r = r->next)
    if (p->val != r->val) ok = false;
  reverse(save);    /* restore the list */
  return ok;
}
```

For `1 → 2 → 3 → 2 → 1` the middle is 3, the reversed half is `1 → 2 → 3`, and the comparison `1=1, 2=2, 3=3` succeeds (the middle compares with itself). `reverse` is the function of §1.6 (p. 27). **Complexity.** O(*n*) time, O(1) space.

Q1.59 Add two numbers stored as linked lists

ARISTA · AMAZON · TECH-1 · ■■□ · S

Each list holds a non-negative number with the least significant digit first. Return their sum as a new list.

30-second answer: add digit by digit with a carry, as on paper, appending to a result list through a dummy head; continue while either list or a carry remains.

```
Node *add_lists(Node *a, Node *b) {
  Node dummy = {0, NULL}, *t = &dummy;
  int carry = 0;
  while (a || b || carry) {
    int s = carry + (a ? a->val : 0)
                  + (b ? b->val : 0);
    carry = s / 10;
    Node *n = malloc(sizeof *n);
    if (!n) return NULL;  /* leaks partial */
    n->val = s % 10;
    n->next = NULL;
    t->next = n;
    t = n;
    if (a) a = a->next;
    if (b) b = b->next;
  }
  return dummy.next;
}
```

`[2,4,3] + [5,6,4]` gives `[7,0,8]` (342 + 465 = 807). **Complexity.** O(max(*m*, *n*)). **Follow-ups.** When the most significant digit comes first, reverse both inputs (or push digits on stacks), add, and reverse the result. **Mistakes.** Dropping a final carry (`5 + 5` must give `[0,1]`).

Q1.60 Reverse a doubly linked list; delete a given node

ARISTA · TECH-1 · ■□□ · S

Reverse a doubly linked list in place. Then delete a node given its pointer.

30-second answer: swap the `next` and `prev` pointers of every node and return the last node visited; deleting a node relinks its two neighbours and updates the head if the node was first.

```
typedef struct D { int val;
          struct D *prev, *next; } D;
D *dl_reverse(D *h) {
  D *cur = h, *last = NULL;
  while (cur) {
    D *nx = cur->next;
    cur->next = cur->prev;
    cur->prev = nx;
    last = cur;
    cur = nx;
  }
  return last;
}
D *dl_delete(D *h, D *n) {
  if (n->prev) n->prev->next = n->next;
  else h = n->next;
  if (n->next) n->next->prev = n->prev;
  free(n);
  return h;
}
```

Complexity. O(*n*) and O(1). **Mistakes.** Following `cur->next` after swapping (it now points backwards; hence the saved `nx`); forgetting the head and tail cases in deletion.

Q1.61 Reverse nodes in groups of k

ARISTA · TECH-2 · ■■■ · T

Reverse a list `k` nodes at a time; a final group shorter than `k` stays as it is.

30-second answer: check that `k` nodes remain, reverse the rest of the list recursively, then reverse this group onto the result.

```
Node *rev_k(Node *head, int k) {
  Node *p = head;
  for (int i = 0; i < k; i++) {
    if (!p) return head;   /* fewer than k */
    p = p->next;
  }
  Node *prev = rev_k(p, k), *cur = head;
  for (int i = 0; i < k; i++) {
    Node *nx = cur->next;
    cur->next = prev;
    prev = cur;
    cur = nx;
  }
  return prev;
}
```

The reversal loop is the ordinary one of §1.6 (p. 27) with `prev` starting at the already processed remainder. **Complexity.** O(n) time, O(n/k) stack; an iterative version keeps the tail of the previous group and uses O(1) space. **Mistakes.** Reversing a short final group.

Q1.62 Copy a list with random pointers

AMAZON · MICROSOFT · TECH-2 · ■■□ · T

Each node has `next` and `rnd` (any node or `NULL`). Make a deep copy.

30-second answer: interleave a copy after every original node, set each copy's `rnd` from its original's `rnd->next`, then split the two lists.

```
typedef struct R { int val;
        struct R *next, *rnd; } R;
R *copy_random(R *h) {   /* checks omitted */
  if (!h) return NULL;
  for (R *p = h; p; p = p->next->next) {
    R *c = malloc(sizeof *c);
    c->val = p->val;
    c->next = p->next;
    c->rnd = NULL;
    p->next = c;
  }
  for (R *p = h; p; p = p->next->next)
    if (p->rnd) p->next->rnd = p->rnd->next;
  R *nh = h->next;
  for (R *p = h; p; p = p->next) {
    R *c = p->next;
    p->next = c->next;
    if (c->next) c->next = c->next->next;
  }
  return nh;
}
```

Complexity. O(n) time, O(1) extra beyond the copy. **Follow-ups.** The alternative is a hash map from original to copy, which uses O(n) memory but is simpler to write.

Q1.63 Merge k sorted lists

AMAZON · GOOGLE · TECH-2 · ■■■ · T

Merge `k` sorted linked lists into one sorted list.

30-second answer: keep the head of each list in a min-heap; repeatedly take the smallest and push its successor.

```
struct LN { int val; LN *next; };
LN *mergeK(vector<LN*> ls) {
  auto cmp = [](LN* a, LN* b) {
    return a->val > b->val; };
  priority_queue<LN*, vector<LN*>,
                 decltype(cmp)> pq(cmp);
  for (LN* l : ls) if (l) pq.push(l);
  LN dummy{0, nullptr}, *t = &dummy;
  while (!pq.empty()) {
    LN* n = pq.top(); pq.pop();
    t->next = n; t = n;
    if (n->next) pq.push(n->next);
  }
  return dummy.next;
}
```

Complexity. O(N log k) for N total nodes. **Follow-ups.** Merging pairwise in rounds (divide and conquer) has the same bound without a heap. This is the same k-way merge that combines sorted runs of an external sort.

Q1.64 Min stack

AMAZON · MICROSOFT · OA · ■□□ · T

A stack with `push`, `pop`, `top` and `getMin`, all O(1).

30-second answer: store with every element the minimum of the stack up to that element.

```
class MinStack {
  vector<pair<int,int>> s;  // value, min
public:
  void push(int x) {
    int m = s.empty() ? x
        : min(x, s.back().second);
    s.push_back({x, m});
  }
  void pop() { s.pop_back(); }
  int top() { return s.back().first; }
  int getMin() { return s.back().second; }
};
```

Complexity. O(1) per operation, O(n) space. **Mistakes.** Keeping a single `min` variable, which cannot be restored after popping the minimum.

Q1.65 Queue using two stacks

MICROSOFT · AMAZON · OA · ■□□ · T

Implement a FIFO queue using only stack operations.

30-second answer: push onto an input stack; pop from an output stack, refilling it from the input stack only when it is empty. Each element moves at most twice.

```
class MyQueue {
  stack<int> in, out;
  void shift() {
    if (out.empty())
      while (!in.empty()) {
        out.push(in.top()); in.pop();
      }
  }
public:
  void push(int x) { in.push(x); }
  int pop() {
    shift(); int v = out.top();
    out.pop(); return v;
  }
  int peek() { shift(); return out.top(); }
  bool empty() {
    return in.empty() && out.empty();
  }
};
```

Complexity. Amortised O(1) per operation. **Mistakes.** Refilling `out` while it still holds elements, which breaks the order.

Q1.66 Evaluate a reverse Polish expression

AMAZON · OA · ■■□ · T

Evaluate tokens such as `["2","1","+","3","*"]` *(that is, (2 + 1) × 3 = 9). Division truncates towards zero.*

30-second answer: push numbers; on an operator pop two operands (the right one first), apply, and push the result.

```
int evalRPN(const vector<string>& t) {
  vector<long long> st;
  for (auto& s : t) {
    if (s.size() == 1 &&
        strchr("+-*/", s[0])) {
      long long b = st.back(); st.pop_back();
      long long a = st.back(); st.pop_back();
      char o = s[0];
      st.push_back(o == '+' ? a + b
                  : o == '-' ? a - b
                  : o == '*' ? a * b : a / b);
    } else st.push_back(stoll(s));
  }
  return (int)st.back();
}
```

Complexity. O(*n*). **Mistakes.** Swapping the operand order (matters for `-` and `/`); treating `"-3"` as an operator (hence the test on the token length).

Q1.67 Sliding window maximum

GOOGLE · AMAZON · TECH-2 · ■■■ · T

For every window of `k` *consecutive elements, output its maximum, in O(*n*).*

30-second answer: keep a deque of indices whose values decrease from front to back; the front is the maximum of the window. Drop the front when it leaves the window, and drop smaller values from the back before pushing the new index.

```
vector<int> maxWindow(const vector<int>& a,
                      int k) {
  deque<int> dq;
  vector<int> r;
  for (int i = 0; i < (int)a.size(); ++i) {
    if (!dq.empty() && dq.front() <= i - k)
      dq.pop_front();
    while (!dq.empty() &&
           a[dq.back()] <= a[i])
      dq.pop_back();
    dq.push_back(i);
    if (i >= k - 1)
      r.push_back(a[dq.front()]);
  }
  return r;
}
```

For `[1,3,-1,-3,5,3,6,7]` and $k = 3$ the output is `[3,3,5,5,6,7]`. Each index enters and leaves the deque once, so the cost is O(*n*). **Mistakes.** Storing values rather than indices (the expiry test needs the index); using `<` so that equal values stay and the deque grows.

Trees and tries

Q1.68 Invert a binary tree; test for symmetry

GOOGLE · AMAZON · OA · ■□□ · T

Swap the left and right child of every node. Then decide whether a tree is a mirror image of itself.

30-second answer: recurse: swap at each node; for symmetry compare the left subtree with the mirror of the right subtree.

```
TreeNode* invert(TreeNode* t) {
  if (!t) return t;
  swap(t->left, t->right);
  invert(t->left);
  invert(t->right);
  return t;
}
bool mirror(TreeNode* a, TreeNode* b) {
  if (!a || !b) return a == b;
  return a->val == b->val
      && mirror(a->left, b->right)
      && mirror(a->right, b->left);
}
// symmetric iff root == null ||
//   mirror(root->left, root->right)
```

Complexity. O(*n*) time, O(*h*) stack. **Mistakes.** Comparing `a->left` with `b->left` in the mirror test.

Q1.69 Lowest common ancestor

MICROSOFT · AMAZON · TECH-1 · ■■□ · T

Find the lowest node that has both `p` *and* `q` *in its subtree, first in a BST and then in a general binary tree.*

30-second answer: in a BST walk down, going left when both keys are smaller and right when both are larger; the first node that splits them is the answer. In a general tree, recurse and return the node where both sides report a hit.

```
TreeNode* lcaBST(TreeNode* t, int p, int q) {
  while (t) {
    if (p < t->val && q < t->val)
      t = t->left;
    else if (p > t->val && q > t->val)
      t = t->right;
    else return t;
  }
  return nullptr;
}
TreeNode* lca(TreeNode* t, TreeNode* p,
              TreeNode* q) {
  if (!t || t == p || t == q) return t;
  TreeNode* l = lca(t->left, p, q);
  TreeNode* r = lca(t->right, p, q);
  return l && r ? t : (l ? l : r);
}
```

Complexity. O(*h*) for the BST, O(*n*) for the general tree. **Mistakes.** Assuming both nodes exist in the general version; the code above then returns the one found.

Q1.70 Kth smallest element in a BST

GOOGLE · AMAZON · TECH-1 · ■■□ · T

Return the k*-th smallest key of a BST.*

30-second answer: in-order traversal visits keys in ascending order; stop at the *k*-th visit using an explicit stack.

```
int kthSmallest(TreeNode* t, int k) {
  vector<TreeNode*> st;
  while (t || !st.empty()) {
    while (t) {
      st.push_back(t); t = t->left;
    }
    t = st.back(); st.pop_back();
    if (--k == 0) return t->val;
    t = t->right;
  }
  return -1;
}
```

Complexity. O(*h* + *k*) time, O(*h*) space. **Follow-ups.** With frequent updates and queries, store the subtree size in each node and descend in O(*h*).

Q1.71 Diameter of a binary tree

GOOGLE · MICROSOFT · TECH-1 · ■■□ · T

Find the number of edges on the longest path between two nodes of a tree.

30-second answer: post-order recursion that returns the node's height to its parent while updating a global best with the left height plus the right height.

```
int diam(TreeNode* t, int& best) {
  if (!t) return 0;
  int l = diam(t->left, best);
  int r = diam(t->right, best);
  best = max(best, l + r);
  return 1 + max(l, r);
}
// call: int best = 0; diam(root, best);
```

The value returned upward (a height) differs from the value recorded (a path through the node); mixing them up is the classic bug. **Complexity.** O(*n*) time, O(*h*) space.

Q1.72 Serialise and deserialise a binary tree

GOOGLE · AMAZON · TECH-2 · ■■■ · T

Convert a binary tree to a string and back, losing no structure.

30-second answer: pre-order traversal with an explicit marker for each null child; the same order reads it back recursively.

```
void ser(TreeNode* t, string& o) {
  if (!t) { o += "#,"; return; }
  o += to_string(t->val) + ",";
  ser(t->left, o);
  ser(t->right, o);
}
TreeNode* de(istringstream& in) {
  string s;
  getline(in, s, ',');
  if (s == "#") return nullptr;
  auto* t = new TreeNode{stoi(s)};
  t->left = de(in);
  t->right = de(in);
  return t;
}
```

The tree `1 (2, 3)` becomes `1,2,#,#,3,#,#,`. Without the null markers the string would not determine the shape. **Complexity.** O(*n*). **Follow-ups.** Level-order form with a queue avoids deep recursion.

Q1.73 Implement a trie

GOOGLE · AMAZON · TECH-1 · ■■□ · S

Support `insert(word)`, `search(word)` *and* `startsWith(prefix)` *for lowercase words.*

30-second answer: a tree with 26 children per node and an end flag; each operation walks one edge per letter.

```
struct Trie {
  struct N {
    N* c[26] = {};
    bool end = false;
  };
  N root;
  void insert(const string& w) {
    N* t = &root;
    for (char ch : w) {
      N*& nx = t->c[ch - 'a'];
      if (!nx) nx = new N;
      t = nx;
    }
    t->end = true;
  }
  N* walk(const string& p) {
    N* t = &root;
    for (char ch : p) {
      t = t->c[ch - 'a'];
      if (!t) return nullptr;
    }
    return t;
  }
  bool search(const string& w) {
    N* t = walk(w); return t && t->end;
  }
  bool startsWith(const string& p) {
    return walk(p) != nullptr;
  }
};
```

Complexity. O(*L*) per operation; memory up to 26 pointers per node. **Follow-ups.** A hash map of children saves memory for large alphabets; deletion needs the pruning of the next problem. **Mistakes.** Treating a prefix as a word (`search("app")` when only `"apple"` is stored).

Q1.74 Delete a prefix from the routing trie

GOOGLE · JUNIPER · TECH-2 · ■■■ · T

Remove a route from the binary trie of §1.9 (p. 41)*, freeing nodes that no longer lead anywhere.*

30-second answer: recurse down the prefix; clear the next hop at its node; on the way back, free any child that has no route and no children.

```
static bool prune(Node *t, uint32_t p,
                  unsigned len, unsigned i) {
  if (i == len) t->hop = -1;
  else {
    unsigned b = (p >> (31 - i)) & 1u;
    if (t->c[b] &&
        prune(t->c[b], p, len, i + 1)) {
      free(t->c[b]);
      t->c[b] = NULL;
    }
  }
  return t->hop < 0 && !t->c[0] && !t->c[1];
}
void trie_delete(Node *root, uint32_t p,
                 unsigned len) {
  prune(root, p, len, 0);
}
```

The function returns whether its node is now useless; the root is never freed. Deleting 10.1.2.0/24 from the table of §1.9 (p. 41) removes the nodes below 10.1.0.0/16 but keeps the /16 route. **Complexity.** O(*len*). **Mistakes.** Freeing a node that still carries a shorter prefix's route; clearing the hop without pruning (a slow memory leak under route churn).

Q1.75 CIDR firewall rules: the most specific rule wins

GOOGLE · CISCO · TECH-2 · ■■■ · T

A rule list maps CIDR blocks to allow or deny. For a packet's source address, apply the most specific matching rule; if two rules have the same prefix, the earlier rule wins; with no match, deny.

30-second answer: store the action at the prefix's node in the trie of §1.9 (p. 41)**, ignore a later rule for an already stored prefix, and answer with a longest-prefix lookup.**

```
static Node *find_exact(Node *t, uint32_t p,
                        unsigned len) {
  for (unsigned i = 0; t && i < len; i++)
    t = t->c[(p >> (31 - i)) & 1u];
  return t;
}
bool add_rule(Node *root, uint32_t p,
              unsigned len, int act) {
  Node *e = find_exact(root, p, len);
  if (e && e->hop >= 0) return true;
  return trie_insert(root, p, len, act);
}
int decide(const Node *root, uint32_t ip) {
  int a = trie_lookup(root, ip);
  return a < 0 ? 0 : a;  /* 1 = allow */
}
```

With the rules `10.0.0.0/8 allow` and `10.1.0.0/16 deny`, the address 10.1.5.5 is denied and 10.2.0.1 is allowed. **Complexity.** O(32) per rule and per lookup. **Mistakes.** Using first-match-in-list semantics (a scan) when the specification says most specific; a rule `0.0.0.0/0` is the default and sits at the root.

Q1.76 Longest-prefix match without a trie

GOOGLE · TECH-2 · ■■□ · S

The interviewer rejects the trie: "use hash tables".

30-second answer: one hash table per prefix length, mapping the masked prefix to the next hop; probe from /32 down to /0 and return the first hit.

```
struct LPM {
  unordered_map<uint32_t, int> t[33];
  static uint32_t mk(uint32_t a, int len) {
    return len ? a & (~0u << (32 - len)) : 0;
  }
  void add(uint32_t p, int len, int hop) {
    t[len][mk(p, len)] = hop;
  }
  int lookup(uint32_t a) {
    for (int len = 32; len >= 0; --len) {
      auto it = t[len].find(mk(a, len));
      if (it != t[len].end())
        return it->second;
    }
    return -1;
  }
};
```

Complexity. At most 33 hash probes per lookup; O(1) insertion. **Follow-ups.** Keep a list of the prefix lengths that occur (real tables use few distinct lengths) and probe only those; binary search over lengths with "marker" entries gives O(log 32) probes. **Mistakes.** The shift `~0u << 32` for the /0 case (handled by `len ?`).

Q1.77 Maximum XOR of two numbers

GOOGLE · TECH-2 · ■■■ · T

Find the maximum of `a[i] ^ a[j]` *over all pairs of a non-negative integer array in O(*n*).*

30-second answer: insert every number into a binary trie, most significant bit first; for each number walk the trie choosing, at each bit, the child with the opposite bit when it exists.

```
int findMaxXor(const vector<int>& a) {
  vector<array<int,2>> t(1);   // 0 = root
  for (int x : a) {
    int u = 0;
    for (int i = 30; i >= 0; --i) {
      int b = (x >> i) & 1;
      if (!t[u][b]) {
        t[u][b] = t.size();
        t.push_back({0, 0});
      }
      u = t[u][b];
    }
  }
  int best = 0;
  for (int x : a) {
    int u = 0, cur = 0;
    for (int i = 30; i >= 0; --i) {
      int b = ((x >> i) & 1) ^ 1;
      if (t[u][b]) {
        cur |= 1 << i;
        u = t[u][b];
      } else u = t[u][b ^ 1];
    }
    best = max(best, cur);
  }
  return best;
}
```

A higher bit set to one outweighs all lower bits, so the greedy choice of the opposite bit is optimal. **Complexity.** O(31 *n*) time and space. **Mistakes.** Starting at bit 31 for non-negative `int` (harmless but wasted), or building the trie with node pointers that reallocate (use indices, as here).

Heaps, large data and graphs

Q1.78 Kth largest element

AMAZON · GOOGLE · OA · ■■□ · T

Return the k*-th largest value of an unsorted array (counting duplicates).*

30-second answer: keep a min-heap of the *k* largest values seen; its root is the answer.

```
int kthLargest(const vector<int>& a, int k) {
  priority_queue<int, vector<int>,
                 greater<int>> h;
  for (int x : a) {
    h.push(x);
    if ((int)h.size() > k) h.pop();
  }
  return h.top();
}
```

Complexity. O(*n* log *k*) time, O(*k*) space. **Follow-ups.** `nth_element` (quickselect) gives O(*n*) on average but modifies the array; a heap also handles a stream. **Mistakes.** Using a max-heap of size *k* (the trap of §1.10 (p. 45)).

Q1.79 Median of a data stream

GOOGLE · AMAZON · TECH-2 · ■■■ · T

Support `add(x)` *and* `median()` *on a growing stream.*

30-second answer: a max-heap for the lower half and a min-heap for the upper half, kept equal in size (or one larger below); the median sits at the roots.

```
class Median {
  priority_queue<int> lo;   // lower half
  priority_queue<int, vector<int>,
                 greater<int>> hi;
public:
  void add(int x) {
    lo.push(x);
    hi.push(lo.top()); lo.pop();
    if (hi.size() > lo.size()) {
      lo.push(hi.top()); hi.pop();
    }
  }
  double median() {   // after >= 1 add
    return lo.size() > hi.size()
      ? lo.top()
      : ((double)lo.top() + hi.top()) / 2;
  }
};
```

Each `add` sends the new value through `lo` to `hi`, which keeps every value of `lo` no larger than every value of `hi`, and then rebalances. **Complexity.** O(log *n*) per `add`, O(1) per `median`.

Q1.80 Top 10 source IPs by bytes in a 10 GB flow log

GOOGLE · AMAZON · TECH-2 · ■■■ · T

A flow log of `src_ip bytes` *lines is far larger than memory. Report the ten sources with the most bytes.*

30-second answer: if the distinct addresses fit in memory, one pass with a hash map of totals and a size-10 min-heap suffices; if not, hash-partition the log into files by address, total each file separately, and merge the per-file top tens.

Approach. Memory is set by the number of *distinct* keys, not by the log size: 10^7 addresses need about $10^7 \times (4 + 8)$ bytes for the totals, which is modest. If even that is too large, partition: a hash of the address picks one of *P* files, so every record of an address lands in the same file, and each file then fits in memory. The top ten overall are among the union of the per-file top tens. When an approximate answer is acceptable, a count-min sketch with a heap of candidates uses fixed memory.

```
using P = pair<uint64_t, uint32_t>;
vector<P> top10(
    unordered_map<uint32_t, uint64_t>& m) {
  priority_queue<P, vector<P>, greater<P>> h;
  for (auto& [ip, b] : m) {
    h.push({b, ip});
    if (h.size() > 10) h.pop();
  }
  vector<P> r;
  while (!h.empty()) {
    r.push_back(h.top()); h.pop();
  }
  reverse(r.begin(), r.end());
  return r;
}
```

Complexity. O(*N*) to read, O(*d* log 10) for *d* distinct keys, and two passes of disk I/O when partitioning. **Mistakes.** Partitioning by line number or by byte range (one address then appears in many files and its total is split); sorting all *d* totals when only ten are needed.

Q1.81 Reorder packets by sequence number

ARISTA · BROADCOM · TECH-1 · ■■□ · T

Packets arrive out of order with sequence numbers. Deliver the data in sequence order, as soon as the next expected number is available, ignoring duplicates.

30-second answer: hold arrivals in a min-heap keyed by sequence number; after each arrival release the top while it equals the next expected number.

```
struct Reorder {
  using Pk = pair<uint32_t, string>;
  uint32_t next = 0;
  priority_queue<Pk, vector<Pk>,
                 greater<Pk>> pq;
  vector<string> push(uint32_t seq,
                      string data) {
    vector<string> out;
    pq.push({seq, move(data)});
    while (!pq.empty() &&
           pq.top().first <= next) {
      if (pq.top().first == next) {
        out.push_back(pq.top().second);
        ++next;
      }
      pq.pop();   // delivered or duplicate
    }
    return out;
  }
};
```

Arrivals 1, 0, 2 deliver nothing, then `[d0, d1]`, then `[d2]`. **Complexity.** O(log *n*) per packet. **Follow-ups.** Real sequence numbers wrap; compare with `(int32_t)(a - b) < 0`. A bounded window and a timeout decide when to skip a lost packet. **Mistakes.** Letting a duplicate with a smaller number block the heap top forever.

Q1.82 Minimum dice throws to finish a snake-and-ladder board

CISCO · OA · ■■□ · S

A board has cells 1 to N. *Landing on a ladder foot or snake head moves you to its other end. A throw advances 1 to 6 cells. Find the fewest throws from cell 1 to cell* N.

30-second answer: model cells as vertices and each throw as an edge of weight one, so breadth-first search from cell 1 gives the minimum.

```
int minThrows(const vector<int>& jump) {
  int N = jump.size() - 1;
  vector<int> dist(N + 1, -1);
  queue<int> q;
  dist[1] = 0; q.push(1);
  while (!q.empty()) {
    int u = q.front(); q.pop();
    if (u == N) return dist[u];
    for (int d = 1; d <= 6 && u + d <= N;
         ++d) {
      int v = jump[u + d] > 0
              ? jump[u + d] : u + d;
      if (dist[v] < 0) {
        dist[v] = dist[u] + 1;
        q.push(v);
      }
    }
  }
  return -1;
}
```

With $N = 30$, ladders $3 \to 22$, $5 \to 8$, $11 \to 26$, $20 \to 29$ and snakes $27 \to 1$, $21 \to 9$, $17 \to 4$, $19 \to 7$, the answer is 3: throw 2 (to 3, up to 22), then 6 (to 28), then 2. **Complexity.** O(6*N*). **Mistakes.** Plain DFS (finds a path, not the shortest); applying a second jump after landing on a jump's end (one throw gives one jump).

Q1.83 Number of islands

AMAZON · MICROSOFT · OA · ■■□ · T

In a grid of `'1'` *(land) and* `'0'` *(water), count the groups of land cells connected horizontally or vertically.*

30-second answer: scan the grid; on each unvisited land cell, count one island and flood-fill it (DFS or BFS) so its cells are not counted again.

```
int numIslands(vector<string>& g) {
  int R = g.size(), C = R ? g[0].size() : 0;
  int n = 0;
  function<void(int,int)> fill =
      [&](int r, int c) {
    if (r < 0 || c < 0 || r >= R || c >= C
        || g[r][c] != '1') return;
    g[r][c] = '0';   // mark visited
    fill(r + 1, c); fill(r - 1, c);
    fill(r, c + 1); fill(r, c - 1);
  };
  for (int r = 0; r < R; ++r)
    for (int c = 0; c < C; ++c)
      if (g[r][c] == '1') {
        ++n; fill(r, c);
      }
  return n;
}
```

Complexity. O(*RC*) time; the recursion depth can reach *RC*, so for large grids use the explicit stack of §1.11 (p. 47). **Mistakes.** No visited marking (infinite recursion); checking bounds after indexing.

Q1.84 Can all tasks be finished (course schedule)

AMAZON · GOOGLE · OA · ■■□ · T

There are `n` *tasks and pairs* `[a, b]` *meaning "task* `b` *must precede task* `a` *". Is there an order that satisfies all pairs?*

30-second answer: build the dependency graph and run Kahn's topological sort; all tasks can be finished exactly when every vertex is output, i.e. there is no cycle.

```
bool canFinish(int n,
               const vector<vector<int>>&
               pre) {
  vector<vector<int>> g(n);
  for (auto& p : pre)
    g[p[1]].push_back(p[0]);
  return (int)topo(n, g).size() == n;
}
```

`topo` is the function of §1.11 (p. 47); returning its order instead of a boolean answers the "find a valid order" variant. **Complexity.** O(*V* + *E*). **Mistakes.** Reversing the edge direction (harmless for the yes/no answer, wrong for the order).

Q1.85 Network delay time

CISCO · GOOGLE · TECH-1 · ■■□ · T

A signal starts at router `k`*. Directed links* `[u, v, w]` *take* `w` *time units. How long until all* `n` *routers (numbered 1 to* `n`*) have the signal, or −1 if some never do?*

30-second answer: run Dijkstra from `k`; the answer is the largest shortest distance, or −1 if any router is unreachable.

```
int networkDelay(
    const vector<vector<int>>& times,
    int n, int k) {
  Graph g(n + 1);
  for (auto& t : times)
    g[t[0]].push_back({t[1], t[2]});
  auto d = dijkstra(g, k);
  long long worst = 0;
  for (int i = 1; i <= n; ++i)
    worst = max(worst, d[i]);
  return worst >= LLONG_MAX / 4
         ? -1 : (int)worst;
}
```

`Graph` and `dijkstra` are those of §1.11 (p. 47). The signal reaches the last router at the *maximum* of the shortest arrival times, not their sum. **Complexity.** O((*V* + *E*) log *V*). **Mistakes.** Using BFS on weighted links; forgetting that vertex 0 does not exist.

Q1.86 Cheapest route with at most k stops

CISCO · AMAZON · TECH-2 · ■■■ · T

Given directed priced links `[u, v, price]`*, find the cheapest route from* `src` *to* `dst` *using at most* `k` *intermediate stops, or −1.*

30-second answer: Bellman–Ford with exactly *k* + 1 rounds, each using only the previous round's distances, so a path never exceeds *k* + 1 links.

```
int cheapest(int n, vector<vector<int>>& f,
             int src, int dst, int k) {
  vector<Edge> es;
  for (auto& x : f)
    es.push_back({x[0], x[1], x[2]});
  auto d = bellmanFord(n, es, src, k + 1);
  return d[dst] >= LLONG_MAX / 4
         ? -1 : (int)d[dst];
}
```

`Edge` and `bellmanFord` are those of §1.11 (p. 47); the round count is the hop limit, as in a protocol that counts hops. **Complexity.** O((*k* + 1)*E*). **Mistakes.** Dijkstra with a plain distance array (the cheapest route may use too many stops, and a dearer route with fewer stops is then lost).

Q1.87 First link that creates a loop

CISCO · JUNIPER · TECH-1 · ■■□ · T

Links between nodes 1 to `n` *are added in order. Report the first link that connects two nodes that are already connected.*

30-second answer: union–find over the nodes; the first link whose `unite` returns false closes a loop.

```
vector<int> firstLoop(
    int n, const vector<vector<int>>& e) {
  DSU d(n + 1);
  for (auto& x : e)
    if (!d.unite(x[0], x[1])) return x;
  return {};
}
```

`DSU` is the structure of §1.11 (p. 47). In a switched network the same test finds the link that a spanning-tree protocol must block. **Complexity.** O(E α(n)). **Follow-ups.** For a graph that is a tree plus one extra link, the answer is the last link of the input that lies on the cycle (the variant asked on some sites).

Q1.88 Make a network connected

CISCO · AMAZON · TECH-1 · ■■□ · T

`n` *computers and cables* `[a, b]`*. In one operation you move a cable to join two computers. Return the fewest operations to connect everything, or −1 if impossible.*

30-second answer: connecting c components needs $c − 1$ spare cables; there are enough exactly when there are at least $n − 1$ cables, so count the components with union–find.

```
int makeConnected(
    int n, const vector<vector<int>>& c) {
  if ((int)c.size() < n - 1) return -1;
  DSU d(n);
  int comps = n;
  for (auto& x : c)
    if (d.unite(x[0], x[1])) --comps;
  return comps - 1;
}
```

Every cable that does not merge two components is spare, and with at least $n − 1$ cables there are always enough spares. **Complexity.** O(E α(n)).

Q1.89 Links whose failure splits the network (bridges)

GOOGLE · ARISTA · TECH-2 · ■■■ · T

Find every link whose removal disconnects the network (a bridge*).*

30-second answer: depth-first search records the entry time `tin` of each vertex and `low`, the earliest entry time reachable from its subtree using one back edge; a tree edge (u, v) is a bridge when `low[v] > tin[u]`.

```
vector<vector<int>> bridges(int n,
    const vector<vector<int>>& edges) {
  vector<vector<int>> g(n), out;
  for (auto& e : edges) {
    g[e[0]].push_back(e[1]);
    g[e[1]].push_back(e[0]);
  }
  vector<int> tin(n, -1), low(n, 0);
  int timer = 0;
  function<void(int,int)> dfs =
      [&](int u, int par) {
    tin[u] = low[u] = timer++;
    for (int v : g[u]) {
      if (v == par) continue;
      if (tin[v] >= 0)
        low[u] = min(low[u], tin[v]);
      else {
        dfs(v, u);
        low[u] = min(low[u], low[v]);
        if (low[v] > tin[u])
          out.push_back({u, v});
      }
    }
  };
  for (int i = 0; i < n; ++i)
    if (tin[i] < 0) dfs(i, -1);
  return out;
}
```

If nothing in the subtree below `v` can reach `u` or above except through the edge (u, v), the edge is a bridge. **Complexity.** O($V + E$). **Mistakes.** Skipping a parent edge when links are duplicated (parallel links are never bridges, so a multigraph needs the edge identifier, not the parent vertex).

Q1.90 Minimum cost to connect all points

GOOGLE · AMAZON · TECH-2 · ■■□ · T

Points on a plane are joined by cables costing the Manhattan distance between endpoints. Find the least total cost that connects all points.

30-second answer: this is a minimum spanning tree on a dense graph; Prim's algorithm in O(n^2), or Kruskal on sorted pair edges with union–find.

```
int minCostConnect(
    const vector<vector<int>>& p) {
  int n = p.size(), total = 0;
  vector<int> d(n, INT_MAX);
  vector<char> in(n, 0);
  d[0] = 0;
  for (int it = 0; it < n; ++it) {
    int u = -1;
    for (int i = 0; i < n; ++i)
      if (!in[i] && (u < 0 || d[i] < d[u]))
        u = i;
    in[u] = 1;
    total += d[u];
    for (int v = 0; v < n; ++v)
      if (!in[v])
        d[v] = min(d[v],
          abs(p[u][0] - p[v][0]) +
          abs(p[u][1] - p[v][1]));
  }
  return total;
}
```

`d[v]` is the cheapest cable from the tree to `v`; each round adds the nearest outside point. **Complexity.** O(n^2) time, O(n) space. **Follow-ups.** For sparse graphs, Kruskal costs O(E log E).

Q1.91 Number of provinces (connected components)

NOKIA · AMAZON · OA · ■■□ · S

A matrix `m[i][j] = 1` states that nodes `i` and `j` are directly connected. Count the groups of connected nodes.

30-second answer: depth-first search with an explicit stack from every unvisited node; the number of starts is the number of groups.

```
int provinces(const vector<vector<int>>& m) {
  int n = m.size(), cnt = 0;
  vector<char> seen(n, 0);
  for (int s = 0; s < n; ++s) {
    if (seen[s]) continue;
    ++cnt;
    vector<int> st{s};
    seen[s] = 1;
    while (!st.empty()) {
      int u = st.back(); st.pop_back();
      for (int v = 0; v < n; ++v)
        if (m[u][v] && !seen[v]) {
          seen[v] = 1;
          st.push_back(v);
        }
    }
  }
  return cnt;
}
```

Complexity. O(n^2) for the matrix form. **Follow-ups.** Union–find gives the same count and also handles edges that arrive over time.

Backtracking, searching, intervals and dynamic programming

Q1.92 Permutations and combination sum

GOOGLE · MICROSOFT · TECH-1 · ■■□ · T

Generate all permutations of distinct integers. Then, from positive integers, list all combinations that sum to a target, each value reusable.

30-second answer: choose, explore, un-choose. For permutations, swap each remaining element into position `i` and recurse. For combinations, either take `c[i]` again or skip to `c[i+1]`.

```
void perm(vector<int>& a, int i,
          vector<vector<int>>& out) {
  if (i == (int)a.size()) {
    out.push_back(a); return;
  }
  for (int j = i; j < (int)a.size(); ++j) {
    swap(a[i], a[j]);
    perm(a, i + 1, out);
    swap(a[i], a[j]);   // un-choose
  }
}
void comb(const vector<int>& c, int i,
          int rem, vector<int>& path,
          vector<vector<int>>& out) {
  if (rem == 0) {
    out.push_back(path); return;
  }
  if (rem < 0 || i == (int)c.size()) return;
  path.push_back(c[i]);
  comb(c, i, rem - c[i], path, out);
  path.pop_back();
  comb(c, i + 1, rem, path, out);
}
```

Complexity. O($n \cdot n!$) for permutations; combinations are bounded by the search tree. **Mistakes.** Forgetting the second `swap` or the `pop_back`; for repeated inputs, duplicates need a sorted array and a skip rule.

Q1.93 Search in a rotated sorted array

GOOGLE · MICROSOFT · TECH-1 · ■■□ · T

A sorted array of distinct values has been rotated at an unknown point (`4 5 6 7 0 1 2`). Find the index of a target in O(log n*).*

30-second answer: in each step one half is sorted; test whether the target lies in the sorted half and discard the other half.

```
int searchRot(const vector<int>& a, int x) {
  int lo = 0, hi = (int)a.size() - 1;
  while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == x) return mid;
    if (a[lo] <= a[mid]) {   // left sorted
      if (a[lo] <= x && x < a[mid])
        hi = mid - 1;
      else lo = mid + 1;
    } else {                 // right sorted
      if (a[mid] < x && x <= a[hi])
        lo = mid + 1;
      else hi = mid - 1;
    }
  }
  return -1;
}
```

Searching 0 in `4 5 6 7 0 1 2`: at mid = 3 (value 7) the left half `4..7` is sorted and excludes 0, so `lo = 4`; at mid = 5 (value 1) the left half `0 1` is sorted and contains 0, so `hi = 4`; mid = 4 then finds index 4. **Complexity.** O(log *n*). **Mistakes.** `<` instead of `<=` in `a[lo] <= a[mid]` when `lo == mid`; duplicates break the test (worst case O(*n*)).

Q1.94 Merge overlapping IP address ranges

GOOGLE · AMAZON · TECH-1 · ■■□ · T

Given ranges as strings `"a.b.c.d-e.f.g.h"`*, merge overlapping ranges and print them in order.*

30-second answer: convert each end to a 32-bit integer, sort and merge as intervals (§1.13 (p. 55)**), and convert back.**

```
extern "C" {
bool parse_ipv4(const char*, uint32_t*);
void format_ipv4(uint32_t, char[16]);
}
vector<string> mergeRanges(
    const vector<string>& in) {
  auto fmt = [](long long x) {
    char b[16];
    format_ipv4((uint32_t)x, b);
    return string(b);
  };
  vector<array<long long, 2>> v;
  for (auto& s : in) {
    size_t d = s.find('-');
    uint32_t a = 0, b = 0;
    parse_ipv4(s.substr(0, d).c_str(), &a);
    parse_ipv4(s.substr(d + 1).c_str(), &b);
    v.push_back({a, b});
  }
  vector<string> out;
  for (auto& r : mergeIntervals(v))
    out.push_back(fmt(r[0]) + "-" +
                  fmt(r[1]));
  return out;
}
```

`10.0.0.1-10.0.0.9` and `10.0.0.5-10.0.0.20` merge to `10.0.0.1-10.0.0.20`. **Complexity.** O(*n* log *n*). **Mistakes.** Comparing the strings (lexicographic order is not numeric order); a signed 32-bit integer for the address (use 64-bit or unsigned). **Follow-ups.** Ask whether adjacent ranges (`…9` and `…10`) should merge.

Q1.95 Peak number of simultaneous flows

CISCO · AMAZON · TECH-1 · ■■□ · T

Given flow intervals `[start, end)`*, find the largest number that overlap at one instant (the meeting-rooms problem).*

30-second answer: sweep line: turn each interval into a +1 event at its start and a −1 event at its end, sort the events, and track the running total.

```
int maxConcurrent(
    const vector<array<int,2>>& iv) {
  vector<pair<int,int>> ev;
  for (auto& x : iv) {
    ev.push_back({x[0], +1});
    ev.push_back({x[1], -1});
  }
  sort(ev.begin(), ev.end());
  int cur = 0, best = 0;
  for (auto& e : ev) {
    cur += e.second;
    best = max(best, cur);
  }
  return best;
}
```

Sorting the pairs puts −1 before +1 at equal times, so an interval ending at *t* frees its slot before another starting at *t* takes it. **Complexity.** O(*n* log *n*). **Follow-ups.** A min-heap of end times gives the same count and the actual assignment.

Q1.96 Climbing stairs and house robber (one-dimensional DP)

AMAZON · MICROSOFT · OA · ■□□ · T

(a) Count the ways to climb `n` stairs taking 1 or 2 steps at a time. (b) Choose non-adjacent houses to maximise the total loot.

30-second answer: (a) `ways[i] = ways[i-1] + ways[i-2]`; (b) the best up to house *i* either skips it or takes it with the best up to *i* − 2. Two variables replace the table.

```
int climb(int n) {
  int a = 1, b = 1;   // ways[0], ways[1]
  for (int i = 2; i <= n; ++i) {
    int c = a + b; a = b; b = c;
  }
  return b;
}
int rob(const vector<int>& h) {
  int take = 0, skip = 0;
  for (int x : h) {
    int t = skip + x;
    skip = max(skip, take);
    take = t;
  }
  return max(take, skip);
}
```

`climb(5)` is 8. For houses `[2,7,9,3,1]` the answer is 12 (2 + 9 + 1). **Complexity.** O(*n*) time, O(1) space. **Mistakes.** Taking the maximum of adjacent pairs greedily.

Q1.97 Longest increasing subsequence

GOOGLE · MICROSOFT · TECH-2 · ■■□ · T

Find the length of the longest strictly increasing subsequence (not necessarily contiguous).

30-second answer: keep `t[i]`, the smallest possible last value of an increasing subsequence of length `i + 1`; each new value replaces the first entry that is not smaller, found by binary search.

```
int lis(const vector<int>& a) {
  vector<int> t;
  for (int x : a) {
    auto it = lower_bound(t.begin(),
                          t.end(), x);
    if (it == t.end()) t.push_back(x);
    else *it = x;
  }
  return t.size();
}
```

For `[10, 9, 2, 5, 3, 7, 101, 18]` the array `t` evolves 10; 9; 2; 2,5; 2,3; 2,3,7; 2,3,7,101; 2,3,7,18, so the length is 4. Note that `t` is not itself a subsequence of the input. **Complexity.** O(*n* log *n*). The quadratic DP, `dp[i] = 1 + max dp[j]` over smaller `a[j]`, is the form to state first. **Mistakes.** `upper_bound` (allows equal values, giving the non-decreasing variant).

Q1.98 Edit distance

GOOGLE · MICROSOFT · TECH-2 · ■■■ · T

Find the fewest single-character insertions, deletions and replacements that turn one string into another.

30-second answer: `d[i][j]` is the distance between the first `i` characters of `s` and the first `j` of `t`; if the last characters match it equals `d[i-1][j-1]`, otherwise one plus the best of replace, delete and insert.

```
int editDistance(const string& s,
                 const string& t) {
  int n = s.size(), m = t.size();
  vector<vector<int>> d(n + 1,
                        vector<int>(m + 1));
  for (int i = 0; i <= n; ++i) d[i][0] = i;
  for (int j = 0; j <= m; ++j) d[0][j] = j;
  for (int i = 1; i <= n; ++i)
    for (int j = 1; j <= m; ++j)
      d[i][j] = s[i-1] == t[j-1]
        ? d[i-1][j-1]
        : 1 + min({d[i-1][j-1],
                   d[i-1][j], d[i][j-1]});
  return d[n][m];
}
```

`"kitten"` to `"sitting"` costs 3 (two replacements and one insertion). **Complexity.** O(*nm*) time and space, reducible to O(min(*n*, *m*)) space by keeping two rows. **Follow-ups.** The longest common subsequence uses the same table with `d[i][j] = d[i-1][j-1] + 1` on a match and `max(d[i-1][j], d[i][j-1])` otherwise.

Design, rate limiting and finite-state machines

Q1.99 LRU cache with a time-to-live (an ARP cache), thread-safe

MICROSOFT · AMAZON · TECH-2 · ■■■ · S

Extend the LRU cache of §1.15 (p. 60)*: every entry expires* `ttl` *time units after it was written, and the cache is used by several threads.*

30-second answer: store an expiry time in each node and treat an expired entry as a miss; guard every method with one mutex, because even `get` **reorders the list.**

```
class ArpCache {
  struct Item { int key; string mac;
                long long expires; };
  size_t cap; long long ttl;
  list<Item> items;
  unordered_map<int,
      list<Item>::iterator> pos;
  mutex m;
public:
  ArpCache(size_t c, long long t)
      : cap(c), ttl(t) {}
  optional<string> get(int ip,
                       long long now) {
    lock_guard<mutex> g(m);
    auto it = pos.find(ip);
    if (it == pos.end()) return nullopt;
    if (it->second->expires <= now) {
      items.erase(it->second);
      pos.erase(it);
      return nullopt;
    }
    items.splice(items.begin(), items,
                 it->second);
    return it->second->mac;
  }
  void put(int ip, string mac,
           long long now) {
    lock_guard<mutex> g(m);
    if (!cap) return;
    auto it = pos.find(ip);
    if (it != pos.end()) {
      items.erase(it->second);
      pos.erase(it);
    } else if (items.size() == cap) {
      pos.erase(items.back().key);
      items.pop_back();
    }
    items.push_front({ip, move(mac),
                      now + ttl});
    pos[ip] = items.begin();
  }
};
```

The caller supplies `now`, which makes the class testable and lets it use a monotonic clock. Expiry is lazy: an expired entry is removed when it is looked up. **Complexity.** O(1) average per call. **Follow-ups.** A background sweep, or a min-heap of expiry times, frees memory for entries never looked up again; shard the cache by key hash, each shard with its own mutex, to reduce contention; a reader-writer lock does not help because `get` writes. **Mistakes.** Locking only `put`; refreshing the expiry on `get` when the specification says the TTL runs from the write.

Q1.100 LFU cache

AMAZON · GOOGLE · TECH-2 · ■■■ · T

Like an LRU cache, but evict the least frequently used key; among equals, evict the least recently used. `get` *and* `put` *must be O(1).*

30-second answer: map each key to its value, count and list position; keep one list of keys per count, and the minimum count so that the victim is the back of that list.

```
class LFU {
  struct E { int v, f;
             list<int>::iterator it; };
  int cap, minf = 0;
  unordered_map<int, E> kv;
  unordered_map<int, list<int>> fl;
  void touch(int k) {
    E& e = kv[k];
    fl[e.f].erase(e.it);
    if (fl[e.f].empty()) {
      fl.erase(e.f);
      if (minf == e.f) ++minf;
    }
    ++e.f;
    fl[e.f].push_front(k);
    e.it = fl[e.f].begin();
  }
public:
  explicit LFU(int c) : cap(c) {}
  int get(int k) {
    if (!kv.count(k)) return -1;
    touch(k);
    return kv[k].v;
  }
  void put(int k, int v) {
    if (!cap) return;
    if (kv.count(k)) {
      kv[k].v = v; touch(k); return;
    }
    if ((int)kv.size() == cap) {
      int old = fl[minf].back();
      fl[minf].pop_back();
      if (fl[minf].empty()) fl.erase(minf);
      kv.erase(old);
    }
    minf = 1;
    fl[1].push_front(k);
    kv[k] = {v, 1, fl[1].begin()};
  }
};
```

Complexity. O(1) average per operation. **Mistakes.** Not updating `minf` when a frequency list empties; resetting `minf` to 1 only after eviction, not on a new insertion; erasing the key from `kv` but not its list node.

Q1.101 Fixed-window versus sliding-window rate limiting

MICROSOFT · AMAZON · TECH-1 · ■■□ · T

Implement a limit of `lim` requests per window of `w` time units in two ways, and show a request trace on which the fixed-window version lets through twice the limit.

30-second answer: a fixed window keeps one counter per aligned window and resets it at the boundary; a sliding log keeps the timestamps of accepted requests and drops those older than `w`.

```
struct Fixed {
  long long w, win = -1;
  int lim, n = 0;
  Fixed(long long w, int lim)
      : w(w), lim(lim) {}
  bool allow(long long t) {
    if (t / w != win) { win = t / w; n = 0; }
    return n < lim && ++n;
  }
};
struct SlidingLog {
  long long w; size_t lim;
  deque<long long> q;
  SlidingLog(long long w, size_t lim)
      : w(w), lim(lim) {}
  bool allow(long long t) {
    while (!q.empty() && q.front() <= t - w)
      q.pop_front();
    if (q.size() >= lim) return false;
    q.push_back(t);
    return true;
  }
};
```

With *lim* = 5 and *w* = 10: five requests at time 9 and five at time 11. The fixed window starts a new window at time 10, so it accepts all ten (ten requests in two time units). The sliding log still holds the five requests of time 9 at time 11 (since 9 > 11 − 10), so it rejects the second batch. **Complexity.** Fixed: O(1) time and space. Log: O(1) amortised time, O(*lim*) space. **Follow-ups.** The token bucket of §1.15 (p. 60) is a middle course: two numbers of state and bounded bursts.

Q1.102 Rate-limit repeated log messages

MICROSOFT · AMAZON · OA · ■□□ · T

A logger prints a message only if the same message has not been printed in the last 10 seconds. Timestamps are non-decreasing.

30-second answer: a hash map from message to the time of its last print.

```
class Logger {
  unordered_map<string, int> last;
public:
  bool shouldPrint(int t, const string& m) {
    auto it = last.find(m);
    if (it != last.end() &&
        t - it->second < 10)
      return false;
    last[m] = t;
    return true;
  }
};
```

Complexity. O(1) average. **Follow-ups.** The map grows without bound; drop entries older than ten seconds with a queue of (time, message) pairs from the front. **Mistakes.** Updating the time of a message that was suppressed (the window would then never end for a steady stream).

Q1.103 Insert, delete and get-random in O(1)

GOOGLE · AMAZON · TECH-1 · ■■□ · T

Design a set with `insert(x)`, `remove(x)` *and* `getRandom()` *(uniform over the elements), all O(1) on average.*

30-second answer: keep the elements in a vector for random access and a hash map from value to its index; remove by moving the last element into the removed slot.

```
class RandomSet {
  vector<int> v;
  unordered_map<int, int> pos;
public:
  bool insert(int x) {
    if (pos.count(x)) return false;
    pos[x] = v.size();
    v.push_back(x);
    return true;
  }
  bool remove(int x) {
    auto it = pos.find(x);
    if (it == pos.end()) return false;
    int last = v.back(), i = it->second;
    v[i] = last;
    pos[last] = i;
    v.pop_back();
    pos.erase(x);
    return true;
  }
  int getRandom() {
    return v[rand() % v.size()];
  }
};
```

Moving the last element avoids shifting the vector. When `x` is itself the last element, `v[i] = last` and `pos[last] = i` are harmless self-assignments, and `pos.erase(x)` then removes it. **Complexity.** O(1) average. **Mistakes.** Erasing from the middle of the vector (O(*n*)); forgetting to update the index of the moved element.

Q1.104 Valid number (a finite-state scan)

JUNIPER · MICROSOFT · TECH-2 · ■■■ · T

Decide whether a string is a valid decimal number: optional sign, digits with an optional point (at least one digit overall), and an optional exponent `e` *or* `E` *with an optional sign and at least one digit.*

30-second answer: scan in the order sign, integer part, point and fraction, exponent; each stage is a state, and a stage needing a digit fails if none follows.

```
bool isNumber(const string& s) {
  size_t i = 0, n = s.size();
  bool dig = false;
  auto D = [&](size_t k) {
    return k < n &&
        isdigit((unsigned char)s[k]);
  };
  if (i < n && (s[i] == '+' || s[i] == '-'))
    ++i;
  while (D(i)) { ++i; dig = true; }
  if (i < n && s[i] == '.') {
    ++i;
    while (D(i)) { ++i; dig = true; }
  }
  if (!dig) return false;
  if (i < n && (s[i] == 'e' ||
                s[i] == 'E')) {
    ++i;
    if (i < n && (s[i] == '+' || s[i] == '-'))
      ++i;
    if (!D(i)) return false;
    while (D(i)) ++i;
  }
  return i == n;
}
```

Accepted: `"2"`, `".5"`, `"3."`, `"-1.2e+10"`. Rejected: `"."`, `"e3"`, `"1e"`, `"1e+"`, `"--1"`, `"1 "`. The same logic as a table over (state, character class) is the form to present for a state-machine question: the states are start, sign, integer, point, fraction, exponent, exponent sign and exponent digits. **Complexity.** O(*n*). **Mistakes.** Accepting a bare point, or an exponent with no digits.

Revision Spread

THE 15-MINUTE VERSION

Ten facts. (1) Read the constraints first: $n \le 10^5$ means O(n log n), $n \le 10^3$–10^4 means O(n^2). (2) Say the six steps aloud: understand, examples, brute force, optimise, code, test. (3) Sorted input suggests two pointers or binary search; "contiguous" suggests a window or prefix sums. (4) Mask for prefix p: `p ? ~0u << (32-p) : 0`; network is `ip & mask`, broadcast is `net | ~mask`. (5) Delete from a list with `Node **pp`; test `fast` before `fast->next`. (6) A trie reads one bit per level; longest-prefix match keeps the deepest marked node and costs O(32). (7) BFS marks on push; Dijkstra skips stale entries and needs non-negative weights. (8) LRU is a hash map to doubly-linked-list nodes; every list change is mirrored in the map. (9) Token bucket: `tokens = min(B, tokens + r·Δt)`, pass if `tokens ≥ size`; integers and a monotonic clock. (10) Binary search: half-open `[lo, hi)`, `mid = lo + (hi-lo)/2`, never `lo = mid`.

Three pictures to redraw. The binary trie with a walk that remembers the deepest mark (§1.9 (p. 41)); the pointer-to-pointer delete and the reversal rewiring (§1.6 (p. 27)); the LRU map-plus-list diagram (§1.15 (p. 60)).

Constraint to complexity

Largest n	Aim for	Typical tools
10–12	O(n!)	permutations, backtracking
20–25	O(2^n)	subsets, bitmask
100	O(n^4)	small DP
500	O(n^3)	Floyd–Warshall
2000–5000	O(n^2)	2-D DP, all pairs
10^5–2×10^5	O(n log n)	sort, heap, binary search
10^6	O(n)	windows, prefix sums, hashing
10^9 and up	O(log n) or O(1)	binary search on the answer, formula

A judge performs about 10^8 simple operations per second; in Python step down one row. Values up to 10^9 mean no array indexed by value, and sums of 10^5 such values need 64 bits.

Keyword to pattern

If the problem says	Think	See
sorted; pair or triple with sum	two pointers, binary search	§1.2 (p. 12)
longest or shortest contiguous ... with a condition	variable sliding window	§1.2 (p. 12)
subarray of size k	fixed window	§1.2 (p. 12)
subarray sum equals k, negatives	prefix sums and a map	§1.2 (p. 12)
seen before, count, group, duplicates	hash set or map	§1.3 (p. 16)
valid, parse, split IP or MAC	single-scan validator	§1.4 (p. 19)
mask, flag, single number, power of two	bit tricks	§1.5 (p. 24)
cycle, middle, k-th from end	fast and slow pointers	§1.6 (p. 27)
reverse, merge, delete in a list	rewire with `prev`, dummy head, `**pp`	§1.6 (p. 27)
matching, undo, next greater	stack, monotonic stack	§1.7 (p. 33)
fixed-size FIFO, driver buffer	ring buffer with a count	§1.7 (p. 33)
window maximum	monotonic deque	§1.7 (p. 33)
tree depth, path, subtree	DFS, post-order return	§1.8 (p. 36)

BST, *k*-th smallest, validate	in-order, interval bounds	§1.8 (p. 36)
prefix, autocomplete, IP route lookup	trie	§1.9 (p. 41)
top *k*, median of a stream, merge *k* lists	heap	§1.10 (p. 45)
fewest hops; grid regions	BFS; flood fill	§1.11 (p. 47)
cheapest path, non-negative	Dijkstra	§1.11 (p. 47)
at most *k* edges; negative weights	Bellman–Ford	§1.11 (p. 47)
prerequisites, order of tasks	topological sort	§1.11 (p. 47)
loop in a network, components	union–find	§1.11 (p. 47)
all combinations or permutations	backtracking	§1.12 (p. 54)
minimise the maximum, "at most *D* days"	binary search on the answer	§1.13 (p. 55)
overlapping ranges, IP ranges	sort by start, merge	§1.13 (p. 55)
number of ways, min cost, can reach	dynamic programming	§1.14 (p. 58)
O(1) get and recency	hash map and doubly linked list	§1.15 (p. 60)
allow *r* per second with bursts	token bucket	§1.15 (p. 60)
validate a sequence of protocol events	state table	§1.16 (p. 65)

Big-O at a glance

Structure	Access	Search	Insert	Delete
array	O(1)	O(n)	O(n)	O(n)
dynamic array	O(1)	O(n)	O(1) amort. at end	O(n)
singly linked list	O(n)	O(n)	O(1) after a node	O(1) after predecessor
stack, queue, deque, ring	end only	O(n)	O(1)	O(1)
hash table	–	O(1) avg	O(1) avg	O(1) avg
BST balanced / skewed	–	O(log n) / O(n)	same	same
binary heap	min: O(1)	O(n)	O(log n)	O(log n)
trie, key length L	–	O(L)	O(L)	O(L)
union–find	–	O(α(n))	O(α(n))	–

Sort	Time (typical)	Worst	Extra space	Stable
insertion	O(n^2), O(n) if nearly sorted	O(n^2)	O(1)	yes
merge	O(n log n)	O(n log n)	O(n)	yes
quick	O(n log n)	O(n^2)	O(log n)	no
heap	O(n log n)	O(n log n)	O(1)	no
counting, radix	O(n + k)	same	O(k)	yes

C++ `std::sort` is introsort (not stable); `stable_sort` is a merge sort; Python's `sorted` is Timsort (stable).

Graph algorithm	Time
BFS, DFS, topological sort	O(V + E)
Dijkstra with a binary heap	O((V + E) log V)
Bellman–Ford	O(VE)
Floyd–Warshall	O(V^3)
Kruskal; Prim with a heap	O(E log E); O(E log V)

Recurrences: $T(n) = 2T(n/2) + O(n)$ gives O(n log n); $T(n) = T(n/2) + O(1)$ gives O(log n); $T(n) = 2T(n-1) + O(1)$ gives $O(2^n)$. Landmarks: $\log_2 10^6 \approx 20$, $\log_2 10^9 \approx 30$, $2^{10} \approx 10^3$, `INT_MAX` $\approx 2.1\times10^9$, `LLONG_MAX` $\approx 9.2\times10^{18}$.

Edge-case checklist

product, `abs(INT_MIN)`.

- **Order:** sorted, reverse sorted, rotated.
- **Strings:** empty, spaces, case, one character, leading zeros.
- **Lists:** `NULL` head, one node, cycle, operating on the head or the tail.
- **Trees:** `NULL` root, single node, skewed (depth n), duplicate keys.
- **Graphs:** disconnected, self-loop, parallel edges, cycle, unreachable target.
- **Intervals:** touching endpoints, nested intervals, ask whether adjacent ranges merge.
- **Binary search:** target below all, above all, absent, at both ends.
- **Networking:** `01.2.3.4`, `256.1.1.1`, a missing octet, /0 and /32, network and broadcast addresses, port 0 and 65535.

Code skeletons in one line each

- Prefix mask `p ? ~0u << (32-p) : 0`; same subnet `(a & m) == (b & m)`; IP to integer `a << 24 | b << 16 | c << 8 | d`.
- Bit idioms: `x & (x-1)` clears the lowest set bit; `x & -x` isolates it; `x && !(x & (x-1))` tests a power of two; `(x + a - 1) & ~(a - 1)` aligns up.
- List: `Node **pp = &head; while (*pp && (*pp)->val != k) pp = &(*pp)->next;` then `*pp = (*pp)->next`.
- Reverse: save `next`, set `head->next = prev`, advance `prev` and `head`.
- LPM walk: `best = root->hop; for i in 0..31: t = t->c[bit]; if (t && t->hop >= 0) best = t->hop`.
- Dijkstra: heap of (distance, vertex); `if (d > dist[u]) continue;` relax edges.
- Union–find: `find` with path compression; `unite` returns false if already joined.
- LRU: map key to list node; `get` moves the node to the front; `put` evicts the back and erases its map entry.
- Token bucket: refill on demand from a monotonic clock, cap at B, pass if the level covers the packet.

Flashcards

Q	A
Complexity of a loop that halves n? §1.1 (p. 9)	O(log n)
Why is `vector::push_back` O(1) amortised?	Copies at doublings total less than $2n$
Two Sum in O(n)? §1.1 (p. 9)	Hash map value to index; look up `target - a[i]`, then insert
Variable window: `if` or `while` to shrink? §1.2 (p. 12)	`while`, until the window is valid again
Seed of the prefix-sum map? §1.2 (p. 12)	`{0: 1}`, for subarrays starting at index 0
Why is `m[key]` unsafe for lookup in C++? §1.3 (p. 16)	It inserts a default value
Why reject `01.2.3.4`? §1.4 (p. 19)	Leading zeros are ambiguous (octal)
Is `strncpy` safe? §1.4 (p. 19)	No: it does not terminate when truncating
Hosts in a /27? §1.5 (p. 24)	$2^5 - 2 = 30$
Why does `x & 1 == 0` fail? §1.5 (p. 24)	`==` binds tighter; write `(x & 1) == 0`
Why special-case /0 in a mask?	Shifting a 32-bit value by 32 is undefined
Popcount by loop? §1.5 (p. 24)	`x &= x - 1` until zero
Broadcast from network and mask?	`net | ~mask`
Why use `Node **pp` for delete? §1.6 (p. 27)	The head needs no special case
Floyd: why does resetting to the head find the cycle start? §1.6 (p. 27)	$a = (k-1)L + c$, so both pointers arrive together
Order of the loop test in Floyd?	`fast` first, then `fast->next`
Ring buffer, full or empty? §1.7 (p. 33)	Keep a count (or waste a slot, or use free-running indices)
Monotonic stack stores?	Indices, not values
BST delete with two children? §1.8 (p. 36)	Copy the in-order successor, delete it from the right subtree
Why does a sorted insertion hurt a BST?	The tree becomes a chain; operations cost O(n)
Validate a BST correctly?	Pass open bounds down, or check that in-order is increasing
LPM data structure and cost? §1.9 (p. 41)	Binary trie, O(32) per lookup, independent of the table size
LPM without a trie?	One hash table per length, probe /32 down to /0
Heap array parent and children? §1.10 (p. 45)	Parent $(i-1)/2$; children $2i+1$ and $2i+2$

BFS: when to mark visited? §1.11 (p. 47)	When pushed
Dijkstra stale check?	`if (d > dist[u]) continue;`
Dijkstra with a negative edge?	Wrong; use Bellman–Ford
Bellman–Ford with a hop limit?	Run *k* rounds, relaxing from a copy
OSPF versus RIP, algorithmically?	Dijkstra on the whole graph versus distributed Bellman–Ford
Union–find: `unite` returns false means?	The edge closes a loop
Topological sort detects a cycle how?	Fewer than *V* vertices are output
Backtracking steps? §1.12 (p. 54)	Choose, explore, unchoose
Binary-search midpoint? §1.13 (p. 55)	`lo + (hi - lo) / 2`
Binary search on the answer needs?	A monotone yes/no test
Merge intervals: sort by?	Start; compare with the last merged end
The five DP steps? §1.14 (p. 58)	State, recurrence, base, order, answer cell
Why is greedy wrong for coins {1,3,4}, amount 6?	Greedy gives 4+1+1; the best is 3+3
Why is LRU O(1)? §1.15 (p. 60)	The map finds the node; the node unlinks itself from a doubly linked list
Common LRU bug?	Evicting from the list but not the map
Token-bucket refill formula?	`tokens = min(B, tokens + r·Δt)`
Fixed-window flaw?	Up to twice the limit across a window boundary
Table-driven FSM benefit? §1.16 (p. 65)	Transitions are data, reviewable and testable

PART I · PILLAR 2

2

Programming: C, Python, C++

Network software is written close to the machine; we learn to see the bytes behind every line.

2.1 Memory layout of a process	MUST
2.2 Types, sizes and conversions	MUST
2.3 Pointers and pointer arithmetic	MUST
2.4 Arrays, pointers and strings	MUST
2.5 Declarations, const and function pointers	MUST
2.6 Storage classes and qualifiers	MUST
2.7 Structs, unions, padding and alignment	MUST
2.8 Bit-fields and bit manipulation	MUST
2.9 Endianness and byte order	MUST
2.10 Dynamic memory and leaks	MUST
2.11 The preprocessor and macros	MUST
2.12 Undefined behaviour	MUST
2.13 Writing the library: memcpy to itoa	MUST
2.14 Parsing an IPv4 header	MUST
2.15 The gotcha gallery: 36 output traps	MUST
2.16 Python: the data model	SHOULD
2.17 Python: comprehensions and generators	SHOULD
2.18 Python: struct, re, subprocess, files, exceptions	MUST

2.19 C++: references, classes and RAII	SHOULD
2.20 C++: virtual functions and the vtable	SHOULD
2.21 C++: STL containers and smart pointers	SHOULD

2.0 Why this pillar matters, and how to read it

Switch operating systems, routing daemons and data-plane software are written largely in C and C++, because they must control memory layout and run without a garbage collector. Automation around them is mostly Python. Interviewers therefore test C hardest: pointers, structures, bytes and output prediction. Part A (sections 2.1–2.15) is the bulk of the pillar and is entirely Must. Part B covers the Python language only, and Part C the C++ that interviews reach.

Three boundaries keep the book free of repetition. The sockets API belongs to Pillar 7 (p. 414), shell scripting to Pillar 8 (p. 483), Python's network libraries to Pillar 10 (p. 586), and linked-list and tree algorithms to Pillar 1 (p. 7). Here we meet only the language features those chapters rely on.

All code assumes 64-bit Linux (the LP64 data model), `gcc` 13 and an x86-64 processor, unless stated otherwise. Compile with `gcc -Wall -Wextra -g file.c -o file`. Every output printed below was produced by running the code under these assumptions. Where the standard leaves behaviour open, the text says so and does not print an output.

> BIG IDEA
>
> A C program is a sequence of bytes at addresses; every type, pointer and structure is a rule for interpreting those bytes, and every trap in this pillar is a place where the rule and the bytes disagree.

2.1 Memory layout of a process MUST · ■□□

When Linux starts a program it gives the process a private *virtual address space*, a range of addresses that the hardware maps to physical memory page by page. The kernel divides this space into regions called *segments* or *sections*, and the C compiler decides which region each object lives in. Knowing the regions answers a surprising share of interview questions, among them "where does a static local live?" and "why did this big array crash my program?".

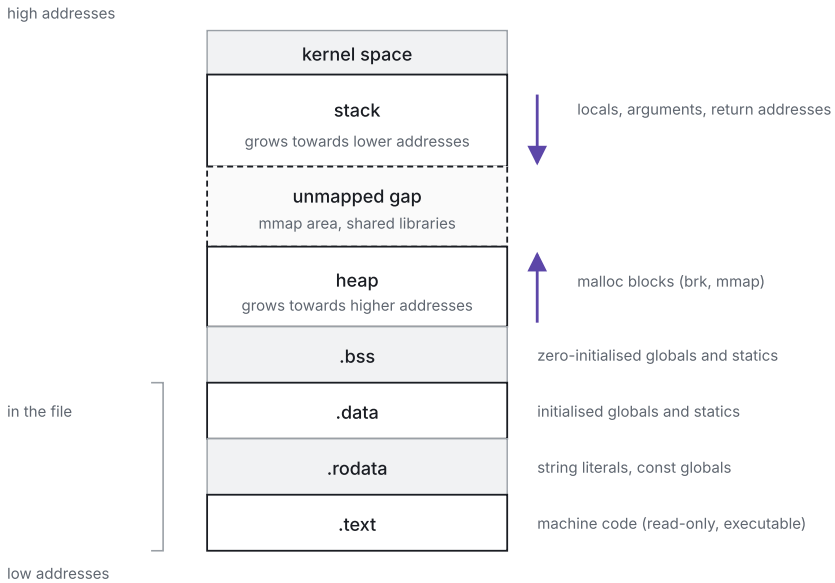


Figure 2.1 The layout of a Linux process. The three lowest sections come from the executable file; .bss occupies no space in the file.

Figure 2.1 shows the regions from the high addresses downwards. The *text* section holds machine code and is read-only. The *rodata* section holds string literals and `const` globals. The *data* section holds globals and statics with a non-zero initial value, and the *BSS* (block started by symbol) holds those initialised to zero, which is why the file records only their total size. The *heap* serves `malloc`, and the *stack* serves function calls. Between them lies unmapped space, so that neither can grow into the other without a fault.

Object	Section	Lifetime	Initial value
`int g = 7;` (global)	.data	whole program	7
`int g;` (global)	.bss	whole program	0
`static int s;` (file or function scope)	.bss (.data if initialised)	whole program	0
`const int k = 3;` (global)	.rodata	whole program	3
`"hello"` (string literal)	.rodata	whole program	the bytes
`int x;` (local)	stack	until the block ends	indeterminate
`malloc(n)`	heap	until `free`	indeterminate

WORKED EXAMPLE

Classify the objects in a small program.

1. Compile the program below and print the address of one object from each row of the table.
2. Read the addresses in order. Code, literals and globals sit close together near the bottom, the heap lies above them, and the stack is far higher.
3. Check the sizes with `size`: the BSS figure counts the zero-initialised bytes, while `text` includes the read-only data.

```
int g_init = 7;  int g_zero;  const int g_ro = 3;
int main(void) {
    static int s_init = 9;      /* .data */
    int local = 1;              /* stack */
    char *lit = "hello";        /* pointer: stack, bytes: .rodata */
    int *h = malloc(sizeof *h); /* heap */
    printf("%p %p %p\n", (void *)&g_ro, (void *)h, (void *)&local);
    free(h); (void)s_init; (void)lit; return 0;
}
```

Result: on one run the three addresses were `0x5595455f9004`, `0x55954f44b2a0` and `0x7fffa525c4f4`, in increasing order. Exact values change on every run because of address-space randomisation; only the order is stable.

TRAP

Claim: a `static` local array is stored on the stack like any other local. *Why it is wrong:* `static` changes the lifetime, not just the visibility, so the array is placed in .data or .bss and exists for the whole run. *Correct statement:* making a large local array `static` removes the stack-overflow risk, but the function is then no longer re-entrant, because every call shares the same bytes.

INTERVIEW LENS

QUALCOMM · ARISTA · CISCO

> “What kind of variable is stored where?” is a reported Qualcomm question, and Arista has asked candidates to print and classify the addresses of a global, a static, a local, a heap object and a literal. A strong answer draws Figure 2.1 from memory, names the section for each object, and adds that the stack and heap grow towards each other. Follow-ups: what is the BSS for, and what happens on stack overflow (a segmentation fault from the guard page).

CHECK YOURSELF

1. In which section does `static int n = 0;` inside a function live? [1]
2. Why does the BSS take no room in the executable file? [2]
3. Where is the pointer `char *p = "abc";` stored, and where are the characters? [3]

2.2 Types, sizes and conversions MUST · ■□□

The size of `int` is not fixed by the language. The standard guarantees only that `char` is one byte and that `sizeof(short) <= sizeof(int) <= sizeof(long)`. On 64-bit Linux (the *LP64* model) `int` is 4 bytes while `long` and pointers are 8; on 32-bit systems (*ILP32*) all three are 4. The full size table is in the Revision Spread (p. 159). Code that handles packet fields must therefore use the fixed-width types of `<stdint.h>`: `uint8_t`, `uint16_t`, `uint32_t` and `uint64_t`, printed with the macros `PRIu32` and its relatives from `<inttypes.h>`.

Arithmetic does not happen in the narrow type. Two rules govern every mixed expression. First, the *integer promotions* widen `char` and `short` operands to `int`. Second, the *usual arithmetic conversions* bring the two operands to a common type: if one is `unsigned int` and the other `int`, the signed one is converted to unsigned. This is the origin of several famous surprises.

Integer promotion — in an expression, any type narrower than `int` is first widened to `int`, so `char + char` is computed as `int`.

```
uint8_t x = 200, y = 100;
printf("%d\n", x + y);                 /* 300, not 44: computed as int */
unsigned int u = 1; int b = -1;
printf("%s\n", b < u ? "true" : "false"); /* false: -1 becomes 4294967295 */
```

The first line prints `300` because both operands are promoted; the sum is truncated to 44 only when stored back into a `uint8_t`. The second prints `false` because `-1` converted to `unsigned int` is `UINT_MAX`. Hence, comparing a signed and an unsigned value is a defect unless one has first proved the signed value is non-negative; `gcc -Wextra` warns about it.

Signedness of `char` is implementation-defined: it is signed on x86 and unsigned on most ARM Linux builds. Thus `char c = 200;` stores -56 on x86 and 200 on ARM, and any code that indexes a table with a plain `char` is a latent bug. Use `unsigned char` or `uint8_t` for bytes.

Overflow behaves differently by signedness. Unsigned arithmetic wraps modulo 2^n and is fully defined, which is exactly what a 32-bit byte counter needs: with `uint32_t before, after`, the expression `after - before` yields the correct delta even after the counter wraps. Signed overflow is undefined behaviour (2.12 (p. 121)).

2.3 Pointers and pointer arithmetic MUST · ■□□

A *pointer* is a variable that holds an address. If `int x = 5;` then `&x` is the address of `x`, and for `int *p = &x;` the expression `*p` is the object at that address, so `*p = 9;` changes `x`. A pointer that points nowhere holds `NULL`, and dereferencing it is undefined behaviour that in practice raises a segmentation fault. A pointer of type `T *` knows the size of `T`, and this size drives all pointer arithmetic.

BIG IDEA

Adding `n` to a `T *` moves the address by `n * sizeof(T)` bytes, so pointer arithmetic counts elements, not bytes.

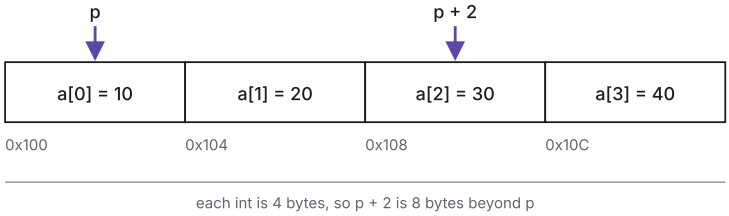


Figure 2.2 With `int *p = a;`, the pointer `p + 2` holds 0x108, because the address advances by two elements of four bytes.

The rules are short. Adding or subtracting an integer moves the pointer by that many elements. Subtracting two pointers into the same array gives the number of elements between them, of type `ptrdiff_t`. Comparing pointers is meaningful only within one array. A `void *` carries no element size, so it cannot be dereferenced or incremented in standard C; it converts implicitly to and from any object pointer type in C, which is why `malloc` returns it. To step through raw bytes, cast to `unsigned char *`.

```
int a[4] = {10, 20, 30, 40};
int *p = a;
printf("%d %d\n", *(p + 2), p[3]);                    /* 30 40 */
printf("%ld\n", (long)(&a[3] - &a[0]));               /* 3: elements   */
printf("%ld\n", (long)((char *)(p + 1) - (char *)p)); /* 4: bytes      */
```

A function receives copies of its arguments, so to modify the caller's variable it must receive its address. The same idea with one more level gives the pointer to pointer, needed whenever a function must change a pointer itself, as when it allocates memory for the caller.

```
void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }

int make_buffer(char **out, size_t n) {   /* caller's pointer is changed */
    *out = malloc(n);
    return *out ? 0 : -1;
}
```

TRAP

Claim: `p + 1` always adds one to the address. *Why it is wrong:* the increment is scaled by the pointee size, so for `int *p` the address grows by 4, and for `struct ethhdr *` by 14. *Correct statement:* to move by bytes, cast to `unsigned char *` first, as in `(uint8_t *)p + 1`.

CHECK YOURSELF

1. If `long *q` holds 0x2000, what does `q + 3` hold? [4]
2. What does `&a[3] - &a[0]` evaluate to for `int a[4]`, and what is its type? [5]

2.4 Arrays, pointers and strings MUST · ■□□

An array is a contiguous block of elements, and a pointer is a single address; they are different things that the language lets one mix. In nearly every expression the name of an array *decays* to a pointer to its first element, so `a[i]` is defined as `*(a + i)`. The exceptions are the operand of `sizeof`, the operand of `&`, and a string literal that initialises an array. Because `a[i]` means `*(a + i)` and addition commutes, even `i[a]` is legal and means the same.

```
int m[2][3] = {{1, 2, 3}, {4, 5, 6}};
printf("%d %d %zu\n", m[1][2], *(*(m + 1) + 2), sizeof m);  /* 6 6 24 */
printf("%zu %zu\n", sizeof(m[0]), sizeof(&m[0]));           /* 12 8   */
```

A two-dimensional array is an array of arrays laid out row by row, so `m[0]` is itself an array of 3 ints (12 bytes) and `&m[0]` is a pointer to it (8 bytes). Decay has one consequence that every interviewer tests: an array *parameter* is really a pointer. In `void f(int a[10])` the `10` is ignored and `sizeof(a)` is 8. Hence a function that needs the length must receive it as a separate argument.

A C *string* is an array of `char` ending with a zero byte `'\0'`. Its length excludes the terminator, while its storage includes it. Two declarations look alike and differ completely, as Figure 2.3 shows.

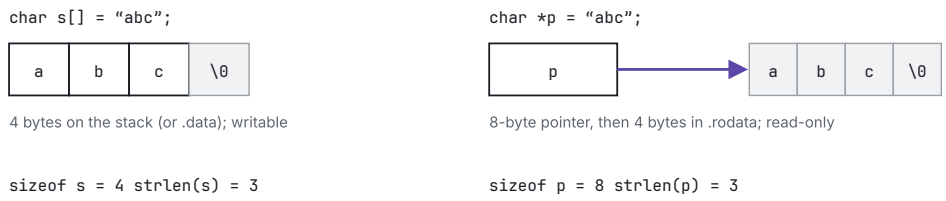


Figure 2.3 An array owns its characters; a pointer to a literal merely refers to read-only characters.

Writing through `p[0] = 'A'` in the second form is undefined behaviour and normally faults, because the literal is in read-only memory. With the array form the write is legal, since the array is a private, writable copy.

The standard string functions all walk to the terminator. `strlen(s)` counts characters and is O(n). `strcpy(d, s)` copies until and including the terminator and does not check that `d` is large enough; this is the classic buffer overflow. `strncpy(d, s, n)` is not its safe version: if `s` has `n` or more characters it writes no terminator, and if it has fewer it pads `d` with zeros up to `n`. The reliable idiom is `snprintf(d, sizeof d, "%s", s)`, which always terminates. `strcmp` returns zero for equal strings and otherwise a negative or positive value, so `if (strcmp(a, b))` is true when the strings *differ*.

> **LAB**
>
> `LAB 2.1 · 15 MIN`
>
> ```
> gcc -Wall -g -o strdemo strdemo.c      # strdemo.c: the two declarations of Figure 2.3
> ./strdemo
> gcc -g -fsanitize=address -o strdemo strdemo.c && ./strdemo
> ```

With `p[0] = 'A'` uncommented, the plain build prints `Segmentation fault`. The sanitised build prints `AddressSanitizer: SEGV on unknown address` and a stack trace whose top frame names the line.

What you should notice: the array form never faults, and the sanitiser reports the line, which is quicker than reading a core dump.

CHECK YOURSELF

1. What does `sizeof(a)` return inside `void f(int a[10])` on a 64-bit machine? [6]
2. After `strncpy(d, "abcdef", 3)`, is `d` a valid string? [7]

2.5 Declarations, const and function pointers MUST · ■■□

C declarations are read from the identifier outwards: start at the name, look right, then left, and respect the parentheses. In `int *const p` the name `p` is a constant, and it is a pointer to `int`. The qualifier `const` applies to the thing immediately on its left, or to the type on its right when nothing stands on its left. Three forms cover every interview question.

Declaration	Reads as	May change `*p`?	May change `p`?
`const int *p` (same as `int const *p`)	`p` is a pointer to a constant `int`	no	yes
`int *const p`	`p` is a constant pointer to `int`	yes	no
`const int *const p`	constant pointer to a constant `int`	no	no

TRAP

Claim: `const char *s` means the string cannot change, so the pointer cannot move either. *Why it is wrong:* here `const` binds to `char`, the pointee; the pointer is free to move, as in `s++`. *Correct statement:* to forbid moving the pointer one writes `char *const s`, and to forbid both, `const char *const s`. Functions that only read a buffer should take `const T *`, which documents intent and lets the compiler reject accidental writes.

A *function pointer* holds the address of code. Its declaration mirrors a function prototype with the name wrapped as `(*name)`: `int (*fp)(int, int);` declares `fp` as a pointer to a function taking two `int` and returning `int`. The parentheses matter, because `int *fp(int, int)` declares a function returning `int *`. A `typedef` makes such types readable, and calling through the pointer needs no dereference: `fp(2, 3)` and `(*fp)(2, 3)` are the same.

Function pointers appear in two interview patterns. The first is the *callback*: the library calls code the caller supplies, as `qsort` does with its comparator. The second is the *dispatch table*: an array of function pointers indexed by a small integer, replacing a long `switch`. Network code uses it to route a packet to a handler by IP protocol number.

1. .bss, because the initial value is zero; `static int n = 5;` would be in .data.↩
2. Its contents are known to be zero, so the file records only the size and the loader supplies zeroed pages.↩
3. The pointer follows the rule for its own storage (stack for a local, .data for a global); the characters are in .rodata.↩
4. 0x2018, since 3 elements of 8 bytes add 24 bytes.↩
5. 3, of type `ptrdiff_t` (a signed integer); the unit is elements, not bytes.↩
6. 8, because the parameter is a pointer; the length 10 is ignored.↩
7. Not necessarily: three characters were copied and no terminator was written, so `d` is terminated only if its fourth byte happened to be zero.↩

```c
typedef int (*handler_fn)(const uint8_t *pkt, size_t len);

static int on_icmp(const uint8_t *p, size_t n) { (void)p; printf("icmp %zu\n",
n); return 0; }
static int on_tcp (const uint8_t *p, size_t n) { (void)p; printf("tcp %zu\n",
n); return 0; }
static int on_udp (const uint8_t *p, size_t n) { (void)p; printf("udp %zu\n",
n); return 0; }

static handler_fn table[256] = { [1] = on_icmp, [6] = on_tcp, [17] = on_udp };

int dispatch(uint8_t proto, const uint8_t *pkt, size_t len) {
    handler_fn fn = table[proto];        /* unset entries are NULL        */
    if (!fn) { printf("proto %u: no handler\n", proto); return -1; }
    return fn(pkt, len);
}
```

Calling `dispatch(6, buf, 40)`, `dispatch(17, buf, 8)` and `dispatch(47, buf, 0)` prints `tcp 40`, `udp 8` and `proto 47: no handler`. The designated initialisers `[1] = on_icmp` set only the named slots, and a static array zeroes the rest, so the `NULL` test is reliable. The lookup costs one indexed load and one indirect call, independent of the number of protocols.

The comparator of `qsort` receives `const void *` and must cast back. The safe return expression is `(x < y) - (x > y)`, because the shortcut `x - y` overflows for extreme values, which is undefined behaviour (2.12 (p. 121)).

```c
int cmp_desc(const void *a, const void *b) {
    int x = *(const int *)a, y = *(const int *)b;
    return (x < y) - (x > y);              /* descending order */
}
/* int v[] = {3, 9, 1, 7};  qsort(v, 4, sizeof v[0], cmp_desc);  ->  9 7 3 1 */
```

Sorting a routing table needs two keys, longest prefix first and then address ascending. The comparator tests the length first and falls through to the address only on a tie: `if (x->len != y->len) return (int)y->len - (int)x->len;`. With prefixes `10.0.0.0/8`, `192.168.1.0/24`, `10.1.0.0/16` and `192.168.0.0/24` the sorted order is `192.168.0.0/24`, `192.168.1.0/24`, `10.1.0.0/16`, `10.0.0.0/8`.

> INTERVIEW LENS
>
> CISCO · JUNIPER
>
> “What are function pointers? Show how you would dispatch packets by protocol number.” is reported as an interview topic at both companies. A strong answer shows the `typedef`, the table with designated initialisers, the `NULL` check, and says why a table beats a `switch` when handlers are registered at run time. Follow-up: how would a module register its own handler (a `register_handler(proto, fn)` function that writes the slot).

> CHECK YOURSELF
>
> 1. What does `const char *const s` forbid? [1]
> 2. Why is `int *fp(int, int)` not a function pointer? [2]

2.6 Storage classes and qualifiers MUST · ■■□

The *storage class* of an object fixes its lifetime and linkage, and a *qualifier* restricts how the compiler may treat accesses. Both are interview staples, and the word `static` alone has three meanings.

`static`. (i) On a local variable it gives the variable static storage duration: one instance, created before `main`, zero-initialised, keeping its value between calls. (ii) On a global variable or function it gives *internal linkage*: the name is visible only inside that translation unit (one `.c` file after preprocessing), so two files may each define `static int count;` without a clash. (iii) A `static` array inside a function therefore moves from the stack to .data or .bss, as noted in 2.1 (p. 107).

`extern`. It declares an object defined elsewhere and allocates nothing. A header should declare (`extern int g;`) and exactly one `.c` file should define (`int g = 0;`). Defining a variable in a header gives each including file its own copy, or a linker error for duplicates; marking it `static` in the header silently gives every file a private copy, which is almost never intended.

`auto` **and** `register`. `auto` is the default for locals and is never written. `register` is a historical hint that forbids taking the object's address; compilers now ignore it for optimisation.

`const`. It makes writes through that name an error. A global `const` usually lives in .rodata, so casting `const` away and writing to it is undefined behaviour. A `const` local is still on the stack but the compiler may assume it never changes.

`volatile`. It tells the compiler that an object may change in ways the code does not show, so every access in the source must really happen, in order. Three situations need it: a memory-mapped hardware register, a flag set by a signal handler or interrupt service routine, and a variable modified by `setjmp`/`longjmp` paths. Consider the loop `while (!flag) ;` waiting for an interrupt.

```
int flag;            void wait_plain(void) { while (!flag) ; }
volatile int vflag;  void wait_vol(void)   { while (!vflag) ; }
```

LAB

LAB 2.2 · 10 MIN

```
gcc -O2 -S -o - v.c | grep -v '^\s*\.'
```

```
wait_plain:  movl flag(%rip),%eax ; testl %eax,%eax ; jne .L1 ; jmp .L3
wait_vol:    movl vflag(%rip),%eax ; testl %eax,%eax ; je .L6 ; ret
```

What you should notice: `wait_plain` loads `flag` once and then spins on `jmp .L3` without reading memory again, so it never sees the change. `wait_vol` branches back to the load on each pass (the `.L6` label), because every access to a `volatile` object must happen.

A `const volatile` object is legal and useful: the program may not write it, but hardware may change it, as with a read-only status register. In an embedded driver one writes `volatile const uint32_t *status = (volatile const uint32_t *)0x40021000;`.

TRAP

> *Claim:* `volatile` makes a shared flag safe between two threads. *Why it is wrong:* `volatile` prevents the compiler from caching a value but provides neither atomicity nor memory ordering between processors, so a read may see a stale value or a torn one on a multi-core machine. *Correct statement:* use `_Atomic` (C11) or the primitives of Pillar 7 for threads; reserve `volatile` for hardware registers and signal handlers (and there prefer `volatile sig_atomic_t`).

`restrict` and `inline`. `restrict` is a promise by the programmer that, during the pointer's lifetime, no other pointer reaches the same object; `memcpy` is declared with `restrict` parameters, which is exactly why overlapping use is undefined. `inline` suggests substituting the function body at the call site; the compiler decides. Together with `static` it is the type-safe alternative to a function-like macro (2.11 (p. 120)). The two tables that summarise qualifiers and storage classes are in the Revision Spread (p. 159).

> CHECK YOURSELF
>
> 1. Two files each contain `static int n;`. Does the linker report a clash? [3]
> 2. Why can a `volatile` read not be removed by the optimiser even when its result is unused? [4]

2.7 Structs, unions, padding and alignment MUST · ■■□

A `struct` groups members laid out in declaration order. The compiler may not reorder them, but it may insert unused bytes called *padding*, because processors read an `n`-byte value most efficiently, or only correctly, at an address divisible by `n`. This divisibility rule is the member's *alignment requirement*. Three rules give the layout of any structure.

> BIG IDEA
>
> Each member sits at the next offset divisible by its own alignment, and the structure's size is rounded up to a multiple of its largest member's alignment so that arrays of the structure stay aligned.

On LP64 the alignment of each basic type equals its size: `char` 1, `short` 2, `int` 4, `long` and pointers 8, `double` 8. Figure 2.4 applies the rules to one structure and to its reordering.

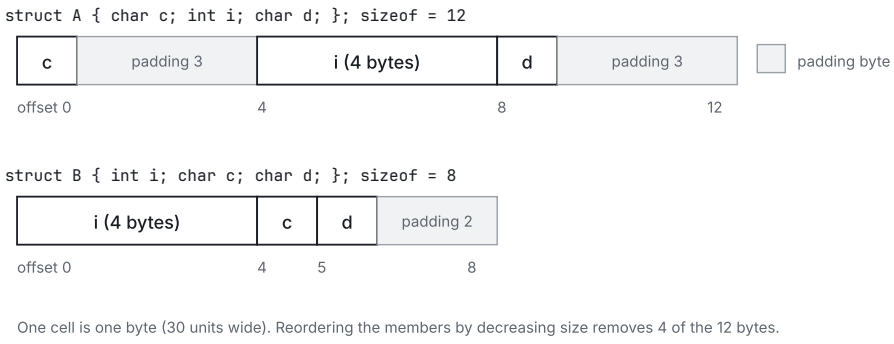


Figure 2.4 Padding in `struct A` and its reordering. Shaded cells are padding and hold unspecified values.

In `struct A` the `int` cannot start at offset 1, so three padding bytes follow `c`. The last `char` ends at offset 9, but the size must be a multiple of 4, so three more bytes follow it. Placing the members in decreasing size order, as in `struct B`, needs only two. The same method answers

the reported Qualcomm questions: for `struct { char a; int b; short c; }` the offsets are 0, 4 and 8 and the size is 12; reordered as `int, short, char` it is 8. For `struct { short s; long l; char c; }` the offsets on LP64 are 0, 8 and 16 and the size is 24; on a 32-bit x86 system `long` is 4 bytes with alignment 4, so the offsets are 0, 4 and 8 and the size is 12.

A `union` stores all its members at offset 0, so its size is the largest member rounded up to the strictest alignment. For `union { int i; char c[5]; }` the size is 8, not 5. Only the member last written is meaningful in the portable sense; reading another member reinterprets the bytes (*type punning*), which C99 permits through unions but C++ does not. The usual use is the *tagged union*, a structure with a tag and a union of alternatives, which stores one of several message formats in the space of the largest.

```
struct msg { enum { MSG_HELLO, MSG_DATA } type;
             union { struct { uint32_t id; }            hello;
                     struct { uint16_t len; uint8_t b[64]; } data; } u; };
```

Padding can be removed with `__attribute__((packed))` or `#pragma pack(1)`, which gives `struct A` the size 6. The price is misaligned member access: x86 handles it at some cost, but strict-alignment processors may fault or take a trap, and taking a pointer to a packed member is a warning for this reason. Packing is therefore acceptable for describing a layout that a protocol dictates, provided members are read with `memcpy` or byte shifts. Two further hazards are worth knowing. Comparing structures with `memcmp` is wrong, because the padding bytes are unspecified. Sending a structure straight over a socket is wrong, because padding, member sizes and byte order all differ between machines (2.9 (p. 117)); fields must be serialised one by one.

The standard macro `offsetof(type, member)` from `<stddef.h>` gives a member's byte offset. Its inverse, `container_of`, recovers the enclosing structure from a pointer to a member, and is the basis of the intrusive lists of the Linux kernel.

```
#define container_of(ptr, type, member) \
    ((type *)((char *)(ptr) - offsetof(type, member)))
```

WORKED EXAMPLE

Size of a structure without `sizeof` (a reported Qualcomm question).

1. Declare one object `struct A a;`.
2. Take the address one element past it, `&a + 1`; pointer arithmetic scales by the whole structure.
3. Cast both addresses to `char *` and subtract, so that the unit becomes bytes.

Result: `(char *)(&a + 1) - (char *)&a` **evaluates to 12 for** `struct A`**.**

INTERVIEW LENS

QUALCOMM · CISCO · BROADCOM

Structure versus union, padding with a worked size, and reordering to minimise size are the most frequently reported C questions. A strong answer states the three rules, draws the byte map as in Figure 2.4, and mentions packing and its cost. The follow-up is nearly always "can you send this over the network?".

CHECK YOURSELF

1. What is `sizeof(struct { char a; short b; char c; })` on LP64? [5]
2. What is the minimum size after reordering? [6]

2.8 Bit-fields and bit manipulation MUST · ■■□

Embedded and protocol code reads and writes individual bits constantly. The four basic operations on an unsigned integer `x` use a mask with a single 1 at bit `n`: set with `x |= 1u << n`, clear with `x &= ~(1u << n)`, toggle with `x ^= 1u << n`, and test with `(x >> n) & 1u`. The `u` suffix matters, because `1 << 31` shifts into the sign bit of an `int`, which is undefined (2.12 (p. 121)). As macros, with every argument parenthesised (2.11 (p. 120)):

```
#define SET_BIT(x, n)    ((x) |=  (1u << (n)))
#define CLEAR_BIT(x, n)  ((x) &= ~(1u << (n)))
#define TOGGLE_BIT(x, n) ((x) ^=  (1u << (n)))
#define TEST_BIT(x, n)   (((x) >> (n)) & 1u)
```

A multi-bit *field* is read by shifting and masking, and written by clearing the field before inserting the value. This read-modify-write helper is the classic register-access question.

```
static inline uint32_t get_field(uint32_t reg, unsigned shift, unsigned width) {
    return (reg >> shift) & ((1u << width) - 1u);
}
static inline uint32_t set_field(uint32_t reg, unsigned shift, unsigned width,
uint32_t val) {
    uint32_t mask = ((1u << width) - 1u) << shift;     /* width < 32 assumed */
    return (reg & ~mask) | ((val << shift) & mask);
}
```

Two idioms recur. Kernighan's population count loops `while (x) { x &= x - 1; n++; }`, because `x & (x - 1)` clears the lowest set bit, so the loop runs once per set bit. A power-of-two test is `x != 0 && (x & (x - 1)) == 0`, since a power of two has a single set bit. Further bit-level exercises appear in Pillar 1 (p. 7).

Bit-fields let the compiler do the shifting: `struct { unsigned ihl:4, version:4; }`. They are convenient, yet the standard leaves the order of fields within a byte, the byte order of larger units and the behaviour of fields straddling units to the implementation. The experiment below shows the same value assigned to the same two fields in two declaration orders.

```
struct bf1 { unsigned int version:4, ihl:4; };   /* version declared first */
struct bf2 { unsigned int ihl:4, version:4; };   /* ihl declared first     */
/* with version = 4, ihl = 5, first byte on x86-64 gcc:  bf1 -> 0x54,  bf2 ->
0x45 */
```

On this platform gcc places the first-declared field in the low-order bits, so only `bf2` produces the wire byte `0x45` of an IPv4 header, and the correct declaration order is reversed on a big-endian target. Hence, bit-fields are a poor choice for wire formats and are best kept for private data and hardware registers of a known compiler; portable code extracts fields with shifts and masks, as in 2.14 (p. 124).

2.9 Endianness and byte order MUST · ■■□

A 32-bit integer occupies four bytes, and *endianness* is the order in which they are stored. A *big-endian* machine stores the most significant byte at the lowest address; a *little-endian* machine

stores the least significant byte first. x86 and most ARM systems run little-endian, whereas the Internet protocols are defined in big-endian order, termed *network byte order*.

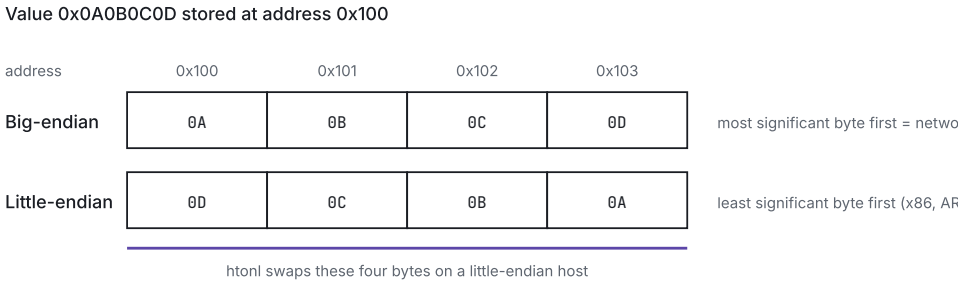


Figure 2.5 The same 32-bit value in memory on a big-endian and a little-endian machine.

Byte order concerns bytes, not bits: the bits inside a byte are not reordered by any API. The headers `<arpa/inet.h>` provide four conversions: `htons` and `htonl` (host to network, 16- and 32-bit) and `ntohs` and `ntohl` (network to host). On a big-endian host they do nothing, so code that calls them is correct on both kinds of machine. If the call is forgotten, the program works against itself and fails against others: the port 80 is `0x0050`, which a little-endian host stores as the bytes `50 00`, and a peer reading those bytes in network order sees `0x5000`, which is 20480.

```
uint16_t port = 80;
printf("%04x %04x\n", port, htons(port));              /* 0050 5000 */
uint8_t w[2] = {0x00, 0x50};  uint16_t raw;  memcpy(&raw, w, 2);
printf("%u %u\n", raw, ntohs(raw));                    /* 20480 80  */
```

To detect the host order, store 1 in an integer and inspect the byte at the lowest address. The union form is defined behaviour; reading through an `unsigned char *` also works, because character types may alias any object.

```
int is_little_endian(void) {
    union { unsigned int i; unsigned char c[4]; } u = { .i = 1 };
    return u.c[0] == 1;                /* least significant byte stored first
*/
}
uint32_t swap32(uint32_t x) {
    return (x >> 24) | ((x >> 8) & 0x0000FF00u) | ((x << 8) & 0x00FF0000u) | (x
<< 24);
}
```

For `x = 0x0A0B0C0D`, `swap32(x)` returns `0x0D0C0B0A`, which is what `htonl` produces on this host. A final rule governs reading wire data: do not cast a byte pointer to `uint32_t *` and dereference, because the address may be misaligned and the cast violates aliasing rules. Assemble the value from bytes, `(p[0] << 24) | (p[1] << 16) | (p[2] << 8) | p[3]`, which is correct on every host without any conversion call. The socket calls that use `htons` for ports are treated in Pillar 7 (p. 414).

> INTERVIEW LENS
>
> CISCO · JUNIPER · ARISTA · BROADCOM
>
> “Write a program to detect endianness” and “why do we need `htons`?” are reported at Cisco, and byte-order questions are common at every systems employer. A strong answer draws Figure 2.5, writes the union test, implements `swap32` with shifts, and explains that the conv-

ersion functions are portable no-ops on big-endian hosts. Follow-up: are the bits inside a byte also swapped (no, bits are not addressable, so only bytes move).

CHECK YOURSELF

1. What are the four bytes of 0x12345678 in memory on x86? [7]
2. Why is `*(uint32_t *)buf` a poor way to read a header field? [8]

2.10 Dynamic memory and leaks MUST · ■■□

Automatic variables vanish when their block ends, so data that must outlive a call, or whose size is known only at run time, comes from the heap. `malloc(n)` returns a block of `n` uninitialised bytes or `NULL`; `calloc(k, size)` returns `k * size` zeroed bytes and fails cleanly if the product overflows; `realloc(p, n)` resizes a block, possibly moving it; `free(p)` returns it. All blocks come from the allocator, which keeps a few header bytes before each block, and `free(NULL)` is defined to do nothing.

```
int *a = malloc(100 * sizeof *a);      /* sizeof *a stays correct if the type
changes */
if (!a) return -1;                     /* always test */
int *tmp = realloc(a, 200 * sizeof *a);
if (!tmp) { free(a); return -1; }      /* on failure the old block is still
valid */
a = tmp;
free(a); a = NULL;                     /* NULL makes a second free harmless */
```

The idiom with `tmp` is mandatory. In `p = realloc(p, n)` a failure overwrites `p` with `NULL` and the only reference to the old block is lost. When `realloc` cannot grow a block in place it allocates a new one, copies the contents and frees the old block, so every pointer into the old block becomes invalid.

Five defects account for nearly all heap bugs, and each has a tool that finds it.

Defect	What happens	Tool that reports it
Memory leak	the last pointer to a block is lost or `free` is never called	valgrind, LeakSanitizer
Use after free	the block is read or written after `free`	AddressSanitizer, valgrind
Double free	`free` on the same pointer twice; the allocator's lists are corrupted	AddressSanitizer, glibc abort message
Buffer overflow	a write past the end of the block	AddressSanitizer, valgrind
Dangling pointer	a pointer outlives its object, whether freed heap or an expired local	AddressSanitizer (heap and stack)

A *leak* costs nothing in a short program and exhausts memory in a router process that runs for months; it is therefore treated as a defect. The program below loses its only pointer.

```
char *a = malloc(16);
strcpy(a, "leak");
a = NULL;                  /* last reference lost */
```

LAB

1. Both modifying the characters through `s` and reassigning `s` itself.↩
2. Without parentheses around `*fp` the declarator binds as a function returning `int *`; a pointer needs `int (*fp)(int, int)`.↩
3. No; `static` gives internal linkage, so each file has its own `n`.↩
4. The access may itself have an effect, such as clearing a status bit in a device register, so the standard treats it as observable behaviour.↩
5. Offsets 0, 2, 4, so the members end at 5; rounding to a multiple of 2 gives 6.↩
6. Order `short, char, char` gives offsets 0, 2, 3, ending at 4, and the size is 4.↩
7. `78 56 34 12`, in increasing address order, since x86 is little-endian.↩
8. The address may be misaligned, the cast breaks the aliasing rules, and the value is in network order and still needs `ntohl`.↩

> LAB 2.3 · 10 MIN
>
> ```
> gcc -g -fsanitize=address -o leak leak.c && ./leak
> ```
>
> ```
> ERROR: LeakSanitizer: detected memory leaks
> Direct leak of 16 byte(s) in 1 object(s) allocated from:
>     #1 ... in main leak.c:4
> SUMMARY: AddressSanitizer: 16 byte(s) leaked in 1 allocation(s).
> ```
>
> **What you should notice:** the report names the size and the line of the `malloc`. With valgrind, `valgrind --leak-check=full ./leak` prints the same fact as “definitely lost: 16 bytes in 1 blocks”.

Returning a pointer to a local is the stack counterpart. `char *f(void) { char buf[16] = "x"; return buf; }` returns the address of storage that is dead the moment `f` returns; the compiler warns, and the behaviour is undefined. The remedies are to let the caller pass the buffer, to allocate on the heap, or to return a `static` buffer (not re-entrant).

> TRAP
>
> *Claim:* after `free(p)` the pointer is `NULL`, or at least safe to test. *Why it is wrong:* `free` receives a copy of `p` and cannot change the caller’s variable, so `p` still holds the old address. *Correct statement:* assign `p = NULL` yourself after `free`; then a second `free(p)` is harmless and a dereference faults immediately instead of corrupting memory silently.

> CHECK YOURSELF
>
> 1. Why does `calloc(n, 4)` differ from `malloc(n * 4)` apart from zeroing? [1]
> 2. What happens to other pointers into a block after `realloc` moves it? [2]

2.11 The preprocessor and macros MUST · ■■□

Before compilation the *preprocessor* performs textual substitution; it knows nothing of types or scopes. A macro `#define NAME text` replaces each occurrence of `NAME` by `text`, and a function-like macro `#define F(x) text` does so with its arguments pasted in unevaluated. Everything that goes wrong follows from this single fact.

```
#define SQ(x)  x*x
#define SQ2(x) ((x)*(x))
SQ(2+3)    /* 2+3*2+3  = 11 */      SQ2(2+3)  /* ((2+3)*(2+3)) = 25 */
10/SQ(2)   /* 10/2*2   = 10 */      #define MAX(a,b) ((a)>(b)?(a):(b))
MAX(i++, j)/* i++ is evaluated twice: with i = 1, j = 0 the result is 2 and i
becomes 3 */
```

Hence three rules: parenthesise every parameter and the whole body; never pass an argument with side effects; and prefer a `static inline` function when types are known, because it evaluates its arguments once and is type-checked. A macro remains the right tool for code that must stay generic, for `__FILE__` and `__LINE__` capture, and for conditional compilation.

A multi-statement macro must be wrapped as `do { ... } while (0)`. Without it, `#define SWAP(a,b) t=a; a=b; b=t;` used as `if (c) SWAP(x,y); else ...` breaks the `if` and fails to compile, because only the first statement belongs to the `if`. The wrapper turns the body into one statement that takes the trailing semicolon.

Two operators help. The `#` operator turns an argument into a string and `##` pastes two tokens. To stringify the *value* of a macro one needs two levels, because `#` stops expansion.

```
#define N 4
#define STR(x)  #x
#define XSTR(x) STR(x)
#define CAT(a,b) a##b
STR(N)  /* "N" */    XSTR(N)  /* "4" */    int CAT(x,1) = 5;   /* declares x1 */
```

Include guards prevent a header being processed twice: wrap it in `#ifndef NET_H`, `#define NET_H`, …, `#endif`, or use `#pragma once`. Conditional compilation (`#ifdef DEBUG`) selects code at build time.

The *X-macro* keeps a list in one place and generates several tables from it, so that an enumeration and its printable names can never fall out of step. It is a favourite way to write protocol state tables.

```
#define STATES(X) X(CLOSED) X(LISTEN) X(ESTABLISHED)
#define AS_ENUM(n) S_##n,
#define AS_STR(n)  #n,
enum state { STATES(AS_ENUM) S_COUNT };              /* S_CLOSED, S_LISTEN,
..., 3 */
static const char *names[] = { STATES(AS_STR) };     /* "CLOSED", "LISTEN", ...
*/
/* names[S_LISTEN] is "LISTEN" */
```

> INTERVIEW LENS
>
> CISCO · QUALCOMM
>
> “What is wrong with `#define SQUARE(x) x*x`? What happens with `SQUARE(i++)`?” and “macro versus inline function” are reported Cisco questions. A strong answer shows `SQ(2+3)` giving 11, the parenthesised fix, and that `SQ2(i++)` multiplies `i++` by `i++`, an unsequenced double modification and therefore undefined. Follow-up: when is a macro better than an inline function (generic code, stringification, `__LINE__`).

> CHECK YOURSELF
>
> 1. What does `XSTR(N)` give when `N` is defined as `4`, and why does `STR(N)` differ? [3]
> 2. Why wrap a multi-statement macro in `do { } while (0)`? [4]

2.12 Undefined behaviour MUST · ■■□

The standard distinguishes three kinds of unspecified outcome. *Implementation-defined* behaviour varies but must be documented, such as the signedness of `char` or the result of right-shifting a negative number. *Unspecified* behaviour is a choice among allowed outcomes that need not be documented, such as the order in which function arguments are evaluated. *Undefined behaviour* (UB) places no requirement at all on the program: it may crash, print the expected result, or be compiled to something else entirely.

> BIG IDEA
>
> The compiler assumes that undefined behaviour never occurs and optimises on that assumption, so a program with UB can fail in ways unrelated to the faulty line.

The common sources are the following. (i) Signed integer overflow. (ii) Dereferencing `NULL`, a dangling or an out-of-bounds pointer, including one past the end. (iii) Reading an uninitialised automatic variable. (iv) Modifying an object twice between sequence points, as in `i = i++ + ++i`. (v) Shifting by a negative count or by the type's width or more, and left-shifting into the sign bit. (vi) Modifying a string literal or a `const` object. (vii) Accessing an object through a pointer of an incompatible type (the *strict aliasing* rule). (viii) Using memory after `free`, or freeing twice. (ix) Falling off the end of a non-`void` function and using the result. (x) A data race between threads.

The consequences are visible in generated code. The function `int f(int x) { return x + 1 > x; }` is compiled by gcc to `movl $1, %eax; ret`: signed overflow cannot happen, so the comparison is always true, although for `x = INT_MAX` the machine addition would wrap. A check written as `if (x + 1 < x) overflow();` can be removed in the same way, so the correct test is `if (x == INT_MAX)`, or arithmetic done in unsigned types.

> LAB
>
> `LAB 2.4 · 10 MIN`
>
> ```
> gcc -g -fsanitize=undefined -o ub ub.c && ./ub     # prints x + 1 with x = INT_MAX
> ```
>
> ```
> ub.c:3:75: runtime error: signed integer overflow: 2147483647 + 1 cannot be represented in type 'int'
> -2147483648
> ```
>
> **What you should notice:** the plain build prints only the wrapped value. The sanitiser reports the file, line and operands, so that a silent wrap becomes a located error. Run test suites with `-fsanitize=address,undefined` before concluding a program is correct.

An interview answer about UB has three parts: name the category, say why the standard leaves it open (portability to processors that trap, and permission to optimise), and give the tool that detects it. Unsigned overflow, by contrast, is defined and wraps, which makes it the right type for counters, hashes and sequence numbers.

> CHECK YOURSELF
>
> 1. Is `a[i] = i++;` defined? [5]
> 2. Is `-8 >> 1` defined? [6]

2.13 Writing the library: memcpy to itoa MUST · ■■□

Interviewers ask for the standard functions because each hides one subtle decision. The versions below are written for clarity; production libraries copy a word at a time but must give the same results.

```c
size_t my_strlen(const char *s) { const char *p = s; while (*p) p++; return
(size_t)(p - s); }

char *my_strcpy(char *d, const char *s) {
    char *r = d;
    while ((*d++ = *s++)) ;               /* copies the terminator, then the
test is 0 */
    return r;
}

int my_strcmp(const char *a, const char *b) {
    while (*a && *a == *b) { a++; b++; }
    return (unsigned char)*a - (unsigned char)*b;   /* compare as unsigned bytes
*/
}
```

`strcmp` compares as `unsigned char`, which is why the cast is present: with a plain signed `char`, a byte above 127 would compare below an ASCII letter. The pitfalls of `my_strcpy` are the missing size check and overlapping arguments.

`memcpy` copies `n` bytes and requires that the regions do not overlap. `memmove` allows overlap, so it must choose the copying direction: if the destination lies *before* the source, copy forwards; if it lies inside the source region *after* its start, copy backwards, otherwise a forward copy would overwrite source bytes that have not yet been read.

```c
void *my_memcpy(void *dst, const void *src, size_t n) {
    unsigned char *d = dst; const unsigned char *s = src;
    while (n--) *d++ = *s++;
    return dst;
}
void *my_memmove(void *dst, const void *src, size_t n) {
    unsigned char *d = dst; const unsigned char *s = src;
    if (d == s || n == 0) return dst;
    if ((uintptr_t)d < (uintptr_t)s || (uintptr_t)d >= (uintptr_t)s + n) {
        while (n--) *d++ = *s++;           /* forwards: safe when no harmful
overlap */
    } else {
        d += n; s += n;
        while (n--) *--d = *--s;           /* backwards: destination overlaps
the tail */
    }
    return dst;
}
```

With `char s[16] = "abcdef"`, calling `my_memmove(s+2, s, 4)` leaves `"ababcd"`, and `my_memmove(s, s+2, 4)` on a fresh copy leaves `"cdefef"`. Applying `my_memcpy` (or the real `memcpy`) to the first case is undefined, and a forward byte loop would give `"abababab"` because the first copied bytes are overwritten before they are read. Casting through `uintptr_t` before comparing pointers avoids comparing unrelated objects, which the standard does not define.

`atoi` hides four decisions: skip leading whitespace, accept one sign, stop at the first non-digit, and refuse to overflow. The version below clamps, as the common LeetCode variant requires; the overflow test is written so that it never overflows itself.

```c
int my_atoi(const char *s) {
    while (*s == ' ' || (*s >= '\t' && *s <= '\r')) s++;
    int neg = 0;
    if (*s == '-' || *s == '+') neg = (*s++ == '-');
    int v = 0;
    while (*s >= '0' && *s <= '9') {
        int d = *s++ - '0';
        if (v > (INT_MAX - d) / 10) return neg ? INT_MIN : INT_MAX;   /* v*10+d
would overflow */
        v = v * 10 + d;
    }
    return neg ? -v : v;
}
```

`my_atoi("  -42xyz")` returns -42, `"+7"` returns 7, `"2147483648"` returns 2147483647, `"-2147483649"` returns -2147483648 and `"abc"` returns 0. The test `v > (INT_MAX - d) / 10` is equivalent to `v * 10 + d > INT_MAX` but is computed without overflow.

`itoa` is not standard, and the task is to build it. Digits come out least-significant first, so they are collected and reversed. The sign needs care: `-n` overflows for `INT_MIN`, so the magnitude is computed in unsigned arithmetic, where negation is defined.

```c
char *my_itoa(int n, char *buf) {            /* buf must hold at least 12 bytes
*/
    char tmp[12]; int i = 0;
    unsigned u = n < 0 ? 0u - (unsigned)n : (unsigned)n;
    do { tmp[i++] = (char)('0' + u % 10); u /= 10; } while (u);   /* do-while: 0
gives "0" */
    char *o = buf;
    if (n < 0) *o++ = '-';
    while (i) *o++ = tmp[--i];
    *o = '\0';
    return buf;
}
```

For 0, -1234 and `INT_MIN` the results are `0`, `-1234` and `-2147483648`. The buffer size is 12 because `INT_MIN` has ten digits, a sign and the terminator. The conversion of an address to dotted-quad text and back is the same exercise with four numbers and is solved in the Problem Set (p. 144).

> CHECK YOURSELF
>
> 1. Why does `my_memmove(s+2, s, 4)` copy backwards? [7]
> 2. Why compute `0u - (unsigned)n` rather than `-n`? [8]

2.14 Parsing an IPv4 header from a byte buffer MUST · ■■□

A packet arrives as an array of bytes. The task is to extract the fields safely: check the length before reading, read multi-byte fields in network order, and verify the checksum. The meaning of each field belongs to Pillar 3 (p. 162); here we consider only offsets and C technique.

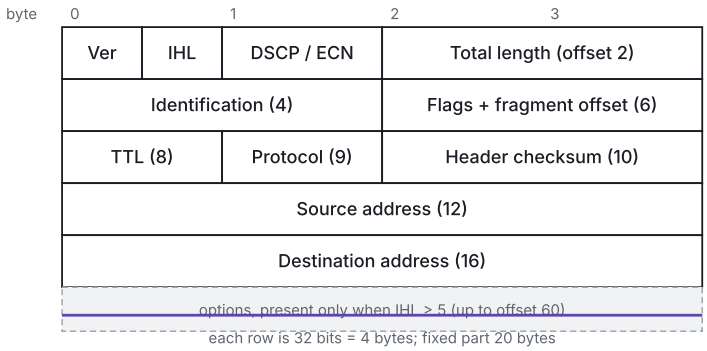


Figure 2.6 The IPv4 header as a C programmer reads it. Offsets in parentheses are in bytes; every multi-byte field is big-endian.

Two designs exist. One may declare a packed structure and cast the buffer to it, which is short but wrong for the reasons of 2.7 (p. 115): the buffer may be misaligned, bit-fields vary, and every field still needs `ntohs`. The robust design reads each field from its offset with small helpers that assemble bytes, and fills an ordinary structure. Version and header length share byte 0 and are split with a shift and a mask.

```c
struct ipv4 { uint8_t version, ihl; uint16_t total_len, id, frag;
              uint8_t ttl, proto; uint16_t csum; uint32_t src, dst; };

static inline uint16_t rd16(const uint8_t *p) { return (uint16_t)((p[0] << 8) |
p[1]); }
static inline uint32_t rd32(const uint8_t *p) {
    return ((uint32_t)p[0] << 24) | ((uint32_t)p[1] << 16) | ((uint32_t)p[2] <<
8) | p[3];
}

uint16_t inet_csum(const uint8_t *p, size_t n) {     /* ones-complement sum, RFC
1071 */
    uint32_t sum = 0;
    for (; n > 1; p += 2, n -= 2) sum += rd16(p);
    if (n) sum += (uint32_t)p[0] << 8;               /* odd trailing byte, pad-
ded with 0 */
    while (sum >> 16) sum = (sum & 0xFFFF) + (sum >> 16);   /* fold carries back
in */
    return (uint16_t)~sum;
}

int ipv4_parse(const uint8_t *buf, size_t len, struct ipv4 *h) {
    if (len < 20) return -1;                         /* too short to read the
fixed part */
    h->version = buf[0] >> 4;  h->ihl = buf[0] & 0x0F;
    if (h->version != 4 || h->ihl < 5) return -2;    /* wrong version or im-
possible IHL */
    size_t hlen = (size_t)h->ihl * 4;                /* IHL counts 32-bit words
*/
    if (len < hlen) return -1;
    h->total_len = rd16(buf + 2);
    if (h->total_len < hlen || h->total_len > len) return -3;
    h->id = rd16(buf + 4);  h->frag = rd16(buf + 6);
    h->ttl = buf[8];  h->proto = buf[9];  h->csum = rd16(buf + 10);
    h->src = rd32(buf + 12);  h->dst = rd32(buf + 16);
    return inet_csum(buf, hlen) == 0 ? 0 : -4;       /* a valid header sums to
zero */
}
```

The helper `u32_to_dotted` prints an address held in host order: `snprintf(out, 16, "%u.%u.%u.%u", ip >> 24, (ip >> 16) & 255, (ip >> 8) & 255, ip & 255)`. For the header bytes `45 00 00 73 00 00 40 00 40 11 b8 61 c0 a8 00 01 c0 a8 00 c7`, supplied in a buffer of at least 115 bytes, `ipv4_parse` returns 0 and the structure holds version 4, IHL 5 (20 bytes), total length 115, TTL 64, protocol 17 (UDP), source 192.168.0.1 and destination 192.168.0.199. With the checksum field zeroed, `inet_csum` returns `0xb861`, the stored value. Passing a length of 19 returns -1, and changing the TTL byte to `0x3f` without updating the checksum returns -4.

The order of the checks is the point of the exercise. Every read is guarded by a prior length test, and `total_len` is trusted only after it has been compared with the number of bytes actually received. A parser that reads `ihl` and then indexes options without checking is the template of a remote vulnerability. The more general loop for variable-length options, a type-length-value parser, appears in the Problem Set (p. 144).

> INTERVIEW LENS
>
> CISCO · JUNIPER · ARISTA · BROADCOM

> “Parse this header from a hex dump” is reported as a networking-software exercise, and “Can you send a C struct directly over a socket?” as a conceptual follow-up. A strong answer reads fields by offset, validates lengths first, converts byte order, and explains why the packed-structure cast is fragile. Follow-ups: how to extend the parser to TCP or UDP (the next header starts at offset `ihl * 4`), and how to verify the checksum incrementally.

CHECK YOURSELF

1. A packet has `buf[0] = 0x47`. What are the version and the header length in bytes? [9]
2. Why must `len` be tested before `total_len` is trusted? [10]

1. `calloc` detects overflow of the multiplication and returns `NULL`, whereas `n * 4` may wrap and yield a block that is too small.↩
2. They dangle, because the old block has been freed.↩
3. `"4"`; the argument of `STR` is not macro-expanded before `#` is applied, whereas the extra level in `XSTR` expands it first.↩
4. It makes the body a single statement, so the macro behaves correctly after `if` without braces and with a trailing semicolon.↩
5. No; `i` is read and modified without sequencing, so the behaviour is undefined.↩
6. It is implementation-defined, not undefined; gcc performs an arithmetic shift and yields -4.↩
7. The destination begins inside the source region, so a forward copy would overwrite source bytes before they are read.↩
8. `-INT_MIN` overflows a signed `int`, which is undefined; unsigned negation is defined and gives 2147483648.↩
9. Version 4 (high nibble) and IHL 7, so the header is 28 bytes.↩
10. The field comes from the sender and may claim more bytes than were received; using it unc-

2.15 The gotcha gallery: 36 output traps MUST · ■■□

hecked allows reads beyond the buffer.↩

Output-prediction questions are a fixed ritual in online assessments at Cisco and Juniper and in Qualcomm and Arista interviews: a snippet of three to eight lines, and the question “what does this print?”. The skill is to classify the snippet first as *defined*, *implementation-defined* or *undefined*, and to compute an output only in the first two cases. This section is the single home of the book’s C traps. Each entry gives the code, the output on 64-bit Linux with gcc (so `int` is 4 bytes and `char` is signed), and the reason. Entries marked UB print nothing definite; any output seen is an accident.

> TRAP
> *Claim:* when a snippet compiles and runs, whatever it prints is “the answer”. *Why it is wrong:* a program with undefined behaviour has no correct output, and the same source may print different results at `-O0` and `-O2`, or on the next compiler release. *Correct statement:* in an interview, say “undefined behaviour” and name the rule; computing a number from an unsequenced expression is the failure the question is designed to find.

Expressions and sequencing

T1. Two writes to one variable.

```
int i = 1;
i = i++ + ++i;
```

Answer: undefined. `i` is modified twice with no sequence point between the modifications.

T2. `sizeof` does not evaluate.

```
int i = 5;
printf("%zu %d\n", sizeof(i++), i);
```

Answer: `4 5`. The operand of `sizeof` is not evaluated (except for variable-length arrays), so `i` is unchanged.

T3. `printf` returns a count.

```
printf("%d\n", printf("hello"));
```

Answer: `hello5` followed by a newline. The inner call prints five characters and returns 5, which the outer call then prints.

T4. Signed meets unsigned.

```
puts(-1 < 1u ? "true" : "false");
```

Answer: `false`. The usual arithmetic conversions turn `-1` into `UINT_MAX`, which is larger than 1 (2.2 (p. 109)).

T5. Plain `char` above 127.

```
char c = 200;
printf("%d\n", c);
```

Answer: `-56` on x86-64, where `char` is signed (implementation-defined). On ARM Linux, where `char` is unsigned, it prints 200.

T6. Array parameter.

```
void f(int a[10]) { printf("%zu\n", sizeof(a)); }
int main(void) { int a[10]; f(a); printf("%zu\n", sizeof a); }
```

Answer: `8` then `40`. Inside `f` the parameter is a pointer (8 bytes on LP64); in `main` the array has 10 elements of 4 bytes.

T7. Macro without parentheses.

```
#define SQ(x) x*x
printf("%d %d\n", SQ(2+3), 10/SQ(2));
```

Answer: `11 10`. The first expands to `2+3*2+3`; the second to `10/2*2`, which evaluates left to right as `(10/2)*2`.

T8. Fall-through.

```
int x = 2;
switch (x) { case 1: putchar('a');
             case 2: putchar('b');
             case 3: putchar('c'); break;
             default: putchar('d'); }
```

Answer: `bc`. Execution enters at `case 2` and continues into `case 3` until `break`.

T9. Comma operator.

```
int x = (1, 2, 3);
printf("%d\n", x);
```

Answer: `3`. The comma operator evaluates its operands left to right and yields the last. Without the parentheses, `int x = 1, 2;` is a compile error, since `2` would be read as a second declarator.

T10. Short-circuit evaluation.

```
int a = 0, b = 5;
if (a && b++) { }
printf("%d %d\n", a, b);
```

Answer: `0 5`. Because `a` is false, the right operand of `&&` is never evaluated. The same applies to `||` when the left side is true.

T11. A static local keeps its value.

```
int cnt(void) { static int c; return ++c; }
for (int i = 0; i < 3; i++) printf("%d ", cnt());
```

Answer: `1 2 3`. The static `c` is zero-initialised once, before `main`, and survives between calls. Printing three calls in a single `printf` would be unspecified, because the order of evaluating arguments is unspecified.

Pointers and declarations

T12. Increment and dereference.

```
int a[] = {10, 20, 30}, *p = a;
int x = *p++;        /* post-increment the pointer, read the old target */
int y = (*p)++;      /* increment the value that p points to           */
int z = ++*p;        /* pre-increment the value                        */
printf("%d %d %d %d %ld\n", x, y, z, a[1], (long)(p - a));
```

Answer: `10 20 22 22 1`. Postfix `++` binds tighter than `*`, so `*p++` is `*(p++)`: the pointer moves, the old element is read. In `(*p)++` and `++*p` the value changes and the pointer does not.

T13. Array address versus element address.

```
int a[5];
printf("%ld %ld\n", (long)((char *)(a + 1) - (char *)a),
                    (long)((char *)(&a + 1) - (char *)a));
```

Answer: `4 20`. `a + 1` steps over one `int`; `&a` has type `int (*)[5]`, so `&a + 1` steps over the whole array.

T14. Where `const` binds.

```
int x = 1, y = 2;
const int *p = &x;   *p = 3;     /* line A */
p = &y;                          /* line B */
int *const q = &x;   q = &y;     /* line C */
```

Answer: line A is a compile error (`assignment of read-only location '*p'`), line B is accepted, and the assignment in line C is a compile error (`assignment of read-only variable 'q'`). `const int *` protects the pointee; `int *const` protects the pointer (2.5 (p. 112)).

T15. Writing to a literal.

```
char *p = "arista";  p[0] = 'A';
char  q[] = "arista"; q[0] = 'A';
```

Answer: the first write is undefined and normally ends in a segmentation fault, because the literal lives in .rodata. The second is defined: `q` is a writable copy.

T16. Pointer to a dead local.

```
char *name(void) { char buf[8] = "eth0"; return buf; }
printf("%s\n", name());
```

Answer: undefined. `buf` ceases to exist when `name` returns. The compiler warns (`returns address of local variable`), and the output may look right, garbage, or crash.

T17. Postfix decrement in recursion.

```
int sum(int n) { return n == 0 ? 0 : n + sum(n--); }
printf("%d\n", sum(3));
```

Answer: the program overflows the stack and ends with a segmentation fault. `n--` passes the *old* value, so the argument never decreases and the recursion never reaches 0. The fix is `sum(n - 1)`.

T18. Padding.

```
struct A { char c; int i; char d; };
struct B { int i; char c; char d; };
printf("%zu %zu\n", sizeof(struct A), sizeof(struct B));
```

Answer: `12 8`. See Figure 2.4 on padding (p. 115): three padding bytes after `c` and three after `d` in `A`, two at the end of `B`.

T19. Union overlay.

```
union { int i; char c[4]; } u = { 1 };
printf("%zu %d\n", sizeof u, u.c[0]);
```

Answer: `4 1` on a little-endian machine, since the least significant byte of 1 is stored first. On a big-endian machine the second number is 0.

T20. A flag polled in a loop. Read the assembly in Lab 2.2 of 2.6 (p. 114): without `volatile`, `while (!flag);` is compiled to read `flag` once. This is a question about the optimiser, not about the printed value.

Numbers

T21. Overflow, signed and unsigned.

```
printf("%u\n", UINT_MAX + 1u);
int m = INT_MAX;  m = m + 1;
```

Answer: the first line prints `0`, because unsigned arithmetic wraps modulo 2^32 (defined). The second is undefined; the UBSan message appears in Lab 2.4 of 2.12 (p. 121).

T22. Floating-point equality.

```
printf("%d\n", 0.1 + 0.2 == 0.3);
float f = 0.1;  printf("%d\n", f == 0.1);
```

Answer: `0` and `0`. The decimal fractions are not exactly representable in binary, and the sum differs from `0.3` in the last bit. In the second case `f` holds the nearest `float`, which differs from the nearest `double` that `0.1` denotes. Compare floating-point values with a tolerance.

T23. Type of a character constant.

```
printf("%d %zu\n", 'A', sizeof('A'));
```

Answer: `65 4`. A character constant has type `int` in C; in C++ it has type `char` and the size is 1.

T24. Shifts.

```
printf("%d\n", -8 >> 1);
int a = 1 << 31;   unsigned b = 1u << 32;
```

Answer: the first prints `-4`: right-shifting a negative value is implementation-defined, and gcc shifts arithmetically. `1 << 31` shifts into the sign bit of an `int` and is undefined; `1u << 32` shifts by the full width and is undefined.

T25. `strlen` versus `sizeof`.

```
char s[] = "abc";
printf("%zu %zu\n", strlen(s), sizeof s);
```

Answer: `3 4`. `strlen` stops before the terminator; the array includes it.

T26. Uninitialised local.

```
int n;
printf("%d\n", n);
```

Answer: undefined for an automatic variable (an indeterminate value). Had `n` been `static` or global, it would be 0. Compilers warn with `-Wall -O2`.

T27. Double free.

```
char *p = malloc(8);  free(p);  free(p);
```

Answer: undefined. With glibc 2.39 the program aborts with `free(): double free detected in tcache 2`. Use after free and a missing `free` are the two related defects (2.10 (p. 119)).

T28. A countdown that never ends.

```
for (unsigned i = 3; i >= 0; i--) printf("%u ", i);
```

Answer: the loop prints `3 2 1 0 4294967295 4294967294 ...` without end, because an unsigned value is always `>= 0` and `0 - 1` wraps. Use `for (unsigned i = n; i-- > 0; )`.

T29. Leading zero.

```
printf("%d %d\n", 010, 010 + 1);
```

Answer: `8 9`. A leading `0` makes an octal constant, and `08` would not compile.

T30. Types in the conditional operator.

```
puts((1 ? -1 : 0u) > 0 ? "pos" : "neg");
```

Answer: `pos`. The two branches are converted to a common type, which is `unsigned int`, so `-1` becomes `UINT_MAX` even though the first branch is chosen.

T31. A mismatched format specifier.

```
printf("%d\n", 3.5);          printf("%s\n", 5);
```

Answer: undefined. `printf` reads its arguments according to the format, not according to what was passed. The compiler warns with `-Wformat`, which is part of `-Wall`.

T32. Division and remainder of negative numbers.

```
printf("%d %d\n", -7 / 2, -7 % 2);
```

Answer: `-3 -1`. Since C99 division truncates towards zero, and the remainder takes the sign of the dividend, so that `(a / b) * b + a % b == a`.

T33. Assignment in a condition.

```
int x = 0;
if (x = 5) puts("yes");
printf("%d\n", x);
```

Answer: `yes` and then `5`. The expression `x = 5` assigns and has the value 5, which is true. The compiler suggests parentheses with `-Wall`.

T34. Precedence of `==` over `&`.

```
int x = 4;
printf("%d\n", x & 1 == 0);
```

Answer: `0`. The comparison binds tighter than `&`, so the expression is `x & (1 == 0)`, which is `4 & 0`. The intended test is `(x & 1) == 0`; the precedence table is in the Revision Spread (p. 159).

T35. Partial initialisation, and `i[a]`.

```
int a[5] = {1};
for (int i = 0; i < 5; i++) printf("%d ", a[i]);
int b[] = {10, 20, 30};   printf("%d\n", 2[b]);
```

Answer: `1 0 0 0 0` followed by `30`. Members without an initialiser are zeroed, and `2[b]` is `*(2 + b)`, the same as `b[2]`.

T36. Where does each variable live? The classification of globals, statics, locals, literals and `malloc` blocks is the table in 2.1 (p. 107), with the effect of `static` on a large local array in the trap that follows it.

> INTERVIEW LENS
>
> CISCO · JUNIPER · QUALCOMM · ARISTA
>
> A set of ten to fifteen of these, usually T1, T2, T7, T10 to T19, T25 and T28, makes up the C section of many online assessments. Qualcomm adds `const` placement, structure against union, padding, `volatile` and `static` (T11, T14 to T20); Arista adds the debugging variants, namely uninitialised variables (T26), returning a local (T16), `n--` in recursion (T17) and writing to a literal (T15). A strong answer names the category first and then the mechanism. If the output is undefined, one gives the rule that is broken and the tool that detects it (UBSan, ASan or a compiler warning).

> LAB
>
> `LAB 2.5 · 20 MIN`
>
> Type the six snippets T2, T4, T12, T13, T18 and T25 into one file, predict each result on paper, then run:
>
> ```
> gcc -Wall -Wextra -O0 -o gal gal.c && ./gal
> ```

Repeat with `-O2`. The defined traps print identical output at both levels; add T1 and T26 and observe the compiler warnings and, possibly, different output.

What you should notice: warnings from `-Wall -Wextra` already flag several entries (T1, T16, T26, T31, T33), and T14 is rejected outright, so reading them is the cheapest debugging step.

CHECK YOURSELF

1. Which entries of this section rely on implementation-defined behaviour rather than undefined behaviour? [1]
2. Why is T11 safe with a loop but not with three calls in one `printf`? [2]

2.16 Python: the data model SHOULD · ■□□

This part treats the Python language only. The libraries for networks and devices are in Pillar 10 (p. 586), sockets in Pillar 7 (p. 414), and shell pipelines in Pillar 8 (p. 483). Examples assume Python 3.10 or later.

In Python every value is an object, and a variable is a *name* bound to an object, not a box that holds a value. Assignment `b = a` therefore binds a second name to the same object. Objects are either *mutable* (list, dict, set, bytearray) or *immutable* (int, float, str, bytes, tuple), and the difference explains most interview traps.

BIG IDEA

Names refer to objects; assignment copies the reference, never the object, so a change made through one name is visible through every name bound to the same mutable object.

```
a = [1, 2]; b = a; c = a[:]            # b is an alias; c is a shallow copy
b.append(3)
print(a, c, a is b, a == c, a is c)    # [1, 2, 3] [1, 2] True False False
grid = [[0] * 2] * 2; grid[0][0] = 9
print(grid)                            # [[9, 0], [9, 0]]: both rows are one
list
t = (1, [2]); t[1].append(3); print(t) # (1, [2, 3]): the tuple holds the same
list
```

The operator `==` compares values and `is` compares identity. Use `is` only for `None` (`if x is None`), because the identity of small integers and strings is an implementation detail. A *shallow copy* (`a[:]`, `list(a)`, `a.copy()`) copies the outer container but shares the elements; `copy.deepcopy` copies recursively.

The two most reported Python traps are both consequences of the model. A default argument is evaluated once, when the function is defined, so a mutable default is shared by all calls. Removing items from a list while iterating over it skips elements, because the iterator tracks a position, not the items.

1. T5 (signedness of `char`) and T24 (right shift of a negative value); the others print a definite value or are undefined. ↩
2. The loop sequences the calls statement by statement, whereas the order of evaluating the arguments of one call is unspecified. ↩

```
def bad(x, acc=[]):                    # one list, created at definition time
    acc.append(x); return acc
print(bad(1), bad(2))                  # [1, 2] [1, 2]  (the same list twice)
def good(x, acc=None):
    if acc is None: acc = []
    acc.append(x); return acc
print(good(1), good(2))                # [1] [2]

nums = [1, 2, 2, 3]
for n in nums:
    if n == 2: nums.remove(n)
print(nums)                            # [1, 2, 3]: the second 2 was skipped
```

The fix for the second case is to build a new list, `nums = [n for n in nums if n != 2]` (2.17 (p. 135)). The first output line shows `[1, 2] [1, 2]` because both calls return the same list object, and `print` shows its final state.

Built-in containers have the complexity one expects from their implementations, which is a frequent question: `x in list` scans linearly and costs O(n), whereas `x in set` and `x in dict` hash the key and cost O(1) on average; `list.append` is amortised O(1), `list.insert(0, x)` is O(n), and `collections.deque` offers O(1) at both ends. Dictionary keys must be *hashable*, which means immutable: a tuple is a valid key, a list is not.

Special methods, written with double underscores, let a class join the language's protocols: `__repr__` for printing, `__eq__` and `__hash__` for equality and hashing, `__lt__` for sorting, `__len__`, `__iter__` and `__contains__` for containers. A class that defines `__eq__` must define `__hash__` consistently, or it becomes unhashable.

```
class Prefix:
    __slots__ = ("addr", "length")
    def __init__(self, addr, length): self.addr, self.length = addr, length
    def __repr__(self): return f"Prefix({self.addr!r}, {self.length})"
    def __eq__(self, o): return (self.addr, self.length) == (o.addr, o.length)
    def __hash__(self): return hash((self.addr, self.length))
    def __lt__(self, o): return (-self.length, self.addr) < (-o.length, o.addr)

ps = [Prefix("10.0.0.0", 8), Prefix("10.1.0.0", 16), Prefix("10.0.0.0", 8)]
print(sorted(ps), len(set(ps)))
# [Prefix('10.1.0.0', 16), Prefix('10.0.0.0', 8), Prefix('10.0.0.0', 8)] 2
```

Python's integers have arbitrary precision, so there is no overflow, and `//` floors (`-7 // 2` is `-4`, `-7 % 2` is `1`), unlike C, where division truncates towards zero (2.15 (p. 128), T32). The conversion `int("ff", 16)` gives 255, `int.from_bytes(b"\x00\x50", "big")` gives 80 and `(80).to_bytes(2, "big")` gives `b'\x00P'`.

> CHECK YOURSELF
>
> 1. What does `bool("0")` return, and why? [1]
> 2. Why is `{[1, 2]: "x"}` an error? [2]

2.17 Python: comprehensions and generators SHOULD · ■■□

A *comprehension* builds a list, set or dictionary from an iterable in one expression: `[x * x for x in range(5) if x % 2 == 0]` is `[0, 4, 16]`, `{x % 3 for x in range(10)}` is `{0, 1, 2}`, and `{w: len(w) for w in words}` maps each word to its length. A comprehension is

shorter than the equivalent loop and avoids the repeated attribute lookup of `append`, but one that needs more than one condition is clearer as a loop.

A *generator* produces its values one at a time instead of storing them. The generator expression `(x * x for x in range(5))` looks like a list comprehension in round brackets, and a function containing `yield` is a generator function: each call to `next` runs the body until the next `yield`, then pauses with its local state intact. A generator is *exhausted* after one pass.

```
sq = (x * x for x in range(5))
print(next(sq), next(sq), sum(sq), sum(sq))   # 0 1 29 0: the second sum finds
nothing left

def fib():
    a, b = 0, 1
    while True:                                # infinite, yet uses constant
memory
        yield a
        a, b = b, a + b
print(list(islice(fib(), 8)))                 # [0, 1, 1, 2, 3, 5, 8, 13]
(from itertools import islice)
```

The benefit is memory. Reading a 50 GB log into a list needs 50 GB, whereas a generator pipeline holds one line at a time. A `for` loop over a file object already reads lazily, so a pipeline chains generators, each consuming the previous one.

```
from collections import Counter

def read_lines(path):
    with open(path) as f:                      # closed even if the consumer
stops early
        for line in f:
            yield line.rstrip("\n")

def src_ips(lines):
    for line in lines:
        yield line.split(None, 1)[0]           # first whitespace-separated
field

counts = Counter(src_ips(read_lines("access.log")))
print(counts.most_common(3))                   # the three most frequent ad-
dresses with counts
```

For lines `10.0.0.1 GET /a`, `10.0.0.2 GET /b`, `10.0.0.1 POST /c`, `10.0.0.3 GET /a`, `10.0.0.1 GET /d`, `10.0.0.2 GET /e` the call prints `[('10.0.0.1', 3), ('10.0.0.2', 2), ('10.0.0.3', 1)]`. The memory used is proportional to the number of *distinct* addresses, not to the file size, which is the answer to the "what if the file is 50 GB?" follow-up. If even the distinct set is too large, one sorts externally or uses a counting sketch, as discussed in Pillar 1 (p. 7). For the top `k` only, `heapq.nlargest(k, counts.items(), key=lambda kv: kv[1])` costs O(n log k).

> INTERVIEW LENS
>
> GOOGLE · AMAZON · MICROSOFT
>
> "Extract the IP addresses from a log and sort by frequency; what if the file is 50 GB?" is a reported Google question. A strong answer streams with a generator, counts with a `Counter`, and states the memory bound and the point at which one would switch to external sorting.

> Follow-ups: the difference between `most_common(k)` and a heap, and why a list comprehension over the whole file would fail.

CHECK YOURSELF

1. How does `[f(x) for x in xs]` differ from `(f(x) for x in xs)` in memory? [3]
2. What does a generator return when called a second time after exhaustion? [4]

2.18 Python: struct, re, subprocess, files, exceptions MUST · ■■□

Binary data with `struct`. The module `struct` converts between Python values and packed bytes, using a format string. The first character selects byte order and alignment: `!` means network (big-endian) order with no padding, `<` little-endian, and no prefix means *native* order with C-style padding. The letters are `B` (1 byte), `H` (2), `I` (4), `Q` (8) and `Ns` for `N` raw bytes. Unpacking the IPv4 header of 2.14 (p. 124) takes one line.

```
hdr = bytes.fromhex("450000730000400040011b861c0a80001c0a800c7")
fmt = "!BBHHHBBH4s4s"                          # 1+1+2+2+2+1+1+2+4+4 = 20 bytes
v_ihl, tos, total, ident, frag, ttl, proto, csum, src, dst = struct.unpack(fmt,
hdr)
print(v_ihl >> 4, v_ihl & 15, total, ttl, proto, hex(csum))   # 4 5 115 64 17
0xb861
print(".".join(map(str, src)), struct.calcsize(fmt))           # 192.168.0.1 20
print(struct.pack("!HI", 80, 1), struct.calcsize("BI"), struct.calcsize("!BI"))
# b'\x00P\x00\x00\x00\x01' 8 5
```

The last line shows the padding rule of 2.7 (p. 115) in Python: `"BI"` uses native alignment and occupies 8 bytes, while `"!BI"` is packed into 5. For a wire format the prefix `!` is always correct.

Regular expressions with `re`. A regular expression describes a text pattern. The functions are `re.search` (first match anywhere), `re.match` (match at the start), `re.findall` and `re.sub`. Named groups `(?P<name>...)` make a log parser self-documenting, and `re.compile` avoids recompiling the pattern in a loop.

```
log = "Oct 29 10:01:02 sw1 %LINK-3-UPDOWN: Interface Gi0/1, changed state to
down"
pat = re.compile(r"(?P<host>\S+) %(?P<fac>[A-Z]+)-(?P<sev>\d)-(?P<mn>[A-Z_]+): "
                 r"Interface (?P<ifc>\S+), changed state to (?P<st>up|down)")
m = pat.search(log)
print(m["host"], m["sev"], m["ifc"], m["st"])      # sw1 3 Gi0/1 down
```

Use raw strings (`r"..."`) so that backslashes reach the regular-expression engine. Quantifiers are greedy by default, so `.*` consumes as much as it can; `.*?` is the minimal form. When the input has fixed whitespace-separated columns, `line.split()` is faster and clearer than a regular expression, as in the table-parsing exercise of the Problem Set (p. 144).

Running programs with `subprocess`. `subprocess.run` starts a command, waits, and returns a result. Pass the command as a *list* so that no shell is involved and no argument can be interpreted as shell syntax; this is the main defence against command injection. `check=True` raises `CalledProcessError` on a non-zero exit status, `capture_output=True` collects the output, `text=True` decodes it, and `timeout` raises `TimeoutExpired` for a hung command.

```
r = subprocess.run(["echo", "hi"], capture_output=True, text=True, check=True,
timeout=5)
print(repr(r.stdout), r.returncode)                # 'hi\n' 0
try:
    subprocess.run(["false"], check=True)
except subprocess.CalledProcessError as e:
    print("rc", e.returncode)                       # rc 1
```

Files. Open files with a `with` block, which closes the file even when an exception occurs. `for line in f` reads lazily; `f.read()` loads everything. Open in text mode (`"r"`) for logs and in binary mode (`"rb"`) for packets and captures. The encoding defaults to the platform's, so give `encoding="utf-8"` for portable code.

Exceptions. `try` runs the code, `except E as e` handles a named error type, `else` runs only when no exception occurred, and `finally` always runs. Catch the narrowest type that you can handle; a bare `except:` also swallows `KeyboardInterrupt`. To translate an error and keep the original, use `raise New(...) from e`.

```
class ParseError(Exception): pass
def parse(s):
    try:
        return int(s)
    except ValueError as e:
        raise ParseError(f"bad number {s!r}") from e
try: parse("x")
except ParseError as e: print(e, type(e.__cause__).__name__)   # bad number 'x'
ValueError
```

> TRAP
>
> *Claim:* `subprocess.run(f"ping -c1 {host}", shell=True)` is fine because the hostnames come from our own inventory. *Why it is wrong:* the string is parsed by the shell, so a value such as `8.8.8.8; rm -rf ~` runs a second command, and inventories do get corrupted. *Correct statement:* pass an argument list, `["ping", "-c1", host]`, without `shell=True`.

> CHECK YOURSELF
>
> 1. What is the byte string produced by `struct.pack("!H", 80)`? [5]
> 2. When does the `else` clause of a `try` statement run? [6]

2.19 C++: references, classes and RAII SHOULD · ■■□

C++ is C with abstractions that cost nothing when unused. Interviews ask for a small, repeatable core: references, constructors and destructors, resource ownership, and move semantics. Examples assume `g++ -std=c++17`.

A *reference* is a second name for an existing object: `int &r = x; r = 9;` changes `x`. Unlike a pointer, a reference must be bound when created, cannot be rebound, and cannot be null. Its main use is a parameter: `void f(const Packet &p)` passes without copying and without allowing modification, and `void g(Packet &p)` lets the callee modify the caller's object.

A *class* bundles data and functions and controls access with `private` and `public`. A *constructor* initialises an object and a *destructor* `~T()` runs when the object's lifetime ends, whether the scope exits normally or by an exception. This guaranteed call is the basis of **RAII** (resource

acquisition is initialisation): acquire a resource in a constructor and release it in the destructor, so that the resource cannot leak on any path.

> BIG IDEA
>
> Tie every resource, whether memory, file, lock or socket, to the lifetime of an object, because the destructor runs on every exit path, including exceptions.

```
struct Guard {
    const char *name;
    explicit Guard(const char *n) : name(n) { std::printf("acquire %s\n",
name); }
    ~Guard() { std::printf("release %s\n", name); }
    Guard(const Guard &) = delete;            // a guard must not be copied
    Guard &operator=(const Guard &) = delete;
};
void work(bool fail) {
    Guard a("A");  Guard b("B");
    if (fail) throw std::runtime_error("boom");
    std::puts("work done");
}
```

Calling `work(false)` prints `acquire A`, `acquire B`, `work done`, `release B`, `release A`. Calling `work(true)` inside a `try` prints `acquire A`, `acquire B`, `release B`, `release A` and then, in the handler, `caught boom`. Destructors run in reverse order of construction, and they run during the unwinding caused by the exception. The standard library applies the same pattern: `std::lock_guard` holds a mutex, `std::ifstream` closes its file, `std::vector` frees its memory.

The compiler generates five special members when asked or needed: the copy constructor, copy assignment, move constructor, move assignment and destructor. The *rule of zero* says to write none of them and let members such as `std::vector` and `std::unique_ptr` manage themselves. The *rule of three/five* says that if a class defines a destructor, copy operation or move operation to manage a raw resource, it should define or delete all five. A *move* transfers a resource from a temporary or an `std::move`d object instead of copying it, leaving the source empty but valid.

```
class File {
    std::FILE *f_;
public:
    explicit File(const char *p) : f_(std::fopen(p, "r")) { if (!f_) throw
std::runtime_error("open"); }
    ~File() { if (f_) std::fclose(f_); }
    File(File &&o) noexcept : f_(o.f_) { o.f_ = nullptr; }   // ownership moves
    File(const File &) = delete;                             // two owners would
close twice
};
```

> TRAP
>
> *Claim:* in C++ it is enough to call `delete` at the end of the function. *Why it is wrong:* an exception or an early `return` before that line skips it, and the memory leaks. *Correct statement:* never write a bare `new` in application code; use `std::make_unique<T>()` or a container, so that the destructor frees the memory on every path.

> CHECK YOURSELF
>
> 1. In which order are local objects destroyed? [7]

> 2. Why delete the copy constructor of a class that owns a `FILE *`? [8]

2.20 C++: virtual functions and the vtable SHOULD · ■■□

Inheritance lets a derived class reuse a base class, and *polymorphism* lets code written against the base call the derived behaviour. By default C++ binds a member function at compile time, according to the static type of the pointer. Declaring the function `virtual` makes the call dispatch at run time, according to the dynamic type of the object.

```
struct Shape  { virtual double area() const { return 0; }
                virtual ~Shape() { std::puts("~Shape"); } };
struct Circle : Shape { double r; explicit Circle(double r) : r(r) {}
                double area() const override { return 3 * r * r; }
                ~Circle() override { std::puts("~Circle"); } };

Shape *s = new Circle(2);
std::printf("%g\n", s->area());      // 12: Circle::area, chosen at run time
delete s;                            // ~Circle, then ~Shape
```

The standard implementation is the *vtable*. For each class with virtual functions the compiler builds one table of function addresses, and each object carries a hidden pointer to its class's table, the *vptr*, stored in the object (typically at offset 0). A virtual call loads the vptr, indexes the table and calls through the result, which costs one extra load and an indirect call. Figure 2.7 shows two objects sharing one table per class.

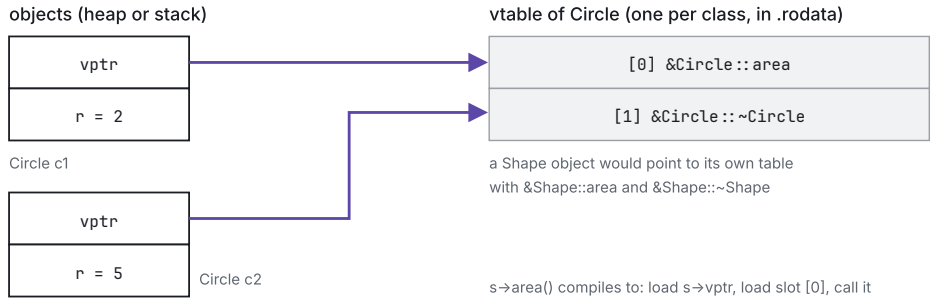


Figure 2.7 Two `Circle` objects point to the same vtable; the call chooses its target through the pointer, not the type of `s`.

A consequence is visible in the size of objects. `struct Plain { int x; void f() {} };` is 4 bytes, because non-virtual functions are not stored in the object, while `struct Virt { int x; virtual void f() {} };` is 16 bytes on LP64: an 8-byte vptr, the 4-byte `int` and 4 bytes of padding. An empty-looking `Shape` with only virtual functions is 8 bytes, just the vptr.

> **TRAP**
>
> *Claim:* a base class needs no virtual destructor if the derived class adds no data. *Why it is wrong:* `delete` on a `Base *` that points to a `Derived` calls only `~Base` when the destructor is not virtual, which is undefined behaviour and skips the derived cleanup. *Correct statement:* any class meant to be used polymorphically gets a `virtual` destructor. The experiment with `NShape` and `NCircle` (no `virtual`) prints only `~NShape`, and `s->area()` returns the base result 0, not 3.

Two related hazards. *Slicing*: assigning a `Circle` to a `Shape` object, as in `Shape sl = c;`, copies only the base part; `sl.area()` then returns 0, the base version, because `sl` is a real `Shape`, so polymorphism needs a pointer or reference. *Calling virtual functions in constructors*: while a base constructor runs the object is still a base, so the base version is called. A `= 0` after the declaration makes a function *pure virtual* and the class *abstract*, which cannot be instantiated and serves as an interface.

> INTERVIEW LENS
>
> NOKIA · CISCO · MICROSOFT
>
> "What is a virtual function and how is it implemented? Why a virtual destructor?" is a reported C++ question. A strong answer draws Figure 2.7, states the cost (one pointer per object, one indirect call), and explains the destructor rule with the `delete base_ptr` case. Follow-ups: the size of an object with virtual functions, what a pure virtual function is, and why virtual calls resist inlining.

> CHECK YOURSELF
>
> 1. How many vtables exist for ten `Circle` objects? [9]
> 2. What does `sizeof(Shape)` report on LP64 for a class with only virtual functions? [10]

2.21 C++: STL containers and smart pointers SHOULD · ■■□

The Standard Template Library supplies containers, iterators and algorithms. The interview skill is to choose the container by the operations needed, and to state the cost. The table gives the usual figures; "amortised" means averaged over a sequence of operations, and "average" for hashed containers assumes a good hash function.

Container	Access	Insert / erase	Find	Notes
`vector`	O(1) by index	back: amortised O(1); middle: O(n)	O(n)	contiguous; best default
`deque`	O(1) by index	both ends O(1); middle O(n)	O(n)	blocks of memory; no contiguity
`list`	O(n)	O(1) given an iterator	O(n)	doubly linked; stable iterators
`set`, `map`	none by position	O(log n)	O(log n)	red-black tree; sorted order
`unordered_set`, `unordered_map`	none	average O(1), worst O(n)	average O(1), worst O(n)	hash table; no order
`priority_queue`	top O(1)	push, pop O(log n)	none	binary heap; max-heap by default

`std::map` keeps keys sorted, so iteration yields `a=1 b=2` for keys inserted as `b`, `a`, and range queries (`lower_bound`) are possible; `unordered_map` is faster for plain lookup. `vector::push_back` doubles the capacity when full: in libstdc++ 1000 insertions cause 11 reallocations, which is why the cost is amortised O(1). `map::operator[]` inserts a default element if the key is absent, so use `count` or `find` to test membership without modifying the map.

> TRAP
>
> *Claim:* a pointer or iterator into a `vector` stays valid while the vector grows. *Why it is wrong:* when the capacity is exceeded, the vector allocates a new block, copies the elements

> and frees the old block, so every saved pointer, reference and iterator dangles. *Correct statement:* after any operation that may reallocate (`push_back`, `insert`, `reserve`), re-obtain iterators, or store indices instead. In the test `v{1,2,3}; p = &v[0]; v.reserve(100);` the comparison `p == &v[0]` is false.

Smart pointers apply RAII to heap objects. A `std::unique_ptr<T>` has exactly one owner, costs nothing beyond the raw pointer, cannot be copied and can be moved; create it with `std::make_unique<T>(args)`. A `std::shared_ptr<T>` shares ownership through a reference count and frees the object when the last owner is destroyed. A `std::weak_ptr<T>` observes a `shared_ptr` object without owning it.

```
auto up  = std::make_unique<int>(5);
auto up2 = std::move(up);            // up is now null; ownership moved
auto sp  = std::make_shared<int>(7);
auto sp2 = sp;                       // sp.use_count() == 2
```

`shared_ptr` has a failure mode that garbage-collected languages do not: a reference cycle. If two nodes hold `shared_ptr` to each other, neither count reaches zero and both leak. In the test, the destructors of two cyclic `Node` objects never run, whereas replacing the back-pointer by a `weak_ptr` (as in a doubly linked list's `prev`) prints `~NodeW` twice. Hence the guideline: `unique_ptr` by default, `shared_ptr` only for genuinely shared ownership, and `weak_ptr` for back-references and caches.

> CHECK YOURSELF
>
> 1. Which container gives sorted iteration, and what does a lookup cost? [11]
> 2. Why do two `shared_ptr` nodes pointing at each other leak? [12]

1. `True`; any non-empty string is true, whatever its characters.↩
2. A list is mutable and therefore unhashable, so it cannot be a dictionary key.↩
3. The list comprehension builds all results at once; the generator expression computes one at a time on demand.↩
4. Nothing: the iteration is already finished, so `next` raises `StopIteration` and a `for` loop ends immediately.↩
5. `b'\x00P'`, the two bytes `00 50`, since `P` is the character 0x50.↩
6. Only when the `try` body finished without raising an exception.↩
7. In reverse order of construction, when the scope ends.↩

8. A copy would give two owners of one handle, and both destructors would close it, which is a double free.↩
9. One for the class `Circle`; the ten objects each hold a pointer to it.↩
10. 8, the size of the vptr, since the class has no data members.↩
11. `std::map` or `std::set`, with O(log n) per lookup.↩
12. Each holds a reference to the other, so neither count falls to zero and neither destructor runs.↩

Problem Set

Every interview question of this pillar is solved once, here, in the compact template: a 30-second answer first, then the approach, code, follow-ups and common mistakes where they apply. Tags give the companies and round where the question was reported, the difficulty (■□□ easy, ■■□ medium, ■■■ hard), and **S** when a specific question was reported by candidates or **T** when only the topic was reported and the wording is ours. Questions whose subject belongs to another pillar are listed at the end of the second part with a pointer.

Q2.1 What is the difference between a structure and a union? Give a use for a union.

QUALCOMM · CISCO · TECH-1 · ■□□ · S

30-second answer: a structure stores every member one after another; a union stores all members at the same address, so its size is that of its largest member and only one is valid at a time.

Answer. `struct { int i; char c; }` is 8 bytes on LP64; `union { int i; char c; }` is 4. Use a union for one storage that holds alternative message formats, selected by a tag (2.7 (p. 115)), or to view the same bytes as different types (endianness test). **Mistakes.** Reading a member other than the last written is reinterpretation of bytes, not conversion.

Q2.2 What is structure padding and why does the compiler add it?

QUALCOMM · CISCO · TECH-1 · ■□□ · S

30-second answer: the compiler inserts unused bytes so that each member starts at an address divisible by its alignment, because misaligned access is slower or faults on some processors.

Answer. `struct { char c; int i; char d; }` has offsets 0, 4, 8 and size 12, with 6 bytes of padding (Figure 2.4). The size is rounded to a multiple of the largest alignment so arrays of the structure stay aligned. **Follow-ups.** Reorder by decreasing size, or pack with `__attribute__((packed))` at the cost of misaligned access.

Q2.3 What is `sizeof(struct { char a; int b; short c; })`? Reorder it to minimise the size.

QUALCOMM · TECH-1 · ■■□ · S

30-second answer: 12 bytes; ordering as `int b; short c; char a;` gives 8.

Approach. `a` at 0; `b` needs a multiple of 4, so 4; `c` needs a multiple of 2, so 8 to 10; round to a multiple of 4: 12. Reordered: `b` at 0, `c` at 4, `a` at 6, end 7, round to 8. **Mistake.** Adding the member sizes (7) and forgetting alignment.

Q2.4 What is the size of a structure with a `short`, a `long` and a `char` on 32-bit and 64-bit machines?

QUALCOMM · TECH-1 · ■■□ · S

30-second answer: 24 bytes on 64-bit Linux (LP64) and 12 on 32-bit x86, because `long` is 8 bytes there and 4 here.

Approach. For `{ short s; long l; char c; }`: LP64 offsets 0, 8, 16, end 17, rounded to 24. ILP32 offsets 0, 4, 8, end 9, rounded to 12. Order `long, short, char` gives 16 and 8. **Mistake.** Assuming `long` is 4 bytes everywhere; on 64-bit Windows (LLP64) it is 4, so the answer is machine-specific.

Q2.5 Find the size of a structure without using `sizeof`.

QUALCOMM · TECH-1 · ■■□ · S

30-second answer: take two consecutive objects and subtract their addresses as bytes.

Code. For a variable: `(char *)(&x + 1) - (char *)&x`. For an array element: `(char *)&a[1] - (char *)&a[0]`. **Why.** `&x + 1` advances by one whole object, and the `char *` cast makes the difference count bytes. **Follow-up.** The macro form `((size_t)((T *)0 + 1))` relies on a null pointer and is formally undefined.

Q2.6 Worksheet: sizes of six structures on LP64.

QUALCOMM · OA · ■■□ · S

30-second answer: 24, 16, 12, 4, 8, 16.

Structure	Offsets	Size
`{char a; double d; char b;}`	0, 8, 16	24
`{double d; char a, b;}`	0, 8, 9	16
`{char a[3]; short s; int i;}`	0, 4, 8	12
`{short s; char c;}`	0, 2	4
`{int i; struct {char c; short s;} in;}`	0, 4 (inner size 4)	8
`{char *p; char c;}`	0, 8	16

Q2.7 What does `volatile` do? Give a scenario where you must use it.

CISCO · QUALCOMM · TECH-1 · ■■□ · S

30-second answer: `volatile` tells the compiler that an object may change outside the program's control, so every source-level access must really be performed and not cached in a register or removed.

Answer. Scenarios: a memory-mapped device register, a flag set in a signal handler or interrupt routine, and variables across `setjmp`. Evidence: at `-O2` a plain `while (!flag);` loads `flag` once and spins on a `jmp`, whereas the `volatile` version reloads (Lab 2.2, 2.6 (p. 114)). **Mistake.** Using it for thread synchronisation (Q2.9).

Q2.8 Can a variable be both `const` and `volatile`? When?

QUALCOMM · TECH-1 · ■■□ · S

30-second answer: yes; the program must not write it, but hardware may change it, as with a read-only status register.

Answer. `volatile const uint32_t *status = (volatile const uint32_t *)0x40021000;` forbids stores through `status` and forces a fresh load on each read. `const` alone would allow the compiler to assume the value never changes.

Q2.9 Is `volatile` enough to share a flag between two threads?

ARISTA · QUALCOMM · TECH-2 · ■■■ · T

30-second answer: no; it stops compiler caching but gives neither atomicity nor ordering between processors.

Answer. Another core may see the write late or out of order relative to other writes, and a multi-word update may be seen torn. Use `_Atomic` (C11) or `atomic_flag`, or a mutex with a condition variable (Pillar 7). **Follow-up.** For a single-core signal handler, `volatile sig_atomic_t` is the correct pairing.

Q2.10 Write a program to detect whether the machine is little- or big-endian.

CISCO · OA · ■□□ · S

30-second answer: store 1 in a 32-bit integer and read its first byte; 1 means little-endian.

Code. `union { unsigned i; unsigned char c[4]; } u = { .i = 1 }; return u.c[0] == 1;` An equivalent form is `*(unsigned char *)&one`, which is allowed because character types may alias anything. **Mistake.** Testing with bit shifts: shifts act on values and give the same result on both kinds of machine.

Q2.11 Why do we need `htons` and `htonl`? What breaks if we forget them?

CISCO · JUNIPER · TECH-1 · ■□□ · S

30-second answer: the network uses big-endian order and x86 is little-endian, so multi-byte fields must be converted; otherwise the peer reads the bytes reversed.

Answer. Port 80 (`0x0050`) stored little-endian is `50 00`, which the peer reads as `0x5000`, i.e. 20480. On a big-endian host the functions do nothing, so the code is portable. **Mistakes.** Converting twice (which restores the wrong order), converting a value that is not in host order, and using `htons` on a 32-bit value.

Q2.12 Swap the byte order of a 32-bit integer without library calls.

CISCO · BROADCOM · TECH-1 · ■■□ · T

30-second answer: shift each byte to its mirrored position and OR them together.

Code. `(x >> 24) | ((x >> 8) & 0xFF00) | ((x << 8) & 0xFF0000) | (x << 24)` with `uint32_t x`; `swap16` is `(uint16_t)((x << 8) | (x >> 8))`. `swap32(0x0A0B0C0D)` is `0x0D0C0B0A`. **Mistake.** Using signed `int`, so that `x >> 24` sign-extends. GCC offers `__builtin_bswap32`.

Q2.13 Explain the memory layout of a C program. Where do globals, statics, locals, `malloc` memory and literals live?

ARISTA · QUALCOMM · TECH-1 · ■□□ · S

30-second answer: text holds code, rodata holds literals and const globals, data holds initialised globals and statics, BSS holds zero-initialised ones, the heap holds `malloc` blocks, and the stack holds locals and return addresses.

Answer. Draw Figure 2.1 in the order text, rodata, data, BSS, heap (growing up), gap, stack (growing down). A `static` local is in data or BSS. **Follow-ups.** Why BSS is not stored in the file; why the heap and stack grow towards each other; what happens on stack overflow (a fault on the guard page).

Q2.14 What is the difference between a stack and a heap allocation?

CISCO · QUALCOMM · TECH-1 · ■□□ · T

30-second answer: stack storage is automatic, fast and fixed in size per thread, and ends with the function; heap storage is explicit, slower, large, and lives until `free`.

Answer. The stack is typically 8 MB per thread on Linux (`ulimit -s`), so a large local array overflows it; the heap is limited by memory. Stack allocation is a pointer adjustment; `malloc` searches free lists and can fail. Heap errors are leaks and double frees; stack errors are overflow and dangling pointers to dead frames.

Q2.15 What causes a segmentation fault?

ARISTA · CISCO · TECH-1 · ■□□ · T

30-second answer: an access to an address that is unmapped or not permitted, such as a null or wild pointer, a write to read-only memory, or a stack overflow.

Answer. Typical C causes: dereferencing `NULL`, use after free, writing to a string literal, indexing far outside an array, and unbounded recursion. Method: compile with `-g -fsanitize=address`, rerun, and read the first frame of the report; or run under `gdb` and type `bt`.

Q2.16 What is the difference between a static global variable, a static local variable and a static function?

CISCO · BROADCOM · TECH-1 · ■□□ · S

30-second answer: a static local keeps its value across calls; a static global or function is visible only within its file.

Answer. Local: static storage duration, zero-initialised, one instance, lives in data or BSS. Global and function: internal linkage, so other files cannot link to the name and the same name may be reused in another file. **Mistake.** Putting `static int x;` in a header: each including file gets a private copy.

Q2.17 What is wrong with `#define SQUARE(x) x*x`? Fix it. What does `SQUARE(i++)` do?

CISCO · OA · ■□□ · S

30-second answer: the argument is pasted without parentheses, so `SQUARE(2+3)` is `2+3*2+3 = 11`; the fix is `((x)*(x))`, and `i++` is then evaluated twice, which is undefined.

Answer. Also `10/SQUARE(2)` expands to `10/2*2 = 10`, which shows why the whole body needs parentheses. For side effects use a `static inline` function, which evaluates its argument once (2.11 (p. 120)).

Q2.18 Macro versus inline function: when would you use each?

CISCO · TECH-1 · ■■□ · S

30-second answer: prefer `static inline` for typed computation; use a macro only for generic code, stringification, token pasting, source location capture and conditional compilation.

	Macro	`static inline`
Type checking	none	full
Argument evaluation	each use re-evaluates	once
Debugging	expanded text, no symbol	steppable
Generic over types	yes	no (C11 `_Generic` aside)

Q2.19 Predict the output: `int a[] = {1,2,3}; int *p = a; printf("%d %d", *p++, *++p);`

CISCO · OA · ■■□ · S

30-second answer: undefined behaviour; `p` is modified twice without sequencing, and the order of evaluating arguments is unspecified.

Answer. Say so first, then show the defined rewrite: `int x = *p++; int y = *++p;` gives `x = 1` and `y = 3`, printed as `1 3`, because the statements are sequenced and `p` visits `a[1]` between them.

Q2.20 Explain `const int *p`, `int *const p` and `const int *const p`. Quiz of six declarations.

CISCO · QUALCOMM · TECH-1 · ■□□ · I

30-second answer: read right to left; `const` applies to what is on its left, so the first protects the pointee, the second the pointer, the third both.

Declaration	`*p = x` allowed?	`p = &y` allowed?
`const int *p`	no	yes
`int const *p`	no	yes
`int *const p`	yes	no
`const int *const p`	no	no
`const char *s`	no	yes
`char *const s`	yes	no

Q2.21 What is a dangling pointer? A memory leak? How would you find each on Linux?

CISCO · TECH-1 · ■□□ · S

30-second answer: a dangling pointer refers to storage that has been freed or has gone out of scope; a leak is allocated storage with no remaining pointer. AddressSanitizer finds the first; valgrind or LeakSanitizer finds the second.

Answer. Build with `-fsanitize=address`: a use after free prints a report with the allocation and free stacks; a leak prints the allocation line (Lab 2.3). Prevention: set pointers to `NULL` after `free`, never return the address of a local, and own each block in exactly one place.

Q2.22 `malloc` versus `calloc`. What does `realloc` do if it cannot grow in place?

ARISTA · CISCO · OA · ■□□ · S

30-second answer: `malloc` returns uninitialised memory; `calloc` returns zeroed memory and checks the multiplication for overflow; `realloc` moves the block, copying the data and freeing the old one.

Answer. If it fails it returns `NULL` and leaves the old block valid, so `p = realloc(p, n)` leaks on failure; use a temporary. Pointers into the old block dangle after a move. `free(NULL)` is a no-op.

Q2.23 Implement `memmove`. Why is `memcpy` unsafe for overlapping regions?

QUALCOMM · BROADCOM · TECH-1 · ■■□ · T

30-second answer: when the destination starts inside the source, a forward copy overwrites bytes before they are read, so `memmove` copies backwards in that case.

Code. The function of 2.13 (p. 122): if `d < s` or `d >= s + n` copy forwards, otherwise from the end. `memmove(s+2, s, 4)` on `"abcdef"` gives `"ababcd"`. Compare addresses through `uintptr_t`. **Follow-up.** `memcpy` may assume no overlap (`restrict`) and use wide copies.

Q2.24 Implement an aligned `malloc` that returns memory aligned to any power of two.

QUALCOMM · BROADCOM · CISCO · TECH-2 · ■■■ · T

30-second answer: allocate extra, round the address up to the alignment, and store the original pointer just before the returned address so that `aligned_free` can recover it.

```
void *aligned_malloc(size_t size,
size_t align) {      /* align: power of
two */
    void *raw = malloc(size + align - 1
+ sizeof(void *));
    if (!raw) return NULL;
    uintptr_t a = ((uintptr_t)raw +
sizeof(void *) + align - 1) & ~
(uintptr_t)(align - 1);
    ((void **)a)[-1] = raw;
/* remember the real block */
    return (void *)a;
}
void aligned_free(void *p) { if (p)
free(((void **)p)[-1]); }
```

Why. Adding `align - 1` guarantees an aligned address inside the block; the mask `~(align - 1)` rounds down to a multiple. Room for the saved pointer is reserved before rounding. Check: `aligned_malloc(100, 64)` returns an address with `a % 64 == 0`. **Follow-up.** C11 offers `aligned_alloc`; POSIX `posix_memalign`.

Q2.25 Write a function to delete a node from a singly linked list; then do it without special-casing the head.

ARISTA · TECH-1 · ■■□ · S

30-second answer: walk with a pointer to the link, not to the node, so the head and an inner node are handled identically.

```
void list_delete(struct node **pp, int
key) {
    while (*pp && (*pp)->v != key) pp =
&(*pp)->next;   /* pp points at the link
*/
    if (*pp) { struct node *dead = *pp;
*pp = dead->next; free(dead); }
}
```

Complexity. O(n) time, O(1) space. With list `1,2,3`, deleting 1, 3 and 9 leaves `2`. The classic version keeps a `prev` pointer and tests `if (!prev)` for the head. The full list library (reverse, cycle detection) belongs to Pillar 1 (p. 7).

Q2.26 What are function pointers? Show how to dispatch packets to handlers by protocol number.

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: a function pointer holds the address of code; a 256-entry array of them indexed by the protocol field replaces a `switch`, and an empty slot is `NULL`.

Answer. `typedef int (*handler_fn)(const uint8_t *, size_t); static handler_fn table[256] = { [6] = on_tcp, [17] = on_udp };` then `fn = table[proto]; if (fn) fn(pkt, len);`. Handlers can register at run time by writing a slot. Full code and output in 2.5 (p. 112).

Q2.27 What happens in recursion versus an infinite loop?

CISCO · TECH-1 · ■□□ · S

30-second answer: infinite recursion consumes a stack frame per call and ends with a stack-overflow crash, whereas an infinite loop uses constant space and runs forever.

Answer. The default stack of 8 MB holds on the order of 10^5 small frames. Compilers may turn tail calls into loops at `-O2`, in which case recursion also spins. Example: `sum(n--)` never reduces its argument (T17, 2.15 (p. 128)).

Q2.28 Is `i = i++ + ++i;` valid? What does it print?

CISCO · QUALCOMM · OA · ■□□ · T

30-second answer: it compiles but has undefined behaviour, because `i` is modified twice without a sequence point; no output can be called correct.

Answer. Name the rule, say that gcc's `-Wsequence-point` (in `-Wall`) warns, and state that different compilers or optimisation levels may give different values. Similar: `a[i] = i++`, `f(i++, i++)`.

Q2.29 What is `offsetof`? How does `container_of` work?

ARISTA · BROADCOM · TECH-2 · ■■■ · T

30-second answer: `offsetof(T, m)` is the byte offset of member `m` within `T`; `container_of` subtracts that offset from a pointer to the member to recover the enclosing structure.

Answer. `#define container_of(ptr, T, m) ((T *)((char *)(ptr) - offsetof(T, m)))`. If `struct pkt { int id; struct list_head node; }` and `n` points at a `node`, `container_of(n, struct pkt, node)` is the packet. This lets one generic list implementation link any structure (intrusive list), as in the Linux kernel.

Q2.30 Can you send a C structure directly over a socket? What could go wrong?

CISCO · JUNIPER · ARISTA · TECH-1 · ■■□ · T

30-second answer: not safely; padding, member sizes and byte order differ between machines and compilers, and pointers are meaningless to the receiver.

Answer. Serialise field by field into a byte buffer in network order, as in 2.14 (p. 124), and send the buffer. Packing removes padding but not endianness, and risks misaligned access. Also never send structures holding pointers. Sockets themselves are in Pillar 7 (p. 414).

Q2.31 Bit-fields for protocol headers: good idea or not?

QUALCOMM · BROADCOM · TECH-1 · ■■□ · T

30-second answer: not for wire formats; the standard leaves field order and straddling to the implementation, so portable code uses shifts and masks.

Answer. The experiment of 2.8 (p. 117) shows `{version:4, ihl:4}` giving `0x54` and `{ihl:4, version:4}` giving `0x45` on x86-64 gcc; a big-endian compiler reverses this. Bit-fields suit private structures and registers on a known compiler. `(b >> 4)` and `(b & 0x0F)` work everywhere.

Q2.32 How would you test the memory bit-cells of a memory segment using C?

BROADCOM · TECH-2 · ■■■ · S

30-second answer: write walking-one patterns and their complements to every byte, read them back, and report the first mismatch; use a `volatile` pointer so the accesses really happen.

```c
int memtest(volatile uint8_t *p, size_t n) {
    for (int bit = 0; bit < 8; bit++) {
        uint8_t pat = (uint8_t)(1u << bit);
        for (size_t i = 0; i < n; i++) p[i] = pat;
        for (size_t i = 0; i < n; i++) if (p[i] != pat) return -1;
        for (size_t i = 0; i < n; i++) p[i] = (uint8_t)~pat;
        for (size_t i = 0; i < n; i++) if (p[i] != (uint8_t)~pat) return -1;
    }
    return 0;
}
```

Complexity. O(16 n) accesses. A healthy region returns 0. **Follow-ups.** A single pattern misses stuck-at and coupling faults, hence both polarities; address-line faults need a separate test that writes a unique value to addresses that are powers of two. Run it with caches disabled or on uncached memory, or the cache hides the cell.

Q2.33 Edit, compile and run this program from the Linux command line. It crashes; debug it.

ARISTA · TECH-1 · ■■□ · S

30-second answer: compile with `-g -Wall -Wextra -fsanitize=address,undefined`, run, and read the first report frame; if needed use `gdb` with `run`, `bt`, `frame`, `print`.

Approach. (i) Read the compiler warnings: uninitialised variables, returning a local, format mismatches. (ii) Run under ASan: it names the line of an out-of-bounds or use-after-free access. (iii) In `gdb ./a.out`: `run`, then at the crash `bt` (call chain), `frame 1`, `print *node`, `info locals`; `watch x` stops when `x` changes. (iv) Fix, rebuild, and rerun. **Mistake.** Adding `printf` calls without understanding, which can hide a race or an overflow by changing layout.

Q2.34 `uint8_t a = 200, b = 100;` What does `printf("%d", a + b)` print, and why?

CISCO · OA · ■□□ · T

30-second answer: 300, because both operands are promoted to `int` before the addition, so nothing wraps.

Answer. Only `uint8_t c = a + b;` would truncate to 44. Contrast `unsigned a = 1; int b = -1; b < a` which is false because `b` converts to unsigned (2.2 (p. 109)).

Q2.35 `char *s = "abc"` versus `char s[] = "abc"`: what differs?

ARISTA · QUALCOMM · OA · ■□□ · T

30-second answer: the pointer refers to read-only characters in .rodata and has size 8; the array is a writable 4-byte copy.

Answer. `sizeof` gives 8 and 4; writing `s[0]` faults in the first case. The pointer can be reassigned, the array name cannot. Figure 2.3.

Q2.36 What do `restrict` and `inline` mean?

BROADCOM · NOKIA · TECH-1 · ■■□ · T

30-second answer: `restrict` promises that no other pointer accesses the same object during the pointer's lifetime, enabling optimisation; `inline` suggests substituting the function body at call sites.

Answer. `memcpy(void *restrict, const void *restrict, size_t)` carries the promise, which is why overlap is undefined. A broken promise is undefined behaviour. `static inline` in a header avoids multiple definitions.

Q2.37 Why use `<stdint.h>` types for packet fields?

BROADCOM · QUALCOMM · TECH-1 · ■□□ · T

30-second answer: `int` and `long` change size between platforms, whereas `uint16_t` and `uint32_t` have exactly the width the protocol specifies.

Answer. A TCP port is `uint16_t`, an IPv4 address `uint32_t`, a byte `uint8_t`. Print with `PRIu32` from `<inttypes.h>`. Unsigned types also give defined wrap-around for sequence numbers.

Q2.38 What is the effect of making a big local array `static`?

QUALCOMM · TECH-1 · ■■□ · S

30-second answer: the array moves from the stack to .data or .bss, which removes the stack-overflow risk and keeps its contents between calls, but the function is no longer re-entrant or thread-safe.

Answer. The binary's BSS grows but its file size does not. Two callers, or two threads, share the same bytes. The alternative is `malloc`, freed by the caller.

Problem Set, continued: Python, C++ and coding items

Q2.39 Python: given `show ip interface brief` text, list the interfaces whose status or protocol is not up.

AMAZON NDE · TECH-1 · ■□□ · S

30-second answer: split each line into whitespace-separated fields, take the name from the first and the status and protocol from the last fields, and filter.

```
def parse_brief(text):
    out = {}
    for line in text.splitlines():
        f = line.split()
        if len(f) < 6 or not re.match(r"
[A-Za-z]+\d", f[0]):
            continue
# skips the header and blank lines
        out[f[0]] = (" ".join(f[4:-1]),
f[-1])         # status may be two words
    return out
bad = [i for i, sp in
parse_brief(text).items() if sp !=
("up", "up")]
```

For the three-interface sample (Gi0/1 up/up, Gi0/2 `administratively down`/down, Gi0/3 up/down) the result is `['GigabitEthernet0/2', 'GigabitEthernet0/3']`. **Mistake.** `split(" ")` instead of `split()`, which breaks on runs of spaces. **Follow-up.** Structured output (JSON) is preferable when the device offers it; see Pillar 10 (p. 586).

Q2.40 Python: extract IP addresses from a log and sort by frequency. What if the file is 50 GB?

GOOGLE · TECH-1 · ■■□ · S

30-second answer: stream the file line by line into a `Counter`; memory depends on the number of distinct addresses, not on the file size.

Answer. The pipeline `Counter(src_ips(read_lines(path))).most_common(k)` of 2.17 (p. 135) runs in one pass, O(n) time. If the distinct set does not fit, partition addresses by hash into several files and count each, or sort externally. For the top `k` only, `heapq.nlargest` costs O(n log k). **Mistake.** `f.readlines()`, which loads everything.

Q2.41 Python: time complexity of `x in list` versus `x in set`? Why?

GOOGLE · AMAZON · OA · ■□□ · I

30-second answer: O(n) for a list, because it compares each element; O(1) on average for a set, because it hashes the key and inspects one bucket.

Answer. Hence membership tests inside loops should use a set built once. The set requires hashable (immutable) elements, and its worst case is O(n) with many collisions.

Q2.42 Python: explain the mutable default argument bug.

GOOGLE · MICROSOFT · OA · ■□□ · I

30-second answer: a default value is evaluated once at definition time, so a default list or dict is shared by all calls that omit the argument.

Answer. `def f(x, acc=[])` returns `[1, 2]` for both `f(1)` and `f(2)` as printed after the second call. Fix: `acc=None` and `if acc is None: acc = []` (2.16 (p. 134)).

Q2.43 Python: what is the difference between `is` and `==`? What is a shallow copy?

MICROSOFT · GOOGLE · OA · ■□□ · T

30-second answer: `==` compares values and `is` compares object identity; a shallow copy duplicates the container but shares the elements.

Answer. `a = [[0]] * 2` makes two references to one inner list; `a[0].append(1)` changes both rows. Use `is` only against `None`. For independent nested copies use `copy.deepcopy` or a comprehension `[[0] for _ in range(2)]`.

Q2.44 Tell me about an automation script you wrote: what it did, how you tested it and how it failed.

AMAZON NDE · TECH-2 · ■■□ · S

30-second answer: describe one concrete script in four parts: the manual task it replaced, how it handled bad input and device errors, how you tested it safely, and one failure with the fix and what you now do differently.

Approach. Choose a script of your own, for example one that collects `show` output from several devices and reports mismatches. State the scale (devices, minutes saved). Testing: a dry-run mode, a lab or a single device first, unit tests on saved outputs. Failure: a timeout on one device that stopped the run, fixed with per-device `try/except`, a timeout and a summary. Use the STAR frame of Part II. **Mistake.** Claiming no failures.

Q2.45 C++: what is a virtual function and how is it implemented? Why a virtual destructor?

NOKIA · CISCO · TECH-1 · ■■□ · I

30-second answer: a virtual function is chosen by the object's dynamic type through a hidden vptr to a per-class vtable; the destructor must be virtual so that `delete` through a base pointer runs the derived destructor.

Answer. Figure 2.7. Cost: one pointer per object and one indirect call. A class with `virtual` members and an `int` is 16 bytes on LP64, without them 4. Without a virtual destructor, `delete base_ptr` runs only `~Base`, which is undefined. **Follow-up.** Pure virtual (`= 0`) makes the class abstract.

Q2.46 C++: what is RAII? Give an example with locks or files.

MICROSOFT · GOOGLE · TECH-1 · ■■□ · I

30-second answer: tie a resource to an object's lifetime, acquiring in the constructor and releasing in the destructor, so that every exit path, including an exception, releases it.

Answer. `std::lock_guard<std::mutex> g(m);` locks at construction and unlocks when `g` leaves scope; `std::ifstream` closes its file; `std::unique_ptr` frees memory. Destruction order is the reverse of construction (output of `work(true)` in 2.19 (p. 138)).

Q2.47 C++: `map` versus `unordered_map`: implementation and complexity?

GOOGLE · MICROSOFT · TECH-1 · ■□□ · I

30-second answer: `map` is a balanced tree, ordered, O(log n) per operation; `unordered_map` is a hash table, unordered, O(1) on average and O(n) worst case.

Answer. Choose `map` when sorted iteration or range queries (`lower_bound`) are needed, `unordered_map` for plain lookup. `operator[]` inserts a default value for a missing key; use `find` or `count` to test. Keys of `unordered_map` need a hash function.

Q2.48 C++: when are iterators and pointers into a `vector` invalidated?

GOOGLE · TECH-1 · ■■□ · T

30-second answer: whenever the vector reallocates (growth beyond capacity, `reserve`), and for elements at or after the point of an `insert` or `erase`.

Answer. With `v{1,2,3}`, `p = &v[0]` and `v.reserve(100)`, the comparison `p == &v[0]` is false. Fixes: reserve before taking pointers, keep indices, or use `list`. **Mistake.** Erasing inside a range-for; use `it = v.erase(it)` or the erase-remove idiom.

Q2.49 C++: explain `unique_ptr`, `shared_ptr` and `weak_ptr`. How do cycles leak?

MICROSOFT · GOOGLE · TECH-1 · ■■□ · T

30-second answer: `unique_ptr` is a sole, movable owner; `shared_ptr` shares ownership by reference count; `weak_ptr` observes without owning and breaks cycles.

Answer. Two `shared_ptr` nodes pointing at each other keep both counts at least 1, so neither destructor runs; making the back-link a `weak_ptr` frees both. Default to `unique_ptr`; create with `make_unique` or `make_shared`.

Q2.50 C++: what is object slicing, and what are the rules of zero, three and five?

NOKIA · MICROSOFT · TECH-2 · ■■□ · T

30-second answer: slicing copies only the base part of a derived object into a base object, losing derived data and virtual behaviour; the rules say which special members to write when a class owns a resource.

Answer. `Shape sl = circle;` makes `sl.area()` return the base result 0. Pass polymorphic objects by pointer or reference. Rule of zero: own resources through members such as `vector` and `unique_ptr` and write no special members. Rules of three and five: if you write a destructor, copy or move operation, define or delete all of copy constructor, copy assignment, destructor and (five) the two move operations.

Q2.51 Describe what happens on the stack when a function is called.

CISCO · QUALCOMM · TECH-1 · ■■□ · T

30-second answer: the caller places arguments (in registers on x86-64, the rest on the stack) and a call instruction pushes the return address; the callee saves the frame pointer and reserves space for its locals; return reverses this.

Answer. A frame holds the return address, the saved frame pointer, locals and spills. A buffer overflow of a local array can overwrite the return address, which is why `gets` and unchecked `strcpy` are exploitable and why stack protectors place a canary value before the return address. Details of the call stack and virtual memory: Pillar 7 (p. 414).

Q2.52 Swap two integers in three ways. Why does the XOR swap fail when both pointers are the same?

QUALCOMM · TECH-1 · ■□□ · T

30-second answer: with a temporary, with addition and subtraction, and with XOR; the XOR swap sets the value to zero if both pointers alias the same object.

```
void swap_tmp(int *a, int *b)       {
int t = *a; *a = *b; *b = t; }
void swap_add(unsigned *a, unsigned *b)
{ *a = *a + *b; *b = *a - *b; *a = *a -
*b; }
void swap_xor(unsigned *a, unsigned *b)
{ *a ^= *b; *b ^= *a; *a ^= *b; }
```

Why. If `a == b`, the first XOR makes the object `x ^ x = 0`, and the rest keep it zero. The unsigned types avoid signed overflow in the addition version. The temporary version is the one to write: it is clearest and compilers optimise it.

Q2.53 Count the set bits of a 32-bit value; test for a power of two.

QUALCOMM · TECH-1 · ■□□ · S

30-second answer: `x &= x - 1` clears the lowest set bit, so count the iterations; a power of two has exactly one set bit.

```
int popcount(uint32_t x) { int n = 0;
while (x) { x &= x - 1; n++; } return
n; }
int is_pow2(uint32_t x)  { return x && !
(x & (x - 1)); }
```

Complexity. O(number of set bits). `popcount(0xF0F0)` is 8, `is_pow2(64)` is 1 and `is_pow2(96)` is 0. GCC: `__builtin_popcount`.

Q2.54 Reverse a string in place.

BROADCOM · OA · ■□□ · T

30-second answer: swap characters from both ends with two pointers until they meet.

```
void reverse(char *s) {
    if (!*s) return;
    char *i = s, *j = s + strlen(s) - 1;
    while (i < j) { char t = *i; *i++ =
*j; *j-- = t; }
}
```

`"router"` becomes `"retuor"`. **Mistakes.** Passing a string literal (read-only; use an array) and computing `strlen(s) - 1` for the empty string, where `size_t` wraps.

Q2.55 Convert a dotted quad to a `uint32_t` with validation, and back.

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: parse four decimal fields of at most three digits and value at most 255, separated by single dots, shifting each into place; reject anything else.

```
int dotted_to_u32(const char *s,
uint32_t *out) {
    uint32_t addr = 0; int parts = 0;
    for (;;) {
        if (*s < '0' || *s > '9') re-
turn -1;
        unsigned v = 0; int digits = 0;
        while (*s >= '0' && *s <= '9') {
            v = v * 10 + (unsigned)
(*s++ - '0');
            if (++digits > 3 || v >
255) return -1;
        }
        addr = (addr << 8) | v;
        if (++parts == 4) break;
        if (*s++ != '.') return -1;
    }
    if (*s != '\0') return -1;
    *out = addr; return 0;
}
```

`"192.168.1.10"` returns 0 with `0xC0A8010A` (host order; apply `htonl` for the wire). `"1.2.3"`, `"1.2.3.256"` and `"1.2.3.4.5"` return -1. The reverse is `u32_to_dotted` in 2.14 (p. 124). **Mistake.** Accepting empty fields (`1..2.3`) or trailing junk.

Q2.56 Implement a circular buffer with power-of-two size.

BROADCOM · QUALCOMM · TECH-1 · ■■□ · I

30-second answer: keep two free-running counters and mask them with `size - 1`; the buffer is empty when they are equal and full when they differ by the size.

```
#define RB_SIZE 8
/* power of two */
struct ring { uint8_t buf[RB_SIZE]; un-
signed head, tail; };
int rb_put(struct ring *r, uint8_t b) {
    if (r->head - r->tail == RB_SIZE)
return -1;   /* full  */
    r->buf[r->head++ & (RB_SIZE - 1)] =
b; return 0;
}
int rb_get(struct ring *r, uint8_t *b) {
    if (r->head == r->tail) return -1;
/* empty */
    *b = r->buf[r->tail++ & (RB_SIZE -
1)]; return 0;
}
```

Ten puts into an empty ring succeed 8 times; the first get returns 0. Unsigned subtraction keeps `head - tail` correct after the counters wrap. **Why the mask.** `& (N-1)` replaces `% N` and wastes no slot. One producer and one consumer need no lock if the counters are atomic.

Q2.57 Write a bounds-safe TLV (type-length-value) parser that rejects truncated and overlong items.

CISCO · JUNIPER · TECH-2 · ■■■ · T

30-second answer: before reading each item, check that two header bytes remain, then that the declared length fits in what remains; subtract rather than add, to avoid overflow.

```
int tlv_walk(const uint8_t *p, size_t
len) {
    size_t off = 0;
    while (off < len) {
        if (len - off < 2) return -1;
/* no room for type and length */
        unsigned t = p[off], l = p[off
+ 1];
        if (l > len - off - 2) return
-1;          /* value would run past the
end */
        printf("type %u len %u\n", t,
l);
        off += 2 + l;
    }
    return 0;
}
```

For `{1,2,0xAA,0xBB,2,0}` it prints `type 1 len 2` and `type 2 len 0`, returning 0; for `{1,5,0xAA}` it returns -1. **Mistake.** Testing `off + 2 + l > len`, which can overflow for wide length fields. Use a 16- or 32-bit length the same way.

Q2.58 Build a MAC learning table with a hash and ageing.

ARISTA · BROADCOM · TECH-2 · ■■□ · T

30-second answer: hash the 6-byte MAC into a bucket of chained entries; a frame refreshes or inserts the entry for its source MAC, lookups return the port, and an ageing sweep removes entries not seen for the timeout.

```
struct mac_ent { uint8_t mac[6];
uint16_t port; uint32_t seen; struct
mac_ent *next; };
static struct mac_ent *tab[8];
static unsigned h(const uint8_t *m) {
uint32_t x = 2166136261u; for (int i =
0; i < 6; i++) { x ^= m[i]; x *=
16777619u; } return x % 8; }
void learn(const uint8_t *m, uint16_t
port, uint32_t now) {
    struct mac_ent **pp = &tab[h(m)];
    for (; *pp; pp = &(*pp)->next)
        if (!memcmp((*pp)->mac, m, 6))
{ (*pp)->port = port; (*pp)->seen =
now; return; }
    struct mac_ent *e = calloc(1,
sizeof *e); if (!e) return;
    memcpy(e->mac, m, 6); e->port =
port; e->seen = now; *pp = e;
}
void age(uint32_t now, uint32_t max) {
    for (int i = 0; i < 8; i++)
        for (struct mac_ent **pp =
&tab[i]; *pp; ) {
            if (now - (*pp)->seen >
max) { struct mac_ent *d = *pp; *pp = d-
>next; free(d); }
            else pp = &(*pp)->next;
        }
}
```

Learning MAC A on port 3 at time 100, B on port 4 at 200, and A again on port 5 at 210, a lookup gives 5 and 4; `age(305, 100)` removes B (105 s old) and keeps A (95 s). The lookup loop is the same as in `learn`. A station moving between ports is handled by the update branch. Why the switch behaves this way: Pillar 4 (p. 229).

Q2.59 Write a fixed-size pool allocator for packet buffers.

BROADCOM · CISCO · TECH-2 · ■■■ · T

30-second answer: carve a static array into equal blocks and keep free blocks on a singly linked list threaded through the blocks themselves; allocation and free are O(1) pointer swaps.

```
#define POOL_N 4
#define BUF_SZ 64
static union blk { union blk *next;
uint8_t data[BUF_SZ]; } pool[POOL_N];
static union blk *freelist;
void  pool_init(void)   { for (int i =
0; i < POOL_N; i++) { pool[i].next =
freelist; freelist = &pool[i]; } }
void *pool_alloc(void)  { union blk *b
= freelist; if (b) freelist = b->next;
return b; }
void  pool_free(void *p){ union blk *b
= p; b->next = freelist; freelist = b; }
```

The union lets a free block store the link where an allocated block stores data, so there is no per-block overhead. After four allocations the fifth returns `NULL`; after `pool_free(y)` the next allocation returns `y`. **Benefits.** No fragmentation, no `malloc` latency, bounded memory. **Follow-ups.** Thread safety (a lock or per-core pools) and alignment of `BUF_SZ`.

Q2.60 Write a re-entrant tokeniser in the style of `strtok_r`.

BROADCOM · QUALCOMM · TECH-2 · ■■■ · T

30-second answer: keep the scan position in a caller-supplied pointer instead of a static variable; skip leading delimiters, mark the token's end with a NUL, and save the position after it.

```
char *my_strtok_r(char *s, const char
*delim, char **save) {
    if (!s) s = *save;
    s += strspn(s, delim);
/* skip leading delimiters */
    if (!*s) { *save = s; return NULL; }
    char *tok = s;
    s += strcspn(s, delim);
/* find the end of the token */
    if (*s) *s++ = '\0';
    *save = s;
    return tok;
}
```

On `"a,b;;c"` with delimiters `",;"` the tokens are `[a][b][c]`; consecutive delimiters give no empty token. The input is modified, so it must be writable. Plain `strtok` keeps its state in a hidden static, so it is not thread-safe.

Q2.61 Write a `printf`-lite supporting `%d %x %s %c` using `stdarg.h`.

BROADCOM · QUALCOMM · TECH-2 · ■■■ · T

30-second answer: walk the format string; copy ordinary characters, and on `%` read the matching argument with `va_arg` and print it in the right base.

```
int my_printf(const char *fmt, ...) {
    va_list ap; va_start(ap, fmt); int n
= 0;
    for (; *fmt; fmt++) {
        if (*fmt != '%') {
putchar(*fmt); n++; continue; }
        switch (*++fmt) {
        case 'd': { int v = va_arg(ap,
int); char b[12]; int i = 0;
                    unsigned u = v < 0
? 0u - (unsigned)v : (unsigned)v;
                    do { b[i++] = (char)
('0' + u % 10); u /= 10; } while (u);
                    if (v < 0) {
putchar('-'); n++; }
                    while (i) {
putchar(b[--i]); n++; } break; }
        case 'x': { unsigned u =
va_arg(ap, unsigned); char b[9]; int i
= 0;
                    do { b[i++] =
"0123456789abcdef"[u & 15]; u >>= 4; }
while (u);
                    while (i) {
putchar(b[--i]); n++; } break; }
        case 's': { const char *s =
va_arg(ap, const char *); while (*s) {
putchar(*s++); n++; } break; }
        case 'c': putchar(va_arg(ap,
int)); n++; break;
        case '%': putchar('%'); n++;
break;
        default:  putchar('%');
putchar(*fmt); n += 2;
        }
    }
    va_end(ap); return n;
}
```

`my_printf("%s=%d %x %c %% %d\n", "ttl", 64, 255, 'z', -5)` prints `ttl=64 ff z % -5` and returns 17. `char` is read as `int` because of the default argument promotions. **Mistake.** Reading with the wrong type, which is undefined, exactly as in a real `printf`.

Q2.62 Python: count the lines containing `ERROR`; build a dictionary from `show` output lines.

GOOGLE · AMAZON · OA · ■□□ · T

30-second answer: a generator expression with `sum` counts lazily; a dictionary comprehension pairs the first two fields of each line.

```
n = sum(1 for line in open(path) if
"ERROR" in line)          # 2 for lines
ERROR a, ok, ERROR b
states = {k: v for k, v in (l.split()
[:2] for l in lines)}    # {'Gi0/1':
'up', 'Gi0/2': 'down'}
```

Use `with open(path) as f` in production so the file is closed. For the dictionary, skip blank lines first, since `l.split()[:2]` of an empty line unpacks to the wrong length.

Q2.63 Python: unpack an IPv4 header with `struct` and verify its checksum.

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: unpack with `"!BBHHHBBH4s4s"`, then add all 16-bit words with end-around carry; a valid header sums to `0xFFFF`.

```
def csum_ok(hdr: bytes) -> bool:
    if len(hdr) % 2: hdr += b"\x00"
    total = sum(struct.unpack(f"!
{len(hdr) // 2}H", hdr))
    while total >> 16: total = (total &
0xFFFF) + (total >> 16)    # fold the
carries
    return total == 0xFFFF
```

For the header of 2.14 (p. 124) it returns `True`; clearing the checksum bytes (positions 10 and 11) gives `False`. The unpack is in 2.18 (p. 137). **Mistake.** Native format `"BBHH..."` instead of `!`, giving the wrong order and padding.

Q2.64 Python: a log-to-`Counter`-to-top-k report with `argparse`.

GOOGLE · TECH-1 · ■■□ · S

30-second answer: parse arguments, stream the file into a `Counter` keyed by the first field, and print `most_common(k)`.

```
def main(argv=None):
    ap =
argparse.ArgumentParser(description="Top
talkers")
    ap.add_argument("logfile");
ap.add_argument("-k", type=int,
default=3)
    a = ap.parse_args(argv)
    c = Counter()
    with open(a.logfile) as f:
        for line in f:
            parts = line.split()
            if parts: c[parts[0]] += 1
# skip blank lines
    for ip, n in c.most_common(a.k):
print(f"{n:6d}  {ip}")
```

On a file with `1.1.1.1` three times, `2.2.2.2` twice and `3.3.3.3` once, `-k 2` prints `3 1.1.1.1` and `2 2.2.2.2`. `argparse` supplies `--help` and error messages free. Add `if __name__ == "__main__": main()`.

Q2.65 Python: a streaming parser for a 10 GB flow log, reporting the top sources by bytes.

GOOGLE · AMAZON NDE · TECH-2 · ■■■ · T

30-second answer: a generator yields (source, bytes) pairs and skips malformed lines; a dictionary accumulates totals; `heapq.nlargest` selects the top k.

```
def flow_bytes(lines):
    for l in lines:
        p = l.split()
        if len(p) == 3 and
p[2].isdigit(): yield p[0], int(p[2])
# src dst bytes

tot = defaultdict(int)
for src, b in flow_bytes(open(path)):
tot[src] += b
top = heapq.nlargest(2, tot.items(),
key=lambda kv: kv[1])
```

With lines `10.0.0.1 10.0.0.9 1500`, `10.0.0.2 10.0.0.9 40`, `10.0.0.1 10.0.0.8 900`, `10.0.0.3 10.0.0.9 2000` and a malformed line, the result is `[('10.0.0.1', 2400), ('10.0.0.3', 2000)]`. Memory is proportional to the number of sources. **Follow-up.** For billions of sources use sampling or a count-min sketch; neither is needed at interview depth.

Q2.66 C++: implement `RingBuffer<T, N>` as a template.

BROADCOM · QUALCOMM · TECH-2 · ■■□ · T

30-second answer: the C ring buffer of Q2.56 with the element type and capacity as template parameters and a compile-time check that `N` is a power of two.

```
template <typename T, std::size_t N>
class RingBuffer {
    static_assert(N > 0 && (N & (N -
1)) == 0, "N must be a power of two");
    std::array<T, N> buf_{};
    std::size_t head_ = 0, tail_ = 0;
// free-running counters
public:
    bool push(const T &v) { if (size()
== N) return false; buf_[head_++ & (N -
1)] = v; return true; }
    bool pop(T &out)      { if
(empty()) return false; out =
buf_[tail_++ & (N - 1)]; return true; }
    std::size_t size() const { return
head_ - tail_; }
    bool empty() const { return head_
== tail_; }
};
```

For `RingBuffer<int, 4>`, six pushes give four successes; the first `pop` yields 0 and `size()` is then 3. Storage is inside the object, so there is no allocation.

Q2.67 C++: implement a move-only `unique_ptr`-lite.

MICROSOFT · GOOGLE · TECH-2 · ■■■ · T

30-second answer: own a raw pointer, delete it in the destructor, delete the copy operations and implement the move operations by stealing the pointer and nulling the source.

```
template <typename T> class UniquePtr {
    T *p_;
public:
    explicit UniquePtr(T *p = nullptr)
noexcept : p_(p) {}
    ~UniquePtr() { delete p_; }
    UniquePtr(const UniquePtr &) = de-
lete;
    UniquePtr &operator=(const UniquePtr
&) = delete;
    UniquePtr(UniquePtr &&o) noexcept :
p_(o.p_) { o.p_ = nullptr; }
    UniquePtr &operator=(UniquePtr &&o)
noexcept {
        if (this != &o) { delete p_; p_
= o.p_; o.p_ = nullptr; }
        return *this;
    }
    T &operator*() const { return *p_; }
    T *operator->() const { return p_; }
    T *release() { T *t = p_; p_ = null-
ptr; return t; }
    explicit operator bool() const { re-
turn p_ != nullptr; }
};
```

For an object that prints in its constructor and destructor, `UniquePtr<Tr> b = std::move(a);` leaves `a` empty and `b` owning, and "dtor" is printed exactly once. **Mistakes.** Forgetting the self-assignment check and `noexcept` on the move operations (containers use the copy path otherwise).

Questions solved in other pillars. Several topics reported for this pillar's companies belong elsewhere and are solved there once. The mutex and deadlock question ("What is a mutex? How do you prevent deadlock?"), thread-safe flags and the socket API belong to Pillar 7 (p. 414). The Bash one-liner for the ten most frequent source addresses, and the scripting items, belong to Pillar 8 (p. 483). The linked-list library, the least-recently-used cache and the longest-prefix-match trie belong to Pillar 1 (p. 7). Python libraries for devices (`ipaddress`, `netmiko`, concurrent reachability checks) belong to Pillar 10 (p. 586).

Revision Spread

THE 15-MINUTE VERSION

Ten facts. (1) Sections: text, rodata, data, BSS, heap, stack; a `static` local is in data or BSS. (2) `p + n` moves `n * sizeof(*p)` bytes; an array parameter is a pointer. (3) `const` binds to what is on its left: `const int *p` protects the pointee, `int *const p` the pointer. (4) Padding: each member at a multiple of its alignment; size rounded to the largest alignment; reorder by decreasing size. (5) Network order is big-endian; `0x0A0B0C0D` is `0D 0C 0B 0A` in little-endian memory. (6) `volatile` is not atomic; signed overflow, null dereference, unsequenced writes and shifts by the width are undefined. (7) Macros: parenthesise everything, never pass side effects, wrap statements in `do { } while (0)`. (8) `realloc` through a temporary; `free` then `NULL`; ASan for memory errors, UBSan for undefined behaviour, valgrind for leaks. (9) Parse packets by offset with length checks first; never cast a byte buffer to a structure. (10) Python names are references, defaults are evaluated once, generators are lazy; C++ releases resources in destructors, and a polymorphic base needs a virtual destructor.

Three figures to redraw. The process layout (Figure 2.1 (p. 107)), the padding byte map (Figure 2.4 (p. 115)) and the byte-order diagram (Figure 2.5 (p. 117)).

Sizes of the basic types, in bytes

Type	ILP32	LP64 (Linux)	LLP64 (Win64)
`char`	1	1	1
`short`	2	2	2
`int`	4	4	4
`long`	4	8	4
`long long`	8	8	8
pointer, `size_t`	4	8	8
`float`, `double`	4, 8	4, 8	4, 8

Alignment equals size for scalars on LP64. `uint8_t` .. `uint64_t` are exact on all three.

Operator precedence, high to low

Level	Operators	Assoc.
1	`() [] . ->` `x++ x--`	L to R
2	`! ~ ++x --x` unary `+ - *p &x` `sizeof` cast	R to L
3	`* / %`	L to R
4	`+ -`	L to R
5	`<< >>`	L to R
6	`< <= > >=`	L to R
7	`== !=`	L to R
8 to 10	& then ^ then |	L to R
11, 12	&& then ||	L to R
13	`?:`	R to L
14	`= += <<= ...`	R to L
15	`,`	L to R

Infamous: `*p++` is `*(p++)`; `x & 1 == 0` is `x & (1 == 0)`; `a << 1 + 2` is `a << 3`; `a + b << c` is `(a + b) << c`.

Format specifiers

Spec	Type
`%d` `%i`	`int`
`%u` `%x` `%o`	`unsigned` (decimal, hex, octal)
`%ld` `%lu`	`long`, `unsigned long`
`%lld` `%llu`	`long long`
`%zu` `%zd`	`size_t`, `ssize_t`
`%c` `%s`	`char`, string
`%f` `%e` `%g`	`double`
`%p`	`void *`
`%02x` `%5.2f` `%-8s` `%*d`	zero-pad, width.precision, left-align, width from argument
`PRIu32` `PRId64`	`<inttypes.h>` for `uint32_t`, `int64_t`

A mismatch is undefined; `uint8_t` promotes to `int`, so `%d` is correct for it.

Storage classes

Declaration	Scope	Lifetime	Linkage	Default
local (`auto`)	block	block	none	indeterminate
`static` local	block	program	none	0
global	file and others	program	external	0
`static` global or function	file	program	internal	0
`extern`	declares, no storage	n/a	external	n/a
`malloc` block	by pointer	until `free`	n/a	indeterminate

Qualifiers

Qualifier	Promise or effect	Does not mean
`const`	no writes through this name	not stored in ROM unless global
`volatile`	every access really happens, in order	not atomic, not a barrier
`restrict`	no other pointer to the same object	checked by the compiler
`_Atomic`	indivisible access, ordering	free of locks
`inline`	body may be substituted	guaranteed inlining
`register`	historical hint; address not allowed	faster today

Padding rule, endianness, macros

Offset of a member is the next multiple of its alignment; `sizeof` is a multiple of the largest alignment. `{char; int; char}` is 12; `{int; char; char}` is 8; `{char a[3]; short; int}` is 12. `packed` removes padding at the price of misaligned access. `htons` and `htonl` convert host to network order and are no-ops on big-endian machines. Safe macro: `#define MAX(a,b) ((a)>(b)?(a):(b))` still evaluates an argument twice.

Memory bug to tool

Symptom	Tool
leak	`valgrind --leak-check=full`, LeakSanitizer
overflow, use after free	`-fsanitize=address`
undefined behaviour	`-fsanitize=undefined`
crash, wrong value	`gdb`: `run`, `bt`, `print`, `watch`
warnings	`-Wall -Wextra -Werror`

STL: which container?

Use `vector` unless you can say why not. Need sorted order or range queries: `map` or `set` (O(log n)). Need fastest lookup and no order: `unordered_map` (O(1) average). Need stable iterators and cheap insertion in the middle: `list`. Need both ends: `deque`. Need the largest element repeatedly: `priority_queue`. Complexity table: 2.21 (p. 141).

Python quick reference

`struct` formats: `!` network order, `<` little, `B` 1 byte, `H` 2, `I` 4, `Q` 8, `Ns` N bytes; native alignment only without a prefix. `list`: append O(1) amortised, `insert(0, x)` O(n), `x in` O(n). `set` and `dict`: O(1) on average, keys hashable. `Counter.most_common(k)`, `defaultdict(list)`, `deque` for queues, `heapq.nlargest`. `subprocess.run([...], check=True, capture_output=True, text=True, timeout=t)`; never `shell=True` with data. `//` floors, `%` takes the divisor's sign.

Flashcards

Question	Answer
`sizeof(struct{char; int; short;})` on x86-64?	12
`sizeof` of `{short; long; char;}` on LP64 and ILP32?	24 and 12

Size of `union { int i; char c[5]; }`?	8
`const int *p`: what is constant?	the pointee
`int *const p`: what is constant?	the pointer
Where does a static local live?	data or BSS
Where does `"abc"` live?	rodata (read-only)
Why is BSS free in the file?	zero contents; only the size is stored
`0x0A0B0C0D` in little-endian memory?	`0D 0C 0B 0A`
Network byte order?	big-endian
Is `volatile` atomic?	no
Can a variable be `const volatile`?	yes: read-only hardware register
`sizeof(a)` for an array parameter?	pointer size, 8 on LP64
`strlen("abc")` and `sizeof("abc")`?	3 and 4
Result of `-1 < 1u`?	false
`uint8_t` 200 + 100?	300 (promoted to `int`)
`char c = 200` prints?	-56 where `char` is signed
`SQ(2+3)` with `x*x`?	11
Why `do { } while (0)` in macros?	one statement, safe after `if`
`STR(N)` versus `XSTR(N)`?	name versus expanded value
Signed overflow?	undefined
Unsigned overflow?	defined, wraps modulo 2^n
`1 << 31` on `int`?	undefined
`i = i++ + ++i;`?	undefined
`memcpy` versus `memmove`?	`memmove` handles overlap
Safe `realloc` pattern?	assign to a temporary first
`strncpy` guarantee?	none: may not terminate
`free` twice?	undefined; glibc aborts
Static array in a function: effect?	off the stack; not re-entrant
Offset of the header version nibble?	byte 0, high four bits
Valid IPv4 header checksum sum?	`0xFFFF` (zero after complement)
Fix for `for (unsigned i = n; i >= 0; i--)`?	`for (unsigned i = n; i-- > 0; )`
What does `container_of` compute?	enclosing struct from member pointer
`*p++` means?	`*(p++)`: read, then advance
Python default `acc=[]`: problem?	one list shared by all calls
`x in list` versus `x in set`?	O(n) versus O(1) average
`!` in a `struct` format?	network (big-endian) order
Why `subprocess` with a list?	no shell parsing, no injection
C++ vptr: how many per object?	one, pointing to the class vtable
Why virtual destructor?	`delete` via base runs derived destructor
`map` versus `unordered_map`?	O(log n) ordered versus O(1) average
Smart pointer for sole ownership?	`unique_ptr`
Break a `shared_ptr` cycle with?	`weak_ptr`

PART I · PILLAR 3

3

Computer Networks Fundamentals

Every packet tells a story; we learn to read it, layer by layer.

3.1 Networks, devices and delay	MUST
3.2 The OSI and TCP/IP models, and encapsulation	MUST
3.3 The data-link layer: Ethernet, switching, error control and ARQ	MUST
3.4 IPv4 addressing, subnetting, VLSM and CIDR	MUST
3.5 The IPv4 header, fragmentation, MTU and ICMP	MUST
3.6 ARP, DHCP and NAT: the glue protocols	MUST
3.7 IPv6 essentials	SHOULD
3.8 Routing concepts in one page	MUST
3.9 Transport: UDP, TCP, flow control and congestion control	MUST
3.10 The application layer: DNS, HTTP, TLS and QUIC	SHOULD
3.11 Capstone: what happens when you type google.com	MUST
3.12 Where the topics are asked, and a mini-project	OPTIONAL

3.1 Networks, devices and delay MUST · ■□□

A computer network is a set of devices that exchange data over shared links so that they can share resources: files, printers, an internet connection, a database. Every network question in an interview reduces to two things, namely *where the bits go* and *how long they take*, and this section supplies the vocabulary for both.

> BIG IDEA
>
> A network moves bits between devices, and splitting that job into independent layers is what allows each part to be replaced without disturbing the rest.

Sizes and shapes

Networks are named by their reach. A *LAN* (local area network) covers a room, a floor or a campus and is owned by one organisation. A *MAN* covers a city. A *WAN* (wide area network) joins sites across cities or countries and usually runs over links leased from a carrier. The *Internet* is the network of networks that joins all of them using one common protocol, IP.

The *topology* is the pattern in which devices are joined. In a *bus* all devices hang from one shared cable; in a *star* each hangs from a central device; in a *ring* each has two neighbours; in a *full mesh* every pair is joined directly. The last is robust but costs n(n − 1)/2 links for n devices, so ten sites already need 45 links and a hundred need 4,950. Hence real networks are stars of stars, with a few meshed links at the core.

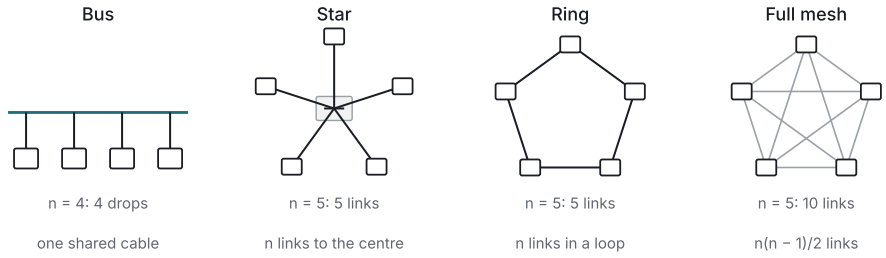


Figure 3.1 Four physical topologies and their link counts. A full mesh grows quadratically, which is why large networks are built as stars of stars.

Devices

Five boxes appear in almost every diagram, and an interviewer expects one sentence on each. A *hub* is a layer-1 repeater: every bit that enters a port leaves all the other ports, so all attached devices share one wire. A *switch* learns which device sits behind which port and forwards a frame only to the port concerned. A *router* joins different networks and forwards packets by their IP destination. A *layer-3 switch* is a switch with routing built into its hardware, which is how campus networks route between VLANs at line rate. A *firewall* permits or drops traffic according to rules, and an *access point* (AP) bridges Wi-Fi radio to the wired network.

Two terms follow from these devices. A *collision domain* is the set of ports whose transmissions can collide on a shared medium; a hub extends it, and every switch port ends it. A *broadcast domain* is the set of devices that receive a frame sent to the broadcast address; a switch extends it, and only a router ends it. Note that modern switched Ethernet is full-duplex, so collisions no longer occur in practice, yet the counting question remains a standard interview exercise.

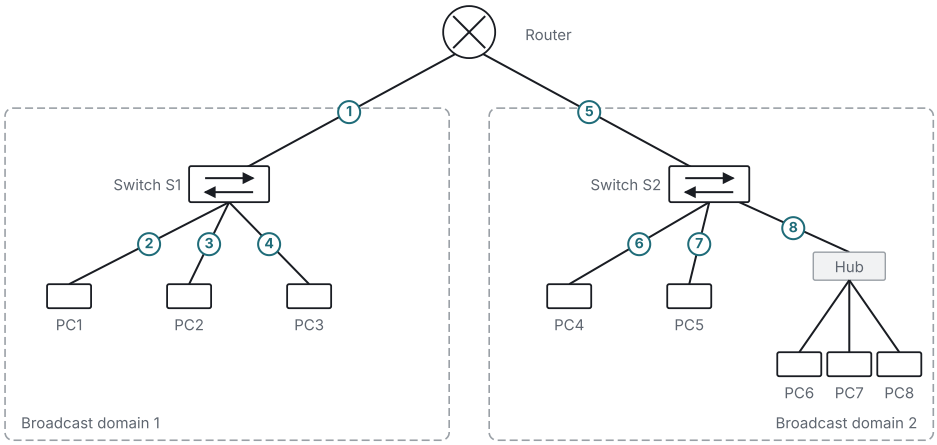


Figure 3.2 Counting domains. The router does not forward broadcasts, so there are two broadcast domains (dashed). Every switch port is its own collision domain, while the hub and PC6 to PC8 share one wire. Link 8 is therefore one domain, and the total is 8 collision domains, numbered on the links.

WORKED EXAMPLE

Count the domains in the figure.

1. A router interface never forwards broadcasts, so each of its two sides is a separate broadcast domain: S1 with PC1–PC3, and S2 with PC4–PC5, the hub and PC6–PC8. That gives 2.
2. Each switch port is a separate collision domain. S1 has four ports in use (router, PC1, PC2, PC3), giving 4.
3. S2 has four ports in use (router, PC4, PC5, hub). The hub and PC6–PC8 share one wire and so form a single domain behind S2's hub port, giving 4.

Result: 2 broadcast domains and 8 collision domains.

Circuit and packet switching

A telephone network of the old kind used *circuit switching*: a path was reserved from end to end for the whole call, whether or not anyone was speaking. The Internet uses *packet switching*: the data is cut into packets, each carrying its destination address, and routers forward them independently. Packet switching wastes nothing on silence and shares links statistically, at the price that packets can queue, be delayed or be lost. Every later topic in this pillar is a mechanism for living with that price.

Four kinds of delay

A packet travelling from host to host accumulates delay at every hop from four sources. *Processing delay* is the time a router takes to read the header and choose the output port; it is microseconds. *Queueing delay* is the time spent waiting for the output link to become free, and it varies with load; it is the cause of jitter, the variation of delay from one packet to the next. *Transmission delay* is the time to push all the bits of the packet onto the link, L/R for a packet of L bits on a link of R bit/s. *Propagation delay* is the time a bit takes to travel the length of the link, d/v, where v is about 2×10^8 m/s in copper or fibre, two-thirds of the speed of light in vacuum.

Bandwidth vs throughput — bandwidth is what the link could carry, in bits per second; throughput is what you actually got. A motorway has six lanes (bandwidth) but you still arrive late in a jam.

Transmission delay depends on packet size and link speed; propagation delay depends only on distance. Confusing the two is the commonest error in this topic.

WORKED EXAMPLE

Send a 1 MB file over a 2,000 km link. Take 1 MB as 8×10^6 bits and ignore queueing and headers.

1. At 10 Mb/s: transmission = $8 \times 10^6 / 10^7 = 0.8$ s.
2. Propagation = 2×10^6 m / 2×10^8 m/s = 0.010 s = 10 ms, whatever the speed of the link.
3. At 10 Mb/s the total is $0.8 + 0.01 \approx 0.81$ s.
4. At 1 Gb/s: transmission = $8 \times 10^6 / 10^9 = 8$ ms, so the total is 8 + 10 = 18 ms.

Result: 0.81 s and 18 ms. Once the link is fast, distance, not bandwidth, sets the floor on latency; no upgrade can beat 10 ms for these 2,000 km.

TRAP

Claim: "A faster link makes every transfer faster by the same factor." *Why it is wrong:* only transmission delay shrinks with bandwidth; propagation delay is fixed by distance and the speed of light in the medium. *Correct statement:* a hundred-fold faster link cuts a large transfer a hundred-fold, but a small request across an ocean barely improves.

INTERVIEW LENS

CISCO · NOKIA

Asked as "hub versus switch versus router", the interviewer wants collision and broadcast domains stated per device, then one scenario, such as why a loop of hubs is harmless but a loop of switches is not (a topic for Pillar 4). A strong answer draws the figure above and counts, without being asked. See Q3.2 in the Problem Set (p. 212).

LAB

`LAB 3.1 · 15 MIN` Cisco Packet Tracer. Build two small networks: four PCs on a hub, and four PCs on a switch. Switch to *Simulation* mode, filter on ICMP, and send one ping from PC0 to PC1 in each network.

What you should notice: on the hub, the event list shows the echo request copied to every other port and the request is seen by PC2 and PC3, which discard it. On the switch, the first request is flooded (the switch has not yet learned PC1), but the reply, and every later ping, travels only between the two ports concerned.

CHECK YOURSELF

1. How many links does a full mesh of 12 routers need? [1]
2. A frame is sent from a PC to another PC in the same VLAN through two switches. How many collision domains does it cross? [2]
3. Which of the four delays grows when the network is congested? [3]

3.2 The OSI and TCP/IP models, and encapsulation MUST · ■□□

Sending a web page involves cables, addresses, reliability, encryption and a document format, and no single protocol does all of this. The standard answer is to divide the work into *layers*, each offering a service to the layer above and using the service of the layer below. A layer on one machine is concerned only with its *peer* layer on the other machine, which it talks to through a header that it adds to the data.

BIG IDEA

> Each layer wraps the data from above in its own header so that its peer layer at the far end can act on it, and this wrapping is why a change at one layer does not disturb the others.

Two models

The *OSI model* (Open Systems Interconnection) has seven layers and is used as vocabulary: engineers say “a layer-2 problem” or “a layer-7 load balancer”. The *TCP/IP model* describes what is actually deployed and has four layers, in which the top three OSI layers merge into one application layer and the bottom two into one link layer. Textbooks that prefer five layers split the link layer again, giving the five-layer model used in this book from now on.

OSI mnemonic — layers 7 to 1: *All People Seem To Need Data Processing* (Application, Presentation, Session, Transport, Network, Data link, Physical). Reverse it for 1 to 7: *Please Do Not Throw Sausage Pizza Away*.

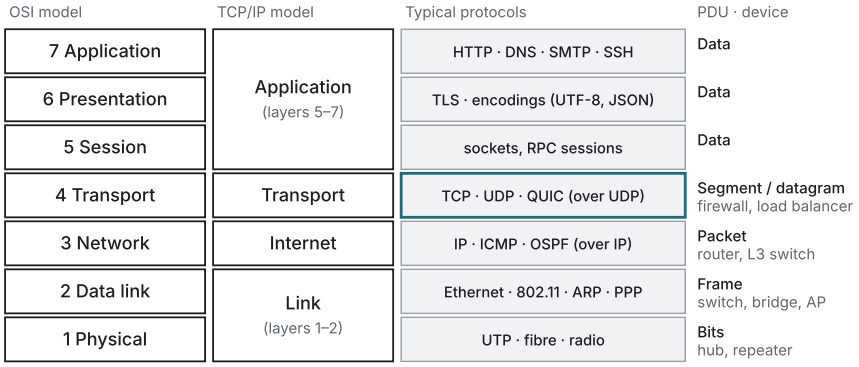


Figure 3.3 The seven-layer OSI model beside the four-layer TCP/IP model. The highlighted row is where TCP and UDP live; the stack below it is the network, the stack above it is the application.

Each layer has a name for its unit of data, the *protocol data unit* (PDU): data at the application layer, a *segment* (TCP) or *datagram* (UDP) at the transport layer, a *packet* at the network layer, a *frame* at the data-link layer, and *bits* on the wire. Interviewers notice when these names are used loosely.

Protocols do not always sit tidily. ARP is carried directly in Ethernet frames yet resolves IP addresses, so it is often called a “layer 2.5” protocol. ICMP is a layer-3 protocol, even though it travels inside IP packets. TLS lives between the transport and application layers and has no home in the TCP/IP model. QUIC runs over UDP but provides what TCP provides, plus encryption. Hence the question “which layer is X?” is best answered with “it is carried in Y and serves Z”.

Encapsulation

When an application writes data to a socket, the transport layer prepends a TCP header, giving a segment; the network layer prepends an IP header, giving a packet; the link layer prepends an Ethernet header and appends a trailer, the *frame check sequence* (FCS), giving a frame. This is *encapsulation*; the receiver strips the headers in reverse order, termed *decapsulation*. A layer knows which protocol is above it through a *demultiplexing key* in its header: the EtherType field in Ethernet (0x0800 for IPv4, 0x86DD for IPv6, 0x0806 for ARP), the protocol field in IP (6 for TCP, 17 for UDP, 1 for ICMP), and the destination port in TCP and UDP.

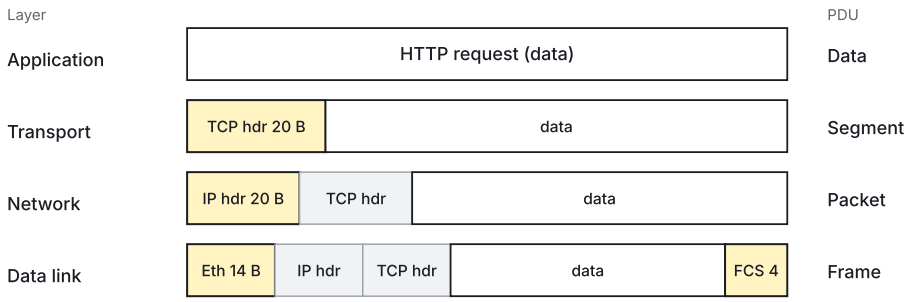


Figure 3.4 Encapsulation. Each layer wraps the data it receives from above in its own header (highlighted). Only the data-link layer also adds a trailer, the frame check sequence (FCS). The receiver reverses the process, which is termed decapsulation.

The headers cost bandwidth, and an interviewer may ask by how much. We measure it below.

WORKED EXAMPLE

Overhead of an HTTP request carrying 100 bytes of data, and of a full-size segment.

1. Header bytes: TCP 20 + IPv4 20 + Ethernet 14 + FCS 4 = 58 bytes.
2. On the wire there is also an 8-byte preamble and start delimiter and a 12-byte inter-frame gap, which together occupy the medium for 20 bytes. The total per frame is 100 + 58 + 20 = 178 byte-times.
3. Efficiency = 100 / 178 ≈ 56%.
4. With 1,460 bytes of data (the largest that fits a 1,500-byte IP packet), the total is 1,460 + 78 = 1,538 byte-times, and the efficiency is 1,460 / 1,538 ≈ 94.9%.

Result: 56% for the small request and 94.9% for the full-size segment. Small packets waste capacity, which is why bulk transfers use the largest segment the path allows.

What changes where

The stack diagram below shows a packet crossing one router. The hosts build and read all layers, but the router climbs only to layer 3: it removes the incoming frame, decrements the IP time-to-live, looks up the destination, and builds a new frame for the next link. The layer-4 segment inside is never read. Hence the rule that interviewers love, which is stated in Section 3.6 with numbers: *the MAC addresses change at every hop, whereas the IP addresses do not (unless NAT intervenes).*

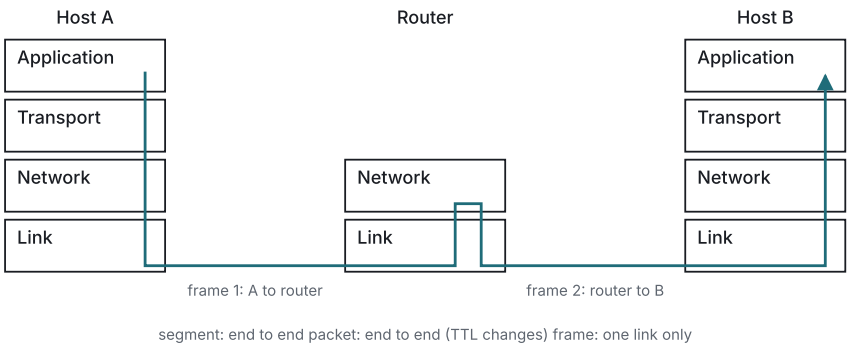


Figure 3.5 A packet crossing one router. The segment is built and read only at the two hosts; the router climbs just to layer 3, then rewrites the frame for the next link.

TRAP

Claim: "A router decrypts and reads the TCP header, which is how it knows the port." *Why it is wrong:* an ordinary router forwards on the IP destination alone and never touches the segment. *Correct statement:* firewalls, NAT devices and load balancers do read layer 4, and are therefore described as layer-4 devices; a plain router is a layer-3 device.

INTERVIEW LENS

CISCO · NOKIA · AMAZON · MICROSOFT · JUNIPER

The question is "name the layers with a protocol and a device each" (Q3.1) or, harder, "which layer does a load balancer work at?". A strong answer gives both L4 (TCP connections, as for a network load balancer) and L7 (HTTP headers, as for an application load balancer), and notes the cost of each.

LAB

`LAB 3.2 · 20 MIN` Wireshark. Run `curl -s http://example.com -o /dev/null` while capturing, stop, and apply the display filter `http`. Select the request and expand the packet-details pane.

```
Frame 14: 142 bytes on wire
Ethernet II, Src: 52:54:00:12:34:56, Dst: 52:54:00:12:35:02   (type 0x0800)
Internet Protocol Version 4, Src: 10.0.2.15, Dst: 93.184.216.34  (proto 6)
Transmission Control Protocol, Src Port: 49822, Dst Port: 80, Len: 76
Hypertext Transfer Protocol: GET / HTTP/1.1
```

What you should notice: the four headings are the four layers of Figure 3.4 from the bottom upward; the EtherType, the IP protocol number and the destination port are the three demultiplexing keys. The headers total 66 bytes of the 142 captured: 14 (Ethernet), 20 (IPv4) and 32 (TCP, 20 plus 12 bytes of timestamp options), leaving 76 bytes of HTTP.

CHECK YOURSELF

1. At which layer does a switch operate, and which address does it read? [4]
2. Why is ARP called "layer 2.5"? [5]
3. A frame carries 14 + 20 + 20 header bytes and 1,000 bytes of data. What fraction of the frame (ignoring FCS and preamble) is data? [6]

3.3 The data-link layer: Ethernet, switching, error control and ARQ

MUST · ■■□

The data-link layer delivers frames across a single link, between two devices that are directly connected. It must decide where one frame ends and the next begins (*framing*), which neighbour a frame is for (*addressing*), whether the frame was damaged (*error detection*) and, on some links, how to repair damage (*ARQ*, automatic repeat request). Ethernet, which dominates wired networks, answers the first three; the fourth is done mostly by higher layers, but it is the standard exam topic for sliding-window protocols and reappears in TCP.

BIG IDEA

Layer 2 gets a frame across one link: the MAC address picks the receiver, the CRC detects damage, and a switch forwards by learning where each address lives.

1. 12 × 11 / 2 = 66.↩
2. Three: PC to switch 1, switch 1 to switch 2, switch 2 to PC. Each link is its own domain.↩
3. Queueing delay, because packets wait for the output link; it is also the source of jitter.↩
4. Layer 2; the destination MAC address.↩
5. It is carried in an Ethernet frame (layer 2) but it maps layer-3 addresses to layer-2 addresses.↩
6. 1,000 / 1,054 ≈ 94.9%.↩

The physical layer in one page

Bits travel as electrical signals over copper, as light over fibre or as radio waves. *Unshielded twisted pair* (UTP) is the cable in offices: Cat5e carries 1 Gb/s and Cat6a carries 10 Gb/s to 100 m. *Fibre* carries more over longer distances: *multimode* (MMF) reaches a few hundred metres inside a building, and *single-mode* (SMF) reaches tens of kilometres. Transceivers plug into the switch as SFP (1 Gb/s), SFP+ (10), SFP28 (25) and QSFP (40 to 400).

Two ends of a link must agree on speed and *duplex*. Full duplex means both ends transmit at once; half duplex means they take turns. Auto-negotiation normally settles both, but if one end is forced to full duplex and the other is left on auto, the auto end falls back to half duplex. This *duplex mismatch* is a classic fault: the link comes up and pings work, but throughput collapses under load, the half-duplex side reports late collisions and the full-duplex side reports CRC errors and runts.

Ethernet frames and MAC addresses

Every Ethernet port has a 48-bit *MAC address*, written as six hexadecimal bytes. The first three bytes form the *OUI*, assigned to the manufacturer; the last three are chosen by the manufacturer. Two bits of the first byte carry meaning: the lowest bit (I/G) marks a group address, so any address with an odd first byte, including `ff:ff:ff:ff:ff:ff`, is multicast or broadcast; the next bit (U/L) marks an address that an administrator has set instead of the hardware default.

Unicast, multicast, broadcast — one receiver, a group of receivers, everyone on the segment. Broadcast is the all-ones address.

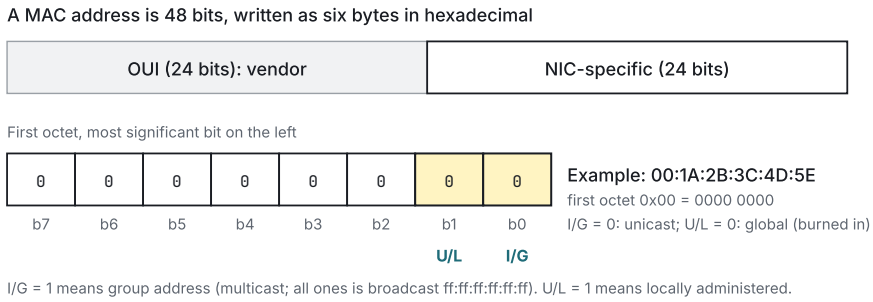


Figure 3.6 MAC address structure. The two low bits of the first octet are flags; the U/L bit is the one flipped when an IPv6 interface ID is built from a MAC (Section 3.7).

The frame format that nearly all networks use is *Ethernet II*. The destination address comes first so that a switch can start forwarding before the rest has arrived. The 2-byte *type* field says what the payload is (0x0800 for IPv4, for example). The payload must be at least 46 bytes, and short payloads are padded, for a reason derived below. The trailer is the 4-byte FCS, a CRC-32 computed over the frame. The *maximum transmission unit* (MTU), the largest payload, is 1,500 bytes.

A *VLAN tag* of 4 bytes, defined in 802.1Q, may be inserted after the source address. It starts with the value 0x8100 (the tag protocol identifier), followed by a 3-bit priority, a drop-eligible bit and a 12-bit VLAN identifier, so up to 4,094 usable VLANs exist. VLANs themselves are treated in Pillar 4.

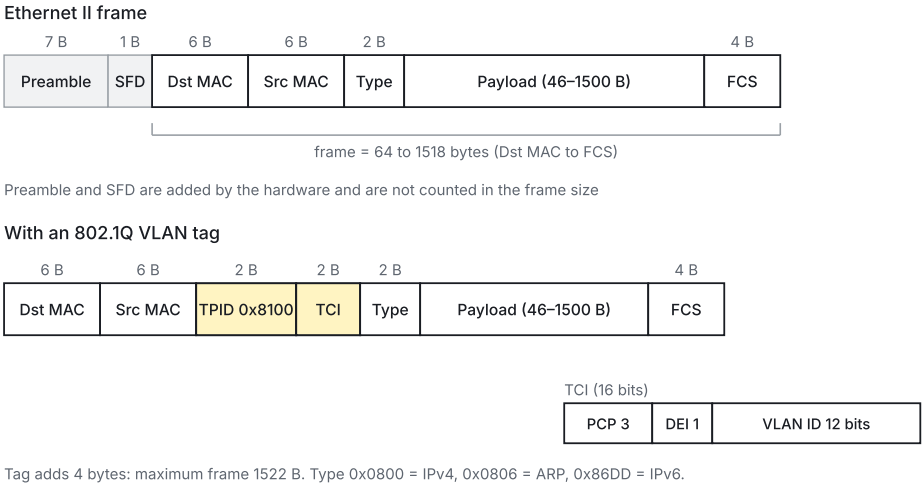


Figure 3.7 Byte-accurate Ethernet II frame (top) and the same frame with an 802.1Q tag (below). The minimum of 64 bytes is the reason for padding short payloads up to 46 bytes.

CSMA/CD and the 64-byte minimum

On the original shared cable, any station could transmit when it heard silence, and two stations could start together. *CSMA/CD* (carrier sense multiple access with collision detection) handles this: a station listens first, transmits if the cable is idle, keeps listening while transmitting, and on hearing a collision stops, sends a short jam signal and waits a random time that grows after each repeated collision (*binary exponential backoff*). For the sender to detect a collision it must still be transmitting when the damaged signal returns from the far end, which sets a lower limit on the frame length.

> WORKED EXAMPLE
>
> **Why is the minimum frame 64 bytes?** A 10 Mb/s Ethernet spans at most 2,500 m, and signals travel at 2×10^8 m/s.
>
> 1. One-way propagation: $T_p = 2{,}500 / (2 \times 10^8) = 12.5$ µs.
> 2. In the worst case, the colliding signal from the far station reaches the sender after one more trip: the sender must still transmit for $2T_p = 25$ µs.
> 3. Bits transmitted in 25 µs at 10 Mb/s: $25 \times 10^{-6} \times 10 \times 10^6 = 250$ bits.
> 4. The standard rounds up to 512 bits (64 bytes, a 51.2 µs slot time) to allow for repeater delays, the jam signal and a safety margin.
>
> **Result: the minimum frame must satisfy $L \geq 2T_p \times R$, giving 250 bits here and 512 bits in the standard. At 1 Gb/s the same rule would demand a much longer frame, so gigabit half-duplex uses carrier extension; full-duplex links have no collisions at all.**

Switching

A *switch* builds a *MAC address table* (also called the CAM table, after the content-addressable memory that stores it) by learning. For each arriving frame it does three things. It reads the source address and records “this address is behind this port”. It then looks up the destination: if the destination is known, the frame is *forwarded* to that port only; if the destination port is the arrival port, it is *filtered* (dropped); if the destination is broadcast, multicast or unknown, the frame is *flooded* out of every port except the arrival port. Entries are removed after an *ageing time* (300 s on Cisco switches), so a device that is moved is relearned.

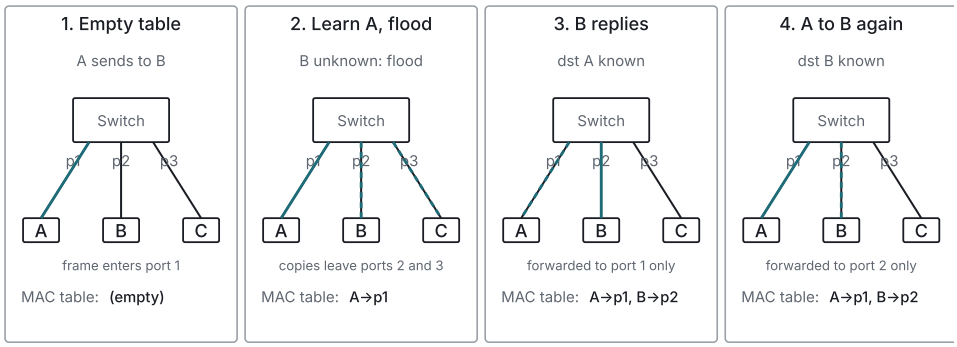


Figure 3.8 A switch learns by reading the source address of every frame and forwards by looking up the destination. Solid accent lines: the frame entering; dashed accent lines: where it leaves.

Flooding an unknown unicast frame is not the same as a broadcast: the frame is still addressed to one MAC, and the receivers other than that one discard it at layer 2. A switch does, however, treat both the same way.

If the table fills, a switch cannot learn and floods every frame for unknown addresses, so it behaves like a hub. An attacker who sends frames with thousands of invented source addresses exploits exactly this to see other people's traffic (*MAC flooding*; mitigation in Pillar 9).

Switches also differ in when they start forwarding. A *store-and-forward* switch receives the whole frame, checks the FCS and then forwards; it adds a delay of one frame time (1,500 bytes take 12 µs at 1 Gb/s and 1.2 µs at 10 Gb/s) but never forwards a corrupted frame. A *cut-through* switch begins forwarding as soon as it has read the destination address, after the 6-byte destination address, so its latency is tens of nanoseconds and independent of frame size, at the price of forwarding damaged frames, which the receiver then discards. Data-centre switches offer cut-through because latency, not error filtering, matters most there.

Framing and error detection

The bits of a frame must be delimited. Ethernet uses a preamble and start delimiter; serial links often use a flag byte. If the flag could appear inside the data, it must be hidden by *stuffing*. In *bit stuffing* (HDLC) the flag is `01111110`, and the sender inserts a 0 after every run of five 1s in the data, which the receiver removes; the data `01111110` is sent as `011111010`. In *byte stuffing* an escape byte is inserted before any flag or escape byte inside the data.

Errors are detected by adding redundancy. A *parity bit* makes the number of 1s even (or odd) and detects any odd number of flipped bits, but misses even numbers. A *checksum* adds the data in words, as the IP header checksum does (Section 3.5). A *cyclic redundancy check* (CRC) treats the data as a polynomial over arithmetic modulo 2 and divides it by a fixed *generator* polynomial; the remainder is the check value. It detects all burst errors no longer than the degree of the generator and almost all longer ones, which is why Ethernet uses the 32-bit generator of CRC-32.

WORKED EXAMPLE

CRC for data 1101011011 with generator 10011 ($x^4 + x + 1$).

1. The generator has 5 bits, so its degree is 4: append four zeros to the data, giving 11010110110000.
2. Divide modulo 2. Wherever the leading bit is 1, XOR the generator under it, aligned to that bit; there is no borrowing. The register after each XOR is shown below.

3. The last four bits are the remainder: 1110. Replace the appended zeros with it and transmit 11010110111110.
4. The receiver divides the whole received word by the same generator. If no bit is damaged the remainder is 0000.

```
start       11010110110000
xor at  0   10011000000000
result      01001110110000
xor at  1   01001100000000
result      00000010110000
xor at  6   00000010011000
result      00000000101000
xor at  8   00000000100110
result      00000000001110      remainder = 1110
```

Result: frame 11010110111110; the receiver's check gives remainder 0. Addition and subtraction are both XOR, which is why the sender can simply replace the zeros with the remainder.

CRC only detects errors. *Hamming codes* correct them. In a Hamming(7,4) code, four data bits are protected by three parity bits placed at positions 1, 2 and 4 of a 7-bit word, each covering the positions whose binary index contains its bit: p1 covers 1, 3, 5, 7; p2 covers 2, 3, 6, 7; p4 covers 4, 5, 6, 7. After a single-bit error the three parity checks, read as a binary number (the *syndrome*), name the position of the faulty bit.

WORKED EXAMPLE

Hamming(7,4) with even parity for data 1011.

1. Place the data at positions 3, 5, 6, 7: d3 = 1, d5 = 0, d6 = 1, d7 = 1.
2. p1 = d3 xor d5 xor d7 = 1 xor 0 xor 1 = 0. p2 = d3 xor d6 xor d7 = 1 xor 1 xor 1 = 1. p4 = d5 xor d6 xor d7 = 0 xor 1 xor 1 = 0.
3. The codeword, positions 1 to 7, is 0110011.
4. Suppose position 5 flips in transit, giving 0110111. The p1 group (1, 3, 5, 7) now sums to an odd number, so c1 = 1; the p2 group (2, 3, 6, 7) is even, so c2 = 0; the p4 group (4, 5, 6, 7) is odd, so c4 = 1.
5. The syndrome c4c2c1 = 101 = 5, so flip position 5 back.

Result: syndrome 5 locates the error, and the corrected word is 0110011.

Reliable delivery on a link: ARQ

Some links, such as Wi-Fi or satellite, must repair losses themselves; the same ideas reappear in TCP (Section 3.9). In *Stop-and-Wait* the sender transmits one frame and waits for its acknowledgement; a timer expires if none arrives, and the frame is resent. It is simple but wasteful on long links. *Go-Back-N* (GBN) lets the sender have a *window* of N unacknowledged frames; the receiver accepts only the next expected frame, discards the rest and repeats the last acknowledgement, and on timeout the sender resends the lost frame and everything after it. *Selective Repeat* (SR) has the receiver buffer out-of-order frames and acknowledge each, so that only the lost frame is resent, at the price of buffers and more complex bookkeeping.

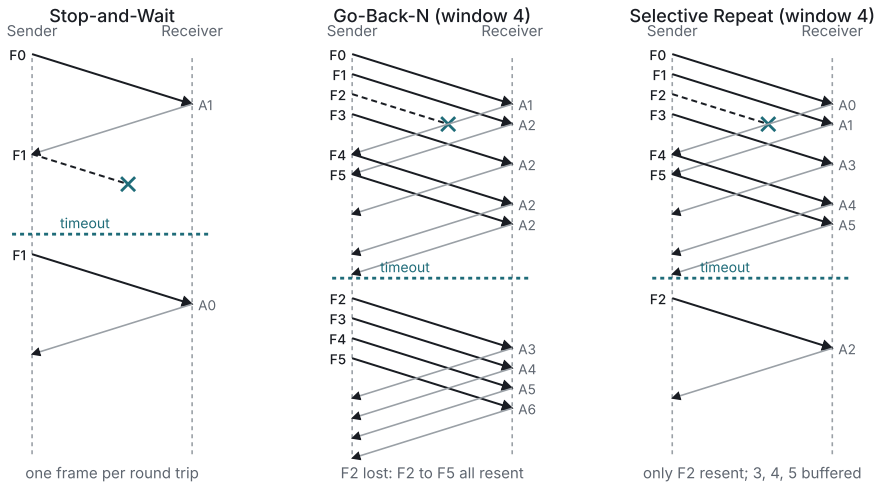


Figure 3.9 Three ARQ schemes after the loss of frame 2. Acknowledgements are shown by the next frame expected (cumulative) in Stop-and-Wait and Go-Back-N, and per frame (A3 means frame 3 received) in Selective Repeat. Go-Back-N discards out-of-order frames and repeats A2; Selective Repeat buffers them.

Let $a = T_p / T_t$ be the ratio of the one-way propagation time to the frame transmission time. The sender can send for T_t, then idles for a round trip of $2T_p$; a window that covers the whole $1 + 2a$ frame times keeps the link busy. Sequence numbers are written in n bits, so they wrap modulo 2^n, and the window must be small enough that an old frame is never mistaken for a new one: in GBN the window may be at most $2^n - 1$, whereas in SR it may be at most 2^{n-1}, because the sender's and receiver's windows must not overlap.

WORKED EXAMPLE

A 1 Mb/s link carries 1,000-bit frames, and the one-way propagation delay is 20 ms.

1. $T_t = 1{,}000 / 10^6 = 1$ ms, so $a = 20$ ms $/ 1$ ms $= 20$.
2. Stop-and-Wait efficiency: $\eta = 1 / (1 + 2a) = 1 / 41 \approx 2.4\%$. The link carries one frame in every 41 ms.
3. A window of $1 + 2a = 41$ frames gives 100% (and any window $W < 41$ gives $\eta = W / 41$).
4. Sequence numbers for $W = 41$: GBN needs $2^n - 1 \geq 41$, so $2^n \geq 42$ and $n = 6$. SR needs $2^{n-1} \geq 41$, so $n - 1 \geq 6$ and $n = 7$.

Result: 2.4%; a window of 41 gives full utilisation; 6 bits for GBN and 7 bits for SR.

TRAP

Claim: "A CRC corrects errors, and a longer CRC corrects more." *Why it is wrong:* a CRC is a detection code; the receiver can only tell that a frame is bad. *Correct statement:* Ethernet drops a frame whose FCS fails and leaves recovery to the higher layers (TCP resends); correction is the job of codes such as Hamming or the forward error correction in Wi-Fi and cellular.

INTERVIEW LENS

QUALCOMM · BROADCOM · NOKIA · ARISTA · CISCO

Qualcomm, Broadcom and Nokia interviews like the numerical problems: the CRC, the window limits and Stop-and-Wait efficiency (Q3.6 to Q3.8). Cisco and Arista ask about learning, flooding and the full CAM table (Q3.5), and Arista adds cut-through (Q3.9). A strong answer states the formula before the arithmetic and checks the units.

LAB

`LAB 3.3 · 25 MIN` Three small tasks. (i) In Packet Tracer, connect three PCs to one switch, run `show mac address-table` before and after a ping from PC0 to PC1, then `clear mac address-table dynamic`. (ii) In Wireshark, apply `eth.dst == ff:ff:ff:ff:ff:ff` and list the protocols that appear. (iii) In Python, check CRC-32.

```
Switch# show mac address-table dynamic
 Vlan    Mac Address       Type        Ports
    1    0001.6446.e5a2    DYNAMIC     Fa0/1
    1    0003.e4a1.b3c4    DYNAMIC     Fa0/2
```

```
import zlib
print(hex(zlib.crc32(b"123456789")))   # 0xcbf43926, the standard check value
```

What you should notice: before the ping the table is empty, and after it holds exactly the source addresses seen, one per port. The broadcast filter shows ARP and DHCP; broadcast frames are never sources, so they never appear in the table.

CHECK YOURSELF

1. A host sends a frame to an address that is not in the switch's table. What does the switch do, and what does it learn? [1]
2. Which Ethernet field makes the receiver pass the payload to IPv4 rather than ARP? [2]
3. For Stop-and-Wait, what happens to the efficiency if the propagation delay is doubled? [3]

3.4 IPv4 addressing, subnetting, VLSM and CIDR MUST · ■■□

An IPv4 address is a 32-bit number that identifies a network interface and tells a router where to send packets for it. It is written as four decimal numbers, one per byte (an *octet*), separated by dots: 172.16.45.200. Subnetting, the art of dividing an address block into smaller networks, is the most frequently examined skill in this book, and it is a mechanical one that anybody can perform in under a minute after a short drill.

BIG IDEA

The mask splits a 32-bit address into a network part and a host part, and subnetting is nothing more than moving that split to fit the number of hosts needed.

Network part, host part, mask

Every address has two parts: the *network* part, shared by all devices in the same subnet, and the *host* part, which distinguishes devices inside it. The *subnet mask* marks the split with ones over the network bits and zeros over the host bits. The *prefix length* is the count of ones, so 255.255.248.0 is written /21 (eight plus eight plus five ones). Two addresses are on the same subnet exactly when they agree on all network bits, which a host tests by bitwise AND with its own mask. In every subnet two addresses are special: the *network address* (host bits all zero) names the subnet itself, and the *broadcast address* (host bits all one) reaches every host in it. Hence a subnet with h host bits holds 2^h addresses and $2^h - 2$ usable hosts.

1. It floods the frame out of every port except the arrival port, and it learns the *source* address on the arrival port.↩
2. The EtherType field: 0x0800 for IPv4, 0x0806 for ARP.↩
3. a doubles, so η = 1 / (1 + 2a) roughly halves when a is large (1/41 becomes 1/81).↩

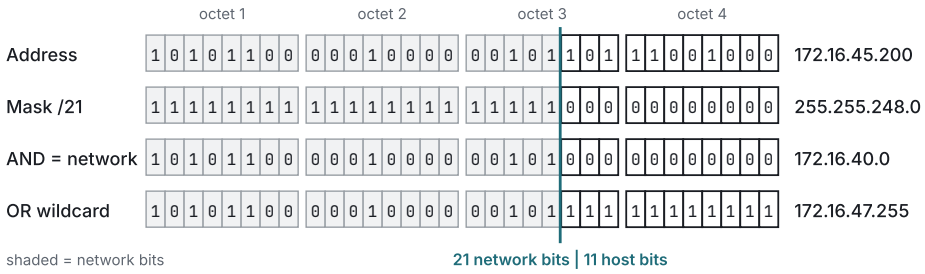


Figure 3.10 The 32-bit ruler for 172.16.45.200/21. The mask line falls inside the third octet, between bit 21 and bit 22; everything left of it is the network, everything right of it is the host.

WORKED EXAMPLE

Network, broadcast, range and hosts for 172.16.45.200/21, by the block-size method.

1. Mask: 21 ones is 255.255.248.0. The interesting octet is the third, where the mask is 248.
2. Block size = 256 − 248 = 8. Subnets in the third octet start at multiples of 8: 0, 8, … 40, 48.
3. The address has 45 in the third octet, and 40 ≤ 45 < 48, so the network address is 172.16.40.0.
4. The broadcast address is one below the next network: 172.16.47.255.
5. Usable range: 172.16.40.1 to 172.16.47.254. Host bits = 32 − 21 = 11, so hosts = $2^{11} - 2$ = 2,046.
6. Wildcard mask (the mask inverted, used in OSPF and ACLs) = 0.0.7.255.

Result: network 172.16.40.0, broadcast 172.16.47.255, 2,046 usable hosts, wildcard 0.0.7.255.

The master table below is worth memorising for the rows from /24 to /30, and the rest follow by the doubling rule: each bit added to the prefix halves the block.

Block-size trick — in the interesting octet, block = 256 − mask octet. List the multiples of the block; the address sits in one of them. Prefix /25 to /30 gives blocks 128, 64, 32, 16, 8, 4 in the fourth octet, and /17 to /23 gives the same series in the third.

Prefix	Mask	Wildcard	Block size (octet)	Addresses	Usable hosts
/8	255.0.0.0	0.255.255.255	1 (octet 1)	16,777,216	16,777,214
/12	255.240.0.0	0.15.255.255	16 (octet 2)	1,048,576	1,048,574
/16	255.255.0.0	0.0.255.255	1 (octet 2)	65,536	65,534
/17	255.255.128.0	0.0.127.255	128 (octet 3)	32,768	32,766
/18	255.255.192.0	0.0.63.255	64 (octet 3)	16,384	16,382
/19	255.255.224.0	0.0.31.255	32 (octet 3)	8,192	8,190
/20	255.255.240.0	0.0.15.255	16 (octet 3)	4,096	4,094
/21	255.255.248.0	0.0.7.255	8 (octet 3)	2,048	2,046
/22	255.255.252.0	0.0.3.255	4 (octet 3)	1,024	1,022
/23	255.255.254.0	0.0.1.255	2 (octet 3)	512	510
/24	255.255.255.0	0.0.0.255	1 (octet 3)	256	254
/25	255.255.255.128	0.0.0.127	128 (octet 4)	128	126
/26	255.255.255.192	0.0.0.63	64 (octet 4)	64	62
/27	255.255.255.224	0.0.0.31	32 (octet 4)	32	30

/28	255.255.255.240	0.0.0.15	16 (octet 4)	16	14
/29	255.255.255.248	0.0.0.7	8 (octet 4)	8	6
/30	255.255.255.252	0.0.0.3	4 (octet 4)	4	2
/31	255.255.255.254	0.0.0.1	2 (octet 4)	2	2 (RFC 3021)
/32	255.255.255.255	0.0.0.0	1 (octet 4)	1	1 (host route)

Classes, private ranges and special addresses

Originally the first bits of an address fixed its size: class A (first octet 1 to 126, /8), class B (128 to 191, /16), class C (192 to 223, /24), class D (224 to 239, multicast) and class E (240 to 255, reserved). The scheme wasted addresses, because an organisation needing 300 hosts had to take a class B with 65,534, and it bloated the routing tables. It was replaced in 1993 by *CIDR* (classless inter-domain routing), in which the prefix length is carried with the address and may be any value. The classes survive only in vocabulary and in some default behaviour of old protocols.

The special ranges below appear in many interview questions.

Range	Meaning
10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16	Private addresses (RFC 1918): not routed on the Internet, reused in every home and company
127.0.0.0/8	Loopback; 127.0.0.1 is the machine itself
169.254.0.0/16	Link-local (APIPA): self-assigned when DHCP fails
100.64.0.0/10	Carrier-grade NAT (shared address space)
0.0.0.0	“This host” at start-up, and the default route as 0.0.0.0/0
255.255.255.255	Limited broadcast, never forwarded by a router
224.0.0.0/4	Multicast
192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24	Reserved for documentation; used in this book’s examples

Two prefixes break the rule “usable hosts = $2^h - 2$”. A /31 has two addresses and, by RFC 3021, both are usable on a point-to-point link, because such a link has no need of a network or broadcast address; this saves half of the addresses on every router-to-router link. A /32 names a single host or interface, and appears as a loopback address or a *host route*.

Splitting and VLSM

Fixed-length subnetting cuts a block into equal subnets. To obtain at least s subnets, borrow b host bits with $2^b \geq s$; the prefix grows by b.

> WORKED EXAMPLE
>
> **Divide 192.168.10.0/24 into six subnets.**
>
> 1. Six subnets need $2^b \geq 6$, so b = 3 (eight subnets). New prefix: 24 + 3 = /27, mask 255.255.255.224.
> 2. Block size = 256 − 224 = 32. Host bits = 5, so $2^5 - 2 = 30$ hosts per subnet.
> 3. The subnets start at .0, .32, .64, .96, .128, .160, .192, .224. For the second, network 192.168.10.32, usable .33 to .62, broadcast .63.

Result: eight /27 subnets of 30 hosts, block size 32; two are left over for growth.

Equal subnets waste addresses when the needs differ. *Variable-length subnet masking* (VLSM) gives each subnet the prefix it needs. The rule is to sort requirements from largest to smallest, allocate the largest first, and place each subnet on a multiple of its own block size so that blocks never straddle.

WORKED EXAMPLE

VLSM from 192.168.10.0/24 for LANs of 100, 50 and 25 hosts and two point-to-point WAN links.

1. 100 hosts: $2^7 - 2 = 126 \geq 100$, so /25. Allocate 192.168.10.0/25: usable .1 to .126, broadcast .127.
2. 50 hosts: $2^6 - 2 = 62 \geq 50$, so /26. Next free address .128: 192.168.10.128/26, usable .129 to .190, broadcast .191.
3. 25 hosts: $2^5 - 2 = 30 \geq 25$, so /27. Next free .192: 192.168.10.192/27, usable .193 to .222, broadcast .223.
4. Each WAN needs 2 addresses: /30. 192.168.10.224/30 (usable .225, .226) and 192.168.10.228/30 (usable .229, .230).
5. What remains is .232 to .255, which is 192.168.10.232/29 and 192.168.10.240/28, both spare.

Result: /25, /26, /27, /30, /30 with 24 addresses spare; allocating the small ones first would have forced the /25 to start at .128 and the plan would not fit.

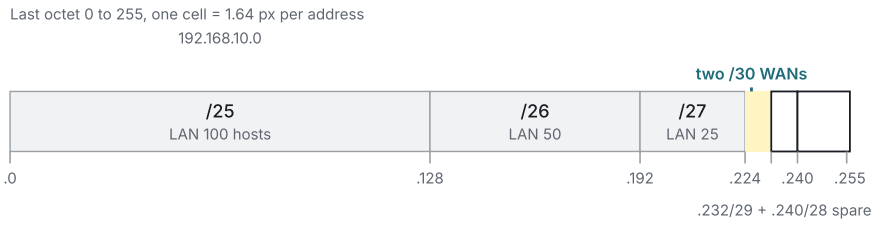


Figure 3.11 The VLSM plan as an address-space bar. The largest subnet is placed first, and each later one starts on a multiple of its own block size; what remains at the top is spare.

Summarisation and the same-subnet test

A router can announce one *summary* (supernet) instead of many routes when the subnets are contiguous and aligned. The method is to write the networks in binary, and find the longest common prefix.

WORKED EXAMPLE

Summarise 10.1.4.0/24 to 10.1.7.0/24, and test 10.1.5.0 to 10.1.8.0.

1. The third octets 4, 5, 6, 7 are 00000100, 00000101, 00000110, 00000111. They share the first six bits, 000001.
2. The prefix is 16 + 6 = 22: the summary is 10.1.4.0/22, which covers exactly 4 × 256 addresses and nothing else.
3. For 10.1.5.0 to 10.1.8.0 the third octets 5, 6, 7, 8 are 00000101 … 00001000, which share only the first four bits, 0000. The smallest single prefix is 10.1.0.0/20, which also claims 10.1.0.0 to 10.1.3.255 and 10.1.9.0 to 10.1.15.255. That over-claims addresses the router does not have.

> **Result: 10.1.4.0/22 summarises the first set exactly; the second cannot be summarised without over-claiming, and must be advertised as three routes: 10.1.5.0/24, 10.1.6.0/23 and 10.1.8.0/24.**

The same-subnet test is a cousin of this: two hosts are on one subnet if their addresses agree on the mask's network bits. For 10.10.10.130/26 and 10.10.10.190/26, the block size is 64 and both fall in 128 to 191, so they share a subnet; 10.10.10.193 falls in 192 to 255, so it does not. Interviewers at Cisco and Arista sometimes ask for this as code (Q3.14).

Finally, a word on how routers use all of these prefixes: when several routes match a destination, the router chooses the one with the *longest prefix*, the most specific. The algorithm and its data structure are treated in Pillar 1 (the trie), and the idea is applied in Section 3.8.

TRAP

Claim: "Every subnet has $2^n - 2$ usable hosts, so a /31 has none and a /32 has none." *Why it is wrong:* the "minus two" removes the network and broadcast addresses, which a /31 point-to-point link does not need (RFC 3021). *Correct statement:* a /31 gives two usable addresses on a point-to-point link, a /32 identifies one host, and $2^h - 2$ applies from /30 downwards.

INTERVIEW LENS

CISCO · AMAZON · JUNIPER · ARISTA · NOKIA

The question is "network, broadcast and range for X/Y without a calculator" (Q3.10), or a VLSM design (Q3.12). The examiner watches the method: state the block size first, then the multiples, then the answer. A strong candidate also mentions /31 and checks that the VLSM plan fits before presenting it.

LAB

`LAB 3.4 · 30 MIN` (i) On paper, solve the five drills below against a stopwatch; the target is 45 seconds each. (ii) In Python, check your answers with the standard library.

```
import ipaddress
n = ipaddress.ip_network("172.16.45.200/21", strict=False)
print(n.network_address, n.broadcast_address, n.num_addresses - 2,
n.hostmask)
```

```
172.16.40.0 172.16.47.255 2046 0.0.7.255
```

What you should notice: the library agrees with the block-size method. Then rewrite the same calculator with only integer shifts and masks (`mask = (0xFFFFFFFF << (32 - p)) & 0xFFFFFFFF`) and compare; that version is the one to write in an interview.

CHECK YOURSELF

1. Network and broadcast of 10.20.30.77/26. [1]
2. Usable range of 192.168.5.130/25. [2]
3. How many usable hosts in 172.20.0.0/19? [3]
4. Are 200.1.1.65/27 and 200.1.1.95/27 on the same subnet? Is the second a valid host address? [4]
5. The smallest prefix that holds 500 hosts. [5]

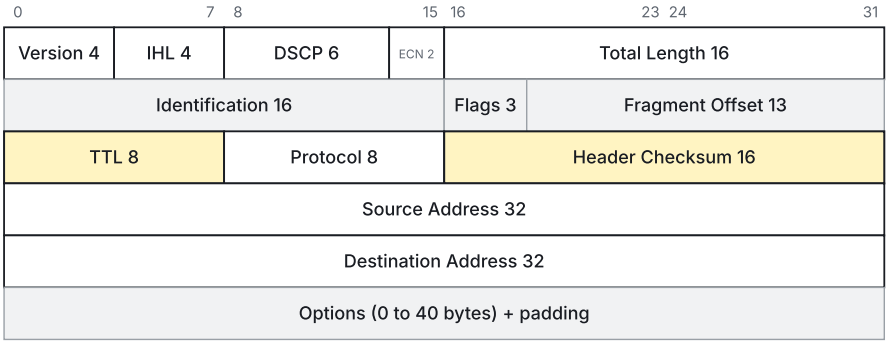
3.5 The IPv4 header, fragmentation, MTU and ICMP MUST · ■■□

Every IPv4 packet starts with a header that holds what a router needs in order to forward it, and little else. Knowing the header field by field lets one read a hex dump, explain why traceroute works and diagnose the fault that makes some websites refuse to load.

BIG IDEA

The IP header carries just enough for each router to forward the packet, and when a link is too small for the packet, the packet is cut into fragments or, if forbidden to fragment, dropped with an error message.

The header, field by field



Yellow: rewritten at every router hop (TTL, hence the checksum).
Grey: fragmentation fields (set by the sender; changed only when a router fragments), and options.
Flags: bit 0 reserved, DF = do not fragment, MF = more fragments. IHL counts 32-bit words, so 5 means 20 bytes.

Figure 3.12 The IPv4 header, 32 bits wide, as it appears on the wire. The minimum is 20 bytes (five rows); the Total Length covers header and data.

Version is 4. *IHL* (header length) counts 32-bit words, so the usual value is 5. *DSCP* and *ECN* carry the traffic class and congestion marks (Pillar 4 and Section 3.9). *Total length* is the size of header plus data in bytes, so a packet is at most 65,535 bytes. *Identification*, *flags* and *fragment offset* support fragmentation, described below. *TTL* (time to live) is decremented by every router and the packet is discarded at zero, which stops packets circulating for ever in a routing loop. *Protocol* names the payload: 1 for ICMP, 6 for TCP, 17 for UDP, 89 for OSPF. The *header checksum* protects the header only; it is the 16-bit one's-complement sum of all header words, and because TTL changes, every router must recompute it. The two 32-bit addresses close the fixed part.

WORKED EXAMPLE

Decode `45 00 00 3c 1c 46 40 00 40 06 9c bc c0 a8 00 68 c0 a8 00 01`.

1. `45`: version 4, IHL 5, so the header is 20 bytes. `00`: DSCP 0, ECN 0.
2. `00 3c`: total length 60 bytes. `1c 46`: identification 0x1c46.
3. `40 00`: flags and offset. 0x4000 is 0100 0000 0000 0000 in binary, so DF = 1, MF = 0, offset 0.
4. `40`: TTL 64. `06`: protocol 6 (TCP). `9c bc`: header checksum.
5. `c0 a8 00 68` is 192.168.0.104 and `c0 a8 00 01` is 192.168.0.1, the source and destination.

1. Block 64, 77 lies in 64 to 127: network 10.20.30.64, broadcast 10.20.30.127.↩
2. Block 128: network .128, broadcast .255, usable 192.168.5.129 to 192.168.5.254.↩
3. 13 host bits, so $2^{13} - 2 = 8{,}190$.↩
4. Block 32, both lie in 64 to 95, so they share a subnet, but .95 is its broadcast address and is not a valid host address.↩
5. $2^9 - 2 = 510 \geq 500$ and $2^8 - 2 = 254$ is too few, so 9 host bits: /23.↩

6. Verify the checksum: add the ten 16-bit words, 0x4500 + 0x003c + 0x1c46 + 0x4000 + 0x4006 + 0x9cbc + 0xc0a8 + 0x0068 + 0xc0a8 + 0x0001 = 0x2FFFD. Fold the carry back in: 0xFFFD + 0x2 = 0xFFFF.

Result: a 60-byte TCP packet from 192.168.0.104 to 192.168.0.1, TTL 64, DF set; the one's-complement sum is 0xFFFF, so the header is intact.

A receiver verifies the checksum by summing all words including the checksum; an undamaged header gives 0xFFFF. To compute it, set the field to zero, sum, fold the carries and complement: here the sum without the checksum is 0x6343 and its complement is 0x9cbc.

Fragmentation, MTU and MSS

Every link has a *maximum transmission unit* (MTU), the largest IP packet it carries: 1,500 bytes on Ethernet and 1,492 on PPPoE. If a packet is larger, a router may *fragment* it, cutting the data into pieces that each carry a copy of the IP header. The Identification field is the same in all fragments, and the *fragment offset* gives the position of each piece in units of 8 bytes, so every piece but the last must have a data size that is a multiple of 8. The *MF* (more fragments) flag is 1 on all but the last. Only the destination reassembles, because routers on the path may take different routes for different fragments.

WORKED EXAMPLE

Fragment a 4,000-byte datagram (20-byte header, 3,980 bytes of data) for a link with MTU 1,500.

1. Room for data per fragment: 1,500 − 20 = 1,480 bytes. That is a multiple of 8 (1,480 = 185 × 8), so it can be used as is.
2. Number of fragments: 3,980 / 1,480 = 2.69, so three: 1,480 + 1,480 + 1,020 = 3,980 bytes.
3. Offsets in 8-byte units: 0, 1,480 / 8 = 185, 2,960 / 8 = 370.

Fragment	Data	Total Length	Offset field	MF
1	1,480	1,500	0	1
2	1,480	1,500	185	1
3	1,020	1,040	370	0

Result: three fragments with offsets 0, 185, 370; the last has MF = 0 and a Total Length of 1,040. If a later link has a smaller MTU, each fragment is fragmented again, with offsets relative to the original.

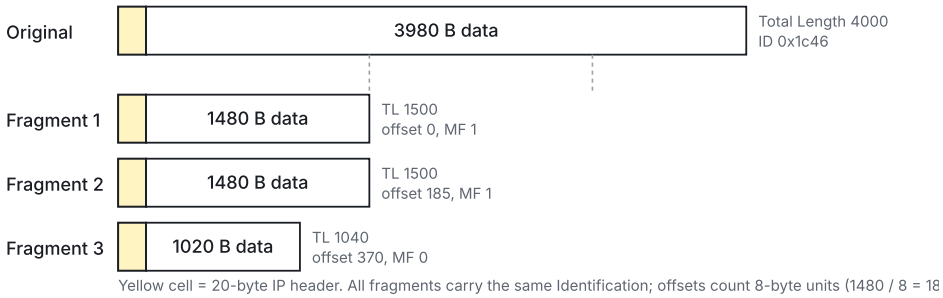


Figure 3.13 Fragmenting a 4,000-byte datagram for a link with MTU 1,500. Only the destination reassembles; routers on the way treat each fragment as a separate packet.

Fragmentation is costly: one lost fragment forces the loss of the whole datagram, and routers and firewalls dislike fragments. Hence modern hosts avoid it. They set the *DF* (don't fragment) flag and use *path MTU discovery* (PMTUD): a router that cannot forward a DF packet drops it and returns ICMP type 3 code 4, "fragmentation needed", with the next-hop MTU, and the sender then shrinks its packets. TCP expresses the result as the *maximum segment size* (MSS), the largest amount of TCP data in one segment, which excludes the IP and TCP headers: MSS = MTU − 20 − 20 = 1,460 on Ethernet. MTU and MSS are different: MTU includes the IP header and excludes the Ethernet header and FCS; MSS excludes both IP and TCP headers.

PMTUD fails when something blocks the ICMP message. The symptom is characteristic: the TCP handshake completes, small requests succeed, but large responses hang, because only full-size segments exceed the MTU. A common cause is a PPPoE or tunnel link with an MTU of 1,492 or less behind a firewall that drops all ICMP. The standard remedy is *MSS clamping*: the router rewrites the MSS option in passing SYN packets so that the endpoints never send oversize segments, for instance `ip tcp adjust-mss 1452` on a PPPoE interface (1,492 − 40).

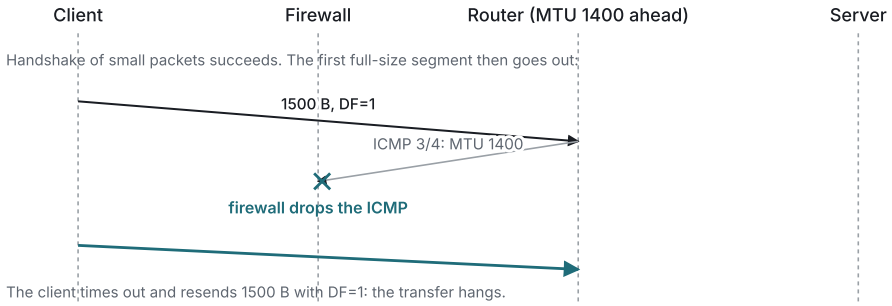


Figure 3.14 Path-MTU black hole. The router cannot forward a 1,500-byte packet with DF set onto its MTU-1,400 link, so it sends ICMP type 3 code 4; a firewall that blocks all ICMP discards it, and the client never learns to send smaller segments.

ICMP, ping and traceroute

The *Internet Control Message Protocol* (ICMP) carries errors and diagnostics, inside IP packets with protocol number 1. The types to know are: 0 echo reply and 8 echo request (ping); 3 destination unreachable, whose code says which kind (0 network, 1 host, 3 port, 4 fragmentation needed with DF set); 5 redirect; and 11 time exceeded.

Traceroute exploits the TTL. It sends probes with TTL 1, 2, 3 and so on. The first router decrements TTL 1 to zero, discards the probe and returns ICMP time exceeded, whose source address names that router; the next probe reaches the second router, and so forth until a probe reaches the destination, which answers differently. Linux sends UDP probes to ports from 33434 upwards, and the destination replies ICMP port unreachable (type 3 code 3); Windows `tracert` sends ICMP echo requests and the destination replies with an echo reply. A line of `* * *` means no answer arrived in time: the router may not generate time exceeded, rate-limit it or sit behind a firewall that drops the probe or its reply; it does not by itself mean that the path is broken.

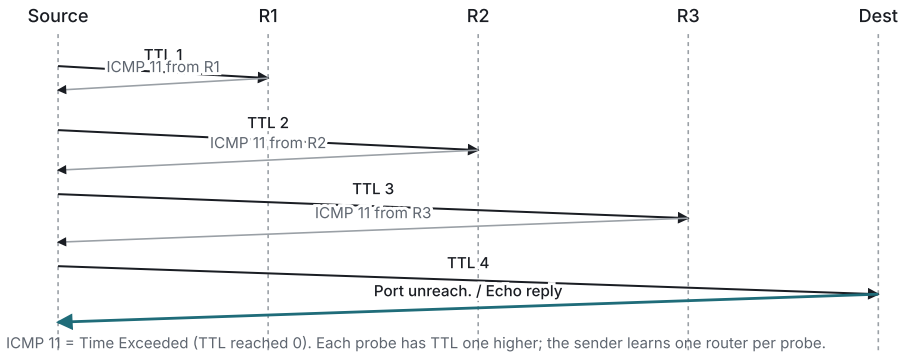


Figure 3.15 How traceroute works. A router that decrements TTL to zero discards the packet and returns an ICMP Time Exceeded message, revealing its own address. The last hop answers with a different message, because the probe finally arrives.

TRAP

Claim: "Fragment offset 1480 means the second fragment starts at byte 1480." *Why it is wrong:* the field counts 8-byte units, so a value of 1480 would mean byte 11,840. *Correct statement:* the second fragment of the example above has offset field 185, which is byte 1,480.

INTERVIEW LENS

AMAZON · MICROSOFT · CISCO · QUALCOMM · BROADCOM · JUNIPER

Q3.15 (fragmentation arithmetic) and Q3.16 (MTU versus MSS) are routine. Q3.17 is the scenario "ping works, the TCP handshake completes, the page never loads". A strong answer names PMTUD, says what to look for (ICMP 3/4 absent; retransmissions of the same large segment; a successful `ping -M do -s 1372`), and gives both fixes (allow ICMP type 3 code 4; clamp the MSS).

LAB

`LAB 3.5 · 20 MIN` On Linux, probe the path MTU with DF set. The ICMP header adds 8 bytes and the IP header 20, so a 1,472-byte payload is exactly a 1,500-byte packet.

```
$ ping -c1 -M do -s 1472 8.8.8.8
1480 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=9.8 ms
$ ping -c1 -M do -s 1473 8.8.8.8
ping: local error: message too long, mtu=1500
$ traceroute -n 8.8.8.8          # then: traceroute -I -n 8.8.8.8
 1  192.168.1.1  0.9 ms
 2  * * *
```

What you should notice: the largest unfragmented payload is 1,472 bytes, confirming the 1,500-byte MTU. In the traceroute, the `-I` (ICMP) option may succeed where the default UDP probes give `* * *` at a firewall; capture both in Wireshark with `ip.ttl == 1` to see the probes and the type-11 replies.

CHECK YOURSELF

1. A 1,500-byte packet (20-byte header) must cross a link with MTU 576. How many fragments, and what are their data sizes? [1]
2. Why is the header checksum recomputed at every hop? [2]
3. Which ICMP message makes PMTUD work, and what does it contain? [3]

3.6 ARP, DHCP and NAT: the glue protocols MUST · ■■□

Three small protocols turn a plugged-in laptop into a working Internet host. ARP finds the MAC address of the next hop, DHCP supplies the laptop's address, mask, gateway and DNS server, and NAT lets a private network share one public address. They are tested together because a single "packet walk" question exercises all three.

> BIG IDEA
>
> IP addresses say where a packet is going, MAC addresses say who gets it on the next link, and ARP, DHCP and NAT are the three devices that keep those two worlds consistent.

ARP

A host that wants to send an IP packet knows the destination IP address but needs a destination MAC address for the frame. If the destination is on the same subnet it needs the destination's MAC; otherwise it needs the MAC of the *default gateway*, the router that leads off the subnet. The *Address Resolution Protocol* (ARP) finds it: the host broadcasts "who has 192.168.1.1? tell 192.168.1.10" (opcode 1, destination MAC `ff:ff:ff:ff:ff:ff`), and only the owner replies by unicast (opcode 2) with its MAC. Both sides record the mapping in an *ARP cache*, which expires after a time that ranges from seconds on Linux to four hours on Cisco IOS.

Mnemonic — ARP asks "Who has this IP?" by broadcast and is answered by unicast; DHCP's first message, Discover, is also a broadcast, because the sender has nothing else to use.

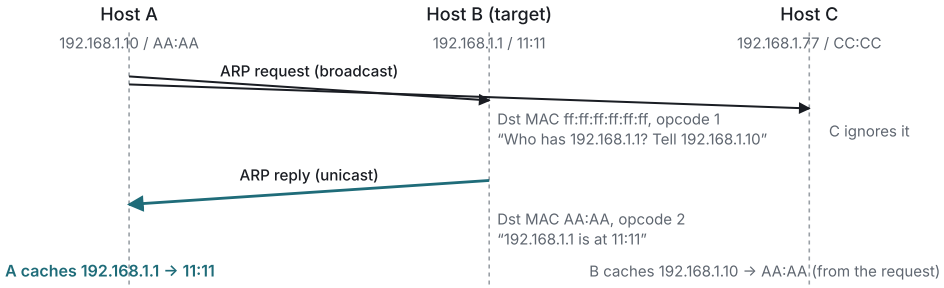


Figure 3.16 ARP resolves a next-hop IP address to a MAC address. The request is broadcast; only the owner of the target address replies, by unicast. MAC addresses are abbreviated to two bytes.

A *gratuitous ARP* is an ARP message in which the sender asks for its own address. A host sends it after configuring an address to detect a duplicate (if anyone replies, the address is taken) and to update other hosts' caches; a router in a first-hop redundancy group sends it on failover so that the switch and hosts learn that the virtual address has moved. *Proxy ARP* is a router answering ARP requests for addresses it can reach on behalf of the real owner. It lets a host with a wrong mask work, and so hides misconfiguration; it enlarges ARP tables and is normally disabled. *ARP spoofing*, the forging of replies, is treated in Pillar 9.

DHCP

DHCP (Dynamic Host Configuration Protocol) hands out addresses on lease. A new host has no address and does not know where the server is, so it must use broadcasts, and the exchange has four messages, remembered as DORA. The client broadcasts a **D**iscover (UDP source port 68, destination 67, source IP 0.0.0.0, destination 255.255.255.255). Every server replies with an **O**ffer holding an address and lease time. The client broadcasts a **R**equest naming the server it chose (option 54) and the address wanted (option 50), which also tells the other servers to withdraw. The chosen server confirms with an **A**cknowledgement, which carries the options: 1 subnet

mask, 3 default gateway, 6 DNS servers, 51 lease time. If no server answers, the client may give itself a link-local 169.254.x.y address, which signals a DHCP failure to anyone reading `ipconfig`.

The lease is renewed in stages. At T1, 50% of the lease, the client unicasts a Request to its server; if that fails, at T2, 87.5%, it broadcasts to any server; if the lease runs out, it starts again with Discover.

Broadcasts do not cross routers, but a company seldom has one DHCP server per subnet. A *relay agent*, configured on the router with `ip helper-address`, receives the broadcast, stores its own interface address in the *giaddr* field, and forwards the message by unicast to the server. The server uses giaddr to choose the right pool and sends its reply to that address, and the relay delivers it to the client.

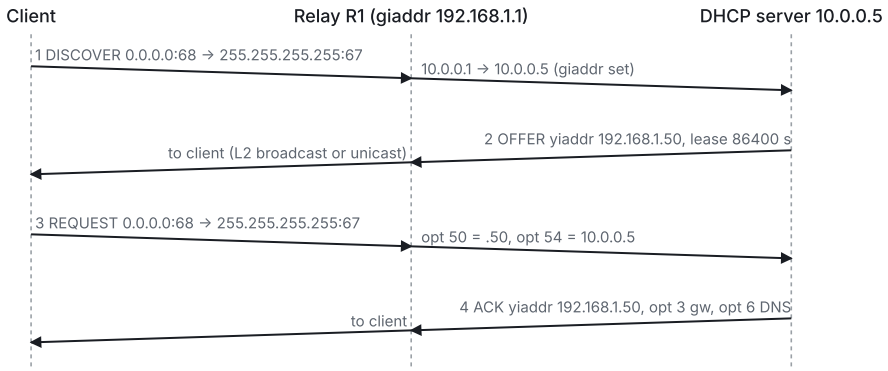


Figure 3.17 DHCP DORA through a relay. The client has no address and no idea where the server is, so Discover and Request are broadcasts (UDP 68 to 67). The relay turns them into unicasts, stamping its own interface address in the giaddr field so the server can pick a pool and reply.

NAT and PAT

Network address translation lets private addresses (Section 3.4) reach the Internet by rewriting the headers at a border device. In *static NAT* one private address is mapped to one public address permanently, which suits a server. In *dynamic NAT* the router maps hosts to addresses from a pool, one-to-one, for as long as they are in use. In *port address translation* (PAT, also *NAT overload*), by far the most common form, thousands of hosts share one public address, and the router tells their connections apart by the source port; it records each mapping in a *translation table*.

Cisco names the four addresses by two questions: is the host inside or outside, and is the address viewed as local (from inside) or global (from outside).

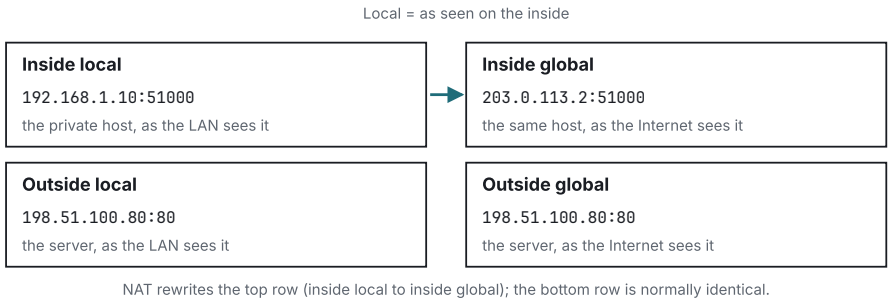


Figure 3.18 The four NAT address names. “Inside” and “outside” say where the host lives; “local” and “global” say from which side the address is viewed.

NAT has costs. A host behind it cannot be reached from outside unless a mapping exists, and so *port forwarding* (a static entry for one port) is needed to publish a server. Protocols that carry addresses in their payload, such as active FTP and SIP, break unless the router rewrites the payload too (an *application-level gateway*). Two hosts behind different NATs cannot open a connection to each other without help (see Q3.24). *Carrier-grade NAT* (CGNAT), which uses 100.64.0.0/10, adds a second NAT inside the provider. IPv6 (Section 3.7) removes the shortage that made all this necessary.

The packet walk

The following worked example is the canonical "show me every hop" question. PC-A (192.168.1.10/24, MAC AA:AA) sends an HTTP request to a web server at 198.51.100.80. R1 has an inside interface 192.168.1.1 (MAC 11:11) and an outside interface 203.0.113.2 (MAC 12:12) with PAT. The ISP router has 203.0.113.1 (MAC 21:21) towards R1 and 198.51.100.1 (MAC 22:22) towards the server, whose MAC is BB:BB. MAC addresses are abbreviated to two bytes.

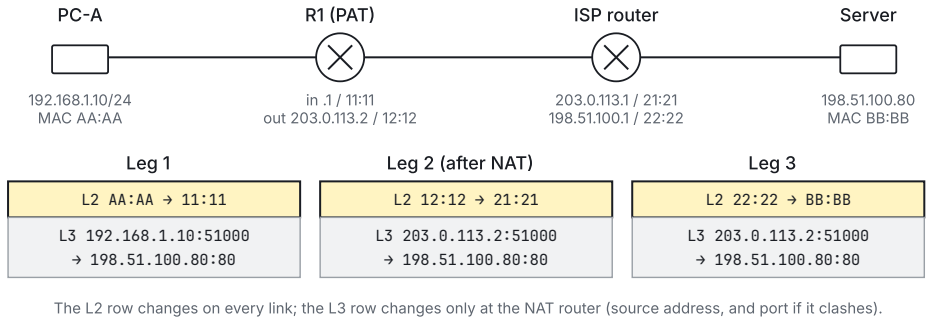


Figure 3.19 The packet walk of Section 3.6. The yellow row (MAC addresses) is rewritten at every hop; the grey row (IP address and port) is rewritten only where the NAT sits. The return packets use the same cards with source and destination swapped, and R1 translates the destination back.

WORKED EXAMPLE

Follow a TCP SYN from PC-A, source port 51000, to the server, port 80.

1. PC-A compares 198.51.100.80 with its own subnet 192.168.1.0/24. The destination is outside, so the frame must go to the default gateway, 192.168.1.1. PC-A has no cache entry and sends an ARP request; R1 replies "192.168.1.1 is at 11:11".
2. **Leg 1.** Frame: source MAC AA:AA, destination 11:11. Packet: 192.168.1.10:51000 to 198.51.100.80:80.
3. R1 decapsulates, decrements the TTL, looks up the route (default via 203.0.113.1) and applies PAT: the source 192.168.1.10:51000 becomes 203.0.113.2:51000, and the mapping is stored. R1 resolves 203.0.113.1 by ARP (21:21) if necessary.
4. **Leg 2.** Frame: 12:12 to 21:21. Packet: 203.0.113.2:51000 to 198.51.100.80:80.
5. The ISP router decrements TTL, finds 198.51.100.0/24 connected, and ARPs for the server (BB:BB).
6. **Leg 3.** Frame: 22:22 to BB:BB. Packet: unchanged, 203.0.113.2:51000 to 198.51.100.80:80.
7. The reply reverses the cards: server to 203.0.113.2:51000, then at R1 the table is consulted and the destination is rewritten to 192.168.1.10:51000, and the frame goes to AA:AA.

```
R1# show ip nat translations
Inside global      Inside local        Outside local     Outside global
203.0.113.2:51000  192.168.1.10:51000  198.51.100.80:80  198.51.100.80:80
```

Result: the MAC addresses change on every leg; the IP addresses are constant except at R1, which rewrites only the source address and port outward and the destination address and port inward. The TTL falls by one at each of the two routers.

TRAP

Claim: "The destination MAC stays the same from PC-A to the server, and the IP addresses change at every router." *Why it is wrong:* it is the other way round; a MAC address has meaning only on one link. *Correct statement:* L2 addresses are rewritten at every hop, L3 addresses end to end, except where NAT rewrites them.

INTERVIEW LENS

CISCO · AMAZON · GOOGLE · MICROSOFT · JUNIPER

Q3.23 is the walk above, usually with PAT. Interviewers push with "what if PC-A has no ARP entry for the gateway?", "what if the server is on the same subnet?" (no gateway: ARP for the server itself) and "what if two hosts use source port 51000?" (R1 picks another global port). Q3.20 and Q3.21 test DHCP with and without a relay.

LAB

`LAB 3.6 · 30 MIN` In Packet Tracer, configure R1 as DHCP server for 192.168.1.0/24 and PAT on its outside interface, then use `debug ip nat`. In Wireshark, filter `arp || dhcp` while renewing a lease with `sudo dhclient -r && sudo dhclient`; on Linux, view the cache with `ip neigh` and provoke a conflict with `arping`.

```
$ ip neigh show 192.168.1.1
192.168.1.1 dev eth0 lladdr 00:11:22:33:44:55 REACHABLE
1 DHCP Discover - Transaction ID 0x3a7c1e02
2 DHCP Offer    - Your IP address: 192.168.1.50
3 DHCP Request  - Requested IP: 192.168.1.50
4 DHCP ACK      - Lease 86400 s; Router 192.168.1.1
```

What you should notice: the first two packets use the Ethernet broadcast, and the DHCP transaction ID ties all four together. After the ACK, the client sends a gratuitous ARP; `show ip nat translations` fills only when traffic leaves the inside.

CHECK YOURSELF

1. Which MAC address does PC-A put as destination when it pings 8.8.8.8? [4]
2. A lease is 8 hours. When does the client first try to renew, and when does it start broadcasting? [5]
3. Why does a DHCP relay need to set giaddr? [6]

1. Data = 1,480; room per fragment = 576 − 20 = 556, rounded down to a multiple of 8 = 552. 1,480 / 552 = 2.68, so

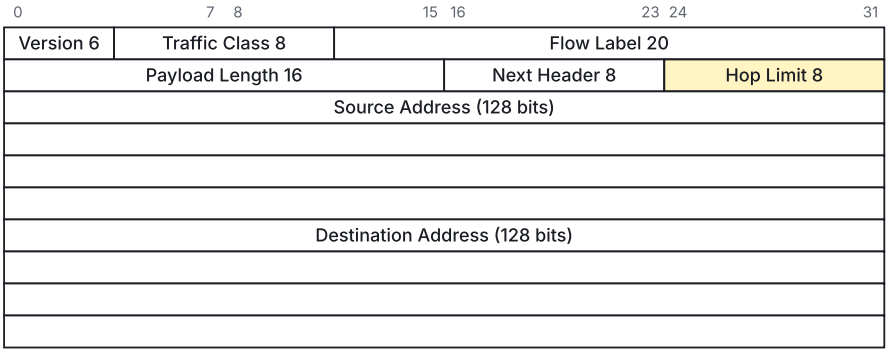
3.7 IPv6 essentials SHOULD · ■■□

IPv4 has about 4.3 billion addresses, and they ran out at the registries between 2011 and 2015. IPv6 widens the address to 128 bits, a space so large that every subnet can be given 2^{64} addresses. Its designers also simplified the header and removed several IPv4 mechanisms: there is no broadcast, no header checksum and no fragmentation by routers, and ARP is replaced by a neighbour discovery protocol built on ICMPv6. For Qualcomm, Nokia and Juniper this section is a Must.

> BIG IDEA
>
> IPv6 solves the shortage by making addresses 128 bits and solves the overhead by making the header fixed at 40 bytes, leaving everything else to extension headers and to ICMPv6.

Notation and address types

An IPv6 address is eight groups of four hexadecimal digits separated by colons. Two rules shorten it: leading zeros in a group may be dropped, and one run of consecutive all-zero groups may be replaced by `::`, once only, because two would make the address ambiguous. If two runs tie, compress the longest, and the leftmost if they are equal.



Fixed 40 bytes. Hop Limit (yellow) is the IPv4 TTL. No header checksum, no fragmentation fields, no options: those moved to extension headers chained by Next Header (6 = TCP, 17 = UDP, 58 = ICMPv6, 44 = Fragment).

Figure 3.20 The IPv6 header. Compared with IPv4 (Figure 3.12), the checksum, the header length, the three fragmentation fields and the options are gone; the addresses are four times larger and a Flow Label is added.

Type	Prefix	Notes
Global unicast (GUA)	2000::/3	Routable on the Internet; sites receive a /48 and subnets are /64
Unique local (ULA)	fc00::/7 (in practice fd00::/8)	Private, like RFC 1918
Link-local	fe80::/10	Valid on one link only; every interface has one
Multicast	ff00::/8	ff02::1 all nodes, ff02::2 all routers; replaces broadcast
Loopback, unspecified	::1/128, ::/128	Like 127.0.0.1 and 0.0.0.0
Documentation	2001:db8::/32	Used in this book

An *anycast* address is one assigned to several machines, and a packet goes to the nearest; it is how public DNS services and CDNs are reached.

three fragments: 552, 552, 376.↩

2. Because TTL, which is inside the header, changes at every hop.↩
3. ICMP type 3 code 4 (fragmentation needed and DF set); it carries the next-hop MTU and the start of the original packet.↩
4. The MAC address of its default gateway, learned by ARP, not any address for 8.8.8.8.↩
5. T1 = 50%, so after 4 hours it unicasts to the server; T2 = 87.5%, so after 7 hours it broadcasts to any server.↩
6. It tells the server which subnet the client is on, so that the right pool is used, and where to send the reply.↩

WORKED EXAMPLE

Compress `2001:0DB8:0000:0000:0000:FF00:0042:8329`, and `2001:db8:0:0:1:0:0:1`.

1. Drop leading zeros: `2001:DB8:0:0:0:FF00:42:8329`. The longest run of zero groups is three groups; replace it with `::`.
2. Lower case is conventional: `2001:db8::ff00:42:8329`.
3. In `2001:db8:0:0:1:0:0:1` there are two runs of two zero groups. They tie, so the left-most is compressed: `2001:db8::1:0:0:1`.

Result: 2001:db8::ff00:42:8329 and 2001:db8::1:0:0:1. Compressing both runs, as in `2001:db8::1::1`, is illegal.

Interface identifiers and SLAAC

Hosts in the same /64 need to choose the remaining 64 bits, the *interface identifier*. The older method, EUI-64, derives it from the MAC address: split the MAC in the middle, insert `FF:FE`, and invert the universal/local bit of the first byte.

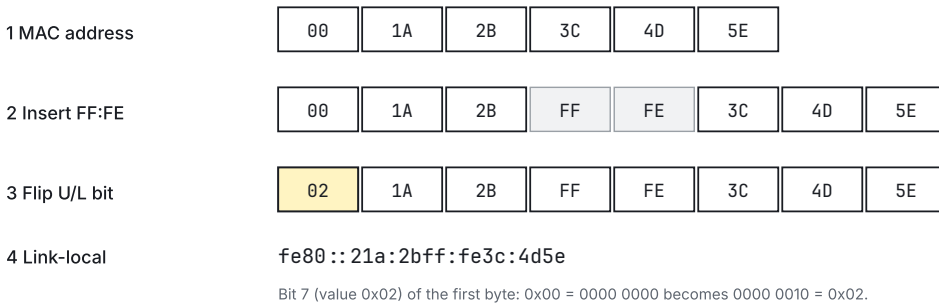


Figure 3.21 Building an EUI-64 interface identifier from a MAC address. The middle step splits the MAC between the vendor and device halves; the flip inverts the universal/local bit, so that a burned-in address is marked "universal" with a 1.

WORKED EXAMPLE

Link-local address and solicited-node multicast for MAC 00:1A:2B:3C:4D:5E.

1. Split the MAC after 3 bytes and insert FFFE: 00:1A:2B:FF:FE:3C:4D:5E.
2. Invert bit 7 of the first byte: 0x00 becomes 0x02. Interface ID = 021A:2BFF:FE3C:4D5E.
3. Link-local: `fe80::21a:2bff:fe3c:4d5e` (leading zero of 021A dropped).
4. The *solicited-node multicast* address is `ff02::1:ff` followed by the last 24 bits of the unicast address: `ff02::1:ff3c:4d5e`.
5. Its Ethernet multicast MAC is `33:33` followed by the last 32 bits of the IPv6 address: `33:33:ff:3c:4d:5e`.

Result: fe80::21a:2bff:fe3c:4d5e; solicited-node ff02::1:ff3c:4d5e; MAC 33:33:ff:3c:4d:5e.

Because EUI-64 exposes the hardware address and lets one be tracked between networks, modern systems use random, changing identifiers (*privacy extensions*) for global addresses. *Stateless address autoconfiguration* (SLAAC) lets a host build an address without a server: it forms a link-local address, checks for a duplicate (*DAD*, duplicate address detection), sends a Router Solicitation, receives a Router Advertisement carrying the prefix, and combines prefix and identifier. Two flags in the advertisement send the host to DHCPv6: M (managed) means "get the add-

ress by stateful DHCPv6", and O (other) means "build the address by SLAAC, but get DNS and other options by stateless DHCPv6". The prefix itself carries an A flag that permits SLAAC.

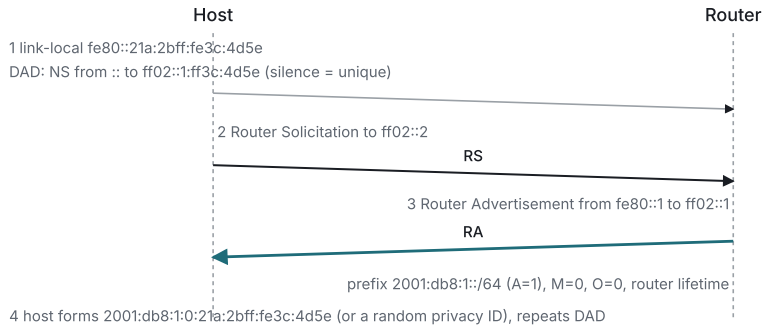


Figure 3.22 SLAAC. The host needs no server: it derives a link-local address, checks it for duplicates (DAD), asks for a router, and combines the advertised /64 prefix with its own interface identifier.

Neighbour discovery

NDP (neighbour discovery protocol) is a set of five ICMPv6 messages: router solicitation (type 133), router advertisement (134), neighbour solicitation (135), neighbour advertisement (136) and redirect (137). Address resolution is by 135 and 136: instead of a broadcast the host sends the neighbour solicitation to the *solicited-node multicast* address of the target, which only the few hosts sharing the last 24 bits listen to, and the target replies with its MAC in a neighbour advertisement. The network interrupts uninterested hosts far less than ARP does.

Header, fragmentation and transition

The 40-byte header has a fixed layout, and everything optional is moved into *extension headers* chained by the Next Header field. Routers never fragment IPv6 packets: only the source may, using a Fragment extension header, so a router with a too-large packet drops it and returns ICMPv6 "packet too big" (type 2). Every IPv6 link must carry at least 1,280 bytes. Hence ICMPv6 must not be blocked wholesale: unlike IPv4, without "packet too big", path MTU discovery fails, and without NDP nothing works at all.

Networks migrate in three ways. *Dual stack* runs both protocols side by side and is the preferred approach. *Tunnels* carry IPv6 packets inside IPv4 packets across an IPv4-only stretch. *NAT64* with *DNS64* lets IPv6-only hosts reach IPv4-only servers: DNS64 synthesises an AAAA record from the server's A record using the prefix 64:ff9b::/96, and NAT64 translates the packet.

WORKED EXAMPLE

Split 2001:db8:acad::/48 into /64 subnets.

1. The prefix borrows 64 − 48 = 16 bits, so there are 2^{16} = 65,536 subnets.
2. The subnet ID is the fourth group. The first three subnets are 2001:db8:acad:0::/64, 2001:db8:acad:1::/64 and 2001:db8:acad:2::/64.
3. The last is when the subnet ID is all ones: 2001:db8:acad:ffff::/64.

Result: 65,536 subnets, from 2001:db8:acad::/64 to 2001:db8:acad:ffff::/64. Subnetting IPv6 is easier than IPv4: it moves on hexadecimal digit boundaries (4 bits).

TRAP

Claim: "The EUI-64 transformation inverts the last bit of the MAC address." *Why it is wrong:* the bit flipped is the universal/local bit, bit 7 of the first byte (value 0x02), not the last bit of

the address. *Correct statement:* 00:1A:… becomes 02:1A:…, and `::` may be used only once in an address.

INTERVIEW LENS

QUALCOMM · NOKIA · JUNIPER · CISCO · AMAZON · GOOGLE

Q3.26 to Q3.29 are mechanical (compress, EUI-64, solicited-node, SLAAC versus DHCPv6), and each is solved in the Problem Set. Q3.30, "why don't IPv6 routers fragment, and what breaks if ICMPv6 is blocked?", rewards the candidate who links it back to PMTUD in Section 3.5.

LAB

`LAB 3.7 · 20 MIN` On Linux, inspect the addresses, neighbours and the router advertisement; in Wireshark use the filter `icmpv6.type == 134`.

```
$ ip -6 addr show dev eth0
    inet6 2001:db8:1:0:21a:2bff:fe3c:4d5e/64 scope global dynamic mngtmpaddr
    inet6 fe80::21a:2bff:fe3c:4d5e/64 scope link
$ ping -c1 ff02::1%eth0
```

What you should notice: the link-local address has the EUI-64 form derived above (some systems print a random identifier instead). The ping to ff02::1 is answered by every IPv6 host on the link, which is the multicast replacement for a broadcast ping.

CHECK YOURSELF

1. How many bits are in an IPv6 subnet ID if a site has a /48 and uses /64 subnets? [1]
2. Which multicast address does a router listen to for solicitations? [2]
3. Why does IPv6 need no ARP? [3]

3.8 Routing concepts in one page MUST · ■■□

Forwarding is the per-packet action of a router: look up the destination and send the packet out of one interface. *Routing* is the process that fills the table used for the lookup, either by hand (static routes) or by protocols that exchange information. The routing table is searched by *longest-prefix match*: of all entries whose prefix matches the destination, the router chooses the one with the longest mask, and the default route 0.0.0.0/0 matches everything with length zero, so it is chosen last. The data structure for a fast lookup is the trie of Pillar 1; this section needs only the rule.

WORKED EXAMPLE

Look up four destinations in a table with 10.0.0.0/8 to R1, 10.1.0.0/16 to R2, 10.1.1.0/24 to R3 and 0.0.0.0/0 to R4.

1. 10.1.1.5 matches /8, /16 and /24, and the longest is /24: next hop R3.
2. 10.1.2.5 matches /8 and /16 only (the third octet is not 1): R2.
3. 10.2.0.1 matches only /8: R1.
4. 11.0.0.1 matches nothing but the default: R4.

Result: R3, R2, R1, R4.

Routing protocols divide into two families by what they tell their neighbours. A *distance-vector* protocol sends its neighbours a list of destinations with a distance to each, and every router computes shortest paths by the Bellman-Ford rule; a *link-state* protocol floods the description of

every router's links to all routers, and each runs Dijkstra's algorithm on the full map. The algorithms themselves are solved in Pillar 1 (shortest paths) and the protocols are configured in Pillar 4 (RIP, OSPF, EIGRP, BGP).

	Distance-vector (RIP)	Link-state (OSPF, IS-IS)
Knowledge	Neighbours' distances only	Complete topology map
Algorithm	Bellman-Ford	Dijkstra (SPF)
Traffic	Periodic table to neighbours	Flooded updates on change
Convergence	Slow; count-to-infinity risk	Fast
Memory, CPU	Low	Higher

BGP, the protocol between organisations, is a third kind, *path-vector*: each route carries the list of autonomous systems it crossed, which prevents loops. An *autonomous system* (AS) is a network under one administration with an AS number; routing inside an AS uses an *interior* protocol and routing between ASes uses BGP.

The weakness of distance-vector is the *count to infinity*. A router believes a neighbour's claim that a network is reachable, and when the real route breaks, two routers may each point to the other for a route that no longer exists, adding one to the metric at each exchange.

WORKED EXAMPLE

Network N hangs off router C in the chain A, B, C; the link to N fails.

1. Before failure the metrics to N are C = 1, B = 2, A = 3.
2. C loses N but, before it announces anything, B's regular update says "N is 2 away". C accepts: N = 3 via B.
3. B hears from C that N is 3 and updates to 4; C then hears 4 and takes 5; the values grow 5, 6, 7 and so on until RIP's infinity of 16, when all three declare N unreachable.
4. With *split horizon*, B does not advertise N back to C, because B learned it from C; C hears nothing and N expires at once. *Poison reverse* sends N with metric 16 back instead. *Triggered updates* spread the bad news immediately, and *hold-down timers* ignore worse news for a while.

Result: without safeguards the metric climbs one step per exchange up to 16; split horizon removes the loop between two neighbours. Larger loops still need hold-down and triggered updates, one reason why link-state protocols replaced RIP.

CHECK YOURSELF

1. A table holds 192.168.0.0/16 and 192.168.4.0/22. Which entry forwards 192.168.5.9? [4]
2. Which family floods the topology, and which algorithm does each router run? [5]

1. 64 − 48 = 16 bits.↩
2. ff02::2, the all-routers address.↩
3. Neighbour discovery (ICMPv6 types 135 and 136, sent to the solicited-node multicast address) resolves IPv6 addresses to MAC addresses.↩

3.9 Transport: UDP, TCP, flow control and congestion control MUST · ■■■

The network layer delivers packets from one machine to another, but it does not say which program should receive them, nor does it promise that they arrive, in order, once. The transport layer adds exactly these things. It is also the layer where interviewers spend the most time, because a TCP question can start at a header field and end at the behaviour of a data-centre fabric. This section is therefore the longest in the pillar: it covers ports, UDP, the TCP header, connection set-up and teardown, the state machine and reliability here, and flow control and congestion control in the next pages.

> BIG IDEA
>
> UDP is a thin service that adds ports and a checksum to IP, while TCP builds a reliable, ordered, flow-controlled and congestion-controlled byte stream on top of an unreliable network by using sequence numbers, acknowledgements, timers and two windows.

Ports and sockets

An IP address names a machine; a *port number*, a 16-bit value, names a program on it. A *socket* is one endpoint, an address-and-port pair, and a TCP connection is identified by the four values (source IP, source port, destination IP, destination port), the *4-tuple* (with the protocol, the 5-tuple). Hence one server port such as 443 can serve millions of clients at once, because every client gives a different tuple. Ports below 1,024 are *well-known* (80 for HTTP, 443 for HTTPS, 53 for DNS, 22 for SSH; the full list is in the Revision Spread), 1,024 to 49,151 are *registered*, and 49,152 to 65,535 are *ephemeral*, picked by the operating system for the client side of a connection (Linux uses 32,768 to 60,999 by default). The sockets API that programs use is treated in Pillar 7.

UDP

The *User Datagram Protocol* is the smallest possible transport: the 8-byte header, shown with the TCP header below, holds the two ports, a length and a checksum that covers the data and parts of the IP header. There is no connection, no retransmission, no ordering and no congestion control: a datagram is sent and may arrive, or not. That is exactly right when a late packet is worthless, or when the application has its own recovery: DNS queries (one question, one answer), voice and video calls, online games and QUIC, the modern transport that the web is moving to (Section 3.10). UDP also suits broadcast and multicast, which a connection cannot.

4. 192.168.4.0/22 covers 192.168.4.0 to 192.168.7.255, so it is the longest match.↩
5. Link-state; Dijkstra's shortest-path-first.↩

The TCP header

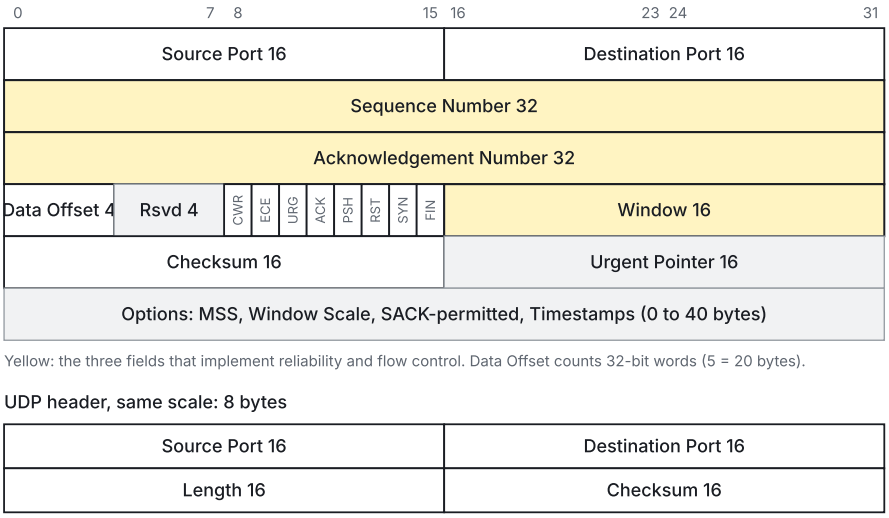


Figure 3.23 The TCP header (20 bytes plus options) above the UDP header (8 bytes). The extra twelve bytes of TCP buy sequence numbers, acknowledgements, a window and control flags.

Source and *destination ports* identify the endpoints. The *sequence number* is the byte-stream position of the first data byte in the segment; the *acknowledgement number* is the next byte the sender of the acknowledgement expects, so it confirms everything before it (a *cumulative* acknowledgement). *Data offset* gives the header length in 32-bit words, since options vary. The eight flag bits are: CWR and ECE, used for explicit congestion notification; URG, nearly obsolete; ACK, set on every segment after the first SYN; PSH, a hint to deliver to the application now; RST, which aborts the connection; SYN, which synchronises sequence numbers at the start; and FIN, which ends one direction. The *window* field is the receiver's advertised free buffer in bytes (Section 3.9, flow control). The checksum covers header, data and an IP pseudo-header.

The *options* appear mostly on SYN segments: the *maximum segment size* (MSS, the largest data per segment, usually 1,460), *window scale* (a shift count that multiplies the window, needed on fast long paths), *SACK-permitted* (enables selective acknowledgements) and *timestamps* (used to measure round-trip time and to protect sequence numbers on fast links).

Connection set-up: the three-way handshake

TCP connections have a beginning and an end, and both sides keep *state* (variables such as the next sequence number, the window and timers) while the connection lasts. The *three-way handshake* creates that state at both ends. The client sends a SYN with its *initial sequence number* (ISN); the server answers with a SYN,ACK that acknowledges the client's ISN and announces its own; the client acknowledges that in turn. The ISN is not zero but a random number: it prevents old segments of a former connection from being accepted by the new one, and it makes it hard for an attacker to guess sequence numbers and inject data (RFC 6528).

WORKED EXAMPLE

Handshake, data and teardown with ISNs 1000 (client) and 5000 (server).

1. SYN: seq = 1000. The SYN consumes one sequence number, so the next client byte is 1001.
2. SYN,ACK: seq = 5000, ack = 1001. The server's SYN also consumes one number.

3. ACK: seq = 1001, ack = 5001. The connection is established on both sides.
4. The client sends 500 bytes, seq = 1001, ack = 5001. The server replies with a pure ACK, seq = 5001, ack = 1501 (1,001 + 500). The pure ACK consumes no sequence number.
5. The client closes: FIN,ACK with seq = 1501. The server ACKs with ack = 1502, because the FIN counts as one byte.

Result: SYN (1000), SYN,ACK (5000, ack 1001), ACK (1001, ack 5001); after 500 bytes the acknowledgement is 1501; the FIN advances it to 1502. A SYN and a FIN each use one sequence number; a pure ACK uses none.

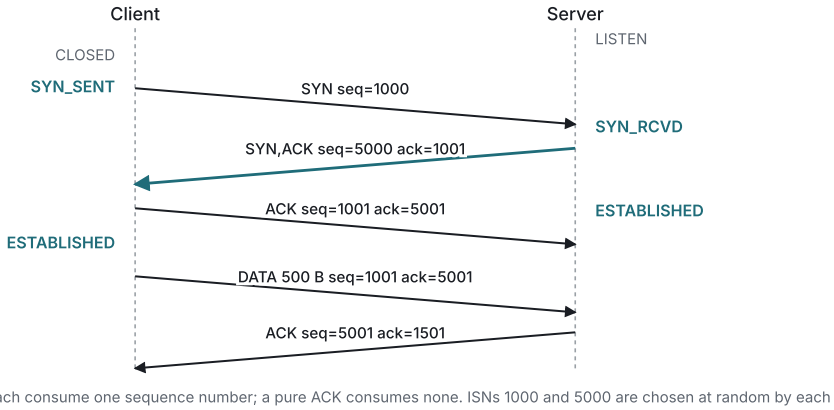


Figure 3.24 The three-way handshake with the initial sequence numbers of the worked example. The client is in SYN_SENT until the SYN,ACK arrives; the server is in SYN_RCVD until the final ACK arrives.

Two messages would not be enough, because the server could not tell whether its own SYN,ACK had arrived, nor distinguish a fresh SYN from a delayed duplicate of an old one. The third segment confirms to the server that the client heard it, and that the request is current. If two hosts send SYNs at the same moment, a *simultaneous open*, they still end up with one connection after exchanging SYN,ACKs.

Why three? — A and B each need to be sure the other heard their ISN. Two messages leave one direction unconfirmed; four are not needed, because the server combines its ACK and SYN in one segment.

A server must remember each half-open connection, and this is exploited by a *SYN flood*: an attacker sends a stream of SYNs from forged addresses and never answers the SYN,ACKs, filling the queue of half-open connections until real clients are refused. *SYN cookies* defeat it by keeping no state: the server encodes a hash of the 4-tuple, a secret and a coarse timestamp (and a code for the MSS) in the sequence number of its SYN,ACK, and forgets the connection. A legitimate client's ACK returns that number plus one; the server verifies the hash and rebuilds the connection. Only genuine clients cost the server any memory, at the price that some options of the SYN, such as the window scale, may be lost.

Teardown and TIME_WAIT

Each direction of a connection is closed separately: a FIN means "I have no more to send", and the other side acknowledges it. The result is the *four-way teardown* of the figure below, in which the second and third messages may be merged. The side that has received a FIN but not yet closed itself can keep sending (a *half-close*), which is why the server's ACK and FIN are separate in general.

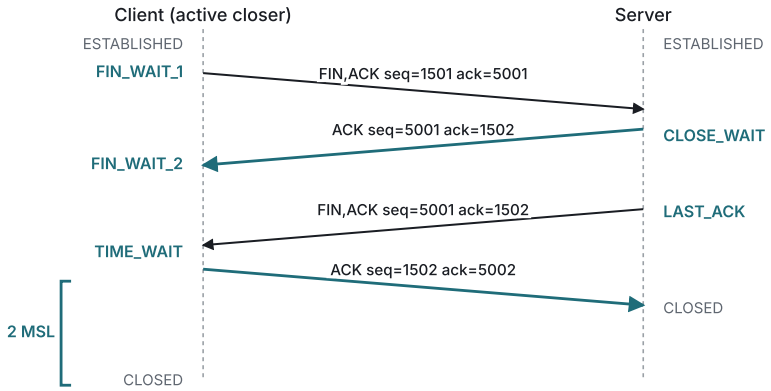


Figure 3.25 Four-way teardown. Each direction is closed separately, so the server may keep sending between its ACK and its own FIN (half-close). The side that sends the first FIN waits 2 MSL in TIME_WAIT before releasing the connection.

The side that sends the first FIN, the *active closer*, ends in the *TIME_WAIT* state and waits for twice the *maximum segment lifetime* (2 MSL; Linux uses a fixed 60 seconds) before it forgets the connection. There are two reasons for this. If the final ACK is lost, the peer retransmits its FIN, and the active closer must still be there to acknowledge it. And any stray segments of the old connection, still wandering the network, must die before the same 4-tuple is reused, or they could be mistaken for data of the new connection.

The passive closer sits in *CLOSE_WAIT* between receiving the FIN and calling `close()`. That state lasts only as long as the application takes to notice the closed connection; a server with thousands of CLOSE_WAIT sockets therefore has a bug in its own code, which forgets to close sockets, and no network setting will cure it. TIME_WAIT, in contrast, is normal; it becomes a nuisance only on a busy client or proxy, which opens many short connections to one destination and runs out of ephemeral ports (about 28,000 ports and 60 seconds allow roughly 470 new connections per second to a single destination). The remedies are connection reuse, more source addresses or ports and, on Linux, `tcp_tw_reuse` for outgoing connections.

A TCP stack sends an *RST* (reset) when it receives a segment for which no connection exists, which includes a SYN to a port where nothing listens, so that the sender fails at once instead of timing out. An RST is also sent when an application aborts (closing a socket with unread data, or with a zero linger time), when a firewall or an intrusion-prevention box injects one to kill a flow, and when a segment violates the connection state, such as data arriving after the peer has crashed and restarted.

The state machine

The behaviour above is summarised by a *state machine* of eleven states. Reading the figure along the solid line gives the client’s life, and along the dashed line the server’s.

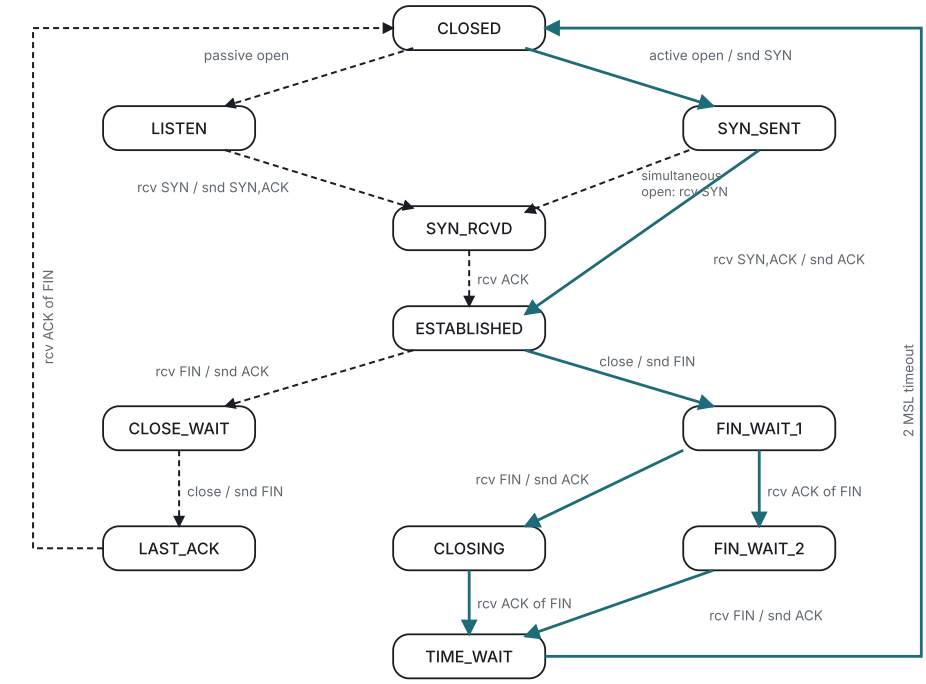


Figure 3.26 The TCP state machine (the eleven states of RFC 9293). Arrows are labelled "event / action". The three states that matter in operations are ESTABLISHED, CLOSE_WAIT (the application has not closed) and TIME_WAIT (the active closer waiting 2 MSL).

> TRAP
>
> *Claim:* "TIME_WAIT is on the client, and a server never has it." *Why it is wrong:* TIME_WAIT belongs to whichever side closes first, the *active closer*. *Correct statement:* a web server that closes idle connections itself accumulates TIME_WAIT sockets, while a client that closes first has them on its side; the passive closer goes through CLOSE_WAIT and LAST_ACK.

Reliability: acknowledgements, timers and fast retransmit

TCP numbers bytes, and the receiver acknowledges the next byte it wants. To avoid an acknowledgement for every segment, a receiver may delay the ACK, typically until a second full segment arrives or about 40 ms (Linux) to 200 ms (older systems) have passed. If a segment is lost, later segments arrive out of order and each one makes the receiver repeat the same acknowledgement, a *duplicate ACK*.

The sender starts a timer when it sends data. The *retransmission timeout* (RTO) must exceed the round-trip time (RTT), or packets are resent needlessly, but not by much, or recovery is slow. It is therefore computed from measured samples by the method of Jacobson and Karels (RFC 6298). With sample R, a smoothed average SRTT and a mean deviation RTTVAR, the update is RTTVAR = (1 − β) RTTVAR + β |SRTT − R|, then SRTT = (1 − α) SRTT + α R, with α = 1/8 and β = 1/4, and RTO = SRTT + 4 RTTVAR. *Karn's algorithm* adds that samples from retransmitted segments are ignored, since it is unclear which transmission the ACK answers; and after each timeout the RTO is doubled (*exponential backoff*).

> WORKED EXAMPLE
>
> **RTO update from SRTT = 100 ms and RTTVAR = 25 ms with samples 120 ms and 80 ms.**

1. First sample R = 120: |SRTT − R| = 20; RTTVAR = 0.75 × 25 + 0.25 × 20 = 23.75 ms; SRTT = 0.875 × 100 + 0.125 × 120 = 102.5 ms. RTO = 102.5 + 4 × 23.75 = 197.5 ms.
2. Second sample R = 80: |102.5 − 80| = 22.5; RTTVAR = 0.75 × 23.75 + 0.25 × 22.5 = 23.44 ms; SRTT = 0.875 × 102.5 + 0.125 × 80 = 99.69 ms. RTO = 99.69 + 4 × 23.44 = 193.4 ms.

Result: RTO = 197.5 ms, then 193.4 ms. RTTVAR is updated first, using the old SRTT; real stacks then clamp the RTO to a minimum, 200 ms on Linux (RFC 6298 recommends 1 s).

Waiting for a timeout is slow, so TCP also uses *fast retransmit*: when three duplicate ACKs arrive, the sender retransmits the missing segment at once. The number three is a compromise. One or two duplicates can be produced by simple reordering in the network, which would cause needless retransmissions; waiting for more costs time on small windows. *Selective acknowledgements* (SACK) refine this: the receiver lists, in an option, the blocks of data it holds above the hole, so that the sender repairs several holes in one round trip and resends nothing that has arrived.

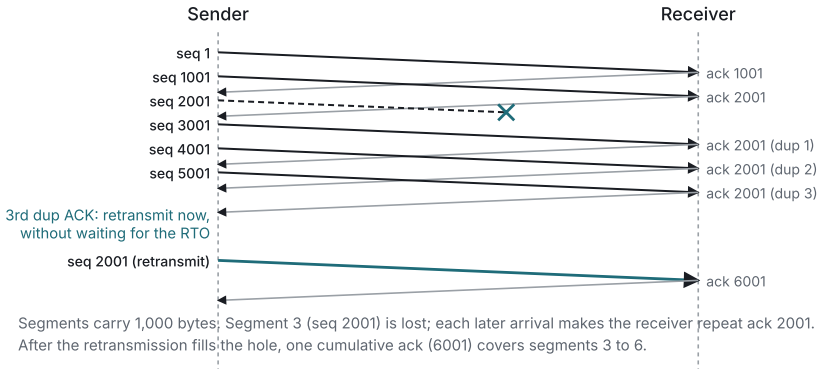


Figure 3.27 Fast retransmit. Three duplicate acknowledgements are strong evidence of loss rather than reordering, so the sender repairs the hole after about one round trip instead of waiting for the retransmission timeout.

INTERVIEW LENS

CISCO · AMAZON · MICROSOFT · GOOGLE · QUALCOMM · JUNIPER

Q3.32 to Q3.35 and Q3.42 (handshake, why three-way, TIME_WAIT, CLOSE_WAIT, SYN flood) are among the most reliably asked questions in the whole book, and each can be answered in thirty seconds. The follow-up decides the grade: “which side holds TIME_WAIT?” and “what would you check on a server with thousands of CLOSE_WAIT?” separate those who have memorised a diagram from those who have debugged a service. Q3.44 asks when a stack sends RST.

LAB

`LAB 3.9a · 25 MIN` Capture a handshake and a close with `tcpdump`, then provoke an RST and a CLOSE_WAIT leak.

```
$ sudo tcpdump -nn -S -i any 'tcp port 8080'
IP 10.0.2.15.49822 > 203.0.113.10.8080: Flags [S],  seq 1000, win 64240, op-
tions [mss 1460,sackOK,TS val 1 ecr 0,nop,wscale 7]
IP 203.0.113.10.8080 > 10.0.2.15.49822: Flags [S.], seq 5000, ack 1001, win
65160, options [mss 1460,sackOK,TS,nop,wscale 7]
IP 10.0.2.15.49822 > 203.0.113.10.8080: Flags [.],  ack 5001, win 502
$ sudo hping3 -S -p 9 -c 1 203.0.113.10        # a closed port
IP 203.0.113.10.9 > 10.0.2.15.2412: Flags [R.], seq 0, ack 1, win 0
$ ss -tan state close-wait                      # server that never calls
close()
Recv-Q Send-Q  Local Address:Port   Peer Address:Port
1      0       10.0.2.15:8080       10.0.2.99:51234
```

(The sequence numbers are shortened; real ISNs are random 32-bit values and `-S` shows them absolute.) **What you should notice:** `S` is SYN, `S.` is SYN,ACK and `.` is ACK. The closed port answers a SYN with `R.`, the RST,ACK described above. In Wireshark, *Statistics → TCP Stream Graphs → Stevens* plots sequence number against time; the preference *Relative sequence numbers* reproduces 0 and 1 instead of the random ISNs. The CLOSE_WAIT socket appears as soon as the client closes and stays until the server process closes its end (Pillar 7).

CHECK YOURSELF

1. A client with ISN 7000 sends a SYN, and later 300 bytes. What ACK does the server send after the data? [1]
2. Which endpoint is in TIME_WAIT if the server calls `close()` first? [2]
3. Three duplicate ACKs arrive for ack = 4001. What does the sender do? [3]

Flow control: protecting the receiver

Flow control stops a fast sender from overrunning a slow receiver. The receiver advertises in every segment how many more bytes its buffer can take, the *receive window* (rwnd), and the sender never has more than that many bytes unacknowledged. Because the sender may transmit a whole window before it needs an acknowledgement, the scheme is termed a *sliding window*: the left edge advances with each acknowledgement, and the right edge advances with it.

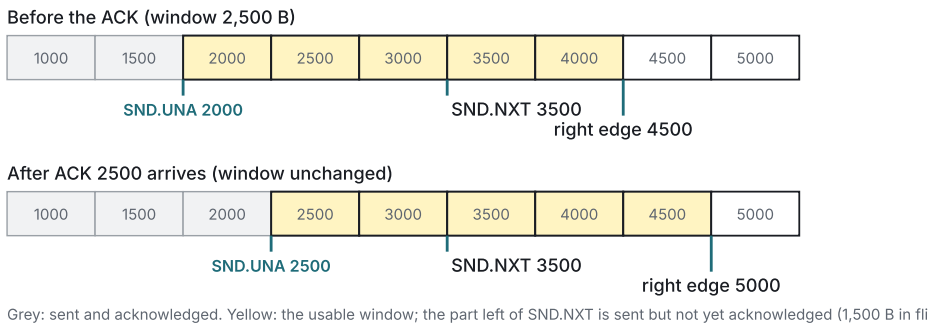


Figure 3.28 The sliding window on the byte stream, drawn in 500-byte cells. An acknowledgement slides the left edge right, and the right edge follows by the same amount if the receiver's advertised window is unchanged.

If the receiver's buffer fills completely it advertises a window of zero and the sender stops. To avoid deadlock should the window update be lost, the sender then sends tiny *window probes* on a *persist timer*, with growing intervals. A related fault is the *silly window syndrome*, in which a

1. The SYN used 7000, the data starts at 7001, so the ACK is 7001 + 300 = 7301.↩
2. The server, since it is the active closer.↩
3. It retransmits the segment that starts at byte 4001 immediately (fast retransmit) without waiting for the RTO, and enters fast recovery.↩

slow receiver advertises tiny windows and a sender fills them with tiny segments, so that headers dominate: the receiver should not advertise a window until it can offer a full segment or half its buffer, and the sender should not send tiny segments when it can wait.

Nagle's algorithm is the sender-side rule. While any sent data is unacknowledged, a small write is held back and merged with later writes until the acknowledgement arrives or a full segment accumulates. This saves many small packets, but combined with the receiver's delayed ACK it can stall an interactive application: if the program writes a request in two small pieces, the second piece waits for the ACK of the first, and the ACK waits, for up to 40 to 200 ms, for the receiver to delay it. The usual fix is to disable Nagle with the socket option `TCP_NODELAY`, or to assemble the request in one write.

Throughput limits and the window scale

Since at most one window is in flight per round trip, a connection cannot exceed throughput ≤ W / RTT, where W is the smaller of the windows. The window needed to keep a path full is the *bandwidth-delay product* (BDP), the capacity R times the round-trip time: the amount of data "in the pipe". The 16-bit window field holds at most 65,535 bytes, which the *window scale* option multiplies by 2^S, with s up to 14, giving windows to about 1 GiB.

WORKED EXAMPLE

A 64 KB window on a 100 ms path; then a 1 Gb/s path of the same length.

1. Ceiling with W = 65,535 B and RTT = 100 ms: 65,535 × 8 / 0.1 = 5.24 Mb/s, whatever the link speed.
2. For a 1 Gb/s path, BDP = 10^9 × 0.1 / 8 = 12.5 MB.
3. Required scale: 65,535 × 2^S ≥ 12.5 × 10^6, so 2^S ≥ 191, so s = 8 (2^8 = 256 gives 16.8 MB; s = 7 gives only 8.4 MB).
4. For a 1 Gb/s path with RTT 190 ms (Mumbai to Virginia): BDP = 23.75 MB, so s = 9 (33.5 MB).

Result: 5.24 Mb/s without scaling; a shift of 8 for the 100 ms path and 9 for the 190 ms path. The receive buffer, not just the option, must be that large.

Congestion control: protecting the network

Flow control protects the receiver; *congestion control* protects the network. In October 1986 the early Internet suffered *congestion collapse*: when queues filled, packets were dropped, senders retransmitted, the retransmissions filled the queues further, and throughput fell by a factor of a thousand. Van Jacobson's remedy, adopted in 1988, is that the sender keeps a second window, the *congestion window* (cwnd), which it grows when things go well and cuts when loss signals trouble. The sender may have at most min(cwnd, rwnd) bytes in flight. Loss is treated as the signal that a queue somewhere has overflowed, and cwnd is counted here in segments (MSS) for clarity.

The algorithm has two phases. In *slow start* cwnd starts small (1 segment in the classical description, 10 in modern stacks) and increases by one segment for every ACK, which doubles it every round trip, an exponential growth that is "slow" only compared with sending a whole window immediately. When cwnd reaches the *slow-start threshold* ssthresh, *congestion avoidance* begins, and cwnd grows by one segment per round trip, a linear increase. On loss the window is cut. This pattern, *additive increase, multiplicative decrease* (AIMD), is what lets many flows share a link fairly: each increase is equal, but each cut removes a share proportional to the flow's size, so unequal flows converge.

The variants differ in how they react to loss. *Tahoe*, the original, treats every loss alike: ssthresh = cwnd / 2, cwnd = 1, slow start again. *Reno* distinguishes the signals. A timeout is serious, so it acts like Tahoe. Three duplicate ACKs mean that later segments are still arriving, so the network is not dead: Reno sets ssthresh = cwnd / 2 and cwnd = ssthresh + 3 (the three duplicate ACKs each mark a segment that has left the network), retransmits the lost segment, and inflates cwnd by one per further duplicate ACK; when the new ACK arrives, cwnd deflates to ssthresh and congestion avoidance continues. This is *fast recovery*. *NewReno* improves recovery when several segments of one window are lost: it stays in fast recovery until all of them are acknowledged, treating a *partial ACK* as a sign of the next hole.

WORKED EXAMPLE

Trace cwnd (in segments) with initial cwnd = 1 and ssthresh = 16, when loss is detected at cwnd = 20.

1. Slow start, one value per RTT: RTT 1 to 5 give 1, 2, 4, 8, 16. At 16 = ssthresh the phase changes.
2. Congestion avoidance, +1 per RTT: RTT 6 to 9 give 17, 18, 19, 20.
3. Loss at cwnd = 20, so ssthresh = 20 / 2 = 10.
4. *Tahoe* (and any variant after a timeout): cwnd = 1, then slow start 1, 2, 4, 8, then 10 (capped at ssthresh), then 11, 12, 13 and so on.
5. *Reno* after three duplicate ACKs: cwnd = 10 + 3 = 13 during recovery, then 10 when the new ACK arrives, and then 11, 12, 13 … in congestion avoidance.

Result: ssthresh = 10 in both cases; Tahoe restarts from 1 and needs four extra round trips to reach the threshold, while Reno resumes at 10. Reno halves; a timeout resets to 1. Do not conflate the two.

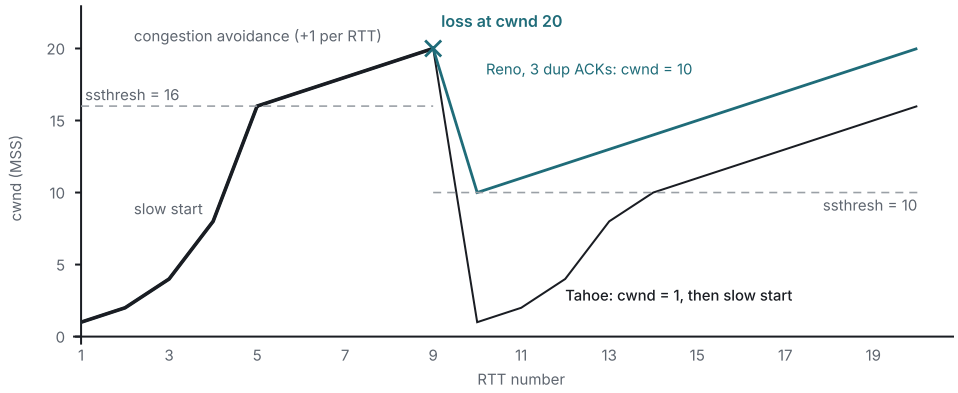


Figure 3.29 The congestion window over 20 round trips. Both variants share the first nine RTTs. At the loss, Tahoe (and any variant on a timeout) restarts from 1; Reno, having seen three duplicate ACKs, halves the window and continues in congestion avoidance.

Both Tahoe and Reno are loss-based and increase by one segment per round trip, which is too timid on a long fast path, where thousands of round trips are needed to recover after a single loss. Two modern algorithms address this in different ways.

CUBIC, the default in Linux (and in Windows and macOS), replaces the linear increase by a cubic function of the time since the last loss: $W(t) = C\,(t - K)^3 + W_{max}$, where W_{max} is the window when the loss occurred and $K = (W_{max}\,\beta / C)^{1/3}$ is the time needed to climb back to it. After a loss the window is cut to $0.7\,W_{max}$; it then rises quickly (the concave part), flattens around W_{max}

where the last loss occurred, and finally accelerates again to look for more capacity (the convex part). Since the function depends on time and not on the number of ACKs, flows with different round-trip times grow at the same rate, which is fairer than Reno.

BBR (bottleneck bandwidth and round-trip propagation time), from Google, drops the idea that loss is the signal. It builds a model of the path from two measurements: the maximum delivery rate seen recently, BtlBw, and the minimum round-trip time, RTprop. It then *paces* packets at about BtlBw and limits the data in flight to a small multiple of the BDP = BtlBw × RTprop, thus operating at the corner of the figure below, with a full pipe and an empty queue, instead of filling the buffer as loss-based algorithms do. Its phases are Startup (exponential search for BtlBw), Drain, ProbeBW (a cycle of eight round-trip phases with gains 1.25, 0.75 and six of 1, to detect more bandwidth) and ProbeRTT (a brief reduction every ten seconds to measure RTprop). BBR performs well on paths with random loss, where CUBIC collapses, and keeps queues short. Its first version, however, ignores loss and can take an unfair share when sharing a shallow buffer with CUBIC flows, which is why versions 2 and 3 add responses to loss and to ECN marks.

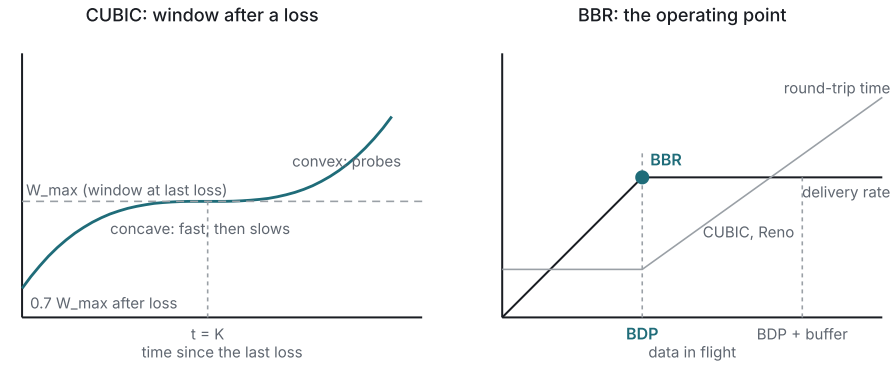


Figure 3.30 Left: CUBIC grows the window as a cubic function of the time since the last loss, flat around W_max. Right: delivery rate (black) saturates at the bottleneck bandwidth, while round-trip time (grey) starts to grow once the pipe is full; BBR operates at the corner, loss-based algorithms at the buffer limit.

Two further mechanisms lie close to this topic. *Explicit congestion notification* (ECN) lets a router mark a packet instead of dropping it when its queue grows; the receiver echoes the mark in the ECE flag and the sender reduces its window and answers with CWR, which signals congestion without loss. *Active queue management* (AQM), such as RED and CoDel, makes routers drop or mark early, so that queues stay short; it is taken up in Pillar 4's treatment of quality of service.

	UDP	TCP	QUIC
Runs over	IP	IP	UDP (port 443)
Connection set-up	None	1 RTT (SYN), plus TLS 1 to 2 RTT	1 RTT with TLS 1.3 built in; 0-RTT on resumption
Reliability and order	None	Whole stream, in order	Per stream
Congestion control	None (application's duty)	In the kernel	In user space (CUBIC, BBR)
Head-of-line blocking	Not applicable	Yes, across the stream	Per stream only
Header	8 bytes	20 to 60 bytes	UDP + QUIC header; payload encrypted

Typical use	DNS, voice, games	Web (HTTP/1.1, 2), email, SSH	HTTP/3

TRAP

Claim: “Flow control and congestion control are the same window, and TCP sends min(rwnd, cwnd) because the smaller one is safer.” *Why it is wrong:* the two windows protect different things, the receiver’s buffer (advertised in the header) and the network’s queues (estimated by the sender and never transmitted). *Correct statement:* the effective window is min(rwnd, cwnd); the slower of the two limits is the binding constraint, and each is changed by a different event.

INTERVIEW LENS

GOOGLE · MICROSOFT · AMAZON · QUALCOMM · CISCO

Q3.36 to Q3.41 are the TCP performance cluster: effective window, the cwnd trace (Q3.37), CUBIC versus BBR (Q3.38), the throughput ceiling (Q3.39), fast retransmit and SACK (Q3.40) and the Nagle stall (Q3.41). Google and Microsoft look for a mental model: which signal does the algorithm use (loss, or delay and bandwidth), and what does it assume about the buffer? Q3.43 is a coding variant: simulate a sliding-window sender; the solution is in the Problem Set.

LAB

`LAB 3.9b · 40 MIN` Compare CUBIC and BBR on an emulated path with two Linux network namespaces. The path has 50 ms of delay and 1% random loss.

```
sudo ip netns add c1; sudo ip netns add c2
sudo ip link add v1 type veth peer name v2
sudo ip link set v1 netns c1; sudo ip link set v2 netns c2
sudo ip -n c1 addr add 10.9.0.1/24 dev v1; sudo ip -n c2 addr add 10.9.0.2/24
dev v2
sudo ip -n c1 link set v1 up; sudo ip -n c2 link set v2 up
sudo ip netns exec c1 tc qdisc add dev v1 root netem delay 50ms loss 1%
sudo ip netns exec c2 iperf3 -s -D
sudo modprobe tcp_bbr
sudo ip netns exec c1 iperf3 -c 10.9.0.2 -t 20 -C cubic
sudo ip netns exec c1 iperf3 -c 10.9.0.2 -t 20 -C bbr
sudo ip netns exec c1 ss -ti dst 10.9.0.2      # during a run
```

```
bbr wscale:7,7 rto:252 rtt:51.2/1.1 mss:1448 cwnd:1043 bytes_retrans:...
pacing_rate 790Mbps delivery_rate 640Mbps
```

What you should notice: with 1% loss the Mathis estimate for a Reno-like flow, (MSS / RTT) × 1.22 / √p = (11,680 bit / 0.05 s) × 12.2, is about 2.9 Mb/s, and CUBIC lands in the same low range, whereas BBR is not governed by loss and reaches a large share of the veth capacity. Plot cwnd from repeated `ss -ti` samples to see CUBIC’s sawtooth against BBR’s flat, model-based window; set loss to 0 and watch the two converge. Exact numbers depend on the machine.

CHECK YOURSELF

1. cwnd = 12 segments, rwnd = 8 segments. How many segments may be in flight? [1]
2. After a timeout at cwnd = 30, what are ssthresh and cwnd? After three duplicate ACKs at cwnd = 30 under Reno? [2]
3. A path has 200 Mb/s and RTT 80 ms. What is its BDP, in bytes? [3]
4. Name the signal each of Reno, CUBIC and BBR uses. [4]

3.10 The application layer: DNS, HTTP, TLS and QUIC SHOULD

· ■■□

The application layer is where users' programs meet the network. This section covers the four protocols that every web request touches: DNS to find the address, HTTP to ask for the page, TLS to protect it and QUIC to carry it faster. Google, Microsoft and Amazon treat it as a Must; for the other companies it is background for the capstone that follows.

> BIG IDEA
>
> DNS turns names into addresses through a hierarchy of servers that cache their answers, and the HTTP, TLS and transport protocols beneath a web page have been reorganised, step by step, to need fewer round trips.

DNS

The *Domain Name System* is a distributed, hierarchical database that maps names to data, chiefly addresses. The hierarchy is read right to left: the *root* (written as a final dot) delegates to *top-level domains* such as `com.`, which delegate to organisations such as `google.com.`, whose own servers hold the records. Three roles take part. A *stub resolver*, inside every host, sends a single question to a configured resolver (the address is normally supplied by DHCP). A *recursive resolver*, run by the ISP, a company or a public service such as 8.8.8.8 or 1.1.1.1, does the work of finding the answer and caches it. An *authoritative server* holds the records of a zone and answers for it. The client's query is *recursive* ("find it for me"); the resolver's queries to the hierarchy are *iterative* ("tell me who to ask next"), because each server replies with an answer or a *referral*.

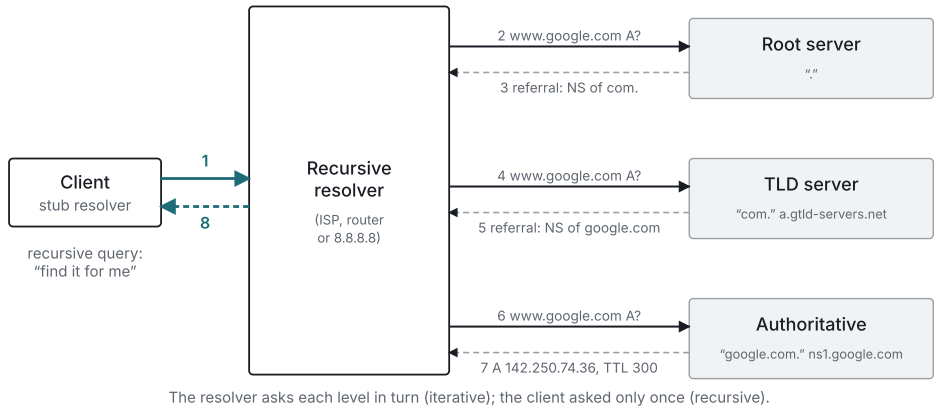


Figure 3.31 DNS resolution of www.google.com on a cold cache. Solid arrows are queries, dashed arrows replies. The resolver caches every reply for its TTL, so the next client is answered at step 1 without steps 2 to 7. The A record is an example value.

Every record has a type and a *time to live* (TTL), the number of seconds that a cache may keep it. The types to know are in the table. Caches also store failures: a *negative answer* (the name does not exist) is cached for a time derived from the zone's SOA record, which saves repeated queries for mistyped names.

Type	Holds	Note
A, AAAA	IPv4, IPv6 address	The usual answer

1. min(12, 8) = 8 segments.↩
2. Timeout: ssthresh = 15, cwnd = 1. Reno on three duplicate ACKs: ssthresh = 15, cwnd = 15 after recovery (18 during it).↩
3. 200 × 10^6 × 0.08 / 8 = 2 MB.↩
4. Reno and CUBIC react to loss (CUBIC with a cubic growth law after the cut); BBR uses measured bandwidth and minimum RTT.↩

CNAME	Alias to another name	Cannot coexist with other records on the same name
NS	Names of a zone's authoritative servers	Used in referrals
MX	Mail servers with a priority	
PTR	Name for an address	Reverse lookup, under in-addr.arpa
TXT	Free text	SPF, domain verification
SOA	Zone parameters, negative-cache time	One per zone
SRV, CAA	Service location; certificate authorities allowed	

DNS uses UDP port 53 for ordinary queries, because a question and answer fit in one datagram each. It falls back to TCP port 53 when an answer is too large and the server sets the truncation flag, and always for zone transfers between servers. The extension mechanism EDNS0 allows UDP answers beyond the original 512 bytes. DNSSEC adds digital signatures to records so that a resolver can detect forgery; DNS over TLS (port 853) and DNS over HTTPS (port 443) encrypt the queries. A CNAME cannot be placed at a zone apex such as `example.com.`, since the apex must also hold SOA and NS records, and a CNAME may not share its name with other records. Hosting providers therefore offer a flattened alias (ALIAS or ANAME), which the authoritative server resolves itself and returns as ordinary A records.

WORKED EXAMPLE

Follow `dig +trace www.google.com`, then watch the cache.

1. The resolver starts with the root hints and asks a root server. The reply is a referral: the NS records of `com.`, with a TTL of 172800 s (two days), such as `a.gtld-servers.net`.
2. It asks a `com.` server, which refers it to the NS records of `google.com.` (`ns1.google.com` and others), again with a TTL of 172800 s.
3. It asks `ns1.google.com`, which is authoritative and returns the A record, with a short TTL such as 300 s, because Google may change it often.
4. Query `dig www.google.com @8.8.8.8` twice, ten seconds apart: the first answer shows TTL 300 or less, and the second shows about 10 less: the resolver is serving its cache and decrementing the TTL.

Result: three iterative steps for a cold cache, none for a warm one. The answer is cached for 300 s; the NS records are cached for two days, so later lookups under `.com` skip the root and TLD steps.

HTTP

The *Hypertext Transfer Protocol* is a request-response protocol over TCP (or QUIC). A request names a *method* (GET fetches, POST submits, PUT replaces, DELETE removes, HEAD fetches headers only), a path and headers; the response carries a *status code* and a body. The code classes are 1xx informational, 2xx success (200), 3xx redirection (301 permanent, 302 temporary, 304 not modified), 4xx client error (400, 401, 403, 404, 429 too many requests) and 5xx server error (500, 502 bad gateway, 503 unavailable). HTTP/1.0 opened a new TCP connection per request. HTTP/1.1 keeps the connection open (*persistent connections*), but answers requests in order, so browsers open six or so connections per site.

HTTP/2 sends binary *frames* belonging to numbered *streams*, so that many requests share one TCP connection, interleaved, and compresses the headers (HPACK). It fixes the blocking at the HTTP level but not at the TCP level: a lost segment holds back every stream behind it. *HTTP/3*

therefore runs over *QUIC*, a transport built on UDP (port 443) in which each stream is delivered independently, loss recovery and congestion control run in the application, TLS 1.3 is part of the handshake, and a *connection ID* lets a connection survive a change of IP address, as when a phone moves from Wi-Fi to cellular. A client that has talked to the server before may send data in the first flight (*0-RTT*), at the price of replay risk for that data.

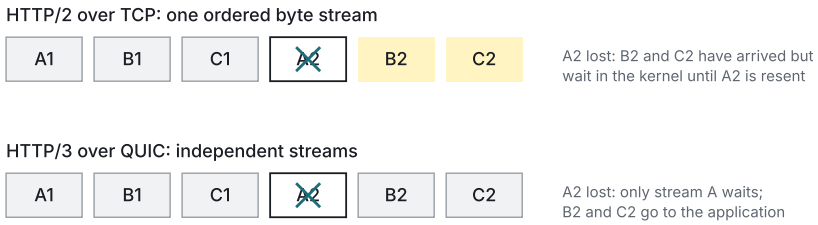


Figure 3.32 Head-of-line blocking. Three streams (A, B, C) share one connection, and the second packet of stream A is lost. Yellow: delivered late for no fault of its own.

TLS, in one paragraph

TLS encrypts and authenticates the connection. The client sends its supported versions and ciphers, a key share and the *server name indication* (SNI), which names the site wanted so that one address can serve many; the server answers with its key share and a *certificate* signed by a certificate authority that the client trusts, proving that the key belongs to the name, and both sides derive the session keys from the key shares. TLS 1.3 completes this in one round trip, whereas TLS 1.2 needs two. The details, certificate chains and attacks are treated in Pillar 9, Section 9.4; here only the round-trip count matters, because it adds to the time to first byte, as Section 3.11 shows.

Other application protocols appear in questions about ports. They are collected below, and the ports are repeated in the Revision Spread.

Protocol	Port	Purpose
SMTP; POP3; IMAP	25 (587 submission); 110; 143	Send mail; fetch and delete; synchronised mailbox
FTP	21 control, 20 data (active)	Active: the server connects back to the client. Passive: the client opens the data connection, which works through NAT
SSH; Telnet	22; 23	Encrypted remote shell; the same in clear text
SNMP; NTP	161/162; 123 (UDP)	Monitoring; time synchronisation

> INTERVIEW LENS
>
> GOOGLE · MICROSOFT · AMAZON
>
> The standard questions are recursive versus iterative DNS with TTLs and negative caching (Q3.46), CNAME versus A versus ALIAS (Q3.47), TLS 1.2 versus 1.3 with SNI (Q3.48) and head-of-line blocking in HTTP/2 versus HTTP/3 (Q3.49). Amazon adds operations: “which record would you change to move a site, and when will users see it?” (answer: the A record, after the old TTL expires; lower the TTL beforehand).

> LAB
>
> `LAB 3.10 · 25 MIN` Run the trace, then capture the protocols.

```
$ dig +trace www.google.com | grep -v '^;' | grep -E 'NS|A\s'
.            518400  IN  NS  a.root-servers.net.
com.         172800  IN  NS  a.gtld-servers.net.
google.com.  172800  IN  NS  ns1.google.com.
www.google.com. 300  IN  A   142.250.74.36
$ dig www.google.com @8.8.8.8 +noall +answer     # run twice, 10 s apart
www.google.com.  253  IN  A  142.250.74.36
$ curl -sI --http2 https://www.google.com | head -1
HTTP/2 200
```

What you should notice: the TTL falls between the two queries. In Wireshark, the filters `dns` and `tls.handshake.type == 1` show the query and the ClientHello, whose SNI extension holds the host name in clear text unless Encrypted Client Hello is used. For TLS decryption set the environment variable `SSLKEYLOGFILE` before starting the browser, and give the file to Wireshark under *Preferences → Protocols → TLS*. The addresses and TTLs are examples and change over time.

CHECK YOURSELF

1. A resolver has just cached `www.example.com` with TTL 600. The authoritative server changes the record. When do clients using that resolver see the change at the latest? [1]
2. Why can a resolver's reply to a large query arrive over TCP? [2]

3.11 Capstone: what happens when you type google.com and press Enter MUST · ■■□

This is the most commonly asked networking question in the book, because every concept in the pillar occurs in it. It is a poor question for a memorised monologue, since the interviewer will interrupt at the point where one is weakest. We therefore give a scaffold of eleven steps, then three versions of the answer at different depths, then the interruptions to expect.

BIG IDEA

A strong answer is ordered, layered and quantified: it follows the packets in time, names the protocol and the address at each step, and says roughly how long each step takes.

The eleven steps

1. **Parse and local checks.** The browser splits the URL into scheme, host and path, consults its HSTS list (a site that demands HTTPS is upgraded before any request), and looks in its own DNS cache, the operating system's cache and the hosts file.
2. **DNS.** If the name is unknown, the stub resolver sends a UDP query to port 53 of the resolver learnt from DHCP. The resolver, if its cache is cold, resolves iteratively through root, `com.` and `google.com.` servers (Section 3.10) and returns the A and AAAA records.
3. **ARP (or NDP).** The destination is off-subnet, so the frame goes to the default gateway; if its MAC address is not cached the host broadcasts an ARP request first (Section 3.6).
4. **TCP.** A three-way handshake to port 443 (about one round trip); or, if the browser knows that the site speaks HTTP/3, a QUIC handshake over UDP 443.
5. **TLS 1.3.** ClientHello with SNI, ServerHello with key share, certificate validation against trusted authorities, Finished messages: one more round trip.

6. **HTTP.** The browser sends `GET / HTTP/2`. Google's address is *anycast*, so the packets reach the nearest front end, which may answer `301` or `200`.
7. **At every router:** the TTL is decremented and the checksum updated, a longest-prefix match chooses the next hop, and the Ethernet header is rewritten (Section 3.8). At the home router, PAT rewrites the source address and port (Section 3.6).
8. **Inside Google:** the front end terminates TLS and forwards to a load balancer and then to backend servers (Pillar 5).
9. **The response** returns by the reverse path, through the NAT table to the laptop; the sender's initial window of about ten segments determines how much arrives in the first round trip (Section 3.9).
10. **Rendering.** The browser parses the HTML and requests the subresources (images, scripts, styles), which reuse the open connection or open new streams.
11. **Reuse and teardown.** The connection stays open for later requests (keep-alive, HTTP/2 multiplexing) and is closed after an idle period with FIN or, for QUIC, with a close frame.

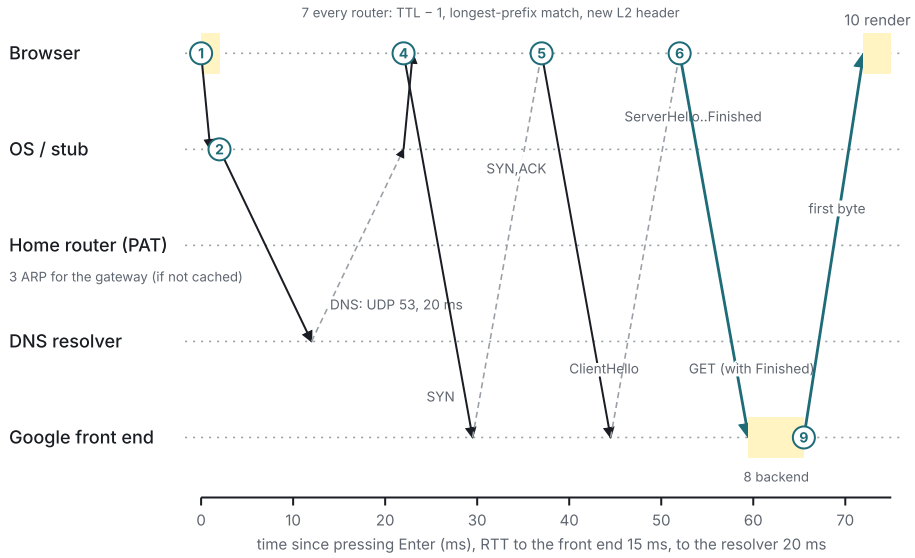


Figure 3.33 The google.com walk on a time axis (illustrative values). Numbers match the eleven steps of Section 3.11; steps 7 and 11 happen continuously. The accent arrows are the HTTP request and the first byte of the response.

Version 1: the 60-second answer

"The browser checks its caches and HSTS list, then DNS turns `www.google.com` into an IP address: my stub resolver asks the resolver given by DHCP, which asks root, `com.` and Google's servers if it has no cached answer. My host sees that the address is outside my subnet, so it ARPs for the default gateway and sends frames to its MAC address. A TCP handshake opens a connection to port 443, then a TLS handshake agrees keys and checks Google's certificate. The browser sends an HTTP GET; each router along the way decrements the TTL and forwards by longest-prefix match, rewriting the Ethernet header, and my home router translates my private address by PAT. Google's nearest front end answers, the reply comes back through the NAT table, and the browser renders the page, reusing the connection for the other files."

Version 2: the 5-minute answer

Add the following to the scaffold, in this order. Say that DNS usually asks for both A and AAAA, and that a browser prefers IPv6 but races both ("happy eyeballs") so that a broken IPv6 path costs

milliseconds, not seconds. At the link layer, name the frame fields: destination MAC is the gateway's, source MAC is the laptop's, and the IP header has TTL 64 and DF set. At TCP, give the sequence numbers' behaviour (random ISN, SYN consuming one number), the options in the SYN (MSS 1,460, window scale, SACK, timestamps) and the initial congestion window of 10 segments. At TLS, state that the client lists `h2` and `http/1.1` in ALPN, so the protocol is chosen inside the handshake, and that a 1.3 handshake costs one round trip. Finish with the numbers of the timeline: with a 15 ms round trip to the front end and 20 ms to a cold DNS resolver, DNS costs 20 ms, TCP 15 ms and TLS 15 ms, so the request leaves at about 50 ms and the first byte arrives at about 70 ms; QUIC saves one round trip by merging transport and TLS set-up, and a warm DNS cache saves nearly all of the first 20 ms.

WORKED EXAMPLE

Packet x-ray: the first TLS ClientHello leaving the laptop. Values are typical, with relative sequence numbers.

Layer	Fields
Ethernet II	dst 52:54:00:12:35:02 (gateway), src 52:54:00:12:34:56 (laptop), type 0x0800
IPv4	10.0.2.15 to 142.250.74.36, TTL 64, protocol 6, DF set, ID 0x1c46, total length 569
TCP	49822 to 443, flags PSH,ACK, seq 1, ack 1, window 502, header 32 bytes (timestamps)
TLS record	content type 22 (handshake), record version 0x0301, length 512
ClientHello	version 0x0303, 32 random bytes, ciphers (TLS_AES_128_GCM_SHA256 first), SNI `www.google.com`, ALPN `h2, http/1.1`, supported_versions 1.3, key_share x25519

1. Payload: the TLS record is 5 + 512 = 517 bytes.
2. TCP segment: 32 + 517 = 549 bytes; IP total length: 20 + 549 = 569.
3. Frame: 14 + 569 = 583 bytes (plus the 4-byte FCS on the wire).

Result: the whole frame holds only 517 bytes of the application's data. Note that the SNI is visible to anyone on the path.

Version 3: the 15-minute answer, with interruptions

At this depth, the interviewer interrupts. The answers below are the expected ones; each points to its home section.

"What if DNS is cached?" The query never leaves the machine (browser or OS cache), or stops at the resolver (about 1 ms on a home network), and the 20 ms of the example disappears. The TTL decides how long this lasts (Section 3.10).

"What if the gateway's MAC is unknown?" The host holds the packet, broadcasts an ARP request on the LAN, and sends the frame when the reply arrives; this adds a fraction of a millisecond. If nobody replies, ARP fails and the connection attempt reports "host unreachable" after a few seconds (Section 3.6).

"What if the certificate has expired, or names another host?" The handshake completes at the TCP level but the client aborts at validation with a fatal alert and the browser shows a warning; no HTTP is sent. For a site with HSTS the user cannot click through (Pillar 9).

"How does the reply find my laptop behind NAT?" The reply is addressed to the router's public address and port; the router looks up the port in its translation table, rewrites the destination to the laptop's private address and port, and sends the frame to the laptop's MAC (Section 3.6).

"What changes with HTTP/3?" Steps 4 and 5 merge into one QUIC round trip over UDP 443; streams are independent; and the browser learned of HTTP/3 from an Alt-Svc header or an HTTPS DNS record (Section 3.10).

"What if the first packet is bigger than the path MTU?" The ClientHello is small, but a large response is not: TCP segments of at most the MSS avoid it, and a blocked ICMP 3/4 would cause the black hole of Section 3.5.

"Why is the first page load slower than the second?" The first load pays for DNS, TCP and TLS set-up and a small congestion window; the second reuses the connection, whose window has grown (Section 3.9).

"Where does the load balancer sit?" After TLS termination, at the front end; the choice of backend is made by a layer-7 or layer-4 balancer (Pillar 5).

> INTERVIEW LENS
>
> GOOGLE · MICROSOFT · AMAZON · CISCO · JUNIPER · QUALCOMM
>
> Q3.45 is asked at nearly every company; Google, Microsoft and Amazon expect depth. The grading criteria are the order of events, the correct protocol and address at each step (MAC at the link, IP end to end, port at the transport layer) and numbers. The cheapest way to impress is to volunteer the failure cases yourself, in the order of the interruptions above.

> LAB
>
> `LAB 3.11 · 40 MIN` Record the whole sequence in one capture. Flush the caches first, then open the page.
>
> ```
> $ resolvectl flush-caches ; sudo ip neigh flush all     # Windows: ipconfig
> /flushdns ; arp -d *
> $ sudo tcpdump -nn -w walk.pcap -i any &  curl -sv https://www.google.com -o
> /dev/null
> $ tshark -r walk.pcap -Y 'arp or dns or tcp.flags.syn==1 or tls.handshake' -T
> fields -e frame.number -e _ws.col.Protocol -e _ws.col.Info
> ```
>
> Find the frame number of each step: (1) none, it is local; (2) the DNS query and response; (3) the ARP request and reply; (4) SYN, SYN-ACK, ACK; (5) ClientHello, ServerHello; (6) the HTTP/2 request; (9) the first data segment of the response; (11) FIN. **What you should notice:** the gaps between frames reproduce the timeline of the figure, scaled to your own network, and the ARP frames appear before the DNS query.

> CHECK YOURSELF
>
> 1. Which two steps of the walk would disappear on a second visit within a minute? [3]
> 2. At which step does the browser first learn the server's IP address? [4]

3.12 Where the topics are asked, and a mini-project OPTIONAL · ■■□

The table below shows how often each topic is reported at each company, from aggregated, self-reported interviews and role descriptions. It is an indication, not a guarantee. H marks a topic that is frequently reported or central to the role, M one that is sometimes asked, and L one that is rare.

The companies are Cisco (CSC), Qualcomm (QCM), Google (GOO), Microsoft (MSF), Amazon network development (AMZ), Juniper (JNP), Arista (ARI), Nokia (NOK) and Broadcom (BRC).

Topic	CSC	QCM	GOO	MSF	AMZ	JNP	ARI	NOK	BRC
OSI, TCP/IP, encapsulation	H	H	M	H	H	H	M	H	M
The google.com walk	H	M	H	H	H	M	M	M	L
Subnetting and VLSM by hand	H	M	M	M	H	H	H	H	M
Ethernet, MAC, switching	H	M	L	M	M	H	H	H	H
CRC, ARQ, sliding window	M	H	M	M	L	M	L	H	H
IPv4 header, fragmentation, MTU	H	H	M	M	H	H	H	H	H
ARP, DHCP, NAT	H	M	M	H	H	H	M	M	M
IPv6 and NDP	M	H	M	M	M	H	M	H	M
TCP handshake, states, TIME_WAIT	H	H	H	H	H	H	H	M	M
TCP congestion control	M	H	H	H	H	M	M	M	M
DNS, HTTP, TLS, QUIC	M	L	H	H	H	L	L	L	L
Routing algorithms	H	M	H	M	M	H	H	H	M

Qualcomm and Broadcom lean towards protocol-stack internals (ARQ, windows, headers, IPv6 for cellular); Google and Microsoft towards TCP behaviour and application performance; Amazon towards MTU, TCP and DNS operations; and Juniper, Arista and Nokia towards layer-2 and layer-3 forwarding.

WORKED EXAMPLE

Mini-project: PacketScope, a teaching packet dissector and subnet toolkit (a weekend). Build a Python command-line tool with three sub-commands, using only the standard library for the core.

1. `pscope subnet 172.16.45.200/21` prints the network, broadcast, range and wildcard, and draws the 32-bit ruler of Section 3.4 in ASCII.
2. `pscope parse capture.pcap` reads the file with `struct` and decodes Ethernet, 802.1Q, IPv4, IPv6, TCP, UDP, ICMP, ARP and DNS; it flags TCP handshakes and computes the round-trip time of each flow from SYN to SYN-ACK. A stretch goal is to detect retransmissions and duplicate ACKs, reproducing Wireshark's "TCP Analysis Flags".
3. `pscope cwnd log.txt` reads `ss -ti` samples taken during the Lab 3.9b runs and plots cwnd against time for CUBIC and BBR.

Result: a repository that exercises bit manipulation (Cisco, Arista), protocol parsing (Qualcomm, Broadcom) and TCP performance (Google, Amazon). Résumé bullet template: "Built a Python packet dissector that decodes N protocols from raw bytes and reports per-flow RTT and retransmissions; validated against Wireshark on M captures."

1. Within 600 seconds of the moment the resolver cached the old record.↩
2. The UDP answer did not fit, so the server set the

truncation flag and the resolver retried over TCP.↩

3. DNS (the answer is cached) and, if the connection is kept alive, the TCP and TLS handshakes; ARP is also cached.↩
4. At step 2, from the DNS answer.↩

Problem Set

Q3.1 Name the OSI layers and give one protocol and one device for each.

ALL · TECH-1 · ■□□ · S

30-second answer: 7 Application (HTTP; gateway), 6 Presentation (TLS), 5 Session (RPC), 4 Transport (TCP; firewall, L4 load balancer), 3 Network (IP; router), 2 Data link (Ethernet; switch, AP), 1 Physical (cable, 802.3 PHY; hub, repeater). PDUs: data, segment, packet, frame, bits.

Follow-ups. Where is ARP (between 2 and 3, carried in Ethernet)? Which layers does TCP/IP merge (5 to 7, and 1 to 2)? **Mistakes.** Calling a switch a layer-3 device (only an L3 switch is); placing TLS in the TCP/IP model without saying it sits between transport and application. Details: §3.2 (p. 165).

Q3.2 Hub, switch, router: collision and broadcast domains?

CISCO · NOKIA · TECH-1 · ■□□ · S

30-second answer: a hub is one collision domain and one broadcast domain for all its ports; a switch gives one collision domain per port but keeps one broadcast domain (per VLAN); a router gives each interface its own collision and broadcast domain.

Approach. Collision domain: who can collide on a shared medium. Broadcast domain: who hears a broadcast. **Answer.** Hub repeats bits to all ports, so everything shares one wire. Switch forwards per port by MAC, so full-duplex ports never collide, but floods broadcasts everywhere in the VLAN. Router does not forward broadcasts, so it ends the domain. For the counting exercise (2 broadcast, 8 collision domains) see Figure 3.2 in §3.1 (p. 163). **Mistake.** Saying a switch splits broadcast domains; VLANs do, not the switch itself.

Q3.3 Which IP header fields change at each router hop, and which never change?

CISCO · AMAZON · JUNIPER · TECH-1 · ■□□ · S

30-second answer: at every hop the TTL falls by one and the header checksum is recomputed, and the Ethernet source and destination MAC are rewritten. Source and destination IP, protocol and the TCP or UDP ports never change, unless NAT rewrites them.

Answer. Constant end to end: IP source and destination, protocol, identification, data. Changing per hop: TTL, checksum, and the layer-2 header and FCS. Changing only when something special happens: fragment fields (a router fragments), DSCP and ECN (a router re-marks or signals congestion), addresses and ports (NAT). **Mistake.** Saying the IP address changes and the MAC does not; the correct statement is the reverse (§3.6 (p. 183)).

Q3.4 Why is the minimum Ethernet frame 64 bytes? Derive it from CSMA/CD.

QUALCOMM · BROADCOM · TECH-1 · ■■□ · S

30-second answer: a sender must still be transmitting when a collision signal returns from the far end, so frame time must be at least twice the one-way delay: $L \geq 2T_pR$. For 10 Mb/s and 2,500 m, $2T_p = 25$ µs gives 250 bits; the standard uses 512 bits (64 bytes, 51.2 µs) to allow for repeaters and jam signal.

Approach and answer. $T_p = 2{,}500 / (2 \times 10^8) = 12.5$ µs; $2T_p \times 10$ Mb/s = 250 bits; round up to 512 bits. Payload is padded to 46 bytes, so that 14 + 46 + 4 = 64. **Follow-ups.** What changes at 1 Gb/s (carrier extension; full duplex has no collisions)? **Mistake.** Dividing by T_p, not $2T_p$. See §3.3 (p. 168).

Q3.5 How does a switch learn MAC addresses? What happens when the CAM table is full?

CISCO · ARISTA · BROADCOM · TECH-1 · ■■□ · S

30-second answer: it records the source MAC of every frame against the arrival port, forwards by the destination MAC, and floods broadcast, multicast and unknown unicast. When the table is full it cannot learn, so unknown destinations are flooded and the switch behaves like a hub, which an attacker exploits with MAC flooding.

Answer. Entries age out (300 s on Cisco) and are refreshed by traffic, so moves are relearned; a frame whose destination is on its arrival port is filtered. A full table does not drop traffic; it degrades to flooding, so the attacker sees other hosts' frames. **Mitigation.** Port security with a MAC limit per port, sticky addresses, and storm control (Pillar 9). **Mistake.** Saying that a full table drops frames. See §3.3 (p. 168).

Q3.6 Compute the CRC for data 1101011011 with generator 10011.

QUALCOMM · NOKIA · OA · ■■□ · S

30-second answer: append four zeros (the generator's degree), divide modulo 2, and the remainder is 1110; the transmitted word is 11010110111110, and the receiver's division gives remainder 0.

Approach. The register after each XOR, and the check, are worked out in the CRC example of §3.3 (p. 168). **Check.** 11010110111110 divided by 10011 leaves 0000, as verified by running the same loop on the full word. **Mistake.** Forgetting to append the zeros first, or using subtraction with borrows instead of XOR.

Q3.7 With 3-bit sequence numbers, what are the maximum Go-Back-N and Selective Repeat windows?

QUALCOMM · BROADCOM · TECH-1 · ■■□ · S

30-second answer: Go-Back-N $2^3 - 1 = 7$; Selective Repeat $2^{3-1} = 4$.

Reason. In GBN the receiver window is 1, so the sender window may use all but one of the numbers: if all 8 were outstanding, an ACK of 0 could mean "all 8 received" or "none received". In SR both windows have size W and must not overlap modulo 2^n, so $2W \le 2^n$. **Mistake.** Quoting 2^n for both.

Q3.8 Stop-and-Wait, 1 Mb/s, 1,000-bit frames, T_p = 20 ms: efficiency, and the window for 100%?

QUALCOMM · TECH-1 · ■■□ · S

30-second answer: T_t = 1 ms, a = 20, $\eta = 1/(1 + 2a) = 1/41 \approx 2.4\%$; a window of 41 frames gives 100%.

Follow-ups. Sequence bits: GBN needs 6, SR needs 7 (the full working is in §3.3 (p. 168)). If the frame error rate is p, Stop-and-Wait efficiency becomes $(1 - p)/(1 + 2a)$. **Mistake.** Using the round-trip time as T_p.

Q3.9 Cut-through versus store-and-forward: which is faster, which propagates errors, and why do data-centre switches offer cut-through?

ARISTA · BROADCOM · TECH-2 · ■■■ · T

30-second answer: cut-through is faster, because it starts forwarding after the destination MAC (latency of tens of nanoseconds, independent of frame size) whereas store-and-forward waits for the whole frame and its FCS (1,500 bytes cost 1.2 µs at 10 Gb/s per hop). The price is that cut-through forwards corrupted frames, which the receiver then drops. Data-centre switches use it because small latencies add up over several hops and matter to storage and HPC traffic.

Answer. Store-and-forward checks the FCS, can convert speeds and queue-shape, and never forwards a bad frame. Cut-through needs equal ingress and egress speed and an uncongested egress; otherwise the switch must buffer, and falls back. Many switches count FCS errors per port and fall back to store-and-forward above a threshold. **Mistake.** Saying cut-through checks the FCS.

Q3.10 Network, broadcast and usable range for 172.16.45.200/21, without a calculator.

CISCO · AMAZON · TECH-1 · ■□□ · S

30-second answer: block size 8 in the third octet, 45 lies in 40 to 47: network 172.16.40.0, broadcast 172.16.47.255, usable 172.16.40.1 to 172.16.47.254, 2,046 hosts.

Approach. Mask 255.255.248.0, block = 256 − 248 = 8; the full working and the wildcard (0.0.7.255) are in §3.4 (p. 174). **Mistake.** Writing 172.16.45.0 as the network address.

Q3.11 How many usable hosts on a /27 and a /31? Why is /31 allowed?

CISCO · JUNIPER · TECH-1 · ■□□ · S

30-second answer: /27 has 5 host bits, $2^5 - 2 = 30$ usable. A /31 has 2 addresses and both are usable, because RFC 3021 allows a point-to-point link to dispense with the network and broadcast addresses.

Answer. A /32 holds one address and names a host. The saving matters: a /31 per link uses half the addresses of a /30. Not all equipment supports /31 on all interface types, so verify before deploying. **Mistake.** Saying that /31 has zero hosts.

Q3.12 VLSM plan from 192.168.10.0/24 for LANs of 100, 50 and 25 hosts and two WAN links.

CISCO · NOKIA · TECH-1 · ■■□ · S

30-second answer: /25 at .0, /26 at .128, /27 at .192, then /30 at .224 and /30 at .228, leaving .232/29 and .240/28 spare.

Approach. Largest first, each on a multiple of its block size. The usable range of each is in §3.4 (p. 174): .1 to .126, .129 to .190, .193 to .222, .225 and .226, .229 and .230. **Mistake.** Starting with the small subnets, which leaves no aligned room for the /25.

Q3.13 Summarise 10.1.4.0/24 to 10.1.7.0/24. Can 10.1.5.0 to 10.1.8.0 become one /22?

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: 10.1.4.0/22. No: the third octets 5 to 8 share only four bits, so one summary would be 10.1.0.0/20, which over-claims 12 other /24s; use 10.1.5.0/24, 10.1.6.0/23 and 10.1.8.0/24.

Approach. Binary alignment: the block must start at a multiple of its size, and contain only the required networks. 4 to 7 is a four-block range starting at a multiple of 4; 5 to 8 is not. See §3.4 (p. 174).

Q3.14 Write `in_subnet(ip, cidr)` using only bitwise operations.

CISCO · GOOGLE · ARISTA · TECH-1 · ■■□ · S

30-second answer: build the mask by shifting, and compare the two addresses under the mask: `(ip & mask) == (net & mask)`. Guard the prefix 0 case, because shifting a 32-bit value by 32 is undefined in C.

```c
#include <stdint.h>
#include <stdio.h>
int parse(const char *s, uint32_t *o) {
  unsigned a, b, c, d;
  if (sscanf(s, "%u.%u.%u.%u",
             &a, &b, &c, &d) != 4
      || (a|b|c|d) > 255) return -1;
  *o = a << 24 | b << 16 | c << 8 | d;
  return 0;
}
int in_subnet(uint32_t ip,
              uint32_t net, int p) {
  uint32_t m = p ? ~0u << (32 - p) : 0;
  return (ip & m) == (net & m);
}
```

Complexity. O(1). **Follow-ups.** Return the network and broadcast: `net & mask` and `net | ~mask`. Support IPv6 by comparing 16 bytes with a partial last byte. **Mistake.** Using signed integers (right shifts then sign-extend), or string comparison.

Q3.15 Fragment a 4,000-byte datagram for MTU 1,500: offsets and MF flags.

QUALCOMM · BROADCOM · JUNIPER · TECH-1 · ■■□ · S

30-second answer: three fragments with data 1,480, 1,480 and 1,020 bytes, Total Length 1,500, 1,500 and 1,040, offset field 0, 185 and 370 (in 8-byte units), MF 1, 1 and 0.

Approach. Data per fragment = 1,500 − 20 = 1,480, a multiple of 8. The table and the second-link variant (MTU 576) are in §3.5 (p. 179). **Mistake.** Writing 1,480 in the offset field.

Q3.16 MTU versus MSS: what is the MSS over PPPoE, and what does `ip tcp adjust-mss` do?

AMAZON · CISCO · TECH-1 · ■■□ · S

30-second answer: MTU is the largest IP packet on a link; MSS is the largest TCP payload, MTU − 40 for IPv4. PPPoE has MTU 1,492, so MSS = 1,452. `ip tcp adjust-mss 1452` makes the router rewrite the MSS option in transit SYNs, so that endpoints never send segments that exceed the link, which avoids the stalls of a broken PMTUD.

Answer. Each endpoint advertises an MSS from its own interface MTU and cannot know of a smaller link in the middle; clamping fixes that without relying on ICMP. IPv6 subtracts 60 (40 + 20). **Mistake.** Including the Ethernet header in the MTU.

Q3.17 Scenario: pages do not load, though ping and the TCP handshake work. What is happening, and how do you prove it?

AMAZON · MICROSOFT · CISCO · TECH-2 · ■■■ · T

30-second answer: a path-MTU black hole. Small packets pass, but a full-size packet with DF set is too big for a link in the path, and the ICMP "fragmentation needed" message is blocked, so the sender keeps resending the same large segment. Prove it with DF-set pings of decreasing size, and a capture that shows repeated retransmissions and no ICMP type 3 code 4.

Approach. (i) `ping -M do -s 1472 host` fails but `-s 1372` succeeds: bisect for the largest working size. (ii) Capture at the client: SYN, SYN-ACK and the request succeed; the response's large segments never arrive or arrive after retransmissions; no ICMP 3/4. (iii) Locate it with `tracepath` or by pinging each hop with DF. (iv) Fix: permit ICMP 3/4 at firewalls; clamp MSS at the tunnel or PPPoE edge; lower the interface MTU. **Mistakes.** Blaming DNS or the server; leaving ICMP blocked "for security". See Figure 3.14 and §3.5 (p. 179).

Q3.18 How does traceroute work on Linux and on Windows? Why `* * *`?

AMAZON · CISCO · TECH-1 · ■■□ · S

30-second answer: it sends probes with TTL 1, 2, 3 and so on, and each router that decrements the TTL to zero returns ICMP time exceeded, revealing itself. Linux sends UDP probes to ports from 33434 and the destination answers ICMP port unreachable; Windows `tracert` sends ICMP echo requests and the destination answers echo reply. `* * *` means no reply arrived in time.

Answer. Reasons for stars: the router rate-limits or suppresses ICMP, a firewall drops probes of this type (try `traceroute -I` or `-T -p 443`), the return path differs, or the destination ignores the probe. A star in the middle with replies afterwards is harmless. **Mistake.** Reading a star as an outage. See §3.5 (p. 179).

Q3.19 What is APIPA (169.254.0.0/16), and what does it tell you about DHCP?

MICROSOFT · CISCO · TECH-1 · ■□□ · S

30-second answer: a host that gets no DHCP reply gives itself a link-local 169.254.x.y address, after probing with ARP that it is unused. Seeing it means the host could not reach a DHCP server: check the cable and VLAN, the relay (`ip helper-address`), the pool (exhausted?) and the server.

Mistake. Treating it as a working configuration: such a host can talk only to others on its own link.

Q3.20 Explain DHCP DORA with ports and addresses. Why is Discover a broadcast?

CISCO · MICROSOFT · TECH-1 · ■□□ · S

30-second answer: Discover (client 0.0.0.0:68 to 255.255.255.255:67), Offer, Request (broadcast, names the chosen server and the address wanted), Acknowledgement (carries mask, gateway, DNS, lease). Discover is a broadcast because the client has no address and does not know any server.

Answer. Offer and Ack may be unicast to the client MAC or broadcast, depending on the broadcast flag. The Request is broadcast too, so that losing servers learn that their offer was declined. Renewal: unicast at T1 (50%), broadcast at T2 (87.5%). The relay case is in §3.6 (p. 183).

Q3.21 DHCP server on another subnet: how does it work, and what is giaddr?

CISCO · JUNIPER · TECH-2 · ■■□ · S

30-second answer: a relay agent on the client's router (`ip helper-address`) turns the broadcast into a unicast to the server and fills giaddr with the address of the interface the request arrived on. The server uses giaddr to select the pool for that subnet and replies to giaddr; the relay delivers the reply to the client.

Answer. Addresses: Discover 0.0.0.0 to 255.255.255.255; relayed copy relay-IP to server-IP, UDP 67 to 67. Option 82 can add circuit identifiers. **Mistake.** Forgetting that without giaddr the server cannot know which subnet to allocate from. See Figure 3.17.

Q3.22 What is gratuitous ARP, and where is it used?

CISCO · ARISTA · TECH-2 · ■■□ · T

30-second answer: an ARP message in which a host announces its own address (sender and target IP equal). It detects duplicate addresses, refreshes neighbours' caches after a change, and is sent by a new active router in first-hop redundancy protocols (HSRP, VRRP) or by a failed-over server so that switches and hosts relearn the virtual address.

Mistake. Thinking it requests anything; no reply is expected. If one arrives, the address is a duplicate. Security: it is also the tool of ARP spoofing (Pillar 9).

Q3.23 Walk a packet from PC-A to a web server through a PAT router. Give MAC, IP and port at each hop.

CISCO · AMAZON · GOOGLE · TECH-1 · ■■□ · S

30-second answer: PC-A ARPs for its gateway; the frame goes AA:AA to 11:11 with 192.168.1.10:51000 to 198.51.100.80:80. R1 applies PAT, so the second leg is 12:12 to 21:21 with 203.0.113.2:51000 to the same destination; the third is 22:22 to BB:BB with the same IPs. MACs change at every link, IPs only at the NAT.

Answer. The complete walk, the NAT table row and the return path are in §3.6 (p. 183). **Follow-ups.** Server on the same subnet: no gateway, ARP for the server. Two hosts with the same source port: R1 assigns another global port. **Mistake.** Reversing the roles of MAC and IP.

Q3.24 Scenario: two hosts behind different NATs need peer-to-peer VoIP. Why does it fail, and how do STUN, TURN and ICE help?

QUALCOMM · MICROSOFT · TECH-2 · ■■■ · T

30-second answer: each NAT drops unsolicited inbound packets, since it has no mapping for them, and each host knows only its private address. STUN lets a host learn its public mapping from a server; both sides exchange these and send simultaneously (hole punching), which works for most NATs. TURN relays media through a server when punching fails (symmetric NATs). ICE gathers all candidates, tests the pairs and picks the best.

Answer. Candidate types: host (private), server-reflexive (from STUN), relayed (TURN). A signalling channel (SIP, WebRTC) carries the candidates. Carrier-grade NAT often forces TURN. IPv6 removes NAT but not firewalls. **Mistake.** Forgetting that TURN costs bandwidth, which is why it is the last resort.

Q3.25 Proxy ARP: what is it, and what can it hide?

CISCO · TECH-2 · ■■□ · T

30-second answer: a router answers ARP requests for addresses it can route to, with its own MAC, so a host that believes a remote address is local still reaches it. It hides wrong masks and missing default gateways, inflates ARP tables, and breaks when the router fails; it is usually disabled (`no ip proxy-arp`).

Q3.26 Compress 2001:0DB8:0000:0000:0000:FF00:0042:8329

CISCO · NOKIA · TECH-1 · ■□□ · S

30-second answer: 2001:db8::ff00:42:8329.

Rules. Drop leading zeros in each group; replace the longest run of all-zero groups by `::`, once only; on a tie, the leftmost run. Hence `2001:db8:0:0:1:0:0:1` becomes `2001:db8::1:0:0:1`. **Mistake.** Using `::` twice.

Q3.27 Derive the EUI-64 link-local address from MAC 00:1A:2B:3C:4D:5E.

CISCO · JUNIPER · TECH-1 · ■■□ · S

30-second answer: fe80::21a:2bff:fe3c:4d5e. Insert FF:FE in the middle of the MAC and flip bit 7 (U/L) of the first byte, so 00 becomes 02.

Steps. 00:1A:2B | FF:FE | 3C:4D:5E gives 001A:2BFF:FE3C:4D5E; flipping gives 021A:2BFF:FE3C:4D5E; prefix fe80::/64. **Mistake.** Flipping the last bit, or not flipping at all. See §3.7 (p. 187).

Q3.28 How does IPv6 replace ARP? What is a solicited-node multicast address?

QUALCOMM · NOKIA · JUNIPER · TECH-1 · ■■□ · S

30-second answer: by NDP, using ICMPv6 neighbour solicitation (135) and neighbour advertisement (136). The solicitation goes to the solicited-node multicast address, ff02::1:ff plus the last 24 bits of the target address, so that only the few hosts sharing those bits are interrupted.

Example. For fe80::21a:2bff:fe3c:4d5e the group is ff02::1:ff3c:4d5e, and the Ethernet multicast MAC is 33:33:ff:3c:4d:5e. The reply is unicast and carries the MAC. The same mechanism runs duplicate address detection. **Mistake.** Saying that IPv6 broadcasts.

Q3.29 SLAAC versus DHCPv6. What do the RA M and O flags mean?

CISCO · QUALCOMM · TECH-2 · ■■□ · T

30-second answer: in SLAAC the host builds its address from the advertised /64 prefix and its own interface identifier, needing no server. M = 1 tells the host to use stateful DHCPv6 for the address; O = 1 tells it to use stateless DHCPv6 for other settings such as DNS, while still using SLAAC for the address.

Answer. The prefix option has its own A flag, which permits SLAAC. RDNSS in the RA can also carry DNS, making DHCPv6 unnecessary. DHCPv6 gives no default gateway; only the RA does. **Mistake.** Expecting DHCPv6 to provide the gateway.

Q3.30 Why don't IPv6 routers fragment? What breaks if ICMPv6 is blocked?

AMAZON · GOOGLE · TECH-2 · ■■■ · T

30-second answer: fragmenting costs router time, so only the source may fragment, using an extension header, and it learns the path MTU from ICMPv6 "packet too big" (type 2). If ICMPv6 is blocked, PMTUD fails with the black hole of Q3.17, and neighbour discovery, router advertisements and duplicate address detection stop, so the link does not work at all.

Answer. Every link must support 1,280 bytes, which is the safe size to use when PMTUD is uncertain. Firewalls must permit types 1 to 4, 128 and 129 (echo) as needed, and 133 to 137 on the local link. **Mistake.** Copying an IPv4 rule that drops all ICMP.

Q3.31 TCP versus UDP: three applications of each, with reasons.

ALL · TECH-1 · ■□□ · S

30-second answer: TCP for correctness and ordering: web (HTTP/1.1 and 2), email, file transfer, SSH. UDP when late data is useless or the application recovers itself: DNS, voice and video, online games, QUIC.

Answer. TCP: connection set-up, reliable ordered stream, flow and congestion control. UDP: no state, no delay, multicast and broadcast, and the ability to build a custom protocol (QUIC reimplements reliability, streams and congestion control in user space). The comparison table is in §3.9 (p. 192). **Mistake.** Saying that UDP is "unreliable so it is bad".

Q3.32 Draw the three-way handshake with ISNs 1000 and 5000.

ALL · TECH-1 · ■□□ · S

30-second answer: SYN seq 1000; SYN,ACK seq 5000 ack 1001; ACK seq 1001 ack 5001. States: client SYN_SENT then ESTABLISHED; server LISTEN, SYN_RCVD then ESTABLISHED.

Follow-ups. After 500 bytes from the client: seq 1001, ack from the server 1501; a FIN takes the number 1501 and is acknowledged with 1502. Both SYN and FIN consume one number; a pure ACK consumes none. The diagram is in Figure 3.24.

Q3.33 Why is the handshake three-way and not two-way?

GOOGLE · MICROSOFT · QUALCOMM · TECH-1 · ■■□ · S

30-second answer: each side must prove it received the other's initial sequence number. With two messages the server cannot tell whether its SYN,ACK arrived, and cannot distinguish a new SYN from a delayed duplicate of an old one, so it would hold state for connections nobody wants. The third segment confirms both.

Answer. Four messages are unnecessary, since the server's ACK and SYN travel together. Random ISNs complete the defence against old duplicates and blind injection. **Mistake.** Saying that "it saves a message"; the point is mutual confirmation.

Q3.34 Explain TIME_WAIT: why 2 MSL, which side enters it, and what happens on a busy proxy?

GOOGLE · AMAZON · MICROSOFT · TECH-1 · ■■□ · S

30-second answer: the side that closes first (the active closer) waits 2 MSL (60 s on Linux) so that it can re-acknowledge a retransmitted FIN if its last ACK was lost, and so that stray segments of the old connection die before the same 4-tuple is reused. A proxy that opens many short connections to one backend accumulates TIME_WAIT sockets and can exhaust ephemeral ports.

Answer. With about 28,000 ephemeral ports and 60 s, one source IP can make about 470 new connections per second to one destination. Remedies: persistent connections and pooling, more source IPs or ports, `tcp_tw_reuse` for outgoing connections (needs timestamps), and letting the client close first. Do not shorten 2 MSL or use the removed `tcp_tw_recycle`, which broke NAT users. **Mistake.** Saying that the client always has TIME_WAIT. See §3.9 (p. 192).

Q3.35 A server shows thousands of CLOSE_WAIT sockets. Whose bug is it?

MICROSOFT · AMAZON · TECH-1 · ■■□ · S

30-second answer: the server application's. CLOSE_WAIT means that the peer sent a FIN, the kernel acknowledged it, and the application has not called `close()` on its socket; so the program leaks sockets, usually on an error path or a missing close of a pooled connection.

Approach. `ss -tanp state close-wait` to find the process; check the code for paths that skip `close()`, such as exceptions, timeouts and abandoned connections; look at file-descriptor use. No kernel or network tuning fixes it, and the leak ends when the process exits. **Mistake.** Confusing it with TIME_WAIT.

Q3.36 Flow control versus congestion control: which window is which, and what is the effective send window?

QUALCOMM · GOOGLE · AMAZON · TECH-1 · ■■□ · S

30-second answer: flow control protects the receiver's buffer and uses the advertised window rwnd, carried in every segment. Congestion control protects the network and uses cwnd, which the sender maintains from loss or delay signals. The sender may have min(rwnd, cwnd) bytes in flight.

Answer. rwnd changes with the application's reads; cwnd changes with ACKs and losses. The binding window is the smaller. **Mistake.** Believing that cwnd is sent in a header; it is local to the sender. See §3.9 (p. 192).

Q3.37 Trace cwnd: initial 1, ssthresh 16, loss at cwnd 20. Tahoe, and Reno on three duplicate ACKs.

QUALCOMM · MICROSOFT · TECH-1 · ■■□ · S

30-second answer: slow start 1, 2, 4, 8, 16, then +1 per RTT 17, 18, 19, 20. At the loss ssthresh = 10. Tahoe sets cwnd = 1 and runs slow start (1, 2, 4, 8, 10, 11, …); Reno on three duplicate ACKs sets cwnd = 10 (13 while in recovery) and continues 11, 12, ….

Follow-ups. A timeout in Reno gives cwnd = 1, like Tahoe. NewReno handles several losses in one window. The plot is in Figure 3.29.

Q3.38 CUBIC versus BBR: which signal does each use, and why can BBR v1 be unfair to CUBIC?

GOOGLE · MICROSOFT · AMAZON · TECH-2 · ■■■ · T

30-second answer: CUBIC reacts to loss and grows its window as a cubic function of the time since the last loss. BBR builds a model from the measured bottleneck bandwidth and minimum RTT, paces at that rate and keeps about one BDP in flight, so it does not need loss. BBR v1 ignores loss, so against CUBIC in a shallow buffer it keeps sending while CUBIC backs off, and takes more than its share; v2 and v3 respond to loss and ECN.

Answer. BBR keeps queues short and tolerates random loss; CUBIC fills the buffer, which causes bufferbloat but is simple and robust. In deep buffers BBR v1 can instead lose share, because its inflight cap is about 2 × BDP. **Mistake.** Saying that BBR "is better": it depends on loss pattern and buffer depth. See §3.9 (p. 192).

Q3.39 Maximum throughput for rwnd 64 KB and RTT 100 ms. How do you fill a 1 Gb/s, 100 ms path?

GOOGLE · AMAZON · TECH-2 · ■■■ · T

30-second answer: throughput ≤ window / RTT = 65,535 × 8 / 0.1 = 5.24 Mb/s. The BDP of a 1 Gb/s, 100 ms path is 12.5 MB, so use window scaling with shift 8 (16.8 MB) and receive and send buffers of that size.

Answer. The shift is 9 for RTT 190 ms (BDP 23.75 MB; Mumbai to Virginia). Also: large enough `tcp_rmem` and `tcp_wmem` maxima, a congestion algorithm that survives a little loss at this rate (CUBIC, BBR), or several parallel streams. **Mistake.** Raising the link speed and expecting more throughput. See §3.9 (p. 192).

Q3.40 Explain fast retransmit and SACK. Why three duplicate ACKs, not one?

QUALCOMM · GOOGLE · TECH-1 · ■■□ · S

30-second answer: when three duplicate ACKs arrive, the sender resends the missing segment immediately instead of waiting for the RTO. SACK lets the receiver list the blocks it already holds, so that the sender repairs several losses per round trip and resends only what is missing. One or two duplicates can result from mere reordering, so three avoids spurious retransmissions at a small cost in delay.

Answer. On small windows there may never be three duplicates, and then only the timeout helps (tail loss probes and RACK address this). See Figure 3.27.

Q3.41 Nagle and delayed ACK: why do interactive applications see 40 to 200 ms stalls, and how are they fixed?

GOOGLE · MICROSOFT · TECH-2 · ■■■ · T

30-second answer: Nagle holds a small segment while earlier data is unacknowledged; the receiver delays its ACK for up to 40 to 200 ms hoping to piggyback it; each waits for the other. The stall appears in write-write-read patterns. Fix by setting `TCP_NODELAY`, by sending the message in one write (`writev`), or by `TCP_QUICKACK` on the receiver.

Answer. Nagle helps bulk and telnet-style traffic by reducing small packets, so do not disable it blindly: batch writes first. **Mistake.** Blaming the network.

Q3.42 What is a SYN flood? How do SYN cookies work?

CISCO · AMAZON · MICROSOFT · TECH-1 · ■■□ · S

30-second answer: an attacker sends many SYNs, usually with forged sources, filling the server's half-open queue so that real clients are refused. With SYN cookies the server keeps no state: it encodes a hash of the 4-tuple, a secret and a timestamp (plus an MSS code) in its initial sequence number, and rebuilds the connection only when a valid ACK returns that number plus one.

Answer. Cost: some SYN options such as window scale may be lost unless encoded in the timestamp. Other defences: bigger backlog, shorter SYN-ACK retries, rate limiting and upstream filtering. See §3.9 (p. 192).

Q3.43 Coding: simulate a sliding-window sender. Given a stream of ACKs and timeouts, output which segments are retransmitted.

GOOGLE · QUALCOMM · TECH-2 · ■■■ · T

30-second answer: keep `base` (oldest unacknowledged) and `next` (next to send). A cumulative ACK for k advances `base` to k and releases new segments up to `base + N`; a repeated ACK of `base` three times, or a timer expiry, retransmits segment `base`.

```c
#include <stdio.h>
/* ev[i] >= 0: cumulative ACK k (segments < k received); ev[i] == -1: timer expiry */
void run(int total, int N, const int *ev, int n) {
    int base = 0, next = 0, dup = 0;
    while (next < base + N && next < total) printf("send %d\n", next++);
    for (int i = 0; i < n; i++) {
        if (ev[i] == -1) {
/* RTO: resend the oldest segment */
            if (base < next) printf("retransmit %d (timeout)\n", base);
        } else if (ev[i] > base) {
/* new data acknowledged */
            base = ev[i]; dup = 0;
            while (next < base + N && next < total) printf("send %d\n", next++);
        } else if (ev[i] == base && base < next && ++dup == 3) {
            printf("retransmit %d (fast)\n", base);
        }
    }
}
```

Complexity. O(total + events). **Follow-ups.** Go-Back-N would resend `base .. next - 1` on timeout; Selective Repeat needs a per-segment ACK flag; the timer restarts whenever `base` advances. **Mistakes.** Counting duplicates across new ACKs (reset `dup` when `base` moves), and sending beyond `base + N`.

Q3.44 When does a TCP stack send RST?

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: when a segment arrives for which no connection exists, which includes a SYN to a closed port; when an application aborts (closing with unread data, or a zero linger time); when a firewall or IPS injects one to kill a flow; and when a segment violates the connection state, such as data after the peer has restarted.

Answer. An RST needs no acknowledgement and the connection is dropped at once. UDP has the analogue ICMP port unreachable. A SYN to a filtered port gets no answer, while a closed port gets RST, which helps in diagnosis. **Mistake.** Mixing RST with FIN, which closes gracefully.

Q3.45 What happens when you type google.com and press Enter? Start at the keyboard.

ALL · TECH-1 · ■□□ · S

30-second answer: the browser checks caches and HSTS; DNS resolves the name (stub resolver, recursive resolver, root, `com.`, `google.com.`); the host ARPs for its gateway; TCP (or QUIC) connects to port 443; TLS 1.3 agrees keys and validates the certificate; HTTP GET goes out through routers that decrement the TTL and forward by longest-prefix match, with PAT at the home router; Google's nearest front end replies; the reply returns through the NAT table and the browser renders and reuses the connection.

Answer. The eleven-step scaffold, the 5- and 15-minute versions, the timeline and the answers to the usual interruptions are in §3.11 (p. 206). **Mistake.** Starting with the network and forgetting HSTS, DNS caching or the NAT return path.

Q3.46 Recursive versus iterative DNS. What do TTL and negative caching do? When does DNS use TCP?

AMAZON · GOOGLE · MICROSOFT · TECH-1 · ■■□ · S

30-second answer: the client asks its resolver recursively ("find it for me"); the resolver asks the root, TLD and authoritative servers iteratively, following referrals. The TTL says how long a cache may keep a record; negative caching keeps "no such name" for a time from the SOA. DNS uses TCP when an answer is truncated (too large for UDP) and for zone transfers.

Answer. Hence a change takes effect after the old TTL: lower it before a migration. Negative caching saves repeated queries but delays a newly created name. DNS over TLS and HTTPS also use TCP. **Mistake.** Saying that DNS is UDP only. See §3.10 (p. 203).

Q3.47 CNAME versus A versus ALIAS. Why can't you put a CNAME at a zone apex?

AMAZON · MICROSOFT · TECH-2 · ■■□ · T

30-second answer: A maps a name to an address; CNAME makes a name an alias of another name, and the resolver restarts the lookup on the target; ALIAS (or ANAME) is a provider feature that flattens a target into A records at query time. A CNAME may not coexist with other records at its name, and the apex must hold SOA and NS, so a CNAME there is illegal.

Answer. Use ALIAS for `example.com` pointing at a CDN or load balancer, and CNAME for `www`. **Mistake.** A CNAME chain that is long, which adds lookups.

Q3.48 TLS 1.2 versus 1.3: how many round trips does each need, and what is SNI?

GOOGLE · MICROSOFT · TECH-2 · ■■□ · S

30-second answer: TLS 1.2 needs two round trips after the TCP handshake; TLS 1.3 needs one, because the client sends its key share in the first message; a resumed 1.3 session can send data in 0-RTT, with replay risk. SNI is the extension in the ClientHello that names the host wanted, so that one IP address can serve many certificates.

Answer. Time to first byte over TCP: 1 RTT (TCP) + 2 (TLS 1.2) or + 1 (TLS 1.3), before the request. TLS 1.3 also removes weak ciphers and encrypts the certificate. The SNI is visible unless Encrypted Client Hello is used. Details in Pillar 9; see also §3.10 (p. 203).

Q3.49 HTTP/2 versus HTTP/3: what is head-of-line blocking, and how does QUIC remove it?

GOOGLE · MICROSOFT · TECH-2 · ■■■ · S

30-second answer: head-of-line blocking is the delay of data that has arrived because earlier data is missing. HTTP/2 multiplexes streams on one TCP connection, so one lost segment stalls all streams behind it. QUIC runs over UDP and has independent streams: loss on one stream delays only that stream.

Answer. QUIC also integrates TLS 1.3 (one handshake round trip, 0-RTT on resumption), uses connection IDs that survive address changes, and runs congestion control in user space. Costs: UDP is blocked or throttled on some networks (browsers fall back to TCP), and CPU use is higher. See Figure 3.32.

Q3.50 Scenario: a website is slow only for users in one city. Give a step-by-step plan.

AMAZON · GOOGLE · TECH-2 · ■■■ · T

30-second answer: narrow by layer and by hop. Reproduce from that city, then check in order: DNS (is the user sent to a distant region?), the anycast or routing path (traceroute and BGP), packet loss and RTT along the path, MTU problems, and the CDN or origin for that region.

Approach. (i) Compare measurements from that city and from a good one: DNS answer and TTL (`dig` through the local resolver and `@8.8.8.8`; geo-routing or ECS misconfiguration), time to first byte split into DNS, TCP, TLS and server (`curl -w`). (ii) Path: `mtr` both ways; look for loss starting at one hop, an unexpected detour, or a changed BGP route or anycast catchment. (iii) Loss and MTU: DF-set pings, retransmission rate in a capture; an ISP peering problem looks like loss at specific hours. (iv) Edge: is the local CDN node unhealthy, over capacity or serving from cache misses? (v) Mitigate: shift the traffic (weighted DNS, withdraw the anycast prefix at that site), then open a ticket with the ISP with evidence. **Mistake.** Restarting servers before locating the problem.

L3.1 Long form: a six-department campus from 10.20.0.0/22

CISCO · NOKIA · TECH-2 · ■■■ · T

30-second answer: sort the needs, give each the smallest block that fits, and place the blocks largest first on aligned boundaries. Six LANs of 450, 250, 120, 60, 28 and 12 hosts need /23, /24, /25, /26, /27 and /28, which use 1,008 of the 1,024 addresses.

Department	Hosts	Prefix	Network	Usable range	Broadcast
Computer science	450	/23	10.20.0.0	10.20.0.1 to 10.20.1.254	10.20.1.255
Electronics	250	/24	10.20.2.0	10.20.2.1 to 10.20.2.254	10.20.2.255
Library	120	/25	10.20.3.0	10.20.3.1 to 10.20.3.126	10.20.3.127
Administration	60	/26	10.20.3.128	10.20.3.129 to 10.20.3.190	10.20.3.191
Laboratories	28	/27	10.20.3.192	10.20.3.193 to 10.20.3.222	10.20.3.223
Guest Wi-Fi	12	/28	10.20.3.224	10.20.3.225 to 10.20.3.238	10.20.3.239

Answer. The spare 10.20.3.240/28 holds four /30 router links (or eight /31s). The whole /22 can be advertised as one summary. **Check.** 512 + 256 + 128 + 64 + 32 + 16 = 1,008. **Mistake.** Giving 450 hosts a /24, or leaving no room for growth; in practice add 20 to 30% headroom before choosing prefixes.

L3.2 Long form: decode a captured SYN frame, byte by byte

QUALCOMM · BROADCOM · TECH-2 · ■■□ · T

30-second answer: split the 54 bytes into 14 (Ethernet), 20 (IPv4) and 20 (TCP) and read each field in turn; it is a TCP SYN from 192.168.0.104:49822 to 192.168.0.1:80, ISN 1000, TTL 64, DF set.

```
5254 0012 3502 5254 0012 3456 0800
4500 0028 1c46 4000 4006 9cd0 c0a8 0068
c0a8 0001
c29e 0050 0000 03e8 0000 0000 5002 faf0
6c61 0000
```

Ethernet. Destination 52:54:00:12:35:02 (the gateway), source 52:54:00:12:34:56, type 0x0800 (IPv4). **IPv4.** `45`: version 4, IHL 5; total length 0x28 = 40 (20 + 20, no data); ID 0x1c46; `4000`: DF; TTL 0x40 = 64; protocol 6; checksum 0x9cd0 (the sum of all ten words including it folds to 0xFFFF); addresses c0a80068 = 192.168.0.104 and c0a80001 = 192.168.0.1. **TCP.** Source port 0xc29e = 49822; destination 0x0050 = 80; sequence 0x3e8 = 1000; ack 0; `50`: offset 5 (20 bytes); `02`: SYN only; window 0xfaf0 = 64,240; checksum 0x6c61 over the pseudo-header, header and data; urgent 0. **Mistake.** Reading the flags byte as the offset byte: `5002` is offset-and-reserved `50`, flags `02`.

L3.3 Long form: fragments across two links, MTU 1,500 then 576

QUALCOMM · BROADCOM · JUNIPER · TECH-2 · ■■■ · T

30-second answer: a 4,000-byte datagram becomes three fragments on the 1,500 link, and each is fragmented again on the 576 link, giving eight fragments; the offsets stay relative to the original datagram, and only the final piece has MF = 0.

Approach. On the 576 link data per fragment = 576 − 20 = 556, rounded down to a multiple of 8: 552 bytes (69 units). The first-stage fragments carry 1,480, 1,480 and 1,020 bytes at offsets 0, 185 and 370.

First-stage fragment	Second-stage data	Offsets	MF
1 (offset 0, 1,480 B)	552, 552, 376	0, 69, 138	1, 1, 1
2 (offset 185, 1,480 B)	552, 552, 376	185, 254, 323	1, 1, 1
3 (offset 370, 1,020 B, MF 0)	552, 468	370, 439	1, 0

Answer. The 376-byte pieces keep MF = 1 because more of the datagram follows. The data adds up: (552 + 552 + 376) × 2 + 552 + 468 = 2,960 + 1,020 = 3,980. **Mistake.** Restarting the offsets at 0 for each fragment, or clearing MF on the last piece of a non-final fragment.

L3.4 Long form: cwnd over 25 round trips with a 3-duplicate-ACK loss and a timeout

QUALCOMM · MICROSOFT · TECH-2 · ■■■ · T

30-second answer: slow start to ssthresh 32, linear growth to 36, a fast-recovery cut to 18, linear growth to 22, then a timeout that sets ssthresh to 11 and cwnd to 1, and slow start back up to 11 and linear growth.

- RTT 1 to 6: 1, 2, 4, 8, 16, 32 (slow start up to ssthresh = 32).
- RTT 7 to 10: 33, 34, 35, 36 (congestion avoidance); three duplicate ACKs arrive in RTT 10, so ssthresh = 18.
- RTT 11 to 15: 18, 19, 20, 21, 22 (fast recovery has ended; congestion avoidance); a timeout occurs in RTT 15, so ssthresh = 11 and cwnd = 1.
- RTT 16 to 20: 1, 2, 4, 8, 11 (slow start, capped at ssthresh).
- RTT 21 to 25: 12, 13, 14, 15, 16 (congestion avoidance).

Approach. Reno halves on the duplicate-ACK loss (36 / 2 = 18) and resumes there; the timeout at 22 gives ssthresh = 11 and cwnd = 1, and slow start is capped at ssthresh (8 doubles to 16, which is cut to 11). **Mistake.** Resetting to 1 on the duplicate-ACK loss, or continuing slow start beyond ssthresh.

L3.5 Subnetting sprint: twenty drills (target 45 seconds each)

CISCO · AMAZON · JUNIPER · ARISTA · NOKIA · OA · ■□□ · T

30-second answer: for each address, find the block size in the interesting octet, take the multiple below, and subtract one from the next multiple for the broadcast. Answers are listed for self-marking.

Address/prefix	Network	Broadcast	Usable
10.20.30.77/26	10.20.30.64	10.20.30.127	62
192.168.5.130/25	192.168.5.128	192.168.5.255	126
172.16.200.9/20	172.16.192.0	172.16.207.255	4,094
10.0.0.200/27	10.0.0.192	10.0.0.223	30
172.31.255.1/18	172.31.192.0	172.31.255.255	16,382
192.168.100.77/28	192.168.100.64	192.168.100.79	14
10.8.130.5/21	10.8.128.0	10.8.135.255	2,046
172.20.15.200/22	172.20.12.0	172.20.15.255	1,022
203.0.113.77/29	203.0.113.72	203.0.113.79	6
192.168.7.250/30	192.168.7.248	192.168.7.251	2
10.255.3.4/13	10.248.0.0	10.255.255.255	524,286
100.64.77.9/10	100.64.0.0	100.127.255.255	4,194,302
172.17.33.33/19	172.17.32.0	172.17.63.255	8,190
192.0.2.129/26	192.0.2.128	192.0.2.191	62
10.4.5.6/9	10.0.0.0	10.127.255.255	8,388,606
198.51.100.99/27	198.51.100.96	198.51.100.127	30
172.16.31.254/21	172.16.24.0	172.16.31.255	2,046
10.10.10.10/12	10.0.0.0	10.15.255.255	1,048,574
192.168.1.1/23	192.168.0.0	192.168.1.255	510
10.1.2.3/16	10.1.0.0	10.1.255.255	65,534

Mistakes. Using the wrong octet for the block (a /13 works in the second octet), and forgetting that /30 leaves two hosts.

Revision Spread

THE 15-MINUTE VERSION

Ten facts. (1) MAC changes at every hop; IP does not, except at NAT. (2) Block size = 256 − mask octet; hosts = $2^h - 2$, except /31 (2) and /32 (1). (3) Fragment offset counts 8-byte units; MSS = MTU − 40. (4) TTL and header checksum change at every hop. (5) SYN and FIN each use one sequence number; a pure ACK uses none. (6) TIME_WAIT is on the active closer; CLOSE_WAIT is an application bug. (7) Effective window = min(rwnd, cwnd); throughput ≤ window / RTT; BDP = rate × RTT. (8) Reno: 3 duplicate ACKs give ssthresh = cwnd / 2; a timeout gives cwnd = 1. (9) CUBIC and Reno react to loss; BBR models bandwidth and RTT. (10) DNS: recursive from the client, iterative from the resolver; TTL decides when a change is seen; TCP for truncation and zone transfers.

Three figures to redraw from memory. The three-way handshake and teardown ladders with sequence numbers (Figure 3.24, Figure 3.25); the cwnd sawtooth with ssthresh (Figure 3.29); the packet-walk strip with the L2 row changing and the L3 row constant (Figure 3.19).

MODELS, DEVICES, PDUS

- 7 Application, 6 Presentation, 5 Session, 4 Transport (segment), 3 Network (packet), 2 Data link (frame), 1 Physical (bits). Mnemonic: All People Seem To Need Data Processing.
- Hub: L1, one collision and one broadcast domain. Switch: L2, collision domain per port. Router: L3, a broadcast domain per interface. Firewall and load balancer: L4 to L7.
- Demultiplexing keys: EtherType, IP protocol, port. Say in an interview: “each layer’s header tells its peer what to do.”

LAYER 2

- Ethernet II: dst 6, src 6, type 2, payload 46 to 1,500, FCS 4 (frame 64 to 1,518; +4 with an 802.1Q tag). Preamble 7 + SFD 1 and gap 12 are outside the frame.
- MAC: OUI 3 + device 3 bytes; I/G = low bit of the first byte (1 = group); U/L = next bit.
- Switch: learn source, forward or flood on destination, age after 300 s. Cut-through starts after the destination MAC; store-and-forward checks the FCS.
- Min frame: $L \geq 2T_pR$. CRC: append r zeros, divide mod 2, remainder replaces them. Hamming: syndrome = position of the error.
- ARQ: SW efficiency $1/(1 + 2a)$, $a = T_p/T_t$. GBN window $\leq 2^n - 1$; SR window $\leq 2^{n-1}$.

IPV4 AND SUBNETTING

- Header 20 to 60 bytes (IHL × 4); fields: ver, IHL, DSCP/ECN, length, ID, flags (DF, MF), offset, TTL, protocol, checksum, src, dst.
- Block-size method: network = largest multiple of the block not above the address; broadcast = next network − 1.
- Private: 10/8, 172.16/12, 192.168/16. Special: 127/8, 169.254/16, 100.64/10, 224/4, 0.0.0.0, 255.255.255.255.
- VLSM: largest first, each on a multiple of its block. Summary: common leading bits of aligned, contiguous blocks.

Prefix	Mask	Block	Addresses	Usable
/24	255.255.255.0	1	256	254
/25	255.255.255.128	128	128	126
/26	255.255.255.192	64	64	62
/27	255.255.255.224	32	32	30
/28	255.255.255.240	16	16	14
/29	255.255.255.248	8	8	6
/30	255.255.255.252	4	4	2
/31	255.255.255.254	2	2	2 (RFC 3021)
/32	255.255.255.255	1	1	1

ARP, DHCP, NAT

- ARP: broadcast request (opcode 1), unicast reply (2). Gratuitous ARP: duplicate check and cache refresh. Proxy ARP: router answers for others.
- DHCP: Discover, Offer, Request, Ack; UDP 68 to 67; T1 50%, T2 87.5%; relay sets giaddr; options 1, 3, 6, 51, 54, 82.
- NAT: static 1:1; dynamic pool; PAT many:1 by port. Inside local, inside global, outside local, outside global. Port forwarding for servers; STUN, TURN and ICE for peer-to-peer.

IPV6

- 128 bits; header 40 bytes; no checksum, no router fragmentation; minimum MTU 1,280. Compress: drop leading zeros, one `::` on the longest run.

- 2000::/3 global, fc00::/7 unique local, fe80::/10 link-local, ff00::/8 multicast, ::1 loopback, 2001:db8::/32 documentation.
- EUI-64: insert FF:FE, flip bit 7. Solicited-node: ff02::1:ff + last 24 bits; MAC 33:33 + last 32 bits.
- NDP (ICMPv6): RS 133, RA 134, NS 135, NA 136, Redirect 137; PTB is type 2. RA flags: M (stateful DHCPv6), O (other options).

TCP AND UDP

- UDP header 8 bytes: ports, length, checksum. TCP header 20 to 60 bytes: ports, seq, ack, offset, flags CWR ECE URG ACK PSH RST SYN FIN, window, checksum, urgent pointer, options (MSS, window scale, SACK, timestamps).
- Handshake: SYN, SYN-ACK, ACK. Teardown: FIN, ACK, FIN, ACK; active closer waits 2 MSL (60 s on Linux). RTO = SRTT + 4 RTTVAR, minimum 200 ms on Linux (1 s in RFC 6298), doubled on each timeout.
- Fast retransmit after 3 duplicate ACKs. Slow start doubles per RTT up to ssthresh, then +1 MSS per RTT. Initial window 10 MSS.
- Window scale shift s multiplies 65,535 by 2^s. Mathis: rate ≈ (MSS / RTT) × 1.22 / √p.

State	Who	Meaning
LISTEN	server	waiting for a SYN
SYN_SENT, SYN_RCVD	client, server	handshake in progress
ESTABLISHED	both	data flows
FIN_WAIT_1, FIN_WAIT_2	active closer	FIN sent; ACK received
CLOSE_WAIT, LAST_ACK	passive closer	FIN received; waiting for the application to close
CLOSING	both closing	simultaneous close
TIME_WAIT	active closer	2 MSL before CLOSED

PORTS, PROTOCOL NUMBERS, ETHERTYPES

Port	Service	Port	Service
20/21	FTP data/control	123	NTP (UDP)
22	SSH	143	IMAP
23	Telnet	161/162	SNMP
25	SMTP	179	BGP
53	DNS (UDP/TCP)	389	LDAP
67/68	DHCP server/client	443	HTTPS, QUIC
69	TFTP	514	syslog
80	HTTP	520	RIP
110	POP3	3389	RDP

IP protocol numbers: 1 ICMP, 2 IGMP, 6 TCP, 17 UDP, 47 GRE, 50 ESP, 51 AH, 58 ICMPv6, 89 OSPF. EtherTypes: 0x0800 IPv4, 0x0806 ARP, 0x86DD IPv6, 0x8100 802.1Q. ICMP: 0 echo reply, 3 unreachable (code 4 fragmentation needed), 5 redirect, 8 echo request, 11 time exceeded.

DNS, HTTP, TLS

- Records: A, AAAA, CNAME, NS, MX, PTR, TXT, SOA, SRV, CAA. UDP 53; TCP for truncation and zone transfer; DoT 853; DoH 443.
- Status classes: 2xx success, 3xx redirect (301, 302, 304), 4xx client (401, 403, 404, 429), 5xx server (500, 502, 503).
- TLS 1.3: 1 RTT; TLS 1.2: 2 RTT; SNI names the host. HTTP/2: streams over TCP, blocked by one loss. HTTP/3: QUIC over UDP 443, per-stream recovery.

FORMULAS

Transmission L / R. Propagation d / v ($v \approx 2 \times 10^8$ m/s). Total delay = processing + queueing + transmission + propagation. Throughput ≤ W / RTT. BDP = R × RTT. Hosts = $2^h - 2$. Subnets = 2^b. Frame efficiency = payload / (payload + headers + 20).

FLASHCARDS

Q	A	Q	A
Layer of a router; of a switch?	3; 2	Compress 2001:0db8:0000:0000:0000:0000:0000:0001	2001:db8::1
PDU names, layer 4 down to 1	Segment, packet, frame, bits	EUI-64 steps	Insert FF:FE; flip bit 7
Fields that change per hop	TTL, header checksum, MAC addresses	Solicited-node multicast	ff02::1:ff + last 24 bits
Minimum and maximum Ethernet frame	64 and 1,518 bytes (1,522 tagged)	RA flags M and O	Stateful DHCPv6; other options by DHCPv6
Why 64 bytes?	Sender must still transmit when a collision returns: $L \geq 2T_pR$	ICMPv6 NDP types	133 to 137
Switch, unknown destination	Flood out of all ports except the arrival port	UDP header size; TCP minimum	8; 20 bytes
CRC remainder bits for generator 10011	4	What does a SYN consume?	One sequence number
Stop-and-Wait efficiency	1 / (1 + 2a)	Which side has TIME_WAIT?	The active closer
GBN and SR window, n-bit numbers	$2^n - 1$; 2^{n-1}	Many CLOSE_WAIT sockets mean	Application does not call close()
/27: block, hosts	32; 30	Effective window	min(rwnd, cwnd)
/30: hosts; /31: hosts	2; 2 (RFC 3021)	Slow start growth	Doubles each RTT
/22 mask	255.255.252.0	Reno on 3 duplicate ACKs	ssthresh = cwnd / 2; cwnd = ssthresh (+3 inflation)
Wildcard of /21	0.0.7.255	Reno on timeout	cwnd = 1
Private IPv4 ranges	10/8, 172.16/12, 192.168/16	CUBIC reacts to	Loss; cubic growth about W_{max}
APIPA range	169.254.0.0/16	BBR models	Bottleneck bandwidth and minimum RTT
CGNAT range	100.64.0.0/10	64 KB window, 100 ms RTT	5.24 Mb/s
Fragment offset unit	8 bytes	BDP of 1 Gb/s, 100 ms	12.5 MB
MSS on Ethernet; on PPPoE	1,460; 1,452	Why Nagle plus delayed ACK stalls	Each waits for the other; use TCP_NODELAY
ICMP for PMTUD	Type 3 code 4	SYN cookie holds	Hash of 4-tuple, secret and time in the ISN
Traceroute probes, Linux; Windows	UDP from port 33434; ICMP echo	TLS 1.3 handshake	1 RTT
ARP request destination MAC	ff:ff:ff:ff:ff:ff	DNS over TCP when	Truncation, zone transfer

DHCP ports	67 server, 68 client	CNAME at the apex	Not allowed (SOA and NS there)
DHCP T1; T2	50%; 87.5% of the lease	HTTP/3 transport	QUIC over UDP 443
Role of giaddr	Identifies the client subnet to the server	First step of the google.com walk	Browser caches and HSTS, then DNS
PAT distinguishes hosts by	Source port		

PART I · PILLAR 4

4

Routing and Switching, CCNA to CCNP ENCOR

A packet crosses a campus, an ISP and the Internet; we learn who decides each hop, and why.

4.1 Inside the box: planes and the CLI	SHOULD
4.2 Switching, VLANs, trunking, inter-VLAN routing	MUST
4.3 Spanning Tree and EtherChannel	MUST
4.4 The routing table, static and default routes	MUST
4.5 OSPF	MUST
4.6 EIGRP	OPTIONAL
4.7 BGP	MUST
4.8 First-hop redundancy	SHOULD
4.9 Route control: prefix lists, route maps, redistribution, VRF-lite	SHOULD
4.10 MPLS and L3VPN	SHOULD
4.11 Quality of Service	SHOULD
4.12 IP multicast	OPTIONAL
4.13 IP services and access lists on IOS	MUST
4.14 Tunnels and enterprise architecture	OPTIONAL
4.15 Mini-project: Campus-to-Internet	OPTIONAL

Every topology and lab in this chapter belongs to one fictional organisation, **Campus Net**: a university network with a sales-and-engineering office floor (VLANs 10 and 20), two border routers, BR1 and BR2, and two upstream providers, ISP-A (AS 100) and ISP-B (AS 200). Subnetting, NAT theory, the longest-prefix-match algorithm, Dijkstra, access-list theory and the internals of the forwarding hardware are taught in their home chapters, and are cited here by one sentence and a page reference. This chapter teaches what is configured on the devices, what the protocols do on the wire, and how each is verified. The CCNA topics are mixed with the CCNP ENCOR topics; the tags tell you which to read first.

4.1 Inside the box: planes and the CLI SHOULD · ■□□

A router or switch performs three jobs that run at three different speeds. The *management plane* (the CLI, SSH, SNMP, NETCONF) is how humans and tools configure it. The *control plane* (OSPF, BGP, spanning tree, ARP) runs on the CPU and builds tables. The *data plane* (the forwarding hardware) uses those tables to move transit packets at line rate, without involving the CPU. The tables are the routing information base (RIB) and the forwarding information base (FIB); how the FIB is held in TCAM is taught in §5.8 (p. 320). Here it is enough to know that a packet *addressed to the router* (an OSPF hello, an SSH session) is punted to the CPU, whereas a *transit* packet never leaves the fast path.

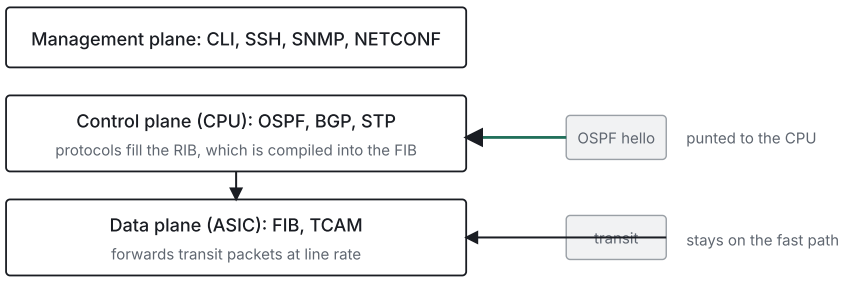


Figure 4.1 Three planes. A packet for the router (accent arrow) is punted to the control plane; a transit packet never leaves the data plane.

The Cisco IOS command line has modes that are visible in the prompt: `Router>` (user), `Router#` (privileged), `Router(config)#` (global configuration) and `Router(config-if)#` or `Router(config-router)#` (a sub-mode). `enable` and `configure terminal` go up a level, `exit` goes back one and `end` returns to privileged mode. `show running-config` displays the active configuration in RAM, whereas `show startup-config` displays the copy in NVRAM that survives a reload; `copy running-config startup-config` saves. Prefix a privileged command with `do` to run it from a configuration mode, and use `?` and the Tab key freely, because abbreviations (`sh ip int br`) are accepted when unambiguous.

WORKED EXAMPLE

The first fifteen minutes on a new Campus Net router.

1. Name the device and protect privileged mode with a hashed secret, because `enable password` is stored readably.
2. Create a local user and enable SSH, which needs a hostname, a domain name and an RSA key.
3. Restrict the virtual terminal lines to SSH, address the interfaces, add a banner and save.

```
hostname BR1
ip domain-name campus.example
enable secret S3cr3t!
username admin privilege 15 secret Adm1n!
crypto key generate rsa modulus 2048
ip ssh version 2
line vty 0 4
 login local
 transport input ssh
interface GigabitEthernet0/1
 ip address 192.0.2.2 255.255.255.252
 no shutdown
banner motd # Authorised use only #
end
copy running-config startup-config
```

```
BR1# show ip interface brief
Interface           IP-Address     OK? Method Status  Protocol
GigabitEthernet0/1  192.0.2.2      YES manual up      up
BR1# show ip ssh
SSH Enabled - version 2.0
```

Result: Status is the Layer 1 state and Protocol the Layer 2 state; “up/up” is the only healthy pair.

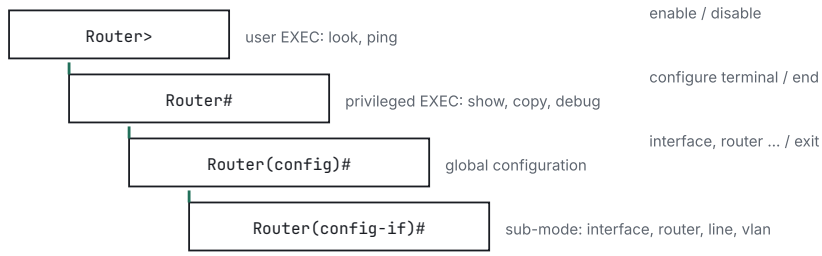
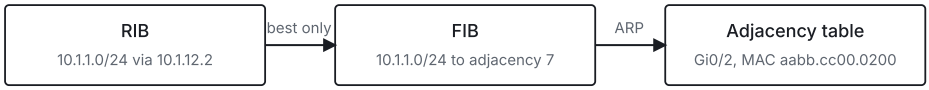


Figure 4.2 The CLI modes. The prompt names the mode: > user, # privileged, (config…)# a configuration mode.

A device boots in a fixed order: the ROM monitor runs a power-on self-test, loads the operating-system image from flash, and then reads the saved configuration from NVRAM; if none exists it offers the setup dialogue. The *configuration register* (`show version` prints it, normally 0x2102) controls this. Setting it to 0x2142 makes the router ignore the saved configuration at the next boot, which is the basis of password recovery: boot with 0x2142, copy the startup configuration into running configuration, change the secret, restore 0x2102 and save. Physical access is therefore equivalent to administrative access, which is why console ports are kept in locked rooms.

Packet forwarding has also evolved in three generations. *Process switching* sends every packet to the CPU, which looks up the route and rebuilds the header; it is the slowest and is used today only for packets addressed to the router. *Fast switching* caches the result of the first packet of a flow. *Cisco Express Forwarding* (CEF) precomputes two tables, the FIB (a copy of the best routes, organised for longest-prefix lookup) and the *adjacency table* (the Layer 2 rewrite string for each next hop, including its MAC address learnt by ARP), and forwards every packet by a single lookup in each. On a switch or high-end router those tables sit in hardware. A missing adjacency, for instance because ARP failed, produces packet drops that `show ip cef` and `show adjacency` reveal even when the route looks correct.



Control plane builds the RIB; CEF derives the FIB and the Layer 2 rewrite; the data plane reads only the last two.

Figure 4.3 From routing table to forwarding. A transit packet needs one FIB lookup and one adjacency rewrite.

The same three planes exist on every vendor, but the commands differ. The table maps the five commands one uses most.

Junos and FRR — Junos has a candidate configuration: you edit it, run `commit`, and can `rollback 1`. IOS applies each line instantly. FRR's `vtysh` is IOS-like, and Arista EOS is almost identical to IOS.

Task	Cisco IOS-XE	Junos	Arista EOS	FRR (`vtysh`)
Interface summary	`show ip interface brief`	`show interfaces terse`	`show ip interface brief`	`show interface brief`
Routing table	`show ip route`	`show route`	`show ip route`	`show ip route`
OSPF neighbours	`show ip ospf neighbor`	`show ospf neighbor`	`show ip ospf neighbor`	`show ip ospf neighbor`
BGP summary	`show ip bgp summary`	`show bgp summary`	`show ip bgp summary`	`show bgp summary`
Save configuration	`copy run start`	`commit`	`write memory`	`write memory`

Configuration is managed by copying it. `copy running-config tftp:` (or `scp:`) saves it off the device; `copy tftp: flash:` loads a new image; `show version` reports the image, the uptime and the register; `show running-config | section ospf` filters output. On a production device, change windows are protected by `configure replace` or by a rollback timer, because a mistyped command over SSH can cut the session that is needed to correct it.

CDP (Cisco) and LLDP (the open standard, IEEE 802.1AB) advertise a device's name, port and platform to its neighbours. `show cdp neighbors detail` is the quickest way to learn what is on the other end of a cable. Both are best disabled on ports facing untrusted users.

> LAB
>
> `LAB 4.1 · 15 MIN` Packet Tracer: two routers and one switch, base configuration and SSH as in the worked example. Then run the multi-vendor task on Containerlab, with one Nokia SR Linux and one Arista cEOS node.
>
> ```
> BR1# show cdp neighbors detail | include Device|IP address
> Device ID: SW1
>   IP address: 192.168.99.2
> ```
>
> **What you should notice:** CDP reveals the neighbour's address without a routing protocol. Fill in the equivalence table above for the other two vendors; the *concepts* (interface, route, neighbour) are the same, and only the commands differ.

> CHECK YOURSELF
>
> 1. What does it mean when `show ip interface brief` prints "up/down"? [1]
> 2. Why is `enable secret` preferred to `enable password`? [2]

4.2 Switching, VLANs, trunking and inter-VLAN routing

MUST · ■□□

BIG IDEA

A VLAN divides one physical switch into several broadcast domains, because the switch refuses to forward a frame to a port of a different VLAN; a router, or a Layer 3 switch, is therefore required to carry traffic between them.

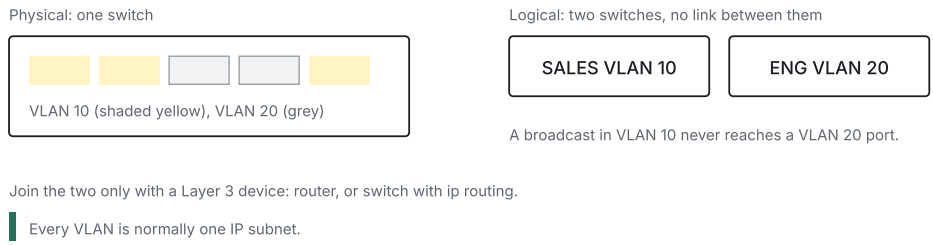


Figure 4.4 A VLAN is a logical switch. Ports of one colour behave as if they belonged to a separate device.

A Layer 2 switch forwards on destination MAC address. It *learns* by reading the source MAC of every arriving frame and recording the (MAC, port, VLAN) triple in its MAC address table, which entries expire after 300 seconds of silence by default. For a known destination the frame leaves one port; for a broadcast, a multicast (without snooping) or an *unknown unicast* the switch floods the frame out of every other port in the VLAN. Thus, without VLANs, a single switch is a single broadcast domain, and every host hears every ARP request.

```
SW1# show mac address-table dynamic vlan 10
Vlan    Mac Address       Type     Ports
  10    0050.7966.6801    DYNAMIC  Gi0/1
  10    0050.7966.6802    DYNAMIC  Gi0/24
```

The forwarding decision of a switch can be summarised in a table. It also explains the failure modes met later: a flood is cheap in a small VLAN, but ruinous in a loop.

Frame arriving in VLAN V	Destination	The switch
Unicast, MAC known	learnt on port P	forwards out of P only (filters if P is the arrival port)
Unicast, MAC unknown	not in table	floods out of all other ports in V (*unknown unicast flooding*)
Broadcast (ff:ff:ff:ff:ff:ff)	all hosts	floods out of all other ports in V
Multicast, no snooping	group	floods like a broadcast; with IGMP snooping only to joined ports
Source MAC new or moved	learning	records or updates the (MAC, port, VLAN) entry

Two consequences follow. First, a MAC address appearing on two ports in quick succession (“MAC flapping”) signals either a loop or a duplicated address. Secondly, the MAC table has finite size (a few thousand to hundreds of thousands of entries), so an attacker who sends frames

with random source addresses can fill it and force the switch to flood everything; port security limits the number of addresses per port (§9.5 (p. 549)).

An *access port* belongs to one VLAN and sends and receives untagged frames. A *trunk port* carries several VLANs on one link, and marks each frame with an IEEE 802.1Q tag so that the far end knows its VLAN. The tag is four bytes inserted between the source MAC and the EtherType, so the frame check sequence is recomputed and the maximum frame grows from 1518 to 1522 bytes.

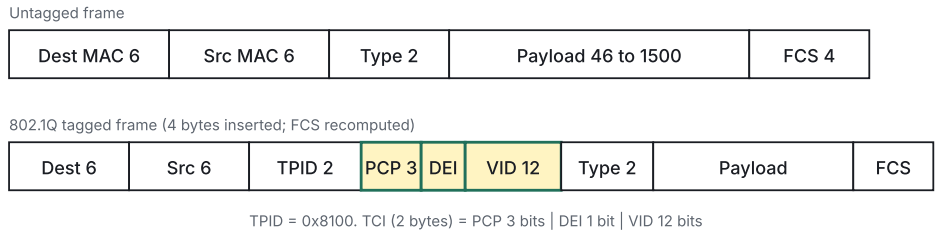


Figure 4.5 The 802.1Q tag. Twelve VID bits give 4096 values, of which 0 and 4095 are reserved, hence 4094 usable VLANs.

VLAN 1 is the default; 2 to 1001 are normal-range; 1002 to 1005 are reserved for legacy FDDI and Token Ring; 1006 to 4094 are extended. One VLAN on a trunk is the *native VLAN* (VLAN 1 by default), whose frames cross the trunk *untagged*. If the two ends disagree about it, frames leak between VLANs and CDP and spanning tree print a native-VLAN mismatch warning. Best practice is to set the native VLAN to an unused one (for example 99) on both ends and to prune the allowed list to the VLANs that need to cross.

WORKED EXAMPLE

Two VLANs, one trunk, router on a stick. Campus Net has SALES (VLAN 10, 192.168.10.0/24) and ENG (VLAN 20, 192.168.20.0/24). A SALES PC on SW1 and an ENG PC on SW2 must communicate. SW1 trunks to SW2 and to R1.

```
! SW1 and SW2
vlan 10
 name SALES
vlan 20
 name ENG
interface GigabitEthernet0/1
 switchport mode access
 switchport access vlan 10        ! PC on SW1 (SW2: access vlan 20)
interface GigabitEthernet0/24
 switchport mode trunk
 switchport trunk allowed vlan 10,20
 switchport trunk native vlan 99
! R1: one physical link, two sub-interfaces
interface GigabitEthernet0/0
 no shutdown
interface GigabitEthernet0/0.10
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.0
interface GigabitEthernet0/0.20
 encapsulation dot1Q 20
 ip address 192.168.20.1 255.255.255.0
```

1. The SALES PC has no route to 192.168.20.0/24, so it ARPs for its gateway 192.168.10.1 and sends the packet to R1’s MAC (untagged on the access port).

2. SW1 tags the frame with VID 10 on the trunk to R1. R1 reads the tag and delivers it to sub-interface .10.
3. R1 looks up 192.168.20.0/24, a connected route out of .20, rewrites the Layer 2 header and sends the packet back on the same cable, now tagged VID 20.
4. SW1 forwards the VID 20 frame over the trunk to SW2, which removes the tag and delivers it on the ENG access port.

Result: the packet crosses the R1 link twice, which is why the design is called a hairpin and why that link limits inter-VLAN throughput.

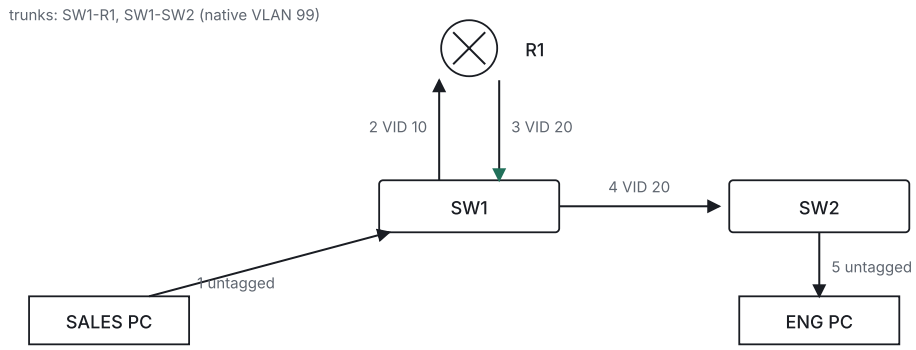


Figure 4.6 The router-on-a-stick hairpin. The packet enters R1 tagged 10 and leaves tagged 20 on the same link.

A *Layer 3 switch* removes the hairpin. Enabling `ip routing` and creating a *switched virtual interface* (SVI) per VLAN lets the switch route in hardware between its own VLANs: `interface Vlan10` with `ip address 192.168.10.1 255.255.255.0` replaces the sub-interface, and hosts keep the same gateway address. SVIs scale better than sub-interfaces, because the forwarding is done by the switching ASIC rather than by a single router link. A *routed port* (`no switchport`) is a Layer 3 port with no VLAN at all.

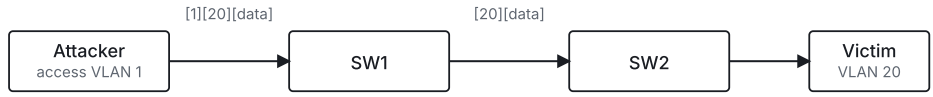
```
SW1(config)# ip routing
SW1(config)# interface Vlan10
SW1(config-if)# ip address 192.168.10.1 255.255.255.0
SW1# show ip route connected
C  192.168.10.0/24 is directly connected, Vlan10
```

Voice VLAN. An IP phone and the PC behind it share one access port. The port is given a second, *voice* VLAN (`switchport voice vlan 150`); CDP or LLDP-MED tells the phone which VLAN to tag, while the PC's frames stay untagged in the access VLAN. The port behaves as a trunk with just two VLANs, and the voice traffic can then be placed in a separate IP subnet and a priority queue (§4.11 (p. 274)).

```
interface GigabitEthernet0/5
 switchport mode access
 switchport access vlan 10
 switchport voice vlan 150
 spanning-tree portfast
 spanning-tree bpduguard enable
```

Two Layer 2 mechanisms deserve suspicion. The *Dynamic Trunking Protocol* (DTP) lets two ports negotiate a trunk; the outcome is in the table (a port set to `mode trunk` or `desirable` initiates, `auto` only agrees). An attacker who sends DTP frames to a port left on `dynamic auto`

can turn it into a trunk and see every VLAN. The *VLAN Trunking Protocol* (VTP) distributes the VLAN database from servers to clients, and a switch that joins with a higher revision number and an empty database overwrites the VLANs of the whole domain (the "VTP bomb"). The remedy is to configure ports explicitly (`switchport mode access` or `trunk`, plus `switchport nonegotiate`), to use VTP transparent or off, and to shut down unused ports. Port security, DHCP snooping and VLAN-hopping defences are taught in §9.5 (p. 549).



SW1 strips the outer tag (1) because VLAN 1 is the native VLAN of the trunk and is sent untagged, leaving tag 20 for SW2.
Defence: native VLAN unused (not 1) on every trunk; never put hosts in the native VLAN.

Figure 4.7 VLAN hopping by double tagging. The attack is one-way (the reply cannot return) and works only when the attacker's access VLAN equals the trunk's native VLAN. Further defences are in §9.5.

Local Peer	access	trunk	dynamic auto	dynamic desirable
access	access	mismatch	access	access
trunk	mismatch	trunk	trunk	trunk
dynamic auto	access	trunk	access	trunk
dynamic desirable	access	trunk	trunk	trunk

TRAP

Claim: two ports on `dynamic auto` will form a trunk. *Why it is wrong:* `auto` only responds to a negotiating peer and never initiates, so two `auto` ports both wait and settle on access mode. *Correct statement:* at least one end must be `trunk` or `desirable`; better, configure both ends statically.

WORKED EXAMPLE

Converting the design to an L3 switch. Campus Net upgrades the trunk-to-R1 design: SW1 becomes the gateway, and R1 is kept only for the border.

1. Enable routing in the switch: `ip routing`.
2. Create `interface Vlan10` and `interface Vlan20` with the addresses that R1's sub-interfaces had (192.168.10.1 and 192.168.20.1). Hosts need no change.
3. Remove the R1 sub-interfaces. Make SW1's uplink a routed port with `no switchport` and a /30 to R1, and exchange routes with OSPF or a static route.
4. Confirm: the VLAN must exist and have at least one active port in the VLAN, or the SVI stays down.

```
SW1# show ip interface brief | include Vlan
Vlan10   192.168.10.1  YES manual up   up
Vlan20   192.168.20.1  YES manual up   up
```

Result: inter-VLAN traffic never leaves SW1, and its capacity is the switching ASIC's, not one 1 Gb/s link's.

INTERVIEW LENS

CISCO · JUNIPER · ARISTA

Expected: the difference between access and trunk, what the native VLAN is, and the symptom of a mismatch; why 4094; ROAS against SVI. A strong answer volunteers the security point (DTP, native VLAN, VTP) and the hairpin bottleneck. Follow-ups: what is on the wire for a frame from VLAN 10 to VLAN 20; what happens when the trunk's allowed list omits a VLAN.

LAB

`LAB 4.2 · 15 MIN` Build Figure 4.6 in Packet Tracer, then break it twice.

```
SW1(config-if)# switchport trunk native vlan 1      ! on one end only
SW1(config-if)# switchport trunk allowed vlan remove 20
```

```
%CDP-4-NATIVE_VLAN_MISMATCH: Native VLAN mismatch discovered on Gi0/24 (1),
with SW2 Gi0/24 (99).
SALES-PC> ping 192.168.20.10
Request timed out.   <-- VLAN 20 no longer crosses the trunk
```

What you should notice: the native mismatch is only a log message and traffic may still flow; the pruned VLAN silently fails. `show interfaces trunk` shows the third table ("VLANs in spanning tree forwarding state and not pruned") shrinking.

CHECK YOURSELF

1. Why does a trunk frame grow to 1522 bytes? [3]
2. A host on VLAN 10 can ping its gateway but not a host in VLAN 20 on another switch. Name two causes. [4]

4.3 Spanning Tree and EtherChannel MUST · ■■□

BIG IDEA

Ethernet frames carry no hop count, so a loop in the Layer 2 topology circulates a broadcast forever; the Spanning Tree Protocol (STP) therefore elects one root switch and blocks the minimum number of ports needed to leave a loop-free tree.

Redundant switch links are desirable, yet a loop produces three failures at once: a *broadcast storm* (frames multiply and fill every link), *MAC table instability* (the same source appears on alternating ports) and duplicate frame delivery. Switches exchange *bridge protocol data units* (BPDUs) every two seconds to agree on a tree. The agreement uses four steps in this order:

1. **Root bridge.** The switch with the lowest *bridge ID* (BID) wins. The BID is the priority followed by the MAC address.
2. **Root port**, one on every non-root switch: the port with the lowest *root path cost*; ties go to the lowest *sender BID*, then the lowest *sender port ID*.
3. **Designated port**, one on every segment: the port of the switch advertising the lowest root path cost on that segment (ties again by sender BID, then port ID). Every port of the root bridge is designated.
4. **Blocked** (alternate) ports: all remaining ports.

Figure 4.8 The bridge ID. The VLAN number occupies the low 12 bits, which is why the configurable priority must be a multiple of 4096.

A BPDU carries the fields needed for the election: the *root bridge ID* the sender believes in, the *root path cost* from the sender, the *sender's own bridge ID* and the *sender port ID* (port priority, 128 by default, and port number), plus the timers (hello, forward delay, max age) set by the root. Only the root generates BPDUs; every other switch relays an updated copy through its designated ports, adding the cost of the link on which it was received. That is why a non-root switch's root path cost is, in effect, the sum of the costs of the links its BPDU crossed. When a switch stops hearing the root's BPDUs on its root port for max-age (20 s), it assumes the path is lost and starts a new election. A *topology change* (a port moving to or from forwarding) is signalled up to the root, which tells every switch to shorten its MAC-table ageing to 15 s, so stale entries do not blackhole traffic.

Link costs depend on speed. The classic (short) values are 10 Mb/s = 100, 100 Mb/s = 19, 1 Gb/s = 4 and 10 Gb/s = 2; the long values used by `spanning-tree pathcost method long` and by MST are 1 Gb/s = 20,000 and 10 Gb/s = 2,000. Because per-VLAN spanning tree (PVST+) runs an independent tree per VLAN, the BID includes the VLAN, and the default priority for VLAN 1 is displayed as 32769.

WORKED EXAMPLE

Root election on the Campus Net triangle. Three switches in VLAN 1, all at default priority 32768 (shown 32769), MACs ending 0001 (SW1), 0002 (SW2) and 0003 (SW3), joined pairwise by 1 Gb/s links (cost 4).

Part 1. Default priorities.

1. Priorities tie, so the lowest MAC wins: SW1 is root. All its ports are designated.
2. SW2 and SW3 each reach the root by their direct link at cost 4 (the path through the third switch costs 8), so each direct port is a root port.
3. On the SW2-SW3 segment both switches offer root path cost 4. The tie goes to the lower sender BID, SW2. SW2's port is designated; SW3's port is not a root or designated port, hence **alternate (blocking)**.

Part 2. Set SW3's priority to 4096 (BID 4097 for VLAN 1).

4. SW3 now has the lowest BID and becomes root. SW1 and SW2 each reach SW3 directly at cost 4.
5. On the SW1-SW2 segment both offer cost 4; SW1's BID is lower, so SW1 is designated and SW2's port on that segment blocks.

Part 3. Two parallel links SW1-SW2 (Gi0/1 and Gi0/2), SW1 root again.

6. SW2 has two candidate root ports with equal cost 4 and an identical sender BID. The tie goes to the lowest *sender port ID*: SW1's Gi0/1 (128.1) beats Gi0/2 (128.2). SW2's port that hears 128.2 becomes alternate.

Result: three outcomes, each decided by a different step of the election: lowest MAC, lowest sender BID, lowest sender port ID.

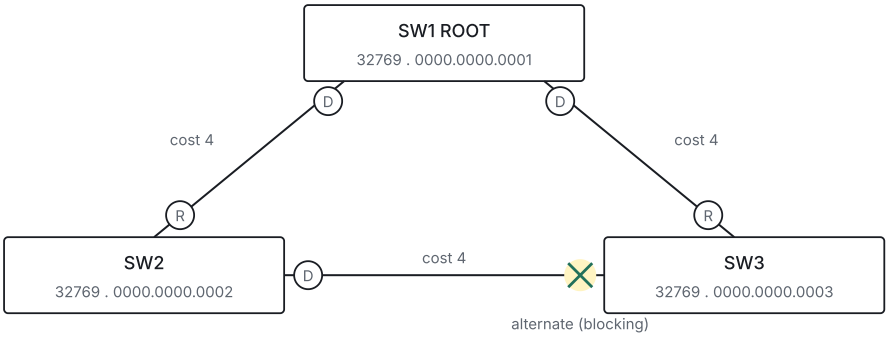


Figure 4.9 Part 1 of the election. R = root port, D = designated port, cross = blocked. Every port on the root is designated; SW3 blocks because SW2 has the lower BID.

TRAP

Claim: a root port is on the root switch and a designated port is the one that forwards toward it. *Why it is wrong:* the root bridge has no root port; the root port is on the *non-root* switch and faces the root. Designated ports are chosen per *segment*, not per switch. *Correct statement:* the root has only designated ports; each non-root switch has exactly one root port.

WORKED EXAMPLE

Load sharing with PVST+. The Campus Net triangle carries VLANs 10 and 20. Rather than let one tree decide, the designer makes SW2 root for VLAN 10 (`spanning-tree vlan 10 priority 4096`, BID 4106) and SW3 root for VLAN 20 (`spanning-tree vlan 20 priority 4096`, BID 4116). All links remain cost 4.

1. VLAN 10: root SW2. SW1 and SW3 use their direct links to SW2 as root ports. On the SW1-SW3 segment both offer cost 4; SW1's BID (32778, 0001) is lower than SW3's (32778, 0003), so SW1's port is designated and **SW3's port blocks**.
2. VLAN 20: root SW3. SW1 and SW2 use their links to SW3. On the SW1-SW2 segment SW1's BID is lower, so SW1 is designated and **SW2's port blocks**.
3. Result per link: SW1-SW2 forwards VLAN 10 and blocks VLAN 20 at SW2; SW1-SW3 forwards VLAN 20 and blocks VLAN 10 at SW3; SW2-SW3 forwards both.

```
SW3# show spanning-tree vlan 10 | include Gi0/1
Gi0/1   Altn BLK 4  128.1  P2p      <-- blocked for VLAN 10 only
SW3# show spanning-tree vlan 20 | include Gi0/1
Gi0/1   Desg FWD 4  128.1  P2p      <-- forwards VLAN 20
```

Result: every link carries traffic for at least one VLAN, which a single tree cannot do. The price is two trees to reason about, so the root of each VLAN should be aligned with the first-hop gateway of that VLAN.

Convergence. In 802.1D a port moves Blocking → Listening (15 s) → Learning (15 s) → Forwarding, and a topology change is detected only after the *max-age* timer (20 s) expires, so recovery takes up to 50 seconds. *Rapid STP* (802.1w) removes the timers. Its ports have three states (discarding, learning, forwarding) and four roles: root, designated, *alternate* (a backup path to the root through another switch) and *backup* (a second port on the same segment). When a link fails, the alternate port becomes the root port at once. On a new point-to-point link a *proposal/agreement* handshake lets a designated port go to forwarding after its neighbour blocks its other non-edge ports (the "sync"), which takes a fraction of a second. A neighbour is declared

dead after three missed hellos (6 s), not after max-age. An *edge port* (PortFast) goes straight to forwarding because it faces an end host.

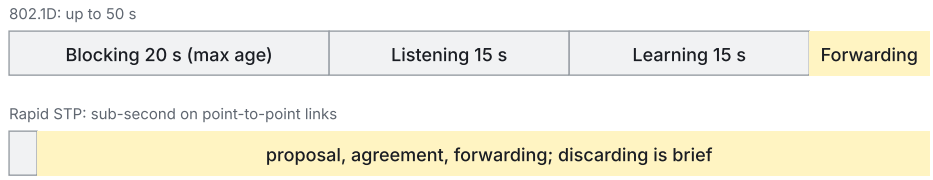


Figure 4.10 Convergence after a link failure, drawn to scale.

	802.1D	Rapid STP (802.1w)	PVST+ / Rapid PVST+	MST (802.1s)
Trees	one for all VLANs (CST)	one for all VLANs	one per VLAN	one per *instance*, each carrying many VLANs
Convergence	up to 50 s	under a second	802.1D / 802.1w speed	802.1w speed
Load sharing	none	none	by choosing a different root per VLAN	by mapping VLANs to instances
CPU and BPDU load	low	low	grows with VLAN count	low, scales to thousands of VLANs

MST groups VLANs into instances, so that 1000 VLANs can share two trees. Switches belong to the same *region* only if their region name, revision number and VLAN-to-instance mapping all match. On Cisco, `spanning-tree mode rapid-pvst` is the usual choice for a campus with up to a few hundred VLANs.

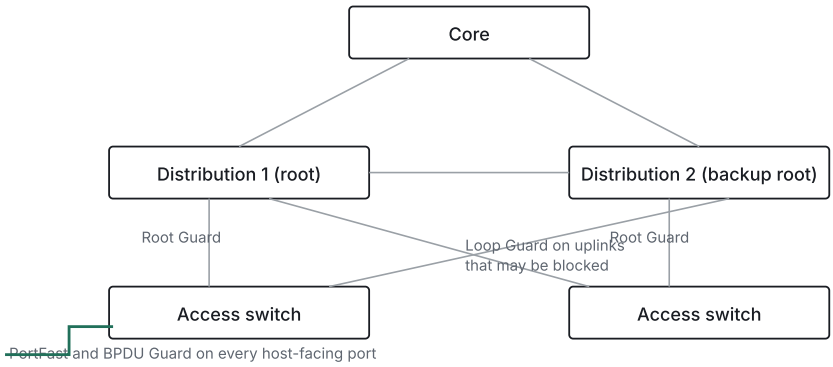


Figure 4.11 Where the guard features go in a campus: Root Guard downward from the distribution layer, Loop Guard on access uplinks, PortFast with BPDU Guard on edge ports.

Several safeguards complete the design. **PortFast** with **BPDU Guard** goes on access ports: PortFast skips the wait, and BPDU Guard puts the port in err-disabled state if a BPDU arrives, which stops a rogue switch. **Root Guard** goes on distribution downlinks to keep access switches from becoming root; a port that hears a superior BPDU moves to root-inconsistent. **Loop Guard** goes on non-designated uplinks, so that a port that suddenly stops receiving BPDUs (a unidirectional link) does not start forwarding. **UDLD** detects the unidirectional fibre itself.

```
spanning-tree mode rapid-pvst
spanning-tree vlan 1,10,20 priority 4096      ! root (use "root primary" to com-
pute it)
spanning-tree portfast default                ! all access ports become edge
ports
spanning-tree portfast bpduguard default
interface GigabitEthernet0/48                 ! distribution downlink
 spanning-tree guard root
interface GigabitEthernet0/1                  ! access uplink
 spanning-tree guard loop
```

```
SW3# show spanning-tree vlan 1
VLAN0001
  Spanning tree enabled protocol rstp
  Root ID    Priority    32769
             Address     0000.0000.0001
             Cost        4
             Port        1 (GigabitEthernet0/1)   <-- root port
Interface        Role Sts Cost      Prio.Nbr Type
Gi0/1            Root FWD 4         128.1    P2p
Gi0/2            Altn BLK 4         128.2    P2p   <-- alternate
```

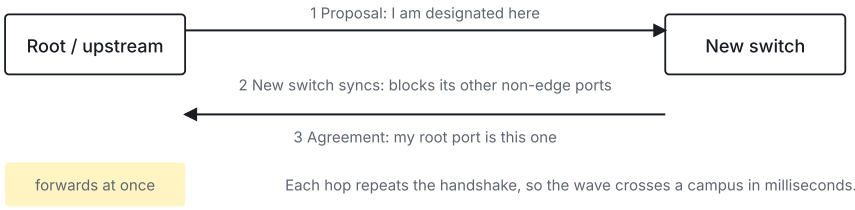


Figure 4.12 The Rapid STP proposal and agreement handshake on a point-to-point link.

EtherChannel bundles up to eight parallel links into one logical port-channel, which spanning tree treats as one link, so that no member is blocked and the bandwidth adds. Members must match in speed, duplex, VLAN list and trunk mode. Negotiation is by LACP (IEEE 802.1AX, modes `active` and `passive`), by Cisco's PAgP (`desirable`, `auto`) or statically with `on`. A bundle forms when at least one end initiates and both use the same protocol; `on` never negotiates, so it pairs only with `on`.

MST needs the instance map to be identical across the region. The mapping is configured once, and unmapped VLANs fall into instance 0 (the *internal spanning tree*, which also represents the region to the outside):

```
spanning-tree mode mst
spanning-tree mst configuration
 name CAMPUS
 revision 1
 instance 1 vlan 10,30
 instance 2 vlan 20,40
spanning-tree mst 1 priority 4096
spanning-tree mst 2 priority 8192
```

	active	passive	desirable	auto	on
active	yes	yes	no	no	no

passive	yes	no	no	no	no
desirable	no	no	yes	yes	no
auto	no	no	yes	no	no
on	no	no	no	no	yes

```
SW1(config)# interface range GigabitEthernet0/1-2
SW1(config-if-range)# channel-group 1 mode active
SW1# show etherchannel summary
Group  Port-channel  Protocol    Ports
1      Po1(SU)       LACP        Gi0/1(P)  Gi0/2(P)
```

The flags read: S = Layer 2, U = in use; P = bundled in the channel; `s` = suspended (the member's settings disagree), D = down. Traffic is spread over members by a hash of header fields (`port-channel load-balance src-dst-ip`), so one flow always uses one member, and a single flow never exceeds one link's speed. Multi-chassis versions (vPC, MLAG) that terminate a bundle on two switches are taught in §5.3 (p. 307).

A bundle is also a design tool: because the channel is one logical port, spanning tree sees one link and keeps it forwarding, and the bundle survives the failure of a member with no topology change. A *Layer 3 EtherChannel* (`no switchport` on the port-channel, then an IP address) carries routed traffic between switches without VLANs. Typical causes of a bundle that does not form or suspends members are tabulated.

Symptom in `show etherchannel summary`	Likely cause	Check
Members (`s`)	member settings differ from the bundle	`show run interface`, speed, duplex, trunk list
Members (`I`) stand-alone	partner is not running the same protocol, or silent	`show lacp neighbor`, partner mode
Port-channel (`SD`)	all members down	cabling, partner shutdown
Bundle up, one link saturated	hash puts many flows on one member	`port-channel load-balance`, `show etherchannel load-balance`
MAC flapping across members	one side `on`, other LACP	make both LACP

```
SW1(config)# port-channel load-balance src-dst-ip
SW1# test etherchannel load-balance interface Po1 ip 192.168.10.5 10.0.0.80
Would select Gi0/2 of Po1
```

> INTERVIEW LENS
>
> CISCO · JUNIPER · ARISTA
>
> STP is the most reliably asked Layer 2 topic. Expect a triangle or a square, and the request to name the root, root ports, designated ports and the blocked port. Show the four-step order aloud, because the examiner is checking the method more than the answer. Follow-ups: why 4096, why RSTP is fast, where each guard goes, and a "network freezes, MACs flap" scenario (a loop, found with `show spanning-tree` and the interface counters).

> LAB
>
> `LAB 4.3 · 20 MIN` Build the triangle, then fail the root port.

```
SW3(config)# spanning-tree mode rapid-pvst
SW3(config)# spanning-tree vlan 1 priority 4096
SW2(config)# interface GigabitEthernet0/1
SW2(config-if)# shutdown
```

```
SW3# show spanning-tree vlan 1 | include Root
  This bridge is the root
```

What you should notice: `show spanning-tree` on SW1 and SW2 now names SW3 as root. With the default 802.1D mode the alternate port takes 30 to 50 s to forward; with `rapid-pvst` pings recover in about a second. To see a storm, disable spanning tree on all three switches (`no spanning-tree vlan 1`) in simulation only.

CHECK YOURSELF

1. Why does the root bridge never have a root port? [5]
2. SW2's two links to the root carry cost 4 each. Which port blocks? [6]

4.4 The routing table, static and default routes MUST · ■□□

BIG IDEA

A router chooses among routes by three tests in a fixed order (the longest matching prefix, then the lowest administrative distance, then the lowest metric), because each test answers a different question: how specific, how trustworthy and how short.

The *routing table* holds one best route per prefix. When a packet arrives, the router finds the entry with the longest prefix that contains the destination address (the algorithm and its trie implementation are in §1.9 (p. 41)). Only if several *sources* offer the same prefix and mask does the *administrative distance* (AD) decide: it is a local trust rating from 0 to 255 assigned to each source of routes, and lower wins. Only if one protocol offers several paths to the same prefix does the *metric*, the protocol's own measure of path length, decide. Metrics from different protocols are never compared, since OSPF cost and RIP hop count measure different things.

Source	AD	Source	AD
Connected interface	0	IS-IS	115
Static route (next hop or interface)	1	RIP	120
EIGRP summary route	5	EIGRP external	170
eBGP	20	iBGP	200
EIGRP internal	90	DHCP-learned default	254
OSPF	110	Unusable	255

```
BR1# show ip route
Gateway of last resort is 192.0.2.1 to network 0.0.0.0
S*    0.0.0.0/0 [1/0] via 192.0.2.1
      10.0.0.0/8 is variably subnetted, 3 subnets, 2 masks
O        10.1.1.0/24 [110/20] via 10.1.12.2, 00:04:10, GigabitEthernet0/2
C        10.1.12.0/30 is directly connected, GigabitEthernet0/2
L        10.1.12.1/32 is directly connected, GigabitEthernet0/2
```

1. The line is up at Layer 1 but the protocol (Layer 2) is down, for example from a keepalive or encapsulation mismatch.↩
2. It stores a hash; `enable password` stores a readable (or weakly obfuscated) string.↩
3. The 4-byte 802.1Q tag is added to the maximum 1518-byte untagged frame.↩
4. VLAN 20 is missing from a trunk's allowed list or from SW2's VLAN database; or inter-VLAN routing (sub-interface or SVI) for VLAN 20 is down.↩
5. The root port is the best path *toward* the root; the root is already there.↩
6. The one connected to the root's higher sender port ID (for example 128.2 rather than 128.1).↩

Figure 4.13 One line of the routing table, token by token.

Read each line from the left. The code names the source (`C` connected, `L` local address of the router itself as a /32, `S` static, `O` OSPF, `B` BGP, `*` candidate default). Next come the prefix, then `[AD/metric]`, then the next hop, the age and the exit interface. The “gateway of last resort” is the default route.

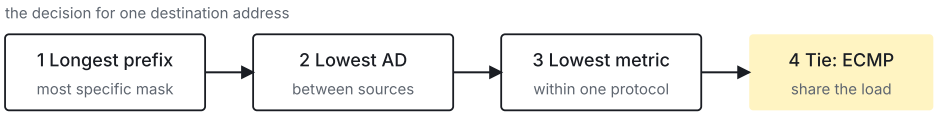


Figure 4.14 The route-selection order. Steps 2 and 3 matter only among candidates for the same prefix and mask.

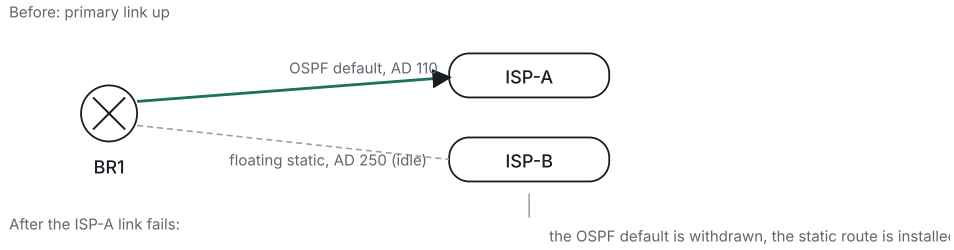


Figure 4.15 A floating static route as a standby. The route in the table changes from *OE2 to S* without any protocol exchange.

A destination that is covered by a *summary* static route can still be reached while the specific routes exist, and `Null0` prevents a loop: if a router advertises 10.0.0.0/8 but holds specifics for only part of it, a packet for an unused address would be sent to the default route and might return. The line `ip route 10.0.0.0 255.0.0.0 Null0` makes it drop instead; because the specifics are longer, they still win.

```
BR1(config)# ip route 10.0.0.0 255.0.0.0 Null0
BR1(config)# ipv6 route 2001:db8:9::/48 2001:db8:12::2
BR1(config)# ip route 172.16.0.0 255.255.0.0 10.1.12.2 10 name BACKUP-VIA-R2
BR1# show ip route 10.1.1.9
Routing entry for 10.1.1.0/24
  Known via "static", distance 1, metric 0
  Routing Descriptor Blocks:
  * 10.1.12.2
```

A *static route* is configured by hand. Pointing it at a next-hop address (`ip route 10.9.0.0 255.255.0.0 10.1.12.2`) needs a *recursive lookup* to find the exit interface; pointing it at an interface alone is acceptable only on point-to-point links, since on Ethernet the router would ARP

for every destination. A *default route* is the prefix 0.0.0.0/0, which matches everything with the shortest possible prefix, so it is used only when nothing else matches. A static route to `Null0` discards traffic, and is used to anchor a summary. The IPv6 forms are `ipv6 route 2001:db8:9::/48 2001:db8:12::2` and `ipv6 route ::/0 ...`. Equal-cost paths are installed together (ECMP, four by default on many IOS releases) and the load is spread per flow.

WORKED EXAMPLE

The routing-table puzzle. R1 holds four candidates: OSPF 10.1.0.0/16 [110/20], EIGRP 10.1.0.0/16 [90/3072], static 10.1.1.0/24 [1/0] and RIP 10.0.0.0/8 [120/2]. Where do three packets go?

1. Installed routes. The OSPF and EIGRP /16 entries have the same prefix and mask, so AD decides: EIGRP (90) is installed and OSPF (110) is held back. The static /24 and the RIP /8 have different masks, so both are installed.
2. 10.1.1.9 matches the /24, /16 and /8. The longest prefix is the /24, so it goes by the static route; AD is not consulted at all, because the candidates differ in mask.
3. 10.1.5.5 fails the /24 and matches the /16 (EIGRP) and the /8. The /16 wins.
4. 10.200.0.1 matches only the /8, so it goes to RIP.

Result: static /24, EIGRP /16, RIP /8; AD was used once, to choose between OSPF and EIGRP.

A *floating static route* is a static route given an AD higher than the dynamic protocol's, so it sits idle until the dynamic route disappears. For Campus Net's backup path to ISP-B, with the primary default learned by OSPF from the border at AD 110:

```
BR1(config)# ip route 0.0.0.0 0.0.0.0 203.0.113.9 250
```

```
BR1# show ip route 0.0.0.0                  ! primary present
O*E2  0.0.0.0/0 [110/1] via 192.0.2.1
BR1# show ip route 0.0.0.0                  ! after the primary link fails
S*    0.0.0.0/0 [250/0] via 203.0.113.9     <-- floating route installed
```

When two or more routes of the same prefix, AD and metric exist, the router installs all of them (up to `maximum-paths`) and shares traffic by a per-flow hash, so that packets of one TCP connection stay in order. This is *equal-cost multipath* (ECMP). A static route can be given a lower or higher AD to engineer which is used: `ip route 172.16.0.0 255.255.0.0 10.1.12.2 10` is installed at AD 10, so it beats an OSPF route (110) for 172.16.0.0/16 but loses to a connected route. The same lever is used for protocols (`distance 105 ...`).

WORKED EXAMPLE

A routing table with eight entries and seven lookups. Table of R1 after convergence:

Prefix	Source	AD/metric
0.0.0.0/0	OSPF E2	110/1
10.0.0.0/8	static to Null0	1/0
10.1.0.0/16	OSPF	110/20
10.1.1.0/24	connected	0/0
10.1.1.0/25	static	1/0
10.1.2.0/24	EIGRP	90/3072

10.1.2.0/24	OSPF	110/30
192.168.0.0/16	eBGP	20/0

1. 10.1.1.10 matches the /24, /16, /8 and the /25 (0 to 127): the **/25 static** wins.
2. 10.1.1.200 is outside the /25, so the **connected /24** wins.
3. 10.1.2.5: the same /24 appears from EIGRP and OSPF; AD 90 beats 110, so **EIGRP**.
4. 10.1.9.9 matches only the /16 and /8: **OSPF /16**.
5. 10.50.0.1 matches only the /8: it goes to **Null0** (discarded), even though a default route exists, because the /8 is longer than the default.
6. 192.168.5.5: **eBGP**; and 8.8.8.8 matches only the default, so **OSPF E2 default**.

Result: specifics always beat defaults; AD arbitrated only for 10.1.2.0/24.

TRAP

Claim: a /16 connected route beats a /24 static route because connected has AD 0. *Why it is wrong:* AD is consulted only between routes for the same prefix and mask. *Correct statement:* the longest prefix is tested first, so a /24 static is used for traffic it matches whatever the AD of the shorter /16.

INTERVIEW LENS

CISCO · AMAZON · JUNIPER

Asked as “order the routing decision” or as a puzzle like the one above. State the order, then say that AD is evaluated once at install time and metric only inside one protocol. Follow-ups: why a floating static can be unreliable when the primary link stays up but the path is broken (use IP SLA tracking, §4.14 (p. 282)), and what a recursive lookup is.

LAB

`LAB 4.4 · 20 MIN` Packet Tracer: four routers in a square (R1-R2-R3-R4-R1), loopbacks 10.255.0.N/32, static routes only. Then add a floating route and test the failover.

```
R1(config)# ip route 10.255.0.3 255.255.255.255 10.1.12.2
R1(config)# ip route 10.255.0.3 255.255.255.255 10.1.14.4 200
R1(config)# interface GigabitEthernet0/1
R1(config-if)# shutdown
R1# traceroute 10.255.0.3
  1 10.1.14.4 4 msec
  2 10.1.34.3 8 msec                 <-- the path changed to the floating
route
```

What you should notice: before the shutdown the first hop is 10.1.12.2 (R2); afterwards `show ip route 10.255.0.3` shows `S [200/0]`. Repeat with `ipv6 route` and compare the table codes.

CHECK YOURSELF

1. A router learns 172.16.0.0/16 by eBGP and by OSPF. Which is installed? [1]
2. Why is `ip route 10.9.0.0 255.255.0.0 Gi0/1` unwise on an Ethernet link? [2]

4.5 OSPF MUST · ■■□

BIG IDEA

> OSPF is a link-state protocol: every router floods a description of its own links to all routers in its area, so that each builds the same map and independently computes shortest paths, which makes convergence fast and loop-free inside an area.

Each router describes itself in a *link-state advertisement* (LSA); the set of LSAs it holds is its *link-state database* (LSDB). Because every router in an area holds an identical LSDB, each runs the same shortest-path-first (SPF) computation, which is Dijkstra's algorithm (§1.11 (p. 47)), with itself as the root. The link metric is the *cost*, the reference bandwidth divided by the interface bandwidth. Routers become *neighbours* by exchanging Hello packets; neighbours that go on to synchronise databases are *adjacent*. OSPF runs directly over IP (protocol 89, TTL 1), sending to the multicast groups 224.0.0.5 (all OSPF routers) and 224.0.0.6 (DR and BDR only). Five packet types exist: Hello, Database Description (DBD, a catalogue of LSA headers), Link-State Request (LSR), Link-State Update (LSU, carrying full LSAs) and Link-State Acknowledgement (LSAck).

Hello and dead timers are 10 s and 40 s on broadcast and point-to-point links, and 30 s and 120 s on non-broadcast multi-access links. The *router ID* (RID), a 32-bit identifier, is chosen in this order: the `router-id` command, else the highest loopback address, else the highest active interface address.

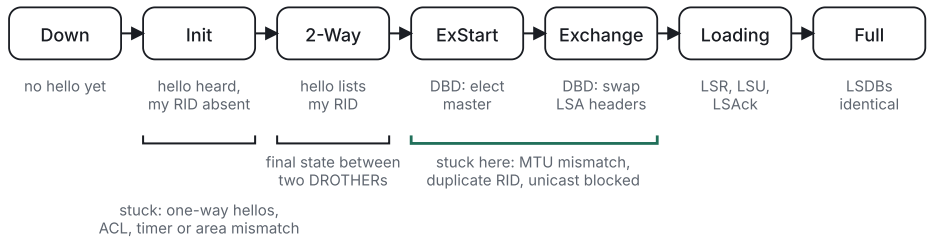


Figure 4.16 The OSPF neighbour state machine. A neighbour is adjacent only at Full; on a multi-access segment, 2-Way is the normal final state between two routers that are neither DR nor BDR.

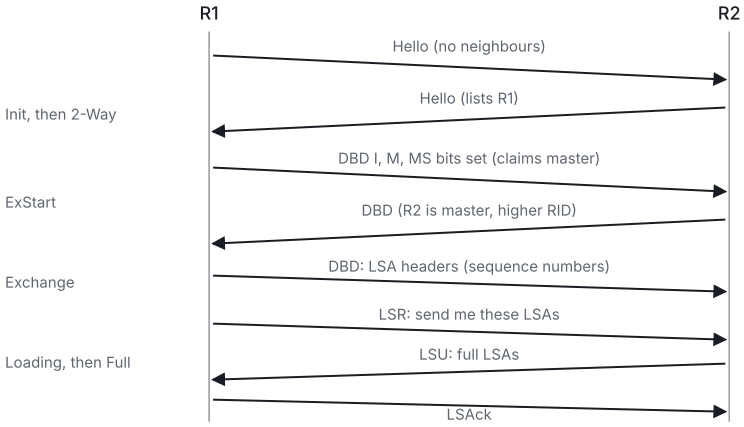


Figure 4.17 The packets of an OSPF adjacency, which Wireshark shows with the filter ospf. Bits in the DBD: I = initial, M = more, MS = master.

States, in order: *Down*; *Attempt* (non-broadcast networks only); *Init* (a hello has been heard, but it does not yet list our RID); *2-Way* (hellos are bidirectional; DR and BDR are elected here); *ExStart* (the two choose a master and initial sequence number); *Exchange* (DBDs list the LSA headers each holds); *Loading* (missing LSAs are fetched with LSR and delivered by LSU); *Full*.

IDs must differ. To reach Full, the interface MTU must also agree, since the MTU is carried in the DBD.

WORKED EXAMPLE

An adjacency stuck in EXSTART. BR1's Gi0/1 has MTU 1500 and the ISP-side router's has MTU 1400.

1. In the DBD, each side advertises its interface MTU. A router that receives a DBD advertising a *larger* MTU than its own interface can carry discards it.
2. Thus the 1400-byte side rejects BR1's DBDs. BR1 accepts the smaller ones and may reach Exchange, but its DBDs are never acknowledged, so it retransmits until the neighbour is declared down.
3. The symptom is a neighbour that cycles EXSTART/EXCHANGE and drops with "too many retransmissions". Hellos work, so the neighbour appears briefly healthy.

```
BR1# show ip ospf neighbor
Neighbor ID  Pri State        Dead Time  Address    Interface
2.2.2.2      1   EXSTART/DR   00:00:36   10.1.12.2  Gi0/1      <-- stuck
BR1# debug ip ospf adj
OSPF-1 ADJ Gi0/1: Nbr 2.2.2.2 has smaller interface MTU
```

Result: the fix is to make the MTUs equal (`ip mtu 1400` on both). `ip ospf mtu-ignore` removes the check but hides a real path-MTU problem, so treat it as a smell.

On a broadcast segment (Ethernet) with *n* routers, full adjacency among all would need n(n-1)/2 adjacencies and flood each LSA many times. OSPF therefore elects a *designated router* (DR) and a *backup* (BDR). Every other router (a *DROTHER*) becomes adjacent only with the DR and BDR: it sends updates to 224.0.0.6, and the DR re-floods them to 224.0.0.5. The election uses the interface *priority* (default 1; 0 means ineligible), then the highest RID. It is *non-preemptive*: a router joining later never displaces the current DR, whatever its priority. Point-to-point links need no election, and `ip ospf network point-to-point` on an Ethernet link between two routers avoids one.

WORKED EXAMPLE

DR election. R1 (priority 1, RID 1.1.1.1), R2 (1, 2.2.2.2), R3 (0), R4 (1, 4.4.4.4) boot together on one LAN.

1. R3 is excluded by priority 0. Among R1, R2 and R4 the priorities tie, so the highest RID wins: DR = R4, BDR = R2 (the next highest).
2. R5 joins with priority 255. Election is not preemptive, so nothing changes: R4 stays DR.
3. R4 fails. The BDR, R2, is promoted to DR, and a new BDR is elected among the rest, where R5's priority 255 beats R1's 1: BDR = R5.

Result: DR R4 then R2; BDR R2 then R5; R3 is never DR or BDR.

Network type	Default on	DR election	Hello / dead	Notes
Broadcast	Ethernet	yes	10 / 40	multicast hellos
Point-to-point	serial, tunnels	no	10 / 40	no DR; faster adjacency
Non-broadcast (NBMA)	Frame Relay, legacy	yes	30 / 120	neighbours configured by hand

Point-to-multipoint	configured	no	30 / 120	each neighbour a /32 route
Loopback	loopback	n/a	n/a	advertised as a /32 host route

Both ends of a link must agree on the *behaviour* implied by the type, for example both using or both skipping the DR election, although the hello timers can differ only if the dead timers are adjusted too (the timers are compared, not the type). A broadcast type on one end and point-to-point on the other forms an adjacency that does not work correctly, because one end expects the DR and the other does not.

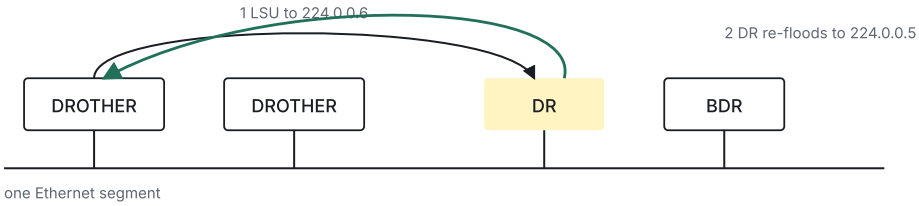


Figure 4.18 DR and BDR. A DROTHER sends to the DR and BDR group (224.0.0.6); the DR re-floods to all (224.0.0.5), so each LSA crosses the segment twice instead of n times.

The cost of an interface is the reference bandwidth divided by the interface bandwidth, with a minimum of 1. The default reference is 100 Mb/s, so Fast Ethernet is 1, and both Gigabit and 10 Gigabit Ethernet are also 1, because the quotient is truncated. The command `auto-cost reference-bandwidth 100000` (100 Gb/s) gives 1 Gb/s a cost of 100 and 10 Gb/s a cost of 10; it must be the same on every router in the domain.

WORKED EXAMPLE

An SPF calculation. Link costs: R1-R2 10, R1-R3 5, R3-R2 3, R2-R4 1, R3-R4 10. Which path does R1 use to reach R4, where a loopback 10.255.0.4/32 of cost 1 lives?

1. Candidate paths: R1-R2-R4 costs 10 + 1 = 11; R1-R3-R4 costs 5 + 10 = 15; R1-R3-R2-R4 costs 5 + 3 + 1 = 9.
2. Dijkstra settles R3 first (cost 5), then R2 via R3 (8, better than the direct 10), then R4 (9).
3. The loopback adds its own cost of 1.

```
R1# show ip route ospf
O   10.255.0.4/32 [110/10] via 10.1.13.3, 00:00:21, GigabitEthernet0/2
```

Result: next hop is R3; the metric is 9 + 1 = 10. The longer-looking path wins because its links are cheaper.

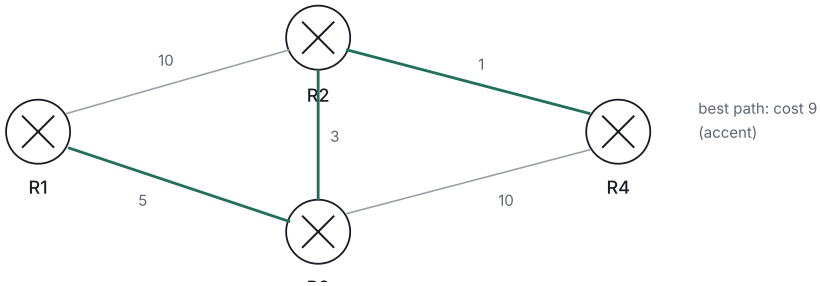


Figure 4.19 The shortest-path tree rooted at R1. The path R1-R3-R2-R4 costs 9; R1-R2-R4 would cost 11.

Large domains are divided into *areas* to limit the size of each LSDB and to confine the effect of flapping links. Area 0, the *backbone*, must connect every other area, because inter-area routes travel as distance vectors through it. A router with interfaces in two areas is an *area border router* (ABR); a router that injects routes from outside OSPF is an *autonomous system boundary router* (ASBR). If an area cannot physically touch area 0, a *virtual link* through a transit area patches the gap.

Type	Name	Originated by	Describes	Flooded
1	Router	every router	its own links and costs	within the area
2	Network	the DR	the routers on a multi-access segment	within the area
3	Summary	an ABR	a prefix from another area (with cost)	into adjacent areas
4	ASBR summary	an ABR	the *location* of an ASBR	into adjacent areas (not the ASBR's own)
5	AS external	an ASBR	a route from outside OSPF	whole domain except stub areas
7	NSSA external	an ASBR inside an NSSA	an external route	inside the NSSA; the ABR converts it to type 5

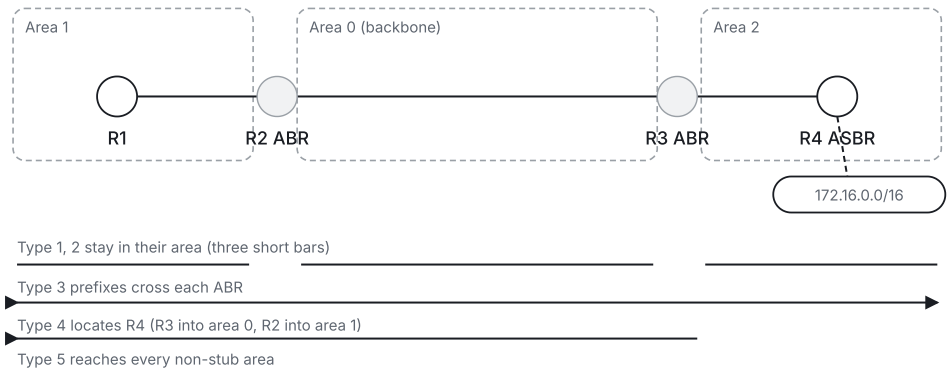


Figure 4.20 Flooding scope of the LSA types in a three-area network. R1 in area 1 holds type 3 LSAs for area 2's prefixes, type 4 for R4 and type 5 for 172.16.0.0/16.

The area types control which LSAs may enter, to shrink the LSDB of small routers. A *stub area* blocks types 4 and 5 and the ABR injects a default route as a type 3. A *totally stubby area* (Cisco: `stub no-summary`) also blocks types 3, leaving only intra-area LSAs and one default. A *not-so-stubby area* (NSSA) blocks types 4 and 5 but lets an ASBR inside the area originate type 7, which the ABR translates to type 5 on entering the backbone. A *totally NSSA* also blocks type 3, with a default from the ABR.

Area type	LSA 1, 2	LSA 3	LSA 4, 5	LSA 7	Default route
Normal	yes	yes	yes	no	none (optional)
Stub	yes	yes	no	no	type 3, automatic
Totally stubby	yes	default only	no	no	type 3, automatic

NSSA	yes	yes	no	yes	type 7, if configured
Totally NSSA	yes	default only	no	yes	type 3, automatic

WORKED EXAMPLE

Converting area 1 to totally stubby. In Figure 4.20, R1 holds type 1 (R1, R2), type 3 (prefixes of areas 0 and 2), type 4 (R4) and type 5 (172.16.0.0/16).

1. Configure `area 1 stub` on R1 and on the ABR R2. Hello packets carry a stub flag, so both must agree or the adjacency fails.
2. On R2 only, add `no-summary`. R2 stops flooding types 3, 4 and 5 into area 1 and injects a single type 3 default instead.

```
R2(config-router)# area 1 stub no-summary
R1# show ip route ospf
O*IA  0.0.0.0/0 [110/2] via 10.1.12.2, GigabitEthernet0/1
```

Result: R1's database shrinks to type 1 and 2 LSAs plus one default; traffic to everything else leaves through the ABR.

WORKED EXAMPLE

An LSA inventory before and after a stub conversion. Take Figure 4.20, with an extra internal prefix 10.2.0.0/24 in area 2 and the external prefix 172.16.0.0/16 injected by R4.

Router	Type 1	Type 2	Type 3	Type 4	Type 5
R1 (area 1), normal	1, 2	none	area 0 and 2 prefixes (via R2)	R4	172.16.0.0/16
R1 (area 1), stub	1, 2	none	same, plus default	none	none
R1 (area 1), totally stubby	1, 2	none	default only	none	none
R3 (ABR, areas 0 and 2)	its own LSAs per area	if multi-access	area-1 and area-2 prefixes	R4	172.16.0.0/16

1. In the normal state R1 must run SPF on a database that includes every inter-area prefix, so every flap in area 2 reaches R1.
2. As a stub, the type 4 and 5 LSAs vanish and R2 originates a default (type 3, cost 1 by default).
3. As totally stubby, a single default replaces all type 3 LSAs: the area survives any change elsewhere.

```
R2# show ip ospf database
            Router Link States (Area 1)
Link ID     ADV Router    Age   Seq#        Link count
1.1.1.1     1.1.1.1       412   0x80000007  2
2.2.2.2     2.2.2.2       403   0x80000009  2
            Summary Net Link States (Area 1)
0.0.0.0     2.2.2.2       380   0x80000001               <-- the default in-
jected into the stub
            Type-5 AS External Link States
172.16.0.0  4.4.4.4       951   0x80000003
```

Result: the database of the stub router shrinks from every prefix in the domain to its own area's LSAs plus one default; this is the mechanism behind the flapping-routes answer in Q4.26.

WORKED EXAMPLE

An NSSA with its own ASBR. Campus Net's engineering branch (area 2) has a small partner network attached through R5, which redistributes 172.20.0.0/16 into OSPF. A stub area cannot contain an ASBR, because type 5 LSAs are forbidden there, so area 2 is made an NSSA.

1. R5 (ASBR) originates the prefix as a **type 7** LSA, flooded only inside area 2.
2. The NSSA ABR (R3) translates it into a **type 5** LSA and floods that into area 0, so the rest of the domain sees an ordinary external route, `O E2`.
3. Inside area 2 the same route shows as `O N2`.

```
R3(config-router)# area 2 nssa
R5(config-router)# redistribute static subnets
R5# show ip route ospf | include 172.20
O N2  172.20.0.0/16 [110/20] via 10.2.0.1
R1# show ip route ospf | include 172.20
O E2  172.20.0.0/16 [110/20] via 10.1.12.2
```

Result: R1 never saw a type 7; the NSSA ABR is the only place it exists in translated form.

WORKED EXAMPLE

Summarising at the ABR. Area 3 holds 200 customer routes in 10.3.0.0/16. Without summarisation each of them becomes a type 3 LSA in area 0 and beyond.

```
R3(config-router)# area 3 range 10.3.0.0 255.255.0.0
R3(config-router)# area 3 range 10.3.0.0 255.255.0.0 cost 50
R3(config)# ip route 10.3.0.0 255.255.0.0 Null0
```

1. The ABR now advertises only 10.3.0.0/16 (type 3) into area 0, so a flap of one /24 inside area 3 stays in area 3.
2. The summary is kept as long as at least one component prefix is up; its cost is the highest of the components, unless a fixed cost is set.
3. The Null0 route stops a loop for addresses inside the /16 that have no specific route (the ABR discards instead of following its default route).

Result: backbone routers hold one LSA instead of 200, and area 3's instability is invisible to them.

Authentication of OSPF messages is configured per interface or per area. Modern IOS-XE supports SHA keychains (`ip ospf authentication key-chain`), and older MD5 uses `ip ospf message-digest-key 1 md5 SECRET`. Authentication failures keep the neighbour in Down or Init because the hellos are discarded. For IPv6, the equivalent is:

```
ipv6 unicast-routing
interface GigabitEthernet0/2
 ipv6 address 2001:db8:12::1/64
 ipv6 ospf 1 area 0
 ipv6 ospf authentication ipsec spi 256 sha1 <key>
ipv6 router ospf 1
 router-id 1.1.1.1
```

Routes carry a code that shows how they were learnt: `O` (intra-area), `O IA` (inter-area, from a type 3), `O E1` and `O E2` (external, type 5) and `O N1` / `O N2` (NSSA). A path's preference is `O`, then `O IA`, then `E1`, then `E2`, regardless of cost. An E2 route carries only the external cost, which is the default and is what is compared by all routers; an E1 route adds the internal cost to the ASBR, and is preferred when there are several ASBRs at different distances.

Three techniques contain flapping. *Summarisation* at the ABR (`area 3 range 10.3.0.0 255.255.0.0`) or at the ASBR (`summary-address`) hides detail so that one flapping /24 inside the range does not trigger SPF elsewhere. A *stub area* removes external LSAs. *Throttling* (`timers throttle spf 50 200 5000`) delays repeated SPF runs, and BFD detects failures in milliseconds so that timers can stay conservative. OSPFv3 uses the same states and algorithm for IPv6, but runs per link rather than per subnet, uses link-local addresses as next hops, sends to ff02::5 and ff02::6, and moves addresses out of the router and network LSAs into new LSA types (8, the link LSA, and 9, the intra-area-prefix LSA).

```
BR1(config)# router ospf 1
BR1(config-router)# router-id 1.1.1.1
BR1(config-router)# auto-cost reference-bandwidth 100000
BR1(config-router)# passive-interface default
BR1(config-router)# no passive-interface GigabitEthernet0/2
BR1(config)# interface GigabitEthernet0/2
BR1(config-if)# ip ospf 1 area 0
BR1(config-if)# ip ospf network point-to-point
```

Verification uses three commands: `show ip ospf neighbor` (state and DR role), `show ip ospf database` (the LSAs by type, and their age) and `show ip ospf interface` (cost, timers, network type, DR). The commonest adjacency faults are, in order: mismatched hello or dead timers, mismatched area, mismatched MTU, a passive interface, duplicate router IDs and a wrong authentication key.

Junos and FRR — Junos: `set protocols ospf area 0 interface ge-0/0/2.0`. FRR: `router ospf` then `network 10.1.12.0/30 area 0`, or `ip ospf area 0` on the interface.

TRAP

Claim: a router with priority 255 becomes DR as soon as it is configured. *Why it is wrong:* the DR election is not preemptive, so the current DR stays until it fails or OSPF is cleared. *Correct statement:* priority matters at election time; to force a change, clear the process (`clear ip ospf process`) after setting priorities. Similarly, 2-Way between two DROTHERs is correct and not a fault.

WORKED EXAMPLE

The six-fault adjacency challenge. A pre-built Campus Net lab has six independent faults; each appears as a neighbour that is missing or stuck. For each, the fastest diagnostic and the fix are below.

Symptom	Evidence	Cause	Fix
No neighbour, interface up	`show ip ospf interface` says hello 10, dead 40 on one side and 5, 20 on the other	hello/dead mismatch	match `ip ospf hello-interval` and `dead-interval`
No neighbour	`%OSPF-4-ERRRCV: mismatch area ID`	area mismatch	`ip ospf 1 area N` the same on both
Stuck in EXSTART	`debug ip ospf adj`: smaller interface MTU	MTU mismatch	equalise `ip mtu`
No neighbour, passive	`show ip protocols`: interface listed as		

	passive	passive interface	`no passive-interface`
Neighbour flaps in 2-Way/Down	log: duplicate router ID	two routers share a RID	set unique `router-id`, `clear ip ospf process`
Down or Init, hellos dropped	`debug ip ospf events`: mismatched authentication	wrong key or type	correct the key-chain

Result: in each case one `show` or `debug` line identifies the fault; always read the debug before changing configuration.

INTERVIEW LENS

CISCO · JUNIPER · NOKIA · AMAZON

The state list and the "stuck in EXSTART/INIT/2-WAY" question are asked at all of these. Strong answers give the cause for each (MTU, one-way hello or filter, normal for DROTHERs) and a command that proves it. The LSA-types table is a near certainty at Cisco and Juniper; learn who originates each and where it floods. The reported scenario "200 routes in one area flap and SPF runs everywhere" is solved by summarisation at the ABR, stub area and throttling.

LAB

`LAB 4.5 · 30 MIN` FRR Containerlab, three routers R1, R2, R3 in areas 1, 0, 2.

```
vtysh -c "show ip ospf database"
vtysh -c "configure terminal" -c "router ospf" -c "area 1 stub no-summary"
vtysh -c "show ip ospf database"
```

```
Summary Link States (Area 1)        <-- before: many; after: only 0.0.0.0
Type-5 AS External Link States      <-- absent in area 1 after conversion
```

What you should notice: the database of the stub router shrinks, and a default route appears. Then capture with Wireshark (`ospf` display filter) while you bring up a link, and match the DBD, LSR, LSU and LSAck packets to Figure 4.16.

CHECK YOURSELF

1. With the default reference bandwidth, what is the cost of a 1 Gb/s link, and why does it matter? [3]
2. Which LSA type does R1 in a totally stubby area still see besides types 1 and 2? [4]

1. eBGP (AD 20), because the prefix and mask are identical and 20 is below OSPF's 110.↩
2. The router treats 10.9.0.0/16 as directly connected and ARPs for every destination in it; this loads the segment and needs proxy ARP on the neighbour.↩
3. It is 1, because 100 / 1000 is rounded up to the

4.6 EIGRP OPTIONAL · ■■□

BIG IDEA

EIGRP is Cisco's advanced distance-vector protocol: it keeps a pre-verified loop-free backup path, the feasible successor, so that when the best path fails the router switches at once without asking its neighbours.

EIGRP (IP protocol 88, multicast 224.0.0.10) forms neighbours when the autonomous-system number, the K values, the subnet and any authentication match. It sends Hello, Update, Query, Reply and ACK packets, using its own reliable transport. The *DUAL* algorithm works on these quantities for each destination: the *reported distance* (RD) is the neighbour's own metric to the destination; the *feasible distance* (FD) is our best total metric; the *successor* is the neighbour on the best path. A neighbour is a *feasible successor* if its RD is below our FD, which is the *feasibility condition*. This guarantees the neighbour's path does not pass through us, because a path through us would have an RD at least as large as our FD, and so it is loop-free by construction.

With the default K values (K1 = K3 = 1) the classic metric is 256 × (10^7 / lowest bandwidth in kb/s + sum of delays in tens of microseconds), each part as an integer. Wide metrics (64-bit) replace this on named-mode configurations so that 10 Gb/s and faster links remain distinguishable.

WORKED EXAMPLE

An EIGRP metric and a feasibility test.

1. *Metric.* A path crosses a T1 (1544 kb/s, delay 20,000 µs) and a Gigabit link (delay 10 µs). Bandwidth term: 10^7 / 1544 = 6476. Delay term: (20,000 + 10) / 10 = 2001. Metric = 256 × (6476 + 2001) = 2,170,112.
2. *Feasibility.* Paths to 10.9.9.0/24 are: via R2, FD 3000 (RD 1000), the successor; via R3, RD 2500; via R4, RD 3500. R3's RD 2500 is below 3000, so R3 is a feasible successor. R4's RD 3500 is not, so it is not feasible (it might loop).
3. *Variance.* With `variance 2` EIGRP may also use paths whose FD is below 2 × 3000 = 6000. R3 qualifies if its FD is below 6000. R4 is never used, because unequal-cost load balancing still requires feasibility.

Result: 2,170,112; the feasible successor is R3; the used paths are R2 and R3.

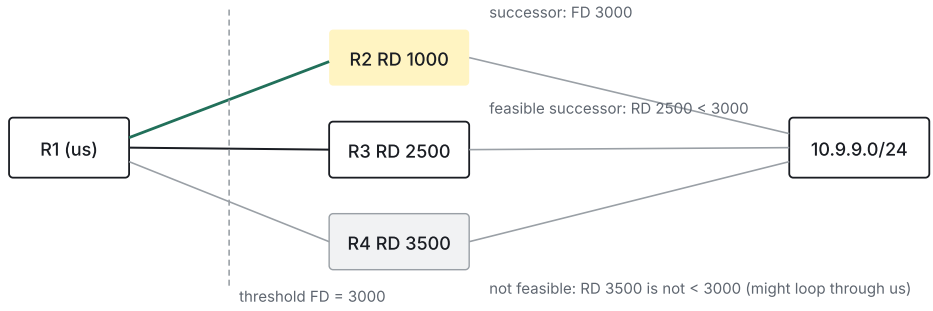


Figure 4.21 The DUAL feasibility condition. A neighbour is a safe backup only if it is closer to the destination than our own best distance.

If the successor fails and a feasible successor exists, it is promoted instantly and no packet is sent. If none exists, the router goes *active* for the route and sends *queries* to its neighbours, which may query theirs in turn. A route whose replies do not arrive in three minutes becomes *stuck in active*

minimum of 1; hence 1 Gb/s, 10 Gb/s and 100 Mb/s links all look equal and OSPF cannot prefer the faster one.↩

4. One type 3 LSA, the default route from the ABR.↩

(SIA) and the neighbour is reset. Two defences limit this: *summarisation* (a query for a prefix covered by a summary is answered at once) and *EIGRP stub* routers, which are not asked at all.

If no feasible successor exists, the router does not simply fail over; it queries. The query spreads outwards until a router either has an answer or is a *stub*. Thus the size of the *query domain* determines how long convergence takes and how likely SIA is, which is why large EIGRP networks use summarisation at distribution layers and stubs at branches.

```
R1# show ip eigrp topology 10.9.9.0/24
P 10.9.9.0/24, 1 successors, FD is 3000
        via 10.1.2.2 (3000/1000), GigabitEthernet0/1       <-- successor
        via 10.1.3.3 (5400/2500), GigabitEthernet0/2       <-- feasible suc-
cessor (2500 < 3000)
```

```
router eigrp CAMPUS
 address-family ipv4 unicast autonomous-system 100
  network 10.1.0.0 0.0.255.255
  af-interface default
   passive-interface
  exit-af-interface
  af-interface GigabitEthernet0/2
   no passive-interface
```

LAB

`LAB 4.6 · 25 MIN` Packet Tracer, five routers in a ring with a chord. Run EIGRP AS 100, inspect the topology table, then shut the successor link.

```
R1# show ip eigrp topology all-links
R1(config)# interface GigabitEthernet0/1
R1(config-if)# shutdown
R1# debug eigrp packets query
```

What you should notice: when a feasible successor exists the route flips with no debug output (no queries). Remove the chord so none exists, shut again, and queries appear. Make the far router an `eigrp stub` and the query disappears.

	OSPF	EIGRP
Algorithm	link-state, SPF	DUAL (distance-vector with loop-free backups)
Standard	open (RFC 2328)	Cisco-origin, informational RFC 7868
Metric	cost from bandwidth	bandwidth and delay (load, reliability optional)
Load sharing	equal-cost only	also unequal-cost (variance)
AD	110	90 internal, 170 external
Hierarchy	areas, strict	summarisation anywhere, no areas

4.7 BGP: the routing protocol of the Internet MUST · ■■■

BIG IDEA

> BGP is a path-vector protocol between autonomous systems that carries policy attributes with each route, and the best route to each prefix is chosen by an ordered list of tie-breakers; it is thus the tool with which operators choose where traffic goes, rather than simply find the shortest path.

An *autonomous system* (AS) is a set of networks under one administration with a single routing policy, identified by an AS number: 16-bit (1 to 65535, with 64512 to 65534 private) or 32-bit (private range 4,200,000,000 and above). BGP between routers of different ASes is *eBGP*; between routers of the same AS it is *iBGP*. BGP runs over TCP port 179, which provides reliable delivery and removes the need for BGP to retransmit. A route is advertised as *NLRI* (a prefix and length) together with *path attributes*, the most important of which, the AS_PATH, is the list of ASes the advertisement has crossed. A router rejects any route whose AS_PATH contains its own AS, which is the loop-prevention rule.

Messages: OPEN (version, AS number, hold time, BGP identifier, capabilities), UPDATE (withdrawn prefixes, path attributes, NLRI), KEEPALIVE, NOTIFICATION (an error; the session closes) and ROUTE-REFRESH. Cisco's default timers are keepalive 60 s and hold time 180 s, and the lower of the two peers' hold times is used.

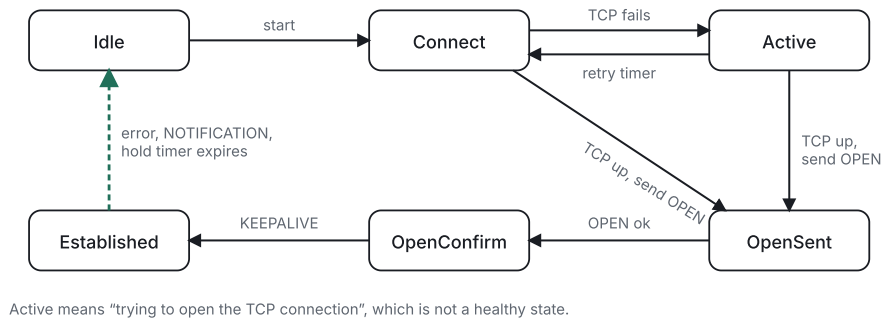


Figure 4.22 The BGP finite-state machine. Only Established exchanges UPDATEs; any error returns the session to Idle.

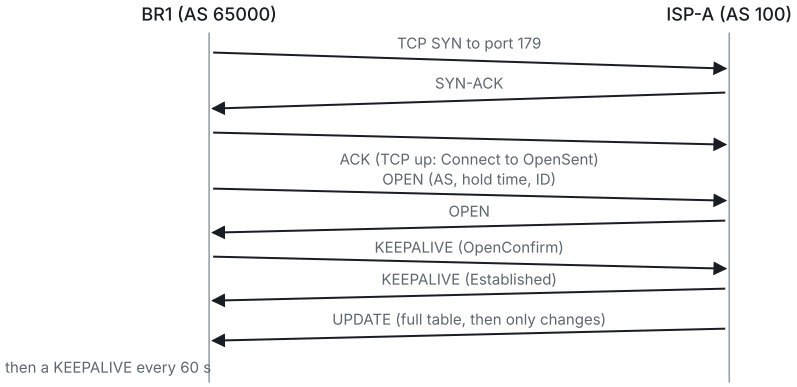


Figure 4.23 A BGP session. If both sides open a TCP connection at once, the one initiated by the router with the higher BGP identifier is kept.

Neighbours are configured explicitly (BGP does not discover them). An eBGP session between directly connected addresses needs only the address and the AS number. An iBGP session is best run between *loopbacks*, because a loopback stays up as long as any path exists, so `update-source Loopback0` tells BGP to originate the TCP session from that address. eBGP sessions

between loopbacks (to use several parallel links) need `ebgp-multihop 2`, since the default TTL of 1 would expire at the first router.

```
BR1(config)# router bgp 65000
BR1(config-router)# bgp router-id 10.255.0.1
BR1(config-router)# neighbor 192.0.2.1 remote-as 100           ! eBGP, direct
BR1(config-router)# neighbor 10.255.0.2 remote-as 65000        ! iBGP, loopback
BR1(config-router)# neighbor 10.255.0.2 update-source Loopback0
BR1# show ip bgp summary
BGP router identifier 10.255.0.1, local AS number 65000
Neighbor     V   AS MsgRcvd MsgSent TblVer InQ OutQ Up/Down  State/PfxRcd
192.0.2.1    4  100    1021     988     12   0    0 02:11:07        1500   <--
Established
10.255.0.2   4 65000      95      93     12   0    0 01:20:44           2
10.255.0.9   4 65000       0       0      0   0    0 never      Active   <--
trying
```

The states are: *Idle* (no resources allocated; waiting for a start event, and a persistent Idle often means no route to the neighbour or a refused session); *Connect* (a TCP connection is being attempted); *Active* (the attempt failed and the router is retrying and listening: not a good state); *OpenSent* (OPEN sent, awaiting the peer's); *OpenConfirm* (OPEN accepted, awaiting a KEEPALIVE); *Established*. In `show ip bgp summary` the last column shows the number of received prefixes when Established and the state name otherwise.

Reading the BGP table is a skill in its own right. The status codes at the left (`*` valid, `>` best, `i` learnt from iBGP, `r` RIB failure, `s` suppressed) and the columns are explained in the annotated output below.

```
BR1# show ip bgp
   Network          Next Hop        Metric LocPrf Weight Path
*> 0.0.0.0          192.0.2.1            0    200      0 100 i
*> 198.51.100.0/24  0.0.0.0              0         32768 i
*  203.0.113.0/24   192.0.2.5           10    100      0 200 300 i
*>i                 10.255.0.2           0    100      0 100 300 i
 |  |                  |                |     |       |   |    |
 |  |                  |                |     |       |   |    +-- origin: i
IGP, e EGP, ? incomplete
 |  |                  |                |     |       |   +------- AS_PATH,
nearest AS first
 |  |                  |                |     |       +----------- weight (32768
= locally originated)
 |  |                  |                |     +------------------- LOCAL_PREF
(iBGP only)
 |  |                  |                +------------------------- MED
 |  |                  +-------------------------------------------- next hop
(0.0.0.0 = local)
 |  +------------------------------------------------------------ > best, * valid,
i via iBGP
```

	eBGP	iBGP
Peers in	different ASes	the same AS
Administrative distance	20	200
TTL of packets	1 (use `ebgp-multihop` or loopbacks)	255
AS_PATH on advertisement	own AS prepended	unchanged

NEXT_HOP on advertisement	changed to the sender	**unchanged**
Re-advertise to a peer of the same type	yes	**no** (iBGP-learned routes are not passed to iBGP peers)

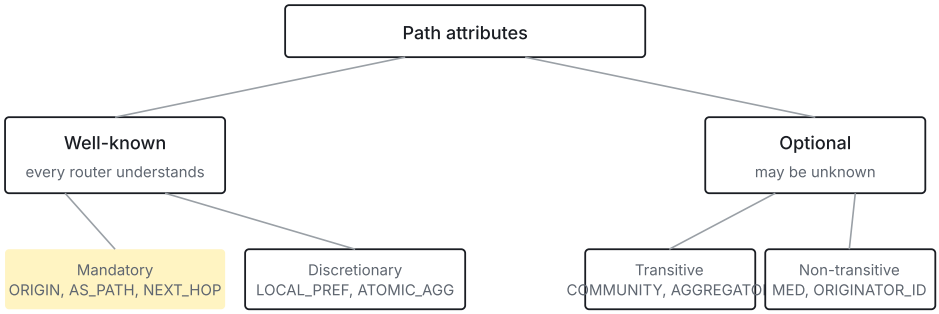


Figure 4.24 Taxonomy of path attributes. Well-known attributes must be recognised; an optional transitive one is passed on even if unknown, an optional non-transitive one is dropped.

The last row explains the iBGP *full mesh*. Because a route learnt from one iBGP peer is not passed to another, every iBGP router must peer with every other, which needs n(n-1)/2 sessions: 45 for ten routers. Two remedies exist. A *route reflector* (RR) is allowed to break the rule: it reflects routes from *clients* to other clients and non-clients, and from non-clients to clients only, while a route learnt by eBGP goes to everyone. To avoid loops the RR tags a reflected route with an ORIGINATOR_ID (the first router's ID) and prepends its CLUSTER_ID to a CLUSTER_LIST; a router discards a route with its own ID, and an RR discards a route listing its own cluster. *Confederations* split the AS into sub-ASes that behave like eBGP among themselves.

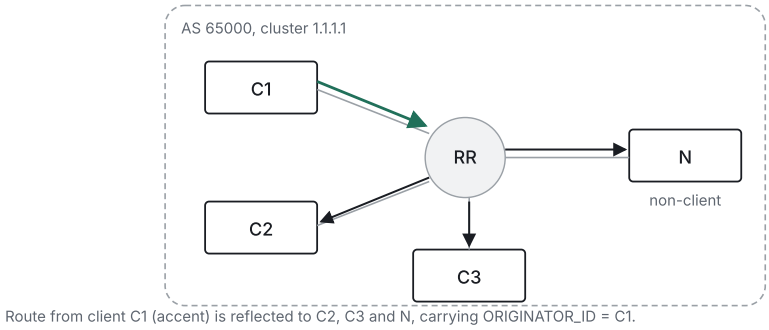


Figure 4.25 A route reflector. A client route goes to all peers; a route from the non-client N would go to clients only. The configuration is one line per client: `neighbor 10.255.0.11 route-reflector-client`.

WORKED EXAMPLE

Counting sessions. Ten routers in a full mesh need 10 × 9 / 2 = 45 iBGP sessions. With two route reflectors and eight clients, each client peers with both RRs (16 sessions) and the RRs peer with each other (1): 17 sessions.

Result: 45 against 17. The saving grows quadratically, which is why large networks use RRs.

Route reflection is configured on the reflector only; clients need no special configuration, and they cannot tell a reflector from a normal iBGP peer. The next hop is not changed by the RR, so an eBGP route reflected to a client still needs a reachable next hop.

WORKED EXAMPLE

A reflector trace. BR1 learns 203.0.113.0/24 from ISP-A and is a client of RR1 (router ID 10.255.0.100, cluster 10.255.0.100). RR1 also serves clients R3 and BR2.

1. BR1 sends the route to RR1 by iBGP with next hop BR1's loopback (`next-hop-self`).
2. RR1 reflects it to R3 and BR2, adding ORIGINATOR_ID = 10.255.0.1 and CLUSTER_LIST = {10.255.0.100}.
3. If a faulty configuration lets BR2 send the route back to RR1, RR1 sees its own cluster ID in the CLUSTER_LIST and discards it. If BR1 received its own route reflected back, it would see its own ORIGINATOR_ID and discard it.

```
R3# show ip bgp 203.0.113.0/24
  100 300
    10.255.0.1 (metric 20) from 10.255.0.100 (10.255.0.100)
      Origin IGP, metric 0, localpref 100, valid, internal, best
      Originator: 10.255.0.1, Cluster list: 10.255.0.100
```

Result: the two attributes give the loop protection that the AS_PATH cannot, since the AS_PATH does not change inside the AS.

A *confederation* is the other answer to the mesh. The AS is split into member sub-ASes (private numbers), full-meshed or reflected inside, with eBGP-like sessions between them that preserve LOCAL_PREF and next hop. The outside world sees only the confederation identifier. It is used rarely and mostly in legacy networks.

Path attributes come in four categories. *Well-known mandatory* attributes (ORIGIN, AS_PATH, NEXT_HOP) must be understood and sent with every route; *well-known discretionary* (LOCAL_PREF, ATOMIC_AGGREGATE) must be understood but need not be sent; *optional transitive* (COMMUNITY, AGGREGATOR) are passed on even if not understood; *optional non-transitive* (MED, ORIGINATOR_ID, CLUSTER_LIST) are dropped if not understood. **WEIGHT** is a Cisco-local value that is never sent to a peer.

Attribute	Scope and use	Winner
WEIGHT	one router; Cisco only; default 0 (32768 for local routes)	highest
LOCAL_PREF	whole AS; sent only in iBGP; default 100; steers *outbound* traffic	highest
AS_PATH	list of ASes crossed; length is a distance proxy	shortest
ORIGIN	IGP (i) < EGP (e) < incomplete (?)	lowest
MED	sent to a neighbouring AS to suggest an entry point; steers *inbound*	lowest
COMMUNITY	a 32-bit tag that carries policy requests	n/a

When a router holds several routes to a prefix, it selects *one* by this list; the first test that gives a unique winner stops the process. The Cisco order is shown. A short mnemonic is **"N WLLA OMNI"** for steps 0 to 8, then **"Old Routers Clear Neighbours"** for the final four.

Step	Test	Rule
0	**N**ext hop reachable	unreachable routes are ignored
1	**W**eight	highest
2	**L**ocal preference	highest
3	**L**ocally originated	prefer own (network, aggregate, redistribute)
4	**A**S_PATH length	shortest
5	**O**rigin	lowest (IGP, then EGP, then incomplete)
6	**M**ED	lowest, compared only between routes from the *same neighbouring AS*
7	**N**eighbour type	eBGP over iBGP
8	**I**GP metric to next hop	lowest
9	multipath check	install several if `maximum-paths` allows
10	**O**ldest	the oldest eBGP route, for stability
11	**R**outer ID	lowest
12	**C**luster-list length	shortest
13	**N**eighbour address	lowest

WORKED EXAMPLE

A path-selection race. BR1 (AS 65000) holds three paths to 203.0.113.0/24. (a) from ISP-A by eBGP: AS_PATH 100 300, LOCAL_PREF 100, MED 50. (b) from ISP-B by eBGP: AS_PATH 200 300, LOCAL_PREF 100, MED 10. (c) from the iBGP peer BR2: AS_PATH 100 300, LOCAL_PREF 150.

1. Weight is unset on all three, so no decision at step 1.
2. Step 2: local preference 150 beats 100. **(c) wins** and the process stops.

Now remove the local-preference setting on (c), so all are 100.

3. Steps 3 to 5 tie: no path is local, all three have AS_PATH length 2 and the same origin.
4. Step 6, MED. MED is compared only between routes from the same neighbouring AS. (a) and (c) both come from AS 100, but (b) comes from AS 200, so (b)'s lower MED is *not* compared against (a).
5. Step 7: eBGP is preferred to iBGP, so (c) is eliminated. (a) and (b) tie through step 8 (equal IGP cost to directly connected peers).
6. Step 10: the oldest eBGP path wins, so whichever was learnt first stays, which is unpredictable by design.

Result: (c) on LOCAL_PREF; without it, (a) or (b) by age. With `bgp always-compare-med`, step 6 would compare (a) and (b) too, and (b) would win with MED 10.

Other vendors — FRR follows nearly the same list (weight, local preference, ...). Junos has no weight; it compares route preference, then LOCAL_PREF, then AS_PATH, and the order of the later steps differs, so consult the vendor documentation.

WORKED EXAMPLE

Dual-homing Campus Net. BR1 connects to ISP-A (AS 100) and BR2 to ISP-B (AS 200); Campus Net (AS 65000 in this teaching topology) owns 198.51.100.0/24. The goal is for ISP-

A to be primary in both directions.

1. *Outbound*: local preference is the tool, since it is shared across the AS by iBGP. Set 200 on routes learnt from ISP-A.
2. *Inbound*: remote ASes decide, so we make the ISP-B path look longer by prepending our AS twice when advertising to ISP-B.

```
! BR1
ip route 198.51.100.0 255.255.255.0 Null0
ip prefix-list OUT seq 5 permit 198.51.100.0/24
route-map FROM-A permit 10
 set local-preference 200
router bgp 65000
 neighbor 192.0.2.1 remote-as 100
 neighbor 192.0.2.1 route-map FROM-A in
 neighbor 192.0.2.1 prefix-list OUT out
 neighbor 192.0.2.1 maximum-prefix 1000 80
 neighbor 10.255.0.2 remote-as 65000
 neighbor 10.255.0.2 update-source Loopback0
 neighbor 10.255.0.2 next-hop-self
 network 198.51.100.0 mask 255.255.255.0
! BR2
route-map TO-B permit 10
 set as-path prepend 65000 65000
router bgp 65000
 neighbor 192.0.2.5 remote-as 200
 neighbor 192.0.2.5 route-map TO-B out
```

```
BR1# show ip bgp 203.0.113.0/24
   Network          Next Hop     Metric LocPrf Weight Path
*> 203.0.113.0/24   192.0.2.1         0    200      0 100 300 i    <-- best
(>), via ISP-A
* i                 10.255.0.2        0    100      0 200 300 i
```

Result: Campus Net sends via ISP-A (LOCAL_PREF 200). Remote networks see paths “100 65000” and “200 65000 65000 65000” and prefer the shorter one via ISP-A.

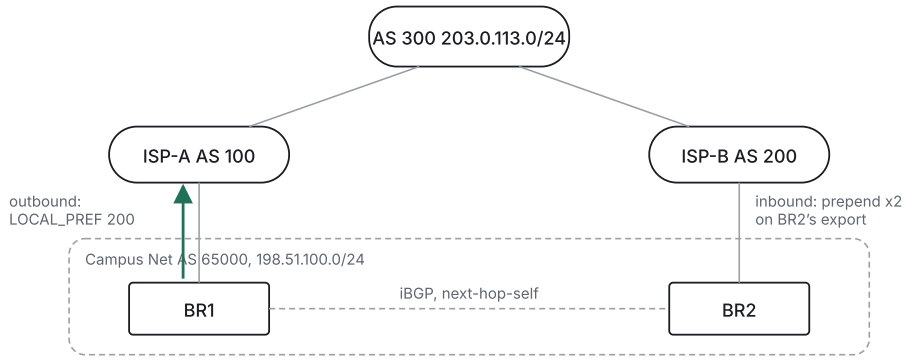


Figure 4.26 Dual-homed Campus Net. Outbound traffic is steered by LOCAL_PREF (inside our AS); inbound traffic by making the ISP-B path longer (outside our AS).

WORKED EXAMPLE

The iBGP next-hop problem. R3, an internal Campus Net router, is an iBGP client of BR1 and has no eBGP session. BR1 learns 203.0.113.0/24 with next hop 192.0.2.1 and passes it to R3 *unchanged*. R3’s IGP has no route to 192.0.2.1, so the next hop is unreachable.

```
R3# show ip bgp
   Network          Next Hop     Metric LocPrf Weight Path
 * i203.0.113.0/24  192.0.2.1          0    100      0 100 300 i   <-- no
">": not best, not in RIB
```

Result: add `neighbor 10.255.0.3 next-hop-self` on BR1, so the next hop becomes BR1's loopback, which the IGP knows. Alternatively advertise the ISP link into the IGP as a passive interface.

WORKED EXAMPLE

A five-path race. BR1 has five candidates for 203.0.113.0/24. None of them has a weight set. Step through the list.

Path	LOCAL_PREF	Local	AS_PATH	Origin	MED	Type	IGP metric to NH	Router ID
P1	100	no	100 300	i	20	eBGP	0	10.0.0.9
P2	100	no	100 300	i	5	eBGP	0	10.0.0.7
P3	100	no	200 400 500	i	0	eBGP	0	10.0.0.5
P4	100	no	100 300	i	5	iBGP	10	10.255.0.2
P5	90	no	100	e	0	eBGP	0	10.0.0.3

1. Step 0 and 1: all next hops reachable; weights tie.
2. Step 2: LOCAL_PREF 100 beats 90. **P5 is eliminated**, though it has the shortest AS_PATH: policy comes before distance.
3. Step 3: no local routes. Step 4: AS_PATH length: P1, P2 and P4 have 2, P3 has 3. **P3 is eliminated.**
4. Step 5: origin ties (all IGP). Step 6: MED, compared among paths from the same neighbouring AS (all three remaining come from AS 100): P2 and P4 have 5, P1 has 20. **P1 is eliminated.**
5. Step 7: eBGP over iBGP: **P4 is eliminated**; P2 remains.

Result: P2 wins at step 7. Had P2 and P4 both been eBGP, step 8 (IGP metric), then step 10 (age) and step 11 (router ID 10.0.0.7 against another) would have decided.

Communities and *aggregation* complete everyday practice. A community is set with a route map (`set community 65000:100 additive`) and sent to the neighbour only if `neighbor X send-community` is configured; the receiving AS then acts on it, for example by assigning local preference 80 to routes tagged 65000:80 (a *backup* community) or by blackholing a destination. `aggregate-address 198.51.100.0 255.255.255.0 summary-only` creates a summary (and suppresses the more-specific components); an aggregate that loses detail sets ATOMIC_AGGREGATE and may carry an AGGREGATOR attribute (and an AS_SET).

```
ip as-path access-list 10 permit ^100_            ! learnt from AS 100
ip as-path access-list 20 permit ^$               ! originated locally
ip community-list standard BACKUP permit 65000:80
route-map FROM-ISP permit 10
 match community BACKUP
 set local-preference 80
route-map FROM-ISP permit 20
router bgp 65000
 neighbor 192.0.2.5 route-map FROM-ISP in
 neighbor 192.0.2.5 send-community
```

Convergence is mostly a timer problem. With default timers a silent failure is noticed only after the hold time (180 s). `fast-external-fallover` (on by default) resets an eBGP session the moment its directly connected interface goes down, and *BFD* (`neighbor X fall-over bfd`) detects loss in a fraction of a second. After a policy change `clear ip bgp X soft in` applies the new inbound policy using a stored copy or a ROUTE-REFRESH, without resetting the session. Two further knobs are the advertisement interval (30 s for eBGP by default) and graceful restart.

Policy tools complete the picture. *Outbound* traffic is influenced by weight (one router) and local preference (the whole AS). *Inbound* traffic can only be influenced indirectly: AS-path prepending, MED (honoured only by the neighbouring AS), advertising more-specific prefixes (most of the Internet filters beyond /24), and communities that ask the provider to act. The well-known communities are `no-export` (do not advertise outside this AS) and `no-advertise` (do not advertise to any peer). Filtering uses prefix lists (§4.9 (p. 268)), AS-path access lists with regular expressions (`^$` means locally originated, `_65001_` any path through AS 65001) and route maps, with `maximum-prefix` as a safety valve, and `soft-reconfiguration inbound` or route refresh to re-apply a policy without resetting the session.

The `network` command in BGP advertises a prefix only if an *exact* match is present in the RIB (which is why the Null0 route above is configured), unlike the `network` command of OSPF, which selects interfaces. `maximum-paths 4` installs several equal paths, provided the weight, local preference, AS_PATH length, origin, MED, neighbouring AS and IGP cost all match; `bgp bestpath as-path multipath-relax` relaxes the requirement that the AS numbers be identical, which suits spine-leaf fabrics where each leaf has its own AS. Those designs (RFC 7938) use eBGP alone because of its policy control and its simple failure behaviour; see §5.4 (p. 310).

BGP is trusted by configuration, not by proof. In 2008 a Pakistani ISP's attempt to block YouTube leaked a more-specific prefix to the world, and most of the Internet followed the longer prefix; in 2021 a configuration change withdrew a large content provider's prefixes and took its DNS servers off the Internet with them. The defences are origin validation with RPKI (route origin authorisations decide whether a route is *valid*, *invalid* or *not found*), prefix and `maximum-prefix` limits, MD5 or TCP-AO session authentication, and the generalised TTL security mechanism (`ttl-security hops 1`).

A note on AS numbers: Campus Net uses the private AS 65000 in this book's examples. A private AS cannot be used on the Internet's global table, so a provider strips it (`neighbor X remove-private-as`) before advertising the customer's prefix. That suits a customer attached to one provider; a customer that is dual-homed, as in Figure 4.26, needs a public AS number, since otherwise the prefix would appear to originate from two different ASes and origin validation could not be applied.

TRAP

Claim: a lower MED always wins, so (b) with MED 10 beats (a) with MED 50. *Why it is wrong:* by default MED is compared only between routes from the same neighbouring AS, and these come from AS 200 and AS 100. *Correct statement:* the comparison is skipped unless `bgp always-compare-med` is configured. Note also that MED is lower-wins, whereas weight and local preference are higher-wins.

TRAP

Claim: "BGP neighbour Active" means the session is working. *Why it is wrong:* Active means the router is *trying* to open the TCP connection and has failed. *Correct statement:* a healthy session is Established; Active or Idle is a fault, usually reachability, ACL, a wrong AS number or a missing `update-source`.

INTERVIEW LENS

AMAZON · CISCO · JUNIPER · ARISTA · NOKIA

The best-path order is among the most frequently reported questions in this book; recite it with the mnemonic and then explain two of the steps unprompted (why MED is limited to one neighbour AS; why eBGP beats iBGP). Expect the dual-homing scenario (outbound by local preference, inbound by prepending or communities), the iBGP next-hop failure, the RR loop-prevention attributes and the full-mesh count. A coding variant asks you to implement the first eight steps; its solution is Q4.45.

LAB

`LAB 4.7 · 40 MIN` Containerlab with FRR: AS 65000 (BR1, BR2, R3), AS 100, AS 200 and AS 300. Configure eBGP, iBGP, `next-hop-self`, the two route maps above, then verify.

```
vtysh -c "show bgp ipv4 unicast 203.0.113.0/24"
vtysh -c "show bgp summary"
```

```
Paths: (2 available, best #1)
  100 300  from 192.0.2.1 (ISP-A)
    Origin IGP, localpref 200, valid, external, best    <-- best
```

What you should notice: shut the ISP-A link and the best path moves to ISP-B within the hold time (180 s, or at once with BFD or fast external fall-over). Capture TCP 179 in Wireshark and decode the AS_PATH and LOCAL_PREF in the UPDATE.

CHECK YOURSELF

1. A route is in the BGP table but absent from the routing table. Name the first thing you check. [1]
2. Why does a route reflector add a CLUSTER_LIST to a reflected route? [2]

4.8 First-hop redundancy: HSRP, VRRP and GLBP SHOULD · ■■□

BIG IDEA

A host has one default gateway address, so two routers share a *virtual* IP address and MAC address, and whichever is active answers for it; when it fails the other takes over and the hosts notice nothing.

1. The next hop: if it is unreachable in the IGP, the route is not installed (also check a better-AD route for the same prefix).↩
2. A route that returns to a reflector carrying its own cluster ID has looped and is discarded.↩

In *HSRP* (Cisco's Hot Standby Router Protocol) one router is *Active* and forwards for the virtual IP, another is *Standby*, and the rest *Listen*. Routers exchange hellos every 3 s (hold time 10 s) and the highest priority (default 100) becomes active. HSRP *preemption* is **off** by default, so a recovered higher-priority router does not take back the role unless `standby preempt` is configured. *VRRP* (RFC 5798) is the open standard, with a *master* and *backups*; its preemption is **on** by default, and a router that owns the virtual IP as its real address has priority 255. *GLBP* adds load balancing: one *active virtual gateway* hands out different virtual MACs, and up to four *active virtual forwarders* each carry part of the load.

	HSRP v1	HSRP v2	VRRP v2/v3	GLBP
Standard	Cisco	Cisco	IETF	Cisco
Transport and group address	UDP 1985, 224.0.0.2	UDP 1985, 224.0.0.102	IP protocol 112, 224.0.0.18	UDP 3222, 224.0.0.102
Virtual MAC	0000.0c07.acXX	0000.0c9f.fXXX	0000.5e00.01XX	0007.b400.XXYY
Default priority / preempt	100 / off	100 / off	100 / on	100 / off
Load sharing	per group	per group	per group	per host (several forwarders)

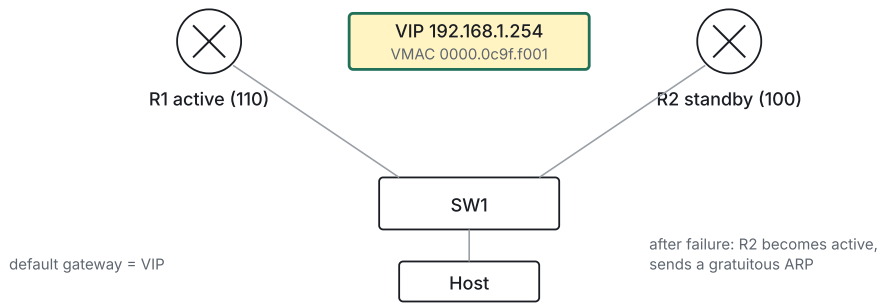


Figure 4.27 First-hop redundancy. Both routers know the VIP; only the active one answers ARP for it with the virtual MAC.

HSRP routers move through states: *Initial* (starting), *Learn* (waiting to learn the virtual IP from the active router), *Listen* (knows the virtual IP, hears hellos, is neither active nor standby), *Speak* (takes part in the election, sending hellos), *Standby* (next in line) and *Active*. The router that becomes active sends a gratuitous ARP and, since the virtual MAC is learnt by the switches from the HSRP hellos themselves, forwarding resumes at once. Failure of the active router is detected after the hold time (10 s by default), and a recoverable improvement is to shorten hello and hold (`standby 1 timers msec 200 msec 750`) or to use BFD.

WORKED EXAMPLE

HSRP with interface tracking. R1 has priority 110 and tracks its uplink Gi0/1 with a decrement of 20; R2 has priority 100. Both use `preempt`. The VIP is 192.168.1.254.

```
R1(config-if)# standby version 2
R1(config-if)# standby 1 ip 192.168.1.254
R1(config-if)# standby 1 priority 110
R1(config-if)# standby 1 preempt
R1(config-if)# standby 1 track GigabitEthernet0/1 20
```

1. Normal state: R1 is active with priority 110.

2. R1's uplink fails. The tracking decrement lowers R1 to 110 - 20 = 90, which is below R2's 100. Because R2 has `preempt`, it takes over.
3. The uplink returns. R1 climbs back to 110 and, with `preempt`, takes the role again.

```
R1# show standby brief
Interface   Grp  Pri P State   Active    Standby      Virtual IP
Gi0/0       1    90  P Standby 192.168.1.3 local      192.168.1.254   <--
after failure
```

Result: tracking changes priority only; the standby takes over only if it has `preempt`. A decrement smaller than the priority gap (here less than 10) would not cause a switch.

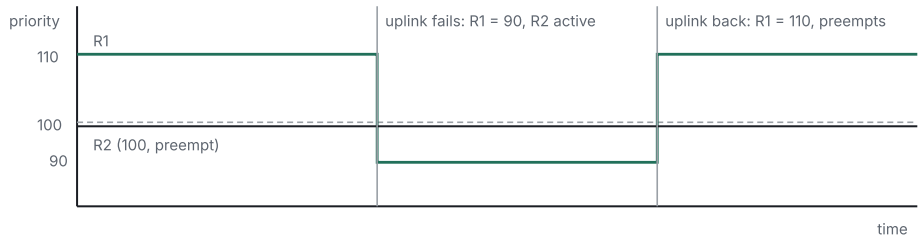


Figure 4.28 Priority against time for the tracking example. The roles swap when the lines cross, provided the lower-priority router has preempt enabled.

VRRP and *GLBP* are configured in the same way. The VRRP version on IOS-XE and on FRR/Linux (`vrrpd`) uses the group, priority and address; if the router's real interface address equals the virtual address it becomes the owner with priority 255. GLBP distributes different virtual MACs to different hosts, so both routers forward.

```
! VRRP (IOS-XE)                          ! GLBP
interface GigabitEthernet0/0             interface GigabitEthernet0/0
 vrrp 1 ip 192.168.1.254                  glbp 1 ip 192.168.1.254
 vrrp 1 priority 110                      glbp 1 priority 110
 vrrp 1 track 1 decrement 20              glbp 1 preempt
```

Align the Layer 2 and Layer 3 roles: make the HSRP active router also the spanning-tree root for the VLAN, so that traffic does not cross the inter-switch link to reach it. On failover the new active router sends a gratuitous ARP so that switches relearn the virtual MAC.

LAB

`LAB 4.8 · 20 MIN` Packet Tracer, two routers and a host, HSRPv2 with tracking as in the example. Start a continuous ping and shut R1's uplink.

```
C:\> ping -t 192.168.1.254
Request timed out.   <-- count the lost packets
R2# show standby brief
Gi0/0   1  100 P Active   local   192.168.1.2   192.168.1.254
```

What you should notice: with default timers up to ten seconds of loss when R1 is *disconnected*, but only a few pings when tracking lowers the priority and R2 has preempt, because the standby no longer waits for the hold timer. Convert to VRRP and compare the lost-ping counts.

CHECK YOURSELF

1. R1 loses its uplink but stays active. Name two causes. [1]

4.9 Route control: prefix lists, route maps, redistribution and VRF-lite SHOULD · ■■□

BIG IDEA

Policy tools decide which routes are accepted, which are modified and which packets are steered; redistribution joins routing domains but, without tagging, joins them into a loop.

A *prefix list* matches prefixes by address and length. `ip prefix-list X permit 10.0.0.0/8 ge 16 le 24` matches any prefix inside 10.0.0.0/8 whose length is between 16 and 24. Thus 10.0.0.0/8 does not match (length 8 is below 16), 10.1.0.0/16 and 10.1.1.0/24 match, and 10.1.1.0/25 does not (length 25 is above 24). With no `ge` or `le` only the exact prefix matches. Lists end with an implicit deny.

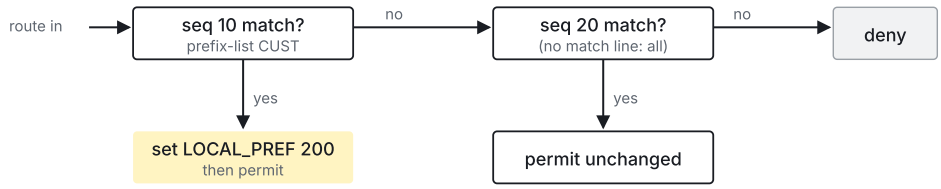


Figure 4.29 How a route map is evaluated. Clauses are tried in sequence-number order.

A *route map* is an ordered list of numbered clauses, each with `match` conditions and `set` actions, and a `permit` or `deny`. The first clause that matches decides; if a permit clause has no match line it matches everything; at the end there is an implicit deny. Route maps are used for BGP policy, for redistribution and for policy-based routing.

```
ip prefix-list CUST seq 5 permit 10.0.0.0/8 ge 16 le 24
route-map SET-LP permit 10
 match ip address prefix-list CUST
 set local-preference 200
route-map SET-LP permit 20          ! no match: everything else passes unchanged
```

Redistribution imports routes of one protocol into another (`redistribute eigrp 100 subnets route-map X` under OSPF). The importing protocol needs a *seed metric*, since the foreign metric means nothing: OSPF assigns cost 20 by default (E2), EIGRP and RIP need explicit values. Done in one direction at one router it is safe. Done in both directions at two routers it is not: a route that enters OSPF from EIGRP at R1 can leave OSPF back into EIGRP at R2 with a better AD or metric, and so on, forming a loop. The fix is to *tag* routes as they are redistributed and to deny the tag in the reverse direction.

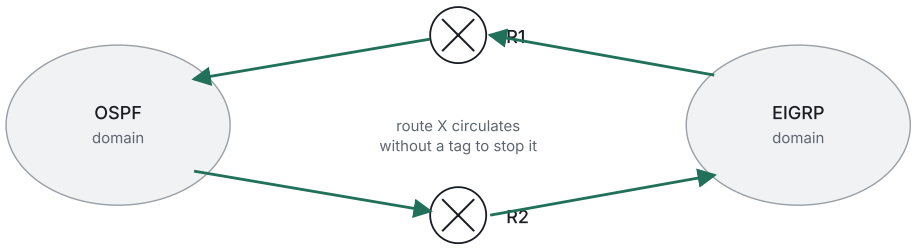


Figure 4.30 Two-way redistribution at two points. A route learnt from EIGRP re-enters EIGRP through OSPF at the other router.

```
route-map EIGRP-TO-OSPF deny 10
 match tag 110                      ! came from OSPF originally: do not send back
route-map EIGRP-TO-OSPF permit 20
 set tag 90
route-map OSPF-TO-EIGRP deny 10
 match tag 90
route-map OSPF-TO-EIGRP permit 20
 set tag 110
router ospf 1
 redistribute eigrp 100 subnets route-map EIGRP-TO-OSPF
router eigrp CAMPUS
 address-family ipv4 unicast autonomous-system 100
  topology base
   redistribute ospf 1 metric 100000 10 255 1 1500 route-map OSPF-TO-EIGRP
```

A seed metric is needed because the receiving protocol cannot interpret the sender's metric. The defaults differ and are a frequent source of surprise.

Redistributed into	Seed metric	Notes
OSPF	cost 20 (E2); 1 for BGP routes	`subnets` needed for classless prefixes on older IOS; `metric-type 1` for E1
EIGRP	none; route not installed without it	`metric bandwidth delay reliability load mtu` or `default-metric`
RIP	none (infinite)	`metric` or `default-metric`
BGP	metric copied (MED)	`redistribute ospf` brings only internal routes unless `match` is used

Policy-based routing (PBR) routes on something other than the destination: `ip policy route-map WEB-VIA-B` on an interface, with `match ip address` for the traffic and `set ip next-hop 192.0.2.5`. *VRF-lite* gives one router several independent routing tables, each with its own interfaces, so two customers can both use 10.0.0.0/24; commands accept a `vrf` keyword (`ping vrf A 10.0.0.1`).

> WORKED EXAMPLE
>
> **Policy-based routing.** Campus Net sends web traffic from SALES via ISP-B while everything else uses the default route toward ISP-A.

```
ip access-list extended WEB
 permit tcp 192.168.10.0 0.0.0.255 any eq 80
 permit tcp 192.168.10.0 0.0.0.255 any eq 443
route-map WEB-VIA-B permit 10
 match ip address WEB
 set ip next-hop 192.0.2.5
interface GigabitEthernet0/0.10
 ip policy route-map WEB-VIA-B
```

```
R1# show route-map WEB-VIA-B
route-map WEB-VIA-B, permit, sequence 10
  Policy routing matches: 412 packets, 307311 bytes
```

Result: matching packets leave via 192.0.2.5 regardless of the routing table; unmatched packets follow normal routing. PBR is stateless, so the return path may differ; add tracking so that it stops when the next hop is dead.

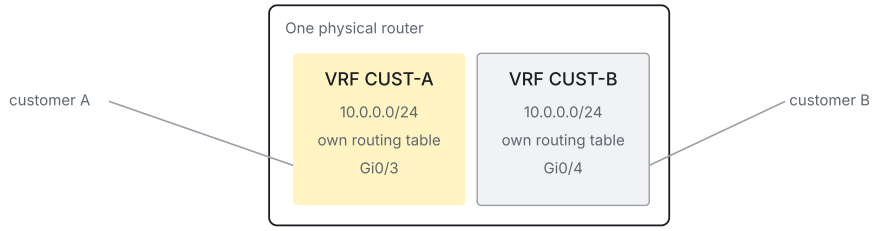


Figure 4.31 VRF-lite. The same prefix lives in two tables; a packet is looked up in the table of the interface it arrived on.

```
vrf definition CUST-A
 address-family ipv4
interface GigabitEthernet0/3
 vrf forwarding CUST-A
 ip address 10.0.0.1 255.255.255.0
R1# show ip route vrf CUST-A
```

TRAP

Claim: redistributing in both directions between two protocols is safe because AD stops loops. *Why it is wrong:* AD protects only routes for the same prefix learnt natively; a redistributed route is seen as a native one by the second protocol, with another AD. *Correct statement:* tag at the first redistribution and deny the tag at the second.

INTERVIEW LENS

CISCO · JUNIPER · AMAZON

Asked as “two-way redistribution loop and how do you stop it” and as “what does this route map do”. Show the sequence evaluation and the implicit deny, state that a permit clause with no match matches everything, and distinguish a deny clause in a route map (the route is not redistributed) from a deny in a prefix list.

LAB

`LAB 4.9 · 40 MIN` GNS3 with IOSv or FRR: three routers, OSPF on one side and EIGRP on the other, mutual redistribution at two routers. Create the loop first, then fix it with tags; finally build VRF-lite with an overlapping 10.0.0.0/24 in two VRFs.

```
R1# show ip route 10.9.9.0
Routing entry for 10.9.9.0/24
  Known via "ospf 1", distance 110, metric 20, type extern 2, tag 110
```

What you should notice: a tag on the route proves where it entered. Without the deny-tag clause, the router prefers the redistributed copy of its own route learned from the other router (an effect of AD); `traceroute` to the prefix bounces between the two redistribution points.

4.10 MPLS and L3VPN SHOULD · ■■■

BIG IDEA

MPLS forwards packets on short labels instead of IP addresses, which lets a provider carry many customers' overlapping address spaces across one core: inner labels identify the customer, outer labels identify the path.

A *label* is a 32-bit shim header inserted between the Layer 2 and IP headers (EtherType 0x8847): a 20-bit label, a 3-bit traffic class, a 1-bit bottom-of-stack flag and an 8-bit TTL. Labels are *pushed*, *swapped* and *popped*. The edge router (*PE*, provider edge) pushes labels; core routers (*P*) swap them using the *LFIB* and never look at the IP header; customer routers are *CE*s. Labels 0 to 15 are reserved: 0 (IPv4 explicit null), 1 (router alert), 2 (IPv6 explicit null), 3 (*implicit null*). Labels are distributed hop by hop by LDP (UDP 646 for discovery, TCP 646 for sessions), one per IGP prefix. With *penultimate-hop popping* (PHP) the router before the egress pops the transport label, because the egress PE advertises implicit null (3) for its own address: the egress then does a single lookup rather than two.

32 bits; S = 1 marks the last label of the stack, so a two-label VPN packet has S = 0 on the outer label.

Figure 4.32 The MPLS shim.

Each router keeps two label tables. The *LIB* (label information base) holds every label it has learnt from neighbours; the *LFIB* (label forwarding information base) holds the one chosen for each prefix, according to the IGP's best next hop. Label *swap* replaces the top label by the one the next hop advertised; the TTL field is decremented as in IP (and the ingress copies the IP TTL into it, unless `no mpls ip propagate-ttl` hides the core from `traceroute`). On the control plane, LDP (label distribution protocol) runs in a hop-by-hop fashion: each router assigns a local label for every prefix in its IGP and advertises it to its LDP neighbours, so that a *label-switched path* follows the IGP's shortest path.

```
P1# show mpls ldp neighbor
    Peer LDP Ident: 10.255.0.3:0; Local LDP Ident 10.255.0.2:0
        TCP connection: 10.255.0.3.646 - 10.255.0.2.43211
        State: Oper; Msgs sent/rcvd: 40/39
P1# show mpls forwarding-table
Local  Outgoing   Prefix           Bytes Label   Outgoing   Next Hop
Label  Label      or Tunnel Id     Switched      interface
24     35         10.255.0.5/32    0             Gi0/2      10.1.23.3     <--
swap 24 to 35
```

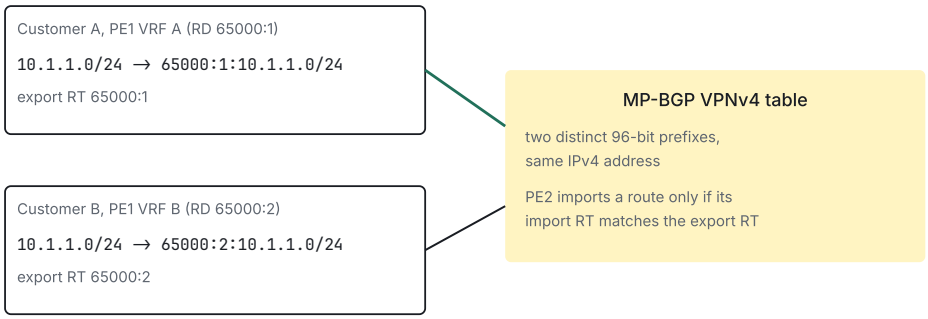


Figure 4.33 RD and RT. The RD only makes the prefixes distinct; the RT is the membership rule that decides which VRF receives each one.

An *L3VPN* (RFC 4364) uses three ideas. Each customer has a *VRF* on the PE. A *route distinguisher* (RD, 64 bits) is prefixed to each customer IPv4 route to make a 96-bit *VPNv4* prefix, so that two customers' 10.1.1.0/24 stay distinct in BGP. A *route target* (RT, a BGP extended community) controls which VRFs import which routes: export RTs are attached when a route is advertised, import RTs select it at the far PE. PEs exchange VPNv4 routes over MP-BGP; the transport path between them is given by LDP labels.

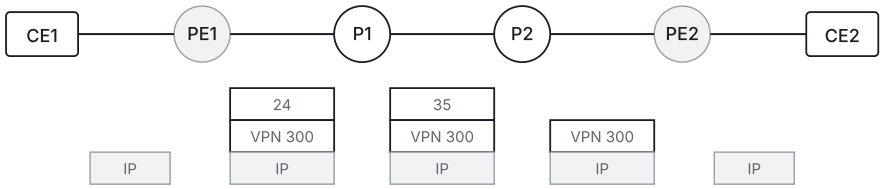
	RD	RT
Purpose	makes a prefix unique in the provider's BGP	decides which VRFs install the route
Where carried	prefix of the VPNv4 NLRI	extended community attribute
Typical value	65000:1	65000:1 (import and export)

WORKED EXAMPLE

An L3VPN packet walk. Customer A site 1 (CE1, 10.1.1.0/24) sends to site 2 (CE2, 10.2.2.0/24) through PE1, P1, P2, PE2. Customer B also uses 10.1.1.0/24 (RD and RT 65000:2).

1. CE1 to PE1: an ordinary IP packet. PE1 looks up the destination in VRF A, finds a VPNv4 route from PE2 with VPN label 300 and BGP next hop PE2's loopback.
2. PE1 pushes two labels: VPN label 300 (inner, bottom of stack) and the LDP label 24 for PE2's loopback (outer).
3. P1 swaps 24 for 35. P1 never reads label 300 or the IP header.
4. P2 is the penultimate hop. PE2 advertised implicit null, so P2 *pops* the transport label and forwards a packet carrying only label 300.
5. PE2 looks up 300, which belongs to VRF A, pops it, performs an IP lookup in VRF A and forwards the packet to CE2.

Result: customer A and B never meet, because each VPN label points to a VRF and each VRF has its own table.



Highlighted block: transport (LDP) label. Open block below it: VPN label. P2 pops the transport label (PHP).
Control plane: LDP between neighbours; MP-BGP VPNv4 between PE1 and PE2.

Figure 4.34 The label stack at each link of the L3VPN walk.

```
vrf definition CUST-A
 rd 65000:1
 address-family ipv4
  route-target both 65000:1
interface GigabitEthernet0/1
 vrf forwarding CUST-A
 ip address 10.1.1.1 255.255.255.0
router bgp 65000
 neighbor 10.255.0.5 remote-as 65000
 neighbor 10.255.0.5 update-source Loopback0
 address-family vpnv4
  neighbor 10.255.0.5 activate
  neighbor 10.255.0.5 send-community extended
```

Verification at the provider edge follows the control plane: first the VRF routing table at each PE, then the VPNv4 table, and finally the label path.

```
PE1# show ip route vrf CUST-A 10.2.2.0
B  10.2.2.0/24 [200/0] via 10.255.0.5, 00:12:30          <-- learnt by MP-BGP
from PE2
PE1# show bgp vpnv4 unicast all 10.2.2.0/24
  Route Distinguisher: 65000:1 (default for vrf CUST-A)
  Local: 10.255.0.5 (metric 20); label 300; Extended Community: RT:65000:1
PE1# traceroute vrf CUST-A 10.2.2.1
  1 10.1.12.2 [MPLS: Labels 24/300 Exp 0] 4 msec         <-- the two-label stack
```

Beyond L3VPN, MPLS traffic engineering and Segment Routing (SR-MPLS, SRv6) steer traffic along explicit paths by stacking labels or segment identifiers at the ingress; they matter mainly for service-provider roles and are outside the ENCOR core.

> **INTERVIEW LENS**
>
> JUNIPER · NOKIA · CISCO
>
> The L3VPN walk is the standard question. Answer with the two-label stack, who pushes, swaps and pops, and why PHP exists. A good answer separates control plane (LDP for transport, MP-BGP for VPN labels) from data plane, and states in one line what RD and RT each do.

> **LAB**
>
> `LAB 4.10 · 40 MIN` FRR Containerlab (Linux MPLS) or GNS3: four-router core, two customers with overlapping 10.1.1.0/24.

```
vtysh -c "show mpls ldp binding"
vtysh -c "show bgp ipv4 vpn"
ip -f mpls route
```

What you should notice: the VPNv4 table shows the same prefix twice under different RDs; the `ip -f mpls route` output shows `swap` at P routers and `pop` at the penultimate hop.

4.11 Quality of Service SHOULD · ■■□

BIG IDEA

When a link is congested, QoS decides who waits and who is dropped, so it is applied where congestion happens, using markings made once at the edge of the network.

Voice needs a one-way delay of no more than about 150 ms, jitter of at most 30 ms and loss below 1%; bulk transfers tolerate none of these constraints being met so strictly, since TCP retransmits. Three models exist: best effort (no distinction), IntServ (reserve per flow with RSVP, which does not scale) and *DiffServ*, in which packets carry a marking and each router applies a *per-hop behaviour* (PHB) by class. DiffServ is the model in use.

The marking is the *DSCP* (differentiated services code point), the top six bits of the IP ToS byte; the bottom two are ECN. Layer 2 has its own 3-bit field, CoS (802.1p), in the 802.1Q tag. Marking is trusted only inside a *trust boundary*, typically at the access switch where an IP phone is recognised; everything else is re-marked.

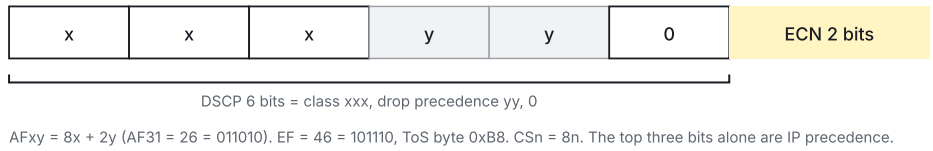


Figure 4.35 The ToS byte. Assured Forwarding classes AF1 to AF4 each have three drop precedences.

PHB	DSCP	Typical use
EF	46	voice
AF41 / AF31 / AF21 / AF11	34 / 26 / 18 / 10	video, critical data, transactional, bulk
CS6 / CS3	48 / 24	network control, signalling
CS1	8	scavenger (below best effort)
default	0	best effort

1. The tracking decrement is too small to drop below the standby's priority, or R2 lacks `preempt` (or the track object is not applied).↩

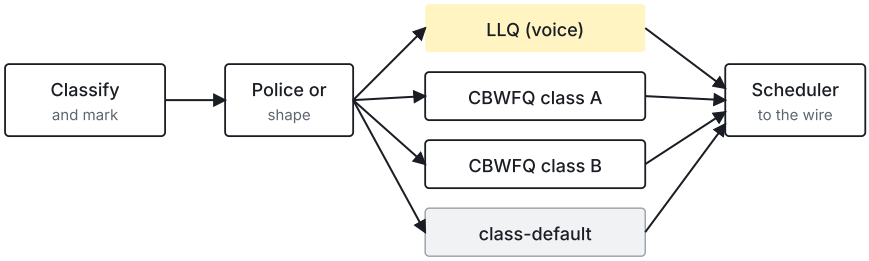


Figure 4.36 The QoS pipeline. Marking happens once at the trust boundary; queuing happens at every congested egress.

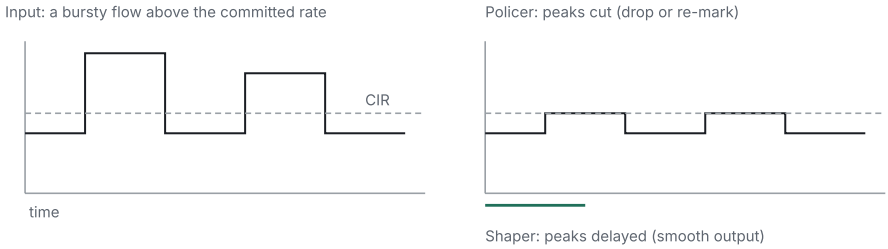


Figure 4.37 Policing against shaping for the same input (left). The policer clips the peaks at the committed rate; the shaper would instead hold the excess and send it later, so total volume is preserved but delay grows.

Queuing then orders the packets. *CBWFQ* gives each class a guaranteed share of bandwidth; *LLQ* adds a strict-priority queue for voice, which has an implicit policer so that it cannot starve the other classes. *WRED* drops packets early and randomly as a queue fills, with a rising probability between a minimum and a maximum threshold, and with higher drop probability for higher drop-precedence markings; it prevents *global synchronisation*, in which tail drop makes all TCP flows back off together. *Policing* and *shaping* both use a token bucket (fills at the committed rate, CIR; capacity is the burst size Bc): a policer discards or re-marks packets that find no tokens, while a shaper delays them in a buffer. Shaping thus adds delay but loses less; policing adds none but TCP sees loss and its window oscillates. A shaper's time interval is Tc = Bc / CIR.

A token bucket works as follows. Tokens, each worth one bit, arrive at the committed rate (CIR) and accumulate up to a limit Bc (a second bucket with Be allows an excess burst). A packet of *n* bits may be sent only if *n* tokens are available; they are then removed. A policer tests the bucket on arrival and acts on the packet at once (transmit, re-mark or drop, giving conform, exceed and violate actions). A shaper keeps the packet in a queue until enough tokens exist. The *time interval* Tc = Bc / CIR is the granularity of the shaper: a smaller Bc means smoother output but more timer work.

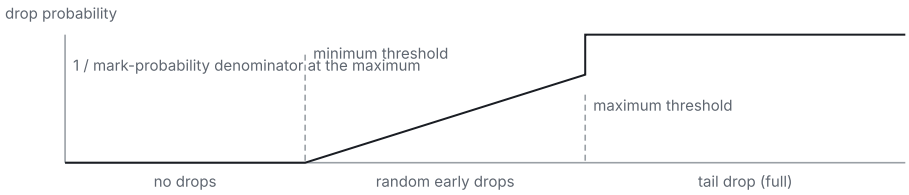


Figure 4.38 The WRED curve. Higher drop-precedence classes (AF13 against AF11) use lower thresholds, so they are dropped first.

WORKED EXAMPLE

Three QoS calculations.

1. *DSCP decode.* ToS 0xB8 = 1011 1000. The top six bits are 101110 = 46 = EF. AF31 has class 3 and drop precedence 1: 8 × 3 + 2 × 1 = 26.
2. *Shaper.* CIR 10 Mb/s with Bc 100,000 bits gives Tc = 100,000 / 10,000,000 = 10 ms, so up to 100 kb may be sent each 10 ms.
3. *Branch WAN policy.* On a 10 Mb/s link: voice 10% priority, video 30%, business-critical 25%, scavenger 1%, the rest default. The reserved sum is 66%, below the default 75% reservation limit, so the policy is accepted.

```
class-map match-all VOICE
 match dscp ef
! VIDEO (af41), CRIT (af31) and SCAV (cs1) are defined in the same way
policy-map WAN-CHILD
 class VOICE
  priority percent 10
 class VIDEO
  bandwidth percent 30
 class CRIT
  bandwidth percent 25
 class SCAV
  bandwidth percent 1
 class class-default
  fair-queue
policy-map WAN-PARENT
 class class-default
  shape average 10000000
  service-policy WAN-CHILD
interface GigabitEthernet0/1
 service-policy output WAN-PARENT
```

Result: the parent shaper makes the Ethernet port behave as a 10 Mb/s link, so the child queues engage; `show policy-map interface Gi0/1` shows per-class packets and drops.

TRAP

Claim: a policer and a shaper are two names for rate limiting. *Why it is wrong:* the policer has no queue, so excess traffic is dropped or re-marked at once; the shaper queues it. *Correct statement:* shape on egress toward a slower service, police to enforce a contract, and never police voice without understanding the consequences.

INTERVIEW LENS

CISCO · BROADCOM · NOKIA

Policing against shaping and the DSCP decode are near-certain. Give the unit conversions without hesitation (ToS byte against DSCP, AFxy = 8x + 2y). Broadcom and Nokia candidates also draw the queue pipeline and say where the buffers and the drop decisions are in hardware.

LAB

`LAB 4.11 · 40 MIN` GNS3 with IOSv and two Linux hosts. Apply the WAN-PARENT policy above to a 10 Mb/s link and mark traffic with iperf3.

```
host$ iperf3 -c 10.0.0.2 -u -b 8M -S 0xB8      ! EF-marked UDP, 8 Mb/s
host$ iperf3 -c 10.0.0.2 -b 50M                ! unmarked TCP
R1# show policy-map interface GigabitEthernet0/1
  Class-map: VOICE (match-all)    priority: 10% (1000 kbps)    drops 4120 <--
exceeds LLQ
```

> **What you should notice:** the EF stream above 1 Mb/s is dropped by the implicit policer of LLQ; the TCP flow takes the rest. Try the same on Linux with `tc qdisc add dev eth0 root tbf rate 10mbit burst 32kbit latency 50ms` and compare the shaper with a policer (`tc filter ... police`).

4.12 IP multicast OPTIONAL · ■■■

> BIG IDEA
>
> Multicast sends one copy of a stream towards many receivers and duplicates it only where paths diverge; receivers announce interest with IGMP, and routers build distribution trees with PIM, accepting packets only from the direction of the source.

Unicast, broadcast and multicast differ in how many copies the network carries. A source sending a stream to ten receivers by unicast sends ten copies over its first link; by broadcast it would reach everyone, wanted or not; by multicast it sends one copy, and the network replicates it only at branch points. Hence video distribution, market-data feeds and routing protocols themselves use multicast.

IPv4 multicast addresses are 224.0.0.0/4. Within it, 224.0.0.0/24 is link-local (never routed; 224.0.0.5 and .6 are OSPF, .10 EIGRP, .13 PIM), 232.0.0.0/8 is source-specific multicast (SSM) and 239.0.0.0/8 is administratively scoped, the private multicast range. A group address maps to a MAC address by placing its low 23 bits after the prefix 01:00:5E; since a group has 28 variable bits, 32 groups share one MAC.

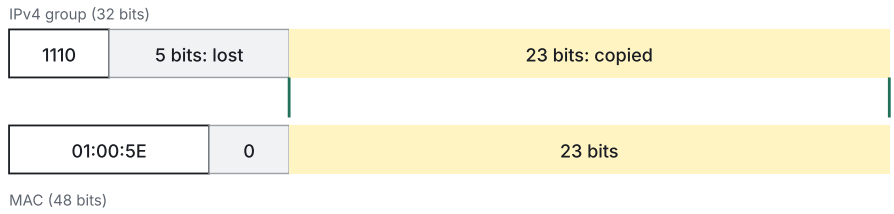


Figure 4.39 Multicast IP-to-MAC mapping. The five lost bits are why 32 groups collide.

> WORKED EXAMPLE
>
> **MAC mapping and ambiguity.** Map 239.1.1.1, and find another group with the same MAC.
>
> 1. The low 23 bits of 239.1.1.1 are the last three octets with the top bit of the second octet dropped: 1 = 0000001, then 1, then 1. The MAC is 01:00:5E:01:01:01.
> 2. 224.129.1.1 has second octet 129 = 1000 0001. Dropping the top bit gives 0000001 = 1, so its MAC is also 01:00:5E:01:01:01.
>
> **Result: 239.1.1.1 and 224.129.1.1 share a MAC; hosts filter again at IP, and 32 groups overlap by this rule.**

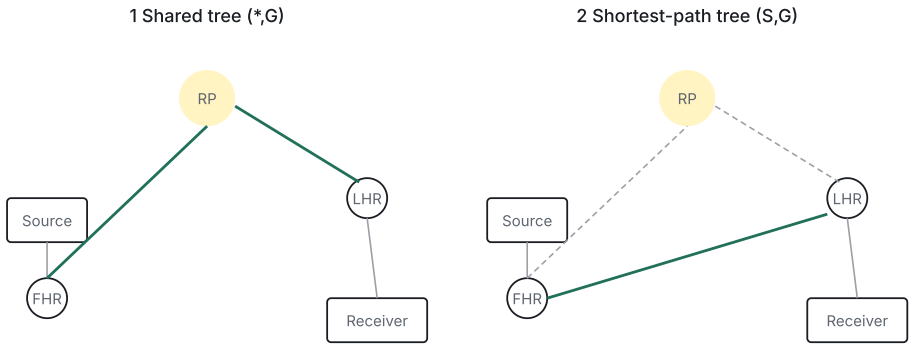


Figure 4.40 PIM sparse mode: shared tree, then switchover to the shortest-path tree. FHR = first-hop router.

Mechanism	Purpose	Notes
IGMPv2	host to router group membership	report to the group; leave to 224.0.0.2; querier election
IGMPv3	adds source filtering	reports to 224.0.0.22; required for SSM
IGMP snooping	switch limits flooding to joined ports	requires a querier in the VLAN
PIM-DM	flood and prune	small networks only
PIM-SM	RP and shared tree, then SPT	the common mode; RP by static, Auto-RP, BSR
PIM-SSM	(S,G) from the start, 232/8	no RP, needs IGMPv3
Bidir-PIM	(*,G) only, traffic both ways	many-to-many applications
MSDP	shares sources between RPs	inter-domain; mostly replaced by anycast RP

Hosts join a group with *IGMP*: v2 uses general queries (to 224.0.0.1), reports sent to the group address and Leave messages (to 224.0.0.2); v3 adds source filtering and is what SSM needs. A switch that does *IGMP snooping* forwards a group only to ports that joined. Between routers, *PIM* in *dense mode* floods and prunes. *PIM sparse mode* (PIM-SM, the standard) uses a *rendezvous point* (RP). A receiver's last-hop router sends a (*,G) join towards the RP, building a shared tree; the source's first-hop router registers with the RP; once traffic flows, the last-hop router switches to a source-rooted shortest-path tree (S,G) and the RP prunes itself off. The RP is found by static configuration, Auto-RP or BSR. Every multicast packet is subject to the *RPF check*: it is accepted only if it arrives on the interface that unicast routing would use to reach the source, which prevents loops. If unicast routing is asymmetric, the RPF interface may differ from the interface the stream uses, and the stream is dropped until a static multicast route corrects it. Bidirectional PIM suits many-to-many traffic, and MSDP connects RPs between domains.

```
R1(config)# ip multicast-routing
R1(config)# ip pim rp-address 10.255.0.1
R1(config-if)# ip pim sparse-mode
R1# show ip mroute 239.1.1.1
(*, 239.1.1.1), 00:03:12/00:02:48, RP 10.255.0.1, flags: S
(10.2.2.5, 239.1.1.1), 00:01:07/00:02:53, flags: T    <-- T: SPT switchover
```

LAB

> `LAB 4.12 · 35 MIN` GNS3 with IOSv: PIM-SM with a static RP on the middle router, a source and a receiver on each side.
>
> ```
> R2# show ip igmp groups
> 239.1.1.1   GigabitEthernet0/1   00:02:10   00:02:49   10.2.2.9      <-- the
> receiver's join
> R2# show ip mroute 239.1.1.1 count
> Group: 239.1.1.1, Source count: 1, Packets forwarded: 5420, Packets received:
> 5420
> ```
>
> **What you should notice:** at first the flag is S (sparse, shared tree) and later T (SPT); break the reverse path with a static route to the source via the wrong interface and `show ip mroute count` reports RPF drops.

> CHECK YOURSELF
>
> 1. Why does a multicast packet fail the RPF check? [1]

4.13 IP services and access lists on IOS MUST · ■■□

> BIG IDEA
>
> A router supplies the services a network depends on (address translation, addressing, time, monitoring, logging) and filters packets with access lists, which are read from the top and stop at the first match.

Address translation. The theory of NAT and PAT is in §3.6 (p. 183); here is the IOS form. Interfaces are marked inside or outside, an ACL selects the inside addresses, and `overload` turns NAT into PAT by multiplexing on port numbers.

```
interface GigabitEthernet0/0.10
 ip nat inside
interface GigabitEthernet0/1
 ip nat outside
access-list 1 permit 192.168.10.0 0.0.0.255
ip nat inside source list 1 interface GigabitEthernet0/1 overload
ip nat inside source static 192.168.10.80 198.51.100.80      ! one-to-one for a
server
```

```
BR1# show ip nat translations
Pro Inside global        Inside local       Outside local      Outside global
tcp 192.0.2.2:1025       192.168.10.5:49152 203.0.113.10:443   203.0.113.10:443
```

DHCP. A router can be a server. A host broadcast does not cross a router, so a router on the client's subnet can *relay* it as a unicast to the real server with `ip helper-address`, filling in the `giaddr` field so that the server chooses the right pool. The helper also forwards some other UDP broadcasts (TFTP, DNS, TACACS, time, NetBIOS) by default.

```
ip dhcp excluded-address 192.168.10.1 192.168.10.20
ip dhcp pool SALES
 network 192.168.10.0 255.255.255.0
 default-router 192.168.10.1
 dns-server 10.0.0.53
interface GigabitEthernet0/0.20
 ip helper-address 10.0.0.5          ! relay to a central server
```

NTP keeps clocks aligned, which makes logs comparable. A server is given a *stratum*, its distance from a reference clock: stratum 1 has the clock attached, each hop adds one, and 16 means unsynchronised. NTP uses UDP 123. *Syslog* (UDP 514) sends log messages to a server, each with a severity from 0 to 7. *SNMP* (UDP 161 for polling, 162 for traps) lets a manager read and set device variables. Version 2c authenticates by a clear-text community string; version 3 adds authentication and encryption (`authPriv`) and should be used.

```
ntp server 10.0.0.5
ntp authentication-key 1 md5 N7pKey
ntp authenticate
ntp trusted-key 1
logging host 10.0.0.6
logging trap warnings
service timestamps log datetime msec
snmp-server group NMS v3 priv
snmp-server user nms NMS v3 auth sha AuthPass priv aes 128 PrivPass
```

```
BR1# show ntp status
Clock is synchronized, stratum 3, reference is 10.0.0.5
```

Level	Name	Level	Name
0	Emergency	4	Warning
1	Alert	5	Notice
2	Critical	6	Informational
3	Error	7	Debugging

Mnemonic — “Every Awesome Cisco Engineer Will Need Ice cream Daily”: Emergency, Alert, Critical, Error, Warning, Notice, Informational, Debugging. `logging trap warnings` sends levels 0 to 4.

Access control lists. The theory and placement of ACLs belong to §9.5 (p. 549); the IOS syntax is below. An ACL is a list of permit and deny lines examined top to bottom; the first match decides, and an invisible `deny ip any any` ends every list. Standard ACLs (1 to 99) match the source address only and are applied close to the destination; extended ACLs (100 to 199) match protocol, source, destination and ports and are applied close to the source. Named ACLs allow sequence numbers, so that lines can be inserted. Masks are *wildcards*: a 0 bit must match, a 1 bit is ignored, so the wildcard is the inverted subnet mask. IPv6 ACLs end with implicit permits for neighbour discovery before the final deny.

WORKED EXAMPLE

An ACL that protects a server. Only SALES (192.168.10.0/24) may reach the web server 10.0.0.80 on TCP 443; other networks must not reach it at all.

1. Place it as close as possible to the destination for a *single* protected host, as an outbound ACL on R1’s server-facing port; alternatively, place an extended ACL inbound on each user-facing interface so that denied packets are discarded at the first router.
2. Permit the allowed flow, deny everything else addressed to the server, and permit the rest.
3. Return traffic needs nothing here, since an outbound ACL on the server’s port inspects only packets *toward* the server. If the ACL were applied inbound on the server port (filtering replies), add `permit tcp host 10.0.0.80 eq 443 192.168.10.0 0.0.0.255 established` .

```
ip access-list extended TO-WEB
 10 permit tcp 192.168.10.0 0.0.0.255 host 10.0.0.80 eq 443
 20 deny   ip any host 10.0.0.80 log
 30 permit ip any any
interface GigabitEthernet0/2
 ip access-group TO-WEB out
```

```
R1# show ip access-lists TO-WEB
    10 permit tcp 192.168.10.0 0.0.0.255 host 10.0.0.80 eq 443 (153 matches)
    20 deny ip any host 10.0.0.80 log (7 matches)
```

Result: line 10 matches the permitted flow, line 20 catches everything else for the server, and line 30 stops the implicit deny from blocking unrelated traffic.

WORKED EXAMPLE

A wildcard mask. Match hosts 192.168.1.16 to 192.168.1.31. The block holds 16 addresses starting at a multiple of 16, so the mask is 16 - 1 = 15 in the last octet: `192.168.1.16 0.0.0.15`. Quick rule: wildcard = 255 minus the subnet-mask octet, so /24 is 0.0.0.255 and a single host is `0.0.0.0` (written `host x`).

TRAP

Claim: an ACL with only `deny` lines blocks those packets and passes the rest. *Why it is wrong:* every ACL ends in an implicit `deny any`, so a list of denies blocks everything. *Correct statement:* finish with `permit ip any any` when the intent is to filter selected traffic only.

INTERVIEW LENS

CISCO · AMAZON · MICROSOFT

Expected at Cisco as a configuration or “what is wrong with this ACL” exercise (order of lines, implicit deny, direction on the interface, wildcard arithmetic). Be able to write the NAT overload line, a DHCP relay and an SNMPv3 user from memory, and to name the verification command for each.

Address translation and filtering are most often debugged with a short routine, which is worth memorising.

Symptom	First check	Likely cause
Inside hosts cannot reach the Internet	`show ip nat translations`, `show ip nat statistics`	no `ip nat inside` or `outside` marking; ACL does not match the inside subnet
Some translations fail, then all	`show ip nat statistics` (misses, pool exhaustion)	too few ports or addresses; missing `overload`
Server unreachable from outside	`show ip nat translations`	no static entry, or ACL blocks the *inside local* address
DHCP clients get no address	`debug ip dhcp server packet`	missing `ip helper-address`; pool exhausted; wrong `giaddr` subnet
ACL seems ignored	`show ip access-lists`, `show ip interface Gi0/2 | include access list`	applied in the wrong direction or on the wrong interface; line order

Clock drifts, logs disagree	`show ntp associations`	NTP unreachable; wrong key; stratum too high

LAB

`LAB 4.13 · 45 MIN` Packet Tracer, integrated: PAT at BR1, a DHCP pool and relay, an NTP server, syslog and an SSH-only vty line, then TO-WEB.

```
BR1(config)# access-list 10 permit 10.0.0.0 0.0.0.255
BR1(config)# line vty 0 4
BR1(config-line)# access-class 10 in
```

What you should notice: `show ip nat translations` fills as hosts browse; a Telnet attempt from outside the admin subnet is refused; the match counters of TO-WEB rise on the permit line only for SALES.

CHECK YOURSELF

1. What does `ip nat inside source list 1 interface Gi0/1 overload` do? [2]
2. A standard ACL is applied near the source and blocks too much. Why? [3]

4.14 Tunnels and enterprise architecture OPTIONAL · ■■□

BIG IDEA

A tunnel wraps a packet inside another to cross a network that would not otherwise carry it, at the price of a header and a smaller usable MTU.

GRE (IP protocol 47) adds a 4-byte header and a new 20-byte IP header, 24 bytes in all, so a 1500-byte link leaves 1476 for the inner packet. GRE does not encrypt; IPsec does (§9.7 (p. 555)), and GRE over IPsec is the standard way to carry routing protocols securely. DMVPN adds dynamic spoke-to-spoke tunnels.

Figure 4.41 GRE encapsulation.

WORKED EXAMPLE

GRE with OSPF, and the recursive-routing fault. HQ-R (203.0.113.1) and Branch-R (198.51.100.1) are joined by a tunnel on 172.31.0.0/30, with OSPF running over it.

```
interface Tunnel0
 ip address 172.31.0.1 255.255.255.252
 tunnel source 203.0.113.1
 tunnel destination 198.51.100.1
 ip mtu 1476
 ip tcp adjust-mss 1436
router ospf 1
 network 172.31.0.0 0.0.0.3 area 0
```

1. The tunnel destination 198.51.100.1 must be reachable over the *underlay* (the real ISP path).
2. If OSPF learns 198.51.100.0/24 *through the tunnel itself*, the router tries to reach the tunnel endpoint through the tunnel. The tunnel then collapses and flaps:

```
%TUN-5-RECURDOWN: Tunnel0 temporarily disabled due to recursive routing
%LINEPROTO-5-UPDOWN: Line protocol on Interface Tunnel0, changed state to
down
```

Result: keep the underlay route to the tunnel destination more specific or of lower AD than anything learnt via the tunnel, for example by running OSPF over the tunnel in a separate process and filtering the underlay prefixes.

Failure detection beyond link state uses *BFD* (millisecond hellos) and *IP SLA* probes that feed a *track* object, for example a floating default route that is withdrawn when a probe to the ISP's gateway fails, not only when the link drops:

```
ip sla 1
 icmp-echo 192.0.2.1 source-interface GigabitEthernet0/1
 frequency 5
ip sla schedule 1 life forever start-time now
track 1 ip sla 1 reachability
ip route 0.0.0.0 0.0.0.0 192.0.2.1 track 1
```

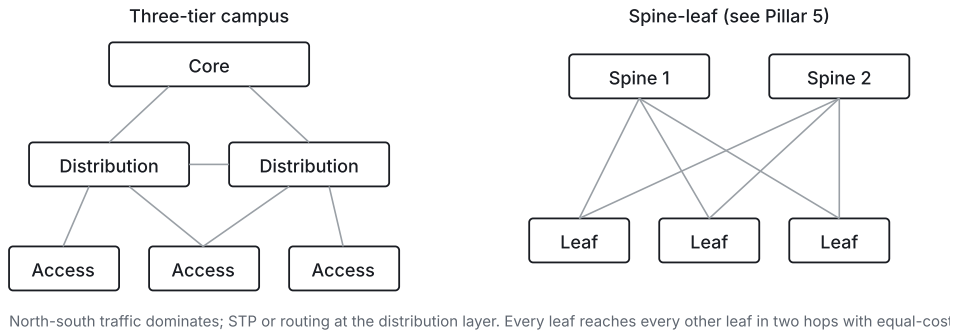


Figure 4.42 The two shapes. The campus hierarchy suits hosts that talk to services outside; the fabric suits servers that talk to each other.

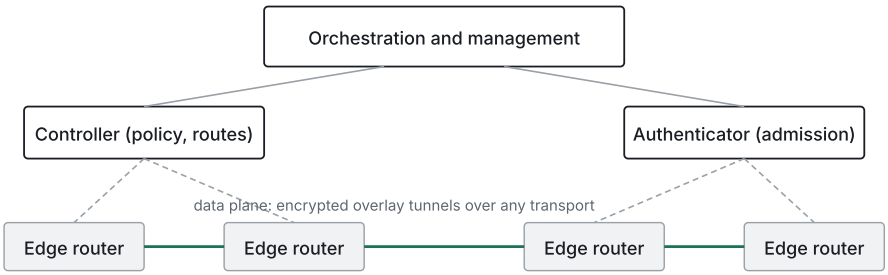


Figure 4.43 The SD-WAN idea: a central control plane (dashed) programs edge devices, which build the data-plane overlay (accent) between themselves.

Enterprise designs have two common shapes. A *three-tier campus* has access, distribution and core layers: access switches terminate hosts (PortFast, BPDU Guard), distribution pairs run the inter-VLAN routing and FHRP, and the core carries traffic between them. A *two-tier (collapsed-core)* design merges the last two; a *spine-leaf* fabric (see §5.1 (p. 303)) replaces both in data centres. *SD-WAN* (for example Cisco's vManage, vSmart, vBond and edge routers) separates a central control plane and orchestration from the edge forwarding devices and builds overlay tunnels over any transport. *SD-Access* builds a campus *fabric* with a LISP control plane and a VXLAN data

plane. These overlays are taught in §5.5 (p. 312); the point here is that they rest on the routing and tunnelling in this chapter.

> CHECK YOURSELF
>
> 1. What MTU should the GRE tunnel use over a 1500-byte link, and why? [4]

4.15 Mini-project: Campus-to-Internet OPTIONAL · ■■■

> BIG IDEA
>
> A dual-homed enterprise built and verified as code shows every protocol of this chapter working together, and a test suite makes the result repeatable evidence of competence.

Topology (Containerlab; FRR plus Arista cEOS or Nokia SR Linux). Campus Net has two access switches (VLANs 10, 20 and a voice VLAN 99) with an LACP uplink to a collapsed-core pair running HSRP or VRRP, with the spanning-tree root aligned to the active gateway. A multi-area OSPF network has area 0 in the core and area 10 for a branch (totally stubby). Two border routers, BR1 and BR2, run eBGP to ISP-A (AS 100) and ISP-B (AS 200), which both peer with a content AS 300, and iBGP between themselves with `next-hop-self`.

Policy requirements.

- ISP-A is primary in both directions: local preference 200 inbound to Campus Net, AS-path prepend twice towards ISP-B.
- Advertise only 198.51.100.0/24, using a prefix list and `maximum-prefix`.
- Tag customer routes with communities and filter by community.
- PAT at the borders, LLQ for voice on the WAN, and a GRE tunnel to a branch carrying OSPF.

Deliverables.

1. A design document: the IP plan using VLSM (method in §3.4 (p. 174)) and a diagram in the style of this book.
2. `topology.clab.yml` and the start-up configurations.
3. A `verify.py` test suite (pytest) that asserts OSPF neighbours are Full, BGP sessions are Established, the best path is via ISP-A, traffic fails over to ISP-B within *N* seconds when the ISP-A link is cut, and DSCP EF is preserved end to end (Pillar 10 (p. 586) shows the Netmiko and `vtysh` calls).
4. A failure-injection runbook with five faults, each with symptom, diagnosis and fix, which feeds the troubleshooting method of Pillar 11 (p. 634).
5. A two-minute explanation of the project, and two résumé bullets.

Milestones. Week 1: L2 and the gateways. Week 2: OSPF and the branch. Week 3: BGP and the policy. Week 4: QoS, GRE, tests, runbook. **Stretch.** Replace the GRE tunnel by an MPLS L3VPN provider core, enable BFD, and measure the convergence improvement.

```
def test_best_path_via_isp_a(bgp):              # pytest fixture runs vtysh on
BR1
    best = bgp.best("203.0.113.0/24")
    assert best.as_path[0] == 100 and best.localpref == 200

def test_failover(lab, bgp):
    lab.link_down("br1-isp-a")
    assert bgp.wait_best("203.0.113.0/24", via_as=200, timeout=15)
```

1. It arrived on an interface that the unicast routing table would not use to reach the source. ↩
2. It translates addresses matched by ACL 1 to the address of Gi0/1, using port numbers to distinguish hosts (PAT). ↩
3. A standard ACL sees only the source address, so near the source it blocks that source's traffic to every destination. ↩
4. 1476, because the 24-byte GRE overhead (20 outer IP, 4 GRE) is subtracted. ↩

Problem Set

Each solution opens with the 30-second answer. Tags give the companies that report the topic, the round, the difficulty (■□□ to ■■■) and the evidence: S for a specific reported question, T for a reported topic with our wording.

Q4.1 Access vs trunk port. What is the native VLAN, and what happens on a mismatch?

CISCO · JUNIPER · TECH-1 · ■□□ · T

30-second answer: an access port carries one VLAN untagged; a trunk carries many VLANs with 802.1Q tags; the native VLAN is the one VLAN sent untagged on a trunk, and if the two ends disagree, frames cross from one VLAN into another.

Answer. A frame of VLAN 1 sent untagged by a switch whose native VLAN is 1 is read by a neighbour with native VLAN 99 as VLAN 99. Hence traffic leaks between VLANs and spanning tree on that VLAN may block the port as inconsistent. CDP prints a mismatch message. **Mistakes.** Treating the message as harmless; leaving the native VLAN as VLAN 1 (set an unused VLAN on both ends).

Q4.2 How many VLANs does 802.1Q support, and why 4094?

CISCO · ARISTA · TECH-1 · ■□□ · T

30-second answer: the VLAN ID is 12 bits, giving 4096 values; 0 (priority tag, no VLAN) and 4095 (reserved) are unusable, leaving 4094.

Answer. On Cisco, 1002 to 1005 are legacy and 1006 to 4094 need extended-range support. Beyond 4094 VLANs, VXLAN's 24-bit VNI is used (§5.5 (p. 312)).

Q4.3 Explain inter-VLAN routing with ROAS and with SVIs. Which scales better?

CISCO · TECH-1 · ■■□ · T

30-second answer: router-on-a-stick uses one trunk and a sub-interface per VLAN on a router, with each packet crossing the link twice; an SVI is a virtual Layer 3 interface on a Layer 3 switch, routed in hardware; SVIs scale better.

Answer. ROAS is bounded by one link and the router's forwarding rate; it suits small branches. SVIs need `ip routing`, one `interface Vlan N` per VLAN, and the VLAN must exist and have an active port for the SVI to come up. The packet walk is in §4.2 (Figure 4.6 (p. 233)). **Follow-up.** Routed ports (`no switchport`) are used on links between Layer 3 switches.

Q4.4 Why is DTP a security risk? How do you harden trunk ports?

CISCO · TECH-1 · ■■□ · T

30-second answer: DTP lets a port become a trunk by negotiation, so an attacker who sends DTP frames can obtain a trunk and see all VLANs; configure every port statically and disable negotiation.

Answer. Access ports: `switchport mode access`, `switchport nonegotiate`, an unused native VLAN, `spanning-tree portfast` with BPDU guard, and shut unused ports into a parking VLAN. Trunks: `switchport mode trunk`, `nonegotiate`, an explicit allowed list, native VLAN not 1. Double-tagging and its defence are in §9.5 (p. 549).

Q4.5 Walk through the STP root election and port roles for the triangle.

CISCO · JUNIPER · ARISTA · TECH-1 · ■■□ · S

30-second answer: lowest bridge ID wins root (SW1, lowest MAC at equal priority); each other switch takes its lowest-cost port to the root as the root port; per segment the lowest-cost, then lowest-BID sender is designated; the remaining port on SW3 blocks.

Answer. The full three-part trace (default priorities, SW3 at 4096, parallel links) is the worked example of §4.3 (Figure 4.9 (p. 237)); learn the four-step order rather than the answer. **Mistake.** Giving the root bridge a root port.

Q4.6 Why must the STP priority be a multiple of 4096?

CISCO · TECH-1 · ■■□ · T

30-second answer: the 16-bit priority field is split into a 4-bit priority and a 12-bit VLAN ID (the extended system ID), so only the top four bits can be set.

Answer. The configurable values are 0, 4096, …, 61440. The VLAN ID is added to the configured value, hence VLAN 10 at default priority displays 32778.

Q4.7 802.1D vs RSTP: port states and roles. Why does RSTP converge in under a second?

CISCO · NOKIA · TECH-1 · ■■□ · T

30-second answer: 802.1D has five states (blocking, listening, learning, forwarding, disabled) and timers of up to 50 s; RSTP has three (discarding, learning, forwarding) and negotiates with proposal and agreement instead of waiting for timers.

Answer. Roles in RSTP are root, designated, alternate and backup. An alternate port is a precomputed path to the root and takes over immediately; a new link is handled by a sync handshake, and edge ports forward at once. Failure is detected after three missed hellos (6 s), or at once on loss of link. **Mistake.** Saying RSTP removes the need for the root election; it keeps it.

Q4.8 PortFast and BPDU Guard, Root Guard, Loop Guard: where does each go?

CISCO · ARISTA · TECH-1 · ■■□ · T

30-second answer: PortFast with BPDU Guard on access ports facing hosts; Root Guard on distribution ports facing access switches that must never be root; Loop Guard on non-designated uplinks.

Answer. BPDU Guard shuts an edge port if a BPDU arrives. Root Guard puts a port in root-inconsistent state if it hears a superior BPDU and recovers when they stop. Loop Guard stops a blocking port from forwarding when BPDUs vanish (a unidirectional link). UDLD detects the unidirectional fibre itself. **Mistake.** PortFast on a switch-facing port invites loops.

Q4.9 Scenario: the network freezes every few minutes, switch CPU is at 100% and MACs flap between ports. Diagnose and fix.

CISCO · AMAZON · TECH-2 · ■■■ · S

30-second answer: a Layer 2 loop or storm; find the loop port from the flapping MAC and the interface counters, shut it, then fix the cause (a missing or broken spanning tree) and add guards.

Approach. (i) `show processes cpu` confirms the load; (ii) `show mac address-table | include <mac>` repeated shows the same MAC on alternating ports; (iii) `show interfaces | include broadcasts` or `show spanning-tree detail | include occurr|from` shows the port with the highest broadcast rate and the topology-change source. **Fix.** Shut that port. Common causes: an unmanaged switch looped by a patch cable, STP disabled or a BPDU filter, an unexpected PortFast, an EtherChannel mismatch. Then enable BPDU Guard, `storm-control broadcast level 1`, Loop Guard, and errdisable auto-recovery.

Q4.10 LACP active/passive vs PAgP vs `on`: which combinations form a bundle?

CISCO · ARISTA · TECH-1 · ■■□ · T

30-second answer: LACP forms with active-active and active-passive, but not passive-passive; PAgP with desirable-desirable and desirable-auto; `on` only with `on`; the protocols do not mix.

Answer. See the matrix in §4.3. `on` does not negotiate, so a mismatch can form a one-sided bundle and a loop. Prefer LACP: it is the open standard and detects a failed partner.

Q4.11 An EtherChannel shows member ports as suspended (s). Give five causes.

CISCO · JUNIPER · TECH-2 · ■■■ · T

30-second answer: the member's settings disagree with the bundle or the partner: speed or duplex, trunk or VLAN configuration, channel mode or protocol, no LACP reply, and differing spanning-tree settings.

Answer. (i) Speed or duplex mismatch; (ii) trunk mode, allowed list or native VLAN differs between members; (iii) LACP partner silent, or both sides passive, or `on` against LACP; (iv) different LACP system or port priority limits (more than eight members); (v) different STP port cost or PortFast settings. Check with `show etherchannel summary` and `show etherchannel detail`. LACP suspends a member that hears no PDUs so that a loop cannot form.

Q4.12 How does MLAG or vPC avoid STP blocking? What is the peer link for?

ARISTA · CISCO · AMAZON · TECH-2 · ■■■ · T

30-second answer: two physical switches present one logical switch to the downstream device, which bundles its uplinks across both; with no loop in the logical topology nothing blocks; the peer link synchronises state and carries traffic when needed.

Answer. The peers share MAC and ARP tables over the peer link, and a keepalive on a separate path detects a split-brain. Orphan ports (single-attached) use the peer link if the other peer holds the path. Internals in §5.3 (p. 307).

Q4.13 What is administrative distance? Give the values for connected, static, eBGP, EIGRP, OSPF, RIP and iBGP.

CISCO · AMAZON · TECH-1 · ■□□ · S

30-second answer: AD is a local trust rating of a routing source, from 0 to 255, used to choose between sources of the same prefix; connected 0, static 1, eBGP 20, EIGRP 90, OSPF 110, RIP 120, iBGP 200.

Answer. Full table in §4.4. AD is not carried in any protocol; it is configurable per route or per protocol (`distance`).

Q4.14 Order the routing decision. Solve the routing-table puzzle.

CISCO · AMAZON · JUNIPER · TECH-1 · ■■□ · S

30-second answer: longest prefix first, then lowest AD, then lowest metric, then equal-cost sharing.

Answer. The puzzle is solved in §4.4: 10.1.1.9 goes to the static /24, 10.1.5.5 to the EIGRP /16 and 10.200.0.1 to the RIP /8. **Twist.** Add a static 10.1.0.0/16 [1/0]: it replaces the EIGRP route (AD 1 against 90) and 10.1.5.5 follows it.

Q4.15 What is a floating static route? Configure a backup default route.

CISCO · TECH-1 · ■■□ · T

30-second answer: a static route with an AD higher than the primary's, which enters the table only when the primary route disappears.

Answer. `ip route 0.0.0.0 0.0.0.0 203.0.113.9 250` as backup to an OSPF default (AD 110). **Mistake.** It reacts to the route vanishing, not to a broken path behind an up link; tie it to an IP SLA track object (§4.14).

Q4.16 Explain RIB, FIB and TCAM. How does a hardware switch do longest-prefix match at line rate?

BROADCOM · ARISTA · JUNIPER · TECH-2 · ■■■ · T

30-second answer: the RIB holds all routes the protocols offer; the FIB is the best-route set compiled for forwarding; hardware stores it in TCAM, which matches all entries in parallel, ordered so the longest prefix wins.

Answer. Detail is in §5.2 (p. 305). A ternary CAM stores each prefix with a mask, so a lookup is a single cycle; entries are sorted by prefix length. Large tables use algorithmic LPM in SRAM (tries) because TCAM is costly and power-hungry. The algorithm is in §1.9 (p. 41).

Q4.17 List the OSPF neighbour states in order.

CISCO · JUNIPER · NOKIA · TECH-1 · ■□□ · S

30-second answer: Down, (Attempt), Init, 2-Way, ExStart, Exchange, Loading, Full.

Answer. Figure 4.16 shows the packet that causes each step: Hello (Init), a Hello listing our ID (2-Way), DBD master election (ExStart), DBD summaries (Exchange), LSR and LSU (Loading). Attempt applies only to non-broadcast networks.

Q4.18 Two routers are stuck in EXSTART. Why? And in INIT, or 2-WAY?

CISCO · AMAZON · JUNIPER · TECH-1 · ■■□ · S

30-second answer: EXSTART/EXCHANGE: MTU mismatch, or a duplicate router ID or blocked unicast; INIT: hellos are heard in one direction only; 2-WAY: normal between two DROTHERs, a fault otherwise.

Answer. Evidence: `show ip interface` for MTU, `debug ip ospf adj` ("smaller interface MTU"). INIT: an ACL or a multicast filter blocks one direction, or one side's hello lacks our ID. 2-Way on a point-to-point link points at a network-type mismatch.

Q4.19 List the parameters that must match for an OSPF adjacency.

CISCO · NOKIA · TECH-1 · ■■□ · T

30-second answer: area ID, subnet and mask, hello and dead timers, authentication, stub-area flags, MTU, compatible network type; router IDs must differ.

Q4.20 How is the DR elected? Is it pre-emptive? Why have a DR?

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: highest interface priority, then highest router ID; priority 0 is ineligible; it is not pre-emptive; the DR limits adjacencies to n-1 and floods each LSA once.

Answer. Without a DR, a segment of n routers needs n(n-1)/2 adjacencies. Worked example in §4.5. Force a change by setting priorities and clearing the process.

Q4.21 Explain LSA types 1, 2, 3, 4, 5, 7: who generates each and its scope.

CISCO · JUNIPER · NOKIA · AMAZON · TECH-1 · ■■□ · S

30-second answer: 1 router (every router, area); 2 network (DR, area); 3 summary (ABR, other areas); 4 ASBR summary (ABR, other areas); 5 external (ASBR, domain-wide except stubs); 7 NSSA external (ASBR in NSSA, converted to 5 by the ABR).

Answer. Table and scope map in §4.5. Mention that type 3 is distance-vector-like (prefix and cost only), which is why every area must hang off area 0.

Q4.22 Stub vs totally stubby vs NSSA: which LSAs are blocked?

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: stub blocks 4 and 5; totally stubby blocks 3, 4 and 5 (one default type 3); NSSA blocks 4 and 5 but allows 7; totally NSSA also blocks 3.

Answer. Use a stub area where no ASBR exists in the area and routers are small; an NSSA where a local ASBR must exist, for example a branch connected to a partner network.

Q4.23 Why must all areas connect to area 0? What is a virtual link?

CISCO · TECH-1 · ■■□ · T

30-second answer: inter-area routing passes through the backbone as prefix-and-cost summaries, so a non-backbone path could loop; a virtual link tunnels area 0 connectivity across a transit area.

Answer. `area 1 virtual-link <ABR RID>` on both ABRs. It is a repair, not a design; a discontiguous backbone is better fixed physically.

Q4.24 Why do 1 Gb/s and 10 Gb/s links have the same OSPF cost by default? How do you fix it?

CISCO · AMAZON · TECH-1 · ■■□ · T

30-second answer: cost = 100 Mb/s divided by bandwidth, with a minimum of 1, so both are 1; set `auto-cost reference-bandwidth` to a higher value on every router.

Answer. With 100000 (100 Gb/s), 1G costs 100 and 10G costs 10. Alternatively set `ip ospf cost` per link. **Mistake.** Changing it on only some routers, which produces inconsistent metrics.

Q4.25 E1 vs E2 external routes: which is preferred, and when does it matter?

CISCO · JUNIPER · TECH-2 · ■■■ · T

30-second answer: E2 (default) keeps the external cost unchanged; E1 adds the internal cost to the ASBR; an E1 route is preferred over E2 for the same prefix, and E1 matters when several ASBRs advertise the same prefix.

Answer. With E2, the nearest exit is chosen only as a tie-break (the lowest internal cost to the ASBR, the forwarding address). With E1 total cost is compared, so the closer ASBR wins unless the external costs differ enough.

Q4.26 Scenario: 200 customer routes in area 3 flap every few minutes and SPF runs network-wide. How do you contain it?

CISCO · TECH-2 · ■■■ · S

30-second answer: summarise at the ABR so the flaps never leave area 3, make area 3 stub or totally stubby, throttle SPF and LSA generation, and fix the root cause.

Approach. Type 1 and 2 LSAs flood within the area, so only area 3's routers recompute; but each flapping prefix also generates a type 3 into area 0 and beyond. `area 3 range 10.3.0.0 255.255.0.0` hides detail: while any prefix inside is up, the summary stays. Then `timers throttle spf 50 200 5000`, `timers throttle lsa all 50 200 5000`, BFD for failure detection and interface dampening (`dampening` on the interface) on the unstable links. **Mistake.** Summarising a range that has holes, which attracts traffic for addresses that do not exist; add a Null0 route.

Q4.28 Define successor, feasible successor, FD and RD. State the feasibility condition.

CISCO · TECH-1 · ■■□ · T

30-second answer: the successor is the next hop of the best path; FD is our best total metric; RD is the neighbour's own metric to the destination; a neighbour is a feasible successor if its RD is lower than our FD.

Answer. The condition guarantees the neighbour's path does not pass through us, so the backup is loop-free without computation. Example in §4.6.

Q4.29 Compute the EIGRP metric for a T1 plus GigE path.

CISCO · TECH-1 · ■■□ · S

30-second answer: 256 × (10^7 / 1544 + (20,000 + 10) / 10) = 256 × (6476 + 2001) = 2,170,112.

Answer. Slowest bandwidth in kb/s, sum of delays in tens of microseconds, integer division at each step, K1 = K3 = 1.

Q4.27 OSPF vs IS-IS: why do many providers and data centres prefer IS-IS?

JUNIPER · NOKIA · GOOGLE · TECH-2 · ■■■ · T

30-second answer: IS-IS runs directly over Layer 2 rather than IP, uses TLVs that extend easily (IPv4, IPv6, segment routing) with one protocol, and scales through a simple two-level hierarchy.

Answer. Differences: OSPF has areas that attach to a backbone through ABRs, IS-IS has areas per router and a contiguous Level 2 backbone; OSPFv2 and v3 are separate protocols, IS-IS carries both families. Both are link-state and use Dijkstra, so say which one you have configured and why.

Q4.30 What is stuck-in-active, and how do stubs and summarisation prevent it?

CISCO · TECH-2 · ■■■ · T

30-second answer: a route with no feasible successor goes active and queries neighbours, who query theirs; if a reply does not return within three minutes the route is stuck in active and the neighbour is reset.

Answer. Stub routers are not queried at all; a summary lets a router answer a query for a covered prefix at once, so queries stop at the summarising boundary. Also: reliable links, and feasible successors by design.

Q4.31 OSPF vs EIGRP: five differences.

CISCO · MICROSOFT · TECH-1 · ■■□ · T

30-second answer: link-state against DUAL; open against Cisco-origin; cost against bandwidth-and-delay; equal-cost only against unequal-cost balancing; areas against summarisation anywhere.

Answer. A fifth point is the AD (110 against 90 and 170) and a sixth the packet transport (IP 89 against 88 with reliable delivery). The comparison table is in §4.6.

Q4.32 Why does BGP use TCP? Which port?

ALL · TECH-1 · ■□□ · T

30-second answer: BGP needs reliable, ordered delivery of large tables; TCP provides it so BGP does not need its own retransmission; the port is 179.

Answer. A BGP session is long-lived and incremental: after the initial table only changes are sent, which relies on a reliable stream. The drawback is that TCP failure detection is slow, so keepalives and BFD supplement it.

Q4.33 List the BGP states. What does Active mean?

CISCO · AMAZON · JUNIPER · TECH-1 · ■□□ · S

30-second answer: Idle, Connect, Active, OpenSent, OpenConfirm, Established; Active means the router is trying to open the TCP connection and has failed, not that the session is up.

Answer. Troubleshoot Idle and Active with reachability (`ping` from the update source), ACLs on TCP 179, the AS number, the update source and `ebgp-multihop`. `show ip bgp summary` prints the state in the last column.

Q4.34 eBGP vs iBGP: AD, TTL, next hop, AS_PATH.

AMAZON · JUNIPER · ARISTA · TECH-1 · ■■□ · T

30-second answer: eBGP AD 20, TTL 1, next hop set to self, own AS prepended; iBGP AD 200, TTL 255, next hop unchanged, AS_PATH unchanged, and no re-advertisement to other iBGP peers.

Answer. The unchanged next hop is why iBGP needs `next-hop-self` or an IGP route to the external link. Table in §4.7.

Q4.35 Recite the BGP best-path selection order.

CISCO · AMAZON · JUNIPER · ARISTA · NOKIA · TECH-1 · ■■□ · S

30-second answer: N WLLA OMNI: next hop reachable, weight (high), local preference (high), locally originated, AS_PATH (short), origin (low), MED (low, same neighbour AS), eBGP over iBGP, lowest IGP metric; then oldest, lowest router ID, shortest cluster list, lowest neighbour address.

Answer. Walk the table of §4.7 aloud, then volunteer two reasons: weight and local preference come first because they are local policy; MED is compared only for one neighbour AS because the values of different ASes are not comparable. This is the single most reported BGP question. **Mistake.** Placing AS_PATH above local preference.

Q4.36 Weight vs local preference vs MED: scope, direction and higher or lower wins.

AMAZON · CISCO · TECH-1 · ■■□ · T

30-second answer: weight is local to one Cisco router, steers outbound, higher wins; local preference is shared in the AS by iBGP, steers outbound, higher wins; MED is sent to a neighbouring AS, hints inbound, lower wins.

	Scope	Steers	Wins
Weight	one router	outbound	highest
Local preference	whole AS	outbound	highest
MED	neighbour AS	inbound	lowest

Q4.37 Why does iBGP need a full mesh? How do route reflectors and confederations solve it?

GOOGLE · ARISTA · JUNIPER · TECH-2 · ■■□ · S

30-second answer: an iBGP router does not pass iBGP-learned routes to other iBGP peers (the AS_PATH does not change inside the AS, so it cannot detect loops), so all must peer directly; a route reflector is permitted to reflect, and a confederation turns sub-ASes into eBGP.

Answer. Ten routers need 45 sessions; two RRs and eight clients need 17. Clients peer only with the RR, which reflects client routes to all and non-client routes to clients. A confederation splits the AS into private sub-ASes with eBGP-like rules between them and a single public AS outward. RRs are more common.

Q4.38 How does a route reflector prevent loops? What is a cluster?

ARISTA · JUNIPER · TECH-2 · ■■■ · T

30-second answer: the RR adds ORIGINATOR_ID (the router that first injected the route) and prepends its CLUSTER_ID to CLUSTER_LIST; a router that sees its own ID, or an RR that sees its own cluster ID, discards the route.

Answer. A cluster is an RR with its clients; the cluster ID defaults to the RR's router ID. If two RRs serve one cluster, give them the same cluster ID so that they ignore each other's reflections; use distinct IDs for more redundancy at the price of more state.

Q4.39 An iBGP router has the route in its BGP table but not in its RIB. Why?

AMAZON · CISCO · TECH-1 · ■■□ · T

30-second answer: usually the BGP next hop is not reachable through the IGP, so the route is not valid for installation; else a better-AD route for the same prefix exists (RIB failure).

Answer. Check `show ip bgp <prefix>` (no `>`; "inaccessible"), `show ip route <next hop>`. Fix with `next-hop-self` on the border router or by carrying the link in the IGP. Also `show ip bgp rib-failure`.

Q4.40 Scenario: dual-homed to ISP-A and ISP-B. Make ISP-A primary for inbound and outbound traffic. Write the policy.

AMAZON · CISCO · MICROSOFT · TECH-2 · ■■■ · S

30-second answer: outbound, set local preference 200 on routes from ISP-A; inbound, prepend our AS two or three times on advertisements to ISP-B (and tag with the provider's low-preference community if offered).

Answer. The configuration and the `show ip bgp` output are in the dual-homing example of §4.7 (Figure 4.26 (p. 256)). **Follow-ups.** Prepending is a hint, because a remote AS with higher local preference ignores it; communities to the provider are stronger. More-specific prefixes (/25s) attract traffic strongly but are filtered by many networks beyond /24. Verify from a looking glass.

Q4.41 How does BGP multipath work? Which conditions must match? What does `multipath-relax` do?

ARISTA · GOOGLE · AMAZON · TECH-2 · ■■■ · T

30-second answer: `maximum-paths N` installs up to N paths that tie through the IGP-metric step; weight, local preference, AS_PATH length, origin, MED and the neighbouring AS must match; `as-path multipath-relax` drops the identical-AS requirement.

Answer. In spine-leaf fabrics each leaf may have its own AS, so equal paths have different AS numbers; relax allows ECMP across them as long as the path lengths are equal.

Q4.42 Why is eBGP used inside large data-centre fabrics instead of OSPF (RFC 7938)?

GOOGLE · MICROSOFT · ARISTA · AMAZON · TECH-2 · ■■■ · T

30-second answer: eBGP has per-neighbour policy, no flooding of the whole topology to every router, scales to very large fabrics, and its failures are easy to reason about; one session per link needs no IGP underneath.

Answer. Typical design: one AS per rack or per leaf pair, one AS for the spines, `allowas-in` or unique private ASNs, ECMP with multipath-relax, BFD for fast failure. Link-state flooding in a fabric of thousands of nodes produces churn that BGP's path-vector updates avoid. See §5.4 (p. 310).

Q4.43 What are BGP communities? Explain no-export and no-advertise.

JUNIPER · NOKIA · TECH-1 · ■■□ · T

30-second answer: a community is a 32-bit tag attached to a route to signal policy; `no-export` keeps a route inside the AS (or confederation); `no-advertise` keeps it from every peer.

Answer. Communities are optional transitive and are only sent if `send-community` is configured. Providers publish communities to set local preference or to blackhole.

Q4.44 Scenario: a customer announces a more-specific prefix of a major content provider. What happens globally? How do RPKI/ROV and max-prefix help?

AMAZON · GOOGLE · MICROSOFT · TECH-2 · ■■■ · S

30-second answer: routers prefer the longest prefix, so traffic for that range is drawn to the hijacker; RPKI origin validation marks the route invalid and filters it; a maximum-prefix limit stops a flood of leaked routes.

Answer. Longest-prefix match, not BGP preference, decides, so the more-specific wins everywhere it is accepted. ROV compares the origin AS and maximum length against signed ROAs. Providers filter customers by prefix list and IRR data, and a max-prefix on each customer session. The 2008 YouTube incident is the textbook case.

Q4.45 Coding: given BGP paths with attributes, implement best-path selection (steps 1 to 8).

GOOGLE · ARISTA · AMAZON · TECH-2 · ■■■ · S

30-second answer: filter the candidate list step by step, keeping only the paths that are best on each attribute, and stop as soon as one remains.

Approach. Eight ordered filters, each a keep-the-best on a key. MED is compared only within one neighbouring AS unless always-compare is set.

```
from dataclasses import dataclass
ORIGIN = {"i": 0, "e": 1, "?": 2}

@dataclass
class Path:
    nh_ok: bool; weight: int; lp: int;
local: bool
    as_path: list; origin: str; med: int
    ebgp: bool; igp: int; peer_as: int;
rid: str

def keep(c, key, best=min):
    v = best(key(p) for p in c)
    return [p for p in c if key(p) == v]

def med_step(c, always):
    if always: return keep(c, lambda p:
p.med)
    low = {}
    for p in c: low[p.peer_as] =
min(low.get(p.peer_as, p.med), p.med)
    return [p for p in c if p.med ==
low[p.peer_as]]

def best_path(paths, always_med=False):
    c = [p for p in paths if p.nh_ok]
    steps = [lambda c: keep(c, lambda p:
p.weight, max),
             lambda c: keep(c, lambda p:
p.lp, max),
             lambda c: keep(c, lambda p:
p.local, max),
             lambda c: keep(c, lambda p:
len(p.as_path)),
             lambda c: keep(c, lambda p:
ORIGIN[p.origin]),
             lambda c: med_step(c,
always_med),
             lambda c: keep(c, lambda p:
p.ebgp, max),
             lambda c: keep(c, lambda p:
p.igp)]
    for s in steps:
        if len(c) <= 1: break
        c = s(c)
    rid = lambda p: tuple(map(int,
p.rid.split(".")))
    return min(c, key=rid) if c else
None
```

Complexity. O(8n) time, O(n) space. **Mistakes.** Comparing MED across neighbour ASes; treating higher MED as better; forgetting step 0 (unreacha-

ble next hop). Real implementations also add the age and cluster-list tie-breaks.

Q4.46 HSRP vs VRRP: which is the open standard? Default preempt?

CISCO · TECH-1 · ■□□ · T

30-second answer: VRRP is the IETF standard; HSRP is Cisco's; HSRP preempt is off by default and VRRP preempt is on.

Answer. Other differences: groups addresses (224.0.0.2 or .102 against 224.0.0.18), virtual MACs and protocol (UDP 1985 against IP 112), VRRP's owner priority 255. Table in §4.8.

Q4.47 With HSRP tracking, R1's uplink fails but R1 stays active. Why?

CISCO · TECH-1 · ■■□ · T

30-second answer: tracking only lowers R1's priority; R2 takes over only if R2 has `preempt` and the new priority is below R2's, so check preempt on R2, the size of the decrement, and the track object.

Answer. `show standby` displays the current priority and the tracked object's state. R1: 110 with decrement 20 becomes 90; with R2 at 100 and preempt, R2 becomes active. A decrement below the gap of 10 leaves R1 active.

Q4.48 Prefix list `10.0.0.0/8 ge 16 le 24`: which of four prefixes match?

CISCO · JUNIPER · TECH-1 · ■■□ · S

30-second answer: 10.0.0.0/8 no, 10.1.0.0/16 yes, 10.1.1.0/24 yes, 10.1.1.0/25 no.

Answer. The prefix must lie inside 10.0.0.0/8 and its length must be between 16 and 24 inclusive.

Q4.49 Explain the loop from two-way redistribution at two points. How do tags fix it?

CISCO · JUNIPER · TECH-2 · ■■■ · T

30-second answer: a route redistributed from protocol A into B at one router is seen as native B by the second router, which redistributes it back into A; tags mark routes at the first redistribution, and the reverse route map denies that tag.

Answer. Configuration in §4.9 (Figure 4.30 (p. 268)). Alternatives: filter by prefix list, or raise the AD of the redistributed routes.

Q4.50 What is an MPLS label stack? Explain PHP and why implicit null exists.

JUNIPER · NOKIA · CISCO · TECH-1 · ■■□ · T

30-second answer: a stack is a sequence of 32-bit shim headers, the top one used for forwarding; the penultimate router pops the top label so that the egress router does one lookup instead of two, and the egress signals this by advertising implicit null (3).

Answer. The bottom-of-stack bit marks the last label. Without PHP the egress router would pop the label and then perform an IP lookup, two lookups in a hot spot.

Q4.51 Walk a packet through an MPLS L3VPN. What do RD and RT do?

JUNIPER · NOKIA · CISCO · TECH-2 · ■■■ · T

30-second answer: the ingress PE looks up the customer's VRF and pushes a VPN label (inner) and a transport label (outer); P routers swap the transport label; the penultimate router pops it; the egress PE uses the VPN label to select the VRF and forwards by IP. RD makes prefixes unique; RT selects which VRFs import them.

Answer. Walk and figure in §4.10 (Figure 4.34 (p. 271)). Control plane: LDP for transport labels, MP-BGP VPNv4 for VPN labels.

Q4.52 Policing vs shaping: the effect on TCP flows.

CISCO · NOKIA · BROADCOM · TECH-1 · ■■□ · T

30-second answer: a policer drops or re-marks excess packets, so TCP sees loss and its window oscillates; a shaper queues them, so TCP sees extra delay but fewer losses.

Answer. Shape on egress towards a slower circuit; police on ingress to enforce a contract. A shaper adds jitter, so voice goes to a priority queue ahead of it.

Q4.53 Decode DSCP 46 and AF31. Why does voice use LLQ?

CISCO · QUALCOMM · NOKIA · TECH-1 · ■■□ · T

30-second answer: DSCP 46 is EF (101110, ToS 0xB8); AF31 is class 3, drop precedence 1: 8 × 3 + 2 × 1 = 26; voice needs low delay and jitter, which a strict-priority queue provides.

Answer. LLQ is CBWFQ plus a priority queue with an implicit policer to stop starvation. Voice targets: delay 150 ms, jitter 30 ms, loss 1%.

Q4.54 How does an IPv4 multicast address map to a MAC address? Why the 32:1 ambiguity?

CISCO · JUNIPER · BROADCOM · TECH-1 · ■■□ · T

30-second answer: the low 23 bits of the group address follow the prefix 01:00:5E; a group has 28 variable bits, so 5 are lost and 32 groups share a MAC.

Answer. 239.1.1.1 and 224.129.1.1 both map to 01:00:5E:01:01:01 (§4.12). The host filters again at IP level; the cost is a little CPU.

Q4.55 Explain the PIM-SM shared tree vs the shortest-path tree and the RPF check. What if unicast routing is asymmetric?

JUNIPER · NOKIA · TECH-2 · ■■■ · T

30-second answer: receivers first join a tree rooted at the RP, (*,G); the last-hop router then switches to a tree rooted at the source, (S,G), which is shorter; the RPF check accepts a packet only on the interface the unicast table would use to reach the source.

Answer. With asymmetric routing the interface that carries the stream differs from the RPF interface, so the stream is dropped. Fix with a static `ip mroute`, a multicast-specific routing table (MP-BGP), or by making the routing symmetric.

Q4.56 Long-form: PVST+ on a four-switch square with a diagonal, VLANs 10 and 20 with different roots.

CISCO · TECH-2 · ■■■ · T

30-second answer: each VLAN runs its own election; with the root at SW1 for VLAN 10 and at SW4 for VLAN 20, SW4 blocks both square links for VLAN 10 and SW2 and SW3 block their links to SW1 for VLAN 20.

Approach. SW1 to SW4 have MACs ending 0001 to 0004. Links: SW1-SW2, SW2-SW4, SW4-SW3, SW3-SW1 and the diagonal SW1-SW4, all 1 Gb/s (cost 4). VLAN 10 keeps the default priority; VLAN 20 has priority 4096 on SW4.

Answer. *VLAN 10* (root SW1, lowest MAC): SW2 and SW3 use their direct links to SW1; SW4 uses the diagonal (cost 4). On SW2-SW4 both offer cost 4, so the lower BID (SW2) is designated and SW4's port blocks; on SW3-SW4 likewise SW3 is designated and SW4 blocks. *VLAN 20* (root SW4): SW2 and SW3 use their links to SW4; SW1 uses the diagonal. On SW1-SW2 and SW1-SW3 SW1 has the lower BID, so it is designated and SW2 and SW3 block. Hence the diagonal forwards both VLANs, and each square link carries one VLAN only.

Mistakes. Using one tree for both VLANs; forgetting that the root has no root port.

Q4.57 Long-form: EIGRP topology table, feasible successors and variance.

CISCO · TECH-2 · ■■■ · T

30-second answer: the successor is the lowest-FD neighbour; every neighbour with RD below that FD is a feasible successor; `variance 2` uses those whose own FD is below twice the best FD.

Approach. R1 has three neighbours for 10.9.9.0/24: R2 (FD 3000, RD 1000), R3 (FD 5400, RD 2500), R4 (FD 4200, RD 3500).

Answer. Successor R2, FD 3000. Feasibility condition RD < 3000: R3 (2500) passes, R4 (3500) fails. Variance 2 gives a limit of 6000: R3's FD 5400 is inside it, so R2 and R3 carry traffic, with a share inversely proportional to the metrics (about 64% and 36%). R4 is never used, though its FD of 4200 is inside the limit, because it is not feasible. If R2 fails, R3 is promoted with no query; R4 would require a query.

```
R1# show ip eigrp topology 10.9.9.0/24
P 10.9.9.0/24, 1 successors, FD is 3000
        via 10.1.2.2 (3000/1000)
        via 10.1.3.3 (5400/2500)
```

Q4.58 Long-form: an HSRP failover timeline.

CISCO · TECH-2 · ■■■ · T

30-second answer: tracking lowers R1's priority at once; with preempt, R2 becomes active in under a second (or at the hold time if R1 simply dies), then R1 preempts back after recovery.

Answer. Times are for hello 3 s, hold 10 s, R1 priority 110 with a decrement of 20 on Gi0/1, R2 priority 100 with preempt.

Time	Event	R1	R2
0	steady state	110, Active	100, Standby
10 s	Gi0/1 fails	90	100, hears a lower priority hello
10 s + 1 hello	R2 preempts	Standby	Active, gratuitous ARP
40 s	Gi0/1 returns	110, preempts after delay	Standby

If R1 were powered off instead, R2 would take over only after the 10 s hold time (about 3 s with `timers msec`). Verification is `show standby brief`. **Mistake.** Forgetting that R2 also needs `preempt` when the cause is a priority change.

Q4.59 Long-form: label tables at every node of an MPLS L3VPN.

JUNIPER · NOKIA · TECH-2 · ■■■ · T

30-second answer: PE1 pushes (24, 300); P1 swaps 24 for 35; P2 pops 35 (PHP); PE2 maps 300 to VRF A and routes by IP.

Answer. Customer A: 10.2.2.0/24 behind PE2, loopback 10.255.0.5/32.

Node	Table	Entry	Action
PE1	VRF A FIB	10.2.2.0/24: VPN label 300, next hop 10.255.0.5	push 300, then LDP label 24
P1	LFIB	in 24, 10.255.0.5/32	swap to 35, out to P2
P2	LFIB	in 35, 10.255.0.5/32	pop (PE2 advertised implicit null), out to PE2
PE2	VPN LFIB	in 300	pop, look up in VRF A, forward to CE2

Customer B's identical 10.1.1.0/24 has a different RD and RT (65000:2) and a different VPN label, so it lands in VRF B. **Mistake.** Saying the P routers look at label 300; they never do.

Q4.60 Long-form: QoS policy for a 10 Mb/s branch WAN with a bandwidth check.

CISCO · NOKIA · TECH-2 · ■■■ · T

30-second answer: shape the interface to 10 Mb/s in a parent policy, and in the child give voice an LLQ of 10%, video 30%, critical data 25%, scavenger 1%, and fair queuing to the rest; the reserved sum is 66%, below the 75% limit.

Answer. The configuration is in §4.11. The check: 10% of 10 Mb/s is 1 Mb/s of strict priority, enough for about ten G.711 calls at roughly 87 kb/s each with overhead. Video 3 Mb/s, critical data 2.5 Mb/s, scavenger 100 kb/s; 3.4 Mb/s remains for default. The shaper makes the Ethernet port behave as a 10 Mb/s link so that the child queues engage. Verify with `show policy-map interface`, and mark with iperf3. **Mistake.** Applying the child policy to a 1 Gb/s port without the parent shaper: it never congests, so it never queues.

Revision Spread

THE 15-MINUTE VERSION

Ten facts. (1) Routing decision: longest prefix, then AD, then metric. (2) STP: lowest bridge ID is root; root port = lowest path cost, then lowest sender BID, then lowest sender port ID; the root has no root port. (3) STP priority moves in steps of 4096; VLAN is added (32768 + 1 = 32769). (4) OSPF states: Down, Init, 2-Way, ExStart, Exchange, Loading, Full; stuck in ExStart means MTU. (5) OSPF DR: priority, then router ID; not pre-emptive; 224.0.0.5 and .6. (6) LSA 1 and 2 stay in the area, 3 and 4 cross ABRs, 5 is domain-wide, 7 is converted to 5. (7) BGP: TCP 179, AD 20 and 200, iBGP never re-advertises iBGP routes, next hop unchanged in iBGP. (8) Best path: **N WLLA OMNI**, then **Old Routers Clear Neighbours**. (9) HSRP preempt off, VRRP on; tracking needs preempt on the standby. (10) EF = 46, AFxy = 8x + 2y, multicast MAC = 01:00:5E + low 23 bits.

Three figures to redraw. The STP triangle with R, D and blocked ports (Figure 4.9); the OSPF state ladder with its stuck points (Figure 4.16); the dual-homed AS diagram with local preference outbound and prepending inbound (Figure 4.26).

Administrative distance

Source	AD	Source	AD
Connected	0	RIP	120
Static	1	EIGRP external	170
EIGRP summary	5	iBGP	200
eBGP	20	DHCP default	254
EIGRP internal	90	Unusable	255
OSPF	110	IS-IS	115

OSPF states, LSAs and areas

State	Meaning
Down / Init	no hello / hello heard, own ID absent
2-Way	bidirectional; DR election; final for DROTHERs
ExStart / Exchange	master elected / DBD headers swapped (MTU)
Loading / Full	LSR and LSU / databases equal

LSA	Origin	Scope
1 Router	each router	area
2 Network	DR	area
3 Summary	ABR	other areas
4 ASBR summary	ABR	other areas
5 External	ASBR	all but stub
7 NSSA	ASBR in NSSA	NSSA, to 5 at ABR

Stub: no 4, 5. Totally stubby: no 3, 4, 5. NSSA: no 4, 5; has 7. Timers 10 and 40 s (broadcast), 30 and 120 s (NBMA). Cost = reference / bandwidth, default reference 100 Mb/s. Route preference: O, O IA, E1, E2.

BGP best path (Cisco)

0 next hop reachable. 1 weight, high. 2 local preference, high (100). 3 locally originated. 4 AS_PATH, short. 5 origin, i < e < ?. 6 MED, low, same neighbour AS. 7 eBGP over iBGP. 8 IGP metric, low. 9 multipath. 10 oldest eBGP. 11 router ID, low. 12 cluster list, short. 13 neighbour address, low.

Attributes: well-known mandatory (ORIGIN, AS_PATH, NEXT_HOP); well-known discretionary (LOCAL_PREF, ATOMIC_AGGREGATE); optional transitive (COMMUNITY, AGGREGATOR); optional non-transitive (MED, ORIGINATOR_ID, CLUSTER_LIST). Keepalive 60 s, hold 180 s. Sessions: full mesh n(n-1)/2.

STP and EtherChannel

802.1D state	Time	RSTP state
Blocking	max age 20 s	Discarding
Listening	15 s	Discarding
Learning	15 s	Learning
Forwarding		Forwarding

Hello 2 s, forward delay 15 s, max age 20 s. Costs (short): 10M 100, 100M 19, 1G 4, 10G 2. Roles: root,

inks; Loop Guard on non-designated uplinks. LACP active or passive, PAgP desirable or auto; passive-passive and auto-auto never form. Flags: SU in use, P bundled, s suspended, D down.

Short notes by section

4.1 CLI. Three planes: management, control, data. Prompts show the mode. `copy run start` saves. Register 0x2142 skips the saved configuration (recovery). CEF = FIB + adjacency table. Say: "the control plane builds the tables; the data plane only reads them."

4.2 VLANs. Access untagged, trunk tagged; native untagged. Tag 4 bytes: TPID 0x8100, PCP 3, DEI 1, VID 12. ROAS hairpins; SVI routes in hardware and needs an active port. DTP: configure statically. VTP: transparent or off. Say: "a VLAN is a broadcast domain; routing is needed between them."

4.3 STP. Four-step election; costs 4 (1G), 2 (10G); Rapid PVST+ is the default choice; MST for many VLANs. BPDU Guard on edge, Root Guard down, Loop Guard on uplinks. EtherChannel: LACP, same speed, duplex, VLANs; one flow, one member. Say: "STP elects a root and blocks what is left."

4.4 Routing table. LPM, then AD, then metric. Floating static: AD above the primary. Null0 anchors summaries. Say: "the most specific route wins, whatever its source."

4.5 OSPF. Link-state, Dijkstra, protocol 89, 224.0.0.5 and .6. Adjacency checklist: area, subnet, timers, authentication, stub flag, MTU, unique RID. Summarise at ABRs; stub areas shrink the database. Say: "every router in an area holds the same database and runs its own SPF."

4.6 EIGRP. DUAL; FC: RD < FD; K1 = K3 = 1; variance only over feasible successors; stubs and summaries limit queries. Say: "a feasible successor is a loop-free backup chosen in advance."

4.7 BGP. Path vector over TCP 179. iBGP: next hop unchanged, no re-advertisement, full mesh or RR. LOCAL_PREF outbound; prepend, MED, communities inbound. `network` needs an exact RIB match. Say: "BGP chooses by policy first and distance second."

4.8 FHRP. Virtual IP and MAC; HSRP preempt off, VRRP on; track and preempt together; align with the STP root.

4.9 Policy. Prefix list `ge` / `le`; route-map first match, implicit deny; tag at redistribution, deny the tag back; PBR is per interface; VRF-lite = separate tables.

4.10 MPLS. 20 / 3 / 1 / 8; push, swap, pop; PHP; VPN label inner, transport label outer; RD unique, RT selects.

4.11 QoS. Mark at the edge; LLQ for voice; CBWFQ shares; WRED avoids synchronisation; policer drops, shaper delays; Tc = Bc / CIR.

4.12 Multicast. IGMP join; PIM-SM via RP, then SPT; RPF check; MAC 01:00:5E + 23 bits.

4.13 Services. NAT overload by ACL; relay with `ip helper-address`; NTP stratum; SNMPv3; syslog 0 to 7; ACL top-down, implicit deny, wildcard = inverse mask.

4.14 Tunnels. GRE 24 bytes, MTU 1476, recursive-routing fault; IP SLA with a tracked static route; SD-WAN separates control from edge.

Top show commands

Task	IOS command	What to look for
Interfaces	`show ip interface brief`	up/up
VLANs and trunks	`show vlan brief`, `show interfaces trunk`	allowed list, native VLAN
STP	`show spanning-tree vlan N`	root, role, state
EtherChannel	`show etherchannel summary`	SU, P, s
Routes	`show ip route`, `show ip cef N`	source, AD/metric, next hop
OSPF	`show ip ospf neighbor`, `... database`, `... interface`	state, LSAs, timers
BGP	`show ip bgp summary`, `show ip bgp N`	state/prefixes, >, attributes
FHRP	`show standby brief`	priority, state
MPLS	`show mpls forwarding-table`, `show bgp vpnv4 unicast all`	labels
QoS	`show policy-map interface`	per-class drops
Multicast	`show ip mroute`, `show`	

	`ip igmp groups`	flags, RPF
Services	`show ip nat translations`, `show ntp status`, `show ip access-lists`	matches, sync

Other facts

Topic	Facts
FHRP	HSRP 224.0.0.2/.102, UDP 1985, 3 s / 10 s; VRRP 224.0.0.18, proto 112; GLBP 224.0.0.102
MPLS	label 20, TC 3, S 1, TTL 8; labels 0 to 3 reserved; LDP 646; RD unique, RT selects
QoS	EF 46, AF11 10, AF21 18, AF31 26, AF41 34, CS1 8; voice 150 ms, 30 ms, 1%; Tc = Bc / CIR
Multicast	224/4, 232/8 SSM, 239/8 private; IGMPv2, v3; PIM-SM; RPF
Services	syslog 0 to 7: EACCEWNID; NTP stratum, UDP 123; SNMPv3 authPriv; GRE 24 bytes

Q	A
Access vs trunk?	One VLAN untagged; many VLANs tagged
Usable VLAN IDs?	4094 (12 bits, 0 and 4095 reserved)
802.1Q tag size?	4 bytes; frame 1522
`auto` + `auto` DTP?	Access
ROAS drawback?	One link carries each packet twice
STP root?	Lowest bridge ID
STP priority step?	4096
Root port?	Lowest root path cost, on non-root only
802.1D worst convergence?	50 s
RSTP port states?	Discarding, learning, forwarding
Where Root Guard?	Distribution downlinks
LACP passive + passive?	No bundle

Q	A
`(s)` in channel summary?	Suspended member
AD of OSPF, eBGP, iBGP?	110, 20, 200
Floating static?	AD above the primary route
/24 static vs /16 connected?	/24: longest prefix
OSPF protocol number?	89
Stuck in ExStart?	MTU mismatch
2-Way between DROTHERs?	Normal
Is the DR preemptive?	No
Default OSPF reference?	100 Mb/s: 1G = 10G = 1
LSA 3 originator?	ABR
LSA 5 blocked in?	Stub, NSSA
Totally stubby gets?	Type 1, 2 and one type 3 default
E1 vs E2?	E1 adds internal cost; E1 preferred
EIGRP FC?	RD < FD
EIGRP K defaults?	K1 = K3 = 1
EIGRP AD?	90, 170 external
BGP port?	TCP 179
BGP Active?	Trying to connect, not up
iBGP next hop?	Unchanged
Full mesh count?	n(n-1)/2
RR loop attributes?	ORIGINATOR_ID, CLUSTER_LIST
Highest wins?	Weight and local preference
MED compared?	Same neighbour AS only
Inbound TE tools?	Prepend, MED, more specifics, communities
`network` in BGP needs?	Exact RIB match
`no-export`?	Not outside the AS
HSRP default preempt?	Off
VRRP owner priority?	255
`ge 16 le 24`?	Length 16 to 24 inclusive
Redistribution loop fix?	Tag and deny the tag

PHP?	Penultimate router pops the label
RD vs RT?	Uniqueness vs import and export
DSCP EF?	46, ToS 0xB8
AF31?	26
Policer vs shaper?	Drops vs delays
Multicast MAC?	01:00:5E + low 23 bits
RPF check?	Accept only from the unicast path to the source
GRE overhead?	24 bytes; MTU 1476

PART I · PILLAR 5

5

Data Centre and Cloud Networking

One request crosses a dozen layers of encapsulation inside a provider; we learn what every hop does and why.

5.1 Why data centres broke the campus model	MUST
5.2 Clos networks and spine-leaf arithmetic	MUST
5.3 ECMP and flow hashing	MUST
5.4 The eBGP underlay, BFD and unnumbered links	MUST
5.5 Overlays: VXLAN	MUST
5.6 EVPN: a control plane for the overlay	SHOULD
5.7 SDN, OpenFlow and P4	SHOULD
5.8 Router and switch internals	MUST
5.9 Load balancers	MUST
5.10 Anycast and content delivery networks	SHOULD
5.11 AWS: the Virtual Private Cloud	MUST
5.12 Azure: the virtual network	SHOULD
5.13 Hybrid connectivity: Direct Connect, ExpressRoute and VPN	SHOULD
5.14 The life of a request in the cloud	MUST
5.15 Mini-project: Pocket Hyperscaler	OPTIONAL

The routing and switching of Pillar 4 (p. 229) serve a building full of people. This chapter serves a building full of machines, and nearly every assumption changes: the traffic runs between servers, the topology is regular, the devices are cheap and numerous, and the tenants are virtual. We keep the book's fictional organisation, **Campus Net**, and give it two additions. First, the university builds a small data centre of its own: two spines, four leaves (L1 to L4) and a pair of border leaves that attach to the existing border routers BR1 and BR2 (§4.7 (p. 256)). Secondly, it rents cloud capacity: a VPC in AWS with the range 10.0.0.0/16 and a hub virtual network in Azure with 10.100.0.0/16, both reached from the campus over private circuits.

The chapter has three movements. Sections 5.1 to 5.4 build the physical fabric and the routing that runs on it; sections 5.5 to 5.7 add virtual networks on top of that fabric; sections 5.8 to 5.10 look inside a forwarding device and at the services that sit in front of servers. Sections 5.11 to 5.14 then show how a public cloud packages all of this as a product. Subnetting (§3.4 (p. 174)), the TCP congestion mechanisms (§3.9 (p. 192)), BGP itself (§4.7 (p. 256)) and the longest-prefix-match algorithm (§1.9 (p. 41)) are taught in their home sections; here we use them.

5.1 Why data centres broke the campus model MUST · ■□□

BIG IDEA

Server-to-server traffic now exceeds client-to-server traffic, and a spanning-tree hierarchy can neither carry it nor survive its failures, so data centres replaced the tree with a routed mesh in which every path is equally short and every link forwards.

Traffic that enters or leaves a data centre is termed *north-south*; traffic between servers inside it is termed *east-west*. A campus is mostly north-south: laptops talk to servers and to the Internet. A data centre is the opposite, because one user request fans out into many internal calls (a front end asks a profile service, a recommendation service, a cache and a database), and because storage replication, backups and distributed computation move bulk data between machines. This is the reason for the change of design: published accounts from the large operators, such as Google's *Jupiter Rising* paper, describe internal traffic as a large multiple of external traffic, so the internal network, not the Internet link, is the bottleneck.

The campus design of Pillar 4 (p. 229) has five properties that fail under this load. (i) *Capacity.* A three-tier tree funnels traffic through a few aggregation and core switches, and spanning tree (§4.3 (p. 237)) blocks redundant links to prevent loops, so a large fraction of the cabling carries nothing. (ii) *Latency.* The number of switches between two servers depends on where they sit, from one switch in the same rack to five across the tree, which makes performance unpredictable. (iii) *Failure scope.* A layer-2 domain is one broadcast domain, so a loop or a broadcast storm takes down everything in it, and spanning-tree reconvergence takes seconds. (iv) *Scale of state.* Every switch in a VLAN learns every MAC address, and thousands of virtual machines overflow the MAC tables. (v) *Segments.* A VLAN identifier has 12 bits, so there are at most 4,094 usable VLANs, too few for thousands of tenants.

The remedy is to give up large layer-2 domains. The data centre is built as a regular, routed mesh (§5.2) in which every link forwards and equal paths are used together (§5.3), the routing protocol is one that scales and isolates failures (§5.4), and tenants are given virtual layer-2 and layer-3 networks that are carried *over* this fabric (§5.5 and §5.6).

WORKED EXAMPLE

Campus Net replaces a two-tier tree with a leaf-spine fabric. The hall has 40 racks. In the tree design each rack switch has two 10 Gb/s uplinks, one to each of two aggregation

switches, and spanning tree blocks one of the two.

1. Physical uplink capacity is 40 racks × 2 × 10 Gb/s = 800 Gb/s.
2. Spanning tree leaves one forwarding uplink per rack: 40 × 10 = 400 Gb/s usable, so 50% of the cabling is idle.
3. Rewire the same cables as a leaf-spine fabric: each rack switch becomes a leaf, the two aggregation switches become spines, and the leaf-to-spine links are routed (layer 3) instead of switched. Both uplinks of every leaf now forward, giving 800 Gb/s.
4. Capacity can now grow without new tiers: a third and fourth spine, with one more uplink per leaf for each, double it again to 1.6 Tb/s.
5. Path length is now uniform: two servers in different racks always cross exactly three switches (leaf, spine, leaf), where the tree needed up to five.

Result: the same cables carry twice the traffic (800 against 400 Gb/s), and the fabric grows by adding spines.

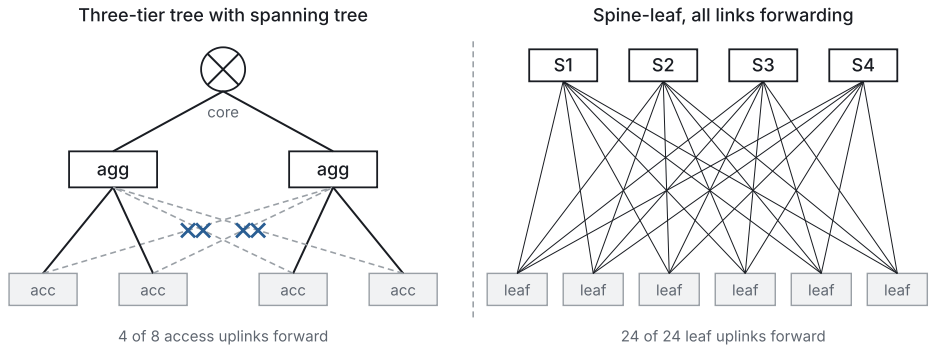


Figure 5.1 The same racks wired two ways. Left: spanning tree blocks one uplink per access switch (crosses), so half of the cabling carries nothing. Right: a routed fabric uses every link at once.

The table summarises the change. Note that every row of the right-hand column is a consequence of one decision, namely to route in the fabric rather than switch.

Problem in a campus tree	Data-centre answer	Where taught
Blocked links, 50% idle	Layer-3 links, all forwarding, ECMP	§5.2, §5.3
Variable hop count	Equal-length leaf-spine-leaf paths	§5.2
Slow, large failure domain	Per-link BGP and BFD, sub-second	§5.4
MAC table and VLAN limits	Overlay with 24-bit segment identifier and a BGP control plane	§5.5, §5.6
Manual box-by-box configuration	Central controllers and automation	§5.7, Pillar 10

TRAP

Claim: “Spine-leaf removes spanning tree entirely.” *Why it is wrong:* the fabric links no longer run it, but the host-facing ports of a leaf may still run spanning-tree guards (BPDU guard, loop protection) because a misconfigured server or a cabling mistake can still create a loop at the edge. *Correct statement:* spanning tree is no longer used for loop prevention between switches; at the edge it survives as a safety device.

CHECK YOURSELF

1. Define east-west traffic and give two sources of it. [1]
2. In Campus Net's tree, what fraction of the uplink capacity does spanning tree waste, and why? [2]
3. How many switches separate two servers in different racks of a leaf-spine fabric? [3]

5.2 Clos networks and spine-leaf arithmetic MUST · ■■□

BIG IDEA

A folded Clos network connects every leaf to every spine, so any two leaves are exactly two hops apart, capacity grows by adding spines, and the size of the fabric follows from three port counts.

In 1953 Charles Clos analysed telephone switches built from small switching elements in several stages, and showed how to obtain a large non-blocking switch from small ones. A data-centre *folded Clos* uses the same idea with the input and output stages folded onto one set of devices. The lower tier consists of *leaf* switches (also called top-of-rack or ToR switches), which have server-facing ports and uplinks; the upper tier consists of *spine* switches, which connect only to leaves. Every leaf has one link to every spine and there are no leaf-to-leaf or spine-to-spine links. Consequently any leaf-to-leaf path is leaf, spine, leaf, and there are as many equal-cost paths as there are spines (Figure 5.2).

Three quantities describe a fabric. The *oversubscription ratio* of a leaf is its server-facing bandwidth divided by its uplink bandwidth; a ratio of 1:1 means the leaf can send everything it receives upwards, and 3:1 means that, if all servers transmit at once, each gets a third of its line rate. The *bisection bandwidth* is the bandwidth between the two halves of the network when it is cut into two equal parts in the worst way; it measures how well any server can talk to any other. The *radix* (port count) of the spine limits how many leaves can attach. Note that oversubscription is a property of the *design*, chosen to save money because servers rarely transmit at line rate together; it is not the same as *blocking*, which is a property of the topology. A Clos network is not blocking in the fabric itself, whatever the leaf ratio.

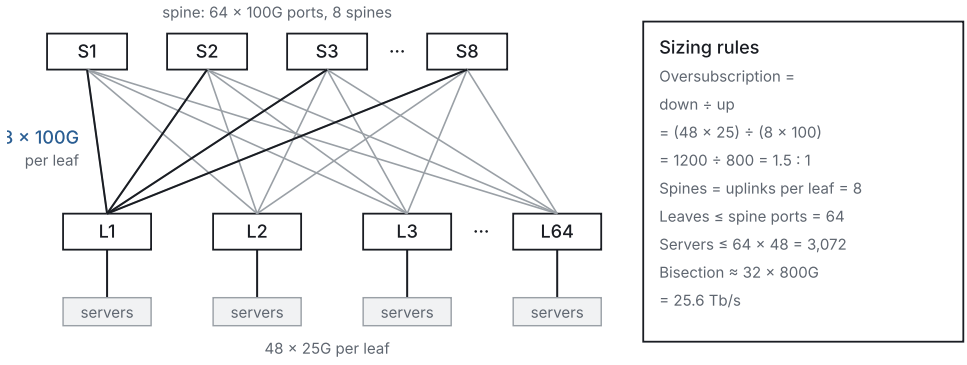


Figure 5.2 A two-tier fabric with its port budget. Every leaf has one link to every spine, so the number of spines equals the uplinks per leaf, and the number of leaves is limited by the ports on a spine.

WORKED EXAMPLE

Size the fabric. A leaf has 48 server ports of 25 Gb/s and 8 uplinks of 100 Gb/s. A spine has 64 ports of 100 Gb/s. How large is the fabric, and what changes for a 1:1 design?

1. Oversubscription: server bandwidth 48 × 25 = 1,200 Gb/s; uplink bandwidth 8 × 100 = 800 Gb/s; ratio 1,200 ÷ 800 = 1.5 : 1.
2. Spines: each leaf needs one link to every spine, so the number of spines equals the uplinks per leaf, which is 8.
3. Leaves: each spine has 64 ports and gives one to every leaf, so at most 64 leaves.
4. Servers: 64 leaves × 48 ports = 3,072 servers.
5. Bisection bandwidth: cut the 64 leaves into two groups of 32; the traffic that crosses is limited by the uplinks of one group, 32 × 800 = 25,600 Gb/s = 25.6 Tb/s. The 1,536 servers in a half could offer 32 × 1,200 = 38.4 Tb/s, so the ratio is again 1.5.
6. For 1:1 there are two routes. Either give each leaf 12 uplinks of 100 Gb/s (12 spines, and each leaf now needs 60 ports), or keep 8 uplinks and populate only 32 server ports (32 × 25 = 800 Gb/s). The first keeps 3,072 servers and costs four more spines; the second halves the servers per leaf.

Result: 1.5 : 1, 8 spines, up to 64 leaves and 3,072 servers; 1 : 1 needs 12 uplinks per leaf.

When 64 leaves are not enough, a third tier is added. Group leaves and spines into *pods*, and connect the spines of different pods through a layer of *super-spines*. Take a pod of 32 leaves and 4 spines. Each leaf has 4 uplinks (one per spine); each spine uses 32 ports down to the leaves and 32 ports up. The super-spines are arranged in 4 *planes*, one per spine position: the first spine of every pod connects to the 32 super-spines of plane 1, and so on. A 64-port super-spine has one port for each of up to 64 pods. The fabric then holds 64 pods × 32 leaves = 2,048 leaves, hence 2,048 × 48 = 98,304 servers, using 128 super-spines. A rule of thumb for identical k-port switches follows from the same argument: a two-tier fabric at 1:1 supports $k^2/2$ server ports (k leaves, each with k/2 down) and a three-tier fabric supports $k^3/4$. With k = 64 these are 2,048 and 65,536. The large operators build exactly this, calling the planes by their own names (Meta's *fabric* design, Google's Jupiter); the principle is the same (Figure 5.3).

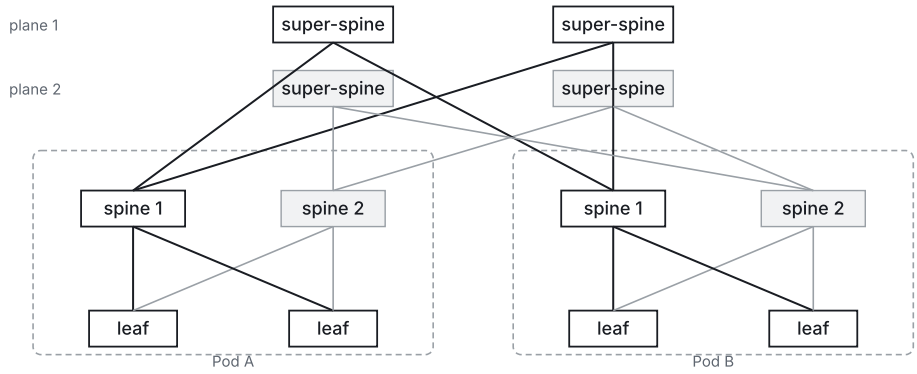


Figure 5.3 A five-stage Clos with two pods and two planes. Spine 1 of every pod reaches only the super-spines of plane 1, and spine 2 only those of plane 2, so each plane is an independent fabric and a failure stays inside it.

DEEP DIVE · MAY BE SKIPPED

Reading *Jupiter Rising* in forty-five minutes. Google's SIGCOMM 2015 paper is the one that candidates most often cite. Read the abstract and introduction first (ten minutes): they list the five generations of the network and the three ideas that recur, namely merchant silicon, Clos topology and centralised control. Then study the figures of the building blocks (fifteen minutes) and redraw one stage by hand, checking the port counts as in the example above. Read the section on centralised control for ten minutes: it explains why the routing is computed by a logically central controller and how switches behave if it is unreachable. Finish with the lessons and operations section, which states what went wrong

in practice (failures of links and of upgrades). The newer *Jupiter Evolving* paper adds optical circuit switches; the sketch of that idea is enough for an interview.

TRAP

Claim: "A 1:1 leaf makes the whole fabric non-blocking." *Why it is wrong:* oversubscription is measured at one tier. If there are fewer spines or slower spine ports than the leaf uplinks require, the next tier throttles traffic, and a single flow can never exceed one link anyway. *Correct statement:* a fabric is non-blocking only if every tier has as much capacity going up as coming down, and a single flow is still limited to the speed of one path.

INTERVIEW LENS

GOOGLE · MICROSOFT · ARISTA · JUNIPER

Expect "size this fabric" with port counts given, and "why not three tiers of chassis switches?". A strong answer states the formulas (oversubscription = down ÷ up; spines = uplinks per leaf; leaves ≤ spine ports; servers = leaves × server ports), gives the 1:1 variant, and then names the costs of going to five stages: more hops, more cables and a harder failure analysis. Candidates report that *Jupiter Rising* (SIGCOMM 2015) is the paper to read; forty-five minutes on its abstract, the Clos figures and the discussion of centralised control are enough for an interview.

CHECK YOURSELF

1. A leaf has 24 × 10G server ports and 4 × 40G uplinks. What is the oversubscription? [4]
2. With 32-port spines, how many leaves can the fabric hold? [5]
3. Why is oversubscription not the same as blocking? [6]

5.3 ECMP and flow hashing MUST · ■■□

BIG IDEA

A fabric has many equal paths, and a router uses them all by hashing each flow's header fields to pick one next hop, so a flow stays in order while different flows spread across the paths.

Equal-cost multipath (ECMP) lets a router install several next hops for one prefix, all with the same routing cost (§4.4 (p. 243)). Which next hop carries a given packet is decided by a hash. The router takes a set of header fields, typically the *5-tuple* (source address, destination address, protocol, source port, destination port), computes a hash with a hardware function such as CRC16 or CRC32 plus a configurable *seed*, and uses the result to index the list of next hops (Figure 5.4). Every packet of one TCP connection has the same 5-tuple, hence the same hash, hence the same path, so the segments arrive in order; different connections have different tuples and spread across the paths. If the router balanced individual packets instead, consecutive segments would take paths with different queueing delays and arrive out of order, which TCP interprets as loss (three duplicate acknowledgements trigger a spurious fast retransmit, see §3.9 (p. 192)).

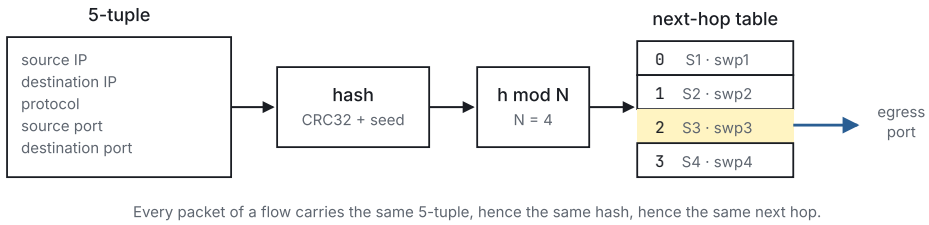


Figure 5.4 The ECMP decision. The hash is computed in hardware from header fields; the remainder indexes a small table of equal-cost next hops (here index 2, spine S3).

WORKED EXAMPLE

Six flows, four spines. Use a deliberately weak toy hash, h = (source port + destination port) mod 4, with next hops S1 to S4 numbered 0 to 3 and 10 Gb/s per uplink. All flows go to port 443 (443 mod 4 = 3).

1. The flows are listed with their rates: A (source port 40001, 1 Gb/s), B (40002, 8), C (40003, 1), D (40004, 1), E (40006, 9), F (40009, 2).
2. Source ports mod 4 are 1, 2, 3, 0, 2, 1. Adding 3 and reducing mod 4 gives hashes 0, 1, 2, 3, 1, 0.
3. Load per spine: S1 (index 0) carries A + F = 3 Gb/s; S2 (index 1) carries B + E = 17 Gb/s; S3 carries C = 1 Gb/s; S4 carries D = 1 Gb/s.
4. Total demand is 22 Gb/s on 40 Gb/s of uplink, yet S2 is asked for 17 Gb/s on a 10 Gb/s link: two large flows (“elephants”) collided, and the other flows (“mice”) sharing that link suffer queueing and loss.
5. Now S3 fails. With a naive “hash mod N”, N changes from 4 to 3, and a flow keeps its index only if h mod 4 = h mod 3, which for uniformly distributed h happens for 3 of the 12 residues; about 75% of all flows change path, including flows that were on healthy spines.
6. With *resilient* (consistent) hashing the hash selects one of a fixed number of buckets, say 16, and each bucket is assigned to a member. When S3 fails only its four buckets are reassigned, so 4 of 16 = 25% of flows move (Figure 5.5).

Result: balance depends on the flow mix, not only on the count of flows; a resilient table limits the disruption of a failure to the flows that had to move.

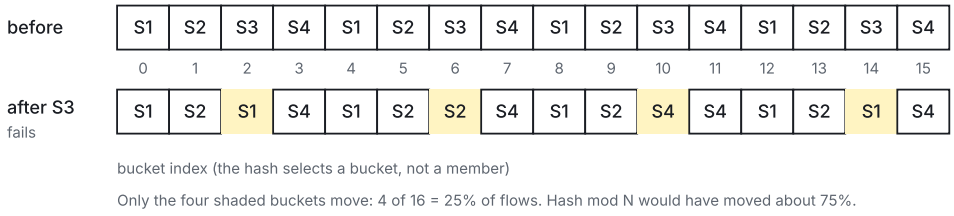


Figure 5.5 Resilient hashing. A fixed table of 16 buckets is filled with the members; when S3 fails, only its own buckets are reassigned, so flows on healthy spines are not reshuffled.

ECMP balances flows, so it is exactly as good as the flow population is uniform. Real traffic is not: a few elephant flows (backups, shuffles) carry most bytes. Operators therefore add refinements. *Weighted* ECMP gives members unequal shares, for example when one spine has fewer links after a failure. *Flowlet switching* splits a flow at idle gaps longer than the path-delay difference and rehashes each burst, which balances better without reordering. *Packet spraying* sends packets of one flow over all paths and is used only where the receiver tolerates reordering, as in some RDMA fabrics. On Linux the sysctl `net.ipv4.fib_multipath_hash_policy` selects the

inputs: 0 hashes the addresses only, 1 the 5-tuple, and 2 the inner headers of encapsulated traffic. In BGP the number of installed paths is limited by `maximum-paths`.

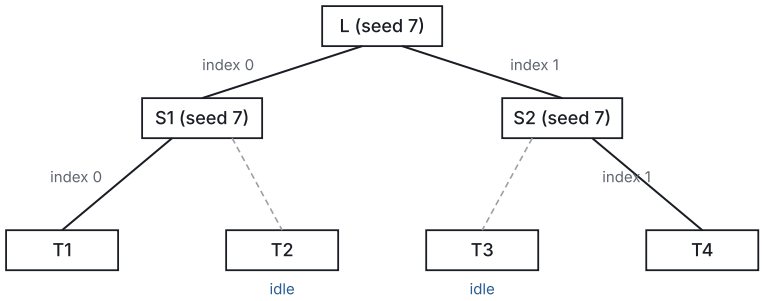


Figure 5.6 Hash polarisation. With the same hash function and seed at both tiers the second decision carries no new information; half of the second-tier links stay idle. A per-device seed breaks the correlation.

A subtle failure is *hash polarisation*. If every tier of the fabric uses the same hash function, the same fields and the same seed, then the decision at the second tier repeats the decision at the first. In Figure 5.6 every flow that reached spine S1 chose its first uplink because h mod 2 = 0; S1 computes the same h and again chooses index 0, so its second link is never used. Polarisation appears as a few links at 100% and their neighbours idle, and it is cured by giving each device its own seed, or by hashing different fields at different tiers.

LAB

`LAB 5.3 · 60 MIN` The lab fabric of §5.4. On a host behind L1 start eight parallel streams to a host behind L4, and count packets on the two spine-facing ports of L1.

```
iperf3 -c 10.1.4.10 -P 8 -t 20
tcpdump -ni swp1 -c 2000 tcp | wc -l      # repeat on swp2
sysctl -w net.ipv4.fib_multipath_hash_policy=0
```

```
swp1: 1237 packets   swp2: 763 packets      (policy 1, 5-tuple)
swp1: 2000 packets   swp2: 0 packets        (policy 0, addresses only)
```

What you should notice: with the 5-tuple the eight streams (eight source ports) split unevenly, perhaps five and three; this is ECMP working, not failing. With policy 0 the hash sees only the address pair, which is the same for all eight, so every packet takes one uplink. (The counts shown are typical, not exact.)

TRAP

Claim: "ECMP load-balances packets." *Why it is wrong:* per-packet balancing reorders segments of one connection and triggers spurious retransmissions. *Correct statement:* ECMP balances flows; individual flows are limited to one path and the balance is statistical.

CHECK YOURSELF

1. Why does every packet of one connection take the same path under ECMP? [7]
2. What fraction of flows moves when one of four members fails under mod-N hashing, and under resilient hashing? [8]
3. Name the symptom of polarisation. [9]

5.4 The eBGP underlay, BFD and unnumbered links MUST · ■■□

BIG IDEA

In a large fabric each switch runs eBGP to its neighbours, because BGP offers simple per-link policy, bounded failure scope and no flooding, and fast failure detection is added with BFD.

The *underlay* is the routed network that carries every packet between the fabric's switches; the *overlay* (§5.5) is the virtual network carried on top of it. Pillar 4 taught OSPF and IS-IS (§4.5 (p. 246)), which would also work here, but RFC 7938 documents why the large operators chose BGP instead. A link-state protocol floods every change to every switch and every switch runs the shortest-path calculation, so thousands of devices produce heavy churn; BGP sends an update only to a neighbour, which limits the effect of a failure to the paths that used it. BGP has a rich policy model (communities, route maps) for draining a device for maintenance, it uses TCP and so needs no extra protocol, and one protocol covers both the fabric and the edge to the outside world. The cost is a more careful design: every switch needs an AS number, and the usual loop-prevention rule must work with it.

The standard plan gives all the spines one AS number and each leaf its own. Campus Net's fabric uses private numbers: spines in AS 65100, and leaves L1 to L4 in AS 65101 to 65104 (Campus Net's border routers remain in AS 65000, §4.7 (p. 256)). When L1 advertises its server subnet 10.1.1.0/24, spine S1 receives it with AS_PATH 65101 and passes it on to L4 with AS_PATH 65100 65101 (Figure 5.7). The shared spine AS has two useful effects. A route can only go *up* to a spine and *down* to a leaf: if L4 tried to advertise it back up to S2, S2 would see its own AS number 65100 in the path and discard it, which prevents "valleys" and the path-hunting that would follow a failure. The leaves, conversely, must have *different* AS numbers; two leaves sharing an AS would reject each other's routes for the same reason, and the fix would be `allowas-in` on the leaves, which weakens the loop protection.

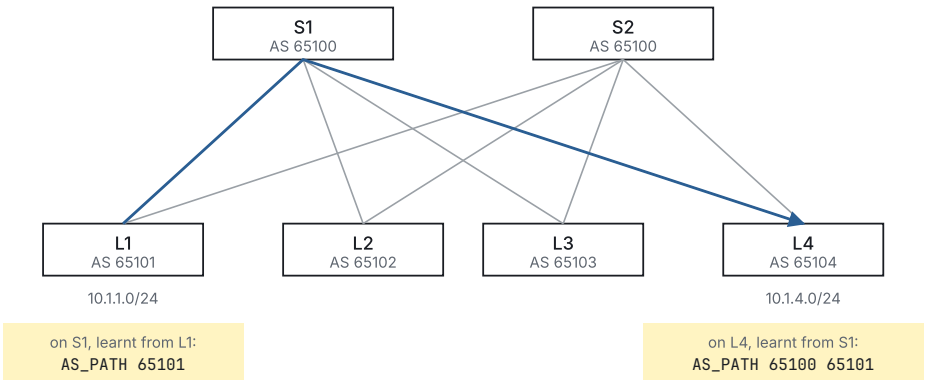


Figure 5.7 An eBGP fabric. Spines share one ASN and each leaf has its own. The route to L1's subnet reaches L4 with the path 65100 65101; S2 would refuse any path that already contains 65100, which keeps routes from bouncing back up.

Two details make ECMP work. By default BGP installs several paths only if their AS_PATH attributes are identical in the relevant parts; with the plan above the two paths from L4 to L1 are both "65100 65101", so `maximum-paths 8` is enough. If the spines had different AS numbers the paths would differ in content though not in length, and the command `bestpath as-path mult-`

1. Traffic between servers in the same data centre; for example request fan-out to internal services and storage replication.↩
2. Half, because each rack has two uplinks and the protocol must block one of them to keep the topology a tree.↩
3. Three: the source leaf, one spine and the destination leaf.↩
4. 240 ÷ 160 = 1.5 : 1.↩
5. 32, one spine port per leaf.↩
6. Oversubscription is a chosen ratio at one tier; blocking is a topological property, and a Clos fabric with enough spine capacity is non-blocking.↩
7. The 5-tuple, and therefore the hash, is the same for every packet of the connection.↩
8. About 75% under mod-N and 25% under resilient hashing.↩
9. Some links saturated while parallel links stay idle, because later tiers repeat the earlier choice.↩

`ipath-relax` is needed. Secondly, private AS numbers are scarce: the 2-byte private range 64512 to 65534 holds only 1,023 numbers, which a large fabric exhausts, so big operators use the 4-byte private range 4200000000 to 4294967294.

Convergence is the other half of the design. A BGP hold timer is 180 s by default, so a neighbour that fails silently would be believed for three minutes. *Bidirectional Forwarding Detection* (BFD, RFC 5880) is a lightweight hello protocol, run in hardware, that tells BGP when a path is dead. With a transmit interval of 300 ms and a multiplier of 3 a failure is detected after 3 × 300 ms = 900 ms.

Mechanism	Detection time	Notes
Interface goes down	milliseconds	only if the physical signal is lost
BFD, 300 ms × 3	0.9 s	independent of the physical layer
BGP keepalive and hold, tuned to 3 s / 9 s	up to 9 s	common data-centre setting
BGP default hold timer	up to 180 s	the default; far too slow

The last refinement removes IP addresses from the links. In *BGP unnumbered* the two ends of a link do not carry IPv4 addresses; the session runs over the IPv6 link-local addresses, which every IPv6 interface has automatically, and IPv4 routes are carried with an IPv6 next hop (RFC 5549, now RFC 8950). A fabric of 64 leaves and 8 spines has 512 links; numbering each with a /31 would consume 1,024 addresses and make 512 subnets to track, whereas unnumbered links need only one loopback address per switch. In FRR the whole neighbour configuration is two lines.

```
router bgp 65101
 bgp router-id 10.0.0.1
 neighbor swp1 interface remote-as external
 neighbor swp2 interface remote-as external
 neighbor swp1 bfd
 neighbor swp2 bfd
 address-family ipv4 unicast
  redistribute connected
  maximum-paths 8
```

LAB

`LAB 5.4 · 3 H` Containerlab with FRR images: two spines (AS 65100) and four leaves (AS 65101 to 65104), loopbacks 10.0.0.1 to 10.0.0.4 on the leaves, one server subnet per leaf. Configure the block above on every leaf and the mirror image on the spines, then verify.

```
leaf4# show bgp summary
leaf4# show ip route 10.1.1.0/24
```

```
Neighbor   V   AS     State/PfxRcd
swp1(S1)   4   65100  6
swp2(S2)   4   65100  6

B>* 10.1.1.0/24 [20/0] via fe80::a8bb:ccff:fe00:100, swp1, weight 1
  *                    via fe80::a8bb:ccff:fe00:200, swp2, weight 1
```

What you should notice: each leaf has exactly two sessions, and the route to another leaf’s subnet has two next hops (ECMP). Shut one spine-facing port and the route falls to one next

hop within about a second when BFD is on; remove the `bfd` lines and the failure of a link whose carrier stays up takes the full hold time to notice.

TRAP

Claim: "Two leaves in the same AS will simply peer through the spines." *Why it is wrong:* the receiving leaf sees its own AS in the AS_PATH and discards the update as a loop. *Correct statement:* give each leaf (or each rack pair) its own AS number, or enable `allowas-in` knowingly and accept the weaker protection.

INTERVIEW LENS

GOOGLE · MICROSOFT · AMAZON · ARISTA · JUNIPER

"Why eBGP and not OSPF in the data centre?" wants three reasons from the text above (smaller failure scope, policy and drain control, one protocol to the edge), the ASN plan and the reason for the shared spine AS, and BFD for sub-second failover. Backbone roles at Amazon and Microsoft also ask about IS-IS and MPLS, taught in §4.10 (p. 271).

CHECK YOURSELF

1. Why do spines share one AS number? [1]
2. How long does BFD with 300 ms and a multiplier of 3 take to detect a failure? [2]
3. What does BGP unnumbered use in place of an IPv4 address on the link? [3]

5.5 Overlays: VXLAN MUST · ■■□

BIG IDEA

VXLAN carries a tenant's Ethernet frame inside a UDP packet between two switches, so tenants obtain layer-2 networks over a routed fabric, with 16 million segment identifiers in place of 4,094 VLANs, at the price of 50 bytes of header per packet.

A virtual machine keeps its IP address when it moves, which is easy only if the new host is in the same layer-2 segment; yet the fabric of §5.4 is routed, and routers do not extend layer-2 segments. The solution is an *overlay*: the endpoints of the virtual network wrap each tenant frame in an ordinary IP packet, the routed *underlay* delivers it, and the far endpoint unwraps it. *VXLAN* (Virtual eXtensible LAN, RFC 7348) is the dominant overlay. The device that wraps and unwraps is the *VTEP* (VXLAN tunnel endpoint); it may be a leaf switch or the virtual switch of a hypervisor, and it has an IP address, usually a loopback, that appears as the outer address. A 24-bit *VXLAN network identifier* (VNI) names the tenant segment, giving 2^{24} = 16,777,216 segments. The outer packet is UDP with destination port 4789.

The encapsulation is shown in Figure 5.8. The VXLAN header itself is eight bytes: a flags byte whose "I" bit (0x08) says that the VNI is valid, three reserved bytes, the three-byte VNI, and one reserved byte. The *source* UDP port is not fixed: the VTEP computes it as a hash of the inner Ethernet, IP and transport headers, within the ephemeral range. This is deliberate. The underlay's ECMP hash (§5.3) sees only the outer 5-tuple, and all packets between two VTEPs would otherwise share one tuple and one path; with the inner flow hashed into the source port, different tenant flows take different spines, and each flow stays in order.

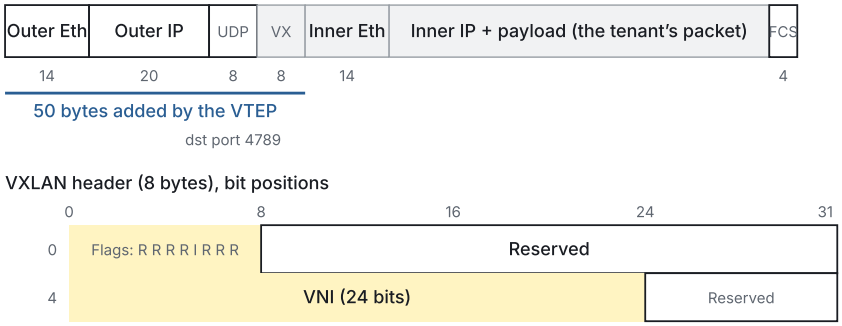


Figure 5.8 VXLAN encapsulation. Byte counts are under each block. The I flag (value 0x08) says the VNI is valid; the 24-bit VNI allows 16,777,216 segments.

WORKED EXAMPLE

Overhead and MTU for a 1,500-byte tenant packet. Campus Net's SALES servers (VLAN 10, 10.1.10.0/24) are given VNI 10010 and appear on two racks; a packet from Host A on L1 to Host B on L2 crosses the fabric (Figure 5.9).

1. The tenant frame is the inner Ethernet header (14 bytes) plus a 1,500-byte IP packet: 1,514 bytes.
2. The VTEP adds a VXLAN header (8), an outer UDP header (8), an outer IPv4 header (20) and an outer Ethernet header (14): 50 bytes in all.
3. The outer IP packet is 20 + 8 + 8 + 1,514 = 1,550 bytes, so the underlay IP MTU must be at least 1,550. On the wire the frame is 1,564 bytes (1,568 with the FCS).
4. If the outer header is IPv6 (40 bytes instead of 20), the overhead is 70 bytes and the minimum underlay MTU is 1,570.
5. Fabrics therefore set the underlay MTU to 9,216 (jumbo frames) and leave the tenant MTU at 1,500 or 9,000. If the underlay stays at 1,500, the usable tenant MTU is 1,500 − 50 = 1,450 for the IP packet, which Linux sets automatically on a VXLAN interface.

Result: 50 bytes of encapsulation (70 over IPv6); underlay IP MTU at least 1,550; jumbo frames avoid the problem.

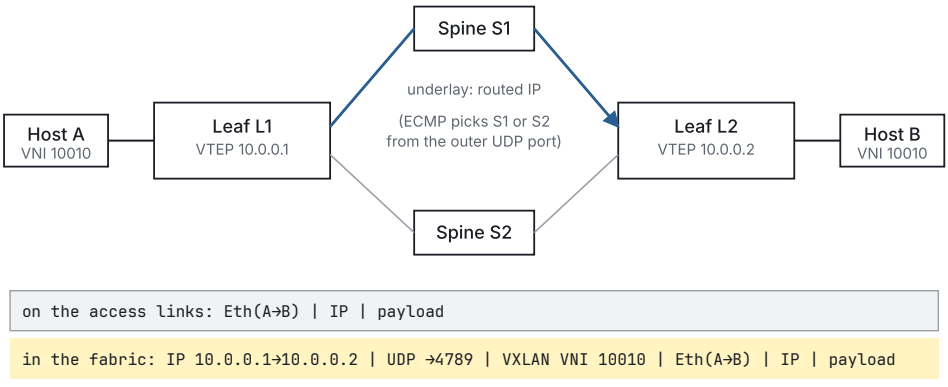


Figure 5.9 A frame from Host A to Host B across the overlay. L1 wraps the tenant frame in an outer IP and UDP packet addressed to L2's loopback; the spines forward it as ordinary IP and never see the inner Ethernet header.

A VTEP must learn where each tenant MAC lives. The original design copies a switch: a VTEP that receives an encapsulated frame learns "inner source MAC X is behind outer source IP Y",

and sends frames for unknown destinations, broadcasts and multicast (together termed *BUM* traffic) to every other VTEP of that VNI. The BUM copies are delivered either by an IP multicast group in the underlay (efficient but requires PIM, §4.12 (p. 277)) or by *ingress replication*, where the source VTEP unicasts one copy to each peer. Flood-and-learn works but does not scale, and §5.6 replaces it with a BGP control plane.

Two other encapsulations appear in interviews. *Geneve* (RFC 8926) uses UDP port 6081 and a variable-length option field, which suits services that need to carry metadata; AWS Gateway Load Balancer (§5.9 (p. 325)) uses it. *NVGRE* (RFC 7637) puts the tenant frame in GRE, which has no UDP port to hash, so it offers poor ECMP entropy. Neither has displaced VXLAN in switch hardware. The comparison table is in the Revision Spread.

LAB

`LAB 5.5 · 90 MIN` Two Linux hosts (or network namespaces) joined by a link with MTU 1500, carrying a point-to-point VXLAN tunnel on VNI 10010.

```
ip link add vxlan10 type vxlan id 10010 dstport 4789 local 10.0.0.1 remote
10.0.0.2
ip addr add 10.1.10.1/24 dev vxlan10 && ip link set vxlan10 up
ip -d link show vxlan10 | grep mtu
ping -M do -s 1472 10.1.10.2
ping -M do -s 1422 10.1.10.2
```

```
vxlan10: ... mtu 1450 ...
ping: local error: message too long, mtu=1450
64 bytes from 10.1.10.2: icmp_seq=1 ttl=64
```

What you should notice: a 1,472-byte payload makes a 1,500-byte packet (1,472 + 28), which the interface refuses; 1,422 makes 1,450 and passes. Raise the underlay to 9000 (`ip link set eth0 mtu 9000`) and the VXLAN interface to 8950, and the large ping succeeds. With the default 1,500 and a tenant that believes its MTU is 1,500, small packets (ping, the TCP handshake) work while large ones vanish: the classic symptom is “ping works, HTTPS hangs”.

TRAP

Claim: “VXLAN extends a VLAN across the data centre.” *Why it is wrong:* the VNI is not a VLAN tag, and the tenant frame travels over a routed network; a VLAN on a leaf port is merely mapped to a VNI (Campus Net maps VLAN 10 to VNI 10010). Further, the inner and outer headers together exceed the link MTU unless it is raised. *Correct statement:* VXLAN carries layer-2 frames in UDP over layer 3, and the underlay MTU must be at least the tenant MTU plus 50.

CHECK YOURSELF

1. How many bytes does VXLAN add over an IPv4 underlay, and which UDP port does it use? [4]
2. Why is the outer UDP source port a hash of the inner headers? [5]
3. What is BUM traffic and how is it delivered without multicast? [6]

5.6 EVPN: a control plane for the overlay SHOULD · ■■■

BIG IDEA

> EVPN lets the same BGP that routes the underlay also advertise which MAC and IP addresses live behind which VTEP, so switches no longer flood to discover hosts and a host move is a routing update.

Flood-and-learn makes every VTEP discover hosts by seeing traffic. *BGP EVPN* (RFC 7432, with RFC 8365 for its use over VXLAN) replaces this with advertisements. It is a BGP address family (AFI 25, SAFI 70, see §4.7 (p. 256)) whose routes are of several *types*, each distinguished by a number. Each leaf learns local MAC and IP addresses from its attached hosts and advertises them with its VTEP address as the next hop; every other leaf installs them. Reachability of all hosts is thus known before a packet is sent. The same sessions as the underlay can carry it, or the spines can act as route reflectors.

Type	Name	Purpose	Key fields	You see it when
1	Ethernet auto-discovery	multihoming: fast withdrawal, aliasing	ESI, Ethernet tag	a server is attached to two leaves
2	MAC/IP advertisement	where a host MAC (and IP) lives	RD, ESI, MAC, IP, L2VNI (Label1), L3VNI (Label2)	a host appears or moves
3	Inclusive multicast	VTEP joins a VNI; builds the BUM list	VTEP address, VNI	a VNI is configured on a leaf
4	Ethernet segment	elects the designated forwarder per segment	ESI, VTEP address	multihoming
5	IP prefix	a whole subnet or external route	RD, prefix and length, gateway, L3VNI	a border leaf injects 0.0.0.0/0, or a subnet is advertised

Types 2 and 5 carry most of the load. A Type-2 route says “this MAC, and optionally this IP, is behind me”; a Type-5 route says “this prefix is behind me” and is how a border leaf announces Campus Net’s default route into a tenant. Types 1 and 4 support a server attached to two leaves (an *Ethernet segment identifier*, ESI) without a proprietary MLAG, and Type 3 builds the list of VTEPs to which BUM traffic is replicated. Routes also carry *route targets* (RT), extended communities that decide which VRF or VNI imports them. Note that with a different AS number on each leaf, automatically derived RTs would not match, so the fabric sets explicit RTs such as 65100:10010.

WORKED EXAMPLE

A host moves from L1 to L3. Host A (MAC aa:aa:aa:aa:aa:11, IP 10.1.10.11, VNI 10010, L3VNI 50001 in VRF CAMPUS) is a virtual machine that migrates.

1. Before the move L1 advertises a Type-2 route of the form shown below (without the mobility community); every leaf installs “MAC aa:aa:aa:aa:aa:11 is at VTEP 10.0.0.1”.
2. After migration the VM sends a gratuitous ARP from its new host. L3 learns the MAC locally and finds that it already holds a *remote* route for it with no sequence number (treated as 0).
3. L3 advertises the same MAC, as shown below, with the *MAC-mobility* extended community and sequence number 1. The rule of RFC 7432 is that the highest sequence number wins.

4. Every leaf, including L1, prefers L3's route. L1 sees that its own local route has been superseded, deletes it and withdraws its advertisement (Figure 5.10).
5. Traffic from L5 reaches the VM through VTEP 10.0.0.3 as soon as the update is installed, typically within milliseconds, rather than after a flood or an aging timer.

```
Route type 2 (MAC/IP advertisement)
  RD            10.0.0.3:10010        ESI    0 (single-homed)
  MAC           aa:aa:aa:aa:aa:11     IP     10.1.10.11
  Label1        10010 (L2VNI)         Label2 50001 (L3VNI)
  Ext. comms    RT 65100:10010  RT 65100:50001
                MAC-mobility seq 1  Router-MAC 00:aa:bb:00:00:03
```

Result: the move is a single BGP update with a higher sequence number, and the old location is withdrawn.

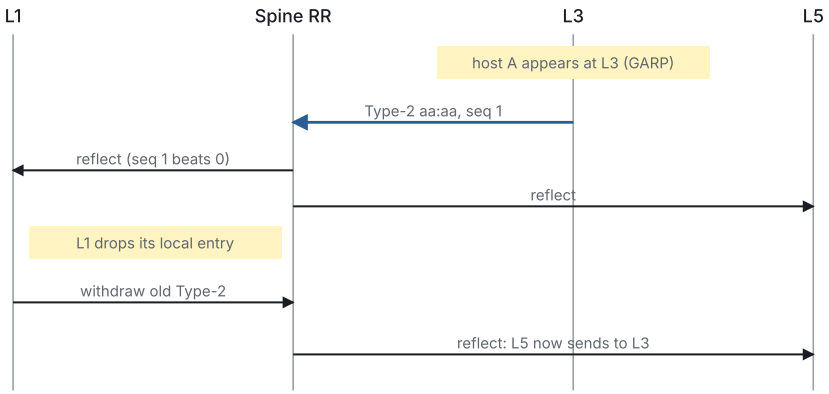


Figure 5.10 A host moves from L1 to L3. The new advertisement carries a higher MAC-mobility sequence number, so every leaf prefers it at once; flood-and-learn would have waited for traffic or for aging.

Three further mechanisms complete the design. *ARP suppression*: a leaf snoops the ARP requests of its local hosts and, because Type-2 routes already tell it the MAC of every known IP address, answers the request itself instead of flooding it across the fabric; only an unknown address is flooded. An *anycast gateway* gives every leaf the same gateway IP address (10.1.10.1) and the same virtual MAC, so a host always finds its default gateway on its own leaf and keeps it after a move; this is the fabric analogue of the first-hop redundancy of §4.8 (p. 265), with no active/standby pair. Finally, *integrated routing and bridging* (IRB, RFC 9135) decides how traffic crosses subnets. In the *symmetric* model, A on VLAN 10 at L1 sends to C on VLAN 20 at L3 like this: A forwards to the anycast gateway on L1; L1 routes the packet in VRF CAMPUS, looks up 10.1.20.31 and finds VTEP L3 with L3's *router MAC*; it encapsulates with the L3VNI 50001 and the inner destination MAC set to L3's router MAC; L3 decapsulates, looks the VRF up by L3VNI, routes into VLAN 20 and bridges to C. Both leaves route (Figure 5.11), and a leaf needs only the VNIs of its own hosts plus the L3VNI. In the *asymmetric* model the ingress leaf routes and then bridges directly in the destination's VNI, so every leaf must hold every VNI even where it has no hosts, which does not scale.

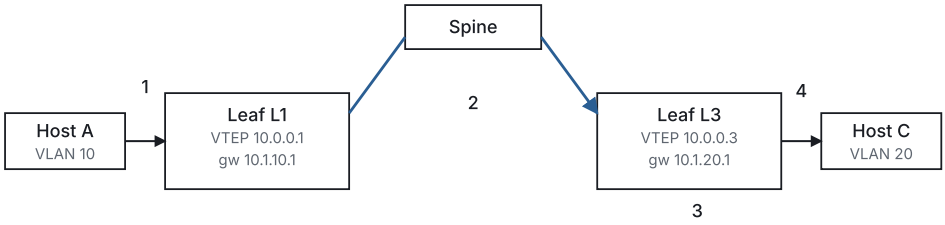


Figure 5.11 Symmetric IRB. Both leaves route, so the packet travels in the tenant's L3VNI and neither leaf needs the other's VLAN to exist locally.

LAB

`LAB 5.6 · 3 H` Containerlab with FRR: the fabric of §5.4 with VNI 10010 on L1 and L3, and EVPN enabled on the same eBGP sessions (`address-family l2vpn evpn`, `advertise-all-vni`). Start a ping from a host on L1 to a host on L3 and read the routes.

```
leaf3# show bgp l2vpn evpn route type macip
leaf3# show evpn mac vni 10010
```

```
*> [2]:[0]:[48]:[aa:aa:aa:aa:aa:11]:[32]:[10.1.10.11]
   10.0.0.1  ... ET:8 RT:65100:10010
```

Now move the host: detach its interface from L1 and attach it to L3, then repeat the command on L5.

```
   10.0.0.3  ... ET:8 RT:65100:10010 MM:1
```

What you should notice: the next hop changes from 10.0.0.1 to 10.0.0.3 and the *MM:1* (MAC-mobility sequence 1) appears; the old route disappears once L1 withdraws it. With ARP suppression on, `tcpdump` on a spine shows no ARP request crossing the fabric.

TRAP

Claim: "Type 5 and Type 2 with an IP address are interchangeable." *Why it is wrong:* a Type-2 route describes one host (a /32 or /128) learned from ARP or ND, and moves with the host; a Type-5 route describes a prefix learned from routing and carries no MAC. *Correct statement:* use Type 2 for hosts, Type 5 for subnets and external routes; importing the same RT into two tenants by accident also leaks their VRFs into each other.

INTERVIEW LENS

ARISTA · JUNIPER · NOKIA · CISCO · MICROSOFT

Arista and Juniper interviews ask for the route types, with one example of each, and for the difference between symmetric and asymmetric IRB. A good answer begins with the motivation (no flooding, mobility, multihoming), draws the Type-2 fields from memory, and names the anycast gateway and ARP suppression as the two features that remove flooding for user traffic. A common scenario is "a VM moves and traffic blackholes for thirty seconds": the answer is the MAC-mobility sequence number, plus a gratuitous ARP from the VM.

CHECK YOURSELF

1. Which EVPN route type carries the L2VNI and L3VNI of a host? [7]
2. What makes the new location of a moved MAC win? [8]
3. What does ARP suppression avoid? [9]

5.7 SDN, OpenFlow and P4 SHOULD · ■■□

BIG IDEA

Software-defined networking moves the decisions out of each switch into a logically central controller and leaves the switch with a fast table lookup; P4 goes one step further and lets the operator define what the table and the packet parser look like.

In a traditional device the *control plane* (routing protocols that decide) and the *data plane* (hardware that forwards) share a box, and each device decides alone. *Software-defined networking* (SDN) separates them: a *controller* holds the network-wide view, computes the forwarding rules and installs them in switches through a *southbound* interface; applications ask the controller for services through a *northbound* interface, often REST or gRPC (Figure 5.12). Two motivations drive it. A central view allows global optimisation that a distributed protocol cannot do, and a programmable interface lets software, not command-line scripts, manage thousands of devices. Google's B4 wide-area network, whose controller computes traffic-engineering paths, is the standard example, and Google's data-centre controller does the same for the Jupiter fabric.

OpenFlow is the protocol that first standardised the southbound interface. A switch holds one or more *flow tables*. Each *flow entry* has a *match* (fields from the packet headers and the ingress port), a *priority*, *counters*, *instructions* (actions such as output to a port, drop, rewrite a field, or go to the next table) and *timeouts*. The switch compares a packet with the entries in priority order and applies the first match. A *table miss*, when no entry matches, is itself handled by a low-priority entry; commonly it sends the packet to the controller as a *packet-in*, and the controller reacts by installing a flow. Installing flows only on demand is *reactive*; installing them in advance is *proactive*. A reactive design stalls when the controller is unreachable, whereas a proactive one keeps forwarding.

WORKED EXAMPLE

Three OpenFlow rules on Open vSwitch. In Mininet, switch `s1` has host h1 on port 1 and host h2 (10.0.0.2) on port 2.

```
ovs-ofctl add-flow s1 "priority=100,ip,nw_dst=10.0.0.2,actions=output:2"
ovs-ofctl add-flow s1 "priority=50,arp,actions=FLOOD"
ovs-ofctl add-flow s1 "priority=0,actions=CONTROLLER:65535"
ovs-ofctl dump-flows s1
```

```
priority=100,ip,nw_dst=10.0.0.2 actions=output:2        n_packets=5
priority=50,arp actions=FLOOD                           n_packets=2
priority=0 actions=CONTROLLER:65535                     n_packets=1
```

1. The first ARP from h1 matches priority 50 and is flooded (2 packets).
2. The echo requests to 10.0.0.2 match priority 100 and go to port 2; the counters rise with each ping (5 here).
3. Anything else, such as a packet to an unknown address, falls to priority 0 and is sent to the controller (1 packet).

1. A route may then only travel up and down, because a spine rejects any path that already contains its own AS; this prevents valley paths. ↩
2. 900 ms. ↩
3. The IPv6 link-local address, with IPv4 prefixes advertised over an IPv6 next hop. ↩
4. 50 bytes (outer Ethernet 14, IP 20, UDP 8, VXLAN 8); destination port 4789. ↩
5. It gives the underlay's ECMP hash entropy, so tenant flows spread over all paths. ↩
6. Broadcast, unknown-unicast and multicast frames; the source VTEP sends a separate unicast copy to each peer (ingress replication). ↩
7. Type 2 (MAC/IP advertisement), in its Label1 and Label2 fields. ↩
8. A higher MAC-mobility sequence number. ↩
9. Flooding ARP requests across the fabric; the leaf answers from the bindings learnt through Type-2 routes. ↩

> **Result: matching is by priority, counters show which rule fires, and the priority-0 entry is the table-miss rule.**

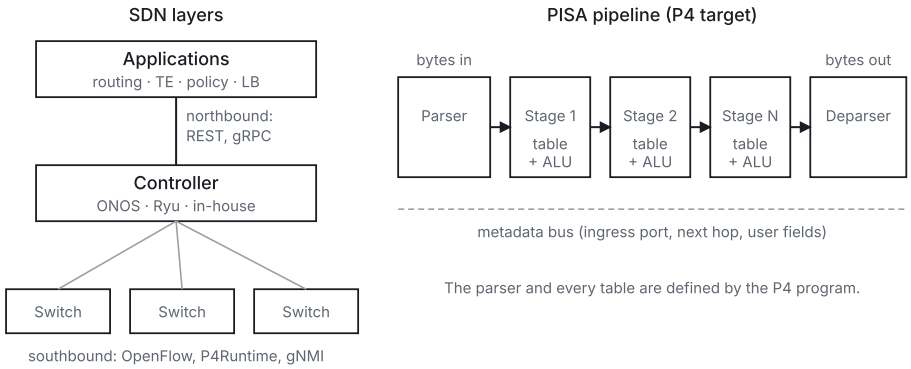


Figure 5.12 Left: the SDN split between applications, a logically central controller and simple switches. Right: the protocol-independent switch architecture that P4 targets, in which the parser, the tables and the deparser are all defined by the program.

OpenFlow fixes the set of headers that a switch can match. *P4* (Programming Protocol-independent Packet Processors) removes that limit. A P4 program describes a *parser* (a state machine that extracts headers from the bytes), a sequence of *match-action tables* and a *deparser* that reassembles the packet; the hardware, a *protocol-independent switch architecture* (PISA) chip, executes it at line rate. The program therefore defines new protocols, for instance a custom telemetry header, without waiting for a silicon revision. The control plane then fills the tables through P4Runtime. The fragment below forwards IPv4 by longest prefix, which is the standard first exercise.

```
parser MyParser(packet_in pkt, out headers hdr, inout metadata meta,
                inout standard_metadata_t std) {
    state start {
        pkt.extract(hdr.ethernet);
        transition select(hdr.ethernet.etherType) {
            0x0800: parse_ipv4;
            default: accept;
        }
    }
    state parse_ipv4 { pkt.extract(hdr.ipv4); transition accept; }
}
control MyIngress(inout headers hdr, inout metadata meta,
                  inout standard_metadata_t std) {
    action drop() { mark_to_drop(std); }
    action forward(bit<48> dst, bit<9> port) {
        std.egress_spec = port;
        hdr.ethernet.srcAddr = hdr.ethernet.dstAddr;
        hdr.ethernet.dstAddr = dst;
        hdr.ipv4.ttl = hdr.ipv4.ttl - 1;
    }
    table ipv4_lpm {
        key = { hdr.ipv4.dstAddr: lpm; }
        actions = { forward; drop; }
        default_action = drop();
    }
    apply { if (hdr.ipv4.isValid()) { ipv4_lpm.apply(); } }
}
```

The `lpm` key kind asks the hardware to do a longest-prefix match, which §5.8 shows to be a ternary or algorithmic lookup. The action `forward` is what a router does after a lookup: rewrite the MAC addresses, decrement the TTL (§3.5 (p. 179)) and pick the port.

> **TRAP**
>
> *Claim:* "SDN means the end of distributed routing protocols." *Why it is wrong:* most production data centres still run BGP for the underlay (§5.4); a controller usually manages policy, traffic engineering or the virtual-network overlay on top. *Correct statement:* SDN centralises selected decisions, and a controller outage need not stop forwarding if the flows were installed proactively.

> **CHECK YOURSELF**
>
> 1. What happens on an OpenFlow table miss? [1]
> 2. What can P4 do that OpenFlow cannot? [2]
> 3. Name the two interface directions of an SDN controller. [3]

5.8 Router and switch internals: RIB, FIB, TCAM and the ASIC MUST · ■■■

> **BIG IDEA**
>
> A router is two machines in one box: a slow, general-purpose computer that decides routes, and a fast, fixed-function chip that forwards packets by looking up a compact copy of those decisions, so the two are joined by a download path and an exception path.

Pillar 4 introduced the *management*, *control* and *data* planes at the command line (§4.1 (p. 230)), and the routing table and administrative distance (§4.4 (p. 243)). This section opens the box and is the canonical place for what happens inside. The *control plane* runs on the *route processor*, an ordinary CPU with DRAM and an operating system: the BGP, OSPF and IS-IS processes exchange messages with neighbours and build tables of routes. The *data plane* runs on one or more *forwarding ASICs* (application-specific integrated circuits) that examine every packet in a fixed, small number of clock cycles. The CPU handles thousands of packets per second; the ASIC handles billions. The structure is drawn in Figure 5.13.

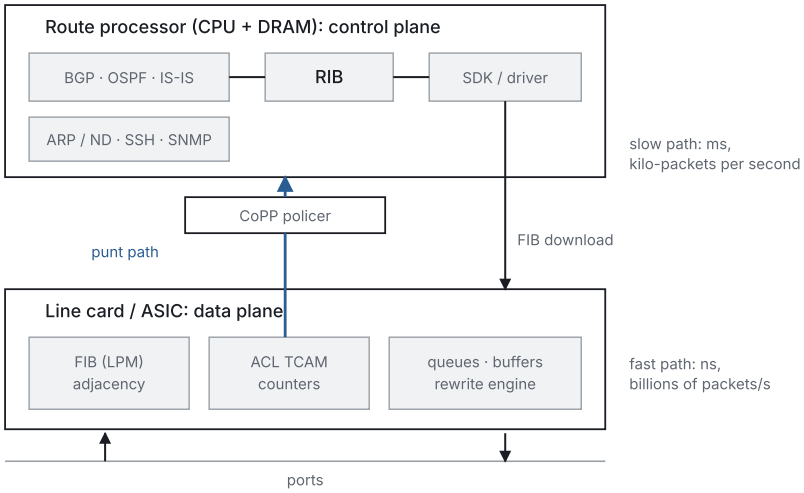


Figure 5.13 Control plane above, data plane below. Routes flow down as FIB entries; packets that the hardware cannot handle (the accent path) flow up, through a policer that protects the CPU.

RIB, FIB and adjacency

The *routing information base* (RIB) is the control plane's table. It holds every route from every source, and when several sources offer the same prefix, the lowest administrative distance chooses the winner. The *forwarding information base* (FIB) is the data plane's table, a compressed copy that contains only the winning routes, in a form the hardware can search quickly. Cisco calls the mechanism *CEF* (Cisco Express Forwarding); it keeps two structures. The FIB maps a destination prefix to a next hop. The *adjacency table* maps that next hop to the egress interface and to the complete layer-2 header to prepend (the *rewrite string*), so that forwarding is one lookup and one copy of precomputed bytes, with no ARP request on the fast path. If the next hop's MAC address is not yet known, the adjacency is a *glean* entry: the first packet is punted to the CPU, which sends the ARP request (§3.6 (p. 183)) and completes the entry.

WORKED EXAMPLE

From RIB to FIB. BR1 learns Campus Net's data-centre aggregate 10.1.0.0/16 twice: from OSPF (AD 110, redistributed by the border leaf) and from eBGP (AD 20, §4.7 (p. 256)), next hop 172.16.255.2.

1. The RIB keeps both candidates. The route-selection rule prefers the lower distance, so the eBGP route is *installed* in the routing table and the OSPF route waits as a backup.
2. The FIB is built from installed routes only: one entry, 10.1.0.0/16 via 172.16.255.2.
3. The next hop 172.16.255.2 is reached over a connected route on Gi0/0/1. *Recursive resolution* (when a route's next hop is itself reached through another route) is flattened in CEF, so the FIB entry points directly to the adjacency for 172.16.255.2 on Gi0/0/1, which holds the neighbour's MAC address and the rewrite string.
4. If the eBGP route is withdrawn, the OSPF route is promoted in the RIB, the FIB entry is replaced, and the hardware table is updated by the driver.

```
BR1# show ip route 10.1.0.0 255.255.0.0
  Known via "bgp 65000", distance 20, metric 0
    * 172.16.255.2, from 172.16.255.2
BR1# show ip cef 10.1.0.0/16 detail
10.1.0.0/16
  nexthop 172.16.255.2 GigabitEthernet0/0/1
```

Result: the RIB holds two routes, the FIB one; Linux shows the same split with `ip route show table all` (the routes) and the kernel's FIB.

Ternary memory and longest-prefix match

Looking up an address in a table of 1,000,000 prefixes by software would take dozens of memory reads (§1.9 (p. 41) gives the trie algorithms). A *ternary content-addressable memory* (TCAM) does it in one cycle by comparing the key with every stored entry at once. Each entry stores a *value* and a *mask*, and a bit position can be 0, 1 or "don't care" (x), which is why the memory is ternary. A prefix such as 10.1.0.0/16 is stored as its 16 significant bits followed by 16 don't-cares. The match lines of all rows go to a *priority encoder*, which returns the index of the lowest matching row, and that index addresses an ordinary SRAM holding the result (the next hop). Since the lowest index wins, entries are stored in order of decreasing prefix length, which makes the first match the longest.

WORKED EXAMPLE

Longest-prefix match in a TCAM. The table has four entries: 10.0.0.0/8 to A, 10.1.0.0/16 to B, 10.1.2.0/24 to C and 0.0.0.0/0 to D. Look up 10.1.2.7.

1. Sort by prefix length, longest first: index 0 is the /24 (C), 1 the /16 (B), 2 the /8 (A), 3 the /0 (D).
2. The /24 is stored as 00001010.00000001.00000010.xxxxxxxx; the key is 00001010.00000001.00000010.00000111.
3. All four rows match, because each prefix covers the key (Figure 5.14). The priority encoder returns index 0, and the SRAM returns next hop C.
4. Cost: a 144-bit-wide entry, 512 K entries deep, is 144 × 524,288 = 75,497,472 bits, about 72 Mbit. A TCAM bit needs roughly twice the storage (value and mask) and considerably more area than an SRAM bit, and *every row* switches on every lookup, so power rises in proportion to depth. This explains why TCAM is the scarcest resource on a switch, and why large tables use *algorithmic* LPM in SRAM and keep TCAM for ACLs.

Result: next hop C (the /24), found in one cycle with the lowest-index rule.

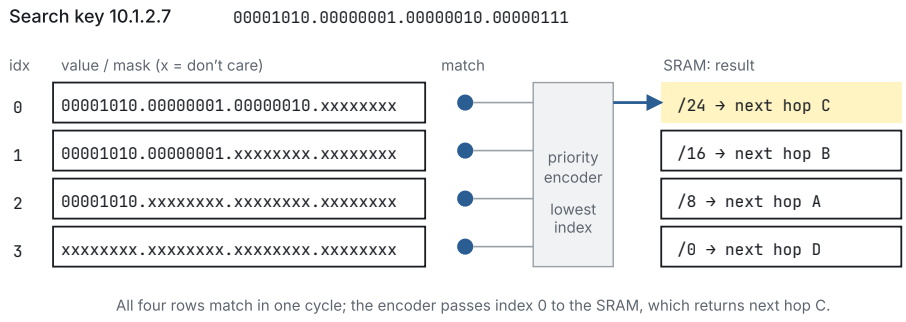


Figure 5.14 A ternary lookup. Every row compares itself with the key in parallel; rows are stored longest prefix first, so the lowest matching index is the longest match.

TCAM serves the *access control lists* of §9.5 (p. 549) for the same reason: an ACL line matches ranges of header bits in priority order. Ranges are awkward, because a ternary entry can only express a power-of-two-aligned block. The port range 1024 to 65535 decomposes into six blocks (1024 to 2047, 2048 to 4095, 4096 to 8191, 8192 to 16383, 16384 to 32767 and 32768 to 65535), so a single line "permit tcp any any range 1024 65535" consumes six entries; a range on both source and destination multiplies the counts. An exact-match table, such as the MAC table, needs no ternary memory and uses a hash in SRAM.

The ASIC pipeline

Modern chips are *pipelines*: a packet passes through a fixed series of stages, each doing one job, and many packets are in flight at once (Figure 5.15). A walk of one packet follows.

1. *Ingress port.* The MAC and physical coding sublayer check the frame and the FCS.
2. *Parser.* It extracts the Ethernet, VLAN, IP and TCP or UDP fields into a *header vector*.
3. *Ingress ACL* (TCAM). Security and QoS classification, and the decision to punt.
4. *L2 lookup* (exact-match hash on VLAN and destination MAC). For a routed packet the destination MAC is the router's own, which sends the packet to the L3 stage.
5. *L3 lookup* (LPM in TCAM or algorithmic SRAM). The result is not a single next hop but an ECMP group.
6. *ECMP resolution and adjacency.* The hash of §5.3 selects a member; the adjacency supplies the new MAC header; the TTL is decremented and the IP checksum updated.
7. *Traffic manager.* The packet is queued, in a *virtual output queue* (VOQ) for its egress port and class, in the buffer; a scheduler (strict priority, weighted round robin or deficit weighted round robin) decides when it leaves; ECN marking happens here.
8. *Egress pipeline.* An egress ACL, VXLAN or MPLS encapsulation if needed, then the deparser rebuilds the headers and the egress MAC sends the frame.

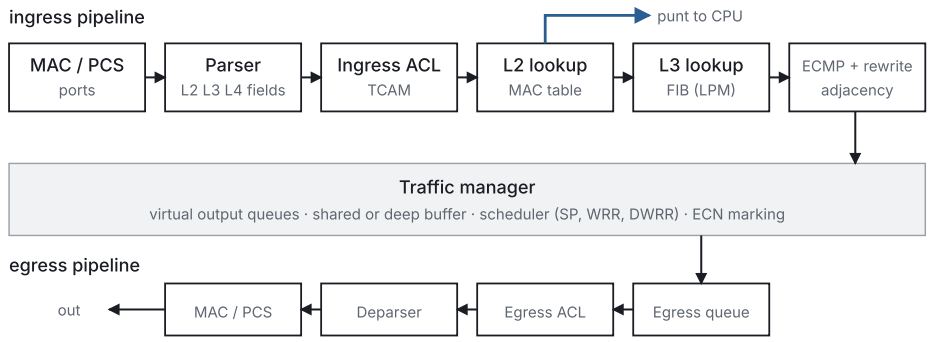


Figure 5.15 A switch ASIC from ingress to egress. Each box is a hardware stage that handles a packet in a fixed number of clock cycles; the traffic manager in the middle is the only place where packets wait.

Two properties of this design matter in practice. *Cut-through* switching starts to transmit a frame as soon as the header has been read, before the rest has arrived, instead of the *store-and-forward* method that waits for the whole frame. For a 1,500-byte frame the saving is the serialisation time: 12,000 bits at 10 Gb/s take 1.2 µs, at 100 Gb/s 0.12 µs. Cut-through needs the egress port to be idle and at least as fast as the ingress port; if the egress is busy or slower (10G in, 100G out would starve), the switch falls back to store-and-forward, and a frame with a bad FCS is already on its way when the error is found. Secondly, the *buffer* absorbs bursts. Shallow shared buffers (tens of megabytes for a whole chip) suit a leaf; deep buffers (gigabytes, with VOQs) suit a router at the edge of a long path. The failure mode of shallow buffers is *incast*: many servers answer one request at the same instant, say 40 senders each returning 64 KiB to one client on a 10 Gb/s port. The burst is 40 × 64 KiB = 2.5 MiB, arriving far faster than the port drains, so a buffer of 1 MiB per port overflows, the tail packets are dropped and the senders wait for the TCP retransmission timer, typically 200 ms, which is a hundred times longer than the transfer itself. Cures are ECN marking with a DCTCP-style sender (§3.9 (p. 192)), a smaller minimum retransmission timeout, deeper buffers, or application-level staggering of the replies.

Punts, CoPP and surviving a restart

Packets that the ASIC cannot or should not forward are *punted*, that is, handed to the route processor: packets addressed to the router itself (BGP, OSPF, SSH), packets whose TTL expires, packets that need an ARP or ND resolution (glean), packets with IP options, and ACL entries marked “log”. Since the CPU is orders of magnitude slower than the ASIC, an unprotected punt path is a denial-of-service target. *Control-plane policing* (CoPP) places a rate limiter on each class of punted traffic, in hardware, before it reaches the CPU.

```
class-map match-any ROUTING
 match access-group name ROUTING-PROTOCOLS
policy-map COPP
 class ROUTING
  police rate 2000 pps conform-action transmit exceed-action drop
 class class-default
  police rate 500 pps conform-action transmit exceed-action drop
control-plane
 service-policy input COPP
```

The control plane may restart (software upgrade or crash) while the data plane keeps working. *Non-stop forwarding* (NSF) with *graceful restart* (GR) does this by cooperation: the ASIC keeps forwarding with the stale FIB, and the neighbours, acting as helpers, keep the restarting router’s routes for an agreed restart time instead of withdrawing them (RFC 4724 for BGP). *Non-stop routing* (NSR) needs no help from neighbours: a standby route processor mirrors the protocol state, including the TCP sessions, and takes over invisibly.

Chips come from two sources. *Merchant silicon* is sold to any vendor and powers most data-centre switches; *custom silicon* is designed by the vendor for its own routers. The classification below is generic and approximate.

Family	Examples	Buffer	Programmability	Typical role
Merchant, shallow shared buffer	Broadcom Trident and Tomahawk, Nvidia Spectrum, Marvell	tens of MB per chip	fixed pipeline, some flexible tables	leaf and spine switches
Merchant, deep buffer	Broadcom Jericho	gigabytes with VOQ	fixed pipeline, large FIB	WAN and aggregation routers
Merchant, programmable	Cisco Silicon One, Tofino (P4)	varies	programmable pipeline	fabric and edge, telemetry
Custom	Juniper Trio and Express, Cisco QuantumFlow	deep	programmable microcode	service-provider routers

> LAB
>
> `LAB 5.8 · 2 H` Implement the binary trie of §1.9 (p. 41) and test it against a brute-force longest-prefix match, which is the specification the trie must agree with.
>
> ```
> for _ in range(100000):
>     ip = random.getrandbits(32)
>     want = max((p for p in prefixes if matches(p, ip)), key=lambda p: p.len,
> default=None)
>     assert trie.lookup(ip) == (want.nexthop if want else None)
> ```

What you should notice: with 10,000 random prefixes the two agree on every address (including the default-route and nested-prefix cases you add on purpose), and the trie answers in at most 32 steps while the scan reads every entry. Time both, then replace the trie's 1-bit stride by 8 bits and count the memory accesses per lookup.

TRAP

Claim: "The TCAM stores the routing table." *Why it is wrong:* it stores the FIB (the winning routes), usually together with algorithmic SRAM for the bulk of a large table, and in the same TCAM the ACL entries compete for space, with a range line taking several entries. *Correct statement:* the RIB lives in the CPU's DRAM; the FIB, ACLs and adjacency live in the ASIC's memories.

INTERVIEW LENS

BROADCOM · ARISTA · JUNIPER · CISCO · NOKIA

Platform and SDK roles probe this section deeply. Expect "RIB versus FIB", "what is TCAM and why is it expensive?", "walk a packet through the ASIC" and, as a coding question, "design a longest-prefix-match structure for one million routes" (the trie and DIR-24-8 answers are in §1.9 (p. 41) and Q5.22). A strong answer names each stage, says where the ECMP hash sits, and explains punt and CoPP unprompted.

CHECK YOURSELF

1. Which table does the ASIC consult, the RIB or the FIB? [4]
2. Why are TCAM entries stored longest prefix first? [5]
3. How many ternary entries does the range 1024 to 65535 need? [6]

5.9 Load balancers MUST · ■■□

BIG IDEA

A load balancer presents one virtual address and spreads connections over many servers; a layer-4 balancer decides from the 5-tuple and is fast, a layer-7 balancer reads the request and is flexible, and hyperscalers build the first kind from ECMP and consistent hashing in software.

A *load balancer* (LB) answers on a *virtual IP address* (VIP), chooses a *backend* server for each new connection and keeps all packets of that connection going to the same backend. A *layer-4* balancer works on addresses and ports: it never reads the payload and sees no URL. A *layer-7* balancer terminates the TCP connection (and usually TLS, §9.4 (p. 546)), parses the HTTP request and opens a second connection to the backend, so it can route by host name or path, rewrite headers, cache and retry. The difference in cost is large, because layer 7 must hold two connections and decrypt each request.

	Layer 4	Layer 7
Decides on	5-tuple	URL, host, headers, cookies
Connection	passes packets (NAT, DSR, tunnel)	terminates and re-opens
TLS	passthrough	terminated here
Speed	millions of packets per second per node	much lower; CPU bound

1. A low-priority table-miss entry decides; commonly the packet goes to the controller as a packet-in.↩
2. Define the parser and the headers themselves, so new protocols are supported without new silicon.↩
3. Northbound toward applications (REST, gRPC) and southbound toward switches (OpenFlow, P4Runtime, gNMI).↩
4. The FIB (with the adjacency table); the RIB lives on the route processor.↩
5. The priority encoder returns the lowest matching index, so the longest prefix then wins.↩
6. Six.↩

AWS	Network Load Balancer (NLB), Gateway Load Balancer (GWLB, Geneve to appliances)	Application Load Balancer (ALB)
Azure	Load Balancer	Application Gateway (regional), Front Door (global)

There are three ways for a layer-4 balancer to forward (Figure 5.16). In *NAT* (proxy) mode the balancer rewrites the destination to the backend and the reply returns through it, which also allows it to rewrite the source (SNAT). In *direct server return* (DSR) the balancer rewrites only the destination MAC (same layer-2 segment) or wraps the packet in a tunnel (IP-in-IP or GRE, any distance), and the backend replies *directly* to the client with the VIP as its source address. For the reply to be accepted by the client, the backend must own the VIP, and it is therefore configured on its loopback interface, with ARP for it suppressed (`arp_ignore=1`, `arp_announce=2` on Linux) so that the backend does not compete with the balancer for the VIP. DSR suits web traffic because responses are far larger than requests: the balancer sees only the small inbound half.

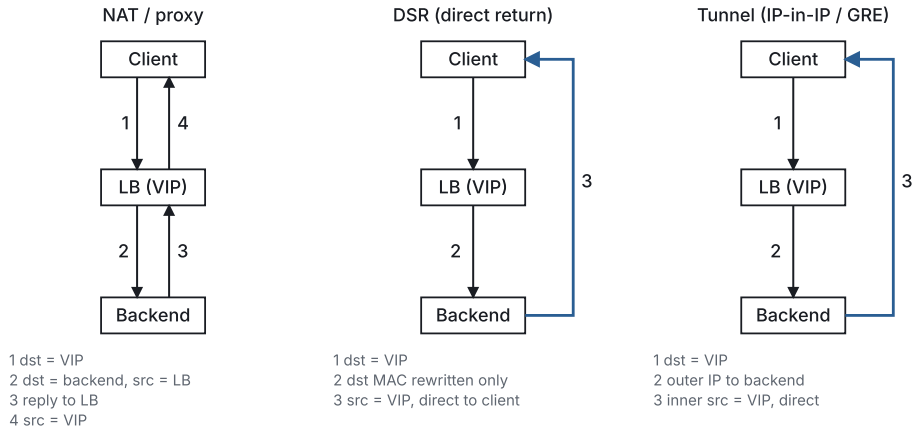


Figure 5.16 Three ways to forward to a backend. In the first the balancer sits on both directions; in the other two the reply bypasses it, so it handles only the (usually much smaller) request traffic.

The balancer must also choose a backend. The common algorithms are in the table; *consistent hashing* is the one that matters at scale, because it keeps most connections on the same backend when the pool changes.

Algorithm	Rule	Weakness
Round robin	next backend in turn	ignores load and request size
Weighted round robin	turns in proportion to weight	static weights
Least connections	fewest open connections	needs shared state
Power of two choices	pick two at random, take the less loaded	none serious; common default
Source or 5-tuple hash	hash of the flow	pool change reshuffles unless consistent
Consistent hash (Maglev)	lookup table built from per-backend permutations	table must be rebuilt on change

WORKED EXAMPLE

A Maglev lookup table. Google's software balancer, Maglev (NSDI 2016), builds a table of M entries, with M prime and much larger than the number of backends; a connection is sent to `table[hash(5-tuple) mod M]`. Take M = 7 and three backends B0, B1 and B2. Each

backend hashes its name into an *offset* and a *skip*, which defines a permutation of the table positions, `(offset + j × skip) mod 7`. The values are B0: offset 3, skip 4; B1: offset 0, skip 2; B2: offset 3, skip 1.

1. Permutations: B0 = 3, 0, 4, 1, 5, 2, 6; B1 = 0, 2, 4, 6, 1, 3, 5; B2 = 3, 4, 5, 6, 0, 1, 2.
2. The backends take turns, each claiming its most preferred position that is still free. Round 1: B0 takes 3, B1 takes 0, B2 wants 3 (taken) and takes 4. Round 2: B0 takes 1 (0 and 4 taken), B1 takes 2, B2 takes 5. Round 3: B0 takes 6.
3. The table is [B1, B0, B1, B0, B2, B2, B0] for positions 0 to 6: B0 has 3 entries and B1, B2 have 2 each.
4. Remove B1 and refill with B0 and B2 only. Round 1: B0 takes 3, B2 takes 4. Round 2: B0 takes 0, B2 takes 5. Round 3: B0 takes 1, B2 takes 6. Round 4: B0 takes 2. The table becomes [B0, B0, B0, B0, B2, B2, B2].
5. Compare the tables: positions 0, 2 and 6 changed, 3 of 7. Two of the changes are unavoidable (B1's own entries); the third is a side effect. A plain `hash mod N`, going from 3 backends to 2, would keep a flow only when h mod 6 is 0 or 1, so about 67% of connections move.

Result: the new table differs in 3 of 7 entries, near the minimum of 2; a hash mod N would have moved about two thirds of the connections.

In production M is large (65,537 in the paper) so that each backend gets nearly the same number of entries, and a *connection-tracking table* on each balancer pins existing connections, so the lookup table matters only for new ones. Maglev machines are stateless peers: the edge routers spread packets over them with ECMP (§5.3), any one of them can fail, and because they all build the same table from the same backend list, a connection that lands on a different Maglev machine after a failure still reaches the same backend.

Health checks decide which backends are in the pool. The balancer probes each backend (a TCP connect, or an HTTP request for a given URL) at an interval and applies hysteresis: a backend moves from Up to Down after, say, three consecutive failures, and from Down back to Up only after two consecutive successes, which prevents flapping. *Session persistence* (sticky sessions) maps one client to one backend by cookie or source address, which is needed only when the application keeps per-user state in memory. A last pitfall concerns NAT mode and outbound connections: a balancer or NAT gateway has one public address and so about 64,000 source ports per destination address and port; many short connections to the same destination exhaust them, and new connections fail. This is *SNAT port exhaustion*, met again in §5.12.

LAB

`LAB 5.9 · 2 H` Linux IPVS in direct-routing mode, with a balancer and two backends in network namespaces on one bridge (VIP 10.1.10.100, backends 10.1.10.21 and .22).

```
ipvsadm -A -t 10.1.10.100:80 -s rr
ipvsadm -a -t 10.1.10.100:80 -r 10.1.10.21 -g
ipvsadm -a -t 10.1.10.100:80 -r 10.1.10.22 -g
ip addr add 10.1.10.100/32 dev lo          # on each backend
sysctl -w net.ipv4.conf.all.arp_ignore=1 net.ipv4.conf.all.arp_announce=2
for i in 1 2 3 4; do curl -s 10.1.10.100/id; done
ipvsadm -E -t 10.1.10.100:80 -s sh
```

```
backend1 backend2 backend1 backend2        (round robin)
backend1 backend1 backend1 backend1        (source hashing)
```

What you should notice: replies carry the VIP as source and never pass through the balancer (`tcpdump` on it shows only requests). Remove the `arp_ignore` setting and the backends answer ARP for the VIP, so requests sometimes skip the balancer altogether.

TRAP

Claim: "An L4 load balancer can route /api and /images to different pools." *Why it is wrong:* it never reads the HTTP request, and with TLS passthrough it could not even decrypt it. *Correct statement:* path-based routing needs a layer-7 balancer (ALB, Application Gateway); a layer-4 balancer routes by address and port.

CHECK YOURSELF

1. Why must a DSR backend carry the VIP on its loopback? [1]
2. What advantage does DSR give the balancer? [2]
3. Why is the Maglev table size a prime much larger than the number of backends? [3]

5.10 Anycast and content delivery networks SHOULD · ■■□

BIG IDEA

Announcing one prefix from many sites lets BGP deliver every user to the nearest site in routing terms, and a content delivery network adds caches at those sites so that most requests never travel to the origin.

In *anycast* the same IP address is announced from several places, and the Internet's routing sends each packet to whichever announcement is best from the sender's position (§4.7 (p. 256)). Nothing in the protocol is special; the routers simply see several paths to one prefix and choose by the usual BGP rules, usually the shortest AS_PATH, then the closest exit by IGP cost. The root name servers, public resolvers and CDNs all use it. It gives short paths, automatic failover (a site that stops announcing is removed from the table) and absorption of attacks, since flood traffic is divided among the sites by where it originates.

WORKED EXAMPLE

Anycast from three sites. The prefix 192.0.2.0/24 (origin AS 64496) is announced from Mumbai, Singapore and Frankfurt. A user in Chennai is behind ISP AS 64510 (Figure 5.17).

1. ISP 64510 learns three paths: AS_PATH 64496 from Mumbai, 64520 64496 from Singapore, 64530 64496 from Frankfurt.
2. BGP prefers the shortest AS_PATH, so Chennai traffic goes to Mumbai.
3. Mumbai is taken down for maintenance and withdraws the prefix. The ISP removes that path, and the best remaining path (a tie in length, broken by IGP cost) takes the traffic to Singapore within the BGP convergence time.
4. A TCP connection that was open to Mumbai now sends its segments to Singapore, which has no state for it and answers with a reset. Anycast therefore suits short exchanges (DNS, short HTTP requests) and long connections only if routing is stable.

Result: the user lands on the BGP-nearest site, and failover is a routing withdrawal.

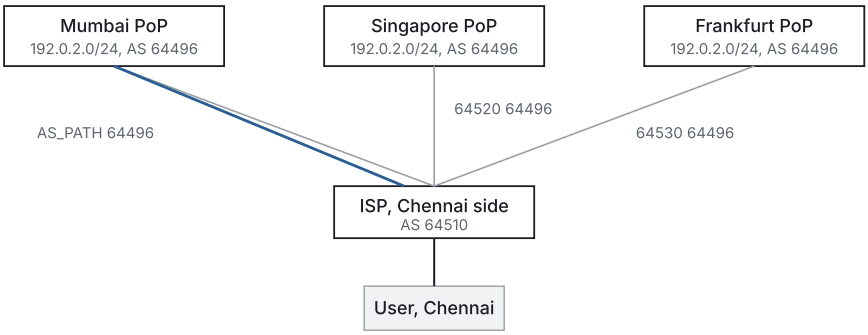


Figure 5.17 One prefix announced from three sites. The ISP's router picks one path by ordinary BGP rules, so each user lands on the site that is nearest in routing terms, not necessarily in kilometres.

A *CDN* places *edge* caches at such sites. A request first needs the CDN's address: the site's name is a CNAME to the CDN's domain, and the CDN's DNS server either returns an anycast address or, in *DNS-based global server load balancing*, returns the address of the edge it judges best for the client's resolver. DNS-based steering can use measured latency and load but sees only the *resolver's* address, so the extension *EDNS Client Subnet* passes a prefix of the client's address to the authoritative server. An edge that holds the object (a *hit*) replies at once; on a *miss* it fetches the object from a mid-tier cache or the origin, stores it according to the `Cache-Control` and TTL headers, and replies.

WORKED EXAMPLE

What a CDN saves. The origin is 220 ms away, the edge 15 ms away, and 90% of requests hit.

1. A hit costs the edge round trip: 15 ms.
2. A miss costs the edge plus the origin trip: 15 + 220 = 235 ms.
3. The expected time is 0.9 × 15 + 0.1 × 235 = 13.5 + 23.5 = 37 ms, against 235 ms with no cache.
4. Raising the hit ratio to 99% gives 0.99 × 15 + 0.01 × 235 = 17.2 ms. Each point of hit ratio is worth about 2.2 ms here.

Result: 37 ms expected, about six times faster than going to the origin every time.

	DNS-based steering	Anycast
Decision made by	the CDN's DNS server, from the resolver's address	BGP, from the user's network position
Failover speed	bounded by DNS TTL and client caching	BGP convergence (seconds)
Granularity	per resolver or client subnet; can use load and latency	coarse; by path length
Weakness	the resolver may be far from the client	route changes break long connections

LAB

`LAB 5.10 · 90 MIN` Containerlab: three "PoP" nodes (FRR) each announce 192.0.2.1/32 by eBGP to one transit router, with a client behind it. Give the PoPs different AS_PATH lengths by prepending.

```
client# traceroute -n 192.0.2.1
vtysh -c 'clear bgp *'   # after shutting the PoP that answered
```

```
 2  10.9.0.2     (transit)
 3  192.0.2.1    pop1
 3  192.0.2.1    pop2        (after pop1 is shut down)
```

What you should notice: all three PoPs hold the same address, and the traceroute shows the one with the shortest path; when it fails the answer moves after BGP converges, and a TCP connection opened before the failure is reset.

TRAP

Claim: "Anycast sends a user to the geographically nearest site." *Why it is wrong:* BGP knows paths, not kilometres; a user may be served from another continent when their ISP peers with the CDN there. *Correct statement:* anycast selects the BGP-nearest site, and operators influence this by choosing where to announce and by prepending.

CHECK YOURSELF

1. What does BGP use to choose between anycast sites? [4]
2. What happens to an open TCP connection when the route changes to another site? [5]
3. Compute the expected latency for a 95% hit ratio, edge 20 ms and origin 180 ms. [6]

5.11 AWS: the Virtual Private Cloud MUST · ■■□

BIG IDEA

A VPC is a private network that the provider simulates in software, scoped to one region, in which routing is explicit (route tables decide what is public) and filtering is done at two levels, a stateful security group at each interface and a stateless network ACL at each subnet.

The fabric of the preceding sections is invisible to a cloud customer; what the customer sees is a virtual network that behaves like the campus network of Pillar 4. In Amazon Web Services this is the *Virtual Private Cloud* (VPC). A VPC is created in one region with an address block between /16 and /28, for example 10.0.0.0/16. It is divided into *subnets*, and each subnet lives in exactly one *availability zone* (AZ), a physically separate data-centre group within the region, so that high availability means placing one subnet in each of two or more AZs. Within the VPC every subnet can reach every other through an implicit router; the *route table* attached to a subnet decides where all other traffic goes. A VPC has a ready-made route "local" for its own block and nothing else; every other route is added by the customer.

A subnet is *public* if its route table contains a default route (0.0.0.0/0) to an *internet gateway* (IGW), a horizontally scaled, redundant gateway that translates between private addresses and public ones. It is *private* if the default route instead points to a *NAT gateway*, which sits in a public subnet and lets instances start outbound connections while blocking unsolicited inbound ones (§3.6 (p. 183)), or if there is no default route at all. Being public also requires the instance to hold a public or Elastic IP address; the route alone is not enough. Note that AWS reserves five addresses in every subnet: the network address (.0), the VPC router (.1), the DNS resolver (.2), one held for future use (.3) and the broadcast address (the last), so a /24 offers 251 usable addresses, not 254.

WORKED EXAMPLE

A three-tier web application across two AZs. The VPC is 10.0.0.0/16 (Figure 5.18).

1. Subnets: public 10.0.1.0/24 (AZ a) and 10.0.2.0/24 (AZ b); private application 10.0.11.0/24 and 10.0.12.0/24; database 10.0.21.0/24 and 10.0.22.0/24. Each has 256 − 5 = 251 usable addresses.
2. Route tables: *public* has 10.0.0.0/16 local and 0.0.0.0/0 to the IGW. Each *application* table has 10.0.0.0/16 local and 0.0.0.0/0 to the NAT gateway in the same AZ (a gateway in a single AZ would be a shared failure point). The *database* table has only the local route, so the database cannot reach the Internet at all.
3. Security groups: ALB-SG allows TCP 443 from 0.0.0.0/0. App-SG allows TCP 8080 *from ALB-SG* (a rule that names a group, not an address range, so it follows instances as they scale). DB-SG allows TCP 5432 from App-SG. No other inbound rule exists.
4. Network ACL on the public subnets: inbound allow TCP 443 from 0.0.0.0/0; outbound allow TCP 1024 to 65535 to 0.0.0.0/0. The outbound rule is the *ephemeral-port* rule: the client's source port is chosen from a high range, and since a network ACL remembers nothing, the reply must be explicitly allowed. Omitting it makes an otherwise correct design hang.
5. Return traffic through the security groups needs no rule, because they are stateful.

Result: only the load balancer is exposed; each tier admits only the tier above it; the database has no route out.

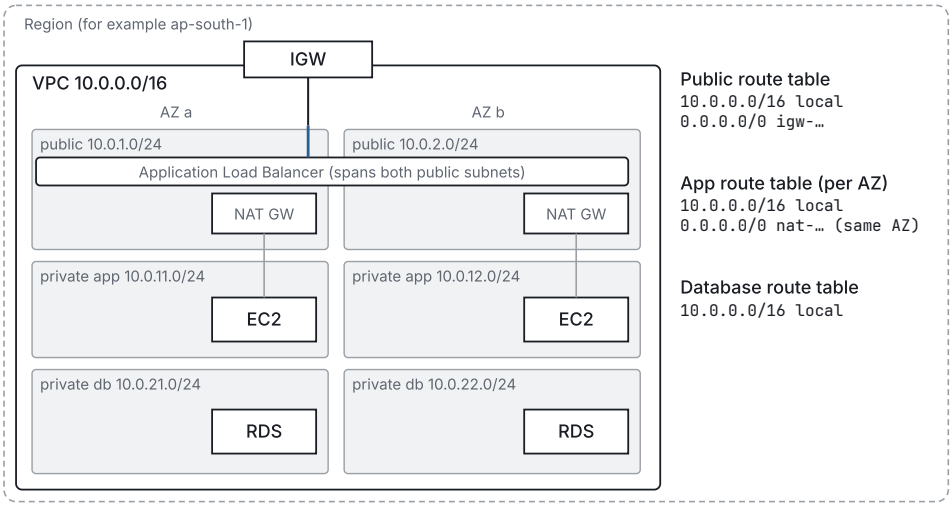


Figure 5.18 The three-tier VPC of the worked example. A subnet is public only because its route table points at the internet gateway; the private subnets reach the internet through the NAT gateway in their own AZ.

	Security group	Network ACL
Attached to	network interface (ENI)	subnet
State	stateful: replies allowed automatically	stateless: both directions need rules
Rule types	allow only	allow and deny
Evaluation	all rules together, no order	by rule number, lowest first, first match wins; final deny
Default	deny inbound, allow outbound	default ACL allows all; a new custom ACL denies all

Can reference	other security groups	address ranges only

Connecting VPCs is the next question. *VPC peering* joins two VPCs with a private link; it needs non-overlapping address blocks and a route in each VPC's tables. Peering is *not transitive*: if A peers with B and B with C, then A cannot reach C through B, and neither can A use B's internet gateway or VPN. Connecting n VPCs in a full mesh needs n(n − 1)/2 peerings: for 50 VPCs that is 1,225. A *transit gateway* (TGW) is a regional hub-and-spoke router: each VPC, VPN and Direct Connect (§5.13) is an *attachment*, and TGW route tables, with *association* and *propagation*, decide which attachment can reach which, so 50 VPCs need 50 attachments and segmentation is expressed in the route tables. *PrivateLink* exposes one *service* (behind a network load balancer) to another VPC through an *interface endpoint*, a network interface with a private address in the consumer's subnet. It is one-way and works even when the address blocks overlap. *Gateway endpoints* reach S3 and DynamoDB through a route-table entry rather than the Internet.

AWS builds these networks by host-based software: the Nitro cards of each server encapsulate packets in a VPC-aware tunnel, apply security groups and answer ARP requests from a mapping service, which is why a VPC has no broadcast and why the underlying switches stay a plain routed fabric (§5.5). *VPC Flow Logs* record accepted and rejected connections per interface and are the first tool for the diagnoses of Pillar 11 (p. 634).

LAB

`LAB 5.11 · 3 H` An AWS free-tier account, console or CLI. Create a VPC 10.0.0.0/16 with a public and a private subnet, an IGW, a bastion in the public subnet and an instance in the private subnet, and enable Flow Logs. Then add a network ACL to the public subnet that allows inbound TCP 22 but no outbound ephemeral ports.

```
aws ec2 create-vpc --cidr-block 10.0.0.0/16
aws ec2 create-route --route-table-id rtb-... --destination-cidr-block
0.0.0.0/0 --gateway-id igw-...
ssh -o ConnectTimeout=5 ec2-user@<bastion-public-ip>
```

What you should notice: with the ephemeral rule missing, SSH hangs at the connect stage while the security group looks correct; Flow Logs show an ACCEPT for the inbound packet and no reply, because the NACL silently drops the response. Adding outbound TCP 1024 to 65535 repairs it. **Cost:** a NAT gateway is billed hourly, so use a NAT instance or skip it, and delete everything afterwards.

TRAP

Claim: "VPC peering is transitive, and I can add a deny to a security group." *Why it is wrong:* peering joins exactly two VPCs, and security groups contain only allow rules. *Correct statement:* use a transit gateway for many VPCs, a network ACL for an explicit deny, and remember that peered VPCs cannot have overlapping blocks.

INTERVIEW LENS

AMAZON · MICROSOFT · GOOGLE

Amazon's network development engineer loops ask "security group versus network ACL" and the scenario "an instance in a private subnet cannot reach the Internet; list the checks in order" (route table, NAT gateway state and placement, the NAT's own route to the IGW, outbound rules, DNS, then the instance's own firewall). Start from the route and finish at the host.

CHECK YOURSELF

1. Why does a /24 subnet in AWS have 251 usable addresses? [7]

2. Which of the two filters needs an ephemeral-port rule, and why? [8]
3. How many peerings does a full mesh of 10 VPCs need? [9]

5.12 Azure: the virtual network SHOULD · ■■□

BIG IDEA

An Azure virtual network offers the same building blocks as a VPC with different edges: subnets span all zones, one filter (the network security group) is stateful and prioritised, and user-defined routes let the customer force traffic through a firewall.

A *virtual network* (VNet) in Azure is scoped to a region, and, unlike a VPC subnet, a subnet is **not** tied to an availability zone; the resources placed in it choose their zones. Azure reserves five addresses per subnet: the network address, the default gateway (.1), two for Azure DNS (.2 and .3) and the broadcast address. Filtering is done by a *network security group* (NSG), a list of rules that may be attached to a subnet, to a network interface, or to both. An NSG is stateful, so unlike an AWS network ACL it needs no return rule, and it has both allow and deny rules. Each rule has a *priority* from 100 to 4096, and the rules are evaluated from the lowest number upward until one matches, after which evaluation stops. The default rules, with priorities 65000 and above, allow traffic inside the VNet and from the load balancer and deny other inbound traffic. *Application security groups* name sets of interfaces so that rules can refer to roles, and *service tags* name Microsoft-managed address sets such as Storage or AzureCloud.

Routing begins with *system routes*: the VNet's own prefix, peered prefixes and a default route to the Internet. A *user-defined route* (UDR) in a route table overrides them, for instance to send 0.0.0.0/0 to a firewall; a *next hop type* of `VirtualAppliance` names the firewall's private address. When several routes share a prefix, a UDR wins over a BGP route, which wins over a system route; as always, the longest prefix is considered before any of this (§4.4 (p. 243)). Where a network virtual appliance (NVA) must learn the VNet's routes and announce its own, the *Azure Route Server* peers with it over BGP (the server uses AS 65515) and injects the learned routes into the VNet, so the UDRs need not be maintained by hand.

WORKED EXAMPLE

A hub-and-spoke network. The hub VNet is 10.100.0.0/16 with `AzureFirewallSubnet` 10.100.1.0/26 (the firewall takes 10.100.1.4) and `GatewaySubnet` 10.100.255.0/27. Two spokes are 10.1.0.0/16 and 10.2.0.0/16 (Figure 5.19).

1. Peering is not transitive, so each spoke is peered with the hub only. Spoke 1 therefore has system routes for 10.1.0.0/16 and 10.100.0.0/16 but none for 10.2.0.0/16.
2. A UDR on every spoke subnet sends 0.0.0.0/0 with next hop `VirtualAppliance` 10.100.1.4.
3. A packet from 10.1.0.5 to 10.2.0.7 matches no system route for 10.2.0.0/16, so it takes the default route, reaches the firewall, which checks its rules and forwards it over the hub-to-spoke-2 peering. Spoke 2 must accept it on the peering link, which needs *Allow forwarded traffic* because the packet did not originate in the hub.
4. The hub's gateway is shared with the spokes by two peering settings: *Allow gateway transit* on the hub side and *Use remote gateway* on each spoke side. Then Campus Net's on-premises network (§5.13) reaches the spokes through the one gateway.
5. For the return traffic the spoke 2 UDR forces the reply through the firewall as well, so the path is symmetric and the stateful firewall sees both directions.

1. The reply leaves with the VIP as its source, and the client only accepts replies from the address it contacted.↩
2. It carries only the (small) request half of the traffic.↩
3. A prime keeps every permutation complete, and a large table gives each backend almost the same share.↩
4. The ordinary best-path rules: usually shortest AS_PATH, then lowest IGP cost to the exit.↩
5. The new site has no state for it and resets it (unless the sites share connection state).↩
6. 0.95 × 20 + 0.05 × 200 = 19 + 10 = 29 ms.↩
7. Five addresses are reserved: network, router, DNS, future use and broadcast.↩
8. The network ACL, because it is stateless and the reply goes to a high source port.↩
9. 10 × 9 ÷ 2 = 45.↩

> **Result: all spoke-to-spoke and spoke-to-Internet traffic crosses the firewall, because of one default route per spoke.**

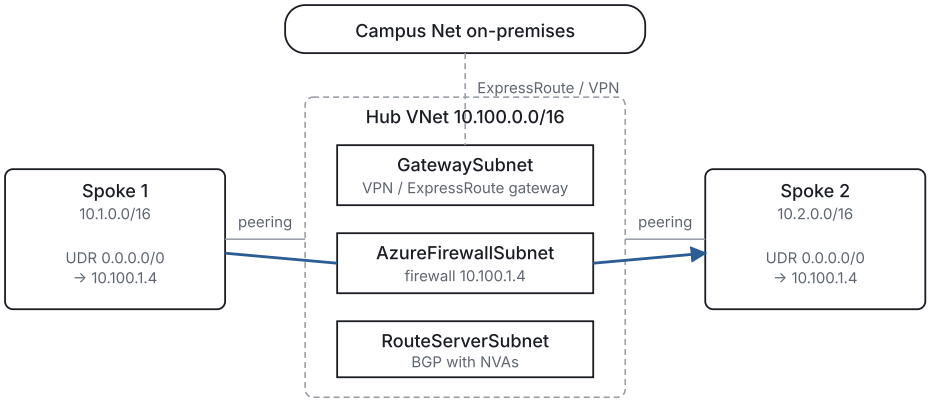


Figure 5.19 Hub and spoke. The hub holds the shared services; the spokes peer only with the hub, so spoke-to-spoke traffic (accent) is forced through the firewall by the default route in each spoke.

The differences from AWS are what interviews test, and the term-by-term mapping is given in the Revision Spread. The behavioural ones are these.

Question	AWS	Azure
Subnet scope	one AZ	whole region (all zones)
Stateless filter at the subnet	network ACL	none (an NSG is stateful)
Rule order	NACL by number; SG unordered	NSG by priority, lowest first
Default internet access	none until IGW and route	outbound by default for most resources (being retired in favour of explicit NAT)
Transitivity hub	transit gateway	Virtual WAN hub or hub VNet with firewall and UDRs
Fast NIC path	Nitro / ENA	Accelerated Networking (SR-IOV, FPGA SmartNIC)

Two cautions. *Accelerated Networking* lets the virtual machine's NIC bypass the host's virtual switch for the data path, so the policy that the vSwitch would apply is offloaded to hardware: the same idea as the ASIC pipeline of §5.8. And SNAT port exhaustion (§5.9) appears in Azure when many instances share the outbound address of a load balancer; the cures are more addresses, a NAT gateway, or connection reuse.

> LAB
>
> `LAB 5.12 · 2 H` An Azure free account and the Azure CLI. Create hub and spoke VNets, peer them, and put a Linux VM as a router (an NVA) in the hub, with a route table on the spokes.

```
az network vnet peering create -g lab -n hub-to-spoke1 --vnet-name hub \
   --remote-vnet spoke1 --allow-forwarded-traffic --allow-vnet-access
az network nic update -g lab -n nva-nic --ip-forwarding true
az network route-table route create -g lab --route-table-name spoke-rt -n de-
fault \
   --address-prefix 0.0.0.0/0 --next-hop-type VirtualAppliance --next-hop-ip-
address 10.100.1.4
sudo sysctl -w net.ipv4.ip_forward=1        # on the NVA
traceroute -n 10.2.0.4                      # from the spoke-1 VM
```

```
 1  10.100.1.4
 2  10.2.0.4
```

What you should notice: the first hop is the NVA, so spoke-to-spoke traffic is forced through the hub. If `--ip-forwarding` is missing on the NIC, Azure drops the forwarded packets even though the Linux kernel forwards them; if *Allow forwarded traffic* is missing, the second spoke drops them.

TRAP

Claim: "NSG rules are checked in the order they were created, like an access list." *Why it is wrong:* they are ordered by priority number, and the lowest number wins; the creation order is irrelevant. *Correct statement:* give specific rules low numbers (100 to 200) and broad rules high ones, leaving gaps for later insertion.

CHECK YOURSELF

1. Which Azure object plays the role of the AWS route table's "next hop to a firewall"? [1]
2. Why must a spoke be peered with the hub and not with other spokes in hub-and-spoke? [2]
3. What does the lowest priority number mean in an NSG? [3]

5.13 Hybrid connectivity: Direct Connect, ExpressRoute and VPN SHOULD · ■■□

BIG IDEA

A private circuit to the cloud is an ordinary BGP interconnection, so its availability and path preference are decided by the redundancy you buy and the attributes you set, and an IPsec VPN over the Internet is the cheaper path or the backup.

Campus Net's cloud networks must be reached from its own routers. There are two families of path. A *private circuit*, AWS Direct Connect (DX) or Azure ExpressRoute, is a layer-2 or layer-3 link rented through a carrier from the customer's site or a colocation facility to the provider's edge; it bypasses the public Internet, so it has predictable latency and no Internet exposure, and it is billed by port and traffic. An *IPsec VPN* (§9.7 (p. 555)) is an encrypted tunnel over the Internet, quick to set up and cheap but with variable latency and bandwidth. Routing on both is BGP (§4.7 (p. 256)), so everything in Pillar 4 about preference, prepending and communities applies.

ExpressRoute terminates on a pair of Microsoft routers (MSEE) in a peering location. A circuit has a *primary* and a *secondary* link, each a BGP session over its own /30. Two *peerings* are used: *private peering* reaches VNets with private addresses; *Microsoft peering* reaches Microsoft 365

gateway or a Direct Connect gateway; a *transit VIF* reaches a transit gateway; a *public VIF* reaches AWS public endpoints. A Site-to-Site VPN to AWS has two tunnels per connection, each good for up to 1.25 Gb/s, and is the usual backup.

WORKED EXAMPLE

ExpressRoute with two links and a VPN backup. Campus Net (AS 65010) advertises 172.16.0.0/16 on-premises and connects through CE1 and CE2 to the primary link (192.168.10.0/30) and the secondary link (192.168.10.4/30), Figure 5.20.

1. Both sessions are eBGP between AS 65010 and AS 12076. Azure learns 172.16.0.0/16 twice.
2. *Inbound preference* (Azure to campus) is decided by what Campus Net advertises. CE2 prepends its own AS twice on the secondary, giving AS_PATH 65010 65010 65010 against 65010 on the primary. Azure prefers the shorter path, so return traffic uses the primary and falls to the secondary on failure.
3. *Outbound preference* (campus to Azure) is decided by what Campus Net sets. Azure advertises 10.100.0.0/16 on both sessions with equal attributes; the campus routers set LOCAL_PREF 200 on routes learnt from CE1 and 100 on CE2, so outbound traffic also uses the primary.
4. The VPN backup is configured with the same prefix. If it is advertised with the same attributes, the best path may differ in the two directions and the traffic is asymmetric, which a stateful firewall in the path drops. Make the VPN less preferred in both directions: prepend towards Azure and set a lower LOCAL_PREF on the campus side.
5. Never advertise a *more specific* prefix over the VPN than over ExpressRoute. The longest prefix is chosen before any preference attribute is read (§4.4 (p. 243)), so a /24 on the VPN beats a /16 on ExpressRoute and moves the traffic onto the backup.

Result: primary preferred in both directions by prepending (inbound) and LOCAL_PREF (outbound); the VPN is a last resort with equal-length prefixes.

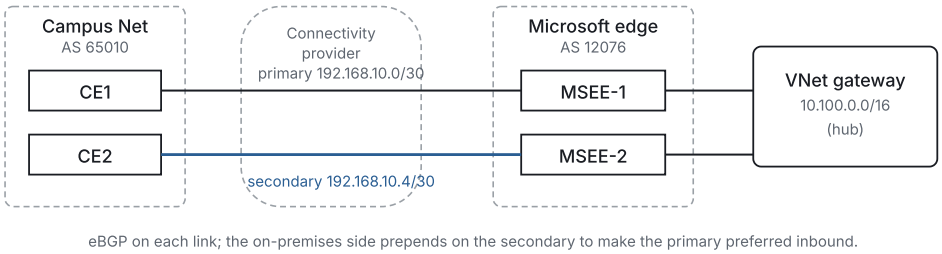


Figure 5.20 ExpressRoute private peering. Two BGP sessions, one per link, run between the customer edge and a redundant pair of Microsoft routers; the pair protects against a device failure, not against loss of the peering location.

BFD can be enabled on ExpressRoute's private peering so that a failure is detected in a second or so instead of at the hold timer. Limits matter in design: each peering accepts only a documented maximum number of advertised prefixes and tears down the session above it, so large enterprises summarise; check the provider's current limits rather than trusting numbers in a book.

Need	Choice
Quick start, low bandwidth, tolerates jitter	IPsec VPN

Predictable latency, high bandwidth, compliance	Direct Connect / ExpressRoute
Many branches, application-aware steering	SD-WAN, with VPN or private circuits as transport
Highest availability	two private circuits in different locations, plus a VPN

TRAP

Claim: "One ExpressRoute circuit is redundant, because it has two links." *Why it is wrong:* both links usually terminate in the same peering location, so a facility failure or a carrier fibre cut can remove both. *Correct statement:* a circuit protects against a device failure; for site-level redundancy buy a second circuit in another location. Advertising the same prefixes over a VPN and ExpressRoute without setting preferences is a second trap, because it gives asymmetric routing.

INTERVIEW LENS

MICROSOFT · AMAZON

Microsoft's cloud network engineer postings list VNets, ExpressRoute, Private Link, UDRs, VPN and NSGs, plus IS-IS, BGP and MPLS for the backbone. Typical scenarios: "design ExpressRoute with VPN failover and keep the routing symmetric" (the example above) and "on-premises advertises 10.0.0.0/8 and a VNet uses 10.1.0.0/16" (Q5.38). Amazon asks the Direct Connect equivalents and BGP troubleshooting.

CHECK YOURSELF

1. How do you make the primary link preferred for traffic from Azure to the campus? [4]
2. Why can a VPN backup advertising a /24 defeat an ExpressRoute that advertises a /16? [5]
3. How many tunnels does an AWS Site-to-Site VPN connection provide? [6]

5.14 The life of a request in the cloud MUST · ■□□

BIG IDEA

One HTTPS request from a phone to a virtual machine uses every mechanism of this chapter in turn, and each hop adds or removes a header, so the request is best learnt as a table of hops.

The student portal of Campus Net is hosted in a cloud region behind a CDN. A student in Chennai opens `https://portal.campus.example`. Figure 5.21 shows the header stack at each stage and the table lists the hops.

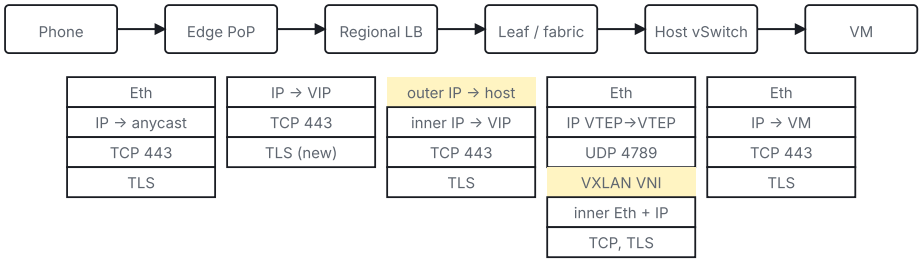


Figure 5.21 Header stacks along the path of one HTTPS request (shaded rows are encapsulation added inside the provider). The inner packet is untouched from the edge to the VM.

#	Hop and device	What happens	Lookup or decision	See
1	Phone and resolver	DNS query; the name is a CNAME to the CDN, the answer is an anycast address 192.0.2.10 with a short TTL	resolver cache; EDNS Client Subnet	§3.10 (p. 203)
2	Phone to ISP	TCP SYN to 192.0.2.10:443; the ISP may translate the source address	default route; NAT	§3.6 (p. 183)
3	ISP and transit routers	the packet follows BGP to the nearest announcement, the Mumbai site	longest prefix in the FIB	§5.10, §5.8
4	Edge PoP	the server's kernel completes the handshake; TLS ends here; layer-7 rules read the Host and path; cache miss	accept queue; cache key	§7.10 (p. 445), §9.4 (p. 546)
5	Edge to region	a new, pooled connection crosses the provider backbone; each router hashes the 5-tuple over equal paths	ECMP	§5.3
6	Regional edge router	the VIP is announced by the balancer tier; ECMP picks one balancer instance	ECMP hash	§5.9
7	Layer-4 balancer	consistent hash and connection table choose a backend; the packet is wrapped in an outer IP header addressed to that host	Maglev table	§5.9
8	Leaf, spine, leaf	the underlay carries the wrapped packet; the outer UDP source port spreads flows over spines	eBGP routes, ECMP	§5.4, §5.5
9	Host NIC or vSwitch	the outer header is removed, the virtual-network identifier selects the tenant, security-group rules are evaluated and a flow entry is created	exact match on a flow table	§5.11, §5.12
10	Virtual machine	the NIC driver hands the packet to the kernel, the TCP stack and the application's `accept`	socket table	§7.10 (p. 445)
11	Reply	the VM answers; the stateful flow entry admits the reply; with DSR it goes straight to the fabric, otherwise back through the balancer	flow table; routing	§5.9
12	Return to the phone	the edge re-encrypts, caches if allowed, and the response retraces the Internet path, which may be different	BGP, possibly asymmetric	§4.7 (p. 256)

When it breaks, the cause is nearly always on this list. Five failures recur, with the first check for each; the method for isolating them is the subject of Pillar 11 (p. 634).

Symptom	Likely cause	First check
Private instance cannot reach the Internet	no default route to a NAT gateway, or the NAT gateway sits in a subnet without an IGW route	route table of both subnets
Connection hangs after the handshake; ping works	MTU blackhole through an overlay or VPN (§5.5)	`ping -M do -s 1422`; lower the MSS
Ingress works, replies vanish	network ACL missing the ephemeral-port rule (a security group is stateful)	Flow Logs: ACCEPT then nothing
Works one way, fails when a second path appears	asymmetric routing through a NAT or firewall that tracks state	compare the paths of both directions
Two VPCs or a VNet and on-premises cannot talk	overlapping address ranges, or peering is not transitive	compare the CIDRs; check the routes in both directions

> INTERVIEW LENS
>
> GOOGLE · MICROSOFT · AMAZON
>
> "What happens when I type an address" is asked at every company (§3.11 (p. 206) answers it for a campus). The cloud version is the follow-up for Google, Microsoft and Amazon: be ready to add the anycast front door, the layer-7 and layer-4 balancers, the overlay in the fabric, and the host's virtual switch. Name the lookup at each hop, as the table does, and say where state lives (the connection table, the flow table, the NAT table).

> CHECK YOURSELF
>
> 1. Where does TLS usually end in this path, and why there? [7]
> 2. Which header lets the leaf-spine fabric spread tenant flows? [8]
> 3. Why may the reply take a different path from the request? [9]

5.15 Mini-project: Pocket Hyperscaler OPTIONAL · ■■■

The project joins the chapter into one story that can be told in an interview. **Brief.** Build, document and break a small EVPN-VXLAN data centre with a cloud-like edge, using Containerlab on a laptop.

1. *Fabric.* Two spines and four leaves (FRR or SR Linux), eight Linux hosts in two tenants (VRF-A and VRF-B), an eBGP unnumbered underlay with BFD (§5.4).
2. *Overlay.* EVPN-VXLAN with symmetric IRB and an anycast gateway for each tenant (§5.5 and §5.6); check the Type-2 and Type-5 routes and the MAC-mobility sequence after a host move.
3. *Edge.* An IPVS direct-routing balancer in front of three web backends in VRF-A, with the VIP announced by BGP from two border leaves (§5.9 and §5.10).
4. *Tooling.* A Python script (see Pillar 10 (p. 586)) that reads the JSON form of `show bgp l2vpn evpn` and prints the table of MAC addresses against VTEPs.
5. *Chaos test.* Fail a spine, a border leaf and a backend in turn; measure loss with `ping -i 0.01` and record the recovery time in a table.

Failure	Detection	Loss (pings)	Recovery time	Cause of the delay
spine S1	BFD			

border leaf BL1	BFD
backend 2	health check

Deliverables. A repository with the topology file and configurations, a diagram, a two-page write-up of the design and the chaos results, and a three-minute demonstration. It is the concrete answer to “tell me about a project” at Arista, Juniper, Nokia, Microsoft and Google.

1. A user-defined route with next hop type VirtualAppliance.↩
2. To force inspection: there is no system route to another spoke, so traffic falls to the default route and the firewall. Peering is also non-transitive.↩
3. The highest precedence; it is evaluated first and a match stops evaluation.↩
4. Prepend the AS path on the secondary advertisement so that the primary has the shorter path.↩
5. Longest-prefix match precedes every BGP preference; the more specific /24 wins.↩
6. Two.↩
7. At the edge: it is close to the user, so the handshake is short, and it can read the request for caching and routing.↩
8. The outer UDP source port, a hash of the inner flow.↩
9. Each direction is routed independently by BGP and ECMP, and DSR deliberately sends the reply on another route.↩

Problem Set

Every interview question of this pillar is solved once, here, in the compact template: a 30-second answer first, then the approach, answer, follow-ups and common mistakes where they apply. The tags give the companies that report the topic, the round, the difficulty (■□□ easy, ■■□ medium, ■■■ hard) and the evidence: **S** if candidates report this specific question, **T** if the topic is reported and the wording is ours. The first twenty questions cover the fabric, the overlay, SDN and the first part of the internals; the rest follow on the next pages. Section numbers refer to this chapter.

Q5.1 What is spine-leaf? Why is every leaf-to-leaf path the same length?

ARISTA · JUNIPER · GOOGLE · TECH-1 · ■□□ · S

30-second answer: a two-tier fabric in which every leaf (rack) switch connects to every spine switch and nothing else; any leaf-to-leaf path is leaf, spine, leaf, so all paths have the same length and the same cost, and there are as many of them as there are spines.

Answer. Servers attach to leaves; leaves have no links to each other, and spines have none either. A path between two leaves must therefore leave through one uplink to a spine and return through that spine's link to the destination: three switches and two links, whichever spine. Equal length makes the paths equal-cost, so ECMP (§5.3) can use all of them, and latency is the same for every pair of racks. **Follow-ups.** For more than one spine's port count of leaves, add a super-spine tier (five switches between racks, §5.2). **Mistake.** Linking spines to each other adds paths of different length and breaks the uniformity.

Q5.2 Define oversubscription. Compute it for 48 × 10G down and 6 × 40G up.

ARISTA · MICROSOFT · TECH-1 · ■□□ · S

30-second answer: oversubscription is the ratio of the bandwidth a switch offers to its servers to the bandwidth it has towards the rest of the fabric; here 480 ÷ 240 = 2 : 1.

Answer. Down: 48 × 10 = 480 Gb/s. Up: 6 × 40 = 240 Gb/s. Ratio 480 ÷ 240 = 2 : 1, so if every server sends at once each receives half its line rate across the fabric. Traffic inside the rack never uses the uplinks. **Follow-up.** For 1 : 1 use 12 uplinks of 40G or reduce the server ports to 24. **Mistake.** Counting the uplinks of one spine only, or mixing gigabits with gigabytes.

Q5.3 What is east-west traffic, and what is north-south?

ALL · TECH-1 · ■□□ · T

30-second answer: north-south crosses the data-centre boundary (clients and the Internet); east-west runs between servers inside it. East-west dominates in modern data centres, which is why the fabric is built for uniform server-to-server bandwidth.

Answer. One user request becomes many internal calls, and replication, storage and analytics add bulk transfers, so internal traffic is a multiple of external traffic (§5.1). Hence the design goals: equal paths, high bisection bandwidth and no blocked links. **Follow-up.** Name a north-south device (the border leaf, the firewall or the edge load balancer).

Q5.4 Why do large data centres prefer eBGP to OSPF in the underlay? Give three reasons.

GOOGLE · MICROSOFT · ARISTA · TECH-2 · ■■□ · T

30-second answer: eBGP limits the effect of a failure to the paths that used it (no network-wide flooding), gives per-neighbour policy for draining and traffic steering, and one protocol serves the fabric and the edge; the cost is an AS-number plan and the need for BFD.

Answer. (i) *Scope.* A link-state protocol floods each change to every router and each recomputes the shortest-path tree; BGP sends updates only to neighbours that need them. (ii) *Policy.* Communities and route maps let an operator drain a switch, steer traffic or filter by tag; OSPF has little equivalent. (iii) *Simplicity.* One protocol runs from the server to the border, with AS-path loop prevention and no redistribution between an IGP and BGP. Also: it runs over TCP, with multipath and unnumbered links (§5.4). **Follow-up.** Fix its slow default timers with BFD at 300 ms × 3. **Mistake.** Claiming BGP converges faster by itself; it does not.

Q5.5 Two leaves share AS 65001. What breaks, and what are two fixes?

ARISTA · JUNIPER · TECH-2 · ■■□ · T

30-second answer: a route learnt from the other leaf carries 65001 in its AS_PATH, so the leaf discards it as a loop and the two racks cannot reach each other; fix with unique AS numbers, `allowas-in` on the leaves, or `as-override` on the spines.

Answer. L1 and L2 (both 65001) exchange routes through spines in AS 65100. L2 receives L1's prefix with AS_PATH 65100 65001, sees its own AS and drops it. Fixes: (i) a unique private AS per leaf (4-byte range if the 1,023 two-byte numbers run out); (ii) `allowas-in` on the leaf, which accepts a bounded number of own-AS occurrences but weakens loop detection; (iii) `neighbor ... as-override` on the spine, which replaces the leaf's AS in the path by the spine's own. **Mistake.** Applying `allowas-in` on the spines, which are not the ones dropping the route.

Q5.6 What does ECMP hash on? Why not per packet?

ALL · TECH-1 · ■■□ · S

30-second answer: normally the 5-tuple (addresses, protocol, ports); every packet of a flow takes the same path, so a TCP stream stays in order. Per-packet balancing reorders segments and causes spurious fast retransmits.

Answer. The hash is computed in hardware, with a per-device seed, and indexes the next-hop group. Some devices let you choose the fields; Linux has `fib_multipath_hash_policy` (0 addresses, 1 5-tuple, 2 inner headers). For tunnels the outer 5-tuple is the same for every packet, so the encapsulating device puts a hash of the inner flow in the outer UDP source port (§5.5). **Follow-up.** Packet spraying is acceptable only where the receiver tolerates reordering. **Mistake.** Saying ECMP "balances load"; it balances flows, statistically.

Q5.7 You see 80% of the traffic on one of four spines. List five possible causes.

GOOGLE · AMAZON · TECH-2 · ■■■ · T

30-second answer: elephant flows, hash polarisation, only one path installed (`maximum-paths`), unequal paths (different AS_PATH length or preference), or a tunnel or LAG with no entropy so that all traffic has one 5-tuple.

Answer. Work from the router outward. (i) *Elephant flows:* a few large flows (backup, shuffle) land on one link; check per-flow telemetry. (ii) *Polarisation:* the same hash and seed at successive tiers (§5.3); change the seed or the fields. (iii) *Multipath not installed:* `show ip route` lists one next hop because `maximum-paths` is 1, or `multipath-relax` is missing where the neighbours' AS numbers differ (§5.4). (iv) *Not equal-cost:* a longer AS_PATH or a higher LOCAL_PREF through three spines leaves them unused. (v) *No entropy:* GRE, IPsec or VXLAN with a fixed source port gives one 5-tuple. Confirm with interface counters per spine, then with a `tcpdump` flow count. **Mistake.** Rebooting spines before reading next-hop counts.

Q5.8 Explain resilient or consistent hashing and why it matters when a spine fails.

GOOGLE · BROADCOM · TECH-2 · ■■■ · T

30-second answer: the hash selects one of many fixed buckets, and the buckets are assigned to next hops; when a next hop fails only its buckets are reassigned, so flows on healthy paths do not move.

Answer. With `hash mod N`, going from N = 4 to 3 changes the index of about 75% of the flows, each of which now takes a different path, arrives out of order or hits a stateful device (a firewall, a load balancer) that has no state for it. With 16 buckets and four members, a failure reassigns 4 of 16 = 25% of the buckets, the minimum possible (Figure 5.5). When the member returns, buckets can migrate back gradually, at an idle gap of the flow, to avoid disturbing flows. **Follow-up.** The same principle at the load balancer is Maglev hashing (§5.9). **Mistake.** Assuming resilient hashing prevents all loss; flows on the failed link still need rerouting.

Q5.9 What is VXLAN? Which UDP port does it use, and how many segment identifiers are there?

ARISTA · CISCO · NOKIA · TECH-1 · ■□□ · S

30-second answer: an overlay that carries Ethernet frames inside UDP/IP between tunnel endpoints (VTEPs), on destination port 4789, with a 24-bit VNI giving 16,777,216 segments.

Answer. The VTEP adds VXLAN (8 bytes), UDP, IP and Ethernet headers (§5.5); the underlay routes the outer IP packet and never sees the tenant frame. A VLAN has 12 bits (4,094 usable), a VNI 24. **Follow-up.** Name Geneve (UDP 6081) as the extensible alternative. **Mistake.** "VXLAN extends a VLAN"; it maps one to a VNI.

Q5.10 Compute the VXLAN overhead and the minimum underlay MTU for a 1,500-byte inner MTU; then for a 9,000-byte tenant MTU over an IPv6 underlay.

ARISTA · JUNIPER · TECH-1 · ■■□ · S

30-second answer: 50 bytes over IPv4 (70 over IPv6); the underlay IP MTU must be at least 1,550 for a 1,500-byte tenant MTU, and 9,070 for 9,000 over IPv6.

Answer. The 1,500 case is worked in §5.5 (inner Ethernet 14 + 1,500 + VXLAN 8 + UDP 8 + IPv4 20 = 1,550). For the second, the inner frame is 9,000 + 14 = 9,014; the outer IPv6 packet adds 40 + 8 + 8, giving 9,070, so a 9,216-byte fabric MTU has room to spare. A 4-byte VLAN tag on the underlay link enlarges the *frame* but not the IP MTU. **Follow-up.** What if you cannot raise the underlay? Lower the tenant MTU and clamp the TCP MSS (§3.5). **Mistake.** Counting the inner Ethernet header twice, or forgetting it.

Q5.11 Why is the outer UDP source port of VXLAN variable?

ARISTA · BROADCOM · TECH-2 · ■■□ · T

30-second answer: so that ECMP in the underlay, which hashes the outer 5-tuple, can spread different tenant flows over different paths; the VTEP sets it to a hash of the inner headers.

Answer. All frames between two VTEPs share outer source and destination addresses, protocol and destination port 4789; a fixed source port would give them a single 5-tuple and one path. The VTEP computes the port from the inner Ethernet, IP and transport fields, typically within 49152 to 65535, so one inner flow keeps one source port (no reordering) and different flows differ. It carries no meaning for the receiver. **Mistake.** Filtering on a fixed source port at a firewall; filter on the destination port.

Q5.12 What does EVPN add over flood-and-learn VXLAN?

ARISTA · JUNIPER · NOKIA · CISCO · TECH-1 · ■■□ · S

30-second answer: a BGP control plane that advertises MAC and IP reachability, so hosts are learnt from routes and not by flooding; it adds mobility, ARP suppression, multihoming, anycast gateways and integrated routing.

Answer. Flood-and-learn sends unknown-unicast and broadcast to every VTEP and learns from traffic, which does not scale and converges by aging. With EVPN each leaf advertises the MACs and IPs it learnt locally (Type 2), its VNIs (Type 3) and prefixes (Type 5); other leaves install them. Consequences: (i) no flooding for known hosts, and ARP answered locally; (ii) a host move is an update with a higher sequence number; (iii) a server can attach to two leaves (Types 1 and 4); (iv) route targets give multi-tenancy. See §5.6.

Q5.13 Explain EVPN route types 2, 3 and 5, with an example of when each is sent.

JUNIPER · NOKIA · ARISTA · TECH-2 · ■■■ · T

30-second answer: Type 2 advertises a host (MAC, optionally IP) with its L2 and L3 VNIs; Type 3 advertises that a VTEP belongs to a VNI, building the flood list; Type 5 advertises an IP prefix, used for subnets and external routes.

Answer. *Type 2:* host 10.1.10.11 with MAC aa:aa:aa:aa:aa:11 appears on L1; L1 sends RD, ESI, MAC, IP, Label1 = 10010 (L2VNI), Label2 = 50001 (L3VNI) and RTs. *Type 3:* VNI 10010 is configured on L3; L3 sends its VTEP address with that VNI, so L1 adds L3 to the replication list for BUM frames of 10010. *Type 5:* the border leaf learns 0.0.0.0/0 from BR1 and sends it into VRF CAMPUS with the L3VNI 50001 and its router MAC. A Type-2 route follows its host; a Type-5 route carries no MAC. **Mistake.** Mixing the two RTs of Type 2 (L2 and L3) or importing one RT in two tenants.

Q5.14 Symmetric versus asymmetric IRB: trade-offs in scale and configuration.

ARISTA · CISCO · TECH-2 · ■■■ · T

30-second answer: in symmetric IRB both leaves route and the packet crosses the fabric in a shared L3VNI, so a leaf needs only its own subnets; in asymmetric IRB only the ingress leaf routes and every leaf needs every VNI, which does not scale.

Answer.

	Symmetric	Asymmetric
Who routes	ingress and egress leaf	ingress leaf only
VNI on the wire	L3VNI of the VRF, both directions	destination's L2VNI, return uses the source's
State per leaf	local subnets plus the L3VNI	every VNI, even without hosts
Configuration	one L3VNI per VRF, router MAC	simple, but a VNI on every leaf
Path	the same both ways	different VNIs each way

Production fabrics use symmetric IRB. **Mistake.** Saying the packet is routed "twice" as if it were a drawback; it is the cheaper design.

Q5.15 A VM moves between racks and traffic blackholes for 30 seconds. Which EVPN mechanisms should have prevented that?

ARISTA · MICROSOFT · TECH-2 · ■■■ · T

30-second answer: the new leaf must learn the MAC quickly (a gratuitous ARP or RARP from the hypervisor), advertise it with a higher MAC-mobility sequence number, and the old leaf must withdraw; a 30-second gap points to a missing announcement or stale caches.

Answer. Check in order. (i) Did the hypervisor send the GARP/RARP on arrival? Without it the leaf learns only from the first data packet. (ii) Does the new leaf's Type-2 route carry sequence 1 (`show bgp l2vpn evpn route type mac-cip`)? (iii) Has the old leaf deleted its local entry and withdrawn it? (iv) Are remote leaves holding a stale ARP or ND entry, or is ARP suppression answering with the old binding? (v) Is *duplicate-MAC detection* freezing the address after several moves in a short window? Thirty seconds suggests an aging or detection timer; flood-and-learn would show the same gap. **Mistake.** Blaming the underlay before reading the Type-2 route.

Q5.16 What is an anycast gateway?

ARISTA · CISCO · TECH-1 · ■□□ · T

30-second answer: every leaf has the same gateway IP address and virtual MAC for a subnet, so a host's default gateway is always on its own leaf, and it keeps working unchanged when the host moves.

Answer. The gateway 10.1.10.1 and one virtual MAC are configured on all leaves; ARP for the gateway is answered locally. There is no active/standby election, no hairpin to a distant gateway, and no gateway change on a VM migration. Compare HSRP and VRRP (§4.8 (p. 265)), which keep one active router per subnet. **Mistake.** Giving each leaf a different physical gateway address for the same subnet.

Q5.17 What is SDN? Name the northbound and southbound interfaces.

CISCO · GOOGLE · TECH-1 · ■□□ · T

30-second answer: separating the control plane (a logically central controller) from the data plane (switches that apply installed rules); applications use the northbound interface (REST, gRPC) and switches the southbound one (OpenFlow, P4Runtime, gNMI, NETCONF).

Answer. The controller has the global topology and computes paths or policies; it programs switches through the southbound protocol. Benefits: global optimisation (Google's B4 traffic engineering) and software-driven operation. Costs: the controller is a critical dependency, so production systems keep forwarding with the last installed state and often keep BGP underneath (§5.7). **Mistake.** "SDN replaces routing protocols."

Q5.18 Describe an OpenFlow flow-table entry and what happens on a table miss.

CISCO · GOOGLE · TECH-2 · ■■□ · T

30-second answer: an entry has a match, a priority, counters, instructions and timeouts; the highest-priority matching entry wins, and on a miss the table-miss entry (priority 0) decides, usually sending a packet-in to the controller.

Answer. Fields: match (ingress port, Ethernet, VLAN, IP, ports), priority, counters, instructions (apply actions such as output or set-field, or goto-table), idle and hard timeouts, a cookie. A packet is matched table by table; if no entry matches and there is no table-miss entry, it is dropped (OpenFlow 1.3). The controller then installs a flow and returns the packet. **Follow-up.** Proactive installation avoids the controller round trip and the outage sensitivity of reactive designs. **Mistake.** Assuming a miss floods by default.

Q5.19 What does P4 let you do that OpenFlow does not?

GOOGLE · BROADCOM · TECH-2 · ■■□ · T

30-second answer: define the parser, the headers, the tables and the actions themselves, so the switch can process a protocol that did not exist when the chip was made; OpenFlow only fills tables over a fixed set of fields.

Answer. A P4 program has a parser (state machine), match-action tables with chosen keys (exact, ternary, lpm) and a deparser; the compiler maps it onto a PISA target. Uses: custom telemetry headers, new tunnels, load-balancing in the switch, in-network aggregation. The control plane (P4Runtime) fills the entries. **Follow-up.** Limits: no loops, a fixed number of stages, and a target-specific architecture. **Mistake.** Treating P4 as a replacement for the controller; it programs the data plane only.

Q5.20 RIB versus FIB: what is the difference?

CISCO · JUNIPER · ARISTA · TECH-1 · ■□□ · S

30-second answer: the RIB holds every route learnt by every source on the control-plane CPU; the FIB is the compact copy of the winning routes, with adjacency data, that the forwarding hardware searches for each packet.

Answer. Several protocols may offer one prefix; the lowest administrative distance (then metric) wins in the RIB, and only the winner is installed in the FIB (§5.8). The FIB also resolves recursive next hops down to an interface and rewrite string. The RIB changes at protocol speed; the FIB is downloaded to the ASIC. Check the two with `show ip route` and `show ip cef` (or `ip route` and the kernel FIB). **Mistake.** Saying the FIB contains backup routes; it normally holds the best path or paths only.

Problem Set, continued: internals, load balancing and the cloud

Q5.21 What is TCAM? Why is it used for LPM and ACLs, and what are its costs?

BROADCOM · CISCO · JUNIPER · TECH-2 · ■■□ · T

30-second answer: a memory that compares the search key with every stored entry at once and stores 0, 1 or "don't care" in each bit; one cycle finds the longest prefix or the first ACL match. It costs area, power and money per bit, so it is small and shared.

Answer. Prefixes and ACL lines are masks over header bits, which is exactly what a ternary cell stores. Entries are kept in priority order and a priority encoder returns the lowest index (Figure 5.14). Costs: a ternary bit needs about twice the storage of an SRAM bit and more area, every row switches on every lookup (power grows with depth), entries must be ordered (an insertion can shift many), and ranges expand into several entries (1024 to 65535 needs six). Large FIBs therefore sit in algorithmic SRAM and TCAM holds ACLs and the rest. **Follow-up.** 144 bits × 512 K entries is about 72 Mbit. Design framing in Q12.33. **Mistake.** "TCAM holds the routing table."

Q5.22 Design a data structure for longest-prefix match supporting 1 million IPv4 prefixes. Discuss memory and lookups per packet.

BROADCOM · ARISTA · JUNIPER · CISCO · TECH-2 · ■■■ · T

30-second answer: a DIR-24-8 table: an array of 2^{24} entries indexed by the top 24 bits of the address, plus 256-entry second-level blocks for the few prefixes longer than /24; a lookup is one memory access, or two, at a cost of about 32 MiB.

Approach. Compare the options by accesses per lookup and memory. A *binary trie* needs up to 32 reads and 1 M prefixes make millions of nodes. A *multibit trie* with stride 8 needs four reads and more memory (prefixes expand to the stride). A *compressed* trie (Linux's LC-trie, a poptrie) uses less memory with a few reads. A TCAM does it in one cycle at high power. DIR-24-8 trades memory for speed: the tables are sized for all /24 blocks, and updates are the price.

Answer.

```
#include <stdint.h>
static uint16_t tbl24[1u << 24];
/* 32 MiB; bit 15 set: index of a tbl8
block */
static uint16_t tbl8[4096][256];
/* 512 bytes per extended /24 block */
uint16_t lookup(uint32_t ip) {
/* returns a next-hop index */
    uint16_t e = tbl24[ip >> 8];
    if (e & 0x8000) e = tbl8[e & 0x7FFF]
[ip & 0xFF];
    return e;
}
```

Insert a prefix of length ≤ 24 by filling 2^(24 − len) entries of `tbl24` (a /8 writes 65,536 entries, a /24 one); keep the longer prefix on collision by inserting in increasing length order or by storing the length. A prefix longer than /24 allocates a `tbl8` block for its /24 and fills the 2^(32 − len) entries. **Complexity.** Lookup O(1): one access (two for long prefixes); memory 32 MiB + 512 B per extended block (4,096 blocks add 2 MiB). With 1 M prefixes, nearly all are ≤ /24, so the table is dense. At about 80 ns per DRAM miss, a core does roughly 12 M lookups per second unprefetched, and far more when it batches lookups and prefetches. **Follow-ups.** IPv6 needs a different structure (a trie on 64-bit prefixes or hashing). Updates of a /8 are slow, so routers batch them. **Mistake.** Forgetting that the 32 MiB array is sparse in use but not in memory.

Q5.23 Walk a packet through a switch ASIC from ingress port to egress.

BROADCOM · ARISTA · JUNIPER · TECH-2 · ■■■ · T

30-second answer: ingress MAC, parser, ingress ACL, L2 lookup, L3 lookup, ECMP resolution and adjacency rewrite, then the traffic manager (queues, buffer, scheduler), egress ACL, deparser and egress MAC.

Answer. The parser builds a header vector; the ACL TCAM can drop, mark or punt; the L2 table (exact match on MAC) finds a routed packet by the router MAC; the L3 LPM returns a next-hop group, a hash picks a member, and the adjacency rewrites the MAC header and the TTL; the traffic manager queues the packet by egress port and class and the scheduler releases it; the egress stages add encapsulation and the checksum. Details and diagram: §5.8. **Follow-ups.** Where is the ECMP hash (after L3)? Where is ECN marked (traffic manager)? Where are punts decided (ACL, L3 or exceptions)? **Mistake.** Putting the queue in front of the lookups; the destination port is needed to choose the queue.

Q5.24 What is a punt? What is CoPP and why is it needed?

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: a punt hands a packet the ASIC cannot forward alone (for the router itself, TTL expiry, an ARP glean) to the CPU; CoPP rate-limits each class of punted traffic so that a flood cannot starve the control plane.

Answer. The CPU handles thousands of packets per second, the ASIC billions, so unprotected punts are a denial-of-service path that can break routing sessions. CoPP applies a hardware policer per class (routing protocols, management, ARP, everything else) before the CPU queue, with generous rates for BGP and OSPF and low ones for the rest (§5.8). **Follow-up.** The data plane also can be protected from the control plane with a router-address ACL. **Mistake.** Applying CoPP to transit traffic; it polices only traffic destined to, or punted to, the router.

Q5.25 Cut-through versus store-and-forward switching. When does cut-through fall back?

ARISTA · BROADCOM · TECH-2 · ■■□ · T

30-second answer: cut-through forwards a frame as soon as the header is read; store-and-forward waits for the whole frame and checks the FCS. Cut-through falls back when the egress port is busy or faster than the ingress port.

Answer. Serialisation time of a 1,500-byte frame is 12,000 bits ÷ 25 Gb/s = 480 ns at 25G (1.2 µs at 10G); cut-through saves most of it per hop. The switch must be able to supply bits to the egress at least as fast as it sends them, so it needs ingress speed ≥ egress speed and an idle port; otherwise the frame is buffered, and the switch is store-and-forward for that frame. Frames with a bad FCS have already left when the error is found; the switch counts them and may poison the frame. **Mistake.** Quoting cut-through latency as if queueing delay vanished.

Q5.26 What is TCP incast? How do deep buffers, ECN/DCTCP or pacing help?

GOOGLE · ARISTA · TECH-2 · ■■■ · T

30-second answer: many servers reply to one client at once, overflowing the shallow buffer of the client's port; tail drops cause retransmission timeouts of about 200 ms and throughput collapses. Larger buffers, early ECN marking with DCTCP, shorter timers or paced replies cure it.

Approach. Quantify the burst against the port buffer. **Answer.** 50 servers each return 20 KB (1 MB in total) to one client on a 10 Gb/s port with 256 KB of buffer. The data needs 1 MB ÷ 1.25 GB/s = 0.8 ms to drain, but the burst arrives almost simultaneously, so about 750 KB is lost. The lost senders wait for a 200 ms timeout, and goodput is 8 Mb ÷ 0.2008 s = about 40 Mb/s, 0.4% of the port. Remedies: (i) *buffers:* deeper or dynamically shared buffers absorb the burst (and cost latency); (ii) *ECN and DCTCP* (§3.9 (p. 192)) mark above a low threshold so senders shrink windows before loss; (iii) *timers:* a retransmission timeout of a few milliseconds; (iv) *application:* limit fan-out, stagger or pace replies; (v) *network:* a priority queue or per-flow fair queueing. **Mistake.** Only adding bandwidth: the port was idle 99% of the time.

Q5.27 What are NSF/Graceful Restart and NSR?

JUNIPER · CISCO · NOKIA · TECH-2 · ■■□ · T

30-second answer: both keep traffic flowing while the control plane restarts. Graceful restart relies on neighbours to keep the router's routes during a restart window; non-stop routing mirrors protocol state on a standby processor, with no help from neighbours.

Answer. *NSF with GR:* the forwarding hardware keeps the stale FIB; the restarting router signals a restart time, and helper neighbours retain its routes and re-learn them (RFC 4724 for BGP, RFC 3623 for OSPF). It needs two route processors or a software restart, and capable neighbours. *NSR:* the standby processor holds a synchronised copy of the protocol state, including TCP sessions for BGP, and takes over unnoticed. GR is cooperative and can be unsafe if the data plane is also broken (traffic is blackholed to a dead next hop for the window); NSR is internal and costs memory and complexity. **Mistake.** Using GR with an aggressive BFD that tears the session down anyway.

Q5.28 L4 versus L7 load balancer.

AMAZON · MICROSOFT · GOOGLE · TECH-1 · ■□□ · S

30-second answer: layer 4 decides from addresses and ports and forwards packets without reading them (fast, no TLS); layer 7 terminates TCP and TLS, reads the HTTP request and routes by host, path or header (flexible, costlier).

Answer. Use the comparison in §5.9: NLB and Azure Load Balancer are layer 4, ALB and Application Gateway layer 7. Choose layer 4 for raw throughput, non-HTTP protocols, source-IP preservation and static addresses; choose layer 7 for path routing, header rewriting, WAF, sticky cookies and retries. Often both are used: a layer-4 tier in front of a layer-7 tier. **Mistake.** Expecting URL routing from a layer-4 balancer.

Q5.29 Explain direct server return. Why does the backend need the VIP on its loopback?

GOOGLE · AMAZON · TECH-2 · ■■□ · T

30-second answer: the balancer forwards the request to the backend without rewriting the destination IP, and the backend replies straight to the client with the VIP as source; the backend must hold the VIP locally to accept the packet, on a loopback with ARP suppressed so that it does not answer for the VIP.

Answer. In layer-2 DSR the balancer rewrites only the destination MAC, so the packet still carries the VIP as destination and the backend's stack drops it unless it owns that address. The loopback holds the VIP; `arp_ignore=1` and `arp_announce=2` stop the backend from advertising it. Over a layer-3 tunnel (IP-in-IP or GRE) the backend decapsulates and sees the same inner packet. The reply source is the VIP, so the client accepts it. Benefits: the balancer handles only requests; limits: no port translation, backends in the same layer-2 domain or tunnelled, and the balancer cannot see the reply to track state. **Mistake.** Putting the VIP on the physical interface, so that the backend answers ARP and steals the VIP.

Q5.30 Design a software L4 load balancer handling 10 Mpps that survives instance failure without breaking connections.

GOOGLE · MICROSOFT · TECH-2 · ■■■ · T

30-second answer: announce the VIP from several stateless balancer nodes; routers spread flows by ECMP; every node uses the same consistent-hash table and a connection table to choose a backend, so a flow that lands on another node after a failure still reaches the same backend.

Approach. Size first, then make failure invisible. **Answer.** *Capacity:* at about 5 Mpps per node (XDP or DPDK on 25 GbE), 10 Mpps needs 2 nodes; with N+2 spare and headroom, 4 to 5. *Flow path:* the border routers hold the VIP route from each node (BGP), ECMP hashes the 5-tuple, ideally with resilient hashing. Each node computes `table[hash(5-tuple) mod M]` with a Maglev table (§5.9), checks its connection table, and forwards by DSR or GRE. *Failure:* BFD removes the dead node's route in about a second; the router re-hashes some flows to other nodes, which hash to the same backend because the tables are identical, so the connections survive. *Backend change:* new connections use the new table; existing ones stay pinned in the connection table until idle. *Health checks* drive the pool; connections drain before removal. **Follow-ups.** SYN floods (SYN cookies at the edge), connection-table size (about 64 bytes per connection), and what breaks (a backend change at the same time as a node failure). **Mistake.** Per-node state that cannot be rebuilt.

Q5.31 How does anycast work? What happens to TCP if routing changes mid-flow?

GOOGLE · AMAZON · TECH-2 · ■■□ · T

30-second answer: the same prefix is announced from many sites and BGP delivers each user to the best (BGP-nearest) one; if the route changes during a connection the packets reach a site with no state for it, which resets the connection.

Answer. Nothing in the protocol is special; the routers see several paths and apply the usual rules (§5.10). A site withdraws its route to leave. Routes are usually stable for the length of a short HTTP or DNS exchange, so TCP works; long flows are exposed to a change of the best path. Mitigations: keep long flows on a unicast address after the handshake, share connection state between sites, use QUIC with connection IDs, and avoid flapping by announcing from stable locations. **Mistake.** Claiming that anycast needs a special protocol, or that it finds the geographically nearest site.

Q5.32 How does a CDN choose which edge serves you?

GOOGLE · AMAZON · MICROSOFT · TECH-1 · ■■□ · T

30-second answer: either DNS returns the address of an edge chosen from the resolver's (or client subnet's) location and measured load, or the CDN announces one anycast address and BGP picks the edge; both aim for the lowest latency with healthy capacity.

Answer. (i) *DNS steering:* the site name is a CNAME to the CDN, whose authoritative server uses the resolver's address or the EDNS Client Subnet, measured latency and load to answer with an edge; TTLs of 20 to 60 s keep it adaptive. (ii) *Anycast:* routing decides, with failover by withdrawal. (iii) Operators combine them, and mid-tier caches shield the origin (§5.10). **Follow-up.** A resolver far from its users causes poor choices; EDNS Client Subnet reduces this at a privacy cost. **Mistake.** Forgetting that a cache miss still pays the origin round trip.

Q5.33 Security group versus network ACL in AWS.

AMAZON · TECH-1 · ■□□ · S

30-second answer: a security group is stateful, allow-only, attached to an interface and evaluated as a whole; a network ACL is stateless, has allow and deny rules in numbered order, and is attached to a subnet.

Answer. Use the table in §5.11. Scenario: a web server accepts TCP 443. The security group needs one inbound rule; the reply is allowed automatically. The network ACL needs an inbound rule for 443 and an outbound rule for the ephemeral ports 1024 to 65535, because it forgets the request. Use security groups for tier-to-tier rules (by group reference) and NACLs for subnet-wide denies, such as blocking an address range. **Mistake.** Writing a deny in a security group.

Q5.34 A private-subnet EC2 instance cannot reach the Internet. List the checks in order.

AMAZON · TECH-1 · ■■□ · S

30-second answer: the subnet's route table (default route to a NAT gateway), the NAT gateway (available, in a public subnet, with an Elastic IP), that subnet's route to the IGW, security group and network ACL rules both ways, DNS, and finally the instance's own firewall.

Answer. (i) Route table of the private subnet: is there `0.0.0.0/0` to a NAT gateway (not a missing or blackholed target)? (ii) NAT gateway: state *available*, an Elastic IP attached, placed in a *public* subnet. (iii) The public subnet's route table: `0.0.0.0/0` to the IGW. (iv) Security group egress of the instance (the default allows all) and the NAT's inbound handling. (v) Network ACLs on both subnets: outbound 443 and inbound ephemeral 1024 to 65535 for the reply. (vi) DNS: `enableDnsSupport`, the resolver at the VPC base + 2. (vii) The host: its default gateway, firewall and proxy settings. Use Flow Logs to see where the packet stops. **Mistake.** Putting the NAT gateway in the private subnet.

Q5.35 Is VPC peering transitive? How do you connect 50 VPCs?

AMAZON · MICROSOFT · TECH-2 · ■■□ · T

30-second answer: no: peering joins exactly two VPCs and does not pass traffic through one to another; for 50 VPCs use a transit gateway, which needs 50 attachments where a full mesh of peerings would need 1,225.

Answer. Peering also needs non-overlapping CIDRs and routes on both sides, and does not allow use of the peer's IGW, NAT or VPN. A transit gateway gives transitive routing, with attachment route tables expressing which VPC reaches which (shared services, isolated tenants). For service-only access use PrivateLink, which tolerates overlapping ranges. Across regions, peer transit gateways. **Follow-up.** Costs: per-attachment and per-GB charges on a TGW, against free peering within an AZ. **Mistake.** Assuming a route through the middle VPC will work if the route tables allow it.

Q5.36 Azure: what is a UDR? Give a scenario that needs one.

MICROSOFT · TECH-1 · ■■□ · T

30-second answer: a user-defined route overrides Azure's system routes for a subnet; for example 0.0.0.0/0 with next hop `VirtualAppliance` to force all spoke traffic through a firewall in a hub.

Answer. A UDR lives in a route table attached to subnets and names a prefix and a next hop type (virtual appliance, virtual network gateway, internet, none). In hub and spoke (§5.12) the spoke default route makes spoke-to-spoke traffic cross the firewall, since peering is non-transitive. Also used to blackhole a prefix (next hop `None`) or to steer traffic to an NVA. Precedence among equal prefixes: UDR, then BGP, then system. **Mistake.** Forgetting *Allow forwarded traffic* on the peering, so that the firewall's forwarded packets are dropped.

Q5.37 Design ExpressRoute with VPN failover. How do you ensure symmetric routing?

MICROSOFT · TECH-2 · ■■■ · T

30-second answer: advertise the same prefixes over both paths with the same length, make ExpressRoute preferred in both directions (higher LOCAL_PREF on-premises, shorter AS_PATH towards Azure) and leave the VPN as a last resort; never advertise a more specific prefix over the VPN.

Approach. Decide preference for each direction separately, then test. **Answer.** The two links of the circuit run to two MSEEs and each is a BGP session (§5.13). *Outbound* (campus to Azure): LOCAL_PREF 200 on routes from the primary CE, 150 from the secondary, 100 from the VPN. *Inbound* (Azure to campus): prepend towards the secondary and more towards the VPN, so the primary has the shortest path. Keep prefix lengths identical on every path, since the longest prefix beats all attributes. Azure prefers ExpressRoute over VPN for the same prefix by default. Add BFD, summarise to stay below the peering route limit, and place a second circuit in another peering location if the facility matters. Rehearse by shutting each link and checking both directions with `traceroute`. **Mistake.** Prepending on only one side, which leaves a path asymmetric through a stateful firewall.

Q5.38 On-premises advertises 10.0.0.0/8 over ExpressRoute while an Azure VNet uses 10.1.0.0/16. What happens and how do you fix it?

MICROSOFT · TECH-2 · ■■■ · T

30-second answer: inside the VNet the more specific local route wins, so VNet traffic stays local, but any on-premises hosts in 10.1.0.0/16 become unreachable from Azure; fix by renumbering, by advertising only the real on-premises prefixes, or by translating one side.

Answer. The VNet has a system route for 10.1.0.0/16 and learns 10.0.0.0/8 through the gateway; longest prefix means 10.1.x.x stays in the VNet and the other 10.x.x.x reaches on-premises. If a campus subnet shares an address with the VNet, say 10.1.5.0/24, traffic to it is delivered inside the VNet to the wrong host or to nothing. In the opposite direction the VNet's 10.1.0.0/16 advertised to the campus competes with the on-premises routes. A VNet cannot be peered with another that overlaps. Fixes, in order of preference: (i) renumber the cloud ranges out of the on-premises space (plan from one IPAM list); (ii) replace the /8 with the prefixes actually in use; (iii) NAT at a firewall for the overlapping part. **Mistake.** Relying on a summary route to "cover" overlapping space.

Q5.39 Explain how Azure or AWS implement the virtual-network data plane: host-based SDN, SmartNICs, encapsulation.

MICROSOFT · AMAZON · TECH-3 · ■■■ · T

30-second answer: the physical network is a plain routed fabric; each host (or its SmartNIC) holds the tenant's flow and mapping tables, encapsulates a tenant packet toward the destination host's physical address, and applies security rules, so the virtual network exists only in the hosts' tables.

Answer. *Publicly described design:* a controller computes a tenant's policy and pushes it to every host; the host's virtual switch (Azure's Virtual Filtering Platform, AWS's Nitro-based software and hardware) holds match-action layers for routing, security-group or NSG rules and NAT. A mapping service translates a virtual address into the physical address of the destination host; the packet is encapsulated (VXLAN-like) and carried by the underlay (§5.5), then decapsulated and filtered at the destination. First packets of a flow take a slower path that builds a flow entry; later packets hit the entry, and with *Accelerated Networking* or Nitro cards the whole fast path is offloaded to the NIC (SR-IOV, FPGA or custom silicon), freeing the CPUs. Stateful rules need a connection tracker. **Follow-up.** Why no broadcast: ARP is answered by the host. **Mistake.** Describing VPCs as VLANs on a switch.

Q5.40 Compare the Google Jupiter approach with a traditional chassis-based data centre.

GOOGLE · TECH-3 · ■■■ · T

30-second answer: Jupiter builds a Clos fabric of many small merchant-silicon switches under centralised control, so capacity scales by adding identical blocks and software is updated centrally; a chassis design buys large proprietary boxes with distributed protocols, which scale less, cost more and fail in bigger domains.

Answer. Per the *Jupiter Rising* paper and its successor: (i) *Topology:* a multi-stage Clos from commodity chips, with later generations adding optical circuit switches in place of part of the spine layer so that capacity can be re-balanced without re-cabling; a chassis tree has a few large aggregation boxes and spanning tree or MLAG. (ii) *Control:* centralised, software-defined routing (Orion) with a logically central view; a chassis network relies on distributed protocols on each device. (iii) *Scale and cost:* many small failures are tolerated and the cost per port is lower; a chassis failure removes a large share of capacity. (iv) *Operations:* incremental upgrade, standard blocks, automation. Limits: the centralised design needs resilient controllers and a fallback. **Mistake.** Saying Jupiter is "just a bigger spine-leaf"; the control plane and the optical layer are the new parts.

Questions solved in other pillars. The congestion-control mechanisms behind Q5.26 are taught in Pillar 3 (p. 162); the ARP, MTU and NAT mechanisms in Q5.10 and Q5.34 are in Pillar 3 (p. 162); BGP attributes in Q5.5, Q5.37 and Q5.38 are in Pillar 4 (p. 229). System-design framings of Q5.21 and Q5.30 (Q12.33, Q12.39) and the slow-user ECMP case (Q11.23) appear in their own pillars and cite this chapter.

Revision Spread

THE 15-MINUTE VERSION

Ten facts. (1) A two-tier Clos: leaves ≤ spine ports, spines = uplinks per leaf, servers = leaves × server ports, oversubscription = down ÷ up; a three-tier fabric of k-port switches holds $k^3/4$ servers at 1 : 1. (2) ECMP balances *flows* by a 5-tuple hash; polarisation comes from the same seed at every tier; resilient hashing moves only the failed member's share (1/N), mod-N moves about 75% when N goes from 4 to 3. (3) RFC 7938: eBGP everywhere, one AS for the spines and one per leaf, BFD 300 ms × 3 = 0.9 s, unnumbered links over IPv6 link-local; the 2-byte private range has only 1,023 numbers. (4) VXLAN is UDP 4789 with a 24-bit VNI and an 8-byte header; it adds 50 bytes over IPv4 (70 over IPv6), so the underlay IP MTU must be at least 1,550; the outer source port is a hash of the inner flow. (5) EVPN is BGP AFI 25 / SAFI 70: Type 2 host, Type 3 flood list, Type 5 prefix; a host move raises the MAC-mobility sequence; use the anycast gateway and symmetric IRB (L3VNI). (6) SDN splits controller from switch; an OpenFlow table miss is the priority-0 entry; P4 programs the parser, the tables and the deparser. (7) RIB on the CPU holds all routes, FIB in the ASIC holds winners with adjacencies; a TCAM stores prefixes longest first and returns the lowest index; the range 1024 to 65535 costs six entries; punts are limited by CoPP. (8) L4 balancers use the 5-tuple (NAT, DSR, tunnel); L7 terminate TLS and read HTTP; DSR needs the VIP on a loopback with ARP suppressed; Maglev uses a prime-sized table. (9) Anycast is BGP-nearest, not nearest in kilometres; expected CDN latency = h × e + (1 − h) × (e + o). (10) AWS: five reserved addresses, security groups stateful and allow-only, NACLs stateless and numbered, peering non-transitive, use a transit gateway; Azure: NSGs stateful with priority 100 to 4096 (lowest first), UDR next hop `VirtualAppliance`, ExpressRoute ASN 12076, one circuit is not site-redundant.

Three figures to redraw. The two-tier fabric with its port budget (Figure 5.2), the VXLAN encapsulation with byte counts (Figure 5.8), and the ASIC pipeline from ingress to egress (Figure 5.15).

Fabric, ECMP and underlay

Item	Rule or value
Oversubscription	down bandwidth ÷ up bandwidth; 48 × 25G over 8 × 100G = 1.5 : 1
Fabric size	spines = leaf uplinks (8); leaves = spine ports (64); servers = 64 × 48 = 3,072
Bisection	half the leaves × their uplink capacity: 32 × 800G = 25.6 Tb/s
Pod and super-spine	32 leaves, 4 spines; spine 32 down + 32 up; 64 pods = 98,304 servers
Hash inputs	src and dst IP, protocol, src and dst port; Linux policy 0/1/2
Mod-N, 4 → 3 members	about 75% of flows change path; resilient: 25% (4 of 16 buckets)
ASN plan	spines 65100, leaves 65101 and up; unique per leaf; `allowas-in` or `as-override` if shared
Failure detection	BFD 300 ms × 3 = 0.9 s; BGP default hold 180 s
`multipath-relax`	needed when equal-length paths differ in AS content

Overlay comparison

Encapsulation	Transport	Header and identifier	Notes
VXLAN (RFC 7348)	UDP 4789	8 bytes; 24-bit VNI	fixed header; source port = inner hash; dominant in switches
Geneve (RFC 8926)	UDP 6081	8 bytes + options; 24-bit VNI	extensible metadata; OVN, NSX, AWS Gateway LB

NVGRE (RFC 7637)	GRE, protocol 47	8 bytes; 24-bit VSID + 8-bit flow ID	no UDP port, so weak ECMP entropy; older Hyper-V
GRE	protocol 47	4 bytes (8 with key)	point-to-point tunnels; carries anything
MPLS L3VPN	labels	4 bytes per label	provider core (§4.10 (p. 271)); label stack, no IP overlay

Overhead over IPv4: VXLAN 50 bytes (14 + 20 + 8 + 8), Geneve 50 plus options, GRE 24 (20 + 4) with no outer Ethernet counted.

EVPN

Type	Name	Use
1	Ethernet auto-discovery	multihoming, fast withdrawal
2	MAC/IP advertisement	host; L2VNI and L3VNI; mobility sequence
3	Inclusive multicast	VNI membership, BUM flood list
4	Ethernet segment	designated forwarder election
5	IP prefix	subnets, default route, external routes

Symmetric IRB: ingress leaf routes into the L3VNI, egress leaf routes out. Asymmetric: every VNI on every leaf. ARP suppression: answered from Type-2 bindings.

Router internals

Item	Fact
RIB	all routes, all sources; control-plane CPU and DRAM
FIB + adjacency	winners and rewrite strings; ASIC memories
Punt reasons	to the router itself, TTL expiry, glean (ARP), IP options, ACL log
TCAM	ternary 0/1/x; priority encoder; 144 bits × 512 K ≈ 72 Mbit; high power
Pipeline	MAC, parser, ingress ACL, L2, L3 (LPM), ECMP and rewrite, traffic manager, egress ACL, deparser, MAC
Cut-through	needs idle egress and ingress speed ≥ egress speed; else store-and-forward
Incast	many senders, shallow buffer, 200 ms RTO; fix with ECN/DCTCP, buffers, fewer or paced senders
GR versus NSR	helper neighbours keep routes; standby processor mirrors state

Load balancers, anycast and CDN

Item	Fact
L4	5-tuple; NAT, DSR or tunnel; no URL
L7	terminates TLS; routes on host, path, header
DSR	VIP on loopback; `arp_ignore=1`, `arp_announce=2`; reply bypasses the balancer
Maglev	table of M entries (M prime); offset and skip per backend; round-robin fill
Health check	up after 2 successes, down after 3 failures (typical)
SNAT limit	about 64,000 ports per source address and destination address and port
Anycast	one prefix, many origins; failover by withdrawal; long TCP flows at risk
CDN hit	0.9 × 15 + 0.1 × 235 = 37 ms (edge 15, origin 220)

AWS versus Azure

Subnet	one AZ	spans zones	AWS: subnet per AZ
Instance filter	security group (stateful, allow)	NSG on NIC (stateful, allow/deny, priority)	NSG lowest number first
Subnet filter	network ACL (stateless)	NSG on subnet	Azure has no stateless filter
Routes	route table	route table with UDRs	UDR > BGP > system
Internet	internet gateway	default outbound / system route	AWS needs IGW and public IP
NAT	NAT gateway (per AZ)	NAT Gateway	cure for SNAT exhaustion
Join two networks	peering (non-transitive)	VNet peering (non-transitive)	overlap is not allowed
Hub	transit gateway	Virtual WAN hub, or hub VNet + firewall	many VPCs: 50 attachments, not 1,225 peerings
Private service access	PrivateLink (interface endpoint)	Private Link (private endpoint)	one-way, overlap allowed
Service by route	gateway endpoint (S3, DynamoDB)	service endpoint	no Internet transit
L4 / L7 balancer	NLB, GWLB / ALB	Load Balancer / Application Gateway	GWLB uses Geneve
Global front door	CloudFront, Global Accelerator	Front Door	anycast edge
Private circuit	Direct Connect (VIFs: private, public, transit)	ExpressRoute (private, Microsoft peering)	circuit is not site-redundant
VPN	Site-to-Site VPN, 2 tunnels	VPN Gateway	up to 1.25 Gb/s per AWS tunnel
Dynamic routes with an NVA	(transit gateway Connect)	Route Server (AS 65515)	BGP into the VNet
Flow records	VPC Flow Logs	NSG flow logs	first diagnostic tool
Reserved addresses	5 per subnet	5 per subnet	.0, .1, .2, .3, last

First five checks when a cloud VM cannot connect

i. Route table of the subnet (and the NAT or gateway state). (ii) Security group or NSG, both directions, by priority. (iii) Network ACL, including the ephemeral ports (AWS). (iv) MTU through the overlay, VPN or peering: `ping -M do -s 1422` . (v) Return path and overlap: is the reply routed back the same way, and do the address ranges overlap?

Flashcards

Q	A
Spine-leaf path length?	leaf, spine, leaf; equal for all pairs
Servers in a two-tier fabric?	leaves (= spine ports) × server ports per leaf
Oversubscription?	down ÷ up bandwidth at the leaf
5-stage Clos adds?	super-spine tier, pods and planes
What does ECMP balance?	flows, not packets
Polarisation cause and cure?	same hash and seed at all tiers; per-device seed
Resilient hashing?	fixed buckets; only the failed member's buckets move
Why eBGP in the DC?	small failure scope, policy, one protocol, simple loop control

Shared leaf ASN breaks?	AS_PATH loop check drops routes; unique AS, `allowas-in` or `as-override`
BFD 300 ms × 3?	0.9 s detection
BGP unnumbered?	IPv6 link-local sessions; IPv4 routes via IPv6 next hop
Private ASN ranges?	64512–65534 (2-byte), 4200000000–4294967294 (4-byte)
VXLAN port and ID?	UDP 4789; 24-bit VNI, 16,777,216 values
VXLAN overhead?	50 bytes IPv4, 70 IPv6; underlay MTU ≥ 1,550
Outer source port?	hash of the inner flow, for ECMP entropy
BUM delivery?	multicast underlay or ingress replication
EVPN address family?	AFI 25, SAFI 70
Type 2, 3, 5?	host MAC/IP; VNI flood list; IP prefix
MAC mobility?	higher sequence number wins; old leaf withdraws
Anycast gateway?	same IP and MAC on every leaf
Symmetric IRB?	both leaves route, shared L3VNI
ARP suppression?	leaf answers from EVPN bindings
OpenFlow table miss?	priority-0 entry, often packet-in
P4 programs what?	parser, match-action tables, deparser
RIB or FIB in the ASIC?	FIB with adjacency
TCAM order?	longest prefix first; lowest index wins
Entries for ports 1024–65535?	six
CoPP?	rate-limits punted traffic per class
Cut-through falls back when?	egress busy or faster than ingress
Incast fix?	ECN/DCTCP, buffers, shorter RTO, paced replies
L4 versus L7?	5-tuple and packets; TLS terminated and HTTP routing
DSR backend needs?	the VIP on a loopback, ARP suppressed
Maglev table size?	prime M much larger than the backends
Anycast picks?	the BGP-nearest site
SG versus NACL?	stateful allow-only on the interface; stateless numbered on the subnet
Is peering transitive?	no; use a transit gateway
NSG rule order?	lowest priority number first
ExpressRoute redundancy?	two links, one location; add a second circuit
VPN over ExpressRoute with a /24?	more specific prefix wins; keep lengths equal

PART I · PILLAR 6

6

Wireless and Cellular Networks

A radio link is a wire with noise, a shared medium and a rulebook; we learn the rulebook frame by frame.

6.1 RF in one sitting	MUST
6.2 PHY essentials: OFDM, QAM and MIMO	MUST
6.3 802.11 architecture and the family tree	MUST
6.4 The MAC: CSMA/CA and DCF timing	MUST
6.5 The 802.11 frame, bit by bit	MUST
6.6 The connection lifecycle	MUST
6.7 Security: WPA2, WPA3 and PMF	MUST
6.8 QoS, aggregation and power save	SHOULD
6.9 Wi-Fi 6, 6E and Wi-Fi 7	MUST
6.10 Roaming and enterprise design	SHOULD
6.11 The Linux and firmware view	SHOULD
6.12 Cellular from LTE to 5G	SHOULD

Wireless engineering is asked in three registers: arithmetic (decibels, timing, data rates), protocol (frames, handshakes, state machines) and system (firmware, scheduling, cellular cores). The sections follow that order. The Must path is about 4.5 days; the Qualcomm and Broadcom WLAN roles expect the whole chapter, and the Nokia and Qualcomm modem roles add §6.12. Ethernet and CSMA/CD are taught in §3.3 (p. 168), the generic 802.1X and EAP exchange in §9.12 (p. 567), and the Linux capture tools in §8.7 (p. 498); this chapter adds only what is specific to the air.

6.1 RF in one sitting MUST · ■□□

A radio link is governed by three quantities: how much power leaves the transmitter, how much of it survives the journey, and how much noise the receiver must compete with. Engineers compare these on a logarithmic scale because the numbers span many orders of magnitude: a transmitter radiates 100 milliwatts, while a receiver can still decode a signal of a ten-billionth of a milliwatt (−100 dBm).

Decibels and dBm

The *decibel* (dB) expresses a ratio of two powers as ten times the base-10 logarithm of that ratio. It is relative: it says “twice as much”, never “so many watts”. Because logarithms turn multiplication into addition, a chain of gains and losses is simply summed. The *dBm* is the same scale with a fixed reference of one milliwatt, and so it is an absolute power. Antenna gain is quoted in *dBi*, relative to an ideal isotropic radiator that sends equally in all directions.

Change in dB	Power ratio	Remember as
0	×1	unchanged
+3	×2	double (exactly 1.995)
+6	×4	double twice
+10	×10	one decade
+20	×100	two decades
+30	×1000	three decades
−3	×½	half
−10	×0.1	one tenth

To convert, use P(dBm) = 10 $\log_{10}$(P in mW). Thus 1 mW is 0 dBm, 100 mW is 20 dBm, 1 W is 30 dBm, and 200 mW is 100 mW doubled, that is 20 + 3 = 23 dBm. A mixed number such as 40 mW is found by splitting it into factors of 10 and 2: 40 = 10 × 2 × 2, so 0 + 10 + 3 + 3 = 16 dBm.

> **TRAP**
>
> *Claim:* “+3 dB means three times the power.” *Why it is wrong:* the decibel is logarithmic; +3 dB is a factor of two and +10 dB a factor of ten. A related slip is to add dBm values: two 20 dBm sources do not make 40 dBm. *Correct statement:* ratios in dB add, absolute levels in dBm combine as powers, so two equal sources give +3 dB (here 23 dBm), and a dBm value plus a dB gain gives a dBm value.

Path loss and the link budget

A signal spreads as it travels, and its power density falls with the square of the distance. For a free-space path, the *free-space path loss* (FSPL) between isotropic antennas is

FSPL (dB) = 20 $\log_{10}$(d) + 20 $\log_{10}$(f) − 147.55, with d in metres and f in hertz.

Two consequences follow. Doubling the distance adds 6 dB, and doubling the frequency also adds 6 dB; hence a 5 GHz signal is about 7 dB weaker than a 2.4 GHz signal at the same distance, which is one reason 5 GHz cells are smaller. Real buildings are worse than free space: walls, floors and people add loss, and indoors the loss grows as 10·n·$\log_{10}$(d) with an exponent n between 3 and 4 rather than 2.

The *link budget* is the bookkeeping of this journey. We start from the transmit power, add every gain, subtract every loss, and compare the received level with what the receiver needs.

WORKED EXAMPLE

Link budget of a 5 GHz link. An AP transmits 20 dBm (100 mW) through a 3 dBi antenna to a laptop 30 m away, with one internal wall of 12 dB and a 3 dBi receive antenna; the channel is 20 MHz wide and the receiver noise floor is −92 dBm.

1. FSPL at 5.5 GHz over 30 m: 20 $\log_{10}$(30) = 29.5, 20 $\log_{10}$(5.5×10⁹) = 194.8, so 29.5 + 194.8 − 147.55 = 76.8 dB.
2. Received power = 20 + 3 − 76.8 − 12 + 3 = −62.8 dBm (Figure 6.1).
3. SNR = −62.8 − (−92) = 29.2 dB.
4. Shannon's bound C = B $\log_2$(1 + SNR) with SNR = 10^2.92 ≈ 832 gives 20 MHz × $\log_2$(833) ≈ 20 × 9.7 = 194 Mb/s for one spatial stream.
5. Compare with the minimum receiver sensitivities of 802.11ax at 20 MHz: MCS 7 (64-QAM 5/6) needs −64 dBm, MCS 8 (256-QAM 3/4) needs −59 dBm, MCS 11 (1024-QAM 5/6) needs −52 dBm. A level of −62.8 dBm therefore guarantees MCS 7, about 86 Mb/s; a good receiver at 29 dB SNR may manage MCS 8, about 103 Mb/s, roughly half of Shannon's bound.

Result: −62.8 dBm received, 29 dB SNR, MCS 7 to 8. To reach 1024-QAM one needs roughly 35 dB SNR, which means being closer or in the same room.

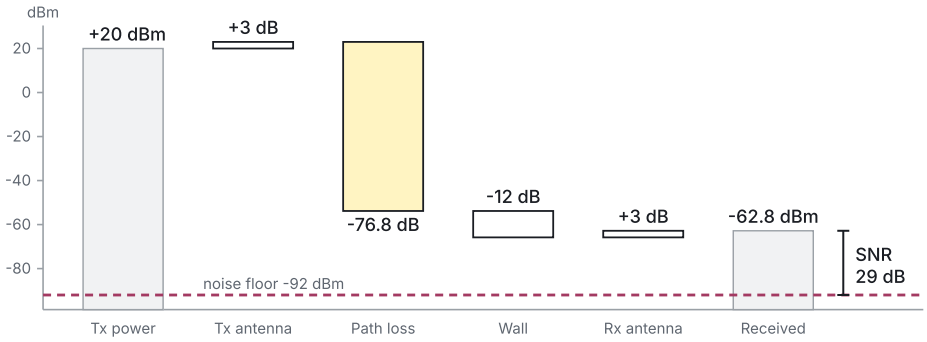


Figure 6.1 Link budget of the worked example: gains raise the level, losses lower it, and the SNR is the gap between the received level and the noise floor.

The noise floor itself has a cause. Thermal noise power is −174 dBm per hertz plus 10 $\log_{10}$(B); for a 20 MHz channel that is −174 + 73 = −101 dBm, and a receiver's electronics add a *noise figure* of several decibels, which is why −92 dBm is a realistic floor. Doubling the channel width doubles the noise power, so an 80 MHz channel has a floor 6 dB higher than a 20 MHz one. This is the first of many trade-offs between speed and robustness.

Shannon and SNR

Shannon's capacity theorem states that the error-free rate of a channel with bandwidth B hertz and signal-to-noise ratio SNR (as a plain ratio, not in dB) cannot exceed $B \log_2(1 + SNR)$. Two readings matter. At high SNR each additional 3 dB of SNR adds about one bit per hertz, because $\log_2$ gains 1 when SNR doubles; at low SNR capacity is nearly linear in power. Capacity is linear in bandwidth. Thus wider channels and cleaner signals both help, but bandwidth pays more, which is why every Wi-Fi generation widened the channel while only slowly raising the QAM order.

Wi-Fi turns this into a table. The transmitter chooses a *modulation and coding scheme* (MCS) from the measured SNR: a low SNR demands a sparse constellation and heavy error coding, a high SNR allows a dense one. The choice of MCS for each frame is called *rate adaptation* and is taught in §6.11 (p. 390).

> INTERVIEW LENS
>
> QUALCOMM · NOKIA · BROADCOM
>
> "Convert 200 mW to dBm" and "what does −3 dB mean" are warm-up questions in WLAN and modem screens, usually followed by a link-budget calculation such as the one above. A strong answer states the formula, writes the chain of additions on the board, and finishes by relating SNR to a modulation order. Follow-ups: why 5 GHz cells are smaller than 2.4 GHz cells, and what a 3 dB change in transmit power does to range (about 41% more distance in free space, since the loss grows with the square).

> CHECK YOURSELF
>
> 1. Convert 25 mW to dBm. [1]
> 2. A receiver has a −90 dBm noise floor and gets −61 dBm. What is the SNR? [2]
> 3. By how much does FSPL change when the distance triples? [3]

6.2 PHY essentials: OFDM, QAM and MIMO MUST · ■■□

The physical layer (PHY) turns bits into radio waves. Modern Wi-Fi, LTE and 5G share one idea at this layer: split the channel into many narrow subcarriers, put a few bits on each, and send them all at once. The reasoning is as follows. A single fast stream of symbols is short in time, and an echo of one symbol arrives while the next is being received; this *multipath* smears the symbols together. Many slow parallel streams have long symbols, and an echo of a few hundred nanoseconds is a small fraction of a long symbol.

OFDM, subcarriers and the cyclic prefix

Orthogonal frequency-division multiplexing (OFDM) places subcarriers at a spacing of exactly 1/T hertz, where T is the useful symbol duration. The spectrum of each subcarrier has the shape sin(x)/x, which is zero at every multiple of 1/T from its centre. Hence at the centre of any one subcarrier all the others contribute exactly zero, and a receiver that integrates over one symbol period separates them perfectly. This is what *orthogonal* means, and it is the answer to the common interview question about how the subcarriers do not interfere (Figure 6.2).

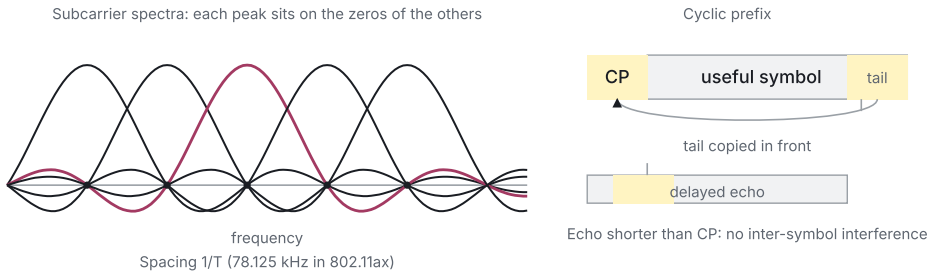


Figure 6.2 Orthogonal subcarriers (left) and the cyclic prefix that absorbs multipath echoes (right).

To keep the symbols apart in time, the transmitter copies the last part of each symbol to its front, the *cyclic prefix* (in 802.11, the *guard interval*, GI). An echo that arrives later than the direct path by less than the prefix falls inside the copy and does not corrupt the next symbol. Because the prefix is a cyclic copy, the received block still looks periodic, so orthogonality survives. The price is time: a 0.8 µs guard interval on a 12.8 µs symbol costs 6%.

The table below lists the numerology of the Wi-Fi generations. The 802.11a/g/n/ac OFDM symbol is 3.2 µs plus a 0.8 µs guard (0.4 µs in the short-GI option) on 312.5 kHz subcarriers, while 802.11ax made the symbol four times longer (12.8 µs, 78.125 kHz spacing) to tolerate outdoor echoes and to make resource units finer.

Generation	Subcarrier spacing	Useful symbol	Guard intervals	Data subcarriers in 20 MHz
802.11a/g	312.5 kHz	3.2 µs	0.8 µs	48 (plus 4 pilots)
802.11n	312.5 kHz	3.2 µs	0.8 or 0.4 µs	52
802.11ac	312.5 kHz	3.2 µs	0.8 or 0.4 µs	52
802.11ax, be	78.125 kHz	12.8 µs	0.8, 1.6 or 3.2 µs	234

Pilot subcarriers carry known symbols; the receiver uses them to track phase and frequency drift.

QAM and the modulation and coding scheme

Quadrature amplitude modulation (QAM) sends a symbol as a point on a two-dimensional grid, by varying the amplitude of two carriers 90° apart. A grid of M points carries $\log_2(M)$ bits per symbol: 4 points (QPSK) carry 2 bits, 16-QAM 4 bits, 64-QAM 6 bits, 256-QAM 8 bits, 1024-QAM 10 bits and 4096-QAM 12 bits (Figure 6.3). A denser grid packs more bits but brings the points closer, so the same noise makes more errors; this is why a high QAM order needs a high SNR. *Forward error correction* adds redundancy, and the *coding rate* (1/2, 2/3, 3/4 or 5/6) is the fraction of bits that are data. 802.11n and earlier use convolutional codes; LDPC codes, which are stronger at high rates, are optional in 802.11n/ac and mandatory in 802.11ax for resource units larger than 242 tones.

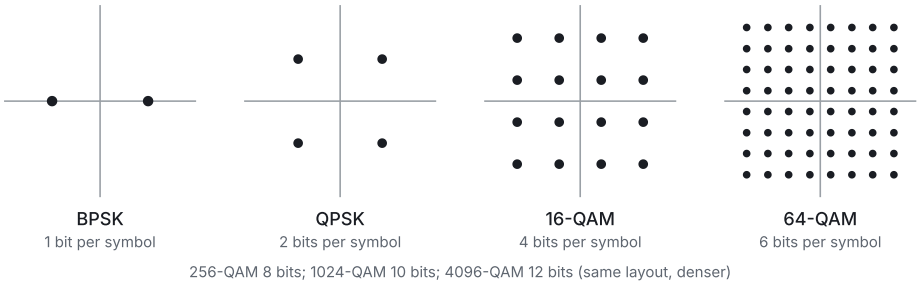


Figure 6.3 Constellations: each dot is one symbol, and a denser grid carries more bits but leaves less distance between neighbours.

Each combination of modulation and coding rate is an MCS index. For a given width, the rate follows from one formula.

Rate = (data subcarriers × bits per subcarrier × coding rate × spatial streams) ÷ (symbol + guard duration).

WORKED EXAMPLE

Data rate of an 802.11ax link. An 80 MHz channel with one 996-tone resource unit has 980 data subcarriers (16 of the 996 are pilots). Use 1024-QAM (10 bits), coding rate 5/6, symbol 12.8 µs plus a 0.8 µs guard interval.

1. Bits per OFDM symbol per stream = 980 × 10 × 5/6 = 8,167.
2. Symbol time = 12.8 + 0.8 = 13.6 µs.
3. Rate per stream = 8,167 ÷ 13.6 µs = 600.5 Mb/s.
4. Two spatial streams give 1,201 Mb/s, the familiar “1.2 Gb/s” label; four streams on 160 MHz (1,960 subcarriers) give 4 × 1,201 = 4,804 Mb/s.
5. For Wi-Fi 7 take 320 MHz (3,920 data subcarriers) and 4096-QAM (12 bits): 3,920 × 12 × 5/6 ÷ 13.6 µs = 2,882 Mb/s per stream; sixteen streams, the standard’s ceiling, give 46.1 Gb/s.

Result: 600.5 Mb/s per stream at 80 MHz; 2,882 Mb/s per stream at 320 MHz with 4096-QAM. Marketing figures are these formulas at the best MCS and the shortest guard interval.

MCS	Modulation	Coding	20 MHz, 1 stream	80 MHz, 1 stream	Typical SNR needed
0	BPSK	1/2	8.6 Mb/s	36.0 Mb/s	5 dB
2	QPSK	3/4	25.8	108.1	12 dB
4	16-QAM	3/4	51.6	216.2	18 dB
7	64-QAM	5/6	86.0	360.3	25 dB
9	256-QAM	5/6	114.7	480.4	31 dB
11	1024-QAM	5/6	143.4	600.5	35 dB

The rates are for 802.11ax with the 0.8 µs guard interval; the SNR column is an engineering rule of thumb, not a standard value, because it depends on the receiver. Wi-Fi 7 adds MCS 12 and 13 (4096-QAM at 3/4 and 5/6).

Spatial streams, MIMO and beamforming

With several antennas at each end a radio can exploit *multiple-input multiple-output* (MIMO) operation. If the transmitter has N_t antennas and the receiver N_r, the channel is a matrix **H** of N_r × N_t complex gains, one per antenna pair. In a scattering environment these paths differ, so the receiver can solve the linear equations y = **H**x and recover up to min(N_t, N_r) independent *spatial streams* sent at the same time on the same frequency. The rate scales with the number of streams, at the cost of needing high SNR on every stream. A "2×2:2" radio has two transmit and two receive antennas and two streams.

Beamforming uses the same antennas differently. The transmitter weights each antenna's phase so that the waves add constructively at the receiver, which raises the SNR by up to 10 $\log_{10}$(N_t) dB (about 3 dB for two antennas, 6 dB for four). The transmitter learns the channel through *explicit sounding*: it sends a null data packet, and the receiver returns a compressed feedback matrix. In *single-user* MIMO (SU-MIMO) all streams go to one client; in *multi-user* MIMO (MU-MIMO) the AP steers different streams to different clients at once, placing the nulls of one client's beam on the other. MU-MIMO needs accurate sounding, which costs airtime, so it pays only with many clients and many AP antennas. The 802.11 generations and their abilities are in §6.3 (p. 365).

Channel width is the last lever. Wi-Fi channels are 20 MHz building blocks that bond to 40, 80, 160 and (Wi-Fi 7) 320 MHz. Each doubling roughly doubles the rate and lowers the SNR by 3 dB, because the noise bandwidth doubles. A wide channel also requires every 20 MHz piece to be idle before transmission, so in a crowded band it is rarely available. The primary 20 MHz channel carries the beacons and the control frames of legacy clients.

TRAP

Claim: "OFDM and OFDMA are the same thing with a different name", or "MU-MIMO and OFDMA are alternatives." *Why it is wrong:* OFDM is the modulation, used by every generation since 802.11a, in which one user occupies all subcarriers of a symbol. OFDMA, new in 802.11ax, divides the subcarriers among several users within one symbol (§6.9 (p. 384)). MU-MIMO separates users in space on the same subcarriers. *Correct statement:* OFDMA shares frequency, MU-MIMO shares space; both can be used together.

INTERVIEW LENS

QUALCOMM · BROADCOM · NOKIA

A Qualcomm candidate was asked to "explain OFDMA and how subcarrier orthogonality is maintained". A strong answer gives the 1/T spacing and the zero-crossing argument, adds the cyclic prefix as the protection against multipath, and only then moves to resource units. Follow-ups: why 802.11ax lengthened the symbol fourfold, and how pilots are used.

CHECK YOURSELF

1. How many bits does a 256-QAM symbol carry, and what happens to the required SNR if 64-QAM is replaced by 256-QAM at the same coding rate? [4]
2. Why does doubling the channel width not double the range? [5]
3. What limits the number of spatial streams? [6]

1. 25 mW is 100 mW divided by four, so 20 − 6 = 14 dBm.↩

6.3 802.11 architecture and the family tree MUST · ■□□

IEEE 802.11 organises stations around an access point. A *station* (STA) is any device with an 802.11 radio. An *access point* (AP) is a station that also bridges to a wired network. The AP and the stations associated with it form a *basic service set* (BSS), the unit of radio coverage; the BSS is identified by a 48-bit *BSSID*, which is the MAC address of the AP's radio for that network. A human-readable network name, the *SSID* (up to 32 bytes), is advertised by the AP; several BSSs that carry the same SSID and are joined by a wired *distribution system* (DS) form an *extended service set* (ESS), within which a client may roam. The ESS identifier is the SSID, which is why the terms *SSID* and *ESSID* are used interchangeably in practice.

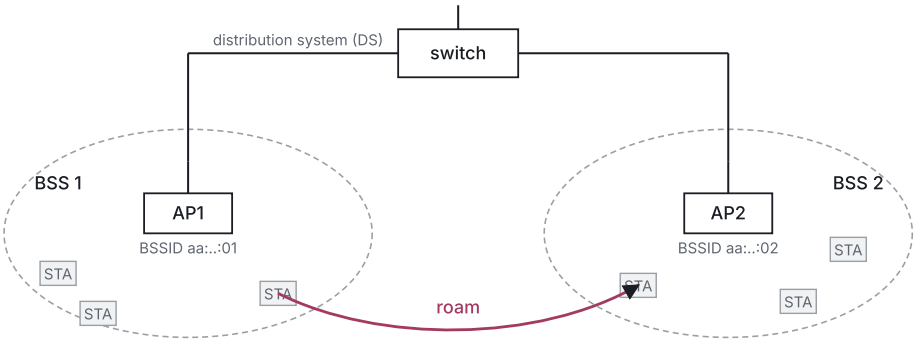


Figure 6.4 An extended service set: two BSSs joined by a wired distribution system under one SSID.

The difference between the two names is the cause of many interview slips. The SSID names a service and is shared by many radios; the BSSID names one radio on one band, and it is the address that appears in the frames. A dual-band home router that advertises "Sharma_Home" on 2.4 and 5 GHz has one SSID and two BSSIDs; a campus with 200 APs, three radios each and two SSIDs on every radio has 200 × 3 × 2 = 1,200 BSSIDs. Each BSSID sends its own beacon about ten times a second, so every extra SSID adds management traffic at the lowest data rate. A common rule is to keep the number of SSIDs per radio to three or fewer.

Besides the infrastructure BSS there are three other modes. In an *independent BSS* (IBSS, "ad hoc") stations talk directly with no AP. *Wi-Fi Direct* lets one device act as a software AP for a peer. *Mesh* (802.11s) joins APs wirelessly so that frames are forwarded over several radio hops, which needs the fourth address field (§6.5 (p. 370)). In all cases the DS is an abstraction, the "wherever frames go when they leave the BSS", and in practice it is a switched Ethernet network.

WORKED EXAMPLE

Counting identifiers. A hostel has 40 dual-band APs, each radio broadcasting two SSIDs ("Hostel" and "Guest"). How many BSSIDs, SSIDs and beacons per second are there?

1. SSIDs: 2 (the ESSs "Hostel" and "Guest").
2. BSSIDs: 40 APs × 2 bands × 2 SSIDs = 160.
3. Beacons: every BSSID beacons once per 102.4 ms, so 160 × 9.77 ≈ 1,560 per second, of which the 2.4 GHz half (780) are sent at the lowest rate and occupy more airtime than the rest.

Result: 2 SSIDs, 160 BSSIDs, about 1,560 beacons per second.

2. SNR = −61 − (−90) = 29 dB.↩
3. 20 $\log_{10}(3)$ = 9.5 dB more.↩
4. 8 bits; the points are closer, so roughly 6 dB more SNR is needed.↩
5. The noise bandwidth doubles, so the SNR falls by 3 dB and the best MCS is lower at the same distance.↩
6. The smaller of the transmit and receive antenna counts, and the rank of the channel matrix, which depends on scattering.↩

The generations

The standard grows by amendments, each a letter. The Wi-Fi Alliance later added numbers for marketing: Wi-Fi 4 (802.11n), 5 (ac), 6 (ax) and 7 (be). The table gives the facts an interview expects.

Name	IEEE	Year	Band	Widths	Highest QAM	Streams	Peak PHY rate	What it introduced
(Wi-Fi 1)	802.11b	1999	2.4 GHz	22 MHz	DSSS/CCK	1	11 Mb/s	first mass-market Wi-Fi
(Wi-Fi 2)	802.11a	1999	5 GHz	20 MHz	64	1	54 Mb/s	OFDM
(Wi-Fi 3)	802.11g	2003	2.4 GHz	20 MHz	64	1	54 Mb/s	OFDM in 2.4 GHz
Wi-Fi 4	802.11n	2009	2.4, 5	20, 40	64	4	600 Mb/s	MIMO, aggregation, block ack
Wi-Fi 5	802.11ac	2013	5	20 to 160	256	8	6.93 Gb/s	wider channels, DL MU-MIMO
Wi-Fi 6, 6E	802.11ax	2021	2.4, 5, (6)	20 to 160	1024	8	9.61 Gb/s	OFDMA, UL MU-MIMO, BSS colour, TWT
Wi-Fi 7	802.11be	2024	2.4, 5, 6	20 to 320	4096	16	46.1 Gb/s	MLO, 320 MHz, puncturing

The peak rates are PHY rates at the best MCS, the maximum number of streams and the shortest guard interval, computed with the formula of §6.2 (p. 361). Real products use two streams, and real throughput is about 60% of the PHY rate at best, and much less under contention, for reasons that are the subject of the next section. Wi-Fi 6E is 802.11ax operating in the 6 GHz band; it is not a different PHY.

> CHECK YOURSELF
>
> 1. Which identifier is a MAC address, the SSID or the BSSID? [1]
> 2. What distinguishes an ESS from a BSS? [2]
> 3. Which generation first used OFDM, and which added MIMO? [3]

6.4 The MAC: CSMA/CA and DCF timing MUST · ■■□

Wi-Fi stations share one radio channel, and only one may transmit successfully at a time. Ethernet resolved the same problem with *carrier sense multiple access with collision detection*, in which a sender listens while it transmits and stops when it hears a collision (§3.3 (p. 168)). A radio cannot do that. Its own transmission is some 90 dB stronger at its antenna than any signal arriving from afar (100 mW against a received −70 dBm), so a station cannot hear a collision while it sends. Wi-Fi therefore *avoids* collisions, rather than detecting them: this is *CSMA with collision avoidance* (CSMA/CA), and the base procedure is the *distributed coordination function* (DCF).

Carrier sense and the interframe spaces

weak as about −82 dBm in 20 MHz) or any energy above about −62 dBm. *Virtual carrier sense* uses the *Duration* field present in every frame: a station that overhears a frame sets its *network allocation vector* (NAV), a timer counting down the microseconds for which the medium is reserved, and regards the medium as busy until the NAV reaches zero, even if it cannot hear the other transmitter.

Waiting times are measured in a small set of *interframe spaces*, whose lengths set priorities. The shortest, the *short interframe space* (SIFS), separates a frame from its immediate response (data and ACK, RTS and CTS); the *DCF interframe space* (DIFS = SIFS + 2 slots) is how long a station must find the medium idle before it may contend for access. Because SIFS is shorter than DIFS, the response to a frame always begins before any contender's DIFS has expired. Hence the receiver of a frame gets the medium back without a fight, which is the cause of the ordering rather than a convention.

PHY	Slot	SIFS	DIFS = SIFS + 2 × slot	CWmin	CWmax
802.11a/n/ac/ax in 5 GHz (and 6 GHz)	9 µs	16 µs	34 µs	15	1023
802.11g/n/ax in 2.4 GHz, short slot	9 µs	10 µs	28 µs	15	1023
802.11g in 2.4 GHz with 802.11b present	20 µs	10 µs	50 µs	15	1023
802.11b (DSSS)	20 µs	10 µs	50 µs	31	1023

Two more spaces complete the set. The PIFS (SIFS + 1 slot) is used by the legacy point coordination function, and the EIFS (SIFS + the time of an ACK at the lowest rate + DIFS) is used after receiving a corrupted frame, to give the unseen exchange time to finish.

The DCF algorithm

The procedure for a unicast frame is as follows.

1. If the medium has been idle for DIFS, transmit at once. Otherwise defer.
2. When the medium becomes idle for a DIFS, draw a random integer *backoff* uniformly from [0, CW], where CW starts at CWmin.
3. Count the backoff down by one for each slot in which the medium stays idle. If the medium becomes busy, *freeze* the counter, wait for the medium to be idle for a DIFS again, and resume.
4. When the counter reaches zero, transmit.
5. The receiver, if the frame arrived intact, sends an ACK after a SIFS. If the sender receives no ACK within the timeout, it assumes a collision or loss, doubles the window ($CW \leftarrow 2 \cdot CW + 1$, up to CWmax), and returns to step 2. After a success, or after the retry limit (7 for short frames) is exceeded and the frame is dropped, CW returns to CWmin.

The sequence of windows is 15, 31, 63, 127, 255, 511 and 1023 for retries 0 to 6, so after six failures the average wait has grown from 7.5 slots to 511 slots. The random draw is what separates two stations that would otherwise sense the idle medium and start together; doubling the window is what keeps the collision probability from rising as the number of contenders grows. The ACK is necessary precisely because the sender cannot hear a collision: it is the only evidence of success.

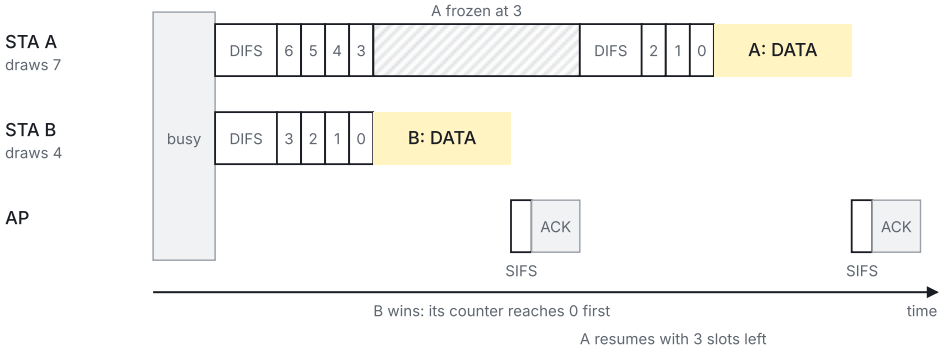


Figure 6.5 Two stations contend: B's counter expires first, A freezes its remaining count and resumes after the acknowledgement and a fresh DIFS.

WORKED EXAMPLE

Contention between two stations (5 GHz, slot 9 µs, SIFS 16 µs, DIFS 34 µs). Stations A and B both have a frame and find the medium idle after another transmission. A draws a backoff of 7, B draws 4.

1. Both wait a DIFS of 34 µs, then count down. After 4 slots (36 µs) B's counter is 0, while A's has fallen from 7 to 3.
2. B transmits. A senses the medium busy and freezes at 3 (Figure 6.5).
3. The AP acknowledges after a SIFS of 16 µs. A must now wait for the medium to be idle for a DIFS, then counts down its remaining 3 slots (27 µs), and transmits.
4. Suppose instead both had drawn 5. Their counters reach zero in the same slot, both transmit, and the frames collide. Neither receives an ACK; each doubles CW from 15 to 31 and draws again from [0, 31]. The probability that they again draw the same value is 1/32.

Result: B sends first; A resumes with 3 slots left, so a loser keeps the progress it made. Collisions cost an entire frame time plus the ACK timeout.

WORKED EXAMPLE

How much of 54 Mb/s is useful? A station sends 1,500-byte packets at 54 Mb/s in 5 GHz, with ACKs at 24 Mb/s and the average backoff of 7.5 slots.

1. MPDU = 1,500 payload + 8 LLC/SNAP + 24 header + 4 FCS = 1,536 bytes = 12,288 bits. With the 16 service and 6 tail bits, 12,310 bits at 216 bits per 4 µs symbol need 57 symbols = 228 µs. Adding the 20 µs preamble gives 248 µs.
2. The ACK (14 bytes) at 24 Mb/s takes the 20 µs preamble plus 2 symbols = 28 µs.
3. One cycle = DIFS 34 + backoff 7.5 × 9 = 67.5 + data 248 + SIFS 16 + ACK 28 = 393.5 µs.
4. Goodput = 12,000 bits ÷ 393.5 µs = 30.5 Mb/s, which is 56% of 54 Mb/s.
5. At 600 Mb/s the data part shrinks to about 70 µs (preamble included) while the overhead of 34 + 67.5 + 16 + 32 stays at about 150 µs: the cycle is 220 µs and the goodput 55 Mb/s, under 10% of the PHY rate.

Result: 56% at 54 Mb/s, under 10% at 600 Mb/s. The fixed overhead does not shrink with the rate, which is why 802.11n introduced aggregation (§6.8 (p. 381)**).**

Hidden nodes and RTS/CTS

Carrier sense assumes that if a transmitter is audible to me, I am audible to it. This fails for the *hidden-node problem*: stations A and C are both within range of the AP but out of range of each other (Figure 6.6). Each finds the medium idle, each transmits, and the frames collide at the AP, repeatedly, because backoff only separates stations that can hear one another.

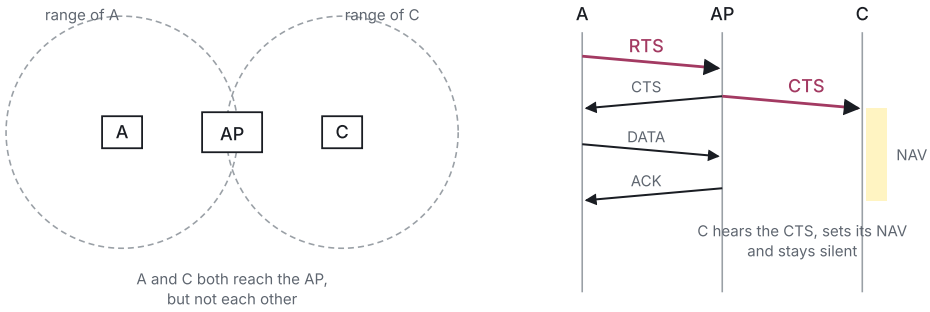


Figure 6.6 The hidden-node problem (left) and its repair (right): the CTS, sent by the AP, reaches the station that could not hear the RTS.

The remedy is a short handshake. Before sending a long frame A sends a 20-byte *request to send* (RTS); the AP replies with a 14-byte *clear to send* (CTS), after a SIFS. Both frames carry a Duration field covering the rest of the exchange: the RTS announces 3 SIFS + CTS + DATA + ACK, and the CTS announces 2 SIFS + DATA + ACK. C cannot hear A, but it hears the AP's CTS, sets its NAV for that duration and stays silent. A collision can now occur only on the short RTS, which is cheap to repeat. The price is the extra RTS and CTS airtime, about 100 µs at the lowest rate, so the handshake is applied only to frames above an *RTS threshold*, which is by default so high that it is off.

The mirror image is the *exposed-node problem*. Carrier sense can forbid a transmission that would have caused no harm: in Figure 6.7, C hears B sending to A and defers, although C's frame to D would not have disturbed A. The standard has no cure within a cell; spatial reuse between cells is addressed by BSS colouring (§6.9 (p. 384)).

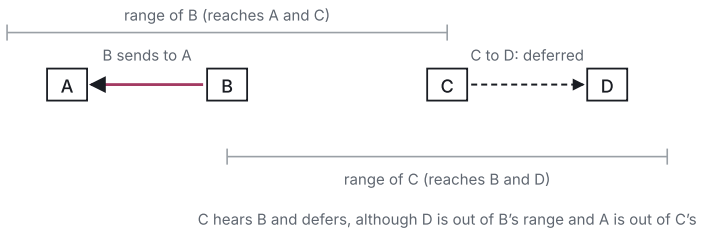


Figure 6.7 The exposed-node problem: carrier sense forbids a transmission that would not have caused any collision.

> **TRAP**
>
> *Claim:* "Wi-Fi uses CSMA/CD like Ethernet", or "RTS/CTS is always on and prevents all collisions." *Why it is wrong:* a radio cannot detect a collision while it transmits, so Wi-Fi uses collision avoidance with a positive acknowledgement; RTS/CTS is applied only above a threshold and merely protects the long frame. *Correct statement:* Wi-Fi is CSMA/CA with ACKs; collisions still occur, but between short RTS frames when the handshake is used.

INTERVIEW LENS

QUALCOMM · BROADCOM · CISCO

"Why CSMA/CA and not CD", "explain DCF with SIFS, DIFS, slot and backoff", and "what is the hidden-node problem" are core screening questions for WLAN firmware roles. A strong answer states the timing values for one PHY, draws the backoff timeline with the frozen counter, explains why SIFS is shorter than DIFS, and mentions that CW doubles on failure and resets on success. Follow-ups: the NAV and its source, the efficiency calculation above, and why aggregation exists.

CHECK YOURSELF

1. Compute DIFS for 802.11b. [4]
2. A frame fails twice in succession (5 GHz). From what range is the third backoff drawn? [5]
3. Who sets the NAV, and from what? [6]

6.5 The 802.11 frame, bit by bit MUST · ■■□

An 802.11 frame has the same three-part shape as any link-layer frame, a header, a body and a trailer, but its header is more elaborate than Ethernet's. This is because a wireless frame must say what kind of frame it is (data, management or control), in which direction it is going relative to the wired network, and which of up to four stations are involved. Unlike IP, whose fields are in network (big-endian) byte order, 802.11 transmits multi-byte fields least-significant byte first. A hand decoder must therefore swap the two bytes of Frame Control before reading it.

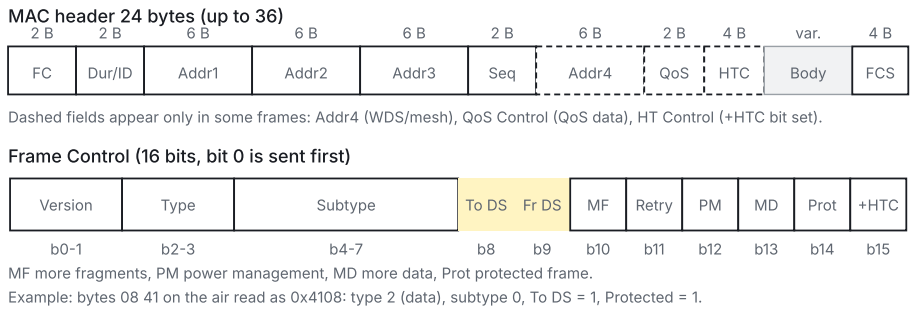


Figure 6.8 The 802.11 MAC header and the bit layout of its Frame Control field.

Frame Control

The first two bytes, the *Frame Control* field, describe the frame. Reading bit 0 first (the first bit sent):

Bits	Field	Meaning
0–1	Protocol version	always 0
2–3	Type	0 management, 1 control, 2 data
4–7	Subtype	the particular frame within the type
8	To DS	frame is going to the distribution system (client to AP)
9	From DS	frame is coming from the distribution system (AP to client)
10	More Fragments	more fragments of this MSDU follow
11	Retry	this is a retransmission; the sender sets it so the receiver may drop duplicates

1. The BSSID; the SSID is a name of up to 32 bytes.↩
2. An ESS joins several BSSs through a distribution system under one SSID, so a client may roam; a BSS is one AP and its clients.↩
3. 802.11a (and g in 2.4 GHz) used OFDM; 802.11n added MIMO.↩
4. DIFS = SIFS + 2 × slot = 10 + 2 × 20 = 50 µs.↩
5. Retry 0 uses [0, 15], retry 1 [0, 31], retry 2 [0, 63]; the third attempt draws from [0, 63].↩
6. Any station that overhears a frame not addressed to it, from the Duration field of that frame (or of the RTS or CTS).↩

12	Power Management	the sender will sleep after this frame (§6.8 (p. 381))
13	More Data	the AP holds more buffered frames for a sleeping STA
14	Protected Frame	the body is encrypted (CCMP, GCMP)
15	+HTC / Order	an HT/VHT/HE Control field is present

WORKED EXAMPLE

Decoding two Frame Controls. The bytes on the air are `08 41` for a data frame and `80 00` for a beacon.

1. Swap the bytes of the first: 0x4108 = 0100 0001 0000 1000 in binary.
2. Bits 0–1 = 00 (version 0); bits 2–3 = 10 (type 2, data); bits 4–7 = 0000 (subtype 0, plain data).
3. Byte 1 is 0x41 = 0100 0001: bit 8 (To DS) = 1, bit 9 (From DS) = 0, bit 14 (Protected) = 1.
4. The beacon: 0x0080, bits 2–3 = 00 (management) and bits 4–7 = 1000 (subtype 8, beacon); no flags.

Result: `08 41` is a protected data frame from a station to its AP; `80 00` is a beacon.

Wireshark and `tcpdump` combine type and subtype into one number, (type × 16) + subtype, which is the value used in the display filter `wlan.fc.type_subtype`. The ones worth memorising are in the next table.

Type	Frame (subtype)	Code	Purpose
Management	Association Request / Response (0 / 1)	0x00 / 0x01	join the BSS; response carries status and AID
Management	Reassociation Request / Response (2 / 3)	0x02 / 0x03	move to another AP of the ESS
Management	Probe Request / Response (4 / 5)	0x04 / 0x05	active scan
Management	Beacon (8)	0x08	periodic advertisement of the BSS
Management	Disassociation (10), Authentication (11), Deauthentication (12)	0x0a, 0x0b, 0x0c	end association, open or SAE exchange, end all
Management	Action (13)	0x0d	block-ack setup, 802.11k/v/r, SA Query
Control	Trigger (2)	0x12	802.11ax, schedules uplink OFDMA
Control	Block Ack Request / Block Ack (8 / 9)	0x18 / 0x19	selective acknowledgement of an aggregate
Control	PS-Poll (10)	0x1a	a sleeping STA asks for a buffered frame
Control	RTS / CTS / ACK (11 / 12 / 13)	0x1b / 0x1c / 0x1d	reservation and acknowledgement
Data	Data (0), Null (4)	0x20, 0x24	payload; Null carries only the power-management bit
Data	QoS Data (8), QoS Null (12)	0x28, 0x2c	payload with a QoS Control field

The addresses: four fields, three meanings

A frame has up to four address fields, because a frame relayed by an AP involves up to four parties: the original source, the final destination, and the two radios that carry it across one hop.

The *receiver address* (RA) and *transmitter address* (TA) are the radio-hop endpoints and are always Address 1 and Address 2. The *source address* (SA) and *destination address* (DA) are the end systems. The To DS and From DS bits tell which role each field plays.

To DS	From DS	Address 1	Address 2	Address 3	Address 4	Typical case
0	0	RA = DA	TA = SA	BSSID	not present	IBSS, management frames
1	0	RA = BSSID	TA = SA	DA	not present	station to AP
0	1	RA = DA	TA = BSSID	SA	not present	AP to station
1	1	RA	TA	DA	SA	AP to AP (WDS, mesh)

Consider a laptop (SA) sending to a file server (DA) behind the AP. The frame to the AP has To DS = 1: Address 1 is the AP's BSSID (the receiver), Address 2 the laptop (the transmitter and source) and Address 3 the server (the destination). The AP, after checking the frame, transmits an Ethernet frame onto the wire; in the reverse direction the AP sends with From DS = 1, Address 1 the laptop, Address 2 its own BSSID and Address 3 the server as source. The fourth address appears only when a frame is relayed wirelessly from AP to AP, because then neither the transmitter nor the receiver is an end system.

Remaining fields and the trailer

The *Duration/ID* field (2 bytes) holds the number of microseconds the exchange will occupy, from which listeners set their NAV (§6.4 (p. 366)); in a PS-Poll it carries the association ID instead. The *Sequence Control* field is a 12-bit sequence number and a 4-bit fragment number, so that duplicates created by retransmissions can be discarded. The *QoS Control* field (2 bytes, present in QoS data frames) holds the traffic identifier (TID, 4 bits), the acknowledgement policy and whether the body is an A-MSDU (§6.8 (p. 381)). The *HT/VHT/HE Control* field (4 bytes) carries link-adaptation feedback. The body is followed by a 4-byte *frame check sequence* (CRC-32) over the whole frame; a receiver silently drops a frame whose FCS is wrong, which is why no negative acknowledgement exists. When encryption is on, CCMP adds an 8-byte header (a 48-bit packet number and the key identifier) and an 8-byte integrity check, so an encrypted frame is 16 bytes longer than a clear one.

The short control frames show how compact the format can be: an ACK or CTS is only Frame Control, Duration, a receiver address and the FCS, 14 bytes; an RTS adds the transmitter address, 20 bytes. A management frame has a 24-byte header, a fixed part and a sequence of *information elements* (IEs), each coded as one byte of element ID, one byte of length and the value. A beacon's body is as follows.

Field or IE	Size	Content in the example
Timestamp	8 B	the AP's microsecond clock, for synchronisation
Beacon interval	2 B	100 TU (1 TU = 1,024 µs, so 102.4 ms)
Capability information	2 B	ESS bit, privacy bit, short preamble
SSID (ID 0)	1 + 6 B	"IITLab"
Supported rates (ID 1), DS parameter set (ID 3)	variable	basic rates; channel 6 (2.4 GHz only)
TIM (ID 5)	variable	DTIM count and period, bitmap of stations with buffered data

RSN (ID 48)	variable	group cipher CCMP, pairwise CCMP, AKM PSK
HT, VHT, HE capabilities	variable	widths, streams, MCS sets, features supported

WORKED EXAMPLE

Decode a QoS data frame header. The first 32 bytes of a captured frame are

`88 41 2c 00 | 02 11 22 33 44 aa | 02 11 22 33 44 bb | 02 00 5e 00 00 01 | 10 07 | 06 00 | 01 00 00 20 00 00 00 00`

1. `88 41`: type 2 (data), subtype 8 (QoS data), To DS = 1, Protected = 1. Because To DS = 1 and From DS = 0, the table gives Address 1 = BSSID, Address 2 = source, Address 3 = destination.
2. `2c 00` is 0x002c = 44 µs of duration, which covers a SIFS and the ACK.
3. Address 1 is `02:11:22:33:44:aa`, the AP; Address 2 is `02:11:22:33:44:bb`, the station; Address 3 is `02:00:5e:00:00:01`, the gateway.
4. `10 07` is 0x0710: fragment number = low 4 bits = 0, sequence number = 0x0710 ≫ 4 = 0x71 = 113.
5. `06 00` is the QoS Control: TID = 6, which maps to the voice access category.
6. The final 8 bytes are the CCMP header: packet number 1 (bytes `01 00` and `00 00 00 00`), and the byte `20`, whose bit 5 is the ExtIV flag, with key identifier 0. The body that follows is ciphertext.

Result: a voice-priority frame from `…:bb` to the gateway `…:01`, relayed by AP `…:aa`, with packet number 1.

LAB

`LAB 6.5 · 15 MIN` Write the Frame Control decoder in C and feed it the two examples above. In a real capture the radiotap header precedes the frame; it begins with a version byte, a pad byte and a 16-bit little-endian length, and that length must be skipped.

```c
#include <stdio.h>
#include <stdint.h>
int main(void) {
    uint8_t wire[2][2] = { {0x08, 0x41}, {0x80, 0x00} };
    for (int i = 0; i < 2; i++) {
        uint16_t fc = wire[i][0] | (wire[i][1] << 8);   /* little-endian */
        printf("fc=0x%04x ver=%u type=%u subtype=%u ", fc,
               fc & 3, (fc >> 2) & 3, (fc >> 4) & 15);
        printf("tods=%u fromds=%u retry=%u prot=%u\n",
               (fc >> 8) & 1, (fc >> 9) & 1, (fc >> 11) & 1, (fc >> 14) & 1);
    }
    return 0;
}
```

```
fc=0x4108 ver=0 type=2 subtype=0 tods=1 fromds=0 retry=0 prot=1
fc=0x0080 ver=0 type=0 subtype=8 tods=0 fromds=0 retry=0 prot=0
```

What you should notice: the byte order is the only trap. Declaring a `struct` with bit-fields would give a layout that depends on the compiler and the host's endianness, so shifts and masks on a `uint16_t` built from the bytes are the portable way. This is the same lesson as `htons` in §7.10 (p. 445), in the opposite direction.

TRAP

Claim: “Address 3 is the destination”, or “the first address is the destination.” *Why it is wrong:* Address 1 is always the receiver of this radio hop, and the meaning of Address 3 depends on To DS and From DS (the destination when To DS = 1, the source when From DS = 1, the BSSID in management frames). *Correct statement:* read the two DS bits first, then the table.

INTERVIEW LENS

QUALCOMM · BROADCOM · JUNIPER

“Explain To DS and From DS and the four addresses” is a standard WLAN question; so is “what is in a beacon”. A strong answer gives the table, walks through one STA-to-server frame in both directions, and notes the little-endian byte order. Follow-ups: what the Retry bit is for, what the sequence number prevents, and why control frames have no sequence number.

CHECK YOURSELF

1. A frame has To DS = 0, From DS = 1. Which address is the source? [1]
2. What is the length of an ACK frame? [2]
3. Which Wireshark subtype codes are a beacon and a deauthentication? [3]

6.6 The connection lifecycle MUST · ■■□

A station connects to a network in five phases: it discovers the AP, performs an initial 802.11 exchange, associates, establishes keys, and only then obtains an IP address. The order matters because each phase unlocks the next. Each is a state of a small state machine kept by the AP: in State 1 the station is unauthenticated and unassociated and may send only a few management frames; in State 2 it is authenticated but not associated; in State 3 it is authenticated and associated, which allows data frames. A *deauthentication* returns the station to State 1 and a *disassociation* to State 2. In a secured network a second gate, the 802.1X *controlled port*, stays blocked in State 3 until the keys are installed, and until then only the key-exchange frames pass.

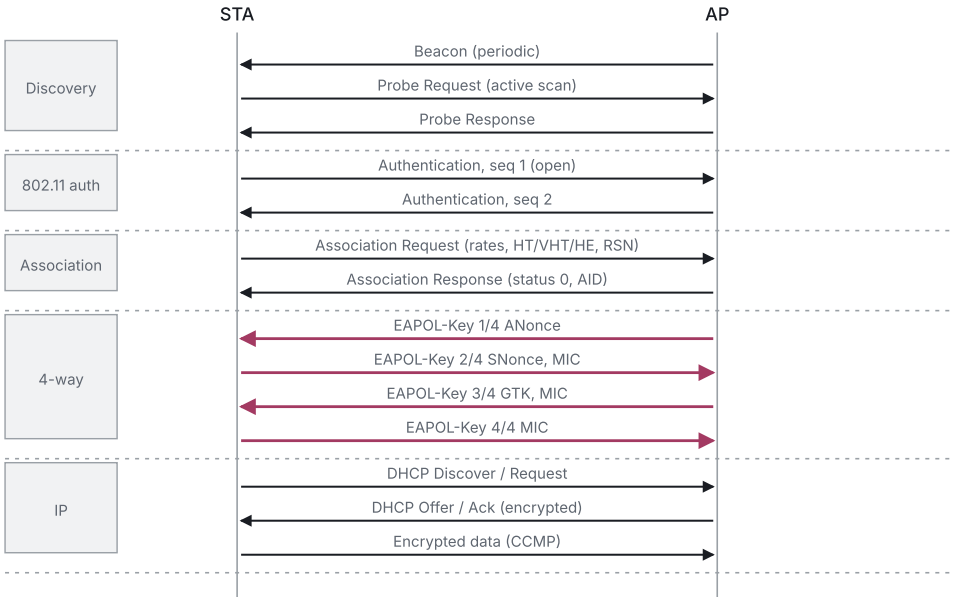


Figure 6.9 The connection lifecycle of a WPA2-Personal station, in the order seen in a monitor-mode capture.

Discovery. In *passive scanning* the station tunes to each channel in turn and listens for beacons, which each BSS sends every 102.4 ms; the dwell per channel must be at least one beacon interv-

al, so a full scan is slow. In *active scanning* the station sends a *probe request* (with a wildcard SSID, or with the SSID of a known network) and every AP that hears it replies with a *probe response* carrying the same capabilities as a beacon; the dwell is only about 20 ms per channel. Phones mix both: active scans on channels where probing is allowed, passive on channels that need radar detection. In 6 GHz a station scans first the *preferred scanning channels*, one in four, and learns about the other channels from reports inside the 2.4 and 5 GHz beacons.

Open authentication. The station sends an *Authentication* frame (algorithm 0, sequence 1) and the AP answers (sequence 2, status 0). In modern networks this exchange is a formality that carries no secret; it survives for compatibility with the original state machine. WEP's shared-key authentication (algorithm 1) is obsolete, and WPA3 replaces the open exchange by the SAE commit and confirm frames (algorithm 3, §6.7 (p. 377)).

Association. The *association request* states the SSID, the supported rates and the station's HT, VHT and HE capabilities (what it can do), a listen interval for power save, and its security choices in the RSN element. The AP answers with a status (0 means success) and an *association ID* (AID), a number from 1 to 2007 that identifies the station in the TIM bitmap. The AP checks that the RSN element matches its policy.

Key establishment. If the network uses a pre-shared key, the pairwise master key is already known to both sides and the *4-way handshake* follows at once (§6.7 (p. 377)). In an enterprise network the station and a RADIUS server first run an EAP method through the AP, which relays the exchange (EAPOL between station and AP, RADIUS between AP and server); the generic exchange is explained in §9.12 (p. 567), and its Wi-Fi-specific output, the PMK delivered to the AP, is covered in §6.7 (p. 377).

IP configuration. With keys installed, the station sends DHCP and ARP (§3.6 (p. 183)), encrypted like any other data. This explains why a monitor-mode capture shows DHCP only when the 4-way handshake has been captured and the passphrase is known to Wireshark.

The table gives a typical capture for a WPA2-Personal client, as a monitor-mode sniffer would see it. Times are indicative.

#	Frame	From → to	Key fields	Why it matters
1	Beacon	AP → broadcast	SSID, RSN (CCMP, PSK), TIM, capabilities	passive discovery; repeated every 102.4 ms
2	Probe Request	STA → broadcast	SSID (wildcard or named), rates	active scan; may use a random MAC
3	Probe Response	AP → STA	as beacon, without TIM	answers only the prober
4–5	Authentication ×2	STA ⇄ AP	algorithm 0, sequence 1 then 2, status 0	opens State 2
6–7	Association Request, Response	STA ⇄ AP	rates, HT/VHT/HE, RSN; status 0, AID	opens State 3; the port is still blocked
8	EAPOL-Key 1 of 4	AP → STA	ANonce, replay counter	STA can now derive the PTK
9	EAPOL-Key 2 of 4	STA → AP	SNonce, MIC, RSN	proves the STA knows the PMK
10	EAPOL-Key 3 of 4	AP → STA	MIC, encrypted GTK, Install	AP proves the same; keys to install
11	EAPOL-Key 4 of 4	STA → AP	MIC	confirms; port opens

12–15	DHCP Discover, Offer, Request, Ack	STA ⇄ server	inside QoS Data, protected	visible only after decryption
16	ARP for the gateway	STA → broadcast	inside protected data	first data frames

The filter `eapol || wlan.fc.type_subtype == 0x0b || wlan.fc.type_subtype == 0x00` isolates the authentication, association and handshake frames in Wireshark (§8.8 (p. 502)). A deauthentication frame carries a *reason code* that says why the connection ended; the common ones are 1 (unspecified), 2 (previous authentication no longer valid), 3 (the station is leaving), 4 (inactivity), 15 (4-way handshake timeout, usually a wrong passphrase) and 23 (802.1X authentication failed).

TRAP

Claim: "802.11 authentication is the security step." *Why it is wrong:* the open exchange has no credentials. Since WPA2, the real authentication is the PSK or 802.1X outcome proved by the 4-way handshake; authentication and association are two state transitions, not two levels of security. *Correct statement:* authentication opens State 2 and association opens State 3; keys come afterwards.

INTERVIEW LENS

QUALCOMM · CISCO · JUNIPER

"How does a station associate with an AP?" and "walk through a WPA2-Personal connection frame by frame" recur in Qualcomm Hyderabad and Bengaluru reports and in Cisco and Juniper wireless rounds. A strong answer lists the phases in order, names the frames, says which state each frame reaches, and distinguishes the 802.11 exchange from the key exchange. Follow-ups: a missing EAPOL message 4 (wrong passphrase or a lost frame), and what a station does if association fails with status 17 (the AP is full).

CHECK YOURSELF

1. In which state may a station first send a data frame? [4]
2. Why can a sniffer see a DHCP exchange only sometimes? [5]
3. What does the AP assign during association? [6]

1. Address 3 (Address 1 is the destination, Address 2 the BSSID).↩
2. 14 bytes: Frame Control 2, Duration 2, receiver address 6, FCS 4.↩
3. 0x08 and 0x0c.↩
4. State 3 (authenticated and associated); in a secured network, user data flows only after the controlled port opens.↩
5. The DHCP packets are encrypted; Wireshark decrypts them only if it has captured the 4-way handshake and knows the passphrase.↩

6.7 Security: WPA2, WPA3 and PMF MUST · ■■■

A radio signal can be received by anyone in range, so a wireless link needs encryption that the wired link does not. Three properties are wanted: only authorised stations may join, traffic must be unreadable to others, and frames must not be forged or replayed. The security of 802.11 has been rebuilt twice because the first two designs failed.

Wired Equivalent Privacy (WEP, 1997) used the RC4 stream cipher with a 24-bit initialisation vector (IV) sent in clear and a static shared key. The IV space is so small that it repeats within hours on a busy network, and weaknesses in the way RC4 consumed the IV let an attacker recover the key from a few tens of thousands of packets, in minutes. Its integrity check was a plain CRC-32, which is linear and cannot detect deliberate edits. WEP is broken and must not be deployed. *WPA* (2003) was a stop-gap that kept the old hardware by wrapping RC4 in TKIP, with a per-packet key and a message integrity code; TKIP is now also deprecated. The two designs that matter are WPA2 and WPA3.

WPA2-Personal: from passphrase to session keys

WPA2 (the RSN of IEEE 802.11i, 2004) encrypts with *CCMP*, which is AES in counter mode for confidentiality and CBC-MAC for integrity. Keys are organised in a hierarchy, shown in Figure 6.10. At the top is the *pairwise master key* (PMK), a secret shared by the station and the AP. In WPA2-Personal the PMK is computed from the passphrase and the network name:

PMK = PBKDF2-HMAC-SHA1(passphrase, SSID, 4096 iterations, 256 bits).

The 4,096 iterations slow down a guesser, and including the SSID makes precomputed tables specific to one network name. The PMK is never transmitted. From it, each association derives a fresh *pairwise transient key* (PTK) by a pseudo-random function, the PRF built on HMAC-SHA1:

PTK = PRF(PMK, “Pairwise key expansion”, min(AA, SPA) ‖ max(AA, SPA) ‖ min(ANonce, SNonce) ‖ max(ANonce, SNonce)),

where AA is the authenticator’s (AP’s) address, SPA the supplicant’s (station’s) address, and the two nonces are random 256-bit numbers chosen by each side for this attempt. For CCMP the output is 384 bits, split into three 128-bit keys: the *key confirmation key* (KCK), which authenticates the handshake messages with a MIC; the *key encryption key* (KEK), which wraps the group key; and the *temporal key* (TK), which encrypts and authenticates the unicast data.

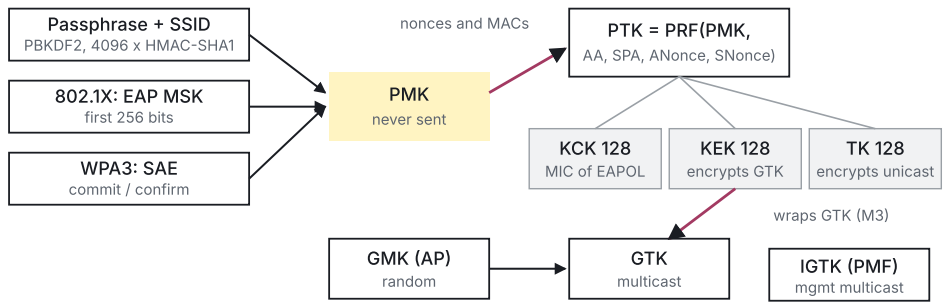


Figure 6.10 The key hierarchy: three ways to obtain a PMK, one PTK per station, and group keys chosen by the AP.

6. An association ID (AID) from 1 to 2007.↩

Why both nonces and both addresses are inputs. The nonces make the PTK fresh: each side contributes randomness, so neither the AP nor an attacker replaying an old message can force the reuse of an earlier key. The addresses bind the key to this pair of devices, so a PTK cannot be moved to another association. Taking the minimum and maximum of each pair is a convention that lets both sides concatenate in the same order without knowing who is who.

The group key. Broadcast and multicast frames go to many stations, so they need a key all share. The AP chooses a *group temporal key* (GTK), derives it from a random group master key and its address, and delivers it, encrypted under the KEK, inside message 3 of the handshake. The GTK is changed when a station leaves the network (so that it can no longer decrypt group traffic) and periodically, by a short two-message group-key handshake.

The 4-way handshake

The handshake has two aims: to prove to each side that the other holds the PMK, and to install the PTK and GTK. It is carried in *EAPOL-Key* frames and runs after association (Figure 6.11).

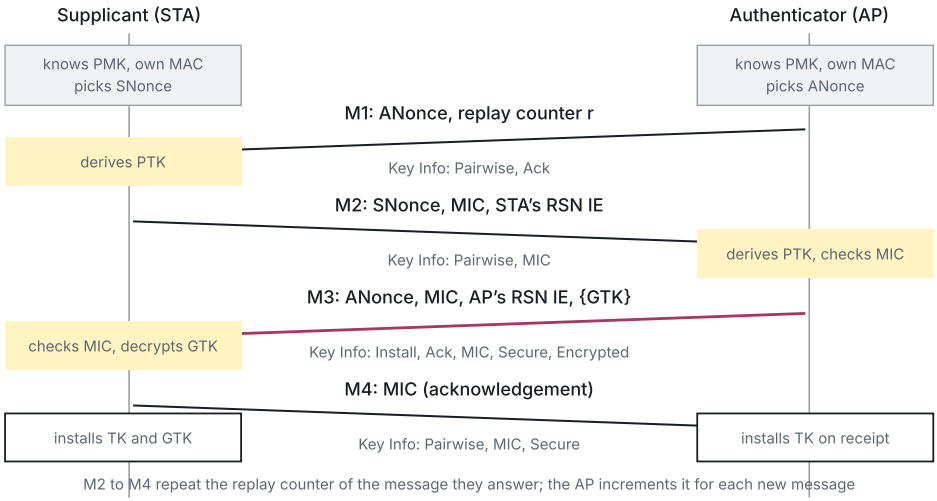


Figure 6.11 The 4-way handshake, with the key-information flags and the moment each side can derive and install keys.

1. **Message 1 (AP → station).** The AP sends its ANonce and a replay counter. The station can now compute the PTK, because it has the PMK, both addresses, its own SNonce and the ANonce.
2. **Message 2 (station → AP).** The station sends its SNonce, its RSN element and a MIC computed with the KCK over the whole message. The AP computes the PTK with the SNonce and verifies the MIC. A wrong passphrase gives a different PMK, hence a different KCK, hence a failed MIC, and the AP drops the exchange; this is the source of reason code 15.
3. **Message 3 (AP → station).** The AP sends the ANonce again, its own RSN element (which the station compares with the beacon, to catch tampering), the GTK encrypted with the KEK, and a MIC. The *Install* flag tells the station to install the TK.
4. **Message 4 (station → AP).** The station acknowledges with a MIC. Both sides now install the TK and the controlled port opens.

The *replay counter* is a number in every EAPOL-Key frame. The AP increases it for each new message, and the station answers with the counter of the message being answered; a receiver discards a frame whose counter is not higher than the last one accepted. It prevents an attacker from replaying captured handshake messages.

KRACK. In 2017 a key reinstallation attack showed a flaw in the way implementations handled message 3. If message 4 is blocked, the AP resends message 3, with a new, higher replay counter and therefore valid. A station that follows the standard literally reinstalls the same TK on receiving it and resets its packet number (the CCMP nonce) to zero. A nonce is then used twice under one key, so an attacker can replay frames or, with a stream-style cipher, recover the keystream and decrypt. The counter does not help, because the retransmitted message is genuine. The remedy is a software change: install a key only once, and ignore the retransmission.

Why WPA2-PSK is vulnerable to an offline dictionary attack. An attacker who captures messages 1 and 2 knows the nonces and both addresses; to test a guessed passphrase he computes the PMK, the PTK and the KCK and checks whether the MIC in message 2 matches. That test requires no interaction with the network, so billions of guesses per second on graphics hardware are possible and weak passphrases fall in hours. A related shortcut reads the PMKID (an HMAC of the PMK and the two addresses) from the first message alone. The defence is a long random passphrase, or no passphrase at all, with 802.1X or SAE.

LAB

`LAB 6.7 · 20 MIN` Derive a PMK and a PTK in Python and see which parts depend on the nonces. The nonces and addresses below are synthetic; in Lab 6-L5 they come from a capture.

```
import hashlib, hmac
def prf(key, label, data, bits):
    out, i = b"", 0
    while len(out) * 8 < bits:
        msg = label + b"\x00" + data + bytes([i])
        out += hmac.new(key, msg, hashlib.sha1).digest()
        i += 1
    return out[:bits // 8]
pmk = hashlib.pbkdf2_hmac("sha1", b"packet123", b"IITLab", 4096, 32)
aa, spa = bytes.fromhex("0211223344aa"), bytes.fromhex("0211223344bb")
an, sn = bytes(range(0x00, 0x20)), bytes(range(0x80, 0xA0))
data = min(aa, spa) + max(aa, spa) + min(an, sn) + max(an, sn)
ptk = prf(pmk, b"Pairwise key expansion", data, 384)
print("PMK", pmk[:8].hex())
print("KCK", ptk[:16].hex())
print("TK ", ptk[32:48].hex())
```

```
PMK f475f5c28b6c9ae2
KCK 274c66d2edb317fe68a08d7c79f78756
TK  5f62b371f013c07535625e7a4e777879
```

What you should notice: change one nonce byte and the KCK and TK change completely, while the PMK stays the same, since it depends only on the passphrase and SSID. In a real capture the same KCK, applied with HMAC-SHA1 (truncated to 16 bytes) to the EAPOL frame of message 2 with its MIC field zeroed, reproduces the MIC in the frame; that match proves the passphrase.

WPA2-Enterprise

In an enterprise network each user has individual credentials, and the PMK comes from an authentication server instead of a passphrase. Three roles take part: the *supplicant* (the station), the *authenticator* (the AP, which blocks all traffic except EAPOL until told otherwise), and the *authentication server* (a RADIUS server). The generic IEEE 802.1X and EAP exchange is taught in

ide EAPOL frames, and between AP and server inside RADIUS Access-Request and Access-Challenge packets. On success the server sends Access-Accept together with the *master session key* (MSK); the first 256 bits of the MSK are the PMK, which the AP then uses in the same 4-way handshake as above. The AP never learns the user's credentials, and each user has a different PMK, so a leaked key exposes one session only.

Two EAP methods matter in interviews. *EAP-TLS* authenticates both ends with certificates, so every client needs one; it is the strongest and the costliest to deploy. *PEAP* builds a TLS tunnel authenticated by the server's certificate alone and runs a password method (MSCHAPv2) inside it; it needs only a server certificate. The common deployment mistake is a client that does not validate the server certificate, which lets a rogue AP harvest password hashes.

WPA3

WPA3 (2018) changes the personal mode and tightens the rest. For the personal mode, *simultaneous authentication of equals* (SAE, a form of the Dragonfly exchange) replaces the PSK derivation. Both sides map the password to a point on an elliptic curve, each picks a random secret, and they swap a *commit* (a scalar and a curve element) and then a *confirm*, in authentication frames with algorithm 3. The result is a PMK that is unique to each session and does not follow from the transcript, with two benefits. A captured exchange offers an attacker no value to test a guess against, so each guess needs a live interaction with the network, which the AP can rate-limit; this ends the offline dictionary attack. And a later compromise of the password does not reveal earlier sessions, a property called *forward secrecy*. The 4-way handshake still follows, as before, to derive the PTK from the SAE-derived PMK. The *hash-to-element* variant avoids timing leaks found in the original password-mapping loop, which were published as the "Dragonblood" weaknesses.

WPA3 also makes *protected management frames* mandatory, offers a 192-bit enterprise suite (GCMP-256 with 384-bit hash functions), and adds *Opportunistic Wireless Encryption* (OWE) for open networks: a Diffie–Hellman exchange at association gives each client its own key with no password, which stops passive eavesdropping, though not an impostor AP.

Transition mode. A network that advertises both WPA2-PSK and WPA3-SAE with the same passphrase lets old devices connect, but also lets an attacker run a rogue WPA2-only AP and force a downgrade, so strong protection exists only when WPA3 alone is configured.

Protected management frames (802.11w)

Before 802.11w the management frames, including deauthentication and disassociation, were unauthenticated. Anyone could forge a deauthentication frame from the AP and eject a station, and repeat it indefinitely. *Protected management frames* (PMF) authenticate robust management frames: unicast ones are encrypted with the TK, and group-addressed ones carry an integrity code made with a separate *integrity group temporal key* (IGTK) delivered in message 3. A station that receives a forged deauthentication simply discards it. If a station has lost its keys and the AP sends an unprotected disassociation, the SA Query exchange verifies that the request is genuine. PMF can be "capable" or "required" in the RSN element; WPA3 and 6 GHz operation require it. PMF does not protect beacons or the radio layer, so jamming remains possible.

	WEP	WPA	WPA2	WPA3
Year	1997	2003	2004	2018
Cipher	RC4, 24-bit IV	RC4 with TKIP	AES-CCMP	AES-CCMP (GCMP-256 for 192-bit mode)

Integrity	CRC-32	Michael MIC	CCM MIC (8 B)	CCM or GCM
Personal key setup	static key	PSK, PBKDF2	PSK, PBKDF2	SAE (Dragonfly)
Enterprise	none	802.1X	802.1X	802.1X, 192-bit option
Management-frame protection	none	none	optional (802.11w)	mandatory
Offline dictionary attack	n/a	yes	yes	no
Forward secrecy	no	no	no	yes (SAE)
Status	broken	deprecated	baseline	current

TRAP

Claim: “The PSK is sent during the 4-way handshake”, or “WPA3 makes weak passwords safe.” *Why it is wrong:* neither the PSK nor the PMK is ever sent; only nonces, MICs and the encrypted GTK are. SAE removes the offline attack but cannot prevent online guessing, which the AP must still limit. *Correct statement:* the handshake proves possession of the PMK without transmitting it; WPA3 raises the cost of a guess from computation to interaction.

INTERVIEW LENS

QUALCOMM · CISCO · JUNIPER · BROADCOM

“What is the 4-way handshake for, and what keys result?” and “how is the PTK derived and why are the nonces included?” are Qualcomm staples, and the follow-ups on KRACK and SAE are common at Cisco and Juniper. A strong answer draws the key hierarchy, states what each of KCK, KEK and TK does, says what each message proves, and explains why the pass-phrase does not cross the air. Follow-ups: PMKID caching, what changes under 802.1X, and the downgrade in transition mode.

CHECK YOURSELF

1. Which key authenticates the handshake messages, and which encrypts the GTK? [1]
2. What stops a forged deauthentication frame under PMF? [2]
3. Under 802.1X, from what is the PMK derived? [3]

6.8 QoS, aggregation and power save SHOULD · ■■□

The DCF of §6.4 (p. 366) treats all frames alike, so a bulk download and a voice call compete equally. Three mechanisms refine it: priority access (EDCA), aggregation to cut overhead, and sleep scheduling.

EDCA and access categories

Enhanced distributed channel access (EDCA), the QoS part of the *Wi-Fi Multimedia* (WMM) profile, keeps contention but changes the parameters. Each station has four queues, the *access categories* (AC), each running its own backoff with its own *arbitration interframe space* (AIFS = SIFS + AIFSN × slot), its own contention window and a transmission opportunity limit. A higher priority means a shorter wait and a smaller window, so voice wins the medium more often, though never with certainty. The default parameters for a non-AP station in 5 GHz are as follows.

Access category	Typical use	AIFSN	CWmin	CWmax	TXOP limit	Mean first wait
AC_VO	voice	2	3	7	1.504 ms	47.5 µs

AC_VI	video	2	7	15	3.008 ms	65.5 µs
AC_BE	best effort	3	15	1023	none	110.5 µs
AC_BK	background	7	15	1023	none	146.5 µs

The last column is SIFS + AIFSN × 9 µs + (CWmin ÷ 2) × 9 µs; for voice, 16 + 18 + 13.5 = 47.5 µs, and for best effort, 16 + 27 + 67.5 = 110.5 µs, so voice starts on average 63 µs earlier. A *TXOP* lets the winner send several frames back to back, separated by SIFS, for up to the limit. If two queues in the same station count down to zero in the same slot, the higher AC gets the medium and the lower one behaves as if it had collided: an *internal collision*.

A frame's category comes from its *user priority* (UP, 0 to 7), carried in the QoS Control field and taken from the IP header's DSCP. UP 1 and 2 map to background, 0 and 3 to best effort, 4 and 5 to video, and 6 and 7 to voice. A trap here is the mapping of Expedited Forwarding (DSCP 46): the older practice of copying the top three bits of the DSCP gives UP 5, which is video; the current recommendation (RFC 8325) maps it to UP 6, which is voice.

Aggregation and the block acknowledgement

The per-frame overhead seen in the efficiency example of §6.4 (p. 366) is independent of the data rate, so the solution is to send more data per access. *Aggregation* does it at two levels. An *A-MSDU* packs several packets into one MAC frame with one header and one FCS, saving header bytes. An *A-MPDU* packs several complete MAC frames, each with its own header, delimiter and FCS, into one PHY transmission.

	A-MSDU	A-MPDU
What is aggregated	packets (MSDUs) in one MPDU	MAC frames (MPDUs) in one PPDU
Headers and FCS	one header, one FCS	one per subframe, plus a 4-byte delimiter
Maximum size	3,839 or 7,935 B (HT), 11,454 B (VHT, HE)	64 KB (HT), 1 MB (VHT), 4 MB (HE)
On a bit error	the whole A-MSDU is lost	only the damaged subframe is repeated
Acknowledgement	one ACK or part of a Block Ack	one Block Ack for all subframes

An A-MPDU survives a noisy channel better, because each subframe has its own FCS and only the failed ones are retransmitted; an A-MSDU has a single FCS and is lost entirely. Practical systems put A-MSDUs inside an A-MPDU.

Acknowledging each subframe separately would defeat the purpose. The *Block Ack* is a single reply, with a bitmap of which subframes in a window arrived: 64 frames in 802.11n and ac, 256 in 802.11ax, 1,024 in 802.11be. The agreement is set up with an ADDBA action frame exchange for each traffic class, after which the sender transmits the aggregate and the receiver replies with a Block Ack, and the sender re-sends only the missing sequence numbers.

WORKED EXAMPLE

Airtime saved by aggregation. A station sends 64 packets of 1,500 B at 600 Mb/s PHY rate, with the overheads of §6.4 (p. 366): DIFS 34 µs, average backoff 67.5 µs, SIFS 16 µs, preamble 50 µs, ACK or Block Ack about 32 µs.

1. Separately: each packet needs 34 + 67.5 + (50 + 20.5) + 16 + 32 ≈ 220 µs, so 64 packets take 64 × 220 = 14.1 ms, a goodput of 64 × 12,000 bits ÷ 14.1 ms ≈ 55 Mb/s.
2. As one A-MPDU: contention 34 + 67.5, preamble 50, data 64 × 20.5 = 1,312 plus about 3 µs of delimiters, SIFS 16 and one Block Ack 32: total ≈ 1.5 ms.
3. Goodput = 768,000 bits ÷ 1.5 ms ≈ 507 Mb/s.

Result: about 9 times the goodput from the same PHY rate. At high rates, aggregation is mandatory, not an option.

Power save

A phone cannot keep its radio on all the time. A station tells the AP that it is about to sleep by setting the Power Management bit in the Frame Control of a frame (often a Null frame), and from then on the AP buffers its frames. The AP lists the stations with waiting frames in the *traffic indication map* (TIM) element of each beacon, one bit per association ID. A sleeping station wakes for selected beacons, in multiples of its listen interval, reads the TIM, and if its bit is set it fetches the data: in legacy mode it sends a PS-Poll and the AP replies with one frame, setting the More Data bit if more remain; with *U-APSD*, designed for voice, the station's own uplink frame triggers the delivery of all buffered frames at once. Broadcast and multicast frames are buffered until the *delivery TIM* (DTIM) beacon, which occurs every DTIM-period beacons; with a beacon interval of 102.4 ms and a DTIM period of 3, a station wishing to receive multicast wakes every 307 ms, and a longer period saves power at the cost of multicast latency.

Target wake time (TWT), introduced in 802.11ax and described in §6.9 (p. 384), replaces this polling with a negotiated schedule: the station and AP agree on when the station will wake, so that it need not listen to every beacon, and many stations can be told to wake at different times, reducing contention.

INTERVIEW LENS

QUALCOMM · BROADCOM · CISCO

"Explain EDCA access categories and how voice gets priority", "what are TIM and DTIM", and "A-MPDU versus A-MSDU" are reported for WLAN firmware roles. A strong answer gives AIFSN and CW for voice and best effort, computes the average first wait, and says priority is statistical. For aggregation, explain the single-FCS risk of A-MSDU; for power save, trace a buffered frame from the TIM bit to the PS-Poll. Follow-ups: how much latency a DTIM period of 3 adds, and what the TXOP limit prevents.

CHECK YOURSELF

1. Which AC does DSCP 46 reach under RFC 8325? [4]
2. Why does a single bit error cost more in an A-MSDU? [5]
3. What does a beacon with a set TIM bit mean for a sleeping station? [6]

1. The KCK authenticates (the MIC); the KEK encrypts the GTK in message 3.↩
2. The frame must carry a valid integrity check or be encrypted under keys the attacker lacks, so the station discards it.↩

6.9 Wi-Fi 6, 6E and Wi-Fi 7 MUST · ■■■

Earlier generations raised the peak rate of one client. By 2015 the limit was no longer the peak but the crowd: an apartment block or a stadium has hundreds of devices, each sending small packets, and each packet pays the contention overhead of §6.4 (p. 366). IEEE 802.11ax turns Wi-Fi from "one talker at a time" into scheduled access in which the AP can orchestrate many stations in one transmission. 802.11be adds more bandwidth and the ability to use several links at once.

OFDMA and resource units

In *OFDMA* the subcarriers of a channel are divided into *resource units* (RUs) of 26, 52, 106, 242, 484 or 996 tones (a tone is one subcarrier, data or pilot), and each RU is assigned to a different station within one transmission (Figure 6.12). Previously a station occupying the channel for 100 bytes wasted the other 99% of the subcarriers' capacity for that symbol; now several stations share them. The number of RUs by width is as follows.

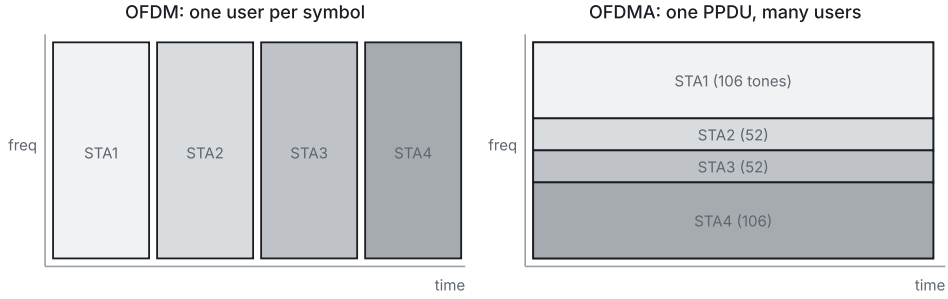


Figure 6.12 Time-frequency grid: OFDM gives the whole channel to one station at a time; OFDMA splits it into resource units shared within a single transmission.

Channel	26-tone	52-tone	106-tone	242-tone	484-tone	996-tone
20 MHz	9	4	2	1		
40 MHz	18	8	4	2	1	
80 MHz	37	16	8	4	2	1
160 MHz	74	32	16	8	4	2

The AP decides the allocation. For the downlink it sends one PPDU with the allocation in its preamble. For the uplink, which needs the stations to start together and to end together, it first sends a *trigger frame* that lists, for each station, the RU, the MCS, the power target and the length. After a SIFS the stations send their *HE trigger-based PPDUs* simultaneously, padded to equal duration, and the AP answers with one *multi-station Block Ack*. A station without an assignment may use *uplink OFDMA random access* (UORA), a backoff over RUs reserved for contention.

3. The first 256 bits of the MSK, delivered to the AP by the RADIUS server.↩
4. User priority 6, hence AC_VO.↩
5. One FCS covers the whole A-MSDU, so the whole aggregate must be repeated; in an A-MPDU only the damaged subframe is.↩
6. The AP has buffered unicast frames for it; it should stay awake and retrieve them (PS-Poll or U-APSD trigger).↩

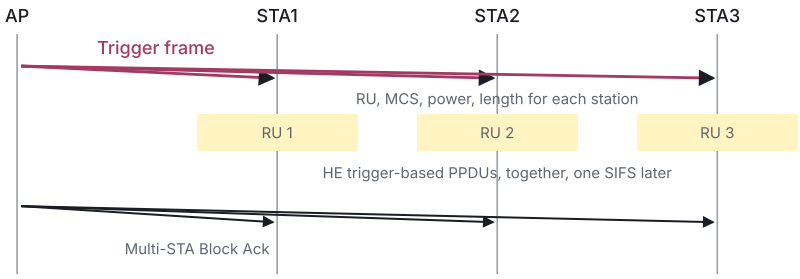


Figure 6.13 Uplink OFDMA: the AP schedules, the stations answer in parallel on their own resource units, and one acknowledgement closes the exchange.

WORKED EXAMPLE

Nine sensors in one 20 MHz channel. Nine IoT sensors each have 100 bytes to send. Compare nine separate accesses with one trigger-based OFDMA exchange, with the 5 GHz timings of §6.4 (p. 366) (DIFS 34 µs, average backoff 67.5 µs, SIFS 16 µs) and these assumptions: a single-user transmission uses a 44 µs preamble and MCS 7, one data symbol (13.6 µs); the ACK takes 32 µs; the trigger frame takes 70 µs; each sensor uses a 26-tone RU (24 data tones) at MCS 5 (64-QAM, 2/3), whose rate is 24 × 6 × 2/3 ÷ 13.6 µs ≈ 7 Mb/s; and the multi-station Block Ack takes 60 µs.

1. Separate accesses: 34 + 67.5 + 44 + 13.6 + 16 + 32 ≈ 207 µs each, so nine take about 1.86 ms, ignoring the collisions that nine contenders would cause.
2. OFDMA: one contention (34 + 67.5), the trigger (70), SIFS (16), the uplink PPDU with 56 µs of preamble and (816 + 22 bits) ÷ 96 bits per symbol = 9 symbols = 122 µs of data, SIFS (16) and the Block Ack (60): about 0.44 ms.

Result: about four times less airtime for the same nine packets, plus no collisions among the nine. The gain is airtime and latency for many small flows; the single-client peak rate does not change.

MU-MIMO, 1024-QAM, BSS colouring and TWT

802.11ac had downlink MU-MIMO only. 802.11ax adds *uplink* MU-MIMO, in which several stations transmit on the same RU and the AP separates them with its antennas, and allows MU-MIMO inside RUs of 106 tones or more, so OFDMA and MU-MIMO can be combined. 1024-QAM (10 bits per symbol) gives 25% more than 256-QAM, but only at a SNR above about 35 dB.

BSS colouring addresses the exposed-node problem between neighbouring cells. Every HE frame carries a 6-bit *colour* of its BSS in the PHY header. A station that detects a frame with a different colour knows that it belongs to an overlapping BSS (OBSS). If the frame is weaker than the *OBSS packet-detect threshold* (the default is −82 dBm and it may be raised up to −62 dBm), the station may ignore that frame, regard the medium as idle and transmit, with its power reduced by as many decibels as the threshold was raised. This is *spatial reuse*. In an example, a frame from another BSS arrives at −78 dBm with a threshold of −72 dBm: the station continues its backoff as though the frame were absent. A frame of the same colour must be respected, which is why a colour collision with a neighbour disables the benefit. The standard keeps two NAVs, one for frames of the own BSS and one for others, so that ignoring a foreign reservation does not affect the own cell.

Target wake time (TWT), mentioned in §6.8 (p. 381), is a negotiated agreement: the station and AP exchange TWT elements that fix a first wake time, a wake interval (a mantissa multiplied by

power of two, in microseconds) and a minimum wake duration. Between its service periods the station can turn its radio off, which suits battery-powered sensors and phones. Because stations wake in separate service periods, contention is also lower. In *broadcast TWT* the AP announces schedules to groups of stations, so no individual negotiation is needed.

Wi-Fi 6E and the 6 GHz band

Wi-Fi 6E is 802.11ax in the 6 GHz band (5,925 to 7,125 MHz in the United States), which adds 1,200 MHz of spectrum, more than the 2.4 and 5 GHz bands combined. The band holds 59 channels of 20 MHz, 29 of 40, 14 of 80 and 7 of 160 MHz, which make wide channels practical; Wi-Fi 7 gets 3 channels of 320 MHz (Figure 6.15). Since no legacy device uses the band, there are no 802.11a/b/g/n/ac clients, no beacons at old rates and no need for protection. Access requires WPA3 or OWE. Indoor low-power access points work without coordination; standard-power outdoor use depends on an automated frequency coordination database that protects incumbent microwave links. Regulators differ by country; India has opened its lower 6 GHz range for indoor low-power use, and the current rules should be checked at the time of reading.

Wi-Fi 7 (802.11be)

802.11be, “Extremely High Throughput”, combines four multipliers: 320 MHz channels (2× over 160 MHz), 4096-QAM (12 bits, 1.2× over 1024-QAM), 16 spatial streams in the standard (2× over 8) and *multi-link operation*, giving the peak rate of 46.1 Gb/s computed in §6.2 (p. 361). Two more features matter in dense networks. *Multi-RU* allows a station to be given several RUs at once, which avoids wasting capacity when the channel is partly occupied. *Preamble puncturing* lets an 80, 160 or 320 MHz transmission skip a 20 MHz subchannel that is busy or occupied by radar, so a single interferer no longer cuts the channel to the width of its clean part.

Multi-link operation (MLO) lets one device use several radios, for example on 5 and 6 GHz, as a single logical device. A *multi-link device* (MLD) has one MAC address that upper layers see and one affiliated station per link, each with its own link address; the stations share one sequence-number space and a Block Ack scoreboard, so frames of one flow may travel on any link. The result is higher throughput (the links add) and lower latency (a frame can use whichever link wins contention first). The mode depends on how well the radios isolate each other.

Mode	Behaviour	Hardware cost	A firmware challenge
STR (simultaneous transmit and receive)	links work independently, one may send while another receives	filters and enough frequency separation; most demanding	in-device interference and independent EDCA and NAV state on each link
NSTR (non-simultaneous)	links are close, so transmitting on one blocks receiving on the other	lower	aligning the end of transmissions on both links, otherwise the sender’s own signal deafens its other link
EMLSR (enhanced multi-link single radio)	one full radio listens on several links with minimal circuits, then switches to the link that receives an initial control frame	lowest; a single radio chain	the switching delay, which must be known to the AP, and the frames lost during it

Another firmware task is the mapping of traffic identifiers to links (for example, voice on the less loaded link), and recovery when a link is lost.

Feature	Wi-Fi 5 (ac)	Wi-Fi 6 (ax)	Wi-Fi 6E	Wi-Fi 7 (be)
Bands	5 GHz	2.4, 5	adds 6 GHz	2.4, 5, 6
Widest channel	160 MHz	160 MHz	160 MHz	320 MHz
QAM	256	1024	1024	4096
OFDMA	no	yes	yes	yes, with multi-RU
MU-MIMO	DL	DL, UL	DL, UL	DL, UL
BSS colour, TWT	no	yes	yes	yes, plus restricted TWT
Multi-link, puncturing	no	no	no	yes

TRAP

Claim: "OFDMA increases the peak speed", or "Wi-Fi 6E is a new standard." *Why it is wrong:* the peak rate of one client comes from width, QAM order and streams; OFDMA improves efficiency and latency when many clients have small amounts of data. 6E is a regulatory and band extension of 802.11ax, with no new PHY. *Correct statement:* OFDMA is about sharing, and 6E is ax in 6 GHz.

INTERVIEW LENS

QUALCOMM · BROADCOM · CISCO · JUNIPER

Qualcomm WLAN postings list MU-MIMO, OFDMA, TWT and rate control, and the reported questions are "OFDM versus OFDMA", "what is a trigger frame", "what is BSS colouring", "list four Wi-Fi 7 features" and "explain MLO modes and a firmware challenge of each". Expect also "how would the firmware schedule RUs across 20 stations with mixed QoS?" (solved in the Problem Set). A strong answer says what is shared (frequency for OFDMA, space for MU-MIMO), draws the trigger sequence, and links each feature to its benefit: airtime, latency, power or throughput.

CHECK YOURSELF

1. How many 26-tone RUs fit in 80 MHz? [1]
2. What does a station do with a frame of another BSS received below the OBSS threshold? [2]
3. By what factors do 320 MHz, 4096-QAM and 16 streams raise the Wi-Fi 6 peak? [3]

6.10 Roaming and enterprise design SHOULD · ■■□

A station that walks from one AP's coverage into another's must change its association without dropping the call. Three facts shape the problem. The *client* decides when and where to roam; the network can only advise, and many clients roam late, remaining attached to a distant AP while a closer one is available (the *sticky client* problem). Roaming consists of a scan, a re-authentication and a (re)association, and each step takes time. And a voice call tolerates a gap of about 50 ms, whereas a full 802.1X authentication takes several round trips to a RADIUS server.

Three amendments help, each aiming at one of these steps.

- **802.11k (radio resource measurement)** supplies information. The AP gives the client a *neighbour report*, a list of nearby APs with their channels, so the client scans only those channels instead of all of them, and can also request beacon reports and load measurements.
- **802.11v (wireless network management)** supplies a suggestion. The AP can send a *BSS transition management request* to a client, naming better candidate APs, or warning that it

will disassociate soon; the client may decline.

- **802.11r (fast BSS transition, FT)** supplies speed. It adds a key level, so that the keys for the next AP can be prepared in advance. The first association derives a PMK-R0 from the master key and the mobility domain; each AP of the domain holds a PMK-R1 derived from it; when the client roams, it derives a fresh PTK for the new AP from the PMK-R1 with new nonces, in the reassociation exchange itself, without a new 802.1X authentication and without a separate 4-way handshake. FT can run *over the air* (two authentication frames, then reassociation) or *over the DS* (the messages are relayed through the old AP).

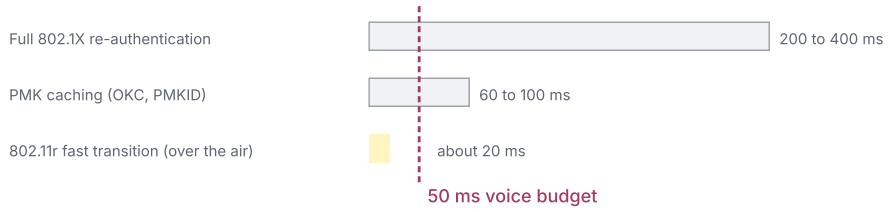


Figure 6.14 Illustrative time to move between access points under three key-management schemes.

WORKED EXAMPLE

Why 802.11r meets the voice budget. Full re-authentication needs about four RADIUS round trips at 20 ms each plus server processing, and a four-message handshake: roughly 200 to 400 ms, well beyond the 50 ms budget. With FT over the air, the exchange is two authentication frames and two reassociation frames, at 1 to 5 ms each including contention, that is about 20 ms. Key caching without 802.11r (PMKID or opportunistic key caching) skips the RADIUS exchange but not the handshake: about 60 to 100 ms.

Result: only 802.11r reliably meets the voice budget. The figures are illustrative; the shape of the comparison is what an interview expects.

A network design must also decide *where* the clients roam. Practical rules follow from the earlier sections.

Channel planning in 2.4 GHz. The band has channels 1 to 13 (14 in Japan only), spaced 5 MHz apart but each about 20 MHz wide. Two channels do not overlap if they are at least 25 MHz apart, so only 1, 6 and 11 can be used together (Figure 6.15). Using channel 3 or 8 overlaps two others partially; the overlapped signal looks like noise rather than a decodable frame, so carrier sense reacts to it unreliably and frames are corrupted, whereas co-channel neighbours at least defer to one another and merely share the airtime. Hence the rule “1, 6, 11 or nothing”.

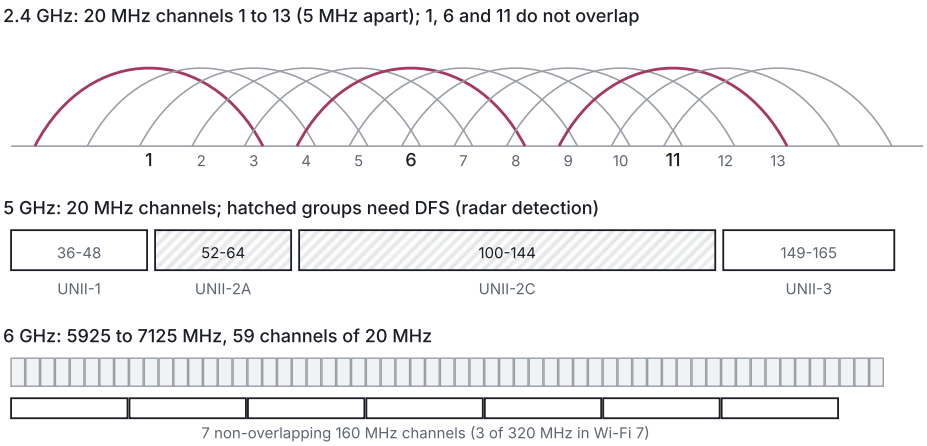


Figure 6.15 Channel maps for the three Wi-Fi bands (US-style numbering; regulatory limits differ by country).

Channel planning in 5 and 6 GHz. There are about 25 channels of 20 MHz in 5 GHz, of which those in the UNII-2A and 2C ranges require *dynamic frequency selection* (DFS): the AP listens for radar for about a minute before using the channel, and must leave within seconds if it detects any, avoiding it for 30 minutes. With 80 MHz channels only five or six exist, which in a dense deployment forces neighbouring APs onto the same channel. A good design uses 20 or 40 MHz channels, a reuse pattern that places the same channel at least two cells apart in every direction, including the floors above and below, and spreads transmit power so that cells overlap by about 15 to 20% for roaming.

Cell size and data rates. An AP can transmit at 20 dBm but a phone only at about 14 dBm, so if the AP's power is not lowered the client hears the AP but cannot answer. Setting the AP power close to the client's, and disabling the lowest data rates (1, 2, 5.5 and 11 Mb/s), makes the beacons shorter, removes slow clients that hold the channel for ages, and shrinks the cell so that the client roams sooner. Typical targets are a signal of −67 dBm or better and an SNR of 25 dB for voice and video.

Controllers. In the *split-MAC* design a *lightweight AP* handles the radio and real-time frames, and a *wireless LAN controller* (WLC) handles management, policy and roaming state. The tunnel between them is CAPWAP (UDP ports 5246 for control and 5247 for data). The controller runs *radio resource management*, which selects channels and powers automatically, detects rogue APs, and keeps the keys that make fast roaming possible. In *local switching* (FlexConnect) the AP puts client traffic directly on the local VLAN so a WAN failure does not stop a branch. Cloud-managed systems (Meraki, Mist) move the controller into a cloud service and keep the data plane local.

WORKED EXAMPLE

Channel plan for a three-floor hostel. Each floor has four APs along the corridor, P1 to P4. Use the eight non-DFS 20 MHz channels of 5 GHz (36, 40, 44, 48, 149, 153, 157, 161), keeping four in reserve.

Floor	P1	P2	P3	P4
3	149	157	36	44
2	44	36	157	149
1	36	44	149	157

1. Along every floor the four neighbours use four different channels.
2. Directly above or below, no cell repeats the channel of the cell next to it (compare floors 1 and 2, then 2 and 3, position by position).
3. Keep power at 14 to 17 dBm and the lowest rate at 12 Mb/s, so the floor slabs and walls finish the job of isolating the reuse.

Result: no two adjacent cells, horizontal or vertical, share a channel. With 80 MHz channels the same building has only two non-DFS channels (36 to 48 and 149 to 161), and the plan is impossible.

INTERVIEW LENS

CISCO · JUNIPER · QUALCOMM

Roaming and design questions come from the enterprise vendors, in the form “how does 802.11r speed up roaming and what do k and v add” and “users complain voice calls drop while walking down a corridor; how do you troubleshoot?”. A strong answer separates the client’s decision from the network’s help, and for the second question works through the layers: signal and SNR at the roam point, AP power versus client power, channel overlap, DFS events, sticky clients, 802.11r compatibility and RADIUS delay. Troubleshooting method in general is the subject of Pillar 11 (p. 634).

CHECK YOURSELF

1. Which amendment supplies a neighbour list? [4]
2. Why must 2.4 GHz plans use 1, 6 and 11? [5]
3. What does DFS require an AP to do? [6]

6.11 The Linux and firmware view SHOULD · ■■□

Everything so far happens in a few microseconds: the acknowledgement of a frame must start 16 µs after the frame ends, the backoff counts in 9 µs slots, and a block-ack bitmap must be built before the sender’s timeout. A general-purpose kernel, whose interrupt latency is of the order of hundreds of microseconds, cannot do this. Hence a Wi-Fi chip contains a small processor with its own *firmware*, and the part of the MAC that is time-critical lives there, while the host runs the policy. This division is the subject of Qualcomm and Broadcom WLAN roles, which combine this chapter with the interrupts, DMA rings and bit manipulation of Pillar 7 (p. 414).

The Linux wireless stack

Linux splits the work into layers, each with a defined interface (Figure 6.16).

1. 37. ↩
2. It ignores the frame for carrier sensing and may transmit, at reduced power (spatial reuse). ↩
3. 2, 1.2 and 2, a total of 4.8 (9.6 to 46.1 Gb/s). ↩
4. 802.11k (the neighbour report). ↩
5. They are the only three 20 MHz channels at least 25 MHz apart, so they do not overlap. ↩
6. Listen for radar before using a channel, and vacate it, for 30 minutes, if radar is detected. ↩

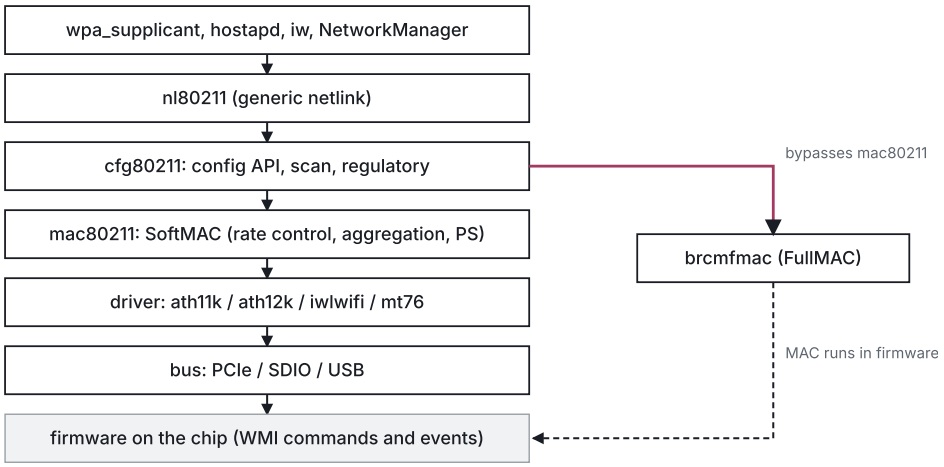


Figure 6.16 The Linux wireless stack, from user space down to the radio, with the FullMAC bypass.

Layer	Role
`wpa_supplicant`, `hostapd`, `iw`, NetworkManager	user-space policy: supplicant and authenticator state machines, the key handshakes, configuration
nl80211	the user-space interface: a generic-netlink family through which user space issues commands (scan, connect, set channel, set type) and receives events
cfg80211	the kernel's configuration layer: validates requests, holds the regulatory domain (which channels and powers are allowed), runs scans and connection management
mac80211	a software MAC framework ("SoftMAC") used by drivers whose hardware leaves the MAC to the host: the state machine, rate control (Minstrel), aggregation sessions, power-save handling, mesh
Driver	`ath9k`, `ath10k`, `ath11k`, `ath12k` (Qualcomm), `iwlwifi` (Intel), `mt76` (MediaTek); programs the chip and moves frames through DMA rings
Bus and firmware	PCIe, SDIO or USB, to the firmware that runs on the chip

The first question is who does the MAC work. In a *SoftMAC* design, the driver implements a thin interface, and mac80211 does most of the MAC, with only timing-critical parts in hardware (older `ath9k`). In a *FullMAC* design the chip's firmware does nearly all of it, and the driver (`brcmfmac`, for Broadcom FullMAC chips) talks to cfg80211 directly, bypassing mac80211. Modern high-rate chips are in between: `ath11k` and `ath12k` register with mac80211 but the firmware does the scheduling, aggregation, retries, rate control and OFDMA, because those decisions must follow each frame by microseconds.

The host and firmware exchange messages through shared-memory rings. Host commands go to the firmware as *WMI* messages (configuration, scan requests, key installation); the firmware answers with events (scan results, connection state, TX completion). The data path uses separate descriptor rings: the host posts empty receive buffers, the firmware writes received frames into them by DMA and raises an interrupt, which a NAPI poll loop services (§7.12 (p. 460)); for transmission the host queues frames and the firmware aggregates, schedules and sends them, then reports completion. The generic firmware concepts, ISRs, bottom halves and DMA, are in §7.13 (p. 463).

What the firmware does. The microsecond tasks are ACK and block-ack generation, backoff and EDCA, aggregation and retry, rate control, OFDMA and MU-MIMO scheduling, beacon generation and the TIM bitmap, hardware encryption, power-save handling and the calibration of the radio. It also performs *offloads* that let the host sleep: answering ARP and neighbour requests, keeping connections alive, filtering packets, scanning for a better AP, and waking the host on a magic packet or a pattern (Wake on WLAN).

Rate adaptation. The transmitter does not know the best MCS; it infers it from acknowledgements. Minstrel, the algorithm in mac80211, keeps for each rate an exponentially weighted success probability and estimates throughput as that probability multiplied by the rate; it sends most frames at the best estimate and about 10% as probes at other rates, and falls back along a retry chain of lower rates when a frame fails. Firmware implementations use richer inputs, such as per-frame SNR feedback.

> LAB
>
> `LAB 6.11 · 20 MIN` Inspect a wireless interface. The output is typical and abridged; yours will differ.
>
> ```
> $ iw dev wlan0 link
> Connected to 02:11:22:33:44:aa (on wlan0)
>         SSID: IITLab
>         freq: 5180
>         signal: -58 dBm
>         rx bitrate: 866.7 MBit/s VHT-MCS 9 80MHz short GI VHT-NSS 2
>         tx bitrate: 780.0 MBit/s VHT-MCS 8 80MHz short GI VHT-NSS 2
> $ iw dev wlan0 station dump | grep -E 'signal|retries|failed'
>         signal:         -58 dBm
>         tx retries:     1042
>         tx failed:      3
> $ iw phy phy0 info | grep -E 'monitor|HE Iftypes|VHT Capabilities'
> ```
>
> **What you should notice:** the bit rate is the formula of §6.2 (p. 361) applied in reverse: VHT-MCS 9 on 80 MHz with 2 streams and the short guard interval is 2 × 234 × 8 × 5/6 ÷ 3.6 µs = 866.7 Mb/s. The two directions differ because each side runs its own rate adaptation. A high count of `tx retries` against a good signal points to interference or hidden nodes, not to range. The third command tells whether the card supports monitor mode and HE.

Monitor mode

A normal (“managed”) interface hands the host only the frames addressed to it, converted to Ethernet. To see the frames of §6.6 (p. 374) one needs *monitor mode*, in which the radio on one channel passes up every 802.11 frame it can decode, management and control frames included, with a *radiotap* header carrying the signal strength, channel and rate. The interface cannot associate while in this mode.

```
$ sudo ip link set wlan0 down
$ sudo iw dev wlan0 set type monitor
$ sudo ip link set wlan0 up
$ sudo iw dev wlan0 set channel 36 80MHz
$ sudo tcpdump -i wlan0 -y IEEE802_11_RADIO -w cap.pcap
```

Wireshark reads the capture and offers the filters in §6.6 (p. 374) (§8.8 (p. 502)); to decrypt the data frames it needs the passphrase and the capture of the station’s 4-way handshake. Only networks one owns or is authorised to test may be captured. Without a monitor-capable adapter, the

which `hostapd` and `wpa_supplicant` can run a complete WPA2 or WPA3 session inside network namespaces, with a capture interface `hwsim0` that shows every frame.

TRAP

Claim: "Monitor mode is the same as promiscuous mode." *Why it is wrong:* promiscuous mode on a managed interface only lifts the filter on destination addresses of frames that the station is already allowed to receive, after decryption and conversion to Ethernet, so other stations' 802.11 frames never reach it. *Correct statement:* monitor mode delivers raw 802.11 frames of all stations on the channel, with radiotap; promiscuous mode does not.

INTERVIEW LENS

QUALCOMM · BROADCOM

WLAN firmware roles at Qualcomm and Broadcom join this section with the C and kernel material of Pillar 7. Reported questions are "what are cfg80211, mac80211 and nl80211", "explain rate adaptation" and, at the low level, bit manipulation, memory management and interrupt handling. A strong answer places each layer on the diagram, explains why the MAC cannot live on the host (the 16 µs SIFS), and names what the firmware offloads. Follow-ups: the difference between FullMAC and SoftMAC, how a host command reaches the firmware, and how a frame travels from the air to a socket.

CHECK YOURSELF

1. What do cfg80211 and mac80211 each do? [1]
2. Why is the ACK generated in hardware or firmware? [2]
3. Which command puts a card on channel 36 in monitor mode? [3]

6.12 Cellular from LTE to 5G SHOULD · ■■□

A cellular network has the same two halves in every generation. The *radio access network* (RAN), a set of base stations, serves the *user equipment* (UE) over a licensed radio channel; the *core network* authenticates the subscriber, assigns an IP address, keeps track of the location, and connects to the Internet. Two planes cross both halves: the *control plane* (signalling about attachment, mobility and sessions) and the *user plane* (the IP packets). The UE and the core exchange *non-access stratum* (NAS) messages, which the base station relays without reading; the UE and base station exchange *access stratum* (AS) messages, the radio resource control (RRC), which set up radio bearers. Unlike Wi-Fi, the base station schedules every transmission, uplink as well as downlink, so there is no contention among UEs.

LTE: the evolved packet system

The LTE architecture, the *evolved packet system* (EPS), has a RAN made of one kind of node and a core, the *evolved packet core* (EPC), made of a few boxes (Figure 6.17).

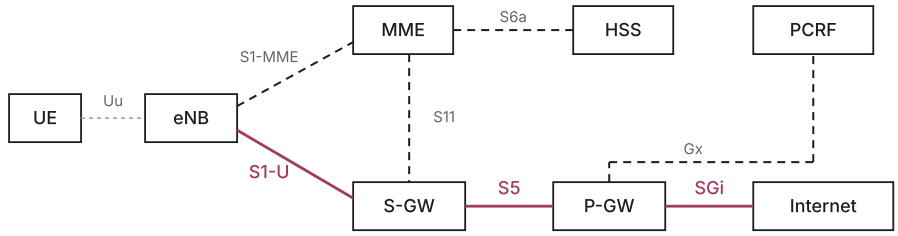


Figure 6.17 The LTE Evolved Packet System: MME and HSS handle signalling, while S-GW and P-GW carry the user plane.

Node	Plane	Function
eNodeB (eNB)	both	the base station: radio scheduling, header compression, ciphering, handover decisions; connects to the MME (S1-MME) and the S-GW (S1-U)
MME (mobility management entity)	control	NAS signalling: attach, authentication, tracking-area updates, paging, bearer set-up; selects the S-GW and P-GW
S-GW (serving gateway)	user	the anchor for the user plane while the UE moves between eNBs
P-GW (PDN gateway)	user	assigns the UE's IP address, applies policy and charging, and links to the Internet (SGi)
HSS (home subscriber server)	control	the subscriber database and the keys; used in authentication (S6a)
PCRF	control	policy and charging rules for the bearers (Gx)

The user's traffic is carried in *bearers*, logical channels with a defined treatment. At attach the network creates a *default EPS bearer*, with best-effort quality (QCI 9) and the UE's IP address, which remains for as long as the UE is attached. A *dedicated bearer* adds a guaranteed bit rate and a low delay for a service such as voice over LTE (QCI 1). An EPS bearer is a chain of three pieces: a radio bearer between UE and eNB, an S1 bearer between eNB and S-GW and an S5 bearer between S-GW and P-GW.

GTP-U. Between the nodes the user's IP packets are carried inside the *GPRS tunnelling protocol, user plane*, a UDP-based tunnel (port 2152). Each packet gets an outer IP header, a UDP header and an 8-byte GTP header whose *tunnel endpoint identifier* (TEID) tells the receiver which bearer the packet belongs to. The reason for tunnelling is mobility: the UE's IP address is anchored in the P-GW, and when the UE moves from one eNB to another only the tunnel's far end changes, while the UE's address stays the same and the routers in between need not know the UE exists. The overhead is 20 + 8 + 8 = 36 bytes with IPv4 (more with extension headers), an instance of the encapsulation theme that returns with VXLAN in Pillar 5 (p. 302).

5G: the new core and radio

5G's *standalone* (SA) architecture replaces the EPC by the *5G core* (5GC) and the eNB by the *gNodeB* (gNB). The design changes in two ways. The control and user planes are fully separated: all user-plane forwarding is in one function, the *UPF*, and the rest is control. And the control-plane functions form a *service-based architecture* (SBA) in which each function offers services to the others over an HTTP/2 interface with JSON bodies and registers itself with a repository,

the NRF; this is how cloud-native software is built, and it lets operators scale and replace the functions independently (Figure 6.18).

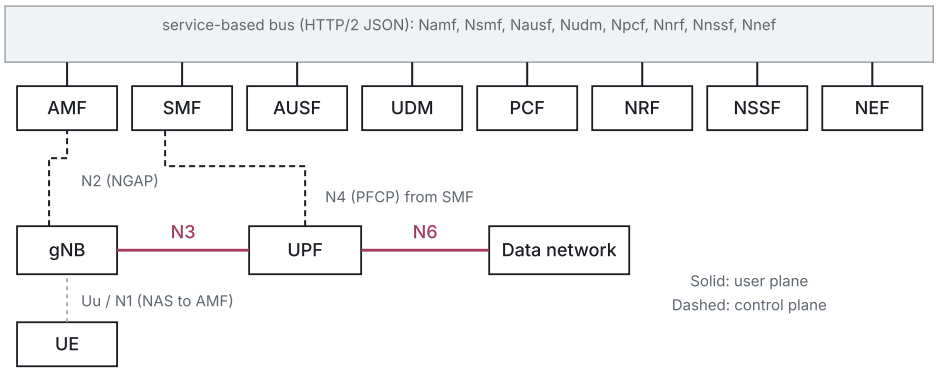


Figure 6.18 The 5G core: control-plane functions on a service bus, and a single user-plane function, the UPF.

5G function	What it does	Closest LTE counterpart
AMF (access and mobility management)	registration, NAS security, mobility, paging; the UE's single point of contact	MME (the mobility half)
SMF (session management)	allocates the UE's IP address, selects and programs the UPF over N4	MME (session half) with the control parts of S-GW and P-GW
UPF (user-plane function)	forwards packets, applies QoS and usage rules, connects to the data network (N6)	S-GW-U and P-GW-U
AUSF and UDM (with UDR)	authentication server and subscriber data	HSS
PCF	policy	PCRF
NRF, NSSF, NEF	service registry and discovery; slice selection; exposure of capabilities to third parties	none

Control-plane interfaces get names such as N1 (UE to AMF, NAS), N2 (gNB to AMF, NGAP), N3 (gNB to UPF, GTP-U), N4 (SMF to UPF, PFCP), N6 (UPF to the data network) and N11 (AMF to SMF). The 5G user plane carries IP in *PDU sessions*, and each session is divided into *QoS flows* that carry a QoS flow identifier (QFI), which the SMF maps to radio bearers; the default flow has the best-effort profile, 5QI 9. An operator can offer *network slices*, separate logical networks identified by a *single network slice selection assistance information* (S-NSSAI), on the same infrastructure.

WORKED EXAMPLE

Registration and PDU session in ten steps (5G SA).

1. The UE finds a cell, performs random access and opens an RRC connection with the gNB; its first NAS message rides along.
2. The NAS *Registration Request* carries the SUCI, the subscriber identity concealed with the home network's public key so that the permanent identity is never sent in clear. The gNB forwards it to an AMF over N2.
3. The AMF asks the AUSF, which asks the UDM, to authenticate the UE (5G-AKA): the network sends a random challenge and an authentication token, and the UE, using the key in its SIM, checks the token (so that it knows the network is genuine) and returns a response.

4. The AMF and UE agree on NAS ciphering and integrity (Security Mode Command); the gNB does the same for the radio (AS security).
5. The AMF sends the *Registration Accept* with a temporary identity (5G-GUTI) and the permitted slices; the UE answers Registration Complete.
6. The UE sends a *PDU Session Establishment Request* (data network name and slice); the AMF forwards it to an SMF.
7. The SMF obtains the subscription from the UDM and policy from the PCF, selects a UPF, and opens an N4 session with it (PFCP rules).
8. The UPF's N3 tunnel endpoint (an address and a TEID) is passed through the AMF to the gNB, which configures the radio bearer and returns its own TEID, so both ends of the N3 GTP-U tunnel are known.
9. The UE receives the *PDU Session Establishment Accept* with its IP address and the QoS rules.
10. User data now flows UE, gNB, N3 tunnel, UPF, N6, on the default QoS flow.

Result: control-plane steps 1 to 9 involve AMF, AUSF/UDM, SMF and PCF; the user plane is touched only by gNB and UPF. The LTE attach has the same shape with the MME, HSS, S-GW and P-GW and ends with the default bearer (QCI 9).

The radio: numerology

LTE used a fixed subcarrier spacing of 15 kHz, with OFDMA in the downlink and SC-FDMA (a power-efficient variant of OFDM) in the uplink, a slot of 0.5 ms and a subframe (the scheduling unit) of 1 ms. 5G NR keeps OFDM but makes the spacing a parameter, the *numerology* μ: the spacing is 15 × 2^μ kHz, and the slot, which always has 14 OFDM symbols, shortens in proportion.

μ	Subcarrier spacing	Slot duration	Slots per 1 ms subframe	Typical use
0	15 kHz	1 ms	1	sub-1 GHz coverage, as in LTE
1	30 kHz	0.5 ms	2	mid-band (3.5 GHz), the main 5G band
2	60 kHz	0.25 ms	4	mid-band, mmWave
3	120 kHz	0.125 ms	8	mmWave (24 to 52 GHz)

For μ = 3 the slot is 1 ms ÷ 2^3 = 125 μs, and the useful symbol is 1/120 kHz = 8.33 μs. A wider spacing means shorter symbols, hence lower latency and more tolerance of the phase noise and Doppler of high frequencies, at the cost of a shorter cyclic prefix. The two frequency ranges differ in the channel width: FR1 (410 MHz to 7.125 GHz) carriers go up to 100 MHz, and FR2 (24.25 to 52.6 GHz) up to 400 MHz, compared with LTE's 20 MHz.

	LTE	5G NR
Waveform	OFDMA downlink, SC-FDMA uplink	CP-OFDM both ways (optionally DFT-s-OFDM in uplink)
Subcarrier spacing	15 kHz	15 × 2^μ kHz
Carrier bandwidth	up to 20 MHz	up to 100 MHz (FR1), 400 MHz (FR2)
Core	EPC	5GC, service-based
Slicing	no	yes (S-NSSAI)

Latency targets	tens of ms	URLLC down to about 1 ms

Two more points complete the picture. In *non-standalone* (NSA) deployment (option 3x) the 5G radio is added to an LTE network, with the LTE eNB as anchor and the EPC as core; *standalone* (option 2) uses the gNB with the 5GC. A *handover* moves the UE between base stations: with X2 in LTE or Xn in 5G the eNBs or gNBs coordinate directly, and with S1 or N2 the core is involved. Finally, operators split a gNB into a *central unit* (CU), a *distributed unit* (DU) and a *radio unit* (RU), the O-RAN architecture, so that the real-time radio functions can sit at the antenna and the rest in a data centre; the CU and DU talk over the F1 interface.

TRAP

Claim: "The AMF is the MME in 5G", or "5G is just faster LTE." *Why it is wrong:* the MME's session functions moved to the SMF, so the AMF handles access and mobility only; and 5G changes the core (service-based, split user plane), the radio (flexible numerology, mmWave) and the service model (slices). *Correct statement:* the AMF is the mobility half of the MME; session management and user-plane control belong to the SMF.

INTERVIEW LENS

NOKIA · QUALCOMM · CISCO

Nokia (RAN and core software) and Qualcomm (modem) examine this section in depth; Cisco (private 5G) and Microsoft (operator cloud) ask for the architecture only. A Qualcomm candidate was asked for the UE network attach procedure. A strong answer names the nodes and interfaces, goes through registration in order, distinguishes NAS from AS and control from user plane, and says what the GTP-U tunnel is for. Follow-ups: the LTE counterpart of each 5G function, μ = 3 slot length, and NSA against SA.

CHECK YOURSELF

1. Which 5G function replaces the S-GW and P-GW user-plane parts? [4]
2. What is the slot duration for μ = 1? [5]
3. What is carried over N3, and by which protocol? [6]

1. cfg80211 holds configuration, regulatory rules and scanning and is the kernel side of nl80211; mac80211 is the SoftMAC implementation used by drivers that leave the MAC to the host.↩
2. It must start 16 μs after the frame ends, which a general-purpose kernel's interrupt latency cannot guarantee.↩
3. `iw dev wlan0 set type monitor`, then, with the interface up, `iw dev wlan0 set channel 36`.↩
4. The UPF.↩
5. 1 ms ÷ 2 = 0.5 ms, with a 30 kHz subcarrier

spacing.↩

6. The user's IP packets between gNB and UPF, in GTP-U tunnels (UDP port 2152).↩

Problem Set

Every interview question of this pillar is solved once, here, in the compact template: a 30-second answer first, then the approach, answer, follow-ups and common mistakes where they apply. Tags give the companies and round where the question was reported, the difficulty (■□□ easy, ■■□ medium, ■■■ hard), and **S** when a specific question was reported by candidates or **T** when only the topic was reported and the wording is ours. Section references point to the teaching text; the answers add the reasoning an interviewer wants to hear and do not repeat the whole section.

Q6.1 What is the difference between SSID, BSSID and ESSID?

CISCO · QUALCOMM · TECH-1 · ■□□ · T

30-second answer: the SSID is the network's name (up to 32 bytes); the BSSID is the 48-bit MAC address of one AP radio serving that name; the ESSID is the SSID of an extended service set, that is, the same name used by several BSSs joined by a distribution system.

Answer. A frame carries the BSSID, not the name; the name appears only in beacons, probes and association frames. A dual-band AP with one name has one SSID and two BSSIDs, and a campus has one SSID for hundreds of BSSIDs. **Follow-ups.** A hidden SSID is a beacon with an empty name, not a security measure; one radio can broadcast several SSIDs by deriving a BSSID per SSID from its base address (§6.3 (p. 365)). **Mistakes.** Saying that the BSSID is the name, or that the SSID is unique to one AP.

Q6.2 Why does Wi-Fi use CSMA/CA instead of CSMA/CD?

QUALCOMM · BROADCOM · CISCO · TECH-1 · ■□□ · T

30-second answer: a radio cannot listen while it transmits, because its own signal is some 90 dB stronger than a received one; and in a wireless cell the sender cannot see a collision at the receiver anyway (hidden nodes). So 802.11 avoids collisions with random backoff and confirms success with an ACK.

Answer. Ethernet detects a collision as a voltage anomaly on the shared wire. A half-duplex radio has a transmitter at +20 dBm and a receiver expecting −70 dBm; even if it could listen, a collision occurs at the receiver, which the sender may not hear. Hence collisions are avoided (carrier sense, DIFS, random backoff, NAV) and detected only by the missing ACK (§6.4 (p. 366)). **Follow-ups.** The cost: ACK airtime and backoff overhead; full-duplex radios with self-interference cancellation are a research topic. **Mistakes.** Writing "CSMA/CD" for Wi-Fi.

Q6.3 Explain DCF with SIFS, DIFS, slot time and backoff. Why is SIFS shorter than DIFS?

QUALCOMM · BROADCOM · TECH-1 · ■■□ · T

30-second answer: a station with a frame waits until the medium has been idle for DIFS = SIFS + 2 slots (34 µs in 5 GHz), then counts down a random backoff in [0, CW] in 9 µs slots, freezing when the medium is busy; at zero it sends and the receiver answers with an ACK after SIFS = 16 µs. SIFS is shorter so that a response always begins before any contender's DIFS expires.

Approach. State the timing for one PHY, then the five steps. **Answer.** Initial CW is 15; a failed frame (no ACK) doubles it to 31, 63, …, 1023; success resets it. SIFS-spaced frames form an atomic exchange (data and ACK, RTS and CTS, fragments), which is protected because no station can finish a DIFS wait in between. **Follow-ups.** PIFS, EIFS and AIFS; how the NAV protects the whole exchange; why the backoff is frozen and not redrawn (fairness: the loser keeps its progress). **Mistakes.** Using 2.4 GHz and 5 GHz values together: in 2.4 GHz with 802.11b present, slot 20 µs, SIFS 10 µs, DIFS 50 µs.

Q6.4 What is the hidden-node problem? How do RTS/CTS and the NAV solve it? What is the exposed-node problem?

QUALCOMM · CISCO · TECH-1 · ■■□ · T

30-second answer: two stations that both reach the AP but not each other cannot sense each other's transmissions and collide at the AP. A short RTS/CTS exchange makes the AP announce the reservation: stations that hear the CTS set their NAV (a countdown of the Duration field) and stay silent. The exposed node is the reverse: a station defers although its transmission would not interfere.

Answer. The RTS Duration covers 3 SIFS + CTS + DATA + ACK and the CTS Duration covers 2 SIFS + DATA + ACK, so everyone within earshot of either frame is silenced. Collisions can then occur only on the 20-byte RTS. RTS/CTS costs airtime, so it is used only above an RTS threshold (default off). The exposed node (B sends to A, C hears B, C wants to send to D) is not solved inside a cell; across cells it is mitigated by BSS colouring and spatial reuse (§6.9 (p. 384)). **Follow-ups.** CTS-to-self (used to protect OFDM frames from 802.11b stations); why NAV is "virtual" carrier sense. **Mistakes.** Saying RTS/CTS eliminates all collisions.

Q6.5 What happens to the contention window after a failed transmission and after a success?

QUALCOMM · TECH-1 · ■■□ · T

30-second answer: after each failure (no ACK) the contention window doubles, CW ← 2·CW + 1, from 15 up to 1023; after a success, or after the frame is dropped at the retry limit, it returns to CWmin.

Answer. Binary exponential backoff adapts to load without anyone measuring it: with few stations a small window means little wasted time; with many stations collisions raise the window until two stations rarely draw the same slot. The mean wait grows from 7.5 slots (CW = 15) to 511 slots (CW = 1023) after six failures. **Follow-ups.** Why not reset to CWmin after a failure (it would keep colliding); the retry limits (7 short, 4 long); Bianchi's model of saturation throughput against the number of stations. **Mistakes.** Doubling the window on a success, or confusing "double" with "double plus one".

Q6.6 Name the three 802.11 frame types and give two subtypes of each.

QUALCOMM · BROADCOM · TECH-1 · ■□□ · T

30-second answer: management (beacon, probe request, authentication, association request, deauthentication), control (RTS, CTS, ACK, Block Ack, PS-Poll) and data (data, QoS data, null).

Answer. The type is Frame Control bits 2–3 (0, 1, 2) and the subtype bits 4–7; Wireshark's combined code is type × 16 + subtype, for example beacon 0x08, ACK 0x1d, QoS data 0x28 (§6.5 (p. 370)). Management frames set up and tear down the relationship; control frames help deliver data (reservation, acknowledgement); data frames carry payload. **Follow-ups.** Which frames carry four addresses (data with To DS = From DS = 1); which have no sequence number (control). **Mistakes.** Putting RTS or ACK among the management frames.

Q6.7 Explain the To DS and From DS bits and the meaning of the four address fields in each case.

QUALCOMM · BROADCOM · TECH-1 · ■■□ · T

30-second answer: Address 1 is always the receiver and Address 2 the transmitter of this hop; Address 3 and 4 complete the end-to-end picture. (0,0): DA, SA, BSSID. (1,0) station to AP: BSSID, SA, DA. (0,1) AP to station: DA, BSSID, SA. (1,1) AP to AP: RA, TA, DA, SA.

Answer. To DS = 1 means the frame is addressed to the distribution system, so the receiver is the AP; From DS = 1 means it comes from the DS, so the transmitter is the AP. When both are 1 neither radio is an end system and a fourth address is needed (§6.5 (p. 370)). A worked example: laptop to server via AP uses (1,0) with A1 = AP, A2 = laptop, A3 = server. **Follow-ups.** The frame header lengths (24, 30 with the fourth address, plus 2 for QoS, 4 for HT Control); WDS and mesh. **Mistakes.** Treating Address 3 as always the destination.

Q6.8 What information is in a beacon frame?

QUALCOMM · CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: a timestamp, the beacon interval (default 100 TU = 102.4 ms), capability bits, the SSID, supported rates, the channel and HT/VHT/HE operation, the TIM (power-save indications), the country, and the RSN element with the security suites.

Answer. A beacon is a management frame with the broadcast address as receiver and the BSSID as transmitter. The *timestamp* synchronises the stations' timers (needed for TWT and power save); the TIM tells sleeping stations whether frames are buffered; the RSN element says which ciphers and key management (PSK, SAE, 802.1X) are offered; the HT/VHT/HE capability and operation elements advertise widths and features; vendor elements add WMM parameters and load. **Follow-ups.** Why beacons are sent at the lowest basic rate and the cost of many SSIDs; what a probe response adds (nothing but the TIM) (§6.5 (p. 370)). **Mistakes.** Forgetting the TIM, or saying that the beacon carries the passphrase.

Q6.9 Walk through how a station connects to a WPA2-Personal AP, frame by frame.

QUALCOMM · CISCO · JUNIPER · TECH-1 · ■■□ · S

30-second answer: beacon or probe request/response (discovery); authentication seq 1 and 2 (open); association request and response (status 0, AID); EAPOL-Key messages 1 to 4 (ANonce, SNonce + MIC, GTK + MIC, MIC) which derive and install keys; then DHCP and ARP in encrypted data frames.

Answer. After the beacon or probe the station is in State 1. The open authentication moves it to State 2 and the association to State 3, but the controlled port stays closed. The station already has the PMK from the passphrase; messages 1 and 2 exchange nonces so that both derive the PTK, message 2 proves the station's knowledge by its MIC, message 3 delivers the GTK, and message 4 confirms. Then both install the TK, the port opens, and DHCP follows (§6.6 (p. 374)). **Follow-ups.** What a wrong passphrase looks like in the capture (message 2 rejected, then reason code 15 after retries); where 802.1X would insert itself; why DHCP is invisible without decryption. **Mistakes.** Putting DHCP before the 4-way handshake.

Q6.10 What is the 4-way handshake for, and what keys result from it?

QUALCOMM · CISCO · BROADCOM · TECH-1 · ■■□ · S

30-second answer: it proves that both sides hold the PMK without sending it, derives a fresh PTK (KCK, KEK, TK) from the PMK and two nonces, and delivers the GTK (and IGTK) encrypted, so that unicast and group traffic can be protected.

Answer. The AP sends ANonce (1); the station replies with SNonce and a MIC under the KCK (2); the AP sends the encrypted GTK, its own RSN element and a MIC with the Install flag (3); the station acknowledges (4). Keys: KCK (authenticates EAPOL messages), KEK (encrypts the GTK), TK (encrypts unicast data), plus the GTK for group traffic (§6.7 (p. 377)). **Follow-ups.** What message 3 repeats the RSN element for (detecting a downgrade by comparing it with the beacon); why four messages (mutual proof, plus the group key). **Mistakes.** Saying that the PSK is sent, or that the handshake negotiates the PMK.

Q6.11 How is the PTK derived? Why are both nonces and both MAC addresses included?

QUALCOMM · TECH-2 · ■■■ · T

30-second answer: PTK = PRF-384(PMK, "Pairwise key expansion", min(AA, SPA) ‖ max(AA, SPA) ‖ min(ANonce, SNonce) ‖ max(ANonce, SNonce)), split into KCK, KEK and TK of 128 bits each. The nonces make it fresh and the addresses bind it to this pair.

Approach. Give the formula, then the role of each input. **Answer.** The PRF is HMAC-SHA1 iterated with a counter until 384 bits are produced. If only the AP chose the nonce, an attacker could replay an old message 1 and force the station to reuse an old PTK; because each side adds its own random value, neither alone controls the key. The two addresses prevent a PTK from being moved to another station or AP. Ordering by min and max lets both sides compute the same string without agreeing on roles (§6.7 (p. 377); Lab 6.7 computes one). **Follow-ups.** The PMK itself comes from PBKDF2 (4,096 iterations) or from the MSK; for SHA-256-based key management the PRF is HMAC-SHA256. **Mistakes.** Including the PMK in the message or the passphrase in the PRF input.

Q6.12 What is the replay counter in EAPOL-Key frames? What was KRACK?

QUALCOMM · BROADCOM · TECH-2 · ■■■ · T

30-second answer: the replay counter increases with every authenticator message and must be echoed by the reply, so that old messages are rejected. KRACK (2017) made a station reinstall an already-installed key when message 3 was retransmitted, which reset the packet number and so reused the encryption nonce.

Answer. The attacker blocks message 4; the AP, getting no answer, resends message 3 with a higher counter, so it is valid. A client that installs the TK on every message 3 resets the CCMP packet number to zero; keystream is then reused, allowing replay of frames and, with other ciphers or implementations, decryption or forgery. The fix is on the client: install a key once and ignore later copies, or reinstall without resetting the counters. The replay counter could not help, since the retransmission was genuine. **Follow-ups.** Why it affected also the group-key and FT handshakes; why WPA3 does not remove the issue by itself (the same handshake). **Mistakes.** Claiming that the attacker learned the passphrase.

Q6.13 Difference between PMK, PTK and GTK. When is the GTK rotated?

QUALCOMM · CISCO · TECH-1 · ■■□ · T

30-second answer: the PMK is the long-lived shared secret (from the passphrase or EAP); the PTK is the per-station, per-association session key derived from it; the GTK is the AP's key for broadcast and multicast, shared by all stations. The GTK is rotated periodically and when a station leaves.

Answer. The PMK is never used to encrypt data; it only seeds the PTK, which gives each station a separate key, so one station cannot read another's unicast traffic. The GTK must be common, so any associated station can decrypt group frames (and could forge them, which is why group traffic is the weaker part). The AP renews it with a two-message group-key handshake after the rekey interval, and when a station disassociates so that the departed station loses access. **Follow-ups.** The IGTK for management frames; PMKID caching of the PMK. **Mistakes.** Saying that all stations share one PTK.

Q6.14 Why is WPA2-PSK vulnerable to offline dictionary attacks, and how does SAE fix it?

QUALCOMM · CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: an eavesdropper who captures messages 1 and 2 can test passphrase guesses offline: guess, derive PMK, PTK and KCK, recompute the MIC and compare. SAE gives each exchange fresh random secrets, so the transcript contains nothing to check a guess against, and every guess needs an online attempt.

Answer. PBKDF2 slows each guess by 4,096 HMACs, but a GPU does billions of hashes a second, so an 8-character passphrase of common words falls in hours. In SAE both parties map the password to a curve point, send a commit (a scalar and an element built from random values) and a confirm; without the password and a live peer one learns nothing, and the AP can rate-limit attempts. A bonus is forward secrecy: a later leak of the password does not decrypt old sessions (§6.7 (p. 377)). **Follow-ups.** The PMKID attack; Dragonblood and hash-to-element; why transition mode is weaker. **Mistakes.** Saying that SAE makes weak passwords safe.

Q6.15 What is PMF (802.11w) and which attack does it stop?

CISCO · QUALCOMM · TECH-1 · ■□□ · T

30-second answer: protected management frames authenticate and encrypt robust management frames (deauthentication, disassociation, action frames), stopping forged deauthentication and disassociation attacks that disconnect clients.

Answer. Before 802.11w anyone could spoof the AP's address in a deauthentication frame and eject a client at will, or force it to re-handshake for an offline attack. With PMF, unicast management frames are encrypted under the TK, and group-addressed ones carry an integrity check under the IGTK, so a spoofed frame fails verification. It is mandatory in WPA3 and in 6 GHz, and optional in WPA2. **Follow-ups.** The SA Query procedure for the case in which one side has lost its keys; what is not protected (beacons, radio-level jamming). **Mistakes.** Believing that PMF encrypts data frames or prevents jamming.

Q6.16 Explain 802.1X roles and name two EAP methods. Which needs client certificates?

CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: the supplicant (client) wants access, the authenticator (the AP or switch) controls a port, and the authentication server (RADIUS) decides. EAP-TLS needs a certificate on both sides; PEAP needs only a server certificate and uses a password inside the TLS tunnel.

Answer. The AP relays EAP messages between EAPOL toward the client and RADIUS toward the server; the port is blocked except for EAPOL until Access-Accept arrives, which also carries the MSK from which the PMK comes. EAP-TLS is the strongest, because there is no password to steal, but needs a certificate infrastructure; PEAP-MSCHAPv2 is the commonest. The generic flow is in §9.12 (p. 567); the Wi-Fi-specific part is in §6.7 (p. 377). **Follow-ups.** Why a client that skips server-certificate validation is vulnerable to a rogue AP; eduroam as a federation of RADIUS servers. **Mistakes.** Saying that the AP authenticates the user.

Q6.17 What is the difference between authentication and association in 802.11?

QUALCOMM · TECH-1 · ■□□ · T

30-second answer: authentication (open system) is a two-frame formality that moves a station to State 2 and has no credentials; association negotiates the capabilities, assigns an association ID and moves it to State 3. Real security comes afterwards, with SAE or 802.1X and the 4-way handshake.

Answer. The old state machine had to be preserved for compatibility, so every station still performs the open exchange. Association carries what matters for the data path: rates, HT/VHT/HE capabilities, the RSN selection and power-save parameters. Deauthentication returns the station to State 1 and disassociation to State 2 (§6.6 (p. 374)). **Follow-ups.** SAE reuses the authentication frames (algorithm 3) for its commit and confirm; FT uses algorithm 2. **Mistakes.** Calling the open exchange “authentication” in the security sense.

Q6.18 Explain OFDM and why the subcarriers are orthogonal. What is the cyclic prefix for?

QUALCOMM · NOKIA · TECH-1 · ■■□ · S

30-second answer: OFDM sends many low-rate symbols on narrow subcarriers spaced exactly 1/T apart, where T is the symbol duration; each subcarrier’s spectrum is zero at the centres of all others, so a receiver integrating over T separates them. The cyclic prefix, a copy of the symbol’s tail placed in front, absorbs multipath echoes so symbols do not overlap.

Answer. Long symbols make multipath echoes a small fraction of the symbol. Orthogonality is exact if the subcarrier spacing equals 1/T and the receiver samples over one useful symbol; the cyclic prefix keeps the block periodic so this still holds with echoes shorter than the prefix. The IFFT/FFT pair makes the transform cheap (§6.2 (p. 361)). **Follow-ups.** The price of the prefix (0.8 µs in 12.8 µs is 6%); what breaks orthogonality (frequency offset and Doppler, which is why oscillator accuracy matters, and why NR uses wider spacing at mmWave); pilots. **Mistakes.** Saying that the subcarriers do not overlap in frequency: they do, but are orthogonal.

Q6.19 OFDM versus OFDMA.

QUALCOMM · CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: in OFDM, one transmission occupies all the subcarriers of the channel; in OFDMA (802.11ax) the AP divides them into resource units and gives different units to different stations within the same symbol.

Answer. OFDMA wins when many stations have small amounts of data: nine 100-byte frames take about a quarter of the airtime and cause no collisions (§6.9 (p. 384)). It needs scheduling by the AP (a trigger frame in the uplink), so it adds complexity, and it does not raise the single-user peak rate. **Follow-ups.** The RU sizes (26 to 996 tones) and number per width; how cellular networks have used OFDMA since LTE. **Mistakes.** Claiming that OFDMA increases peak throughput.

Q6.20 SU-MIMO versus MU-MIMO. What did 802.11ax add over 802.11ac for MU-MIMO?

QUALCOMM · BROADCOM · TECH-1 · ■■□ · T

30-second answer: SU-MIMO sends several spatial streams to one client; MU-MIMO sends different streams to different clients at once, separated by beamforming. 802.11ac had downlink MU-MIMO only; 802.11ax added uplink MU-MIMO, up to eight users, and combination with OFDMA resource units of 106 tones or more.

Answer. MU-MIMO needs channel knowledge (sounding), so that the AP can place a null of each client's beam on the others; the sounding airtime and the quality of the feedback limit the gain, which appears with many clients and an AP with many antennas. Uplink MU-MIMO adds the need to synchronise the clients, which the trigger frame provides (§6.2 (p. 361), §6.9 (p. 384)). **Follow-ups.** Why the number of streams is bounded by the AP's antennas; the stream count of a typical phone (1 or 2). **Mistakes.** Mixing up the roles: MU-MIMO shares space, OFDMA shares frequency.

Q6.21 What is a trigger frame? Describe an uplink OFDMA exchange.

QUALCOMM · BROADCOM · TECH-2 · ■■■ · T

30-second answer: a trigger frame is a control frame by which the AP solicits uplink transmissions: it lists, for each station, its RU, MCS, power target and length. After SIFS the stations send HE trigger-based PPDUs simultaneously, and the AP replies with one multi-station Block Ack.

Answer. The AP first gathers buffer status (from earlier frames or a buffer-status poll), then sends the trigger. All responders must end together, so shorter ones pad. The AP controls the target receive power, which keeps the signals within a few decibels at its antenna and so makes the shared symbol decodable. Stations without a grant may use the random-access RUs (UORA). A trigger can also carry MU-BAR to solicit acknowledgements (§6.9 (p. 384)). **Follow-ups.** The cost of the trigger and the padding; frequency and timing synchronisation (the stations align to the trigger within the guard interval). **Mistakes.** Forgetting that uplink OFDMA is scheduled, not contended.

Problem Set, continued: Wi-Fi 6 and 7, power, roaming and cellular

Q6.22 What is BSS colouring?

QUALCOMM · CISCO · TECH-1 · ■■□ · T

30-second answer: every HE frame carries a 6-bit colour identifying its BSS. A station that hears a frame of another colour below the OBSS packet-detect threshold may ignore it and transmit, with reduced power; this is spatial reuse and it cuts needless deferral between overlapping cells.

Answer. Without colours, any frame above the −82 dBm preamble-detect level makes the medium busy, even if it comes from a distant neighbour on the same channel. With colours, the station sees that the frame is inter-BSS; if its level is below the threshold (the default −82 dBm, adjustable up to −62 dBm), it treats the medium as idle and may transmit at a power lowered by the amount that the threshold was raised. Two NAVs (intra-BSS and basic) keep own-cell reservations intact (§6.9 (p. 384)). **Follow-ups.** Colour collisions between neighbouring APs, and the AP's ability to change colour; why gains are largest in dense, same-channel deployments. **Mistakes.** Saying that colouring avoids interference; it only avoids unnecessary deferral.

Q6.23 What is TWT and which devices benefit?

QUALCOMM · BROADCOM · TECH-1 · ■■□ · T

30-second answer: target wake time is a negotiated schedule in which the AP and a station agree when the station will wake; it sleeps in between without listening to beacons. Battery devices (IoT sensors, phones, wearables) benefit, and contention falls because stations wake in separate slots.

Answer. The agreement fixes the first wake time, the interval (mantissa × 2^exponent µs) and the minimum wake duration; it may be individual or, in broadcast TWT, announced by the AP for a group, and it may be trigger-enabled so that the AP polls the station inside its window. Compared with legacy power save, the station avoids waking for every beacon or DTIM, and the AP knows when to buffer. Wi-Fi 7 adds restricted TWT to protect latency-sensitive flows (§6.8 (p. 381), §6.9 (p. 384)). **Follow-ups.** The firmware challenge of honouring many agreements at once; clock drift between AP and station. **Mistakes.** Describing TWT as a faster beacon.

Q6.24 List four Wi-Fi 7 features and the benefit of each.

QUALCOMM · CISCO · BROADCOM · TECH-1 · ■■□ · T

30-second answer: 320 MHz channels (double the bandwidth, in 6 GHz); 4096-QAM (20% more bits per symbol at high SNR); multi-link operation (aggregate rate, lower latency and better reliability across bands); multi-RU and preamble puncturing (use of fragmented or partly occupied spectrum).

Answer. The factors 2 (320 MHz), 1.2 (4096-QAM) and 2 (16 streams) take the peak from 9.6 to 46.1 Gb/s. Realistically a 2-stream 320 MHz client has 5.8 Gb/s. 4096-QAM needs an even better SNR than 1024-QAM (and a low error-vector magnitude in the transmitter), so it benefits clients near the AP only. Restricted TWT and multi-AP coordination target latency (§6.9 (p. 384)). **Follow-ups.** The PHY rate for 2 streams (2 × 2,882 Mb/s); which of the features need new hardware in the AP. **Mistakes.** Quoting 46 Gb/s as a user-visible rate.

Q6.25 Explain MLO modes (STR, NSTR, EMLSR) and a firmware challenge each introduces.

QUALCOMM · BROADCOM · TECH-2 · ■■■ · T

30-second answer: in STR a multi-link device transmits on one link while receiving on another; in NSTR the links are too close, so it cannot; in EMLSR a single full radio listens on several links and switches to the one that receives an initial control frame. Challenges: independent per-link state (STR), aligned end times (NSTR), switching delay (EMLSR).

Answer. STR needs radio isolation, and its firmware must keep a separate EDCA, NAV and back-off state for each link yet share one sequence-number space and a block-ack scoreboard. NSTR must align PPDU ends on both links, otherwise transmitting on one deafens the other and the acknowledgements are lost. EMLSR saves power and cost; the AP must know the transition delay and not send to the station until it has switched, and frames during the switch are lost. Common tasks: TID-to-link mapping, link recovery and the single MLD address (§6.9 (p. 384)). **Follow-ups.** Where in the stack the link choice is made; effect on rate adaptation. **Mistakes.** Treating MLO as band steering: the same flow uses several links at once.

Q6.26 What is A-MPDU versus A-MSDU? Which survives better on a noisy channel, and why?

QUALCOMM · BROADCOM · TECH-1 · ■□□ · T

30-second answer: an A-MSDU puts several packets in one MAC frame with one header and one FCS; an A-MPDU sends several complete MAC frames, each with its own FCS, in one PHY burst. The A-MPDU survives better: a bit error costs only its subframe, while an A-MSDU is lost entirely.

Answer. An A-MSDU saves header bytes but its single FCS makes it all-or-nothing; its maximum size is 3,839 or 7,935 bytes for HT and 11,454 for VHT and HE. An A-MPDU subframe has a 4-byte delimiter and is padded to 4 bytes, but is acknowledged selectively by Block Ack and can reach 64 KB (HT), 1 MB (VHT) or 4 MB (HE). Systems use both: A-MSDUs inside an A-MPDU (§6.8 (p. 381)). **Follow-ups.** The efficiency effect of aggregation (the 9× goodput example); the latency cost of waiting to fill an aggregate. **Mistakes.** Reversing the two, or saying that A-MSDU needs a Block Ack per packet.

Q6.27 What is Block Ack?

QUALCOMM · BROADCOM · TECH-1 · ■■□ · T

30-second answer: a single acknowledgement frame with a bitmap that reports which MPDUs of a window arrived, so that an aggregate needs one reply and only the missing MPDUs are resent.

Answer. The agreement is set up per traffic identifier by ADDBA request and response action frames, fixing the window size (64 in 802.11n and ac, 256 in 802.11ax, 1,024 in 802.11be) and the starting sequence number. The sender transmits an A-MPDU, the receiver replies with a Block Ack after SIFS (or the sender solicits it with a Block Ack Request), and the sender slides the window past acknowledged frames. The receiver reorders frames, so that the upper layers still see order (§6.8 (p. 381)). **Follow-ups.** Immediate versus delayed; the multi-STA Block Ack in OFDMA; the reorder-buffer memory on a chip. **Mistakes.** Saying that Block Ack replaces the ACK for single frames.

Q6.28 Explain EDCA access categories and how voice gets priority.

QUALCOMM · CISCO · TECH-1 · ■■□ · T

30-second answer: four queues (voice, video, best effort, background), each with its own AIFSN, CWmin and CWmax and TXOP limit; voice has AIFSN 2 and CW 3 to 7, best effort AIFSN 3 and CW 15 to 1023. A shorter wait and a smaller window make voice win more often, and its TXOP allows a short burst.

Answer. The average first wait is SIFS + AIFSN × 9 µs + (CWmin/2) × 9 µs: 47.5 µs for voice against 110.5 µs for best effort. Priority is therefore statistical, not guaranteed; if two queues in one station expire together, the higher one wins (internal collision). The user priority is carried in the QoS Control field and derived from the DSCP; EF should map to UP 6 (voice) under RFC 8325, not to UP 5 (§6.8 (p. 381)). **Follow-ups.** Why high-priority traffic can still starve others (admission control, TXOP limits); WMM certification. **Mistakes.** Quoting wired-network strict priority.

Q6.29 What are TIM and DTIM? How does a sleeping station get buffered unicast frames?

QUALCOMM · BROADCOM · TECH-1 · ■■□ · T

30-second answer: the TIM in each beacon has one bit per association ID marking stations with buffered unicast frames; every DTIM-period-th beacon is a DTIM, after which buffered broadcast and multicast are sent. A sleeping station wakes for the beacon, sees its bit, and fetches its frames with a PS-Poll (or a U-APSD trigger).

Answer. The station tells the AP that it will sleep by setting the power-management bit; the AP buffers. After a PS-Poll the AP sends one frame, with More Data set if more are queued; with U-APSD, one uplink frame releases all. With a 102.4 ms beacon and DTIM period 3 a station that wants multicast wakes every 307 ms (§6.8 (p. 381)). **Follow-ups.** Listen interval; the latency and power trade-off of a long DTIM; why phones set a DTIM of 1 to 3. **Mistakes.** Thinking that the DTIM concerns unicast.

Q6.30 How does 802.11r speed up roaming? What do 802.11k and 802.11v add?

CISCO · JUNIPER · QUALCOMM · TECH-1 · ■■□ · T

30-second answer: 802.11r (fast transition) derives the keys for the next AP in advance from a key hierarchy (PMK-R0, PMK-R1), so a roam costs two authentication and two reassociation frames, about 20 ms, with no 802.1X re-authentication. 802.11k gives the client a neighbour list; 802.11v lets the AP suggest a better AP.

Answer. The client decides; k and v only help. k shortens the scan by naming the candidate channels; v moves sticky clients and balances load with BSS transition requests; r removes the long part, the key exchange (§6.10 (p. 387)). **Follow-ups.** Over-the-air versus over-the-DS; opportunistic key caching as the vendor alternative; clients that fail when 802.11r is enabled. **Mistakes.** Believing that the AP can force a client to roam.

Q6.31 Users complain that voice calls drop while walking down a corridor. How do you troubleshoot?

CISCO · JUNIPER · TECH-2 · ■■■ · T

30-second answer: reproduce on a walk test with a client log and a capture; find whether the call drops at the roam (gap, sticky client, key exchange time), or from poor signal (hole, high retries), or from the channel (interference, DFS evacuation); fix coverage, power, minimum rate, 802.11k/v/r and QoS in that order, and verify again.

Approach. Follow the method of Pillar 11 (p. 634): scope, measure, hypothesise, change one thing, verify. **Answer.** (i) Scope: one floor, one phone model or all? Time-dependent? (ii) Walk with the phone, record RSSI and SNR, the BSSID and the time of each roam, and capture on the channel. (iii) Read the evidence. A long gap at the roam suggests 802.1X re-authentication or no 802.11r; a late roam at −80 dBm suggests a sticky client (raise the minimum RSSI, enable 802.11v); a hole between APs shows as falling SNR and rising retries; an AP that disappears on a DFS channel points to radar events; one-way audio points to an AP that is louder than the client (reduce transmit power). (iv) Check that voice is marked UP 6 end to end, and that rates below 12 Mb/s are disabled. **Follow-ups.** Target values: −67 dBm, SNR 25 dB, retries under 10%, roam under 50 ms. **Mistakes.** Changing power and channels before looking at the roam events.

Q6.32 Convert 200 mW to dBm. What does −3 dB mean?

QUALCOMM · NOKIA · TECH-1 · ■□□ · T

30-second answer: 200 mW is 23 dBm (100 mW is 20 dBm and doubling adds 3 dB). −3 dB is a factor of one half in power.

Answer. dBm = 10 $\log_{10}$(mW), so 10 $\log_{10}$(200) = 23.0. A decibel is a ratio: +3 dB is ×2, +10 dB is ×10, −3 dB is ×0.5, −10 dB is ×0.1; a −3 dB loss halves the power, so 23 dBm becomes 20 dBm. In amplitude (voltage) terms the factors are the square roots (§6.1 (p. 359)). **Follow-ups.** Add 17 dBm and 20 dBm (that is 50 mW + 100 mW = 150 mW = 21.8 dBm); the FSPL change for twice the distance (+6 dB). **Mistakes.** Adding dBm values, or reading −3 dB as “minus three watts”.

Q6.33 Why are only channels 1, 6 and 11 recommended in 2.4 GHz?

CISCO · TECH-1 · ■■□ · T

30-second answer: the 2.4 GHz channels are 5 MHz apart but each is about 20 MHz wide, so two channels are free of overlap only if their numbers differ by 5 or more; 1, 6 and 11 are the only three such channels in the 1 to 11 range that all regions allow.

Answer. Co-channel neighbours hear each other’s preambles and share airtime by CSMA/CA; partly overlapping neighbours do not decode each other and merely raise noise, so frames are corrupted and retried. Hence one plans three non-overlapping channels and repeats them. In some regions the plan 1, 5, 9, 13 is used for OFDM-only networks, accepting a little overlap; the general recommendation stays 1, 6, 11. 40 MHz channels in 2.4 GHz leave room for no more than one cell (§6.10 (p. 387)). **Follow-ups.** Why 5 GHz relieves this; how the interference from microwave ovens and Bluetooth enters. **Mistakes.** Choosing channels 3 and 8 “to avoid the crowd”.

Q6.34 Throughput is low even though RSSI is −50 dBm. List causes.

CISCO · QUALCOMM · TECH-2 · ■■■ · T

30-second answer: RSSI measures only the signal; throughput also depends on SNR (noise and interference), contention (airtime shared with neighbours and slow clients), retries, the negotiated MCS, channel width and streams, legacy-client protection, and the wired side.

Answer. Check, in order: (i) noise floor and SNR: a −50 dBm signal with a −60 dBm interferer is a 10 dB SNR, which gives a low MCS; (ii) channel utilisation and co-channel APs (each shares the airtime); (iii) retry rate and the MCS in `iw station dump`; (iv) slow legacy stations (an 802.11b client at 1 Mb/s occupies the channel for ages, and protection modes add overhead); (v) a width mismatch (80 MHz on a congested band, 40 MHz in 2.4 GHz) or a single-stream client; (vi) power-save or TXOP behaviour; (vii) the backhaul or the server. **Follow-ups.** The expected goodput (about 60% of the PHY rate at best). **Mistakes.** Treating RSSI as quality.

Q6.35 What is rate adaptation? Describe one algorithm.

QUALCOMM · BROADCOM · TECH-2 · ■■■ · T

30-second answer: the transmitter does not know which MCS the channel supports, so it picks one per frame from the outcome of earlier frames. Minstrel keeps, for every rate, a smoothed delivery probability, estimates throughput as rate × probability, uses the best, and probes other rates for about 10% of frames.

Approach. Rates are an ordered set; a high rate has more throughput if it succeeds and none if it fails, so the algorithm explores and exploits.
Answer. Each interval (100 ms in Minstrel) the success statistics update the probability with an exponentially weighted average (75% old, 25% new); the retry chain falls back to lower rates when the first choice fails, and a probe is only sent at a rate not obviously worse. Firmware versions add SNR feedback and aggregation length.

```c
#include <stdlib.h>
#define NR 8
struct rate { int kbps; double p; };

int pick(const struct rate *t,
         unsigned sent)
{
  if (sent % 10 == 9)   /* probe */
    return rand() % NR;
  int best = 0;
  for (int i = 1; i < NR; i++)
    if (t[i].kbps * t[i].p >
        t[best].kbps * t[best].p)
      best = i;
  return best;
}

void update(struct rate *r,
            int ok, int tries)
{
  double s = (double)ok / tries;
  r->p = 0.75 * r->p + 0.25 * s;
}
```

Follow-ups. Why not use the highest rate always (the loss collapses throughput); the effect of aggregation (a loss costs a whole burst); how OFDMA and MU-MIMO complicate it (per-RU rates). **Mistakes.** Judging a rate on one frame, or never probing upwards.

Q6.36 In firmware, how would you schedule RUs across 20 stations with mixed QoS? State the inputs, the constraints and a greedy heuristic.

QUALCOMM · TECH-2 · ■■■ · T

30-second answer: inputs are each station's queue per access category, its rate per tone and its age; constraints are the RU tree of the channel (26 to 996 tones), the PPDU length limit and the number of users per PPDU. Greedy: sort by priority, then queue; give each the smallest RU that drains its queue, keeping a minimum RU for those not yet served; carry unserved stations over with a growing age.

Approach. Pick the metric, then the allocation, then fairness. **Answer.** With 80 MHz there are 37 26-tone RUs, so 20 stations all fit; the problem is how big each RU is. Time is bounded by the TXOP, so a station needs queue ÷ bpt tones, where bpt is the number of bytes one tone carries in the PPDU duration at the station's MCS, rounded up to an allowed size (26, 52, 106, 242, 484, 996). Reserving 26 tones per later station prevents a large voice or video queue from starving the rest.

```
struct sta { int id, prio, bytes, bpt;
};

static int cmp(const void *a,
               const void *b)
{
  const struct sta *x = a, *y = b;
  if (x->prio != y->prio)
    return y->prio - x->prio;
  return y->bytes - x->bytes;
}

void schedule(struct sta *s, int n,
              int *ru)
{
  static const int sz[] =
    {26, 52, 106, 242, 484, 996};
  int left = 996;  /* 80 MHz */
  qsort(s, n, sizeof *s, cmp);
  for (int i = 0; i < n; i++) {
    ru[i] = 0;
    if (left < 26) continue;
    /* keep 26 per later STA */
    int room = left - 26 * (n-i-1);
    if (room < 26) room = 26;
    int need = s[i].bytes/s[i].bpt + 1;
    int k = 0;
    while (k < 5 && sz[k] < need) k++;
    while (k > 0 && sz[k] > room) k--;
    ru[i] = sz[k];
    left -= sz[k];
  }
}
```

The function returns sizes; a second step places them in the RU tree (largest first, so that no 26-tone gap is stranded) and chooses an MCS per RU from the station's recent SNR. **Complexity.** O(n log n). **Follow-ups.** Uplink needs buffer-status reports; deadline-aware (earliest-deadline-first) scheduling for voice; deficit counters for long-term fairness; when to prefer MU-MIMO or a single wide RU. **Mistakes.** Ignoring the trigger and acknowledgement overhead, which makes tiny RUs wasteful.

Q6.37 Describe the LTE architecture (MME, S-GW, P-GW, HSS).

NOKIA · QUALCOMM · TECH-1 · ■■□ · T

30-second answer: the UE connects over the radio to an eNodeB; the eNB talks to the MME for signalling (S1-MME) and to the S-GW for user data (S1-U). The MME handles attach, security and mobility using subscriber data from the HSS (S6a); the S-GW anchors user data during handover; the P-GW assigns the IP address and connects to the Internet (SGi). PCRF supplies policy.

Answer. Control and user planes are separate: MME and HSS carry signalling, S-GW and P-GW carry packets in GTP-U tunnels (S1-U, S5). The user's session is an EPS bearer: the default one has QCI 9, and a dedicated bearer (QCI 1 for voice) adds guaranteed rate (§6.12 (p. 393)). **Follow-ups.** X2 handover between eNBs; tracking areas and paging; EPS-AKA. **Mistakes.** Putting the IP address allocation in the MME.

Q6.38 What are AMF, SMF and UPF in 5G? What is a service-based architecture?

NOKIA · QUALCOMM · TECH-1 · ■■□ · T

30-second answer: the AMF handles registration, NAS security and mobility; the SMF manages PDU sessions and allocates the IP address and programs the UPF; the UPF forwards user packets. In a service-based architecture the control functions expose services to each other over HTTP/2 and register in a repository (NRF), instead of using fixed point-to-point interfaces.

Answer. The 4G MME was split: its mobility half is the AMF and its session half the SMF, and the user-plane parts of S-GW and P-GW became the UPF, controlled over N4 (PFCP). The service-based design allows each function to scale and update on its own, as cloud software does; discovery is by the NRF. Other functions: AUSF and UDM (authentication and subscriber data), PCF (policy), NSSF (slice selection) (§6.12 (p. 393)). **Follow-ups.** Why CUPS lets the UPF sit near the edge; what a slice is. **Mistakes.** Saying that the AMF is a renamed MME.

Q6.39 Walk through the UE attach or registration procedure and PDU session establishment.

QUALCOMM · NOKIA · TECH-2 · ■■■ · S

30-second answer: RRC connection; Attach (LTE) or Registration (5G) request to the MME or AMF; authentication with the HSS or AUSF/UDM; security mode; accept; then session set-up: in LTE the default bearer through S-GW and P-GW, in 5G a PDU session through the SMF and UPF. At the end the UE has an IP address and a GTP-U tunnel to the core.

Answer for LTE. (1) Random access and RRC connection setup; the attach request with a PDN connectivity request travels to the MME in the S1AP initial UE message. (2) The MME fetches the authentication vector from the HSS over S6a and runs EPS-AKA with the UE. (3) NAS security mode command. (4) The MME updates the HSS with its location and receives the subscription. (5) The MME sends a Create Session Request to the S-GW and then the P-GW (GTP-C), which allocates the UE's IP address and the default bearer (QCI 9). (6) The MME sends an initial context setup request to the eNB, carrying the attach accept; the eNB reconfigures the radio bearer. (7) The UE replies attach complete; the MME gives the eNB's tunnel endpoint to the S-GW (Modify Bearer), and downlink data can flow. **5G.** The steps are the ten of §6.12 (p. 393): registration with SUCI, 5G-AKA through AUSF and UDM, security, accept with 5G-GUTI, then the PDU session through the SMF, an N4 session at the UPF and an N3 tunnel to the gNB. **Follow-ups.** NAS against AS; what changes at re-registration or handover; why SUCI hides the permanent identity. **Mistakes.** Forgetting that the core, not the eNB, authenticates.

Q6.40 What is GTP-U? Why tunnel user data between gNB and UPF?

NOKIA · QUALCOMM · TECH-1 · ■■□ · T

30-second answer: the GPRS tunnelling protocol, user plane, carries the user's IP packets inside UDP (port 2152), with an 8-byte header whose TEID identifies the bearer or session. Tunnelling hides the UE's address from the transport network and keeps it constant while the UE moves.

Answer. The UE's IP address belongs to the core (the P-GW or UPF), not to the topology of the radio network. A tunnel from the base station to the anchor lets the transport network route on outer addresses only, so a handover merely changes the tunnel end; it also gives per-session accounting and QoS marking. Overhead is 20 + 8 + 8 = 36 bytes with IPv4, which reduces the effective MTU of the UE to avoid fragmentation (§6.12 (p. 393)). **Follow-ups.** The TEID allocation (each end assigns the ID its peer must use); GTP-C on port 2123; the similarity to VXLAN. **Mistakes.** Thinking that GTP encrypts: it does not, and IPsec or the radio security does.

Q6.41 Compare 5G NR numerology with LTE. What does μ = 3 mean for slot duration?

QUALCOMM · NOKIA · TECH-2 · ■■■ · T

30-second answer: LTE has a fixed 15 kHz subcarrier spacing and 1 ms subframe; NR's spacing is 15 × 2^μ kHz, and the slot of 14 symbols lasts 1 ms ÷ 2^μ. For μ = 3 the spacing is 120 kHz and the slot is 125 μs.

Answer. Wider spacing means shorter symbols (8.33 μs useful for 120 kHz), so lower latency and more tolerance of the phase noise and Doppler at mmWave, at the cost of a shorter cyclic prefix and so less delay spread tolerated. There are 8 slots in a 1 ms subframe at μ = 3 and 80 in a 10 ms frame. Different numerologies can coexist on one carrier, for example eMBB and URLLC services (§6.12 (p. 393)). **Follow-ups.** The PRB width (12 × the spacing: 1.44 MHz at 120 kHz); the maximum carrier bandwidth (100 MHz in FR1, 400 MHz in FR2); mini-slots. **Mistakes.** Taking the slot as 7 symbols (LTE) or as a fixed 1 ms.

Q6.42 In Linux, what are cfg80211, mac80211 and nl80211?

QUALCOMM · BROADCOM · TECH-1 · ■□□ · T

30-second answer: nl80211 is the netlink interface through which user space (iw, wpa_supplicant, hostapd) configures wireless devices; cfg80211 is the kernel configuration layer, with regulatory rules and scanning; mac80211 is the SoftMAC framework that implements the MAC state machine, rate control and aggregation for drivers that need it.

Answer. A command such as `iw dev wlan0 scan` goes through nl80211 to cfg80211, which validates it and calls the driver (directly for FullMAC chips like brcmfmac, or through mac80211 for SoftMAC drivers like ath9k and, in part, ath11k). The firmware on modern chips does the time-critical work; the host and firmware exchange WMI commands and events (§6.11 (p. 390)). **Follow-ups.** FullMAC versus SoftMAC; monitor mode and radiotap; `mac80211_hwsim` for simulation. **Mistakes.** Describing mac80211 as a driver, or placing the regulatory database in user space only.

Questions solved in other pillars. The generic 802.1X and EAP exchange behind Q6.16 is in Pillar 9 (p. 538); the troubleshooting method behind Q6.31 is Pillar 11 (p. 634); the interrupt, DMA and receive-path questions that WLAN firmware interviews add are solved in Pillar 7 (p. 414).

Revision Spread

THE 15-MINUTE VERSION

Ten facts. (1) +3 dB is ×2 and +10 dB is ×10; dBm is absolute (200 mW = 23 dBm); FSPL = 20 log d + 20 log f − 147.55 (m, Hz) and adds 6 dB per doubling of distance or frequency. (2) Shannon: $C = B \log_2(1 + SNR)$; doubling the width costs 3 dB of SNR. (3) Rate = data subcarriers × bits × coding × streams ÷ symbol time; 802.11ax at 80 MHz, 1024-QAM, 5/6 gives 600.5 Mb/s per stream. (4) Wi-Fi is CSMA/CA with ACKs; in 5 GHz slot 9, SIFS 16, DIFS 34 µs; CW doubles 15 → 1023 on failure and resets on success; SIFS < DIFS so responses win. (5) RTS/CTS and the NAV fix hidden nodes; exposed nodes remain. (6) Frame Control is little-endian; type 0/1/2 = management/control/data; Address 1 is the receiver, Address 3 depends on To DS/From DS. (7) Lifecycle: discovery, open auth (State 2), association (State 3, AID), 4-way handshake, DHCP; real authentication is PSK/SAE/802.1X. (8) PTK = PRF(PMK, AA, SPA, ANonce, SNonce) = KCK | KEK | TK; the PMK never crosses the air; the GTK travels encrypted in message 3. (9) WPA3 = SAE (no offline dictionary, forward secrecy) + mandatory PMF; transition mode can be downgraded. (10) OFDMA shares frequency (RUs, trigger frames), MU-MIMO shares space, BSS colour allows spatial reuse, TWT schedules sleep; Wi-Fi 7 adds 320 MHz, 4096-QAM, MLO, multi-RU and puncturing; 5G NR: spacing 15 × 2^µ kHz, slot 1 ms ÷ 2^µ, AMF = mobility, SMF = sessions, UPF = user plane.

Three figures to redraw. The DCF timeline with the frozen backoff (Figure 6.5), the 4-way handshake with its keys (Figure 6.11), and the OFDM versus OFDMA grid (Figure 6.12).

Standards at a glance

Standard	Band · width	QAM · peak	Essence
802.11b / a / g	2.4 / 5 / 2.4 GHz · 22 / 20 / 20 MHz	CCK / 64 / 64 · 11 / 54 / 54 Mb/s	DSSS; OFDM
802.11n (Wi-Fi 4)	2.4, 5 GHz · 20, 40	64 · 600 Mb/s	MIMO, A-MPDU
802.11ac (Wi-Fi 5)	5 GHz · to 160	256 · 6.93 Gb/s	DL MU-MIMO
802.11ax (Wi-Fi 6, 6E)	2.4, 5, 6 GHz · to 160	1024 · 9.61 Gb/s	OFDMA, UL MU-MIMO, colour, TWT
802.11be (Wi-Fi 7)	2.4, 5, 6 GHz · to 320	4096 · 46.1 Gb/s	MLO, multi-RU, puncturing
LTE / 5G NR	licensed · 20 / 100 to 400 MHz	256 / 1024	scheduled OFDMA; EPC / 5GC

Timing, sizes and constants

Item	Value
5 GHz OFDM	slot 9 µs, SIFS 16, PIFS 25, DIFS 34; CWmin 15, CWmax 1023
2.4 GHz	OFDM short slot: 9, 10, 28; with 802.11b: 20, 10, 50; 802.11b CWmin 31
Contention window	15, 31, 63, 127, 255, 511, 1023; mean backoff 7.5 slots at CW 15
EDCA (AIFSN, CWmin–CWmax)	VO 2, 3–7; VI 2, 7–15; BE 3, 15–1023; BK 7, 15–1023; AIFS = SIFS + AIFSN × slot
Beacon	100 TU, 1 TU = 1,024 µs, so 102.4 ms; DTIM period 3 gives 307.2 ms
Frame sizes	ACK, CTS 14 B; RTS 20 B; header 24 B (30 with Addr4, +2 QoS, +4 HTC); FCS 4 B; CCMP +16 B
CCA thresholds	preamble detect about −82 dBm, energy detect about −62 dBm (20 MHz)
Sensitivity (HE, 20 MHz)	MCS 0 −82, MCS 7 −64, MCS 9 −57, MCS 11 −52 dBm
Wireshark codes	beacon 0x08, probe req/resp 0x04/0x05, assoc req/resp 0x00/0x01, auth 0x0b,

	0x1b/0x1c/0x1d, BA 0x19, QoS data 0x28
OFDM numerology	312.5 kHz / 3.2 µs (a to ac); 78.125 kHz / 12.8 µs (ax, be); GI 0.8, 1.6, 3.2 µs
RUs in 80 MHz	37 × 26, 16 × 52, 8 × 106, 4 × 242, 2 × 484, 1 × 996
6 GHz	5,925 to 7,125 MHz; 59 × 20, 29 × 40, 14 × 80, 7 × 160 MHz; 3 × 320 MHz
Cellular ports	GTP-U 2152, GTP-C 2123, PFCP 8805 (UDP); S1AP 36412, NGAP 38412 (SCTP)

Security comparison

	WEP	WPA	WPA2	WPA3
Cipher	RC4	TKIP	CCMP	CCMP, GCMP-256
Personal	static	PSK	PSK	SAE
Offline guess	n/a	yes	yes	no
PMF	no	no	optional	required
Fwd secrecy	no	no	no	yes

Key chain and lifecycle in one line each

- **Keys.** Passphrase + SSID → PBKDF2 → PMK (or EAP MSK[0:256], or SAE); PMK + AA + SPA + ANonce + SNonce → PTK = KCK 128 | KEK 128 | TK 128; GMK → GTK, delivered encrypted under the KEK; IGTK protects management multicast.
- **Handshake.** M1 ANonce; M2 SNonce + MIC; M3 GTK + MIC + Install; M4 MIC. The replay counter rises with each new message; a wrong passphrase fails the MIC in M2.
- **Lifecycle.** Beacon or probe, authentication (algorithm 0, SAE 3, FT 2), association (status 0, AID 1 to 2007), EAPOL 1 to 4, DHCP, ARP.
- **Roaming.** k = neighbour report, v = BSS transition request, r = FT with PMK-R0 and PMK-R1; about 20 ms against 200 to 400 ms for full 802.1X.
- **LTE to 5G.** MME → AMF (mobility) + SMF (sessions); S-GW/P-GW user plane → UPF; HSS → UDM + AUSF; PCRF → PCF; S1-U → N3; S1-MME → N2; X2 → Xn; QCI 9 → 5QI 9.
- **Numerology.** µ = 0, 1, 2, 3 gives 15, 30, 60, 120 kHz and slots of 1, 0.5, 0.25, 0.125 ms.

Flashcards

Q	A	Q	A
200 mW in dBm	23 dBm	Why CSMA/CA, not CD	Radio is deaf while sending
−3 dB	Half power	DIFS, 5 GHz	34 µs
FSPL, double distance	+6 dB	CW after 3 failures	127
Thermal floor, 20 MHz	−101 dBm	What sets the NAV	Duration field
Bits in 1024-QAM	10	RTS/CTS solves	Hidden nodes
Wider channel, SNR	−3 dB per doubling	Frame Control order	Little-endian
Subcarrier spacing	1/T	To DS = 1, From DS = 0	STA to AP: BSSID, SA, DA
Cyclic prefix	Absorbs echoes	ACK length	14 bytes
SU- vs MU-MIMO	One vs many clients	Message 1 holds	ANonce, counter
ax 80 MHz, 1024-QAM	600.5 Mb/s per stream	Key that wraps GTK	KEK
Beacon interval	102.4 ms	PMKID	HMAC of PMK, AA, SPA
SSID vs BSSID	Name vs radio MAC	KRACK reinstalls	TK, nonce reset
Open auth gives	State 2 only	SAE gives	No offline guessing
AID range	1 to 2007	PMF stops	Forged deauth
Voice AIFSN, CW	2, 3 to 7	Needs client certs	EAP-TLS
A-MPDU vs A-MSDU	FCS per subframe vs one	OFDMA shares	Frequency (RUs)
Block Ack, ax	256 MPDUs	Trigger frame has	RU, MCS, power, length
DTIM 3	307 ms multicast	BSS colour	6 bits, HE preamble
2.4 GHz plan	1, 6, 11	Wi-Fi 7 factors	2 × 1.2 × 2
NR slot, µ = 3	125 µs	AMF vs SMF	Mobility vs sessions

PART I · PILLAR 7

7

Operating Systems and Socket Programming

Beneath every packet is a machine that schedules, locks, maps and interrupts; we learn to follow the packet through it.

7.1 What an operating system does	MUST
7.2 Processes, fork and exec	MUST
7.3 Threads and context switching	MUST
7.4 CPU scheduling	SHOULD
7.5 Synchronisation	MUST
7.6 Deadlock, starvation, priority inversion	MUST
7.7 Memory management and virtual memory	MUST
7.8 Interrupts, system calls and DMA	MUST
7.9 Inter-process communication and signals	SHOULD
7.10 Sockets from first principles	MUST
7.11 Server concurrency: threads, select, poll, epoll	MUST
7.12 The Linux receive path: NIC to recv()	MUST
7.13 Firmware and embedded essentials	SHOULD
7.14 Debugging a slow server	OPTIONAL
7.15 Mini-project: TinyRelay	OPTIONAL

7.0 Why this pillar matters, and how to read it

Networking products are programs running on an operating system, and the questions asked about them are operating-system questions: how a process differs from a thread, why two threads corrupt a counter, what a lock is, what happens when memory runs out, and how a packet gets from the wire to `recv()`. Cisco, Juniper and Arista ask these for software roles; Qualcomm and Broadcom ask them again for firmware, with interrupts, DMA and `volatile` added.

The chapter has three movements. Sections 7.1–7.9 are the classical core: processes, threads, synchronisation, deadlock, memory, interrupts. Sections 7.10–7.11 teach the Berkeley sockets API and the server architectures built on it; this is the canonical home of the API in the book. Sections 7.12–7.14 follow a packet through the Linux kernel, and 7.15 is a mini-project that joins the chapter. Two boundaries keep the book free of repetition. C memory, pointers and byte order belong to Pillar 2 (p. 105), and we use them here without re-teaching them. The behaviour of the TCP protocol itself, its state machine and congestion control, belongs to Pillar 3 (p. 162); here we meet TCP only as the thing a socket call triggers.

Every program in this chapter was compiled with `gcc -Wall -Wextra` on 64-bit Linux and run, and every printed output is from that run. Compile threaded programs with `-pthread`. Exact timings and counts vary between machines; where a number varies, the text gives its range and not a promise.

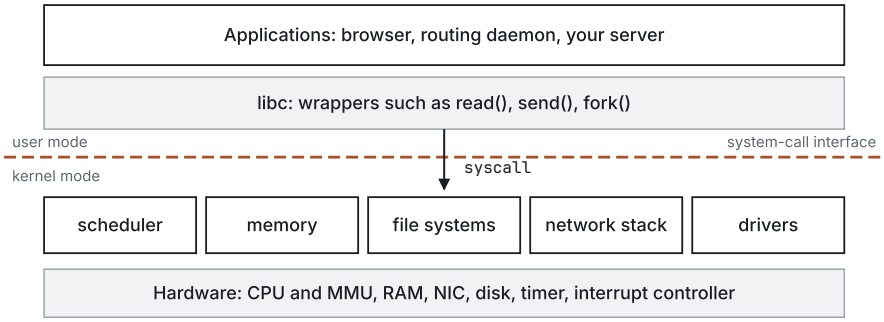
> BIG IDEA
>
> An operating system multiplexes one CPU, one memory and a set of devices among many programs, and it can do so safely only because the hardware gives it a privileged mode that programs do not have.

7.1 What an operating system does MUST · ■□□

An *operating system* is a resource manager and a protection boundary. As a manager it divides the CPU among runnable tasks (scheduling), gives each program its own view of memory (virtual memory), and shares disks and network cards through uniform abstractions (files, sockets). As a boundary it prevents a faulty or hostile program from reading another program's memory or taking over a device. The mechanism is a *mode bit* in the processor: in *user mode* the CPU refuses privileged instructions and accesses to kernel memory; in *kernel mode* it allows them. The *kernel* is the part of the operating system that runs in kernel mode.

Mode bit — one bit of CPU state. Think of a hostel warden's master key: residents (user mode) cannot copy it.

A program asks the kernel for service through a *system call*. The C library wrapper (`read()`, `send()`, `fork()`) loads the call number and arguments into registers and executes a special instruction (`syscall` on x86-64). The processor switches to kernel mode, jumps to a fixed entry point, switches to a kernel stack, and the kernel indexes a table of handlers by the call number. The result returns in a register, and a return instruction (`sysret`) restores user mode. Figure 7.1 shows the layers and the boundary.



A call crosses the dashed line twice: into the kernel by the syscall instruction, and back by sysret with the result in a register. Interrupts cross it from below, without a request from the program.

Figure 7.1 The user/kernel boundary. System calls cross it on request; interrupts cross it from below without a request.

LAB

`LAB 7.1 · 10 MIN` Count the system calls a trivial program makes.

```
$ strace -c ls /usr > /dev/null
% time     seconds  usecs/call     calls    errors syscall
 18.61    0.000190          27         7           read
 17.34    0.000177          35         5           mprotect
 11.75    0.000120          13         9           close
 10.19    0.000104          14         7           openat
  5.78    0.000059          29         2           getdents64
```

What you should notice: `ls` is a few lines of logic, yet it makes dozens of calls: `openat` to open the directory, `getdents64` to read its entries, `write` to print (not shown), `mprotect` and `mmap` from the dynamic loader. Every line of the table is a mode switch.

TRAP

Claim: "A system call is just a function call." *Why it is wrong:* it changes privilege level, switches stacks, and with the speculative-execution mitigations in current kernels costs from about a hundred nanoseconds to over a microsecond, against a few nanoseconds for an ordinary call. *Correct statement:* a system call is a controlled, expensive entry into the kernel; high-performance I/O designs (batching, `epoll`, `io_uring`, kernel bypass) exist to make fewer of them.

CHECK YOURSELF

1. Why can a user program not simply execute the instruction that disables interrupts? [1]
2. Name two things that move execution from user mode to kernel mode other than a system call. [2]

7.2 Processes, fork and exec MUST · ■■□

A *process* is a program in execution. The kernel represents it by a *process control block*, called `task_struct` in Linux, which records the identifier (PID), the scheduling state, the saved registers, a pointer to the memory map, the open files and the signal settings. Each process owns a private *virtual address space* (the memory model, segment by segment, is in Pillar 2 (p. 107)); Figure 7.2 shows how the control block points to it.

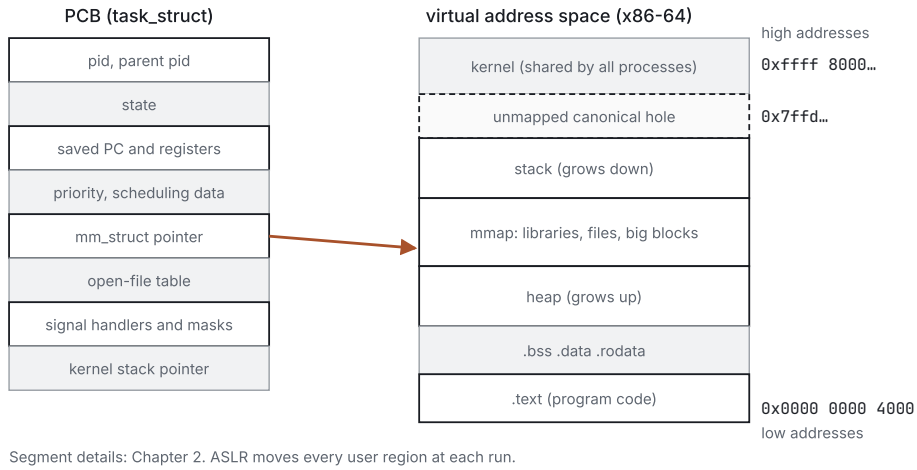


Figure 7.2 The PCB (left) and the address space it describes (right). The kernel half is the same in every process; the user half is private.

A process is always in one of five states, shown in Figure 7.3. It is *ready* when it could run but is waiting for a CPU, *running* when it holds one, and *waiting* (or blocked) when it needs an event such as disk or network data. Only the scheduler moves a task between ready and running; only an event moves it from waiting to ready.

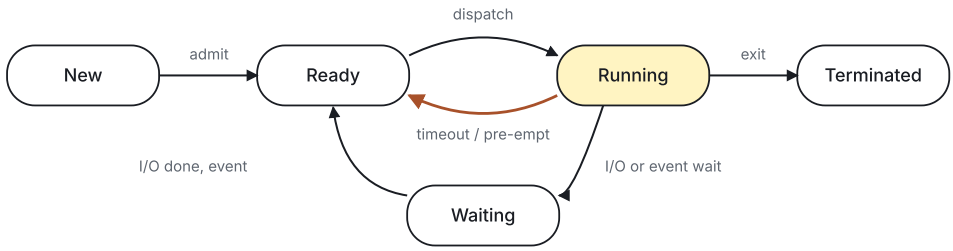


Figure 7.3 Process states. Only the accent-coloured transition, pre-emption, is forced on the running process.

Creating processes: fork, exec, wait

Linux creates a process by duplication. `fork()` returns twice: in the parent it returns the child's PID, and in the child it returns 0. The child receives a copy of the parent's address space, made cheap by *copy-on-write* (7.7 (p. 435)): both processes share the same physical pages, marked read-only, until one writes. `exec()` replaces the calling process's program with a new one while keeping its PID and open files. A shell runs a command as `fork`, then `exec` in the child, then `wait` in the parent. `wait()` returns the child's exit status and lets the kernel discard its control block.

The program below is the core of a shell: it forks, the child becomes the requested command through `execvp`, and the parent waits and reports how the child ended.

```c
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

int main(int argc, char **argv)          /* usage: ./run command [args...] */
{
    if (argc < 2) {
        fprintf(stderr, "usage: run command [args]\n");
        return 2;
    }
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }
    if (pid == 0) {                      /* child: become the command */
        execvp(argv[1], argv + 1);
        perror("execvp");                /* reached only if exec failed */
        _exit(127);
    }
    int st;
    waitpid(pid, &st, 0);                /* parent: wait, read the status */
    if (WIFEXITED(st))
        printf("pid %d exited with status %d\n", (int)pid, WEXITSTATUS(st));
    else if (WIFSIGNALED(st))
        printf("pid %d killed by signal %d\n", (int)pid, WTERMSIG(st));
    return 0;
}
```

```
$ ./run sh -c 'exit 3'
pid 17619 exited with status 3
$ ./run nosuchcmd
execvp: No such file or directory
pid 17620 exited with status 127
$ ./run sh -c 'kill -9 $$'
pid 17623 killed by signal 9
```

Note that the line `perror("execvp")` runs only when `exec` fails, since a successful `exec` never returns; the child then uses `_exit`, which skips the duplicated stdio buffers.

Fork puzzles ask how many lines a program prints. The rule is that each `fork()` doubles the number of processes that execute the code after it. Hence a loop of three forks yields $2^3 = 8$ processes, each printing the line below the loop.

```c
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void)
{
    for (int i = 0; i < 3; i++)
        fork();
    printf("hi\n");
    while (wait(NULL) > 0)      /* reap children so output is complete */
        ;
    return 0;
}
```

The output is eight lines (`./fork3 | wc -l` prints 8). The variant `fork() && fork() || fork()` is subtler, because `&&` and `||` short-circuit and the child sees a different return value from the parent. The parent gets a non-zero PID from both forks, so the `||` is skipped. The first

child gets 0 from its fork, skips the `&&` branch and evaluates the third fork; the second child gets 0 from its own fork and does the same. Counting the survivors gives five processes (Figure 7.4).

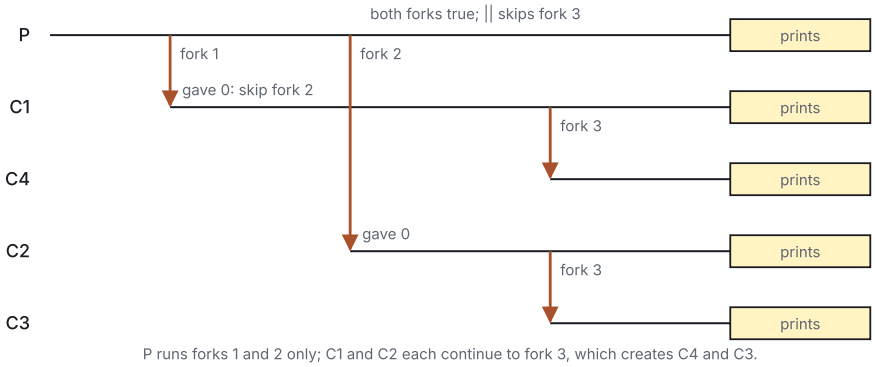


Figure 7.4 Process tree for fork() && fork() || fork(). Each process that finishes the expression prints once; there are five.

> **TRAP**
>
> *Claim:* "`printf("hi")` before a `fork` prints once." *Why it is wrong:* when standard output is a pipe or file it is fully buffered, so the unflushed text sits in the process's memory and is copied into the child. Both processes flush it at exit. *Correct statement:* the text is printed twice; end the line with `\n` on a terminal, or call `fflush(stdout)` before forking.

Zombies and orphans

When a child exits, the kernel keeps its control block until the parent calls `wait()`, so the parent can still read the exit status. Between exit and `wait()` the child is a *zombie*: it uses no memory beyond the block, but it occupies a PID, and a parent that never waits eventually exhausts them. An *orphan* is the opposite case, a child whose parent has exited first. The kernel re-parents it to PID 1 (or to a designated subreaper), which calls `wait()` for it.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void)
{
    pid_t pid = fork();
    if (pid == 0)
        _exit(7);                       /* child exits at once */
    printf("child %d is a zombie for 5 s\n", (int)pid);
    fflush(stdout);
    sleep(5);                           /* run: ps -o pid,ppid,stat,cmd */
    int st;
    waitpid(pid, &st, 0);               /* reaping removes the zombie */
    printf("exit status %d\n", WEXITSTATUS(st));
    return 0;
}
```

> **LAB**
>
> `LAB 7.2 · 10 MIN` Create and reap a zombie. Run `./zombie &`, then in the same second:

```
$ ps -o pid,ppid,stat,cmd --ppid $(pgrep zombie)
  PID  PPID STAT CMD
 8935  8933 Z    [zombie] <defunct>
exit status 7
```

> **What you should notice:** state `Z` and the label `<defunct>` mark the zombie. It cannot be killed, because it is already dead; only its parent's `wait()` (or the parent's death) removes it.

> INTERVIEW LENS
>
> CISCO · JUNIPER · QUALCOMM
>
> "How many times does this print?" with `fork` in a loop, and "what is a zombie?", are reported in Cisco, Juniper and Qualcomm technical rounds. A strong answer states the doubling rule, draws the process tree, mentions stdout buffering if the output is redirected, and gives both the zombie definition and its cause (the parent did not wait). Follow-ups: how `exec` differs from `fork`, and what the shell does between them.

> CHECK YOURSELF
>
> 1. How many processes exist after `fork(); fork(); fork();` executed once by one process? [3]
> 2. What does `fork()` return in the child? [4]
> 3. Does `exec()` create a new process? [5]

7.3 Threads and context switching MUST · ■■□

A *thread* is an execution context inside a process. All threads of a process share the code, the global data, the heap and the open files; each has its own stack, its own saved registers and its own *thread-local storage*, including `errno`. Because the address space is shared, creating a thread needs no new page tables and communicating between threads needs no kernel call: a write by one thread is visible to the others. The same sharing is the hazard.

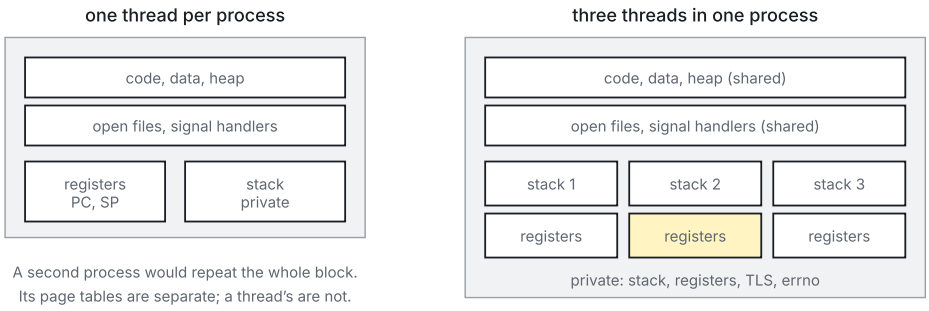


Figure 7.5 Process versus threads. Threads share everything except stack, registers and thread-local storage.

Linux implements a thread as a task that shares its parent's address space (the `clone()` system call with the sharing flags set), which is the *1:1* model: one kernel-scheduled task per user thread, so threads run in parallel on different cores. Runtimes such as Go multiplex many lightweight tasks onto few kernel threads (*M:N*), gaining cheap creation at the cost of a second scheduler.

> BIG IDEA
>
> Sharing memory makes threads cheap to create and to switch, and it is also why a bug in one thread can corrupt, or terminate, every other thread in the process.

The race condition

Two threads that each increment a shared counter one million times should produce two million. They do not, because `counter++` compiles to three steps (load, add, store), and the threads interleave between them. the table below shows how one update is lost.

Step	Thread A	Thread B	counter in memory
1	load counter (5)		5
2		load counter (5)	5
3	add 1 (6)		5
4		add 1 (6)	5
5	store 6		6
6		store 6	6

Both threads read 5 and both store 6, so one increment vanishes.

```
#include <pthread.h>
#include <stdio.h>

#define N 1000000
static volatile long counter;   /* volatile: keep it out of a register */
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

static void *plain(void *arg)
{
    (void)arg;
    for (int i = 0; i < N; i++)
        counter++;                      /* load, add, store: not atomic */
    return NULL;
}

static void *locked(void *arg)
{
    (void)arg;
    for (int i = 0; i < N; i++) {
        pthread_mutex_lock(&m);
        counter++;
        pthread_mutex_unlock(&m);
    }
    return NULL;
}

static void run(void *(*fn)(void *), const char *name)
{
    pthread_t a, b;
    counter = 0;
    pthread_create(&a, NULL, fn, NULL);
    pthread_create(&b, NULL, fn, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("%-8s counter = %ld (expected %d)\n", name, counter, 2 * N);
}

int main(void)
{
    run(plain, "plain");
    run(locked, "mutex");
    return 0;
}
```

```
$ gcc -O1 -pthread race.c -o race && ./race
plain    counter = 1092934 (expected 2000000)
mutex    counter = 2000000 (expected 2000000)
```

The unprotected total falls anywhere between about one and two million, and differs on each run. The `volatile` qualifier is present only to stop the optimiser from keeping the counter in a register, which would hide the race on some compilers; it does not fix it (7.13 (p. 463)). The ThreadSanitizer (`gcc -fsanitize=thread -g`) reports it directly: `WARNING: ThreadSanitizer: data race`, with the two source lines involved.

What a context switch does

A *context switch* stops one task and starts another. The kernel (i) saves the running task's registers and program counter into its control block, (ii) lets the scheduler choose the next task, (iii) if the next task belongs to a different process, loads its page-table base register, which invalidates or retags the TLB entries (7.7 (p. 435)), and (iv) restores the next task's registers. Step (iii) is the reason that switching between threads of the same process is cheaper than switching between processes: the address space does not change, so the cached translations and, to a lesser degree, the data caches stay useful. The direct cost of a switch is a few microseconds; the indirect cost, the cold caches afterwards, is often larger.

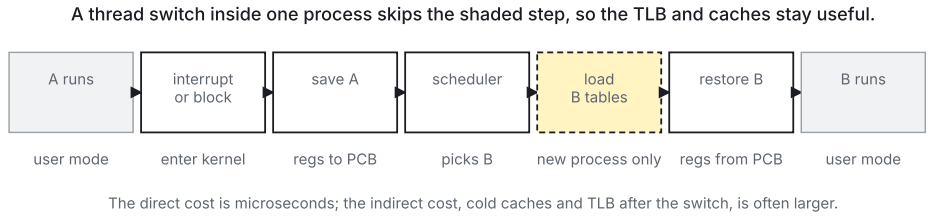


Figure 7.6 The steps of a context switch. The shaded step is skipped when both tasks are threads of one process.

LAB

`LAB 7.3 · 10 MIN` Let ThreadSanitizer find the race, then confirm the fix.

```
$ gcc -fsanitize=thread -g -O1 race.c -o racet -pthread && ./racet
WARNING: ThreadSanitizer: data race
  Read of size 8 ... by thread T2:
    #0 plain race.c:12
SUMMARY: ThreadSanitizer: data race race.c:12 in plain
```

What you should notice: every report names a line in `plain` (line 12, the unprotected increment) and none names `locked`. The sanitizer finds races that did not happen to corrupt the output on this run, which is why it belongs in every threaded project's test build.

TRAP

Claim: "Threads have their own heaps" or "a thread switch flushes the TLB." *Why it is wrong:* the heap belongs to the process, and a switch inside one process keeps the same page tables. *Correct statement:* threads share the heap; only the stack, registers and thread-local storage are private, and a TLB flush happens only when the address space changes.

INTERVIEW LENS

CISCO · QUALCOMM · JUNIPER · ARISTA

"Process versus thread" is asked at nearly every networking employer, usually followed by "what happens if one thread crashes?". A strong answer gives the shared and private lists, states that a fault such as SIGSEGV is delivered to the process and ends all threads, and compares the switch cost with a reason (no address-space change). Follow-ups: thread safety and reentrancy (`strtok` keeps a hidden static pointer, so it is unsafe; `strtok_r` takes the state as an argument), and why a server prefers a pool of threads to one thread per request.

CHECK YOURSELF

1. Which segments are private to a thread? [6]
2. Why is the unprotected counter result different on each run? [7]

7.4 CPU scheduling SHOULD · ■■□

The *scheduler* chooses which ready task runs next, balancing throughput (work finished per second), latency (time to first response) and fairness. A *non-pre-emptive* algorithm lets a task run until it blocks or finishes; a *pre-emptive* one can interrupt it, normally on a timer tick. For one task we define the *turnaround time* TAT = completion − arrival and the *waiting time* WT = TAT − burst, where the burst is the CPU time needed.

We compare four algorithms on five processes: P1 arrives at 0 with burst 8, P2 at 1 with 4, P3 at 2 with 9, P4 at 3 with 5, P5 at 4 with 2.

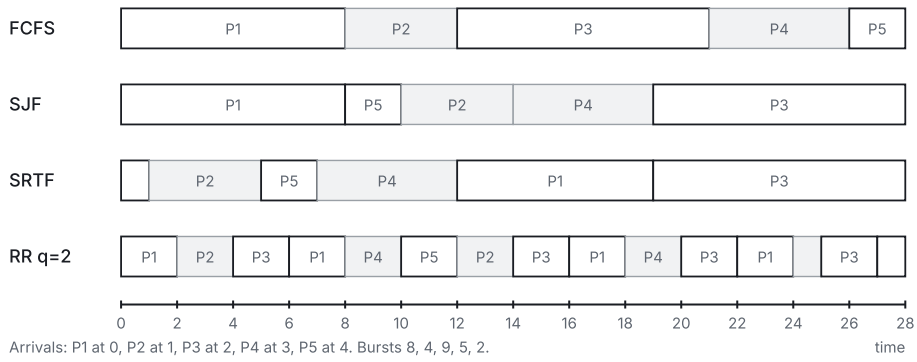


Figure 7.7 Gantt charts for the five processes. Shorter jobs first (SJF, SRTF) cut the average wait; round robin raises it here but bounds the response time.

WORKED EXAMPLE

Average waiting time under FCFS. First-come-first-served runs in arrival order, so P1 runs 0–8, P2 8–12, P3 12–21, P4 21–26, P5 26–28.

1. Waiting time = start − arrival for a task that runs without interruption: P1 0, P2 8 − 1 = 7, P3 12 − 2 = 10, P4 21 − 3 = 18, P5 26 − 4 = 22.
2. Sum = 57, average = 57/5.

Result: average waiting time 11.4, average turnaround 17.0. The short P5 waits behind the long P3, the *convoy effect*.

Algorithm	Pre-emptive?	Avg waiting	Avg turnaround	Property
FCFS	no	11.4	17.0	simple; convoy effect

SJF (shortest job first)	no	8.2	13.8	optimal waiting among non-pre-emptive; needs burst lengths; starves long jobs
SRTF (shortest remaining time first)	yes	6.6	12.2	optimal average waiting overall; many switches
Round robin, q = 2	yes	13.0	18.6	fair; response bounded by (n − 1)q; q too small wastes time in switches

Round robin lets each task run for at most a *quantum* q and then moves it to the back of the queue. Its numbers above assume that a task arriving at the instant a quantum ends joins the queue before the pre-empted task; the opposite convention changes the order and the totals slightly, so a question should state it. *Priority scheduling* runs the highest-priority ready task; low-priority tasks may *starve*, and *ageing*, which slowly raises the priority of waiting tasks, cures it. A *multilevel feedback queue* (MLFQ) combines these ideas: new tasks enter the top queue with a short quantum, a task that uses its whole quantum is demoted, and a periodic boost lifts everything back so that nothing starves.

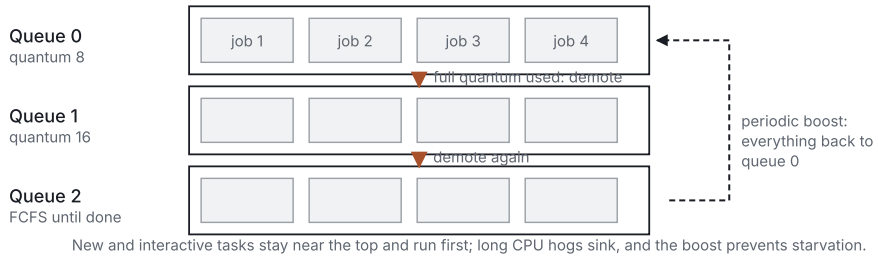


Figure 7.8 A multilevel feedback queue with three levels.

Linux does not use a fixed quantum. The Completely Fair Scheduler kept a *virtual runtime* for each task, advanced more slowly for tasks of higher weight, and always ran the task with the smallest value; two tasks with weights 2 : 1 therefore receive CPU in the ratio 2 : 1. From Linux 6.6 the pick logic is EEVDF (earliest eligible virtual deadline first), which keeps the weighted-fair accounting but also lets latency-sensitive tasks ask for shorter slices. The interview-level statement is that Linux gives each runnable task a weighted fair share and picks the one that has had the least.

> **TRAP**
>
> *Claim:* "SJF is the best algorithm." *Why it is wrong:* it is optimal for average waiting time only if burst lengths are known in advance, which they are not, and it starves long jobs when short ones keep arriving. *Correct statement:* real schedulers estimate the next burst from history (exponential averaging) and use ageing or feedback to prevent starvation.

> **CHECK YOURSELF**
>
> 1. In FCFS, which of TAT and WT is larger for a task, and by how much? [8]
> 2. What does ageing prevent? [9]

1. It is a privileged instruction; in user mode the CPU raises a fault and the kernel decides what

7.5 Synchronisation MUST · ■■□

A *critical section* is a stretch of code that touches shared data and must not be executed by two threads at once. Enforcing this is *mutual exclusion*, and the tools differ in who may release them, whether a waiter sleeps, and where they may be used. We meet five: the mutex, the semaphore, the spinlock, the condition variable and the read-write lock.

A *mutex* (mutual-exclusion lock) has two states and an owner. `pthread_mutex_lock` takes it, sleeping if another thread holds it; `pthread_mutex_unlock` releases it, and only the owner may do so. Figure 7.9 shows the second thread being put to sleep and woken. The race in 7.3 (p. 420) disappears when the increment is placed between the two calls, at the price of slower code. On the four-core test machine, two threads making two million increments in total took about 120 ms under a contended mutex and about 20 ms with `__atomic_fetch_add`, an *atomic* instruction that performs the single-word update without ever sleeping; the unprotected loop was faster still, and wrong. Only the ordering of these figures should be trusted, not their values.

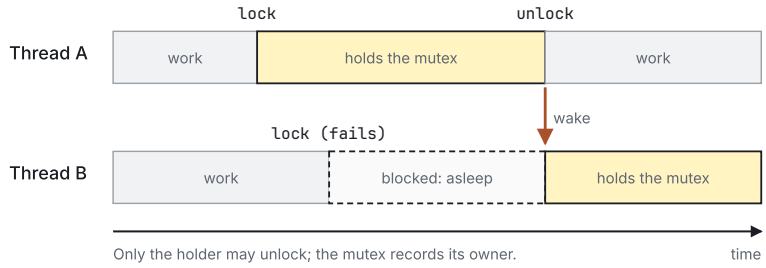


Figure 7.9 A mutex timeline. Thread B fails to lock, sleeps, and is woken when A unlocks.

A *semaphore* is a non-negative counter with two atomic operations: `wait` (also P or down) decrements it, sleeping while it is zero, and `post` (V or up) increments it and wakes a sleeper. It has no owner, so any thread may post. A *counting* semaphore initialised to N admits N threads at once, as a pool of N identical resources; a *binary* semaphore (N = 1) behaves like a lock but may be released by a different thread than the one that took it, which makes it a signalling device between threads.

A *spinlock* does not sleep: a waiter loops on an atomic test until the lock is free. This is wasteful on a single core and fine for very short critical sections on several. It is the only lock usable where sleeping is forbidden, such as an interrupt handler (7.8 (p. 441)), because a sleeping task cannot be scheduled from interrupt context. A naive test-and-set loop writes to the lock's cache line on every attempt, so the line bounces between cores; the *test-and-test-and-set* refinement spins on an ordinary read, which hits the local cache, and attempts the atomic write only when the lock looks free.

happens, normally terminating the program.↩

2. A hardware interrupt (timer, NIC) and an exception such as a page fault or a divide by zero.↩
3. Eight, counting the original: each fork doubles the count.↩
4. Zero. In the parent it returns the child's PID; on failure it returns -1 and no child exists.↩
5. No. It replaces the program image of the calling process; the PID and open descriptors survive.↩
6. The stack, the registers (including the program counter) and thread-local storage.↩
7. The interleaving depends on when the scheduler pre-empts each thread, which varies with load and timing; the lost updates vary with it.↩
8. TAT is larger, by exactly the burst time, since WT = TAT − burst.↩
9. Starvation of low-priority tasks, by raising a task's priority the longer it waits.↩

Semaphore — a counter with two operations. A car park with a "spaces free: 5" board: `wait` takes a space, `post` returns one.

```
#include <pthread.h>
#include <stdio.h>

#if defined(__x86_64__)
#define cpu_relax() __builtin_ia32_pause()   /* ease the sibling thread */
#else
#define cpu_relax() ((void)0)
#endif

typedef struct { volatile char flag; } spinlock_t;

static void spin_lock(spinlock_t *l)
{
    for (;;) {
        /* test-and-set: atomically write 1 and return the old value */
        if (!__atomic_test_and_set(&l->flag, __ATOMIC_ACQUIRE))
            return;
        /* test-and-test-and-set: spin on a plain read, which stays in
           the local cache, and retry the atomic only when it looks free */
        while (__atomic_load_n(&l->flag, __ATOMIC_RELAXED))
            cpu_relax();
    }
}

static void spin_unlock(spinlock_t *l)
{
    __atomic_clear(&l->flag, __ATOMIC_RELEASE);
}

static spinlock_t lock;
static long counter;

static void *work(void *arg)
{
    (void)arg;
    for (int i = 0; i < 1000000; i++) {
        spin_lock(&lock);
        counter++;
        spin_unlock(&lock);
    }
    return NULL;
}

int main(void)
{
    pthread_t t[4];
    for (int i = 0; i < 4; i++) pthread_create(&t[i], NULL, work, NULL);
    for (int i = 0; i < 4; i++) pthread_join(t[i], NULL);
    printf("counter = %ld (expected 4000000)\n", counter);
    return 0;
}
```

The program prints `counter = 4000000 (expected 4000000)`. Note the acquire and release memory orders: acquire on lock and release on unlock ensure that the data protected by the lock is seen consistently by the next holder.

A *condition variable* lets a thread sleep until a predicate on shared data becomes true. It is always used with a mutex: `pthread_cond_wait(&c, &m)` atomically releases the mutex and sleeps, and re-acquires the mutex before returning. The waiter must recheck the predicate in a `while` loop, not an `if`, for two reasons: a wake-up may be *spurious* (the standard permits it), and a wake-up may be *stolen* (another thread takes the item between the signal and the waiter's re-acquisition of the mutex).

BIG IDEA

> A condition variable does not remember signals; the predicate on the shared data is the truth, and the loop around `cond_wait` is how a thread rechecks it.

Worked example: the bounded buffer

The *producer–consumer* problem asks for a fixed-size queue shared by threads that add items and threads that remove them: producers must wait when the queue is full, consumers when it is empty. The solution below uses one mutex and two condition variables. Three producers make 10,000 items between them, two consumers take them, and the program compares the sum put in with the sum taken out.

```
#include <pthread.h>
#include <stdio.h>

#define SLOTS 8
#define ITEMS 10000
#define PRODUCERS 3
#define CONSUMERS 2

static int buf[SLOTS];
static int head, tail, count;
static int produced, consumed;          /* items handed out / taken so far */
static long sum_in, sum_out;
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t not_full = PTHREAD_COND_INITIALIZER;
static pthread_cond_t not_empty = PTHREAD_COND_INITIALIZER;

static void *producer(void *arg)
{
    (void)arg;
    for (;;) {
        pthread_mutex_lock(&m);
        if (produced == ITEMS) {        /* nothing left to make */
            pthread_mutex_unlock(&m);
            return NULL;
        }
        int item = ++produced;          /* claim item number under the lock */
        while (count == SLOTS)          /* while, not if */
            pthread_cond_wait(&not_full, &m);
        buf[tail] = item;
        tail = (tail + 1) % SLOTS;
        count++;
        sum_in += item;
        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&m);
    }
}
```

The consumer mirrors the producer, and `main` starts the threads and compares the sums:

```
static void *consumer(void *arg)
{
    (void)arg;
    for (;;) {
        pthread_mutex_lock(&m);
        while (count == 0 && consumed < ITEMS)
            pthread_cond_wait(&not_empty, &m);
        if (count == 0) {               /* all items consumed: finish */
            pthread_mutex_unlock(&m);
            return NULL;
        }
        int item = buf[head];
        head = (head + 1) % SLOTS;
        count--;
        consumed++;
        sum_out += item;
        pthread_cond_signal(&not_full);
        if (consumed == ITEMS)
            pthread_cond_broadcast(&not_empty);   /* release idle consumers */
        pthread_mutex_unlock(&m);
    }
}

int main(void)
{
    pthread_t p[PRODUCERS], c[CONSUMERS];
    for (int i = 0; i < PRODUCERS; i++)
        pthread_create(&p[i], NULL, producer, NULL);
    for (int i = 0; i < CONSUMERS; i++)
        pthread_create(&c[i], NULL, consumer, NULL);
    for (int i = 0; i < PRODUCERS; i++)
        pthread_join(p[i], NULL);
    for (int i = 0; i < CONSUMERS; i++)
        pthread_join(c[i], NULL);
    printf("sum in = %ld, sum out = %ld, %s\n", sum_in, sum_out,
           sum_in == sum_out ? "match" : "MISMATCH");
    return 0;
}
```

```
$ gcc -Wall -Wextra -O2 -pthread bbuf.c -o bbuf && ./bbuf
sum in = 50005000, sum out = 50005000, match
```

Four details deserve attention. First, `count` is read and written only with the mutex held, so it is consistent. Second, a producer waits on `not_full` and a consumer on `not_empty`, so each signal wakes only a thread that can make progress. Third, the consumers must be released at the end; the `broadcast` when the last item is taken does this, and without it a consumer sleeping on an empty buffer would never return. Fourth, the same problem with semaphores uses `empty` (initially the number of slots) and `full` (initially 0), plus a mutex around the buffer indices:

```c
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>

#define SLOTS 4
#define ITEMS 1000

static int buf[SLOTS], in, out;
static sem_t empty, full;                  /* free slots, filled slots */
static pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
static long sum;

/* Taking m before waiting on empty would deadlock on a full buffer: the
   producer would sleep holding m, and the consumer could never get m. */
static void *producer(void *arg)
{
    (void)arg;
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&empty);              /* first reserve a slot ... */
        pthread_mutex_lock(&m);        /* ... then take the lock */
        buf[in] = i;
        in = (in + 1) % SLOTS;
        pthread_mutex_unlock(&m);
        sem_post(&full);
    }
    return NULL;
}

static void *consumer(void *arg)
{
    (void)arg;
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&m);
        sum += buf[out]; out = (out + 1) % SLOTS;
        pthread_mutex_unlock(&m);
        sem_post(&empty);
    }
    return NULL;
}

int main(void)
{
    pthread_t p, c;
    sem_init(&empty, 0, SLOTS);
    sem_init(&full, 0, 0);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL); pthread_join(c, NULL);
    printf("sum = %ld (expected %d)\n", sum, ITEMS * (ITEMS + 1) / 2);
    return 0;
}
```

The order of the two waits in the producer matters. If the producer took the mutex first and then waited on `empty`, a full buffer would leave it asleep holding the mutex; the consumer could then never take the mutex to remove an item and make room, and both would wait for ever. This is a deadlock (7.6 (p. 431)).

Choosing a primitive

	Mutex	Binary semaphore	Counting semaphore	Spinlock	Condition variable

Owner	yes, the locker	no	no	yes, by convention	not a lock
Waiter sleeps?	yes	yes	yes	no, spins	yes
Usable in an interrupt handler?	no	`post` only	`post` only	yes	no
Typical use	protect shared data	signal an event	limit a pool of N	very short sections, kernel	wait for a predicate
Linux kernel API	`mutex_lock`	`down` / `up`	`down` / `up`	`spin_lock_irqsave`	`wait_event`

A mutex and a binary semaphore are not interchangeable. Because a mutex has an owner, the system can lend the owner a higher priority when a more urgent thread waits (priority inheritance, 7.6 (p. 431)), can detect an unlock by the wrong thread, and can offer recursive and error-checking variants. A semaphore, lacking an owner, can be posted from an interrupt handler or from another thread, which is exactly what signalling needs.

Dining philosophers

Five philosophers sit round a table with one fork between each pair of neighbours; each needs both adjacent forks to eat. If every philosopher takes the left fork and then waits for the right, all five hold one fork and wait for another: the Coffman conditions hold and nobody ever eats. The remedy follows from the prevention table in 7.6 (p. 431): number the forks and make each philosopher take the lower-numbered fork first. Four philosophers still take their left fork first, but the fifth must take fork 0 before fork 4, so the cycle cannot close. Other cures, a waiter who admits at most four philosophers at once or `trylock` with back-off, break hold-and-wait instead.

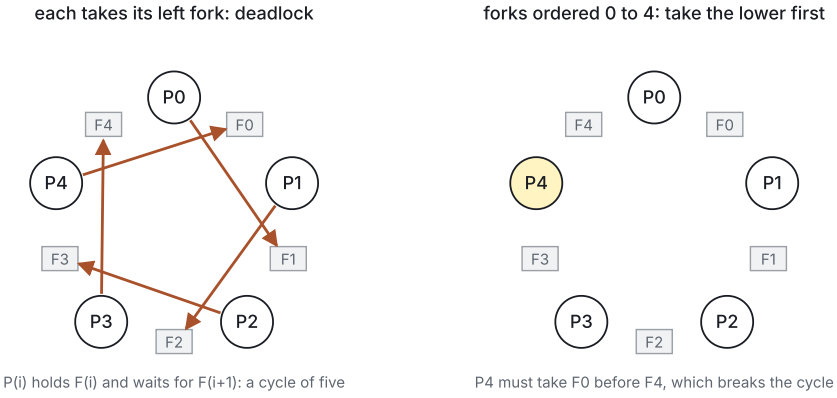


Figure 7.10 Dining philosophers. Left: the circular wait. Right: ordering the forks removes it.

Read-write locks

When many threads read and few write, `pthread_rwlock` lets any number of readers hold the lock together but gives a writer exclusive access. A reader-preferring implementation can *starve* a writer that waits behind a steady stream of readers; a writer-preferring one blocks new readers once a writer is waiting, trading reader latency for freedom from starvation.

> INTERVIEW LENS
>
> JUNIPER · CISCO · QUALCOMM · ARISTA
>
> “Implement a mutex or semaphore”, “mutex versus semaphore” and “write a multithreaded program with a lock” are reported Juniper and Cisco questions, and Qualcomm asks which lock is safe inside an interrupt handler. A strong answer gives the table above in words, states the ownership difference, writes the bounded buffer with a `while` around the wait, and exp-

lains why a spinlock is correct in an interrupt handler (nothing may sleep there) but wasteful in user code. Follow-ups: how to build a semaphore from a mutex and a condition variable (Q7.11), and what happens if `unlock` is called by a thread that does not own a mutex (undefined for a default mutex, an error for an error-checking one).

TRAP

Claim: "`volatile int flag` protects a critical section." *Why it is wrong:* `volatile` stops the compiler from caching the variable but does not make a read-modify-write atomic and does not order other memory accesses. *Correct statement:* use a lock or an atomic operation with the right memory order; `volatile` is for device registers (7.13 (p. 463)).

CHECK YOURSELF

1. A consumer wakes from `cond_wait` and finds the queue empty. Why can this happen? [1]
2. Which of mutex and spinlock may be used in an interrupt handler? [2]
3. Can a thread unlock a mutex that another thread locked? [3]

7.6 Deadlock, starvation and priority inversion MUST · ■■□

A set of tasks is *deadlocked* when each waits for a resource that another task in the set holds, so none can proceed. Four conditions, the *Coffman conditions*, must hold together: (i) *mutual exclusion*, a resource is held by at most one task; (ii) *hold and wait*, a task holds one resource while requesting another; (iii) *no pre-emption*, a resource cannot be taken away; (iv) *circular wait*, a cycle of tasks each waits for the next. Hence preventing deadlock means breaking any one of them.

Condition	Prevention	Cost
Mutual exclusion	make the resource shareable (read-only data, lock-free structures)	not possible for a printer or a write
Hold and wait	request all resources at once, or release before requesting	low utilisation, possible starvation
No pre-emption	if a request fails, release everything held and retry (`pthread_mutex_trylock`)	restart cost; livelock if both retry in step
Circular wait	impose a global order on locks and always acquire in that order	the standard practical answer

A *resource-allocation graph* draws tasks as circles and resources as squares, with a request edge from a task to a resource and an assignment edge from a resource to a task. A cycle is the circular-wait condition made visible.

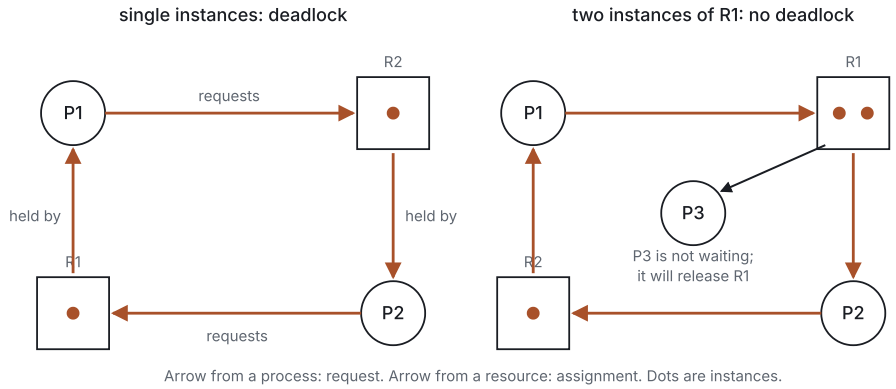


Figure 7.11 Resource-allocation graphs. Left: a cycle with single-instance resources is a deadlock. Right: a cycle with a multi-instance resource is not, since P3 can finish and release R1.

> **TRAP**
>
> *Claim:* "A cycle in the resource graph always means deadlock." *Why it is wrong:* with several instances of a resource, a task outside the cycle may release one and let the cycle dissolve. *Correct statement:* a cycle is necessary for deadlock; it is sufficient only when every resource has a single instance.

Worked example: lock ordering

Thread 1 locks A then B; thread 2 locks B then A. If each takes its first lock before either takes its second, both wait for ever. The program below does this deliberately, with a short sleep to make the bad interleaving certain.

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

static pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;

static void *t1(void *x)
{
    (void)x;
    pthread_mutex_lock(&A);                 /* order: A, then B */
    usleep(100000);                         /* make the bad interleaving certain */
    pthread_mutex_lock(&B);
    puts("t1 done");
    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    return NULL;
}

static void *t2(void *x)
{
    (void)x;
#ifdef FIXED
    pthread_mutex_lock(&A);                 /* same order as t1: no cycle */
    usleep(100000);
    pthread_mutex_lock(&B);
#else
    pthread_mutex_lock(&B);                 /* order: B, then A: a cycle */
    usleep(100000);
    pthread_mutex_lock(&A);
#endif
    puts("t2 done");
    pthread_mutex_unlock(&A);
    pthread_mutex_unlock(&B);
    return NULL;
}

int main(void)
{
    pthread_t a, b;
    pthread_create(&a, NULL, t1, NULL);
    pthread_create(&b, NULL, t2, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    return 0;
}
```

```
$ gcc -g -pthread dl.c -o dl && ./dl &
$ gdb -p $! -batch -ex "thread apply all bt 2"
Thread 3 ...  #1  __lll_lock_wait (futex=... <B>, private=0)
Thread 2 ...  #1  __lll_lock_wait (futex=... <A>, private=0)
Thread 1 ...  #0  __futex_abstimed_wait_common64 ... pthread_join
```

What you should notice: two threads are stuck inside `__lll_lock_wait`, one on lock `B` and one on lock `A`, while the main thread waits in `pthread_join`. Compiled with `-DFIXED`, the second thread takes A first, as the first does, and the program prints both “done” lines: a global lock order removes the cycle.

Worked example: the banker’s algorithm

Deadlock avoidance lets a system grant a request only if the resulting state is *safe*, meaning some order exists in which every task can obtain its maximum claim and finish. The banker’s algorithm

tests this. Take five tasks and three resource types with totals A = 10, B = 5, C = 7. Each task has declared a maximum claim; the *need* is the maximum minus what it holds now.

Task	Allocation (A B C)	Max (A B C)	Need = Max − Alloc
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

WORKED EXAMPLE

Is the state safe? Available = totals − the column sums of Allocation = (10, 5, 7) − (7, 2, 5) = (3, 3, 2).

1. Work = (3, 3, 2). P1's need (1, 2, 2) fits. Run it to completion and take back its allocation: Work = (3, 3, 2) + (2, 0, 0) = (5, 3, 2).
2. P3 needs (0, 1, 1), which fits. Work = (5, 3, 2) + (2, 1, 1) = (7, 4, 3).
3. P4 needs (4, 3, 1), which fits. Work = (7, 4, 3) + (0, 0, 2) = (7, 4, 5).
4. P0 needs (7, 4, 3), which fits. Work = (7, 4, 5) + (0, 1, 0) = (7, 5, 5).
5. P2 needs (6, 0, 0), which fits. Work = (7, 5, 5) + (3, 0, 2) = (10, 5, 7), the totals.

Result: the sequence ⟨P1, P3, P4, P0, P2⟩ finishes everyone, so the state is safe.

Request test. Suppose P1 now requests (1, 0, 2). It is within its need (1, 2, 2) and within Available (3, 3, 2). Pretend to grant it: Available becomes (2, 3, 0), P1 holds (3, 0, 2) and needs (0, 2, 0). P1's need fits in (2, 3, 0), so it can finish and return its holdings, giving (5, 3, 2), after which the sequence above completes. The new state is safe, so the request is granted. By contrast, a request by P4 for (3, 3, 0) leaves Available at (0, 0, 2) with every need larger than that in some column; no task can proceed, the state is unsafe, and the request must wait even though resources are free at this moment.

Detection and recovery are the alternative when avoidance is too costly: build the wait-for graph, look for a cycle, and break it by terminating or rolling back a task. General-purpose systems mostly ignore deadlock and leave lock discipline to programmers; databases detect and abort a victim transaction.

Starvation and priority inversion

Starvation is indefinite postponement of a task that is ready but never chosen: a long job under SJF, a low-priority task under a busy high-priority load, a writer behind endless readers. Unlike deadlock, the system as a whole keeps moving; ageing, fair queues and writer-preferring locks are the cures.

Priority inversion occurs when a high-priority task H waits for a lock held by a low-priority task L, and a medium-priority task M, needing neither, pre-empts L. H is then delayed by M although H has the higher priority. In 1997 this reset the Mars Pathfinder lander's computer repeatedly: a low-priority meteorological task held a shared bus lock, a medium task ran, and the high-priority bus-management task missed its deadline, triggering the watchdog. The remedy is *priority inheritance*: while L holds a lock that H wants, L runs at H's priority, so M cannot pre-empt it; the Pathfinder team enabled this option on the lock remotely.

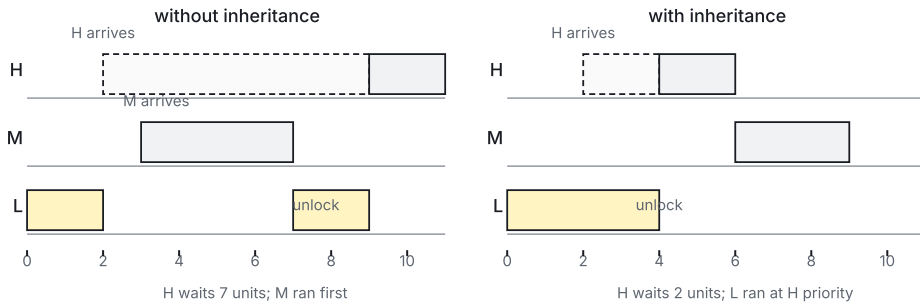


Figure 7.12 Priority inversion. Without inheritance, H waits for M's whole run. With inheritance, L finishes its critical section at H's priority and H resumes after two units.

> INTERVIEW LENS
>
> QUALCOMM · CISCO · JUNIPER
>
> "Name the four conditions for deadlock" and "run the banker's algorithm on this matrix" are reported in written tests and technical rounds; "explain priority inversion" is common for firmware and RTOS roles. A strong answer lists the conditions, gives a lock-ordering example, and for the banker's algorithm shows the Work vector at every step. Follow-ups: deadlock versus livelock (tasks keep changing state but make no progress) and why Linux's `PTHREAD_PRIO_INHERIT` mutexes exist.

> CHECK YOURSELF
>
> 1. Which Coffman condition does a global lock order break? [4]
> 2. Why is a safe state not the same as "no deadlock now"? [5]

7.7 Memory management and virtual memory MUST · ■■□

Each process addresses memory through *virtual addresses*, and a hardware unit, the *memory-management unit* (MMU), translates them to *physical addresses* using tables the kernel maintains. Memory is divided into fixed-size *pages* (4 KB on x86-64 and most ARM systems) in the virtual space and equal-sized *frames* in physical memory; a *page table* maps each virtual page number (VPN) to a physical frame number (PFN). Three benefits follow. Processes are isolated, since a process can name only the pages its table maps. Memory can be fragmented physically yet contiguous virtually. And the kernel can defer work: a page need not exist until it is first used.

> BIG IDEA
>
> Virtual memory is chiefly a translation-and-protection mechanism; using disk as spare memory is a secondary benefit, and a process that never touches a page never owns a frame for it.

Translating an address

A virtual address splits into a page number and an *offset* within the page. With 4 KB pages the offset takes the low 12 bits ($2^{12} = 4096$), and the translation replaces the page number by the frame number and keeps the offset.

> WORKED EXAMPLE

1. The wake-up was spurious, or another consumer took the item first; hence the `while` loop that rechecks the predicate. ↩
2. The spinlock. A mutex may sleep, which is not allowed in interrupt context. ↩
3. Not legitimately; a mutex has an owner. A semaphore has none, so any thread may post it. ↩
4. Circular wait: with a fixed order, no cycle of waiting tasks can form. ↩
5. Safety is a guarantee about the future, that some order lets everyone finish; an unsafe state may not be deadlocked yet but may become so. ↩

Translate 0x00403ABC in a 32-bit address space with 4 KB pages. The page table maps VPN 0x00403 to PFN 0x1A2.

1. Offset = the low 12 bits = 0xABC. VPN = the address shifted right by 12 = 0x00403.
2. Look up VPN 0x00403: PFN = 0x1A2.
3. Physical address = PFN × 4096 + offset = 0x1A2000 + 0xABC.

Result: physical address 0x1A2ABC.

A flat table for 2^{20} pages with 4-byte entries needs 4 MB for every process, even a tiny one. A *multi-level* table avoids this by allocating the lower levels only where addresses are in use. With a 10 + 10 + 12 split the top 10 bits index a first-level table whose entries point to second-level tables, and the next 10 bits index the second-level table; Figure 7.13 follows 0x00403ABC through it (first index 1, second index 3).

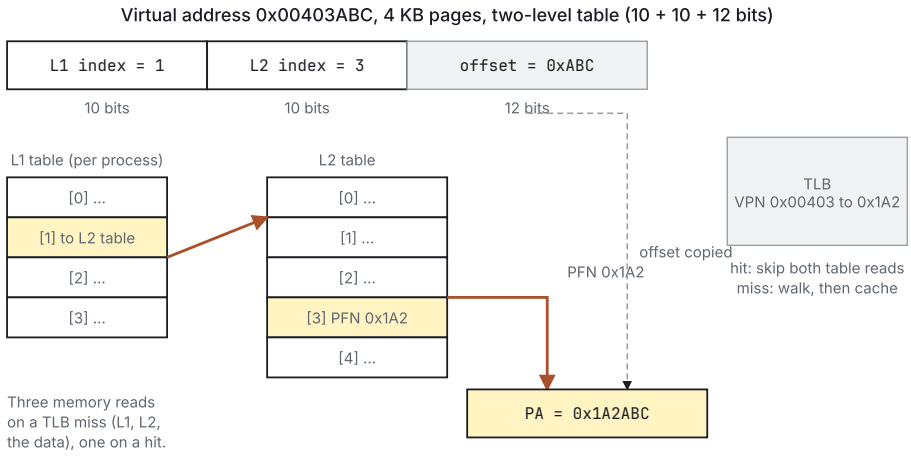


Figure 7.13 Two-level translation of 0x00403ABC. The TLB caches finished translations so that a hit skips both table reads.

WORKED EXAMPLE

Page-table memory for a process using 8 MB. Each second-level table (1024 entries × 4 bytes = 4 KB) maps 1024 pages = 4 MB.

1. 8 MB needs two second-level tables if the region is contiguous and aligned: 2 × 4 KB.
2. One first-level table: 4 KB.

Result: 12 KB of page tables, against 4 MB for the flat table.

On x86-64 with 48-bit virtual addresses, the same idea uses four levels of 9 bits each (512 eight-byte entries per table) plus the 12-bit offset: 4 × 9 + 12 = 48 bits.

The TLB and effective access time

Walking the table costs memory reads, so the MMU caches recent translations in the *translation lookaside buffer* (TLB), a small associative cache. A *hit* gives the frame at once; a *miss* triggers a table walk, after which the entry is cached. Because a process switch changes the page tables, the TLB entries are flushed or tagged with an address-space identifier, which is the hidden cost of switching processes (7.3 (p. 420)).

WORKED EXAMPLE

Effective access time (EAT). TLB lookup 1 ns, memory access 100 ns, TLB hit ratio 95 %, single-level table.

1. A hit costs the lookup and one memory access: 1 + 100 = 101 ns.
2. A miss costs the lookup, one table read and one data read: 1 + 100 + 100 = 201 ns.
3. EAT = 0.95 × 101 + 0.05 × 201 = 95.95 + 10.05.

Result: EAT = 106 ns, 6 % above the raw memory time. At a 50 % hit ratio it would be 151 ns.

Page faults

A *page fault* is the exception raised when the MMU cannot translate an address. The kernel's handler decides what kind it is. A *minor* fault finds the page already in memory or needing only a zeroed frame (first touch of `malloc`'d memory, a copy-on-write copy); it costs microseconds. A *major* fault must read the page from disk (a file mapping, or swap), costing milliseconds. An *invalid* fault is an access to an unmapped address or one that violates permissions, and the kernel delivers SIGSEGV.

LAB

`LAB 7.7 · 10 MIN` Watch first-touch page faults. The program reads `getrusage` before and after a 100 MB `malloc` and a `memset`.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/resource.h>

static long faults(void)
{
    struct rusage r;
    getrusage(RUSAGE_SELF, &r);
    return r.ru_minflt;                    /* minor faults: no disk I/O needed */
}

int main(void)
{
    size_t n = 100u << 20;                 /* 100 MB */
    long f0 = faults();
    char *p = malloc(n);                   /* reserves address space only */
    long f1 = faults();
    memset(p, 1, n);                       /* first touch of every page */
    long f2 = faults();
    printf("after malloc: +%ld faults\n", f1 - f0);
    printf("after touching 100 MB: +%ld faults (%zu pages of 4 KB)\n",
           f2 - f1, n / 4096);
    printf("last byte = %d\n", p[n - 1]);   /* keeps the optimiser honest */
    free(p);
    return 0;
}
```

```
$ gcc -O1 pf.c -o pf && ./pf
after malloc: +3 faults
after touching 100 MB: +25600 faults (25600 pages of 4 KB)
```

What you should notice: `malloc` of 100 MB costs almost nothing; the 25,600 faults arrive only when the pages are first written, one per 4 KB page. With transparent huge pages enabled the count is lower, since one fault then maps 2 MB.

Copy-on-write, mmap and thrashing

After `fork()` the child's page table points at the parent's frames, and both sets of entries are marked read-only. The first write by either process raises a fault; the kernel copies that one frame, maps the copy writable for the writer, and resumes. Hence `fork` followed by `exec` copies almost nothing. `mmap` maps a file, or anonymous zeroed memory, into the address space, and pages are loaded on demand; `malloc` uses it for large blocks. When the total of the processes' actively used pages (their *working sets*) exceeds physical memory, pages are evicted just before they are needed again and the system spends its time faulting: this is *thrashing*.

Page replacement

When no frame is free, the kernel must evict a page. *FIFO* evicts the oldest-loaded page; *LRU* (least recently used) evicts the page unused for longest; *Optimal* evicts the page whose next use is furthest in the future, which needs knowledge of the future and serves as a bound. Real kernels approximate LRU with the *clock* algorithm: frames sit on a circle, each with a reference bit set by the hardware on use; a hand sweeps, clearing set bits and evicting the first frame whose bit is already clear.

Take the reference string 7 0 1 2 0 3 0 4 2 3 0 3 2 with three frames. The table shows the resident pages after each reference (oldest first for FIFO and LRU; slot order for Optimal), and F marks a fault.

Ref	FIFO		LRU		Optimal	
7	7	F	7	F	7	F
0	7 0	F	7 0	F	7 0	F
1	7 0 1	F	7 0 1	F	7 0 1	F
2	0 1 2	F	0 1 2	F	2 0 1	F
0	0 1 2		1 2 0		2 0 1	
3	1 2 3	F	2 0 3	F	2 0 3	F
0	2 3 0	F	2 3 0		2 0 3	
4	3 0 4	F	3 0 4	F	2 4 3	F
2	0 4 2	F	0 4 2	F	2 4 3	
3	4 2 3	F	4 2 3	F	2 4 3	
0	2 3 0	F	2 3 0	F	2 0 3	F
3	2 3 0		2 0 3		2 0 3	
2	2 3 0		0 3 2		2 0 3	
Faults		**10**		**9**		**7**

Bélády's anomaly is the surprising fact that for FIFO more frames can cause more faults. On the string 1 2 3 4 1 2 5 1 2 3 4 5, FIFO has 9 faults with three frames and 10 with four. LRU and Optimal are *stack algorithms*: the pages held with n frames are always a subset of those held with n + 1, so they cannot show the anomaly.

Stack, heap and a small allocator

The stack is fast to allocate because allocation is moving one register, the stack pointer, and freeing is moving it back; memory is reused immediately and is likely to be in cache. The heap must find a block of the right size among live and freed ones, which costs a search and risks fragmentation. *Internal fragmentation* is waste inside an allocated block (rounding up to 16 bytes); *external fragmentation* is free memory scattered in pieces too small to use. Structure padding and alignment, the other memory-layout question asked with these, are taught in Pillar 2 (p. 115).

Embedded targets often have no `malloc`, and Arista and Qualcomm ask candidates to design one. The usual answer is a *free list* over a fixed arena: each block begins with a header holding its size and a free flag; `malloc` takes the first block that fits (first fit) and splits off the remainder; `free` marks the block free and merges it with free neighbours (*coalescing*) so that fragments do not accumulate.

```
#include <stddef.h>
#include <stdio.h>

#define ARENA 4096
struct hdr { size_t size; int free; struct hdr *next; };   /* 24 bytes on LP64 */

static _Alignas(16) unsigned char arena[ARENA];
static struct hdr *first;

static size_t round16(size_t n) { return (n + 15) & ~(size_t)15; }

void *a_malloc(size_t n)
{
    if (!first) {                               /* one big free block */
        first = (struct hdr *)arena;
        *first = (struct hdr){ ARENA - sizeof *first, 1, NULL };
    }
    n = round16(n);
    for (struct hdr *h = first; h; h = h->next) {
        if (!h->free || h->size < n) continue;  /* first fit */
        if (h->size >= n + sizeof *h + 16) {    /* split the remainder */
            struct hdr *r = (struct hdr *)((unsigned char *)(h + 1) + n);
            *r = (struct hdr){ h->size - n - sizeof *h, 1, h->next };
            h->size = n;
            h->next = r;
        }
        h->free = 0;
        return h + 1;
    }
    return NULL;
}

void a_free(void *p)
{
    if (!p) return;
    ((struct hdr *)p - 1)->free = 1;
    for (struct hdr *h = first; h && h->next; ) {   /* coalesce neighbours */
        if (h->free && h->next->free) {
            h->size += sizeof *h + h->next->size;
            h->next = h->next->next;
        } else h = h->next;
    }
}

int main(void)
{
    void *a = a_malloc(100), *b = a_malloc(200), *c = a_malloc(50);
    a_free(b); a_free(a);                       /* a and b merge into one hole */
    void *d = a_malloc(300);                    /* fits only in the merged hole */
    printf("a=%p d=%p same start: %s\n", a, d, a == d ? "yes" : "no");
    a_free(c); a_free(d);
    size_t total = 0; int blocks = 0;
    for (struct hdr *h = first; h; h = h->next) { total += h->size; blocks++; }
    printf("blocks=%d, free bytes=%zu\n", blocks, total);
    return 0;
}
```

```
$ gcc -Wall -Wextra alloc.c -o alloc && ./alloc
a=0x55afdad11038 d=0x55afdad11038 same start: yes
blocks=1, free bytes=4072
```

Freeing `a` and `b` creates one merged hole that the 300-byte request then fits; freeing everything coalesces the arena into a single block of 4096 − 24 bytes. A production allocator adds size classes, thread-local caches and a boundary tag at the block's end so that coalescing needs no list

walk; the kernel's *slab allocator* keeps per-type caches of equal-sized objects, which is why `sk_buff` allocation is quick (7.12 (p. 460)).

TRAP

Claim: "`malloc(100 MB)` gives me 100 MB of physical memory." *Why it is wrong:* it reserves virtual address space; frames are assigned at the first write of each page, as the lab shows, and the kernel may overcommit. *Correct statement:* `malloc` returns virtual memory, and physical frames arrive on first touch.

INTERVIEW LENS

CISCO · JUNIPER · QUALCOMM · ARISTA

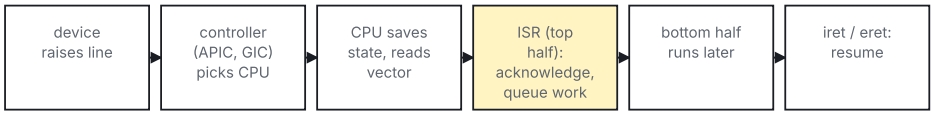
"What is virtual memory and why is it needed?" is asked almost everywhere, with "what happens on a TLB miss, and on a page fault?" as the follow-up. A strong answer names isolation and translation first, draws the VPN and offset split, and separates the TLB-miss path (hardware or kernel walks the table) from the page-fault path (the kernel loads or creates the page, or kills the process). Qualcomm and Arista add "design `malloc` for a system without an OS", which expects a header format, first-fit with splitting, and coalescing on free. Follow-ups: copy-on-write after `fork`, and why a TLB flush hurts.

CHECK YOURSELF

1. How many bits of a 32-bit virtual address are the offset with 16 KB pages? [1]
2. Which page-replacement algorithm shows Bélády's anomaly? [2]
3. Why does `fork` followed by `exec` cost little? [3]

7.8 Interrupts, system calls and DMA MUST · ■■□

A device signals that it needs attention by raising an *interrupt*. The interrupt controller (the APIC on x86, the GIC on ARM) selects a CPU; the CPU finishes the current instruction, saves enough state to resume, switches to kernel mode and jumps through a *vector table* to the handler registered for that interrupt line, the *interrupt service routine* (ISR). When the ISR ends, a special return instruction (`iret` or `eret`) restores the interrupted code. *Exceptions* (page fault, divide by zero) use the same mechanism but arise from the running instruction, and a *system call* is a deliberate software-triggered entry.



Steps 1 to 4 and 6 are fast and run with the interrupt masked; step 5 is ordinary scheduled kernel work.

Figure 7.14 The life of an interrupt. Only the shaded step, the ISR, is time-critical; the heavy work is deferred.

An ISR runs while other interrupts may be masked, so it must be short. It acknowledges the device, grabs what cannot wait (reads a status register, notes the packet's buffer) and schedules the rest as a *bottom half*. Linux offers softirqs, tasklets and workqueues for this. The ISR itself, the *top half*, runs in *interrupt context*, where there is no process to put to sleep: it may not block, take a mutex, or call `kmalloc(GFP_KERNEL)`, which can sleep; it uses spinlocks and `GFP_ATOMIC`.

	Top half (ISR)	Softirq / tasklet	Workqueue
Context	interrupt	softirq (interrupt-like)	process (a kernel thread)
May sleep?	no	no	yes
May be interrupted by another interrupt?	by higher-priority ones only	yes	yes, and pre-empted by the scheduler
Example in a NIC driver	acknowledge, `napi_schedule()`	NAPI poll, protocol processing	reset after a hardware error

Direct memory access (DMA) lets a device read and write main memory without the CPU copying each word. The driver prepares a *descriptor ring*, a circular array of descriptors each holding the address and length of a buffer; it programs the NIC with the ring's base address. The NIC fills buffers and advances its position, the driver follows behind, cleans completed descriptors and hands fresh buffers back (Figure 7.15). On a CPU with caches the driver must also keep the cache consistent with the device's writes, which the kernel's DMA-mapping API arranges.

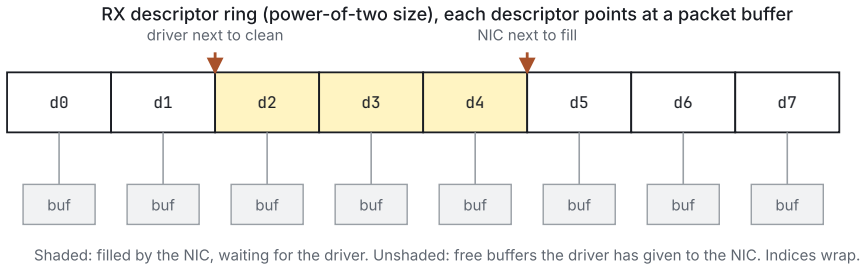


Figure 7.15 An RX descriptor ring. The NIC fills descriptors up to its pointer; the driver processes behind it and refills. Both indices wrap.

Why one interrupt per packet fails

A 10 Gb/s link carrying minimum-size frames moves 64 bytes plus 20 bytes of preamble and inter-frame gap per packet: 84 bytes = 672 bits, hence $10 \times 10^9 / 672 \approx 14.88$ million packets per second, one every 67 ns. An interrupt costs more than that, so a CPU taking one interrupt per packet would do nothing else; at the worst it would be *livelocked*, spending all its time entering and leaving ISRs. The cures are *interrupt coalescing* (the NIC raises one interrupt per batch or per timeout) and *NAPI* (7.12 (p. 460)), which switches the device to polling under load.

	Interrupts	Polling
Latency at low load	best: reacts at once	up to one polling interval
CPU use at low load	none while idle	wasted: the poller runs regardless
Behaviour at high load	overhead per event; livelock risk	efficient: events are batched
Used by	NICs at low rate, keyboards	NAPI under load, DPDK, busy-polling drivers

> LAB
>
> `LAB 7.8 · 10 MIN` Look at the machine's interrupt and receive-processing counters.

```
$ head -4 /proc/interrupts
           CPU0       CPU1       CPU2       CPU3
 26:          0          0          2          0  IO-APIC   4-edge      ttyS0
 32:          0        972          0          0  PCI-MSIX-0000:00:01.0   3-edge
virtio0-stats
$ head -1 /proc/net/softnet_stat
00039a92 00000000 00000019 ...
```

What you should notice: each row of `/proc/interrupts` is one interrupt line with a count per CPU; a multi-queue NIC shows one line per queue, each bound to a core, and the counts climb during an `iperf3` run. In `softnet_stat` the first three hexadecimal columns are packets processed, packets dropped, and `time_squeeze`, the number of times the receive poll ran out of budget with work left; here 0x39a92 packets, 0 drops, 0x19 = 25 squeezes.

TRAP

Claim: "An ISR can call a function that takes a mutex, provided the mutex is rarely contended." *Why it is wrong:* if the mutex is held, the ISR would sleep, but an interrupt handler has no task context to sleep in, and the kernel panics or deadlocks. *Correct statement:* in interrupt context use spinlocks (with interrupts disabled while the lock is held, to avoid self-deadlock) and atomic allocations.

INTERVIEW LENS

QUALCOMM · BROADCOM · CISCO · JUNIPER

"Describe what happens from the device signal to the end of the ISR" and "why can't you sleep in an ISR?" are reported firmware and driver questions, and "polling or interrupts?" was reported for a Cisco internship. A strong answer lists the steps in the figure, states the top-half rule, mentions deferral and DMA, and gives the polling trade-off with the packet-rate arithmetic above. Follow-ups: what NAPI is, and what `volatile` and memory barriers do for device registers (7.13 (p. 463)).

CHECK YOURSELF

1. Why must an ISR be short? [4]
2. What does DMA save the CPU from? [5]

7.9 Inter-process communication and signals SHOULD · ■■□

Processes cannot touch each other's memory, so they communicate through the kernel or through memory the kernel maps into both. A *pipe* is a one-way byte channel between related processes; a *FIFO* (named pipe) is a pipe with a name in the file system, so unrelated processes may open it; a *message queue* preserves message boundaries; *shared memory* maps the same frames into two address spaces; and a *socket* (7.10 (p. 445)) also works across machines.

Mechanism	Direction	Unrelated processes?	Kernel copies per message	Needs synchronisation?
Pipe	one way	no (shared ancestor)	2	no, the kernel blocks
FIFO	one way	yes	2	no
Message queue	any	yes	2	no
Shared memory	any	yes	0	yes, e.g. a process-shared mutex

Unix-domain socket	both	yes (same host)	2	no
Signal	notification only	yes	none, carries a number	n/a

Shared memory is fastest because after setup no kernel call is on the data path; its price is that the processes must synchronise themselves, as threads do.

The shell pipeline `ls | wc -l` shows how the pieces combine. The parent creates a pipe, forks twice, and each child uses `dup2` to make the pipe's end its standard output or input before `exec`.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void)                          /* equivalent of: ls | wc -l */
{
    int fd[2];
    if (pipe(fd) < 0) { perror("pipe"); exit(1); }

    if (fork() == 0) {                  /* writer: ls */
        dup2(fd[1], STDOUT_FILENO);     /* stdout now feeds the pipe */
        close(fd[0]); close(fd[1]);     /* close the originals */
        execlp("ls", "ls", (char *)NULL);
        perror("execlp"); _exit(127);
    }
    if (fork() == 0) {                  /* reader: wc -l */
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]); close(fd[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("execlp"); _exit(127);
    }
    close(fd[0]); close(fd[1]);  /* parent must close, or wc never sees EOF */
    while (wait(NULL) > 0)
        ;
    return 0;
}
```

In each child, `dup2(fd[1], STDOUT_FILENO)` makes descriptor 1 refer to the pipe's write end, and the original two descriptors are closed. The parent must also close both ends: a pipe delivers end-of-file to the reader only when every write end is closed, so a forgotten copy in the parent leaves `wc` waiting for ever.

A *signal* is an asynchronous notification delivered to a process: SIGINT (Ctrl-C), SIGTERM (polite termination), SIGKILL (cannot be caught), SIGSEGV (invalid memory access), SIGCHLD (a child changed state; the standard place to reap zombies with `waitpid(-1, NULL, WNOHANG)`), and SIGPIPE (a write to a pipe or socket whose reader has gone). Because a handler can interrupt the program at any point, it may call only *async-signal-safe* functions: `write` is on the list, `printf` and `malloc` are not, since they may hold internal locks that the interrupted code already owns. The common practice is to set a flag in the handler and act on it in the main loop. A network server normally ignores SIGPIPE (`signal(SIGPIPE, SIG_IGN)`, or `MSG_NOSIGNAL` on `send`) and handles the `EPIPE` error instead.

> CHECK YOURSELF
>
> 1. Which IPC mechanism avoids kernel copies of the data? [6]
> 2. Why can a signal handler not safely call `printf`? [7]

7.10 Sockets from first principles MUST · ■■□

A *socket* is a file descriptor that names one end of a communication channel. The *Berkeley sockets API* is the set of system calls, standard on Linux, macOS and Windows (with small differences), by which a program creates such endpoints, attaches them to addresses, and moves bytes through them. A stream socket (`SOCK_STREAM`, TCP) delivers a reliable, ordered byte stream between two connected endpoints; a datagram socket (`SOCK_DGRAM`, UDP) sends independent messages with no connection and no delivery guarantee. The protocols themselves are taught in Pillar 3 (p. 162); this section teaches the calls that drive them.

A TCP server prepares a *listening socket* and then serves connections one by one: `socket` creates the endpoint, `bind` gives it a local address, `listen` marks it passive, and `accept` waits for a client and returns a *new* descriptor for that client while the listening descriptor remains available for the next. A client calls `socket` and then `connect`. After that both sides use `send` / `recv` (or `write` / `read`) and finally `close`.

Call	What it does	Notes
`socket(domain, type, proto)`	creates an endpoint, returns a descriptor	`AF_INET`/`AF_INET6`; `SOCK_STREAM`/`SOCK_DGRAM`
`bind(fd, addr, len)`	assigns a local address and port	port in network byte order; `EADDRINUSE` if taken
`listen(fd, backlog)`	makes the socket passive, sizes the accept queue	capped by `net.core.somaxconn`
`accept(fd, addr, len)`	takes one completed connection from the queue	returns a new descriptor; blocks if none
`connect(fd, addr, len)`	starts the TCP handshake	returns when established or failed
`send` / `recv`	move bytes	may transfer fewer bytes than asked
`sendto` / `recvfrom`	UDP: with a peer address per datagram	`recvfrom` reports who sent it
`close(fd)`	releases the descriptor, sends FIN	`shutdown(fd, SHUT_WR)` closes one direction only
`setsockopt`	changes behaviour	`SO_REUSEADDR`, `TCP_NODELAY`, `SO_KEEPALIVE`

1. 14, since $2^{14} = 16384$; the remaining 18 bits are the page number.↩
2. FIFO. Stack algorithms such as LRU and Optimal cannot show it.↩
3. Copy-on-write: the pages are shared and not copied, and `exec` discards the child's address space almost at once.↩
4. Interrupts of equal or lower priority are held off while it runs, so a long ISR delays other devices and the scheduler's timer.↩
5. Copying every word between the device and memory; the CPU only sets up descriptors and handles the completion interrupt.↩
6. Shared memory, after the one-off setup; it needs explicit synchronisation.↩
7. `printf` takes an internal lock; if the signal interrupted code holding it, the handler would deadlock.↩

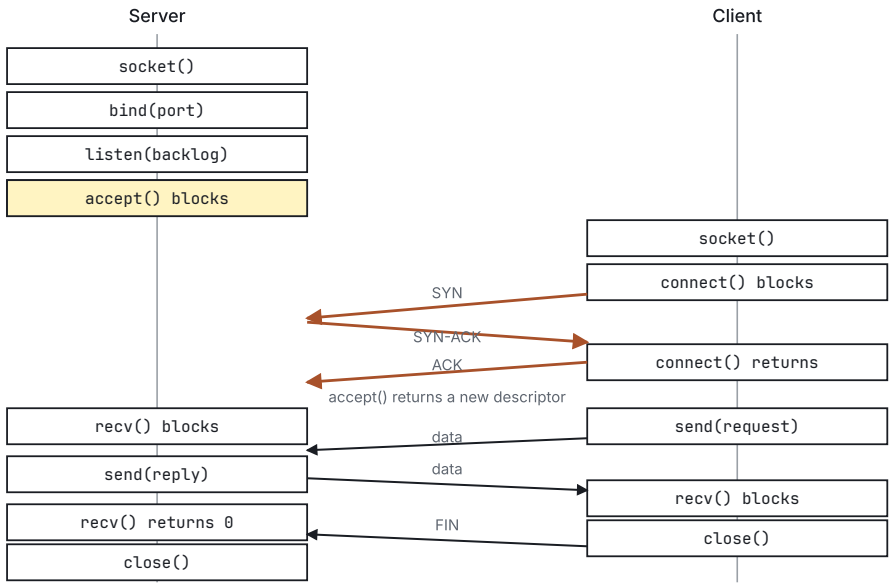


Figure 7.16 The call sequence of a TCP exchange. The kernel completes the three-way handshake without the server's process; accept() merely collects the finished connection.

The mapping to the wire is worth fixing in mind. `connect()` sends a SYN and returns when the SYN-ACK arrives and the final ACK has been sent. On the server the kernel answers SYNs by itself: it keeps half-open connections in a *SYN queue*, moves each to the *accept queue* when its handshake completes, and `accept()` simply removes the oldest entry (Figure 7.17). The `backlog` argument sizes the accept queue, limited by the system maximum; when it is full the kernel drops or ignores further handshake completions, so clients see delays or retransmitted SYNs.

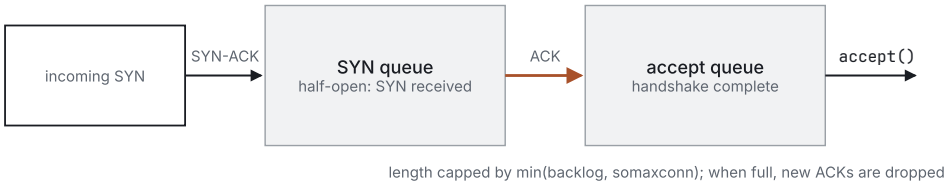


Figure 7.17 The two kernel queues behind a listening socket.

Addresses and byte order

An IPv4 endpoint is a `struct sockaddr_in` holding the family, a 16-bit port and a 32-bit address, both in *network byte order* (big-endian). A PC stores integers little-endian, so a port must be converted with `htons` ("host to network short") and an address with `htonl`; the reverse conversions are `ntohs` and `ntohl`. For port 8080 = 0x1F90, a little-endian host holds the bytes `90 1F` in memory, and the wire needs `1F 90`; `htons(8080)` performs the swap, and on a big-endian host it does nothing. The byte-order rules themselves are in Pillar 2 (p. 117). `INADDR_ANY` (0.0.0.0) binds to every local address, and `inet_pton` / `inet_ntop` convert between text and binary addresses.

A TCP echo server and client

The server below accepts one client at a time and sends back every byte it receives. It shows the three habits that separate a working server from a fragile one: it sets `SO_REUSEADDR` before

`bind`, it checks every return value, and it loops on `send` because a call may transfer fewer bytes than requested.

```
#include <arpa/inet.h>
#include <errno.h>
#include <netinet/in.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

static void die(const char *what) { perror(what); exit(1); }

int main(int argc, char **argv)
{
    int port = argc > 1 ? atoi(argv[1]) : 8080;
    signal(SIGPIPE, SIG_IGN);  /* write to a closed peer gives EPIPE */

    int ls = socket(AF_INET, SOCK_STREAM, 0);
    if (ls < 0) die("socket");
    int on = 1;
    setsockopt(ls, SOL_SOCKET, SO_REUSEADDR, &on, sizeof on);

    struct sockaddr_in a = { .sin_family = AF_INET,
                             .sin_port = htons(port),  /* host to network order */
                             .sin_addr.s_addr = htonl(INADDR_ANY) };
    if (bind(ls, (struct sockaddr *)&a, sizeof a) < 0) die("bind");
    if (listen(ls, 128) < 0) die("listen");
    printf("listening on port %d\n", port);

    for (;;) {
        struct sockaddr_in peer;
        socklen_t len = sizeof peer;
        int c = accept(ls, (struct sockaddr *)&peer, &len);
        if (c < 0) {
            if (errno == EINTR) continue;
            die("accept");
        }
        char ip[INET_ADDRSTRLEN];
        inet_ntop(AF_INET, &peer.sin_addr, ip, sizeof ip);
        printf("client %s:%d\n", ip, ntohs(peer.sin_port));

        char buf[4096];
        ssize_t n;
        while ((n = recv(c, buf, sizeof buf, 0)) > 0) {
            for (ssize_t off = 0; off < n; ) {   /* send may be partial */
                ssize_t w = send(c, buf + off, n - off, 0);
                if (w < 0) { if (errno == EINTR) continue; goto done; }
                off += w;
            }
        }
    done:
        close(c);  /* sends FIN; the next client is accepted */
    }
}
```

`SO_REUSEADDR` lets `bind` succeed on a port whose previous connections are still in `TIME_WAIT`; without it a restarted server fails with `EADDRINUSE` for up to a minute. It does not allow two live servers to share a port, which is a different option, `SO_REUSEPORT` (7.11 (p. 453)). The client is shorter:

```c
#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    const char *ip = argc > 1 ? argv[1] : "127.0.0.1";
    int port = argc > 2 ? atoi(argv[2]) : 8080;

    int s = socket(AF_INET, SOCK_STREAM, 0);
    if (s < 0) { perror("socket"); return 1; }
    struct sockaddr_in a = { .sin_family = AF_INET, .sin_port = htons(port) };
    if (inet_pton(AF_INET, ip, &a.sin_addr) != 1) {
        fprintf(stderr, "bad address\n");
        return 1;
    }
    if (connect(s, (struct sockaddr *)&a, sizeof a) < 0) {
        perror("connect");
        return 1;
    }

    const char *msg = "hello, socket\n";
    send(s, msg, strlen(msg), 0);
    char buf[128];
    ssize_t n = recv(s, buf, sizeof buf - 1, 0);   /* may be partial */
    if (n < 0) { perror("recv"); return 1; }
    buf[n] = '\0';
    printf("echo: %s", buf);
    close(s);
    return 0;
}
```

LAB

`LAB 7.10 · 15 MIN` Run both, then look at the sockets afterwards.

```
$ gcc -Wall -Wextra echosrv.c -o echosrv && gcc -Wall -Wextra echocli.c -o echocli
$ ./echosrv 8085 &
listening on port 8085
$ ./echocli 127.0.0.1 8085
echo: hello, socket
client 127.0.0.1:43614
$ awk 'NR>1 { print $2, $3, $4 }' /proc/net/tcp | grep 1F95
00000000:1F95 00000000:0000 0A
0100007F:AA5E 0100007F:1F95 06
```

What you should notice: `ss -tan` shows the same table in words, but `/proc/net/tcp` is always present. Port 8085 appears as `1F95`, the network-order hexadecimal that `htons` produced. State `0A` is LISTEN. State `06` is TIME_WAIT, and it sits on the *client's* side (local port `AA5E` = 43614): the client called `close()` first, so it is the one that waits, 2 × MSL (60 s on Linux), in case its final ACK was lost. This is deliberate behaviour, not a leak.

The stream has no message boundaries

TCP delivers a byte stream, so one `send()` is not one `recv()`. Three sends of 10, 20 and 5 bytes may arrive as one read of 35 bytes, two reads of 25 and 10, or any other split; likewise a single large `send` may need several `recv` calls. The program below sends three messages, lets them all arrive, and then reads.

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

int main(void)
{
    int ls = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in a = { .sin_family = AF_INET,
                             .sin_addr.s_addr = htonl(INADDR_LOOPBACK) };
    socklen_t len = sizeof a;
    bind(ls, (struct sockaddr *)&a, sizeof a);  /* port 0: kernel picks one */
    listen(ls, 1);
    getsockname(ls, (struct sockaddr *)&a, &len);

    int c = socket(AF_INET, SOCK_STREAM, 0);
    connect(c, (struct sockaddr *)&a, sizeof a);   /* completes at once */
    int s = accept(ls, NULL, NULL);

    char msg[20];
    memset(msg, 'x', sizeof msg);
    send(c, msg, 10, 0);  /* three separate sends */
    send(c, msg, 20, 0);
    send(c, msg, 5, 0);
    usleep(200000);  /* let all three arrive first */

    char buf[100];
    ssize_t n;
    n = recv(s, buf, 25, 0);  printf("first recv(25):  %zd bytes\n", n);
    n = recv(s, buf, 100, 0); printf("second recv(100): %zd bytes\n", n);
    close(c); close(s); close(ls);
    return 0;
}
```

```
first recv(25):  25 bytes
second recv(100): 10 bytes
```

The first read returned 25 bytes spanning two messages, and the second the remaining 10 bytes, spanning the third and the tail of the second. The receiver must therefore *frame* the messages: delimit them with a terminator (a newline, as HTTP/1 headers do) or prefix each with its length. A length prefix with two helpers, `readn` and `writen`, which loop until the whole count is transferred, is the standard pattern:

```c
#include <arpa/inet.h>
#include <errno.h>
#include <stdint.h>
#include <sys/socket.h>
#include <sys/types.h>

/* Read exactly n bytes; return n, 0 on clean EOF before any byte, -1 on error. */
ssize_t readn(int fd, void *buf, size_t n)
{
    size_t got = 0;
    while (got < n) {
        ssize_t r = recv(fd, (char *)buf + got, n - got, 0);
        if (r == 0) return got == 0 ? 0 : -1;   /* EOF mid-message: error */
        if (r < 0) { if (errno == EINTR) continue; return -1; }
        got += r;
    }
    return got;
}

/* Write exactly n bytes; return n or -1. */
ssize_t writen(int fd, const void *buf, size_t n)
{
    size_t sent = 0;
    while (sent < n) {
        ssize_t w = send(fd, (const char *)buf + sent, n - sent, MSG_NOSIGNAL);
        if (w < 0) { if (errno == EINTR) continue; return -1; }
        sent += w;
    }
    return n;
}

/* Frame = 4-byte big-endian length, then payload. */
int send_msg(int fd, const void *p, uint32_t len)
{
    uint32_t be = htonl(len);
    if (writen(fd, &be, 4) < 0) return -1;
    return writen(fd, p, len) < 0 ? -1 : 0;
}

ssize_t recv_msg(int fd, void *p, uint32_t cap)    /* returns payload length */
{
    uint32_t be;
    ssize_t r = readn(fd, &be, 4);
    if (r <= 0) return r;
    uint32_t len = ntohl(be);
    if (len > cap) return -1;  /* never trust a peer's length */
    return readn(fd, p, len) < 0 ? -1 : (ssize_t)len;
}
```

Three details matter. `readn` retries on `EINTR` (a signal interrupted the call) and treats end-of-file inside a message as an error. `writen` uses `MSG_NOSIGNAL` so that a closed peer produces `EPIPE` and not a fatal SIGPIPE. And `recv_msg` refuses a length larger than its buffer, because a peer's length field is untrusted input.

> BIG IDEA
>
> A TCP socket promises the bytes, in order, and nothing about how they are grouped; every protocol on top of it must supply its own message boundaries.

UDP and address-family-neutral code

A UDP socket needs no `listen`, `accept` or `connect`: one `bind`, then `recvfrom` returns each datagram together with the sender's address, and `sendto` replies to it. Each call moves exactly

one datagram, so there is no framing problem, but datagrams may be lost, duplicated or reordered, and one that exceeds the buffer given to `recvfrom` is truncated.

```
#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    int port = argc > 1 ? atoi(argv[1]) : 9090;
    int s = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in a = { .sin_family = AF_INET, .sin_port = htons(port),
                             .sin_addr.s_addr = htonl(INADDR_ANY) };
    if (s < 0 || bind(s, (struct sockaddr *)&a, sizeof a) < 0) {
        perror("udp");
        return 1;
    }
    for (;;) {
        char buf[1500];
        struct sockaddr_in from;
        socklen_t len = sizeof from;
        ssize_t n = recvfrom(s, buf, sizeof buf, 0,
                             (struct sockaddr *)&from, &len);
        if (n < 0) { perror("recvfrom"); continue; }
        sendto(s, buf, n, 0, (struct sockaddr *)&from, len);   /* reply */
    }
}
```

To test it, `printf dgram | nc -u -w1 127.0.0.1 9090` should print `dgram`. To keep a client independent of IPv4 or IPv6, resolve names with `getaddrinfo`, which returns a list of candidate addresses, and try each:

```c
#include <netdb.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

/* Connect to host:port over IPv4 or IPv6, whichever works first. */
int dial(const char *host, const char *port)
{
    struct addrinfo hints, *res, *p;
    memset(&hints, 0, sizeof hints);
    hints.ai_family = AF_UNSPEC;                /* IPv4 or IPv6 */
    hints.ai_socktype = SOCK_STREAM;
    if (getaddrinfo(host, port, &hints, &res) != 0)
        return -1;
    int fd = -1;
    for (p = res; p; p = p->ai_next) {          /* try each address in turn */
        fd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
        if (fd < 0) continue;
        if (connect(fd, p->ai_addr, p->ai_addrlen) == 0) break;
        close(fd); fd = -1;
    }
    freeaddrinfo(res);
    return fd;
}

int main(void)
{
    int fd = dial("localhost", "8083");
    printf("dial returned %s\n",
           fd >= 0 ? "a connected socket" : "-1 (connection refused)");
    return 0;
}
```

With nothing listening on port 8083 the program prints `dial returned -1 (connection refused)`: the kernel answered the SYN with an RST, which `connect` reports as `ECONNREFUSED`. If the server's accept queue is full instead, the SYN is silently dropped and the client keeps retransmitting, so the symptom is a *timeout*, not a refusal.

Errors and options worth knowing

errno	Meaning	Usual response
EINTR	a signal interrupted the call	retry
EAGAIN / EWOULDBLOCK	non-blocking call has nothing to do	wait for readiness
ECONNREFUSED	no listener; RST received	report, maybe retry later
ECONNRESET	peer sent RST	close the socket
EPIPE	write after the peer closed	close; ignore SIGPIPE
EADDRINUSE	port in use or in TIME_WAIT without SO_REUSEADDR	set the option, or pick another port

Option	Effect
SO_REUSEADDR	allow bind while old connections linger in TIME_WAIT
SO_REUSEPORT	several sockets may bind one port; the kernel spreads connections among them

`TCP_NODELAY`	send small writes at once instead of waiting to coalesce (disables Nagle)
`SO_KEEPALIVE`	probe an idle connection to detect a dead peer
`SO_RCVBUF` / `SO_SNDBUF`	size the kernel buffers, which bounds the window the connection can advertise

TRAP

Claim: "If I `send(fd, buf, 1000, 0)` and it returns 1000, the peer has received 1000 bytes." *Why it is wrong:* a successful `send` means only that the kernel accepted the bytes into its send buffer; they may still be in flight, lost and awaiting retransmission, or unread by the application. Likewise one `recv` is not one message. *Correct statement:* `send` returning n means n bytes were queued; delivery is confirmed only by an application-level reply.

INTERVIEW LENS

CISCO · JUNIPER · ARISTA · GOOGLE

"Write a TCP client and server in C" and "explain `listen()`'s backlog" are reported at Cisco, Juniper and Arista, with Google asking the follow-ups on framing and errors. A strong answer writes the call sequence from memory, applies `htons`, checks return values, explains `SO_REUSEADDR` and TIME_WAIT, and volunteers that `recv` returns a partial count. Follow-ups: what `connect` returns when no process listens (`ECONNREFUSED` after an RST) and when the accept queue is full (a timeout), and how a server serves many clients at once (7.11 (p. 453)).

CHECK YOURSELF

1. What does `accept()` return, and what happens to the listening socket? [1]
2. Which side of a connection enters TIME_WAIT? [2]
3. Why must a `recv`-based protocol parser loop? [3]

7.11 Server concurrency: threads, select, poll, epoll MUST · ■■■

The echo server above serves one client at a time; while it waits for client A's data it cannot even accept client B. A real server must handle many connections at once, and there are two families of design. The first gives each connection its own thread or process; the second lets one thread wait on many sockets and act on whichever is ready, termed *I/O multiplexing*.

Thread per connection

The simplest change is to hand each accepted descriptor to a new thread:

```c
#include <arpa/inet.h>
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <unistd.h>

static void *serve(void *arg)
{
    int c = (int)(long)arg;
    char buf[4096];
    ssize_t n;
    while ((n = recv(c, buf, sizeof buf, 0)) > 0)
        send(c, buf, n, MSG_NOSIGNAL);  /* short writes ignored for brevity */
    close(c);
    return NULL;
}

int main(void)
{
    int ls = socket(AF_INET, SOCK_STREAM, 0), on = 1;
    setsockopt(ls, SOL_SOCKET, SO_REUSEADDR, &on, sizeof on);
    struct sockaddr_in a = { .sin_family = AF_INET, .sin_port = htons(8084),
                             .sin_addr.s_addr = htonl(INADDR_ANY) };
    if (bind(ls, (struct sockaddr *)&a, sizeof a) < 0 ||
        listen(ls, 128) < 0) {
        perror("bind/listen");
        return 1;
    }
    for (;;) {
        int c = accept(ls, NULL, NULL);
        if (c < 0) continue;
        pthread_t t;                                /* one thread per connection */
        pthread_attr_t at; pthread_attr_init(&at);
        pthread_attr_setdetachstate(&at, PTHREAD_CREATE_DETACHED);
        pthread_create(&t, &at, serve, (void *)(long)c);
    }
}
```

The program is correct and the logic stays linear, since each thread blocks in `recv` as the single-client server did. Its cost is memory and scheduling: every thread has a stack (8 MB of virtual space by default, though only the touched pages are resident) and a kernel task, and with thousands of mostly idle connections the scheduler and the caches spend their time on switching. This limit gave rise to the *C10K problem*, serving ten thousand concurrent connections on one machine, and its answer, the event loop.

select, poll and epoll

`select` takes three bit sets of descriptors (read, write, exceptional), blocks until one is ready, and returns with the sets modified to show which; the caller rebuilds the sets on every call. The sets have a fixed size, `FD_SETSIZE`, normally 1024, so a descriptor numbered 1024 or more cannot be watched at all. `poll` takes an array of `struct pollfd`, which removes the size limit and the rebuilding of bit sets. Both are O(n) per call: the kernel scans every watched descriptor, and so does the program to find the ready ones.

`epoll` stores the interest list *inside the kernel*. `epoll_ctl` adds, modifies or removes a descriptor once; `epoll_wait` returns only the descriptors that are ready. The kernel keeps the interest list as a red-black tree and attaches a callback to each socket's wait queue; when data arrives, the callback moves that descriptor onto a *ready list* (Figure 7.18). The cost of a wait is therefore proportional to the number of ready descriptors, not the number watched.

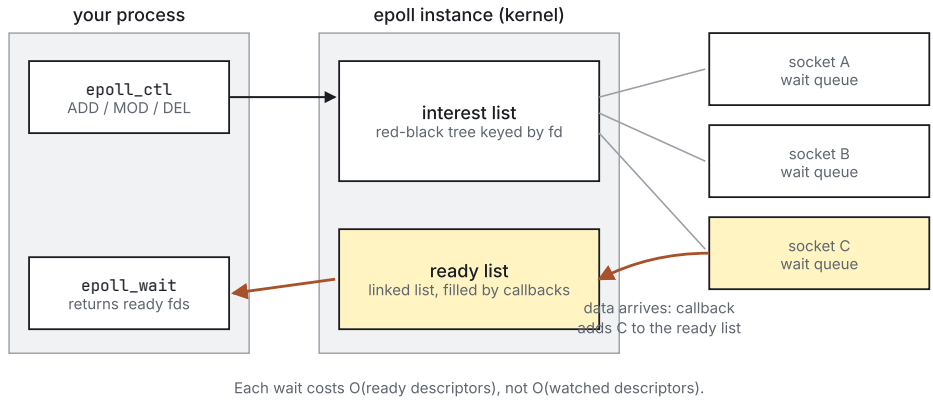


Figure 7.18 Inside epoll. The interest list is built once by epoll_ctl; the socket callbacks fill the ready list; epoll_wait only drains it.

	`select`	`poll`	`epoll`
Descriptor limit	`FD_SETSIZE` (1024)	none	none (resource limit)
Cost per wait	O(n) scan, O(n) copy in	O(n) scan, O(n) copy in	O(ready)
Interest set kept by	caller, rebuilt each call	caller, array each call	kernel, changed by `epoll_ctl`
Edge-triggered mode	no	no	yes
Portability	POSIX everywhere	POSIX	Linux only (BSD: `kqueue`)

Level-triggered and edge-triggered

By default `epoll` is *level-triggered*: `epoll_wait` reports a descriptor on every call for as long as it is ready, that is, while unread data remains. In *edge-triggered* mode (`EPOLLET`) it reports a descriptor only when its state *changes* from not ready to ready: once per arrival of new data. The two behaviours are drawn in Figure 7.19.

Level or edge — level-triggered is a doorbell that rings while the visitor stands there; edge-triggered rings once on arrival, and you must let everyone in.

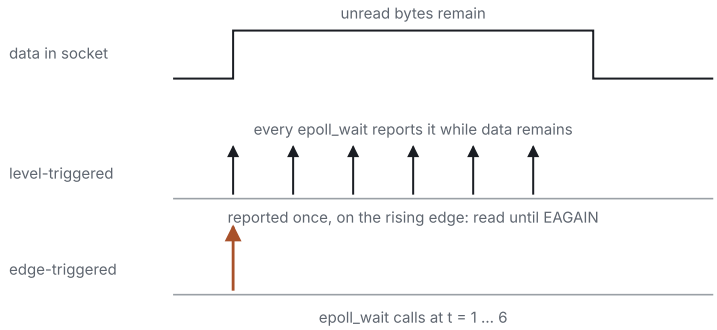


Figure 7.19 Level-triggered reports every wait while data remains. Edge-triggered reports once, at the rising edge.

Edge-triggered mode demands two disciplines. The sockets must be *non-blocking* (`O_NONBLOCK`), and on each notification the program must read until `read` returns `EAGAIN`, because an event is not repeated if data is left behind. Using ET with a blocking socket hangs the server at the last read, and not draining loses the rest of the data until the peer sends more. In return ET needs fewer `epoll_wait` calls.

The epoll server

The listing is a complete echo server serving every client from one thread. The listening socket is level-triggered; each client is registered edge-triggered.

```
#define _GNU_SOURCE
#include <arpa/inet.h>
#include <errno.h>
#include <fcntl.h>
#include <netinet/in.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/epoll.h>
#include <sys/socket.h>
#include <unistd.h>

#define MAX_EVENTS 256

static void die(const char *what) { perror(what); exit(1); }

static void set_nonblock(int fd)
{
    int fl = fcntl(fd, F_GETFL, 0);
    if (fl < 0 || fcntl(fd, F_SETFL, fl | O_NONBLOCK) < 0) die("fcntl");
}

static void add(int ep, int fd, uint32_t events)
{
    struct epoll_event ev = { .events = events, .data.fd = fd };
    if (epoll_ctl(ep, EPOLL_CTL_ADD, fd, &ev) < 0) die("epoll_ctl add");
}

int main(int argc, char **argv)
{
    int port = argc > 1 ? atoi(argv[1]) : 8080;
    signal(SIGPIPE, SIG_IGN);

    int ls = socket(AF_INET, SOCK_STREAM, 0);
    if (ls < 0) die("socket");
    int on = 1;
    setsockopt(ls, SOL_SOCKET, SO_REUSEADDR, &on, sizeof on);
    struct sockaddr_in a = { .sin_family = AF_INET, .sin_port = htons(port),
                             .sin_addr.s_addr = htonl(INADDR_ANY) };
    if (bind(ls, (struct sockaddr *)&a, sizeof a) < 0) die("bind");
    if (listen(ls, SOMAXCONN) < 0) die("listen");
    set_nonblock(ls);

    int ep = epoll_create1(0);
    if (ep < 0) die("epoll_create1");
    add(ep, ls, EPOLLIN);                          /* listener: level-triggered */
    printf("epoll server on port %d\n", port);
```

The event loop follows.

```
    struct epoll_event evs[MAX_EVENTS];
    for (;;) {
        int n = epoll_wait(ep, evs, MAX_EVENTS, -1);
        if (n < 0) { if (errno == EINTR) continue; die("epoll_wait"); }
        for (int i = 0; i < n; i++) {
            int fd = evs[i].data.fd;

            if (fd == ls) {                      /* new connections */
                for (;;) {
                    int c = accept(ls, NULL, NULL);
                    if (c < 0) {
                        if (errno == EAGAIN || errno == EWOULDBLOCK) break;
                        if (errno == EINTR) continue;
                        perror("accept"); break;
                    }
                    set_nonblock(c);
                    add(ep, c, EPOLLIN | EPOLLRDHUP | EPOLLET);   /* ET */
                }
                continue;
            }

            if (evs[i].events & (EPOLLERR | EPOLLHUP)) { close(fd); continue; }

            int closed = 0;
            char buf[4096];
            for (;;) {                           /* ET: drain until EAGAIN */
                ssize_t r = read(fd, buf, sizeof buf);
                if (r > 0) {
                    ssize_t off = 0;
                    while (off < r) {
                        ssize_t w = write(fd, buf + off, r - off);
                        if (w > 0) { off += w; continue; }
                        if (w < 0 && errno == EINTR) continue;
                        /* EAGAIN: the send buffer is full; a real server would
                           queue the rest and register EPOLLOUT. We drop it. */
                        break;
                    }
                } else if (r == 0) { closed = 1; break; }  /* peer closed */
                else if (errno == EAGAIN || errno == EWOULDBLOCK) break;
                else if (errno == EINTR) continue;
                else { closed = 1; break; }
            }
            if (closed || (evs[i].events & EPOLLRDHUP))
                close(fd);  /* close() also removes it from epoll */
        }
    }
}
```

Read it in four steps. The setup is `socket`, `bind`, `listen` and `set_nonblock`. `epoll_create1` makes the instance and `add` registers the listener. The loop calls `epoll_wait`; if the event is on the listener, a loop accepts every pending connection until `EAGAIN`, switching each new socket to non-blocking and registering it with `EPOLLIN | EPOLLRDHUP | EPOLLET`. Otherwise the event is on a client, and the program reads until `EAGAIN`, writing each chunk back, and closes the socket on end-of-file or error; `close` also removes it from the epoll set, provided the descriptor has not been duplicated. One simplification remains: if `write` returns `EAGAIN` because the peer is not reading, the remaining bytes are dropped. A production server queues them in a per-connection buffer and registers `EPOLLOUT` until it drains, which is the back-pressure mechanism.

> LAB
>
> `LAB 7.11 · 20 MIN` Hold thousands of connections open. The load generator `loadgen.c` below opens N connections to the server and sends one echo on each.

```c
#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/socket.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc < 3) {
        fprintf(stderr, "usage: loadgen port connections\n");
        return 1;
    }
    int port = atoi(argv[1]), conns = atoi(argv[2]);
    int *fd = malloc(conns * sizeof *fd);
    struct sockaddr_in a = { .sin_family = AF_INET, .sin_port = htons(port) };
    inet_pton(AF_INET, "127.0.0.1", &a.sin_addr);
    int ok = 0;
    for (int i = 0; i < conns; i++) {
        fd[i] = socket(AF_INET, SOCK_STREAM, 0);
        if (connect(fd[i], (struct sockaddr *)&a, sizeof a) < 0) {
            perror("connect");
            break;
        }
        ok++;
    }
    for (int i = 0; i < ok; i++) {  /* one round trip on every connection */
        char b[16];
        send(fd[i], "ping", 4, 0);
        if (recv(fd[i], b, sizeof b, 0) != 4) {
            fprintf(stderr, "bad echo on %d\n", i);
            break;
        }
    }
    printf("%d of %d connections echoed\n", ok, conns);
    return 0;
}
```

```
$ gcc -O2 epollsrv.c -o epollsrv && gcc loadgen.c -o loadgen
$ ulimit -n 20000
$ ./epollsrv 8082 &
$ ./loadgen 8082 3000
3000 of 3000 connections echoed
```

What you should notice: one thread serves 3,000 simultaneous connections. Raise the count until `connect` fails with `EMFILE` or `ECONNREFUSED`; the first is the per-process descriptor limit (`ulimit -n`), the second a full accept queue (`net.core.somaxconn`). Repeat with `tpc` and compare the memory in `ps -o rss,nlwp`, then fill the table below on your own machine.

Connections	Thread per connection	`select`	`epoll`
10			
100			
1,000			
10,000		n/a (over 1024 descriptors)	

Fill in resident memory and the time to echo all connections; the shape of the curve, not the values, is the lesson.

Further models and pitfalls

An event loop uses one core. To use several, run one loop per core: each worker has its own epoll instance and its own listening socket bound to the same port with `SO_REUSEPORT`, and the kernel hashes incoming connections among them. Sharing a *single* listening socket among several waiters causes the *thundering herd*, where every sleeper wakes for one new connection and all but one go back to sleep; `EPOLLEXCLUSIVE` or the sharded listeners avoid it. Libraries such as libevent and the structure of nginx implement this *reactor* pattern. `io_uring` goes one step further: the application submits operations on a shared ring and collects *completions*, so a burst of I/O needs few system calls, and the model is closer to asynchronous I/O than to readiness notification.

> **TRAP**
>
> *Claim:* "With `EPOLLET` I can read once per event." *Why it is wrong:* edge-triggered mode does not repeat the notification for data left in the buffer. *Correct statement:* read until `EAGAIN`, on a non-blocking socket. Further, `select` cannot watch a descriptor numbered 1024 or above.

> **INTERVIEW LENS**
>
> CISCO · JUNIPER · ARISTA · GOOGLE · NOKIA
>
> "`select` versus `poll` versus `epoll`" is among the most reported systems questions at these employers, and Google SRE and networking interviews raise the C10K problem directly. A strong answer gives the cost model (O(n) per call against O(ready)), the descriptor limit, the interest list kept in the kernel, then level versus edge and the non-blocking requirement. Follow-ups: how to scale a server to several cores (`SO_REUSEPORT`, one loop per core), what the thundering herd is, and why a slow client needs an output buffer and `EPOLLOUT`.

> **CHECK YOURSELF**
>
> 1. Why does `epoll_wait` scale better than `poll`? [4]
> 2. What happens if an edge-triggered server stops reading before `EAGAIN`? [5]

1. A new descriptor for the connected client; the listening socket stays open and queues further connections.↩
2. The side that closes first (the active closer), for 2 × MSL.↩
3. A byte stream may split or merge messages at any point, so a single `recv` can return part of a message or parts of several.↩
4. The interest list lives in the kernel and only ready descriptors are returned, so the cost depends on the number of ready descriptors, not the number watched.↩

7.12 The Linux receive path: NIC to recv() MUST · ■■■

Networking employers ask this section's question in many forms: "how does a packet get from the wire to the application?" The answer joins everything before it: DMA, interrupts, softirqs, the memory allocator, the protocol layers and the socket. We follow one TCP segment through a Linux host and then name where the common acceleration techniques attach.

> BIG IDEA
>
> The Linux receive path is a pipeline in which the hardware fills memory by DMA, a single interrupt starts a batched polling loop, and each packet is carried through the stack as one metadata object, the `sk_buff`, so that no layer copies the payload until the application reads it.

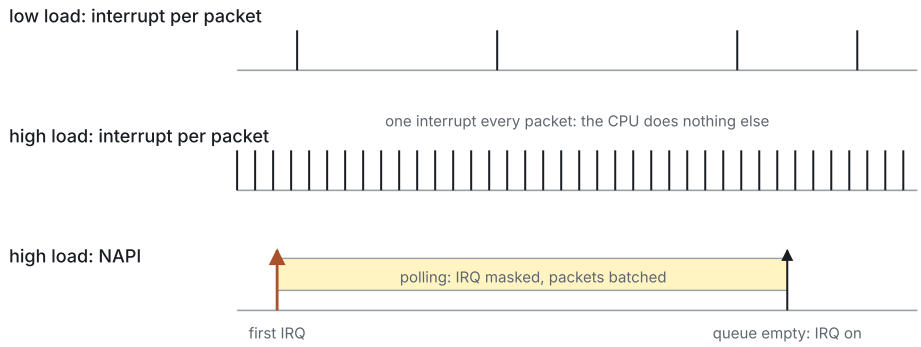


Figure 7.20 Interrupt per packet and NAPI. Under high load NAPI replaces thousands of interrupts with one interrupt and one polling window.

The nine steps

1. **DMA.** The NIC receives a frame, checks its CRC, and writes it by DMA into a buffer that a descriptor in the RX ring (7.8 (p. 441)) points at, then marks the descriptor complete.
2. **Interrupt.** The NIC raises an MSI-X interrupt, normally one per RX queue, each bound to a core. The driver's ISR does almost nothing: it calls `napi_schedule()` and *masks* further interrupts from that queue.
3. **Softirq.** On return from the interrupt, the kernel runs the `NET_RX` softirq, whose handler `net_rx_action()` calls the driver's `poll()` function.
4. **Poll with a budget.** `poll()` takes completed descriptors from the ring, up to its *budget* (64 packets per call by default; `net.core.netdev_budget`, default 300, bounds one softirq run across devices). For each packet it builds an `sk_buff` and sets its protocol. If fewer packets than the budget were waiting, the ring is empty: the driver calls `napi_complete_done()` and re-enables the interrupt. If the budget was used up, polling continues without any new interrupt.
5. **GRO.** `napi_gro_receive()` merges consecutive segments of the same flow into one larger `sk_buff`, so that the upper layers run once for many packets.
6. **Link layer to IP.** `__netif_receive_skb_core()` hands the packet first to taps (`AF_PACKET` sockets; this is where `tcpdump` sees it), then to the protocol handler for its EtherType: `ip_rcv()`.
7. **IP.** The netfilter `PREROUTING` hook runs; a route lookup finds the destination local; `ip_local_deliver()` runs the `INPUT` hook and passes the packet to the transport protocol.

5. The remaining data stays in the socket buffer with no further notification until new data arrives, so the connection stalls.↩

8. **TCP.** `tcp_v4_rcv()` finds the socket from the four-tuple, `tcp_rcv_established()` handles sequence numbers, ACKs and the window (Pillar 3 (p. 162) covers what these do), and the data is appended to the socket's receive queue. A process sleeping in `recv()`, or in `epoll_wait()` on this socket, is made runnable.
9. **User copy.** When the process runs, `recv()` enters `tcp_recvmsg()`, which copies the bytes from the `sk_buff`s into the user's buffer (`copy_to_user`) and frees them. This is the one copy of the payload on the receive path.

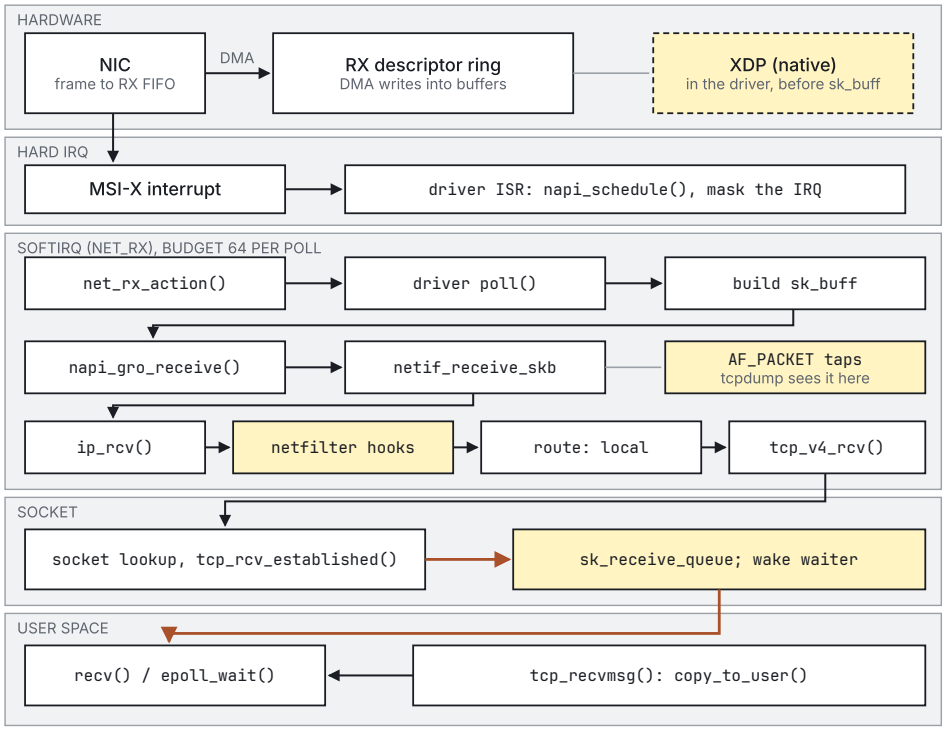


Figure 7.21 The receive path. Yellow marks where an observer can see or alter the packet: native XDP (before any sk_buff exists), the AF_PACKET taps, the netfilter hooks, and the socket queue.

The sk_buff

An `sk_buff` (*socket buffer*, "skb") is the kernel's packet descriptor. It does not hold the packet; it points into a linear data buffer by four pointers, `head` (start of the allocation), `data` (start of the current packet header), `tail` (end of the data) and `end` (end of the allocation), and keeps the lengths, protocol, device, timestamps and the owning socket. Each layer then moves the `data` pointer instead of copying bytes: `skb_pull(n)` advances `data` by n to strip a header as the packet climbs, `skb_push(n)` moves it back to prepend one as the packet descends, `skb_put(n)` extends `tail` to append, and `skb_reserve(n)` leaves headroom at the start so that headers can be pushed later without reallocation. Pulling the 14-byte Ethernet header, as `ip_rcv` expects, is exactly `skb_pull(skb, 14)` (Figure 7.22). Large payloads sit outside the linear area in fragment pages described by `skb_shared_info` at `end`, which is how GRO and scatter-gather avoid copying.

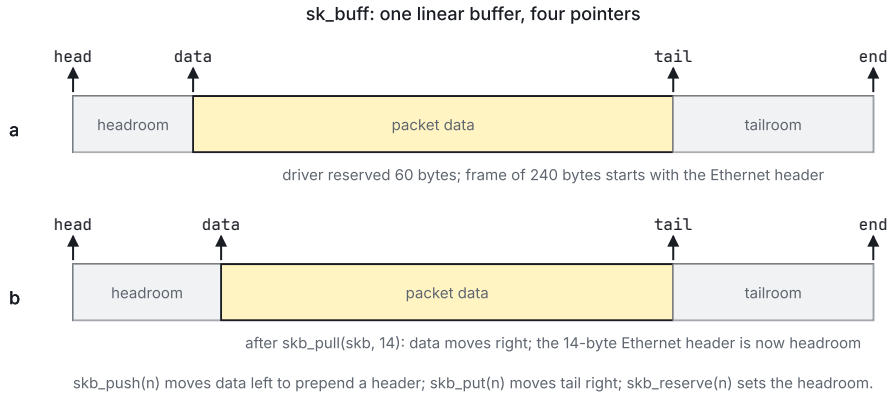


Figure 7.22 The four sk_buff pointers, before (a) and after (b) skb_pull(skb, 14). Only the pointer moves; no bytes are copied.

The kernel allocates `sk_buff` structures from a slab cache (7.7 (p. 435)), which is why allocation is cheap enough for millions of packets per second.

Transmit, and where acceleration attaches

The transmit path is the mirror image: `send()` enters `tcp_sendmsg()`, which copies the data into `sk_buff`s and builds segments; `ip_queue_xmit()` adds the IP header and routes the packet; the queueing discipline (`qdisc`) schedules it; `dev_queue_xmit()` hands it to the driver's `ndo_start_xmit()`, which places a descriptor in the TX ring and tells the NIC; and a completion interrupt, again batched, lets the driver free the `sk_buff`.

Several mechanisms exist because one core cannot keep up with a fast link. *RSS* (receive-side scaling) has the NIC hash each flow's addresses and ports to one of several RX queues, each with its own interrupt, so different flows are processed on different cores while the packets of one flow stay in order. *RPS* and *RFS* do the same steering in software, for NICs without enough queues, and RFS prefers the core on which the consuming application runs. *XDP* runs a small verified program, written in restricted C and loaded into the kernel, in the driver before any `sk_buff` is allocated; it can drop, redirect or pass the packet, which is the fastest place to discard unwanted traffic. Because it acts before the `AF_PACKET` tap, a packet dropped by native XDP is invisible to `tcpdump`. *DPDK* leaves the kernel path altogether: a user-space poll-mode driver reads the NIC's rings directly from huge-page memory, with no interrupts, no system calls and no `sk_buff`, at the cost of dedicating cores and reimplementing the protocol stack above it.

	Kernel stack	XDP	DPDK
Where packets are handled	kernel, full stack	kernel driver, before sk_buff	user space, kernel bypassed
Interrupts	yes, with NAPI polling	yes, same driver	none: polling cores
Copies and system calls	one copy; one call per `recv`	none for dropped or redirected	none
Protocol stack	Linux's own	Linux's, for passed packets	application supplied
Typical use	servers, routers	filtering, load balancing, DDoS drop	forwarding and virtual-switch data planes

LAB

`LAB 7.12 · 20 MIN` Observe the stack's knobs and counters (outputs depend on the NIC and driver, so none is reproduced here).

```
$ ethtool -l eth0                       # RX/TX queue counts (RSS)
$ ethtool -C eth0 rx-usecs 50           # interrupt coalescing: one interrupt per 50
us
$ ethtool -S eth0 | grep -i -E 'drop|miss|rx_queue'
$ cat /proc/net/softnet_stat            # per-CPU: processed, dropped, time_squeeze
$ sudo bpftrace -e 'kprobe:tcp_v4_rcv { @[comm] = count(); }'
```

What you should notice: during an `iperf3` run the packets are spread over the queues listed by `ethtool -l`; a rising third column of `softnet_stat` (`time_squeeze`) means the poll budget ran out and the receive path is falling behind; and the `bpftrace` count attributes `tcp_v4_rcv` calls to whichever process happened to be on the CPU when the softirq ran, a reminder that softirq work is charged to no one in particular.

TRAP

Claim: "`tcpdump` sees every packet the NIC received, and it sees them before the firewall." *Why it is wrong:* `tcpdump` taps at `AF_PACKET`, which is after the driver and any native XDP program (so XDP drops are invisible) and, on ingress, before netfilter (so firewall drops are visible as received). *Correct statement:* ingress capture sits between XDP and netfilter. Further, "NAPI means no interrupts" is also wrong: it means interrupts are masked during polling and re-enabled when the queue empties.

INTERVIEW LENS

CISCO · JUNIPER · ARISTA · NOKIA · BROADCOM · QUALCOMM

"How does a packet travel from the NIC to a user-space application?" and "what is `sk_buff`?" are standard at platform and SDK roles; Qualcomm data-path interviews ask about NAPI and DMA rings, and Cisco and Nokia add XDP versus DPDK. A strong answer walks the nine steps with function names, states why polling replaces per-packet interrupts (the arithmetic in 7.8 (p. 441)), explains that `sk_buff` moves pointers and not bytes, and distinguishes where XDP and DPDK intercept. Follow-ups: where `tcpdump` sees the packet, what RSS does, and how the transmit path differs.

CHECK YOURSELF

1. What does the driver do with the RX interrupt when NAPI starts polling? [1]
2. How does `skb_pull(skb, 14)` remove the Ethernet header without copying? [2]
3. Where does native XDP run relative to `sk_buff` allocation? [3]

7.13 Firmware and embedded essentials SHOULD · ■■□

Network firmware runs on small cores, often under a real-time operating system (RTOS) or on bare metal, close to the hardware this chapter has described. Correctness there rests on a handful of habits: telling the compiler that a location can change without its knowledge, ordering memory accesses, doing little in an ISR, and passing data between interrupt and main code without locks.

volatile and device registers

A *memory-mapped* device register is read or written like memory, but a read may return a different value each time and a write has an effect. The compiler, which assumes memory changes only when the program changes it, may then optimise the access away. `volatile` forbids that: every

source-level access to the object becomes a real access, in program order with respect to other volatile accesses. The qualifier combinations follow from this: `const volatile uint32_t *status` is a read-only register that the hardware may change; `volatile uint32_t *data` is a register the program may write.

```
#include <stdint.h>

/* read-only device register */
#define UART_STATUS (*(const volatile uint32_t *)0x40001004u)
/* read/write device register */
#define UART_DATA   (*(volatile uint32_t *)0x40001000u)
#define TX_READY    (1u << 5)

void uart_putc(char c)
{
    while (!(UART_STATUS & TX_READY))  /* volatile: re-read each pass */
        ;  /* without it the compiler may read once and spin for ever */
    UART_DATA = (uint32_t)c;  /* volatile: this store must really happen */
}

/* ISR-to-main hand-over: a flag set in the handler and polled by the main loop */
static volatile uint32_t rx_flag;

void uart_isr(void)
{
    rx_flag = 1;  /* do little here; leave the work to main */
}

int main_loop_step(void)
{
    if (rx_flag) {                      /* volatile: re-read on each call */
        rx_flag = 0;
        return 1;
    }
    return 0;
}
```

Compiling this with `-O2` and reading the assembly shows the load of the status register *inside* the loop; remove `volatile` and the compiler is entitled to load once and spin for ever. Two limits must be stated. `volatile` does not make an access atomic, and it does not order the access against *non-volatile* accesses or against another core; the flag in the example is safe only because one interrupt handler and one main loop share a single core. Between cores one uses C11 atomics with acquire and release ordering, or memory barriers, which stop the CPU and compiler reordering accesses across them. Storage classes (`static`, `extern`) and the full qualifier rules are in Pillar 2 (p. 114).

Lock-free ring buffers between an ISR and the main loop, or between two cores

The standard hand-over structure is the *single-producer single-consumer* (SPSC) ring. Because each index has exactly one writer, no lock is needed: the producer writes the slot and then publishes it by advancing `tail` with *release* ordering; the consumer reads `tail` with *acquire* ordering, which guarantees that it sees the slot's contents, reads the slot, and frees it by advancing `head`. With a power-of-two size, `index & (size - 1)` replaces the modulo. The same layout, one side owning each index, is the descriptor ring that a NIC and its driver share (7.8 (p. 441)).

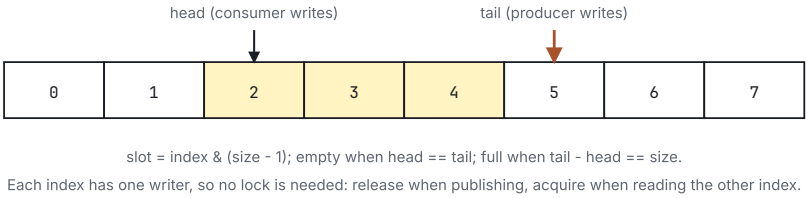


Figure 7.23 An SPSC ring. The consumer owns head, the producer owns tail; full is tail - head = size, empty is head = tail.

```
#include <pthread.h>
#include <stdatomic.h>
#include <stdint.h>
#include <stdio.h>

#define RING 1024                       /* power of two: mask replaces modulo */
#define MASK (RING - 1)

struct ring {
    uint32_t slot[RING];
    _Atomic uint32_t head;              /* written only by the consumer */
    _Atomic uint32_t tail;              /* written only by the producer */
};

static int ring_put(struct ring *r, uint32_t v)       /* producer only */
{
    uint32_t t = atomic_load_explicit(&r->tail, memory_order_relaxed);
    uint32_t h = atomic_load_explicit(&r->head, memory_order_acquire);
    if (t - h == RING)
        return 0;                       /* full */
    r->slot[t & MASK] = v;
    atomic_store_explicit(&r->tail, t + 1, memory_order_release);  /* publish */
    return 1;
}

static int ring_get(struct ring *r, uint32_t *v)      /* consumer only */
{
    uint32_t h = atomic_load_explicit(&r->head, memory_order_relaxed);
    uint32_t t = atomic_load_explicit(&r->tail, memory_order_acquire);
    if (h == t)
        return 0;                       /* empty */
    *v = r->slot[h & MASK];
    atomic_store_explicit(&r->head, h + 1, memory_order_release);  /* free slot */
    return 1;
}

static struct ring q;
#define COUNT 1000000u

static void *prod(void *arg)
{
    (void)arg;
    for (uint32_t i = 1; i <= COUNT; i++)
        while (!ring_put(&q, i))
            ;                           /* spin while full */
    return NULL;
}

int main(void)
{
    pthread_t p;
    uint64_t sum = 0;
    uint32_t v, got = 0;
    pthread_create(&p, NULL, prod, NULL);
    while (got < COUNT)
        if (ring_get(&q, &v)) { sum += v; got++; }
    pthread_join(p, NULL);
    printf("received %u items, sum %llu (expected %llu)\n", got,
           (unsigned long long)sum, (unsigned long long)COUNT * (COUNT + 1) / 2);
    return 0;
}
```

The program prints `received 1000000 items, sum 500000500000 (expected 500000500000)`. The indices are free-running 32-bit counters that wrap harmlessly, because the

difference `t - h` stays correct in unsigned arithmetic; the slot is found by masking. A full ring is reported by `ring_put` returning 0, and the caller decides whether to drop, retry or back-pressure.

Byte order and bit manipulation are the other firmware staples. Whether a header field is big-endian (IP, TCP) or little-endian (802.11 Frame Control) decides whether `ntohs` or a plain read is correct; the endianness test, `__builtin_bswap32`, mask-and-shift field extraction and the set, clear and toggle idioms are all in Pillar 2 (p. 117).

TRAP

Claim: "Declaring the shared variable `volatile` makes the code thread-safe." *Why it is wrong:* `volatile` neither makes read-modify-write atomic nor orders memory accesses between cores. *Correct statement:* it prevents the compiler from removing or caching accesses, which suits device registers and ISR flags on one core; for threads and cores use atomics, barriers or locks.

INTERVIEW LENS

QUALCOMM · BROADCOM · ARISTA

"What does `volatile` do, and is it enough for sharing data between an ISR and a thread?" and "implement a lock-free ring buffer" are reported for firmware and platform roles. A strong answer gives the register example and its assembly consequence, names the single-writer-per-index rule, and chooses acquire and release ordering over a lock. Follow-ups: why a mutex cannot be used inside an ISR, what happens when the producer is faster than the consumer, and why the indices may be free-running.

CHECK YOURSELF

1. What does `const volatile uint32_t *p` declare? [4]
2. Why does the SPSC ring need no lock? [5]

7.14 Debugging a slow server OPTIONAL · ■■■

Consider a TCP service whose latency rises under load although CPU use is low. The method is that of Pillar 11 (p. 634): a symptom, a hypothesis that this chapter makes specific, a command that tests it, and the evidence that confirms or rejects it.

Symptom	Hypothesis	Command	Evidence	Fix
New connections wait, then time out	the accept queue is full: the server does not call `accept` fast enough	`ss -ltn`	`Recv-Q` on the listener near the backlog	find why the loop is blocked; raise backlog and `somaxconn` only after that
One thread at 100 % of a core, others idle	a single-threaded loop does blocking disk work	`top -H`, `strace -p`	the thread sits in `read` on a file, not in `epoll_wait`	keep the event loop non-blocking; push slow work to a thread pool and return results through an `eventfd`
Packets arrive but the application sees gaps	receive processing falls behind	`cat /proc/net/softnet_stat`	third column (`time_squeeze`) or second (dropped) rising	spread flows over queues (RSS), set IRQ affinity, raise `netdev_budget`
Many sockets in TIME_WAIT,				

`EADDRNOTAVAIL` on connect	a client opens and closes connections faster than ports recycle	`ss -tan state time-wait | wc -l`	tens of thousands of entries	reuse connections (keep-alive), widen the ephemeral port range
Throughput stops while `Send-Q` is high	the peer reads slowly; the send buffer is full	`ss -tin`	`Send-Q` large, small advertised window	add per-connection output buffering and `EPOLLOUT`; look at the receiver

A full accept queue with one busy thread is the classic first finding. The cause is in the application, the cheap fix is the event loop of 7.11 (p. 453) with a worker pool, and only then does tuning `softnet_stat` and the NIC queues pay off, because kernel tuning cannot rescue a loop that is not calling `accept`.

7.15 Mini-project: TinyRelay, an epoll chat and relay server

OPTIONAL · ■■■

The project joins the sections of this chapter into one program that can be shown at an interview. It is a single-threaded C server first, then a multi-threaded one, built in the milestones below; each milestone is small enough to finish in an evening.

Milestone	Build	Section
1	an edge-triggered `epoll` loop with non-blocking sockets and length-prefixed frames	7.10 (p. 445), 7.11 (p. 453)
2	chat rooms: `JOIN`, `MSG` and `LEAVE` commands, with fan-out to every member of a room	7.11
3	a per-client output buffer; register `EPOLLOUT` only while it is non-empty (back-pressure)	7.11
4	a thread pool for slow commands, fed by a mutex and condition-variable queue; results return to the loop through an `eventfd`	7.5 (p. 425)
5	a UDP statistics port reporting connected clients, messages per second, bytes in and out	7.10
6	graceful shutdown on SIGINT through `signalfd`	7.9 (p. 443)
7	load test with `loadgen` at 1,000 and 5,000 connections; report latency percentiles	7.11
8	builds clean under ThreadSanitizer and AddressSanitizer, with a Makefile	7.3 (p. 420)

Stretch goals: IPv6 through `getaddrinfo`; `SO_REUSEPORT` sharding across worker processes; an XDP program that drops packets from a blocklisted address before they reach the server (7.12 (p. 460)

). The pay-off is a defensible answer to “design a scalable server”, “explain epoll” and “how did you test the concurrency?”

1. It masks the queue’s interrupt, and re-enables it when polling finds the ring empty.↩
2. It advances the `data` pointer by 14 bytes and reduces `len`; the header bytes remain in memory as headroom.↩
3. Before it, in the driver, which is why XDP is cheaper than any stage that follows.↩
4. A pointer to a 32-bit value that the program may not write through but that may change on its own, such as a hardware status register.↩
5. Each index is written by one side only; release and acquire ordering make the slot contents visible before the index that publishes them.↩

Problem Set

Every interview question of this pillar is solved once, here, in the compact template: a 30-second answer first, then the approach, code, follow-ups and common mistakes where they apply. Tags give the companies and round where the question was reported, the difficulty (■□□ easy, ■■□ medium, ■■■ hard), and **S** when a specific question was reported by candidates or **T** when only the topic was reported and the wording is ours. Where another pillar already solves a question in full, the entry here gives the 30-second answer and a pointer, so that nothing is solved twice.

Q7.1 Process versus thread: what is shared and what is not?

CISCO · JUNIPER · QUALCOMM · ARISTA · TECH-1 · ■□□ · S

30-second answer: threads of one process share its address space (code, globals, heap) and open files; each thread has its own stack, registers and thread-local storage. Processes share nothing unless set up explicitly.

Answer. A process is the unit of resource ownership (address space, descriptors, signal handlers); a thread is the unit of scheduling inside it. Thread creation and switching are cheaper, and communication is a plain memory write, but there is no isolation. **Follow-ups.** The 1:1 model in Linux; `clone()` flags. **Mistakes.** Saying threads have separate heaps (7.3 (p. 420)).

Q7.2 What happens if one thread segfaults?

CISCO · TECH-1 · ■□□ · S

30-second answer: the whole process dies, because SIGSEGV is delivered to the process and its default action terminates it, taking every thread along.

Answer. All threads share one address space, so a wild pointer in one may already have corrupted the others. A handler may catch the signal, but resuming is unsafe. **Follow-ups.** `exit()` from any thread ends the process, while `pthread_exit()` ends only the caller; one process per worker (as browsers and some routing daemons do) contains a crash.

Q7.3 How many times does `hi` print? `fork(); fork(); fork(); printf("hi\n");` Then `fork() && fork() || fork().`

CISCO · JUNIPER · QUALCOMM · TECH-1 · ■■□ · S

30-second answer: 8, because each fork doubles the number of processes; the second expression creates 5 processes.

Approach. Three forks give $2^3 = 8$ processes, each reaching the `printf`. For the expression, the parent gets a non-zero PID from both forks, so `&&` is true and `||` skips the third fork: 1 process. The first child gets 0, so `&&` is false and it runs the third fork: 2 processes. The second child gets 0 from its fork, so it also runs the third fork: 2 processes. Total 1 + 2 + 2 = 5 (Figure 7.4). **Mistakes.** Forgetting stdout buffering: with output redirected, an unflushed `printf("hi")` before the fork is copied and printed twice.

Q7.4 What is a zombie process? What is an orphan?

CISCO · JUNIPER · TECH-1 · ■□□ · S

30-second answer: a zombie is a child that has exited but whose parent has not yet called `wait()`, so its control block and exit status remain; an orphan is a running child whose parent has exited, adopted by PID 1.

Answer. A zombie uses a PID slot and no other memory, shows state `Z` in `ps`, and cannot be killed; it disappears when the parent waits or dies. Reap with `waitpid(-1, NULL, WNOHANG)` in a SIGCHLD handler, or ignore SIGCHLD with `SA_NOCLDWAIT`. **Mistakes.** Calling an orphan a zombie (7.2 (p. 416)).

Q7.5 What happens on a context switch? Why is a thread switch cheaper?

CISCO · QUALCOMM · TECH-1 · ■■□ · S

30-second answer: the kernel saves the running task's registers in its control block, picks the next task, and restores that task's registers; switching to a thread of the same process avoids changing the page tables and so keeps the TLB and caches warm.

Answer. Steps: (i) a timer interrupt, a block or a yield enters the kernel; (ii) save registers and program counter; (iii) the scheduler chooses; (iv) if the address space differs, load the new page-table base (flushing or retagging the TLB); (v) restore registers and return. The indirect cost, refilling caches and TLB, often exceeds the direct few microseconds. **Mistakes.** Claiming a thread switch flushes the TLB.

Q7.6 Compute the average waiting time under round robin with q = 2 for P1 (arrival 0, burst 5), P2 (1, 3), P3 (2, 1), P4 (3, 2).

QUALCOMM · CISCO · OA · ■■□ · S

30-second answer: the average waiting time is 4.5 time units.

Approach. Take the convention that a newly arrived process joins the ready queue before a pre-empted one. Gantt chart: P1 0–2, P2 2–4, P3 4–5, P1 5–7, P4 7–9, P2 9–10, P1 10–11. Completion times: P1 11, P2 10, P3 5, P4 9. Turnaround = completion − arrival: 11, 9, 3, 6. Waiting = turnaround − burst: 6, 6, 2, 4. **Answer.** (6 + 6 + 2 + 4)/4 = 4.5; average turnaround 29/4 = 7.25. **Mistakes.** Using the opposite arrival convention without saying so, which changes the order at t = 2 and the totals.

Q7.7 What is starvation? How does ageing fix it?

QUALCOMM · TECH-1 · ■■□ · S

30-second answer: starvation is a ready task that is never chosen, for example a low-priority task behind a steady stream of high-priority ones; ageing raises a task's priority the longer it waits, so it eventually runs.

Answer. Starvation differs from deadlock: the system as a whole progresses. Sources are priority scheduling, SJF with a stream of short jobs, and reader-preferring locks. Cures are ageing, fair queueing, feedback queues with a periodic boost, and writer-preferring locks (7.4 (p. 423)).

Q7.8 What is a race condition? What is a critical section?

CISCO · JUNIPER · QUALCOMM · MICROSOFT · TECH-1 · ■□□ · S

30-second answer: a race condition is a result that depends on the timing of unsynchronised accesses to shared data, at least one of them a write; a critical section is the code that touches the shared data and must run by one thread at a time.

Answer. `counter++` is load, add, store, so two threads can both load 5 and both store 6 (7.3 (p. 420)). A correct solution gives mutual exclusion, progress (a free section is granted to a waiter) and bounded waiting (no thread is overtaken for ever). **Follow-ups.** Detecting races with ThreadSanitizer; atomics for single-word updates.

Q7.9 Mutex versus binary semaphore: when would you use each?

JUNIPER · CISCO · QUALCOMM · TECH-1 · ■■□ · S

30-second answer: a mutex has an owner and only the locker may unlock it, so it protects shared data; a binary semaphore has no owner and any thread may post it, so it signals events between threads.

Answer. Ownership gives the mutex priority inheritance, recursive and error-checking variants and misuse detection. Use a mutex around a critical section. Use a semaphore when one thread, or an interrupt handler, tells another that something has happened ("buffer ready"), or a counting semaphore to bound a pool. **Mistakes.** Using a semaphore as a lock and then suffering priority inversion with no inheritance available (7.5 (p. 425)).

Q7.10 Mutex versus spinlock. Which is used in an interrupt handler, and why?

QUALCOMM · BROADCOM · CISCO · TECH-1 · ■■□ · S

30-second answer: a mutex puts a waiter to sleep, a spinlock busy-waits; an interrupt handler has no task context to sleep in, so it must use a spinlock, with interrupts disabled while it is held.

Answer. Spin when the critical section is shorter than a context switch and the holder is running on another core. On one CPU a spinlock held by a pre-empted task would spin for ever, which is why the kernel disables pre-emption while one is held. Use `spin_lock_irqsave` if an ISR can also take the lock, otherwise the ISR could interrupt the holder on the same core and spin for ever. **Mistakes.** Sleeping or calling `kmalloc(GFP_KERNEL)` under a spinlock.

Q7.11 Implement a counting semaphore using a mutex and a condition variable.

JUNIPER · ARISTA · TECH-2 · ■■■ · S

30-second answer: keep a counter under a mutex; `wait` sleeps on the condition variable while the counter is zero, then decrements; `post` increments and signals.

Code.

```
#include <pthread.h>

typedef struct {
    int n;
    pthread_mutex_t m;
    pthread_cond_t c;
} sem_t2;

void s_init(sem_t2 *s, int n)
{
    s->n = n;
    pthread_mutex_init(&s->m, 0);
    pthread_cond_init(&s->c, 0);
}

void s_wait(sem_t2 *s)
{
    pthread_mutex_lock(&s->m);
    while (s->n == 0)
        pthread_cond_wait(&s->c, &s->m);
    s->n--;
    pthread_mutex_unlock(&s->m);
}

void s_post(sem_t2 *s)
{
    pthread_mutex_lock(&s->m);
    s->n++;
    pthread_cond_signal(&s->c);
    pthread_mutex_unlock(&s->m);
}
```

Answer. The loop in `s_wait` rechecks the counter because the wake-up may be spurious or stolen by another waiter that decremented first. One `signal` per `post` is enough, since one unit of the resource appeared. **Follow-ups.** Use `sem_timedwait`-style timeouts with `pthread_cond_timedwait`; this is how POSIX semaphores are built on some systems.

Q7.12 Implement producer–consumer with a bounded buffer. Why does the wait use a `while` loop?

JUNIPER · CISCO · GOOGLE · TECH-2 · ■■■ · S

30-second answer: a circular buffer with a mutex, a `not_full` and a `not_empty` condition variable; each side waits in a `while` on its predicate, because a woken thread must recheck that the condition still holds.

Approach. The complete program with three producers and two consumers is in 7.5 (p. 425) (`bbuf.c`), with the semaphore version beside it. Producer: lock; `while (count == N) wait(not_full)`; store; `count++`; signal `not_empty`; unlock. Consumer: the mirror. **Answer.** `while`, not `if`, since the standard allows spurious wake-ups and another thread may consume the item between the signal and the waiter's re-acquisition of the mutex. **Mistakes.** Waiting on the semaphore after taking the mutex; forgetting to wake sleeping consumers at shutdown.

Q7.13 What are the four conditions for deadlock? How do you prevent each?

CISCO · JUNIPER · TECH-1 · ■■□ · S

30-second answer: mutual exclusion, hold and wait, no pre-emption and circular wait must all hold; break any one. In practice, impose a global lock order to break circular wait.

Answer. Mutual exclusion: make the resource shareable. Hold and wait: request everything at once. No pre-emption: on a failed `trylock`, release all and retry. Circular wait: order the locks and acquire in order. Example: threads locking A,B and B,A deadlock; the fixed order A,B for both (7.6 (p. 431)) cures it. **Mistakes.** Treating mutual exclusion as removable for locks that protect writes.

Q7.14 Run the banker's algorithm: totals (6, 3, 4); Allocation P0 (1,0,1), P1 (1,1,0), P2 (1,0,1), P3 (0,1,1); Max P0 (3,2,2), P1 (2,2,1), P2 (2,1,2), P3 (1,2,2). Is the state safe?

QUALCOMM · CISCO · OA · ■■■ · S

30-second answer: yes; the safe sequence is P1, P2, P3, P0.

Approach. Available = (6, 3, 4) − (3, 2, 3) = (3, 1, 1). Need = Max − Allocation: P0 (2,2,1), P1 (1,1,1), P2 (1,1,1), P3 (1,1,1). Work = (3,1,1): P1 fits, Work = (4,2,1). P2 fits, Work = (5,2,2). P3 fits, Work = (5,3,3). P0 needs (2,2,1), fits, Work = (6,3,4), the totals. **Answer.** Safe, in the order ⟨P1, P2, P3, P0⟩. **Follow-ups.** If P0 asked for (2,0,0) now: it is within its need and within Available, and after the pretend grant Available is (1,1,1), still enough to run P1 and then everything else, so grant. **Mistakes.** Comparing Max (not Need) with Work.

Q7.15 What is priority inversion? Explain priority inheritance using the Mars Pathfinder case.

QUALCOMM · CISCO · TECH-1 · ■■□ · S

30-second answer: a high-priority task waits for a lock held by a low-priority task that a medium-priority task keeps pre-empting; priority inheritance lets the lock holder run at the waiter's priority until it unlocks.

Answer. On Pathfinder (1997) a low-priority task held a bus lock, a medium task ran, and the high-priority bus manager missed its deadline and triggered a reset; the engineers enabled inheritance on the lock by remote patch. Alternatives: priority ceiling (the lock has a ceiling priority that its holder inherits), or avoiding the shared lock (7.6 (p. 431)). **Mistakes.** Describing it as deadlock: the system is delayed, not stuck.

Q7.16 Readers–writers: give a writer-preferring solution.

ARISTA · JUNIPER · TECH-2 · ■■■ · S

30-second answer: count readers and waiting writers under a mutex; a reader enters only if no writer is active or waiting; a writer enters when no one is inside; on exit a writer prefers waking another writer.

Code.

```
#include <pthread.h>

typedef struct {
    pthread_mutex_t m;
    pthread_cond_t rd, wr;
    int readers, writer, wwait;
} rw_t;

void r_lock(rw_t *l)
{
    pthread_mutex_lock(&l->m);
    while (l->writer || l->wwait)
        pthread_cond_wait(&l->rd, &l-
>m);
    l->readers++;
    pthread_mutex_unlock(&l->m);
}

void r_unlock(rw_t *l)
{
    pthread_mutex_lock(&l->m);
    if (--l->readers == 0)
        pthread_cond_signal(&l->wr);
    pthread_mutex_unlock(&l->m);
}

void w_lock(rw_t *l)
{
    pthread_mutex_lock(&l->m);
    l->wwait++;
    while (l->writer || l->readers)
        pthread_cond_wait(&l->wr, &l-
>m);
    l->wwait--;
    l->writer = 1;
    pthread_mutex_unlock(&l->m);
}

void w_unlock(rw_t *l)
{
    pthread_mutex_lock(&l->m);
    l->writer = 0;
    if (l->wwait)
        pthread_cond_signal(&l->wr);
    else
        pthread_cond_broadcast(&l->rd);
    pthread_mutex_unlock(&l->m);
}
```

Answer. New readers yield to any waiting writer (`wwait`), so writers cannot starve; the cost is that readers can starve under a steady stream of writers, which is the dual problem. A fair alternative queues arrivals in order. **Follow-ups.**

`pthread_rwlock_t` with `PTHREAD_RWLOCK_PREFER_WRITER_NONRECURSIVE_NP` provides the same policy.

Q7.17 What is virtual memory and why is it needed?

CISCO · JUNIPER · TECH-1 · ■□□ · S

30-second answer: each process sees a private, contiguous address space that the MMU maps to physical frames; it provides isolation, simple linking and loading, and lets pages be allocated on demand or swapped out.

Answer. Benefits: protection (a process cannot name another's memory), relocation (every program may assume the same layout), sharing (libraries, copy-on-write), and over-commitment through demand paging. **Mistakes.** Describing it only as "disk used as RAM" (7.7 (p. 435)).

Q7.18 Explain paging, the page table and the TLB. What happens on a TLB miss and on a page fault?

CISCO · QUALCOMM · ARISTA · TECH-1 · ■■□ · S

30-second answer: memory is split into fixed pages; the page table maps virtual pages to frames; the TLB caches recent translations. A TLB miss causes a table walk; a page fault, when the entry is absent or forbidden, enters the kernel to load the page or kill the process.

Approach. Address = page number + offset. Worked check: 0x00403ABC with 4 KB pages maps page 0x00403 to frame 0x1A2, giving 0x1A2ABC. A TLB miss costs extra memory reads (one per level) and is handled by hardware or a short kernel routine; a page fault costs microseconds (minor) to milliseconds (major, disk). **Answer.** EAT = 0.95 × 101 + 0.05 × 201 = 106 ns for the figures in 7.7 (p. 435). **Mistakes.** Confusing the two: a TLB miss is not an error.

Q7.19 Compute the page faults for LRU with three frames on the reference string 1 2 3 4 2 1 5 6 2 1 2 3 7 6 3 2 1 2 3 6.

QUALCOMM · OA · ■■□ · S

30-second answer: 15 faults in 20 references.

Approach. Track the three most recently used pages; a reference is a hit if the page is among them, and otherwise evicts the least recently used. Outcome per reference (F fault, h hit): F F F F h F F F F F h F F F h F F h h F. Count the Fs: 15. **Check.** With four frames the same string gives 10 faults, so memory helps. **Mistakes.** Not moving a page to most-recent on a hit.

Q7.20 What is Bélády's anomaly?

QUALCOMM · TECH-1 · ■■□ · S

30-second answer: for FIFO replacement, giving a process more frames can increase its page faults; LRU and Optimal are immune because they are stack algorithms.

Answer. On 1 2 3 4 1 2 5 1 2 3 4 5, FIFO takes 9 faults with three frames and 10 with four. The pages in memory under LRU with n frames are always a subset of those with n + 1 frames, which excludes the anomaly (7.7 (p. 435)).

Q7.21 What is copy-on-write and how does `fork()` use it?

CISCO · GOOGLE · TECH-1 · ■■□ · S

30-second answer: parent and child share the same physical pages, marked read-only; the first write by either raises a fault and the kernel copies just that page for the writer.

Answer. This makes `fork` fast and makes `fork` plus `exec` nearly free, since the child's pages are never copied. It also explains why a forked child's memory use grows only as it writes (7.7 (p. 435)). **Follow-ups.** Redis snapshots rely on it; a parent writing heavily during a snapshot can double memory use.

Q7.22 Stack versus heap: which is faster to allocate, and why?

ARISTA · QUALCOMM · TECH-1 · ■□□ · S

30-second answer: the stack, because allocation is one adjustment of the stack pointer and the memory is likely in cache; the heap must search for a suitable block and may fragment.

Answer. Solved in full once in Q2.14 (p. 144) of Pillar 2, with lifetime and size limits. The systems angle added here is that heap allocation may take a lock and, on first touch, a page fault (7.7 (p. 435)).

Q7.23 What is `sizeof` this struct, and how do you reduce it?

ARISTA · QUALCOMM · CISCO · TECH-1 · ■■□ · S

30-second answer: add padding so each member sits at a multiple of its alignment and the total is a multiple of the largest alignment; order members by decreasing size to shrink it.

Answer. Worked in Q2.3 (p. 144) and Q2.2 (p. 144) of Pillar 2 (for example `{char; int; char}` is 12 bytes and 8 when reordered). For wire formats, pack the structure (`__attribute__((packed))`) and accept the unaligned-access cost, or better, parse by offset.

Problem Set, continued: firmware, sockets and the stack

Q7.24 Design a memory allocator (malloc and free) for an embedded system with no OS.

ARISTA · QUALCOMM · BROADCOM · TECH-2 · ■■■ · S

30-second answer: carve blocks from a static arena, each with a header (size, free flag); `malloc` takes the first free block that fits and splits it; `free` marks the block free and merges it with free neighbours.

Approach. The tested first-fit implementation is in 7.7 (p. 435) (`alloc.c`). State the design choices: alignment (round sizes to 16), the fit policy (first fit is fast, best fit wastes less), splitting only when the remainder can hold a header, coalescing on free, and what happens when the arena is exhausted (return NULL, never block). **Answer.** For hard real-time firmware prefer *fixed-size pools*: one free list per size class, O(1) allocate and free, no external fragmentation (Q2.59 (p. 144) builds one). Make the allocator thread- and ISR-safe with a spinlock or by disabling interrupts for the few instructions of list surgery. **Mistakes.** Forgetting alignment, and coalescing only forwards.

Q7.25 What is an interrupt? Describe the steps from device signal to the return from the ISR.

QUALCOMM · CISCO · JUNIPER · TECH-1 · ■■□ · S

30-second answer: a device asserts an interrupt line; the controller selects a CPU, which saves its state, switches to kernel mode, jumps through the vector table to the ISR; the ISR acknowledges the device and defers work, then an `iret` restores the interrupted code.

Answer. Steps: (i) device raises the line; (ii) controller (APIC or GIC) prioritises and delivers; (iii) the CPU finishes its instruction, pushes state, loads the handler address from the vector; (iv) the ISR runs, reads status, acknowledges the device, queues a bottom half; (v) `iret` or `eret` resumes. The ISR runs in interrupt context and must not sleep (7.8 (p. 441)). **Follow-ups.** Maskable versus non-maskable; interrupt latency and nesting.

Q7.26 Top half versus bottom half. Why can't you sleep in an ISR?

QUALCOMM · BROADCOM · TECH-1 · ■■□ · S

30-second answer: the top half is the urgent, short ISR that acknowledges the device; the bottom half (softirq, tasklet, workqueue) does the rest later. An ISR runs on whatever task was interrupted and has no context of its own, so sleeping would block an unrelated task or the scheduler.

Answer. Softirqs and tasklets also cannot sleep; workqueue items run in a kernel thread and can. In an ISR use spinlocks and `GFP_ATOMIC`, never a mutex or `GFP_KERNEL` (7.8 (p. 441)). **Mistakes.** Calling a function that may sleep indirectly, such as `copy_to_user`.

Q7.27 Polling versus interrupts: when is polling better?

CISCO · QUALCOMM · TECH-1 · ■■□ · S

30-second answer: interrupts suit rare events because they cost nothing while idle; polling suits frequent events because it batches them and avoids per-event overhead and interrupt livelock.

Answer. At 14.88 million minimum-size packets per second on a 10 Gb/s link, one interrupt per packet leaves no CPU for work. Hybrid designs interrupt when idle and poll when busy: NAPI. DPDK polls always, trading a burning core for latency. A poll interval bounds the latency of a polled event (7.8 (p. 441)).

Q7.28 What is DMA?

QUALCOMM · BROADCOM · TECH-1 · ■□□ · S

30-second answer: direct memory access lets a device move data to or from main memory itself, so the CPU only sets up a descriptor and handles the completion interrupt.

Answer. A NIC DMAs received frames into buffers named by an RX descriptor ring and the driver processes behind it; the buffers must be physically contiguous or described by a scatter-gather list, and cache coherency with the device must be arranged by the kernel's DMA-mapping calls. **Follow-up.** An IOMMU translates device addresses and limits what a device can reach.

Q7.29 Name the IPC mechanisms. Which is fastest, and why?

CISCO · JUNIPER · TECH-1 · ■■□ · S

30-second answer: pipes, FIFOs, message queues, shared memory, sockets and signals; shared memory is fastest because, after setup, data moves with no system call and no copy.

Answer. The price of shared memory is synchronisation: use a process-shared mutex or a semaphore. Pipes and sockets are simpler and copy twice; signals carry only a number. See the table in 7.9 (p. 443).

Q7.30 What is `volatile`? Give a firmware example. Does it make code thread-safe?

QUALCOMM · ARISTA · TECH-1 · ■□□ · S

30-second answer: `volatile` tells the compiler that every access to the object must really happen, in order, because it may change outside the program, as with a device register; it does not make code thread-safe.

Answer. Worked in Q2.7 (p. 144) and Q2.9 (p. 144) of Pillar 2; the polled UART register and its assembly are in 7.13 (p. 463). It gives neither atomicity nor ordering between cores.

Q7.31 Write a function that detects the machine's endianness.

QUALCOMM · ARISTA · CISCO · TECH-1 · ■□□ · S

30-second answer: store 1 in a 16-bit integer and look at its first byte; if it is 1, the machine is little-endian.

Code.

```c
#include <stdint.h>

int is_little_endian(void)
{
    uint16_t x = 1;
    return *(uint8_t *)&x == 1;
}
```

Answer. Accessing an object through `uint8_t *` (a character type) is always permitted, so this is defined behaviour. A fuller treatment, with a union and `htonl`, is in Q2.10 (p. 144).

Q7.32 Explain the TCP server socket call sequence. What does `listen()`'s backlog mean?

CISCO · JUNIPER · ARISTA · GOOGLE · TECH-1 · ■□□ · S

30-second answer: `socket`, `bind`, `listen`, then `accept` in a loop, with `read`/`write` on the descriptor it returns, and `close`; the backlog sizes the queue of completed connections waiting for `accept`.

Answer. The client mirrors with `socket`, `connect`, `send`/`recv`, `close`. The kernel finishes the three-way handshake itself, parking half-open connections in the SYN queue and completed ones in the accept queue; `accept()` takes the oldest. Linux caps the backlog at `net.core.somaxconn`. When the queue is full, new connection attempts are delayed or dropped (7.10 (p. 445)). **Mistakes.** Reading the backlog as a limit on total connections.

Q7.33 Why do we need `htons` and `htonl`?

CISCO · JUNIPER · TECH-1 · ■■□ · S

30-second answer: the network uses big-endian byte order and most hosts are little-endian, so multi-byte integers in headers and `sockaddr` fields must be converted; forgetting `htons` on a port makes the program use the wrong port.

Answer. 8080 is 0x1F90; without `htons` a little-endian host sends `90 1F`, which the network reads as 0x901F = 36895. Solved in full in Q2.11 (p. 144) of Pillar 2; the lab in 7.10 (p. 445) shows `1F95` in `/proc/net/tcp`.

Q7.34 Why can `recv()` return fewer bytes than were sent? How do you frame messages over TCP?

GOOGLE · ARISTA · CISCO · TECH-1 · ■■□ · S

30-second answer: TCP is a byte stream with no message boundaries, so the kernel may split or merge sends; the receiver loops until it has a whole message, which the sender marks with a length prefix or a delimiter.

Approach. Send a 4-byte length in network order, then the payload; receive with `readn(4)` and then `readn(len)`; `readn` loops over `recv` and retries on `EINTR`. The tested helpers are in 7.10 (p. 445) (`frame.c`), and `coalesce.c` there shows a 25-byte read spanning two messages. **Mistakes.** Trusting the peer's length (cap it), and sending the length in host order.

Q7.35 What is TIME_WAIT, who enters it, and why does `bind` fail with `EADDRINUSE` after a restart?

CISCO · GOOGLE · ARISTA · TECH-1 · ■■□ · S

30-second answer: TIME_WAIT is the state of the side that closes first, lasting 2 × MSL (60 s on Linux); it absorbs delayed segments and lets a lost final ACK be re-sent. A restarted server's `bind` fails because old connections on the port are still in it, unless `SO_REUSEADDR` is set.

Answer. It is intentional protocol behaviour, not a bug; the protocol detail is in Pillar 3 (p. 162). A server that closes connections itself accumulates TIME_WAIT entries; setting `SO_REUSEADDR` before `bind` fixes the restart problem. **Follow-ups.** A client that opens and closes many connections can exhaust ephemeral ports; reuse connections. **Mistakes.** Confusing `SO_REUSEADDR` with `SO_REUSEPORT`.

Q7.36 `select` versus `poll` versus `epoll`.

CISCO · JUNIPER · ARISTA · GOOGLE · TECH-1 · ■■□ · S

30-second answer: all three wait for readiness on many descriptors; `select` and `poll` rescan every descriptor on each call (O(n)), `select` is capped at 1024, and `epoll` keeps the interest list in the kernel and returns only ready descriptors (O(ready)).

Answer. Use the table in 7.11 (p. 453): limit, cost, who holds the set, edge-trigger support, portability. `epoll` is Linux-only; BSD has `kqueue`. For a few descriptors `poll` is as fast and simpler. **Follow-ups.** `io_uring` as a completion-based alternative.

Q7.37 Level-triggered versus edge-triggered epoll. What bug do you get with edge-triggered mode on a blocking socket?

ARISTA · GOOGLE · TECH-2 · ■■■ · S

30-second answer: level-triggered reports a descriptor as long as it is ready; edge-triggered reports only the transition, so you must drain the socket until `EAGAIN`. On a blocking socket the final read blocks for ever and the loop hangs.

Answer. Edge-triggered code: set `O_NONBLOCK`, and on each event loop `read` until it returns `EAGAIN`. If you stop early, the leftover bytes produce no further event until new data arrives, so the connection stalls. Edge-triggered needs fewer system calls; level-triggered is more forgiving (7.11 (p. 453)). **Mistakes.** One read per event.

Q7.38 Design a server handling 100,000 concurrent connections: the threading model, the I/O model and the kernel tuning.

GOOGLE · CISCO · AMAZON · TECH-2 · ■■■ · T

30-second answer: one event loop per core using `epoll` with non-blocking sockets and `SO_REUSEPORT` listeners, a worker pool for blocking work, per-connection state kept small, and raised descriptor and queue limits.

Approach. (i) Model: a thread per core, each with its own epoll instance and listener sharing the port through `SO_REUSEPORT`, so the kernel balances new connections and there is no thundering herd. (ii) I/O: edge-triggered reads drained to `EAGAIN`; per-connection output buffers and `EPOLLOUT` for back-pressure; blocking or CPU-heavy work moved to a pool and returned through an `eventfd`. (iii) Memory: at 100,000 connections, 4 KB of user state and a few KB of socket buffer each is hundreds of megabytes, so bound buffers and use timeouts to reap idle clients. (iv) Limits: `ulimit -n` above 100,000, `net.core.somaxconn` and `tcp_max_syn_backlog` raised, `fs.file-max`. (v) NIC: RSS queues matched to cores with interrupt affinity. **Follow-ups.** What breaks first (descriptors, memory, the accept queue), and how to test (a load generator on several machines). **Mistakes.** Thread per connection; ignoring slow readers.

Q7.39 How does a packet travel from the NIC to a user-space application in Linux?

CISCO · JUNIPER · ARISTA · NOKIA · BROADCOM · TECH-2 · ■■■ · S

30-second answer: the NIC DMAs the frame into an RX ring buffer and interrupts; the driver schedules NAPI, which polls the ring in softirq, builds an `sk_buff`, and passes it through GRO, IP, netfilter and TCP into the socket's receive queue; `recv()` then copies it to user space.

Answer. The chain: NIC → DMA ring → MSI-X IRQ → `napi_schedule` (IRQ masked) → `net_rx_action` → driver `poll` (budget 64) → `sk_buff` → `napi_gro_receive` → `__netif_receive_skb_core` (taps) → `ip_rcv` → netfilter PREROUTING → route (local) → `ip_local_deliver` → INPUT → `tcp_v4_rcv` → socket queue → wake the process → `tcp_recvmsg` → `copy_to_user`. Draw Figure 7.21 of 7.12 (p. 460) from memory and mark XDP and the `tcpdump` tap. **Follow-ups.** The transmit path; RSS; where firewall rules apply.

Q7.40 What is an `sk_buff`? Explain `skb_push` and `skb_pull`.

JUNIPER · ARISTA · BROADCOM · TECH-1 · ■■□ · S

30-second answer: `sk_buff` is the kernel's per-packet descriptor, pointing into a data buffer with `head`, `data`, `tail` and `end`; `skb_pull(n)` moves `data` forward to strip a header, `skb_push(n)` moves it back to prepend one, with no copying.

Answer. As a packet climbs the stack each layer pulls its own header; going down, each pushes its own into headroom reserved by `skb_reserve`. For a received frame, `skb_pull(skb, 14)` exposes the IP header. `skb_put(n)` extends `tail` to append data; large payloads live in fragments after `end` (7.12 (p. 460)). **Mistakes.** Calling `skb_push` without enough headroom, which triggers a panic or a costly reallocation.

Q7.41 What is NAPI and why was it introduced?

QUALCOMM · CISCO · BROADCOM · TECH-1 · ■■□ · S

30-second answer: NAPI is Linux's hybrid receive model: the first packet interrupts, the driver masks the interrupt and polls the ring in softirq up to a budget, then re-enables the interrupt when the ring is empty; it stops interrupt storms at high packet rates.

Answer. Before NAPI one interrupt per packet could livelock a host at line rate (the arithmetic is in 7.8 (p. 441)). With NAPI the cost per packet falls as load rises. Related knobs: interrupt coalescing (`ethtool -C`), the per-poll budget (64) and `netdev_budget` (300). **Mistakes.** "NAPI removes interrupts": it only suppresses them while polling.

Q7.42 What are XDP and DPDK, and how do they differ from the normal kernel path?

CISCO · NOKIA · GOOGLE · TECH-2 · ■■■ · S

30-second answer: XDP runs a small in-kernel program in the driver before an `sk_buff` exists, to drop, redirect or pass packets; DPDK bypasses the kernel with user-space poll-mode drivers that read the NIC rings directly.

Answer. XDP stays inside Linux (the kernel still owns the NIC and the stack handles whatever is passed), so it suits filtering and load balancing; DPDK takes the NIC away from the kernel, dedicates polling cores and huge pages, and the application supplies its own stack, which suits forwarding planes. See the comparison table in 7.12 (p. 460). **Mistakes.** Saying `tcpdump` can see XDP-dropped packets.

Q7.43 Implement a lock-free single-producer single-consumer ring buffer.

ARISTA · BROADCOM · QUALCOMM · TECH-2 · ■■■ · S

30-second answer: a power-of-two array with two free-running indices, each written by exactly one side; the producer stores the slot and then publishes the tail with release ordering, the consumer reads the tail with acquire ordering.

Approach. The compiled C11 program, with a million-item test between two threads, is in 7.13 (p. 463) (`spsc.c`). Key points: empty is `head == tail`; full is `tail - head == size`; the slot is `index & (size - 1)`; unsigned wrap-around keeps the differences right. **Answer.** Only acquire/release is needed because each index has a single writer, and the data written before the release is visible after the matching acquire. Multiple producers or consumers need compare-and-swap loops or locks. **Mistakes.** `volatile` indices without ordering; a non-power-of-two size with free-running counters.

Q7.44 What happens when a client calls `connect()` to a port with no listener? And when the server's accept queue is full?

GOOGLE · CISCO · TECH-1 · ■■□ · S

30-second answer: with no listener the server's kernel answers the SYN with an RST and `connect` fails at once with `ECONNREFUSED`; with a full accept queue the SYN (or the final ACK) is dropped, so the client sees retransmissions and a timeout.

Answer. Refusal is quick and unambiguous. A full queue gives a slow, ambiguous failure: SYN retransmits at roughly 1, 3, 7 seconds and so on, and the symptom resembles packet loss. Diagnose with `ss -ltn` (listener `Recv-Q` at the backlog) and fix the loop that is not calling `accept` (7.14 (p. 467)). **Follow-ups.** A firewall that silently drops gives the same timeout as a full queue, while a firewall that rejects gives a refusal or an ICMP error.

Q7.45 Explain the thundering-herd problem with `accept()` and `epoll`, and its mitigations.

GOOGLE · ARISTA · TECH-2 · ■■■ · T

30-second answer: when several processes or threads wait on one listening socket, a single new connection wakes all of them and all but one go back to sleep, wasting CPU; mitigate with `EPOLLEXCLUSIVE` or by giving each worker its own listener with `SO_REUSEPORT`.

Answer. Modern `accept()` wakes one waiter, but several epoll instances watching the same listener all wake. `EPOLLEXCLUSIVE` asks the kernel to wake one or a few; `SO_REUSEPORT` lets N sockets bind the same port and the kernel hashes connections among them, which also balances load across cores. The trade-off of `SO_REUSEPORT` is that a hashed connection is stuck with its worker even if that worker is busy (7.11 (p. 453)).

Questions solved in other pillars. The TCP state machine and TIME_WAIT timers behind Q7.35 and Q7.44, and congestion control, belong to Pillar 3 (p. 162). The troubleshooting framework behind the slow-server case belongs to Pillar 11 (p. 634). Shell tools such as `ss`, `strace` and `perf` are taught in Pillar 8 (p. 483).

Revision Spread

THE 15-MINUTE VERSION

Ten facts. (1) User mode versus kernel mode is a CPU mode bit; a system call crosses it by the `syscall` instruction and costs far more than a function call. (2) `fork` doubles the processes (n forks give 2^n); a zombie is an exited child not yet waited for, and an orphan is adopted by PID 1. (3) Threads share code, data, heap and descriptors, and own stack, registers and TLS; one thread's SIGSEGV kills the process. (4) A mutex has an owner; a semaphore does not; a spinlock never sleeps and is the only lock usable in an ISR; wait on a condition variable in a `while`. (5) Deadlock needs mutual exclusion, hold and wait, no pre-emption and circular wait; a global lock order breaks the cycle; a cycle in a resource graph is sufficient only for single-instance resources. (6) 4 KB pages: offset 12 bits; PA = frame × 4096 + offset; EAT = h(t + m) + (1 − h)(t + 2m). (7) FIFO can fault more with more frames (Bélády); LRU and Optimal cannot. (8) An ISR is short, cannot sleep, and defers work to a bottom half; DMA moves data with no CPU copy; NAPI polls under load. (9) TCP is a byte stream: loop on `recv`, frame with a length prefix, `htons` the port, set `SO_REUSEADDR`, ignore SIGPIPE. (10) `epoll` costs O(ready); edge-triggered needs non-blocking sockets read to `EAGAIN`; the receive path is NIC, DMA ring, IRQ, NAPI, `sk_buff`, IP, TCP, socket queue, `recv`.

Three figures to redraw. The receive path with XDP and the tcpdump tap (Figure 7.21), the socket call ladder against the TCP handshake (Figure 7.16), and the epoll interest and ready lists (Figure 7.18).

Synchronisation primitives

	Owner	Sleeps?	In ISR?	Use
Mutex	yes	yes	no	protect shared data; priority inheritance
Binary semaphore	no	yes	post only	signal an event
Counting semaphore	no	yes	post only	bound a pool of N
Spinlock	yes	no	yes	very short sections, kernel
Condition variable	no (needs a mutex)	yes	no	wait for a predicate, in a `while`
Read-write lock	yes	yes	no	many readers, few writers
Atomic	none	no	yes	one word, no lock

Deadlock, memory and scheduling formulae

Item	Rule
Coffman conditions	mutual exclusion, hold and wait, no pre-emption, circular wait
Prevention	share, request all at once, release and retry, order the locks
Banker's	Need = Max − Alloc; run any task with Need ≤ Work; Work += its Alloc; safe if all finish
Turnaround, waiting	TAT = completion − arrival; WT = TAT − burst
Response time (RR)	at most (n − 1) × q for a new task
Optimal for average waiting	SJF (non-pre-emptive), SRTF (pre-emptive)
Page split	offset = $\log_2$(page size); VPN = VA ≫ offset bits
Flat table size	2^(VA bits − offset bits) × entry size
Two-level, 10 + 10 + 12	4 KB per table; 8 MB mapped needs 3 tables (12 KB)
x86-64	4 levels × 9 bits + 12 bits = 48-bit addresses

EAT	h × (t + m) + (1 − h) × (t + 2m)
10 Gb/s, 64-byte frames	84 bytes on the wire = 672 bits; 14.88 Mpps

Page replacement on 7 0 1 2 0 3 0 4 2 3 0 3 2, three frames

Policy	Faults	Idea
FIFO	10	evict the oldest loaded
LRU	9	evict the least recently used
Optimal	7	evict the page used furthest ahead
Clock	n/a	LRU approximation with a reference bit and a sweeping hand

Socket calls and what they trigger

Side	Call	Effect
Server	`socket`	create descriptor
Server	`bind`	local address and port (`htons`)
Server	`listen(n)`	passive; accept queue of about n; SYN queue behind it
Server	`accept`	returns a new descriptor after the handshake completed
Client	`connect`	sends SYN; returns on SYN-ACK; `ECONNREFUSED` on RST
Both	`send` / `recv`	partial counts; loop; `EINTR`, `EAGAIN`
Both	`close`	FIN; the first closer enters TIME_WAIT (2 × MSL)
UDP	`sendto` / `recvfrom`	one datagram per call; sender address returned

I/O models and the receive path

Model	Cost per wait	Limit
thread per connection	scheduler and memory per thread	thousands
`select`	O(n)	fd below 1024
`poll`	O(n)	none
`epoll`	O(ready)	descriptors, memory

Item	Value
NAPI poll budget	64 per poll; `netdev_budget` 300
`sk_buff` pointers	head, data, tail, end; `skb_pull` strips, `skb_push` prepends
Hooks in order	native XDP, tcpdump tap, netfilter PREROUTING, INPUT, socket
Tuning knobs	`somaxconn`, `tcp_max_syn_backlog`, `netdev_budget`, `ulimit -n`, RSS queues, IRQ affinity
Errors	`EAGAIN` retry later; `EINTR` retry; `ECONNRESET` close; `EPIPE` close and ignore SIGPIPE

Flashcards

Q	A
Mode bit?	Separates user and kernel mode; privileged instructions fault in user mode
`fork` return values?	Child PID in parent, 0 in child, -1 on error
Processes after `fork(); fork();`?	4
`fork() && fork() || fork()`?	5 processes

Zombie?	Exited child not yet waited for
Orphan?	Running child whose parent exited; adopted by PID 1
Thread-private state?	Stack, registers, TLS (`errno`)
Why is a thread switch cheaper?	Same page tables; TLB and caches stay warm
`counter++` is?	Load, add, store: not atomic
Mutex versus binary semaphore?	Owner and priority inheritance versus signalling
Why a `while` around `cond_wait`?	Spurious and stolen wake-ups
Spinlock in an ISR?	Yes; sleeping is impossible there
Producer–consumer wait order?	Semaphore `empty`, then the mutex; never the reverse
Four deadlock conditions?	Mutual exclusion, hold and wait, no pre-emption, circular wait
Cure for lock-order deadlock?	A global order
Safe state?	Some order lets every task finish
RAG cycle, multi-instance?	Necessary, not sufficient
Priority inversion fix?	Priority inheritance (or ceiling)
Starvation fix?	Ageing
SJF weakness?	Needs burst lengths; starves long jobs
RR quantum too small?	Time lost in context switches
Page, frame?	Virtual block, physical block of the same size
TLB?	Cache of recent translations
Page fault kinds?	Minor (no disk), major (disk), invalid (SIGSEGV)
Copy-on-write?	Shared read-only pages, copied on the first write
Bélády's anomaly?	More frames, more faults, in FIFO
`malloc` returns?	Virtual memory; frames arrive on first touch
Internal versus external fragmentation?	Waste inside a block versus unusable free gaps
ISR rules?	Short, no sleeping, spinlock and `GFP_ATOMIC`, defer the rest
Top and bottom half?	Urgent acknowledge versus deferred work
DMA?	Device reads or writes memory without the CPU
Fastest IPC?	Shared memory, with its own synchronisation
`volatile`?	Forces real accesses; not atomic, not ordered across cores
SIGPIPE?	Write to a closed peer; ignore it and handle `EPIPE`
`listen` backlog?	Size of the accept queue
Why `htons`?	Network order is big-endian
`recv` returned less?	Stream, no boundaries; loop and frame
TIME_WAIT side?	The first closer, 2 × MSL
`SO_REUSEADDR`?	Bind despite TIME_WAIT entries
`epoll` versus `poll`?	Kernel interest list; O(ready)
Edge-triggered rules?	Non-blocking, read until `EAGAIN`
Thundering herd cure?	`EPOLLEXCLUSIVE` or `SO_REUSEPORT`
NAPI?	Interrupt, then poll with a budget, then re-enable
`sk_buff`?	Packet descriptor with head, data, tail, end pointers
XDP versus DPDK?	In-driver kernel program versus kernel bypass
`connect` to a closed port?	RST then `ECONNREFUSED`

PART I · PILLAR 8

8

Linux and Network Tools

The engineer's toolbox: seeing the network from a Linux shell, and reading every field of what it prints.

8.1 Why a network engineer lives in Linux	MUST
8.2 Shell survival kit	MUST
8.3 Text processing for log hunting	MUST
8.4 Bash scripting that survives production	SHOULD
8.5 iproute2 and network namespaces	MUST
8.6 Sockets on the wire: ss	MUST
8.7 Packet capture I: tcpdump	MUST
8.8 Packet capture II: Wireshark	SHOULD
8.9 DNS from the command line	MUST
8.10 Path and reachability tools	MUST
8.11 Link layer and NIC health	SHOULD
8.12 Netfilter: iptables and nftables	MUST
8.13 Performance: iperf3, BDP, buffers	SHOULD
8.14 Systems debugging toolkit	SHOULD

8.1 Why a network engineer lives in Linux MUST · ■□□

Almost every network device that is not a laptop runs Linux, or sits beside a Linux machine that does. Arista EOS, Nokia SR Linux, SONiC, Cumulus and Junos Evolved are Linux distributions with a routing stack on top; Cisco NX-OS offers a Bash shell; servers, hypervisors, containers and cloud instances are Linux throughout. Hence the first action in most incidents is to open a shell, and the interview for a network role usually asks the candidate to do so on a whiteboard. This pillar teaches that shell as a measuring instrument for the network.

The key fact is that every tool in this chapter is a *window onto kernel state*. The kernel holds the interface table, the routing table, the ARP cache, the socket table and the packet-filter rules; the tools merely ask for them over a few interfaces: netlink sockets (`ip`, `ss`), `ioctl` calls into the NIC driver (`ethtool`), a packet tap (`tcpdump`), and the `/proc` and `/sys` file systems. Once one knows which window shows which table, a symptom points directly to a command (Figure 8.1).

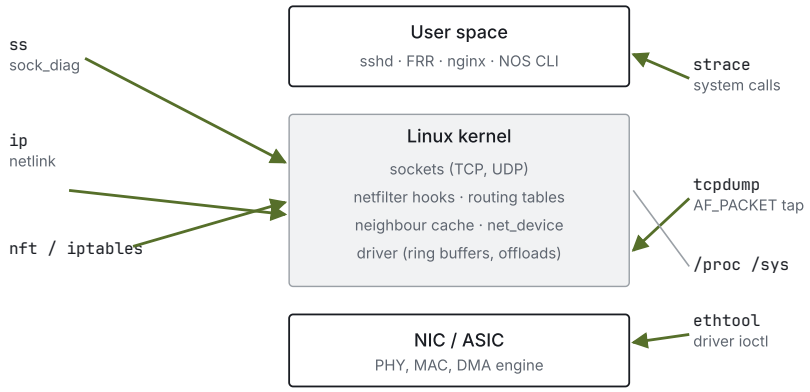


Figure 8.1 Where each tool of this chapter taps into the system. The commands are different faces of one kernel, which is why a Linux host, an Arista switch and a container give the same counters.

Consider how little changes between platforms. On an Arista switch `show interfaces Ethernet1` prints counters from the same `net_device` structure that `ip -s link show et1` prints from the EOS Bash shell, and that `ip -s link show eth0` prints on an Ubuntu server. The legacy tools (`ifconfig`, `route`, `arp`, `netstat`) read the same data through older, slower interfaces and are deprecated; Table 8.1 gives the replacements that interviewers expect.

Legacy (net-tools)	Modern	Notes
`ifconfig`	`ip addr`, `ip -s link`	`ip -br a` gives one line per interface
`route -n`	`ip route`, `ip route get`	`get` shows the route actually chosen
`arp -n`	`ip neigh`	shows NUD states, not only entries
`netstat -tulpn`	`ss -tulpn`	`ss` reads the kernel directly through netlink
`brctl`, `vconfig`	`ip link … type bridge`, `type vlan`	

`iptables`	`nft`	same netfilter hooks, one syntax (8.12 (p. 510))

INTERVIEW LENS

AMAZON · GOOGLE · ARISTA · JUNIPER · CISCO

Interviewers rarely ask for the syntax of one command in isolation. They give a symptom ("the service is unreachable") and watch which command the candidate types first and what the candidate does with the output. A strong answer names the layer being tested before the command, uses the modern tool, and states what a good and a bad result look like. The command ladder of the Revision Spread at the end of this chapter is built for exactly this.

8.2 Shell survival kit MUST · ■□□

Linux treats nearly everything as a *file*: a document, a terminal, a pipe, a device, even a running process (under `/proc`). Programs are small and do one job, and the shell joins them by connecting the output of one to the input of the next. Three streams are open in every process: *standard input* (descriptor 0), *standard output* (1) and *standard error* (2). Descriptors are explained in 7.9 (p. 443) and 7.10 (p. 445); here they matter because redirection simply re-points them.

Operator	Effect	Example
`>` / `>>`	send stdout to a file (truncate / append)	`ip -br a > addr.txt`
`<`	read stdin from a file	`sort < hosts.txt`
`2>`	send stderr to a file	`ping x 2> err.txt`
`2>&1`	make stderr follow stdout	`cmd > all.log 2>&1`
`2>/dev/null`	discard stderr	`find / -name x 2>/dev/null`
`|`	stdout of left becomes stdin of right	`ss -tan | wc -l`
`&&` / `||`	run next only if previous succeeded / failed	`make && ./run`
`;` and `&`	run in sequence / run in background	`sleep 9 &`

Every command returns an *exit status* in `$?`: 0 means success and anything else means failure, which is why `&&` and `||` work. The *pipeline* is the central idea. Consider the standard question "which five clients hit the web server most?", answered by `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -5`. Each stage is a separate process; the kernel connects them with pipes (small in-memory buffers), and a slow reader makes the writer wait, so no stage needs the whole file in memory (Figure 8.2).

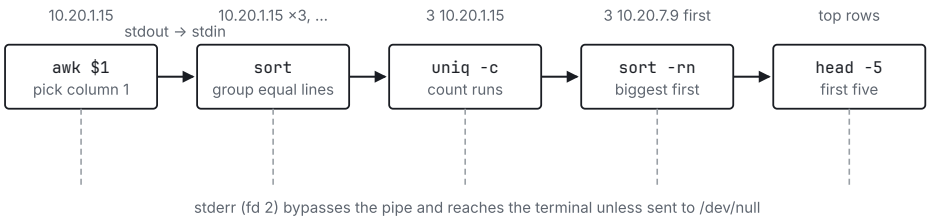


Figure 8.2 A pipeline: five processes, four pipes. Data passes between stages as text lines; error messages travel on the separate stderr stream.

Files and directories. The file system is one tree rooted at `/`. The directories worth knowing are `/etc` (configuration), `/var/log` (logs), `/proc` and `/sys` (kernel state, 8.14 (p. 516)), `/tmp` (scratch space) and `/home`. The commands used daily are few: `ls -lah`, `cd`, `pwd`, `cp`, `mv`, `rm`, `mkdir -p`, `cat`, `less`, `head`, `tail -f` (follow a growing log), `find / -name '*.conf' -mtime -1`, `wc -l`, and `df -h` and `du -sh * | sort -h` for disk space.

Permissions. Each file has an owner, a group and three permission triplets: read (4), write (2) and execute (1) for the *user*, the *group* and *others*. The first character of `ls -l` gives the type (`-` file, `d` directory, `l` link). For a directory, *execute* means "may enter" and *write* means "may create or delete entries". Octal digits add the bit values, so `-rwxr-x---` is 7 5 0 (Figure 8.3), set with `chmod 750 file`.

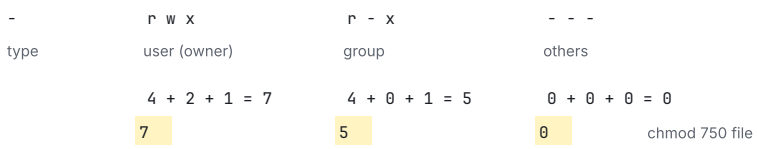


Figure 8.3 The mode string `-rwxr-x---` decoded. Octal 750 means the owner may read, write and execute, the group may read and execute, and everyone else has no access.

Three special bits exist. The *setuid* bit (4000) makes a program run with its owner's identity (`passwd` needs it); *setgid* (2000) does the same for the group and, on a directory, makes new files inherit the group; the *sticky* bit (1000) on a directory allows a user to delete only their own entries, which is why `/tmp` shows as `drwxrwxrwt`. A process that needs privilege runs through `sudo`, which logs the command.

Processes and job control. `ps aux` lists all processes and `top` or `htop` shows them live. A command started with `&` runs in the background; `Ctrl-Z` suspends the foreground job, `bg` resumes it in the background, `fg` brings it back, `jobs` lists them. `kill PID` sends SIGTERM, which the program may handle in order to exit cleanly; `kill -9` sends SIGKILL, which cannot be caught and leaves no chance to clean up, so it is a last resort (7.9 (p. 443)). `nohup cmd &` or a `tmux` session keeps work alive after the terminal closes, which matters over a flaky SSH link.

> **TRAP**
>
> *Claim:* "`kill -9` is the proper way to stop a process." *Why it is wrong:* SIGKILL gives the process no chance to flush buffers, remove lock files or close connections cleanly. *Correct statement:* send SIGTERM first, wait, and use SIGKILL only if the process ignores it.

> **CHECK YOURSELF**
>
> 1. What does `cmd > out.txt 2>&1` do, and why does `cmd 2>&1 > out.txt` differ? [1]
> 2. A file has mode 640. Who can read it? [2]

8.3 Text processing for log hunting MUST · ■■□

Logs are text, one event per line, so a few tools answer most operational questions. *grep* filters lines, *awk* treats each line as columns, *sed* edits streams, and *cut*, *sort*, *uniq* and *wc* slice and count. For structured data from an API, *jq* does the same for JSON. The analogy is a sieve, a spreadsheet and a find-and-replace; the skill is combining them in a pipeline.

The examples use two files from Campus Net: `access.log`, the combined-format log of the student portal's web server, and `syslog.txt`, the switch log of `sw-core1`. Each line of the web log

is split by *awk* on whitespace into numbered fields (Figure 8.4): `$1` is the client, `$4` the timestamp, `$7` the path, `$9` the status and `$10` the bytes; `$NF` is always the last field.

Figure 8.4 A web-log line as awk sees it. Fields are separated by runs of blanks; the quoted request counts as three fields (`$6 $7 $8`) unless the separator is changed with `-F`.

grep selects lines. The options worth memorising are `-E` (extended regular expressions), `-v` (invert), `-c` (count), `-o` (print only the match), `-i`, `-n`, `-r`, and `-A`/`-B`/`-C n` for context lines around a match, which is how one sees what happened *before* an error.

```
$ grep -c 'changed state to down' syslog.txt
5
$ grep -B1 -A1 'Gi1/0/7' syslog.txt            # context around one port
Oct 29 09:00:33 sw-core1 %LINEPROTO-5-UPDOWN: Line protocol on Interface Gi1/0/12,
changed state to up
Oct 29 09:01:02 sw-core1 %LINEPROTO-5-UPDOWN: Line protocol on Interface Gi1/0/7, changed
state to down
Oct 29 09:01:05 sw-core1 %LINEPROTO-5-UPDOWN: Line protocol on Interface Gi1/0/12,
changed state to down
$ grep -oE 'Gi[0-9/]+' syslog.txt | sort | uniq -c
      8 Gi1/0/12
      1 Gi1/0/3
      1 Gi1/0/7
```

awk reads a line, splits it into fields, and runs `pattern { action }` for every line; `BEGIN` and `END` blocks run before and after. Variables need no declaration, arrays are associative, and `NR` is the line number. This is enough for a “group by” query.

```
$ awk '$9 >= 500 {print $1, $7, $9}' access.log        # server errors only
10.20.1.15 /portal/grades 500
10.20.4.31 /portal/grades 502
10.20.4.31 /portal/grades 502
$ awk '{c[$9]++} END {for (s in c) print s, c[s]}' access.log | sort    # status histo-
gram
200 5
302 1
304 1
500 1
502 2
$ awk '{b[$1]+=$10} END {for (i in b) print b[i], i}' access.log | sort -rn | head -2   #
bytes per client
10551 10.20.1.15
2736 10.20.7.9
```

WORKED EXAMPLE

Which switch ports flapped more than three times? The `%LINEPROTO-5-UPDOWN` message is logged each time a port changes state, so a flapping port produces many “down” lines.

1. Keep only the down events: `grep 'LINEPROTO.*to down' syslog.txt`.
2. The port is the tenth field, `Gi1/0/12,`, with a trailing comma, which `sub()` removes: `awk '{sub(/,/,"",$10); print $10}'`.
3. Count per port with an array, and print those over the threshold in the `END` block.

```
$ grep 'LINEPROTO.*to down' syslog.txt \
    | awk '{sub(/,/,"",$10); n[$10]++} END {for (p in n) if (n[p] > 3) print p, n[p]}'
Gi1/0/12 4
```

Result: Gi1/0/12 went down four times in under two minutes; Gi1/0/7 went down once. For the last hour, add a time filter on `$3` before the count.

sed applies edit commands to a stream. The commonest is substitution, `s/old/new/`, with `g` to replace every match on a line; with `-n` and the `p` flag it prints only the lines that matched, and `-i` edits a file in place. **cut** extracts columns by delimiter (`cut -d: -f1 /etc/passwd`), **sort** orders lines (`-n` numeric, `-r` reverse, `-h` human sizes, `-u` unique, `-k2` by column), and **uniq -c** counts *adjacent* repeats, which is why it must follow a `sort`.

```
$ sed -n 's/.*Interface \([^,]*\),.*to \(up\|down\)$/\1 \2/p' syslog.txt | head -3
Gi1/0/12 down
Gi1/0/12 up
Gi1/0/3 up
$ sed 's/10\.20\.\([0-9]*\)\.[0-9]*/10.20.\1.x/g' access.log | head -1      # anonymise
last octet
10.20.1.x - - [29/Oct/2026:09:01:12 +0530] "GET /portal/login HTTP/1.1" 200 5120 "-"
"Mozilla/5.0"
```

jq filters JSON. A dot selects a key, `.[]` iterates an array, `select(...)` filters, `-r` removes quotes from strings, and `@csv` formats a row. Network APIs (controllers, cloud CLIs, `ip -j`) return JSON, so `jq` joins the same pipelines.

```
$ jq -r '.interfaces[] | select(.in_errors>0) | "\(.name) \(.in_errors)"' ifaces.json
Gi1/0/12 412
Gi1/0/3 3
$ ip -j -br a | jq -r '.[] | "\(.ifname) \(.addr_info[0].local)"'     # structured output
from iproute2
```

TRAP

Claim: "`sort | uniq -c` counts the distinct values wherever they appear." *Why it is wrong:* `uniq` compares only neighbouring lines, so unsorted input yields several small counts for the same value. *Correct statement:* always `sort` first (or use an awk array, which needs no ordering). Further, `grep` with an unquoted pattern lets the shell expand `*` and `$` before grep sees them; quote patterns in single quotes.

LAB

`LAB 8.1 · 30 MIN` Log-hunting dojo. Save the two sample logs shown above (or any nginx log and switch syslog) and write one pipeline for each question.

```
1. Which client sent the most bytes?          (awk array, sort -rn)
2. What share of requests failed with 5xx?    (awk END with printf "%.0f%%")
3. Which port flapped, and at what times?     (grep, awk '{print $3}')
4. Which URLs returned 502, with counts?      (awk '$9==502', sort, uniq -c)
```

What you should notice: each answer is a filter (`grep` or an awk pattern), a grouping (array or `sort | uniq -c`) and a ranking (`sort -rn | head`). Once the three stages are familiar, a new question is a change of one stage, not a new program.

CHECK YOURSELF

1. Print the last field of every line of a file. [3]

> 2. Why does `grep -c` differ from `grep ... | wc -l` on a file with long lines? [4]

8.4 Bash scripting that survives production SHOULD · ■■□

A script is a recipe that a junior colleague can follow at three in the morning. It begins with a *she-bang*, `#!/usr/bin/env bash`, which names the interpreter, and runs after `chmod +x`. Bash is the right tool for gluing commands, files and exit codes; the rule of thumb is that a script of more than about fifty lines, or one that needs a real data structure, JSON handling or a network protocol, belongs in Python (Pillar 10 (p. 586)).

Variables and quoting. Assign without spaces (`host=10.0.1.2`) and read with `$host`. The most common production bug is the unquoted expansion: the shell splits an unquoted value on spaces and expands wildcards in it, so `rm -rf $dir/` with an empty `dir` deletes `/`. Always write `"$dir"`. Command output is captured with `$(cmd)`, and arithmetic uses `$(( a + b ))`.

Conditionals and loops. Use `[[ ... ]]` in Bash (it does not word-split and supports patterns and `&&`). Tests: `-f` file exists, `-d` directory, `-r` readable, `-s` non-empty, `-z` empty string, `==` string equality, `-eq -lt -gt` for integers.

```
if [[ -r "$file" && $count -gt 0 ]]; then echo ok; else echo "bad" >&2; fi

case $state in up) echo good ;; down|dead) echo bad ;; *) echo '?' ;; esac

for h in sw1 sw2 sw3; do ping -c1 -W1 "$h" >/dev/null && echo "$h up"; done
while IFS= read -r line; do echo "got: $line"; done < hosts.txt     # reads a file line
by line
for i in {1..5}; do ...; done                                        # brace expansion
```

Functions and arguments. A function takes `$1`, `$2`, … like a script does; `$#` is the count, `"$@"` the list of all arguments with each one preserved as a word, and `${1:-default}` supplies a default. `getopts` parses `-c 3 -w 1` style options. A function returns an exit status (`return 1`) and prints its result for the caller to capture.

Failing loudly. By default Bash carries on after an error, so a script can report success after it broke. The preamble `set -euo pipefail` changes that: `-e` exits when a command fails, `-u` treats an unset variable as an error, and `-o pipefail` makes a pipeline fail if *any* stage fails (without it the pipeline's status is that of the last stage only). Further, `trap 'rm -f "$tmp"' EXIT` runs clean-up on any exit, and every error message goes to stderr (`>&2`) with a non-zero status, so that cron or a monitor can detect it.

Bug	Symptom	Fix
Unquoted `$var`	breaks on spaces; dangerous with `rm`	`"$var"`
`[ $x == y ]` with empty `x`	"unary operator expected"	`[[ $x == y ]]`
`cmd | grep x` hides `cmd` failure	script continues after failure	`set -o pipefail`
`cd dir; rm *`	deletes in the wrong place if `cd` fails	`cd dir && rm *`, or `set -e`
`for f in $(ls)`	splits on spaces	`for f in *` or `find -print0`
`while read` in a pipeline	variables lost (subshell)	redirect: `done < file`
Reading `$?` too late	wrong status	save it: `rc=$?`

WORKED EXAMPLE

`reach.sh`**: report unreachable hosts and fail if any exist.** The script reads a file of hosts, pings each with a timeout, appends one CSV row per host, and exits non-zero if any host was down, so that cron or a monitoring agent can alert on the status alone.

```
#!/usr/bin/env bash
# reach.sh - ping every host in a file; log CSV; exit 1 if any is down.
# usage: reach.sh [-c count] [-w secs] [-o out.csv] hosts.txt
set -euo pipefail

count=2 wait=1 out=reach.csv
usage() { echo "usage: ${0##*/} [-c count] [-w secs] [-o out.csv] hosts.txt" >&2;
exit 2; }

while getopts ":c:w:o:h" opt; do
  case $opt in
    c) count=$OPTARG ;;
    w) wait=$OPTARG ;;
    o) out=$OPTARG ;;
    *) usage ;;
  esac
done
shift $((OPTIND - 1))
[[ $# -eq 1 && -r $1 ]] || usage
hosts=$1

probe() {                       # probe HOST -> prints "up" or "down"
  if ping -c "$count" -W "$wait" -q "$1" >/dev/null 2>&1; then echo up; else echo
down; fi
}

down=0
[[ -s $out ]] || echo "time,host,status" > "$out"
while IFS= read -r host || [[ -n $host ]]; do
  [[ -z $host || $host == \#* ]] && continue      # skip blanks and comments
  state=$(probe "$host")
  printf '%s,%s,%s\n' "$(date -Is)" "$host" "$state" >> "$out"
  if [[ $state == down ]]; then down=$((down + 1)); echo "DOWN  $host" >&2; fi
done < "$hosts"

echo "checked $(grep -cv '^\s*\(#\|$\)' "$hosts") hosts, $down down"
(( down == 0 ))                 # exit status: 0 only if nothing was down
```

```
$ cat hosts.txt
# campus
10.0.1.2
127.0.0.1
192.0.2.77
10.20.99.1
$ ./reach.sh -c1 -w1 hosts.txt; echo "exit=$?"
DOWN  192.0.2.77
DOWN  10.20.99.1
checked 4 hosts, 2 down
exit=1
$ cut -d, -f2,3 reach.csv | tail -4
10.0.1.2,up
127.0.0.1,up
192.0.2.77,down
10.20.99.1,down
```

Result: the summary line goes to stdout, the failures to stderr, and the exit status carries the verdict. The `|| [[ -n $host ]]` clause keeps a final line that lacks a newline.

For a whole subnet the sweep is parallel instead of sequential. Pinging 254 addresses one at a time with a one-second timeout takes over four minutes when most addresses are empty; 32 at a

time takes under ten seconds.

```
#!/usr/bin/env bash
# sweep.sh 10.20.1 - ping A.B.C.1-254, 32 at a time, print the live hosts
set -euo pipefail
prefix=${1:?usage: sweep.sh A.B.C}
seq 1 254 | xargs -P 32 -I{} sh -c \
  'ping -c1 -W1 -q '"$prefix"'.{} >/dev/null 2>&1 && echo '"$prefix"'.{} || true' | sort
-V
```

```
$ ./sweep.sh 10.0.1
10.0.1.1
10.0.1.2
```

Note the `|| true`. Without it `xargs` returns 123 whenever one of the commands fails, and under `set -e` every dead address would abort the script; this fault was found while testing the version above. The second tool is a log summary, which reuses the awk idioms of 8.3 (p. 486):

```
#!/usr/bin/env bash
# logsum.sh - summarise an nginx/Apache combined-format access log.
set -euo pipefail
top=5
while getopts ":n:" o; do [[ $o == n ]] && top=$OPTARG || { echo "usage: ${0##*/} [-n
top] FILE" >&2; exit 2; }; done
shift $((OPTIND - 1))
log=${1:?usage: ${0##*/} [-n top] FILE}
[[ -r $log ]] || { echo "cannot read $log" >&2; exit 2; }

echo "== requests: $(wc -l < "$log")"
echo "== status classes"
awk '{c[substr($9,1,1) "xx"]++} END {for (k in c) printf "%s %d\n", k, c[k]}' "$log" |
sort
echo "== top $top clients"
awk '{print $1}' "$log" | sort | uniq -c | sort -rn | head -n "$top"
echo "== paths with 5xx"
awk '$9 ~ /^5/ {print $9, $7}' "$log" | sort | uniq -c | sort -rn | head -n "$top"
```

```
$ ./logsum.sh -n 3 access.log
== requests: 10
== status classes
2xx 5
3xx 2
5xx 3
== top 3 clients
      3 10.20.7.9
      3 10.20.1.15
      2 10.20.4.31
== paths with 5xx
      2 502 /portal/grades
      1 500 /portal/grades
```

Scheduling. `cron` runs a command at fixed times. A line `*/5 * * * * /opt/netops/reach.sh /opt/netops/hosts.txt || logger -t reach "hosts down"` runs the check every five minutes and writes to the system log if it fails; note the five time fields (minute, hour, day of month, month, day of week). Cron has a minimal `PATH`, so scripts use absolute paths. The modern alternative is a systemd timer (8.14 (p. 516)).

> INTERVIEW LENS
>
> AMAZON NDE · ARISTA · JUNIPER · GOOGLE

> The usual live task is “read hostnames from a file and report which do not respond”. A strong solution quotes every expansion, sets `-euo pipefail`, handles blank lines and comments, takes the file as an argument rather than hard-coding it, prints failures to stderr and returns a non-zero status. Follow-ups: how to make it parallel (`xargs -P`), how to avoid a flood of simultaneous pings, and when to switch to Python (data structures, JSON, retries). The solved version is Q8.8 in the Problem Set.

CHECK YOURSELF

1. What does `set -o pipefail` change? [5]
2. Why write `"$@"` rather than `$@`? [6]

8.5 iproute2 and network namespaces MUST · ■■□

iproute2 is the single tool, `ip`, for layer-2 and layer-3 state on Linux. It talks to the kernel over *netlink* sockets and has one object per table: `ip link` (interfaces), `ip addr` (addresses), `ip route` (routing table), `ip neigh` (ARP and neighbour discovery cache), `ip rule` (policy routing), `ip netns` (network namespaces). Each object takes the verbs `show`, `add`, `del`, `set`; options `-br` (brief), `-s` (statistics), `-j` (JSON) and `-n NS` (act inside a namespace) go before the object. The protocols behind these tables are taught in 3.6 (p. 183) and 4.4 (p. 243); this section teaches how to read and change them.

```
$ ip -br a                                # one line per interface: name, state, addresses
lo               UNKNOWN        127.0.0.1/8
r-eth0@if7       UP             10.0.1.2/24
$ ip route
default via 10.0.1.1 dev r-eth0
10.0.1.0/24 dev r-eth0 proto kernel scope link src 10.0.1.2
$ ip route get 10.0.2.2                   # what the kernel WOULD do for this destination
10.0.2.2 via 10.0.1.1 dev r-eth0 src 10.0.1.2 uid 0
```

`ip route show` lists the table, while `ip route get` asks the kernel to perform the lookup and reports the chosen next hop, output interface and *source address*. The difference matters because the table may hold several overlapping prefixes, policy rules and a default route; the lookup applies longest-prefix match (1.9 (p. 41)) and returns the one that wins. The `proto kernel` line was added automatically when the address was configured, and `scope link` marks a directly connected network.

Interface counters. `ip -s link show eth0` prints receive and transmit counters (bytes, packets, errors, drops, overruns). Counters are cumulative since the interface came up, so one reads the *difference* between two snapshots (8.11 (p. 509)). The flags in angle brackets matter: `UP` is the administrative state, `LOWER_UP` means the carrier is present, and a port that is `UP` without `LOWER_UP` has no cable or no peer.

```
$ ip -s link show r-eth0
8: r-eth0@if7: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
    link/ether ee:da:da:f1:92:5d brd ff:ff:ff:ff:ff:ff link-netns rtr
    RX:  bytes packets errors dropped  missed   mcast
           238       3      0       0       0       0
    TX:  bytes packets errors dropped carrier collsns
           434       5      0       0       0       0
```

The neighbour table. Before sending to a next hop the host needs its MAC address, which it learns by ARP (IPv4) or Neighbour Discovery (IPv6) and keeps in the *neighbour cache*. Each

1. The first sends stdout to the file and then points stderr at the same place; the second points stderr at the old stdout (the terminal) before stdout moves, so errors still reach the screen.↩
2. The owner (read and write) and the group (read); others have no access.↩
3. `awk '{print $NF}' file`.↩
4. It does not: both count matching lines. They differ only for `grep -o`, where `-c` still counts lines, not matches.↩
5. A pipeline returns a failure status if any command in it fails, not only the last.↩
6. `"$@"` passes each argument as one word, preserving embedded spaces; unquoted `$@` splits them again.↩

entry has a state from the Neighbour Unreachability Detection (NUD) machine, and the state is the diagnostic.

State	Meaning	What it tells you
`INCOMPLETE`	ARP request sent, no reply yet	resolution in progress; a long stay means no answer
`REACHABLE`	confirmed recently (about 30 s)	healthy
`STALE`	known but not confirmed lately	normal; used without delay, re-checked on next use
`DELAY`	stale entry just used; waiting 5 s for upper-layer proof	transient
`PROBE`	unicast ARP probes being sent	transient; reachability being tested
`FAILED`	probes unanswered; entry given up	L2 fault: host down, wrong VLAN or subnet, cable
`PERMANENT`	set by hand (`ip neigh add … nud permanent`)	static ARP

```
$ ip neigh flush all; ping -c1 -W1 10.0.1.77       # an address nobody owns
$ ip neigh
10.0.1.77 dev r-eth0 INCOMPLETE                    # no lladdr yet: nobody answered the
ARP
$ ping -c1 10.0.1.1 >/dev/null; sleep 40; ip neigh
10.0.1.1 dev r-eth0 lladdr 46:4f:48:75:38:92 STALE # confirmed once, aged out after ~30 s
```

Hence “Destination Host Unreachable” printed by `ping` *from the local address* is not an ICMP message from a router at all: the host itself gave up because ARP failed for an on-link destination. If the same words arrive *from a router’s address*, the router had no route.

Policy routing. A packet normally consults one table, `main`. `ip rule` adds rules that choose a table by source address, incoming interface, mark or other fields, evaluated in priority order. To send traffic from 10.1.0.0/16 through ISP-B and everything else through ISP-A, one adds a second table with its own default route and a rule that selects it:

```
$ ip route add default via 192.0.2.1 dev p0                        # ISP-A, main table
$ ip route add default via 198.51.100.1 dev p1 table 100           # ISP-B, table 100
$ ip rule add from 10.1.0.0/16 lookup 100 priority 1000
$ ip rule show
0:      from all lookup local
1000:   from 10.1.0.0/16 lookup 100
32766:  from all lookup main
$ ip route get 8.8.8.8
8.8.8.8 via 192.0.2.1 dev p0 src 192.0.2.2
$ ip route get 8.8.8.8 from 10.1.5.5
8.8.8.8 from 10.1.5.5 via 198.51.100.1 dev p1 table 100
```

The rules are checked top to bottom, `local` first (addresses of the host itself), then ours, then `main`. The `ip route get … from` form is the quickest test of a policy. Persisting it across reboots belongs to the network manager (netplan, NetworkManager or `systemd-networkd`), not to the shell.

Other link types. The same `ip link add` creates virtual interfaces. A VLAN sub-interface carries one 802.1Q tag (4.2 (p. 233)): `ip link add link eth0 name eth0.10 type vlan id`

`eth0 master br0`. A VRF (`type vrf table 10`) gives a separate routing table per tenant (4.9 (p. 268)), and a `veth` pair is a virtual patch cable whose two ends may live in different namespaces.

Network namespaces and veth

A *network namespace* is an independent copy of the network stack: its own interfaces, addresses, routing table, neighbour cache, firewall rules and `/proc/net`. A process in a namespace sees only those. Containers are built from this idea, and so is every lab in this book. A `veth` pair behaves like a cable with a plug at each end; whatever enters one end leaves the other, so putting the ends in two namespaces joins them. The lab below builds the topology of Figure 8.5, and Campus Net's student VLAN and server VLAN are the two end networks.

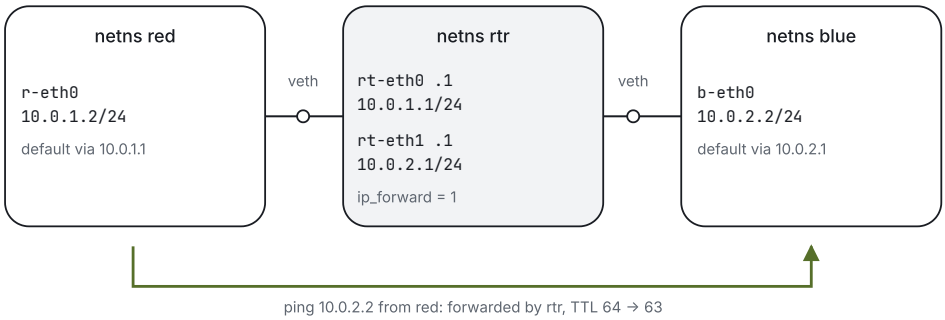


Figure 8.5 The three-namespace lab. Each rounded box has private interfaces and routing tables; the two veth pairs are the cables. Without forwarding enabled in `rtr` the middle box drops transit packets.

LAB

`LAB 8.2 · 40 MIN` Two end hosts and a router, built with namespaces (run as root; nothing needs a hypervisor).

```
ip netns add red; ip netns add blue; ip netns add rtr
ip link add r-eth0 type veth peer name rt-eth0       # cable 1
ip link add b-eth0 type veth peer name rt-eth1       # cable 2
ip link set r-eth0 netns red;  ip link set rt-eth0 netns rtr
ip link set b-eth0 netns blue; ip link set rt-eth1 netns rtr
ip -n red  addr add 10.0.1.2/24 dev r-eth0; ip -n red  link set r-eth0 up
ip -n blue addr add 10.0.2.2/24 dev b-eth0; ip -n blue link set b-eth0 up
ip -n rtr  addr add 10.0.1.1/24 dev rt-eth0; ip -n rtr addr add 10.0.2.1/24 dev rt-
eth1
ip -n rtr  link set rt-eth0 up;  ip -n rtr link set rt-eth1 up
ip -n red  route add default via 10.0.1.1
ip -n blue route add default via 10.0.2.1
```

Now test, then break and repair things one at a time.

```
$ ip netns exec rtr sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 0
$ ip netns exec red ping -c2 -W1 10.0.2.2
2 packets transmitted, 0 received, 100% packet loss
$ ip netns exec rtr sysctl -w net.ipv4.ip_forward=1
$ ip netns exec red ping -c2 -W1 10.0.2.2
64 bytes from 10.0.2.2: icmp_seq=1 ttl=63 time=0.556 ms
```

Breakages to try: (i) delete the default route in `red`; (ii) remove the address from `rt-eth1`; (iii) `ip -n red neigh add 10.0.1.1 lladdr 02:00:00:00:00:99 dev r-eth0 nud`

> `anent` (a wrong static ARP entry); (iv) re-disable forwarding. For each, write down the symptom of `ping`, `ip route get 10.0.2.2` and `ip neigh`.
>
> **What you should notice:** with forwarding off, `red` reaches the router (10.0.1.1 answers) but not `blue`, so the fault is in the router and not in `red`. TTL 63 in the reply confirms exactly one router hop. A wrong static ARP entry makes ping fail although every route is correct; `ip neigh` exposes it.

Containers. Docker and Kubernetes use the same machinery. Each container gets its own namespace, one end of a veth pair is moved inside, the other end is attached to a host bridge (`docker0`, itself an `ip link … type bridge`), and an iptables MASQUERADE rule on the host rewrites container source addresses on the way out (8.12 (p. 510)). Published ports (`-p 8080:80`) are DNAT rules in the same tables. Hence a container's "network problem" is diagnosed with `nsenter -t PID -n ip a`, `ip route`, `ss` and `tcpdump` on the veth, exactly as above.

> TRAP
>
> *Claim:* "`ip addr add` makes the address permanent." *Why it is wrong:* the command changes the running kernel only; a reboot, a link flap handled by a network manager, or a DHCP renewal discards it. *Correct statement:* test with `ip`, then write the setting in netplan, NetworkManager or `systemd-networkd`. The same applies to routes and to `sysctl -w` (8.13 (p. 514)).

> CHECK YOURSELF
>
> 1. A host's `ip neigh` shows the gateway as `FAILED`. Which layer is suspect? [1]
> 2. What does `ip route get 8.8.8.8` print that `ip route` does not? [2]

8.6 Sockets on the wire: ss MUST · ■■□

`ss` (socket statistics) lists the kernel's socket table through the `sock_diag` netlink interface; it replaced `netstat`, which parsed `/proc/net/tcp` and became slow with many connections. A socket is the endpoint of a conversation (7.10 (p. 445)); `ss` shows its state, queues, addresses and, with options, the process that owns it and the internals of its TCP connection. The TCP state machine is drawn in 3.9 (p. 192); here each state is a row in a table.

Option	Meaning
`-t -u -x`	TCP, UDP, Unix sockets
`-l` / `-a`	listening only / all states
`-n`	numeric ports and addresses; without it every line causes a DNS or services lookup
`-p`	owning process (needs root for other users' sockets)
`-i`	TCP internals: rtt, cwnd, retransmissions
`-e -m`	extended info; socket memory
`-s`	summary counts by state
`state X` / `dport = :443`	filter expressions

The first question on any host is *who is listening, on which address*. The mnemonic is `ss -tulpn` (TCP, UDP, listening, process, numeric).

```
$ ss -tlnp
State  Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 0      5           10.0.2.2:8080     0.0.0.0:*     users:
(("python3",pid=21426,fd=3))
```

The local address is the diagnosis. `10.0.2.2:8080` accepts connections only to that address; `127.0.0.1:5000` accepts only local clients, so a Flask or Node server started in development mode "works on my machine and nowhere else" until it binds `0.0.0.0` (all IPv4 addresses) or `[::]`. For a listening socket `Recv-Q` is the current accept queue length and `Send-Q` its maximum (the `backlog`); a `Recv-Q` equal to `Send-Q` means the application is not calling `accept()` fast enough and new connections are being dropped. For an established socket `Recv-Q` is unread data and `Send-Q` is unacknowledged data.

States as signals. Each connection moves through the TCP states (3.9 (p. 192)). Counting them with `ss -tan | awk 'NR>1{c[$1]++} END{for(s in c) print s,c[s]}'` summarises a host's health.

State	Normal?	Signals
`LISTEN`	yes	server waiting; check bind address
`SYN-SENT`	brief	client waited for SYN-ACK; many for long means filtered port or dead peer
`SYN-RECV`	brief	half-open; a pile suggests SYN flood or a full accept queue
`ESTAB`	yes	data phase
`FIN-WAIT-2`	brief	we closed, waiting for the peer's FIN; the peer has not closed yet
`TIME-WAIT`	yes	on the side that closed first; lasts 60 s on Linux (2 × MSL), protecting against old segments
`CLOSE-WAIT`	**no**	peer closed, our application has not called `close()`: a program bug
`LAST-ACK`	brief	we sent our FIN after CLOSE-WAIT, waiting for the final ACK

The two halves of a close show clearly in one experiment on a single host: the server side closes first, so it enters `FIN-WAIT-2` and then `TIME-WAIT`, while the other end stays in `CLOSE-WAIT` until its program closes.

```
$ ss -tan | grep :9                                  # server closed its end first
LISTEN     0      1          10.0.1.2:9          0.0.0.0:*
CLOSE-WAIT 1      0          10.0.1.2:43508     10.0.1.2:9           <- client has not
called close()
FIN-WAIT-2 0      0          10.0.1.2:9         10.0.1.2:43508       <- server waits for
client's FIN
$ ss -tan | grep :9                                  # after the client's close()
TIME-WAIT  0      0          10.0.1.2:9         10.0.1.2:43508       <- the first closer
waits 60 s
```

`CLOSE-WAIT` entries never disappear by themselves, because the kernel is waiting for the application; thousands of them mean a descriptor leak (a missing `close()` on an error path), and the fix is in the code, not in the network. `TIME-WAIT` is not a leak: it is the price of reliable closing, paid by whichever side closes first. A busy HTTP/1.0 server, which closes after every response, therefore shows many `TIME-WAIT` entries on the *server*, and a client making many short connections shows them on the *client*, where they can exhaust ephemeral ports.

Filtering. `ss -tan state established '( dport = :443 or sport = :443 )'`, `ss -tn dst 10.0.2.0/24`, and `ss -tan state time-wait | wc -l` select by state, port or address. The summary `ss -s` prints totals, which is a quick check for a connection explosion.

TCP internals. With `-i`, `ss` prints the kernel's own measurements for each connection (3.9 (p. 192) explains what they control).

```
$ ss -tin dst 10.0.2.2
State      Recv-Q Send-Q Local Address:Port  Peer Address:Port
CLOSE-WAIT 11118  0           10.0.1.2:48232     10.0.2.2:8080
     bbr wscale:10,10 rto:204 rtt:0.066/0.036 ato:40 mss:1448 pmtu:1500 cwnd:11
     bytes_sent:18 bytes_received:11118 segs_out:7 segs_in:12 send 1930666667bps
     pacing_rate 4534978280bps delivery_rate 482666664bps app_limited minrtt:0.024
```

Field	Reading
`bbr` / `cubic`	congestion-control algorithm in use
`rtt:0.066/0.036`	smoothed RTT / mean deviation, in milliseconds
`rto:204`	retransmission timeout, ms (minimum 200 ms on Linux)
`mss`, `pmtu`	segment size negotiated; path MTU known
`cwnd:11`	congestion window in segments; with `mss` gives bytes in flight allowed
`retrans:0/3` (when present)	retransmits in flight / total; rising total means loss
`send`, `delivery_rate`	window-limited rate / measured goodput
`app_limited`	the application, not the network, is the limit
`rwnd_limited`, `snd_wnd`	the receiver's window is the limit (zero-window problems)

Reading it is a short argument. High `rtt` with small `cwnd` and repeated `retrans` points to loss; small `rtt`, large `cwnd` and `app_limited` points to a slow application; `rwnd_limited` points to the receiver. The same fields are drawn as a Stevens graph in Wireshark (8.8 (p. 502)).

> LAB
>
> `LAB 8.3 · 30 MIN` The socket safari. Write a short Python server and client (or reuse the echo programs of 7.10 (p. 445)), and make `ss` show each of the states LISTEN, ESTAB, FIN-WAIT-2, CLOSE-WAIT and TIME-WAIT.
>
> ```
> $ python3 -m http.server 8080 --bind 127.0.0.1 &
> $ ss -tlnp | grep 8080                # LISTEN, and the bind address
> $ curl -s localhost:8080 >/dev/null; ss -tan state time-wait    # HTTP/1.0 server
> closes first
> ```
>
> Then open a connection from Python, read the reply but do not call `close()`, and run `ss -tan state close-wait`.

> **What you should notice:** which side is in TIME-WAIT depends only on who closed first, and CLOSE-WAIT appears on the side that has *received* a FIN but not yet closed. Binding to `127.0.0.1` makes the same server unreachable from another host with no change of firewall.

> CHECK YOURSELF
>
> 1. `ss -tlnp` shows `127.0.0.1:5000` for a web app. Can a remote client connect? [3]
> 2. Which state accumulates when an application forgets `close()`? [4]

8.7 Packet capture I: tcpdump MUST · ■■□

`tcpdump` copies packets from the kernel and prints one line per packet. It is the ground truth of a network investigation: logs and counters say what a program or device believes happened, while the capture shows what crossed the wire. Capture uses a packet tap (`AF_PACKET`) inside the driver path. On *ingress* the copy is taken before netfilter, so tcpdump sees packets that the firewall then drops; on *egress* the copy is taken after the firewall has accepted them (8.12 (p. 510), Figure 8.7). The analogy is a camera at the gate of the housing society: it records everyone who walks up, including those the guard turns away.

The commonest invocation is `tcpdump -i eth0 -nn -s 0 -w file.pcap 'filter'`.

Option	Effect
`-i eth0` / `-i any`	interface; `any` captures all, but with a pseudo link header (no real MAC addresses)
`-nn`	no name or service resolution: faster and unambiguous
`-s 0`	capture whole packets (the default is 262144 bytes in current versions; older ones cut at 96)
`-w f.pcap` / `-r f.pcap`	write raw packets / read them back
`-c N`	stop after N packets
`-e`	print link-layer headers (MAC addresses, VLAN tag)
`-v` `-vv`	more decoding (TTL, IP id, checksums)
`-S`	absolute sequence numbers (default is relative)
`-X` / `-A`	hex-and-ASCII / ASCII payload
`-G secs -W n`	rotate output files: a ring buffer for long captures
`-ttt`	time since the previous packet, which makes delays visible

Filters. The expression is compiled to *BPF* (Berkeley Packet Filter) bytecode that runs inside the kernel, so unwanted packets are never copied to user space. This is what makes capture on a busy link practical; `tcpdump -d 'expr'` prints the compiled program. The grammar is built from *primitives* joined by `and`, `or`, `not`, with parentheses quoted against the shell.

Primitive	Matches
`host 10.0.1.2`, `src host`, `dst host`	IPv4/IPv6 address, either or one direction
`net 10.20.0.0/16`	any address in the prefix

1. Layer 2: ARP received no reply, so look at the cable, VLAN, subnet mask or whether the gateway is up.↩
2. The route actually selected after policy rules and longest-prefix match, with the output interface and the source address the kernel would use.↩
3. No. The socket accepts only connections addressed to the loopback address; bind `0.0.0.0` or the interface address.↩
4. `CLOSE-WAIT`, on the side that received the peer's FIN.↩

`port 443`, `src port`, `dst port`, `portrange 8000-8100`	TCP or UDP port
`tcp`, `udp`, `icmp`, `arp`, `ip6`	protocol
`ether host aa:bb:cc:dd:ee:ff`	MAC address
`vlan 10`	frames tagged with VLAN 10
`greater 1400`, `less 64`	frame length
`tcp[tcpflags] & (tcp-syn|tcp-ack) == tcp-syn`	SYN set, ACK clear: a connection attempt, not the reply
`tcp[tcpflags] & (tcp-rst) != 0`	resets
`icmp[icmptype] == icmp-echo`	pings only
`ip[8] < 5`	TTL below 5

The expression `tcp[tcpflags]` reads the flag byte of the TCP header (byte 13). The mask-and-compare form is needed because several flags can be set in one packet: `& (tcp-syn|tcp-ack) == tcp-syn` keeps a SYN with ACK clear, a connection attempt, and rejects the SYN-ACK reply, whereas `& tcp-syn != 0` would match both.

```
$ tcpdump -i eth0 -nn 'tcp dst port 443 and tcp[tcpflags] & (tcp-syn|tcp-ack) == tcp-syn'
# new connections to HTTPS
$ tcpdump -i any -nn 'not port 22 and host 10.20.1.15' -w /tmp/h.pcap
# one host, without our own SSH
$ tcpdump -nn -r /tmp/h.pcap 'tcp[tcpflags] & tcp-rst != 0'
# filter a saved file
```

Quote the whole expression; otherwise the shell interprets parentheses, `&` and `|`. To capture on a busy interface for hours without filling the disk, use `-G 300 -W 12 -w 'cap-%H%M.pcap'`, which keeps the latest hour.

Reading a line

A TCP line has the same shape in every capture (Figure 8.6). The flags field is the most useful: `[S]` is a SYN, `[S.]` a SYN-ACK (the dot stands for ACK), `[.]` a bare ACK, `[P.]` data with PSH and ACK, `[F.]` a FIN, `[R]` or `[R.]` a reset.

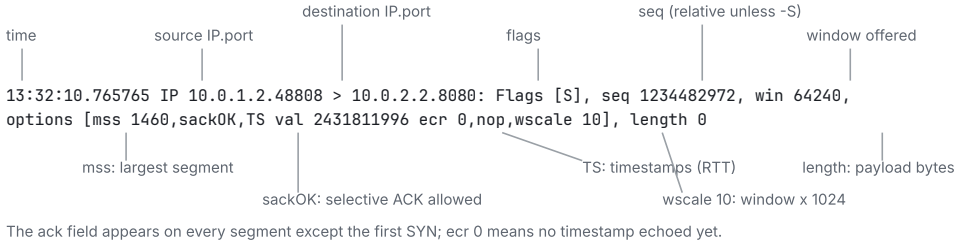


Figure 8.6 Anatomy of a tcpdump line for a SYN. Everything after the flags is TCP detail: the options carried only on SYNs fix the segment size, selective acknowledgements, timestamps and window scaling for the whole connection.

WORKED EXAMPLE

Reading a complete HTTP request. The capture below is `curl http://10.0.2.2:8080/` taken on the router between the two end hosts, with `-ttt` (delta times) and repeated host names shortened to `C` (client, 10.0.1.2) and `S` (server, 10.0.2.2:8080). Sequence numbers are relative.

```
 delta      direction  flags    sequence / ack            note
 0.000000   C > S  Flags [S],   seq 0, win 64240, [mss 1460,sackOK,TS,nop,wscale 10]
1  SYN
 0.000041   S > C  Flags [S.],  seq 0, ack 1, win 65160, [mss 1460,sackOK,TS,wscale
10]  2  SYN-ACK
 0.000015   C > S  Flags [.],   ack 1, win 63
3  ACK: connected
 0.000083   C > S  Flags [P.],  seq 1:77, ack 1, length 76: HTTP: GET / HTTP/1.1
4  request
 0.000008   S > C  Flags [.],   ack 77
5  request received
 0.002830   S > C  Flags [P.],  seq 1:158, ack 77, length 157: HTTP/1.0 200 OK
6  response headers
 0.000035   C > S  Flags [.],   ack 158
7
 0.000048   S > C  Flags [P.],  seq 158:7033, ack 77, length 6875
8  body
 0.000043   S > C  Flags [F.],  seq 7033, ack 77
10 server closes first
 0.000197   C > S  Flags [F.],  seq 77, ack 7034
11 client closes
 0.000033   S > C  Flags [.],   ack 78
12 final ACK
```

1. Lines 1 to 3 are the three-way handshake (3.9 (p. 192)). The SYN carries the options; `win 64240` is the offered window, and with `wscale 10` it is later read as the value times 1024 (line 3's `win 63` is 63 × 1024 bytes).
2. Line 4 carries 76 bytes of data: relative sequence 1 to 77, so the next byte expected is 77, which the ACK of line 5 confirms.
3. The 2.8 ms gap before line 6 is the server's *think time*, the gap between the request and the first response byte; it is the number to watch when a service is slow.
4. The server (HTTP/1.0) closes first with `[F.]`; the client answers, and the final ACK completes the four-way close. The server, as the first closer, is the side that holds `TIME-WAIT` (8.6 (p. 495)).

Result: a healthy connection shows the SYN, SYN-ACK, ACK within one round-trip time, data acknowledged promptly, and two FINs. Every departure from that is a symptom.

A failing handshake. The most informative failures have two forms. A closed port answers at once with a reset: the client sees `Connection refused` and the capture shows two lines.

```
$ tcpdump -i rt-eth0 -nn 'tcp port 9999'
IP 10.0.1.2.46782 > 10.0.2.2.9999: Flags [S], seq 654362669, win 64240, options [...]
IP 10.0.2.2.9999 > 10.0.1.2.46782: Flags [R.], seq 0, ack 654362670, win 0, length 0
```

A port silently dropped by a firewall (or a dead host) gives no reply at all, and the client retransmits the SYN on a timer until it gives up. With a DROP rule in the path (8.12 (p. 510)) the capture on the *ingress* interface of the router shows the client retrying, while the capture on the egress interface shows nothing.

```
$ tcpdump -i rt-eth0 -nn -tt 'tcp port 8080'       # ingress: packets arrive and are
dropped inside the router
1793280763.217828 IP 10.0.1.2.41010 > 10.0.2.2.8080: Flags [S], seq 1916356639, ...
1793280764.224517 IP 10.0.1.2.41010 > 10.0.2.2.8080: Flags [S], seq 1916356639, ...
(+1.007 s)
1793280765.248314 IP 10.0.1.2.41010 > 10.0.2.2.8080: Flags [S], seq 1916356639, ...
(+1.024 s)
1793280766.272388 IP 10.0.1.2.41010 > 10.0.2.2.8080: Flags [S], seq 1916356639, ...
(+1.024 s)
$ tcpdump -i rt-eth1 -nn 'tcp port 8080'           # egress: no packets leave
0 packets captured
```

Three points. The sequence number is the same on every line, which identifies these as retransmissions of one SYN. The curl timed out after its 8 s limit, with no error message from the network: “packets leave and nothing returns” is the signature of a drop or a dead path, whereas an RST or an ICMP error is a deliberate answer. And the spacing is 1 s, 1 s, 1 s here rather than the doubling 1, 2, 4 s that textbooks give: Linux 6.1 and later keep the first retries at the initial timeout (`net.ipv4.tcp_syn_linear_timeouts`, 4 on this machine) before backing off exponentially. Both patterns mean the same thing, so one reads the lack of an answer, not the exact spacing.

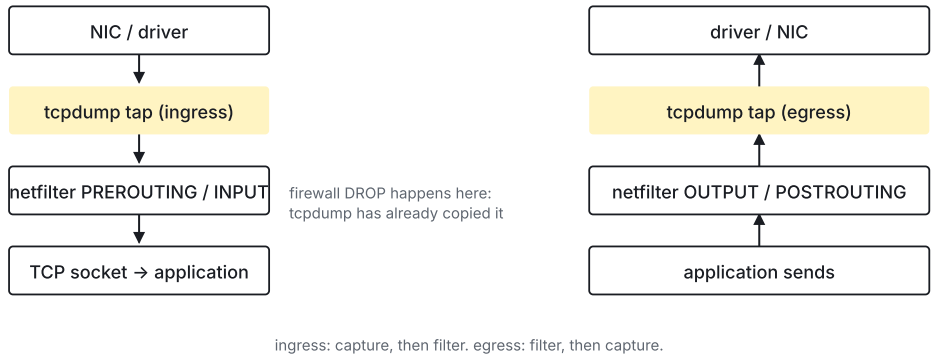


Figure 8.7 Where tcpdump sits relative to netfilter. A packet visible in a capture may still be discarded before the application sees it; a packet absent from an egress capture may have been dropped by the local firewall.

> **TRAP**
>
> *Claim:* “tcpdump shows the packet, so the firewall allowed it.” *Why it is wrong:* on ingress the tap is before netfilter. *Correct statement:* a packet seen on ingress has reached the NIC; whether it reaches the application is shown by `iptables -L -v` counters or the socket, not by tcpdump. Further, a capture on a VM or a router sees only its own interfaces.

Offloads and big packets. Capture on the *sending* host and tcpdump may show segments of 30 000 or 64 000 bytes on a 1500-MTU link. This is not a bug: TCP segmentation offload (TSO) and generic segmentation offload (GSO) let the kernel hand one large buffer to the stack and have the NIC (or the last software stage) cut it into 1448-byte wire segments, and generic receive offload (GRO) merges them on receipt. The capture tap sees the buffer before it is cut (8.11 (p. 509)). Likewise, checksums show as “incorrect” on locally generated packets because the NIC computes them after the tap.

> **LAB**
>
> `LAB 8.4 · 30 MIN` Handshake forensics. In one terminal run `sudo tcpdump -i any -nn -s 0 -w /tmp/h.pcap 'port 8080 or port 9999 or port 53'`; in another, start `python3 -m http.server 8080` and run `curl localhost:8080`, `curl localhost:9999` (refused)

and `dig @127.0.0.53 example.com`. Stop the capture and read it with `tcpdump -nn -r /tmp/h.pcap`.

What you should notice: the three-way handshake lines for 8080; an `[R.]` reply for 9999 within a microsecond; a UDP query and response for port 53 with the DNS transaction id. With `-i any` the packets appear once per interface they cross, and loopback traffic never has a real MAC address.

CHECK YOURSELF

1. Write a filter for packets leaving 10.0.1.2 to TCP port 443 that are not SYNs. [1]
2. In `[P.]`, what do P and the dot mean? [2]

8.8 Packet capture II: Wireshark SHOULD · ■■□

Wireshark decodes the same packets graphically and understands hundreds of protocols; `tshark` is its command-line form. Where tcpdump is the camera, Wireshark is the forensic laboratory. Its distinctive features are three. *Display filters* select what is shown after capture; *Follow TCP stream* reassembles a conversation into readable text; and the *Expert Information* window lists anomalies (retransmissions, resets, zero windows) found by the dissector.

The two kinds of filter use different languages and must not be confused. A *capture filter* is BPF, as in 8.7 (p. 498), and decides what is recorded; a *display filter* is Wireshark's own field language (`protocol.field operator value`) and decides what is shown, with no loss of the underlying capture.

Display filter	Shows
`ip.addr == 10.0.1.2`	traffic to or from the address
`ip.addr == 10.0.1.2 && !(tcp.port == 22)`	the host, excluding SSH
`tcp.flags.syn == 1 && tcp.flags.ack == 0`	connection attempts
`tcp.flags.reset == 1`	resets
`tcp.analysis.retransmission`	retransmitted segments
`tcp.analysis.fast_retransmission || tcp.analysis.duplicate_ack`	loss recovery
`tcp.analysis.zero_window`	receiver buffer full
`tcp.stream == 3`	one TCP conversation
`dns.flags.rcode != 0`	DNS errors
`dns.qry.name contains "campus"`	queries by name
`http.response.code >= 400`	HTTP errors
`http.request.method == "POST"`	form submissions
`tls.handshake.type == 1`	TLS ClientHello (see the server name)
`icmp.type == 3`	destination unreachable
`frame.time_delta > 1`	gaps longer than a second

Analysis tools. *Statistics → Conversations* and *Endpoints* rank hosts by bytes and packets, which finds the top talkers. *I/O Graph* plots throughput or packets per second over time, optionally with a display filter on a second line, so that "bytes" and "retransmissions" can be overlaid. *TCP*

Stream Graphs include the *Stevens* time-sequence graph (sequence number against time): a steady straight line is healthy; flat steps are stalls; a jump back down is a retransmission. *Expert Info* groups findings by severity.

```
$ tshark -r cap.pcap -q -z conv,tcp                          # conversation table
10.0.1.2:45784 <-> 10.0.2.2:8080     6 7555 bytes   6 480 bytes   12 8035 bytes
0.000000   0.0040
$ tshark -r cap.pcap -Y http -T fields -e frame.number -e http.request.method -e
http.response.code -e http.request.uri
4   GET        /
8          200
$ tshark -r cap.pcap -q -z expert                            # anomalies found
   Notes (2)   Sequence  TCP  This frame initiates the connection closing
   Chats (6)   Sequence  TCP  Connection establish request (SYN): server port 8080
```

Reading a slow transfer. The two signatures to separate are loss and receiver limits. Packet loss appears as clusters of *duplicate ACKs* (the receiver repeating the same acknowledgement number because a segment is missing) followed by a *fast retransmission*, and the Stevens graph shows a sawtooth where throughput drops after each loss event. A receiver bottleneck appears as *TCP Window Full* or *Zero Window* flags and a `win` value shrinking to 0 in the receiver's ACKs: the sender is healthy but has been told to stop, and the Stevens graph shows flat stalls without retransmission. The congestion-control reasoning behind these shapes is in 3.9 (p. 192). The fix differs completely: loss means a network path investigation (8.10 (p. 506)); a closing window means the application is slow to read, and the remedy is in the receiving program or its socket buffer (8.13 (p. 514)).

Decrypting TLS. An HTTPS capture shows the TLS handshake in clear (ClientHello with the Server Name, certificates, the negotiated cipher; see Pillar 9 (p. 538)) and the data encrypted. Browsers and curl can write the session secrets to a file named by `SSLKEYLOGFILE`; pointing Wireshark at that file under *Preferences → Protocols → TLS → (Pre)-Master-Secret log filename* decrypts the capture for the owner of the client. It works for a test of one's own application and is a legitimate debugging method; it does not weaken TLS for anyone else, because it needs the secrets from the endpoint itself.

> **TRAP**
>
> *Claim:* "Wireshark's red and black lines mean the network is broken." *Why it is wrong:* the colouring and the Expert Info flag *protocol events*; a few retransmissions or one RST at connection close are common on healthy paths, and a capture taken on the sender can show checksum errors caused by offload (8.7 (p. 498)). *Correct statement:* judge by rate and by pattern: many retransmissions on one stream, or resets at the start of connections, matter.

> **CHECK YOURSELF**
>
> 1. Which filter language does the capture dialog use, and which the filter bar? [3]
> 2. What distinguishes a receiver-limited stall from packet loss? [4]

8.9 DNS from the command line: dig, nslookup, host MUST · ■■□

`dig` (domain information groper) sends a DNS query and prints the whole reply, which makes it the standard instrument for DNS faults; `host` and `nslookup` give shorter answers. The protocol, caching and the resolver hierarchy are explained in 3.10 (p. 203); this section teaches the output.

The examples query an authoritative server for the zone `campus.example`, set up inside the lab namespace at 10.0.2.2.

```
$ dig @10.0.2.2 www.campus.example A
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 45184
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1
;; WARNING: recursion requested but not available

;; QUESTION SECTION:
;www.campus.example.        IN  A

;; ANSWER SECTION:
www.campus.example. 300 IN  A   10.0.2.10

;; Query time: 4 msec
;; SERVER: 10.0.2.2#53(10.0.2.2) (UDP)
;; MSG SIZE  rcvd: 91
```

The header holds the verdict. `status` is the response code: **NOERROR** (the query worked; an empty answer with NOERROR means the name exists but has no record of that type), **NXDOMAIN** (the name does not exist), **SERVFAIL** (the server failed to obtain an answer: a broken upstream, unreachable authoritative servers or DNSSEC validation failure) and **REFUSED** (policy: the server will not answer this client or this zone). The *flags* describe the reply: `qr` (this is a response), `aa` (authoritative answer, from the zone's own server), `rd` (recursion desired, set by the client), `ra` (recursion available, set by a recursive resolver), `ad` (data authenticated by DNSSEC), `tc` (truncated: retry over TCP). The number after the name in the answer, `300`, is the *TTL*, the seconds for which a cache may keep the record; querying a caching resolver twice shows it counting down.

Options that matter.

Command	Purpose
`dig +short www.campus.example`	only the answer data
`dig @10.0.2.2 name`	ask a specific server, bypassing `/etc/resolv.conf`
`dig campus.example MX` / `NS` / `TXT` / `SOA` / `AAAA`	other record types
`dig -x 10.0.2.10`	reverse lookup (PTR)
`dig +noall +answer name`	only the answer section, with TTL
`dig +trace name`	walk from the root: root, TLD, authoritative
`dig +norecurse @server name`	ask without recursion (what a resolver does to others)
`dig +cd name`	disable DNSSEC checking (distinguishes validation failure)
`dig +tcp name`	force TCP
`dig +tries=1 +time=1`	short timeout for scripts

```
$ dig @10.0.2.2 +noall +answer portal.campus.example
portal.campus.example.  300 IN  CNAME   www.campus.example.
www.campus.example. 300 IN  A   10.0.2.10
$ dig @10.0.2.2 nosuch.campus.example | grep -E 'status|SOA'
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 9668
campus.example.     300 IN  SOA ns1.campus.example. noc.campus.example. 2026102901 3600
600 86400 300
$ dig @10.0.2.2 www.other.example | grep status        # a zone this server does not hold
;; ->>HEADER<<- opcode: QUERY, status: REFUSED, id: 48082
$ dig @10.0.2.2 +tries=1 +time=1 www.campus.example     # server stopped
;; no servers could be reached
```

An NXDOMAIN reply carries the zone's SOA record in the *authority* section; its last field (here 300) is the negative-caching TTL, the time for which resolvers remember that the name does not exist. The CNAME in the first example shows the alias followed by the final address. The distinction between "no servers could be reached" (a timeout, with nothing heard) and a status (a server answered) is the first split in any DNS diagnosis.

NXDOMAIN versus SERVFAIL. For NXDOMAIN, check the spelling, the zone for a missing record, and a search-domain suffix appended by the resolver (`ndots`). For SERVFAIL, query the authoritative servers directly (`dig @ns1 name`); if they answer normally, the failure lies in the resolver's path or validation: `dig +cd name` succeeding where `dig name` fails means DNSSEC validation is rejecting the answer (expired signatures or a broken DS record at the parent).

Tracing. `dig +trace` performs the iterative resolution itself instead of asking a recursive resolver: it starts from the root servers, follows the referral to the top-level domain and then to the zone's authoritative servers. The shape of the output (abridged; addresses and signatures omitted) is a sequence of delegations.

```
$ dig +trace www.campus.example
.                   518400 IN NS   a.root-servers.net.  ...        <- the root zone's NS
set (from the hints file)
example.             172800 IN NS   a.nic.example.  ...             <- referral from a
root server to the TLD
campus.example.       86400  IN NS   ns1.campus.example.            <- referral from the
TLD to the zone
www.campus.example.   300    IN A    10.0.2.10                      <- the answer, from
the authoritative server
```

Each stage returns a *referral* (an NS set in the authority section, with glue addresses in the additional section) until a server answers with the `aa` flag. A trace that stops at one stage identifies the broken delegation.

Where the host looks first. A program calls `getaddrinfo()`, and the C library consults the sources listed in `/etc/nsswitch.conf`, normally `hosts: files dns`. `files` means `/etc/hosts`, so a line there overrides DNS. `dns` reads `/etc/resolv.conf`, which on a modern Ubuntu holds only the local stub `nameserver 127.0.0.53` run by `systemd-resolved`; `resolvectl status` shows the real upstream servers and `resolvectl query name` shows what the system resolver, as opposed to `dig`, would answer. Note that `dig` ignores `/etc/hosts` and `nsswitch.conf`: if `ping name` and `dig name` disagree, the cause is likely an `/etc/hosts` entry or a search-domain difference. `host` and `nslookup` are shorter forms:

```
$ host www.campus.example 10.0.2.2
www.campus.example has address 10.0.2.10
$ getent hosts www.campus.example             # what applications see: files, then DNS
```

LAB

> `LAB 8.5 · 30 MIN` DNS detective. Plant three faults and diagnose each with only `dig`, `getent hosts` and `cat`: (i) add `10.9.9.9 www.campus.example` to `/etc/hosts`; (ii) point `/etc/resolv.conf` at an address that does not run DNS; (iii) query a name that has no record.
>
> **What you should notice:** `getent` returns the `/etc/hosts` address while `dig` returns the real one; a dead resolver gives timeouts ("no servers could be reached", about 2 s per try and attempt), not a status; and the missing name gives `NXDOMAIN` with an SOA in the authority section.

> CHECK YOURSELF
>
> 1. `dig` shows `status: NOERROR` and an empty ANSWER section. What does that mean? [5]
> 2. Which two header flags show that the reply came from the zone's own server and that a resolver offered recursion? [6]

8.10 Path and reachability tools MUST · ■■□

A connectivity complaint is answered by a ladder of probes that each test one more layer than the last: ARP, ICMP, a TCP handshake, an application request. The tools below are the rungs. Their protocols (ICMP, TTL, path MTU, ARP) are in 3.5 (p. 179) and 3.6 (p. 183).

ping sends ICMP echo requests and reports the round-trip time and loss. Useful options are `-c N` (count), `-i` (interval), `-W` (reply timeout), `-s` (payload size), and `-M do` (set the Don't-Fragment bit). Replies show `ttl`, which reveals the hop count when the sender's initial value is known (64 on Linux: a reply with ttl 63 crossed one router). `ping` proves that ICMP works on the path, nothing more: a server can answer ping with its web service down, or drop ICMP while serving traffic.

Path MTU is found with DF pings. The payload plus 28 bytes of IP and ICMP header must fit the MTU, so `-s 1472` fills a 1500-byte packet. With a smaller link in the path (the lab set the link towards `blue` to 1400), the router answers with ICMP "fragmentation needed", naming the limit:

```
$ ping -c1 -M do -s 1472 10.0.2.2
From 10.0.1.1 icmp_seq=1 Frag needed and DF set (mtu = 1400)
$ ping -c1 -M do -s 1372 10.0.2.2                # 1372 + 28 = 1400
1380 bytes from 10.0.2.2: icmp_seq=1 ttl=63 time=0.046 ms
$ tracepath -n 10.0.2.2                          # discovers the same limit hop by hop
 1:  10.0.1.1                                             0.067ms
 2:  10.0.1.1                                             0.012ms pmtu 1400
 2:  10.0.2.2                                             0.027ms reached
     Resume: pmtu 1400 hops 2 back 2
```

Ordinary packets are small and work, but large transfers hang when the ICMP message is filtered: the "black hole" symptom, where a login works and a large file stalls. `tracepath` needs no root and reports `pmtu` as it learns it.

traceroute reveals the routers on the path by exploiting the TTL field: it sends probes with TTL 1, 2, 3, …; each router that decrements the TTL to zero discards the probe and returns ICMP *Time Exceeded*, whose source address names that hop (Figure 8.8). The last probe reaches the destination, which answers differently (an ICMP port-unreachable for UDP probes, an echo reply for ICMP probes, a SYN-ACK or RST for TCP probes), ending the trace.

1. `src host 10.0.1.2 and dst port 443 and tcp and not (tcp[tcpflags] & tcp-syn != 0)`.↩
2. P is the PSH flag (deliver data now) and the dot represents the ACK flag.↩
3. The capture dialog takes BPF capture filters; the filter bar takes Wireshark display filters.↩
4. Receiver-limited stalls show zero-window or window-full flags and no retransmissions; loss shows duplicate ACKs and retransmissions.↩
5. The name exists but has no record of the type requested (for example an AAAA query for an IPv4-only host).↩
6. `aa` (authoritative answer) and `ra` (recursion available).↩

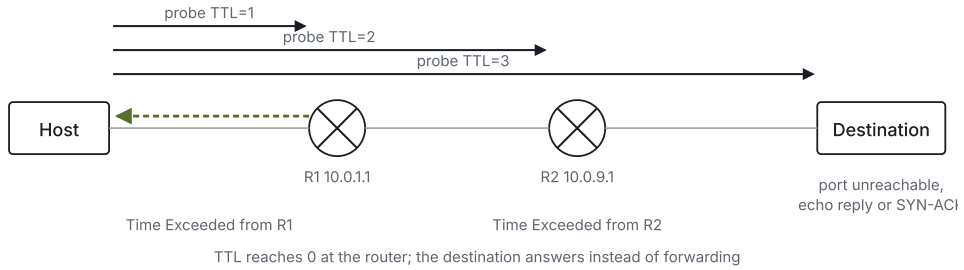


Figure 8.8 Traceroute. Each probe lives one router longer than the last; the source address of each Time Exceeded message names the hop. Reverse paths may differ from forward paths, and each hop may rate-limit its replies.

Linux `traceroute` sends UDP datagrams to high, unlikely ports by default (so that the destination answers port-unreachable), while Windows `tracert` uses ICMP echo; the choice is historical, and either may be filtered. Options switch the probe type: `-I` (ICMP), `-T -p 443` (TCP SYN to port 443, which passes a firewall that allows only HTTPS), `-n` (no reverse DNS, much faster) and `-q 1` (one probe per hop). A line of `* * *` means no reply came within the timeout: the router may be configured not to send Time Exceeded, a firewall may drop the probes or the replies, or the router rate-limits ICMP. It does not mean the router is down, because traffic that is not addressed to the router itself passes through it normally.

```
$ traceroute -n -q1 10.0.2.2
 1  10.0.1.1  0.059 ms
 2  10.0.2.2  0.036 ms
```

mtr combines ping and traceroute: it probes every hop continuously and reports loss and latency statistics per hop, which exposes intermittent problems that one traceroute misses. The report mode is `mtr -rwnc 100 host` (report, wide, numeric, 100 cycles).

```
$ mtr -rwnc 5 10.0.2.2
HOST: vm            Loss%   Snt   Last   Avg  Best  Wrst StDev
  1.|-- 10.0.1.1     0.0%     5    1.1   0.3   0.1   1.1   0.5
  2.|-- 10.0.2.2     0.0%     5    0.1   0.1   0.1   0.2   0.0
```

The rule for reading it is that *loss counts only if it persists to the last hop*. Routers prioritise forwarding over generating ICMP, and many apply control-plane policing (CoPP) to the CPU path that builds Time Exceeded messages. A hop that shows 50 % loss while later hops show none is merely slow to answer *its own* probes; the forwarded traffic is unaffected. Real loss at hop *k* appears at hop *k* and every hop after it. The two reports below (typical shapes, not captured from the lab) show the contrast.

```
False alarm (rate limit)          Real loss from hop 3
 1 10.20.0.1    0.0%               1 10.20.0.1    0.0%
 2 10.30.0.1    0.0%               2 10.30.0.1    0.0%
 3 10.30.4.1   50.0%  <- ignore     3 10.30.4.1   12.0%  <- real
 4 10.40.1.1    0.0%               4 10.40.1.1   11.0%
 5 10.40.9.9    0.0%  <- last hop   5 10.40.9.9   12.0%  <- last hop
```

Two further cautions. Latency that jumps at a hop and stays high thereafter is real queueing or a long link (for example a sea cable); a spike at one hop only is again the router's slow ICMP path. And the forward and return paths can differ, so loss on the return path shows at the hops that carry the replies; `mtr` from both ends settles it.

arping tests layer 2 directly with ARP, bypassing IP routing and any ICMP filter: `arping -I eth0 10.0.1.1` prints the gateway's MAC address if it is alive on the segment. It is also the tool for detecting duplicate addresses (two replies with different MACs).

nc (netcat) tests a TCP or UDP port, and `curl` tests the application. `nc -zv host 443` completes a handshake and exits (`-z` no data, `-v` verbose, `-w 2` timeout); the message distinguishes the failure kinds, as the earlier capture showed.

```
$ nc -zv -w2 10.0.2.2 8080
Connection to 10.0.2.2 8080 port [tcp/http-alt] succeeded!
$ nc -zv -w2 10.0.2.2 9999
nc: connect to 10.0.2.2 port 9999 (tcp) failed: Connection refused      <- RST: host up,
nothing listening
```

"Connection refused" means a reset came back (host reachable, port closed); "timed out" means silence (drop, dead host or path). `curl -v` prints the request and response headers and the connection steps, and `curl -w` prints timings that split a slow request into stages:

```
$ curl -s -o /dev/null -w 'dns=%{time_namelookup} connect=%{time_connect} tls=%
{time_appconnect} start=%{time_starttransfer} total=%{time_total} code=%{http_code}\n'
http://10.0.2.2:8080/
dns=0.000033 connect=0.000235 tls=0.000000 start=0.003609 total=0.004089 code=200
```

The timings are cumulative from the start: DNS finished at 0.03 ms, the TCP handshake at 0.24 ms, the first response byte at 3.6 ms (the server's think time is `start - connect`), the whole body at 4.1 ms. A large `dns` points to the resolver, a large `connect - dns` to the network round trip or a SYN drop, a large `tls - connect` to certificate or handshake costs, a large `start - tls` to the application, and a large `total - start` to the transfer.

> **TRAP**
>
> *Claim:* "Loss shown in the middle of an mtr report means a router is dropping my traffic." *Why it is wrong:* the percentage counts only unanswered probes to that router's own control plane, which is rate limited. *Correct statement:* trust loss only if it continues to the final hop. Likewise `* * *` in traceroute is a missing reply, not a dead router, and `ping` working does not show that TCP port 443 or the application works.

> **INTERVIEW LENS**
>
> CISCO · JUNIPER · ARISTA · AMAZON · GOOGLE
>
> "Explain traceroute", "mtr shows loss at hop 3 but not at the end" and "traceroute through a firewall that only allows 443" are near-universal. A strong answer states the TTL and Time Exceeded mechanism, the UDP, ICMP and TCP probe types and why `-T -p 443` passes a port-443 filter, the rate-limiting rule for intermediate hops, and asymmetric paths. Follow-ups: ECMP making different probes take different paths (Paris traceroute keeps the flow tuple fixed), and MPLS hops that hide or reveal labels. The solved forms are Q8.10 and Q8.11.

> **CHECK YOURSELF**
>
> 1. A DF ping of 1472 bytes fails and 1372 succeeds. What is the path MTU? [1]
> 2. Which message does `nc` print for a closed port, and which for a filtered one? [2]

8.11 Link layer and NIC health: ethtool, ip -s link, nstat

SHOULD · ■□□

A large share of “network” problems are physical: a bad cable, a speed or duplex mismatch, a failing transceiver, a full receive ring. `ethtool` talks to the NIC driver and reports what the hardware negotiated and counted. The analogy is a mechanic’s diagnostic plug: the counters are the odometer, cumulative since the device came up, so one reads a *rate*, not a total.

Command	Reports
`ethtool eth0`	link detected, speed, duplex, auto-negotiation, supported modes
`ethtool -S eth0`	driver counters: `rx_crc_errors`, `rx_missed`, `tx_dropped`, per-queue statistics
`ethtool -i eth0`	driver name, version, firmware, bus address
`ethtool -k eth0`	offload features on or off (TSO, GSO, GRO, checksums)
`ethtool -g eth0`	ring buffer sizes, current and maximum
`ethtool -a eth0`	pause-frame (flow-control) settings
`ethtool -m eth0`	transceiver (SFP) optical power and temperature, if supported
`ip -s link show eth0`	generic RX/TX packets, errors, drops, overruns
`cat /proc/net/dev`	the same counters for every interface, plain text
`nstat -az`	protocol counters: `TcpRetransSegs`, `IpInDiscards`, `TcpExtListenDrops`

```
$ ethtool eth0 | grep -E 'Speed|Duplex|Auto|Link'
    Speed: 1000Mb/s
    Duplex: Full
    Auto-negotiation: on
    Link detected: yes
$ ethtool -g eth0 | sed -n 2,13p
Pre-set maximums:
RX:         256
TX:         256
Current hardware settings:
RX:         256
TX:         256
$ ethtool -k eth0 | grep -E '^(tcp-seg|generic-seg|generic-rec)'
tcp-segmentation-offload: on
generic-segmentation-offload: on
generic-receive-offload: on
```

(The 1000 Mb/s block is the usual output of a physical NIC; the virtual NIC of the book’s lab VM reports `Speed: Unknown!` because it has no medium. The ring and offload lines above are real output from it.) The first line to check on a suspect port is the negotiated speed and duplex: a link that negotiates 100 Mb/s half-duplex on a gigabit port points to a damaged cable or an auto-negotiation mismatch (one side forced, the other automatic: the automatic side falls back to half-duplex), and half-duplex produces *late collisions* and CRC errors under load. In that case the repair is a new cable or both ends set to auto.

Counters need two snapshots. A counter such as `rx_crc_errors 5000` says nothing alone; it may have grown during a fault last month. The method is to read twice and divide the difference by the interval (rate = Δcounter / Δtime).

```
$ ip -s link show eth0 | awk '/RX:/{getline; a=$3} END{print a}'; sleep 10; ip -s link
show eth0 | awk '/RX:/{getline; b=$3} END{print b}'
311919
312190
```

Here 271 packets arrived in 10 s, 27 packets per second. The same two-snapshot approach applied to `rx_crc_errors` or to the `errors` column separates "errors since boot" from "errors now". In the generic counters, `errors` are damaged frames (CRC, length, alignment: physical), `dropped` means the host discarded frames (ring full, no buffer, filter), and `overruns` or `missed` mean the NIC had no free descriptor: the CPU did not drain the ring fast enough. A rising `rx_missed` with a small ring is the case for `ethtool -G eth0 rx 4096` or for spreading interrupts across queues (7.12 (p. 460)).

Offloads and capture. TSO lets the kernel pass a large TCP buffer (up to 64 KB) to the NIC, which cuts it into MSS-sized segments with correct headers and checksums; GSO does the same in software when the NIC cannot; GRO merges consecutive received segments of one flow before the stack sees them, saving per-packet cost. One 64 KB buffer becomes about 45 wire segments of 1448 bytes (65 536 / 1448 ≈ 45). These features explain why a capture taken *on the host* shows super-sized packets and unverified checksums, whereas a capture on a mirror port or another machine shows the real wire frames. For protocol debugging one turns them off temporarily: `ethtool -K eth0 tso off gso off gro off`.

> TRAP
>
> *Claim:* "5,000 CRC errors, so the link is bad." *Why it is wrong:* counters are cumulative since the device started or was cleared. *Correct statement:* take two readings and compare the rate; a link is bad if errors increase while traffic flows, and the cure is confirmed when the rate falls to zero.

> CHECK YOURSELF
>
> 1. A gigabit port negotiated 100 Mb/s half-duplex and shows late collisions. Likely cause? [3]
> 2. Why does tcpdump on a sender show a 64 KB segment on a 1500-MTU link? [4]

8.12 Netfilter: iptables and nftables MUST · ■■□

Netfilter is the packet-filtering framework inside the Linux kernel. It defines *hooks*, fixed points on a packet's path, at which registered rules may accept, drop, modify or redirect it. `iptables` and `nftables` are two user-space front-ends to the same hooks; `ufw` and `firewalld` are friendlier front-ends over them. Connection-tracking state, address translation and filtering all live here, which is why the framework underlies host firewalls, NAT gateways, Docker and Kubernetes networking. The concepts of NAT and ACLs are taught in 3.6 (p. 183) and Pillar 9 (p. 538); this section teaches the Linux mechanism.

A packet entering the host passes **PREROUTING**; the routing decision then decides whether it is for this host (**INPUT**, then the local process) or for forwarding (**FORWARD**). A locally generated packet passes **OUTPUT**. Everything leaving the host passes **POSTROUTING** (Figure 8.9). Hence the answers to the standard questions: a packet *routed through* the box traverses PREROUTING, FORWARD, POSTROUTING; a packet *for* the box traverses PREROUTING and INPUT.

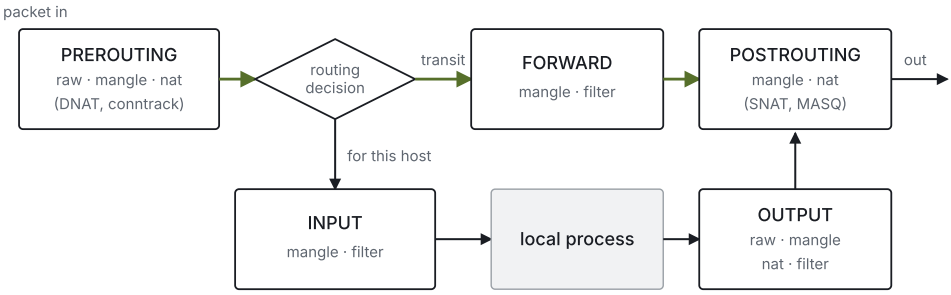


Figure 8.9 The netfilter path, simplified. Boxes are hooks with the tables registered there; the diamond is the routing decision. Transit traffic crosses PREROUTING, FORWARD and POSTROUTING; local traffic enters through INPUT and leaves through OUTPUT and POSTROUTING.

Tables and targets. In iptables, rules live in *tables* by purpose: `filter` (accept or drop: the default), `nat` (address translation), `mangle` (header changes, marks), `raw` (exempt from connection tracking). Each table has some of the *chains* above. A chain is an ordered list of rules; a packet is compared with the rules *in order*, the first match decides, and if none matches the chain's *policy* (ACCEPT or DROP) applies. A rule's target is `ACCEPT`, `DROP` (discard silently), `REJECT` (discard and send an error), `LOG` (write a log line and continue), `DNAT`, `SNAT` and `MASQUERADE` (translation), or a jump to another chain.

Connection tracking. The kernel's *conntrack* module records each flow, and rules can match its state: `NEW` (first packet), `ESTABLISHED` (traffic in both directions seen), `RELATED` (new flow tied to an existing one: an ICMP error, an FTP data channel) and `INVALID`. This allows a *stateful* firewall: accept replies to connections the host started, and judge only new connections. The rule `-m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT` is placed first, for two reasons: nearly all packets match it, so it is also the fastest path, and it permits return traffic without opening any inbound port.

WORKED EXAMPLE

A host firewall for a Campus Net server. Policy: replies are allowed; SSH only from the campus range 10.0.0.0/8; HTTPS from anywhere; everything else dropped, with a rate-limited log.

1. Set the default policy to DROP on INPUT, after making sure that the SSH session will survive: the ESTABLISHED rule comes first.
2. Accept loopback, so that local services can talk to each other.
3. Add the SSH and HTTPS rules, then the logging rule, which sits last so that only unmatched packets are logged.

```
iptables -P INPUT DROP
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -p tcp -s 10.0.0.0/8 --dport 22  -j ACCEPT
iptables -A INPUT -p tcp --dport 443 -j ACCEPT
iptables -A INPUT -m limit --limit 5/min -j LOG --log-prefix "campus-fw drop: "
```

```
$ iptables -L INPUT -n -v --line-numbers          # after one ping and one SSH-port
attempt from 10.0.1.2
Chain INPUT (policy DROP 1 packets, 84 bytes)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     0    --  *      *       0.0.0.0/0            0.0.0.0/0
ctstate RELATED,ESTABLISHED
2        0     0 ACCEPT     0    --  lo     *       0.0.0.0/0            0.0.0.0/0
3        1    60 ACCEPT     6    --  *      *       10.0.0.0/8           0.0.0.0/0
tcp dpt:22
4        0     0 ACCEPT     6    --  *      *       0.0.0.0/0            0.0.0.0/0
tcp dpt:443
5        1    84 LOG        0    --  *      *       0.0.0.0/0            0.0.0.0/0
limit: avg 5/min burst 5 LOG
```

Result: the packet counters prove which rule matched: the SSH SYN (60 bytes) hit rule 3, and the ping (84 bytes) fell through to the LOG rule and then to the DROP policy. If a rule has zero hits when traffic should match it, an earlier rule is taking the packets.

The usual mistakes are all about order. A broad `DROP` placed above a specific `ACCEPT` makes the later rule dead; the counters then stay at zero, which is how one finds it. Rules are inserted at a position with `-I INPUT 3`, deleted with `-D INPUT 3`, and the whole set is dumped or restored with `iptables-save` and `iptables-restore`. A change made at the shell is lost at reboot until saved by the distribution's mechanism (`netfilter-persistent`).

DROP versus REJECT. With `DROP` the sender receives nothing and retransmits until its timeout (the SYN retries seen in 8.7 (p. 498)), so a legitimate client waits a long time for the failure while a scanner is slowed down. With `REJECT` the sender at once receives a TCP RST (or ICMP port-unreachable for UDP) and fails fast, "connection refused". Neither is "more secure": inside a trusted network REJECT gives faster diagnosis and avoids hanging applications; on an Internet-facing interface DROP gives away less to scanners.

NAT. Source NAT rewrites the source of outgoing packets; `MASQUERADE` is SNAT that takes the address of the outgoing interface, the right choice when that address is dynamic. Destination NAT rewrites the destination of incoming packets to expose an internal service. Conntrack remembers the translation, so replies are translated back automatically. The theory of PAT is in 3.6 (p. 183).

```
iptables -t nat -A POSTROUTING -s 10.0.1.0/24 -o rt-eth1 -j MASQUERADE
iptables -t nat -A PREROUTING -i rt-eth0 -p tcp --dport 8081 -j DNAT --to-destination
10.0.2.2:8080
```

```
$ conntrack -L                                  # the translation is stored with both
directions
tcp 6 116 SYN_SENT src=10.0.1.2 dst=10.0.1.1 sport=58188 dport=8081 [UNREPLIED]
src=10.0.2.2 dst=10.0.2.1 sport=8080 dport=58188
```

In that line the first tuple is the original packet (client to the router's port 8081) and the second is the expected reply (server 10.0.2.2:8080 to the translated source). The flow stayed `[UNREPLIED]` in the experiment because the destination host had a firewall that accepted port 8080 only from 10.0.1.0/24, and MASQUERADE had changed the client's apparent address to the router's 10.0.2.1: a NAT gateway changes the source seen by every firewall behind it, so those rules must be written for the translated address. Forwarding also requires `net.ipv4.ip_forward=1` (8.5 (p. 492)).

nftables

nftables is the successor to iptables, with one tool (`nft`), one syntax for IPv4, IPv6 and other families (`inet` covers both), sets and maps that match many addresses in a single rule, and atomic loading of a whole ruleset. A rule set has *tables* containing *chains*, which are attached to a hook with a `type`, `hook` and `priority`. The bastion-host rule set below does what the iptables example did, with the campus range held in a set, and loads in one step.

```
table inet campus_fw {
  set mgmt { type ipv4_addr; flags interval; elements = { 10.0.1.0/24 } }
  chain input {
    type filter hook input priority 0; policy drop;
    ct state established,related accept
    ct state invalid drop
    iif "lo" accept
    ip protocol icmp icmp type { echo-request, destination-unreachable, time-exceeded }
limit rate 10/second accept
    ip saddr @mgmt tcp dport 8080 accept          # management service from the set only
    tcp dport 443 accept
    counter limit rate 5/minute log prefix "campus-fw drop: "
  }
}
```

```
$ nft -f bastion.nft && nft list chain inet campus_fw input | tail -3     # after a probe
to the closed port 9999
        tcp dport 443 accept
        counter packets 3 bytes 180 limit rate 5/minute burst 5 packets log prefix "cam-
pus-fw drop: "
```

The counter before the `limit` counts every packet that reaches the end of the chain (here three SYN retries to port 9999), while the log is limited to five a minute and the policy drops them. The ruleset is shown by `nft list ruleset`, and the iptables-to-nftables translation is mechanical:

Task	iptables	nftables
Allow replies	`-m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT`	`ct state established,related accept`
Allow SSH from a range	`-p tcp -s 10.0.0.0/8 --dport 22 -j ACCEPT`	`ip saddr 10.0.0.0/8 tcp dport 22 accept`
Several ports	`-m multiport --dports 80,443 -j ACCEPT`	`tcp dport { 80, 443 } accept`
Default drop	`iptables -P INPUT DROP`	`policy drop;` in the chain definition
Masquerade	`-t nat -A POSTROUTING -o eth0 -j MASQUERADE`	`oifname "eth0" masquerade` in a `nat` chain
DNAT	`-t nat -A PREROUTING -p tcp --dport 8081 -j DNAT --to 10.0.2.2:8080`	`tcp dport 8081 dnat to 10.0.2.2:8080`
List with counters	`iptables -L -n -v --line-numbers`	`nft list ruleset` (counters if declared)
Save / load	`iptables-save` / `iptables-restore`	`nft list ruleset > f` / `nft -f f`

A bastion rule set follows from this pattern: default-drop INPUT, ESTABLISHED/RELATED first, loopback, SSH from the management set, ICMP limited to the types needed for path MTU discovery and diagnostics, and a rate-limited log. The ICMP types matter: dropping all ICMP breaks path MTU discovery (8.10 (p. 506)).

> TRAP
>
> *Claim:* "I added the ACCEPT rule, so it works." *Why it is wrong:* rules are evaluated top to bottom and the first match wins; an earlier DROP, or the packet counters showing zero, proves the rule is never reached. *Correct statement:* check order with `--line-numbers` and the pkts column, and test with the real traffic. Further, mixing `iptables` and `nft` on the same host can leave two rule sets competing at the same hook.

> LAB
>
> `LAB 8.7 · 40 MIN` Firewall builder. In the three-namespace lab of 8.5 (p. 492), run a web server in `blue` on port 8080 and apply the bastion rule set to `blue`. Test a matrix of flows from `red`: TCP 8080, TCP 9999, ping, and the same after `iptables -t nat -A POSTROUTING -o rt-eth1 -j MASQUERADE` in `rtr`.
>
> **What you should notice:** 8080 succeeds from the management range and fails once masquerading changes the source; 9999 hangs (silent drop) and the counter rises; ping is allowed by the ICMP rule. Capture with `tcpdump` on both sides of `rtr` to watch the translated source address appear.

> CHECK YOURSELF
>
> 1. Which chains does a packet forwarded through a Linux router traverse? [5]
> 2. Why is the ESTABLISHED,RELATED rule first? [6]

8.13 Performance: iperf3, BDP, socket buffers, sysctl

SHOULD · ■■□

Throughput questions need a measurement before a theory. `iperf3` generates traffic between two hosts and reports the achieved rate, which separates "the network is slow" from "the application is slow". The theory of windows and congestion control is in 3.9 (p. 192); this section covers the tools and the Linux settings that decide whether the theory is reached.

```
server$ iperf3 -s                                  # listen on TCP/UDP 5201
client$ iperf3 -c 10.0.2.2 -t 30                   # one TCP stream for 30 s
client$ iperf3 -c 10.0.2.2 -P 4                    # four parallel streams
client$ iperf3 -c 10.0.2.2 -R                      # reverse: server sends (tests the
other direction)
client$ iperf3 -c 10.0.2.2 -u -b 200M              # UDP at a fixed rate: reports
loss and jitter
client$ iperf3 -c 10.0.2.2 -w 64K                  # fix the socket buffer size
```

```
$ iperf3 -c 10.0.2.2 -t 3 -u -b 200M
[  5]   0.00-3.00   sec  71.5 MBytes   200 Mbits/sec  0.000 ms  0/55626 (0%)    sender
[  5]   0.00-3.00   sec  71.5 MBytes   200 Mbits/sec  0.011 ms  20/55626 (0.036%)
receiver
```

The TCP line shows bitrate and `Retr`, the retransmission count. The UDP line shows `Jitter` and `Lost/Total datagrams` at the receiver: loss in UDP is measured, not hidden. (Lab figures such as 16 Gb/s with two streams, over a virtual link with a round-trip time under 0.1 ms, show only the host's speed; they say nothing about a wide-area path.)

1. Between 1400 and 1499 bytes: a packet of 1372 + 28 = 1400 fits and one of 1500 does not. The router's "mtu = 1400" message gives the exact value.↩
2. Closed: "Connection refused" (a RST). Filtered or dead: a timeout, with no message from the network.↩
3. A damaged cable (only two pairs working) or a duplex mismatch from forced settings on one side.↩
4. The tap sees the buffer before TSO/GSO splits it; the wire carries 1448-byte segments.↩
5. PREROUTING, FORWARD and POSTROUTING.↩
6. Most packets belong to existing flows, so it is the fastest path, and it allows replies without opening inbound ports.↩

Bandwidth-delay product. A TCP sender may have at most one window of unacknowledged data in flight, so its throughput is at most *window* ÷ *RTT*. To fill a link the window must reach the *bandwidth-delay product* (BDP), the amount of data in the pipe: *BDP* = *bandwidth* × *RTT*. The analogy is a long pipe: the longer and wider it is, the more water must be poured in before the first drop comes out. For a 1 Gb/s path with an 80 ms RTT (a Mumbai to Frankfurt circuit), BDP = $10^9 \times 0.08 / 8 = 10$ MB. A window of 64 KB, the largest without window scaling, allows $65\,536 \times 8 / 0.08 \approx 6.5$ Mb/s, under 1 % of the link.

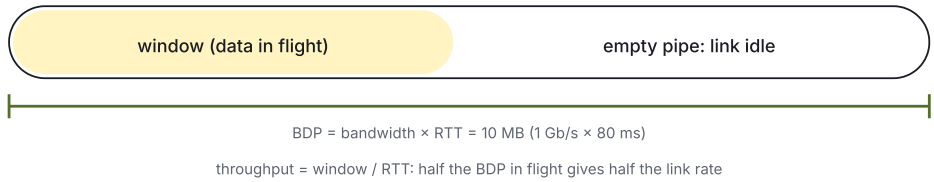


Figure 8.10 The pipe model. The link is full only when the data in flight equals the bandwidth-delay product; a window smaller than that leaves part of the pipe empty.

Link	RTT	BDP	Throughput with a 64 KB window
100 Mb/s LAN	1 ms	12.5 KB	100 Mb/s (window is larger than BDP)
1 Gb/s campus WAN	10 ms	1.25 MB	52 Mb/s
1 Gb/s Mumbai to Frankfurt	80 ms	10 MB	6.5 Mb/s
10 Gb/s data-centre interconnect	100 ms	125 MB	5.2 Mb/s

What decides the window. The sender's window is limited by the smaller of the congestion window and the *receiver's advertised window*, which is bounded by the receive buffer. Linux sizes it automatically between the limits of `net.ipv4.tcp_rmem` (minimum, default, maximum, in bytes) for reading and `net.ipv4.tcp_wmem` for writing; `net.core.rmem_max` and `wmem_max` cap what an application may request with `SO_RCVBUF` (7.10 (p. 445)). Window scaling (`net.ipv4.tcp_window_scaling=1`, on by default) multiplies the 16-bit window field by up to 2^{14}, so windows above 64 KB are possible only if both ends negotiated it in the SYN (the `wscale` option seen in 8.7 (p. 498)). On a long fat pipe, check that the maximum of `tcp_rmem` and `tcp_wmem` is at least the BDP; the lab VM used for this chapter has a maximum of 32 MB, enough for about 2.7 Gb/s at 100 ms.

```
$ sysctl net.ipv4.tcp_rmem net.ipv4.tcp_wmem net.core.rmem_max
net.ipv4.tcp_congestion_control
net.ipv4.tcp_rmem = 4096    131072  33554432
net.ipv4.tcp_wmem = 4096    16384   4194304
net.core.rmem_max = 4194304
net.ipv4.tcp_congestion_control = bbr
$ sysctl net.ipv4.tcp_available_congestion_control
net.ipv4.tcp_available_congestion_control = reno bbr cubic
```

sysctl. The kernel's tunable parameters appear as files under `/proc/sys`, and `sysctl` reads and writes them: `sysctl -w net.ipv4.ip_forward=1` changes the running kernel immediately and is lost at reboot. To make a change permanent one writes it into a file such as `/etc/sysctl.d/90-campus.conf` (`net.ipv4.ip_forward = 1`) and loads it with `sysctl --system`. The settings met most often are `ip_forward` (route between interfaces), `net.core.somaxconn` and `tcp_max_syn_backlog` (accept and SYN queues),

`tcp_congestion_control` (cubic or bbr), `ip_local_port_range` (ephemeral ports; 32768 to 60999 here, about 28 000 simultaneous connections to one destination) and `tcp_tw_reuse`.

Congestion control and queues. `cubic` is the long-standing default and `bbr` estimates bandwidth and RTT directly rather than reacting to loss; `sysctl -w net.ipv4.tcp_congestion_control=bbr` switches new connections, and `ss -ti` shows which one a given connection uses. To reproduce a long path in a laboratory one adds delay and loss with the `netem` queueing discipline, `tc qdisc add dev eth0 root netem delay 40ms loss 0.1%`, and removes it with `tc qdisc del dev eth0 root`. (The kernel of the book's test VM lacked `netem`, so no measurement is printed; a laptop with Ubuntu 24.04 has it.)

WORKED EXAMPLE

100 ms, 10 Gb/s, but 50 Mb/s per flow. Compute the window the transfer actually uses and compare it with the BDP.

1. BDP = $10^{10} \times 0.1 / 8 = 125$ MB.
2. The window in use: $50 \times 10^6 \times 0.1 / 8 = 625$ KB, only 0.5 % of the BDP.
3. Two explanations give this size. (i) A buffer cap: a maximum receive buffer of about 625 KB, or window scaling absent (which would give 64 KB and 5 Mb/s, so scaling is present but the cap is low). (ii) Loss: by the Mathis relation, rate ≈ 1.22 × MSS / (RTT × √p), loss $p = (1.22 \times 1448 \times 8 / (0.1 \times 50 \times 10^6))^2 \approx 8 \times 10^{-6}$, one packet in 125 000.
4. Distinguish them with `ss -ti` during the transfer: `cwnd` small with `retrans` rising means loss, `rwnd_limited` or `snd_wnd` pinned means a buffer cap, and a capture shows window-full or zero-window flags (8.8 (p. 502)).

Result: raise `tcp_rmem`/`tcp_wmem` maxima towards 128 MB if buffer-limited; find and remove the loss otherwise, or try bbr, which tolerates random loss better than cubic. Several parallel streams (`-P`) also work around either limit.

CHECK YOURSELF

1. A 622 Mb/s link has an RTT of 40 ms. What is the BDP? [1]
2. Why does `sysctl -w` not survive a reboot? [2]

8.14 Systems debugging toolkit: /proc, lsof, strace, systemd SHOULD · ■■□

When the network looks clean the fault is on the host: a process that died, a disk that filled, a service that never bound its port, a limit on open files. The tools in this section answer "what is this process doing" and "what does the system believe". Process, descriptor and system-call concepts are taught in 7.2 (p. 416) and 7.8 (p. 441).

/proc. The `/proc` file system is a window onto the kernel, presented as files that are generated when read. `/proc/<pid>/` describes a process: `cmdline`, `status` (state, memory, threads), `fd/` (one symbolic link per open descriptor), `limits`, `net/`. System-wide files include `/proc/meminfo`, `/proc/loadavg`, `/proc/net/dev` (interface counters), `/proc/net/tcp` (the socket table in hexadecimal, from which `ss` once read), `/proc/net/route`, `/proc/net/arp` and `/proc/sys/net/*` (the tunables of 8.13 (p. 514)). Five worth remembering for networking are `/proc/net/dev`, `/proc/net/tcp`, `/proc/net/arp`, `/proc/sys/net/ipv4/ip_forward` and `/proc/<pid>/fd`.

```
$ ls -l /proc/21426/fd                       # the web server of the lab
lr-x------ 1 root root 64 0 -> /dev/null
l-wx------ 1 root root 64 1 -> /dev/null
lrwx------ 1 root root 64 3 -> socket:[125319]       <- descriptor 3 is the listening
socket
$ grep -E 'Name|State|VmRSS|Threads' /proc/21426/status
Name:   python3
State:  S (sleeping)
VmRSS:     19784 kB
Threads:    1
$ cat /proc/sys/net/ipv4/ip_forward
0
```

lsof ("list open files") shows which process holds which file or socket. Because sockets are files, it answers "who is using port 8080": `lsof -nP -i :8080`, and `lsof -nP -iTCP -sTCP:LISTEN` lists all listeners. `ss -tlnp` gives the same for sockets alone; `lsof +L1` finds deleted files that still occupy disk, a common cause of a "full" disk with no large file visible.

```
$ lsof -nP -i :8080
COMMAND   PID USER   FD   TYPE DEVICE SIZE/OFF NODE NAME
python3 21426 root    3u  IPv4 125319      0t0  TCP 10.0.2.2:8080 (LISTEN)
$ kill 21426                                    # SIGTERM first: lets the program close its
connections
```

strace prints the system calls a process makes, with arguments and return values, which turns "it hangs" into a specific call. `strace -f -tt -e trace=network,connect,poll curl …` follows children (`-f`), adds timestamps (`-tt`) and filters to the network calls. For a hanging `curl`, two patterns separate the causes. A DNS hang shows `connect` to the resolver's address on port 53 (or `sendto` on a UDP socket) followed by a long `poll` with no reply before any `connect` to the target. A TCP hang shows the name resolved, then `connect(5, {sin_port=htons(8080), sin_addr=inet_addr("10.0.2.2")}, 16) = -1 EINPROGRESS` followed by a `poll` that waits until the timeout, the signature of a dropped SYN. A refused port instead returns the error at once (`ECONNREFUSED`).

```
$ strace -f -tt -e trace=connect,ppoll curl -s -m2 http://10.0.2.2:8080/ -o /dev/null
13:35:39.483272 connect(3, {sa_family=AF_UNIX, sun_path="/var/run/nscd/socket"}, 110) =
-1 ENOENT
13:35:39.484290 connect(5, {sa_family=AF_INET, sin_port=htons(8080),
sin_addr=inet_addr("10.0.2.2")}, 16) = -1 EINPROGRESS
```

(The first line is the C library looking for a name-service cache daemon that is not installed, a harmless failure; the second, in a run with the port blocked by a firewall, is followed by a wait with no answer.) Attaching to a running process uses `strace -p PID`; `strace -c` prints a table of calls and time, which finds the slow call.

systemd and the journal. Most distributions start services with *systemd*. `systemctl status nginx` shows whether a unit is active, its main process, the last log lines and why it stopped; `systemctl restart|reload|enable --now nginx`, `systemctl list-units --failed` and `systemctl list-timers` are the others in daily use. Service logs go to the *journal*, read with `journalctl`: `journalctl -u nginx --since "10 min ago"`, `-f` (follow), `-p err` (priority error and above), `-b` (since the last boot) and `-k` (kernel messages, the equivalent of `dmesg`). (The lab container ran without systemd as its first process, so these commands cannot be shown with real output here; they behave as described on any standard Ubuntu host.)

Resource tools. `top` or `htop` shows CPU and memory per process; `vmstat 1` prints one line per second with run-queue length (`r`), swap (`si`, `so`), blocked I/O (`b`) and CPU split (`us sy id wa`); `free -m` shows memory (the `available` column is what applications can use, not `free`); `df -h` shows file systems and `du -sh /var/log/*` finds what fills one; `iostat -x 1` shows disk utilisation; `uptime` or `/proc/loadavg` show load averages. Two limits cause strange failures. The *open files* limit, `ulimit -n` (20 000 in the lab shell, often 1024 by default), caps descriptors and therefore connections: "too many open files" in a server log is this. A *full disk* makes services fail to write logs, sockets or temporary files, and the symptom can be a 502 from the proxy in front.

WORKED EXAMPLE

A website returns 502 Bad Gateway. The reverse proxy is up, so it cannot reach the application behind it.

1. `systemctl status app` shows the unit `failed`; the last lines show it exited.
2. `journalctl -u app --since "10 min ago"` shows `OSError: [Errno 28] No space left on device`.
3. `df -h` shows `/var` at 100 %; `du -sh /var/log/* | sort -h | tail -3` finds a 40 GB log file written by a debug option left on.
4. Truncate the file (`: > /var/log/app/debug.log`; deleting it while the process runs would not free space until the process exits, which `lsof +L1` would reveal), restart the unit, and configure `logrotate` with size limits.

Result: the 502 was the symptom of a disk problem two layers away; none of the network tools would have found it, which is why the host checks are part of the method in Pillar 11 (p. 634)**.**

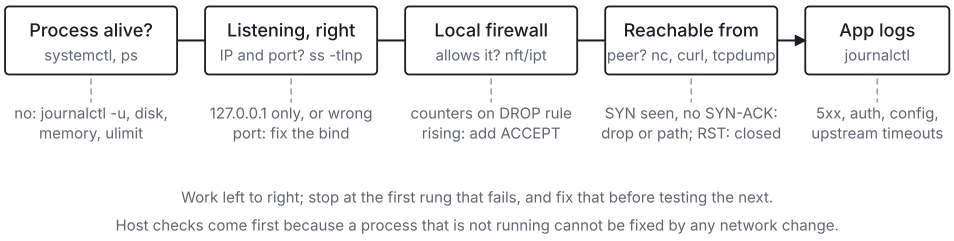


Figure 8.11 A decision ladder for "service unreachable on Linux". Each rung has one command and one typical finding.

INTERVIEW LENS

AMAZON SYSDE · GOOGLE SRE · ARISTA · JUNIPER

"What is in /proc", "find the process on port 8080 and stop it", "strace a hanging curl" and "disk is full, service is down" are standard live-shell questions. A strong answer uses the specific file or command, explains what it reads (kernel state, not stored files), names the signal sequence (TERM, then KILL) and checks the cause afterwards (the 40 GB log, the missing `close()`). Follow-ups: `ulimit -n` and `/proc/<pid>/limits`, zombie processes, and `strace -c` for a profile. Solved forms are Q8.20, Q8.21, Q8.22 and Q8.28.

CHECK YOURSELF

1. How does `strace` distinguish a DNS hang from a TCP connect hang? [3]
2. A deleted 10 GB log is still filling the disk. Which command finds the holder? [4]

1. $622 \times 10^6 \times 0.04 / 8 \approx 3.1$ MB.↩
2. It changes only the running kernel's value; the persistent source is a file in `/etc/sysctl.d/` loaded at boot.↩
3. A DNS hang waits on port 53 before any connect to the target; a TCP hang shows `connect` to the target returning `EINPROGRESS` and then waiting.↩
4. `lsof +L1` (files with a link count below one) shows the process holding the deleted file open.↩

Problem Set

Every interview question of this pillar is solved once, here, in the compact template: a 30-second answer first, then the approach, code, follow-ups and common mistakes where they apply. Tags give the companies and round where the topic was reported, the difficulty (■□□ easy, ■■□ medium, ■■■ hard), and **S** when a specific question was reported by candidates or **T** when only the topic was reported and the wording is ours. Output shown comes from the Campus Net lab of this chapter. Where a section already teaches the full reasoning, the entry gives the answer and a pointer, so that nothing is explained twice.

Q8.1 How do you list all listening TCP and UDP ports and the owning process?

CISCO · AMAZON · ARISTA · TECH-1 · ■□□ · T

30-second answer: `ss -tulpn`, which prints TCP and UDP (`-t -u`), listening sockets (`-l`), the owning process (`-p`) and numeric ports and addresses (`-n`). Run it as root to see other users' processes.

Answer. `-n` matters because without it `ss` resolves addresses and port numbers to names, which is slow and may hide the port number being searched for. Read the *Local Address* column: `0.0.0.0:22` or `*:22` accepts on every IPv4 address, `127.0.0.1:5000` only on loopback, `[::]:80` on every IPv6 (and usually IPv4) address. Filter with `ss -tlnp 'sport = :8080'`. **Follow-ups.** `lsof -i -P -n` gives the same through the file table; `netstat -tulpn` is the legacy form (8.6 (p. 495)). **Mistakes.** Forgetting `sudo` and concluding that no process owns the port.

Q8.2 What is the difference between `ip addr` and `ifconfig`? Why was net-tools deprecated?

CISCO · QUALCOMM · MICROSOFT · TECH-1 · ■□□ · T

30-second answer: `ifconfig` belongs to the unmaintained net-tools package and reads `/proc` and uses old `ioctl` calls; `ip` (iproute2) uses netlink and covers everything the kernel supports: several addresses per interface, policy routing, VRFs, namespaces, tunnels and modern IPv6.

Answer. net-tools has seen no significant development since the early 2000s. Its `ifconfig` shows only one address per alias label, cannot show or set many link types or the neighbour states, and has no equivalent of `ip rule`, `ip netns` or `ip route get`. `ip` reads and writes through netlink, which is faster on hosts with many routes or sockets and returns consistent snapshots. The replacements are listed in 8.1 (p. 484). **Follow-ups.** `ip -br a`, `ip -j` for JSON, `ip -s link` for counters. **Mistakes.** Saying `ifconfig` is "removed": it still runs where installed, but is not installed by default on current Debian and Ubuntu.

Q8.3 How do you check disk usage and find the biggest directory?

AMAZON · MICROSOFT · TECH-1 · ■□□ · T

30-second answer: `df -h` shows file-system usage; `du -sh * | sort -h` (or `du -xh --max-depth=1 / | sort -h | tail`) shows which directory is biggest. Run the second from the top and descend into the largest.

Approach. `df -h` first, because it names the full file system; then `du -xh --max-depth=1 /var | sort -h | tail -5`, with `-x` to stay on that file system, repeated one level down. **Follow-ups.** If `df` says 100 % but `du` finds less, deleted files held open by a process (`lsof +L1`) or inode exhaustion (`df -i`) are the usual causes; `ncdu` gives an interactive view. **Mistakes.** Running `du` on `/` without `-x`, which counts `/proc` and mounted volumes.

Q8.4 What do `/etc/resolv.conf`, `/etc/hosts` and `/etc/nsswitch.conf` contain, and in what order are they consulted?

GOOGLE · AMAZON · MICROSOFT · TECH-1 · ■□□ · T

30-second answer: `nsswitch.conf` sets the order of name sources, normally `hosts: files dns`; so `/etc/hosts` is read first, then DNS using the servers named in `/etc/resolv.conf`.

Answer. `/etc/hosts` maps names to addresses locally. `/etc/resolv.conf` lists `nameserver` addresses, a `search` list of suffixes to try on short names and `options` (`timeout`, `attempts`, `ndots`); on current Ubuntu it holds the stub `127.0.0.53` of `systemd-resolved`, and `resolvectl status` shows the real upstreams. Applications call `getaddrinfo()`, which follows `nsswitch.conf`. **Follow-ups.** `dig` bypasses hosts and nsswitch, so `getent hosts name` is the right test of what applications see (8.9 (p. 503)). **Mistakes.** Editing `resolv.conf` on a system where it is regenerated, so the change vanishes.

Q8.5 What does `ping` actually send? What does "Destination Host Unreachable" from your own IP mean?

CISCO · QUALCOMM · MICROSOFT · TECH-1 · ■□□ · T

30-second answer: ICMP Echo Request (type 8, code 0) with an identifier, a sequence number and a payload, answered by Echo Reply (type 0). "Destination Host Unreachable" whose source is the sender's own address means ARP found no owner for an on-link destination.

Answer. The default payload is 56 bytes, so with an 8-byte ICMP header and 20-byte IP header the packet is 84 bytes. If the destination is on the local subnet the host must ARP for it; when there is no reply, the host's own IP layer reports the failure. `ip neigh` shows the entry as `INCOMPLETE` and then `FAILED` (8.5 (p. 492)). The same words from a *router's* address mean that router had no route. **Follow-ups.** A reply of "Request timeout" or no reply is silence (drop or dead host); TTL in the reply estimates the hop count. **Mistakes.** Treating a successful ping as proof that the service works.

Q8.6 Explain permissions `754` and the sticky bit on `/tmp`.

CISCO · AMAZON · TECH-1 · ■□□ · T

30-second answer: 754 is rwx for the owner (7), r-x for the group (5) and r– for others (4). The sticky bit on a directory lets only a file's owner (or root) delete or rename it, which is why world-writable `/tmp` shows `drwxrwxrwt`.

Answer. Read is 4, write 2, execute 1, summed per class (Figure 8.3 in 8.2 (p. 485)). On a directory, execute means permission to enter and use names inside, and write permission to create or delete entries there. Without the sticky bit, anyone with write access to a directory could remove other users' files in it. **Follow-ups.** `chmod 1777 dir` sets the sticky bit; setuid (4000) runs a program as its owner (`passwd`), setgid (2000) on a directory makes new files inherit its group. `umask 022` gives new files 644. **Mistakes.** Using `chmod 777` to "fix" a permission problem.

Q8.7 Write a one-liner to print the top 10 IPs by request count from an access log.

AMAZON · ARISTA · GOOGLE · TECH-1 · ■■□ · S

30-second answer: `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10`. The first field is the client address; `sort` groups equal lines so that `uniq -c` can count them; `sort -rn` ranks by count.

Answer.

```
$ awk '{print $1}' access.log | sort |
uniq -c | sort -rn | head -3
      3 10.20.7.9
      3 10.20.1.15
      2 10.20.4.31
$ awk '{c[$1]++} END {for (i in c) print
c[i], i}' access.log | sort -rn | head
-10     # no initial sort
```

Complexity. The pipeline sorts the whole input, O(n log n) in time, spilling to disk when memory is short (`sort -S 1G`); the awk array version is O(n) and holds only one entry per distinct address, so it is better for a very large log. **Follow-ups.** Requests per URL (`$7`), per status (`$9`), per minute (cut the timestamp with `cut -c14-18`); only errors (`awk '$9>=500'`); behind a proxy the client is in `X-Forwarded-For`, not `$1`. **Mistakes.** Omitting the first `sort` (`uniq` counts only adjacent lines) and sorting counts as text (`sort -n`, not `sort`).

Q8.8 Write a Bash script that reads a file of hostnames and reports which are unreachable, exiting non-zero if any fail.

AMAZON · ARISTA · JUNIPER · TECH-1 · ■■□ · T

30-second answer: loop over the file with `while read`, ping each host once with a timeout, print the failures to stderr, and `exit 1` at the end if any failed. The version with options, CSV output and a log is `reach.sh` in 8.4 (p. 489)**.**

Code.

```
#!/usr/bin/env bash
set -uo pipefail                        #
no -e: a failing ping is expected, not
fatal
fail=0
while IFS= read -r h || [[ -n $h ]]; do
  [[ -z $h || $h == \#* ]] && continue
  ping -c1 -W1 "$h" >/dev/null 2>&1 ||
{ echo "UNREACHABLE $h" >&2; fail=1; }
done < "${1:?usage: ${0##*/} hostsfile}"
exit "$fail"
```

```
$ ./q8.sh hosts.txt; echo rc=$?
UNREACHABLE 192.0.2.77
UNREACHABLE 10.20.99.1
rc=1
```

Follow-ups. For hundreds of hosts use `xargs -P 32` (parallel), as in `sweep.sh`; separate DNS failure from ping failure with `getent hosts`; above fifty lines or with retries, JSON output or threads, use Python. **Mistakes.** `for h in $(cat file)` (word splitting), no quoting of `"$h"`, and `set -e` that aborts at the first dead host.

Q8.9 What does `ip route get 1.1.1.1` tell you that `ip route show` does not?

GOOGLE · MICROSOFT · AMAZON · TECH-1 · ■■□ · T

30-second answer: it performs the actual lookup and prints the route the kernel would use, with the next hop, the output interface and the source address, after policy rules and longest-prefix match. `show` only lists table contents.

Answer. `ip route show` lists the main table. The real decision also depends on `ip rule` (other tables), marks, the input interface and the best-matching prefix. `get` applies all of these:

```
$ ip route get 10.0.2.2
10.0.2.2 via 10.0.1.1 dev r-eth0 src
10.0.1.2 uid 0
$ ip route get 8.8.8.8 from 10.1.5.5
# test a policy rule
8.8.8.8 from 10.1.5.5 via 198.51.100.1
dev p1 table 100
```

The `src` field answers “which address will the packet carry”, which is what a remote firewall will see and which ACLs and NAT rules match. **Follow-ups.** `iif eth0` simulates a forwarded packet; an error “Network is unreachable” means no route at all. **Mistakes.** Reading the default route in `show` and assuming it is used for every destination.

Q8.10 How does traceroute work? Why does Linux use UDP and Windows ICMP? How do you trace through a firewall that allows only 443?

CISCO · JUNIPER · ARISTA · GOOGLE · TECH-1 · ■■□ · T

30-second answer: it sends probes with TTL 1, 2, 3, …; each router that decrements the TTL to zero returns ICMP Time Exceeded, revealing its address, until a probe reaches the destination. The UDP or ICMP choice is historical; for a firewall that allows only 443, use TCP probes: `traceroute -T -p 443 host`.

Answer. Figure 8.8 shows the exchange (8.10 (p. 506)). The destination answers UDP probes (sent to unlikely high ports) with ICMP port unreachable, ICMP echo probes with an echo reply, and TCP SYN probes with SYN-ACK or RST; any of these ends the trace. Both UDP and ICMP work; Unix tradition used UDP and Windows' `tracert` ICMP echo, and either may be filtered. A TCP SYN to the permitted port passes the filter and reveals each hop until the firewall. **Follow-ups.** ECMP makes successive probes take different paths, so Paris traceroute keeps the flow tuple constant; `* * *` is a missing reply, not a failure; MPLS can hide hops. **Mistakes.** Reading the latency of a middle hop as the latency of the path.

Q8.11 `mtr` shows 50 % loss at hop 3 but 0 % at the destination. Is there a problem?

CISCO · JUNIPER · AMAZON · GOOGLE · TECH-1 · ■■□ · T

30-second answer: no. Loss at an intermediate hop that does not continue to later hops is the router rate-limiting its own ICMP replies (control-plane policing), not dropping forwarded traffic. Only loss that persists to the final hop is real.

Answer. Hop 3 is probed by the packets whose TTL expires there, and the router must generate the reply on its slow path. Forwarded traffic uses the fast path and is untouched; if it had been dropped at hop 3, hops 4 onwards would show at least the same loss. Confirm with a different protocol (`mtr -T -P 443`) or by testing the real application. **Follow-ups.** The mirror case: loss that begins at hop 6 and stays through the destination is real, located at or just after hop 6; loss only at the last hop may be the destination rate-limiting ICMP, so test with TCP. **Mistakes.** Sending a ticket to the owner of hop 3.

Q8.12 A server has thousands of sockets in TIME-WAIT. Is that a problem? Which side ends up there, and why does it last 2 × MSL?

GOOGLE · AMAZON · CISCO · TECH-1 · ■■□ · T

30-second answer: usually not a problem. TIME-WAIT is held by whichever side closed first, for 60 s on Linux (2 × MSL), so that a lost final ACK can be answered with a repeated FIN and old segments of the connection expire before the port pair is reused.

Answer. The active closer sends the first FIN, receives the peer's FIN and sends the last ACK; it must keep the connection tuple for 2 × MSL in case that ACK was lost, and so that delayed duplicates from the old connection die in the network. Thousands of entries cost a few hundred bytes each. They matter only to a *client* opening many short connections to one destination: the tuple space is limited by ephemeral ports (28 232 by default), so about 470 new connections per second to one server address and port exhaust it. Remedies: reuse connections (keep-alive, pooling), widen `ip_local_port_range`, enable `net.ipv4.tcp_tw_reuse` for outgoing connections, spread over more server addresses. **Follow-ups.** `tcp_tw_recycle` was removed from the kernel because it broke clients behind NAT; `SO_REUSEADDR` lets a restarted server bind while old connections wait (7.10 (p. 445)). **Mistakes.** Calling TIME-WAIT a leak, or lowering it by recompiling.

Q8.13 Many sockets are in CLOSE-WAIT. What is the cause and whose fault is it?

GOOGLE · AMAZON · ARISTA · TECH-1 · ■■□ · T

30-second answer: the peer has closed (sent FIN) and the local application has not called `close()`. It is an application bug, a descriptor leak, not a network problem; the kernel waits for the program indefinitely.

Answer. CLOSE-WAIT is the state after receiving a FIN and before the local FIN; only the application can move the socket on, by `close()`. Find the process with `ss -tanp state close-wait`, then count descriptors in `/proc/PID/fd` and read the code for error paths where `close()` is skipped (a `return` before cleanup, an exception, a connection pool that never evicts). The growth eventually hits the open-files limit and the service fails with "too many open files". **Follow-ups.** Stopgap: restart the process; permanent: close in a `finally` or use RAII; contrast with FIN-WAIT-2, which is the *other* side of the same half-close. **Mistakes.** Tuning TCP timers, which have no effect on CLOSE-WAIT.

Q8.14 Write a tcpdump filter to capture only SYN packets (not SYN-ACK) to port 443.

CISCO · JUNIPER · QUALCOMM · TECH-1 · ■■□ · T

30-second answer: `tcpdump -nn 'tcp dst port 443 and tcp[tcpflags] & (tcp-syn|tcp-ack) == tcp-syn'`. The mask keeps the SYN and ACK bits, and the comparison requires SYN set and ACK clear.

Answer. A SYN-ACK has both bits set, so testing `tcp-syn` alone would match it too; masking with `(tcp-syn|tcp-ack)` and comparing with `tcp-syn` selects only the first packet of a handshake. The filter runs in the kernel as BPF (8.7 (p. 498)). **Follow-ups.** Resets: `tcp[tcpflags] & tcp-rst != 0`; SYN-ACKs: `== (tcp-syn|tcp-ack)`; the older form is `tcp[13] & 18 == 2`. **Mistakes.** Writing `tcp[tcpflags] == tcp-syn` for a pure SYN (it works, but breaks with ECN flags set), and forgetting to quote the expression.

Q8.15 You see 64 KB TCP segments in tcpdump on a 1500-MTU link. Is it a bug?

QUALCOMM · BROADCOM · ARISTA · TECH-1 · ■■□ · T

30-second answer: no. TCP segmentation offload (TSO) and generic segmentation offload (GSO) let the stack hand one large buffer to the driver, which is cut into MSS-sized frames later; the capture tap sees the buffer. On receive, GRO merges frames before the tap.

Answer. The wire frames are 1500 bytes or fewer; the 64 KB figure is a software view that saves per-packet CPU cost. Check with `ethtool -k eth0 | grep -E 'tcp-seg|generic'`; disable temporarily with `ethtool -K eth0 tso off gso off gro off` to see real frames, or capture on another machine or a mirror port. A 64 KB buffer becomes about 45 segments of 1448 bytes (8.11 (p. 509)). **Follow-ups.** Checksums appear wrong on the sender because the NIC fills them in after the tap; a packet of 64 KB never crosses the wire, so path-MTU reasoning is unaffected. **Mistakes.** Filing a bug, or concluding that the MTU is 64 KB.

Problem Set, continued: DNS, firewalls, namespaces and walk-throughs

Q8.16 Walk me through the output of `dig +trace google.com`.

GOOGLE · AMAZON · CISCO · TECH-1 · ■■□ · T

30-second answer: `+trace` makes dig do the iterative resolution itself: it takes the root servers from its hints, asks one for `google.com`, receives a referral to the `.com` servers, asks one of those, receives a referral to Google's authoritative servers, and finally receives the answer with the `aa` flag.

Answer. Each stage prints an NS set (the referral) and the server that supplied it. The sequence is root `.` then TLD `com.` then the zone `google.com.` then the A record. The referral carries glue addresses in the additional section so that the next server can be reached without resolving its name. A resolver that serves clients does exactly this on a cache miss, then caches each answer for its TTL; the stub on the client just sends one recursive query. Reading the trace locates a fault: a stage that returns no referral or times out identifies a broken delegation or a dead authoritative server. The sequence is drawn in 3.10 (p. 203) and the output is annotated in 8.9 (p. 503). **Follow-ups.** DNSSEC adds `DS` and `RRSIG` records and the `ad` flag; `+norecurse` mimics what a resolver sends. **Mistakes.** Expecting `+trace` to use the system resolver's cache; it ignores it.

Q8.17 What is the difference between NXDOMAIN and SERVFAIL? What would you check for each?

GOOGLE · AMAZON · MICROSOFT · TECH-1 · ■■□ · T

30-second answer: NXDOMAIN is a definitive answer that the name does not exist. SERVFAIL means the server could not produce an answer: a failure on the way. Check the name and zone for NXDOMAIN; check the authoritative servers and DNSSEC for SERVFAIL.

Answer.

	NXDOMAIN	SERVFAIL
Meaning	name does not exist (authoritative)	server could not resolve
Cached	yes, for the SOA's negative TTL	briefly
Typical causes	typo, record never created, wrong zone, search-suffix expansion	authoritative servers unreachable or lame, broken delegation, DNSSEC validation failure, resolver overload
First checks	`dig @authoritative name` shows the same? check zone file and the `search` list	query the authoritative servers directly; `dig +cd name` (succeeds if DNSSEC is the cause); `dig +trace`

NOERROR with an empty answer is a third case: the name exists but not with that record type. **Follow-ups.** REFUSED means a policy denial; a timeout is neither. **Mistakes.** Flushing caches for SERVFAIL when the authoritative side is broken.

Q8.18 Explain iptables tables and chains. Which chain sees a packet routed through the box, and which a packet destined to it?

CISCO · AMAZON · JUNIPER · TECH-1 · ■■□ · T

30-second answer: tables group rules by purpose (`filter`, `nat`, `mangle`, `raw`); chains are the hook points (PREROUTING, INPUT, FORWARD, OUTPUT, POSTROUTING). A forwarded packet crosses PREROUTING, FORWARD and POSTROUTING; a packet for the host crosses PREROUTING and INPUT.

Answer. Each table registers only in some chains: `filter` in INPUT, FORWARD, OUTPUT; `nat` in PREROUTING, OUTPUT, POSTROUTING (for the first packet of each flow only; the rest follow conntrack); `mangle` in all five; `raw` in PREROUTING and OUTPUT. Within a chain, rules run top to bottom and the first terminating target wins; a chain without a match uses its policy. Local packets leave through OUTPUT and POSTROUTING (Figure 8.9, 8.12 (p. 510)). **Follow-ups.** Where does DNAT happen (PREROUTING, before the routing decision) and SNAT (POSTROUTING, after it)? Why does a forwarding firewall need `ip_forward=1`? **Mistakes.** Putting a filtering rule for forwarded traffic in INPUT.

Q8.19 DROP versus REJECT: what does the client see in each case, and which is better for internal networks?

CISCO · AMAZON · TECH-1 · ■■□ · T

30-second answer: with DROP the client gets nothing and times out after retransmitting; with REJECT it gets an immediate TCP RST or ICMP port-unreachable and fails at once. Internally REJECT is usually better, because it makes failures fast and obvious; DROP suits untrusted edges.

Answer. A dropped SYN is retried at about 1 s intervals before the connection attempt gives up after roughly two minutes, so applications appear to hang (8.7 (p. 498)). REJECT lets a monitoring system or a developer see “connection refused” immediately. On the Internet, DROP withholds the information that a host exists and slows scanners, but a determined scanner learns the same from timing, so the security gain is small. **Follow-ups.** `REJECT --reject-with tcp-reset` or `icmp-port-unreachable`; rate-limit ICMP errors. **Mistakes.** Saying one of them is “more secure” without context.

Q8.20 How do you find which process is using port 8080 and kill it gracefully?

AMAZON · GOOGLE · ARISTA · TECH-1 · ■■□ · T

30-second answer: `ss -tlnp 'sport = :8080'` or `lsof -nP -i :8080` gives the PID; `kill PID` sends SIGTERM; wait and confirm with `ss`; use `kill -9` only if it ignores SIGTERM.

Approach. Identify first, because the owner may be a service manager that restarts it. If it is a unit, stop it properly with `systemctl stop name` instead of killing the PID. Otherwise:

```
$ lsof -nP -i :8080
python3 21426 root  3u  IPv4 125319  0t0
TCP 10.0.2.2:8080 (LISTEN)
$ kill 21426; sleep 2; ss -tlnp 'sport =
:8080'     # SIGTERM first; the listener
disappears
$ kill -9 21426
# only if still present
```

Follow-ups. `fuser -k 8080/tcp` does both steps; a restarted port may be held in TIME-WAIT and needs `SO_REUSEADDR` in the new server (7.10 (p. 445)); signals are in 7.9 (p. 443). **Mistakes.** `kill -9` as the first move, and killing the parent supervisor that respawns it.

Q8.21 What is in `/proc`? Name five files you would look at for networking.

AMAZON · GOOGLE · ARISTA · TECH-1 · ■■□ · S

30-second answer: `/proc` is a virtual file system in which the kernel exposes process and system state as files that are generated on read. For networking: `/proc/net/dev`, `/proc/net/tcp`, `/proc/net/arp`, `/proc/net/route` and `/proc/sys/net/ipv4/ip_forward`.

Answer. `/proc/net/dev` holds per-interface byte, packet, error and drop counters; `/proc/net/tcp` is the socket table in hexadecimal (state `0A` is LISTEN, ports in hex: `1F90` = 8080); `/proc/net/arp` the neighbour cache; `/proc/net/route` the IPv4 routing table; `/proc/sys/net/*` the tunables, writable (8.13 (p. 514)). Per process: `/proc/PID/fd` (open files and sockets), `/proc/PID/status`, `/proc/PID/limits`, `/proc/PID/net/`. `/proc/meminfo` and `/proc/loadavg` give system state. Nothing in `/proc` takes disk space; it is the kernel's own data. **Follow-ups.** `/sys` for devices and driver parameters (`/sys/class/net/eth0/speed`); `ss` and `ip` are friendlier readers of the same data. **Mistakes.** Saying that `/proc` contains logs or that its files are stored on disk.

Q8.22 How do you make a sysctl change persistent? Give an example.

AMAZON · GOOGLE · TECH-1 · ■■□ · T

30-second answer: `sysctl -w` changes only the running kernel. For persistence, put the setting in a file under `/etc/sysctl.d/`, for example `net.ipv4.ip_forward = 1` in `90-forward.conf`, and apply it with `sysctl --system`; it is also read at boot.

Answer.

```
$ sudo sysctl -w net.ipv4.ip_forward=1
# now, until reboot
$ echo 'net.ipv4.ip_forward = 1' | sudo
tee /etc/sysctl.d/90-forward.conf
$ sudo sysctl --system
# reload every file
$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
```

Equivalent: write to `/proc/sys/net/ipv4/ip_forward` (also temporary). Files are read in lexical order, so a later name overrides an earlier one. **Follow-ups.** Per-interface keys such as `net.ipv4.conf.all.rp_filter`; some settings are applied by a network manager and overwritten; inside a container only namespaced keys can be changed. **Mistakes.** Editing `/etc/sysctl.conf` on a distribution that ships `sysctl.d`, or forgetting that a kernel module must be loaded before its keys exist.

Q8.23 A Linux host can ping its gateway but not 8.8.8.8, and the gateway can ping 8.8.8.8. Walk through your diagnosis.

CISCO · GOOGLE · AMAZON · MICROSOFT · TECH-2 · ■■■ · T

30-second answer: the host-to-gateway link works, so the fault is on the path through the gateway: the host's default route, the gateway forwarding or translating the host's packets, or the return path. Test with `ip route get 8.8.8.8`, then `tcpdump` on both gateway interfaces to see where the packet stops.

Approach. Work outwards, one layer at a time, and decide which side the fault is on before touching anything.

1. **Host.** `ip route get 8.8.8.8` must say `via <gateway> dev eth0 src <own address>`. A missing default route gives "Network is unreachable" (a route problem on the host); a wrong source address or interface points to policy routing or a second NIC. `ping 8.8.8.8` fails with silence, not an error, so the packet leaves.
2. **Gateway receives?** On the gateway, `tcpdump -ni LAN icmp` shows the echo request arriving. If it does not, the host is sending to the wrong MAC (`ip neigh` , a stale entry) or the VLAN is wrong.
3. **Gateway forwards?** If the request arrives but `tcpdump -ni WAN icmp` shows nothing leaving, the gateway discards it: `sysctl net.ipv4.ip_forward` is 0, a FORWARD chain rule drops it (`iptables -L FORWARD -nv`), or reverse-path filtering (`rp_filter`) rejected the source. If a packet leaves with the host's private source address, NAT is missing, so the reply cannot come back: check `iptables -t nat -L POSTROUTING -nv` for MASQUERADE.
4. **Return path.** If the request leaves translated but no reply arrives, the fault is upstream (ISP route, filtering). If the reply arrives at the gateway but not at the host, check conntrack (`conntrack -L`) and the FORWARD rules for the reverse direction.
5. **Why the gateway can ping.** The gateway pings from its own public address, which needs no forwarding or translation. That the gateway works and the host does not is the clue that the missing step is exactly one of those two.

Follow-ups. If `ping 8.8.8.8` works but `ping google.com` does not, the fault moves to DNS (8.9 (p. 503)); if small packets work and large ones hang, check the path MTU (8.10 (p. 506)). The complete method, with the other layers, is in the cases of 11.8 (p. 649); NAT is in 3.6 (p. 183) and static routes in 4.4 (p. 243). **Mistakes.** Changing the gateway's configuration before capturing at both interfaces.

Q8.24 Throughput between two data centres on a 10 Gb/s link with 100 ms RTT is only 50 Mb/s per flow. Diagnose.

GOOGLE · AMAZON · CISCO · ARISTA · TECH-2 · ■■■ · T

30-second answer: the BDP is 125 MB, and 50 Mb/s means a window of about 625 KB, so either the window is capped (receive buffer, no window scaling) or loss is holding the congestion window down. `ss -ti` and a capture decide which.

Approach. The full calculation is the worked example in 8.13 (p. 514): BDP = $10^{10} \times 0.1 / 8 = 125$ MB; window in use = $50 \times 10^{6} \times 0.1 / 8 = 625$ KB. Then:

Evidence	Reading	Fix
`wscale` absent in the SYN, or `snd_wnd`/`rcv_space` near 64 KB	window scaling disabled by a middlebox or setting	enable `tcp_window_scaling`; find the device that strips the option
`rwnd_limited` or window-full flags; receive buffer maximum below BDP	buffer cap	raise `tcp_rmem`/`tcp_wmem` maxima and `rmem_max` towards 128 MB
`cwnd` small, `retrans` rising, duplicate ACKs	loss: by the Mathis relation 50 Mb/s at 100 ms needs $p \approx 8 \times 10^{-6}$, one packet in 125 000	locate the loss (errors on a port, microbursts, a policer); try BBR; fix the cause
`app_limited`	the application sends too little	application tuning, more streams

Follow-ups. Parallel streams (`iperf3 -P 8`) show whether the limit is per-flow; a policer or shaper on the path produces a sawtooth; check `tc -s qdisc` on the sender. **Mistakes.** Upgrading the link, which does not change a window-limited flow.

Q8.25 Where does tcpdump capture relative to netfilter on ingress and egress? Why might you see a packet in tcpdump that the application never receives?

CISCO · JUNIPER · ARISTA · QUALCOMM · TECH-2 · ■■■ · T

30-second answer: on ingress the tap is before netfilter; on egress it is after. So a captured inbound packet may still be dropped by a firewall rule, by reverse-path filtering, by a full socket queue or because no socket matches.

Answer. The capture point is `AF_PACKET`, in the receive path before PREROUTING and in the transmit path after POSTROUTING (Figure 8.7). Reasons a seen packet is not delivered: (i) an INPUT, FORWARD or PREROUTING rule drops it (the DROP counter rises in `iptables -L -nv`); (ii) `rp_filter` discards a source address that would not route back through that interface (`nstat -az | grep -i filter`, or `IPReversePathFilter`); (iii) no listening socket, so the kernel sends RST (visible in the capture) or ICMP unreachable; (iv) the accept queue or receive buffer is full (`ss -ltn` with `Recv-Q` equal to `Send-Q`, `TcpExtListenDrops` rising); (v) a bad checksum or conntrack `INVALID` state; (vi) the packet was for another namespace or MAC (promiscuous capture). Conversely an *outbound* packet missing from the capture was dropped before the tap, by the local OUTPUT chain or a routing failure. **Follow-ups.** `nft monitor trace` or `iptables -t raw -j TRACE` follows one packet through the rules; XDP programs run even earlier (7.12 (p. 460)). **Mistakes.** Treating the capture as proof of delivery.

Q8.26 Design an nftables rule set for a bastion host. Explain conntrack and why ESTABLISHED,RELATED comes first.

AMAZON · GOOGLE · ARISTA · TECH-2 · ■■■ · T

30-second answer: default-drop input; accept established and related first, then loopback, SSH from the management network and any needed ICMP; log the rest at a limited rate; allow outbound by default. Conntrack remembers flows, so replies are accepted without opening inbound ports, and the check comes first because it matches almost every packet.

Answer. The complete rule set is in 8.12 (p. 510) (table `inet campus_fw`). Design decisions: (i) `policy drop` on `input` so that anything forgotten is closed; (ii) `ct state established,related accept` first, for speed and so that the administrator's own SSH session survives a reload; (iii) `ct state invalid drop`; (iv) a named set `mgmt` for the permitted source ranges, changed without editing rules; (v) ICMP limited to echo-request, destination-unreachable and time-exceeded at a limited rate, since blocking all ICMP breaks path-MTU discovery; (vi) a counter and a rate-limited log at the end; (vii) a `forward` chain with `policy drop` so that the host cannot be used as a router unless intended; (viii) load atomically with `nft -f`, and test from a second session with `at now + 5 minutes` rolling the rules back. **Follow-ups.** SSH hardening is in Pillar 9 (p. 538); the equivalent iptables rules differ only in syntax. **Mistakes.** Setting the policy to drop before adding the ESTABLISHED rule, which cuts off one's own session.

Q8.27 How do network namespaces work, and how does Docker give a container connectivity with veth, a bridge and NAT?

GOOGLE · AMAZON · MICROSOFT · NOKIA · TECH-2 · ■■■ · T

30-second answer: a network namespace is a private copy of interfaces, addresses, routes, ARP cache and firewall rules. Docker puts each container in its own namespace, connects it with a veth pair to a host bridge (`docker0`), and uses iptables MASQUERADE for outbound and DNAT for published ports.

Answer. Namespaces are one of the kernel's isolation primitives (alongside PID, mount, UTS, IPC, user and cgroup). A process belongs to one namespace of each kind; the network one gives it a separate stack, starting with only a down `lo`. A veth pair is a virtual cable: Docker moves one end into the container as `eth0` and attaches the other to the bridge, which acts as a switch among containers and as their gateway (172.17.0.1). Outbound packets leave through the host's routing table; a MASQUERADE rule in `nat POSTROUTING` rewrites their source to the host's address, and conntrack translates the replies back (8.12 (p. 510)). `-p 8080:80` adds a DNAT rule in PREROUTING from host port 8080 to the container's address. Figure 8.5 and the lab of 8.5 (p. 492) build the same structure by hand, with `rtr` as the bridge. **Follow-ups.** `nsenter -t PID -n` or `ip netns exec` to debug inside; host networking removes the namespace; Kubernetes pods share one namespace among containers; overlay networks encapsulate (VXLAN). **Mistakes.** Describing containers as lightweight VMs, which have their own kernel.

Q8.28 `strace` a `curl` that hangs. What do you expect to see, and how do you tell a DNS hang from a TCP connect hang?

AMAZON · GOOGLE · ARISTA · TECH-2 · ■■■ · T

30-second answer: `strace -f -tt -e trace=network,poll curl URL`. A DNS hang waits after a send to port 53 (or a connect to the resolver) and never reaches a connect to the target; a TCP hang shows `connect(…) = -1 EINPROGRESS` to the target followed by a poll that times out.

Answer. `curl` first loads configuration and looks for the name-service cache (`connect` to `/var/run/nscd/socket` failing with `ENOENT` is harmless), then resolves the name by `sendto` / `recvfrom` or `connect` on a UDP socket to the resolver. After resolution it creates a non-blocking socket and calls `connect` , which returns `EINPROGRESS` , then waits in `poll` / `ppoll` for writability; success is signalled by `getsockopt(SO_ERROR) = 0` , failure by an error such as `ECONNREFUSED` (immediate RST) or by the timeout alone (dropped SYN). If the hang is in `poll` after a UDP send to port 53, check `resolvectl status` and `dig` ; if it follows a `connect` to the target, switch to `tcpdump` to see whether a SYN-ACK returns (8.7 (p. 498)). **Follow-ups.** `strace -c` for a profile; `-T` for time per call; `ltrace` for library calls; compare `curl -w` timings (8.10 (p. 506)). **Mistakes.** Forgetting `-f` (misses child processes) and `-tt` (no timing).

Q8.29 Explain the `ip neigh` states. What does FAILED versus INCOMPLETE indicate?

CISCO · ARISTA · JUNIPER · TECH-1 · ■■■ · T

30-second answer: the states track whether a neighbour's MAC address is known and recently confirmed: INCOMPLETE (resolving, no reply yet), REACHABLE (confirmed), STALE (not recently confirmed), DELAY and PROBE (re-verifying), FAILED (resolution gave up). INCOMPLETE is in progress; FAILED is the verdict that nobody answered.

Answer. The full table is in 8.5 (p. 492). After an ARP request is broadcast, the entry sits in INCOMPLETE while up to three solicitations are sent (one a second by default); if none is answered it becomes FAILED, and packets for that address are discarded with "Destination Host Unreachable". For a gateway, FAILED or a long INCOMPLETE means a layer-2 problem: the gateway is down, in another VLAN, on a different subnet from what the mask implies, or filtering ARP. A REACHABLE entry ages to STALE after about 30 s, which is normal; on the next use the kernel goes DELAY (5 s) and PROBE (unicast ARP) to confirm. **Follow-ups.** Capture with `tcpdump -nn -e arp` ; duplicate IP addresses show as an entry whose `lladdr` changes; `arping -D` detects duplicates; IPv6 uses neighbour discovery with the same states. **Mistakes.** Treating STALE as a fault.

Q8.30 How would you implement policy-based routing so that traffic from 10.1.0.0/16 uses ISP-B and everything else uses ISP-A?

CISCO · JUNIPER · AMAZON · NOKIA · TECH-2 · ■■■ · T

30-second answer: put ISP-B's default route in a separate table (`ip route add default via <B> table 100`) and select it with a rule (`ip rule add from 10.1.0.0/16 lookup 100`); the main table keeps ISP-A as default. Verify with `ip route get … from …`.

Answer. The commands and a verified run are in 8.5 (p. 492). A production design adds four items. (i) **NAT per uplink:** masquerade on each outgoing interface, since each ISP expects its own addresses (`-t nat -A POSTROUTING -o ispB0 -j MASQUERADE`). (ii) **Reverse-path filtering:** with asymmetric paths set `net.ipv4.conf.all.rp_filter=2` (loose) or the reply may be discarded. (iii) **Failover:** the rules do not notice a dead ISP; a script or routing daemon pings through each link and replaces the default route or removes the rule. (iv) **Marks instead of sources**, when the selector is not an address: mark the packet and select on the mark.

```
iptables -t mangle -A PREROUTING -p tcp
--dport 443 -j MARK --set-mark 1
ip rule add fwmark 1 lookup 100 priority
1000
```

Follow-ups. Rules are checked in priority order and `main` is consulted last; routers do this with PBR or VRF route leaking (4.9 (p. 268)).
Mistakes. Forgetting that rules are lost on reboot unless written to the network configuration.

Revision Spread

THE 15-MINUTE VERSION

Ten facts. (1) Every tool is a window onto kernel state: `ip` and `ss` use netlink, `ethtool` the driver, `tcpdump` a packet tap, `/proc` the rest. (2) Pipes connect stdout to stdin; `2>&1` after `>file` sends both to the file; exit status 0 means success. (3) `sort | uniq -c | sort -rn | head` is the top-N idiom; `uniq` counts adjacent lines only. (4) Scripts begin `set -euo pipefail`, quote every `"$var"`, send errors to stderr and exit non-zero on failure. (5) `ip route get` shows the route actually chosen with its source address; `ip neigh FAILED` or `INCOMPLETE` is an ARP (layer 2) fault. (6) `ss -tlnp` shows who listens and on which address; many `CLOSE-WAIT` is an application bug; `TIME-WAIT` is normal on the side that closed first. (7) tcpdump captures before netfilter on ingress and after it on egress; `[S]` with no `[S.]` is a drop, `[R.]` is a closed port. (8) Loss at an mtr hop counts only if it persists to the last hop. (9) Counters are cumulative: take two readings and divide by the interval. (10) Throughput is at most window ÷ RTT; the window must reach bandwidth × RTT.

Three figures to redraw. The netfilter path (Figure 8.9), the three-namespace lab with its routing tables (Figure 8.5), and the anatomy of a tcpdump SYN line (Figure 8.6).

Is it the network? The command ladder

Layer	Question	Command	Healthy result
L1	Is there a link?	`ethtool eth0`, `ip -br link`	`Link detected: yes`, `LOWER_UP`, expected speed and full duplex
L1	Errors now?	`ip -s link` twice, `ethtool -S`	error counters not increasing
L2	Is the gateway's MAC known?	`ip neigh`, `arping -I eth0 GW`	`REACHABLE` or `STALE` with `lladdr`
L3	Address, mask, route?	`ip -br a`, `ip r`, `ip route get X`	via the expected gateway, right `src`
L3	Does the path work?	`ping -c3 X`, `mtr -rwnc 100 X`, `tracepath X`	loss ends at 0 % at the destination
L3	MTU ok?	`ping -M do -s 1472 X`	reply; otherwise "Frag needed (mtu = N)"
L4	Does the port answer?	`nc -zv X 443`, `ss -tan`	`succeeded`; `Connection refused` = closed, timeout = filtered
L4	Who listens locally?	`ss -tlnp`	right address and port for the service
DNS	Does the name resolve?	`dig +short X`, `dig @ns X`, `getent hosts X`	expected address; `NXDOMAIN` / `SERVFAIL` otherwise
Firewall	Is it filtered?	`nft list ruleset`, `iptables -L -n -v --line-numbers`, `conntrack -L`	counters rise on the expected rule
L7	Does the application answer?	`curl -sv -o /dev/null -w '%{time_connect} %{time_starttransfer}\n' URL`	2xx/3xx; timings in range
Host	Process, disk, logs?	`systemctl status`, `journalctl -u`, `df -h`, `lsof -i`, `strace -p`	active, space free, no errors

Commands by task

Task	Command
Listening ports with process	`ss -tulpn`
Connections to a port, by state	`ss -tan state established '( dport = :443 )'`
TCP internals (rtt, cwnd, retrans)	`ss -ti dst X`

	`END{for(s in c) print s,c[s]}'`
Interface counters / rates	`ip -s link show eth0`, `cat /proc/net/dev`, `nstat -az`
Legacy to modern	`ifconfig→ip a`, `route -n→ip r`, `arp -n→ip n`, `netstat→ss`, `brctl→ip link … type bridge`
Add address / route / namespace	`ip addr add 10.0.1.2/24 dev eth0`, `ip route add 10.0.2.0/24 via 10.0.1.1`, `ip netns add red`
Run inside a namespace	`ip netns exec red CMD`, `ip -n red a`
Policy routing	`ip route add default via GW table 100`, `ip rule add from P lookup 100`
Capture to file, rotate	`tcpdump -i eth0 -nn -s 0 -w f.pcap 'filter'`, `-G 300 -W 12`
Packets by DNS / HTTP	`tcpdump -nn port 53`, `tcpdump -nn -A port 80`
Trace the path (TCP)	`traceroute -n -T -p 443 X`, `mtr -rwnc 100 X`
Time a request	`curl -w '%{time_namelookup} %{time_connect} % {time_appconnect} % {time_starttransfer} % {time_total}\n' -o /dev/null -s URL`
Throughput test	`iperf3 -s`; `iperf3 -c X -P 4 -t 30 -R`; UDP: `-u -b 200M`
Persistent sysctl	`/etc/sysctl.d/90-x.conf` then `sysctl --system`
Who owns port 8080	`ss -tlnp 'sport = :8080'`, `lsof -nP -i :8080`
Trace system calls	`strace -f -tt -e trace=network,poll CMD`, `strace -c -p PID`
Logs of a service	`journalctl -u NAME --since "10 min ago" -f`
Disk	`df -h`, `du -xh –max-depth=1 / | sort -h | tail`, `lsof +L1`

Text-processing idioms

Need	Idiom
Top 10 clients	`awk '{print $1}' f | sort | uniq -c | sort -rn | head -10`
Group by field	`awk '{c[$9]++} END{for(k in c) print k,c[k]}' f`
Filter on a number	`awk '$9>=500 {print $1,$7,$9}' f`
Last field	`awk '{print $NF}' f`
Replace	`sed 's/old/new/g' f`, in place `sed -i`
Print matching part	`grep -oE 'Gi[0-9/]+' f`; context `grep -B2 -A2 PATTERN f`
Columns	`cut -d: -f1,3 /etc/passwd`
JSON	`jq -r '.items[] | select(.up==false) | .name'`
Count lines / uniques	`wc -l`; `sort -u | wc -l`

tcpdump (BPF) and Wireshark display filters

Goal	tcpdump (capture filter)	Wireshark (display filter)
One host	`host 10.0.1.2`	`ip.addr == 10.0.1.2`
One port	`port 443`	`tcp.port == 443`
Network	`net 10.20.0.0/16`	`ip.addr == 10.20.0.0/16`
Exclude SSH	`not port 22`	`!(tcp.port == 22)`
New connections	`tcp[tcpflags] & (tcp-syn|tcp-ack) == tcp-syn`	`tcp.flags.syn==1 && tcp.flags.ack==0`
Resets	`tcp[tcpflags] & tcp-rst != 0`	`tcp.flags.reset == 1`
Retransmissions	(not available)	`tcp.analysis.retransmission`
Zero window	(not available)	`tcp.analysis.zero_window`
Pings	`icmp[icmptype] == icmp-echo`	`icmp.type == 8`
DNS	`udp port 53`	`dns`; errors `dns.flags.rcode != 0`
HTTP errors	`tcp port 80` then read	`http.response.code >= 400`
TLS ClientHello	`tcp port 443`	`tls.handshake.type == 1`
VLAN 10	`vlan 10`	`vlan.id == 10`
One conversation	`host A and host B and port P`	`tcp.stream == N`
Slow gaps	`-ttt` option	`frame.time_delta > 1`

TCP states as traffic lights, and ICMP types

State	Light	Meaning
`LISTEN`, `ESTAB`	green	normal
`TIME-WAIT`	green	first closer waits 60 s; many = short connections
`SYN-SENT`	amber	waiting for SYN-ACK; many for long = filtered or dead peer
`SYN-RECV`	amber / red	many = SYN flood or full accept queue
`FIN-WAIT-2`	amber	peer has not closed its side
`CLOSE-WAIT`	red	application did not `close()`
`LAST-ACK`	amber	final ACK pending

ICMP type / code	Meaning	Seen when
0	echo reply	ping answered
3 / 0, 1	network, host unreachable	no route, ARP failure at a router
3 / 3	port unreachable	UDP to a closed port; ends a UDP traceroute
3 / 4	fragmentation needed, DF set	path MTU smaller than the packet
3 / 13	administratively prohibited	firewall reject
5	redirect	a better gateway exists
8	echo request	ping
11 / 0	time exceeded in transit	TTL reached zero (traceroute hop)

Netfilter, BDP and DNS in one place

Item	Rule
Forwarded packet	PREROUTING → FORWARD → POSTROUTING
Packet for the host	PREROUTING → INPUT; local output: OUTPUT → POSTROUTING
DNAT / SNAT	PREROUTING (before routing) / POSTROUTING (after)
Conntrack states	NEW, ESTABLISHED, RELATED, INVALID
First rule of a stateful firewall	`ct state established,related accept`
DROP / REJECT	silence and retries / immediate RST or ICMP error
BDP	bandwidth (bit/s) × RTT (s) ÷ 8 bytes
Window-limited rate	window × 8 ÷ RTT; 64 KB at 80 ms = 6.5 Mb/s
Mathis (loss-limited)	rate ≈ 1.22 × MSS ÷ (RTT × √p)
DNS status	NOERROR (empty = no such type), NXDOMAIN (no name), SERVFAIL (failed), REFUSED (policy)
DNS header flags	`qr` response, `aa` authoritative, `rd` / `ra` recursion desired / available, `ad` DNSSEC valid, `tc` truncated
Resolution order	`nsswitch.conf` `hosts: files dns` → `/etc/hosts`, then `resolv.conf`
Neighbour states	INCOMPLETE → REACHABLE → STALE → DELAY → PROBE → FAILED
Ephemeral ports	32768-60999 (28 232)

Flashcards

Q	A
`ip neigh` shows FAILED for the gateway	ARP got no reply: layer-2 problem

	(VLAN, cable, gateway down, wrong mask)
Which chain sees forwarded traffic?	FORWARD, after PREROUTING and before POSTROUTING
Filter for a pure SYN	`tcp[tcpflags] & (tcp-syn|tcp-ack) == tcp-syn`
mtr loss at a middle hop only	ICMP rate limit; ignore unless it persists to the destination
What does `ip route get X` add to `ip route`?	The chosen route, output interface and source address
`ss` option for listening sockets with PID	`-lp` (usually `ss -tulpn`)
Why `-n` in ss, tcpdump, traceroute	skips name lookups: faster, no hidden port numbers
Thousands of CLOSE-WAIT	the application is not closing sockets: a code bug
Which side holds TIME-WAIT?	The side that closed first, 60 s on Linux
Pipe versus redirect	pipe joins two commands; redirect connects a stream to a file
`cmd 2>&1 > f` versus `cmd > f 2>&1`	the first leaves stderr on the terminal; the second sends both to the file
Why quote `"$var"`	to stop word splitting and globbing; empty `$var` in `rm -rf $var/`
`set -euo pipefail`	exit on error, error on unset variable, fail a pipeline if any stage fails
Permissions 750	owner rwx, group r-x, others none
Sticky bit on /tmp	only the owner can delete their files in a shared directory
`kill` default signal	SIGTERM (15), catchable; SIGKILL (9) cannot be caught
`uniq -c` needs what first?	`sort`, because it counts adjacent duplicates only
Linux traceroute default probe	UDP to high ports; `-I` ICMP, `-T` TCP
Why does `* * *` appear?	no ICMP reply: filtered, rate-limited or not sent; not proof of a dead hop
ICMP 3/4	fragmentation needed with DF set: path MTU discovery
Ping with DF to test MTU	`ping -M do -s 1472 X` (payload + 28 = packet size)
DNS: NXDOMAIN vs SERVFAIL	name absent vs failed lookup
`dig +cd` succeeds, plain `dig` fails	DNSSEC validation failure
`dig +trace` shows	root, TLD and authoritative referrals walked iteratively
Does `dig` read /etc/hosts?	No; use `getent hosts` for what applications see
tcpdump on ingress is before or after the firewall?	before; on egress, after
64 KB packet seen on a 1500-MTU host	TSO/GSO: the tap sees the buffer before it is split
`ethtool -S` counters	cumulative since boot: take a delta
Duplex mismatch symptom	late collisions, CRC errors, speed falls under load
REJECT vs DROP	immediate error vs silence
`MASQUERADE` vs `SNAT`	masquerade uses the outgoing interface's current address
Why ESTABLISHED,RELATED first	matches most packets and lets replies in without open inbound ports
BDP of 1 Gb/s, 80 ms	10 MB
`sysctl -w` persistence	none; write `/etc/sysctl.d/*.conf` and `sysctl --system`
Find the process on port 8080	`ss -tlnp 'sport = :8080'` or `lsof -nP -i :8080`
DNS hang vs TCP hang in strace	wait after UDP send to :53 vs `connect` to target returning EINPROGRESS then a long poll
Policy routing in two lines	`ip route add default via B table 100`; `ip`

```
rule add from P lookup 100
```

PART I · PILLAR 9

9

Network Security

Locks, keys and liars: we name every major attack from Layer 2 to Layer 7, the control that stops it, and walk through TLS 1.3 and IKEv2 packet by packet.

9.1 Security mindset: CIA, AAA, STRIDE, defence in depth	SHOULD
9.2 Cryptography you must be able to explain	MUST
9.3 PKI and certificates	MUST
9.4 The TLS 1.3 handshake, step by step	MUST
9.5 ACLs: stateless packet filtering	MUST
9.6 Stateful firewalls, NGFW and NAT interplay	MUST
9.7 IPsec and IKE	MUST
9.8 VPN families and remote access	SHOULD
9.9 Layer-2 attacks and switch hardening	MUST
9.10 Layer-3 and Layer-4 attacks, and DDoS	MUST
9.11 DNS security	SHOULD
9.12 AAA, RADIUS, TACACS+ and 802.1X	SHOULD
9.13 Detection and device hardening	OPTIONAL
9.14 Zero trust	SHOULD

This chapter assumes the protocols of Pillar 3 (p. 162) (ARP, DHCP, TCP, DNS) and the switching and routing of Pillar 4 (p. 229); it adds the adversary. The examples belong to **Campus Net**, the fictional university network of Pillar 4: SALES (VLAN 10, 192.168.10.0/24) and ENG (VLAN 20, 192.168.20.0/24) on the office floor, a server network 10.0.0.0/24 holding the web server 10.0.0.80, two border routers BR1 and BR2, and the public address 203.0.113.5 used for translation. NAT itself is taught in Section 3.6 (p. 183) and IOS access-list syntax in Section 4.13 (p. 279); here we teach the reasoning behind them. Candidate reports show that Cisco, Juniper and Nokia ask about ACLs, IPsec and Layer-2 hardening, whereas Google, Microsoft and Amazon ask about TLS, DNS and DDoS; these are patterns reported by candidates, not official syllabi.

9.1 Security mindset: CIA, AAA, STRIDE, defence in depth

SHOULD · ■□□

BIG IDEA

Security is the management of risk, not the removal of it, because every control costs money or convenience and every asset has a finite value; we therefore stack independent controls so that one failure is not fatal.

Three properties define what is being protected. *Confidentiality* means that only authorised parties can read the data; *integrity* means that it cannot be altered undetected; *availability* means that it can be reached when needed. Together they form the *CIA triad*. Three further services answer who is asking: *authentication* proves identity, *authorisation* decides what that identity may do, and *accounting* records what it did; the three are called *AAA* (Section 9.12). *Non-repudiation* adds that the sender cannot later deny having sent a message, which only a digital signature gives (Section 9.2).

A breach of each property has a network example. A packet capture on shared Wi-Fi breaks confidentiality; a router that rewrites a downloaded file breaks integrity; a SYN flood against the web server breaks availability. Note that the three can conflict: encrypting everything raises confidentiality but blinds the intrusion-detection system, and replicating data raises availability but multiplies the places it can leak.

To reason about attackers systematically we use *STRIDE*, a threat classification from Microsoft. Each letter is a way of violating one security property, and each has a standard family of controls.

Bank branch — guard at the door (firewall), CCTV (IDS), locker keys (encryption), visitor register (logging), two keys for the vault (separation of duty).

Threat	Violates	Campus Net example	Typical control
Spoofing	authentication	forged ARP reply for the gateway	DAI, 802.1X, certificates
Tampering	integrity	modified packet on an unencrypted link	TLS, IPsec, HMAC
Repudiation	non-repudiation	an admin denies deleting a VLAN	AAA accounting, signed logs
Information disclosure	confidentiality	Telnet password sniffed on a trunk	SSH, TLS, encryption at rest
Denial of service	availability	SYN flood on 10.0.0.80	SYN cookies, rate limits, scrubbing
Elevation of privilege	authorisation	a student account gains enable access	least privilege, TACACS+ command sets

A *threat model* lists the assets, the entry points to them and one threat per STRIDE letter with its control. It is the first step of any design, because a control without a named threat cannot be justified or tested.

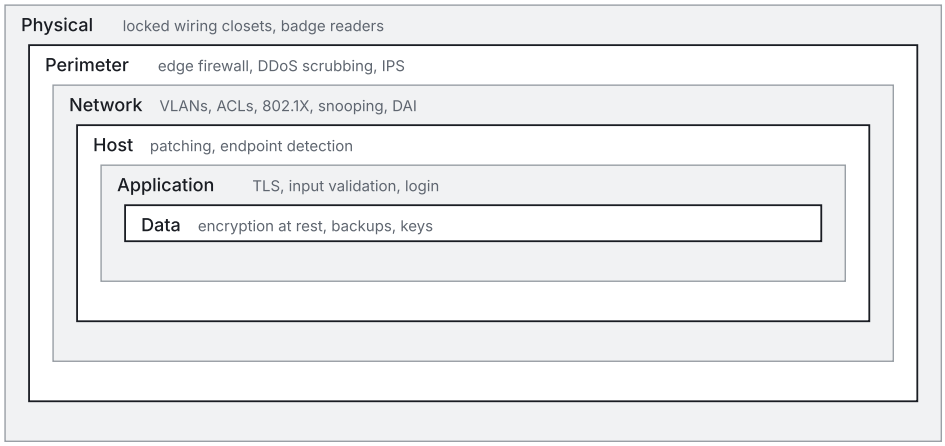


Figure 9.1 Defence in depth for Campus Net: independent controls on every ring, so that the failure of one does not expose the data.

Defence in depth places controls on several rings (Figure 9.1), each assumed to fail some day. The rings should be independent, meaning that one flaw (a stolen password, say) does not defeat two rings at once. Further, each ring should fail closed where possible and should log, so that the next ring and the operators learn of the failure.

WORKED EXAMPLE

A threat model for the Campus Net Wi-Fi and lab network. Assets: student records on 10.0.0.0/24, the lab switches, and the guest Wi-Fi's reputation. Entry points: the Wi-Fi, the lab sockets, the VPN and the public web server.

1. Spoofing: someone joins the staff SSID with a shared password. Control: WPA3-Enterprise with 802.1X, so that identity is per user (Section 9.12).
2. Tampering: a rogue DHCP server hands out a malicious gateway. Control: DHCP snooping on access ports (Section 9.9).
3. Repudiation: a technician changes an ACL and denies it. Control: TACACS+ accounting to a syslog server.
4. Information disclosure: the lab's HTTP management page sends passwords in clear. Control: HTTPS only, HTTP disabled (Section 9.13).
5. Denial of service: a student floods the DHCP pool. Control: DHCP rate limiting and port security.
6. Elevation of privilege: a guest laptop reaches the server VLAN. Control: guest VLAN with an ACL or firewall rule denying 10.0.0.0/24.

Result: six threats, six controls, each with an owner and a test; the table is the audit trail.

INTERVIEW LENS

CISCO · AMAZON · MICROSOFT

"Name the CIA triad and give a network breach of each" is a warm-up; "How would you secure a campus?" is the real question. A strong answer names the assets first, then the rings of Figure 9.1, then one control per threat, and ends with logging and a way to test each control.

9.2 Cryptography you must be able to explain MUST · ■■□

BIG IDEA

Every secure protocol in this chapter is built from five primitives (symmetric ciphers, hashes, MACs, signatures and key exchange), because each solves exactly one problem that the others cannot.

Symmetric encryption uses one secret key for both directions. It is fast (AES runs at many gigabits per second, helped by hardware instructions), so it carries all bulk data. Its problem is distribution: both parties must already share the key. A *block cipher* such as AES transforms 128-bit blocks; a *mode of operation* says how to handle longer messages. ECB encrypts each block alone, so equal plaintext blocks give equal ciphertext blocks and patterns survive; it must never be used. CBC chains blocks through an initialisation vector (IV) but gives no integrity. CTR turns the cipher into a stream. GCM is CTR with an authentication tag, termed *authenticated encryption with associated data* (AEAD): one operation gives confidentiality and integrity. ChaCha20-Poly1305 is the other AEAD in common use, faster than AES where hardware support is missing. Note that GCM fails badly if a key and nonce pair is ever reused, so protocols derive nonces from counters.

Hash functions map any input to a fixed-size digest (SHA-256 gives 256 bits). Three properties matter: given a digest one cannot find an input (*pre-image resistance*), cannot find a second input with the same digest, and cannot find any colliding pair. MD5 and SHA-1 are broken for collisions and are not used for security. A hash alone does not authenticate a message, because an attacker who alters the message can recompute the hash. For stored passwords a fast hash is also wrong; one uses a salted, deliberately slow function such as bcrypt or Argon2.

HMAC fixes this by mixing a shared key into the hash: HMAC(K, m) = H((K $\oplus$ opad) ‖ H((K $\oplus$ ipad) ‖ m)). Only holders of K can produce the tag, so it gives integrity and origin authentication. The nested form is used because the naive H(K ‖ m) is open to *length-extension* attacks on SHA-2. HMAC is how IPsec (older suites), TLS 1.2 non-AEAD suites and many API signatures work.

Asymmetric cryptography gives each party a *key pair*: a public key that may be published and a private key that is never shared. RSA rests on the difficulty of factoring large numbers; elliptic-curve cryptography (ECC) rests on the discrete logarithm on a curve and reaches the same strength with much smaller keys (a 256-bit curve matches about 3072-bit RSA). Asymmetric operations are around a thousand times slower than AES, hence the standard design: use them briefly to agree on keys or prove identity, then switch to symmetric encryption.

A **digital signature** proves who produced a message. The signer hashes the message and signs the digest with the private key; anyone verifies it with the public key (Figure 9.2). Encryption and signing use the pair in opposite directions: to send a secret to Bob one encrypts with Bob's public key, and to sign one uses one's own private key.

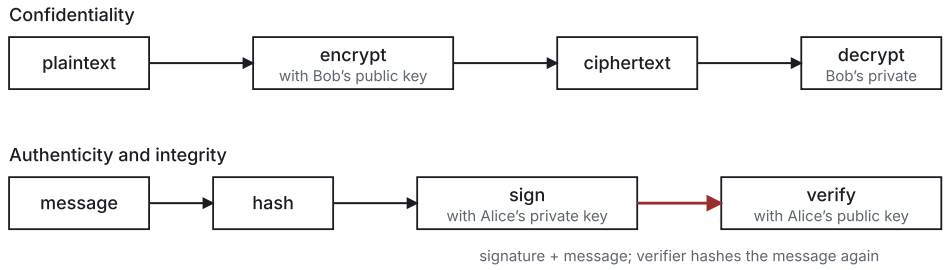


Figure 9.2 Encryption and signature use the key pair in opposite directions: encrypt to the public key, sign with the private key.

TRAP

Claim: a signature is "encryption with the private key". *Why it is wrong:* this holds only for textbook RSA; ECDSA and EdDSA are not encryption at all, and real RSA signatures add padding (PSS). *Correct statement:* one signs with the private key and verifies with the public key; one encrypts with the public key and decrypts with the private key.

Diffie-Hellman (DH) lets two parties create a shared secret over a public channel. Both agree on public numbers: a prime p and a generator g. Each picks a secret, publishes g raised to it modulo p, and raises the other's value to its own secret. Since $(g^a)^b = (g^b)^a$, both reach the same number, while an eavesdropper who sees only the published values must solve the *discrete logarithm problem*, which is infeasible for large p. In the paint-mixing analogy, each side mixes a public colour with a private one, swaps the mixtures, and adds the private colour again; nobody can unmix the shared result.

WORKED EXAMPLE

Toy Diffie-Hellman. Take p = 23, g = 5. The client chooses a = 6 and the server b = 15.

1. Client publishes $A = 5^6 \bmod 23$. Now $5^2 = 25 \equiv 2$, so $5^6 \equiv 2^3 = 8$. A = 8.
2. Server publishes $B = 5^{15} \bmod 23$. Using $5^2 \equiv 2$, $5^4 \equiv 4$, $5^8 \equiv 16$: $5^{15} = 5^8 \cdot 5^4 \cdot 5^2 \cdot 5 \equiv 16 \cdot 4 \cdot 2 \cdot 5 = 640 \equiv 19$. B = 19.
3. Client computes $B^a = 19^6 \bmod 23$. Since $19 \equiv -4$, $(-4)^6 = 4096 \equiv 2$.
4. Server computes $A^b = 8^{15} \bmod 23$. Since $8 = 2^3$ and $2^{11} \equiv 1 \pmod{23}$, $8^{15} = 2^{45} \equiv 2^1 = 2$.

Result: shared secret K = 2 on both sides; Eve sees only 23, 5, 8 and 19.

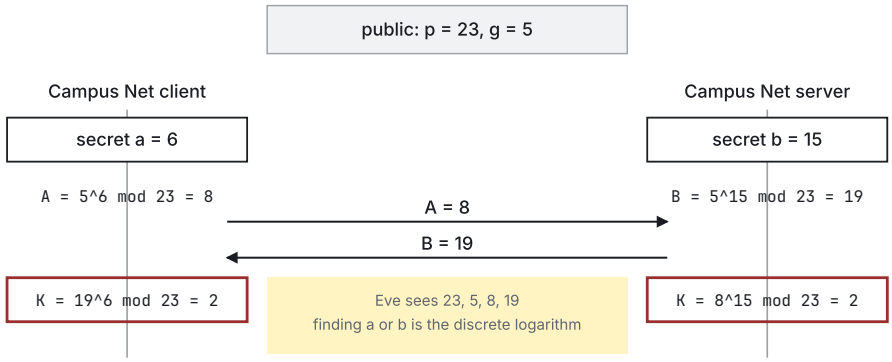


Figure 9.3 Diffie-Hellman with p = 23 and g = 5. The eavesdropper sees p, g, 8 and 19 only; both ends compute 2.

With p = 23 an attacker simply tries all exponents. With a 2048-bit prime, or an elliptic curve such as X25519, the search space is far beyond reach. The shared number is never used directly as a key; a *key derivation function* (HKDF in TLS 1.3) expands it into the keys needed. In *ephemeral* DH (DHE, or ECDHE on curves) fresh secrets are generated for every session and discarded afterwards. This gives *forward secrecy*: if the server's long-term private key is stolen later, recorded sessions still cannot be decrypted, because the session secrets no longer exist. DH alone does not authenticate, so an active attacker can sit in the middle; the signature on the handshake (Section 9.4) closes that gap.

Primitive	Purpose	Example	What breaks if misused
Symmetric cipher	confidentiality	AES-GCM, ChaCha20-Poly1305	ECB leaks patterns; nonce reuse in GCM leaks the key stream
Hash	integrity fingerprint	SHA-256	MD5, SHA-1 collisions; fast hashes for passwords
HMAC	integrity and origin	HMAC-SHA256	a short or reused key; H(K ‖ m) instead of HMAC
Signature	origin, non-repudiation	ECDSA, Ed25519, RSA-PSS	reused nonce in ECDSA reveals the key
Key exchange	shared secret	ECDHE (X25519)	no authentication: man in the middle; static keys: no forward secrecy
Key pair encryption	key transport	RSA-OAEP	RSA key transport has no forward secrecy; removed from TLS 1.3

Quantum computers running Shor's algorithm would break RSA, DH and ECC; hybrid key exchanges that combine X25519 with a lattice scheme (ML-KEM) are already deployed in browsers. Symmetric ciphers and hashes are weakened only by doubling the key length.

> CHECK YOURSELF
>
> 1. Why does TLS use both symmetric and asymmetric cryptography? [1]
> 2. Why is HMAC preferred to H(K ‖ m)? [2]
> 3. What does ephemeral DH give that static RSA key transport does not? [3]

9.3 PKI and certificates MUST · ■■□

> BIG IDEA
>
> A certificate binds a public key to a name by a signature from a party the verifier already trusts, so that trust in one root key extends to millions of servers it has never met.

Public-key encryption has a gap: when the client receives "my public key is K", how does it know that K belongs to the site and not to an attacker? The answer is a *public key infrastructure* (PKI). A *certificate authority* (CA) checks that the applicant controls the name and signs a document that lists the name and the key. That document is an *X.509 certificate*. The Aadhaar analogy

fits: a central authority (the root CA) authorises enrolment agencies (intermediate CAs), who issue the cards (leaf certificates), and a verifier checks the chain of authority and not only the face.

X.509 field	Meaning
Subject, Subject Alternative Name (SAN)	the identity; browsers check the SAN list (DNS names) and ignore the old Common Name
Issuer	the CA that signed it
Validity (notBefore, notAfter)	the dates; public certificates now last at most about 13 months and the maximum is falling
Public key	the key being certified (RSA or ECDSA)
Basic Constraints	CA:TRUE for authorities, with an optional path length limit
Key Usage, Extended Key Usage	permitted uses, such as digitalSignature and serverAuth
CRL and OCSP locations, SCTs	where revocation is checked; proof of logging in Certificate Transparency
Signature	the issuer's signature over all the above

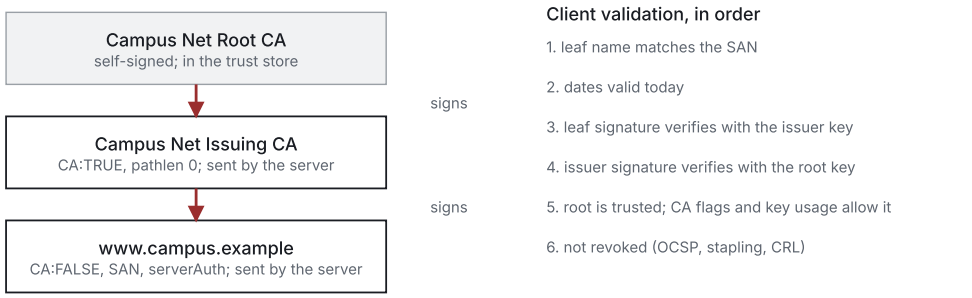


Figure 9.4 The chain of trust. The browser holds only the root; the server sends the leaf and the intermediate.

Browsers and operating systems ship a *trust store* of root certificates. A root is self-signed, so its authority comes from being in the store and not from any signature. Roots are kept offline and sign only a few *intermediate* CAs; the intermediates issue the leaf certificates. This limits the damage of a compromise, because an intermediate can be revoked without replacing the root in every device. The server sends its leaf and the intermediates, and the client builds the path to a root it holds (Figure 9.4).

Validation checks the six conditions in the figure. *Revocation* handles keys that leak before expiry. A *certificate revocation list* (CRL) is a signed list of serial numbers, large and slow to fetch. The *Online Certificate Status Protocol* (OCSP) asks the CA about one certificate, but leaks browsing to the CA and adds a round trip; hence *OCSP stapling*, in which the server fetches a signed, time-stamped response and attaches it to the handshake. The *must-staple* flag makes stapling mandatory. *Certificate Transparency* requires CAs to log every certificate in public append-only logs, whose signed timestamps (SCTs) appear in the certificate, so that a mis-issued certificate for campus.example can be detected by its owner. *Pinning*, in which an application accepts only a specific key, defends against rogue CAs but breaks easily when keys rotate; browsers have abandoned it for CT. *ACME*, the protocol of Let's Encrypt, automates issuance: the applicant proves control of a domain by serving a token over HTTP or by publishing a DNS record, receives a certificate valid for 90 days, and renews it by script. In *mutual TLS* (mTLS) the client

also presents a certificate, which is how service-to-service traffic and managed devices authenticate.

WORKED EXAMPLE

Reading a certificate. `openssl x509 -in leaf.crt -noout -text` prints the fields of the table above. The lines that matter for the Campus Net leaf are:

```
Issuer: O = Campus Net, CN = Campus Net Issuing CA
    Not Before: Oct 29 13:37:50 2026 GMT
    Not After : Jan 27 13:37:50 2027 GMT
Subject: CN = www.campus.example
    Public Key Algorithm: id-ecPublicKey   (ASN1 OID: prime256v1)
    CA:FALSE
    Digital Signature
    TLS Web Server Authentication
    DNS:www.campus.example
Signature Algorithm: ecdsa-with-SHA256
```

1. The issuer is the intermediate, not the root: the chain has three certificates.
2. The validity window is 90 days, so renewal must be automated (ACME).
3. `CA:FALSE` forbids this certificate from signing others, and the key usage limits it to signing handshakes for a web server.
4. Only the SAN `www.campus.example` is valid; a request for `campus.example` would fail the name check.

Result: five lines decide whether a client accepts the certificate: issuer chain, dates, SAN, CA flag and usage.

LAB

`LAB 9.1 · 30 MIN` Build a two-level CA for Campus Net and verify a server certificate. **Use only machines you own; unauthorised access to a computer system is an offence under sections 43 and 66 of the Indian IT Act 2000, and the attack labs of this chapter must never be run on a college or public network.**

```
openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:prime256v1 -nodes \
  -keyout root.key -out root.crt -days 365 \
  -subj "/O=Campus Net/CN=Campus Net Root CA"
# intermediate: CSR, signed by the root (basicConstraints CA:TRUE,pathlen:0)
# leaf: CSR for www.campus.example, signed by the intermediate, SAN in ext-
file
openssl verify -CAfile root.crt leaf.crt
openssl verify -CAfile root.crt -untrusted int.crt leaf.crt
openssl s_server -accept 8446 -cert leaf.crt -cert_chain int.crt \
  -key leaf.key -www &
openssl s_client -connect 127.0.0.1:8446 -servername www.campus.example \
  -CAfile root.crt -verify_hostname www.campus.example
```

```
error 20 at 0 depth lookup: unable to get local issuer certificate
leaf.crt: OK                          (second command, with -untrusted)
 0 s:CN = www.campus.example
   i:O = Campus Net, CN = Campus Net Issuing CA
 1 s:O = Campus Net, CN = Campus Net Issuing CA
   i:O = Campus Net, CN = Campus Net Root CA
Verification: OK
```

Now start the server with `-cert leaf.crt` only (no `-cert_chain`): the client reports `verify error:num=21:unable to verify the first certificate`.

What you should notice: the leaf alone cannot be validated, because the client has the root but not the intermediate. Many browsers cache intermediates or fetch them from the AIA extension, so the same site works in a browser and fails in `curl` or a Java client; the fix is to send the full chain. Against a public site, `-showcerts` prints the same structure, with `s:` the subject and `i:` the issuer of each certificate.

CHECK YOURSELF

1. Why does a browser trust a root certificate that is self-signed? [4]
2. What does OCSP stapling remove? [5]

9.4 The TLS 1.3 handshake, step by step MUST · ■■■

BIG IDEA

TLS 1.3 completes in one round trip because the client guesses the key-exchange group and sends its key share in the very first message, so the server can derive keys and start encrypting at once.

Transport Layer Security (TLS) gives a TCP connection confidentiality, integrity and server authentication (RFC 8446). It runs above TCP, so the TCP handshake of Pillar 3 (p. 162) comes first. Think of booking a sealed conversation: in version 1.3 you bring your half of the room key in your first sentence, so the receptionist can lock the door at once. The handshake has four goals: agree on a cipher suite, agree on keys by ECDHE (Section 9.2), prove the server's identity with a certificate (Section 9.3), and confirm that nobody altered the negotiation.

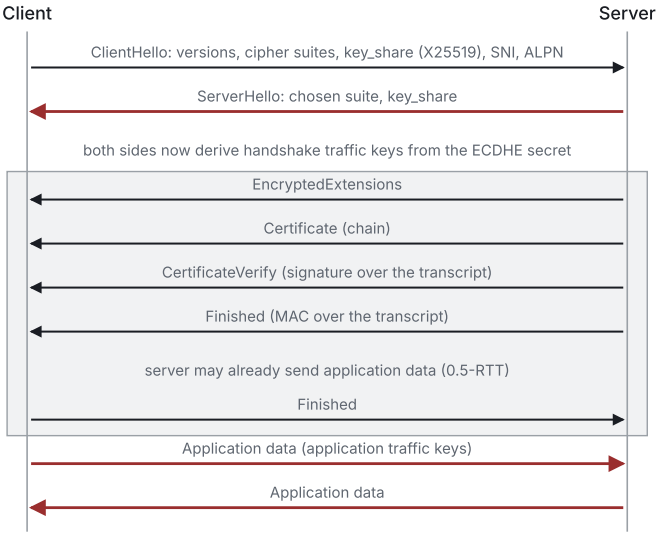


Figure 9.5 The TLS 1.3 full handshake. The shaded part is encrypted; the application can send data after its own Finished.

1. **ClientHello.** The client lists the versions it accepts (`supported_versions`), its cipher suites, the groups it supports and, as a guess, a *key share* for one of them, usually an X25519 public value. Further extensions carry the *server name indication* (SNI), the signature algorithms it accepts and ALPN, the application protocol wanted (`h2`, `http/1.1`). The message is in clear.

1. Asymmetric operations are slow but need no shared secret; symmetric ones are fast but need one. TLS uses the first to agree a key and the second for the data. ↩
2. H(K ‖ m) allows length extension on SHA-2: an attacker who knows the tag for m can compute a valid tag for m ‖ padding ‖ x without K. ↩
3. Forward secrecy: the session keys are never derived from a long-term key, so stealing that key later does not expose recorded traffic. ↩
4. It does not trust the signature; it trusts the root's presence in its trust store, which the vendor curates. ↩
5. The client's separate query to the CA, which saves a round trip and stops the CA learning which sites the client visits. ↩

2. **ServerHello.** The server picks one suite and answers with its own key share. From this moment both sides hold the ECDHE shared secret and, through the key schedule (HKDF), derive the *handshake traffic keys*. Everything the server sends from now on is encrypted. A legacy ChangeCipherSpec record is also sent, only so that middleboxes that expect the TLS 1.2 shape do not drop the connection.
3. **EncryptedExtensions.** The remaining negotiated parameters, such as the chosen ALPN protocol, now travel encrypted.
4. **Certificate.** The server's leaf and intermediates, encrypted, so a passive observer no longer sees which certificate was presented.
5. **CertificateVerify.** A signature, made with the certificate's private key, over a hash of the whole handshake so far. This is the proof of identity: only the key holder can produce it, and it binds the key exchange to the certificate, which defeats a man in the middle who relays the ECDHE values.
6. **Finished (server).** A MAC over the transcript under a key from the handshake secret. It confirms that both sides saw the same messages, so a downgrade attempt would be detected. The server derives the *application traffic keys* and may send data immediately.
7. **Finished (client).** The client verifies items 5 and 6, then sends its own Finished and its first request under the application keys. One round trip has elapsed.

If the server does not support the group guessed in the key share, it replies with a *HelloRetryRequest* naming a group it accepts, and the client sends a new ClientHello: this costs one extra round trip. A client certificate (mutual TLS) is requested by a CertificateRequest message after EncryptedExtensions and answered by the client's Certificate and CertificateVerify before its Finished.

DEEP DIVE · MAY BE SKIPPED

The key schedule in four lines. TLS 1.3 chains HKDF extractions. (i) The *early secret* comes from the PSK (zeros if there is none). (ii) The *handshake secret* mixes in the ECDHE shared secret; from it, hashes of the transcript up to ServerHello give the client and server *handshake traffic secrets*. (iii) The *master secret* is extracted next; hashes of the transcript up to the server's Finished give the *application traffic secrets*. (iv) Each direction expands its traffic secret into a record key and IV, and into a *finished key* for the Finished MAC. Because every secret depends on the transcript, a tampered ClientHello changes all later keys and the Finished check fails.

Resumption avoids the full handshake. After the first connection the server sends a *NewSessionTicket* that carries a pre-shared key (PSK) value. On a later connection the client offers it in the ClientHello; if accepted, no certificate is exchanged, and with a fresh ECDHE share forward secrecy is kept. With a PSK the client may also send *0-RTT early data* in its first flight, encrypted under a key from the PSK. This gives zero extra round trips, but early data has no forward secrecy and can be *replayed*: an attacker records the first flight and sends it again, and the server may process it twice. Hence servers should accept 0-RTT only for idempotent requests (a GET, never a payment).

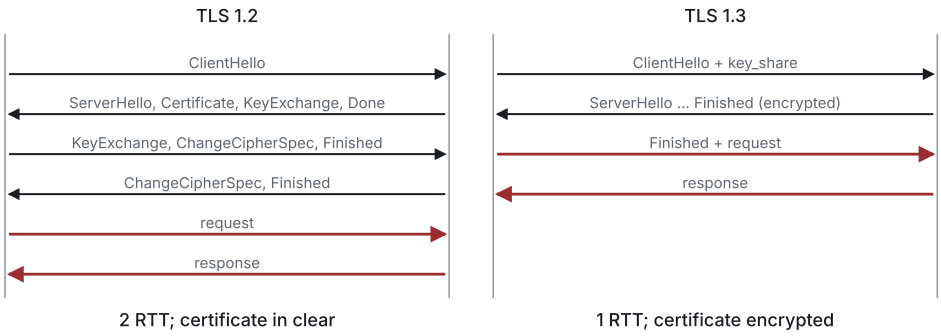


Figure 9.6 Round trips before the first request: two for TLS 1.2, one for TLS 1.3 (each after the TCP handshake).

	TLS 1.2	TLS 1.3
Round trips before the request	2	1 (0 with PSK and early data)
Key exchange	RSA key transport or (EC)DHE	(EC)DHE or PSK only
Forward secrecy	only with DHE suites	always, for certificate-based handshakes
Certificate visible to observers	yes	no, it is encrypted
Cipher suites	many, including CBC, RC4, 3DES	five AEAD suites (AES-GCM, ChaCha20-Poly1305, AES-CCM)
Renegotiation, compression	present, and a source of attacks	removed
Signature over handshake	ServerKeyExchange only	CertificateVerify over the entire transcript

TRAP

Claim: HTTPS hides which site a user visits. *Why it is wrong:* the SNI in the ClientHello is plaintext unless Encrypted Client Hello (ECH) is used, and the DNS lookup may also leak the name. *Correct statement:* TLS 1.3 hides the certificate and the content; it hides the destination name only with ECH and encrypted DNS (Section 9.11), and the destination address is always visible.

LAB

`LAB 9.2 · 20 MIN` Watch the messages of TLS 1.3 with `-msg`, using the server of Lab 9.1.

```
openssl s_client -connect 127.0.0.1:8446 -servername www.campus.example \
  -CAfile root.crt -tls1_3 -msg 2>&1 | grep 'Handshake \['
```

(Add `-tls1_2` for the TLS 1.2 contrast: the server's certificate then appears in clear, and a `ServerKeyExchange` message precedes `ServerHelloDone`.)

```
>>> TLS 1.3, Handshake [length 00f7], ClientHello
<<< TLS 1.3, Handshake [length 007a], ServerHello
<<< TLS 1.3, Handshake [length 0006], EncryptedExtensions
<<< TLS 1.3, Handshake [length 03cc], Certificate
<<< TLS 1.3, Handshake [length 004e], CertificateVerify
<<< TLS 1.3, Handshake [length 0034], Finished
>>> TLS 1.3, Handshake [length 0034], Finished
```

For a browser-style capture of an HTTPS request, set `SSLKEYLOGFILE=$HOME/keys.log`, run `curl https://www.campus.example/`, capture with `tcpdump -w tls.pcap port 443`, and give Wireshark the log under Preferences, Protocols, TLS, "(Pre)-Master-Secret log filename".

What you should notice: only ClientHello and ServerHello are readable on the wire; the other five messages sit in encrypted records, and OpenSSL prints them because it holds the keys. The `Certificate` is by far the longest (0x3cc = 972 bytes). With the key log, Wireshark shows the same list under each encrypted record, which is the paper variant of this lab.

INTERVIEW LENS

GOOGLE · MICROSOFT · AMAZON

"Walk me through the TLS 1.3 handshake. Where does encryption start?" Give the seven steps above, say that encryption starts immediately after the ServerHello, and explain why this saves a round trip. Expect: "What was removed from 1.2?" (RSA key transport, renegotiation, weak ciphers), "What is 0-RTT's risk?" (replay) and "What if the certificate is expired or names another host?" (the client aborts at validation with a fatal alert).

9.5 ACLs: stateless packet filtering MUST · ■■□

BIG IDEA

An access control list judges every packet on its own header fields against an ordered list, so it is fast and simple, but it has no memory of conversations and therefore cannot know that a packet is a reply.

An *access control list* (ACL) is an ordered list of permit and deny rules, applied to an interface in one direction. Think of a bouncer with a printed list: he reads it from the top, stops at the first line that names you, and if you are not on it you are out. Four rules govern every ACL. (i) The list is read top to bottom and the *first match* decides; later lines are never consulted. (ii) An invisible *implicit deny any* ends every list, so a list of denies alone blocks everything. (iii) An ACL examines only packets passing through the interface in the direction applied: an outbound ACL does not filter traffic that the router itself originates. (iv) One ACL may be applied per interface, per direction and per protocol (IPv4 and IPv6 separately). The syntax, numbering and verification commands are in Section 4.13 (p. 279); here we concentrate on design.

	Standard ACL	Extended ACL
Matches	source address only	protocol, source, destination, ports, flags
Numbers (classic)	1 to 99, 1300 to 1999	100 to 199, 2000 to 2699
Placement	close to the **destination**	close to the **source**
Reason	it cannot name the destination, so near the source it would block the source's other traffic as well	it can name everything, so dropping early saves bandwidth on the path

The placement rule follows from what each type can express. A standard ACL that denies ENG from reaching the servers, if placed on ENG's own interface, would also deny ENG access to the Internet, because it sees only the source. Placed on the server-facing port, it affects only what

reaches the servers. An extended ACL has no such problem and is better near the source, where it saves the transit links and the core routers the work (Figure 9.7).

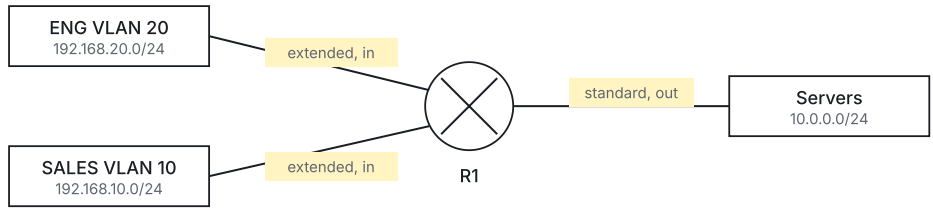


Figure 9.7 ACL placement at R1 in Campus Net: the extended ACL near the source, the standard ACL near the destination.

A *wildcard mask* says which address bits must match: 0 means "must match", 1 means "ignore". It is the inverse of the subnet mask for a contiguous block, so one writes 255 minus each mask octet. Because it is an independent bit pattern, it can also be discontiguous, which a subnet mask cannot: `192.168.1.0 0.0.254.255` matches every address with an odd third octet and any fourth octet, a trick used to select odd-numbered VLAN subnets. Note that a wildcard mask is not a subnet mask; confusing the two gives a rule that matches a different set of addresses.

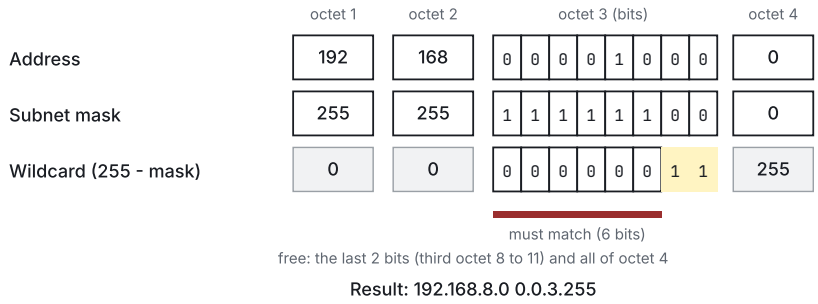


Figure 9.8 Wildcard mask for 192.168.8.0/22. Only the first 22 bits must match; the last 10 are free.

WORKED EXAMPLE

An ACL for ENG. Policy: ENG (192.168.20.0/24) may use HTTP and HTTPS to the web server 10.0.0.80 and may ping it; everything else addressed to the server network 10.0.0.0/24 is denied; all other traffic is permitted.

1. Choose an extended ACL because the policy names protocols, ports and a destination. Place it *inbound* on R1's ENG-facing interface (close to the source). Replies from the server enter R1 on another interface and are never filtered by this list, so no `established` line is needed.
2. Order the lines from the most specific to the most general. The permits for the server come first; the deny for the whole server network follows; the permit for everything else is last.
3. Write the wildcard for 10.0.0.0/24 as 0.0.0.255 and for a single host as `host`.

```
ip access-list extended ENG-IN
 10 permit tcp 192.168.20.0 0.0.0.255 host 10.0.0.80 eq 80
 20 permit tcp 192.168.20.0 0.0.0.255 host 10.0.0.80 eq 443
 30 permit icmp 192.168.20.0 0.0.0.255 host 10.0.0.80 echo
 40 deny   ip any 10.0.0.0 0.0.0.255
 50 permit ip any any
```

4. Test it by tracing eight flows through the list, as in the table.

Flow from ENG	First match	Result
TCP 443 to 10.0.0.80	line 20	permit
TCP 22 to 10.0.0.80	line 40	deny
ping 10.0.0.80	line 30	permit
ping 10.0.0.81	line 40	deny
TCP 443 to 10.0.0.81	line 40	deny
UDP 53 to 8.8.8.8	line 50	permit
TCP 443 to SALES host 192.168.10.5	line 50	permit (intended? the policy never mentioned SALES)
TCP 443 to 10.0.0.80 with source 192.168.99.9	line 40	deny (not in ENG's range)

Result: the list implements the policy; the seventh row shows that a final `permit ip any any` makes the list a deny-list, so every unlisted path is open.

TRAP

Claim: adding `permit ip any any` early "just in case" does no harm. *Why it is wrong:* the first match decides, so every line after it becomes dead code. *Correct statement:* place specific lines first, the general permit last, and check the hit counts (`show access-lists`), since a line with zero matches may be shadowed.

Three limits follow from statelessness. First, return traffic must be allowed explicitly: an inbound ACL on an Internet-facing interface needs `permit tcp any any established` (which matches ACK or RST, and so is easily forged) or a stateful device. Second, an ACL cannot follow protocols that open dynamic ports, such as active FTP. Third, an ACL cannot see the application. On management lines it works well: `access-class 10 in` on the VTY lines with a standard list limits who may log in. IPv6 lists end with implicit permits for neighbour discovery before the final deny, since ND replaces ARP. ACLs are executed in hardware (TCAM) on switches and routers, at line rate, which is why they remain the first filter for DDoS traffic and for Layer-3 segmentation, while the firewall of Section 9.6 handles sessions.

INTERVIEW LENS

CISCO · JUNIPER · AMAZON

"Write an ACL that lets one VLAN reach a web server and nothing else" is asked at the whiteboard. A strong answer states the placement and direction first, orders the lines specific to general, writes the wildcard correctly, mentions the implicit deny, and says how to verify (hit counters). Expect: "Where would you put it if it were standard?" and "How do replies get back?"

CHECK YOURSELF

1. Where is a standard ACL placed, and why? [1]
2. An ACL consists of three `deny` lines for specific hosts. What does it permit? [2]
3. What is the wildcard mask for a single host? [3]

9.6 Stateful firewalls, NGFW and NAT interplay MUST · ■■□

BIG IDEA

A stateful firewall remembers each connection in a session table, so it can allow replies automatically and drop everything that does not belong to a permitted conversation.

A *stateful firewall* creates an entry when it permits the first packet of a flow and then admits packets that match it in either direction. The entry holds the *5-tuple* (protocol, source and destination address, source and destination port), the connection state, a timer and counters. For TCP the firewall follows the handshake and the sequence-number window, so that a forged RST or out-of-window segment is dropped; for UDP and ICMP, which have no state, it keeps a pseudo-session with a short idle timeout. The housing-society guard analogy applies: he remembers that you invited the delivery rider, so he lets him leave again without checking the list, whereas a stateless guard (the ACL) re-reads the list for every passage.

Protocol	Inside address	Translated address	Destination	State	Idle timer
TCP	192.168.20.15:51514	203.0.113.5:40001	198.51.100.7:443	ESTABLISHED	3600 s
UDP	192.168.10.7:49152	203.0.113.5:40002	8.8.8.8:53	replied	30 s
ICMP	192.168.20.15 id 77	203.0.113.5 id 77	198.51.100.7	echo sent	10 s

Firewalls group interfaces into *zones* (Campus Net: inside, servers, outside) and write rules between zones. Traffic inside one zone is usually permitted, traffic between zones is denied unless a rule allows it, and the usual defaults are that inside may initiate towards servers and outside, servers may reply, and outside may reach only published services. Some vendors use numeric *security levels* instead (Cisco ASA: 100 for inside, 0 for outside), with traffic from high to low allowed by default.

A *next-generation firewall* (NGFW) adds visibility above Layer 4: *application identification* (recognising a protocol by its behaviour, not its port, so that a service on TCP 443 is told apart from another), *user identity* (rules by directory user or group, not by address), an *intrusion prevention* engine (Section 9.13), URL and category filtering, malware sandboxing, and *TLS inspection*. For the last, the firewall acts as a proxy: it terminates the client's TLS session using a certificate that it forges on the fly, signed by an enterprise CA that the client's devices are configured to trust, and opens a second session to the real server. Content can then be scanned, but it breaks certificate pinning and mutual TLS, concentrates risk in the firewall's CA key, and raises privacy and legal questions, so banking and health categories are normally bypassed and users are informed by policy.

WORKED EXAMPLE

A packet walk through the edge firewall. Campus Net translates 192.168.20.0/24 to 203.0.113.5. A laptop 192.168.20.15 opens a connection to 198.51.100.7:443.

1. The SYN arrives on the inside interface. The firewall finds no session, checks the policy (inside to outside, HTTPS permitted), translates the source to 203.0.113.5:40001 and creates the session of the first row above.
2. The SYN-ACK from 198.51.100.7 is addressed to 203.0.113.5:40001. It matches the session in reverse, so it bypasses the rule base; the destination is translated back to

192.168.20.15:51514.

3. An unsolicited SYN from the Internet to 203.0.113.5:22 matches no session, and no rule permits it. It is dropped (usually silently, so that a scanner learns nothing).
4. Suppose BGP now sends the reply of a *new* connection via the second border router, whose firewall has no session. That firewall sees a SYN-ACK with no SYN and drops it; the client's connection times out.

Result: stateful inspection depends on seeing both directions of a flow through the same device.

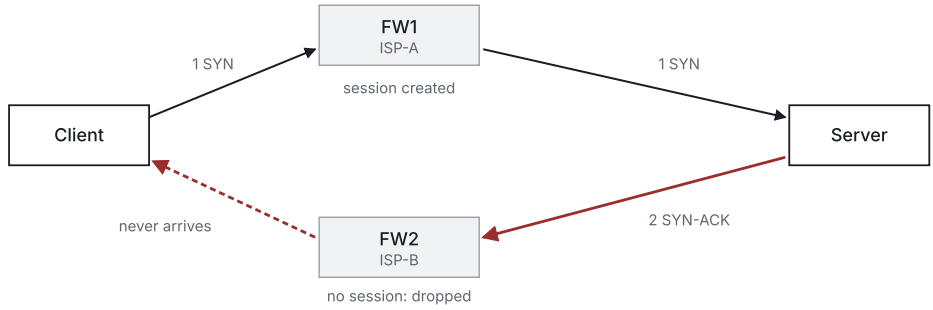


Figure 9.9 Asymmetric routing: the SYN opens a session on FW1; the SYN-ACK returns through FW2, which has no session and drops it.

Step 4 is the *asymmetric routing* problem (Figure 9.9). It arises when load sharing or BGP policy (Pillar 4) sends the outgoing and returning packets over different paths. The cures are to force symmetry (prefer one border for both directions, as with AS-path prepending or local preference), to place the firewalls in a high-availability pair that synchronises its session table (active/standby, in which only one unit forwards; an active/active pair needs symmetric flows or state replication between the units), or to place a single firewall behind both borders. A pair that drops 30 % of replies (Q9.27) is the textbook symptom.

NAT interplay. Translation and filtering interact in an order that is vendor-specific, so one must know which addresses a rule sees. In Linux Netfilter, *destination* NAT happens in `prerouting`, before the forward filter, so rules match the translated (internal) address; *source* NAT happens in `postrouting`, after it, so rules match the original internal address. Hence a rule for a published server names 10.0.0.80, not the public address. The connection-tracking entry stores both directions, so replies are un-translated without a rule. A stateful firewall is also itself a target: its session table has a finite size, which is why a flood of new connections can exhaust it (Section 9.10), and why no firewall alone stops a volumetric attack. The NAT techniques themselves are in Section 3.6 (p. 183).

The rule base of the Campus Net edge firewall is best planned as a matrix of zones before any rule is typed. The default for every empty cell is deny.

From to	Users	Servers	Outside
Users	permit (same zone)	TCP 80 and 443	permit, source-translated to 203.0.113.5
Servers	replies only	permit (same zone)	TCP 443 to package mirrors and DNS to the resolver

Outside	deny	TCP 443 to 10.0.0.80 only (after destination NAT)	not applicable

For availability, two firewalls form a *high-availability pair*. In *active/standby* only one unit forwards; it sends its session table over a dedicated link, together with heartbeats, so that the standby takes over without dropping established connections. If the heartbeat link fails while both units are healthy, both become active (*split brain*), which is why the heartbeat uses two paths. In *active/active* both forward, which brings back the symmetry problem above.

LAB

`LAB 9.3 · 25 MIN` A stateful edge firewall for Campus Net with nftables, in a Linux VM (or three network namespaces) with `eth0` outside, `eth1` users and `eth2` servers. Run only on a lab VM you own.

```
table inet campus {
  chain input { type filter hook input priority 0; policy drop;
    ct state established,related accept
    iif lo accept
    iifname "eth1" tcp dport 22 accept
  }
  chain forward { type filter hook forward priority 0; policy drop;
    ct state established,related accept
    ct state invalid drop
    iifname "eth1" oifname "eth0" accept
    iifname "eth1" oifname "eth2" tcp dport { 80, 443 } accept
    iifname "eth0" oifname "eth2" ip daddr 10.0.0.80 tcp dport 443 accept
  }
}
table ip nat {
  chain prerouting { type nat hook prerouting priority -100;
    iifname "eth0" tcp dport 443 dnat to 10.0.0.80
  }
  chain postrouting { type nat hook postrouting priority 100;
    oifname "eth0" snat to 203.0.113.5
  }
}
```

The three forward rules, in order, let users reach the Internet, let users reach the web ports of the server network, and let the Internet reach the published server only (after DNAT). Load it with `nft -f campus.nft`, make a connection from a user host, and list the session with `conntrack -L`.

```
tcp 3598 ESTABLISHED src=192.168.20.15 dst=198.51.100.7 sport=51514 dport=443
  src=198.51.100.7 dst=203.0.113.5 sport=443 dport=40001 [ASSURED]
```

What you should notice: the second half of the entry is the reply tuple: its destination is the translated address 203.0.113.5:40001, which is how the reply is matched and un-translated. No rule allows replies; `ct state established,related` does that for every flow. The forward rule for the published server names 10.0.0.80, the address after DNAT. From the outside, `nc -zv 203.0.113.5 22` gets no answer, since the default policy is drop.

CHECK YOURSELF

1. What does a stateful firewall check that an ACL does not? [4]
2. Why does an active/active firewall pair need symmetric routing? [5]

9.7 IPsec and IKE MUST · ■■■

BIG IDEA

IPsec protects traffic at the IP layer for every application at once, and it separates the work into a slow, rare negotiation (IKE) and a fast, per-packet protection (ESP), because the keys must be agreed before any packet can be encrypted.

IPsec is a suite of protocols that authenticates and encrypts IP packets. It has two data-plane protocols and one control protocol. *ESP* (Encapsulating Security Payload, IP protocol 50) gives confidentiality, integrity and replay protection. *AH* (Authentication Header, protocol 51) gives integrity and replay protection only, and it authenticates parts of the outer IP header, so any NAT that rewrites an address breaks it; hence ESP with integrity is the norm and AH is rarely seen. *IKE* (Internet Key Exchange, UDP 500) authenticates the peers and creates the keys.

Each protected direction of traffic is described by a *Security Association* (SA): the algorithms, the keys, the lifetime and a 32-bit *Security Parameter Index* (SPI) that the receiver uses to look up the SA in its database. SAs are one-way, so a tunnel needs a pair. The *Security Policy Database* (SPD) says which traffic is to be protected, bypassed or dropped; the *Security Association Database* (SAD) holds the live SAs.

IPsec works in two modes (Figure 9.10). In *transport mode* the original IP header is kept and ESP is inserted between it and the transport header; this protects host-to-host traffic. In *tunnel mode* the whole original packet is encrypted and a new outer IP header added; this is used between gateways, and it hides the inner addresses. When a NAT device lies on the path, ESP has no ports for the NAT to translate. *NAT traversal* (NAT-T) solves this by wrapping ESP in a UDP header, port 4500, after IKE detects the translation from hashes of the addresses it sends.

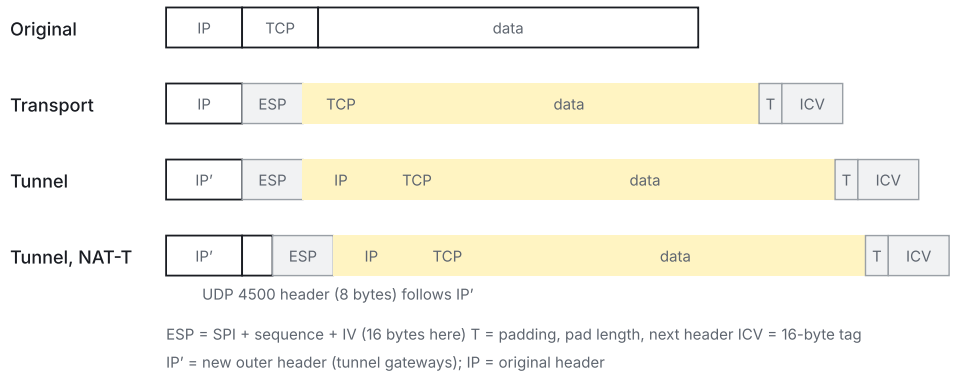


Figure 9.10 Packet formats, drawn to scale (1 byte = 1.9 px, inner payload 100 bytes). Encryption covers the shaded middle; ESP authentication covers from the ESP header to the trailer.

IKE negotiates in two stages. In *IKEv1*, Phase 1 builds a protected IKE channel, in six messages in *main mode*, or three in *aggressive mode*, which is faster but exposes the peer identities and a hash that allows offline guessing of a pre-shared key; Phase 2, *quick mode*, uses that channel to build the IPsec SAs in three messages. The embassy analogy fits: Phase 1 is two embassies agreeing on a secure diplomatic pouch, and Phase 2 is the individual sealed letters sent in it. *IKEv2* (RFC 7296) simplifies this (Figure 9.11). `IKE_SA_INIT` (two messages) agrees on the algorithms, exchanges DH values and nonces and creates the IKE SA. `IKE_AUTH` (two messages), encrypted, exchanges identities, proves them by pre-shared key, certificate or EAP, and negotiates the first *Child SA* (the ESP tunnel) with the *traffic selectors*. Four messages thus replace six

1. Close to the destination, because it matches only the source and would otherwise block that source's traffic to every other destination.↩
2. Nothing: the implicit `deny any` blocks all other packets.↩
3. 0.0.0.0, written `host x.x.x.x`.↩
4. Whether a packet belongs to an existing, permitted session (and for TCP, whether its state and sequence number are valid).↩
5. Each unit holds only the sessions it created unless state is synchronised, so a reply returning through the other unit matches no session and is dropped.↩

or nine. `CREATE_CHILD_SA` later adds a Child SA or rekeys either SA, and liveness checks and EAP support are built in.

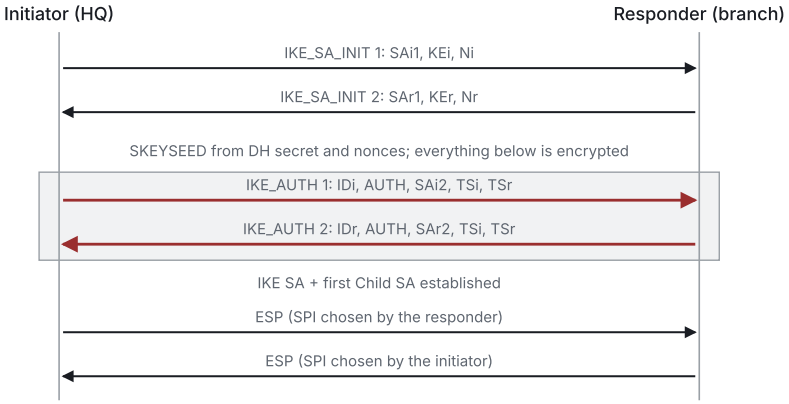


Figure 9.11 IKEv2 in four messages, then ESP. A later CREATE_CHILD_SA message pair rekeys or adds a Child SA.

	IKEv1	IKEv2
Messages to first tunnel	6 main + 3 quick = 9 (aggressive: 3 + 3 = 6)	4
Phases	Phase 1 then Phase 2	IKE SA, then Child SA together
Authentication	pre-shared key or certificate	PSK, certificate, EAP
Liveness, NAT-T	extensions (DPD, NAT-T)	built in
Remote access	XAUTH, mode config (non-standard)	EAP and configuration payload

For two peers to bring a tunnel up, the following must match: the encryption and integrity algorithms (or the AEAD), the pseudo-random function, the DH group, the authentication method and key, and the lifetimes; if *perfect forward secrecy* (PFS) is configured, a new DH exchange is performed for each Child SA, so that compromise of the IKE keys does not expose later Child SAs. The traffic selectors (the *proxy-IDs* or crypto ACL of policy-based VPNs) must mirror each other. A *policy-based* VPN protects the traffic that matches an ACL; a *route-based* VPN creates a virtual tunnel interface (VTI), and whatever the routing table sends into it is protected, which is simpler and supports routing protocols over the tunnel.

> WORKED EXAMPLE
>
> **A site-to-site tunnel from Campus Net to its hostel annex.** BR1 (203.0.113.2) protects traffic between ENG, 192.168.20.0/24, and the annex, 172.16.50.0/24, whose router has public address 198.51.100.20. Route-based IKEv2 on IOS XE:

```
crypto ikev2 proposal P1
 encryption aes-gcm-256
 prf sha256
 group 19
crypto ikev2 policy POL1
 proposal P1
crypto ikev2 keyring KR1
 peer ANNEX
  address 198.51.100.20
  pre-shared-key <secret>
crypto ikev2 profile PROF1
 match identity remote address 198.51.100.20 255.255.255.255
 authentication local pre-share
 authentication remote pre-share
 keyring local KR1
crypto ipsec transform-set TS1 esp-gcm 256
 mode tunnel
crypto ipsec profile IPSEC1
 set transform-set TS1
 set ikev2-profile PROF1
interface Tunnel0
 ip address 10.255.0.1 255.255.255.252
 tunnel source GigabitEthernet0/0
 tunnel destination 198.51.100.20
 tunnel mode ipsec ipv4
 tunnel protection ipsec profile IPSEC1
ip route 172.16.50.0 255.255.255.0 Tunnel0
```

1. The proposal fixes the algorithms: AES-256 in GCM mode (so no separate integrity algorithm), SHA-256 as the PRF and DH group 19 (the 256-bit NIST curve).
2. The profile names the peer and the authentication; the IPsec profile binds the transform set to it; `tunnel protection` attaches both to the interface.
3. The static route sends annex traffic into the tunnel, and the router encrypts it after the routing decision.
4. Verify in two places, since the IKE SA and the Child SA are separate:

```
BR1# show crypto ikev2 sa
 Tunnel-id  Local            Remote           Status
 1          203.0.113.2/500  198.51.100.20/500 READY
BR1# show crypto ipsec sa | include encaps|decaps
  #pkts encaps: 124, #pkts encrypt: 124
  #pkts decaps: 122, #pkts decrypt: 122
```

Result: status READY and both counters rising mean the tunnel carries traffic in both directions; READY with counters at zero, or only one rising, points to routing or a Child SA problem.

A planted fault. Suppose the annex router is configured with a policy-based selector 172.16.50.0/25 while BR1 proposes 172.16.50.0/24. Phase 1 completes, `show crypto ikev2 sa` says READY, yet no traffic passes; the debug shows the responder answering the Child SA with the notification `TS_UNACCEPTABLE`. The cause is that the selectors do not mirror each other. This is why “phase 1 up” never means “the VPN works”. The check order is then: the IKE SA state, the Child SA and its selectors, the transform set and PFS group, the routing into the tunnel, and finally NAT exemption (traffic to the annex must not be translated before it is encrypted). The full troubleshooting case is in Pillar 11 (p. 634).

TRAP

Claim: AH is more secure than ESP because it authenticates the whole packet. *Why it is wrong:* AH gives no confidentiality, and because it covers the outer header it fails when NAT changes an address. *Correct statement:* ESP with an authenticated mode such as AES-GCM is the standard; AH is retained for completeness.

INTERVIEW LENS

CISCO · JUNIPER · NOKIA

"Explain IKEv1 phases and IKEv2, then tell me what you check when Phase 1 is up and nothing passes." A strong answer gives the message counts (6 or 3 plus 3, against 4), says what each exchange establishes, and then walks the check order of the planted fault: Child SA and selectors, transform and PFS, routing, NAT exemption, counters. Expect: "Why NAT-T?" and "Why does AH fail through NAT?"

9.8 VPN families and remote access SHOULD · ■■□

BIG IDEA

A VPN builds a private path over a public network with encryption as the tunnel wall, and each family differs in the layer at which it works, the client it needs and how it survives NAT.

A *site-to-site* VPN joins two networks and is invisible to users; IPsec is the classical choice. *Remote-access* VPNs connect one user device to the enterprise: either IPsec with IKEv2 and EAP, or an *SSL/TLS VPN* (such as Cisco Secure Client) that runs over TCP 443 and uses DTLS over UDP for speed. The TLS form passes almost every firewall because it looks like HTTPS, and some products offer a clientless portal. *WireGuard* is a newer design: peers are identified by public keys, handshake uses the Noise protocol with Curve25519, data uses ChaCha20-Poly1305 over UDP, and there is no negotiation of algorithms at all, which keeps the code small enough to audit. Cisco DMVPN and GETVPN are Cisco-specific ways of scaling IPsec to many sites; we mention the names only.

	IPsec site-to-site	IPsec remote access	SSL/TLS VPN	WireGuard
Layer	3	3	4 to 7	3
Transport	ESP or UDP 4500	ESP or UDP 4500	TCP or UDP 443	UDP (any port)
NAT traversal	NAT-T	NAT-T	native	native (UDP, keepalive)
Client	none (gateways)	native or vendor	vendor client or browser	small client, one key per peer
Authentication	PSK, certificate	certificate, EAP	certificate, SAML, MFA	public keys only
Typical use	branch to HQ	managed laptops	contractors, restrictive networks	site links, servers, personal VPN

Split tunnelling sends only enterprise prefixes through the VPN and everything else directly to the Internet. It saves bandwidth, but the device is then on two networks at once and the enterprise cannot inspect its Internet traffic; full tunnelling reverses both properties.

Every encapsulation consumes bytes of the packet, so the inner MTU must shrink. For ESP in tunnel mode with AES-GCM over IPv4, the extra bytes are:

Item	Bytes
New outer IP header	20
ESP header (SPI + sequence number)	8
GCM initialisation vector	8
Padding (0 to 3) + pad length + next header	2 to 5
Integrity check value (ICV)	16
Total	**54 to 57**
NAT-T UDP header	+ 8

WORKED EXAMPLE

Why `ip tcp adjust-mss 1360`. On a 1500-byte path the worst case with NAT-T is 57 + 8 = 65 bytes, so the largest inner packet is 1500 − 65 = 1435 bytes. A TCP segment carries 40 bytes of IP and TCP headers, so the largest segment payload (MSS) is 1435 − 40 = 1395 bytes. Without a correction, hosts negotiate an MSS of 1460, their packets exceed the tunnel and either fragment or vanish (when DF is set and ICMP is filtered). The router therefore rewrites the MSS in SYNs that cross the tunnel. The common value of 1360 is conservative: it leaves 35 bytes of margin for GRE (24 bytes), PPPoE (8 bytes) or IPv6 on the path.

WireGuard adds 60 bytes over IPv4 (20 IP, 8 UDP, 16 for its header and 16 for the authentication tag, plus padding of the inner packet to a multiple of 16), so its default tunnel MTU is 1420, which also suits IPv6 paths (80 bytes). The comparison for large deployments is made in Q9.32.

LAB

`LAB 9.5 · 20 MIN` WireGuard between two network namespaces on a Linux VM you own. The sizes below follow from the protocol; your timestamps and counters will differ.

```
ip netns add a; ip netns add b
ip link add va type veth peer name vb; ip link set va netns a; ip link set vb
netns b
ip -n a addr add 10.9.0.1/24 dev va; ip -n b addr add 10.9.0.2/24 dev vb
ip -n a link set va up; ip -n b link set vb up
for n in a b; do ip link add wg0 type wireguard; ip link set wg0 netns $n;
done
wg genkey | tee a.key | wg pubkey > a.pub; wg genkey | tee b.key | wg pubkey
> b.pub
ip netns exec a wg set wg0 private-key a.key listen-port 51820 \
  peer $(cat b.pub) allowed-ips 10.200.0.2/32 endpoint 10.9.0.2:51820
ip netns exec b wg set wg0 private-key b.key listen-port 51820 \
  peer $(cat a.pub) allowed-ips 10.200.0.1/32 endpoint 10.9.0.1:51820
ip -n a addr add 10.200.0.1/24 dev wg0; ip -n b addr add 10.200.0.2/24 dev
wg0
ip -n a link set wg0 up; ip -n b link set wg0 up
ip netns exec a tcpdump -ni va -c 3 udp port 51820 &
ip netns exec a ping -c 1 10.200.0.2
```

```
10.9.0.1.51820 > 10.9.0.2.51820: UDP, length 148     (handshake initiation)
10.9.0.2.51820 > 10.9.0.1.51820: UDP, length 92      (handshake response)
10.9.0.1.51820 > 10.9.0.2.51820: UDP, length 128     (the ping: 84 bytes,
padded to 96, plus 32)
```

> **What you should notice:** the handshake is two messages, started by the first packet that needs it; the ping travels as one UDP datagram on port 51820, and 128 − 84 = 44 bytes of the datagram are WireGuard's (16 header, 12 padding, 16 tag). The peers are identified only by their keys, so the filter on the cable shows no algorithm negotiation at all.

9.9 Layer-2 attacks and switch hardening MUST · ■■□

> BIG IDEA
>
> Layer-2 protocols trust every host on the segment, because ARP, DHCP, DTP and STP were designed for friendly networks; hardening replaces that trust with state kept by the switch.

An attacker inside the building needs no firewall bypass: a laptop on a wall socket already shares a broadcast domain with its victims. The switch features of this section are cheap, run in hardware and are expected knowledge for any Cisco, Juniper or Nokia interview. The attacks and defences are collected in one table in the Revision Spread; here we explain how each works.

ARP spoofing. ARP (Section 3.6 (p. 183)) has no authentication: any host may announce that an IP address is at its MAC address, and neighbours accept the latest claim. ARP spoofing exploits this. The attacker sends gratuitous ARP replies claiming that the gateway's address is at the attacker's MAC and that the victim's address is also at the attacker's MAC (Figure 9.12), so both ends send their frames to the attacker, who relays them: a *man-in-the-middle*. As a wedding guest shouting "I am the caterer!" collects all the dishes, the attacker collects the traffic.

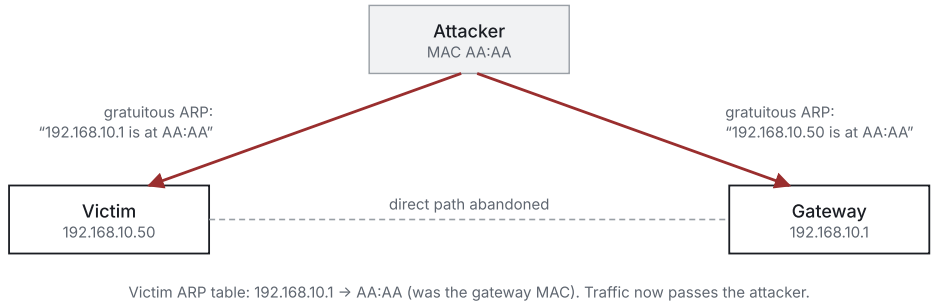


Figure 9.12 ARP spoofing on a Campus Net VLAN: the attacker answers for the gateway and for the victim, and relays between them.

DHCP attacks. In *DHCP starvation* the attacker requests addresses with forged MAC addresses until the pool is empty; then a *rogue DHCP server* answers new clients with the attacker as gateway and DNS server, which is another man-in-the-middle. **MAC flooding** floods the switch with frames from random source MACs until the table is full; the switch then floods unknown unicast frames out of every port in the VLAN, like a hub, and the attacker captures them (Section 3.2 (p. 165)).

The defences work together.

- *Port security* limits the MAC addresses per access port (`maximum`), can learn them as `sticky` and reacts to a violation by `protect` (drop silently), `restrict` (drop and log) or `shutdown` (the default; the port is err-disabled until reset or recovered). It stops MAC flooding and starvation by forged MACs.
- *DHCP snooping* classifies ports as *trusted* (uplinks, the DHCP server) or *untrusted* (access). On untrusted ports it drops DHCP server messages (OFFER, ACK), so a rogue server is sil-

ent, and it can rate-limit DHCP requests, which blunts starvation. It also builds the *binding table*: MAC, IP address, VLAN, port and lease for every address handed out.

- *Dynamic ARP Inspection* (DAI) checks each ARP packet on an untrusted port against the binding table and drops those whose sender IP and MAC do not match a binding. It therefore *depends on DHCP snooping* (or on static ARP ACLs for hosts with fixed addresses). Uplinks are marked trusted.
- *IP Source Guard* uses the same table to drop packets whose source IP is not bound to the port, which limits spoofing from the inside.

WORKED EXAMPLE

Hardening a Campus Net access switch. SW1 serves SALES (VLAN 10) and ENG (VLAN 20) on Gi1/0/2 to Gi1/0/24, with the uplink to R1 on Gi1/0/1.

```
ip dhcp snooping
ip dhcp snooping vlan 10,20
ip arp inspection vlan 10,20
interface GigabitEthernet1/0/1
 description uplink to R1
 ip dhcp snooping trust
 ip arp inspection trust
interface range GigabitEthernet1/0/2 - 24
 switchport mode access
 switchport nonegotiate
 switchport port-security
 switchport port-security maximum 2
 switchport port-security violation restrict
 switchport port-security mac-address sticky
 ip dhcp snooping limit rate 15
 spanning-tree portfast
 spanning-tree bpduguard enable
```

With this in place the attack from the figure fails: the attacker's ARP reply claims 192.168.10.1 at AA:AA, but the binding table has no entry for that pair on the attacker's port, so DAI drops it and logs:

```
%SW_DAI-4-DHCP_SNOOPING_DENY: 1 Invalid ARPs (Reply) on Gi1/0/5, vlan 10.
```

Result: the victim's ARP table keeps the real gateway MAC, and the log names the port to investigate.

LAB

`LAB 9.6 · 30 MIN` ARP spoofing and its defeat, in an isolated lab (two Linux hosts and a managed switch, or a virtual switch with DHCP snooping). **Isolated lab only; never on a college or public network.**

```
attacker$ sudo sysctl -w net.ipv4.ip_forward=1
attacker$ sudo arpspoof -i eth0 -t 192.168.10.50 192.168.10.1
victim$   ip neigh show 192.168.10.1
```

```
192.168.10.1 dev eth0 lladdr 52:54:00:aa:00:01 REACHABLE     (before)
192.168.10.1 dev eth0 lladdr 52:54:00:ee:ee:05 REACHABLE     (during: the
attacker's MAC)
```

Now enable the commands of the example above and repeat. On the switch:

```
SW1# show ip dhcp snooping binding
MacAddress          IpAddress       Lease(sec)  Type           VLAN
Interface
52:54:00:aa:10:50   192.168.10.50   86212       dhcp-snooping  10    Gi1/0/3
SW1# show ip arp inspection statistics vlan 10
 Vlan  Forwarded  Dropped  DHCP Drops  ACL Drops
   10        214       96          96          0
```

What you should notice: the victim's entry keeps the gateway's real MAC, the `Dropped` counter climbs with the attacker's replies, and the drops are DHCP drops, meaning that no binding matched. A host with a static address would be dropped too, until it is given an ARP ACL or a static binding.

VLAN hopping lets an attacker reach a VLAN to which the port does not belong, in two ways. In *switch spoofing* the attacker's host speaks DTP (the dynamic trunking protocol) and negotiates a trunk with the switch if the port is `dynamic auto` or `desirable`, after which it receives the traffic of all VLANs. The cure is to set access ports to `switchport mode access` and `switchport nonegotiate`. In *double tagging* the attacker sits in the trunk's native VLAN (VLAN 1 by default) and sends a frame with two 802.1Q tags: the outer carries the native VLAN, the inner the target VLAN (Figure 9.13). The first switch removes the outer tag, since native-VLAN frames cross the trunk untagged, and forwards the frame still carrying the inner tag; the second switch reads it and delivers the frame into the victim VLAN. The attack is one-way (no reply can return the same way) and works only if the native VLAN matches an access VLAN in use. Hence the native VLAN of every trunk is set to an unused VLAN (for example VLAN 999), unused ports are shut down, and tagging of the native VLAN may be enforced.

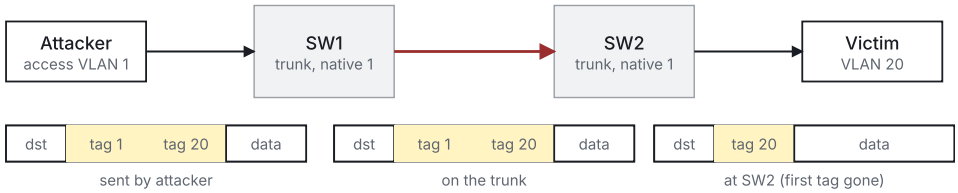


Figure 9.13 Double tagging. The first switch strips the outer native-VLAN tag; the second reads the inner tag and delivers into VLAN 20. One way only.

TRAP

Claim: VLANs are a security boundary on their own. *Why it is wrong:* a misconfigured trunk, DTP, a shared native VLAN or an unfiltered router lets traffic cross, and traffic between VLANs passes the router, which has no filter unless an ACL is applied. *Correct statement:* VLANs separate broadcast domains; security needs hardened trunks, native-VLAN hygiene and filtering at Layer 3 (Section 9.5).

STP manipulation. An attacker who sends BPDUs with a low bridge priority can become the root bridge and steer the whole network's traffic through his port. *BPDU Guard* on PortFast edge ports puts the port in err-disabled state on receiving any BPDU; *Root Guard* on distribution-facing ports refuses a superior BPDU and puts the port in a root-inconsistent state until it stops. Storm control additionally caps broadcast, multicast and unknown-unicast rates per port. STP itself is taught in Section 4.3 (p. 237).

INTERVIEW LENS

CISCO · JUNIPER · NOKIA

"Explain VLAN hopping" and "How does DAI work and what does it depend on?" are the usual Layer-2 questions. A strong answer names the attack, draws the frame or the triangle, gives the one-line command, and states the dependency (DAI needs the snooping table). Expect: "What if the host has a static address?" (an ARP ACL or static binding) and "Why does double tagging work only one way?"

CHECK YOURSELF

1. What does DAI check an ARP packet against? [1]
2. Why does double tagging fail if the native VLAN is unused? [2]

9.10 Layer-3 and Layer-4 attacks, and DDoS MUST · ■■□

BIG IDEA

Most network floods succeed because a packet's source address is never checked and because the server commits memory before the client proves it exists; the cures are to verify the source and to postpone the commitment.

IP spoofing. The IP header carries whatever source address the sender writes, and a router forwards on the destination only. Spoofing hides the attacker, and it is the basis of reflection attacks. The cure is *ingress filtering* (BCP 38, RFC 2827): the provider drops packets arriving from a customer whose source address is not in the customer's allocation. Inside a network, *unicast reverse path forwarding* (uRPF) does it automatically by asking, for each packet, whether the routing table would send traffic to the source address out of the arrival interface. In *strict* mode the best route must use that very interface, which suits single-homed edges. In *loose* mode any route to the source suffices, which is required where routing is asymmetric, as on a multihomed border such as BR1 and BR2, where a strict check would drop valid traffic. On IOS the commands are `ip verify unicast source reachable-via rx` (strict) and `reachable-via any` (loose).

SYN flood. The TCP handshake (Pillar 3 (p. 162)) makes a server allocate a half-open entry when a SYN arrives, and hold it for tens of seconds while waiting for the ACK. An attacker sends SYNs from random spoofed sources at a high rate. The entries fill the SYN queue (Figure 9.14, and Figure 7.17 for the queues of Section 7.10 (p. 445)), the SYN-ACKs go to hosts that never asked, and genuine clients are refused. As a prankster who books every table at a restaurant and never arrives, the attacker uses very little of his own resources.

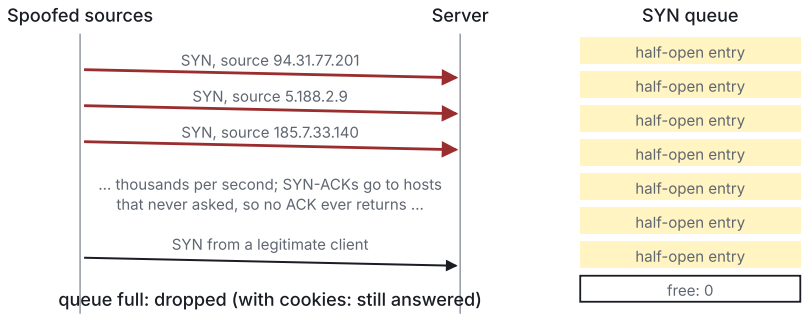


Figure 9.14 A SYN flood fills the SYN queue with half-open entries; with cookies the server keeps none.

1. The DHCP snooping binding table (or a static ARP ACL).↩
2. The attacker must be in the native VLAN for the first switch to strip the outer tag, and an unused native VLAN has no access ports for him to occupy.↩

Defences: enlarge the backlog and shorten the SYN-ACK retry count (they only delay exhaustion); filter at the edge; place a *SYN proxy* in a firewall, which completes the handshake itself and passes the connection on only when the client answers; or use **SYN cookies**. With cookies the server, when its queue is full, stores nothing. It encodes the state in the initial sequence number of its SYN-ACK. In the original design the 32 bits hold 5 bits of a minute counter, 3 bits that index a small table of MSS values, and 24 bits of a keyed hash of the addresses, ports and counter; Linux uses a close variant that adds the counter and mixes the MSS index into a second hash. A genuine client returns acknowledgement number = ISN + 1, from which the server recomputes the hash, validates it and rebuilds the connection. A spoofed source never returns, so the attacker consumes no memory. The price is that options such as window scaling must be carried in other ways, so cookies are used only under load.

DEEP DIVE · MAY BE SKIPPED

A cookie with numbers (illustrative layout). Let the MSS table be {536, 1300, 1440, 1460}, the minute counter 17, and let the keyed hash of the two addresses, the two ports and the counter be 0x2A91F3.

1. Cookie = $17 \times 2^{27} + 3 \times 2^{24} + 0x2A91F3 = 2{,}281{,}701{,}376 + 50{,}331{,}648 + 2{,}789{,}875 = 2{,}334{,}822{,}899$. This is the sequence number of the SYN-ACK; the acknowledgement number is the client's ISN plus 1.
2. The honest client replies with acknowledgement number 2,334,822,900.
3. The server subtracts 1, reads the counter (17, accepted if it is within the last two or so minutes) and the MSS index (3, so 1460), recomputes the hash and compares it with the low 24 bits.

Result: a valid ACK recreates the connection with the right MSS, and nothing was stored in between.

LAB

`LAB 9.4 · 30 MIN` SYN flood against SYN cookies on two lab VMs on an isolated host-only network (attacker and a victim running a web server on 10.0.0.80:443). **Never run this on a college or public network.** The packet rates are typical, not exact.

```
victim$   sudo sysctl -w net.ipv4.tcp_syncookies=0
attacker$ sudo hping3 -S -p 443 --flood --rand-source 10.0.0.80
victim$   sudo tcpdump -ni eth0 -c 4 'tcp port 443 and tcp[13] & 2 != 0'
victim$   ss -tan state syn-recv | wc -l
victim$   nstat -az TcpExtListenDrops
```

```
94.31.77.201.1827 > 10.0.0.80.443: Flags [S], seq 1384720, win 512
10.0.0.80.443 > 94.31.77.201.1827: Flags [S.], seq 2905177321, ack 1384721
10.0.0.80.443 > 94.31.77.201.1827: Flags [S.], seq 2905177321   (1 s later)
128                              <- SYN-RECV entries: the queue is full
TcpExtListenDrops                1200000
```

From a third host `curl -m 5 https://10.0.0.80/` now times out. Re-enable cookies with `sysctl -w net.ipv4.tcp_syncookies=1`:

```
kernel: TCP: request_sock_TCP: Possible SYN flooding on port 443. Sending
cookies.
```

What you should notice: each spoofed SYN has a different source and the SYN-ACK is retransmitted with the *same* sequence number, because it belongs to a stored entry. The queue sits at its limit. With cookies, `curl` succeeds again while the flood continues, and the kernel logs that it is sending cookies; legitimate sessions complete, because their ACK validates the cookie.

INTERVIEW LENS

GOOGLE · MICROSOFT · AMAZON

"What is a SYN flood and how do SYN cookies work?" is a favourite because it tests TCP knowledge and engineering judgement together. A strong answer draws the SYN queue, explains why spoofed sources never complete, gives the cookie's contents, and states the cost (options) and the limits (a volumetric flood still saturates the link). Expect: "What else can you do upstream?" and "How would you know it is a SYN flood and not a busy day?" (many SYN-RECV, few ESTABLISHED).

Other floods. UDP and ICMP floods need no handshake and simply saturate a link or a CPU. *Smurf* attacks (an ICMP echo to a broadcast address with the victim's spoofed source) are historic because routers no longer forward directed broadcasts. *Application-layer* attacks send valid but expensive requests: an HTTP flood requests heavy pages, and *Slowloris* holds many connections open with partial requests until the server's connection table is full.

Reflection and amplification. The attacker sends a small query with the *victim's* address as source to many public servers, which answer the victim with a much larger reply (Figure 9.15). Protocols that use UDP and return more than they receive are used: DNS (reported factors of tens), NTP's old `monlist` command (hundreds), and memcached (up to tens of thousands). The attacker's own bandwidth is multiplied and the packets arrive from legitimate servers. The defences are in three places: BCP 38 stops the spoofing at the source network; server operators close open resolvers and disable `monlist` and UDP memcached; the victim needs capacity upstream.

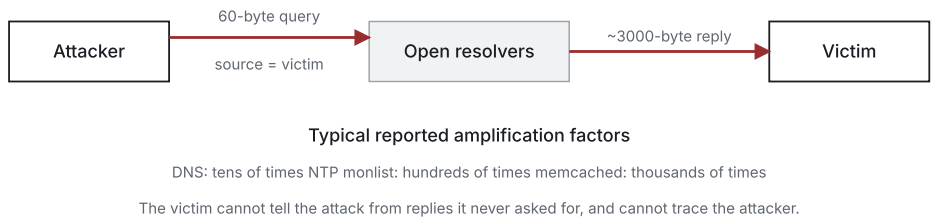


Figure 9.15 Reflection and amplification: a small spoofed query becomes a large reply aimed at the victim.

BGP hijacking. An AS can announce a prefix it does not own, or a more specific one, and neighbours who do not check will route traffic to it (Section 4.7 (p. 256)). The defence is *RPKI route origin validation* (ROV). The holder of a prefix signs a *ROA* (route origin authorisation) naming the prefix, its maximum length and the AS allowed to originate it. A router that validates classifies each route as *valid* (matches a ROA), *invalid* (covered by a ROA but with the wrong origin or too long) or *not found*, and drops the invalid ones. Limits: ROV checks only the origin, not the path, so an attacker who forges the legitimate origin at the end of the path is not caught (path validation by ASPA or BGPsec is still in development); and coverage is incomplete, since unsigned prefixes are "not found". Prefix filters, max-prefix limits and IRR-based filters supplement it.

DDoS attacks fall into three classes, which need different defences.

Class	Example	Target	Main defence

Volumetric	UDP flood, amplification	bandwidth	upstream scrubbing, anycast, RTBH
Protocol	SYN flood, fragment flood	state tables, CPU	SYN cookies, SYN proxy, rate limits
Application	HTTP flood, Slowloris	the application	CDN and WAF, rate limiting per client, autoscaling

A *DDoS defence stack* is layered, since no single device is large enough. At the edge, ACLs, uRPF and rate limits remove obvious junk. At the provider, *remotely triggered black hole* (RTBH) routing advertises the victim's address with a community that makes upstream routers discard its traffic, at the cost of giving the attacker the outage; *BGP Flowspec* distributes finer filters (by port or packet size). A *scrubbing centre* receives the victim's traffic through a BGP diversion or an anycast address, removes the attack and returns the clean traffic by a tunnel. A CDN and WAF absorb application floods, and at the origin cookies and autoscaling provide the last margin. Note that a stateful firewall is not on this list: its session table is itself a target, and a volumetric flood saturates the link before the firewall sees it.

9.11 DNS security SHOULD · ■■□

BIG IDEA

Classical DNS answers are accepted on the strength of a 16-bit transaction ID and a UDP port, so security has to come either from making these unguessable or from signing the data.

DNS (Pillar 3 (p. 162)) runs over unauthenticated UDP. In *cache poisoning* the attacker makes a recursive resolver ask for a name and then floods it with forged answers; the first answer that matches the query's name, transaction ID and source port is cached, and every client of the resolver is then sent to the attacker's address (Figure 9.16). It is like slipping a fake entry into the society telephone directory before the real one arrives.

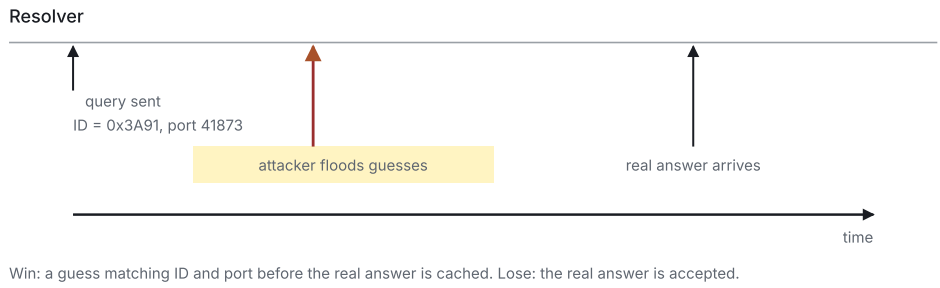


Figure 9.16 The poisoning race. The attacker must land a forged answer, with the right transaction ID and source port, before the real one.

The *Kaminsky attack* (2008) made this practical. A normal attacker has one chance per record, then must wait until the cached answer expires. Kaminsky queried *random* names under the target domain (aaaa1.example.com, aaaa2.example.com), which can never be cached, so each query started a fresh race; the forged answer carried an extra "authority" record that rewrote the name servers of the whole domain.

WORKED EXAMPLE

Why source-port randomisation mattered. Suppose the attacker sends 1,000 forged answers inside each race.

1. With a fixed port, only the 16-bit ID must be guessed: 1,000 / 65,536 ≈ 1.5 % per race. The chance of at least one win reaches 50 % after about 0.69 / 0.0153 ≈ 45 races, which takes seconds.
2. With a random port among about 64,000, the guess space is 65,536 × 64,000 ≈ 4.2×10^9, so the chance per race is 1,000 / 4.2×10^9 ≈ 2.4×10^{-7}. Reaching 50 % takes about three million races, which is hours or days of sustained traffic and easy to notice.

Result: 16 more random bits raise the attacker's work by about 65,000 times. Case randomisation of the query name (0x20) adds a few more bits.

Randomisation only raises the cost; *DNSSEC* removes the problem by signing the data (RFC 4033). Each zone has a key pair: the *DNSKEY* records publish the public keys, and an *RRSIG* record signs every record set. The parent zone vouches for the child's key with a *DS* record, which is a hash of the child's key-signing key, so the chain runs from the root (the resolver's *trust anchor*) through `com` to `example.com` (Figure 9.17). A validating resolver checks the chain and sets the AD (authenticated data) flag; if it fails, the client receives SERVFAIL. Denial of existence is signed too (NSEC and NSEC3 records). DNSSEC authenticates the origin of the data but encrypts nothing, and it does not protect the last hop between the client and its resolver unless that resolver is validating and trusted.

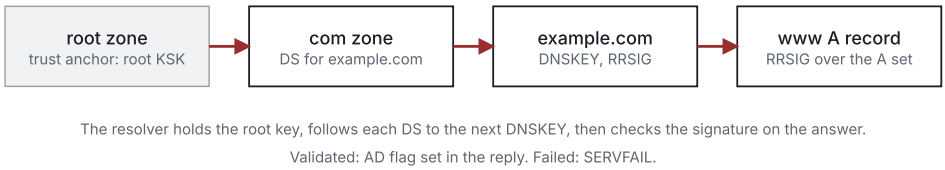


Figure 9.17 The DNSSEC chain of trust. Each DS record is a hash of the child zone's key, signed by the parent.

DNS over TLS (DoT, TCP 853) and *DNS over HTTPS* (DoH, TCP 443) encrypt the hop between client and resolver, which hides the queries from the local network and prevents tampering on that path. They do not prove that the answer is true, which is the job of DNSSEC. Because DoH looks like HTTPS, it also bypasses DNS-based filters, which is a concern for enterprise policy. Other threats: *DNS tunnelling* hides data in long, random-looking query names to an attacker's domain, detected by query length and volume; *open resolvers* are used as amplifiers and should answer only their own clients.

TRAP

Claim: DoH is DNSSEC, or makes DNS secure. *Why it is wrong:* DoH protects only the connection to the resolver; the resolver may still hold a poisoned answer. *Correct statement:* DoH and DoT give privacy and integrity on one hop; DNSSEC gives authenticity of the data from the zone owner. The two are complementary.

```
$ dig +dnssec cloudflare.com A
;; flags: qr rd ra ad;                  <- ad: chain validated
cloudflare.com. 300 IN RRSIG A 13 2 ... <- algorithm 13: ECDSA P-256
```

9.12 AAA, RADIUS, TACACS+ and 802.1X SHOULD · ■■□

BIG IDEA

> Network devices should not decide who may log in or connect; a central server does, so that identities, permissions and records live in one place and can be changed in one place.

AAA (Section 9.1 (p. 539)) is delivered by a central server, using one of two protocols. *RADIUS* is the standard for network access (users joining Wi-Fi, VPN or a switch port); *TACACS+* is preferred for administrators logging in to devices, because it can authorise each command.

	RADIUS	TACACS+
Transport	UDP 1812 (authentication), 1813 (accounting)	TCP 49
AAA separation	authentication and authorisation combined	three separate functions
Encryption	password attribute only	whole packet body
Per-command authorisation	no (coarse privilege level)	yes (command sets)
Standard	open (RFC 2865)	originally Cisco-proprietary
Typical use	802.1X, VPN, Wi-Fi	device administration

For device administration, `aaa authentication login default group tacacs+ local` tries the server first and falls back to a local account only when the server is unreachable; `aaa authorization commands 15` and `aaa accounting commands 15` then check and record every privileged command. A strong local fallback account is essential, or an outage of the AAA server locks everyone out.

IEEE 802.1X controls access at the port. There are three roles: the *supplicant* (the laptop), the *authenticator* (the switch or access point), and the *authentication server* (a RADIUS server). Before authentication the port passes only EAPOL frames (EAP over LAN), like a college gate where the student shows an identity card to the guard, who telephones the administration office before opening. The switch does not interpret the credentials; it relays *EAP* messages in RADIUS packets (Figure 9.18). The EAP *method* decides what is proved. *EAP-TLS* uses certificates on both sides and is the strongest, but needs a certificate on every device. *PEAP* builds a TLS tunnel authenticated by the server's certificate and carries a password exchange (MSCHAPv2) inside it.

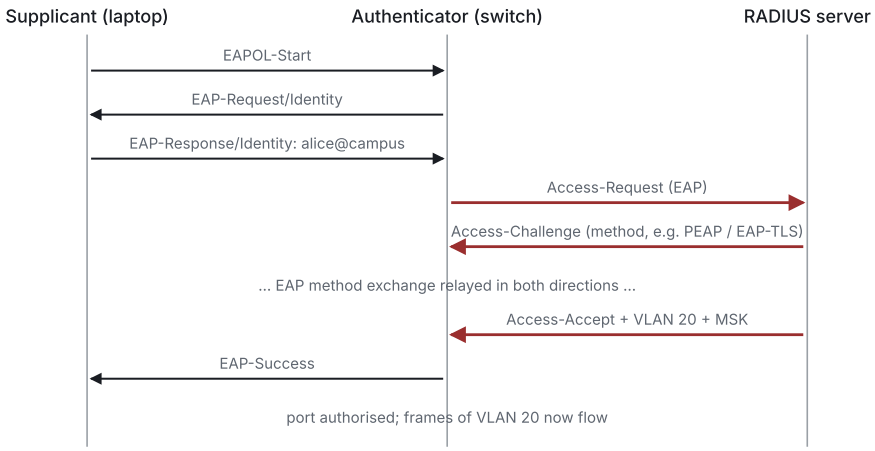


Figure 9.18 The 802.1X exchange for a wired port in Campus Net. The switch only relays EAP; the RADIUS server decides.

On success the RADIUS reply can include attributes that set the VLAN of the port (Tunnel-Type VLAN, Tunnel-Medium-Type 802 and Tunnel-Private-Group-ID), so that staff and students land in different VLANs on the same socket. Devices that cannot run a supplicant, such as printers, fall back to *MAC Authentication Bypass* (MAB), where the MAC address is the credential; it is weak, as MAC addresses are easily forged, so such devices go in a restricted VLAN. A policy server such as Cisco ISE adds *network access control*: it also checks device posture (patch level, antivirus) and may quarantine a non-compliant device.

```
aaa new-model
radius server ISE1
 address ipv4 10.0.0.20 auth-port 1812 acct-port 1813
 key <secret>
aaa authentication dot1x default group radius
dot1x system-auth-control
interface GigabitEthernet1/0/5
 switchport mode access
 authentication port-control auto
 authentication order dot1x mab
 dot1x pae authenticator
```

9.13 Detection and device hardening OPTIONAL · ■□□

BIG IDEA

Prevention fails sometimes, so we add detection to learn of the failure, and we harden devices to make the failure less likely in the first place.

An *intrusion detection system* (IDS) examines a copy of the traffic, taken from a SPAN port or an optical TAP, and raises alerts; an *intrusion prevention system* (IPS) sits inline and can drop or reset a flow (Figure 9.19). An IDS is like CCTV that alerts, an IPS is the guard who blocks. The price of blocking is risk: a false positive stops legitimate traffic, and an inline device that fails can cut the link unless it is set to fail open. *Signature-based* detection matches known patterns and has few false positives but misses novel attacks; *anomaly-based* detection flags deviations from a learned baseline and finds new attacks but raises more false alarms. Snort and Suricata are signature engines, and Zeek logs protocol behaviour for analysis. *NetFlow* and IPFIX records, which summarise flows without payload, reveal scans and exfiltration cheaply. A *SIEM* collects logs and flows from all sources and correlates them, for example “twenty failed logins, then a success, from a new country”.

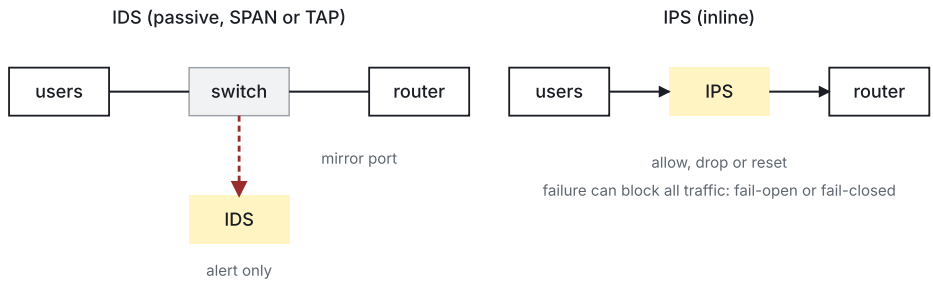


Figure 9.19 IDS watches a copy of the traffic and alerts; IPS sits in the path and drops.

A signature is a rule. This Suricata rule alerts when one source opens ten SSH connections to the web server within a minute, and the alert it produces names the rule, the class and the flow.

```
alert tcp $EXTERNAL_NET any -> 10.0.0.80 22 (msg:"SSH brute force";
flow:to_server;
  threshold:type both, track by_src, count 10, seconds 60; classtype:attempted-
admin; sid:1000001; rev:1;)
```

```
10/29/2026-14:02:11.512344  [**] [1:1000001:1] SSH brute force [**]
  [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP}
198.51.100.77:50122 -> 10.0.0.80:22
```

WORKED EXAMPLE

Why false positives dominate. Suppose 1 flow in 10,000 is an attack, and the sensor detects 99 % of attacks but wrongly flags 1 % of benign flows. In 1,000,000 flows there are 100 attacks, of which 99 are caught, and 999,900 benign flows, of which 9,999 are flagged. Of the 10,098 alerts only 99 are real, a precision of about 1 %.

Result: because attacks are rare, even a 1 % false-positive rate buries the true alerts, so rules must be tuned and alerts correlated before anyone is paged.

Device hardening reduces the attack surface of the routers and switches themselves. The following diff shows an insecure Campus Net router and its hardened version, with the reason for each change.

```
! before
service password-encryption
enable password cisco
username admin password cisco
line vty 0 4
 transport input telnet
snmp-server community public RO
ip http server
```

```
! after
enable algorithm-type scrypt secret <pw>
username admin privilege 15 algorithm-type scrypt secret <pw>
crypto key generate rsa modulus 3072
ip ssh version 2
line vty 0 4
 transport input ssh
 access-class MGMT-ONLY in
 exec-timeout 5 0
snmp-server group MON v3 priv
no ip http server
ip http secure-server
logging host 10.0.0.30
ntp authenticate
interface range GigabitEthernet1/0/2 - 24
 no cdp enable
```

Change	Reason
`scrypt` (type 9) or `sha256` (type 8) secrets	type 7 from `service password-encryption` is reversible in seconds and type 5 (MD5) is weak
SSH version 2 only, Telnet off	Telnet sends passwords in clear
`access-class` on VTY	only the management network may even attempt a login

SNMPv3 `priv`	v1 and v2c send the community string in clear; v3 authenticates and encrypts
HTTPS only	the HTTP server sends passwords in clear
Syslog and authenticated NTP	consistent timestamps and a record that an attacker cannot easily erase
CDP and LLDP off on user ports	they disclose model, software version and addresses

Further measures: *control-plane policing* (CoPP) rate-limits traffic addressed to the router so that a flood cannot starve routing protocols; a *management VRF* keeps management traffic off the production routing table; unused services and ports are shut down.

> TRAP
>
> *Claim:* `service password-encryption` protects the configuration's passwords. *Why it is wrong:* it applies the reversible type-7 obfuscation, which any tool decodes immediately. *Correct statement:* use `secret` commands with type 8 or 9 hashing and keep configuration files access-controlled.

9.14 Zero trust SHOULD · ■■□

> BIG IDEA
>
> Zero trust removes the assumption that a request from "inside" the network is safe, because attackers who get one foothold would otherwise move freely, so every request is decided on identity, device and context.

The classical design is a castle with a moat: a hard perimeter (firewall, VPN) and trust inside it. Its weakness is that one stolen laptop or phished password gives an intruder the whole interior. *Zero trust* (NIST SP 800-207) treats the network as hostile everywhere. Instead of one gate to the IT park, every office door has a badge reader that checks who you are and whether your laptop is patched, each time. Its seven tenets are listed in the Revision Spread.

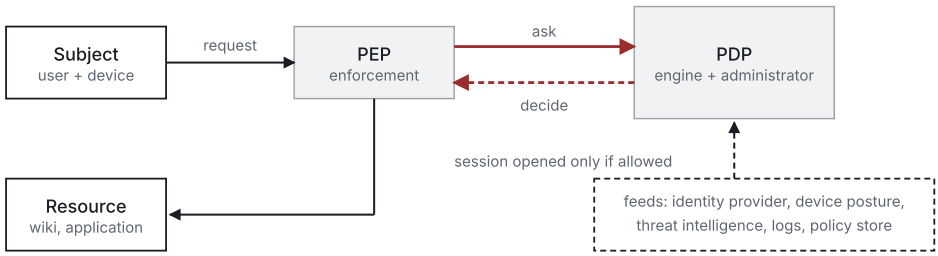


Figure 9.20 NIST SP 800-207 logical components: every request passes a Policy Enforcement Point, which asks the Policy Decision Point.

The architecture has three logical parts (Figure 9.20). The *Policy Enforcement Point* (PEP) sits in front of each resource and opens or closes the session. The *Policy Decision Point* is made of a *Policy Engine*, which computes the decision from the inputs, and a *Policy Administrator*, which tells the PEP and sets up the session. The inputs include the identity provider, device posture and compliance data, threat intelligence, activity logs, and the policy store itself. Decisions are made per request, not once per login.

	Perimeter model	Zero trust
Trust granted by	network location	identity, device state and context
Access given	to the network segment	to one application, per session
Lateral movement	easy once inside	limited by micro-segmentation
Remote user	full VPN	identity-aware proxy
Example	firewall plus VPN	Google BeyondCorp

Micro-segmentation applies the same idea inside data centres: workloads may talk only on explicitly permitted flows, enforced by host firewalls or a service mesh using mutual TLS. *SASE* and *SSE* are names for cloud services that deliver the PEP, secure web gateway and related functions to remote users; we need only the names.

WORKED EXAMPLE

A contractor needs one internal wiki. The VPN design gives the contractor's laptop an address on the staff network, from where every server is reachable. The zero-trust design publishes the wiki through an identity-aware proxy.

1. The contractor signs in at the proxy with the identity provider and multi-factor authentication.
2. The proxy (the PEP) asks the decision point. Inputs: the contractor's group, "contractor"; the device certificate and patch status; the time and location; the resource, "wiki"; the threat score of the session.
3. If all conditions hold, the proxy opens a session to the wiki only, for a limited time. Every request is checked again, and every decision is logged.
4. If the laptop's patch level changes, the next request is refused without any change to the network.

Result: access follows the person and the device to one application, not the address to a segment.

Migration from a VPN is gradual: inventory applications and identities first, put identity and device inventory under central control, publish the least critical application through the proxy, then move others, and retire the broad VPN last. The migration is mapped to the NIST components in Q9.30.

INTERVIEW LENS

GOOGLE · MICROSOFT · AMAZON

"What is zero trust?" The one-line answer is "never trust, always verify, per request", but the interviewer wants the mechanism: PEP, policy engine and administrator, the inputs to the decision, and what changes for a remote user. Expect: "Is it a product?" (no, an architecture) and "What does it not solve?" (a compromised but compliant device, and policy errors).

Problem Set

Every interview question of this pillar is solved once, here, in the compact template: a 30-second answer first, then approach, answer, follow-ups and mistakes where they apply. The tags give the companies and round where the topic was reported, the difficulty (■□□ easy, ■■□ medium, ■■■ hard), and **S** when candidates reported the specific question or **T** when only the topic was reported and the wording is ours. The Campus Net examples are those of the chapter. Q9.1 to Q9.16 follow; Q9.17 to Q9.32 are in the continuation.

Q9.1 What is the CIA triad? Give a network example of a breach of each.

CISCO · AMAZON · MICROSOFT · OA · ■□□ · S

30-second answer: confidentiality (only authorised readers), integrity (no undetected change), availability (reachable when needed); a sniffed Telnet password, a rewritten download, and a SYN flood breach them in that order.

Answer. Add authentication, authorisation, accounting and non-repudiation for who is acting; STRIDE maps each threat to the property it breaks (9.1 (p. 539)). **Follow-ups.** Which property does encryption raise and which may it lower? (Confidentiality up; the ability of an IDS to inspect down.) **Mistakes.** Treating availability as an operations matter only: DoS is a security attack.

Q9.2 Symmetric versus asymmetric encryption: why does TLS use both?

GOOGLE · CISCO · MICROSOFT · OA · ■□□ · S

30-second answer: symmetric is fast but needs a shared key; asymmetric needs no shared secret but is about a thousand times slower, so TLS uses asymmetric methods (ECDHE, signatures) once to agree keys and prove identity, then AES-GCM or ChaCha20 for the data.

Answer. Symmetric: one key, bulk data, AEAD modes give integrity too. Asymmetric: key pair, used for key agreement, signatures and certificates; RSA by factoring, ECC by discrete logarithm on curves, with smaller keys (9.2 (p. 541)). **Mistakes.** Saying that TLS encrypts the data with RSA.

Q9.3 What is a hash? Why is a hash alone not enough to authenticate a message?

AMAZON · CISCO · TECH-1 · ■□□ · S

30-second answer: a hash maps any input to a fixed-size digest that is one-way and collision-resistant; but anyone can recompute it after altering the message, so authentication needs a key: HMAC.

Answer. HMAC(K, m) = H((K $\oplus$ opad) ‖ H((K $\oplus$ ipad) ‖ m)) gives integrity and origin authentication to holders of K, and avoids the length-extension weakness of H(K ‖ m). A signature does the same without a shared key and adds non-repudiation. **Follow-ups.** Password storage needs a salted slow hash (Argon2, bcrypt), not HMAC. **Mistakes.** Using MD5 or SHA-1 for security.

Q9.4 What is a digital certificate and who signs it?

CISCO · GOOGLE · TECH-1 · ■□□ · S

30-second answer: an X.509 certificate binds a public key to a name (the SAN), and is signed by a certificate authority whose own certificate chains to a root in the client's trust store.

Answer. Fields: subject, SAN, issuer, validity, public key, key usage, Basic Constraints, revocation pointers, SCTs, signature. The leaf is signed by an intermediate CA; the intermediate by the root (9.3 (p. 543)). **Follow-ups.** How is a certificate obtained? (CSR, domain validation, ACME.) **Mistakes.** Thinking that a self-signed certificate is weaker in cryptography; it is simply untrusted by third parties.

Q9.5 What is the implicit rule at the end of every ACL? Where are standard and extended ACLs placed?

CISCO · JUNIPER · AMAZON · OA · ■□□ · S

30-second answer: an implicit `deny any`; standard ACLs (source only) go close to the destination, extended ACLs close to the source.

Answer. The list is read top to bottom, the first match decides, and nothing matching means deny. A standard ACL near the source would block that source's traffic to every destination, whereas an extended ACL can name the destination, so dropping early saves bandwidth (9.5 (p. 549)). The IOS syntax is in 4.13 (p. 279). **Follow-ups.** Inbound versus outbound; an outbound ACL does not filter router-originated packets. **Mistakes.** A list of denies with no final permit.

Q9.6 Compute the wildcard mask for 172.16.32.0/20.

CISCO · JUNIPER · OA · ■□□ · S

30-second answer: 0.0.15.255, matching 172.16.32.0 to 172.16.47.255.

Approach. The wildcard is 255 minus each subnet-mask octet. /20 is 255.255.240.0, so the wildcard is 0.0.(255 − 240).255 = 0.0.15.255. **Check.** The block size in the third octet is 256 − 240 = 16, starting at 32 (a multiple of 16), so the third octet runs 32 to 47. The method is that of Figure 9.8. **Mistakes.** Writing the subnet mask in the ACL, or choosing a network address that is not on a block boundary.

Q9.7 Stateless versus stateful firewall.

CISCO · JUNIPER · AMAZON · NOKIA · TECH-1 · ■□□ · S

30-second answer: a stateless filter judges each packet alone by header fields; a stateful firewall keeps a session table and automatically admits packets that belong to a permitted flow, in particular the replies.

Answer. The session entry holds the 5-tuple, state, timers; for TCP it checks the handshake and sequence window. Stateful gives safer defaults and simpler rules but costs memory per flow and breaks under asymmetric routing; the ACL is fast and hardware-based but cannot recognise replies (9.6 (p. 552)). **Mistakes.** Calling `established` in an ACL stateful; it matches only the ACK or RST flag.

Q9.8 IDS versus IPS.

CISCO · NOKIA · TECH-1 · ■□□ · T

30-second answer: an IDS watches a copy of the traffic and alerts; an IPS sits inline and can drop or reset flows, at the price that a false positive or a failure interrupts traffic.

Answer. Detection by signature (known patterns, few false positives) or by anomaly (baseline, finds novel attacks, more noise). Placement: SPAN or TAP for the IDS, in the path for the IPS, with fail-open or fail-closed decided in advance (9.13 (p. 569)). **Mistakes.** Putting an IPS in the path without a bypass.

Q9.9 Walk me through the TLS 1.3 handshake. Where does encryption start?

GOOGLE · MICROSOFT · AMAZON · TECH-1 · ■■□ · S

30-second answer: ClientHello with a key share; ServerHello with its key share, after which everything is encrypted (EncryptedExtensions, Certificate, CertificateVerify, Finished); client Finished; application data. One round trip.

Answer. (i) ClientHello: versions, suites, key share, SNI, ALPN. (ii) ServerHello: chosen suite and share; both derive handshake keys by ECDHE and HKDF. (iii) Server sends EncryptedExtensions, the certificate chain, a CertificateVerify signature over the transcript and a Finished MAC. (iv) The client verifies and sends Finished with the first request under application keys (9.4 (p. 546), Figure 9.5). **Follow-ups.** HelloRetryRequest when the guessed group is wrong; mutual TLS adds CertificateRequest. **Mistakes.** Saying that encryption starts after Finished; it starts after ServerHello.

Q9.10 Why is TLS 1.3 one round trip instead of two? What was removed from 1.2, and why?

GOOGLE · MICROSOFT · AMAZON · TECH-1 · ■■□ · S

30-second answer: the client sends its key share in the first message, so the server can finish key agreement in its first reply. Removed: static RSA key exchange (no forward secrecy), renegotiation, compression and weak ciphers (CBC, RC4, 3DES).

Answer. In 1.2 the client had to wait for the server's chosen parameters before sending its key exchange, which cost the second round trip. Version 1.3 allows only (EC)DHE or PSK, five AEAD suites, and encrypts the certificate. The table in 9.4 (p. 546) lists the differences. **Mistakes.** Forgetting that TCP's handshake comes first: total time to first byte is still 2 RTT before the request is answered.

Q9.11 What is forward secrecy and how does ECDHE provide it?

GOOGLE · MICROSOFT · AMAZON · TECH-2 · ■■□ · S

30-second answer: past sessions stay secret even if the server's long-term key is stolen later, because the session keys come from ephemeral Diffie-Hellman values that are deleted after use.

Answer. With RSA key transport the client encrypts the secret to the server's certificate key, so one stolen key decrypts every recorded session. With ECDHE both sides generate a fresh key pair per session; the certificate key only signs the handshake, and never protects a secret. A recorded session cannot be decrypted without the discarded ephemeral values (9.2 (p. 541)). **Follow-ups.** Session tickets can weaken it if their keys are long-lived. **Mistakes.** Thinking forward secrecy stops a live man in the middle; authentication does that.

Q9.12 What is 0-RTT and why is it dangerous for non-idempotent requests?

GOOGLE · AMAZON · TECH-2 · ■■□ · S

30-second answer: on resumption the client sends application data in its first flight under a key from the earlier PSK, saving a round trip; an attacker can replay that flight, so a server may process it twice.

Answer. Early data has no replay protection and no forward secrecy of its own. A replayed GET is harmless; a replayed "transfer 1,000 rupees" is not. Mitigations: accept 0-RTT only for idempotent methods, one-time tickets, a strike register in the server, and an `Early-Data` header so that the application can reject with 425 (9.4 (p. 546)). **Mistakes.** Enabling it on a payments endpoint.

Q9.13 How does a browser validate a certificate? What happens if the intermediate is missing?

GOOGLE · MICROSOFT · CISCO · TECH-1 · ■■□ · S

30-second answer: it builds a chain from the leaf to a trusted root, checking signatures, validity dates, name against the SAN, key usage and revocation; with a missing intermediate the path cannot be built and strict clients fail.

Answer. The six checks of Figure 9.4. Some browsers cache intermediates or fetch them through the AIA extension, so the site appears fine in a browser and fails in `curl`, Java or a mobile library with `unable to verify the first certificate`. The fix is to serve the full chain, leaf first (9.3 (p. 543), Lab 9.1). **Follow-ups.** An expired leaf and a hostname mismatch give different errors; the client aborts with a fatal alert. **Mistakes.** Installing the root on the server instead of the intermediate.

Q9.14 Explain IPsec tunnel versus transport mode and AH versus ESP. Why does AH not work through NAT?

CISCO · JUNIPER · NOKIA · TECH-1 · ■■□ · S

30-second answer: transport mode protects the payload and keeps the original IP header (host to host); tunnel mode encrypts the whole packet inside a new IP header (gateway to gateway). ESP (protocol 50) encrypts and authenticates; AH (51) authenticates only, and its integrity check covers the outer IP header, so NAT rewriting the address breaks it.

Answer. Figure 9.10 shows the formats. ESP in tunnel mode is the norm; NAT traversal wraps ESP in UDP 4500 because ESP has no ports for a NAT to translate (9.7 (p. 555)). **Mistakes.** Calling AH "more secure" for authenticating more of the header.

Q9.15 Explain IKEv1 Phase 1 and Phase 2. How does IKEv2 differ? How many messages does each use?

CISCO · JUNIPER · NOKIA · TECH-2 · ■■□ · S

30-second answer: IKEv1 Phase 1 builds the IKE SA (6 messages in main mode, 3 in aggressive) and Phase 2 quick mode builds the IPsec SAs (3 messages); IKEv2 does both in 4 messages, IKE_SA_INIT and IKE_AUTH, with CREATE_CHILD_SA for rekeying.

Answer. Aggressive mode is faster but sends identities in clear and exposes a hash for offline PSK guessing. IKEv2 adds built-in liveness, NAT-T and EAP, and a simpler state machine (Figure 9.11, 9.7 (p. 555)). **Follow-ups.** What does PFS add? (A fresh DH exchange per Child SA.) **Mistakes.** Stating nine messages for IKEv2.

Q9.16 A site-to-site VPN shows Phase 1 up, but no traffic passes. What do you check?

CISCO · JUNIPER · NOKIA · TECH-2 · ■■□ · S

30-second answer: Phase 1 only proves that the peers authenticated; check the Child SA (selectors or crypto ACL mirroring, transform set, PFS group), the routing into the tunnel, NAT exemption, and the encaps/decaps counters.

Approach. (i) `show crypto ikev2 sa` then `show crypto ipsec sa`: are there inbound and outbound SPIs, and do both counters rise? Encaps rising with zero decaps means the far end does not return. (ii) Selectors: 172.16.50.0/24 on one side and /25 on the other gives `TS_UNACCEPTABLE`. (iii) Transform and PFS mismatches show as `NO_PROPOSAL_CHOSEN`. (iv) A route or interface-bound policy must send the traffic into the tunnel. (v) A NAT rule must exempt VPN traffic, so that it is not translated before encryption. (vi) Reduce MSS if large packets die while pings work (9.8 (p. 558)). The full case is Case 15 (p. 667). **Mistakes.** Concluding "the tunnel is up" from Phase 1.

Problem Set, continued

Q9.17 What is NAT-T and which ports does IPsec use?

CISCO · JUNIPER · NOKIA · TECH-1 · ■■□ · S

30-second answer: IKE uses UDP 500; ESP is IP protocol 50 and has no ports; NAT traversal detects a NAT during IKE and moves IKE and ESP to UDP 4500, with ESP wrapped in a UDP header.

Answer. IKE sends hashes of the addresses and ports in its first messages; if they differ from what the peer sees, a NAT lies between them. ESP packets then carry an 8-byte UDP header (RFC 3948), and keepalives hold the NAT mapping open. AH cannot work at all (9.7 (p. 555)). **Mistakes.** Opening only UDP 500 on the firewall, so that Phase 1 succeeds and data fails.

Q9.18 Explain ARP spoofing and how DAI stops it. What does DAI depend on?

CISCO · JUNIPER · NOKIA · TECH-1 · ■■□ · S

30-second answer: the attacker sends forged ARP replies claiming the gateway's IP at his MAC (and the victim's IP at his MAC), becoming a man in the middle; DAI checks each ARP packet on untrusted ports against the DHCP snooping binding table and drops mismatches.

Answer. ARP has no authentication and hosts accept gratuitous replies. DAI therefore depends on the snooping table (IP, MAC, VLAN, port) or on static ARP ACLs for fixed-address hosts; uplinks and the DHCP server port must be trusted, and a rate limit protects the CPU. Without snooping DAI drops everything, including valid ARP (9.9 (p. 560)). **Mistakes.** Enabling DAI before snooping has built its table.

Q9.19 What is VLAN hopping? Explain both methods and their mitigations.

CISCO · JUNIPER · NOKIA · TECH-1 · ■■□ · S

30-second answer: reaching a VLAN one does not belong to: by switch spoofing (negotiating a trunk with DTP) or double tagging (an outer native-VLAN tag stripped by the first switch exposes an inner tag); disable DTP on access ports and use an unused native VLAN.

Answer. Switch spoofing: the port is `dynamic auto` or `desirable`, so the attacker's host forms a trunk and sees all VLANs; use `switchport mode access` and `nonegotiate`. Double tagging: the attacker is in the native VLAN, the first switch strips the outer tag, the second delivers by the inner tag; one-way only. Use an unused native VLAN, shut unused ports, tag the native VLAN where supported (9.9 (p. 560)). **Mistakes.** Believing that VLAN 1 is harmless because it is "default".

Q9.20 What is a SYN flood? How do SYN cookies work?

GOOGLE · MICROSOFT · AMAZON · CISCO · TECH-1 · ■■□ · S

30-second answer: a flood of SYNs from spoofed sources fills the server's half-open queue so that real clients are refused; with SYN cookies the server keeps no state and encodes a counter, an MSS index and a keyed hash of the 4-tuple in the SYN-ACK's sequence number, validating it when the ACK returns.

Answer. The valid ACK (acknowledgement = cookie + 1) proves the client received the SYN-ACK, which a spoofed source cannot do. Cost: TCP options such as window scale must be encoded or lost, so the kernel enables cookies only when the queue overflows. Other defences: larger backlog, SYN proxy at a firewall, rate limits, upstream filtering (9.10 (p. 563), Lab 9.4). **Mistakes.** Saying cookies increase the queue.

Q9.21 What is DNS cache poisoning? Why did source-port randomisation help so much?

GOOGLE · MICROSOFT · AMAZON · TECH-2 · ■■□ · S

30-second answer: forged answers that match the query's name, 16-bit ID and port, arriving before the real one, are cached and served to all users; randomising the source port adds about 16 more unknown bits, multiplying the attacker's work by about 65,000.

Answer. The worked probabilities in 9.11 (p. 566): about 1.5 % per race with a fixed port against about 2×10^{-7} with a random one, for 1,000 forged packets. Kaminsky's random sub-domains defeated the cache's protection by making each attempt a new race. Further defences: 0x20 case randomisation, query rate limits, DNSSEC. **Mistakes.** Claiming that randomisation prevents poisoning; it only makes it expensive.

Q9.22 RADIUS versus TACACS+: which for device administration, and why?

CISCO · JUNIPER · NOKIA · TECH-1 · ■■□ · S

30-second answer: TACACS+ for device administration: it separates authentication, authorisation and accounting, authorises each command, runs over TCP 49 and encrypts the whole body; RADIUS (UDP 1812 and 1813) suits network access such as 802.1X and VPN.

Answer. RADIUS combines authentication and authorisation and hides only the password attribute, but it is an open standard supported by every vendor and by Wi-Fi and VPN gear; TACACS+ was Cisco's, now widely supported. Keep a local fallback account for the day the server is unreachable (9.12 (p. 567)). **Mistakes.** Using RADIUS where command authorisation is required.

Q9.23 Explain the 802.1X roles and the message flow.

CISCO · MICROSOFT · NOKIA · TECH-2 · ■■□ · S

30-second answer: the supplicant (client) and authenticator (switch) exchange EAPOL; the switch relays EAP to the authentication server (RADIUS), which accepts or rejects and may return a VLAN; only then is the port authorised.

Answer. Figure 9.18: EAPOL-Start, identity request and response, Access-Request, an EAP method exchange (EAP-TLS or PEAP) relayed in Access-Challenges, Access-Accept with VLAN attributes, EAP-Success. Before that only EAPOL passes. Devices without a supplicant use MAB, which is weak (9.12 (p. 567)). **Follow-ups.** Which method is strongest? (EAP-TLS.) **Mistakes.** Saying the switch checks the password.

Q9.24 What is uRPF? Strict versus loose mode, and where is loose required?

CISCO · JUNIPER · AMAZON · TECH-2 · ■■□ · S

30-second answer: uRPF drops a packet whose source address has no route back through the arrival interface (strict) or no route at all (loose), implementing BCP 38; loose is required on multihomed borders where return paths are asymmetric.

Answer. Strict suits single-homed customer edges. On BR1 and BR2 with two providers, a valid packet from a remote network may arrive on the interface that is not the best path back, so strict mode would drop it; loose mode still drops unrouted (bogon or unallocated) sources (9.10 (p. 563)). **Mistakes.** Applying strict mode where routing is asymmetric.

Q9.25 A 400 Gb/s amplification DDoS hits your customer's IP. Walk through mitigation from the first minute to steady state.

GOOGLE · AMAZON · MICROSOFT · TECH-2 · ■■■ · S

30-second answer: confirm and classify from flow data; protect capacity by diverting to a scrubbing centre (or black-holing the one target if it cannot be saved); filter the reflected protocol with Flowspec and ACLs; coordinate with upstreams; then harden and tune.

Structured answer.

1. *Minute 0 to 5, detect and classify.* NetFlow or IPFIX and SNMP show a surge to one address, UDP from source port 53 or 123 or 11211, huge packets. Declare an incident and name an owner.
2. *Contain.* The link is saturated, so local filtering cannot help; only upstream capacity can. Advertise the victim prefix to the scrubbing provider (BGP diversion or anycast), or, if the customer is expendable for now, RTBH the single /32 (it completes the denial of service but saves everything else).
3. *Filter.* Ask upstreams to apply Flowspec or ACLs on UDP source port 11211 and 123, fragments and oversized packets; rate-limit the reflected services.
4. *Steady state.* Return clean traffic over a tunnel, confirm the customer's service, keep the diversion until the attack stops, then withdraw it.
5. *After.* Check BCP 38 and open resolvers on one's own network, publish the timeline, add always-on protection.

Mistakes. Relying on the customer's firewall, which sees the flood after the link is full. **Follow-ups.** Why not block at the origin? (Spoofing hides it.)

Q9.26 How does DNSSEC establish trust? What does it not protect against? How is DoH different?

GOOGLE · MICROSOFT · AMAZON · TECH-2 · ■■■ · S

30-second answer: a chain of signed DS and DNSKEY records from the root trust anchor down to the zone, so a validating resolver can verify the RRSIG on an answer; it does not encrypt queries, hide them from observers, or help against a compromised resolver or registrar. DoH encrypts the client-resolver hop and proves nothing about the data.

Answer. Figure 9.17 shows the chain. Not covered: privacy, availability (signed answers make amplification worse), unsigned zones, and the last hop if the client does not validate. DoH and DoT give privacy and integrity on one hop; with a malicious resolver they offer no truth (9.11 (p. 566)). **Mistakes.** Equating the two.

Q9.27 A stateful firewall pair in active/active drops 30 % of return traffic. Diagnose.

CISCO · JUNIPER · AMAZON · NOKIA · TECH-2 · ■■■ · S

30-second answer: asymmetric routing: some replies return through the unit that did not see the SYN and so has no session; fix with symmetric paths, state synchronisation, or a single logical firewall.

Approach. The drop fraction (about a third) suggests that one of several equal paths or ECMP hash outcomes returns through the other unit. Check, in order: (i) drop counters and logs naming "no session" or "invalid state"; (ii) the session table of each unit for the failing flow; (iii) whether state sync is enabled and fast enough for the first packets (a reply can overtake the sync); (iv) the routing on both sides, whether ECMP or BGP policy sends return traffic via the second border. **Fix.** Make the paths symmetric (local preference and AS-path prepending, or a hash by source and destination that is symmetric); or active/standby; or session sync with a proper cluster (9.6 (p. 552), Figure 9.9). The corresponding case is Case 13 (p. 664). **Mistakes.** Disabling inspection to "make it work".

Q9.28 Design the network security for a new 2,000-student campus.

CISCO · AMAZON · MICROSOFT · TECH-2 · ■■■ · T

30-second answer: segment by role, authenticate every port and SSID with 802.1X, filter between segments with a firewall and ACLs, protect the edge, harden the switches, and log everything to a SIEM.

Structured answer.

Layer	Design
Segmentation	VLANs and subnets for staff, students, servers, guests, IoT, management; routing between them only through the firewall or filtered at the distribution layer
Access	802.1X (EAP-TLS for staff, PEAP or a portal for students), MAB for printers, dynamic VLAN, guest Wi-Fi isolated and rate-limited
Switch hardening	DHCP snooping, DAI, port security, BPDU Guard, unused ports off, native VLAN unused (9.9 (p. 560))
Edge	active/standby NGFW pair with NAT, IPS, URL filtering, uRPF, BGP filters and RPKI, DDoS service
Remote access	TLS or IKEv2 VPN with MFA; or identity-aware proxy (9.14 (p. 571))
Management	TACACS+ with local fallback, SSH only, management VRF, SNMPv3, NTP, syslog to a SIEM
Data and apps	TLS everywhere, certificates by ACME, backups offline

Follow-ups. How are students on personal devices handled? (Separate VLAN, posture check, no access to servers except by proxy.) **Mistakes.** Starting with products instead of assets and threats (9.1 (p. 539)).

Q9.29 Explain BGP hijacking and how RPKI route-origin validation mitigates it. What are its limits?

GOOGLE · AMAZON · JUNIPER · TECH-2 · ■■■ · S

30-second answer: an AS announces a prefix it does not own (or a more specific one) and attracts traffic; with RPKI the owner signs a ROA naming the allowed origin AS and maximum length, and validating routers drop invalid announcements. ROV checks only the origin, so a forged origin or a path attack passes.

Answer. Valid, invalid and not-found are the three states; coverage is incomplete, so not-found routes are still accepted. Limits: no path validation (ASPA and BGPsec address it), loose `maxLength` allows a more specific hijack inside the ROA, and route leaks are not hijacks. Add prefix lists from the IRR, max-prefix limits and monitoring (9.10 (p. 563), 4.7 (p. 256)). **Mistakes.** Thinking that RPKI encrypts or authenticates BGP sessions.

Q9.30 What is zero trust? How would you migrate a VPN-based company to it? Map it to NIST SP 800-207.

GOOGLE · MICROSOFT · AMAZON · TECH-2 · ■■■ · S

30-second answer: no implicit trust from network location; every request is authenticated, authorised and checked against device state, per session. Migrate application by application behind an identity-aware proxy, then retire the broad VPN.

Structured answer.

1. *Inventory* users, devices, applications and flows.
2. *Identity first:* central identity provider with MFA; device certificates and an inventory with posture signals (the identity management and CDM feeds of the 800-207 model).
3. *Put a Policy Enforcement Point in front of the first, least critical applications:* an identity-aware proxy. The policy engine and administrator are its decision service; policy is written per application and group.
4. *Observe, then enforce:* log decisions in a SIEM first, then block.
5. *Move the rest*, micro-segment the data centre, and retire full-tunnel access.

Follow-ups. What stays? (Firewalls and segmentation remain; the change is that location does not grant trust.) **Mistakes.** Buying “a zero-trust product” and keeping flat internal access (9.14 (p. 571)).

Q9.31 TLS inspection on an NGFW: how does it work, what breaks, and what are the privacy and legal considerations?

CISCO · JUNIPER · MICROSOFT · TECH-2 · ■■■ · T

30-second answer: the firewall terminates the client's TLS session with a certificate it forges on the fly under an enterprise CA the clients trust, inspects the plaintext, and opens a second TLS session to the server; pinning and mutual TLS break, and the firewall's CA key becomes a prize.

Answer. Breaks: apps with certificate pinning, client-certificate (mTLS) services, and TLS 1.3 features such as ECH. Practice: exempt banking, health and other sensitive categories, keep the CA key in an HSM, tell users by policy, keep logs short, and verify server certificates on the outbound leg, since otherwise the firewall hides the server's errors from the user (9.6 (p. 552)). Legal: consent and data-protection duties apply to employee and student traffic. **Mistakes.** Inspecting everything and storing plaintext.

Q9.32 Compare IPsec, WireGuard and TLS VPNs for 10,000 remote employees: performance, NAT traversal, management.

CISCO · JUNIPER · AMAZON · TECH-2 · ■■■ · T

30-second answer: for managed laptops at scale choose a TLS/DTLS or IKEv2 client with certificates and MFA and central policy; IPsec performs well and is hardware-accelerated; WireGuard is the fastest and simplest, but its static key model needs a management layer; TLS VPN passes the most firewalls.

	IPsec (IKEv2)	WireGuard	TLS/DTLS VPN
Throughput	high, hardware offload on gateways	very high, in-kernel, small code	good with DTLS, lower over TCP
NAT and firewall traversal	NAT-T on UDP 4500; may be blocked	UDP only, any port	TCP 443 passes nearly everywhere
Management at 10,000 users	mature: certificates, EAP, per-user policy	peers are keys, so provisioning, revocation and per-user logging need extra tooling	mature: SAML, MFA, posture, per-app policy
Failure modes	MTU, selector mismatches	no built-in identity beyond keys	TCP-over-TCP stalls

Answer. Decide by what must be managed: identity and posture (TLS or IKEv2 products), performance and simplicity for site links (WireGuard), and reach through hostile networks (TLS on 443). **Follow-ups.** Why prefer an identity-aware proxy for individual apps? (9.14 (p. 571)). **Mistakes.** Judging only by raw throughput.

Revision Spread

THE 15-MINUTE VERSION

Ten facts. (1) CIA plus AAA; STRIDE maps threats to properties; defence in depth means independent rings. (2) Symmetric for bulk (AES-GCM, ChaCha20), asymmetric to agree keys and sign, hash for integrity, HMAC for keyed integrity; ephemeral DH gives forward secrecy. (3) A certificate is a CA signature over a name and a key; the server must send leaf and intermediates; the client checks the SAN, the dates, the chain and revocation. (4) TLS 1.3: one round trip, encryption starts after ServerHello, certificate encrypted, 0-RTT can be replayed. (5) ACL: first match, implicit deny, extended near the source, standard near the destination, wildcard is the inverse of the mask. (6) Stateful firewalls need both directions of a flow through one device. (7) IPsec: ESP 50, AH 51, IKE UDP 500, NAT-T UDP 4500; IKEv2 is four messages; "phase 1 up" never means the VPN works. (8) Layer-2 controls: DHCP snooping feeds DAI; port security stops MAC flooding; `nonegotiate` and an unused native VLAN stop hopping. (9) SYN cookies keep no state; amplification is stopped at the source by BCP 38 and at the victim by upstream scrubbing; DNSSEC signs data, DoH hides the hop. (10) TACACS+ for admins, RADIUS for access, 802.1X relays EAP; zero trust decides per request.

Three figures to redraw. The TLS 1.3 ladder with the encrypted part shaded (Figure 9.5), the IKEv2 four-message ladder (Figure 9.11), and the ESP packet formats (Figure 9.10).

Attack, defence and command

Attack and defence	Command
ARP spoofing: Dynamic ARP Inspection	`ip arp inspection vlan 10,20`; uplink `ip arp inspection trust`
Rogue DHCP, starvation: DHCP snooping	`ip dhcp snooping vlan 10,20`; uplink `ip dhcp snooping trust`; `limit rate 15`
MAC flooding: port security	`switchport port-security maximum 2`; `violation restrict`; `mac-address sticky`
Switch spoofing: no DTP	`switchport mode access`; `switchport nonegotiate`
Double tagging: unused native VLAN	`switchport trunk native vlan 999`
STP root takeover: BPDU Guard, Root Guard	`spanning-tree bpduguard enable`; `spanning-tree guard root`
Spoofed sources: uRPF, BCP 38, IP Source Guard	`ip verify unicast source reachable-via rx` (or `any`); `ip verify source`
SYN flood: SYN cookies	`sysctl -w net.ipv4.tcp_syncookies=1`
Amplification: BCP 38, closed resolvers, scrubbing, RTBH	upstream; Flowspec
BGP hijack: RPKI ROV, prefix filters, max-prefix	`neighbor ... maximum-prefix`; validating RPKI session
DNS poisoning: port randomisation, 0x20, DNSSEC	validating resolver; `dig +dnssec`
Telnet, clear passwords: SSH 2, secret type 8 or 9	`transport input ssh`; `algorithm-type scrypt secret`
Weak SNMP: SNMPv3 with privacy	`snmp-server group MON v3 priv`
Rogue device on a port: 802.1X	`authentication port-control auto`
Lateral movement: segmentation, zero trust	firewall zones; identity-aware proxy

Ports and protocols for security services

Service	Port or protocol
SSH · Telnet (clear)	TCP 22 · TCP 23
DNS · DoT · DoH	53 · TCP 853 · TCP 443
HTTPS · QUIC	TCP 443 · UDP 443

IKE · NAT-T	UDP 500 · UDP 4500; ESP is IP protocol 50, AH 51
WireGuard	UDP 51820 (default)
RADIUS · TACACS+	UDP 1812 and 1813 · TCP 49
Syslog · SNMP · NTP	UDP 514 · UDP 161 and 162 · UDP 123
LDAP · LDAPS · Kerberos	389 · 636 · 88
DHCP (watched by snooping)	UDP 67 and 68

Cryptography at a glance

Primitive	Gives, and the pitfall
Symmetric AEAD (AES-GCM, ChaCha20)	confidentiality and integrity; nonce reuse, ECB
Hash (SHA-256)	fingerprint; not authentication, not for passwords
HMAC	keyed integrity; short or reused key
Signature (ECDSA, Ed25519, RSA-PSS)	origin, non-repudiation; not “encrypt with the private key”
(EC)DH (X25519, P-256)	shared secret; no authentication alone
RSA key transport	secret to a public key; no forward secrecy, gone from TLS 1.3
Equal strength	256-bit EC, 3072-bit RSA, 128-bit AES (RSA-2048 is about 112 bits)

TLS, IKE, ACL, VPN quick facts

Topic	Facts
TLS 1.3 against 1.2	1 RTT against 2; no RSA key transport, renegotiation, compression or CBC; certificate encrypted; five AEAD suites
TLS 1.3 messages	ClientHello, ServerHello, then encrypted EncryptedExtensions, Certificate, CertificateVerify, Finished, client Finished
IKEv1 against IKEv2	6 (main) or 3 (aggressive) plus 3 quick; IKEv2 is 4: IKE_SA_INIT ×2, IKE_AUTH ×2; CREATE_CHILD_SA rekeys
IPsec mismatch checklist	encryption, hash or PRF, DH group, authentication and PSK, lifetime, selectors (proxy-IDs), PFS, NAT exemption, routing
ESP tunnel overhead (IPv4, GCM)	54 to 57 bytes, plus 8 for NAT-T; MSS about 1395 on a 1500 link, commonly 1360
ACL placement	extended near the source, standard near the destination; inbound drops before routing
Wildcard masks	/8 0.255.255.255 · /16 0.0.255.255 · /20 0.0.15.255 · /22 0.0.3.255 · /24 0.0.0.255 · /25 0.0.0.127 · /26 0.0.0.63 · /27 0.0.0.31 · /28 0.0.0.15 · /29 0.0.0.7 · /30 0.0.0.3 · host 0.0.0.0
VPN choice	site link: IPsec or WireGuard; managed laptops: IKEv2 or TLS; hostile networks: TLS on 443
DDoS classes	volumetric (scrubbing, RTBH), protocol (cookies, proxies), application (CDN, WAF)

Zero-trust tenets (NIST SP 800-207). (1) All data sources and computing services are resources. (2) All communication is secured regardless of network location. (3) Access is granted per session. (4) Access is decided by dynamic policy (identity, device, behaviour). (5) The enterprise monitors the integrity and posture of all assets. (6) Authentication and authorisation are strictly enforced before access. (7) The enterp-

rise collects as much information as it can about the state of assets and traffic to improve policy.

Flashcards

Q	A
CIA?	Confidentiality, integrity, availability
STRIDE?	Spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege
Sign or encrypt with which key?	Sign with the private key; encrypt with the public key
Why HMAC, not H(K‖m)?	Length-extension attack on SHA-2
Toy DH: p 23, g 5, a 6, b 15?	A = 8, B = 19, K = 2
Forward secrecy?	Ephemeral DH keys; a stolen long-term key opens no past session
Chain of trust?	Leaf, intermediate, root in the trust store
Missing intermediate?	`unable to verify the first certificate`; send the full chain
Revocation methods?	CRL, OCSP, OCSP stapling (must-staple)
Certificate Transparency?	Public logs of every certificate; SCTs in the certificate
Where does TLS 1.3 encryption start?	After ServerHello
What is in the TLS 1.3 ClientHello?	Versions, suites, key share, SNI, ALPN
0-RTT risk?	Replay of early data
What does SNI leak?	The site name, unless ECH is used
Implicit ACL rule?	`deny any`
Standard ACL placement?	Near the destination
Wildcard of /20?	0.0.15.255
Stateless versus stateful?	Per-packet against session table
Asymmetric routing and firewalls?	The reply meets a unit with no session; dropped
Order of NAT and filter in Linux?	DNAT in prerouting before the filter; SNAT in postrouting after
ESP versus AH?	Encrypts and authenticates (50) versus authenticates only, NAT-hostile (51)
IKE and NAT-T ports?	UDP 500 and UDP 4500
IKEv2 messages?	Four
Phase 1 up, no traffic?	Selectors, transform and PFS, routing, NAT exemption
DAI depends on?	DHCP snooping binding table
Port security violation modes?	Protect, restrict, shutdown
Two VLAN-hopping methods?	DTP switch spoofing; double tagging
What does BPDU Guard do?	Err-disables an edge port that receives a BPDU
SYN cookie contents?	Counter, MSS index, keyed hash of the 4-tuple
uRPF strict versus loose?	Same interface versus any route; loose for multihoming
Kaminsky attack?	Random sub-domains give unlimited poisoning races
DNSSEC records?	DNSKEY, RRSIG, DS (plus NSEC or NSEC3)
DoH is not?	DNSSEC; it protects one hop only
RADIUS versus TACACS+?	UDP 1812 combined AAA, password-only encryption, versus TCP 49 separated AAA, full encryption
802.1X roles?	Supplicant, authenticator, authentication server
ROV limit?	Checks only the origin AS, not the path
Type 7 password?	Reversible; use type 8 or 9 secrets

Self-test

1. Which TLS 1.3 message is the first to be encrypted? (a) ClientHello (b) ServerHello (c) EncryptedExtensions (d) client Finished.
2. The wildcard mask for 10.8.0.0/13 is (a) 0.7.255.255 (b) 0.8.0.0 (c) 0.0.7.255 (d) 0.255.255.255.
3. Which feature stops ARP spoofing on an access switch? (a) port security (b) BPDU Guard (c) Dynamic ARP Inspection (d) uRPF.
4. A VPN shows IKE SA READY but no traffic. The first thing to compare is (a) the DH group of IKE_SA_INIT (b) the traffic selectors of the Child SA (c) the NTP time (d) the VLAN.
5. TACACS+ runs over (a) UDP 49 (b) TCP 49 (c) UDP 1812 (d) TCP 1812.
6. SYN cookies store the connection state in (a) a hash table (b) the SYN queue (c) the initial sequence number (d) a DNS record.
7. A standard ACL goes (a) near the source (b) near the destination (c) on the VTY only (d) outbound on any port.
8. DNSSEC gives (a) privacy of queries (b) authenticity of the data (c) protection of the client-resolver hop (d) availability.
9. AH fails through NAT because (a) it uses UDP (b) its check covers the outer IP header (c) it has no SPI (d) it encrypts ports.
10. ROV validates (a) the AS path (b) the origin AS of a prefix (c) the BGP session (d) the next hop.

Answers. 1 c; 2 a (/13 gives 255.248 in the second octet, so 255 − 248 = 7); 3 c; 4 b; 5 b; 6 c; 7 b; 8 b; 9 b; 10 b.

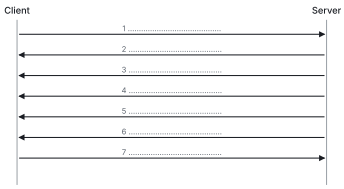


Figure 9.21 Self-test: name the seven numbered messages and mark where encryption begins. The answer is in the TLS 1.3 ladder of this chapter.

Answer to the figure. 1 ClientHello; 2 ServerHello (encryption begins after it); 3 EncryptedExtensions; 4 Certificate; 5 CertificateVerify; 6 server Finished; 7 client Finished.

PART I · PILLAR 10

10

Network Automation and Programmability

From typing on one switch to pushing, checking and rolling back configuration on fifty devices from Git, and defending every choice in an interview.

10.1 Why automate, and the maturity ladder	MUST
10.2 JSON, YAML and XML	MUST
10.3 Paramiko: SSH from first principles	OPTIONAL
10.4 Netmiko: multi-vendor SSH	MUST
10.5 Parsing CLI output	SHOULD
10.6 Jinja2 and the source of truth	MUST
10.7 NAPALM	SHOULD
10.8 Nornir	OPTIONAL
10.9 Ansible for networks	SHOULD
10.10 YANG data models	MUST
10.11 NETCONF and RESTCONF	MUST
10.12 gNMI and streaming telemetry	SHOULD
10.13 Git for network engineers	MUST
10.14 CI/CD and testing	SHOULD
10.15 Safety at scale	OPTIONAL

Every example in this chapter belongs to **Campus Net**. Its access layer is three Cisco IOS XE switches, `acc1` to `acc3`; its distribution layer is `dist1` and `dist2` (IOS XE) and its border router is `br1` (IOS XE). Its data-centre extension (5.2 (p. 305)) is two Arista cEOS leaves, `leaf1` and `leaf2`, under one FRR spine, `spine1`. Management addresses are `10.255.0.0/24`, and the VLANs are SALES (10, 192.168.10.0/24) and ENG (20, 192.168.20.0/24). The Python language itself is taught in 2.16 (p. 134) to 2.18 (p. 137), and the shell and `jq` in 8.2 (p. 485); this chapter teaches the *network* libraries and the practice around them. Every script has been written to be syntactically valid Python 3.10 or later; the libraries are third-party and must be installed with `pip` in a virtual environment.

10.1 Why automate, and the maturity ladder MUST · ■□□

A network is changed by people typing commands, and the commonest cause of an outage is a change that went wrong (11.1 (p. 635)). Typing does not scale because each additional device costs the same minutes again, and it does not leave evidence because the only record is the shell history on the engineer's laptop. Automation replaces the typing with a program, and in doing so offers four things: *consistency* (every device receives the same lines), *speed* (forty devices take the time of one), *auditability* (the program, its input and its output are stored) and *repeatability* (a change that worked in the lab can be replayed exactly). The risk is the mirror image: a wrong change is also applied consistently and quickly, which is why §10.15 occupies a section of its own.

WORKED EXAMPLE

Adding VLAN 30 (GUEST) to 40 access switches.

1. By hand. Each switch needs a login, `vlan 30`, `name GUEST`, a trunk check and `write memory`: about six minutes with care. Forty switches take 40 × 6 = 240 min, four hours.
2. Typing errors. Suppose each switch has a 3 % chance of a slip (wrong VLAN on a trunk, a missed save). The chance that *all* forty are right is 0.97^40 ≈ 0.30, so there is a 70 % chance of at least one wrong switch, and an expected 1.2 wrong ones.
3. By script. Each switch takes about 9 s (connect 4 s, configure 5 s). With 8 worker threads the 40 switches finish in 5 rounds × 9 s = 45 s; reading the log and the diff adds about five minutes. The script gives the same lines to every switch, so the typo rate is the rate of one typo, in one file, where a reviewer can see it.

Result: about 6 min against 4 h, and the residual risk is one reviewable mistake instead of forty unrecorded ones.

BIG IDEA

Automation does not remove mistakes; it moves them from forty unrecorded keystroke sessions into one artefact that can be reviewed, tested and replayed.

The maturity ladder. Teams do not jump to full automation; they climb. We number the steps L0 to L4, and each step is defined by *where the intended state of the network lives*.

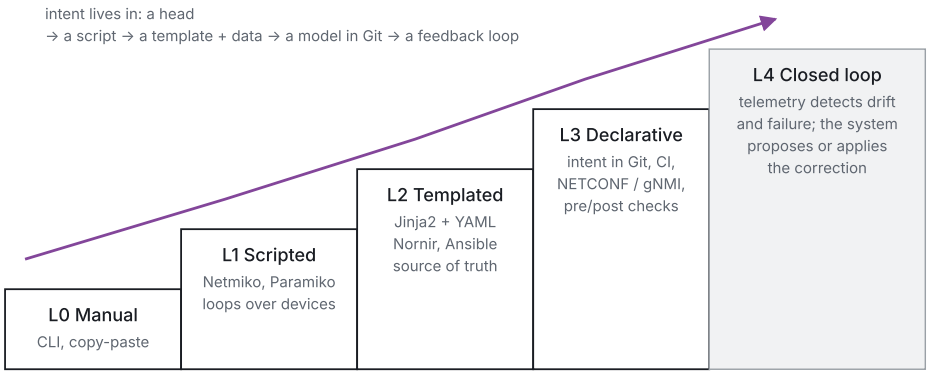


Figure 10.1 The maturity ladder. Each step changes where the intended state is kept, and therefore what can be reviewed, tested and replayed.

Step	Intended state lives in	Typical tools	Fails when
L0 manual	the engineer's head	CLI, notepad	people leave; changes differ per device
L1 scripted	a script of commands	Paramiko, Netmiko	the script is run twice or on the wrong model
L2 templated	data files plus templates	Jinja2, Ansible, Nornir	the data drift from the devices
L3 declarative	models and intent in Git, validated in CI	NETCONF, gNMI, NAPALM, Batfish	the tests do not model the real network
L4 closed loop	the running network, watched by telemetry	gNMI, alerting, remediation runbooks	automatic remediation acts on a false signal

The reference architecture that the rest of the chapter assembles is a loop: the *source of truth* (data in Git, or an inventory such as NetBox) feeds a *renderer* (templates), which feeds a *pipeline* (tests and approval), which pushes over a *transport* (SSH, NETCONF, gNMI) to the devices, whose *telemetry* returns to a collector and, through alerting, back to a person or a pipeline. Sections 10.2 to 10.9 build the left half; 10.10 to 10.12 the transports and the return path; 10.13 to 10.15 the pipeline and its safety.

> INTERVIEW LENS
>
> CISCO · AMAZON · JUNIPER · ARISTA
>
> "Why automate?" is asked as a warm-up and judged on specificity. A strong answer names three benefits with a mechanism each (consistency from one template, speed from concurrency, audit from Git) and one risk (blast radius), then adds a number from experience, such as the 4 h against 6 min comparison above. A likely follow-up is "what would you not automate?": one-off emergency changes whose cost of building the tooling exceeds the change, and anything nobody can yet test.

> CHECK YOURSELF
>
> 1. At which ladder step does a Jinja2 template with a YAML file first appear? [1]
> 2. Why is "the device's running config" a poor place to keep intent? [2]

10.2 JSON, YAML and XML MUST · ■□□

CLI output is text written for a human reading a terminal: columns align by spaces, headings vary between releases and a script must guess where a field starts. *Structured data* names every field and nests related ones, so a program asks for `interface["description"]` instead of counting columns. Three formats appear in network work, and all three describe the same tree of keys, lists and values. *JSON* is the default of web APIs and maps directly onto Python dictionaries and lists. *YAML* is a superset that is pleasant to edit by hand and is used for inventories and playbooks. *XML* is verbose but supports namespaces, which NETCONF needs to say which YANG module a tag belongs to (10.10 (p. 604)).

Consider one interface on `dist1`, first as the CLI shows it and then in the three formats.

```
interface GigabitEthernet1/0/1
 description to-acc1
 ip address 10.0.0.1 255.255.255.252
 no shutdown
```

```
{"interface": {"name": "GigabitEthernet1/0/1", "description": "to-acc1",
               "enabled": true, "ipv4": {"address": "10.0.0.1", "prefix-length": 30}}}
```

```
interface:
  name: GigabitEthernet1/0/1
  description: to-acc1
  enabled: true
  ipv4: {address: 10.0.0.1, prefix-length: 30}
```

```
<interface xmlns="urn:ietf:params:xml:ns:yang:ietf-interfaces">
  <name>GigabitEthernet1/0/1</name>
  <description>to-acc1</description>
  <enabled>true</enabled>
  <ipv4 xmlns="urn:ietf:params:xml:ns:yang:ietf-ip">
    <address><ip>10.0.0.1</ip><prefix-length>30</prefix-length></address>
  </ipv4>
</interface>
```

Property	JSON	YAML	XML
Comments	no	yes (#)	yes (`<!-- -->`)
Human editing	tolerable	best	poor
Types	string, number, boolean, null, list, object	the same plus dates; **implicit** typing	all text; types come from the schema
Namespaces	no	no	yes
Used for	REST, RESTCONF, gNMI JSON, `show … | json`	inventories, playbooks, CI files	NETCONF, older APIs
Python	`json` (standard library)	`yaml` (PyYAML)	`xml.etree`, `xmltodict`, `lxml`

Conversion between them is a three-line job in Python, because each parser returns the same kind of object.

```
import json, yaml, xmltodict

with open("interfaces.yml") as f:
    data = yaml.safe_load(f)                  # always safe_load: plain load can run code
for ifc in data["interfaces"]:
    if not ifc.get("enabled", True):          # missing key counts as enabled
        print(ifc["name"], "is administratively down")

print(json.dumps(data, indent=2))             # YAML -> JSON
xml_text = xmltodict.unparse({"interfaces": {"interface": data["interfaces"]}},
pretty=True)
back = xmltodict.parse(xml_text)              # XML -> dict (all values become strings)
```

Given `interfaces.yml` listing `Gi1/0/1` (enabled), `Gi1/0/2` (`enabled: false`) and `Gi1/0/3` (enabled), the script prints `Gi1/0/2 is administratively down`. Note that the XML round trip returns the string `"true"`, not the boolean, because XML carries no types: this is the usual reason a comparison silently fails after a conversion.

Why structure beats screen-scraping. A scraper depends on the exact layout of text that the vendor may change in any release; a column that widens, a new header line or a localised message breaks the regular expression, and it breaks silently by returning the wrong field. Structured output depends on a *schema*, which the vendor versions and documents. Hence the order of preference is: (i) a native API returning JSON or XML (`show interfaces | json` on Arista EOS, RESTCONF, NETCONF, gNMI); (ii) a parser with a maintained template (10.5 (p. 594)); (iii) a hand-written regular expression, as a last resort.

Validation. Data typed by people contain mistakes, and a template renders a mistake as faithfully as a fact. A *schema* states what is allowed, and the check runs before anything touches a device. `jsonschema` validates JSON-shaped data (including loaded YAML) against a schema; `pydantic` does the same with Python classes. A worked example is in 10.6 (p. 596).

> TRAP
>
> *Claim:* "YAML is just JSON without braces, so indentation is cosmetic." *Why it is wrong:* indentation is the syntax. A tab is illegal, two spaces fewer turn a key from a child into a sibling, and YAML 1.1 loaders read the unquoted words `no`, `off` and `yes` as booleans and `010` as an octal number. *Correct statement:* indent with spaces only, quote any string that could be read as another type (`"no"`, `"010"`) and always read data back with `safe_load` to check what was parsed.

> CHECK YOURSELF
>
> 1. Which of the three formats can carry a namespace, and which protocol needs it? [3]
> 2. What does `yaml.safe_load("vlan: 010")` return for `vlan`? [4]

10.3 Paramiko: SSH from first principles OPTIONAL · ■□□

Paramiko is a pure-Python implementation of the SSH protocol. SSH runs over TCP port 22 and proceeds in layers: the transport layer negotiates algorithms and exchanges keys, the user-authentication layer proves the client's identity, and the connection layer multiplexes *channels* over the one session, such as a command (`exec`), a terminal (`shell`) and file transfer (`sftp`). Paramiko exposes each of them, which gives full control and also full responsibility, much as driving a manual car does.

Two ways of running commands matter. `exec_command` opens an `exec` channel, runs one command and closes it; it suits Linux hosts and the few network operating systems that accept it. Most network devices want an interactive terminal, so one opens a `shell` channel with `invoke_shell()`, sends text, waits and reads whatever has arrived. The pager is the first obstacle: IOS stops after one screen with `--More--`, so the script sends `terminal length 0` first.

```
import time
import paramiko

def run_show(host: str, user: str, password: str, cmd: str) -> str:
    client = paramiko.SSHClient()
    client.load_system_host_keys()                      # trust ~/.ssh/known_hosts
    client.set_missing_host_key_policy(paramiko.RejectPolicy())   # refuse unknown keys
    client.connect(host, username=user, password=password, timeout=10,
                   look_for_keys=False, allow_agent=False)
    try:
        chan = client.invoke_shell()
        chan.send("terminal length 0\n")
        time.sleep(0.5)
        chan.recv(65535)                                # discard banner and prompt
        chan.send(cmd + "\n")
        time.sleep(1.0)                                 # crude: wait for the device
        out = b""
        while chan.recv_ready():
            out += chan.recv(65535)
        return out.decode(errors="replace")
    finally:
        client.close()

if __name__ == "__main__":
    text = run_show("10.255.0.21", "netops", "secret", "show version")
    with open("acc1-version.txt", "w") as f:
        f.write(text)
```

The weakness is visible in the line marked *crude*: Paramiko does not know where the device's answer ends, so the script sleeps and hopes. A slow `show tech-support` is truncated; a fast command wastes a second. Recognising the prompt is exactly what Netmiko adds, and for that reason one rarely uses Paramiko directly on network devices. It remains the right tool for SFTP transfers (`client.open_sftp().put(local, remote)`) and for Linux hosts.

> **TRAP**
>
> *Claim: "*`AutoAddPolicy` is fine in a lab, so it is fine in the script." *Why it is wrong:* it accepts any host key without checking, so an attacker who intercepts the connection presents his own key and receives the password (9.8 (p. 558)). *Correct statement:* load `known_hosts` (or a managed key store) and use `RejectPolicy`, or pin the fingerprint, and accept a new key only through a deliberate, recorded step.

10.4 Netmiko: multi-vendor SSH made sane MUST · ■■□

Netmiko wraps Paramiko with per-vendor knowledge: how to log in, how to recognise the prompt, how to disable paging, how to enter and leave configuration mode and how to save. It is the automatic gearbox fitted to the manual car. A connection is created by `ConnectHandler` with a `device_type` (`cisco_ios` for IOS and IOS XE, `cisco_xr`, `cisco_nxos`, `arista_eos`, `juniper_junos`, `linux`), and the object then offers the calls below.

Call	Purpose	Notes

`send_command(cmd)`	run one show command, return text	waits for the prompt; `read_timeout=` for slow commands; `expect_string=` for odd prompts
`send_command_timing(cmd)`	same, but waits on silence	use when no stable prompt appears (confirmations)
`send_config_set([lines])`	enter config mode, send lines, leave	does **not** save; returns the session text
`send_config_from_file(path)`	as above, lines from a file	
`save_config()`	`write memory` or the vendor equivalent	separate step on purpose
`enable()`	enter privileged mode	needs `secret=`
`disconnect()`	close	use a `with` block

The distinction that interviewers ask about (Q10.4 (p. 619)) is that `send_command` expects the *user-mode or privileged prompt* and returns output, while `send_config_set` first enters `configure terminal`, sends each line, and leaves; the lines are configuration, not queries, and nothing is persisted until `save_config()`.

A multi-device backup. The task: back up the running configuration of every device in the inventory, in parallel, with a time-stamped file per device, retries for transient timeouts, and a summary at the end. The script reads `inventory.yml` (hostname, address, `device_type`) and takes credentials from the environment, never from the file.

```
import csv, os, time, logging
from concurrent.futures import ThreadPoolExecutor, as_completed
from datetime import datetime
from pathlib import Path

import yaml
from netmiko import ConnectHandler
from netmiko.exceptions import NetmikoTimeoutException, NetmikoAuthenticationException

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
OUT = Path("backups") / datetime.now().strftime("%Y%m%d-%H%M%S")
RETRIES, WORKERS = 3, 8

def backup(dev: dict) -> tuple[str, str, str]:
    """Return (name, status, detail). Never raises: one failure must not stop the
rest."""
    params = {"device_type": dev["device_type"], "host": dev["host"],
              "username": os.environ["NET_USER"], "password": os.environ["NET_PASS"],
              "conn_timeout": 15}
    for attempt in range(1, RETRIES + 1):
        try:
            with ConnectHandler(**params) as conn:       # disconnects on leaving the
block
                cfg = conn.send_command("show running-config", read_timeout=60)
            (OUT / f"{dev['name']}.cfg").write_text(cfg)
            return dev["name"], "ok", f"{len(cfg.splitlines())} lines"
        except NetmikoAuthenticationException:
            return dev["name"], "auth-failed", "not retried: repeated bad logins lock ac-
counts"
        except (NetmikoTimeoutException, OSError) as exc:
            logging.warning("%s attempt %d: %s", dev["name"], attempt, exc)
            time.sleep(2 ** attempt)                     # back off: 2 s, 4 s, 8 s
    return dev["name"], "timeout", f"gave up after {RETRIES} attempts"

def main() -> None:
    devices = yaml.safe_load(open("inventory.yml"))["devices"]
    OUT.mkdir(parents=True, exist_ok=True)
    results = []
    with ThreadPoolExecutor(max_workers=WORKERS) as pool:
        futures = [pool.submit(backup, d) for d in devices]
        for fut in as_completed(futures):
            results.append(fut.result())
    with open(OUT / "summary.csv", "w", newline="") as f:
        csv.writer(f).writerows([("device", "status", "detail"), *sorted(results)])
    bad = [r for r in results if r[1] != "ok"]
    logging.info("%d ok, %d failed", len(results) - len(bad), len(bad))
    raise SystemExit(1 if bad else 0)                    # non-zero exit lets CI or cron
notice

if __name__ == "__main__":
    main()
```

For `acc1` to `acc3`, `dist1`, `dist2`, `br1`, `leaf1` and `leaf2` the run prints `8 ok, 0 failed` and writes eight `.cfg` files and `summary.csv`. The design decisions are the ones an interviewer probes. *Threads* suit the job because each worker spends nearly all its time waiting for the network, so Python's global interpreter lock, which is released during network waits, does not limit it. *Eight workers* is a cap, not a goal: unlimited parallelism would open hundreds of simultaneous logins and can overload the TACACS+ server (9.12 (p. 567)). With 20 devices at 6 s each, serial work takes 120 s and eight workers about 3 rounds × 6 s = 18 s. *Authentication failures are not retried*, because each retry is another bad login. *The function never raises*, so one dead switch does not hide the other seven results. *The exit status* reports failure to the scheduler (8.4 (p. 489)).

LAB

`LAB 10.1 · 45 MIN` Run the backup against a containerlab topology of `leaf1`, `leaf2` (cEOS) and `spine1` (FRR) plus two IOS XE images or sandboxes.

```
export NET_USER=admin NET_PASS=admin
python3 backup.py
```

```
INFO 5 ok, 0 failed
$ cat backups/*/summary.csv
device,status,detail
leaf1,ok,87 lines
```

What you should notice: shut one container with `docker stop` and rerun. The script logs three timeouts with growing gaps, reports `timeout` for that device and the other devices still succeed; the exit status is 1.

CHECK YOURSELF

1. Which Netmiko call saves the configuration, and why is it separate from `send_config_set`? [5]
2. Why are threads, not processes, adequate here? [6]

10.5 Parsing CLI output: regex, TextFSM, TTP, Genie SHOULD · ■■□

Some devices and some commands have no structured output, so the script receives text and must turn it into data. We compare four methods on one command, `show ip interface brief` on `acc1`, and ask a question that a monitoring script really needs: which interfaces are administratively up but have a line protocol that is down? That combination usually means a cable or a far-end fault.

```
Interface              IP-Address      OK? Method Status                Protocol
Vlan1                  unassigned      YES unset  administratively down down
Vlan10                 192.168.10.2    YES manual up                    up
Vlan20                 192.168.20.2    YES manual up                    up
GigabitEthernet1/0/1   unassigned      YES unset  up                    up
GigabitEthernet1/0/2   unassigned      YES unset  down                  down
GigabitEthernet1/1/1   10.0.0.2        YES manual up                    down
```

Regular expressions. A named-group pattern, with the `re` module of 2.18 (p. 137), is the fastest to write and the first to break. The status column holds one word or two (“administratively down”), so a pattern that assumes single words fails on exactly the rows that matter.

```
import re
ROW = re.compile(r"^(?P<intf>\S+)\s+(?P<ip>\S+)\s+\w+\s+\w+\s+"
                 r"(?P<status>administratively down|up|down)\s+(?P<proto>up|down)\s*$",
re.M)
rows = [m.groupdict() for m in ROW.finditer(raw)]        # raw holds the text above
```

TextFSM. A *finite-state machine* reads the text line by line; a template declares the variables (`Value`) and, per state, the line patterns and what to do on a match. The community library *ntc-templates* holds hundreds of such templates for the commonest commands, and Netmiko applies them with `send_command(cmd, use_textfsm=True)`. Reading a template is a skill in itself, so we read one in full.

1. L2. The data and the logic are separate files, so the data can be reviewed and the template reused. ↩
2. It records what is configured, not what was meant; a manual change, a typo or a partial failure are all silently “correct” by that definition. ↩
3. XML; NETCONF, to name the YANG module that each element belongs to. ↩
4. The integer 8 under YAML 1.1 rules (a leading zero means octal); quote the value to keep the string. ↩
5. `save_config()`; saving is a decision (one may wish to test first and roll back by reload), so the library does not do it implicitly. ↩
6. The work is waiting on network I/O, during which the interpreter lock is released; processes would add memory and start-up cost for no gain. ↩

```
Value Required INTF (\S+)
Value IPADDR (\S+)
Value STATUS (up|down|administratively down)
Value PROTO (up|down)

Start
  ^${INTF}\s+${IPADDR}\s+\w+\s+\w+\s+${STATUS}\s+${PROTO}\s*$$ -> Record
```

Each `Value` line names a variable and gives its regular expression in brackets; `Required` discards a record without it. Under `Start`, the pattern begins with `^`, each `${NAME}` inserts the variable's expression, and `$$` is a literal end-of-line anchor. The header row does not match because `OK?` contains a question mark that `\w+` cannot consume, so no special case is needed. `-> Record` writes one record per match. The Python side is short.

```
import io, textfsm

with open("show_ip_int_brief.textfsm") as t:
    fsm = textfsm.TextFSM(t)
rows = [dict(zip(fsm.header, r)) for r in fsm.ParseText(raw)]
bad = [r["INTF"] for r in rows if r["STATUS"] == "up" and r["PROTO"] == "down"]
print("up/down:", bad)
```

With the output above, `rows` has six dictionaries and the script prints `up/down: ['GigabitEthernet1/1/1']`: the uplink to `dist1` is enabled but the protocol is down. The same dictionaries are what `use_textfsm=True` returns, with lower-case keys (`intf`, `ipaddr`, `status`, `proto`) in the ntc-templates convention.

TTP. The *Template Text Parser* writes the template as the sample itself, replacing the variable parts with `{{ name }}`, which is easier to author and review than a state machine.

```
<group name="interfaces">
{{ intf }}   {{ ip }}   YES {{ method }}   {{ status | ORPHRASE }}   {{ proto }}
</group>
```

`ORPHRASE` lets a value contain spaces. The call is `ttp(data=raw, template=tmpl)`, then `.parse()` and `.result()`; Netmiko accepts `use_ttp=True, ttp_template=path`.

Genie. Cisco's *pyATS/Genie* library ships maintained parsers for several hundred commands across IOS XE, NX-OS and others, returning nested dictionaries that follow one schema per command. `device.parse("show ip interface brief")` returns `{"interface": {"Vlan10": {"ip_address": "192.168.10.2", "status": "up", "protocol": "up", ...}}}`, and the same call exists in Netmiko as `use_genie=True`. The cost is a large dependency and Cisco-centred coverage.

Method	Strength	Weakness	Choose when
Regex	no dependency; quick	fragile on wrapped lines and optional columns	one command, one release, one-off
TextFSM / ntc-templates	many templates exist; multi-line state	writing new templates is slow	the command is already in ntc-templates
TTP	template looks like the output	smaller community	writing your own, for a vendor without templates

Genie	maintained by the vendor; consistent schema	heavy install; Cisco-focused	a Cisco estate already using pyATS

TRAP

Claim: "A parser written against today's output will keep working." *Why it is wrong:* vendors add columns, reword headings and change spacing between releases, and a regular expression that no longer matches simply returns fewer rows, so the script reports a healthy network. *Correct statement:* prefer an API (10.11 (p. 607)), pin the software release a parser was tested against, and treat "zero rows parsed" as an error, not as "nothing found".

10.6 Jinja2 templating and the source of truth MUST · ■■□

A template separates three things that a hand-edited configuration mixes: the *data* (which VLANs, which ports), the *logic* (what a port configuration looks like) and the *transport* (how the text reaches the device). *Jinja2* is the Python templating language used by Ansible, Nornir and many internal tools. It replaces `{{ expression }}` with a value, runs `{% statement %}` blocks such as `for` and `if`, and applies *filters* with `|`, as in `{{ name | upper }}`. Like a wedding-card template, the names and dates change while the layout stays.

A *source of truth* (SoT) is the single place that records what the network is meant to be. It must be separate from the devices, because a device records only what is configured; a manual change, a typo or a half-applied push is "correct" by that definition. A YAML file in Git is the simplest SoT; *NetBox* (and its relatives Nautobot and Infrahub) adds a database, an API and relationships between devices, prefixes and VLANs. The data below is Campus Net's `sot.yml`.

```
vlans:
  10: {name: SALES}
  20: {name: ENG}
  30: {name: GUEST}
switches:
  acc1:
    uplink: GigabitEthernet1/1/1
    ports:
      - {name: GigabitEthernet1/0/1, vlan: 10, desc: sales-pc-01}
      - {name: GigabitEthernet1/0/2, vlan: 20, desc: eng-pc-07}
      - {name: GigabitEthernet1/0/3, vlan: 30, desc: guest-ap}
  acc2:
    uplink: GigabitEthernet1/1/1
    ports:
      - {name: GigabitEthernet1/0/1, vlan: 10, desc: sales-pc-02}
```

The template `access_switch.j2` follows. The flags `trim_blocks` and `lstrip_blocks`, set in the script, remove the blank line that each `{% … %}` line would leave behind.

```
hostname {{ name }}
!
{% for id, v in vlans.items() | sort %}
vlan {{ id }}
 name {{ v.name }}
{% endfor %}
!
{% for p in ports %}
interface {{ p.name }}
 description {{ p.desc }}
 switchport mode access
 switchport access vlan {{ p.vlan }}
 spanning-tree portfast
 no shutdown
{% endfor %}
interface {{ uplink }}
 description trunk-to-dist1
 switchport mode trunk
 switchport trunk allowed vlan {{ vlans.keys() | sort | join(",") }}
```

The renderer does three jobs in order: load, **validate**, render. Validation matters because a template renders a mistake as faithfully as a fact. `StrictUndefined` makes a missing variable an error rather than an empty string, and the checks below catch a VLAN that is out of range or not declared.

```
import ipaddress
import yaml
from jinja2 import Environment, FileSystemLoader, StrictUndefined
from jsonschema import validate

SCHEMA = {"type": "object", "required": ["vlans", "switches"], "properties": {
    "vlans": {"type": "object", "additionalProperties": {
        "type": "object", "required": ["name"], "properties": {"name": {"type":
"string"}}}},
    "switches": {"type": "object", "additionalProperties": {
        "type": "object", "required": ["uplink", "ports"], "properties": {
            "ports": {"type": "array", "items": {"type": "object",
                "required": ["name", "vlan", "desc"],
                "properties": {"vlan": {"type": "integer", "minimum": 1, "maximum":
4094}}}}}}}}}

def check(sot: dict) -> list[str]:
    errors = []
    for sw, spec in sot["switches"].items():
        for p in spec["ports"]:
            if p["vlan"] not in sot["vlans"]:
                errors.append(f"{sw} {p['name']}: VLAN {p['vlan']} is not declared in
vlans")
    return errors

def main() -> None:
    sot = yaml.safe_load(open("sot.yml"))
    validate(sot, SCHEMA)                                  # raises ValidationError on
type or range
    if errs := check(sot):
        raise SystemExit("\n".join(errs))
    env = Environment(loader=FileSystemLoader("templates"), trim_blocks=True,
                      lstrip_blocks=True, undefined=StrictUndefined)
    env.filters["netmask"] = lambda p: str(ipaddress.ip_network(p, strict=False).netmask)
    tpl = env.get_template("access_switch.j2")
    for name, spec in sot["switches"].items():
        text = tpl.render(name=name, vlans=sot["vlans"], **spec)
        open(f"rendered/{name}.cfg", "w").write(text + "\n")

if __name__ == "__main__":
    main()
```

For `acc1` the first lines of `rendered/acc1.cfg` are:

```
hostname acc1
!
vlan 10
 name SALES
vlan 20
 name ENG
vlan 30
 name GUEST
!
interface GigabitEthernet1/0/1
 description sales-pc-01
 switchport mode access
 switchport access vlan 10
```

If a person types `vlan: 40` for a port, the run stops before any file is written, with `acc2 GigabitEthernet1/0/1: VLAN 40 is not declared in vlans`. This is the central economy of templating: the error is found in a file, in seconds, at review time, instead of on a switch, after the change.

Three further features are used constantly. A *filter* is any function applied with `|`; the `netmask` filter registered above turns `10.0.0.0/30` into `255.255.255.252`. A *macro* (`{% macro port(p) %}…{% endmacro %}`) is a function inside a template, used to avoid repeating a block. *Inheritance* (`{% extends "base.j2" %}` with `{% block %}`) lets a platform template override only the lines that differ. Note that whitespace in a template is the whitespace of the output, and IOS treats indentation of sub-commands as significant to a human reader but not to the parser; Arista and Junos templates are less forgiving, so one compares the rendered file with the real configuration before trusting a template.

WORKED EXAMPLE

Fix this template. The following fragment should print one `vlan` stanza per VLAN but prints a stray blank line and fails on `acc2`.

```
{% for id, v in vlans %}
vlan {{ id }}
 name {{ v.name }}
{% endfor %}
```

1. `vlans` is a dictionary; iterating it yields keys only, so unpacking `id, v` fails. The fix is `vlans.items()`.
2. The blank lines come from the newline after each `{% … %}` tag. Set `trim_blocks=True`, or write `{%- … -%}` to strip whitespace on one side.
3. With `StrictUndefined`, `v.name` raises if a VLAN has no `name`; the schema above already requires it.

Result: `{% for id, v in vlans.items() | sort %}` **plus** `trim_blocks=True`.

CHECK YOURSELF

1. Which Jinja2 setting turns a missing variable into an error? [1]
2. Why validate the data before rendering? [2]

10.7 NAPALM: vendor-neutral getters and safe config replace SHOULD · ■■□

NAPALM (Network Automation and Programmability Abstraction Layer with Multivendor support) puts one Python API in front of several operating systems. A *driver* per platform (`ios`, `iosxr`, `nxos`, `eos`, `junos`) translates the common calls into whatever that platform offers, as a universal remote controls televisions of different brands. Its two halves are *getters*, which return normalised dictionaries, and a *configuration workflow* with a candidate, a diff, a commit and a rollback.

Getter	Returns
`get_facts()`	vendor, model, OS version, serial, hostname, uptime, interface list
`get_interfaces()`	per interface: `is_up`, `is_enabled`, `speed`, `mtu`, `mac_address`, `description`
`get_interfaces_counters()`	octets, packets, errors, discards per interface
`get_interfaces_ip()`	IPv4 and IPv6 addresses per interface
`get_lldp_neighbors()`	neighbour host and port per local interface

`get_arp_table()`, `get_mac_address_table()`	ARP and MAC entries
`get_bgp_neighbors()`	per VRF: router ID, peers, `is_up`, `remote_as`, address-family prefix counts
`get_route_to(destination)`	the routes that match
`get_config(retrieve="running")`	the configuration text
`is_alive()`, `cli([cmds])`, `ping()`	liveness, raw commands, ping from the device

The first example compares BGP on `br1` (IOS XE) and `leaf1` (EOS) with the same code. Without NAPALM the script would parse two different outputs (10.5 (p. 594)); with it the field names are identical, which is the entire value of the library. The protocol itself is taught in 4.7 (p. 256).

```
import os
from napalm import get_network_driver

def connect(driver_name: str, host: str):
    drv = get_network_driver(driver_name)
    dev = drv(hostname=host, username=os.environ["NET_USER"],
password=os.environ["NET_PASS"],
              timeout=30)
    dev.open()
    return dev

for drv, host in [("ios", "10.255.0.1"), ("eos", "10.255.0.31")]:
    dev = connect(drv, host)
    try:
        for vrf, data in dev.get_bgp_neighbors().items():
            for peer, p in data["peers"].items():
                state = "up" if p["is_up"] else "DOWN"
                print(f"{host:12} {vrf:8} {peer:15} AS{p['remote_as']:<6} {state}")
    finally:
        dev.close()
```

```
10.255.0.1   global   203.0.113.1     AS100    up
10.255.0.31  global   10.0.0.1        AS65000  up
```

The configuration workflow. Changing a device through NAPALM has five steps, and the order is the safety property.

1. `load_merge_candidate(config=text)` stages *additional* lines; `load_replace_candidate(config=text)` stages a *complete* configuration that will replace the running one.
2. `compare_config()` returns a diff between running and candidate. An empty string means nothing would change.
3. `commit_config()` applies the candidate; `discard_config()` throws it away.
4. The script verifies (a getter, a ping) that the device behaves.
5. `rollback()` returns to the previous configuration if verification fails.

```
leaf1 = connect("eos", "10.255.0.31")
leaf1.load_merge_candidate(config="ntp server 10.255.0.5\n")
diff = leaf1.compare_config()
print(diff)                                  # shows "+ntp server 10.255.0.5"
if not diff:
    leaf1.discard_config()
elif input("commit? [y/N] ") == "y":
    leaf1.commit_config()
    ok = any("10.255.0.5" in l for l in leaf1.cli(["show ntp associations"])["show ntp
associations"].splitlines())
    if not ok:
        leaf1.rollback()                     # back to the pre-commit configuration
leaf1.close()
```

The mechanisms underneath differ, and the difference explains the failure modes. EOS and Junos have native candidate sessions or commit models, so the diff and the rollback are exact. On IOS the driver copies a file to the device by SCP and runs `configure replace` (which requires the archive feature and SCP enabled), so a replace is only as safe as that feature.

> **TRAP**
>
> *Claim:* "`load_replace_candidate` is just a bigger merge." *Why it is wrong:* replace makes the running configuration *equal* to the candidate, so everything absent from the file is removed, including the management ACL, the AAA lines or the SSH keys if the file was rendered without them. The device then loses the very access the script needs to roll back. *Correct statement:* use merge for adding lines; use replace only for a full, rendered configuration that includes management access, read the diff first, and commit with a timer (`commit_config(revert_in=…)` where the driver supports it).

NAPALM also offers `compliance_report()`, which compares getters against a YAML file of expected values (for example, "BGP neighbour 10.0.0.1 is up"), so the same tool is used to enforce intent as well as to push it.

10.8 Nornir: Python-native inventory and concurrency

OPTIONAL · ■■□

Nornir is a framework, not a language: it supplies an *inventory* (hosts, groups, defaults), a *runner* that executes a Python function against every selected host in a thread pool, and *result* objects, while the tasks themselves are ordinary Python. It resembles Ansible's inventory with the full control of Python; the conductor hands each musician the score at once and collects how each performed. Plug-ins add tasks: `nornir_netmiko`, `nornir_napalm`, `nornir_jinja2`, `nornir_utils`.

An attribute is resolved by inheritance: a host's own value wins, then its groups, then the defaults.

```
defaults.yaml   ->  groups.yaml (leaf: platform eos, data.role leaf)  ->  hosts.yaml
(leaf1: hostname 10.255.0.31)
```

```
# inventory/hosts.yaml
leaf1:
  hostname: 10.255.0.31
  groups: [leaf]
  data: {asn: 65101, loopback: 10.0.0.11}
leaf2:
  hostname: 10.255.0.32
  groups: [leaf]
  data: {asn: 65102, loopback: 10.0.0.12}

# inventory/groups.yaml
leaf:
  platform: eos
  data: {role: leaf}
```

```
# config.yaml
inventory:
  plugin: SimpleInventory
  options: {host_file: inventory/hosts.yaml, group_file: inventory/groups.yaml,
defaults_file: inventory/defaults.yaml}
runner:
  plugin: threaded
  options: {num_workers: 8}
```

The worked task renders a template per host, pushes it as a *dry run*, shows every diff, and commits only if every diff is non-empty or approved and nothing failed.

```
import os
from nornir import InitNornir
from nornir.core.filter import F
from nornir.core.task import Result, Task
from nornir_jinja2.plugins.tasks import template_file
from nornir_napalm.plugins.tasks import napalm_configure

def deploy(task: Task, dry_run: bool) -> Result:
    rendered = task.run(task=template_file, template="leaf.j2", path="templates",
**task.host.data)
    task.run(task=napalm_configure, configuration=rendered.result, dry_run=dry_run,
replace=False)
    return Result(host=task.host, result="done")

nr = InitNornir(config_file="config.yaml")
nr.inventory.defaults.username = os.environ["NET_USER"]    # secrets from the environ-
ment,
nr.inventory.defaults.password = os.environ["NET_PASS"]    # never from inventory files
leaves = nr.filter(F(role="leaf"))

plan = leaves.run(task=deploy, dry_run=True)
for host, multi in plan.items():
    print(f"=== {host} ===")
    print(multi.failed and multi[0].exception or multi[-1].diff)

if plan.failed:
    raise SystemExit("dry run failed on: " + ", ".join(plan.failed_hosts))
if input("apply to %d leaves? [y/N] " % len(leaves.inventory.hosts)) == "y":
    result = leaves.run(task=deploy, dry_run=False)
    print("failed:", list(result.failed_hosts))
```

`F(role="leaf")` filters on the `data` dictionary, so selecting a *canary* (`F(name="leaf1")`) is one line, which is why a Nornir script grows into the staged rollout of 10.15 (p. 616) without restructuring. Note that `multi[0]` is the parent task's result and the sub-task results follow it. Choose Nornir over Ansible when the logic is genuinely programmatic (loops with decisions,

calls to a database, unit tests); choose Ansible when the team writes declarative tasks and the logic is a checklist (10.9 (p. 603)).

10.9 Ansible for networks SHOULD · ■■□

Ansible is an agentless automation tool: the control machine reads a YAML *playbook*, connects to each device in an *inventory* and runs *tasks*, each calling a *module*. For network devices nothing is installed on the target, because the connection plug-in speaks SSH (`ansible.netcommon.network_cli`), NETCONF (`netconf`) or HTTPS (`httpapi`). Modules are shipped in *collections* per vendor: `cisco.ios`, `arista.eos`, `junipernetworks.junos`. The playbook is a printed checklist that every shift follows, readable by people who do not program.

```
# inventory.yml
all:
  children:
    access:
      hosts:
        acc1: {ansible_host: 10.255.0.21}
        acc2: {ansible_host: 10.255.0.22}
        acc3: {ansible_host: 10.255.0.23}
      vars:
        ansible_network_os: cisco.ios.ios
        ansible_connection: ansible.netcommon.network_cli
        ansible_user: netops
        ansible_password: "{{ vault_netops_password }}"   # from ansible-vault, never
plain text
```

The playbook makes sure VLANs 10, 20 and 30 exist with the right names on every access switch, and sets one interface description. A *play* selects hosts; *tasks* run in order; each module call states a desired result, and the module decides whether anything must change.

```
# vlans.yml
- name: Campus Net access VLANs
  hosts: access
  gather_facts: false
  tasks:
    - name: Ensure VLANs exist
      cisco.ios.ios_vlans:
        config:
          - {vlan_id: 10, name: SALES, state: active}
          - {vlan_id: 20, name: ENG,   state: active}
          - {vlan_id: 30, name: GUEST, state: active}
        state: merged

    - name: Describe the uplink
      cisco.ios.ios_config:
        parents: interface GigabitEthernet1/1/1
        lines: [description trunk-to-dist1]
        save_when: modified
```

Run it twice. The first run changes the switches that lack the VLANs; the second must change nothing, and that is the test of *idempotence*, the property that applying the same change twice leaves the same state as applying it once.

1. `undefined=StrictUndefi` on the `Environment`.↩
2. A template renders wrong data as readily as right data; validation moves the failure from the device to the pull request.↩

```
$ ansible-playbook -i inventory.yml vlans.yml
TASK [Ensure VLANs exist] ***   changed: [acc1]   changed: [acc2]   ok: [acc3]
TASK [Describe the uplink] ***  changed: [acc1]   changed: [acc2]   changed: [acc3]
PLAY RECAP  acc1 : ok=2 changed=2   acc2 : ok=2 changed=2   acc3 : ok=2 changed=1
$ ansible-playbook -i inventory.yml vlans.yml
PLAY RECAP  acc1 : ok=2 changed=0   acc2 : ok=2 changed=0   acc3 : ok=2 changed=0
```

A dry run is `ansible-playbook -i inventory.yml vlans.yml --check --diff -l acc1`: `--check` computes the change without applying it, `--diff` prints it, and `-l acc1` limits the run to one host, which is the first canary step (10.15 (p. 616)). Secrets are stored with `ansible-vault encrypt_string`, and the run is given `--ask-vault-pass` or a vault-id file kept out of Git.

Resource modules (`ios_vlans`, `ios_interfaces`, `ios_l3_interfaces`, `ios_bgp_global`) model one feature as structured data, read the device's current state, and compare it with the request, so they are idempotent by design. Their `state` decides what happens to anything not listed.

`state`	Effect
`merged`	add or update what is listed; leave everything else
`replaced`	make each *listed resource* equal to the request (one VLAN or interface at a time)
`overridden`	make the *whole feature* equal to the request; remove unlisted resources
`deleted`	remove the listed resources (or all of them if none is listed)
`gathered`	read the device and return structured data; no change
`rendered`, `parsed`	produce CLI from data, or data from CLI text, offline

> **TRAP**
>
> *Claim:* "Ansible is idempotent by magic." *Why it is wrong:* idempotence belongs to the module, not to the tool. `ios_command` always runs, so it always reports `changed`; `ios_config` compares text lines and reports `changed` every run if a line is written differently from the way the device echoes it (an abbreviated command, a `parents` list that does not match the hierarchy). *Correct statement:* prefer resource modules, write `ios_config` lines exactly as `show running-config` prints them, and treat a second run with `changed` above zero as a bug.

Choose Ansible when the work is a repeatable checklist, the team is not mainly programmers, and vendor modules cover the feature. Choose Python (10.8 (p. 601)) when the logic needs loops with decisions, calls to other systems, or unit tests. Many teams use both: Ansible for the standard changes, Python for the tools that wrap them.

10.10 YANG data models MUST · ■■□

YANG (RFC 7950) is a language for describing a tree of configuration and state data: its names, types and constraints. It is the building plan approved by the authority, and NETCONF, RESTCONF and gNMI are three different lorries that deliver furniture which must fit the plan. A

YANG file is called a *module*, and it declares a namespace so that identical names from different modules do not collide. The building blocks are few.

Statement	Meaning
`container`	a named group of children; has no key; at most one instance
`list`	a collection of entries, each identified by its `key` leaf (or leaves); a configuration list must have a key
`leaf`	one value of one type (`string`, `uint16`, `boolean`, `enumeration`, `inet:ipv4-address`)
`leaf-list`	a list of plain values with no sub-structure
`typedef`, `grouping`/`uses`	a reusable type; a reusable group of nodes
`augment`	one module adds nodes to another module's tree
`deviation`	a vendor states that it does not implement a node, or implements it differently
`config false`	a read-only node: operational state, such as counters

A small module for Campus Net's switch ports shows each of them.

```
module campus-demo {
  yang-version 1.1;
  namespace "urn:example:campus-demo";
  prefix cd;
  typedef vlan-id { type uint16 { range "1..4094"; } }
  grouping port-config {
    leaf name        { type string; }
    leaf description { type string; }
    leaf vlan        { type vlan-id; }
  }
  container campus {
    list switch {
      key "hostname";
      leaf hostname { type string; }
      list port { key "name"; uses port-config; }
      leaf uptime { type uint32; config false; }
    }
  }
}
```

The tool `pyang -f tree campus-demo.yang` prints the shape of the data compactly. The symbols are the vocabulary one must be able to read at sight.

```
module: campus-demo
  +--rw campus
     +--rw switch* [hostname]
        +--rw hostname    string
        +--rw port* [name]
        |  +--rw name           string
        |  +--rw description?   string
        |  +--rw vlan?          vlan-id
        +--ro uptime?     uint32
```

Symbol	Meaning	Symbol	Meaning

`rw`	configuration data (read-write)	`ro`	state data (read-only)
`*`	list or leaf-list	`?`	optional node
`[name]`	the list's key	`!`	presence container: its existence means something
`(choice)`	exactly one of the cases	`augment …`	nodes added by another module

The IETF module `ietf-interfaces` (RFC 8343), abridged, has the same shape, and its `ietf-ip` augment (RFC 8344) adds the addresses.

```
module: ietf-interfaces
  +--rw interfaces
  |  +--rw interface* [name]
  |     +--rw name           string
  |     +--rw description?   string
  |     +--rw type           identityref
  |     +--rw enabled?       boolean
  |     +--ro oper-status    enumeration
  |     +--ro statistics
  |        +--ro in-octets?   yang:counter64
  augment /if:interfaces/if:interface:
     +--rw ipv4!
        +--rw address* [ip]
           +--rw ip               inet:ipv4-address-no-zone
           +--rw prefix-length?   uint8
```

The tree maps onto the CLI by path. The node `/interfaces/interface[name='GigabitEthernet1/0/1']/description` is the line `description to-acc1` under `interface GigabitEthernet1/0/1`; `/interfaces/interface[name=…]/ietf-ip:ipv4/address[ip='10.0.0.1']/prefix-length` is the mask of the `ip address` line. The bracketed key `[name=…]` selects one list entry, exactly as a primary key selects one row.

Three families of models. *IETF* models (`ietf-interfaces`, `ietf-routing`) are standardised but lag the features of real devices. *OpenConfig* models are written by large operators for operational use, are vendor-neutral and are the usual language of gNMI. *Native* models (`Cisco-IOS-XE-native`, Arista's EOS native) mirror each vendor's CLI one-to-one, so they are complete but not portable.

Family	Coverage	Portability	Typical use
IETF	basic features	high, but vendors implement subsets	NETCONF/RESTCONF baseline
OpenConfig	interfaces, BGP, LLDP, telemetry paths	high across Arista, Juniper, Nokia, Cisco	gNMI telemetry and multi-vendor config
Native	everything the device does	none	features absent from the other two

Modules are found at yangcatalog.org and in each vendor's GitHub repository; the YANG Suite and `pyang` explore them. A *deviation* is the reason that two devices claiming the same model do not behave identically: the first thing to check when a standard path returns an error is that device's deviation file.

CHECK YOURSELF

1. In a configuration `list`, what is mandatory? [1]
2. What does `ro` mean in a tree, and where would it be used? [2]

10.11 NETCONF and RESTCONF MUST · ■■□

NETCONF (RFC 6241) is a protocol for installing, manipulating and deleting configuration. It runs as an SSH subsystem on **TCP port 830** (RFC 6242), exchanges XML, and treats the configuration as a transaction with a result for every request, like registered post with a receipt. The protocol is layered, and the layers are worth drawing.

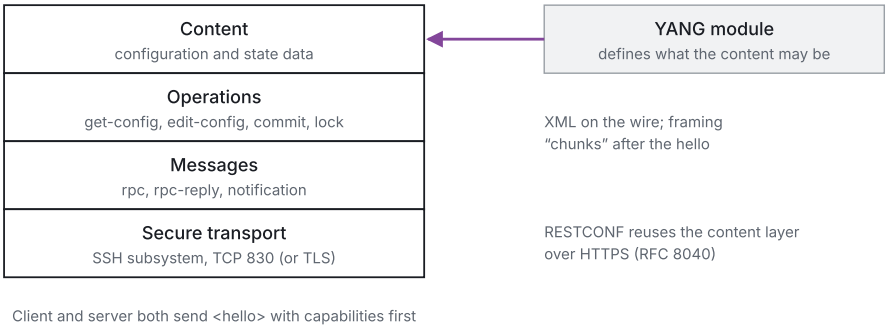


Figure 10.2 The NETCONF layers (RFC 6241). The YANG module constrains only the top layer; the three layers beneath carry any content.

The hello exchange. On connecting, each side immediately sends a `<hello>` listing its *capabilities*, URIs naming the protocol version and optional features and every YANG module the server supports. The session proceeds only on the capabilities both sides share, which is how a client learns whether the server has a candidate datastore, supports `:validate` or `:confirmed-commit`, and which models it speaks.

```
<hello xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <capabilities>
    <capability>urn:ietf:params:netconf:base:1.1</capability>
    <capability>urn:ietf:params:netconf:capability:candidate:1.0</capability>
    <capability>urn:ietf:params:netconf:capability:validate:1.1</capability>
    <capability>http://openconfig.net/yang/interfaces?module=openconfig-
interfaces</capability>
  </capabilities>
</hello>
```

Datastores. A *datastore* is a complete configuration. `running` is what the device is using; `startup` is what it loads at boot; `candidate` is an editable scratch copy that does nothing until it is committed. The candidate is valuable because several edits can be made, validated and applied together, or discarded, so the change is all-or-nothing; many IOS XE releases offer only `running`, and the client must check the capability.

Operation	Purpose
`<get>`	read configuration and state data
`<get-config>`	read configuration only, from a named datastore, optionally filtered
`<edit-config>`	change a datastore; the `operation` attribute is `merge` (default), `replace`, `create`, `delete` or `rem-`

	ove
`<copy-config>`, `<delete-config>`	copy or delete a whole datastore (for example running to startup)
`<lock>`, `<unlock>`	give one session exclusive write access to a datastore
`<validate>`	check a datastore for consistency without applying it
`<commit>`	copy candidate to running; with `confirmed` it reverts unless confirmed within a timeout
`<discard-changes>`	reset the candidate to the running configuration
`<close-session>`, `<kill-session>`	end this or another session

The difference between `delete` and `remove` is a favourite question: `delete` fails if the node does not exist, `remove` succeeds either way, so `remove` is the idempotent choice for automation.

A safe change is a transaction: **lock** the datastore, **edit** the candidate, **validate**, **commit**, **unlock**; if any step returns an `<rpc-error>`, **discard** the changes and unlock, and the running configuration has never been altered. The `ncclient` library implements this directly.

```
from ncclient import manager
from ncclient.operations import RPCError

NS = "urn:ietf:params:xml:ns:yang:ietf-interfaces"
SELECT = f'<interfaces xmlns="{NS}"><interface><name>GigabitEthernet1/0/1</name>
</interface></interfaces>'
CHANGE = f"""<config><interfaces xmlns="{NS}"><interface>
  <name>GigabitEthernet1/0/1</name><description>to-acc1 (uplink)</description>
</interface></interfaces></config>"""

with manager.connect(host="10.255.0.11", port=830, username="netops", password="secret",
                     hostkey_verify=True, device_params={"name": "iosxe"}, timeout=30)
as m:
    print([c for c in m.server_capabilities if "ietf-interfaces" in c])   # does it speak
the model?
    before = m.get_config(source="running", filter=("subtree", SELECT)).data_xml
    use_candidate = ":candidate" in m.server_capabilities
    target = "candidate" if use_candidate else "running"
    try:
        with m.locked(target):
            m.edit_config(target=target, config=CHANGE, default_operation="merge")
            if use_candidate:
                m.validate(source="candidate")
                m.commit()
    except RPCError as err:
        if use_candidate:
            m.discard_changes()
        raise SystemExit(f"change refused, running config untouched: {err}")
    after = m.get_config(source="running", filter=("subtree", SELECT)).data_xml
    print("changed" if before != after else "no change")
```

Expected result on `dist1`: the capability list contains the `ietf-interfaces` URN, the script prints `changed`, and `show running-config interface Gi1/0/1` shows `description to-acc1 (uplink)`.

RESTCONF (RFC 8040) offers the same YANG data over HTTPS, so any HTTP client works. It is not arbitrary REST: URIs are derived from the model, and the media types are `application/yang-data+json` and `application/yang-data+xml`. The URI pattern is

```
https://<host>/restconf/data/<module>:<container>/<list>=<key>/<leaf>
```

with list keys written after `=`; a `/` inside a key must be percent-encoded (`GigabitEthernet1%2F0%2F1`).

```
$ curl -sk -u netops:$PW -H "Accept: application/yang-data+json" \
    https://10.255.0.11/restconf/data/ietf-
interfaces:interfaces/interface=GigabitEthernet1%2F0%2F1
{"ietf-interfaces:interface": {"name": "GigabitEthernet1/0/1", "description": "to-acc1
(uplink)",
 "type": "iana-if-type:ethernetCsmacd", "enabled": true}}
$ curl -sk -u netops:$PW -X PATCH -H "Content-Type: application/yang-data+json" \
    -d '{"ietf-interfaces:interface":{"name":"GigabitEthernet1/0/1","description":"to-
acc1"}}' \
    https://10.255.0.11/restconf/data/ietf-
interfaces:interfaces/interface=GigabitEthernet1%2F0%2F1 -w '%{http_code}\n'
204
```

(`-k` skips certificate checking and belongs in a lab only; see 9.3 (p. 543).) The verbs map to NETCONF operations as follows.

HTTP	NETCONF equivalent	Meaning in RESTCONF
GET	`get` / `get-config`	read a resource; `?content=config` or `nonconfig` selects the kind
POST	`edit-config` operation `create`	create a child; fails if it exists
PUT	`edit-config` operation `replace`	replace the target resource
PATCH	`edit-config` operation `merge`	merge the supplied data
DELETE	`edit-config` operation `delete`	remove the resource

RESTCONF has no candidate datastore, no lock and no commit: each request applies immediately to running. It is therefore simpler for single changes and for people who already know HTTP; NETCONF is the choice when a multi-step change must be atomic.

> **TRAP**
>
> *Claim:* "NETCONF runs on port 22." *Why it is wrong:* port 22 is plain SSH for the CLI. The IANA-assigned port for the NETCONF subsystem is 830, and some devices additionally accept the subsystem on 22. *Correct statement:* NETCONF is SSH subsystem `netconf` on TCP 830; RESTCONF is HTTPS on 443.

> **INTERVIEW LENS**
>
> CISCO · JUNIPER · NOKIA · GOOGLE
>
> "NETCONF versus RESTCONF versus gNMI" is asked by name. A strong answer gives transport, encoding and best use for each (compared in 10.12 (p. 609)), then states the deciding property: NETCONF for transactional configuration with candidate and commit, RESTCONF for simple HTTP integrations, gNMI for streaming telemetry and for configuration at scale. Expect "what is in the hello?" and "why a candidate datastore?"

10.12 gNMI and streaming telemetry SHOULD · ■■□

For three decades monitoring meant *polling*: a management station asks each device for its counters every five minutes using SNMP. Polling is like calling the plumber every five minutes to ask whether there is a leak. A micro-burst that lasts two seconds, a link that flaps and recovers, a queue that fills for a moment, all fall between two polls and leave no trace. *Streaming telemetry*

inverts the model: the device sends the data to a collector at a set interval or when a value changes, like a smart water meter that reports as soon as the flow changes.

Property	SNMP polling	gNMI streaming
Direction	collector pulls	device pushes (after one subscription)
Granularity	1 to 5 min typical	1 to 10 s typical; or on change
Data model	MIBs, OIDs	YANG paths, mostly OpenConfig
Encoding and transport	ASN.1 BER over UDP	protobuf over gRPC and HTTP/2, TLS
Load on the device	one request per object per poll	one subscription, batched updates
Events	traps, unreliable (UDP)	`ON_CHANGE` updates, reliable (TCP)

gNMI (the gRPC Network Management Interface) defines four RPCs on one gRPC service, over TCP with TLS. The IANA port is 9339; vendors use their own defaults (Arista EOS 6030, Nokia SR Linux 57400), so confirm on the target.

RPC	Purpose
`Capabilities`	the models, encodings and gNMI version the target supports
`Get`	one-shot read of one or more paths
`Set`	transactional change with `update`, `replace` and `delete` operations
`Subscribe`	open a stream of updates

A *path* is a slash-separated route through the YANG tree with keys in brackets, for example `/interfaces/interface[name=Ethernet1]/state/counters/in-octets`. *Encodings* are `JSON_IETF` (readable), `PROTO` (compact) and others. A subscription has a *mode*: `STREAM` keeps sending; `ONCE` sends the current values and closes; `POLL` sends values each time the client asks. Within a stream, each path has its own mode.

Stream mode	Behaviour	Use for
`SAMPLE` (with an interval)	send the value every N seconds	counters, rates, utilisation
`ON_CHANGE`	send only when the value changes	oper-status, BGP state, LLDP neighbours
`TARGET_DEFINED`	the device chooses the better of the two per leaf	when one does not know

In **dial-in** (the model above) the collector opens the gRPC connection to each device and subscribes. In **dial-out** the device opens the connection to a configured collector and pushes; this suits devices behind NAT or firewalls and avoids the collector holding thousands of sessions. The tool `gnmic` is a command-line client and collector.

```
# on leaf1 (Arista cEOS): enable gNMI
management api gnmi
   transport grpc default
$ gnmic -a 10.255.0.31:6030 -u admin -p admin --insecure capabilities
$ gnmic -a 10.255.0.31:6030 -u admin -p admin --insecure get --path
/interfaces/interface[name=Ethernet1]/state/oper-status
$ gnmic -a 10.255.0.31:6030 -u admin -p admin --insecure subscribe \
    --path "/interfaces/interface[name=Ethernet1]/state/counters" \
    --mode stream --stream-mode sample --sample-interval 10s
{"source": "10.255.0.31:6030", "subscription-name": "default-1",
 "timestamp": 1792000000000000000, "updates": [{"Path":
"interfaces/interface[name=Ethernet1]/state/counters/in-octets", "values": {"...":
9124031}}]}
```

(`--insecure` disables TLS and belongs in the lab only.) Replacing `--stream-mode sample` by `--stream-mode on-change` with the path `/interfaces/interface[name=Ethernet1]/state/oper-status` prints nothing until the port is shut, then prints one update `UP → DOWN` within a second.

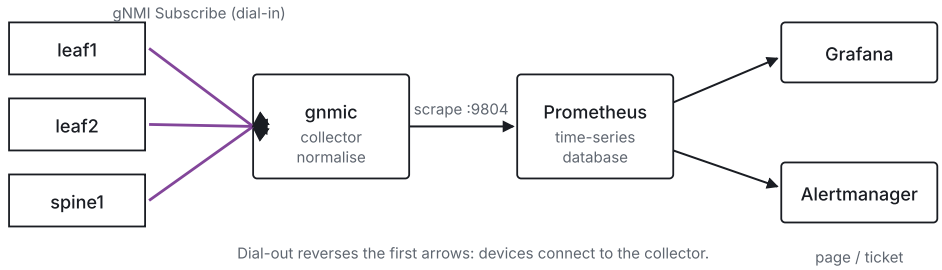


Figure 10.3 A model-driven telemetry pipeline. Devices stream; the collector turns updates into metrics; the database is scraped by Grafana and alert rules.

Sizing a pipeline is arithmetic, and Q10.25 uses it. A single `SAMPLE` leaf at 10 s produces 6 updates per minute; with 8 counters per port, 10,000 ports and 100 bytes per update, the stream is 10,000 × 8 × 6 × 100 B / 60 s = 0.8 MB/s in steady state, about 70 GB per day before compression, which is why sample intervals, leaf selection and `ON_CHANGE` for state matter. Telemetry does not replace SNMP everywhere: traps and legacy monitoring systems persist, so a real estate runs both for a time.

> LAB
>
> `LAB 10.2 · 60 MIN` Stream and flap. Start the telemetry stack from the containerlab topology (cEOS `leaf1`, `leaf2`; `gnmic`, Prometheus, Grafana containers).
>
> ```
> gnmic -a leaf1:6030 -u admin -p admin --insecure subscribe \
>   --path "/interfaces/interface[name=Ethernet1]/state/oper-status" --stream-mode on-
> change
> # in another terminal:
> docker exec clab-campus-leaf1 Cli -c "configure; interface Ethernet1; shutdown"
> ```
>
> ```
> ... "values": {"interfaces/interface/state/oper-status": "DOWN"}
> ```
>
> **What you should notice:** the update arrives within about a second of the command, with no poll in between. With `--stream-mode sample --sample-interval 10s` on the counters instead, the same flap shows only as a dip in the next sample.

10.13 Git for network engineers MUST · ■□□

Git is a distributed version-control system: it stores every version of a set of files, who changed them, when and why, and lets several people work on separate copies and merge them. It is the registrar's office for configuration: the old version is always retrievable and each change has a name against it. A *commit* is a saved snapshot with a message; a *branch* is a movable label on a line of commits; a *remote* is a shared copy (GitHub, GitLab) that people `push` to and `pull` from. For network work the repository holds the source of truth (10.6 (p. 596)), the templates, the tests and the pipeline definition.

Task	Command
Copy a repository	`git clone URL`
Start a change on a branch	`git switch -c feature/vlan-40`
See what changed	`git status`, `git diff`
Stage and save	`git add sot.yml` then `git commit -m "Add VLAN 40 (LAB) to acc1-acc3"`
Publish the branch	`git push -u origin feature/vlan-40`
Bring in others' work	`git fetch` (download only) then `git merge`; `git pull` is both
Undo a merged change	`git revert COMMIT` (adds a new commit that reverses it)
Label a release	`git tag -a v2026.10.29-1 -m "VLAN 40"`

`git fetch` updates the local copy of the remote branches without touching the working files, so it is always safe; `git pull` is `fetch` followed by a merge, which may stop on a conflict. A `commit` records history locally; a `push` sends it to the remote. *Merge* joins two lines of history with a merge commit; *rebase* replays one branch's commits on top of another for a straight line. Either is acceptable provided the team agrees, and one never rebases commits that others already hold.

The pull-request workflow is how a change is reviewed before it can reach a device.

```
main     o---o-----------------M---o       tag v2026.10.29-1 on M
          \                   /
feature    o---o---o---------o             pipeline runs on every push to the branch
```

1. Create the branch `feature/vlan-40` and edit `sot.yml` (add VLAN 40 and ports).
2. Commit with a message that says what and why, and push.
3. Open a *pull request* (PR). The pipeline (10.14 (p. 613)) lints, renders and tests, and posts the rendered diff as a comment.
4. A reviewer reads the YAML diff *and* the rendered configuration diff, and asks for changes or approves. A protected `main` branch refuses merges without a green pipeline and one approval.
5. Merge; the pipeline deploys. Tag the release so that "what was running last Tuesday" has an exact answer.

Secrets. A password committed to Git stays in the history even after the file is edited, and every clone has it; once committed, the only remedy is to *rotate* the secret. Hence `.gitignore` lists

1. A `key`, naming the leaf or leaves that identify an entry.↩
2. Read-only state data, for example `oper-status` and the counters; it cannot be written by `edit-config`.↩

`*.vault`, `.env` and `backups/`; a pre-commit hook (`detect-secrets`, `git-secrets`) rejects commits that contain a key; and credentials are injected from the environment or a secrets manager at run time (10.15 (p. 616)).

GitOps is the practice of making Git the only way to change the network. The repository states the intended configuration; the pipeline is the only actor with device credentials; and a scheduled job compares what the devices run with what Git says. The difference is *drift*, and finding it is as valuable as pushing changes.

```
import difflib, pathlib
from napalm import get_network_driver

def drift(name: str, driver: str, host: str, user: str, pw: str) -> str:
    dev = get_network_driver(driver)(hostname=host, username=user, password=pw)
    dev.open()
    try:
        running = dev.get_config(retrieve="running")["running"].splitlines()
    finally:
        dev.close()
    intended = pathlib.Path(f"rendered/{name}.cfg").read_text().splitlines()
    return "\n".join(difflib.unified_diff(intended, running, "git", "device",
lineterm=""))
```

A non-empty result raises a ticket (or, with approval, re-pushes the intended file). Real configurations contain lines the template does not own, such as certificates and timestamps, so a practical check ignores a list of patterns or compares sections only.

> CHECK YOURSELF
>
> 1. Why is `git revert` preferred to deleting a commit on a shared branch? [1]
> 2. A password was committed yesterday and the file was fixed today. Is it safe? [2]

10.14 CI/CD and testing for networks SHOULD · ■■□

Continuous integration (CI) runs a fixed set of checks automatically on every change; *continuous delivery* (CD) carries a change that has passed those checks to the devices, with a human or a rule approving each gate. The principle is a mock exam before the board exam: a failure in the laboratory costs minutes, the same failure in production costs an outage.

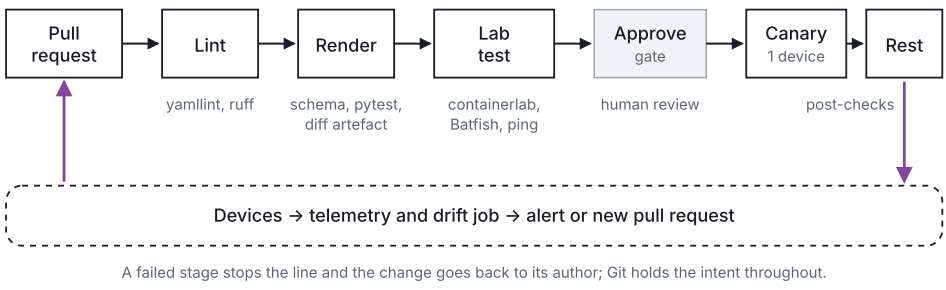


Figure 10.4 The GitOps pipeline. Gates fail cheaply early; the loop at the bottom closes the ladder of Figure 10.1.

Stages and what each catches.

Stage	Tool	Catches

Lint	`yamllint`, `ansible-lint`, `ruff`	syntax, style, deprecated modules
Schema	`jsonschema`, `pydantic`	wrong types, out-of-range VLANs, missing keys
Unit test	`pytest` on rendered text	a template that drops a line
Static analysis	Batfish	routing and ACL errors across the whole network, offline
Lab test	containerlab + pyATS or scripts	behaviour: sessions, reachability
Approval	protected environment	the human decision
Post-check	pyATS, NAPALM `compliance_report`	the change harmed something

A unit test on the rendered output is cheap and needs no device. The test below fails the pull request if any rendered access port lacks `no shutdown`.

```
import pathlib, re
import pytest

CONFIGS = sorted(pathlib.Path("rendered").glob("acc*.cfg"))

@pytest.mark.parametrize("cfg", CONFIGS, ids=lambda p: p.stem)
def test_every_access_port_is_enabled(cfg):
    blocks = re.split(r"(?m)^interface ", cfg.read_text())[1:]
    for block in blocks:
        if "switchport mode access" in block:
            assert " no shutdown" in block, f"{cfg.stem}: {block.splitlines()[0]} stays
shut"
```

Batfish builds a model of the network from the configuration files alone and answers questions without any device or packet. It finds undefined references, unreachable prefixes and ACLs that block what they should not, and so it checks a change *before* it exists on a device.

```
from pybatfish.client.session import Session
from pybatfish.datamodel import HeaderConstraints

bf = Session(host="localhost")
bf.set_network("campus")
bf.init_snapshot("snapshots/candidate", name="candidate", overwrite=True)
missing = bf.q.undefinedReferences().answer().frame()
assert missing.empty, missing                       # an ACL or route-map name that does
not exist
flow = bf.q.testFilters(headers=HeaderConstraints(srcIps="192.168.30.10",
dstIps="192.168.10.5",
                                                  applications="https"),
nodes="dist1").answer().frame()
assert (flow["Action"] == "DENIED").all(), "GUEST must not reach SALES"
```

The **lab test** builds a *digital twin*, a small copy of the topology in containers, so the real configuration is applied to something that behaves like the network. *Containerlab* describes the topology in YAML and starts cEOS, FRR and Linux containers in seconds.

```
# lab/campus.clab.yml
name: campus
topology:
  nodes:
    leaf1:  {kind: ceos,  image: ceos:4.32.0F}
    leaf2:  {kind: ceos,  image: ceos:4.32.0F}
    spine1: {kind: linux, image: quay.io/frrouting/frr:9.1.0}
  links:
    - endpoints: ["leaf1:eth1", "spine1:eth1"]
    - endpoints: ["leaf2:eth1", "spine1:eth2"]
```

The pipeline, in GitHub Actions syntax, chains the stages with `needs`; the `production` environment is configured in the repository to require a named approver, which is the gate of Figure 10.4.

```
# .github/workflows/network.yml
name: network-ci
on: {pull_request: {}, push: {branches: [main]}}
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install yamllint ruff && yamllint sot.yml && ruff check .
  render:
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt && python render.py && pytest -q tests/unit
      - uses: actions/upload-artifact@v4
        with: {name: rendered, path: rendered/}
  lab-test:
    needs: render
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/download-artifact@v4
        with: {name: rendered, path: rendered/}
      - run: sudo containerlab deploy -t lab/campus.clab.yml
      - run: python tests/push_and_check.py
      - if: always()
        run: sudo containerlab destroy -t lab/campus.clab.yml --cleanup
  deploy:
    if: github.ref == 'refs/heads/main'
    needs: lab-test
    environment: production
    runs-on: self-hosted
    steps:
      - uses: actions/checkout@v4
      - run: python deploy.py --canary leaf1 --then-rest
        env: {NET_USER: "${{ secrets.NET_USER }}", NET_PASS: "${{ secrets.NET_PASS }}"}
```

WORKED EXAMPLE

A pull request that the lab catches. A reviewer-approved change to `access_switch.j2` tidies the port block and, by accident, deletes the line `no shutdown`.

1. The linter and the schema check pass, because the YAML data are unchanged and the template is syntactically valid.
2. The unit test of the previous listing fails on `acc1`: `acc1: GigabitEthernet1/0/1 stays shut`. The pull request is blocked before the lab starts.

3. Suppose that test did not exist. The lab test pushes the configuration to a cEOS host-facing port and its ping from host `h1` to `h2` fails: `0 received, 100% packet loss`. The job fails and `deploy` never runs.

Result: the defect is found in a container, at the cost of a failed pull request, rather than on thirty access ports on a Monday morning.

Pre- and post-checks. Before the push the script records a baseline, and after it compares; the comparison is the proof that the change did what was intended and nothing else.

Check	Before	After	Verdict rule
BGP sessions established	4	4	must be equal
Routes in the global table	128	128	within ±2
Interfaces up	24	24	equal, except the one changed
Reachability matrix (ping)	all 12 pairs	all 12 pairs	no pair lost
Config lines changed		3	equals the reviewed diff

If any rule fails, the script rolls back (10.15 (p. 616)) and exits with an error. Such a pre/post framework is what Cisco's *pyATS* provides: `pre = dev.learn("bgp")` and `post = dev.learn("bgp")` return structured snapshots, and `Diff(pre.info, post.info)` prints only the differences.

INTERVIEW LENS

CISCO · AMAZON · GOOGLE · ARISTA

"What would you put in a CI pipeline for network changes?" The strong answer lists stages in order with the defect each one catches (lint, schema, unit tests, Batfish, digital-twin test, approval, canary, post-check, rollback) and says which stages are cheap and run on every push. A follow-up is "how do you test what you cannot emulate?": hardware features such as ASIC limits and optics are covered by the canary and the post-check, not by the lab.

10.15 Safety at scale, and the platforms you will meet

OPTIONAL · ■■□

Automation raises the speed of a mistake as well as the speed of a fix, so the safety design matters more than the tool. The principles are few and each answers a way things go wrong.

Control	Meaning	Prevents
Idempotence	applying twice equals applying once	duplicate or oscillating changes after a retry
Dry run and diff	compute the change, show it, apply nothing	surprises; the reviewer sees the lines
Approval gate	a person accepts the diff	an unreviewed change reaching production

Canary and waves	one device, then a few, then the rest, with a pause and a check between	a defect hitting everything at once
Pre/post-check	measure state before and after	silent damage
Automatic rollback	revert when a check fails or access is lost	a long outage after a bad push
Concurrency cap	limit simultaneous sessions (for example 8)	overload of AAA servers and device control planes
Secrets management	credentials from a vault or CI secret; short-lived where possible	credentials in Git or in logs
Audit log	who, what, when, from which commit	an unanswerable post-mortem

Rollback has to work *without* the network the change may have broken. On IOS, `reload in 10` before the change and `reload cancel` after a successful check restore the saved configuration if the device becomes unreachable; EOS has `commit timer 00:10:00` in a configuration session, Junos has `commit confirmed`, and NETCONF has a confirmed-commit (10.11 (p. 607)). A NAPALM `rollback()` needs a reachable device, and so cannot rescue the change that cut the path.

Waves. A rollout is a sequence of groups with a soak time and a gate between them. For Campus Net's 40 access switches a plan might be:

Wave	Targets	Soak	Gate to continue
0	lab twin	—	CI green
1	`acc1` (canary)	10 min	post-checks pass, no new alarms
2	4 switches in one building	15 min	same, plus error counters flat
3	10 % of the rest	15 min	same
4	remainder, in groups of 10	5 min each	same; stop at first failure

WORKED EXAMPLE

A story with five lessons. A script was run against 120 branch routers. The template had been edited so that the default-route line was dropped, and the script's thread pool was unbounded. It opened 120 sessions at once, took out the TACACS+ server, pushed a configuration that removed `ip route 0.0.0.0 0.0.0.0` and saved it. The branches could not be reached to repair them.

1. *Diff review.* A dry-run diff would have shown `- ip route 0.0.0.0 0.0.0.0 …` on the first device; a rule "refuse any diff that removes a route or more than N lines" would have stopped the run mechanically.
2. *Canary.* The first wave of one router would have lost its default route alone.
3. *Post-check with automatic rollback.* A reachability check from the management system to the router's loopback, with `reload in 10` armed, restores the saved configuration if the check fails.
4. *Concurrency cap.* Eight workers would not have overloaded AAA, and the failure would not have blocked emergency logins.
5. *Save only after verification.* Writing to NVRAM only after the post-check passed means a reload returns the old, working configuration.

> **Result: any one of the five contains the damage to one router; together they make this class of incident close to impossible.**

Platforms. Some vendors offer a controller with its own API, so a script talks to the controller rather than to each device. Knowing the names lets one place a platform in a conversation.

Platform	Vendor	Interface	Role
Catalyst Center (DNA Center)	Cisco	REST, intent API	campus automation and assurance
Meraki Dashboard	Cisco	REST (cloud)	cloud-managed access and security
CloudVision	Arista	gRPC, REST, Python	change control and telemetry for EOS
Junos PyEZ, Apstra	Juniper	NETCONF-based Python library; intent-based data-centre controller	Junos automation
NX-API	Cisco	JSON-RPC over HTTP on Nexus	scripting Nexus switches
SONiC	open source	Redis, gNMI, REST	the switch OS of several hyperscalers

Software-defined networking separates the control plane from the data plane (5.7 (p. 318)); OpenFlow was the first protocol to program forwarding tables directly, P4 describes the forwarding pipeline itself, and in practice the hyperscalers run standard BGP with custom automation and telemetry rather than a central OpenFlow controller. Credentials and key handling in the automation system follow the rules of 9.12 (p. 567) and 9.13 (p. 569); for the live operation of a network that automation has broken, the method of 11.1 (p. 635) applies.

> TRAP
>
> *Claim:* "The script worked in the lab, so it is safe to run on everything." *Why it is wrong:* the lab has no TACACS+ load, no old software release, no device with a hand-made exception and no human traffic. *Correct statement:* production access to a change is earned in waves, with a gate between each, and the first production device is the first real test.

1. History that others have already pulled must not be rewritten; `revert` adds a new commit, so every clone stays consistent and the audit trail is kept.↩
2. No. The old commit still holds the password in every clone; rotate the secret, and then remove it from history if policy requires.↩

Problem Set

Every interview question of this pillar is solved once, here, in the compact template: a 30-second answer first, then approach, answer, code, follow-ups and mistakes where they apply. Tags give the companies and round where the topic was reported, the difficulty (■□□ easy, ■■□ medium, ■■■ hard) and the evidence (S: a specific question reported by candidates; T: a topic reported, our wording). All code is Python and has been checked for syntax; the tests use only the standard library.

Q10.1 Why automate networks? Give three benefits and one risk.

CISCO · AMAZON · JUNIPER · TECH-1 · ■□□ · T

30-second answer: consistency (one template gives every device the same lines), speed (concurrency turns hours into minutes) and auditability (the change, its input and its output are in Git and logs). The risk is blast radius: a wrong change is applied everywhere just as quickly.

Answer. Attach a number to each: forty switches by hand take about four hours with a 70 % chance of at least one slip at a 3 % per-switch error rate (10.1 (p. 587)); a script takes six minutes. Reproducibility is a fourth benefit, since a change tested in the lab is replayed byte for byte. **Follow-ups.** How do you contain the risk? Dry run, canary, post-checks, rollback (10.15 (p. 616)). **Mistakes.** Answering only “it saves time”, or ignoring the risk.

Q10.2 JSON, YAML or XML for a human-edited inventory?

CISCO · GOOGLE · TECH-1 · ■□□ · T

30-second answer: YAML, because it allows comments, has little punctuation and is easy to diff. JSON suits machine-to-machine APIs; XML is used where a protocol requires namespaces, as in NETCONF.

Answer. All three describe the same tree, and a parser converts between them (10.2 (p. 589)). People edit YAML in review, so comments (the reason for an exception) and a clean line-by-line diff matter. YAML’s weaknesses are implicit typing (`no` becomes `false`, `010` octal) and whitespace sensitivity, handled by quoting strings, using `safe_load` and validating against a schema. **Mistakes.** Using JSON for a file people must annotate; loading YAML with the unsafe loader.

Q10.3 Paramiko or Netmiko: what problem does Netmiko solve?

CISCO · AMAZON · TECH-1 · ■□□ · S

30-second answer: Paramiko is raw SSH, so the script must sleep and guess when output ends. Netmiko adds per-vendor knowledge: login sequence, prompt detection, paging off, configuration mode entry and exit, and save.

Answer. With Paramiko one writes `send`, `sleep`, `recv` and parses the prompt by hand; a slow command is truncated and a fast one wastes time. Netmiko’s `send_command` reads until the device prompt returns, so it knows the answer is complete. It also normalises dozens of platforms through `device_type`. **Follow-ups.** When would you still use Paramiko? SFTP and Linux hosts. **Mistakes.** Saying Netmiko is “faster”: it is more reliable, not faster.

Q10.4 What does `send_config_set` do differently from `send_command`?

CISCO · AMAZON · ARISTA · TECH-1 · ■□□ · S

30-second answer: `send_command` runs one operational command and returns its output, waiting for the prompt. `send_config_set` enters configuration mode, sends a list of lines, leaves, and returns the session text; it does not save.

Answer. The first expects a query; the second changes state. After `send_config_set`, call `save_config()` once the change has been verified, because saving is a separate decision. For a configuration command sent through `send_command`, the device would answer `Invalid input` because the session is not in configuration mode. **Follow-ups.** How do you detect an error in a config push? Inspect the returned text for `Invalid input` or `%` markers, or use `error_pattern`. **Mistakes.** Forgetting that unsaved changes vanish at reload; saving blindly.

Q10.5 What is idempotency, and why does it matter for Ansible playbooks?

CISCO · JUNIPER · NOKIA · TECH-1 · ■□□ · S

30-second answer: an operation is idempotent if applying it twice leaves the same state as applying it once. It lets a playbook be re-run safely after a failure, and `changed=0` on the second run is the proof.

Answer. Declarative modules (`ios_vlans`) read the current state and act only on the difference. Command modules (`ios_command`) always run and so always report `changed` . Idempotence matters because automation retries: a half-failed run is repaired by running it again. **Follow-ups.** Which operation is not idempotent? `ip route add` without a check, or appending a line to a list. **Mistakes.** Believing Ansible makes everything idempotent (10.9 (p. 603)).

Q10.6 What is Git? Fetch versus pull; commit versus push.

CISCO · AMAZON · GOOGLE · TECH-1 · ■□□ · S

30-second answer: Git is a distributed version-control system that records every version of the files with author and message. `fetch` downloads remote changes without applying them; `pull` is fetch plus merge. A `commit` saves locally; `push` sends commits to the remote.

Answer. Distributed means each clone holds the full history, so work continues offline and any clone can restore the repository. A branch is a movable label on commits; a merge or rebase joins branches (10.13 (p. 612)). **Mistakes.** Treating a commit as shared before it is pushed; committing secrets.

Q10.7 Return the devices whose uptime is under one day, from `show version` text.

AMAZON · CISCO · TECH-1 · ■■□ · S

30-second answer: parse the uptime sentence with one regular expression that captures every `N unit` pair, convert units to seconds and compare with 86,400. Handle both IOS (“3 hours, 12 minutes”) and EOS (“5 weeks, 4 days”) wording.

Approach. The uptime line lists units in descending order, and any may be absent, so a loop over `(\d+)\s+(year|week|day|hour|minute)s?` is more robust than a fixed pattern. Weights: year 365 days (IOS counts years as 365), week 7 days.

```
import re

UNIT = {"year": 31536000, "week":
604800, "day": 86400, "hour": 3600,
"minute": 60}
LINE = re.compile(r"uptime is (?
P<t>.+)", re.I)
PAIR = re.compile(r"(\d+)\s+
(year|week|day|hour|minute)s?", re.I)

def uptime_seconds(show_version: str) -
> int | None:
    m = LINE.search(show_version)
    if not m:
        return None
# unknown: caller decides, never guess
    return sum(int(n) * UNIT[u.lower()]
for n, u in PAIR.findall(m.group("t")))

def rebooted_recently(devices:
list[dict]) -> list[str]:
    out = []
    for d in devices:
        s =
uptime_seconds(d["show_version"])
        if s is not None and s < 86400:
            out.append(d["name"])
    return out

devs = [{"name": "acc1", "show_version":
"acc1 uptime is 3 hours, 12 minutes"},
        {"name": "dist1",
"show_version": "dist1 uptime is 1 year,
2 weeks, 1 day"},
        {"name": "leaf1",
"show_version": "no uptime here"}]
assert rebooted_recently(devs) ==
["acc1"]
assert uptime_seconds("x uptime is 1
week, 2 days, 3 hours, 4 minutes") ==
604800 + 172800 + 10800 + 240
```

Complexity. Linear in the text length. **Follow-ups.** Why `None` for unparsable? Treating “unknown” as “recent” or “old” would hide a parse failure (10.5 (p. 594)). **Mistakes.** Parsing only `hours` ; forg-

etting `weeks`, which Cisco prints for uptimes above seven days.

Q10.8 Back up 100 devices fast and robustly. How?

AMAZON · CISCO · MICROSOFT · TECH-1 · ■■□ · S

30-second answer: a thread pool of about eight workers (the work is waiting on I/O), a timeout and bounded retries with back-off per device, no retry on authentication failure, one result per device without raising, logging, credentials from the environment or a vault, and a Git commit only when the configuration changed.

Approach. The pool and retry skeleton is the script of 10.4 (p. 591). What it lacks, and an interviewer expects, is change detection: two backups differ by timestamps and counters, so naive storage creates noise.

```
import re, subprocess
from pathlib import Path

VOLATILE = re.compile(r"^(Building
configuration|Current configuration|!
Last configuration"
                      r"|! NVRAM
config|ntp clock-period|! Time:).*$",
re.M)

def normalise(cfg: str) -> str:
    return VOLATILE.sub("", cfg).strip()
+ "\n"

def store_if_changed(repo: Path, name:
str, cfg: str) -> bool:
    path = repo / f"{name}.cfg"
    new = normalise(cfg)
    if path.exists() and
path.read_text() == new:
        return False
# nothing to record
    path.write_text(new)
    subprocess.run(["git", "-C",
str(repo), "add", path.name],
check=True)
    subprocess.run(["git", "-C",
str(repo), "commit", "-m", f"{name}:
configuration changed"], check=True)
    return True

assert normalise("Building
configuration...\nhostname acc1\n") ==
"\nhostname acc1\n".lstrip("\n")
```

Answer. Eight workers is a ceiling set by the AAA server and device control planes, not by the interpreter; 100 devices at 6 s take 100 / 8 × 6 = 75 s. The exit status is non-zero if any device failed, so the scheduler alerts. **Follow-ups.** How do you find that a device changed unexpectedly? The Git history of that file; this is also drift detection. **Mistakes.** Committing secrets that appear in configurations (`username … secret`, SNMP strings): strip or encrypt them before storing.

Q10.9 How would you parse unstructured CLI output? Compare regex, TextFSM and Genie.

CISCO · AMAZON · ARISTA · TECH-2 · ■■□ · S

30-second answer: prefer an API with structured output; failing that, TextFSM (ntc-templates) for multi-line state-based parsing, Genie for a Cisco estate, TTP when writing your own, and plain regex for a single command on a single release.

Answer. Regex is fastest to write and the most fragile. TextFSM separates variables from a small state machine, and hundreds of templates exist. Genie parsers are maintained by the vendor and return a consistent schema, at the price of a heavy dependency. Whatever the method, pin the release the parser was tested on and treat zero rows as an error (10.5 (p. 594)). **Follow-ups.** What does `use_textfsm=True` do? Netmiko looks up the template for the platform and command in ntc-templates and returns a list of dictionaries. **Mistakes.** Parsing in production without a test that feeds saved real output.

Q10.10 Explain Jinja2 with a loop over interfaces.

GOOGLE · MICROSOFT · AMAZON · TECH-1 · ■■□ · S

30-second answer: Jinja2 combines a template with data. `{{ x }}` prints a value, `{% for %}` repeats a block, and filters transform values. A loop over a list of port dictionaries produces one `interface` stanza per port.

Answer. Given `ports: [{name: Gi1/0/1, vlan: 10}, {name: Gi1/0/2, vlan: 20}]`, the template

```
{% for p in ports %}
interface {{ p.name }}
 switchport access vlan {{ p.vlan }}
{% endfor %}
```

renders two stanzas. Use `StrictUndefined` so a missing key is an error, `trim_blocks` for clean whitespace, and validate the data first (10.6 (p. 596)). **Follow-ups.** Macros, `include` and inheritance for reuse; filters such as `join`, `sort` and custom ones. **Mistakes.** Putting decisions in the template that belong in the data; rendering unvalidated input.

Q10.11 What is a source of truth, and why not the devices' running config?

GOOGLE · MICROSOFT · AMAZON · TECH-2 · ■■□ · T

30-second answer: the SoT is the single authoritative record of intended state (a Git repository of YAML, or NetBox). The running config records only what is configured, so any manual error or half-applied change is "correct" by that definition and cannot be detected as drift.

Answer. Intent and actual state must be separate so that they can be compared. The SoT also holds relations a device does not know: which VLAN belongs to which site, which prefix is allocated to whom. Changes enter through the SoT and the pipeline; devices are checked against it (10.13 (p. 612)). **Follow-ups.** What if the SoT and a device disagree? A drift report decides whether to correct the device or to update the SoT, a human decision. **Mistakes.** Treating NetBox as the SoT of configuration text rather than of data.

Q10.12 NAPALM `load_merge_candidate` versus `load_replace_candidate`; risks of replace?

CISCO · ARISTA · JUNIPER · TECH-2 · ■■□ · S

30-second answer: merge adds the given lines to the running configuration; replace makes the running configuration equal to the given file. Replace deletes anything not in the file, including management access, so a missing line can lock you out.

Answer. Replace is attractive because the result is exactly the rendered file and drift disappears; its price is that the file must be complete. The control is the diff: `compare_config()` before `commit_config()`, a commit timer or `reload in` for recovery, and a canary (10.7 (p. 599)). **Follow-ups.** How does IOS implement replace? SCP plus `configure replace`, which needs the archive feature. **Mistakes.** Rendering a file without the management ACL and AAA lines, then committing it without reading the diff.

Q10.13 What is YANG? Explain container, list, leaf and key with an interface example.

CISCO · JUNIPER · NOKIA · TECH-2 · ■■□ · S

30-second answer: YANG is a language that defines the schema of configuration and state data: a tree of containers, lists, leaves and types. A `list` such as `interface` holds entries identified by a `key` such as `name`; each entry has leaves like `description` and `enabled`.

Answer. `/interfaces` is a container; `interface` is a list keyed by `name`, so `interface[name='GigabitEthernet1/0/1']` picks one entry as a primary key picks a row; `description` is a leaf of type string. `config false` marks state (counters, `oper-status`). Models come in IETF, OpenConfig and native families (10.10 (p. 604)). **Follow-ups.** Augment and deviation. **Mistakes.** Saying YANG is a protocol; it is a data-modelling language, carried by NETCONF, RESTCONF and gNMI.

Q10.14 NETCONF, RESTCONF and gNMI: transport, encoding, use.

CISCO · JUNIPER · GOOGLE · ARISTA · TECH-2 · ■■□ · S

30-second answer: NETCONF is XML over SSH (port 830) with candidate datastore and commit, for transactional configuration. RESTCONF is JSON or XML over HTTPS using HTTP verbs, for simple integrations. gNMI is protobuf over gRPC and HTTP/2, for streaming telemetry and configuration at scale.

	NETCONF	RESTCONF	gNMI
Transport	SSH subsystem, TCP 830	HTTPS, 443	gRPC/HTTP/2 with TLS, 9339 or vendor port
Encoding	XML	JSON or XML	protobuf, JSON_IETF
Operations	get, edit-config, commit, lock	GET, POST, PUT, PATCH, DELETE	Capabilities, Get, Set, Subscribe
Transactions	yes (candidate, commit)	no, each call applies	`Set` is atomic per request
Best for	multi-step safe changes	quick reads and edits, web tooling	telemetry, large fleets

All three use YANG models, so the content is the same; only the carrier differs (10.11 (p. 607), 10.12 (p. 609)). **Mistakes.** Calling RESTCONF "REST on the device": it is YANG-driven.

Q10.15 Which port does NETCONF use, and what happens in `<hello>`?

CISCO · JUNIPER · NOKIA · TECH-1 · ■□□ · S

30-second answer: TCP 830, as the SSH subsystem `netconf`. Each side immediately sends a `<hello>` listing its capabilities (protocol version, optional features such as candidate and validate, and the YANG modules it supports); the session uses the capabilities both share.

Answer. The client learns what the server can do before sending any request: whether a candidate datastore exists, whether `confirmed-commit` is supported, and which modules to use. The session also carries a session ID in the server's hello. Version 1.1 adds chunked framing after the hello. **Follow-ups.** What if the capability is missing? The client falls back to `running` and takes more care (10.11 (p. 607)). **Mistakes.** Saying port 22.

Problem Set, continued: protocols, design and incident questions

Q10.16 What are NETCONF datastores, and why is a candidate valuable?

CISCO · JUNIPER · NOKIA · TECH-2 · ■■□ · S

30-second answer: `running` is the active configuration, `startup` is loaded at boot, `candidate` is an editable scratch copy that has no effect until committed. It lets several edits be made, validated and applied as one atomic change, or discarded.

Answer. Without a candidate, each `edit-config` hits the live configuration at once, so a change of five steps is visible half-done. With one, the sequence is lock, edit, validate, commit, unlock; an error leads to `discard-changes`, and running was never touched. `confirmed-commit` adds a timer that reverts unless the commit is confirmed, which protects against a change that cuts the management path. **Follow-ups.** Which devices lack it? Many IOS XE releases are `writable-running` only. **Mistakes.** Assuming every device has a candidate; check the capability in the hello.

Q10.17 Map RESTCONF HTTP methods to NETCONF operations.

CISCO · JUNIPER · TECH-1 · ■■□ · S

30-second answer: GET is `get` / `get-config`; POST is `edit-config` with `create`; PUT is `replace`; PATCH is `merge`; DELETE is `delete`.

Answer. The URI is `/restconf/data/<module>:<path>`, list keys after `=`, with `/` in a key encoded as `%2F`. The media type is `application/yang-data+json` or `+xml`. Unlike NETCONF there is no lock, candidate or commit: each call takes effect immediately, so RESTCONF is not suitable for a multi-step atomic change. PATCH is the usual choice for “set this leaf” because it leaves siblings alone, whereas PUT replaces the target resource (10.11 (p. 607)). **Mistakes.** Using PUT on a whole interface when only the description was meant, which resets its other leaves.

Q10.18 Streaming telemetry versus SNMP polling: why did hyperscalers move?

GOOGLE · MICROSOFT · ARISTA · TECH-2 · ■■□ · S

30-second answer: polling shows only what happens at the instants of the poll and costs the device a request per object per poll; streaming pushes updates at seconds’ granularity or on change, with a structured YANG model, over a reliable transport.

Answer. Events shorter than the poll interval are invisible to polling; a link that flaps in 30 s between five-minute polls leaves no trace. Polling 100,000 objects per device from several systems loads the control-plane CPU, whereas one subscription batches updates. SNMP traps are UDP and may be lost; `ON_CHANGE` updates ride TCP. The data are named by YANG paths rather than OIDs, so one OpenConfig path serves several vendors (10.12 (p. 609)). **Follow-ups.** Does it replace SNMP? Not entirely: traps and legacy systems remain, and the two coexist for years. **Mistakes.** Claiming telemetry is “faster” without saying why.

Q10.19 Explain gNMI subscription modes. ON_CHANGE or SAMPLE?

GOOGLE · ARISTA · JUNIPER · NOKIA · TECH-2 · ■■□ · S

30-second answer: `STREAM` continues until cancelled, `ONCE` returns the current values and closes, `POLL` returns values on each client request. Within a stream, `SAMPLE` sends at an interval and `ON_CHANGE` sends only when the value changes. Use `SAMPLE` for counters and `ON_CHANGE` for state.

Answer. Counters change every packet, so `ON_CHANGE` would flood; only a time series with an interval is meaningful. Operational state (`oper-status`, BGP session state, LLDP neighbours) changes rarely and matters at once, so `ON_CHANGE` gives delay of about a second and almost no traffic. `TARGET_DEFINED` lets the device choose per leaf. A heartbeat interval re-sends unchanged values so that a collector can tell silence from a dead stream. **Mistakes.** `ON_CHANGE` on counters; very short sample intervals on thousands of ports.

Q10.20 Ansible or Python scripts: when would you pick each?

CISCO · JUNIPER · NOKIA · TECH-1 · ■■□ · S

30-second answer: Ansible for repeatable, declarative checklists that operators who do not program must run, using vendor collections and idempotent modules. Python for logic with branches, calls to other systems, and unit tests.

Criterion	Ansible	Python (Netmiko, NAPALM, Nornir)
Team skill	operators, YAML	programmers
Task shape	state the desired end	loop, decide, compute
Idempotence	by module	by your code
Testing	`--check`, molecule	`pytest`, mocks
Error handling	blocks, `rescue`	exceptions, any logic

Answer. Most teams use both: Ansible runs the standard changes and a Python tool decides what to run. **Mistakes.** Writing loops and conditionals in YAML until the playbook is an unreadable program; or writing Python to do what a resource module already does idempotently.

Q10.21 What goes into a CI pipeline for network configuration changes?

CISCO · AMAZON · GOOGLE · TECH-2 · ■■□ · S

30-second answer: lint, schema validation, render, unit tests on the rendered text, static analysis with Batfish, a digital-twin lab test, a diff artefact for the reviewer, an approval gate, canary deployment, post-checks and automatic rollback.

Answer. Order stages from cheap to expensive so that most failures happen in seconds (10.14 (p. 613)). Each stage has a named defect: lint (syntax), schema (a VLAN 4096), unit test (a dropped `no shutdown`), Batfish (an undefined route-map), lab (BGP does not establish), post-check (route count fell). The pipeline posts the rendered diff on the pull request. **Follow-ups.** What do you not test? Hardware limits and optics; the canary covers them. **Mistakes.** A pipeline that only lints.

Q10.22 Design a rollout of new NTP and AAA configuration to 5,000 devices on three vendors with automatic rollback.

AMAZON · CISCO · GOOGLE · HM · ■■■ · T

30-second answer: model the data once in a source of truth, render per vendor, test in a digital twin, then roll out in waves (1, 1 %, 10 %, the rest) with a concurrency cap, per-device pre/post-checks, and a device-side timer that reverts the change if management access is lost.

Approach. (i) *Data*: one YAML for NTP servers, TACACS+ servers and keys' references, with per-site overrides. (ii) *Render*: a Jinja2 template per OS family (IOS XE, EOS, Junos), unit-tested on rendered text. (iii) *Test*: Batfish for syntax and references; containerlab for each family. (iv) *Rollout*: AAA is the dangerous half, because a wrong key locks everybody out, including the automation. Therefore NTP first, then AAA with the old servers kept as fallback and local-user fallback. (v) *Rollback*: `reload in` or `commit confirmed` or `commit timer` armed before the push, cancelled only after a post-check that logs in *again with the new method* and reads NTP status. (vi) *Pace*: 5,000 devices at 8 per batch of 20 s is 5000 / 8 × 20 = 12,500 s, about 3.5 h; with 32 workers, under an hour, still below the AAA server's session limit. Waves with a soak time dominate the schedule.

Answer. Stop at the first failure in a wave; record all results; leave the pipeline's own jump host out of the first wave so that someone can repair the rest. **Follow-ups.** What if a device is offline? Mark it pending and converge it later; a declarative push is idempotent. **Mistakes.** Changing AAA and NTP in one step; testing only the success path of authentication.

Q10.23 Your automation pushed a bad ACL that isolated 30 sites. Walk through response and process changes.

AMAZON · MICROSOFT · GOOGLE · HM · ■■■ · T

30-second answer: stop the pipeline, restore service first (revert the commit and redeploy, or use out-of-band access), communicate, then run a blameless post-mortem and add the missing control: a diff guard, a canary and a post-check.

Approach. *Contain*: pause all jobs so that nothing else changes; identify the change by the commit and tag. *Restore*: `git revert` the commit and run the pipeline against the 30 sites in groups; where management is cut, use the out-of-band console or the device-side timer, if one was armed. *Communicate*: a status page and named owner every 30 minutes. *Learn*: the timeline from logs (who approved what, which tests passed, which did not exist). *Fix the process*: a rule to refuse diffs that touch management ACLs; Batfish `testFilters` for management reachability in CI; a first wave of one site with a post-check from the management network; a revert timer. **Mistakes.** Blaming the engineer; fixing the ACL forward under pressure instead of reverting to the last known good state (11.1 (p. 635)).

Q10.24 How would you test a configuration change before production?

CISCO · ARISTA · AMAZON · TECH-2 · ■■■ · S

30-second answer: a digital twin in containerlab for behaviour, Batfish for whole-network analysis offline, unit tests of the rendered text, then a canary with pre/post-checks and rollback in production.

Answer. The four layers catch different faults. Unit tests catch template errors in milliseconds. Batfish evaluates routing and filters from configuration files and answers “can GUEST reach SALES?” without a device. The twin runs real control-plane software, so protocol sessions form and pings cross. The canary meets what no emulation has: hardware limits, real optics, real traffic. State the limits honestly: a twin has different performance and some features do not exist in the container images (10.14 (p. 613)). **Mistakes.** Treating a green lab as proof; testing only the happy path.

Q10.25 Design telemetry that detects micro-bursts on 10,000 ports and alerts within 30 s. Estimate the data.

GOOGLE · MICROSOFT · ARISTA · HM · ■■■ · T

30-second answer: sampling counters every 10 s cannot see a burst of milliseconds, so use device-side detection (queue-depth thresholds, drop counters, or a burst-event feature) streamed `ON_CHANGE`, plus 10 s counter samples for context. About 4,000 samples per second, about 35 GB a day raw, and an alert path of about 20 s.

Approach. *What to measure*: a micro-burst is a short queue fill: it shows as tail drops or a high queue-depth watermark, not as average utilisation (a 10 s mean of a 5 ms burst at line rate is 0.05 %). So subscribe to drop counters and the queue high-watermark per port. *Volume*: 10,000 ports × 4 leaves (in-octets, out-octets, discards, queue watermark) every 10 s is 4,000 updates/s; at 100 B each that is 400 kB/s, 34.6 GB/day raw. A time-series database stores a sample in about 1.5 B, so 4,000 × 86,400 × 1.5 B ≈ 0.5 GB/day. *Capacity*: 10,000 ports on 48-port devices is about 210 devices; four `gnmic` collectors at roughly 50 devices each. *Latency budget*: 10 s sample interval, plus 2 s transport and write, plus an alert rule evaluating over 15 s, gives about 27 s, inside 30 s.

Answer. Show the pipeline of Figure 10.3 and say where it scales: collectors are sharded by device, the database by time and label. Alert on rate of drops, not on a single sample, to avoid noise. **Follow-ups.** What if the device cannot detect bursts? Mirror to a probe, or use shorter sample intervals on a small set of suspect ports. **Mistakes.** Promising to see milliseconds with 10 s sampling.

Q10.26 Write a Python `Device` class with `connect`, `get_interfaces` and `push_config(dry_run=True)`, and unit-test it without devices.

AMAZON · CISCO · ARISTA · TECH-2 · ■■■ · S

30-second answer: inject the connection factory so tests can pass a fake, keep parsing in a pure function, and make `dry_run` the default so that sending needs an explicit decision. Tests then assert on calls and results without the network.

Approach. Three decisions make it testable: *dependency injection* (the constructor accepts a `connector`), *pure parsing* (a function from text to data) and *safe default* (`dry_run=True`). The test double records what was sent.

```
from __future__ import annotations
import unittest

def parse_brief(text: str) -> dict[str, dict]:
    """Parse 'show ip interface brief' into {name: {ip, status, proto}}."""
    out = {}
    for line in text.splitlines()[1:]:
        parts = line.split()
        if len(parts) < 6:
            continue
        status = " ".join(parts[4:-1])  # "administratively down" has two words
        out[parts[0]] = {"ip": parts[1], "status": status, "proto": parts[-1]}
    return out

class Device:
    def __init__(self, name: str, host: str, connector=None, device_type: str = "cisco_ios"):
        self.name, self.host, self.device_type = name, host, device_type
        self._connector, self._conn = connector, None

    def connect(self, **creds) -> None:
        if self._connector is None:
            from netmiko import ConnectHandler            # imported late: tests need no library
            self._connector = ConnectHandler
        self._conn = self._connector(device_type=self.device_type, host=self.host, **creds)

    def get_interfaces(self) -> dict[str, dict]:
        return parse_brief(self._conn.send_command("show ip interface brief"))

    def push_config(self, lines: list[str], dry_run: bool = True) -> str:
        if dry_run:
            return "\n".join(lines)  # show what would be sent; send nothing
        return self._conn.send_config_set(lines)

class FakeConn:
    def __init__(self, **kw):
        self.kw, self.sent = kw, []
    def send_command(self, cmd):
        return ("Interface IP-Address OK? Method Status Protocol\n"
                "Vlan10 192.168.10.2 YES manual up up\n"
                "Gi1/0/2 unassigned YES unset administratively down down\n")
    def send_config_set(self, lines):
        self.sent.append(lines)
        return "ok"

class DeviceTest(unittest.TestCase):
```

```
    def setUp(self):        self.dev =
Device("acc1", "10.255.0.21",
connector=FakeConn)
self.dev.connect(username="u",
password="p")    def test_parse(self):
ifs = self.dev.get_interfaces()
self.assertEqual(ifs["Gi1/0/2"]
["status"], "administratively down")
self.assertEqual(ifs["Vlan10"]["ip"],
"192.168.10.2")    def
test_dry_run_sends_nothing(self):
self.dev.push_config(["vlan 40"])
self.assertEqual(self.dev._conn.sent,
[])    def
test_real_push_sends_once(self):
self.dev.push_config(["vlan 40"],
dry_run=False)
self.assertEqual(self.dev._conn.sent,
[["vlan 40"]])if __name__ ==
"__main__":    unittest.main()
```

Follow-ups. Context-manager support and a `disconnect` in `finally`; `unittest.mock.patch("netmiko.ConnectHandler")` is the alternative to injection. **Mistakes.** Importing the library at module top so that tests need it; defaulting `dry_run` to `False`.

Q10.27 How do you handle secrets in automation repositories and CI?

AMAZON · GOOGLE · MICROSOFT · TECH-2 · ■■■ · S

30-second answer: never in Git. Keep credentials in a secrets manager or in CI-encrypted secrets, inject them as environment variables at run time, scope them to the minimum, prefer short-lived tokens, scan commits for keys, and rotate at once if one leaks.

Answer. Layers: (i) `.gitignore` for `.env` and vault files; (ii) a pre-commit hook (`detect-secrets`) and a CI scan; (iii) Ansible Vault or HashiCorp Vault for stored values, with the vault password held by CI, not in the repository; (iv) a service account per pipeline with TACACS+ command authorisation limited to what the job needs, so that a leaked credential has a small reach (9.12 (p. 567)); (v) masked variables so that logs do not print them; (vi) a documented rotation. A secret that reached history is considered public: rotate first, rewrite history second. **Mistakes.** Putting the password in the inventory “for the lab”; echoing environment variables in CI logs.

Q10.28 Explain GitOps for networks and drift detection.

GOOGLE · AMAZON · CISCO · TECH-2 · ■■■ · T

30-second answer: Git holds the intended state; only the pipeline holds device credentials; every change is a pull request. A scheduled job compares the devices with what Git renders; the difference is drift, which is reported or reconciled.

Answer. The loop is: Git, pipeline, device, observation, back to Git. Drift arises from emergency CLI changes, failed pushes and software upgrades. The comparison renders the intended file and diffs it against `get_config` (the function in 10.13 (p. 612)), ignoring volatile lines. Policy decides the response: alert only, open a ticket, or re-apply the intended state at the next window. Reconciliation without review can overwrite a legitimate emergency fix, so many teams first raise a pull request that adopts or reverts the change. **Mistakes.** Auto-reconciling during an incident; comparing unnormalised text and drowning in false positives.

Q10.29 What is SDN? Where do OpenFlow and P4 fit, and what do hyperscalers run?

GOOGLE · BROADCOM · MICROSOFT · TECH-2 · ■■■ · T

30-second answer: SDN separates the control plane (decisions) from the data plane (forwarding) and exposes the network through software APIs. OpenFlow programs flow tables from a controller; P4 describes the forwarding pipeline itself. Hyperscalers mostly run BGP-based fabrics with central automation and telemetry.

Answer. OpenFlow (2008 onward) gave controllers a way to install match-action rules, but table sizes and scale limited it in general networks. P4 programs the parser and match-action stages of a programmable ASIC, for new headers and in-network telemetry. In production, Google’s WAN and data-centre work used custom controllers, and most hyperscaler data centres use eBGP underlays (5.4 (p. 310)) with automation, intent verification and gNMI telemetry, so “SDN” in practice is software-driven operation more than OpenFlow (5.7 (p. 318)). **Mistakes.** Equating SDN with OpenFlow.

Q10.30 A vendor has no API and a quirky CLI whose pager and prompts appear mid-output. Build a robust driver.

AMAZON · CISCO · JUNIPER · TECH-2 · ■■■ · T

30-second answer: write a small driver on Netmiko's base class around one function that sends a command and reads in a loop, answering the pager, accepting confirmations only from an allow-list, and returning only when the prompt regex matches at the end; give it a deadline, and test it with a fake channel.

Approach. Never `sleep` a fixed time. Read in a loop with a short idle step; on each pass look only at the *tail* of the buffer. If it shows a pager marker send a space and delete the marker; if it shows a confirmation, answer only when the command is in an allow-list (an automatic `y` to `reload` would be dangerous); if a prompt ends the buffer, strip the echoed command and return. Use a monotonic clock for the deadline and raise on timeout. Injecting `write`, `read`, `sleep` and `now` makes the function testable.

```
import re
import time

PROMPT = re.compile(r"[\w.\-]+(\
(config[^)]*\))?[#>]\s*$")
MORE = re.compile(r"--More--|<--- More -
-->")
CONFIRM = re.compile(r"\[confirm\]|\
(y/n\)\s*$", re.I)
SAFE_CONFIRM = {"clear counters", "copy
running-config startup-config"}

def run_command(write, read, cmd,
timeout=30.0, idle=0.2,
sleep=time.sleep, now=time.monotonic):
    write(cmd + "\n")
    buf, deadline = "", now() + timeout
    while now() < deadline:
        sleep(idle)
        buf += read()
        tail = buf[-120:]
        if MORE.search(tail):
            buf = MORE.sub("", buf)
            write(" ")
        elif CONFIRM.search(tail):
            if cmd not in SAFE_CONFIRM:
                raise
RuntimeError(f"unexpected confirmation
for {cmd!r}")
            write("y\n")
            buf += "\n"
        elif
PROMPT.search(buf.rstrip("\r\n").splitlines()
[-1] if buf.strip() else ""):
            lines = buf.splitlines()
            return "\n".join(l for l in
lines[1:-1])       # drop echoed command
and final prompt
    raise TimeoutError(f"no prompt after
{timeout}s for {cmd!r}: ...
{buf[-60:]!r}")

# --- test with a scripted fake channel
----------------------------------------
--------------
chunks = iter(["show log\r\nline 1\r\n--
More--", "line 2\r\n", "line
3\r\nsw1#"])
sent, clock = [], iter(range(100))
out = run_command(sent.append, lambda:
next(chunks, ""), "show log",
                  sleep=lambda s: None,
now=lambda: float(next(clock)))
assert sent == ["show log\n", " "], sent
assert out.splitlines() == ["line 1",
"line 2", "line 3"], out
```

Answer. In a Netmiko subclass of `CiscoSSHConnection`, call `run_command(self.write_channel, self.read_channel, cmd)` from the method that replaces `send_command`, and put `terminal length 0` (or the vendor's equivalent) in `session_preparation` so the pager rarely appears. Add bounded retries on `TimeoutError`

only for read-only commands. **Follow-ups.** Unknown prompts? Set the prompt regex per platform, learn the base prompt after login. **Mistakes.** `time.sleep(5)` after every command; auto-answering every confirmation; matching the prompt anywhere in the buffer, which a log line containing `#` will trigger.

Revision Spread

THE 15-MINUTE VERSION

Ten facts. (1) Automation moves intent from heads to scripts to templates and data to models in Git to a feedback loop (L0 to L4); the gain is review, replay and evidence. (2) JSON, YAML and XML carry the same tree; YAML for people, JSON for APIs, XML where namespaces are needed; always `safe_load` and validate. (3) Paramiko is raw SSH; Netmiko adds prompts, paging and config mode; `send_config_set` does not save. (4) Parse text only when no API exists: TextFSM, TTP, Genie, regex last; zero rows parsed is an error. (5) Jinja2 plus a validated YAML source of truth renders configs; `StrictUndefined`, `trim_blocks`. (6) NAPALM: getters, then `load_*_candidate`, `compare_config`, `commit_config`, `rollback`; replace deletes what is missing. (7) Ansible resource modules are idempotent; command modules are not; `--check --diff -l host`. (8) YANG defines the tree; NETCONF (SSH 830, XML, candidate and commit), RESTCONF (HTTPS, verbs) and gNMI (gRPC, streams) carry it. (9) `SAMPLE` for counters, `ON_CHANGE` for state; telemetry pushes, SNMP polls. (10) Git is the only way in; pipeline: lint, schema, test, Batfish, twin, approve, canary, post-check, rollback.

Three figures to redraw. The maturity ladder (Figure 10.1), the NETCONF layers (Figure 10.2) and the GitOps pipeline with its gates (Figure 10.4).

Which tool when?

Need	Choose	Why
One-off command on a Linux host, or SFTP	Paramiko	raw SSH, full control
Show or change on many CLI devices	Netmiko	prompts, paging, vendors handled
Same API across vendors, safe diff and rollback	NAPALM	getters and candidate workflow
Inventory plus Python logic, concurrency	Nornir	threads, filters, plug-ins
Operator-readable checklist, vendor modules	Ansible	declarative, idempotent resource modules
Atomic multi-step change, device supports it	NETCONF	candidate, validate, commit
Simple read or edit from any HTTP client	RESTCONF	verbs and JSON
Counters, state, events at scale	gNMI	streams, OpenConfig paths
Config text from data	Jinja2 + YAML/NetBox	review and reuse
Pre-change reachability and ACL analysis	Batfish	offline, whole-network
Behaviour test of a change	containerlab + pytest/pyATS	digital twin

NETCONF operations and datastores

Operation	Purpose	Notes
`get` / `get-config`	read state and config / config only	subtree or XPath filter
`edit-config`	change a datastore	`operation`: merge, replace, create, delete, remove
`copy-config`, `delete-config`	whole-datastore copy or delete	running to startup
`lock` / `unlock`	exclusive write access	pair them, release on error
`validate`	check without applying	needs `:validate`
`commit`	candidate to running	`confirmed` reverts on timeout

`close-session` / `kill-session`	end own or another session

Datastore	Meaning	Capability
`running`	active configuration	base
`candidate`	scratch copy, applied on commit	`:candidate`
`startup`	loaded at boot	`:startup`

Protocol comparison and ports

	NETCONF	RESTCONF	gNMI
Transport	SSH subsystem	HTTPS	gRPC, HTTP/2, TLS
Port	830	443	9339 (IANA); EOS 6030; SR Linux 57400
Encoding	XML	JSON / XML	protobuf, JSON_IETF
Atomic change	yes	no	`Set` per request
Streaming	notifications	SSE (limited)	`Subscribe`
RFC / spec	6241, 6242	8040	OpenConfig gNMI spec

RESTCONF verb	NETCONF
GET	get / get-config
POST	edit-config create
PUT	edit-config replace
PATCH	edit-config merge
DELETE	edit-config delete

URI: `https://HOST/restconf/data/MODULE:CONTAINER/LIST=KEY/LEAF`; `/` in a key is `%2F`; media type `application/yang-data+json`.

Code and command cards

Item	Form
Netmiko	`with ConnectHandler(device_type="cisco_ios", host=h, username=u, password=p) as c:` then `c.send_command("show ip int br", use_textfsm=True)`, `c.send_config_set([...])`, `c.save_config()`
`device_type`	`cisco_ios` (also IOS XE), `cisco_xr`, `cisco_nxos`, `arista_eos`, `juniper_junos`, `linux`
NAPALM	`get_network_driver("eos")(hostname=h, username=u, password=p)`; `open()`, getters, `load_merge_candidate(config=)`, `compare_config()`, `commit_config()`, `rollback()`, `close()`
Top getters	`get_facts`, `get_interfaces`, `get_interfaces_counters`, `get_interfaces_ip`, `get_lldp_neighbors`, `get_arp_table`, `get_mac_address_table`, `get_bgp_neighbors`, `get_route_to`, `get_config`
Ansible	`ansible-playbook -i inventory.yml site.yml --check --diff -l acc1`
Ansible states	`merged` add; `replaced` per resource; `overridden` whole feature; `deleted`; `gathered`; `rendered`; `parsed`
ncclient	`manager.connect(host=, port=830, username=, password=, hostkey_verify=True)`, `get_config`, `locked(target)`, `edit_config`, `validate`, `commit`, `discard_changes`
gnmic	`capabilities`, `get --path P`, `set --update-path P --update-value V`, `subscribe --path P --mode stream --stream-mode sample --sample-interval 10s`
Git	`clone`, `switch -c`, `add`, `commit`, `push`, `fetch`, `merge`, `revert`, `tag -a`
Rollback timers	IOS `reload in 10` / `reload cancel`; EOS `commit timer 00:10:00`; Junos `commit confirmed`; NETCONF `commit` with `confirmed`

YANG tree legend

Symbol	Meaning
`rw` / `ro`	configuration / state
`*`	list or leaf-list
`?`	optional
`[name]`	key of the list
`!`	presence container
`augment` / `deviation`	another module adds nodes / a vendor departs from the model

Safety checklist

from a vault; audit log with commit ID; refuse diffs that remove routes or management access.

Flashcards

Q	A
Ladder steps L0 to L4	manual, scripted, templated, declarative in Git, closed loop
Why not the running config as source of truth?	it records what is configured, not what was meant
`yaml.load` versus `safe_load`	the first can construct arbitrary objects; always use `safe_load`
YAML `no` unquoted becomes	boolean `false` (YAML 1.1); quote it
XML round trip returns booleans as	strings; XML carries no types
Paramiko `AutoAddPolicy` risk	accepts any host key: man-in-the-middle
`invoke_shell` versus `exec_command`	interactive channel versus one command per channel
Netmiko call that saves	`save_config()`
Why no retry on authentication failure	each retry is another bad login and may lock the account
Thread pool size for backups	small (about 8): capped by AAA and device control planes
TextFSM `Record`	writes the current variables as one row
`use_textfsm=True` needs	a matching ntc-templates entry for platform and command
Jinja2 setting for missing variable error	`undefined=StrictUndefined`
`trim_blocks`	removes the newline after a block tag
NAPALM merge versus replace	add lines versus make running equal to the file
Risk of replace	deletes what is missing, including management access
NAPALM five-step flow	load, compare, commit, verify, rollback
Ansible module that is not idempotent	`ios_command` (and `raw`); `ios_config` if lines mismatch the echo
`overridden` versus `replaced`	whole feature versus listed resource
Idempotence test for a playbook	second run shows `changed=0`
YANG `list` must have	a `key` (for configuration lists)
`config false` means	state data, read-only
IETF, OpenConfig, native models	standard basics, operator-led portable, complete but vendor-specific
NETCONF port and transport	830, SSH subsystem
What does `<hello>` carry?	capabilities and, from the server, a session ID
`delete` versus `remove`	fails if absent versus succeeds either way
Why a candidate datastore	atomic, validated, discardable multi-step change
RESTCONF PATCH equals	NETCONF merge
gNMI four RPCs	Capabilities, Get, Set, Subscribe
`ON_CHANGE` fits	oper-status, BGP state, LLDP
`SAMPLE` fits	counters at an interval
Dial-out means	the device connects to the collector
Fetch versus pull	download only versus download and merge
Secret committed to Git	rotate it; history keeps the old value
Batfish needs	configuration files only; no devices
Canary purpose	meet real hardware and traffic with one device first

PART III · PILLAR 11

11

Troubleshooting Methodology

Given a vague diagram and a one-line complaint, we ask, walk the layers aloud, name the proof, and close with fix, verification and prevention.

11.1 Why method beats memory	MUST
11.2 SCALPEL: the seven-step framework	MUST
11.3 Clarifying questions	MUST
11.4 Strategies: where to start and how to narrow	MUST
11.5 Layer command matrix and reading outputs	MUST
11.6 Symptom look-alikes	MUST
11.7 Incident overlay and post-mortems	SHOULD
11.8 The Case File: eighteen scenarios	MUST · cases 1 to 12
11.9 Mock rounds	SHOULD

This chapter integrates the book. The protocols behind each fault are taught in Pillar 3 (p. 162) and Pillar 4 (p. 229), and the tools (`tcpdump`, `ss`, `mtr`, `iperf3`) in Pillar 8 (p. 483); here we teach only the method, the diagnosis, and the commands each case needs, citing the home section whenever a mechanism is used. Every topology belongs to **Campus Net**, the fictional university network of Pillar 4, with its two border routers BR1 and BR2 and its providers ISP-A (AS 100) and ISP-B (AS 200). Candidate reports describe three recurring formats: a deliberately vague diagram in which the candidate must ask questions (Cisco), a spoken troubleshooting scenario scored on structure (Google), and a mixed Linux, BGP and OSPF discussion (Amazon network development engineer loops). We state these as observed patterns, not as official formats.

11.1 Why method beats memory MUST · ■□□

BIG IDEA

A troubleshooting interview scores the way you reduce uncertainty as much as the answer you reach, because an engineer who cannot show a repeatable method cannot be trusted with a production outage.

OPD doctor — a good doctor takes the history, examines, orders the cheapest decisive test, and only then prescribes. Nobody prescribes antibiotics for "I feel unwell".

Memory fails in a case round for a simple reason: the space of faults is larger than any list of them. The same symptom, "the user cannot browse", has a dozen causes at five layers, and the interviewer chooses the cause after hearing your first move. A candidate who recalls forty commands but has no order for them tests at random, while a candidate with a method tests the cheapest hypothesis that splits the possibilities in half. This is the behaviour the rubric below rewards.

Criterion	Earns the mark	Loses the mark
Clarification	Restates the symptom, asks three to five questions, states assumptions aloud	Types a command before any question
Structure	Names a method and the layer under test at each step	Jumps between layers without saying why
Technical depth	Right command, healthy output expected, correct inference	Lists commands without saying what they prove
Evidence	Each conclusion rests on a quoted output	"It is probably the firewall"
Communication	Thinks aloud in short sentences, summarises	Silence, or an unstructured monologue
Closure	Fix, verification, prevention, communication	Stops at the diagnosis

WORKED EXAMPLE

One prompt, two candidates. The prompt is "Users cannot access the web server."

Minute	Candidate A	Candidate B
0:00	"I would check the firewall, DNS, the web service, the switch ports and the routes."	"Let me restate: users get an error opening the site. Which users, which error, since when?"
1:30	Lists twelve commands, none tied to a hypothesis	Learns: one VLAN, timeout (not refusal), started after yesterday's change; states three hypotheses
4:00	Asked "what does that show?", cannot say what a healthy output looks like	Pings the gateway, then the server: "gateway answers, so L1 to L3 of this VLAN are intact; I test TCP 443 next"

9:00	Proposes restarting the web server	Finds the ACL line, fixes it, re-tests, proposes a template check
Score (0 to 4)	Clarification 0, structure 1, depth 2, evidence 1, communication 1, closure 0	All six criteria at 4

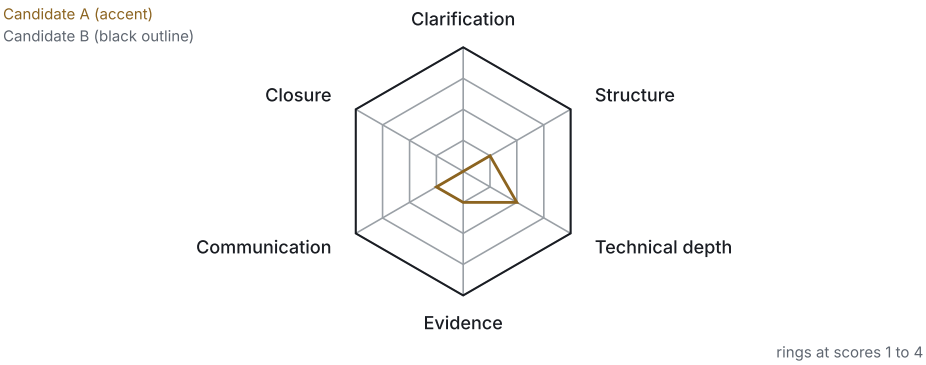


Figure 11.1 The two transcripts scored on the six criteria. Candidate B's area is the whole hexagon; A's is a small sliver near the centre.

TRAP

Claim: "Ping works, so the network is fine." *Why it is wrong:* ICMP proves only that some ICMP packet crossed the path; the application may use a different port, a different policy, or a different path, and a firewall may treat ICMP separately. *Correct statement:* ping proves reachability for ICMP between those two addresses, nothing more.

INTERVIEW LENS

CISCO · GOOGLE · AMAZON

The question is usually a single sentence ("a user cannot reach the internet"). A strong answer restates it, asks before it acts, names the layer under test, and says what output would change its mind. Likely follow-ups: "what if that command shows the opposite?" and "how would you prevent it?"

CHECK YOURSELF

1. Name the first thing a strong candidate does after hearing a one-line complaint. [1]
2. Why is "restart it" a weak answer even when it works? [2]

11.2 SCALPEL: the seven-step framework MUST · ■□□

BIG IDEA

SCALPEL is a fixed order, Symptom, Clarify, Assess, Layer walk, Pinpoint, Execute, Lock in, because the cheap checks that remove the most uncertainty must come before the expensive ones.

The order mirrors the surgical safety checklist: the knife is used only after the checks. Each letter has a spoken form, which is what the interviewer hears and scores.

Figure 11.2 The SCALPEL card. The accent loop is the rule to remember: when a hypothesis fails or explains only part of the evidence, return to Assess, not to a random command.

Step	Say aloud	Do not
Symptom	"So PC-A gets a timeout opening the intranet; let me confirm what 'cannot reach' means."	Accept "slow", "down" or "broken" unmeasured
Clarify	"Three questions first: how many users, since when, what changed?"	Ask more than six; the first three carry most information
Assess	"Given one VLAN and a change yesterday, I suspect trunk list, SVI or DHCP relay, in that order."	Hold an unranked list of ten
Layer walk	"I test the gateway first. A healthy reply shows …; a timeout would mean …"	Run a command without saying what healthy looks like
Pinpoint	"Root cause: VLAN 20 missing from the trunk; this explains why only SW2 hosts fail."	Stop at the first finding without checking it explains everything
Execute	"Add the VLAN with `add`, not by replacing the list; rollback is removing it."	Fix without a rollback, or fix first and ask later
Lock in	"Same ping now passes; I would template the trunk and alert on VLAN drift."	End at "it works now"

The time budget below suits a twenty-minute round. Most of the time belongs to the layer walk, so the first three minutes have to be spent on questions and not on commands.

Figure 11.3 Time budget for a twenty-minute round, in minutes. The letters follow Figure 11.2.

WORKED EXAMPLE

SCALPEL narrated on Case 1. The prompt is "A user says the internet is down." The case itself, with its commands, is in Case 1 (p. 649); this transcript shows pacing and phrasing.

Time	Step	Spoken

0:00	S	"So one user cannot get to websites. Before I touch anything: does 'internet down' mean every site, every protocol?"
0:30	C	"Is it one user or many? What error text? Does a ping to a public IP work? Did anything change?"
1:45	A	"Whole VLAN 10, a name-resolution error, the IP ping works, a DHCP scope was edited yesterday. I rank: wrong DNS from DHCP, DNS server down, port 53 filtered."
2:30	L	"The IP ping succeeded, so L1 to L3 and NAT are proven; I skip them and say so. I test DNS: which server does the client use?"
6:00	P	"The client points at 192.168.10.53, which does not exist; the real server answers. Wrong option 6 in the VLAN 10 scope explains why only VLAN 10 fails."
7:00	E	"Correct the scope, renew leases; users can flush the cache meanwhile."
8:00	L	"Verify a name lookup from two clients; add a post-change test: resolve a name from every scope."

WORKED EXAMPLE

Recovering when a hypothesis fails. The same Case 1 prompt, but the interviewer answers differently: the client's DNS server *is* correct.

Time	Step	Spoken
3:00	L	"The client uses 192.168.50.53, which is right, so hypothesis one is ruled out. I return to Assess."
3:30	A	"Next: the DNS server is down, or port 53 is filtered from VLAN 10. One test separates them: query the server from another VLAN."
4:30	L	"It answers from VLAN 20, so the server is up. I now read the ACL hit counts for UDP 53 on the VLAN 10 interface."
6:00	P	"A deny line for port 53 has rising matches; it was added with the scope change. That explains why only VLAN 10 fails."

INTERVIEW LENS

CISCO · GOOGLE · AMAZON

Interviewers often interrupt to test the loop. When told "that command shows the opposite", answer "that rules out …, so I return to Assess and the next hypothesis is …". Candidates who show the loop are scored higher than those who guess correctly the first time.

CHECK YOURSELF

1. Which SCALPEL step is repeated when a hypothesis fails? [3]
2. Why does Pinpoint require explaining the unaffected cases too? [4]

11.3 Clarifying questions MUST · ■□□

BIG IDEA

The first three minutes of questions decide the next fifteen, because each answer removes whole layers or locations from the search.

Railway enquiry — "Which train, which date, which class?" comes before "Is there a seat?".

Five families cover every complaint: *scope* (who and what is affected), *timeline and change* (since when, what changed), *symptom precision* (the exact error), *environment* (the devices and policies on the path), and *evidence* (what can be observed). The card below holds twenty-five questions; in an interview one asks the three to five highest-yield ones, which are usually numbers 1, 6, 8, 12 and 14.

Family	Questions
Scope	(1) How many users, which VLAN, site or building? (2) One application or all? (3) One direction or both? (4) Wired and wireless alike? (5) Who is unaffected, and what do they share?
Timeline and change	(6) Since when, and did it ever work? (7) Constant, periodic or random? (8) What changed: configuration, cable, upgrade, new device, certificate, policy? (9) Anything nearby: desk move, maintenance, provider work? (10) Any pattern in time: hourly, busy hours, backups?
Symptom precision	(11) Exact error text or code? (12) Does ping work, from which source to which destination? (13) By name or by IP? (14) Timeout, refusal, reset or slowness, and measured how? (15) Which port and protocol?
Environment	(16) Which devices are on the path, and can I log in to them? (17) IP plan, VLANs, routing protocol? (18) Firewalls, NAT, VPN, load balancers? (19) Redundancy: second link, ECMP, LAG, first-hop redundancy? (20) Vendors and software versions?
Evidence	(21) Which logs, counters and graphs exist? (22) Commands at both ends? (23) Any packet capture or flow record? (24) A known-good comparison: another user, site, or last week? (25) What has been tried already?

When the interviewer declines to answer, state an assumption instead and continue: “I will assume a single OSPF area and no firewall on the path; tell me if that is wrong.” Stated assumptions are scored positively, whereas silent assumptions are scored as guessing.

WORKED EXAMPLE

Turning a vague diagram into a scenario. The Cisco-style prompt gives two LANs, two routers, a switch each and a cloud labelled WAN, with the words “PC-A cannot reach PC-B”. Eight questions define it.

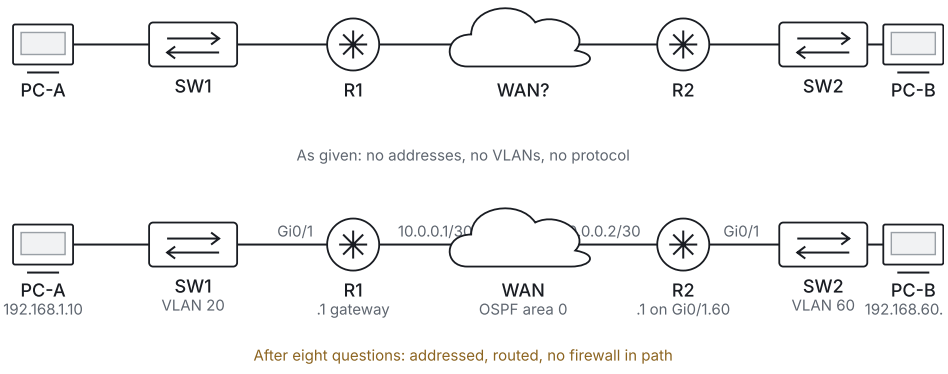


Figure 11.4 The vague diagram as given (top) and after eight questions (bottom). The addresses are as designed; what PC-B actually has is not yet known.

Candidate asks	Interviewer answers
Addresses and gateways?	PC-A 192.168.1.10/24, gateway R1 .1; PC-B should be 192.168.60.20/24, gateway R2 .1
What exactly fails?	Ping from PC-A to PC-B times out; nothing else tested
Does PC-A reach anything behind R2?	Yes, the file server 192.168.50.5 in VLAN 50
Did it ever work?	PC-B was connected for the first time this morning, in the new VLAN 60

How are R1 and R2 joined, and which protocol?	Provider Layer 2 circuit, 10.0.0.0/30, OSPF area 0; no firewall
What changed?	SW2 and R2 were configured for VLAN 60 yesterday
How does R2 reach SW2?	One trunk, one sub-interface per VLAN
May I see outputs on SW2 and R2?	Yes, on request

Result: the WAN is proven by the file server; the search narrows to PC-B, SW2 and R2's VLAN 60 sub-interface. Mock round 1 (p. 672) **continues from here.**

INTERVIEW LENS

CISCO · JUNIPER · GOOGLE

For a vague diagram, candidates report being scored mainly on whether they define the scenario before proposing anything. A strong answer ends the clarifying phase with a one-sentence summary: "so the WAN works, PC-B is new, and the fault is local to VLAN 60 on SW2 or R2".

CHECK YOURSELF

1. Which two questions in the table separate "the network" from "one device"? [5]
2. What do you do if the interviewer says "assume anything"? [6]

11.4 Strategies: where to start and how to narrow MUST · ■□□

BIG IDEA

The best starting point is the one that discards the largest part of the search space with one test, so the strategy depends on what is already known about the fault.

Six strategies are in common use. None is correct in general, so the skill lies in choosing, and in saying why.

Diwali lights — to find one dead bulb in a long string, test the middle first and discard half; the same binary search works on a path of routers.

Strategy	Start	Use when	Risk
Bottom-up	Cable, port, link state (L1), then upward	A physical change, a new link, intermittent loss	Slow when the fault is in the application
Top-down	Application error, then downward	One application fails and others work	Misses a path problem that hits only that port
Divide and conquer	`ping` the gateway or the middle hop (L3), then go up or down	Many hops, no other clue; the default	Needs access to several devices
Follow the packet	`traceroute`, then each device's forwarding decision along the way	A multi-hop path, routing suspected	Forward path may differ from the return path
Compare with working	Diff the failing and the working user, site or configuration	One of two similar things fails	A shared cause is missed
Swap or isolate	Replace the cable, optic or port with a known-good one	Hardware suspected after counters or light levels point to it	Destroys evidence if done first

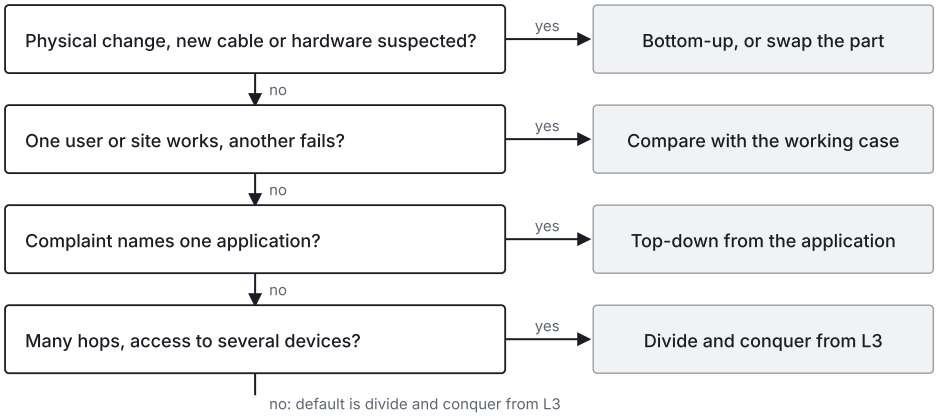


Figure 11.5 Strategy chooser. The default is divide and conquer from Layer 3.

WORKED EXAMPLE

The same fault with four strategies. Campus Net has a six-device path: PC, SW1, R1, R2, R3 and a server. The link between R2 and R3 is administratively down because an engineer shut the wrong interface. The count is the number of tests before the faulty link is isolated.

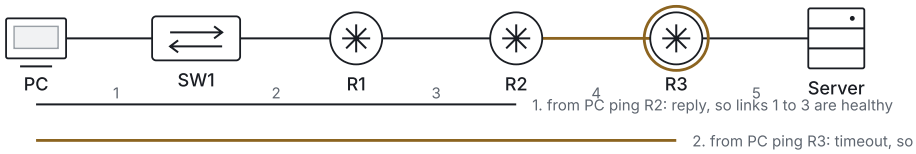


Figure 11.6 Binary search on the path. Two pings from the PC isolate link 4; one `show interface` on R2 then confirms the cause.

Strategy	Sequence	Tests
Bottom-up	PC port, SW1 port, R1 interface, link R1 to R2, link R2 to R3 (down)	5
Top-down	Browser timeout; `curl` by IP shows it is not DNS; `traceroute` stops after R2; `show interface` on R2	4
Divide and conquer	`ping R2` works; `ping R3` fails; `show interface` on R2 confirms	2 plus 1
Follow the packet	`traceroute` shows last hop R2; inspect R2's route and egress interface	2

Result: with n segments, the binary search needs about log2 n tests (here two for five links), whereas the sequential strategies need up to n.

TRAP

Claim: "I always start at Layer 1." *Why it is wrong:* the physical layer is cheap to check only on the local device; on a remote path one `ping` at the middle already tests Layers 1 to 3 together and halves the search. *Correct statement:* use bottom-up after a physical change, and divide and conquer otherwise.

CHECK YOURSELF

1. A single laptop fails while its neighbour on the same switch works. Which strategy is natural? [7]

> 2. Why is a swap done after the counters are saved? [8]

11.5 Layer command matrix and reading outputs MUST · ■■□

> BIG IDEA
>
> A command is only evidence once you know what a healthy output looks like, because the same number can be normal on one link and a fault on another.

The matrix lists, for each layer, the question to settle and the two or three commands that settle it on Cisco IOS XE, Junos and Linux. The mechanisms behind each command are in their home chapters (switching (p. 233), OSPF (p. 246), BGP (p. 256)), and the Linux tools are taught in Pillar 8 (p. 483). Arista EOS and NX-OS accept the IOS forms used here with small differences.

Layer	Cisco IOS XE	Junos	Linux
L1 Link up, clean, speed and duplex	`show interfaces`, `show interfaces status`, `show interfaces transceiver`	`show interfaces extensive`, `show interfaces diagnostics optics`	`ethtool eth0`, `ethtool -S eth0`, `ip -s link`
L2 MAC, VLAN, trunk, loop	`show mac address-table`, `show vlan brief`, `show interfaces trunk`, `show spanning-tree`, `show etherchannel summary`	`show ethernet-switching table`, `show vlans`, `show spanning-tree bridge`, `show lacp interfaces`	`ip neigh`, `bridge fdb show`, `bridge vlan`, `ip -d link`
L3 Address, neighbour, route, path	`show ip interface brief`, `show ip arp`, `show ip route X`, `show ip ospf neighbor`, `show ip bgp summary`, `ping X source Y`, `traceroute X`	`show interfaces terse`, `show arp`, `show route X`, `show ospf neighbor`, `show bgp summary`, `ping X source Y`	`ip addr`, `ip route get X`, `ip neigh`, `ping -I eth0 X`, `traceroute -n X`, `mtr -rwzbc 100 X`
L4 Port, policy, state	`show access-lists`, `show ip nat translations`, `show policy-map control-plane`	`show firewall filter`, `show security flow session`	`ss -tulpn`, `ss -ti`, `conntrack -L`, `nft list ruleset`, `nc -zv X 443`
L7 Name, handshake, service	`show ip dhcp binding`, `show ip dhcp snooping`	`show dhcp server binding`	`dig X`, `curl -v URL`, `openssl s_client -connect X:443`, `journalctl -u svc`
Any On the wire	`monitor capture` (embedded packet capture)	`monitor traffic interface ge-0/0/0`	`tcpdump -ni eth0 'host X and port Y'`

Reading interface counters

Counters on an interface are cumulative since the last clear, so a figure is meaningful only as a *delta*: take two snapshots ten seconds apart and subtract. The pair below contrasts a healthy port with a port on a bad cable; the lines to read are the speed and duplex, and the error counters.

1. Restate the symptom as something measurable and ask clarifying questions, before any command.↩
2. It gives no root cause, so the fault recurs, and it hides the evidence needed to prevent it.↩
3. Assess: the failed result is evidence, so the hypotheses are re-ranked.↩
4. A cause that explains only the failing cases may be a coincidence; one that also explains why others work is much more likely true.↩
5. "Does PC-A reach anything behind R2?" and "Did it ever work?" prove the WAN and the age of the fault.↩
6. State the assumptions aloud, in a list of two or three, then proceed and revisit them if evidence contradicts one.↩
7. Compare with working: diff the laptop's address, VLAN, port configuration and DHCP lease against its neighbour's.↩
8. Replacing the part clears the counters and the fault, so the evidence for the root cause would be lost.↩

Blood report — the reference range matters more than the raw number; a CRC count of 40 means nothing until you know whether it was 40 a minute ago.

```
Healthy
Gi1/0/10 is up, line protocol is up
  Full-duplex, 1000Mb/s
  0 input errors, 0 CRC, 0 frame
  0 output errors, 0 collisions

Faulty (cable or optic)
Gi1/0/10 is up, line protocol is up
  Full-duplex, 1000Mb/s
  2217 input errors, 2217 CRC, 0 frame
  0 output errors, 0 collisions
```

Counter	Reads as	Usual cause
CRC, input errors rising	Frames corrupted on the wire	Bad cable, dirty or weak optic, duplex mismatch (on the full-duplex side)
Runts	Frames under 64 bytes	Collisions, duplex mismatch
Giants	Frames above the interface MTU	MTU or jumbo mismatch, tag overhead
Late collisions	Collision after the 64th byte	Half-duplex port facing a full-duplex peer; cable too long
Output drops	Egress queue overflowed	Congestion, microbursts
Resets, carrier transitions	Link flapped	Cable, optic, remote reboot

Reading trunks, neighbours and sessions

On a trunk, `show interfaces trunk` prints three lists, and the *difference* between them is the diagnosis: a VLAN that is allowed but not active is missing from the VLAN database, and one that is active but not forwarding is blocked by spanning tree or pruning.

```
Vlans allowed on trunk              : 1,10,20          healthy, VLAN 20 present
Vlans allowed and active            : 1,10,20
Vlans in spanning tree forwarding   : 1,10,20

Vlans allowed on trunk              : 10               faulty: list was replaced
```

Two more pairs are worth memorising, since they are the commonest outputs shown in case rounds. The OSPF pair shows the state and the dead timer; the BGP pair shows that the last column is a *state* while the session is down and a *prefix count* once it is up.

```
R1# show ip ospf neighbor                   healthy
Neighbor ID  Pri  State      Dead Time  Address    Interface
2.2.2.2        1  FULL/DR    00:00:36   10.0.12.2  Gi0/2

R1# show ip ospf neighbor                   stuck
2.2.2.2        1  EXSTART/DR 00:00:34   10.0.12.2  Gi0/2

R1# show ip bgp summary                     healthy
Neighbor     V   AS  MsgRcvd MsgSent  Up/Down   State/PfxRcd
192.0.2.1    4  100     5120    4871  3d02h              812

R1# show ip bgp summary                     down
192.0.2.1    4  100        0       0  never          Active
```

The routing protocols have state machines, taught in §4.5 (p. 246) and §4.7 (p. 256); what matters when troubleshooting is *where a session sticks*, since each stuck state points to a different class of fault.

Protocol state	Stuck here means	First test
OSPF Down or Init	Hellos do not arrive, or are one-way (ACL, multicast, passive interface)	`show ip ospf interface`, `debug ip ospf hello`
OSPF 2-Way	Normal between DROthers on a broadcast segment	none needed
OSPF ExStart or Exchange	DBD exchange fails, usually an MTU mismatch	`show interfaces`, MTU line, on both ends
OSPF Loading	LSRs not answered, packets lost	Check MTU, authentication, loss
BGP Idle	Administratively down, or no route to the peer	`show ip bgp neighbors`, `show ip route peer`
BGP Active	TCP connection attempts fail (not "active" in the usual sense)	Ping and `telnet peer 179` from the same source; ACL
BGP OpenSent	Open sent, no valid reply: AS or parameters mismatch	Logs: NOTIFICATION
BGP Established with a number	The number is the prefixes received; the session is healthy	`show ip bgp neighbors X advertised-routes`

Reading Linux evidence

On a Linux host, three outputs carry most of the diagnosis. The pair for `ss -ti` shows a healthy long-haul connection and the window-limited one of Case 17 (p. 670); the `ping` pair shows a path-MTU black hole (Case 5 (p. 654)); the last block shows `ip -s link`, where errors and drops are deltas to be read twice.

```
ss -ti   healthy
 cubic wscale:7,7 rto:284 rtt:81/4 cwnd:640 bytes_retrans:0
ss -ti   window-limited, retransmitting
 cubic rto:284 rtt:81/4 cwnd:10 bytes_retrans:182520 retrans:0/127

ping -M do -s 1372 10.10.5.20    healthy at the path MTU
1380 bytes from 10.10.5.20: icmp_seq=1 ttl=62 time=21.4 ms
ping -M do -s 1472 10.10.5.20    black hole: no reply and no error
--- 10.10.5.20 ping statistics --- 3 packets transmitted, 0 received

ip -s link show eth0
    RX: bytes packets errors dropped  missed  mcast
    8812770112 6120331  2217      40       0   1203
```

Reading ping, traceroute and Linux socket output

Symbol or text	Meaning
IOS ping `!`	Reply received
IOS ping `.`	Timeout: dropped, or no return route
IOS ping `U`	ICMP unreachable received from a router on the path
IOS ping `M`	Packet needed fragmentation and DF was set
traceroute `*`	No reply for that probe (filtered, or rate-limited); not a fault by itself

traceroute `!H`, `!N`, `!A`	Host, network unreachable, or administratively prohibited
`ss -ti` field `retrans`	Retransmitted segments on that connection
`ss -ti` field `wscale`	Window-scale factors negotiated; absent means a 64 KB window limit
`ip -s link` `dropped`, `errors`	Interface-level loss, as deltas

TRAP

Claim: "The counter shows 3,000 errors, so the link is bad." *Why it is wrong:* the counter is cumulative and may have grown a year ago; and `mtr` loss at a middle hop is often a router rate-limiting its own ICMP replies. *Correct statement:* take deltas, and count only loss that persists to the destination.

TRAP

Claim: "`debug all` will show me the problem." *Why it is wrong:* on a busy router it can exhaust the CPU and cut the session. *Correct statement:* use a targeted debug (for example `debug ip ospf adj`) with a condition, or a filtered capture, inside a change window, and switch it off with `undebug all`.

CHECK YOURSELF

1. Which command shows whether a VLAN is blocked by spanning tree on a trunk? [1]
2. What does an OSPF neighbour in 2-Way on a broadcast segment mean? [2]

11.6 Symptom look-alikes MUST · ■■□

BIG IDEA

Precise words for the symptom point at different faults, because a timeout, a refusal and a reset are produced by different devices behaving in different ways.

Interviewers often supply a loose word and wait to see whether the candidate sharpens it. The table fixes the vocabulary; the three ladders show what each of the first three looks like in a capture.

Phone calls — "nobody picked up" is a timeout, "the number does not exist" is a refusal, and "the call dropped" is a reset.

Observed	On the wire	Means	First test
Timeout	SYN retransmitted, nothing returns	Packet dropped on the path: ACL, firewall DROP, no route, dead link	Capture at both ends; see where it stops
Connection refused	SYN answered by RST	Host reached; nothing listens, or a firewall REJECTs	`ss -tlnp` on the server
Reset mid-stream	Established, then RST	Idle timer in a middlebox, application crash, lost state	Who sent the RST (TTL, MAC)
ICMP unreachable	ICMP type 3, codes 0, 1, 3, 13	No route, no ARP answer, UDP port closed, filtered	Read the code and the sender
Loss	Gaps, retransmissions	Errors, congestion, rate limiting	`mtr` to the destination, counters as deltas
Latency	Constant high RTT	Distance, queueing, a detour	`ping`, `mtr` per hop
Jitter	RTT varies	Queueing, ECMP members of unequal quality, wireless	`ping -i 0.2 -c 100`
Low throughput	Window or loss limited	Small window, loss, shaping	`iperf3`, `ss -ti`

By IP only	Name lookups fail	DNS server, DHCP option, filter on port 53	`dig @server name`
One direction only	Request seen, reply absent	Asymmetric route, ACL, NAT, uRPF	Captures at both ends
Intermittent	Pattern in time	Duplicate IP, flapping link, one bad ECMP member, lease expiry	Correlate with logs

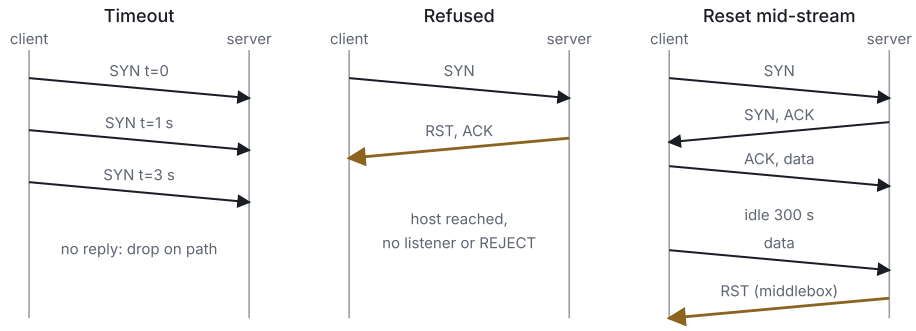


Figure 11.7 Three ladders. In a timeout the sender hears nothing; in a refusal the server answers at once with a reset; in a mid-stream reset a device that lost the state answers later data with a reset.

```
# 1 Timeout: SYN retransmitted at 1 s and 3 s, never answered
10:01:00.000 IP 10.0.1.5.51312 > 10.0.2.10.8080: Flags [S], seq 1001
10:01:01.003 IP 10.0.1.5.51312 > 10.0.2.10.8080: Flags [S], seq 1001
10:01:03.011 IP 10.0.1.5.51312 > 10.0.2.10.8080: Flags [S], seq 1001
# 2 Refused: reset within a millisecond
10:02:00.000 IP 10.0.1.5.51320 > 10.0.2.10.8080: Flags [S], seq 2001
10:02:00.001 IP 10.0.2.10.8080 > 10.0.1.5.51320: Flags [R.], ack 2002
# 3 Reset after 300 s of silence: a middlebox forgot the session
10:08:01.220 IP 10.0.2.10.443 > 10.0.1.5.51400: Flags [R.], seq 5102
```

The excerpts support three inferences. In the first, the server never sees the SYN or its answer never returns; moving the capture to the server's interface decides which. In the second, the server host is alive and reachable, and the question moves to the listener or a REJECT rule. In the third, the interval of exactly five minutes is the clue: it matches an idle timer, and keepalives shorter than that timer remove the symptom.

The table below pairs the commonest first sentences of a complaint with their first suspect and the case in this chapter that works it through.

Symptom sentence	First suspect	Case
Works by IP, not by name	DNS server or DHCP option 6	1 (p. 649)
Only large transfers hang	MTU, blocked ICMP	5 (p. 654)
One direction only, random sessions	Asymmetric path through stateful firewalls	13 (p. 664)
Blips every few minutes	Duplicate IP, flapping	10 (p. 661)
Bulk transfer slow, interactive fine	Duplex mismatch, loss	4 (p. 653)
Slow for some users, always the same ones	One ECMP or LAG member	14 (p. 666)

Neighbour stuck in ExStart	MTU mismatch	6 (p. 656)
BGP stuck in Active	TCP 179 cannot open	8 (p. 658)
Tunnel up, no traffic	Selectors, NAT exemption	15 (p. 667)
Refused then timeout	Listener, then filters	16 (p. 669)
Loss at one traceroute hop only	Control-plane rate limit	18 (p. 671)

CHECK YOURSELF

1. A TCP SYN is answered by an immediate RST. Is the firewall dropping it? [3]
2. Why do reset-after-idle failures appear at exactly the same interval? [4]

11.7 Incident overlay and post-mortems SHOULD · ■■□

BIG IDEA

In a live outage, restoring service comes before finding the cause, because every minute of diagnosis with users down is a minute of avoidable impact.

When the prompt says “production is down”, the order changes from SCALPEL’s: *mitigate*, *communicate*, *diagnose*, *fix*, *review*. Mitigation is any action that restores service without needing the root cause: roll back the last change, fail over to the standby, drain the faulty member, or disable the new feature. If a change was made ten minutes before the outage, rolling it back is faster and safer than reasoning about it, and the evidence is still available afterwards from logs and saved outputs.

Fire brigade — put out the fire first, then look for the short circuit.

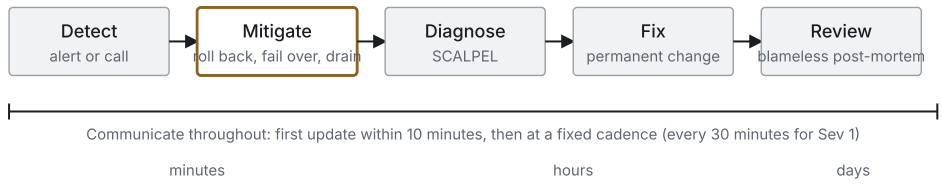


Figure 11.8 The incident overlay. Mitigation is the accent step; communication runs in parallel with every phase.

Mitigation	Use when	Evidence to save first
Roll back the change	An outage follows a change within minutes	Configuration, logs and counters before the rollback
Fail over to the standby	A redundant path or device is healthy	State of both devices
Drain or shut the faulty member	One link, optic or node harms a share of flows	Counters of the member
Disable the new feature	A feature, not the platform, is implicated	The feature’s configuration
Rate-limit or shed load	Overload, not a fault	Utilisation graphs

Severity sets how often stakeholders are updated and who is woken.

Severity	Definition	Example	Update cadence

Sev 1	Widespread loss of a critical service	Campus core down; every VLAN affected	Every 30 minutes
Sev 2	Major degradation or one site down	A floor lost, ISP-A down with ISP-B carrying	Every hour
Sev 3	Limited impact with a workaround	One VLAN slow	Daily
Sev 4	Cosmetic or no user impact	Alarm threshold needs tuning	At closure

Four roles are named at once in a serious incident: an *incident commander* who decides and does not type commands, an *operator* who runs them, a *scribe* who keeps the timeline, and a *communications lead* who updates stakeholders. In an interview, naming these roles scores as maturity; saying "I would do everything myself" does not.

WORKED EXAMPLE

Three incident messages. A good message states impact, action, and the time of the next update, and nothing speculative. The wording below is for the floor outage of Case 11.

Message	Text
First update (10 minutes)	"11:12. The third floor has no network. About 180 users are affected. We suspect a Layer 2 loop and are isolating ports. Next update at 11:40."
Progress update	"11:30. The suspected ports are shut and traffic on the floor is recovering. We are verifying each VLAN. Next update at 12:00."
Resolution	"11:48. Service is restored since 11:31. The cause was a cable loop on edge ports. A review will be circulated within three working days."

WORKED EXAMPLE

A blameless post-mortem for Case 11. The template has seven parts; the filled version is below, abridged.

Part	Content
Summary	11:05 to 11:31, the third floor (about 180 users) lost network.
Timeline	11:05 loop forms after a desk move; 11:07 helpdesk calls; 11:09 broadcast alarm on SW3A; 11:12 incident declared Sev 1; 11:18 MAC-flap logs name Gi1/0/14 and Gi1/0/15; 11:20 both ports shut (mitigation); 11:31 all services normal.
Impact	About 26 minutes of no network on one floor; no data loss.
Root cause	A physical loop on edge ports where BPDU filter hid the loop from spanning tree.
Contributing factors	The access-port template had BPDU filter but no BPDU Guard or storm control; no alert on broadcast rate; desk moves without a ticket.
What went well	The MAC-flap log named the ports at once; the shut-down was quick.
Action items	Template change (network engineering, two weeks); storm control and broadcast alert (monitoring, one week); disable unused wall ports (facilities, one month).

The post-mortem names *systems*, not people. "A user plugged a cable in" is a trigger; the root cause is that the design allowed one cable to take down a floor, and that is what the action items fix. The same structure answers behavioural questions: the timeline is the situation, the mitigation your action, and the action items the result, which is how the incident stories of Pillar 12 (p. 687) are built for Amazon's Ownership, Dive Deep and Bias for Action principles.

TRAP

Claim: "The root cause was human error." *Why it is wrong:* people make errors at a steady rate, so a cause that stops at a person guarantees a repeat. *Correct statement:* the root cause is the missing guard that let an ordinary error become an outage.

CHECK YOURSELF

1. A change was made ten minutes before a Sev 1 outage. What is the first action? [5]
2. Which role does the incident commander not perform? [6]

11.8 The Case File: eighteen scenarios MUST · ■■□

Each case follows the same layout so that it can be rehearsed aloud: the prompt as the interviewer says it, a topology (the accent marks where the fault lies, so cover it first and attempt the case), the clarifying questions with the answers the interviewer gives, the ranked hypotheses, a layer walk with the command and the *decisive output line*, the root cause, and then fix, verification, prevention and likely follow-ups. Cases 1 to 12 are the core; 13 to 16 are for strong candidates; 17 and 18 are optional. The tags after the title give the layers involved and the companies whose candidates report such scenarios.

No.	The complaint	Layers	Level
1 (p. 649)	Internet works by IP, not by name	L3, L7	Must
2 (p. 651)	Same VLAN, different switches, cannot talk	L2	Must
3 (p. 652)	Inter-VLAN routing broken for one VLAN	L2, L3	Must
4 (p. 653)	Slow file transfers after a cable swap	L1, L2	Must
5 (p. 654)	SSH works, big transfers over the VPN hang	L3, L4	Must
6 (p. 656)	OSPF neighbours stuck in ExStart	L3	Must
7 (p. 657)	OSPF neighbour never appears	L3	Must
8 (p. 658)	BGP session stuck in Active	L3, L4	Must
9 (p. 659)	BGP up, but a prefix is not advertised or installed	L3	Must
10 (p. 661)	One server drops for a minute, several times an hour	L2, L3	Must
11 (p. 662)	A whole floor went down when someone plugged in a switch	L2	Must
12 (p. 663)	New VLAN's clients get 169.254 addresses	L2, L3, L7	Must
13 (p. 664)	Traffic works one way through the firewalls	L3, L4	Should
14 (p. 666)	Website slow for some users only	L1 to L3	Should

1. `show interfaces trunk`: compare "allowed and active" with "in spanning tree forwarding state".↩
2. It is normal for two DROthers, which do not become fully adjacent with each other; only the DR and BDR form Full adjacencies.↩
3. No. A drop produces silence; a RST means a host or a REJECT rule answered, so the path is intact.↩
4. A stateful device removes idle sessions after a fixed timer, and the next packet is reset.↩
5. Roll the change back, after saving the current state and logs for the post-mortem.↩
6. Typing commands; the commander decides and coordinates while the operator acts.↩

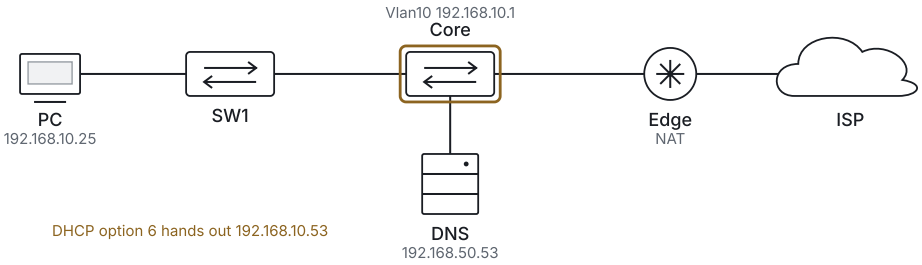


Figure 11.9 Case 1. The VLAN 10 DHCP scope hands out a DNS address in the wrong subnet.

We ask	The interviewer answers
Everyone or one user?	Every user in VLAN 10; other VLANs are fine
What exactly fails?	Browser: “server not found” (`DNS_PROBE_FINISHED_NXDOMAIN`)
Does `ping 8.8.8.8` work?	Yes
Any change?	The VLAN 10 DHCP scope was edited yesterday

Hypotheses, ranked. (1) DHCP hands out a wrong DNS server; (2) the DNS server is down; (3) an ACL blocks port 53 from VLAN 10.

Layer	Command	Decisive output	Inference
L1 to L3	`ping 8.8.8.8`	`Reply from 8.8.8.8: bytes=32 time=18ms TTL=113`	Link, gateway, NAT and ISP work; lower layers are skipped, and we say why
L7	`ipconfig /all`	`DNS Servers . . . : 192.168.10.53`	The client uses an address in its own subnet where no server lives
L7	`nslookup google.com 192.168.50.53`	`Name: google.com`	The real DNS server is healthy
L7	`dig @192.168.10.53 google.com`	`;; connection timed out; no servers could be reached`	The handed-out address is dead
L7	`show running-config`, section dhcp	`dns-server 192.168.10.53`	The scope holds the typo

Root cause. DHCP option 6 in the VLAN 10 scope was typed as 192.168.10.53 instead of 192.168.50.53. This explains why only VLAN 10 fails and why raw IP connectivity is intact.

Fix. `ip dhcp pool VLAN10`, then `dns-server 192.168.50.53`; clients renew with `ipconfig /renew`. As an interim step, users may enter the correct DNS server by hand.

Verify. `nslookup` succeeds from two clients and the browser loads a page.

Prevent. Generate scopes from the address plan, and add a post-change test that resolves a name from a client in every scope. DHCP and DNS are taught in §3.6 (p. 183) and §3.10 (p. 203).

The interviewer asks	A strong answer
If only some names fail?	Suspect split-horizon DNS, a conditional forwarder, or negative caching (the SOA minimum TTL).
Why does a ping to a public IP prove so much?	It crosses the local link, the VLAN, the gateway, the NAT and the ISP; only DNS and the layers above remain.

What if the DNS server itself were down?	`dig @192.168.50.53` would time out as well, and the fix would be there, not in the scope.

Variant. A wireless client reports “connected, no internet”: a captive portal or a DHCP fault on the SSID (Pillar 6 (p. 358)).

Case 2 · Same VLAN, different switches, cannot talk MUST · ■□□

L2 · CISCO · JUNIPER · ARISTA · NOKIA

Prompt. “PC-A and PC-B are both in VLAN 20 but cannot ping each other.”

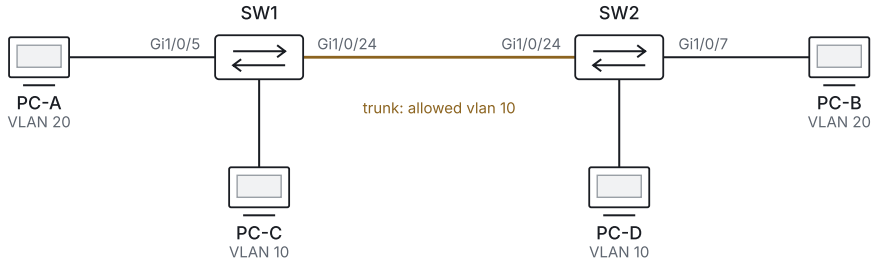


Figure 11.10 Case 2. VLAN 10 hosts cross the trunk; VLAN 20 hosts do not.

We ask	The interviewer answers
Same subnet?	Yes, 10.20.0.0/24
Do VLAN 10 hosts across the trunk work?	Yes
Any change?	The trunk was re-provisioned last night
Same-switch VLAN 20 pings?	They work

Hypotheses, ranked. (1) VLAN 20 missing from the trunk’s allowed list; (2) VLAN 20 not created on SW2; (3) native VLAN mismatch; (4) spanning tree blocking VLAN 20 only (it runs per VLAN).

Layer	Command	Decisive output	Inference
L1	`show interfaces Gi1/0/24`	`up, line protocol is up`; 0 CRC	The link is clean
L2	`show interfaces trunk` on SW1	`Vlans allowed on trunk: 10`	VLAN 20 is not carried
L2	`show vlan brief` on SW2	`20 ENG active Gi1/0/7`	The VLAN exists; hypothesis 2 is ruled out
L2	`show mac address-table vlan 20` on SW1	PC-A listed; PC-B’s MAC absent	Frames from PC-B never arrive
L2	`ip neigh` on PC-A	`10.20.0.7 dev eth0 FAILED`	ARP gets no answer

Root cause. The re-provisioning used `switchport trunk allowed vlan 10`, which *replaces* the list, instead of `add`. Trunks and VLANs are taught in §4.2 (p. 233).

Fix. `switchport trunk allowed vlan add 20` on both ends.

Verify. `show interfaces trunk` lists 20 under allowed, active and forwarding; the MAC of PC-B is learnt; the ping works.

Prevent. Build trunks from a template that sets the list explicitly, and diff the allowed list after every change.

The interviewer asks	A strong answer
What shows a native VLAN mismatch?	`%CDP-4-NATIVE_VLAN_MISMATCH` in the log, and frames leaking between VLANs.
How would you see it on Junos?	`show ethernet-switching interface` and `show vlans`.
Which list on the trunk shows a spanning-tree block?	The third: VLANs in forwarding state and not pruned.

Variant. The VLAN is allowed and active, but absent from the forwarding list: a per-VLAN root disagreement is blocking it on this trunk.

> TRAP
> *Claim:* "`allowed vlan 20` adds VLAN 20 to the trunk." *Why it is wrong:* without the keyword `add`, the command replaces the whole list. *Correct statement:* use `switchport trunk allowed vlan add 20`.

Case 3 · Inter-VLAN routing broken for one VLAN MUST · ■□□

L2 · L3 · CISCO · AMAZON · JUNIPER

Prompt. "Hosts in VLAN 30 can ping each other but nothing outside their VLAN."

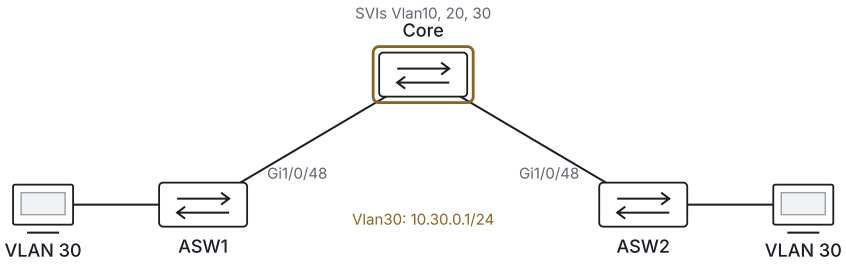


Figure 11.11 Case 3. The core switch has SVIs for three VLANs; Vlan30 is the new one.

We ask	The interviewer answers
Can they ping their gateway 10.30.0.1?	No
Do other VLANs route?	Yes, so `ip routing` is on
Anything new?	VLAN 30 was created this week

Hypotheses, ranked. (1) the SVI is shut or down; (2) VLAN 30 is missing from the core's VLAN database or the uplink trunk; (3) hosts have a wrong gateway or mask.

Layer	Command	Decisive output	Inference
L3	`ipconfig` on a host	Gateway `10.30.0.1`, mask `255.255.255.0`	The host settings are correct
L3	`show ip interface brief`, line Vlan30	`Vlan30 10.30.0.1 YES manual administratively down down`	The SVI is shut
L2	after `no shutdown`: same command	`up down`	Layer 1 up, but the VLAN has no forwarding port

L2	`show vlan brief` on the core	VLAN 30 absent	The VLAN was created only on the access switch

Root cause. Two faults in series: the SVI was shut, and VLAN 30 was neither defined on the core nor allowed on the uplink. An SVI comes up only when its VLAN exists and has at least one port up in that VLAN (the *autostate* rule, §4.2 (p. 233)).

Fix. `vlan 30` on the core; `switchport trunk allowed vlan add 30` on the uplinks; `interface vlan 30`, then `no shutdown`.

Verify. `show ip interface brief` shows `up up`; a host pings the gateway and then the outside; `show ip route connected` lists 10.30.0.0/24.

Prevent. A provisioning playbook that creates the VLAN, the trunk membership and the SVI together, with a post-check.

The interviewer asks	A strong answer
What brings an SVI up?	The VLAN exists, at least one port or trunk in it is up and forwarding, and the SVI is not shut.
Router-on-a-stick variant?	A missing sub-interface, a wrong `encapsulation dot1Q 30`, or a trunk that does not carry the VLAN.
What does `switchport autostate exclude` do?	It stops a port (for example a sniffer) from keeping the SVI up.

Variant. The gateway answers but the hosts carry a /16 mask: ARP for outside addresses is attempted locally and fails.

Case 4 · Slow file transfers after a cable swap MUST · ■■□

L1 · L2 · CISCO · QUALCOMM · BROADCOM · ARISTA

Prompt. “Since yesterday copying files to the file server is painfully slow; web browsing seems fine.”

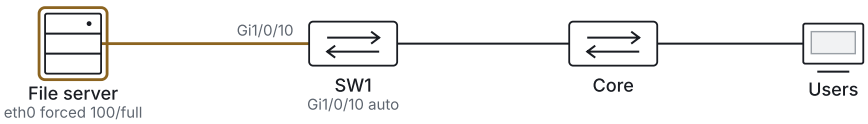


Figure 11.12 Case 4. The server NIC is forced to 100/full; the switch port is left on auto.

We ask	The interviewer answers
All users, or one server?	One server
What changed?	The cable and NIC were replaced and “tuned”
Any error messages?	None
Throughput measured?	About 2 Mb/s against the usual 900 Mb/s

Hypotheses, ranked. (1) duplex mismatch; (2) bad cable (CRC); (3) NIC offload or driver; (4) uplink congestion.

Layer	Command	Decisive output	Inference
L1	`show interfaces Gi1/0/10` on the switch	`Half-duplex, 100Mb/s`; `1554 late collision`, `8412 runts`	The auto side fell back to half duplex
L1	`ethtool eth0` on the server	`Speed: 100Mb/s`, `Duplex: Full`, `Auto-negotiation: off`	The forced side believes it is full duplex
L1	`ethtool -S eth0` for the crc counters, twice, 10 s apart	`rx_crc_errors` rises between snapshots	Proves the fault is current, not historic
L4	`ss -ti` during a copy	`retrans:3/412`	Bulk TCP collapses on loss; interactive traffic hardly notices

Root cause. A duplex mismatch. A port that does not negotiate is not seen by the auto side, which therefore picks the speed by parallel detection but defaults to half duplex. The half-duplex side detects collisions (late collisions, runts) when the full-duplex side transmits at will; the full-duplex side logs CRC errors from the truncated frames. The mechanism is in §3.3 (p. 168).

Fix. Set both ends to auto (preferred), or both forced identically: `ethtool -s eth0 autoneg on`.

Verify. 1000/full on both sides, counters stop rising, and `iperf3` reaches about 940 Mb/s.

Prevent. An auto/auto policy, and an alert on the rate of late collisions and CRC errors.

The interviewer asks	A strong answer
Which side shows late collisions, and which CRC?	The half-duplex side shows late collisions and runts; the full-duplex side shows CRC and frame errors.
CRC errors with matching duplex?	Suspect the cable, the optic (`show interfaces transceiver`) or interference.
Why is browsing fine but the copy slow?	Short flows carry little data, whereas bulk flows meet the loss and TCP backs off.

Variant. A NIC offload or driver bug gives the same slowness with no errors on the switch; compare `ethtool -k` and driver versions.

Case 5 · SSH works, big transfers over the VPN hang MUST · ■■□

L3 · L4 · GOOGLE · AMAZON · CISCO · JUNIPER · MICROSOFT

Prompt. “Branch users can open the intranet login page over the site-to-site VPN, but large pages and downloads hang. SSH is fine.”

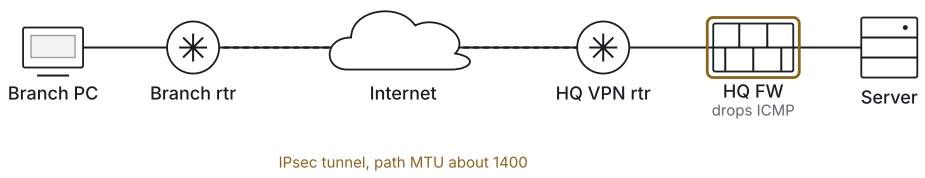


Figure 11.13 Case 5. The HQ firewall behind the VPN router drops all ICMP, including the “fragmentation needed” message.

We ask	The interviewer answers
Small or large?	Small things work, large hang
Only over the VPN?	Yes
Recent change?	The HQ firewall was “hardened”: all ICMP blocked
Which way does the bulk data flow?	Server to client

Hypotheses, ranked. (1) a path-MTU-discovery black hole (ICMP type 3 code 4 blocked); (2) MSS not clamped on the tunnel; (3) servers set the DF bit.

Layer	Command	Decisive output	Inference
L3	`ping -M do -s 1472 10.10.5.20`	`100% packet loss`	A full-size DF packet does not cross
L3	`ping -M do -s 1372 10.10.5.20`	`1380 bytes from 10.10.5.20: ttl=62`	The path MTU is about 1400
L3	`tracepath -n 10.10.5.20`	`pmtu 1500` unchanged	The router’s ICMP never returns, so discovery fails
L4	`tcpdump` at the client	Handshake completes; full-size segments never arrive; server retransmits	The first big segment is the one lost
L4	HQ firewall log	`ICMP type 3 code 4 from branch router dropped`	The proof of the black hole

Root cause. Tunnel encapsulation lowers the usable MTU. The VPN router sends “fragmentation needed” to the server, the HQ firewall drops it, and the server keeps sending 1500-byte DF packets that are discarded: a *PMTUD black hole*. SSH works because its packets are small. Path-MTU discovery is taught in §3.5 (p. 179).

Fix. Permit ICMP type 3 code 4 (and, for IPv6, Packet Too Big); also clamp the MSS on the tunnel, `ip tcp adjust-mss 1360`, or on Linux `iptables -t mangle -A FORWARD -p tcp --tcp-flags SYN,RST SYN -j TCPMSS --clamp-mss-to-pmtu`.

Verify. The large download completes, the SYN carries MSS 1360, and the DF ping at 1372 succeeds.

Prevent. A security policy that lists the essential ICMP types (RFC 4890 for IPv6), and a tunnel template with the MSS clamp.

The interviewer asks	A strong answer
Why does SSH work?	Its packets are small and never reach the MTU limit.
What about UDP applications?	They have no MSS, so they need fragmentation or sizing in the application.
How do you size the clamp?	ESP in tunnel mode adds about 50 to 73 bytes (plus 8 for NAT-T), so configure MTU 1400 and MSS 1360 (MTU minus 40).

Variant. IPv6: routers never fragment, so a filtered Packet Too Big message is fatal for every large flow.

> INTERVIEW LENS
>
> GOOGLE · AMAZON · CISCO

> Asked as a symptom with the cause hidden. A strong answer reaches “size-dependent” within two questions, proposes the DF ping ladder as the test, and names both ICMP and the MSS clamp in the fix. Expect: “why does the ping at 1472 fail but 1372 pass?”

Case 6 · OSPF neighbours stuck in ExStart MUST · ■■□

L3 · CISCO · JUNIPER · NOKIA · ARISTA · AMAZON

Prompt. “We added a link between R1 and R2 and OSPF will not come up; `show ip ospf neighbor` keeps showing ExStart.”

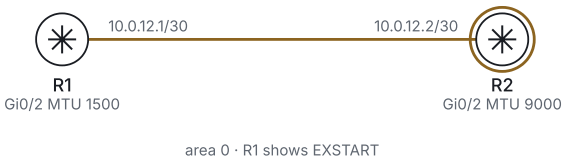


Figure 11.14 Case 6. Jumbo frames are configured on R2 only.

We ask	The interviewer answers
Which state exactly?	ExStart on R1, Exchange then back on R2; it flaps
Do hellos pass?	Yes, 2-Way was reached
Interface MTUs?	“Not sure”
Vendors?	Mixed

Hypotheses, ranked. (1) MTU mismatch; (2) duplicate router ID; (3) unicast problem for the DBD packets.

Layer	Command	Decisive output	Inference
L3	`show ip ospf neighbor`	`2.2.2.2 1 EXSTART/DR 00:00:34 10.0.12.2 Gi0/2`	Hellos work; the database exchange does not
L3	`debug ip ospf adj`	`Nbr 2.2.2.2 has larger interface MTU`	R1 rejects R2’s description packets
L1	`show interfaces Gi0/2`, MTU line, on each router	`MTU 1500` on R1, `MTU 9000` on R2	The mismatch

Root cause. A database description packet carries the sender’s interface MTU, and a router rejects one that advertises a larger MTU than its own, so the exchange never completes (§4.5 (p. 246)).

Fix. Match the MTUs. `ip ospf mtu-ignore` is a stop-gap only, because large updates may then be dropped.

Verify. The neighbour reaches Full, `show ip route ospf` lists the remote prefixes, and the databases agree.

Prevent. Link templates set MTU on both ends, and an automation pre-check compares them.

The interviewer asks	A strong answer

State the OSPF neighbour states.	Down, Init, 2-Way, ExStart, Exchange, Loading, Full.
What keeps a neighbour in Init?	One-way hellos: an ACL, a multicast problem, or a passive interface on the far side.
What does a duplicate router ID look like?	Flapping adjacencies and flushed LSAs, with `%OSPF-4-DUP_RTRID` in the log.

Variant. On a non-broadcast link, a network-type mismatch gives different timers (10 and 40 s against 30 and 120 s) and no adjacency.

> TRAP
>
> *Claim:* "`ip ospf mtu-ignore` fixes it." *Why it is wrong:* it hides the mismatch, and a full-size LSU can later be lost on the smaller side. *Correct statement:* make the MTUs equal, and use the command only as a documented temporary measure.

> INTERVIEW LENS
>
> CISCO · JUNIPER · NOKIA
>
> Often asked as a pure recall question, "stuck in ExStart". The full answer gives the mechanism (the MTU field in the database description), the proof (`debug ip ospf adj`) and the objection to `mtu-ignore`. Expect: "what is checked in a DBD packet?"

Case 7 · OSPF neighbour never appears MUST · ■□□

L3 · CISCO · JUNIPER · NOKIA

Prompt. "R3 and R4 are directly connected and pingable, but no OSPF neighbour forms."

Figure 11.15 Case 7. Three independent mismatches are stacked on one link.

We ask	The interviewer answers
Any neighbour state at all?	None; not even Init
What changed?	R4 is new
Authentication?	Yes, MD5 on R3

Hypotheses, ranked. The *adjacency must-match list* is the hypothesis list: same subnet and mask; same area ID; same hello and dead timers; same authentication type and key; same stub flags; compatible network types; unique router IDs; interface not passive; nothing blocking 224.0.0.5 and 224.0.0.6.

Layer	Command	Decisive output	Inference
L3	`show ip ospf interface Gi0/0` on R4	`No Hellos (Passive interface); Area 1`	R4 sends no hellos, and is in the wrong area
L3	the same on R3	`Area 0`, `Hello 10, Dead 40`, MD5	R3 expects area 0 with authentication

L3	after `no passive-interface`: R3 log	`Received invalid packet: mismatch area ID`	Second fault exposed
L3	after correcting the area: R3 log	`Mismatch Authentication type. Input packet specified type 0, we use type 2`	Third fault exposed

The exact wording of these messages varies a little between releases; the pattern is the point.

Root cause. Three stacked faults: a passive interface, area 1 where area 0 is required, and no authentication on R4. The lesson is to fix one, re-test, and not to stop at the first finding.

Fix. `no passive-interface Gi0/0`; `network 10.0.34.0 0.0.0.255 area 0`; `ip ospf authentication message-digest` and `ip ospf message-digest-key 1 md5 <key>`.

Verify. `show ip ospf neighbor` shows Full; the remote prefixes are in the table.

Prevent. Generate both ends of every link from one source of truth.

The interviewer asks	A strong answer
Which parameters need not match?	Process ID, priority, cost and interface names.
Junos equivalent?	`show ospf interface detail` and `show ospf neighbor`.
Why do network types matter?	Defaults differ between broadcast (10 and 40 s) and non-broadcast (30 and 120 s), so a mismatch appears as a timer mismatch.

Variant. The key identifier differs, although both ends use MD5: the log shows an authentication failure, not a type mismatch.

Case 8 · BGP session stuck in Active MUST · ■■□

L3 · L4 · AMAZON · GOOGLE · CISCO · JUNIPER · ARISTA · NOKIA · MICROSOFT

Prompt. “Our eBGP session to the ISP is stuck in Active.”

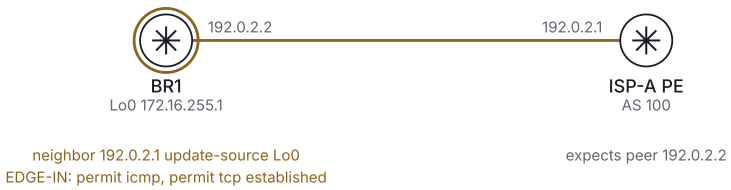


Figure 11.16 Case 8. BR1 sources the session from its loopback while the provider expects the WAN address, and an edge ACL blocks inbound SYNs.

We ask	The interviewer answers
Did it ever work?	No, it is a new circuit
Which addresses do the two sides use?	We use Loopback0; the ISP expects 192.0.2.2
Any filtering?	An edge ACL on the WAN interface
Who configured what?	Both sides are configured

Hypotheses, ranked. (1) update-source or neighbour address mismatch; (2) TCP 179 blocked by an ACL; (3) eBGP TTL 1 with a loopback (needs `ebgp-multihop`); (4) AS or MD5 mismatch

(these show NOTIFICATION messages, not Active).

Layer	Command	Decisive output	Inference
L4	`show ip bgp summary`	`192.0.2.1 4 100 0 0 ... never Active`	TCP connection attempts are failing
L3	`ping 192.0.2.1 source Loopback0`	`.....`	The ISP has no route back to our loopback
L3	`ping 192.0.2.1`	`!!!!!`	The link itself works
L4	`telnet 192.0.2.1 179 /source-interface Gi0/1`	`Open`	From the WAN address the port opens
L4	`show access-lists EDGE-IN`	`30 deny ip any any (87 matches)`	The ISP's SYNs are hitting the deny

Root cause. Two faults: BR1 originates the session from Loopback0 although the provider peers with the WAN address, and the edge ACL permits only established TCP and ICMP, so the ISP's SYN to port 179 is dropped. BGP's session rules are in §4.7 (p. 256).

Fix. Remove `update-source Loopback0`, or agree loopback peering with the ISP (plus `ebgp-multihop` and a static route). Insert `permit tcp host 192.0.2.1 host 192.0.2.2 eq bgp` and the reverse before the deny.

Verify. `show ip bgp summary` shows a number in the last column; `show ip bgp neighbors 192.0.2.1 | include state` shows Established; routes enter the table.

Prevent. An edge-ACL template with a BGP peer object group, and a pre-turn-up checklist.

The interviewer asks	A strong answer
Name the BGP states.	Idle, Connect, Active, OpenSent, OpenConfirm, Established.
How does Idle (admin) differ from Active?	Idle (admin) is shut down by configuration; Active is an attempt that fails.
What does eBGP to a loopback need?	TTL above 1 (`ebgp-multihop 2` or `disable-connected-check`) and a route to both loopbacks.

Variant. The state flaps between Active and Idle with "bad AS" in the log: a `remote-as` mismatch, shown as a NOTIFICATION.

> INTERVIEW LENS
>
> AMAZON · GOOGLE · JUNIPER
>
> Candidates report BGP state definitions being probed directly. Answer Active precisely (connect failing), then give the three checks: address, filter, TTL. Expect: "what would you see if the AS number were wrong?"

Case 9 · BGP up, but a prefix is not advertised or installed MUST · ■■□

L3 · AMAZON · GOOGLE · ARISTA · JUNIPER · NOKIA · MICROSOFT

Prompt. "BGP is up, but the ISP says it does not receive our 198.51.100.0/24, and internally R5 learns a route but does not use it."

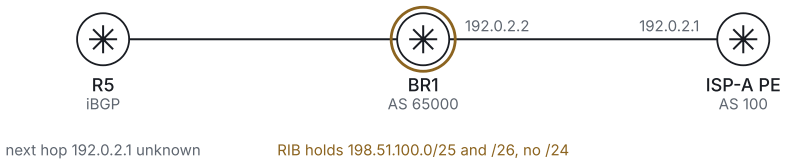


Figure 11.17 Case 9. BR1 owns only two subnets of the /24, and R5 cannot reach the eBGP next hop.

We ask	The interviewer answers
Is the prefix in our table exactly?	"We have subnets of it"
Any outbound policy?	A route map, TO-ISP
iBGP design?	Full mesh, no `next-hop-self`

Hypotheses, ranked. (1) a `network` statement needs an *exact* match in the routing table; (2) outbound policy filters the route; (3) the iBGP next hop is unreachable; (4) the ISP filters by IRR or RPKI.

Layer	Command	Decisive output	Inference
L3	`show ip route 198.51.100.0 255.255.255.0`	`% Network not in table`	No exact /24 to originate
L3	`show ip bgp neighbors 192.0.2.1 advertised-routes`	empty	Nothing is sent
L3	`show ip bgp 8.8.8.0` on R5	`192.0.2.1 (inaccessible) from 10.255.0.1`	The next hop is unusable
L3	`show ip route 192.0.2.1` on R5	`% Network not in table`	The IGP does not carry the WAN link

Root cause. Two faults: the `network` statement needs the exact /24 in the routing table but only the /25 and /26 exist; and iBGP passes the eBGP next hop unchanged, which R5 cannot reach (§4.7 (p. 256)).

Fix. `ip route 198.51.100.0 255.255.255.0 Null0` (an anchor route) or `aggregate-address`; then `neighbor <R5> next-hop-self` on BR1.

Verify. The /24 appears in `advertised-routes`, the ISP confirms, and the R5 route shows the best-path marker `>`.

Prevent. An origination checklist, and a registered RPKI route origin authorisation for the /24.

The interviewer asks	A strong answer
What is the best-path order?	Weight, local preference, locally originated, AS-path length, origin, MED, eBGP over iBGP, IGP metric to the next hop, then tie-breakers (§4.7 (p. 256)).
How do you resend routes without dropping the session?	`clear ip bgp <peer> soft out`.
Why might the ISP drop a /25?	Providers commonly filter prefixes longer than /24.

Variant. The prefix is advertised but the provider marks it RPKI-invalid because no route origin authorisation exists.

Case 10 · One server drops for a minute, several times an hour MUST · ■■□

L2 · L3 · CISCO · AMAZON · GOOGLE

Prompt. “The finance server randomly becomes unreachable for 30 to 60 seconds, several times an hour.”

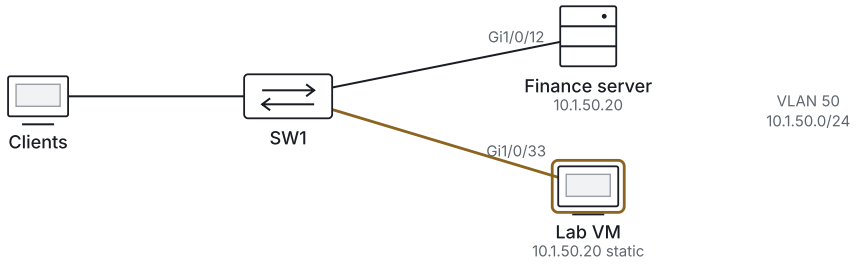


Figure 11.18 Case 10. A lab VM was given the finance server’s static address.

We ask	The interviewer answers
All clients or some?	Some at a time
Pattern?	Random
What does the server log?	Nothing
Any change?	New VMs in VLAN 50 last week

Hypotheses, ranked. (1) duplicate IP address; (2) MAC flapping from a loop; (3) NIC-teaming misconfiguration; (4) ARP cache problems.

Layer	Command	Decisive output	Inference
L3	switch log	`%IP-4-DUPADDR: Duplicate address 10.1.50.20 on Vlan50, sourced by 0050.56aa.bbbb`	Another device uses the address
L2	`show ip arp 10.1.50.20`, repeated	The MAC alternates between two values	Clients’ caches flip
L2	`show mac address-table address 0050.56aa.bbbb`	`Gi1/0/33`	The intruder is on the hypervisor port
L2	`arping -D -I eth0 10.1.50.20`	`Unicast reply from 10.1.50.20 [00:50:56:AA:BB:BB]`	A second MAC answers for the address

Root cause. A duplicate IP address: ARP caches on clients alternate between the two MACs, so the server is reachable only while a client holds the right entry (ARP is in §3.6 (p. 183)).

Fix. Re-address the VM, `clear arp-cache`, and reserve the address in IPAM and the DHCP exclusion list.

Verify. ARP stays stable and no `DUPADDR` message appears for an hour.

Prevent. IPAM, DHCP snooping with IP source guard, dynamic ARP inspection, and gratuitous-ARP conflict detection on servers.

The interviewer asks	A strong answer
How does a MAC flap differ from an ARP flap?	A MAC flap is one MAC seen on two ports (loop, mis-teamed NIC); an ARP flap is one IP resolving to two MACs (duplicate).
What else looks the same?	An HSRP split-brain, where both routers are active; `show standby` shows it.
How do you find the intruder?	The OUI identifies the vendor; `show mac address-table address <mac>` names the port.

Variant. A NIC team configured active-active without a LAG makes the server's own MAC flap between ports.

Case 11 · A whole floor went down when someone plugged in a switch MUST · ■■□

L2 · CISCO · JUNIPER · NOKIA · ARISTA

Prompt. "At 11:05 the entire third floor lost network; the switch CPUs are at 100 per cent and the LEDs flash wildly."

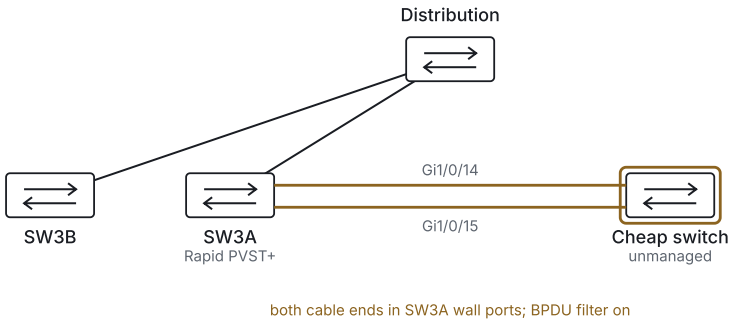


Figure 11.19 Case 11. A cheap switch has a cable looped between two wall ports of SW3A; BPDU filter makes spanning tree blind.

We ask	The interviewer answers
Whole floor or building?	Floor
Any change?	"Nothing", then, when asked about desk moves: a desk move this morning
What does monitoring show?	Broadcast at 100 per cent of the link
Is spanning tree on?	Yes, Rapid PVST+, with PortFast on edge ports

Hypotheses, ranked. (1) a Layer 2 loop and broadcast storm; (2) spanning tree disabled or bypassed on the ports; (3) root-bridge change; (4) a unidirectional link.

Layer	Command	Decisive output	Inference
L2	`show interfaces Gi1/0/14`, rate lines	`5 minute input rate 912345000 bits/sec`	A storm on these ports
L2	switch log	`%SW_MATM-4-MACFLAP_NOTIF: Host 0050.7966.6801 in vlan 10 is flapping between port Gi1/0/14 and port Gi1/0/15`	The same MACs appear on both ports: a loop

L2	`show spanning-tree interface Gi1/0/14 detail`	`Bpdu filter is enabled; BPDU: sent 0, received 0`	Spanning tree is blind on these ports
L1	`show processes cpu sorted`	`CPU utilization for five seconds: 99%`	The control plane is saturated

Mitigate first. `shutdown` on Gi1/0/14, then confirm the rates fall.

Root cause. A physical loop on edge ports where BPDU filter disabled loop detection; with PortFast alone, the port would receive its own BPDUs and block within about two seconds (§4.3 (p. 237)). No BPDU Guard or storm control existed as a backstop, and Ethernet frames have no TTL, so they circulate indefinitely.

Fix. Remove the loop and the BPDU filter from access ports; `spanning-tree portfast bpduguard default`; `storm-control broadcast level 1.00`; an errdisable recovery policy; Loop Guard on uplinks.

Verify. Broadcast rates and CPU return to normal and MAC flaps stop.

Prevent. An access-port template with BPDU Guard and storm control, broadcast-rate alerts, physical security of wall ports, 802.1X. The post-mortem for this case is in §11.7 (p. 647).

The interviewer asks	A strong answer
Why do Ethernet frames loop for ever?	The Ethernet header has no TTL, so nothing removes a frame from a loop.
What does Root Guard do?	It stops a port from accepting a superior BPDU, so an edge switch cannot take over as root.
How does RSTP converge?	A proposal and agreement handshake on point-to-point links, in under a second.

Variant. The *global* `spanning-tree portfast bpdufilter default` is safer than the per-port form, because ports lose the filter when a BPDU arrives.

> INTERVIEW LENS
>
> CISCO · JUNIPER · ARISTA
>
> A mitigation test: the interviewer waits to see whether the candidate shuts the port before explaining the theory. Say the mitigation first, then the cause, then the template change. Expect: “why did spanning tree not stop it?”

Case 12 · New VLAN’s clients get 169.254 addresses MUST · ■□□

L2 · L3 · L7 · CISCO · AMAZON · JUNIPER

Prompt. “Laptops in the new lab VLAN 60 get 169.254 addresses.”

Figure 11.20 Case 12. The SVI has no relay, and the access switch distrusts the uplink on which offers return.

We ask	The interviewer answers
Only VLAN 60?	Yes
Does a static address work?	Yes, so Layers 2 and 3 are fine
Does a scope exist for VLAN 60?	Yes
Is DHCP snooping on?	Yes, on the access layer

Hypotheses, ranked. (1) no relay on the SVI; (2) the scope is missing or exhausted; (3) snooping drops server replies on an untrusted uplink; (4) option 82 trouble.

Layer	Command	Decisive output	Inference
L7	`ipconfig` on a laptop	`Autoconfiguration IPv4 Address: 169.254.37.8`	No lease was obtained
L7	`tcpdump -ni eth0 port 67 or port 68`	Only `BOOTP/DHCP, Request from ...`	Discovers are sent; no offer returns
L3	`show running-config interface vlan 60`	no `ip helper-address`	Nothing forwards the broadcast to the server
L2	after the helper is added: `show ip dhcp snooping statistics`	`Packets Dropped From untrusted ports = 14`	Offers are discarded at the uplink
L2	`show ip dhcp snooping`	`Gi1/0/48 no` (trust)	The uplink is untrusted

Root cause. Two faults: the SVI has no DHCP relay, and the access switch treats its uplink as untrusted, so the offers that return from the server are dropped. A client without a lease assigns itself an address in 169.254.0.0/16 (§3.6 (p. 183)).

Fix. `ip helper-address 10.1.99.10` on Vlan60; `ip dhcp snooping trust` on the uplink; check the server scope for 10.1.60.0/24.

Verify. The four steps of DHCP (Discover, Offer, Request, Acknowledge) appear in a capture, the client holds a lease, and `show ip dhcp snooping binding` lists it.

Prevent. The VLAN provisioning playbook includes the relay and ends with a check that a test client obtains a lease.

The interviewer asks	A strong answer
How does the server pick the scope?	From the relay address (giaddr) in the request.
Which DHCP steps are broadcast?	Discover and, before the address is held, Request; the relay converts them to unicast.
What is the option 82 risk?	Snooping inserts it; some servers drop requests that carry it with giaddr zero.

Variant. *Exhaustion.* A late arrival gets 169.254 while others work: `show ip dhcp pool` shows no free addresses. Shorten the lease, enlarge the scope and reclaim stale bindings.

Case 13 · Traffic works one way through the firewalls SHOULD · ■■■

L3 · L4 · CISCO · JUNIPER · NOKIA · GOOGLE · AMAZON

Prompt. “After adding a second internet link, some users’ sessions to the cloud application break; it looks random.”

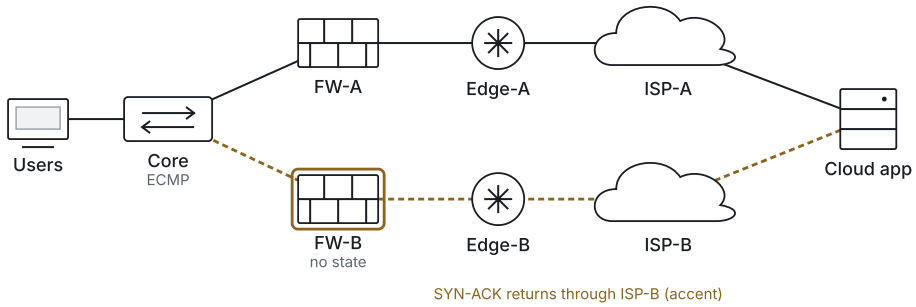


Figure 11.21 Case 13. The request leaves through ISP-A and FW-A; the reply returns through ISP-B and FW-B, which has no state for it.

We ask	The interviewer answers
Which users?	About half, varying by session
Which protocols?	TCP mostly; pings are fine
What do the firewall logs say?	FW-B: “TCP packet without SYN, no connection”
What changed?	A second ISP, with ECMP to both firewalls

Hypotheses, ranked. (1) asymmetric routing through two independent stateful firewalls; (2) a NAT pool mismatch; (3) MTU on the new link.

Layer	Command	Decisive output	Inference
L3	`traceroute` from a user, and from the cloud server (looking glass)	The two paths use different ISPs	The path is asymmetric
L4	FW-B log	`%ASA-6-106015: Deny TCP (no connection) from 203.0.113.50/443 to 198.51.100.77/51514 flags SYN ACK`	A SYN-ACK arrives for a session FW-B never saw
L4	`show conn` on FW-A	The session, with its SYN	FW-A holds the state; FW-B does not
L3	ICMP test	Passes	Echo replies are permitted without a session

Root cause. The return path crosses a different stateful firewall that has no session state, so it discards the reply. Remote networks choose their own return path by BGP policy, so Campus Net cannot force symmetry with outbound routing alone (§4.7 (p. 256)).

Fix. Make the paths symmetric: source-NAT per ISP so replies return to the address of the firewall that sent the request; or policy routing; or an HA pair with state synchronisation or clustering. Do not run ECMP across independent stateful devices.

Verify. Sessions are stable and the “no connection” drop counter falls to zero.

Prevent. A design rule that a stateful device must see both directions, and failover tests of each path.

The interviewer asks	A strong answer
What is TCP state bypass?	A firewall setting that skips state checks for chosen flows; it removes the symptom and some protection.
How does ECMP choose a path?	By hashing the five-tuple, so one flow stays on one path.
Would strict uRPF help?	No, it would also drop asymmetric traffic.

Variant. *NAT pool exhaustion.* New connections fail while old ones work; `show ip nat statistics` shows misses. Enlarge the pool or use PAT.

> INTERVIEW LENS
>
> GOOGLE · AMAZON · CISCO
>
> A design question disguised as a fault. A strong answer ends with a rule (stateful devices must see both directions) and offers two fixes with their trade-offs. Expect: "what would you do if you could not change the ISPs?"

Case 14 · Website slow for some users only SHOULD · ■■■

L1 · L2 · L3 · GOOGLE · MICROSOFT · AMAZON · ARISTA

Prompt. "About a quarter of users say the internal portal is slow; the others are fine. Server CPU is low."

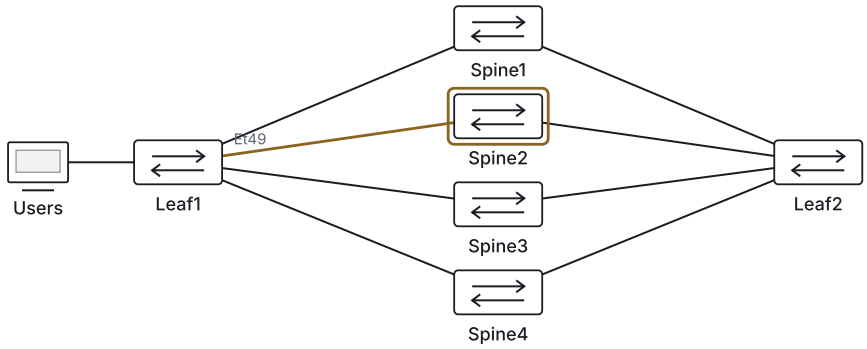


Figure 11.22 Case 14. Flows from Leaf1 are hashed over four spine uplinks; one has a failing optic.

We ask	The interviewer answers
Which users?	No pattern by floor
Consistent per user?	Yes, the same users every time
Measured how?	`curl` shows a long time to first byte, with retransmissions
What is the topology?	A leaf-spine fabric with four equal-cost uplinks

Hypotheses, ranked. (1) one bad ECMP or LAG member; (2) an optic or duplex error; (3) one overloaded pool member behind a load balancer; (4) DNS returning a far replica.

Layer	Command	Decisive output	Inference

L7	`curl -w` with the timing variables (name lookup, connect, first byte), slow vs fast client	DNS and connect equal; transfer slow	Not DNS or setup; loss in the transfer
L3	`mtr -rwzbc 100 portal` from a slow client	Loss from the leaf-to-spine hop, persisting to the end	Real forwarding loss on one path
L1	`show interfaces counters errors` on Leaf1	`Et49 FCS 18342` rising	CRC errors on one uplink
L1	`show interfaces transceiver`	`Et49 Rx power -14.2 dBm`	The optic is weak

Root cause. A faulty optic on one fabric uplink loses packets for exactly those flows whose five-tuple hashes to it. Hashing is consistent per flow, which explains “the same users, always” and the absence of a pattern by floor.

Fix. Drain or shut the member so flows rehash, then clean or replace the optic and check the fibre.

Verify. CRC deltas are zero, light levels normal, and the users’ `curl` timings recover.

Prevent. Telemetry alerts on CRC rate and optical power thresholds (Pillar 10 (p. 586)), and an automated drain.

The interviewer asks	A strong answer
Which fields feed the ECMP hash?	Source and destination address, protocol and ports.
Why is per-packet balancing avoided?	It reorders packets and triggers duplicate ACKs.
What is a gray failure?	A partial fault that health checks miss; end-to-end probes over every path find it.

Variant. A port channel member suspended for a mismatched configuration (flag `s` in `show etherchannel summary`) gives the same “some flows” pattern.

> INTERVIEW LENS
>
> GOOGLE · MICROSOFT · ARISTA
>
> A gray-failure question. The marks are for linking “same users every time” to flow hashing, and for proposing telemetry on every member. Expect: “how would you find this across 500 leaves?”

Case 15 · VPN tunnel up, but no traffic SHOULD · ■■□

L3 · SECURITY · CISCO · JUNIPER · NOKIA

Prompt. “The new site-to-site VPN to Hyderabad shows Phase 1 up, but users cannot reach HQ.”

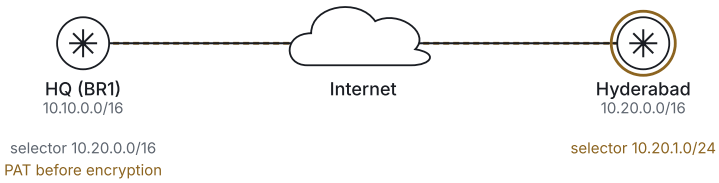


Figure 11.23 Case 15. The two sides disagree on the traffic selectors, and HQ translates VPN-bound traffic before encrypting it.

We ask	The interviewer answers
IKE version?	IKEv2
Is the child SA up?	"Not sure"
What do the logs say?	`TS_UNACCEPTABLE`
Any NAT?	HQ does PAT for internet access

Hypotheses, ranked. (1) traffic-selector or proxy-ID mismatch; (2) transform-set or PFS mismatch; (3) missing NAT exemption; (4) a missing route to the tunnel, if route-based.

Layer	Command	Decisive output	Inference
L3	`show crypto ikev2 sa`	`Status: READY`	Phase 1 is healthy
L3	`show crypto ipsec sa peer 203.0.113.9`	no child SA; debug: `TS_UNACCEPTABLE`	The selectors do not match
L3	after mirroring the selectors: the same command	`#pkts encaps: 412 ... #pkts decaps: 0`	HQ sends; nothing returns
L3	`show ip nat translations`	`icmp 192.0.2.2:5 10.10.4.7:5 10.20.1.9:5 10.20.1.9:5`	HQ translates traffic before encryption

Root cause. Two faults: HQ defines 10.20.0.0/16 and the branch 10.20.1.0/24, so negotiation of the child SA fails; and the NAT rule translates VPN-bound traffic, so it no longer matches the selector.

Fix. Mirror the selectors on both peers; add a NAT exemption (a `deny` for 10.10.0.0/16 to 10.20.0.0/16 at the top of the NAT access list).

Verify. Both sides show encapsulation and decapsulation counts rising; users reach HQ applications.

Prevent. Generate both peers from one definition, and use the checklist of §9.7 (p. 555).

The interviewer asks	A strong answer
What causes Phase 1 to fail?	A wrong pre-shared key, mismatched proposals, or UDP 500 and 4500 blocked.
How do policy-based and route-based VPNs differ in debugging?	The first by comparing selectors, the second by checking the route into the tunnel.
What is NAT-T?	ESP encapsulated in UDP 4500 when a NAT is detected between the peers.

Variant. The tunnel works for small packets only: the MTU problem of Case 5 (p. 654) applies as well.

Case 16 · Linux service: refused, then timeout SHOULD · ■□□

L4 · L7 · AMAZON · GOOGLE · MICROSOFT · ARISTA

Prompt. "Our new API on port 8080 works with `curl` on the VM itself, but not from other machines."

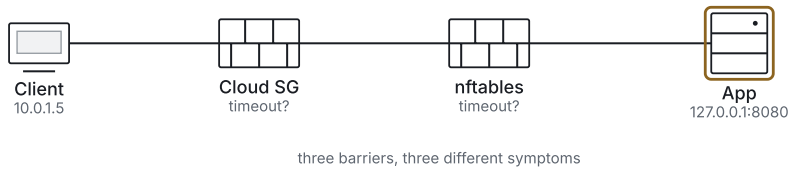


Figure 11.24 Case 16. Three barriers in a row, each with its own symptom; the loopback bind is the first.

We ask	The interviewer answers
What error does the client see?	"Connection refused" once; a timeout after the admin "opened" the firewall
Cloud or on premises?	Cloud, with a security group
Any change?	A new deployment

Hypotheses, ranked. (1) the application binds to loopback; (2) the host firewall; (3) the cloud security group; (4) a wrong port.

Layer	Command	Decisive output	Inference
L4	`ss -tlnp` on the VM, line for 8080	`LISTEN 0 128 127.0.0.1:8080 ... users: (("python3",pid=2211,fd=3))`	Loopback only; the kernel answers other hosts with a RST
L4	after rebinding to `0.0.0.0`: client `curl`	`curl: (28) Connection timed out after 5001 milliseconds`	A drop, not a refusal
L4	`nft list ruleset` or `ufw status`	no rule for 8080; `policy drop`	The host firewall drops it; allow it
L4	`tcpdump -ni eth0 port 8080` on the VM	nothing arrives	The packet dies outside the VM: the security group

Root cause. Three layers in a stack: the loopback bind (refusal), the host firewall (timeout), and the cloud security group (timeout, with the packet never arriving). Placing the capture on the VM is what separates the last two.

Fix. Bind to the interface address, allow 8080 from the client CIDR in nftables and in the security group.

Verify. `curl -v http://10.0.2.10:8080/health` returns 200 from the client, and `ss` shows `0.0.0.0:8080`.

Prevent. A deployment health check run from outside the host, and the security groups held as infrastructure-as-code.

The interviewer asks	A strong answer

Refusal or ICMP unreachable?	TCP is refused with a RST; a closed UDP port returns ICMP port unreachable (type 3 code 3).
What does tcpdump placement prove?	A capture on the VM shows whether the packet arrives; if not, the drop is upstream.
How do `ufw` and nftables relate?	`ufw` is a front end that writes rules into the kernel's netfilter.

Variant. The application binds only to an IPv6 wildcard, or to a Unix socket behind a proxy that is not running.

Case 17 · Throughput between Mumbai and Frankfurt is terrible OPTIONAL · ■■■

L4 · GOOGLE · MICROSOFT · AMAZON · BROADCOM

Prompt. "We have a 1 Gb/s private link, but a single backup stream reaches about 6 Mb/s. Several streams in parallel sum to much more."

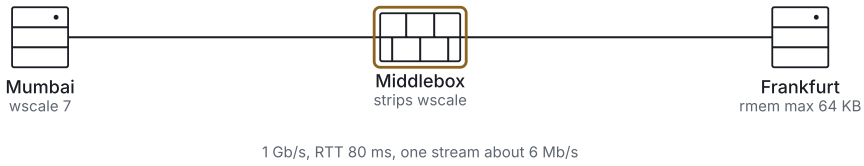


Figure 11.25 Case 17. A middlebox strips the window-scale option and an old script caps the receive buffer; one stream is limited to 64 KB per round trip.

We ask	The interviewer answers
Loss on the link?	Nearly none
Do parallel streams help?	Yes
Round-trip time?	80 ms
Any tuning of the hosts?	A hardening script from years ago

Hypotheses, ranked. (1) a window limit (the bandwidth-delay product); (2) window scaling removed; (3) loss and congestion control; (4) CPU or disk.

Layer	Command	Decisive output	Inference
L4	arithmetic	BDP = 1 Gb/s × 0.08 s = 10 MB; 64 KB / 80 ms is about 6.5 Mb/s	The observation equals a 64 KB window: window-limited
L4	`ss -ti`	`wscale` field absent	Scaling was not negotiated
L4	`tcpdump -ni eth0 'tcp[tcpflags] & tcp-syn != 0'` at both ends	`wscale 7` leaves Mumbai; the SYN arrives without it	A middlebox strips the option
L4	`sysctl net.ipv4.tcp_rmem`	`4096 16384 65536`	The receive buffer maximum is 64 KB
L4	`iperf3 -c host`, then `-P 8`	about 6 Mb/s, then several times more	Each stream is limited separately

Root cause. Without window scaling, the window cannot exceed 64 KB, and the buffer cap enforces the same limit, so a stream carries at most one window per round trip. The relation is in §3.9 (p. 192).

Fix. Stop the option stripping (disable the middlebox's TCP normalisation); raise `tcp_rmem` and `tcp_wmem` maxima to at least 16 MB; consider BBR.

Verify. A single `iperf3` stream reaches about 900 Mb/s.

Prevent. Baseline long-haul throughput tests, and an audit for drift in `sysctl` settings.

The interviewer asks	A strong answer
Why do parallel streams help?	Each stream has its own window, so their sum approaches the link rate.
What if loss, not the window, is the limit?	`ss -ti` shows many retransmissions and a small `cwnd`; the Mathis estimate applies.
Would BBR help?	Yes under loss, since it paces to the measured rate and not to loss events.

Variant. A single stream limited by the application, for example a disk or a single-threaded compressor.

Case 18 · Traceroute shows a scary loss at hop 4 OPTIONAL · ■□□

L3 · GOOGLE · CISCO · AMAZON

Prompt. "A customer sends an `mtr` showing 70 per cent loss at your router, hop 4, and demands a fix."

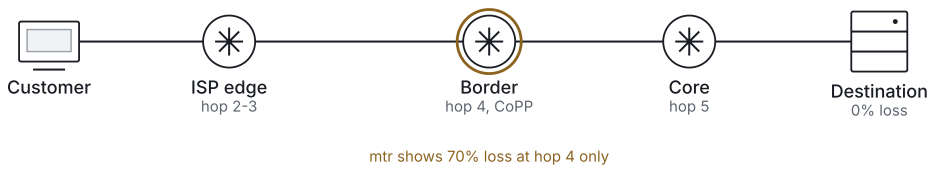


Figure 11.26 Case 18. The border router rate-limits the ICMP it generates; the forwarding path is clean.

We ask	The interviewer answers
Loss at the destination?	0 per cent
Impact the customer can see?	None measured; they "saw the report"
Which protocol?	`mtr` default, ICMP

Hypotheses, ranked. (1) control-plane rate limiting of ICMP replies (benign); (2) real forwarding loss, which would persist at later hops; (3) a slow return path.

Layer	Command	Decisive output	Inference
L3	`mtr -rwzbc 200 dest`	hop 4 `70.0%`, hops 5 and 6 `0.0%`	Loss that does not persist is not forwarding loss
L4	`mtr -T -P 443 -rwzbc 200 dest`	same shape	The pattern is independent of the probe

L3	`show policy-map control-plane`	`Class-map: ICMP-TTL ... exceeded 41230 packets`	CoPP limits the replies the router generates
L1	interface counters on the border router	clean	The data plane is healthy

Root cause. None: the router rate-limits the time-exceeded messages it generates, while it forwards the customer's traffic without loss. The skill tested is communication, since the answer is evidence and a courteous explanation.

Fix. None; explain with the traces. Consider a modestly higher ICMP rate limit.

Verify. The customer's application metrics are normal.

Prevent. A support runbook entry for this report.

The interviewer asks	A strong answer
Which latency pattern is real?	A jump at one hop that *persists* to later hops is queueing; one that appears only at that hop is slow-path ICMP.
How do you answer the customer?	Share the traces, explain that routers answer probes on a slow path, and offer an end-to-end application test.
What would real loss look like?	Loss at hop 4 and at every later hop.

Variant. A different return path for the probe replies inflates the latency shown at one hop.

> TRAP
>
> *Claim:* "Any loss in `mtr` is a fault at that hop." *Why it is wrong:* loss at a middle hop that does not carry on to later hops is the router declining to answer probes. *Correct statement:* only loss persisting to the destination counts.

11.9 Mock rounds SHOULD · ■■■

Three timed rounds follow, for practice with a friend. The partner reads the *interviewer lines* and the *hidden facts*, revealing a fact only when asked for it; the candidate covers the model transcript and attempts the round aloud, then compares. Each round lasts twenty to twenty-five minutes. The same ten-point rubric scores all three.

Criterion	Points	Full marks when
Symptom restated and measurable	1	The vague word is replaced by a test
Clarifying questions (scope, change, precision)	2	At least three, and assumptions stated
Structure and strategy named	2	A method is named and the layer under test stated each time
Technical accuracy	2	Commands, healthy outputs and inferences are right
Evidence for the root cause	1	It explains every symptom, including the unaffected cases
Fix with rollback and verification	1	The same test now passes
Prevention and communication	1	A guard, a monitor or a template is proposed

Round 1 · The Cisco vague-diagram round (cases 2 and 12)

Setting. The diagram is Figure 11.4 (p. 638), and the eight clarifying answers there are already known. The prompt is "PC-A cannot reach PC-B." Time: 25 minutes.

Hidden fact	Reveal only if asked
PC-B shows `169.254.37.8`	For PC-B's IP configuration
Static address 192.168.60.20 on PC-B cannot ping its gateway	After the candidate sets one
On SW2, `show interfaces trunk` toward R2 lists allowed VLANs `1,50`	For the trunk on SW2
R2 has `interface Gi0/1.60` with 192.168.60.1/24 and no helper; the DHCP server is 192.168.50.10	For R2's sub-interface

WORKED EXAMPLE

Model transcript.

C (0:30). "Restating: the ping from PC-A to PC-B times out. The file server behind R2 answers, so the provider circuit, OSPF and R2's routing work. PC-B is new, in VLAN 60. My first suspect is PC-B itself. May I see its IP configuration?"

I. "It shows 169.254.37.8."

C (3:00). "That is an automatic address, so PC-B has no lease and is not in 192.168.60.0/24; the ping fails before routing matters. Two things could cause it: its frames do not reach R2, or the DHCP request is not answered. I test Layer 2 first, with a static address, because it removes DHCP from the picture. I would set 192.168.60.20 and ping the gateway .1."

I. "It times out."

C (6:00). "So Layer 2 or the sub-interface. On SW2 I want `show vlan brief` and `show interfaces trunk`."

I. "VLAN 60 exists and Gi0/7 is in it. The trunk to R2 lists allowed VLANs 1 and 50."

C (8:00). "VLAN 60 is not carried to R2, so its frames never arrive. I would add it with `switchport trunk allowed vlan add 60`, noting that without `add` the list is replaced. Rollback is `remove 60`."

I. "Done. The static address now pings .1. Put PC-B back on DHCP: still 169.254."

C (11:00). "Layer 2 is now proven, so the broadcast Discover reaches R2 but nobody forwards it, because the server is in another VLAN. I would look at R2's sub-interface for a relay."

I. "No `ip helper-address`."

C (13:00). "Add `ip helper-address 192.168.50.10` under Gi0/1.60 and renew. Verification: PC-B holds a lease, pings its gateway, and PC-A pings PC-B. I would also check that Windows Firewall on PC-B permits ICMP, since that is the next cause if the ping still fails. Prevention: the VLAN playbook creates the VLAN, trunk entry, sub-interface and relay together, and finishes by testing a lease."

Watch for. Two stacked faults: a candidate who stops after the trunk fix has not re-tested. See Case 2 (p. 651) and Case 12 (p. 663).

Round 2 · The Google round (cases 14 and 17)

Setting. A spoken scenario with no diagram. Prompt: “Users say the internet is slow. Define slow, then troubleshoot end to end.” Time: 25 minutes.

Hidden fact	Reveal only if asked
About a quarter of users, no pattern by floor, same users each time	Scope
`curl` timings: DNS and connect equal, first byte slow	Measurements
Fabric: leaf-spine, four equal-cost uplinks	Environment
Leaf1 `Et49` shows rising FCS errors and Rx power -14.2 dBm	Counters, optics
Second complaint: a nightly backup to Frankfurt reaches 6 Mb/s on a 1 Gb/s link, RTT 80 ms	After the first fault is closed

WORKED EXAMPLE

Model transcript.

C (0:30). “‘Slow’ needs a number. I would separate latency, loss and throughput, and ask: how many users, which sites, measured how, and since when?”

I. “A quarter of users, always the same ones, no pattern by floor. `curl` shows a long time to first byte.”

C (3:00). “Consistent per user and per flow points at something that depends on the flow’s identity, which in a fabric is ECMP hashing: one of four equal paths is bad, and flows hashed to it suffer. DNS and connect times are equal, so the problem is loss during transfer, not setup. I would run `mtr` from a slow client to see where loss starts, then read error counters on the uplinks.”

I. “Loss starts at the leaf-to-spine hop. Et49 shows FCS errors.”

C (7:00). “A failing optic or fibre. I would check the optical power, drain the member so flows rehash (service recovers at once), then replace the optic. Verification is a delta of the counters and the users’ timings. Prevention: alert on error rate and optical power, with an automated drain.”

I. “Fine. A different team says their backup to Frankfurt reaches only 6 Mb/s on a 1 Gb/s link. Parallel streams are faster.”

C (12:00). “The round-trip is 80 ms, so the bandwidth-delay product is 10 MB. Six megabits per second at 80 ms is about 64 KB per round trip, exactly an unscaled window. I would check whether window scaling was negotiated, with `ss -ti` and a SYN capture at both ends, and the receive buffer with `sysctl net.ipv4.tcp_rmem`.”

I. “The SYN leaves with a scale of 7 and arrives without it.”

C (15:00). “A middlebox is stripping the option. Fix it, raise the buffer maxima to 16 MB, and verify a single stream approaches the line rate. Prevention: periodic long-haul throughput baselines.”

Watch for. Defining "slow" before acting, and linking *consistent per user* to hashing. See Case 14 (p. 666) and Case 17 (p. 670).

Round 3 · The Amazon network-development round (cases 8 and 16)

Setting. A mixed discussion. Prompt: "Our data centre's eBGP session to a cloud edge router is stuck, and separately a new API on a Linux VM is unreachable." Time: 25 minutes.

Hidden fact	Reveal only if asked
`show ip bgp summary` shows `Active`, new circuit	The session state
Our side peers from Loopback0; the cloud side expects the WAN address	The peering addresses
An edge ACL permits only ICMP and established TCP	Filtering
The API answers on `127.0.0.1:8080` only; nftables has no rule for 8080; the security group lacks 8080	Part two, in sequence

WORKED EXAMPLE

Model transcript.

I. "What does Active mean in BGP?"

C (1:00). "Despite its name, Active means the speaker is trying to open the TCP connection and failing. Idle means it is not trying, because it is down or has no route to the peer. OpenSent means TCP succeeded and an Open was sent but no valid answer has come. So Active is a TCP problem: address, path or filter."

I. "It has been Active since turn-up."

C (3:00). "Three questions: which address does each side use, is anything filtering port 179, and is it eBGP between directly connected peers? I would ping the peer from the *same source* the session uses, and test port 179 with `telnet peer 179 /source-interface`."

I. "Our side uses Loopback0; the ping from it fails, the ping from the WAN interface works. The edge ACL permits ICMP and established TCP."

C (6:00). "Then there are two faults. The cloud side has no route to our loopback, and it expects the WAN address; and its SYN to us is dropped by an ACL that allows only established traffic. I would remove `update-source` and permit TCP 179 in both directions, then expect the summary to show a prefix count. Hit counts on the ACL deny line would confirm it first."

I. "Good. Second problem: a new API on a Linux VM works locally and not remotely."

C (10:00). "Locally works, so the service is up. First, `ss -tlnp | grep 8080` to see the bind address. If it is 127.0.0.1, remote clients get a refusal; the fix is to bind to the interface address. After that, a timeout rather than a refusal means a drop, so I would read `nft list ruleset`, then run `tcpdump -ni eth0 port 8080` on the VM. If nothing arrives even with the host rule open, the packet is dying in the cloud, at the security group."

I. "How would you detect both before users did?"

C (15:00). "A synthetic probe from outside the host that opens port 8080 after each deployment; BGP session state and prefix-count alerts; and the ACL and security-group rules held in

> code, so that peers and ports are reviewed together."

Watch for. Precise state definitions, and an explanation that distinguishes refusal from timeout. See Case 8 (p. 658) and Case 16 (p. 669).

Labs and the fault injector

The cases are rehearsed best on a network that can be broken on demand. The lab below needs only Containerlab with FRR routers and Linux hosts; the Cisco switching cases (2, 3, 4, 11, 12) are reproduced in Packet Tracer or CML from the table.

> LAB
>
> `LAB 11.1 · 40 MIN PER FAULT` The break-fix arena. Build six FRR routers (R1 to R4, BR1, BR2), four Linux hosts and one nftables firewall in Containerlab, with OSPF in the core. A partner runs `breakit.sh N` without telling you which fault was injected; you restore service with SCALPEL and then run `check.sh`.

```
#!/bin/sh
# breakit.sh N : inject fault N (the numbers are the cases of 11.8)
case "$1" in
 5)  docker exec clab-campus-fw iptables -I FORWARD -p icmp -j DROP
     docker exec clab-campus-br1 ip link set dev tun0 mtu 1500 ;;
 6)  docker exec clab-campus-r2 ip link set dev eth2 mtu 9000 ;;
 7)  docker exec clab-campus-r4 vtysh -c 'conf t' -c 'router ospf' \
         -c 'passive-interface eth1' ;;
 10) docker exec clab-campus-h2 ip addr add 10.1.50.20/24 dev eth1 ;;
 16) docker exec clab-campus-h3 sh -c 'pkill -f http.server; \
         python3 -m http.server 8080 --bind 127.0.0.1 &' ;;
esac
```

```
#!/bin/sh
# check.sh : the grader; exits non-zero on the first unhealthy item
docker exec clab-campus-h1 ping -c2 -W1 10.1.2.10 >/dev/null || exit 1
docker exec clab-campus-r1 vtysh -c 'show ip ospf neighbor' | grep -q Full ||
exit 2
docker exec clab-campus-h1 curl -s -m3 http://10.1.2.10:8080/health || exit 3
echo healthy
```

> Key output after fault 6: `show ip ospf neighbor` on R1 prints `ExStart/DR`; after the fix, `Full/DR`. **What you should notice:** each fault leaves one decisive line, and `check.sh` passes only when every layer is healthy, so it also catches a stacked second fault.

Lab	Task	Time
L11.2	Packet Tracer or CML. Inject, then solve: case 2 (`switchport trunk allowed vlan 10`), case 3 (`shutdown` on `interface vlan 30`), case 4 (hard-code 100/full on one end), case 11 (`spanning-tree bpdufilter enable` on two ports and a patch cable between them), case 12 (remove `ip helper-address`)	30 min each
L11.3	Partner mock rounds using the three scripts above, scored with the rubric	3 × 25 min
L11.4	Write a blameless post-mortem for one injected fault, using the template of §11.7 (p. 647)	45 min

Mini-project (optional). Extend `breakit.sh` and `fixit.sh` to ten faults with an automatic grader that checks a ping matrix, OSPF and BGP state and an HTTP probe, and write one runb-

ook per fault in SCALPEL format. A line for a résumé: “Built a fault-injection lab with ten reproducible network failures and SCALPEL runbooks.”

Problem Set

Each solution opens with the 30-second answer. Tags give the companies that report the topic, the round, the difficulty (■□□ to ■■■) and the evidence: S for a specific reported question, T for a reported topic with our wording. Cases referred to are in §11.8 (p. 649).

Q11.1 Describe your general approach to troubleshooting a network problem.

CISCO · GOOGLE · AMAZON · TECH-1 · ■□□ · T

30-second answer: restate the symptom, ask scope, timeline and change questions, rank hypotheses, test them layer by layer saying the expected output, name a root cause that explains everything, fix with a rollback, then verify and prevent.

Answer. Name the method (SCALPEL, §11.2 (p. 636)) and the strategy: divide and conquer from Layer 3 by default. **Mistakes.** Listing commands without a purpose; no closure.

Q11.2 A user cannot reach a website. What are the first three checks and why?

CISCO · GOOGLE · TECH-1 · ■□□ · T

30-second answer: (i) scope and error text, to know whether it is one user, one site or all; (ii) `ping` the gateway and then a public IP, which tests Layers 1 to 3 in two steps; (iii) `nslookup` or `dig` the name, to separate DNS from connectivity.

Answer. Only then `curl -v` to see the TCP and HTTP layers. Each check discards a layer: a gateway ping proves the local link and VLAN, a public ping proves routing and NAT, and a successful IP with a failed name is Case 1 (p. 649).

Q11.3 What is the difference between a connection timeout and connection refused?

CISCO · GOOGLE · AMAZON · TECH-1 · ■□□ · S

30-second answer: a timeout means no answer, so the packet was dropped somewhere; a refusal is a RST from a reachable host with nothing listening, or a firewall REJECT.

Answer. On the wire, a timeout is SYN retransmissions at 1 s, 3 s and 7 s; a refusal is a SYN followed within a millisecond by a RST (Figure 11.7 (p. 645)). Hence a refusal proves the path works. **Follow-up.** For UDP, a closed port returns ICMP port unreachable.

Q11.4 Top-down, bottom-up and divide and conquer: when would you use each?

CISCO · GOOGLE · TECH-1 · ■□□ · T

30-second answer: bottom-up after a physical change, top-down when one application fails, and divide and conquer from Layer 3 when many hops are involved and there is no other clue.

Answer. Divide and conquer halves the search with each test; in Figure 11.6 (p. 640) two pings isolate one of five links. Add follow the packet for routing faults, compare with working when one of two similar things fails, and swap when counters implicate hardware. **Mistake.** Always starting at Layer 1.

Q11.5 What do "up/down" and "administratively down" mean on a Cisco interface?

CISCO · JUNIPER · TECH-1 · ■□□ · T

30-second answer: the first word is the physical state and the second the line protocol (Layer 2); "administratively down" means someone typed `shutdown`.

- *up / up*: healthy.
- *up / down*: carrier present, Layer 2 failing (encapsulation, keepalives, clocking); for an SVI, no active port in the VLAN.
- *down / down*: no signal (cable, remote shut, wrong optic).
- *administratively down*: shut by configuration.
- *err-disabled*: a protection (BPDU Guard, port security) closed it.

Q11.6 A host gets a 169.254.x.x address. What does that tell you?

CISCO · AMAZON · TECH-1 · ■□□ · T

30-second answer: the host asked for DHCP and received no answer, so it assigned itself a link-local address (APIPA).

Answer. Layer 2 may be fine; test with a static address. Then check the relay on the SVI, the scope, snooping trust and the server. See Case 12 (p. 663). **Mistake.** Assuming the DHCP server is down before checking the relay.

Q11.7 How do you check whether a switch has learned a host's MAC, and on which port?

CISCO · JUNIPER · ARISTA · TECH-1 · ■□□ · T

30-second answer: `show mac address-table address <mac>` **on IOS,** `show ethernet-switching table` **on Junos,** `bridge fdb show` **on Linux; the host must have sent a frame recently.**

Answer. Find the MAC from the host (`ipconfig /all`) or from `show ip arp` . If absent, generate traffic (a ping) and look again; entries age out after 300 s. The port shown leads you one hop towards the host; repeat on that switch until you reach an access port.

Q11.8 Two hosts in the same VLAN on different switches cannot communicate. Walk through it.

CISCO · JUNIPER · ARISTA · TECH-1 · ■■□ · T

30-second answer: the VLAN is missing somewhere between the two ports: check the trunk allowed list, that the VLAN exists on both switches, the native VLAN, and spanning-tree state for that VLAN.

Answer. `show interfaces trunk` is the decisive command; compare the allowed, active and forwarding lists. The cause in Case 2 (p. 651) was `allowed vlan 10` replacing the list. **Follow-up.** After the fix, check that both MACs are learnt.

Q11.9 OSPF neighbours are stuck in ExStart. Likely cause and fix?

CISCO · JUNIPER · NOKIA · ARISTA · AMAZON · TECH-1 · ■■□ · T

30-second answer: an interface MTU mismatch, because a router rejects database descriptions advertising a larger MTU than its own; make the MTUs equal.

Answer. Prove it with `debug ip ospf adj` ("larger interface MTU") and `show interfaces | include MTU` . `ip ospf mtu-ignore` masks the problem. Other causes of a stall: duplicate router ID, unicast loss on non-broadcast networks. Full case: Case 6 (p. 656).

Q11.10 List everything that must match for two OSPF routers to become neighbours.

CISCO · JUNIPER · NOKIA · TECH-1 · ■■□ · T

30-second answer: subnet and mask, area ID, hello and dead timers, authentication, stub flags, a compatible network type, a unique router ID, a non-passive interface, and nothing blocking 224.0.0.5 and 224.0.0.6.

Answer. MTU must also match for the exchange to complete. These need not match: process ID, priority, cost, interface name. Use the list as the hypothesis list (Case 7 (p. 657)).

Q11.11 BGP Active versus Idle: what does each mean, and how do you troubleshoot?

AMAZON · GOOGLE · CISCO · JUNIPER · ARISTA · TECH-1 · ■■□ · T

30-second answer: Idle means the speaker is not attempting a connection (down, no route, or in back-off); Active means it is trying to open TCP and failing, so check the address, the path and TCP 179 filters.

Answer. Ping and `telnet peer 179` from the same source the session uses; check the update-source, `ebgp-multihop` for loopbacks, and ACL hit counts. NOTIFICATION messages indicate AS, timer or MD5 faults. Full case: Case 8 (p. 658).

Q11.12 BGP is Established but a prefix is not advertised. Why?

AMAZON · GOOGLE · ARISTA · JUNIPER · TECH-1 · ■■□ · T

30-second answer: a `network` statement originates a prefix only if an exact match is in the routing table; otherwise check outbound policy, then the peer's inbound filters.

Answer. `show ip route <prefix> <mask>` , `show ip bgp <prefix>` , `advertised-routes` . If an internal router learns it but does not use it, the next hop is unreachable: use `next-hop-self` (Case 9 (p. 659)). **Follow-up.** RPKI and IRR filters at the provider.

Q11.13 Small packets work and large packets fail. Explain and fix.

GOOGLE · AMAZON · CISCO · JUNIPER · TECH-1 · ■■□ · T

30-second answer: the path MTU is smaller than the packet and the ICMP "fragmentation needed" message is lost, so a black hole forms; allow ICMP type 3 code 4 and clamp the MSS.

Answer. Test with `ping -M do -s <size>`; find the largest passing size and add 28 for the MTU. Tunnels are the usual cause (Case 5 (p. 654)). Clamp the MSS to MTU minus 40. For IPv6, Packet Too Big must not be filtered.

Q11.14 An interface shows increasing CRC errors. What causes it, and how do you prove which?

CISCO · QUALCOMM · ARISTA · BROADCOM · TECH-1 · ■■□ · T

30-second answer: corrupted frames from a bad cable, a weak optic, interference or a duplex mismatch; take two snapshots to prove the counter is rising, check duplex and optical power, then swap the cable.

Answer. Duplex mismatch is shown by late collisions on the other side and a half-duplex entry. An optic fault is shown by `show interfaces transceiver` (Rx power below the threshold). Clean the fibre or swap the part after saving the counters.

Q11.15 Duplex mismatch: symptoms on each side?

CISCO · QUALCOMM · BROADCOM · TECH-1 · ■■□ · T

30-second answer: the half-duplex side shows late collisions and runts; the full-duplex side shows CRC and frame errors; bulk TCP is very slow while interactive traffic seems fine.

Answer. Cause: one end forced and the other on auto. Fix both to auto. Full case: Case 4 (p. 653).

Q11.16 A server is intermittently unreachable for 30 seconds at a time. Causes?

CISCO · AMAZON · GOOGLE · TECH-1 · ■■□ · T

30-second answer: a duplicate IP address, a flapping link or a Layer 2 loop, ARP or lease expiry, a bad ECMP member, or a first-hop redundancy split-brain; correlate the outage times with logs.

Answer. The decisive checks are `%IP-4-DUPADDR`, `show ip arp` flipping between two MACs, and MAC-flap messages (Case 10 (p. 661)). Ask whether the pattern is periodic, because a period often equals a timer.

Q11.17 How would you detect and stop a broadcast storm?

CISCO · JUNIPER · NOKIA · ARISTA · TECH-1 · ■■□ · T

30-second answer: detect by a broadcast-rate alarm, 100 per cent CPU and MAC-flap logs; stop it by shutting the offending port, then prevent with BPDU Guard, storm control and loop protection.

Answer. Mitigate first by shutting the port that shows the highest broadcast input rate. Find the loop from the MAC flaps between two ports. See Case 11 (p. 662). Frames have no TTL, which is why the storm persists.

Q11.18 How would you use `tcpdump` to decide whether a firewall is dropping traffic?

GOOGLE · AMAZON · ARISTA · TECH-1 · ■■□ · T

30-second answer: capture on both sides of the firewall with the same filter; if the SYN is seen on the inside interface and not on the outside, the firewall drops it, and if the SYN leaves but nothing returns, the fault is beyond.

Answer. Use `tcpdump -ni <if> 'host X and port Y'` at each interface, in the same minute. A SYN seen at the server with no SYN-ACK leaving points at the host; a SYN-ACK seen leaving but not arriving back points at the return path or the firewall state (Case 13 (p. 664)).

Q11.19 IPsec Phase 1 is up, Phase 2 is down. Diagnose.

CISCO · JUNIPER · NOKIA · TECH-1 · ■■□ · T

30-second answer: Phase 1 only proves the peers authenticate; Phase 2 fails on mismatched traffic selectors or proxy IDs, transform sets, PFS groups, or lifetimes.

Answer. Read the debug (`TS_UNACCEPTABLE` or `NO_PROPOSAL_CHOSEN`) and compare both configurations. Once the child SA is up, if encapsulation counts rise but decapsulation stays zero, check the NAT exemption and the return route (Case 15 (p. 667)).

Q11.20 Traceroute shows * * * at hop 5 but the destination is reachable. Explain.

GOOGLE · CISCO · AMAZON · TECH-1 · ■□□ · T

30-second answer: that router does not answer TTL-exceeded probes (it filters or rate-limits ICMP) but still forwards transit traffic, so it is not a fault.

Answer. Traceroute depends on the router's co-operation; forwarding does not. Confirm by observing that later hops answer and the destination is reached. See Case 18 (p. 671). **Mistake.** Reporting hop 5 as the break.

Q11.21 (Cisco style) Two networks are joined by routers and switches; PC-A cannot reach PC-B. Ask questions and solve it.

CISCO · JUNIPER · TECH-2 · ■■■ · S

30-second answer: restate the symptom; ask for addresses, what fails, what does work, whether it ever worked, the routing protocol, the change history and the access to outputs; state the scenario back in one sentence; then walk Layer 3 first by pinging gateways.

Answer. The full transcript with timestamps is Round 1 (p. 673) of §11.9 (p. 672), where two stacked faults (a trunk list and a missing relay) are found. The scoring turns on the clarification: eight questions defined the scenario in Figure 11.4 (p. 638). **Mistake.** Proposing a fix after the first finding.

Q11.22 (Google style) Users report "the internet is slow." Define slow, then troubleshoot end to end.

GOOGLE · TECH-2 · ■■■ · S

30-second answer: turn "slow" into latency, loss or throughput with a measurement and a scope; then compare a slow and a fast client with `curl -w` timings, `mtr` and counters, and follow the evidence along the path.

Answer. The timing split (DNS, connect, first byte, transfer) shows which phase is slow. Transfer-phase slowness with equal setup times means loss or a window limit. The transcript is Round 2 (p. 674).

Q11.23 Some users are always slow and others never, in a leaf-spine fabric. How can ECMP cause that, and how do you find the bad link?

GOOGLE · MICROSOFT · AMAZON · ARISTA · TECH-2 · ■■■ · T

30-second answer: ECMP hashes each flow's five-tuple onto one of several equal paths, so a user whose flows hash to a lossy path is slow every time; find it by error counters and optics on each member.

Answer. Consistency per user, with no pattern by floor, is the signature. Confirm with `mtr` from a slow client, then `show interfaces counters errors` and `show interfaces transceiver` on each uplink. Drain the member. See Case 14 (p. 666).

Q11.24 After adding a second ISP, some TCP sessions break. Why, and how do you fix it architecturally?

CISCO · JUNIPER · NOKIA · GOOGLE · AMAZON · TECH-2 · ■■■ · T

30-second answer: replies return through a different stateful firewall that has no state for the session and drops them; make the paths symmetric or share state.

Answer. Options: source NAT per ISP, policy routing, an active-standby pair, or a cluster with state synchronisation. Avoid ECMP across independent stateful devices. Evidence: "no connection" drops on one firewall (Case 13 (p. 664)).

Q11.25 A single TCP flow over a long link is slow, but parallel flows are fine. Diagnose.

GOOGLE · MICROSOFT · AMAZON · BROADCOM · TECH-2 · ■■■ · T

30-second answer: a single stream is window-limited: throughput cannot exceed window divided by round-trip time, so compare the bandwidth-delay product with the window and check scaling and buffers.

Answer. 64 KB over 80 ms is about 6.5 Mb/s, against a BDP of 10 MB on 1 Gb/s. Check `wscale` in `ss -ti`, the SYN at both ends and `tcp_rmem`. Parallel streams each carry a window, hence they sum up. See Case 17 (p. 670).

Q11.26 Production is down for 2,000 users after a change you made ten minutes ago. What do you do, minute by minute?

GOOGLE · AMAZON · MICROSOFT · TECH-2 · HM · ■■■ · S

30-second answer: declare an incident, roll the change back, tell stakeholders, verify recovery, and only then find the cause and write a blameless post-mortem.

Minute	Action
0	Confirm the outage from monitoring, not only the report; state the time of the change
1	Declare Sev 1; name a commander, an operator and a scribe
2	Save current state: `show running-config`, logs, counters, for the post-mortem
3	Roll back the change (`configure replace` or the saved configuration)
5	Post the first update: impact, action under way, next update time
7	Verify with the test that failed (pings, a user check, monitoring green)
10	If not restored, fail over or drain; keep updates every 30 minutes
After	Reproduce in a lab, find the root cause, hold the blameless review (§11.7 (p. 647))

Mistake. Investigating before rolling back, or rolling back without saving the evidence.

Q11.27 How do you troubleshoot a problem you cannot reproduce?

GOOGLE · AMAZON · TECH-2 · ■■■ · T

30-second answer: make the fault observable: add monitoring and a rolling packet capture at the suspected point, correlate the reported times with logs and changes, and form hypotheses from the pattern.

Answer. (i) Ask for exact timestamps and affected hosts. (ii) Keep a circular capture (`tcpdump -C ... -W ...`) or flow records at both ends. (iii) Look for periodicity, which suggests timers, lease expiry or scheduled jobs. (iv) Compare the affected with the unaffected. (v) Change one variable at a time and record it.

Q11.28 Write a script that detects interface-error increases across 100 devices and alerts.

AMAZON · GOOGLE · ARISTA · TECH-2 · ■■■ · T

30-second answer: poll `show interfaces` concurrently, parse the error counter per interface, compare it with the previous poll and alert on a delta above a threshold; handle counter resets.

Code (Python). The fetch is mocked; in production replace it with a Netmiko `send_command`, or a gNMI subscription (Pillar 10 (p. 586)).

```
import json, pathlib, re
from concurrent.futures import (
    ThreadPoolExecutor as Pool)
STATE, LIMIT = pathlib.Path("c.json"),
50
MOCK = {   # host -> "show interfaces"
 "sw1": "Gi1/0/10 is up, line protocol"
        " is up\n 2217 input errors\n",
 "sw2": "Gi1/0/1 is up, line protocol"
        " is up\n 12 input errors\n"}

def fetch(host):     # real: Netmiko
    return MOCK[host]

def parse(text):
    out, name = {}, None
    for ln in text.splitlines():
        m = re.match(r"(\S+) is
(up|down)", ln)
        if m:
            name = m.group(1)
        m = re.search(r"(\d+) input
err", ln)
        if m and name:
            out[name] = int(m.group(1))
    return out

def alerts(now, old):
    for h, ifs in now.items():
        for i, n in ifs.items():
            p = old.get(h, {}).get(i, n)
            d = n - p if n >= p else n
            if d > LIMIT:   # d=n:
cleared
                yield h, i, d

hosts = sorted(MOCK)
with Pool(20) as pool:
    now = dict(zip(hosts, map(parse,
          pool.map(fetch, hosts))))
old = {}
if STATE.exists():
    old = json.loads(STATE.read_text())
for h, i, d in alerts(now, old):
    print(f"ALERT {h} {i}: +{d} errors")
STATE.write_text(json.dumps(now))
```

With a saved state of 1,000 errors for `sw1` Gi1/0/10 and 10 for `sw2` Gi1/0/1, the output is `ALERT sw1 Gi1/0/10: +1217 errors`; `sw2` rose by 2 and stays quiet. The first run only stores

a baseline. **Mistake.** Alerting on the absolute counter, which stays high for ever.

Q11.29 Describe a time you troubleshot a hard problem.

AMAZON · GOOGLE · CISCO · HM · ■■■ · T

30-second answer: use STAR with SCALPEL as the Action: situation and impact, your task, the clarifying questions and layer walk with evidence, and the result with prevention.

Answer. Choose a case with a stacked fault, as in Case 7 (p. 657) or Case 16 (p. 669): it shows that you re-tested after each fix. State numbers (users affected, minutes to mitigate) and the guard you added. Amazon's Dive Deep and Ownership principles are the target (Pillar 12 (p. 687)).

Q11.30 Design monitoring so that the last five cases would have been detected before users noticed.

GOOGLE · AMAZON · MICROSOFT · TECH-2 · ■■■ · T

30-second answer: monitor symptoms at the user layer (synthetic probes, DNS and DHCP success), plus state and rates at the device layer (neighbours, sessions, error and broadcast rates, optics).

Case	Signal that would have fired
14, ECMP optic	CRC delta and optical power threshold on every uplink
15, VPN	Child SA and encaps/decaps rate per tunnel
16, Linux	External synthetic probe on the port after each deploy
17, window	Long-haul throughput baseline
18, mtr	None needed; a runbook answers the customer

Answer. Alert on rate and delta, not on absolute counters; route alerts by severity; test the monitors by injecting the fault (Pillar 10 (p. 586)).

Q11.31 Cold case A: "The branch OSPF neighbour flaps every few minutes."

CISCO · JUNIPER · TECH-2 · ■■■ · T

30-second answer: the neighbour reaches Full on small packets, then drops when large database updates are lost on a tunnel whose real MTU is lower than its configured one; match the tunnel MTU and clamp, and let ICMP through.

Hidden facts (reveal only if asked). Ping works; `ping size 1500 df-bit` to the HQ tunnel address fails; the log shows `Neighbor Down: Dead timer expired` a few seconds after `Loading`; HQ is an IPsec peer; the tunnel interface has MTU 1500.

Model solution. (i) Clarify: since when, what changed (an IPsec tunnel replaced a leased line), scope (one branch). (ii) Hypotheses: MTU on the tunnel, timer mismatch, authentication. (iii) Layer walk: `show ip ospf neighbor` cycles between Loading and Down; small pings pass, `ping 10.9.9.1 size 1500 df-bit` fails and `size 1400` passes. Hello packets are small and survive, while full-size LSUs are lost, so the dead timer expires. (iv) Fix: `ip mtu 1400` on both tunnel ends and `ip tcp adjust-mss 1360`; never `mtu-ignore`. (v) Verify: Full for an hour, LSDB equal. (vi) Prevent: tunnel template with MTU, and a pre-check. Related: Case 5 (p. 654), Case 6 (p. 656).

Q11.32 Cold case B: "After enabling jumbo frames, NFS hangs for some hosts only."

GOOGLE · AMAZON · ARISTA · TECH-2 · ■■■ · T

30-second answer: one switch or trunk in the storage VLAN still has MTU 1500, so large frames are dropped silently at Layer 2 (no ICMP is generated), while small operations work; find the hop with `ping -M do -s 8972` and check each port's MTU.

Hidden facts. `ls` works and `cp` of a large file hangs; affected hosts hang to the same server; the storage VLAN crosses three switches; one was replaced last week.

Model solution. (i) Clarify: which hosts, since when (jumbo enabled yesterday), what is common (the replaced switch). (ii) Hypotheses: an MTU gap in the path; NIC settings; NFS server. (iii) Walk: from a failing host `ping -M do -s 8972 <server>` has 100% loss and `-s 1472` passes. Bisect by pinging each switch's management address with the large size, or compare `show interfaces | include MTU` on each trunk; the replaced switch shows `MTU 1500`. A Layer 2 switch drops an oversize frame without sending ICMP, so the sender can never learn the limit. (iv) Fix: `system mtu jumbo 9216` or the platform equivalent, then reload if needed. (v) Verify: the 8972-byte ping passes and the copy completes. (vi) Prevent: a pre-change check that pings at the jumbo size across the VLAN. Related: Case 5 (p. 654).

Revision Spread

THE 15-MINUTE VERSION

SCALPEL. Symptom: restate it as a measurement. **C**larify: scope, timeline and change, precision, environment, evidence. **A**ssess: annotate the path, rank three hypotheses. **L**ayer walk: command, healthy output, inference; divide and conquer from Layer 3. **P**inpoint: a root cause that explains every symptom. **E**xecute: minimal fix with a rollback. **L**ock in: verify, prevent, communicate. A failed hypothesis returns you to A.

Ten facts. (1) Three first questions: how many, since when, what changed. (2) Timeout means a drop; refusal means a RST from a live host; reset mid-stream means a middlebox or crash. (3) Ping proves ICMP only. (4) Counters are cumulative: take deltas. (5) ExStart means MTU; Init means one-way hellos; BGP Active means TCP failing. (6) Small packets pass and large fail: MTU and blocked ICMP. (7) In a live outage, roll back before diagnosing. (8) `allowed vlan` without `add` replaces the list. (9) Only `mtr` loss that persists to the destination counts. (10) Fix one fault, re-test: stacked faults are common.

Three figures to redraw. The SCALPEL card with its loop (Figure 11.2); the three ladders, timeout, refusal, reset (Figure 11.7); the strategy chooser (Figure 11.5).

Command matrix, condensed

Layer	IOS XE	Linux
L1	`show interfaces`, `show interfaces status`, `... transceiver`	`ethtool eth0`, `ethtool -S eth0`, `ip -s link`
L2	`show mac address-table`, `show interfaces trunk`, `show spanning-tree`	`ip neigh`, `bridge fdb show`, `bridge vlan`
L3	`show ip route X`, `show ip ospf neighbor`, `ping X source Y`	`ip route get X`, `ping -M do -s N`, `mtr -rwzbc 100 X`
L4	`show access-lists`, `show ip nat translations`, `show policy-map control-plane`	`ss -tlnp`, `ss -ti`, `nft list ruleset`
L7	`show ip dhcp binding`, `show ip dhcp snooping`	`dig`, `curl -v`, `tcpdump -ni eth0`

Look-alikes

Observed	Means	First test
Timeout	Drop on the path	Capture at both ends
Refused	RST from live host	`ss -tlnp`
Reset mid-stream	Idle timer or crash	Who sent the RST
By IP only	DNS	`dig @server`
One direction	Asymmetry, ACL, NAT	Captures both ends
Large fails, small works	MTU	`ping -M do -s`
Same users always slow	One ECMP member	Counters per member
Single flow slow	Window limit	`ss -ti`, BDP
Intermittent	Duplicate IP, flap	Correlate with logs

Protocol checklists

OSPF must match. Subnet and mask, area, hello and dead, authentication, stub flags, network type, unique router ID, not passive, MTU. **BGP stuck states.** Idle: down or no route. Active: TCP failing (address, path, port 179). OpenSent: no valid reply. Established with a number: healthy. **DHCP.** Discover, Offer, Request, Acknowledge; relay adds giaddr; 169.254.0.0/16 on failure. **TCP signatures.** SYN repeated: drop. SYN then RST: no listener. Zero window: receiver slow. Duplicate ACKs: loss. **Overheads (bytes).** 802.1Q 4, MPLS 4 per label, GRE 24, VXLAN 50, IPsec ESP about 50 to 73. MSS is the MTU minus 40.

Incident checklist. Mitigate, communicate (first update within 10 minutes), diagnose, fix, post-mortem. Roles: commander, operator, scribe, communications. Save the evidence before rolling back. Name the system gap, not the person.

Recognition lines for the eighteen cases

Case	Recognition line
1	By IP works, by name fails: DHCP option 6 or DNS
2	Same VLAN across a trunk fails: allowed list replaced
3	One VLAN cannot leave: SVI shut, VLAN missing on the core
4	Bulk slow after a swap: half and full duplex mismatch
5	Large packets hang over the VPN: ICMP blocked, no MSS clamp
6	ExStart and Exchange loop: MTU mismatch
7	No neighbour at all: passive, area, authentication
8	BGP Active: loopback source and ACL on port 179
9	Established, prefix absent: no exact route, next hop unreachable
10	Server drops for a minute: duplicate IP
11	Floor down, LEDs flashing: loop with BPDU filter
12	169.254 in a new VLAN: no relay, untrusted uplink
13	Random sessions break: asymmetric return through a stateless firewall
14	Same users slow: one ECMP member with a bad optic
15	Tunnel up, no data: selectors and NAT exemption
16	Refused, then timeout: loopback bind, host firewall, security group
17	One stream 6 Mb/s: window scaling stripped
18	Loss at one hop only: CoPP, not a fault

Q	A
First SCALPEL step?	Symptom: restate it as a measurement
Which step does a failed hypothesis return to?	Assess
The three highest-yield questions?	How many, since when, what changed
Default strategy?	Divide and conquer from Layer 3
What does a timeout mean?	A drop on the path
What does a refusal mean?	RST: host reached, nothing listening
Reset after exactly 5 minutes idle?	Middlebox idle timer
Does ping prove the application works?	No, only ICMP reachability
Are interface counters rates?	No, cumulative: take deltas
`late collision` points to?	Duplex mismatch, on the half-duplex side
CRC on the full-duplex side points to?	The same mismatch, or cable or optic
`up/down` on an SVI?	VLAN has no active port, or VLAN missing
OSPF stuck in ExStart?	MTU mismatch
OSPF stuck in Init?	One-way hellos (ACL, multicast, passive)
BGP Active?	TCP connection failing
BGP state column shows a number?	Established; the prefixes received
A `network` statement originates when?	The exact prefix is in the RIB
iBGP route “inaccessible”?	Next hop unreachable: `next-hop-self`
`169.254.x.x`?	No DHCP reply
Which log names a duplicate IP?	`%IP-4-DUPADDR`
`%SW_MATM-4-MACFLAP_NOTIF`?	A MAC moving between ports: a loop or a duplicate
Why does a loop never end?	Ethernet frames have no TTL
`bpdufilter` on an edge port?	Loop detection off; use BPDU Guard
PMTUD black hole needs?	ICMP type 3 code 4 dropped
MSS on a 1400-byte path?	1360
Asymmetric path and a stateful firewall?	Reply dropped: no session
Why are the same users always slow in ECMP?	The five-tuple hashes to one path
64 KB window at 80 ms?	About 6.5 Mb/s
`mtr` loss at one middle hop only?	Rate-limited ICMP; not a fault

PART I · PILLAR 12

12

Supporting Subjects

Everything else that decides the offer: aptitude, the computer-science core, design, behaviour and your own story.

12A · Aptitude and online assessments (12.1 to 12.7)	MUST
12B · Object-oriented programming (12.8)	MUST
12C · Database management systems (12.9)	MUST
12D · Computer organisation (12.10)	SHOULD
12E · Light system design for networking roles (12.11)	MUST
12F · Behavioural interviews, STAR and Amazon principles (12.12)	MUST
12G · Résumé and project deep-dives (12.13 and 12.14)	MUST

Pillars 1 to 11 teach what a network engineer must know. This pillar teaches what stands between that knowledge and the offer: the online assessment that creates the shortlist, the core-computing viva that every campus panel adds, the design conversation, the behavioural round and the document that starts all of them. None of it is difficult in the way BGP is difficult; all of it is lost by the unprepared through speed, vocabulary and structure. We therefore treat each topic as a drill: a formula card, a shortcut, and worked examples that can be redone on paper in two minutes.

Seven sub-chapters follow, lettered 12A to 12G. Aptitude (12A) is the largest, because it is the first filter in nearly every Indian campus drive and the only one that is purely arithmetic. Networking examples recur through the pillar, with the fictional organisation *Campus Net* from earlier pillars appearing wherever a story needs a setting.

12.1 Online assessments: formats, pacing and guessing

MUST · ■□□

An online assessment, or OA, is a timed test taken on a browser, usually with a webcam running, that decides who is interviewed. The format varies by company and by year, so the table below records what candidates have reported and should be checked with your placement cell before each drive.

Platform	What is reported	Consequence for you
HackerRank	Hidden test cases, partial marks per case, language chosen per question, tab switches logged	Practise reading input fast; submit a brute force early
AMCAT	Adaptive English, quantitative and logical sections, then coding (Automata)	Early accuracy matters more than speed
Mercer Mettl	Employer sets section order, timing, negative marking and locking	Read the instruction screen; a locked section cannot be revisited
CoCubes	Modules for aptitude, fundamentals, coding and English; usually no negative marking	Attempt everything
Company papers (Cisco, Juniper, Qualcomm, Nokia)	30 to 40 MCQs on aptitude and core subjects, plus one to three coding problems, in 60 to 90 minutes	Budget about 45 seconds per MCQ

For the company-by-company papers see Part III; here we need only the consequence, which is that every format rewards the same three habits: a fixed time budget, a two-pass method and sensible guessing.

> BIG IDEA
>
> An assessment is a time-allocation problem before it is a knowledge problem, because the marks lost to one stubborn question exceed the marks gained by solving it.

The two-pass method

In pass one we spend at most 45 seconds on each question, answer those that yield, and flag the rest. In pass two we return to the flagged questions in order of how close they felt. A final five minutes fills every remaining blank when there is no penalty. This works because question difficulty is uneven and marks are not: an easy question and a hard one are worth the same.

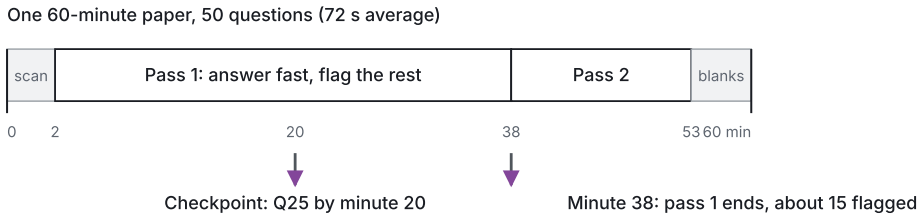


Figure 12.1 A pacing plan for a 50-question, 60-minute paper. If you are behind a checkpoint, skip faster rather than think harder.

Negative marking and guessing

Suppose a paper has four options and deducts a quarter mark for a wrong answer. A blind guess has expected value $1/4 \times 1 - 3/4 \times 1/4 = +0.0625$, so it is marginally positive. Remove one option you know to be wrong and the value becomes $1/3 \times 1 - 2/3 \times 1/4 = +0.167$. With a penalty of one third, the blind guess is exactly neutral, and one eliminated option again turns it positive. Hence the rule: guess whenever you can eliminate at least one option, and leave a question blank only when the penalty is as large as the reward.

> **TRAP**
>
> *Claim:* "Never guess in a paper with negative marking." *Why it is wrong:* the penalty is usually smaller than the reward, so the expectation of a guess after one elimination is positive. *Correct statement:* compute the expectation once, for the stated penalty, and apply it to every question; do not decide mood by mood.

The CS-MCQ section

Most company papers add 15 to 25 questions on computing fundamentals. They are fast marks if the rules are fresh, and the rules are taught elsewhere in this book, so the table gives only the one-line rule and its home.

Topic	One-line rule	Home
C output	Look first for undefined behaviour, then precedence, then `sizeof` and integer promotion	§2.15 (p. 128)
Scheduling	Turnaround = completion − arrival; waiting = turnaround − burst	§7.4 (p. 423)
Page replacement	FIFO evicts oldest, LRU evicts least recently used, Optimal evicts the page used farthest ahead	§7.7 (p. 435)
Deadlock	Four conditions together: mutual exclusion, hold and wait, no preemption, circular wait	§7.6 (p. 431)
Subnetting	Block size = 256 − mask octet; usable hosts = $2^h - 2$	§3.4 (p. 174)
Normal forms, SQL	See this pillar	§12.9 (p. 709)

CHECK YOURSELF

1. A paper has five options and a penalty of 1/4 per wrong answer. What is the expected value of a blind guess? [1]
2. With the same paper, you eliminate two options. What is the expected value? [2]
3. In a 90-minute paper with 40 MCQs and two coding problems, you allot 30 minutes to MCQs. What is the time per MCQ? [3]

12.2 Number system, remainders, HCF and LCM MUST · ■□□

Number-system questions look like tricks but are applications of three ideas: factorisation, cycles and the remainder. They also overlap with networking, because base conversion and divisibility by powers of two underlie subnet masks (§3.4 (p. 174)).

FORMULA CARD: DIVISIBILITY AND DIVISORS

- Divisible by 2, 5, 10: last digit even, 0 or 5, 0. By 4 and 8: last two and last three digits. By 3 and 9: digit sum. By 11: (sum of odd-place digits) − (sum of even-place digits) is 0 or a multiple of 11.
- By 7: remove the last digit d, subtract $2d$ from what remains, repeat; the result is divisible by 7 exactly when the original was.
- If $N = p^a q^b r^c$, the number of divisors is $(a + 1)(b + 1)(c + 1)$ and their sum is the product of $(p^{a+1} - 1)/(p - 1)$ over the primes.
- HCF × LCM = product of the two numbers (two numbers only, not three).

Shortcut. To count divisors, write the factorisation once and do not list them; to test 7, run the doubling trick twice at most.

WORKED EXAMPLE

Divisors of 360.

1. Factorise: $360 = 2^3 \times 3^2 \times 5$.
2. Count: $(3 + 1)(2 + 1)(1 + 1) = 4 \times 3 \times 2 = 24$ divisors.
3. Sum: $(2^4 - 1)/(2 - 1) \times (3^3 - 1)/(3 - 1) \times (5^2 - 1)/(5 - 1) = 15 \times 13 \times 6$.

Result: 24 divisors with sum 1,170.

WORKED EXAMPLE

Is 2,597 divisible by 7?

1. Last digit 7; the rest is 259. Compute $259 - 2 \times 7 = 245$.
2. Repeat: last digit 5, rest 24: $24 - 10 = 14$.
3. 14 is a multiple of 7.

Result: yes; 2,597 = 7 × 371.

FORMULA CARD: CYCLES, REMAINDERS, ZEROS AND BASES

- Unit digits repeat: 0, 1, 5, 6 never change; 4 and 9 have cycle 2; 2, 3, 7, 8 have cycle 4. Take the exponent mod the cycle length, and read 0 as the full cycle.
- Remainders multiply: $(a \times b) \bmod n = ((a \bmod n) \times (b \bmod n)) \bmod n$. If $a^k \equiv 1 \pmod n$ for a small k, reduce the exponent mod k.
- Trailing zeros of $n!$ = $\lfloor n/5 \rfloor + \lfloor n/25 \rfloor + \lfloor n/125 \rfloor + \ldots$
- Base conversion: group binary digits in fours for hex and in threes for octal, from the right.

Shortcut. For a large power, first find the cycle, then reduce the exponent. Never compute the power.

WORKED EXAMPLE

Unit digit of $3^{47} \times 8^{22}$.

1. The cycle of 3 is 3, 9, 7, 1. Since 47 mod 4 = 3, 3^{47} ends in 7.
2. The cycle of 8 is 8, 4, 2, 6. Since 22 mod 4 = 2, 8^{22} ends in 4.
3. Multiply the digits: 7 × 4 = 28.

Result: unit digit 8.

WORKED EXAMPLE

(a) Remainder of 2^{100} divided by 7. (b) Trailing zeros of 125!. (c) 1101101_2 in hex.

1. a. $2^3 = 8 \equiv 1 \pmod 7$. Since $100 = 3 \times 33 + 1$, $2^{100} \equiv 2^1 = 2$.
2. b. 125/5 = 25, 125/25 = 5, 125/125 = 1, so 25 + 5 + 1 = 31.
3. c. Pad to 0110 1101 and read each group: 6 and D.

Result: (a) 2; (b) 31 zeros; (c) 0x6D, which is 109.

FORMULA CARD: HCF AND LCM

- HCF is the largest number dividing all the numbers; LCM is the smallest that all divide. Use prime factors: take the lowest powers for HCF and the highest for LCM.
- "Greatest number that divides a and b leaving remainders r and s": HCF of $(a - r)$ and $(b - s)$, provided the answer exceeds both remainders.
- "Smallest number leaving the same remainder r on division by x, y, z": LCM$(x, y, z) + r$.
- Events that repeat every x, y and z units coincide every LCM units.

Shortcut. For HCF by hand use Euclid: replace (a, b) by $(b, a \bmod b)$ until the remainder is zero.

WORKED EXAMPLE

Three timers fire every 12, 18 and 30 seconds. They fire together at 09:00:00. When next, and how often in one hour?

1. $12 = 2^2 \cdot 3$, $18 = 2 \cdot 3^2$, $30 = 2 \cdot 3 \cdot 5$. The highest powers give LCM $= 2^2 \times 3^2 \times 5 = 180$ seconds.
2. They next coincide at 09:03:00.
3. In 3,600 seconds there are 3,600/180 = 20 further coincidences, plus the first at 09:00:00.

Result: after 3 minutes; 21 times including the start.

WORKED EXAMPLE

Find the greatest number that divides 414 and 536 leaving remainders 6 and 8.

1. Subtract the remainders: 414 − 6 = 408 and 536 − 8 = 528.
2. Euclid: 528 mod 408 = 120; 408 mod 120 = 48; 120 mod 48 = 24; 48 mod 24 = 0. The HCF is 24.
3. Check that 24 exceeds both remainders: 24 > 8. Verify: 414 = 17 × 24 + 6 and 536 = 22 × 24 + 8.

Result: 24.

CHECK YOURSELF

1. Number of divisors of 720. [4]
2. Unit digit of $7^2 \times 9^{13}$. [5]
3. Trailing zeros of 200!. [6]
4. Smallest number leaving remainder 3 on division by 8, 12 and 20. [7]

12.3 Percentages, profit and loss, ratio and averages MUST · ■□□

These four topics are one family: each compares a quantity with a base. Most errors come from choosing the wrong base, so every card below states the base first.

FORMULA CARD: PERCENTAGES

- x% of y = y% of x. A percentage change is (change / original) × 100, with the original as base.
- Successive changes of a% and b% give a net change of $a + b + ab/100$ per cent.
- If price rises by r%, consumption must fall by $100r/(100 + r)$% to keep expenditure fixed. If price falls by r%, consumption can rise by $100r/(100 - r)$%.
- Fractions worth memorising: 1/6 = 16.67%, 1/7 = 14.29%, 1/8 = 12.5%, 1/9 = 11.11%, 1/11 = 9.09%, 1/12 = 8.33%.

Shortcut. Convert the percentage to a fraction (“25% of the price rises” means 1/4 more, so 1/5 less) and reason in parts.

WORKED EXAMPLE

Sugar rises in price by 25%. By what percentage must a household cut consumption to hold spending constant?

1. Let the old price be 100 per kg and use 1 kg: spending 100.
2. New price is 125, so the same 100 buys 100/125 = 0.8 kg.
3. The cut is 0.2 kg out of 1 kg. Formula check: 100 × 25 / 125 = 20.

Result: 20%.

WORKED EXAMPLE

In an exam, 30% of candidates failed mathematics, 25% failed English and 10% failed both. What percentage passed both?

1. Failed at least one subject: 30 + 25 − 10 = 45% (inclusion and exclusion).
2. Passed both: 100 − 45.

Result: 55%.

FORMULA CARD: PROFIT AND LOSS

- Profit% = (profit / cost price) × 100; selling price = cost price × $(1 \pm p/100)$. Discount is always on the marked price; profit is always on the cost price.
- Successive discounts of a% and b% equal a single discount of $a + b - ab/100$.
- A dishonest weight giving e grams short per true kilogram of w grams: gain% = e / ($w - e$) × 100.

Shortcut. Work with multipliers: a 20% discount is × 0.8, a 25% markup is × 1.25.

WORKED EXAMPLE

An item is marked ₹800. Successive discounts of 20% and 10% are given. The cost price is ₹480. Find the profit percentage.

1. Selling price: 800 × 0.8 × 0.9 = ₹576. This equals a single discount of 20 + 10 − 2 = 28%.
2. Profit: 576 − 480 = ₹96.
3. Profit% on cost: 96/480 × 100.

Result: 20% profit.

WORKED EXAMPLE

A trader sells at cost price but uses a 900 g weight for a kilogram. Find the gain.

1. The customer pays for 1,000 g and receives 900 g, so the error is 100 g.
2. The trader's cost for what was delivered is the value of 900 g.
3. Gain% = 100 / 900 × 100.

Result: 11.11%, not 10%, because the base is what was actually given.

FORMULA CARD: RATIO, PROPORTION, PARTNERSHIP AND ALLIGATION

- If $a : b = c : d$ then $ad = bc$. Convert a ratio to parts and find the value of one part.
- Partners share profit in the ratio of (capital × time).
- Alligation: mixing a dearer item at price d with a cheaper one at c to reach mean price m, the quantities are in the ratio $(d - m) : (m - c)$.

Shortcut. Draw the alligation cross: put c and d on top, m in the middle, subtract diagonally, and read the answer along the bottom.

WORKED EXAMPLE

Rice at ₹40 per kg is mixed with rice at ₹55 per kg to give a mixture worth ₹48 per kg. In what ratio?

1. Cheaper: 40; dearer: 55; mean: 48.
2. Cheaper : dearer = (55 − 48) : (48 − 40) = 7 : 8.
3. Check: (7 × 40 + 8 × 55) / 15 = (280 + 440)/15 = 48.

Result: 7 : 8 (cheaper to dearer).

WORKED EXAMPLE

A invests ₹5,000 for 12 months and B invests ₹8,000 for 6 months. They earn ₹9,000. Find each share.

1. Capital × time: A = 60,000; B = 48,000. Ratio 60 : 48 = 5 : 4.
2. Parts total 9; one part is 9,000 / 9 = ₹1,000.

Result: A gets ₹5,000 and B gets ₹4,000.

FORMULA CARD: AVERAGES

- Average = sum / count. A change of Δ in the average over n items means the total changed by $n\Delta$.
- When a new member joins and the average rises by Δ, the new member exceeds the old average by $(n + 1)\Delta$.
- Weighted average = Σ(count × value) / Σ count.

Shortcut. Always convert averages to totals, change the total, and convert back.

WORKED EXAMPLE

The average age of 8 students is 15. When the teacher is included, the average rises by 2. Find the teacher’s age.

1. Total of students: 8 × 15 = 120.
2. New total: 9 × 17 = 153.
3. Teacher = 153 − 120.

Result: 33 years.

WORKED EXAMPLE

A batsman’s average after 10 innings is 42. What must he score in the 11th to raise it to 45?

1. Total after 10 innings: 420. Required total after 11: 11 × 45 = 495.
2. Score = 495 − 420. Check by the shortcut: 45 + 10 × 3 = 75.

Result: 75 runs.

INTERVIEW LENS

CISCO · JUNIPER · NOKIA · QUALCOMM

Percentage and ratio questions are the bulk of the quantitative section in these drives because they are quick to set and quick to mark. The examiner is testing whether you pick the correct base. A strong candidate states the base aloud (“profit on cost, discount on marked”) before computing, which also catches the commonest error.

CHECK YOURSELF

1. A price falls by 20%. By what percentage can consumption rise at constant spending? [8]
2. A number is increased by 30% and then decreased by 30%. Net change? [9]
3. Milk at ₹30 per litre and ₹42 per litre are mixed to make ₹36 per litre. Ratio? [10]
4. The average of five numbers is 18. Two are removed, with average 12. Average of the other three? [11]

1. 1/5 × 1 − 4/5 × 1/4 = 0.2 − 0.2 = 0. A blind guess is neutral.↩
2. Three options remain: 1/3 × 1 − 2/3 × 1/4 = 1/3 − 1/6 = +0.167.↩
3. 30 × 60 / 40 = 45 seconds.↩
4. $720 = 2^4 \times 3^2 \times 5$, so 5 × 3 × 2 = 30.↩
5. $7^2 = 49$ ends in 9; 9^{13} ends in 9 (odd exponent); 9 × 9 = 81, so 1.↩
6. 40 + 8 + 1 = 49.↩
7. LCM(8, 12, 20) = 120, so 123.↩
8. 100 × 20 / (100 − 20) = 25%.↩
9. 30 − 30 − 900/100 = −9%, a fall of 9%.↩

12.4 Work, pipes, speed, trains and boats MUST · ■□□

All five topics in this section are rate problems: a quantity done per unit of time. Work per day, water per hour and distance per hour obey the same equation, quantity = rate × time, so one habit serves them all: convert every statement into a rate before combining.

FORMULA CARD: TIME AND WORK

- If A alone takes a days, A's rate is $1/a$ of the job per day. Rates add when people work together.
- A and B together: $ab/(a + b)$ days.
- LCM method: take total work as the LCM of the days; each person's rate becomes a whole number of units per day.
- Pipes: a filling pipe has a positive rate, an emptying pipe or leak a negative rate; sum them.
- Man-days: $M_1D_1H_1/W_1 = M_2D_2H_2/W_2$.

Shortcut. The LCM method removes fractions: for 10 and 15 days, assume 30 units; A does 3 per day and B does 2.

WORKED EXAMPLE

A can finish a job in 10 days and B in 15 days. All three of A, B and C together finish in 5 days. How long would C take alone?

1. Take the work as LCM(10, 15, 5) = 30 units. A does 3 per day and B does 2 per day.
2. A, B and C together do 30/5 = 6 per day, so C does 6 − 3 − 2 = 1 per day.
3. C alone: 30/1 days.

Result: 30 days.

WORKED EXAMPLE

A can do a job in 20 days and B in 30 days. They start together, but A leaves after 6 days. How long does the whole job take?

1. Work = 60 units; A does 3 per day and B does 2 per day.
2. In 6 days together: 6 × 5 = 30 units, leaving 30.
3. B alone does 30 units in 15 days.

Result: 6 + 15 = 21 days.

WORKED EXAMPLE

Pipe A fills a tank in 6 hours and pipe B in 8 hours. Pipe C empties it in 12 hours. With all three open, how long to fill the empty tank?

1. Capacity = LCM(6, 8, 12) = 24 units. A adds 4 per hour, B adds 3, C removes 2.
2. Net rate = 4 + 3 − 2 = 5 units per hour.
3. Time = 24/5.

Result: 4.8 hours, which is 4 hours 48 minutes.

WORKED EXAMPLE

A tank fills in 4 hours, but because of a leak it takes 5 hours. How long does the leak alone take to empty a full tank?

1. Filling rate 1/4 per hour; with the leak 1/5 per hour.
2. Leak rate = 1/4 − 1/5 = 1/20 per hour.

10. (42 − 36) : (36 − 30) = 6 : 6 = 1 : 1.↩
11. Total 90; removed 24; remaining 66, so 66/3 = 22.↩

Result: 20 hours.

FORMULA CARD: TIME, SPEED AND DISTANCE

- Distance = speed × time. Convert km/h to m/s by multiplying by 5/18, and back by 18/5.
- Average speed over equal distances at speeds x and y: $2xy/(x + y)$, the harmonic mean. It is not $(x + y)/2$. Over equal times it is the arithmetic mean.
- Relative speed: same direction subtract, opposite directions add.
- Catching up: time = initial gap / relative speed.

Shortcut. If the speed ratio is $a : b$ over the same distance, the time ratio is $b : a$.

WORKED EXAMPLE

A car goes to a city at 60 km/h and returns at 40 km/h. Find the average speed.

1. Equal distances, so use the harmonic mean: 2 × 60 × 40 / (60 + 40) = 4,800/100.
2. Check with 120 km each way: time 2 h + 3 h = 5 h for 240 km, giving 48 km/h.

Result: 48 km/h.

WORKED EXAMPLE

Train X leaves a station at 30 km/h. Two hours later train Y leaves the same station on the same track at 50 km/h. When does Y catch X?

1. X's head start: 30 × 2 = 60 km.
2. Relative speed: 50 − 30 = 20 km/h.
3. Time after Y departs: 60/20.

Result: 3 hours after Y leaves, at distance 150 km.

FORMULA CARD: TRAINS, BOATS AND STREAMS

- A train passing a pole or a person covers its own length. Passing a platform or bridge, it covers (train + platform).
- Two trains crossing: the distance is the sum of the lengths, the speed is the relative speed.
- Boat in still water u, stream v: downstream $u + v$, upstream $u − v$. Hence u = (down + up)/2 and v = (down − up)/2.

Figure 12.2 Crossing a platform starts when the front reaches it and ends when the rear leaves it, so the distance is L + P.

WORKED EXAMPLE

A 240 m train runs at 72 km/h. How long does it take to cross a 360 m platform?

1. Speed: 72 × 5/18 = 20 m/s.
2. Distance: 240 + 360 = 600 m.
3. Time: 600/20.

Result: 30 seconds.

WORKED EXAMPLE

A 200 m train at 60 km/h overtakes a 150 m train moving the same way at 42 km/h. How long does the overtaking take?

1. Relative speed: 60 − 42 = 18 km/h = 18 × 5/18 = 5 m/s.
2. Distance: 200 + 150 = 350 m.
3. Time: 350/5.

Result: 70 seconds.

WORKED EXAMPLE

A boat travels 36 km downstream and returns upstream. Its still-water speed is 15 km/h and the stream is 3 km/h. Total time?

1. Downstream speed 18 km/h: 36/18 = 2 h.
2. Upstream speed 12 km/h: 36/12 = 3 h.

Result: 5 hours. The average speed is 72/5 = 14.4 km/h, not 15.

WORKED EXAMPLE

A boat covers 24 km downstream in 2 hours and 24 km upstream in 3 hours. Find the still-water and stream speeds.

1. Downstream 24/2 = 12 km/h; upstream 24/3 = 8 km/h.
2. $u = (12 + 8)/2 = 10$; $v = (12 − 8)/2 = 2$.

Result: boat 10 km/h, stream 2 km/h.

CHECK YOURSELF

1. A can do a job in 24 days and B in 40. How long together? [1]
2. A tap fills a tank in 3 h and another in 6 h; an outlet empties it in 4 h. Time with all open? [2]
3. A 100 m train crosses a pole in 10 s. Its speed in km/h? [3]
4. A boat's speed in still water is 9 km/h; it takes 4 h to go 20 km upstream. Stream speed? [4]

12.5 Interest, counting, probability, ages and the calendar

MUST · ■■□

FORMULA CARD: SIMPLE AND COMPOUND INTEREST

- Simple interest SI = $PRT/100$. Amount A = P + SI.
- Compound interest: $A = P(1 + r/100)^n$; CI = A − P. For half-yearly compounding use rate $r/2$ and $2n$ periods.
- CI − SI for two years = $P(r/100)^2$; for three years = $P(r/100)^2(3 + r/100)$.
- Rule of 72: money doubles in about $72/r$ years at compound rate r%.

Shortcut. For two years at r% the multiplier is $1 + 2r/100 + (r/100)^2$; at 10% it is 1.21.

WORKED EXAMPLE

Find the compound interest and simple interest on ₹10,000 at 10% per annum for 2 years, and verify the difference formula.

1. SI = 10,000 × 10 × 2 / 100 = ₹2,000.
2. Amount at CI = $10,000 \times 1.1^2 = 12,100$, so CI = ₹2,100.

3. Difference = 100 = 10,000 × $(0.1)^2$.

Result: CI ₹2,100, SI ₹2,000, difference ₹100.

WORKED EXAMPLE

₹8,000 is invested at 20% per annum, compounded half-yearly. Find the amount after one year.

1. Half-yearly rate 10%, two periods.
2. Amount = 8,000 × 1.1 × 1.1.

Result: ₹9,680 (annual compounding would give ₹9,600).

FORMULA CARD: PERMUTATIONS AND COMBINATIONS

- Permutations (order matters): P(n, r) = n!/(n − r)!. Combinations: C(n, r) = n!/(r!(n − r)!).
- Arrangements of n objects with repeats p, q: n!/(p!q!).
- Circular arrangements of n people: (n − 1)!. If two named people must be adjacent: treat them as one block and multiply by 2.
- Objects that must be together: glue them as one block, arrange, then multiply by the internal arrangements.
- Distributing r identical items among k people (stars and bars): C(r + k − 1, k − 1).
- "At least" counts: add the cases, or subtract the complement from the total.

Shortcut. Decide first whether order matters. If a question says "select", "choose" or "committee", use C; if it says "arrange" or "form a number", use P.

WORKED EXAMPLE

How many 4-digit numbers can be formed from the digits 1 to 7 with no repetition? How many are even?

1. All: P(7, 4) = 7 × 6 × 5 × 4 = 840.
2. Even: the units digit is 2, 4 or 6 (3 choices). The other three places take P(6, 3) = 6 × 5 × 4 = 120 ways.
3. Even numbers: 3 × 120.

Result: 840 numbers, of which 360 are even.

WORKED EXAMPLE

A committee of 4 is chosen from 6 men and 4 women with at least 2 women. How many committees?

1. Two women and two men: C(4, 2) × C(6, 2) = 6 × 15 = 90.
2. Three women and one man: C(4, 3) × C(6, 1) = 4 × 6 = 24.
3. Four women: C(4, 4) = 1.

Result: 90 + 24 + 1 = 115.

FORMULA CARD: PROBABILITY

- P(event) = favourable / total, for equally likely outcomes. Two dice have 36 outcomes; a deck has 52 cards (4 suits, 12 face cards); n coins have 2^n outcomes.
- Complement: P(at least one) = 1 − P(none). Independent events multiply; mutually exclusive events add.
- Conditional: P(A | B) = P(A and B)/P(B). Without replacement, probabilities change after each draw.

Shortcut. The words “at least one” almost always mean: compute the complement.

WORKED EXAMPLE

Two fair dice are thrown. What is the probability that the sum is 8?

1. Pairs summing to 8: (2,6), (3,5), (4,4), (5,3), (6,2), which is 5 pairs.
2. Total pairs: 36.

Result: 5/36.

WORKED EXAMPLE

A bag has 5 red, 3 blue and 2 green balls. Three are drawn at random. What is the probability that at least one is red?

1. Complement: no red, so all three come from the 5 non-red balls: C(5, 3) = 10.
2. Total ways: C(10, 3) = 120. So P(none) = 10/120 = 1/12.
3. P(at least one red) = 1 − 1/12.

Result: 11/12.

FORMULA CARD: AGES, CLOCKS AND CALENDARS

- Ages: write today’s ages as ratio parts ($5x$, $3x$), then add or subtract the years for the other time.
- Clocks: the minute hand moves 6° per minute and the hour hand 0.5° per minute, so the angle at H hours M minutes is $|30H − 5.5M|$; if above 180° subtract it from 360°. The hands overlap every $720/11 = 65\frac{5}{11}$ minutes.
- Calendars: 100 years have 5 odd days, 200 years 3, 300 years 1, 400 years 0. An ordinary year has 1 odd day and a leap year 2. An *odd day* is a day beyond whole weeks.

Shortcut. Count the days between two dates and take the remainder after dividing by 7.

WORKED EXAMPLE

The ratio of the ages of two engineers is 5 : 3. Six years hence it will be 7 : 5. Find their present ages.

1. Write $(5x + 6) / (3x + 6) = 7/5$ and cross-multiply: $25x + 30 = 21x + 42$.
2. Then $4x = 12$, so $x = 3$.

Result: 15 and 9 years. Check: 21 : 15 = 7 : 5.

WORKED EXAMPLE

(a) What is the angle between the hands at 3:40? (b) At what time between 4 and 5 do the hands overlap? (c) If 26 January 2026 is a Monday, what day is 15 August 2026?

1. a. $|30 \times 3 − 5.5 \times 40| = |90 − 220| = 130°$, which is below 180°.
2. b. At 4:00 the minute hand is 120° behind. It gains 5.5° per minute, so $120/5.5 = 240/11 = 21\frac{9}{11}$ minutes.
3. c. Days from 26 January to 15 August: 5 + 28 + 31 + 30 + 31 + 30 + 31 + 15 = 201. Since $201 = 28 \times 7 + 5$, move five days from Monday.

Result: (a) 130°; (b) about 4:21:49; (c) Saturday.

FORMULA CARD: MENSURATION

- Rectangle: area lb, perimeter $2(l + b)$. Circle: area πr^2, circumference $2\pi r$; use $\pi = 22/7$ when the radius is a multiple of 7.

- Cuboid: volume lbh, surface $2(lb + bh + hl)$. Cube: volume a^3, surface $6a^2$.
- Cylinder: volume $\pi r^2 h$, curved surface $2\pi rh$. Cone: volume $(1/3)\pi r^2 h$. Sphere: volume $(4/3)\pi r^3$.
- 1 m^3 = 1,000 litres; 1 litre = 1,000 cm^3.

Shortcut. Convert every length to one unit first; most errors are unit errors.

WORKED EXAMPLE

A rectangular tank is 4 m long, 3 m wide and 2.5 m deep. How many litres does it hold when full?

1. Volume = 4 × 3 × 2.5 = 30 m^3.
2. Each cubic metre is 1,000 litres.

Result: 30,000 litres.

WORKED EXAMPLE

A cylindrical cable drum has radius 7 cm and height 20 cm. Find its volume and curved surface area (π = 22/7).

1. Volume = $(22/7) \times 7^2 \times 20 = 22 \times 7 \times 20 = 3{,}080$ cm^3.
2. Curved surface = $2 \times (22/7) \times 7 \times 20 = 880$ cm^2.

Result: 3,080 cm^3 and 880 cm^2.

CHECK YOURSELF

1. A sum of ₹10,000 at 10% compound: find CI − SI over three years. [5]
2. Ways to arrange the letters of ROUTER. [6]
3. Ways for 6 people to sit at a round table with two named people adjacent. [7]
4. Probability of exactly two heads in three tosses. [8]
5. A father is three times as old as his son. In 12 years he will be twice as old. Present ages? [9]
6. If 1 January 2000 was a Saturday, what day was 1 January 2001? [10]

1. 24 × 40 / 64 = 15 days.↩
2. Capacity 12: rates 4 + 2 − 3 = 3 per hour, so 12/3 = 4 hours.↩
3. 10 m/s × 18/5 = 36 km/h.↩
4. Upstream speed 20/4 = 5 km/h, so v = 9 − 5 = 4 km/h.↩
5. $P(r/100)^2(3 + r/100)$ = 10,000 × 0.01 × 3.1 = ₹310 (CI 3,310 less SI 3,000).↩
6. 6!/2! = 360, since R repeats.↩
7. Block the pair: (5 − 1)! × 2 = 48.↩

12.6 Logical reasoning MUST · ■□□

Reasoning questions test method more than knowledge. Each family has one drawing or one table that turns the puzzle into bookkeeping; the cards below name it. A full eight-person circular puzzle is solved in Q12.10 (p. 733).

FORMULA CARD: SERIES AND CODING-DECODING

- Number series: test differences, then second differences, then ratios, then squares, cubes or primes. Letter series: convert letters to positions (A = 1, …, Z = 26).
- Common patterns: $n(n + 1)$ gives 2, 6, 12, 20; doubling differences give $2^n + 1$; alternating series are two interleaved series.
- Coding: a shift by k, a reversal (A ↔ Z, so position p becomes $27 - p$), a positional sum, or a word-to-word map found by comparing sentences that share a word.

Shortcut. In a word-map problem, find the word common to two sentences and the code common to the same two; that pair is the match.

WORKED EXAMPLE

Complete: (a) 2, 6, 12, 20, 30, ? (b) 3, 5, 9, 17, 33, ? (c) B, D, G, K, ?

1. a. The differences are 4, 6, 8, 10, so the next difference is 12: 42. (Also 6 × 7.)
2. b. The differences are 2, 4, 8, 16, so the next is 32: 33 + 32 = 65.
3. c. Positions 2, 4, 7, 11; the differences 2, 3, 4 continue with 5, giving 16, which is P.

Result: 42; 65; P.

WORKED EXAMPLE

(a) Under a shift of +1, ROUTER is coded as? (b) In a code, "ping the host" is "ka ro mi", "host is up" is "mi ta ne" and "ping is up" is "ka ta ne". Code for "the"?

1. a. R, O, U, T, E, R become S, P, V, U, F, S.
2. b. The first two sentences share "host" and "mi", so host = mi.
3. The first and third share "ping" and "ka", so ping = ka. Hence "the" is the remaining code of the first sentence, "ro".

Result: SPVUFS; ro.

FORMULA CARD: BLOOD RELATIONS AND DIRECTIONS

- Draw a family tree, one symbol per person. Parse the sentence from the end, replacing each phrase by a person: "the only daughter of my mother" is *me* (if I am female).
- Directions: put the start at the origin, North up, and add the displacement of every leg. A right turn from North faces East; from East faces South.

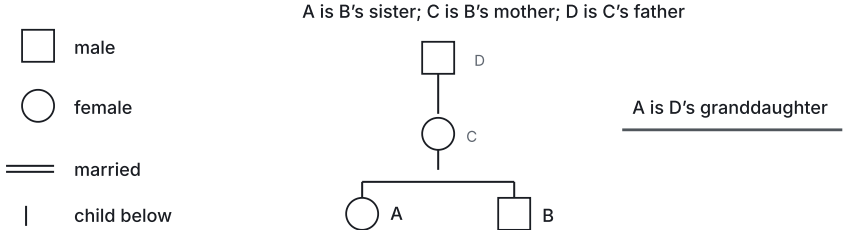


Figure 12.3 The family-tree notation and one solved chain. The sentence is read link by link, never as a whole.

8. C(3, 2)/8 = 3/8.↩
9. F = 3S and F + 12 = 2(S + 12) give S = 12 and F = 36.↩
10. The year 2000 was a leap year, so 2 odd days: Saturday + 2 = Monday.↩

WORKED EXAMPLE

Pointing to a man, Rekha says, "His mother is the only daughter of my mother." How is the man related to Rekha?

1. "The only daughter of my mother" is Rekha herself.
2. So the man's mother is Rekha.

Result: he is her son.

WORKED EXAMPLE

A walks 5 km north, turns right and walks 3 km, turns right and walks 5 km, then turns left and walks 2 km. Where is he relative to the start?

1. North 5, then east 3, then south 5: net position 3 km east, 0 north.
2. Facing south, a left turn faces east: 2 km further east.

Result: 5 km due east of the start.

FORMULA CARD: SEATING ARRANGEMENTS

- Draw the seats first (a row with numbered places, or a circle). Fix the most definite clue, then place each clue in turn; carry the alternatives forward on the same diagram.
- In a row facing north, left is west. In a circle facing the centre, a person's left is clockwise.
- Count "between" carefully: two seats between A and B puts them three places apart.

WORKED EXAMPLE

Five engineers P, Q, R, S, T sit in a row facing north. (i) Q is at one end. (ii) R is two places to the right of Q. (iii) S is not next to R. (iv) T is somewhere to the left of P. Find the order, left to right.

1. From (ii), R is to Q's right, so Q is at the left end: Q _ R _ _.
2. From (iii), S cannot take seat 2 or 4, so S is in seat 5: Q _ R _ S.
3. T and P fill seats 2 and 4; by (iv) T is the left one.

Result: Q, T, R, P, S.

WORKED EXAMPLE

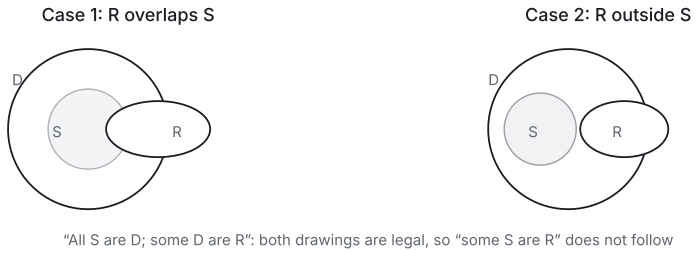
Using the same row, who is third from the right and who sits between T and P?

1. Reading right to left: S, P, R, T, Q, so the third from the right is R.
2. Between T (seat 2) and P (seat 4) is seat 3.

Result: R in both cases.

FORMULA CARD: SYLLOGISMS AND DATA SUFFICIENCY

- Syllogisms: draw Venn circles and keep only conclusions true in *every* drawing. "All A are B" also implies "some A are B" in the usual Indian convention. "No A is B" is symmetrical; "some A are B" is symmetrical; "all A are B" is not.
- A "some" statement never forces a definite conclusion by itself: from "all A are B, some B are C" no relation between A and C follows.
- Data sufficiency options: (A) I alone; (B) II alone; (C) both together, neither alone; (D) each alone; (E) neither, even together. Test each statement alone first, then both.



"All S are D; some D are R": both drawings are legal, so "some S are R" does not follow

Figure 12.4 A conclusion is valid only if it holds in every legal drawing; here the second drawing breaks it.

WORKED EXAMPLE

Statements: All switches are devices. Some devices are routers. Conclusions: I. Some switches are routers. II. Some routers are devices.

1. Draw switches inside devices; routers overlap devices, and may or may not touch switches (Figure 12.4). So I is not certain.
2. Some devices are routers; the relation is symmetrical, so some routers are devices.

Result: only II follows.

WORKED EXAMPLE

(a) Is 10.1.1.130 in the same subnet as 10.1.1.10? I. The mask is 255.255.255.128. II. Both hosts use the same gateway. (b) What is the average speed for a journey? I. The first half of the distance was at 60 km/h. II. The second half was at 40 km/h.

1. a. I alone: block size 128, so .10 lies in 0 to 127 and .130 in 128 to 255: different subnets. II alone proves nothing (a gateway can serve several subnets). Answer A.
2. b. Neither statement alone gives a speed over the whole journey. Together, with equal halves, the average is $2 \times 60 \times 40/100 = 48$ km/h, whatever the distance. Answer C.

Result: (a) A; (b) C.

FORMULA CARD: PUZZLES BY THE GRID METHOD

- Draw a table with the people as rows and each attribute as a column. Mark a cross for every "not" clue and a tick for every "is" clue; each tick forces crosses in its row and column.
- Start with the clue that names a single cell, then use elimination: when a row or column has one empty cell left, it is a tick.

WORKED EXAMPLE

Asha, Ravi and Meena each manage one of a router, a switch and a firewall, and each works one shift: morning, afternoon or night. (i) Ravi works the morning shift. (ii) The router's manager works nights. (iii) Asha does not manage the firewall. (iv) Meena does not manage the switch. (v) Ravi does not manage the firewall. Who manages what, and when?

1. (iv): Meena manages the router or the firewall. (iii): Asha manages the router or the switch. (v): Ravi manages the router or the switch.
2. i. and (ii): Ravi works mornings, so he is not the router's manager; hence Ravi manages the switch.
3. Then Asha has the router, and Meena the firewall. By (ii) Asha, who has the router, works nights, so Meena works afternoons.

Result: Asha, router, night; Ravi, switch, morning; Meena, firewall, afternoon.

CHECK YOURSELF

1. Next term: 3, 8, 15, 24, 35, ? [1]
2. Under reversal (A ↔ Z), the code of CAT. [2]
3. “A is B’s brother. C is A’s mother. D is C’s husband.” How is D related to B? [3]
4. All routers are devices. No device is a cable. Does “No router is a cable” follow? [4]

12.7 Data interpretation and verbal ability SHOULD · ■□□

DATA INTERPRETATION

A data-interpretation question gives a table or chart and asks for growth, ratio or share. Approximation is the whole skill: round to two significant figures, compare fractions by cross-multiplication, and remember that percentage growth is always relative to the earlier value.

Campus	Peak 2025 (Gb/s)	Peak 2026 (Gb/s)
North	12	15
Main	20	26
Hostel	8	11
Lab	15	18
Annexe	5	8

WORKED EXAMPLE

For the table above, find (a) the campus with the highest percentage growth, (b) the highest absolute growth, and (c) the total growth.

1. Growth: North 3/12 = 25%; Main 6/20 = 30%; Hostel 3/8 = 37.5%; Lab 3/15 = 20%; Annexe 3/5 = 60%.
2. Absolute growth is largest for Main (6 Gb/s).
3. Totals: 60 in 2025 and 78 in 2026, so 18/60 = 30%.

Result: Annexe (60%); Main (6 Gb/s); 30% overall.

VERBAL ABILITY

FORMULA CARD: VERBAL

- Subject–verb agreement: the verb agrees with the head noun, not the nearest noun (“the list of requirements *is* long”). “Each”, “every” and “neither” take the singular. With “neither … nor”, the verb agrees with the nearer subject.
- Keep tenses consistent and lists parallel (“configure, test and deploy”, not “configure, testing and deploy”). Use “a” before a consonant *sound* (“a unit”, “an hour”).
- Reading comprehension: read the questions first, skim the passage for the opening and closing sentences, then scan for the keywords. The correct option restates the passage; the wrong ones add, exaggerate or reverse it.
- Para-jumbles: the opening sentence introduces a noun without a pronoun; “this”, “it”, “therefore” and “however” cannot start a paragraph.

WORKED EXAMPLE

Correct: (a) “The list of requirements are long.” (b) “Neither the router nor the switches was configured.” Then order: P. This is why a lost packet is simply sent again. Q. The

Internet delivers packets on a best-effort basis. R. It makes no promise that any given packet will arrive. S. Reliability is therefore added by the hosts.

1. a. The head noun is “list”, so “is”. (b) The nearer subject is “switches”, so “were”.
2. Q is the only sentence with no pronoun or connective, so it opens. R continues with “it”. P explains the consequence (“this is why”), and S concludes (“therefore”).

Result: “is”; “were”; order QRPS.

WORKED EXAMPLE

Passage: “A route flap occurs when a prefix is repeatedly announced and withdrawn. Each change forces neighbouring routers to recompute paths, so one unstable link can burden routers far away. Operators therefore apply damping, which penalises a prefix on every flap and suppresses it once the penalty crosses a threshold. Damping protects routers, but it also delays recovery: a prefix that has genuinely returned stays hidden until its penalty decays. Many operators now use it sparingly.” Q1. The passage mainly argues that (A) flaps are harmless (B) damping protects routers at a cost to recovery (C) damping should be abolished (D) neighbouring routers cause flaps. Q2. Which is implied? (A) Damping never helps (B) Operators weigh damping’s benefit against its delay.

1. Q1: the third sentence introduces damping and the fourth balances protection against delay. Option A contradicts sentence two; C overstates “sparingly”; D reverses cause and effect. Answer B.
2. Q2: “sparingly” implies that the cost is considered. A is an exaggeration (“never”). Answer B.

Result: B and B.

CHECK YOURSELF

1. In the table above, what is the ratio of Main’s 2026 peak to Annexe’s? [5]
2. Choose: “Each of the engineers (is / are) on call.” [6]

12.8 Object-oriented programming for the viva MUST · ■■□

Object-oriented programming, or OOP, organises a program as objects that bundle data with the functions that act on it. Panels ask about it because the vocabulary is short and answers reveal at once whether a candidate has written a program of any size. The C++ mechanics live in §2.19 (p. 138) and §2.20 (p. 140); this section supplies the viva-ready answers and uses Python for code, because network-automation libraries are written in it (Pillar 10 (p. 586)).

BIG IDEA

Every OOP term answers one question: how do we change one part of a program without breaking the rest?

The four pillars with a networking example

Pillar	One-line meaning	Network example	Mechanism to quote
Encapsulation	Data and the code that guards it live together; outsiders use a controlled interface	`Interface.set_mtu()` rejects values outside 68 to 9,216	Access specifiers (`private`, `protected`, `public`); `_name` convention in Python

1. The differences are 5, 7, 9, 11 (each term is $n^2 - 1$), so the next is 48.↩
2. C is 3, becomes 24 (X); A becomes Z; T is 20, becomes 7 (G): XZG.↩
3. D is C’s husband and C is the mother of both A and B, so D is B’s father.↩
4. Yes: every router lies inside devices, and devices lie outside cables.↩
5. 26 : 8 = 13 : 4.↩
6. “Each” is singular, so “is”.↩

Abstraction	Expose what an object does, hide how	`device.get_config()` without knowing the vendor command	Abstract classes and interfaces
Inheritance	A new class reuses and extends an existing one ("is-a")	`CiscoIOS` and `JunosDevice` extend `Device`	`class B : public A;` `class B(A)`
Polymorphism	One call, different behaviour by the object's real type	`for d in devices:` `d.get_config()`	Virtual functions (run time); overloading (compile time)

Encapsulation and abstraction are often confused. Encapsulation is about *protecting* state by bundling and access control; abstraction is about *simplifying* by showing only essentials. A TV remote is the usual analogy: the buttons are the abstraction, and the sealed circuit board is the encapsulation.

Classes, objects, constructors

A *class* is a blueprint that fixes the data members and member functions; an *object* is one instance of it with its own values. A *constructor* runs when the object is created, to establish a valid state; a *destructor* runs when it is destroyed. In C++ constructors run base first, then derived, and destructors run in the reverse order. A class that is meant to be inherited must declare its destructor `virtual`, because otherwise deleting a derived object through a base pointer skips the derived destructor.

Shallow copy — copies pointer values, so two objects share one block; the second `delete` is a double free. A **deep copy** duplicates the block. Rule of three: if you write one of destructor, copy constructor or copy assignment, write all three.

Overloading, overriding and hiding

	Overloading	Overriding
Meaning	Same name, different parameter lists	Derived class replaces a base-class function with the same signature
Binding	Compile time (static)	Run time (dynamic), if the base function is `virtual`
Scope	Within one class	Across a base and a derived class
Return type	May differ (but not alone)	Must match, or be covariant
Python	Not supported; the last definition wins; use default arguments or `functools.singledispatch`	By defining the method again

> **TRAP**
>
> *Claim:* "Polymorphism means many forms," and nothing more. *Why it is wrong:* it names the idea and omits the mechanism, so the examiner learns nothing. *Correct statement:* in C++, a call through a base pointer to a `virtual` function is resolved at run time through the object's *vptr*, which points to the *vtable* of its real class; overloading is resolved by the compiler from the argument types.

Abstract classes and interfaces

	Abstract class	Interface
Can hold data and implemented methods	Yes	Java and C#: default methods only, no instance state
Instantiable	No	No

A class may extend	One (Java), several (C++, Python)	Many
C++	At least one pure virtual function (`= 0`)	No keyword: an abstract class with only pure virtuals
Python	Subclass of `abc.ABC` with `@abstractmethod`	Same, or a `typing.Protocol`
Use when	Related classes share code and a contract	Unrelated classes share only a contract

How virtual dispatch works

Each class with virtual functions has one *vtable*, an array of function pointers; each object stores one hidden pointer, the *vptr*, to its class's table. A virtual call loads the vptr, indexes the table and jumps. The cost is one pointer per object (8 bytes on a 64-bit machine), one extra memory read per call, and the loss of inlining. §2.20 (p. 140) shows the layout with `sizeof` output.

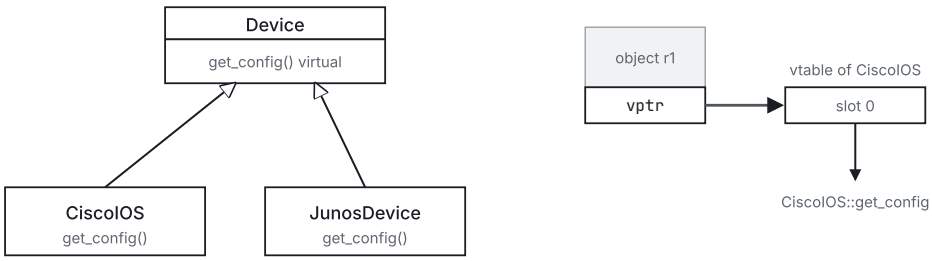


Figure 12.5 Subclasses override one function (hollow arrowheads mean "is a"); at run time each object reaches its own version through its vptr and vtable.

SOLID in brief

Letter	Principle	Violation	Repair
S	Single responsibility: one reason to change	A `Router` class that also parses configs and sends e-mail	Split into `Router`, `ConfigParser`, `Notifier`
O	Open for extension, closed for modification	An `if vendor == ...` chain edited for every new vendor	A new subclass; no existing code changes
L	Liskov substitution: a subclass must work wherever its base does	`ReadOnlyDevice` raises an error in `set_config()`	Separate readable and writable interfaces
I	Interface segregation: no client forced to depend on methods it does not use	One `Device` interface with `bgp_neighbours()` that a switch cannot implement	Small interfaces: `Routing`, `Switching`
D	Dependency inversion: depend on abstractions, not concrete classes	A monitor that constructs `CiscoIOS` itself	The monitor receives a `Device`; a factory supplies it

Five patterns in one table

Pattern	Intent	Network example	Cost or caution

Singleton	One instance, global access	A logger or connection pool	Hidden global state makes tests hard
Factory	Create objects by a key without naming the class	`ConnectHandler(device_type="cisco_ios")` in Netmiko	One more indirection
Strategy	Swap an algorithm at run time	Pluggable route-metric calculators	Many small classes
Observer	Notify subscribers when state changes	Telemetry consumers subscribed to interface counters	Order and failure of callbacks
Adapter	Wrap an incompatible interface	A common API over vendor REST and CLI	The wrapper can leak vendor detail

WORKED EXAMPLE

Replace an `if vendor == ...` chain with a factory that follows the open-closed principle.

1. Define the contract once as an abstract class; each vendor supplies only what differs.
2. Register the classes in a dictionary keyed by vendor name.
3. The caller asks the factory and never names a vendor class.

```
from abc import ABC, abstractmethod

class Device(ABC):
    def __init__(self, host): self.host = host
    @abstractmethod
    def show_config_cmd(self): ...
    def get_config(self):
        return f"{self.host}: {self.show_config_cmd()}"

class CiscoIOS(Device):
    def show_config_cmd(self): return "show running-config"
class Junos(Device):
    def show_config_cmd(self): return "show configuration"

DRIVERS = {"ios": CiscoIOS, "junos": Junos}
def connect(kind, host): return DRIVERS[kind](host)

print(connect("junos", "r1").get_config())   # r1: show configuration
```

Result: adding EOS means one new class and one dictionary entry; `Device("x")` raises `TypeError`, because it is abstract. This is the same shape as NAPALM's drivers (§10.7 (p. 599)).

Questions that recur as MCQs

Question	Answer
Can a constructor be virtual?	No: the vptr is not yet set when it runs
Can a destructor be virtual?	Yes, and it must be in a polymorphic base class
Which function is called on a base pointer to a derived object without `virtual`?	The base version (static binding)
What is the diamond problem?	A class inherits one base through two paths; C++ uses `virtual` inheritance, and Python resolves it

	by the C3 method resolution order
Can an abstract class have a constructor?	Yes; it runs when a derived object is built
`static` member function	Belongs to the class, has no `this`, cannot be virtual
Composition or inheritance?	Prefer composition (“has-a”) unless the relationship is truly “is-a”

INTERVIEW LENS

CISCO · JUNIPER · ARISTA · MICROSOFT

Panels ask “overloading versus overriding” and “abstract class versus interface” as openers, then probe one answer. A strong answer states the difference, names the mechanism, and gives a network example in a sentence. Expect the follow-up “how would you add a new vendor?”, which is the factory above. Questions are solved in Q12.11 to Q12.18 (p. 733).

CHECK YOURSELF

1. Which binding does overloading use? [1]
2. Why must a base class with virtual functions have a virtual destructor? [2]
3. Which SOLID letter does an `if vendor` chain violate? [3]

12.9 Databases: keys, SQL, normalisation, transactions and indexes MUST · ■■□

A database management system, or DBMS, stores data so that many programs can query and change it safely. Network teams use one for inventories, IP address management and monitoring, and campus tests ask about it at multiple-choice depth: keys, normal forms, a join or two, ACID and indexes. We follow one running inventory, drawn from Campus Net, through the whole section.

Relations, keys and the ER model

A *relation* is a table; a *tuple* is a row; an *attribute* is a column. Keys identify rows, and the vocabulary is tested directly.

Key	Definition	Inventory example
Super key	Any set of attributes that is unique per row	(name), (name, vendor)
Candidate key	A minimal super key	`name`
Primary key	The candidate chosen to identify rows; unique and not null	`device.name`
Unique key	Unique, but may be null (one null in some systems)	`device.serial`
Foreign key	Attribute referring to a primary key elsewhere; enforces referential integrity	`device.site_id` refers to `site.site_id`
Composite key	A key of several attributes	(`device`, `interface`)

An *entity-relationship diagram* shows entities as boxes and relationships as lines; the crow’s foot at one end means “many”. One site has many devices and one device has many interfaces, so the

1. Compile-time (static) binding.↩
2. Otherwise `delete base_ptr` runs only the base destructor and leaks the derived part.↩
3. O, open for extension and closed for modification.↩

inventory is a chain of one-to-many links.

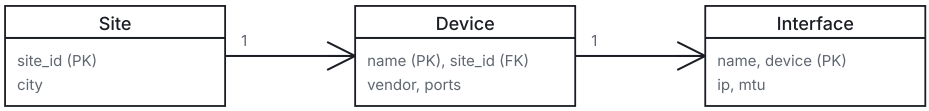


Figure 12.6 Entity-relationship diagram of the Campus Net inventory: three one-to-many links, with the crow's foot on the "many" side.

SQL in one page

SQL statements are written in one order but executed in another, and most wrong answers come from forgetting it: `FROM` and joins, then `WHERE`, `GROUP BY`, `HAVING`, `SELECT`, `ORDER BY`, and last `LIMIT`. Hence a column alias defined in `SELECT` cannot be used in `WHERE`, and `WHERE` filters rows before grouping while `HAVING` filters groups after it.

Our data: `site(1 Main, 2 Hostel, 3 Lab, 4 Annexe)` and `device(name, site_id, vendor, ports)` with `r1`, `r2` (Main, Cisco, 4 ports each), `sw1` (Main, Juniper, 48), `sw2` (Main, Arista, 48), `sw3` (Hostel, Arista, 24), `sw4` (Hostel, Arista, 48) and `fw1` (Lab, Palo Alto, 8).

Join	Rows returned	Typical use
`INNER JOIN`	Only matching pairs	Devices with their city
`LEFT JOIN`	All left rows; right columns null if no match	Sites including empty ones
`RIGHT JOIN`	Mirror image of left	Rarely needed
`FULL JOIN`	All rows from both sides	Reconciling two inventories
`CROSS JOIN`	Every pair (m × n rows)	Generating combinations
Self join	A table joined to itself with aliases	Employees with their managers

WORKED EXAMPLE

Four queries on the inventory, with their exact output.

1. Arista devices and their city: `SELECT d.name, s.city FROM device d JOIN site s ON d.site_id = s.site_id WHERE d.vendor = 'Arista' ORDER BY d.name;` gives (`sw2`, Main), (`sw3`, Hostel), (`sw4`, Hostel).
2. Devices per site, including empty sites: `SELECT s.city, COUNT(d.name) FROM site s LEFT JOIN device d ON d.site_id = s.site_id GROUP BY s.city;` gives Annexe 0, Hostel 2, Lab 1, Main 4. Using `COUNT(*)` here would wrongly give Annexe 1, since the null-padded row still counts.
3. Sites with more than two devices: `... GROUP BY s.city HAVING COUNT(*) > 2;` gives Main (4).
4. Devices with more ports than the average of their own site (a correlated subquery): `SELECT name FROM device d WHERE ports > (SELECT AVG(ports) FROM device WHERE site_id = d.site_id);` gives `sw1`, `sw2`, `sw4`. The site averages are 26 for Main and 36 for Hostel.

Result: `COUNT(column)` skips nulls and `COUNT(*)` does not; that distinction decides query 2.

The window function `ROW_NUMBER() OVER (PARTITION BY site_id ORDER BY ports DESC)` numbers devices within each site without collapsing rows, which is the clean way to answer "top *n* per group". `EXISTS` stops at the first match and handles nulls safely, whereas `NOT IN` returns nothing if the subquery yields a single null. The classic "second-highest salary" and join questions are solved in Q12.21 and Q12.22 (p. 733).

Normalisation

Normalisation splits tables so that each fact is stored once, which removes three anomalies: *insertion* (cannot add a site without a device), *update* (the same city changed in many rows) and *deletion* (removing the last device loses the site). It rests on *functional dependencies*: $X \rightarrow Y$ means that each value of X determines one value of Y.

Normal form	Requirement	Violation fixed by
1NF	Every cell is atomic; no repeating groups	One row per IP address, not "10.0.0.1, 10.0.0.2"
2NF	1NF, and no non-key attribute depends on *part* of a composite key	Moving partial dependencies to a new table
3NF	2NF, and no non-key attribute depends on another non-key attribute (no transitive dependency)	Moving the transitive chain out
BCNF	For every dependency $X \rightarrow Y$, X is a super key	Splitting on the offending determinant

WORKED EXAMPLE

Normalise `Log(device, site, site_city, interface, ip, vendor, vendor_phone)`. The dependencies are: device → site, vendor; site → site_city; vendor → vendor_phone; (device, interface) → ip. The key is (device, interface).

1. 1NF: assume every cell holds a single value (otherwise split the multiple IPs into rows).
2. 2NF: `site` and `vendor` depend on `device` alone, which is part of the key. Split into `Device(device, site, site_city, vendor, vendor_phone)` and `Interface(device, interface, ip)`.
3. 3NF: in `Device`, site → site_city and vendor → vendor_phone are transitive. Split into `Device(device, site, vendor)`, `Site(site, site_city)` and `Vendor(vendor, vendor_phone)`.
4. BCNF: in all four tables every determinant is a key, so the design is in BCNF.

Result: four tables, `Interface`, `Device`, `Site`, `Vendor`; each city and phone number is stored once.

TRAP

Claim: "3NF and BCNF are the same." *Why it is wrong:* take `Teach(student, course, teacher)` where each teacher teaches one course, so teacher → course. The key is (student, course) or (student, teacher). The table is in 3NF, because `course` is part of a key, yet teacher is not a super key. *Correct statement:* BCNF is stricter; the split into `(teacher, course)` and `(student, teacher)` achieves it, but this particular split loses the dependency (student, course) → teacher.

Denormalisation reverses the process deliberately, to avoid joins in read-heavy systems such as monitoring dashboards, at the price of redundant copies.

Transactions, ACID and isolation

A *transaction* is a group of statements that must succeed or fail as one. Transferring money by UPI is the standard illustration: debit and credit happen together or not at all.

Property	Meaning	Mechanism
Atomicity	All or nothing	Undo log, rollback
Consistency	Constraints hold before and after	Keys, checks, triggers
Isolation	Concurrent transactions do not see each other's partial work	Locks, multi-version concurrency control (MVCC)
Durability	Committed data survives a crash	Write-ahead log flushed before commit

Concurrency causes three named anomalies. A *dirty read* sees another transaction's uncommitted change. A *non-repeatable read* reads a row twice and gets two values because another transaction committed between. A *phantom* re-runs a range query and finds new rows. The isolation levels of the SQL standard each forbid more of them.

Isolation level	Dirty read	Non-repeatable read	Phantom
Read uncommitted	possible	possible	possible
Read committed	prevented	possible	possible
Repeatable read	prevented	prevented	possible (standard)
Serializable	prevented	prevented	prevented

Note that engines differ from the standard table: PostgreSQL's repeatable read is a snapshot that also blocks phantoms, and its default is read committed; MySQL InnoDB defaults to repeatable read. *Deadlock* arises when two transactions each hold a lock the other needs; the engine detects the cycle and aborts one, which the application must retry.

Indexes and B+ trees

An *index* is a separate sorted structure that lets the engine find rows without scanning the table, like the index of a book. Most engines use a *B+ tree*: a balanced tree in which internal nodes hold only keys and routing pointers, all rows (or row pointers) sit in the leaves, and the leaves are linked in order. The height stays small because each node is a disk page holding hundreds of keys. With a fan-out of 100, one million rows need about three levels, hence three or four page reads for a lookup, against thousands for a scan. The linked leaves make range queries cheap: find the first key, then walk sideways.

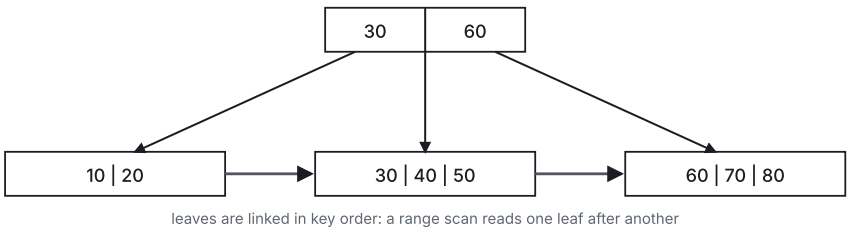


Figure 12.7 A small B+ tree. Keys below 30 go left, 30 to 59 to the middle, 60 and above to the right; the highlighted links are the leaf chain.

A *clustered* index stores the table rows in key order (one per table, normally the primary key); a *non-clustered* index holds keys with pointers to rows. A *composite* index on (`site_id`, `vendor`) serves `WHERE site_id = 1` and `WHERE site_id = 1 AND vendor = 'Arista'`, but not `WHERE vendor = 'Arista'` alone: this is the *leftmost-prefix rule*. An index is ignored when the column is wrapped in a function (`WHERE UPPER(vendor) = ...`) or when the predicate matches most rows, and each index slows inserts and updates and costs storage.

NoSQL and telemetry stores

Key-value, document, wide-column and graph stores relax the relational model to scale across machines. The CAP theorem says that during a network partition a distributed store must choose between consistency and availability. Interface counters are best kept in a *time-series database* such as Prometheus: 10,000 ports sampled every 10 seconds produce 1,000 samples per second, or 86.4 million samples a day, which suits append-only time-ordered storage rather than a relational table.

> INTERVIEW LENS
>
> CISCO · MICROSOFT · NOKIA · AMAZON
>
> Expect "write a query" on paper, a normal-form definition, and one concurrency story. A strong answer states the assumption (which column is the key, whether nulls are possible), writes the query, and traces it on the sample data. The IP-allocation race in Q12.26 (p. 733) is the favourite for network roles.

> CHECK YOURSELF
>
> 1. Which of `WHERE` and `HAVING` can use an aggregate? [1]
> 2. Table `R(A, B, C)` with B → C and key A. Which normal form fails? [2]
> 3. Can an index on (`site_id`, `vendor`) serve `WHERE vendor = 'Cisco'`? [3]
> 4. Samples per day from 2,000 ports polled every 30 seconds? [4]

12.10 Computer organisation for network engineers SHOULD · ■■□

Computer organisation describes how a processor represents numbers, reaches memory and overlaps instructions. Qualcomm, Broadcom and Arista ask about it because a packet processor is a small computer whose speed depends on exactly these three things. Endianness and the memory layout of a process are taught in §2.9 (p. 117) and §7.7 (p. 435); we do not repeat them, and keep to number representation, caches, pipelines and the memory hierarchy.

> BIG IDEA
>
> Memory is slow and the processor is fast, so almost every design in this section is a way of hiding that gap: caches by remembering, pipelines by overlapping.

Number representation

A computer stores n-bit integers in *two's complement*: the top bit has weight -2^{n-1}, so the range is -2^{n-1} to $2^{n-1} - 1$ (−128 to 127 for 8 bits). To negate a number, invert all bits and add one; the same adder then handles signed and unsigned values. Overflow occurs when two operands of the same sign give a result of the opposite sign.

> WORKED EXAMPLE

1. Only `HAVING`, because `WHERE` runs before grouping.↩
2. 3NF: C depends on B, a non-key attribute (a transitive dependency). It is in 2NF, since the key is a single attribute.↩
3. No, the leftmost column is absent from the predicate.↩
4. 2,000 × (86,400/30) = 2,000 × 2,880 = 5,760,000.↩

(a) Write −45 in 8-bit two's complement. (b) What does 100 + 50 give in 8 bits? (c) Encode 10.75 as an IEEE 754 single-precision number.

1. a. 45 = 0010 1101; inverted 1101 0010; plus one 1101 0011 = 0xD3. Check: 0xD3 = 211 = 256 − 45.
2. b. 100 + 50 = 150 = 1001 0110, which read as signed is 150 − 256 = −106. Two positives gave a negative, so signed overflow occurred.
3. c. 10.75 = 1010.11_2 = 1.01011×2^3. The sign is 0; the exponent is 3 + 127 = 130 = 1000 0010; the fraction is 0101 1000 … (23 bits). Grouped: 0 10000010 01011000000000000000000.

Result: 0xD3; −106 with overflow; 0x412C0000.

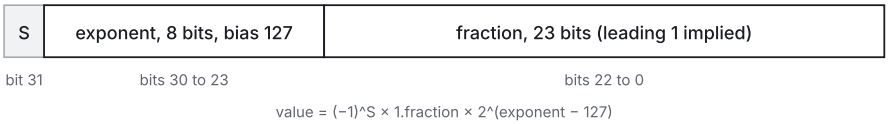


Figure 12.8 IEEE 754 single precision. Double precision uses 11 exponent bits (bias 1,023) and 52 fraction bits.

Two consequences matter in interviews. Floating-point numbers are approximations (0.1 has no exact binary form), so equality tests on them are unsafe. Further, a 32-bit float holds integers exactly only up to 2^{24}, so counters and byte totals must be integers. Conversions between binary, hex and dotted decimal are taught with subnetting in §3.4 (p. 174): for example 0xC0A80101 is 192.168.1.1.

Memory hierarchy

No memory is both large and fast, so computers stack several. Each level holds a subset of the one below it, and a program that reuses nearby data (*locality*) sees mostly the fast levels. Temporal locality means recently used items are used again; spatial locality means neighbours of a used item are used soon.

Level	Typical size	Typical latency	Note
Registers	under 1 KB	about 0.3 ns	Inside the core
L1 cache	32 to 64 KB	about 1 ns	Per core, split instruction and data
L2 cache	256 KB to 2 MB	about 4 ns	Per core
L3 cache	8 to 64 MB	10 to 40 ns	Shared
DRAM	8 to 512 GB	80 to 100 ns	Main memory
NVMe SSD	0.5 to 8 TB	10 to 100 µs	About 1,000 times slower than DRAM
Network, same data centre	n/a	about 0.5 ms round trip	See Pillar 5 (p. 302)

These are orders of magnitude, not specifications; what an interviewer wants is the ratios, namely that each level costs about ten to a hundred times more than the one above.

Cache organisation

A cache holds *blocks* (lines) of, typically, 64 bytes. An address is split into three fields. The *offset* selects the byte within a block: $\log_2$(block size) bits. The *index* selects the set: $\log_2$(number of sets) bits. The *tag*, the remaining high bits, is stored with the block and compared to confirm a hit.

Mapping	Where a block may go	Strength	Weakness
Direct-mapped	Exactly one line (index = block mod lines)	Fast, simple	Conflict misses
k-way set-associative	Any of *k* lines in one set	Balanced; common	Comparators in parallel
Fully associative	Any line	No conflict misses	Costly; used in TLBs and TCAM-like structures

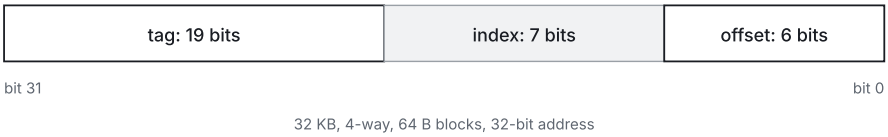


Figure 12.9 The address split of the worked example below; the index picks one of 128 sets, and four tags are compared at once.

WORKED EXAMPLE

A 32 KB, 4-way set-associative cache has 64-byte blocks and a 32-bit address. Find the three fields.

1. Offset: $64 = 2^6$, so 6 bits.
2. Number of blocks = 32,768 / 64 = 512; sets = 512 / 4 = 128 = 2^7, so 7 index bits.
3. Tag = 32 − 7 − 6 = 19 bits.

Result: tag 19, index 7, offset 6.

WORKED EXAMPLE

Block addresses 3, 11, 3, 19, 11 are accessed in an 8-block cache that starts empty. Count the hits for direct-mapped, 2-way (LRU) and fully associative (LRU).

1. Direct-mapped: index = address mod 8. All of 3, 11 and 19 map to line 3, so each access evicts the previous block: 0 hits.
2. 2-way: 4 sets, index = address mod 4, so all map to set 3, which holds two blocks. Sequence: 3 miss, 11 miss, 3 hit, 19 miss (evicts 11, the least recently used), 11 miss (evicts 3): 1 hit.
3. Fully associative: all eight lines are free to hold them, so only 3 distinct misses occur: 3, 11, 19 miss; the second 3 and the second 11 hit: 2 hits.

Result: 0, 1 and 2 hits. More associativity removes conflict misses.

Performance is summarised by the *average memory access time*: AMAT = hit time + miss rate × miss penalty. With a 1 ns hit, a 5% miss rate and a 100 ns penalty, AMAT = 1 + 0.05 × 100 = 6 ns, so the cache hides most but not all of the DRAM delay. On a write, a *write-through* cache updates memory at once (simple, high traffic); a *write-back* cache marks the line dirty and writes it on eviction (less traffic, more complexity). Routers use content-addressable memory (TCAM) for

longest-prefix match because it compares a key against every entry in one cycle, at much higher cost and power per bit than SRAM; the trade-off is solved in Q12.33 (p. 733).

Pipelining

A pipeline overlaps the stages of consecutive instructions, as a car factory overlaps its assembly stations. The classic five-stage RISC pipeline has *fetch*, *decode*, *execute*, *memory* and *write-back*. With k stages and n instructions the pipeline takes $k + (n - 1)$ cycles instead of kn, so the speedup is $kn/(k + n - 1)$. For 100 instructions on five stages: $500/104 \approx 4.8$, approaching k as n grows. Switch ASICs use the same idea: a packet flows through parser and match-action stages, and one packet completes per clock.

Hazard	Cause	Remedy
Structural	Two instructions need the same hardware in one cycle	Duplicate the unit (separate instruction and data caches)
Data (read after write)	An instruction needs a result not yet written	Forwarding; stall one cycle for a load followed by a use
Control	A branch changes the next instruction	Branch prediction; flush on a wrong guess

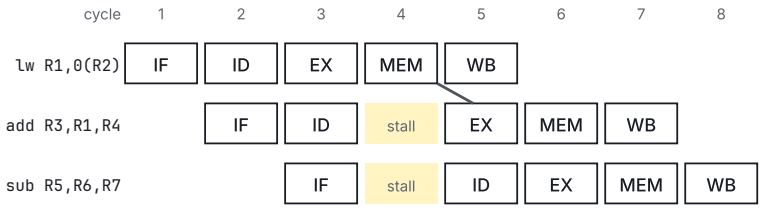


Figure 12.10 A load followed at once by a use of its result: even with forwarding, the add must wait one cycle for the memory stage, so one bubble delays every later instruction.

Interrupts, polling and the NIC

When a packet arrives, the NIC copies it into memory by *DMA* (direct memory access) through a ring of descriptors and then signals the CPU. Signalling by *interrupt* costs a context switch per packet, which is ruinous at millions of packets a second, whereas *polling* wastes cycles when idle. Linux therefore uses NAPI: the first packet raises an interrupt, further interrupts are masked, and the kernel polls in batches until the ring is empty. The mechanism is taught in §7.8 (p. 441); the questions are in Q12.32 (p. 733).

> INTERVIEW LENS
>
> QUALCOMM · BROADCOM · ARISTA · CISCO
>
> Typical asks are two’s complement conversions, the tag/index/offset split, and why a cache helps. A strong answer shows the arithmetic in one line and states the assumption (byte-addressed, block size in bytes). Expect “what happens on a cache miss?” as a follow-up: the line is fetched, an old line is evicted, and the pipeline stalls for the miss penalty.

> CHECK YOURSELF
>
> 1. Range of a 12-bit two’s complement integer. [1]
> 2. A 16 KB direct-mapped cache with 32-byte blocks and 32-bit addresses: how many index bits? [2]
> 3. A 6-stage pipeline runs 200 instructions with no stalls. Speedup over no pipelining? [3]

4. AMAT with a 2 ns hit, 4% miss rate and 60 ns penalty. [4]

12.11 Light system design for networking roles MUST · ■■□

A design question gives a vague goal (“design the network for a new campus”) and thirty-five minutes. Nobody expects a finished product. The panel watches whether the candidate asks before drawing, puts numbers on the problem, names failure modes and states trade-offs. All three designs below follow one six-step method, and each ends in a skeleton that can be redrawn from memory.

> BIG IDEA
>
> A design answer is judged by its reasoning order (requirements, numbers, structure, failure), so a modest design reached in that order beats an elaborate one drawn first.

The six-step template

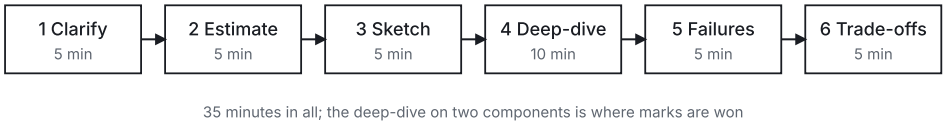


Figure 12.11 The six-step design template with a time budget.

1. **Clarify.** Functional needs, then non-functional ones: scale, latency, availability, security, budget. Ask, then write the answers on the board.
2. **Estimate.** Users, requests per second, bandwidth, storage, with round numbers and stated assumptions.
3. **Sketch.** Five to eight boxes and the arrows between them.
4. **Deep-dive.** Choose the two components that carry the risk and go one level down.
5. **Failure and scale.** What breaks first, and what happens at ten times the load?
6. **Trade-offs.** Say what was given up, and what you would revisit with more time.

The numbers in step 2 are quick if a few conversions are fixed in memory.

Quantity	Value	Use
2^{10}, 2^{20}, 2^{30}, 2^{40}	about 10^3, 10^6, 10^9, 10^{12}	Sizes
Seconds in a day	86,400 (about 10^5)	Daily to per-second
1 Gb/s sustained	10.8 TB per day	Bandwidth to volume
Availability 99.9%, 99.99%	8.76 hours, 52.6 minutes of downtime per year	Targets
Latency: same rack, same city, across India, across the world	0.1 ms, 2 ms, 30 ms, 150 ms or more	Placement

Design a campus network

Clarify. Campus Net: five buildings, 8,000 users, wired and Wi-Fi, laboratories, hostels, guest access, CCTV and IoT, availability 99.9%, two internet providers. Ask about budget, existing cabling, and whether a data-centre room exists.

1. −2048 to 2047.↩
2. 16,384 / 32 = 512 blocks = 2^9, so 9 bits.↩
3. 6 × 200/(6 + 199) = 1,200/205 ≈ 5.85.↩
4. 2 + 0.04 × 60 = 4.4 ns.↩

Estimate. Peak concurrency 40% and 2 Mb/s each gives 8,000 × 0.4 × 2 = 6,400 Mb/s = 6.4 Gb/s of internet demand. Two 10 Gb/s uplinks in active-active use give N+1 protection: after one fails, the survivor still carries 6.4 Gb/s. The 99.9% target allows 8.76 hours of downtime a year, which rules out any single-box core. With two devices per user there are about 16,000 endpoints.

Sketch. A hierarchical design, as in Figure 12.12 (p. 717): access switches in each building, a pair of distribution switches per building, a pair of core switches, and a pair of edge firewalls to the two providers.

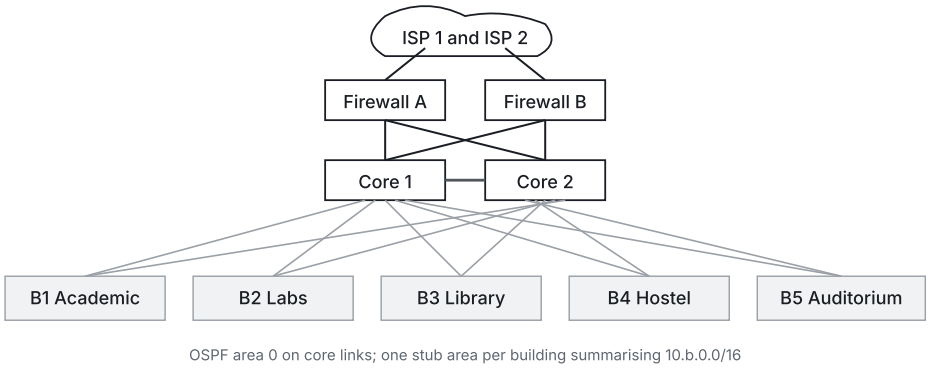


Figure 12.12 Campus Net, collapsed to one line per building: dual cores, dual firewalls and dual uplinks from each building (each box contains its own distribution pair and access layer), so that no single device failure isolates a building.

Deep-dive 1: addressing and VLANs. One /16 per building gives each distribution pair one summary route. The second octet is the building, and the third octet is the VLAN, grouped by role.

Building	Users	Prefix	Third octet by role
B1 Academic and administration	1,500	10.1.0.0/16	10 to 19 staff; 30 to 59 Wi-Fi
B2 Laboratories	1,500	10.2.0.0/16	20 to 29 lab wired; 30 to 59 Wi-Fi
B3 Library and seminar halls	1,000	10.3.0.0/16	30 to 59 Wi-Fi; 60 to 69 CCTV and IoT
B4 Hostels	3,000	10.4.0.0/16	30 to 59 Wi-Fi (24 /24 subnets for 6,000 devices)
B5 Auditorium and sports	1,000	10.5.0.0/16	30 to 59 Wi-Fi
Infrastructure	n/a	10.250.0.0/16	Loopbacks and /31 point-to-point links
Guest, data room	n/a	10.251.0.0/16, 10.100.0.0/16	Guest isolated; servers behind the firewall

The hostel plan is checked by arithmetic: 3,000 users × 2 devices = 6,000 addresses, and at 254 hosts per /24 that needs 24 subnets, which fits in the 30 slots (30 to 59) reserved for Wi-Fi. Subnetting method is in §3.4 (p. 174).

Deep-dive 2: resilience. Each distribution pair runs a first-hop redundancy protocol (HSRP or VRRP, §4.8 (p. 265)) or an anycast gateway, with the active gateway matched to the spanning-

spanning tree is confined to one closet and equal-cost paths are used in parallel. Loss of one core switch then costs a reconvergence of a few hundred milliseconds, which TCP sessions survive; Q12.40 (p. 733) walks through it.

Wi-Fi and security. A wireless controller (or cloud-managed access points) maps each SSID to a VLAN: staff with WPA3-Enterprise and 802.1X, students with 802.1X or a passphrase per user, guests with a captive portal and internet-only access. CCTV and IoT sit in their own VLANs behind ACLs that allow only the recording server. DHCP snooping, dynamic ARP inspection and port security protect access ports, and the firewalls enforce zones, NAT and logging (Pillar 9 (p. 538)).

Failure and scale. The single points that remain are the building closet itself and the provider pair if both enter by one duct; one answer is two physical paths. At ten times the load the first limit is the firewall's throughput, not the links, so we size it by inspected Gb/s.

Trade-offs. A collapsed core (distribution and core on the same pair) saves cost for five buildings; the separate core is chosen here because six or more buildings would make the cabling mesh unmanageable.

Design a rate limiter

A *rate limiter* caps how many requests a client may make in a time window, to protect a service from overload and abuse.

Clarify. Limit per user, per IP and per API key? A limit of, say, 100 requests per minute, with some burst allowed? Enforced on many gateway servers? Fail open (allow when the limiter is down) or fail closed? What should the client see when refused?

Estimate. Ten million registered users, 10% active at peak, 0.5 requests per second each: $10^6 \times 0.5 = 500{,}000$ checks per second. One in-memory store handles about 100,000 operations per second, so we need at least 5 shards; 8 allow headroom. State per active user is a counter and a timestamp, roughly 64 bytes, so $10^6 \times 64$ B = 64 MB in all.

Algorithm	How it works	Strength	Weakness
Token bucket	A bucket of capacity *b* refills at rate *r*; a request takes a token	Allows bursts up to *b*; two numbers per key	Burst can be unwelcome
Leaky bucket	Requests join a queue drained at constant rate	Smooth output	Delays; no burst
Fixed window	Counter per clock minute	One counter	Up to 2 × limit around a boundary
Sliding-window log	Store every timestamp	Exact	Memory per request
Sliding-window counter	Weight the previous window by the overlap	Cheap and close to exact	Approximate

WORKED EXAMPLE

Limit 100 per minute with the sliding-window counter. The previous minute saw 80 requests; the current minute has 30 so far and is 15 seconds old. Allow the next request?

1. Overlap of the sliding window with the previous minute: $(60 - 15)/60 = 0.75$.
2. Estimate = $80 \times 0.75 + 30 = 60 + 30 = 90$.
3. 90 is below 100, so the request is allowed and the counter becomes 31.

> **Result: allowed; a token bucket with *r* = 10/s and *b* = 20 would likewise pass 20 requests at once, then 10 per second, i.e. 70 in the first 5 seconds.**

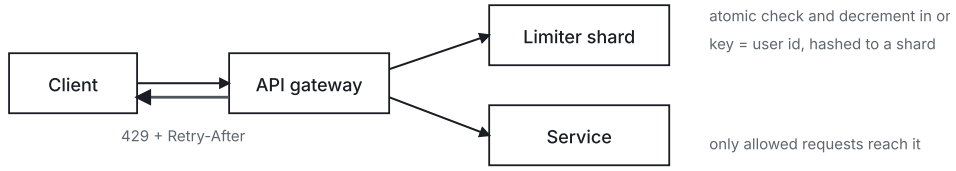


Figure 12.13 The limiter sits at the gateway; each user maps to one shard, so a check is one network round trip.

Deep-dive. With many gateways the counters must be shared. A store such as Redis does the check atomically with a short script or `INCR` plus `EXPIRE`, because a read followed by a write from two gateways would let both pass. A local cache of tokens at each gateway, refilled in batches, trades a small over-admission for fewer round trips. Refused clients receive HTTP 429 with a `Retry-After` header.

Failure. If the store is down, public read endpoints fail open and money-moving endpoints fail closed. A hot key (one abusive client) lives on one shard, which is acceptable since its traffic is mostly refused. The router analogue is the interface policer, which drops excess traffic, versus the shaper, which queues it (§4.11 (p. 274)); the coding version is in §1.15 (p. 60).

Design a CDN

A *content delivery network* stores copies of content in many *points of presence* (PoPs) near users, so that most requests never reach the origin.

Clarify. Static files and video; users across India and a smaller share abroad; a target hit ratio of 95%; TLS everywhere; purge within a minute.

Estimate. Ten million daily users at 50 MB each: $10^7 \times 50$ MB = 500 TB a day. Dividing by 86,400 s and multiplying by 8: $500 \times 10^{12} \times 8 / 86{,}400 \approx 46$ Gb/s average, and about 140 Gb/s at three times the mean for the evening peak. With a 95% hit ratio the origin supplies only 5%: about 2.3 Gb/s average and 7 Gb/s at peak, which one origin cluster can serve.

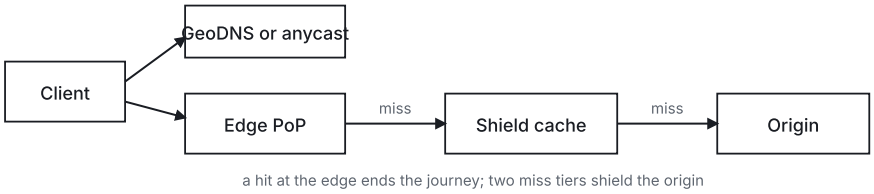


Figure 12.14 Request path in a two-tier CDN: the first hit wins, and every miss goes one tier closer to the origin.

Deep-dive 1: request routing and caching. The client's DNS lookup for `cdn.example.com` returns the address of a nearby PoP, chosen by GeoDNS (by the resolver's location) or by anycast (the same address announced from every PoP, so BGP delivers the nearest); the comparison is in Q12.38 (p. 733). The cache key is the URL plus the headers that change the response; the time to live comes from `Cache-Control: max-age`. Inside a PoP, consistent hashing of the key across cache servers avoids storing every object everywhere and keeps most keys in place when a server joins or leaves.

Deep-dive 2: freshness. Purge by key or tag through a control plane that fans out to every PoP; use versioned file names (`app.3f9a.js`) so that most files never need purging. Popular items are served from memory, long-tail items from disk, and the shield tier collapses simultaneous misses for one object into a single origin request.

Failure and scale. A failed PoP is withdrawn from anycast or DNS and traffic moves to the next nearest, so spare capacity of about 30% is kept. Volumetric attacks are absorbed because capacity is spread over many PoPs. The origin is protected by the shield and by serving stale content while revalidating.

Trade-offs. A longer TTL raises the hit ratio but delays updates; more PoPs reduce latency but increase purge cost and the number of certificates to manage.

Variants

A layer-4 load balancer spreads connections by the five-tuple using ECMP and consistent hashing, and answers on a virtual IP; a layer-7 balancer terminates TLS and routes by URL. Global traffic management combines health-checked DNS with anycast. Details and the 1-million-connections-per-second design are in Q12.39 (p. 733) and Pillar 5 (p. 302).

> INTERVIEW LENS
>
> CISCO · JUNIPER · AMAZON · GOOGLE · NOKIA
>
> For networking roles the campus design is the likeliest question, then a limiter or load balancer. A strong answer opens with requirements and one calculation, draws no more than eight boxes, names a redundancy mechanism at each layer, and closes with failure modes. Expect “what if a core switch dies?”, “how does this scale to ten campuses?” and “where does security sit?”.

> CHECK YOURSELF
>
> 1. A campus of 12,000 users, 35% concurrency at 1.5 Mb/s. Peak demand? [1]
> 2. Hit ratio drops from 95% to 90% on 140 Gb/s of peak traffic. Origin load at peak before and after? [2]
> 3. Which rate-limit algorithm permits a burst of *b* requests? [3]

12.12 Behavioural interviews: STAR, Amazon principles and the offer MUST · ■■□

A behavioural question asks for evidence of past conduct: “Tell me about a time when …”. The panel reasons that what you did before is the best predictor of what you will do, so a vague answer fails even when the candidate is capable. Amazon builds its whole loop on this idea, and Google, Microsoft and Cisco use the same format in their managerial and HR rounds. Preparation consists of a small bank of true stories and one structure for telling them.

> BIG IDEA
>
> A behavioural answer is believed when it is specific, so a story that names your action, a number and a lesson beats a larger story told in generalities.

STAR and STAR-L

STAR is Situation, Task, Action, Result. Situation and Task together take about 20 seconds: where, when, what was at stake, and which part was *yours*. Action takes about 60 seconds and is

1. 12,000 × 0.35 × 1.5 = 6,300 Mb/s = 6.3 Gb/s.↩
2. 5% of 140 = 7 Gb/s before; 10% of 140 = 14 Gb/s after, so the origin load doubles.↩
3. The token bucket, with capacity *b*.↩

the core: three concrete steps in the first person. Result takes 20 seconds and carries a number. For failure questions add *L*, the learning (STAR-L), in 10 seconds. A full answer therefore lasts 90 to 120 seconds; for a 30-second version use CAR (Challenge, Action, Result).

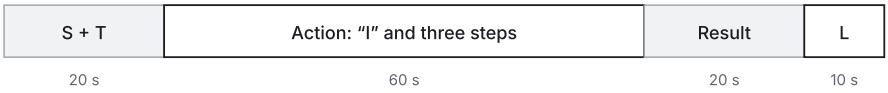


Figure 12.15 The time split of a 110-second STAR-L answer: most of the airtime belongs to what you did.

The template below can be filled in the blanks and then spoken aloud until it sounds natural.

> In [semester or internship], our team of [n] was [situation]. I was responsible for [task]. First I [action 1, with a tool or technique], then I [action 2], and when [obstacle] happened I [action 3]. As a result, [number: latency down 40%, three days saved, a grade, adoption]. Next time I would [learning].

WORKED EXAMPLE

Prompt: “Tell me about a time you helped a team that was falling behind.” The stories here are fictional models; use your own facts.

Weak. “In our final-year project we had some delays, so we all worked hard and managed to finish. It was a good lesson in teamwork.”

Strong.

1. **S.** Four of us were building an SNMP monitor for the hostel network; two weeks before the demo, the polling module, owned by a teammate, lost about one reply in five.
2. **T.** I was in charge of testing, and the demo date was fixed.
3. **A.** First, I wrote a script that sent 1,000 queries and logged the misses; they clustered after a 2-second timeout. Second, I proposed a longer timeout with two retries and paired with him for an evening to implement it. Third, I added the script to the repository as an automated test so the fault could not return.
4. **R.** Loss fell from about 20% to under 1% over 1,000 queries, and the demo ran without a miss.
5. **L.** I would now run a load test in the first week, not the last.

Result: the strong version has a measurable problem, three actions that are all “I”, a number and a lesson; the weak one has none of these.

The Amazon Leadership Principles

Amazon states sixteen *Leadership Principles*, and every interviewer in a loop owns one or two. A designated *Bar Raiser* from outside the hiring team probes the answers and can veto the offer. In Network Development Engineer loops the principles are mixed into technical rounds too: “tell me about an outage you debugged” tests Dive Deep. The wording below is our paraphrase; read Amazon’s own page before the interview.

#	Principle	Plain meaning	What a fresher’s evidence looks like
1	Customer Obsession	Start from the user’s need, and keep their trust	You changed a design after watching users struggle

2	Ownership	Treat the whole outcome as yours, beyond your job description	You fixed what nobody assigned to you
3	Invent and Simplify	Look for a simpler or new way, and accept being misunderstood for a while	You scripted a manual task
4	Are Right, A Lot	Sound judgement, from data and from seeking views that differ	You changed your mind on evidence
5	Learn and Be Curious	Keep learning and exploring	You taught yourself a protocol for a project
6	Hire and Develop the Best	Raise the standard of the people around you	You mentored a junior
7	Insist on the Highest Standards	Refuse to ship what you know is flawed	You redid work that passed but was fragile
8	Think Big	Aim for a bolder result than the obvious one	You designed for ten times the load
9	Bias for Action	Decide when speed matters; most choices can be reversed	You chose with incomplete data and corrected it
10	Frugality	Do more with less	You used free tools to meet a budget
11	Earn Trust	Listen, speak candidly, admit mistakes	You owned an error in front of the team
12	Dive Deep	Know the details; audit the data	You root-caused a fault using packet captures
13	Have Backbone; Disagree and Commit	Challenge respectfully, then commit fully	You objected, lost the vote and delivered anyway
14	Deliver Results	Meet commitments despite obstacles	You shipped by the deadline despite a setback
15	Strive to be Earth's Best Employer	Create a safe, productive, inclusive place	You made a team meeting work for quieter members
16	Success and Scale Bring Broad Responsibility	Consider the wider effects of what you build	You considered privacy or misuse in a project

Building the story bank

Eight true stories, each tagged to two to four principles, cover most questions; two spares are kept for later rounds, because the interviewers in one loop each look for a fresh story. Mine them from projects, internships, hackathons, club work, lab incidents, sport and part-time work. Quantify each (hours saved, latency, users, marks).

#	Theme	Primary	Secondary	Prompt it answers
S1	Helped a user	Customer Obsession	Earn Trust, Deliver Results	"Beyond what was asked"
S2	Beyond my role	Ownership	Bias for Action	"Took on something not yours"
S3	Root-caused an outage	Dive Deep	Highest Standards	"Used data to find the cause"
S4	Failure and recovery	Learn and Be Curious, Earn Trust	Ownership	"A time you failed"

S5	Disagreed with a peer	Have Backbone; Disagree and Commit	Earn Trust	“Disagreed with a senior”
S6	Decided with little data	Bias for Action	Are Right, A Lot	“Incomplete information”
S7	Automated a manual task	Invent and Simplify	Frugality, Deliver Results	“Found a simpler way”
S8	Raised the quality bar	Insist on the Highest Standards	Hire and Develop the Best	“Refused to compromise”

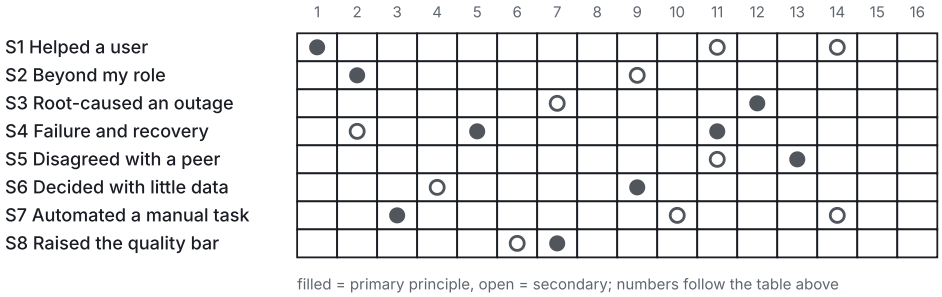


Figure 12.16 The story-by-principle matrix, with example entries. A complete bank has every principle from 1 to 16 covered at least once.

One story serves several principles by changing the angle, not the facts. The SNMP story above, for example, is told for *Ownership* by stressing that the polling module was not yours, for *Dive Deep* by stressing the 1,000-query log and the clustering of misses, and for *Deliver Results* by stressing the fixed demo date.

> TRAP
>
> *Claim:* “We” is more modest and therefore better. *Why it is wrong:* the panel is hiring you, and a story told in “we” gives no evidence of what you did. *Correct statement:* say “we” for the situation and “I” for every action; add the team’s result and your own share of it.

“Tell me about yourself”

This question opens most interviews, and the usual error is to recite a résumé. Use *Present, Past, Future*, in 60 to 90 seconds.

> I am [name], a final-year [branch] student at [college]. **Present:** I am focused on networking and systems; I [one project in one line, with a number]. **Past:** I got here through [an internship, CCNA, research or competitive programming: one or two proofs]. **Future:** I am drawn to [company]’s [specific product] because [link to your skills]. That is why I applied for this role.

Vary the lead by round: the technical round opens with the project to be questioned; the managerial round opens with impact and teamwork; the HR round adds one personal trait with evidence. The “Future” line is what makes the answer yours.

Company	Products to name in the “Future” line
Cisco	Catalyst and Meraki switching and Wi-Fi, SD-WAN, Webex
Juniper	Junos, Mist AI-driven wireless and wired
Qualcomm	Snapdragon modems and connectivity (Wi-Fi, 5G)

Arista	EOS, CloudVision, cloud-scale data-centre switching
Amazon	AWS backbone, VPC and Transit Gateway
Microsoft	Azure networking and the global WAN
Nokia	SR Linux, IP routing and optical
Broadcom	Switch silicon for data centres

Common HR questions and answer frames

Question	Frame
Why this company?	One product or team, one skill of yours it needs, one thing you want to learn
Why networking?	A moment that hooked you (a lab, an outage), then the scale of the field
Strengths	Two, each with a one-line example
Weakness	A real one, plus what you do about it (never "I work too hard")
Where in five years?	Deeper technical ownership of a network, possibly leading a small team
Relocation, shifts, on-call, bond	Answer honestly; networking operations often involve on-call, so show you understand
A conflict	A story from S5, ending in a working relationship
Why should we hire you?	Skills matched to the job description, with one proof
Higher studies	Honest, with a commitment signal ("I plan to work for several years first")
Salary expectation (fresher)	"I would like the company's standard for this role; I am focused on the work"
Questions for us	On-call load, the team's tools, how success is measured at six months

Offers in India

A fresher's offer is a bundle, not a number. *CTC* (cost to company) includes items that never reach you as cash: the employer's provident-fund share, insurance, gratuity, and bonuses that depend on performance. Separate these before comparing.

Component	Offer A, "₹30 LPA"	Offer B, "₹24 LPA"
Fixed base	₹20 L	₹24 L
Variable pay (stated)	₹6 L	none
One-time joining bonus	₹4 L (clawback if you leave within two years)	none
Recurring cash if variable pays 80%	20 + 4.8 = ₹24.8 L	₹24 L
First-year cash	24.8 + 4 = ₹28.8 L	₹24 L
Recurring cash if variable pays 50%	20 + 3 = ₹23 L	₹24 L

Offer A is better in year one and, at 80% payout, in later years by ₹0.8 L; at 50% payout it is worse by ₹1 L every year. Hence compute *guaranteed annual cash* separately from the headline, and ask what the variable actually paid last year. Other warning signs are variable pay above 30% of CTC, shares in an unlisted start-up counted at face value, service bonds, and long clawbacks.

On-campus offers are usually fixed by the company and not negotiable on base, and some placement cells forbid holding two offers, so read the policy first. Off-campus and lateral offers can move on the joining bonus, the joining date, location or team, the share grant, and a written review date. Choose one or two items that matter to you. The script is: thank them, state enthusiasm, give one competing offer or market data, ask for one specific number, and stop speaking. Let the company name the figure first where possible.

> INTERVIEW LENS
>
> AMAZON · GOOGLE · MICROSOFT · CISCO
>
> Amazon scores each answer against its principle and asks follow-ups until the story is exhausted: “What was your part?”, “What was the data?”, “What would you do differently?” Prepare one fact for each. Google and Microsoft look for collaboration and growth; Cisco’s managerial round adds relocation, flexibility and project depth. Model answers to the standard prompts are in Q12.41 to Q12.50 (p. 733).

> CHECK YOURSELF
>
> 1. How many seconds of a 110-second answer should be Action? [1]
> 2. Offer: base ₹18 L, variable ₹4 L paid at 75%, no bonus. Recurring cash? [2]
> 3. Rewrite “We fixed the bug” so that it shows your action. [3]

12.13 Résumé and project deep-dives for networking roles

MUST · ■□□

The résumé does two jobs. It passes a first screen, by a person who reads for about thirty seconds or by software that matches keywords, and it becomes the agenda of the interview, because every line on it may be questioned. Juniper, Arista and Broadcom panels are reported to spend most of a technical round on one or two résumé projects. A résumé is therefore a list of promises, and the interview checks whether each can be kept.

> BIG IDEA
>
> Put on the page only what you can defend for five minutes at a whiteboard, because the interviewer will choose the line you are least ready for.

Layout of the one-page résumé

A fresher’s résumé is one page, in a single column, exported as PDF. Multi-column layouts, tables, icons and skill bars are often misread by applicant-tracking software, and a photograph or marital status is not wanted. Order the sections by strength: for most students, projects come before certificates.

1. About 60 seconds, a little over half. ↩
2. 18 + 0.75 × 4 = ₹21 L. ↩
3. “I reproduced the fault with a packet capture, found the missing retransmission timer and patched it.” ↩

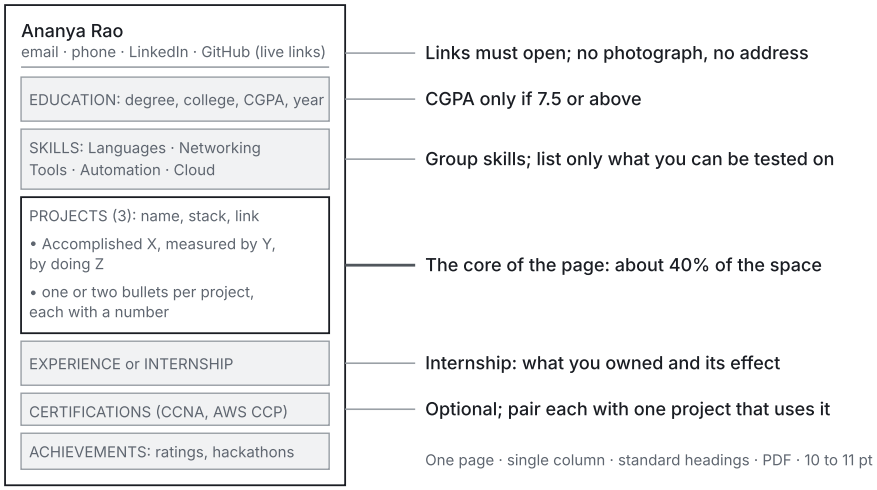


Figure 12.17 The one-page layout of a fresher's networking résumé, with the projects block emphasised.

Skills are grouped so that a reader can find a keyword at once: *Languages* (C, Python), *Networking* (TCP/IP, VLAN, STP, OSPF, BGP, ACLs, NAT), *Tools* (Wireshark, tcpdump, GNS3 or EVE-NG, Linux), *Automation* (Netmiko, NAPALM, Ansible, REST and NETCONF) and *Cloud* (AWS VPC). Align the keywords with the job description for network development, network consulting or software-in-networking roles, but only those you could explain. Red flags include "OSI model" as a skill, percentage bars for skills, vague buzzwords and links that return an error.

The bullet formula

Each bullet follows Google's *XYZ* form: "Accomplished **X**, as measured by **Y**, by doing **Z**." It forces a result and a method into one line and starts with a verb in the past tense.

Weak	Strong
Worked on a network automation project using Python	Cut lab configuration time from 2 hours to 5 minutes (about 96% less) by writing a Python and Netmiko script that pushes templated configurations to 12 routers
Made a file transfer tool	Reduced packet loss in a UDP file-transfer tool in C from 12% to 0.5% by adding selective-repeat retransmission with a 16-packet window
Learnt OSPF and BGP	Built a three-area OSPF and two-AS eBGP lab in GNS3 with route maps that prefer the primary link, and verified failover in under 5 seconds with traceroute
Member of coding club	Solved 450 problems on an online judge and reached a rating of 1,650 (Codeforces)

The percentage in the second row of the table is checked: $1 - 5/120 = 0.958$. Never invent a number; use measurements you actually took, and state the conditions if asked ("12 routers in a virtual lab").

Choosing and presenting projects

Network interviewers prefer a project that is measurable, network-relevant and your own code. Strong choices are a multi-area OSPF and eBGP lab; a packet sniffer in C on raw sockets; a small

TCP over UDP with a sliding window; a DHCP or DNS server in Python; an SDN controller application; a monitoring dashboard fed by SNMP or gNMI; a VXLAN-EVPN lab in containerlab. The labs and mini-projects of Pillars 8 to 11 are designed to be listed here. Give each repository a README with a one-line purpose, an architecture diagram, how to run it, results, and tests; a licence file completes it.

How to defend a project

An interviewer asks, in some order, ten things. Prepare a written answer to each, in a two-minute version and a ten-minute version.

#	Question	What a good answer contains
1	What problem, and why does it matter?	One sentence of need, one of who suffers
2	Draw the architecture	Five boxes and the data path, drawn without notes
3	What did you build, and what did others?	Your share, stated plainly
4	What was the hardest bug and how did you find it?	Symptom, method, tool, cause (a Dive Deep story)
5	What alternatives did you reject, and why?	Two options with a reason each
6	How did you test it?	Unit tests, fault injection, a measured baseline
7	What were the results?	Numbers with conditions
8	What breaks at ten times, and at a thousand times, the load?	The first bottleneck and its fix
9	What about security?	Credentials, input validation, least privilege
10	What would you change?	One honest flaw and the next step

WORKED EXAMPLE

Prepare the answer to “why did you choose Netmiko, and not NETCONF, for the config pusher?”

1. State the facts of the situation: the lab devices were three vendors, two of which had no NETCONF support in the images available.
2. Name the cost of the choice: screen-scraping is brittle, since output formats change between versions.
3. Name the mitigation and the next step: parsing with templates, a test per command, and a plan to move supported devices to NETCONF.

Result: the answer shows a constraint, a trade-off and an improvement, which is what “why” questions look for.

TRAP

Claim: “Listing many technologies makes the résumé stronger.” *Why it is wrong:* each one is a licence for a question, and three missed questions end a project discussion. *Correct statement:* list fewer tools, and be able to explain how each works, why you chose it and what its

limits are. Likewise, claiming the team's work as your own is detected by a single probe: "which part did you write?"

INTERVIEW LENS

JUNIPER · ARISTA · BROADCOM · CISCO · AMAZON

The deep-dive is usually the longest block of a technical round. The panel is estimating your real contribution, your understanding of the layers below your code, and your honesty about limits. A strong candidate draws the architecture unprompted, brings a number, and answers "what would you change?" with a specific flaw. The model answers are in Q12.51 to Q12.54 (p. 733).

CHECK YOURSELF

1. Rewrite "Improved performance of the DNS server" in the XYZ form, with placeholder numbers. [1]
2. A script cut a task from 90 minutes to 6. What is the percentage reduction? [2]

12.14 Where it is asked, drills and the portfolio project

OPTIONAL · ■□□

Knowledge from this pillar decays faster than the networking in earlier pillars, because it is practised in speed and speech. This final section therefore says which company asks what, how to drill in thirty-minute blocks, and how to gather the whole pillar into one portfolio.

Where each topic is asked

The matrix records reported emphasis, not guarantees: ● often, ◖ sometimes, ○ rarely. Verify against your drive's notice.

Topic	Cisco	Qualcomm	Google	Microsoft	Amazon NDE	Juniper	Arista	Nokia	Broadcom
Aptitude in the OA	●	●	○	○	◖	●	◖	●	●
CS MCQs (C, OS, DBMS, networks)	●	●	○	◖	◖	●	◖	●	●
OOP viva	●	●	◖	●	◖	●	●	●	●
DBMS and SQL	●	◖	◖	●	◖	◖	◖	●	◖
Computer organisation, bits	◖	●	◖	◖	○	◖	●	◖	●
Light system design	●	○	●	●	●	◖	◖	◖	○
Behavioural (STAR)	●	◖	●	●	●	◖	◖	◖	◖
Résumé and project deep-dive	●	●	●	●	●	●	●	●	●

1. For example: "Raised throughput of a Python DNS server from 800 to 3,200 queries per second by caching answers for their TTL and using asynchronous sockets."↩
2. 1 − 6/90 = 0.933, about 93%.↩

Drills

ID	Drill	Time
D12A.1 to D12A.12	A 20-question timed set for each topic group of §12.2 (p. 690) to §12.6 (p. 701); use the forty questions after the Problem Set first	25 min each
D12A.M1	Full mock paper: 50 questions, 60 minutes, pacing as in Figure 12.1 (p. 688)	60 min
D12E.1	Three design mocks from §12.11 (p. 717), the six steps timed, a friend as interviewer	3 × 35 min
D12F.1	Story-mining worksheet: list ten incidents, tag each with principles	60 min
D12F.2	Record answers to ten behavioural prompts; score structure, “I”, number, learning, time	60 min
D12G.1	Rewrite your résumé against the twenty-point checklist in the Revision Spread	60 min
D12G.2	Fill the deep-dive table of §12.13 (p. 726) for your main project; time the two-minute version	60 min

The daily routine for aptitude is thirty minutes: ten minutes revising one formula card, fifteen minutes on a timed set, five minutes entering every miss in the error log. Keep the log honestly; its trend tells you which of the three error types to attack.

Date	Topic	Question	Error type	Fix
12 Mar	Time and work	A, B, C: C alone	Careless (read 5 days as 6)	Underline the numbers
13 Mar	P and C	Vowels together	Concept (forgot inner order)	Glue, arrange, multiply

The three types are *concept* (you did not know the rule), *careless* (you knew it) and *speed* (right but too slow). Concept errors need a card; careless errors need a habit such as writing the base before computing; speed errors need timed sets.

> LAB
>
> `LAB 12.1 · 30 MIN` Build the inventory of §12.9 (p. 709) and watch an index change a query plan. Save the statements of that section in `inventory.sql`, then run:

```
import sqlite3
c = sqlite3.connect(":memory:")
c.executescript(open("inventory.sql").read())
q = "SELECT s.city, COUNT(d.name), COALESCE(SUM(d.ports), 0) FROM site s " \
    "LEFT JOIN device d ON d.site_id = s.site_id GROUP BY s.city ORDER BY 3
DESC"
print(c.execute(q).fetchall())
print(c.execute("EXPLAIN QUERY PLAN SELECT * FROM device WHERE vendor =
'Arista'").fetchall())
c.execute("CREATE INDEX ix_vendor ON device(vendor)")
print(c.execute("EXPLAIN QUERY PLAN SELECT * FROM device WHERE vendor =
'Arista'").fetchall())
```

The key output lines are `[('Main', 4, 104), ('Hostel', 2, 72), ('Lab', 1, 8), ('Annexe', 0, 0)]`, then `SCAN device`, then `SEARCH device USING INDEX ix_vendor (vendor=?)`.

What you should notice: the empty site Annexe survives because of the left join, and the plan changes from a scan of every row to a search through the index. On seven rows the difference is invisible in time; on seven million it decides the query.

Then answer ten graded queries on the same data; the expected results are in the table, and writing each before running it is the point of the exercise.

#	Task	Expected result
1	Devices with at least 24 ports	`sw1`, `sw2`, `sw3`, `sw4`
2	Devices per vendor, largest first	Arista 3, Cisco 2, Juniper 1, Palo Alto 1
3	Cities whose total ports exceed 50	Hostel 72, Main 104
4	Devices from Cisco or Juniper	`r1`, `r2`, `sw1`
5	Sites with no device, using `NOT IN`	Annexe
6	Vendors present at more than one site	Arista
7	Share of all ports held by `sw1`	26.09% (48 of 184)
8	The largest device in Hostel	`sw4`
9	Pairs of devices at the same site	7 (six at Main, one at Hostel)
10	Rank by ports with `RANK()`	`sw1`, `sw2`, `sw4` rank 1; `sw3` 4; `fw1` 5; `r1`, `r2` 6

Mini-project: the Placement Readiness Portfolio

The pillar ends with a deliverable that you can send to a recruiter. Gather: (i) the final one-page résumé; (ii) a GitHub profile with two or three polished repositories from the projects of Pillars 8 to 11; (iii) a story bank of eight STAR stories mapped to the principles, in the matrix of Figure 12.16 (p. 721); (iv) one design write-up, the campus network for your own college with real building names and a plausible IP plan; (v) a two-minute and a ten-minute recording of your main project; (vi) your error-log trend, showing the rate of each error type falling over four weeks.

1. In the matrix, which two themes cover most of the technical rounds? [1]
2. Which error type does a timed set cure? [2]

1. OOP viva and résumé or project deep-dive, which are ● for every company listed.↩
2. Speed errors; concept errors need a formula card and careless errors need a checking habit.↩

Problem Set

Q12.1 A sum doubles in 8 years at simple interest. In how many years does it triple?

ALL · OA · ■□□ · T

30-second answer: 16 years.

Fast method. Doubling means the interest equals the principal after 8 years, so each 8 years adds one principal; tripling needs interest of two principals, which is 16 years. **Conventional.** Rate = 100/8 = 12.5% per annum; for tripling, 200 = 12.5 × T, so T = 16. **Mistake.** Answering 12 by adding half as much again.

Q12.2 Find the unit digit of 7^{95}.

ALL · OA · ■□□ · T

30-second answer: 3.

Approach. The unit digits of powers of 7 cycle as 7, 9, 3, 1 with length 4. 95 = 4 × 23 + 3, so take the third entry: 3. **Mistake.** Reading remainder 0 as "first entry" instead of the last. See §12.2 (p. 690).

Q12.3 A can finish a job in 12 days and B in 18. How long together?

ALL · OA · ■□□ · T

30-second answer: 7.2 days (7 days 4.8 hours).

Fast method. Take 36 units; A does 3 and B does 2 per day; together 5, so 36/5 = 7.2. **Conventional.** 1/12 + 1/18 = 5/36 of the job per day, so 36/5 days. **Check.** 12 × 18/(12 + 18) = 216/30 = 7.2.

Q12.4 Trains of 150 m and 100 m run at 54 and 36 km/h in opposite directions. Crossing time?

ALL · OA · ■■□ · T

30-second answer: 10 seconds.

Approach. Opposite directions: speeds add, 54 + 36 = 90 km/h = 90 × 5/18 = 25 m/s. Distance is the sum of the lengths, 250 m. Time = 250/25 = 10 s. **Mistake.** Subtracting the speeds, which is for the same direction (it would give 250/5 = 50 s).

Q12.5 In how many ways can the letters of NETWORK be arranged so that the vowels are together?

ALL · OA · ■■□ · T

30-second answer: 1,440.

Approach. The vowels E and O are glued into one block, giving 6 objects (N, T, W, R, K and the block): 6! = 720 arrangements. The block can be EO or OE: × 2. All letters are distinct, so no division is needed: 720 × 2 = 1,440. **Mistake.** Forgetting the internal order of the block.

Q12.6 Two cards are drawn without replacement. Probability that both are aces?

ALL · OA · ■■□ · T

30-second answer: 1/221.

Approach. (4/52) × (3/51) = 12/2,652 = 1/221. **Check.** C(4, 2)/C(52, 2) = 6/1,326 = 1/221. **Mistake.** Using 4/52 twice, which assumes replacement.

Q12.7 All routers are devices; some devices are switches. Does "Some routers are switches" follow?

ALL · OA · ■■□ · T

30-second answer: No.

Approach. Draw routers inside devices. The switches overlap devices, but nothing forces the overlap to fall inside routers; a legal drawing puts every switch in the part of devices outside routers, so the conclusion fails in that drawing. A conclusion must hold in every drawing (Figure 12.4 (p. 701)). **Mistake.** Treating "some" as transferable through "all".

Q12.8 Round robin, quantum 2: P1 (arrives 0, burst 5), P2 (1, 3), P3 (2, 1), P4 (3, 2). Average waiting time?

CISCO · QUALCOMM · OA · ■■□ · T

30-second answer: 4.5 time units.

Approach. New arrivals join the queue before the pre-empted process does. Order of execution (Gantt chart): P1 0 to 2, P2 2 to 4, P3 4 to 5, P1 5 to 7, P4 7 to 9, P2 9 to 10, P1 10 to 11.

Process	Completion	Turnaround (completion − arrival)	Waiting (turnaround − burst)
P1	11	11	6
P2	10	9	6
P3	5	3	2
P4	9	6	4

Answer. (6 + 6 + 2 + 4)/4 = 4.5. **Mistake.** Queueing the pre-empted process before the new arrival, which gives a different order; state the convention. See §7.4 (p. 423).

Q12.9 What does `int i = 5; printf("%d %d", i++, ++i);` print?

CISCO · QUALCOMM · OA · ■■□ · T

30-second answer: undefined behaviour; no single output is correct.

Answer. Two modifications of `i` occur with no sequence point between them, and the order of evaluating function arguments is unspecified. We tested two compilers: GCC printed `6 7` and Clang printed `5 7`. **What to choose in an OA.** If "undefined" or "compiler dependent" is an option, choose it. If not, choose the output the setter's own compiler most plausibly produced, and note the doubt. **Mistake.** Giving "5 7" as if it were guaranteed. See §2.12 (p. 121).

Q12.10 Eight people A to H sit round a circular table facing the centre. Find the arrangement.

CISCO · NOKIA · OA · ■■■ · T

30-second answer: clockwise from A: A, H, B, C, E, G, D, F.

Clues. (1) A sits opposite E. (2) B is second to the left of A. (3) C sits immediately to the right of E. (4) D sits opposite B. (5) F sits next to D. (6) G does not sit next to A.

Approach. Facing the centre, left is clockwise. Number the seats 0 to 7 clockwise and put A at 0.

1. Clue 1: E is at 4.
2. Clue 2: B is two seats clockwise of A, so seat 2.
3. Clue 3: the right of E is anticlockwise, so C is at 3.
4. Clue 4: D is opposite B, so seat 6.
5. Remaining people F, G, H take seats 1, 5, 7. By clue 5, F is next to D (seat 6), so F is at 5 or 7; by clue 6, G is not next to A (seats 1 and 7 are), so G is at 5. Then F is at 7 and H is at 1.

Answer. Seats 0 to 7: A, H, B, C, E, G, D, F. **Check.** All six clues hold, and each is needed (dropping any one allows at least two arrangements).

Q12.11 Explain the four pillars of OOP with a networking example.

CISCO · JUNIPER · TECH-1 · ■□□ · S

30-second answer: encapsulation bundles data with the code that guards it (`Interface.set_mtu()` validates the value); abstraction exposes what an object does and hides how (`device.get_config()`); inheritance reuses a base class (`CiscoIOS` extends `Device`); polymorphism gives one call different behaviour by real type (looping over devices calling `get_config()`).

Follow-ups. The mechanism of polymorphism (Q12.13); the difference between encapsulation and abstraction (protect versus simplify). **Mistake.** Reciting definitions without one example. Table in §12.8 (p. 705).

Q12.12 Overloading versus overriding.

CISCO · QUALCOMM · TECH-1 · ■□□ · S

30-second answer: overloading is several functions of one name with different parameter lists in one scope, resolved by the compiler; overriding is a derived class replacing a virtual function of the base with the same signature, resolved at run time.

Follow-ups. Python has no overloading (the last definition wins; use default arguments or `singledispatch`). A non-virtual function that is redefined in a derived class is *hiding*, not overriding. **Mistake.** Saying that return type alone can distinguish overloads; it cannot.

Q12.13 How do virtual functions work internally, and what do they cost?

QUALCOMM · ARISTA · TECH-1 · ■■□ · T

30-second answer: each class with virtual functions has a vtable of function pointers; each object holds a hidden vptr to its class's table; a virtual call loads the vptr, indexes the table and jumps. The cost is one pointer per object, one extra memory read per call and no inlining.

Evidence. For `struct A { virtual void f(){} int x; };` against `struct B { int x; };`, `sizeof(A)` is 16 and `sizeof(B)` is 4 on a 64-bit machine: 8 bytes of vptr plus padding to alignment. **Follow-ups.** Why constructors cannot be virtual (vptr not yet set); why base destructors should be. See §2.20 (p. 140).

Q12.14 Abstract class versus interface.

CISCO · MICROSOFT · TECH-1 · ■■□ · T

30-second answer: an abstract class may hold data and implemented methods and cannot be instantiated; an interface is a pure contract with no instance state. In C++ an interface is an abstract class with only pure virtual functions (`= 0`); in Python, `abc.ABC` with `@abstractmethod` , or `typing.Protocol` .

Answer. Use an abstract class when related types share code and a contract; use an interface when unrelated types share only a contract. A class can implement many interfaces but, in Java, extend one class. Table in §12.8 (p. 705).

Q12.15 Shallow versus deep copy; the rule of three and five.

ARISTA · BROADCOM · TECH-1 · ■■□ · T

30-second answer: a shallow copy copies the pointer so two objects share one block, and the second destructor frees it again (double free); a deep copy allocates a new block. Rule of three: if a class needs a destructor, copy constructor or copy assignment, it needs all three; rule of five adds move constructor and move assignment.

```
Buf(const Buf &o) : n(o.n) { p = new
char[n]; memcpy(p, o.p, n); }   // deep
copy
Buf &operator=(const Buf &o) {
    if (this != &o) { char *q = new
char[o.n]; memcpy(q, o.p, o.n);
                      delete[] p; p =
q; n = o.n; }
    return *this;
}
```

Mistake. Self-assignment: freeing `p` before copying from it. Smart pointers and `std::string` remove the need for the three. See §2.19 (p. 138).

Q12.16 Explain SOLID with one example per letter.

MICROSOFT · AMAZON · TECH-1 · ■■□ · T

30-second answer: S, a class has one reason to change (split `Router` from `ConfigParser`). O, extend by adding classes, not editing chains (new vendor, new subclass). L, a subclass works wherever its base does (no `ReadOnlyDevice` that throws in `set_config`). I, small interfaces (`Routing` , `Switching`). D, depend on abstractions (the monitor takes a `Device`).

Follow-up. Which letter does an `if vendor == ...` chain break (O)? Table with violation and repair in §12.8 (p. 705).

Q12.17 Implement a thread-safe Singleton in C++. Why might you avoid it?

MICROSOFT · ARISTA · TECH-1 · ■■□ · T

30-second answer: use a function-local static, which C++11 guarantees is initialised exactly once even with several threads; delete copying. Avoid Singletons because they are hidden global state, which makes unit tests order-dependent and hard to isolate.

```
class Logger {
public:
    static Logger& get() { static Logger
inst; return inst; }
    Logger(const Logger&) = delete;
    Logger& operator=(const Logger&) =
delete;
    void log(const char* m);
private:
    Logger() = default;
};
```

Follow-ups. Python: a module-level object is already a singleton. Alternative: pass the logger in (dependency inversion). **Mistake.** Double-checked locking written by hand, which is easy to get wrong.

Q12.18 Design classes for a routing table with add, delete and longest-prefix lookup.

ARISTA · JUNIPER · CISCO · TECH-2 · ■■■ · T

30-second answer: one `RoutingTable` class holding a dictionary per prefix length; lookup tries lengths from 32 down to 0 and returns the first match, so it costs at most 33 hash probes. For millions of routes use a trie (§1.9 (p. 41)**).**

```
import ipaddress as ip

class RoutingTable:
    def __init__(self):
        self.by_len = {}              #
length -> {network as int: next hop}
    def add(self, prefix, hop):
        n = ip.ip_network(prefix)

self.by_len.setdefault(n.prefixlen, {})
[int(n.network_address)] = hop
    def delete(self, prefix):
        n = ip.ip_network(prefix)
        self.by_len.get(n.prefixlen,
{}).pop(int(n.network_address), None)
    def lookup(self, addr):
        a = int(ip.ip_address(addr))
        for plen in sorted(self.by_len,
reverse=True):     # longest first
            key = (a >> (32 - plen)) <<
(32 - plen)
            if key in self.by_len[plen]:
                return self.by_len[plen]
[key]
        return None
```

With 0.0.0.0/0 to `isp`, 10.0.0.0/8 to `core` and 10.1.0.0/16 to `b1`, the lookups of 10.1.2.3, 10.9.9.9 and 8.8.8.8 return `b1`, `core`, `isp`; after deleting the /16, 10.1.2.3 returns `core`.

Complexity. Add and delete O(1); lookup O(33).

Design note. `Route` could be its own class for attributes (metric, source); composition ("table has routes") is preferred to inheriting from `dict`.

Q12.19 Primary key, unique key and foreign key.

CISCO · NOKIA · TECH-1 · ■□□ · T

30-second answer: a primary key identifies each row and can be neither null nor repeated (one per table); a unique key forbids repeats but may allow a null (several per table); a foreign key is a column that must match a primary or unique key in another table, which enforces referential integrity.

Example. `device.name` is the primary key, `device.serial` unique, `device.site_id` a foreign key to `site`. Deleting a site that devices still reference is refused, or cascades if declared `ON DELETE CASCADE`. Table in §12.9 (p. 709).

Q12.20 What is normalisation? Explain 1NF, 2NF and 3NF.

CISCO · NOKIA · MICROSOFT · TECH-1 · ■□□ · T

30-second answer: normalisation splits tables so that each fact is stored once, removing insertion, update and deletion anomalies. 1NF: atomic cells. 2NF: no non-key attribute depends on part of a composite key. 3NF: no non-key attribute depends on another non-key attribute. Mnemonic: every attribute depends on the key (1NF), the whole key (2NF) and nothing but the key (3NF).

Example. The four-table decomposition of the interface log is worked in §12.9 (p. 709). **Mistake.** Stating that normalisation always improves speed; it reduces redundancy but adds joins.

Q12.21 Write SQL for the second-highest salary, with and without window functions.

MICROSOFT · NOKIA · TECH-1 · ■■□ · T

30-second answer: `SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp);` **returns the second-highest distinct salary.**

Data. `emp(name, salary)`: Asha 50,000; Ravi 70,000; Meena 70,000; Kiran 60,000; Dev 40,000. **Three forms, each returning 60,000.**

```
SELECT DISTINCT salary FROM emp ORDER
BY salary DESC LIMIT 1 OFFSET 1;
SELECT DISTINCT salary FROM
  (SELECT salary, DENSE_RANK() OVER
(ORDER BY salary DESC) r FROM emp) t
WHERE r = 2;
```

Ties. Ravi and Meena tie at 70,000, so `DENSE_RANK` and `DISTINCT` are needed; `ROW_NUMBER` would return 70,000. **Follow-up.** If there is no second salary, the first form returns null and the `LIMIT` form returns no row; wrap it in a subquery to force null.

Q12.22 INNER versus LEFT JOIN, with sample tables and output.

CISCO · MICROSOFT · TECH-1 · ■■□ · T

30-second answer: `INNER JOIN` **keeps only rows that match on both sides;** `LEFT JOIN` **keeps every left row and fills the right columns with null when nothing matches.**

Data. `dept`: 10 NOC, 20 Dev, 30 QA. `emp(ename, did)`: Asha 10, Ravi 10, Meena 20.

Query on `dept d ... emp e ON e.did = d.did`	Rows returned
`INNER JOIN`	(Dev, Meena), (NOC, Asha), (NOC, Ravi)
`LEFT JOIN`	the same three, plus (QA, null)

Use. Add `WHERE e.ename IS NULL` to a left join to find departments with no employees: QA. **Mistake.** Putting a filter on a right-table column in `WHERE`, which turns a left join into an inner join; put it in `ON`.

Q12.23 What is an index? Why a B+ tree? When does an index hurt?

CISCO · NOKIA · TECH-1 · ■■□ · T

30-second answer: an index is a sorted auxiliary structure that avoids scanning the table. A B+ tree is used because it is balanced and shallow (high fan-out means three or four page reads for a million rows) and its linked leaves give cheap range scans. It hurts when it slows writes, costs space, or is ignored.

When ignored. Low selectivity (a column with two values), a function applied to the column, a leading wildcard (`LIKE '%x'`), or a composite index whose leftmost column is not in the query. **Follow-up.** Clustered versus non-clustered (§12.9 (p. 709)).

Q12.24 Explain ACID with a bank-transfer example.

MICROSOFT · AMAZON · TECH-1 · ■■□ · T

30-second answer: moving ₹500 from account 1 to 2 must debit and credit together (atomicity), keep the balance rule intact (consistency), not be seen half-done by others (isolation) and survive a crash after commit (durability).

```
BEGIN;
UPDATE acct SET bal = bal - 500 WHERE
id = 1 AND bal >= 500;
UPDATE acct SET bal = bal + 500 WHERE
id = 2;
COMMIT;
```

Mechanisms. Atomicity by an undo log; consistency by constraints (`CHECK (bal >= 0)`); isolation by locks or MVCC; durability by flushing the write-ahead log before commit. **Mistake.** Ignoring the check of the first update's row count: if the balance is short, zero rows change and the transaction must roll back.

Q12.25 Isolation levels and the anomalies each prevents.

MICROSOFT · NOKIA · TECH-1 · ■■□ · T

30-second answer: read uncommitted allows dirty reads; read committed forbids them; repeatable read also forbids non-repeatable reads; serializable also forbids phantoms.

Answer. Each level adds one prevention, in the order dirty read, non-repeatable read, phantom (matrix in §12.9 (p. 709)). **Defaults.** PostgreSQL and Oracle read committed; MySQL InnoDB repeatable read. **Follow-up.** Why not always serializable? Throughput falls through blocking or retry on conflicts. **Mistake.** Quoting the standard table for PostgreSQL's repeatable read, which also blocks phantoms.

Q12.26 Two engineers allocate "the next free IP" at once and get the same address. Why, and how do you fix it?

AMAZON · CISCO · TECH-2 · ■■■ · T

30-second answer: each transaction reads the same free row before the other writes (a check-then-act race), so both claim it. Fix by making the claim atomic: a conditional update, a lock on the row with `FOR UPDATE SKIP LOCKED`, or a unique constraint that makes the second insert fail.

Interleaving. T1 and T2 both run `SELECT ip FROM ip_pool WHERE owner IS NULL ORDER BY ip LIMIT 1` and see 10.0.0.2; both then `UPDATE ... SET owner = ...` for it. Under read committed both succeed.

Fixes.

1. Conditional update, checking the row count: `UPDATE ip_pool SET owner = :me WHERE ip = :ip AND owner IS NULL;` One row changed means success; zero means retry. We ran this on a pool whose first free address was 10.0.0.2: exactly one row changed.
2. Lock while choosing (PostgreSQL, MySQL 8): `SELECT ip FROM ip_pool WHERE owner IS NULL ORDER BY ip LIMIT 1 FOR UPDATE SKIP LOCKED;` so each transaction takes a different free row.
3. A table `allocation(ip PRIMARY KEY, owner)`: the second insert of the same `ip` violates the key and is retried.

Mistake. Fixing it in application code with a Python lock, which fails with more than one application server; the database is the single point of agreement.

Q12.27 Represent −18 in 8-bit two's complement.

QUALCOMM · BROADCOM · TECH-1 · ■□□ · T

30-second answer: 1110 1110, which is 0xEE.

Steps. 18 = 0001 0010; invert: 1110 1101; add one: 1110 1110. **Check.** 256 − 18 = 238 = 0xEE. **Range.** −128 to 127; the pattern 1000 0000 has no positive counterpart.

Q12.28 What is endianness? What is network byte order, and why do we need `htons`?

QUALCOMM · ARISTA · BROADCOM · TECH-1 · ■■□ · S

30-second answer: endianness is the order in which a multi-byte value's bytes are stored. Little-endian puts the least significant byte first (x86, most ARM); big-endian puts the most significant first. Network byte order is big-endian, so `htons` and `htonl` convert from host order before sending and `ntohs` and `ntohl` convert back.

Example. 10.0.0.1 is 0x0A000001: on the wire the bytes are 0A 00 00 01; in memory on a little-endian host they are 01 00 00 0A. `htons(0x1234)` on such a host gives 0x3412. **Mistake.** Calling network byte order "little-endian". Details in §2.9 (p. 117).

Q12.29 Write a C function to check whether the machine is little-endian.

QUALCOMM · BROADCOM · TECH-1 · ■■□ · S

30-second answer: store 1 in a 16-bit integer and read its first byte through a byte pointer; if it is 1, the least significant byte is first.

```c
#include <stdint.h>
int is_little_endian(void) {
    uint16_t x = 1;
    return *(uint8_t *)&x == 1;
}
```

Output. On x86 the function returns 1. Reading through `uint8_t *` (a character type) is permitted by the aliasing rules, whereas a cast between `int *` types would not be. **Follow-up.** A `union` also works in C.

Q12.30 Count the set bits of an integer (Kernighan's method). Complexity?

QUALCOMM · BROADCOM · ARISTA · TECH-1 · ■■□ · S

30-second answer: repeat `x &= x - 1`, which clears the lowest set bit, and count the iterations; the time is O(*k*) for *k* set bits.

```
int popcount(unsigned x) {
    int c = 0;
    while (x) { x &= x - 1; c++; }
    return c;
}
```

Trace. For 0xF0F0 (eight set bits) the loop runs 8 times; for 0 it runs 0 times, so `popcount(0xF0F0)` is 8 and `popcount(0)` is 0 (compiled and run). **Why it works.** Subtracting 1 flips the lowest set bit and every zero below it, so the AND clears exactly that bit. **Alternatives.** A 256-entry table, or `__builtin_popcount` (hardware instruction).

Q12.31 Find the tag, index and offset bits for a 16 KB direct-mapped cache with 16-byte blocks and 32-bit addresses.

QUALCOMM · BROADCOM · TECH-1 · ■■□ · T

30-second answer: offset 4 bits, index 10 bits, tag 18 bits.

Steps. Offset = $\log_2 16 = 4$. Blocks = 16,384/16 = 1,024, and direct-mapped means one block per set, so index = $\log_2 1{,}024 = 10$. Tag = 32 − 10 − 4 = 18. **Variant.** A 4-way cache of the same size has 256 sets: index 8, tag 20. Method and a hit-count example in §12.10 (p. 713).

Q12.32 Interrupts versus polling versus NAPI. Why do high-speed NICs poll?

ARISTA · BROADCOM · AMAZON · TECH-2 · ■■□ · T

30-second answer: an interrupt per packet costs a context switch, which at millions of packets a second consumes the CPU (receive livelock); polling wastes cycles when idle. NAPI combines them: the first packet interrupts, interrupts are then masked and the kernel polls in batches until the ring is empty.

Detail. The NIC places packets in a ring by DMA; interrupt coalescing delays the signal until several packets accumulate, trading latency for efficiency. Multi-queue NICs spread flows over cores (RSS). `ethtool -c` shows coalescing and `ethtool -S` shows drops. See §7.8 (p. 441).

Q12.33 Why do routers use TCAM for lookups? Trade-offs against SRAM-based algorithmic lookup.

CISCO · JUNIPER · ARISTA · TECH-2 · ■■■ · T

30-second answer: a TCAM compares the key with every entry at once and stores a "don't care" bit per cell, so a longest-prefix or ACL lookup finishes in one cycle regardless of table size. The price is area, power and cost per bit, and limited capacity; algorithmic lookup in SRAM or DRAM (multibit tries) is cheaper and larger but needs several memory accesses.

Detail. Entries are ordered so that longer prefixes win, which makes incremental updates costly. TCAM is also the natural store for ACLs because it matches ranges of header bits. High-end ASICs combine both. See Pillar 5 (p. 302) and §1.9 (p. 41).

Q12.34 Design the network for a new five-building campus with 8,000 users.

CISCO · JUNIPER · NOKIA · TECH-2 · ■■□ · T

30-second answer: a hierarchical design with dual cores, a distribution pair per building and dual uplinks everywhere; one /16 per building summarised in an OSPF stub area; HSRP or VRRP gateways, a wireless controller with 802.1X and a guest portal, segmentation by ACL, and dual edge firewalls to two providers. Peak internet demand is 8,000 × 0.4 × 2 Mb/s = 6.4 Gb/s, so two 10 Gb/s uplinks.

Skeleton to redraw. Clarify; estimate; core, distribution, access; addressing; redundancy; wireless; security; failure; trade-offs. **Full solution** with figure and IP plan: §12.11 (p. 717).

Q12.35 Design a rate limiter for an API used by 10 million users.

AMAZON · GOOGLE · MICROSOFT · TECH-2 · ■■□ · T

30-second answer: a token bucket per API key at the gateway, with counters in a sharded in-memory store updated atomically; 10^6 active users at 0.5 requests per second need 500,000 checks per second, hence at least 5 shards of 100,000 operations per second; refused clients receive 429 and `Retry-After`.

Skeleton. Requirements; algorithm choice (token bucket for bursts); placement at the gateway; distributed state; response; failure policy (open for reads, closed for payments). **Full solution:** §12.11 (p. 717). Code: §1.15 (p. 60).

Q12.36 Token bucket versus leaky bucket, and how they relate to policing and shaping.

CISCO · JUNIPER · ARISTA · TECH-1 · ■■□ · T

30-second answer: a token bucket of capacity *b* refilled at rate *r* lets a quiet sender burst up to *b* and then sends at *r*; a leaky bucket queues arrivals and releases them at constant rate, so its output is smooth. A policer is a token bucket that drops or re-marks excess; a shaper queues the excess and so behaves like a leaky bucket.

Numbers. With $r = 10$ per second and $b = 20$, a full bucket allows $20 + 10 \times 5 = 70$ packets in the first 5 seconds; a leaky bucket at 10 per second allows only 50. **Consequence.** Policing adds no delay but can cause TCP loss; shaping adds delay and needs buffer (§4.11 (p. 274)).

Q12.37 Design a CDN for an Indian video-streaming start-up.

GOOGLE · AMAZON · MICROSOFT · TECH-2 · ■■■ · T

30-second answer: assume 2 million daily viewers watching one hour at 2 Mb/s: 900 MB each, 1.8 PB a day, 167 Gb/s on average and about 500 Gb/s at three times the mean. Serve adaptive-bitrate segments from edge caches in Mumbai, Delhi, Chennai and Kolkata and inside ISP networks, with a shield tier, so that at a 95% hit ratio the origin carries 25 Gb/s at peak.

Components. Request routing by DNS or anycast; segments of 2 to 6 seconds cached under the URL; popular titles in memory, the long tail on disk; versioned manifests; token-signed URLs; TLS at the edge; popular new releases pre-positioned before launch. **Arithmetic.** $2 \times 10^6 \times 0.9$ GB = 1.8 PB; $1.8 \times 10^{15} \times 8/86{,}400 = 167$ Gb/s; × 3 = 500; × 5% = 25. **Trade-off.** Pre-positioning spends bandwidth to save latency. Template: §12.11 (p. 717).

Q12.38 Anycast versus GeoDNS for request routing.

GOOGLE · AMAZON · CISCO · TECH-2 · ■■■ · T

30-second answer: GeoDNS answers each lookup with a nearby server's address, chosen from the resolver's location; anycast announces one address from many sites and lets BGP deliver each packet to the topologically nearest. GeoDNS is controllable but coarse and slow to fail over; anycast fails over at BGP speed but gives less control over load.

	GeoDNS	Anycast
Choice made by	DNS server, by resolver address	BGP, by AS path
Failover	Waits for TTL and health checks	Withdraw the route: seconds
Accuracy	Wrong when the resolver is far from the user	Nearest by network path, not by distance
Load control	Fine (weights per region)	Coarse (prepending, communities)
Risk	Stale caches	A route flap can move live TCP sessions

Practice. Many CDNs combine them: anycast for DNS and TCP entry, DNS steering for capacity.

Q12.39 Design an L4 load balancer for 1 million new connections per second.

AMAZON · GOOGLE · ARISTA · TECH-2 · ■■■ · T

30-second answer: announce one virtual IP from several balancer nodes; routers spread flows over them by ECMP on the five-tuple; each node picks a backend by consistent hashing so that most flows survive a change, forwards by direct server return, and health-checks backends.

Estimate (assumptions stated). About 10 inbound packets per connection gives 10 million packets per second; at 5 million per second per software node (XDP or DPDK on 25 GbE), 2 nodes suffice and 4 give N+2. With 60-second connections, 60×10^6 = 60 million live connections; at 64 bytes of state each that is 3.84 GB spread over the nodes. **Design points.** Direct server return lets replies bypass the balancer; consistent hashing (Maglev) limits remapping when a backend or a node fails; connection draining before removal; SYN-flood protection at the edge. **Follow-up.** L7 balancing terminates TLS and routes by URL, at lower throughput. See Pillar 5 (p. 302).

Q12.40 How would you make the campus network survive a core switch failure without users noticing?

CISCO · JUNIPER · NOKIA · TECH-2 · ■■■ · T

30-second answer: remove every single point of failure above the access layer and make detection fast: dual cores, dual uplinks from each distribution pair, routed links with equal-cost paths, BFD for failure detection in about 300 ms, and a first-hop redundancy protocol with tuned timers. Traffic then reroutes in well under a second, which TCP sessions survive.

Timeline. BFD with a 100 ms interval and multiplier 3 detects the loss in 300 ms; OSPF recomputes in tens of milliseconds; ECMP already holds the alternate path, so only flows on the dead switch move; the gateway moves to its standby within the tuned hold time (for example 750 ms). A TCP sender sees a retransmission delay, not a reset, because the minimum retransmission timeout is about 200 ms and a connection fails only after many. **Also.** Stateful firewall pair with synchronised sessions; dual power feeds; spanning-tree root placed on the active gateway; a tested maintenance procedure. **Mistake.** Relying on default spanning-tree timers, which take seconds. See §12.11 (p. 717) and §4.8 (p. 265).

Q12.41 Tell me about yourself.

ALL · HR · ■□□ · S

30-second answer, model (Ananya, fictional): “I am Ananya Rao, a final-year electronics and communication student at a Pune engineering college. Currently I am focused on networking: I built LabWatch, a Python tool that polls twelve lab switches every 30 seconds and raises an alert when a link stays above 80% for three samples. I got here through my CCNA preparation and a summer internship where I scripted configuration backups for 40 devices. I am drawn to Cisco’s campus switching and Meraki products because they combine the routing and automation I enjoy. That is why I applied for this role.”

Make it yours. Present: project plus a number; Past: one or two proofs; Future: one named product and a link. 60 to 90 seconds. **Probes.** “Tell me more about LabWatch” (have the architecture ready, Q12.51). **Mistake.** Reading the résumé aloud, or starting with school marks.

Q12.42 Why networking? Why this company?

ALL · HR · ■□□ · S

30-second answer, model: “In my second year a campus outage of two hours turned out to be a duplex mismatch on one cable; finding it from interface counters made networks feel like detective work. The field is also where every service eventually depends on the same few protocols. I chose [company] because [one product or team] solves [problem] at a scale I want to work on, and my LabWatch and OSPF lab work are close to [that team’s] daily tasks.”

Make it yours. A real moment, then the company’s specific product, then your matching proof. **Probes.** “What do you know about our products?”, “Which of our competitors do you prefer?” Read the company’s engineering blog the week before. **Mistake.** “Because it is a reputed company”.

Q12.43 (Customer Obsession) A time you went beyond what was asked for a user.

AMAZON · CISCO · BEHAVIOURAL · ■■□ · T

30-second answer, model: S/T. Hostel students complained of slow Wi-Fi each evening; the IT office had closed the tickets as "bandwidth is fine". I wanted the actual cause. A. I surveyed 40 students, then logged client counts per access point every minute for five evenings with a small script. Two of the nine access points carried 60% of the clients. I showed the chart to the IT office and proposed lowering the power of those two and moving one. R. Median ping to the gateway at 9 p.m. fell from 180 ms to 60 ms, and tickets for the month fell from 22 to 7.

Make it yours. Who was the user, what did they say, what did you measure, what changed. **Probes.** "What if IT had refused?" (take the data to the warden). "How did you know the survey was representative?" **Principles.** Also fits Dive Deep.

Q12.44 (Ownership) A time you took responsibility for something outside your role.

AMAZON · MICROSOFT · BEHAVIOURAL · ■■□ · T

30-second answer, model: S. Two days before a networking practical, the simulator images failed to load on 30 lab PCs; the technician was on leave and the repair was queued for later in the week. T. It was not my job, but my batch would lose the exam slot. A. I found that the cache folder had been corrupted, wrote a script to restore the images from the shared drive, tested it on three machines, and wrote a one-page guide so the instructor could repeat it. R. All 30 PCs were working in two hours and the practical went ahead as scheduled.

Make it yours. The cost of doing nothing, what you chose to own, how you made it repeatable. **Probes.** "Did you ask permission?" (say you informed the instructor first). **Mistake.** Taking over someone's work without telling them.

Q12.45 (Dive Deep) A time you found the root cause of a difficult problem.

AMAZON · CISCO · JUNIPER · BEHAVIOURAL · ■■□ · T

30-second answer, model: S. Transfers between two lab PCs ran at about 8 Mb/s on a 100 Mb/s link, on and off. T. I was asked to find out why before a demonstration. A. I first ruled out the cable and the PCs with `iperf` on a different port. Then I read the interface counters on the switch: late collisions and CRC errors were rising on the one port. One end was set to auto-negotiate and the other hard-coded to 100 Mb/s full duplex, so the auto side fell back to half duplex: a duplex mismatch. R. Setting both ends to auto restored 94 Mb/s, and I added the check to the lab handbook.

Make it yours. Symptom, hypotheses ruled out in order, the one datum that located the cause. **Probes.** "What would you have done if the counters were clean?" (packet capture on both sides). Troubleshooting method: Pillar 11 (p. 634).

Q12.46 (Have Backbone; Disagree and Commit) A time you disagreed with a teammate or professor.

AMAZON · GOOGLE · BEHAVIOURAL · ■■□ · T

30-second answer, model: S. For LabWatch, our supervisor asked us to use SNMPv2c; I wanted SNMPv3 because v2c sends the community string in clear text. A. I wrote a short note: three of our five switch models did not support v3 in the lab images, and v2c on an isolated management VLAN limited the risk. I raised my concern once, with the evidence, and the supervisor decided on v2c. I committed fully: I put the poller on the management VLAN, restricted access with an ACL, and documented the risk. R. We delivered on time, and I later enabled v3 on the two models that supported it.

Make it yours. Show respect, data, a clear decision, and full commitment afterwards. **Probes.** "Were you right?" (partly; the constraint was real). **Mistake.** A story in which the other person was simply wrong.

Q12.47 (Bias for Action) A time you decided with incomplete information.

AMAZON · MICROSOFT · BEHAVIOURAL · ■■□ · T

30-second answer, model: S. Two hours before a hackathon demonstration, the server on port 8080 was unreachable from the judges' network; the college firewall rules were unknown and IT was not responding. T. Decide whether to wait or change the plan. A. Waiting was not an option and the change was reversible, so I wrote a ten-line script that tried ports 80, 443 and 8443 from the venue network. Only 443 was open. We moved the demo there and kept the old setting as a fallback. R. The demonstration ran without a failure and we placed second.

Make it yours. State the deadline, what you did not know, why the decision was reversible, and how you cheaply reduced uncertainty. **Probes.** "What if 443 had been closed?" (the phone hotspot, prepared in advance).

Q12.48 (Deliver Results) A time you met a tough deadline.

AMAZON · CISCO · BEHAVIOURAL · ■■□ · T

30-second answer, model: S/T. A reliable file transfer over UDP, with a sliding window, was due in ten days and I had written no code. A. I cut the scope into steps with dates: stop-and-wait by day 3, sliding window by day 7, tests and report by day 10. I used `tc netem` to emulate 10% loss from the first version, so bugs showed early. When the window logic failed on day 6, I dropped the planned congestion control rather than the tests. R. A 1 MB file arrived with a matching hash under 10% loss, submitted on day 10.

Make it yours. The fixed date, the plan, what you cut and why. **Probes.** "What would you have cut next?" **Mistake.** Heroic overnight work as the only action.

Q12.49 (Learn and Be Curious) Something you learnt recently on your own.

AMAZON · GOOGLE · BEHAVIOURAL · ■□□ · T

30-second answer, model: "No course covered BGP, so I taught myself. I read the core sections of RFC 4271, then built a three-AS lab in GNS3 over two weekends, and deliberately broke it: a wrong next hop, a missing network statement, a looping AS path. Each failure taught me one rule, such as why an iBGP route needs the next hop to be reachable. I wrote two pages of notes and explained them to twelve juniors in a one-hour session, which showed me three gaps that I then closed."

Make it yours. What, how (a project, not just videos), what broke, how you shared it. **Probes.** "What will you learn next, and why?" Have an answer that matches the role.

Q12.50 Tell me about a failure. What did you learn?

AMAZON · GOOGLE · CISCO · BEHAVIOURAL · ■■■ · T

30-second answer, model: S/T. I was automating VLAN changes on six lab switches with a Netmiko script. A (failure). I ran it without a dry run, and a template error replaced the VLAN configuration; the lab was down for 40 minutes. I told the instructor at once, and restored the switches from backups that I happened to have saved. Then I added three things to the script: a dry-run mode that prints the diff, an automatic backup before any change, and a rollback command. R. No incident in the next three months, and the instructor adopted the script. L. Change tools must be safe by default, not by my care.

Make it yours. A real failure with your own responsibility, quickly owned, and a system-level fix. **Probes.** "Whose fault was it?" (mine; avoid blaming). **Mistake.** A disguised success ("I worked too hard").

Q12.51 Walk me through your best project. Draw the architecture.

JUNIPER · ARISTA · CISCO · TECH-1 · ■■□ · T

30-second answer, model: "LabWatch monitors twelve lab switches. A Python poller sends SNMP requests to each switch every 30 seconds, stores interface counters in SQLite, and a small web page plots utilisation; an alert rule e-mails me when a link exceeds 80% for three samples."

```
12 switches --SNMP--> poller (asyncio) -
-> SQLite --> web dashboard
                              \--> alert
rule --> e-mail
```

Approach. Draw five boxes before speaking; say the problem, your share (all of the poller, half the dashboard), then one number: 38 samples per second (12 switches × 48 ports × 2 counters / 30 s = 38.4). **Probes.** "Why SNMP and not streaming telemetry?" (supported by all the lab devices); "How is the data stored?" Method in §12.13 (p. 726).

Q12.52 What was the hardest bug in your project and how did you find it?

JUNIPER · ARISTA · BROADCOM · TECH-1 · ■■□ · T

30-second answer, model: the utilisation graph showed impossible negative spikes. The cause was the 32-bit counter `ifInOctets`, which wraps after 2^{32} bytes: at 1 Gb/s that is 34 seconds, so at a 30-second poll the counter often wraps between two readings and the raw difference is negative. I found it by printing consecutive raw values near a spike. I fixed it with the 64-bit `ifHCInOctets` and, for old devices, by computing deltas modulo 2^{32}.

Check. $2^{32} \times 8 / 10^9 = 34.4$ s at 1 Gb/s; 343.6 s at 100 Mb/s. **Probes.** "What if the counter wraps twice between polls?" (poll faster than the wrap time; at 100 Gb/s even 64-bit counters last years). **Method.** Symptom, hypothesis, raw data, cause, fix, test.

Q12.53 If your project had to serve 1,000 times the load, what breaks first?

AMAZON · GOOGLE · CISCO · TECH-2 · ■■■ · T

30-second answer: today LabWatch handles 38.4 samples per second; at 1,000 times (12,000 switches) that is 38,400 per second. The poller breaks first, because it waits for each switch in turn, and the single-writer SQLite database second. The fixes are a pool of asynchronous pollers sharded by device, batched inserts, and a time-series database with down-sampling.

Order of reasoning. Estimate (12,000 × 48 × 2 / 30 = 38,400 per second); find the bottleneck by tracing one cycle; fix it; name the next one (the network: SNMP over UDP loses polls under load, so move to streaming telemetry). **Mistake.** "Just add servers" with no bottleneck named.

Q12.54 What would you do differently if you started the project again?

AMAZON · MICROSOFT · BEHAVIOURAL · ■■■ · T

30-second answer: three changes, each with a reason: use 64-bit counters from the first day (it cost me two days to find the wrap); write tests against a fake SNMP agent before the poller (I tested by hand on real switches); and consider gNMI streaming where devices support it, because polling cannot see changes shorter than the interval.

Make it yours. Name real flaws, not trivial ones ("better UI"). Link each to a lesson and to the next version. **Probes.** "Which would you do first?" (the tests, since they make the other changes safe). **Mistake.** "Nothing; it went well".

Forty further aptitude questions, with worked answers

Each question is single-answer with four options. Attempt a question in 60 seconds, mark the option, and only then read the working; the topics follow the order of §12.2 (p. 690) to §12.6 (p. 701).

NUMBER SYSTEM

M1. Remainder when 7^{100} is divided by 6? (a) 1 (b) 2 (c) 3 (d) 5

Answer (a). $7 \equiv 1 \pmod 6$, so $7^{100} \equiv 1$.

NUMBER SYSTEM

M2. Largest three-digit number divisible by 13? (a) 988 (b) 989 (c) 990 (d) 999

Answer (a). $999 = 13 \times 76 + 11$, so $999 - 11 = 988$.

NUMBER SYSTEM

M3. Sum of the first 20 odd numbers? (a) 380 (b) 400 (c) 420 (d) 441

Answer (b). The sum of the first n odd numbers is $n^2 = 400$.

NUMBER SYSTEM

M4. How many multiples of 7 lie between 100 and 300? (a) 27 (b) 28 (c) 29 (d) 30

Answer (b). First 105 = 7 × 15, last 294 = 7 × 42, so 42 − 15 + 1 = 28.

HCF AND LCM

M5. The HCF of two numbers is 8 and their LCM is 240. One number is 48. The other is? (a) 40 (b) 60 (c) 80 (d) 120

Answer (a). 8 × 240 / 48 = 40.

HCF AND LCM

M6. The smallest number divisible by 15, 20 and 25? (a) 150 (b) 300 (c) 400 (d) 600

Answer (b). 15 = 3·5, 20 = 2^2·5, 25 = 5^2; LCM = $2^2 \times 3 \times 5^2 = 300$.

PERCENTAGES

M7. If 40% of x equals 60% of y, then $x : y$ is? (a) 2 : 3 (b) 3 : 2 (c) 4 : 6 (d) 3 : 4

Answer (b). $0.4x = 0.6y$ gives $x/y = 6/4 = 3 : 2$.

PERCENTAGES

M8. A's salary is 25% more than B's. By what percentage is B's less than A's? (a) 20% (b) 25% (c) 16.67% (d) 24%

Answer (a). Let B = 100, A = 125; difference 25 on base 125 gives 20%.

PERCENTAGES

M9. A town of 20,000 grows 10% a year. Population after two years? (a) 24,000 (b) 24,200 (c) 24,400 (d) 22,000

Answer (b). 20,000 × 1.1 × 1.1 = 24,200.

PROFIT AND LOSS

M10. An article costs ₹250. At what marked price does a 20% discount still leave a 20% profit? (a) ₹360 (b) ₹375 (c) ₹400 (d) ₹300

Answer (b). Required selling price 250 × 1.2 = 300; marked price × 0.8 = 300 gives 375.

PROFIT AND LOSS

M11. Selling at ₹440 gives a loss of 12%. The cost price? (a) ₹480 (b) ₹500 (c) ₹520 (d) ₹388

Answer (b). 440 / 0.88 = 500.

PROFIT AND LOSS

M12. Two articles are each sold at ₹1,200, one at 20% gain and one at 20% loss. Net result? (a) No loss (b) 4% loss (c) 4% gain (d) 2% loss

Answer (b). Costs 1,000 and 1,500, total 2,500; sales 2,400; loss 100 = 4%.

RATIO

M13. ₹1,210 is divided in the ratio 2 : 3 : 6. The largest share? (a) ₹220 (b) ₹330 (c) ₹660 (d) ₹550

Answer (c). Parts total 11, one part is ₹110, largest is 6 × 110.

RATIO

M14. A 60-litre mixture has milk and water in the ratio 2 : 1. Water to add to make it 1 : 1? (a) 10 L (b) 15 L (c) 20 L (d) 30 L

Answer (c). Milk 40 L, water 20 L; add 20 L to equalise.

AVERAGES

M15. The average of 7 numbers is 15. An eighth number raises the average to 16. The eighth number? (a) 21 (b) 22 (c) 23 (d) 24

Answer (c). 8 × 16 − 7 × 15 = 128 − 105 = 23.

AVERAGES

M16. Average of the first 10 even numbers? (a) 10 (b) 11 (c) 12 (d) 10.5

Answer (b). Sum = 10 × 11 = 110, so the average is 11.

TIME AND WORK

M17. 6 men finish a job in 8 days. How long for 12 men? (a) 2 days (b) 4 days (c) 6 days (d) 16 days

Answer (b). Man-days = 48; 48/12 = 4.

TIME AND WORK

M18. A takes 30 days and B takes 45 days. Together? (a) 15 days (b) 18 days (c) 20 days (d) 75 days

Answer (b). 30 × 45 / 75 = 18.

TIME AND WORK

M19. A is twice as efficient as B. Together they take 12 days. B alone takes? (a) 18 days (b) 24 days (c) 36 days (d) 48 days

Answer (c). Rates 2*k* and *k*: 3*k* = 1/12, so *k* = 1/36.

PIPES

M20. A pipe fills a tank in 12 hours and a drain empties it in 18 hours. Both open: time to fill? (a) 30 h (b) 36 h (c) 6 h (d) 72 h

Answer (b). 1/12 − 1/18 = 1/36.

SPEED AND DISTANCE

M21. 90 km/h in m/s? (a) 15 (b) 20 (c) 25 (d) 30

Answer (c). 90 × 5/18 = 25.

SPEED AND DISTANCE

M22. Walking at 5 km/h a man is 12 minutes late; at 6 km/h he is 10 minutes early. Distance? (a) 9 km (b) 10 km (c) 11 km (d) 12 km

Answer (c). *d*/5 − *d*/6 = 22/60 h gives *d*/30 = 11/30, *d* = 11.

SPEED AND DISTANCE

M23. Two trains 100 km apart start towards each other at 40 and 60 km/h. They meet after? (a) 30 min (b) 45 min (c) 1 h (d) 1.5 h

Answer (c). Closing speed 100 km/h, gap 100 km.

TRAINS

M24. A 180 m train crosses a pole in 9 s. Its speed? (a) 54 km/h (b) 60 km/h (c) 72 km/h (d) 80 km/h

Answer (c). 180/9 = 20 m/s = 72 km/h.

TRAINS

M25. Trains of 120 m and 130 m run the same way at 60 and 40 km/h. Time to overtake? (a) 30 s (b) 45 s (c) 50 s (d) 60 s

Answer (b). Relative 20 km/h = 50/9 m/s; 250 / (50/9) = 45.

BOATS

M26. A boat does 12 km/h in still water; the stream is 4 km/h. Time to go 16 km upstream? (a) 1 h (b) 1.5 h (c) 2 h (d) 4 h

Answer (c). Upstream 8 km/h; 16/8 = 2.

BOATS

M27. A boat does 15 km/h downstream and 9 km/h upstream. Still-water speed? (a) 10 km/h (b) 11 km/h (c) 12 km/h (d) 13 km/h

Answer (c). (15 + 9)/2 = 12.

INTEREST

M28. Simple interest on ₹5,000 at 6% for 3 years? (a) ₹800 (b) ₹900 (c) ₹950 (d) ₹1,000

Answer (b). 5,000 × 6 × 3 / 100 = 900.

INTEREST

M29. Compound interest on ₹4,000 at 5% for 2 years? (a) ₹400 (b) ₹410 (c) ₹420 (d) ₹440

Answer (b). 4,000 × 1.1025 = 4,410, so CI = 410.

INTEREST

M30. At simple interest a sum doubles in 10 years. The annual rate? (a) 5% (b) 8% (c) 10% (d) 12.5%

Answer (c). Interest equals principal in 10 years: 100/10 = 10%.

COUNTING

M31. Ways to select 3 members from 8? (a) 24 (b) 56 (c) 112 (d) 336

Answer (b). C(8, 3) = 56.

COUNTING

M32. Arrangements of the letters of PACKET? (a) 120 (b) 360 (c) 720 (d) 5,040

Answer (c). Six distinct letters: 6! = 720.

COUNTING

M33. Handshakes among 12 people, each pair once? (a) 66 (b) 72 (c) 132 (d) 144

Answer (a). C(12, 2) = 66.

PROBABILITY

M34. Probability that a card from a pack is a king or a heart? (a) 1/4 (b) 4/13 (c) 17/52 (d) 1/2

Answer (b). 13 + 4 − 1 = 16 cards; 16/52 = 4/13.

PROBABILITY

M35. Probability that a die shows a prime? (a) 1/3 (b) 1/2 (c) 2/3 (d) 1/6

Answer (b). Primes 2, 3, 5: 3/6.

PROBABILITY

M36. Two dice: probability of sum 7? (a) 1/12 (b) 1/9 (c) 1/6 (d) 5/36

Answer (c). Six pairs out of 36.

CLOCKS AND CALENDARS

M37. Angle between the hands at 2:30? (a) 95° (b) 105° (c) 120° (d) 135°

Answer (b). |30 × 2 − 5.5 × 30| = |60 − 165| = 105.

CLOCKS AND CALENDARS

M38. Today is Wednesday. What day is it 100 days later? (a) Thursday (b) Friday (c) Saturday (d) Sunday

Answer (b). 100 = 14 × 7 + 2; Wednesday + 2 = Friday.

SERIES

M39. 1, 2, 6, 24, 120, ? (a) 240 (b) 360 (c) 720 (d) 840

Answer (c). Factorials: 1!, 2!, 3!, 4!, 5!, so 6! = 720.

CODING

M40. If each letter shifts by +1, SWITCH is coded as? (a) TXJUDI (b) TXIUDI (c) RVHSBG (d) TXJUCI

Answer (a). S→T, W→X, I→J, T→U, C→D, H→I.

Twenty-seven core-subject MCQs, with worked answers

These follow the CS-MCQ section of a typical paper (§12.1 (p. 688)): C output, operating systems, DBMS, networks and data structures. Allow 45 seconds each. Output questions assume a 32-bit `int` and the usual two's-complement behaviour.

C OUTPUT

S1. `printf("%d", 5/2 + 5%2*3);` prints (a) 2 (b) 5 (c) 7 (d) 8

Answer (b). Integer division gives 5/2 = 2; `%` and `*` bind left to right: 5%2 = 1, then 1 × 3 = 3; 2 + 3 = 5.

C OUTPUT

S2. With `int a[] = {10,20,30,40}; int *p = a;`, the value of `*(p+2) + p[1]` is (a) 30 (b) 40 (c) 50 (d) 60

Answer (c). `*(p+2)` is `a[2]` = 30 and `p[1]` is 20; the sum is 50.

C OUTPUT

S3. `unsigned char x = 255; x += 2; printf("%d", x);` prints (a) 257 (b) 1 (c) 2 (d) 0

Answer (b). Unsigned arithmetic wraps modulo 256: 257 mod 256 = 1.

C OUTPUT

S4. `int i = 0; while (i++ < 3); printf("%d", i);` prints (a) 3 (b) 4 (c) 2 (d) infinite loop

Answer (b). The test uses i before incrementing; it succeeds for 0, 1, 2 and fails at 3, but the increment still occurs: i = 4.

C OUTPUT

S5. `printf("%zu", sizeof("net"));` prints (a) 3 (b) 4 (c) 8 (d) undefined

Answer (b). The string literal has an array type of four chars including the terminating null.

C OUTPUT

S6. `int x = 3; printf("%d", x << 2 | 1);` prints (a) 12 (b) 13 (c) 14 (d) 7

Answer (b). Shift binds tighter than OR: 3 << 2 = 12, and 12 | 1 = 13.

C OUTPUT

S7. `printf("%d %d", 10/3, -10/3);` prints (C99 and later) (a) 3 -4 (b) 3 -3 (c) 4 -3 (d) 3 3

Answer (b). Integer division truncates toward zero, so −10/3 = −3.

C OUTPUT

S8. A function has `static int s = 0; s++; printf("%d", s);` and is called three times. The output is (a) 000 (b) 111 (c) 123 (d) 321

Answer (c). A static local keeps its value between calls and is initialised once.

OPERATING SYSTEMS

S9. Processes P1, P2, P3 arrive together with bursts 24, 3, 3 and run first come first served. Average waiting time? (a) 3 (b) 10 (c) 17 (d) 27

Answer (c). Waiting times 0, 24, 27; the average is 51/3 = 17.

OPERATING SYSTEMS

S10. The same three processes under shortest job first. Average waiting time? (a) 3 (b) 5 (c) 10 (d) 17

Answer (a). Order 3, 3, 24: waiting 0, 3, 6; the average is 9/3 = 3.

OPERATING SYSTEMS

S11. FIFO page replacement, 3 frames, references 1 2 3 4 1 2 5 1 2 3 4 5. Page faults? (a) 7 (b) 9 (c) 10 (d) 12

Answer (b). Faults on 1, 2, 3, 4, 1, 2, 5; then 1 and 2 hit; 3 and 4 fault; the final 5 hits. Total 9.

OPERATING SYSTEMS

S12. The same references with 4 frames under FIFO. Page faults? (a) 8 (b) 9 (c) 10 (d) 6

Answer (c). Ten faults: more frames gave more faults, which is Belady's anomaly.

OPERATING SYSTEMS

S13. The same references, 3 frames, LRU. Page faults? (a) 7 (b) 9 (c) 10 (d) 11

Answer (c). Simulation gives 10; LRU is a stack algorithm and so cannot show Belady's anomaly.

OPERATING SYSTEMS

S14. Which is NOT a necessary condition for deadlock? (a) Mutual exclusion (b) Hold and wait (c) Pre-emption of resources (d) Circular wait

Answer (c). Deadlock needs the absence of pre-emption (no pre-emption), not its presence.

OPERATING SYSTEMS

S15. A 32-bit address space with 4 KB pages. How many bits are the page offset? (a) 10 (b) 12 (c) 20 (d) 22

Answer (b). 4 KB = 2^{12}, so 12 offset bits and 20 page-number bits.

DBMS

S16. R(A, B, C, D) with A → B, B → C, C → D and key A. The highest normal form? (a) 1NF (b) 2NF (c) 3NF (d) BCNF

Answer (b). The key is a single attribute so there is no partial dependency (2NF), but B → C and C → D are transitive, so 3NF fails.

DBMS

S17. Column x holds 1, NULL, 3, 3. What does `COUNT(x)` return? (a) 2 (b) 3 (c) 4 (d) 1

Answer (b). `COUNT(x)` skips nulls: 3 (`COUNT(*)` is 4 and `COUNT(DISTINCT x)` is 2).

DBMS

S18. A 4-row table cross-joined with a 5-row table gives how many rows? (a) 9 (b) 20 (c) 5 (d) 4

Answer (b). 4 × 5 = 20.

DBMS

S19. Which isolation level prevents dirty reads but allows non-repeatable reads? (a) Read uncommitted (b) Read committed (c) Repeatable read (d) Serializable

Answer (b). Read committed forbids dirty reads only.

NETWORKS

S20. Usable host addresses in a /27? (a) 30 (b) 32 (c) 62 (d) 14

Answer (a). $2^5 - 2 = 30$.

NETWORKS

S21. The network address of 172.16.45.200/20? (a) 172.16.32.0 (b) 172.16.40.0 (c) 172.16.45.0 (d) 172.16.0.0

Answer (a). Block size 16 in the third octet; 45 lies in 32 to 47.

NETWORKS

S22. DNS queries normally use which transport and port? (a) TCP 53 (b) UDP 53 (c) UDP 67 (d) TCP 80

Answer (b). UDP port 53 (TCP 53 for large responses and zone transfers).

NETWORKS

S23. With an MTU of 1,500 and 20-byte IP and 20-byte TCP headers, the MSS is (a) 1,500 (b) 1,480 (c) 1,460 (d) 1,440

Answer (c). 1,500 − 20 − 20 = 1,460.

NETWORKS

S24. Which layer does a router operate at? (a) Data link (b) Network (c) Transport (d) Session

Answer (b). Layer 3, the network layer.

DATA STRUCTURES

S25. Time to search a sorted array of n elements by binary search? (a) $O(n)$ (b) $O(\log n)$ (c) $O(n \log n)$ (d) $O(1)$

Answer (b). The range halves at each step.

DATA STRUCTURES

S26. Push 1, push 2, push 3, pop, push 4, pop, pop. The popped values in order? (a) 3 4 2 (b) 3 2 1 (c) 3 4 1 (d) 1 2 3

Answer (a). Pops return 3, then 4, then 2.

DATA STRUCTURES

S27. The in-order traversal of a binary search tree visits keys in (a) Descending order (b) Ascending order (c) Level order (d) Insertion order

Answer (b). Left, node, right gives ascending order.

Revision Spread

THE 15-MINUTE VERSION

Ten facts. (1) With a 1/4 penalty and four options, guess whenever one option is eliminated: expected value 1/3 − 1/6 = +0.17. (2) Average speed over equal distances is $2xy/(x + y)$; over equal times it is the mean. (3) Work problems: take the LCM of the days as total units. (4) CI − SI over two years is $P(r/100)^2$. (5) Unit digits of 2, 3, 7, 8 cycle in 4; reduce the exponent mod 4. (6) Polymorphism is a call through the vptr to the vtable; overloading is compile time, overriding is run time. (7) SQL runs FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT. (8) BCNF: every determinant is a super key; 3NF allows a prime attribute as the dependent. (9) Cache address: offset = $\log_2$(block), index = $\log_2$(sets), tag = the rest. (10) Design in six steps and put a number on step two; tell stories in STAR with 60% of the time on your own actions.

Three figures to redraw from memory. The two-pass pacing bar (Figure 12.1); the address split with tag 19, index 7 and offset 6 (Figure 12.9); the dual-core campus with one summary per building (Figure 12.12).

APTITUDE MASTER TABLE

Topic	Formula or rule	Watch for
Divisors	$N = p^a q^b$: $(a+1)(b+1)$; HCF × LCM = product of two	Three numbers: not valid
Unit digit	Cycle 4 (2, 3, 7, 8), 2 (4, 9); 0, 1, 5, 6 fixed	Exponent remainder 0 means last of cycle
Remainders	$(a \cdot b) \bmod n = ((a \bmod n)(b \bmod n)) \bmod n$	Reduce before multiplying
Trailing zeros	$n!$ has $\lfloor n/5 \rfloor + \lfloor n/25 \rfloor + \ldots$	Count 5s, not 2s
Percent change	$a + b + ab/100$; price up r%: cut $100r/(100+r)$%	Base is the old value
Profit	On cost; discount on marked price	Successive: $a + b - ab/100$
False weight	Gain% = error/(true − error) × 100	Base is what was given
Alligation	(dearer − mean) : (mean − cheaper)	Answer is cheaper : dearer
Partnership	Share ∝ capital × time	Equal times: capitals
Work	Rate $1/a$; together $ab/(a+b)$	Leak or drain is negative
Speed	km/h × 5/18 = m/s; equal distance: $2xy/(x+y)$	Not the simple mean
Relative speed	Same way subtract; opposite add	Trains: distance = sum of lengths
Platform	(train + platform)/speed	Pole: train only
Boats	down $u+v$, up $u-v$; $u = (d+u)/2$, $v = (d-u)/2$	Round trip speed is not u
SI, CI	$PRT/100$; $P(1+r/100)^n$; half-yearly: $r/2$, $2n$	CI − SI (2 y) = $P(r/100)^2$
P and C	$P(n,r)$, $C(n,r)$; circular $(n-1)!$; repeats $n!/(p!q!)$	"Together": glue, then × internal
Stars and bars	r identical to k: $C(r+k-1, k-1)$	Zero allowed
Probability	Favourable/total; at least one: 1 − P(none)	Without replacement: change denominator
Clock	Angle	30H − 5.5M
Calendar	100 y: 5 odd days; 400 y: 0; leap year: 2	Count days, divide by 7
Ages	Ratio parts $5x$, $3x$; add the years to each	Same years added to both

SQL CHEAT TABLE

	HAVING, SELECT, ORDER BY, LIMIT
INNER, LEFT	Matches only; all left rows with nulls. Filter right columns in `ON` for left joins
COUNT	`COUNT(*)` counts rows; `COUNT(col)` skips nulls
WHERE, HAVING	Rows before grouping; groups after
IN, EXISTS	`NOT IN` fails with a null in the subquery; prefer `NOT EXISTS`
Window	`fn() OVER (PARTITION BY a ORDER BY b)`: `ROW_NUMBER`, `RANK` (gaps), `DENSE_RANK`, `LAG`
Top *n* per group	`ROW_NUMBER() OVER (PARTITION BY g ORDER BY x DESC)`, keep ≤ *n*
Second highest	`... WHERE salary < (SELECT MAX(salary) ...)` or `DENSE_RANK() = 2`
Transactions	`BEGIN; ...; COMMIT;` or `ROLLBACK`. Claim rows with a conditional `UPDATE` and check the row count
Isolation	Dirty, non-repeatable, phantom: each level forbids one more

OOP, ORGANISATION AND DESIGN IN BRIEF

- Pillars: encapsulation (protect), abstraction (simplify), inheritance (is-a), polymorphism (one call, many behaviours). Prefer composition over inheritance.
- Virtual: one vptr per object, one vtable per class; constructors cannot be virtual, polymorphic bases need a virtual destructor. Abstract class holds code and a contract; interface is a contract only.
- SOLID: one reason to change; extend, do not edit; subclass substitutes for base; small interfaces; depend on abstractions.
- Normal forms: atomic cells; whole key; nothing but the key; every determinant a super key.
- B+ tree: shallow (fan-out 100: three levels per million rows), linked leaves for ranges, leftmost-prefix rule; indexes slow writes.
- Two's complement: negate by invert and add one; range -2^{n-1} to $2^{n-1} - 1$; overflow when same-sign operands give an opposite-sign result. Float: 1, 8, 23 bits, bias 127.
- Hierarchy: L1 about 1 ns, DRAM about 100 ns, SSD about 100 µs: each step ten to a thousand times slower. AMAT = hit + miss rate × penalty. Pipeline speedup $kn/(k + n - 1)$; hazards: structural, data, control.
- Design: clarify, estimate, sketch, deep-dive two parts, failures, trade-offs. Campus: 8,000 × 0.4 × 2 Mb/s = 6.4 Gb/s. Limiter: token bucket for bursts, sliding-window counter ≈ previous × overlap + current, 429 and `Retry-After`. CDN: 500 TB/day is 46 Gb/s mean, three times at peak.

BEHAVIOUR, RÉSUMÉ AND OFFER

- STAR: S and T 20 s, Action 60 s in "I", Result 20 s with a number, Learning 10 s. Eight stories tagged to two to four principles of the sixteen; a fresh story per interviewer.
- Tell me about yourself: Present, Past, Future in 60 to 90 s, with a named product in the last sentence.
- Offer: compare *guaranteed annual cash*; ask what variable pay paid last year; one-time bonuses are not recurring; on-campus offers rarely negotiate.
- Résumé checklist: (1) one page, (2) single column, (3) PDF, (4) no photograph, (5) live links, (6) CGPA only if 7.5 or above, (7) skills grouped, (8) only defensible skills, (9) three projects, (10) a number in each bullet, (11) every bullet begins with a past-tense verb, (12) XYZ form, (13) stack named for each project, (14) README and diagram on GitHub, (15) tests visible, (16) certifications matched to a project, (17) achievements with ratings, (18) keywords match the job, (19) no skill bars, (20) consistent dates.
- Project defence: ten questions (problem, architecture, your share, hardest bug, trade-offs, tests, results, ten and a thousand times load, security, change); prepare a two-minute and a ten-minute version.

FLASHCARDS

Q	A	Q	A
Net of $+a\%$ then $+b\%$	$a + b + ab/100$	Unit-digit cycle of 7	7, 9, 3, 1
Price up $r\%$: cut consumption by	$100r/(100 + r)\%$	False weight gain	error/(true − error) × 100
Alligation ratio	(dearer − mean) : (mean − cheaper)	A and B together	$ab/(a + b)$ days
Average speed, equal distances	$2xy/(x + y)$	km/h to m/s	× 5/18
Crossing a platform	(train + platform)/speed	CI − SI, two years	$P(r/100)^2$

Circular arrangements of n	$(n-1)!$	P(at least one)	1 − P(none)
Trailing zeros of $n!$	$\lfloor n/5 \rfloor + \lfloor n/25 \rfloor + \ldots$	Angle between clock hands	$\lvert 30H - 5.5M \rvert$
Divisors of $p^a q^b$	$(a+1)(b+1)$	HCF × LCM of two numbers	Their product
How polymorphism works	vptr to vtable	Overloading binds at	Compile time
Why a virtual destructor	`delete` through a base pointer	Open-closed principle	Extend, do not edit
Rule of three	Destructor, copy constructor, copy assignment	An interface in C++	Abstract class of pure virtuals
SQL execution order	FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY	`COUNT(*)` against `COUNT(col)`	Rows against non-nulls
2NF fails when	A partial dependency on a composite key	BCNF requires	Every determinant a super key
A phantom read	New rows in a repeated range query	Leftmost-prefix rule	A composite index serves queries from its left columns
Durability comes from	The write-ahead log	Range of n-bit two's complement	-2^{n-1} to $2^{n-1}-1$
Negate in two's complement	Invert and add one	IEEE 754 single	1, 8, 23 bits; bias 127
Cache fields	Offset $\log_2 B$, index $\log_2$ sets, tag the rest	Pipeline speedup	$kn/(k+n-1)$
AMAT	Hit time + miss rate × penalty	Six steps of design	Clarify, estimate, sketch, deep-dive, failures, trade-offs
1 Gb/s for a day	10.8 TB	Token bucket against leaky bucket	Burst against smooth
STAR time split	20 s, 60 s, 20 s, 10 s	The XYZ bullet	Accomplished X, measured by Y, by doing Z

II

Cheat Codes

Decoders and quick tables for the minutes when the clock is running and the explanation is already learnt.

A1 DSA: constraints, patterns and the 45-minute round ★

A2 Networking: mnemonics, speed tricks and reference tables ★

A3 Online assessments: formats, pacing and elimination ★

A4 HR: the question bank and the offer ★

II.0 How to use this part MUST · ■□□

This part is a *decoder* layer. A table marked ★ exists nowhere else in the book and is printed here in full. Everything else is a single line that names the pillar section where the idea is taught, so that a topic is never explained twice. Consequently, if a row looks too short to learn from, the page printed beside it is the place to read.

Wherever a table states what a company asks or how its test is laid out, the source is a set of candidate reports. We therefore write "reported" and give the year, and we never present a report as a rule. Use the tables to plan; confirm on the day.

Candidate reports — anonymous, self-reported accounts on sites such as GeeksforGeeks, Glassdoor, Jointaro and Medium. They span 2016 to 2026, formats change every year, and a drive's own invitation always overrides this book.

You have	Open
30 seconds in a coding round	the decoders of A1 (p. 754)
the night before a networking round	the mnemonics and tables of A2 (p. 760)
an online assessment tomorrow	A3 (p. 765), then the mock papers of Part III
an HR round or an offer letter	A4 (p. 768)

A1 DSA cheat codes MUST · ■□□

> BIG IDEA
>
> A coding problem announces its algorithm twice, once in the size of its input and once in its wording, and reading both before writing anything removes most wrong starts.

The constraint decoder

The basic table, from $n \le 12$ to $n \ge 10^9$, is printed in the Revision Spread of Pillar 1 (the table (p. 101)). Its rule of thumb is that a judge performs about 10^8 simple operations per second. For example, at $n = 10^5$ an $O(n \log n)$ method costs about 1.7×10^6 steps, an $O(n\sqrt{n})$ method about 3.2×10^7, and an $O(n^2)$ method 10^{10}, which is far too slow. The statement carries other numbers besides n, and these select the technique as well.

★ Constraint signals other than *n*.

Signal in the statement	It implies	Reach for
$n \le 40$	2^{40} is too large, but two halves of 2^{20} are not	meet in the middle: enumerate each half, combine by sorting or hashing
$k \le 20$ with large n	state is a subset of k items	bitmask DP over k, $2^k \times k$
values up to 10^9	no array can be indexed by value	hash map, or coordinate compression after sorting
"return the answer modulo $10^9 + 7$"	the count is astronomically large	DP or combinatorics; reduce after every product, in 64 bits
sum of n over all tests $\le 2 \times 10^5$	per-test $O(n \log n)$ is fine	never re-allocate a large array per test

grid up to 1000 × 1000	10^6 cells	one O(cells) BFS; avoid deep recursion
string length ≤ 10^5	comparing all substrings is hopeless	window, hash, KMP; never O(n^2)
n up to 10^{18}	the loop cannot run n times	O(log n): binary search on the value, fast exponentiation
memory limit 256 MB	about 6 × 10^7 32-bit integers in all	no $10^5 \times 10^5$ table; roll the DP rows
Python, 1 second	about 10^7 simple operations	built-ins (`sort`, `Counter`, `heapq`), not nested loops
a stream, or "one pass"	memory must stay bounded	heap of size k, reservoir sampling, a counter table

The keyword decoder

The first 31 rows of the decoder, from "sorted" to "validate a sequence of protocol events", are in the same Revision Spread (the keyword table (p. 101)) with the section of each template. The table below holds the rows that one does not find there, with LeetCode numbers as anchors: each number names one standard problem for practice, and none is a prediction of what will be asked.

★ Keyword decoder, second half.

Signal in the problem	Think	Anchors
minimum meeting rooms; peak overlap	sort starts, min-heap of end times; or a sweep line	253, 1094
insert into sorted intervals; remove fewest to avoid overlap	one linear merge; greedy by earliest end	57, 435
add v to every element of [l, r], many times	difference array, then one prefix pass	1094, 1109
1 to n with one missing or repeated	cyclic sort, sign marking, or XOR	268, 287, 448
rotated sorted array; minimum; peak	binary search on which half is sorted	33, 153, 162
median of two sorted arrays	binary search on the partition	4
smallest window containing all of ...; permutation in a string	variable window with need/have counters	76, 567
at most K distinct; replace at most K	variable window with counts	424, 904, 1004
product or sum below K, positives only	shrink the window while it is too large	713, 209
trapped water; container	two pointers with running maxima, or a monotonic stack	42, 11
longest palindromic substring	expand around each centre, O(n^2)	5
matrix: rotate, spiral, zero rows	index arithmetic; transpose then reverse rows	48, 54, 73
grid: find a word or a path with moves	DFS that marks and unmarks cells	79
nearest gate; rot spreads each minute	multi-source BFS from all sources at once	542, 994
minimum steps between words or states	BFS over an implicit graph	127
edge weights are only 0 or 1	0-1 BFS with a deque	1368

minimise the largest edge or effort on a path	Dijkstra with max instead of sum; or binary search plus BFS	1631, 778
number of paths in a grid or DAG	DP in topological order	62, 63
copy a structure with arbitrary pointers	hash map from old node to new node	133, 138
custom alphabet order	build edges from adjacent words, topological sort	269, 953
each item once, can a total be reached	0/1 knapsack, capacity looped downwards	416, 494
unlimited coins: fewest, or number of ways	unbounded knapsack; for ways, coins in the outer loop	322, 518
stock trades with cooldown or *k* trades	state-machine DP	121, 309, 188
split a string into valid words; decode ways	1-D DP over prefixes	139, 91
k-th largest in an unsorted array or stream	heap of size *k*, or quickselect	215, 703
no two adjacent equal; cooldown between tasks	max-heap by count, greedy	767, 621
most frequent words	counter, then heap or bucket sort	347, 692
random element with equal probability from a stream	reservoir sampling	382, 398
insert, delete and random in O(1)	array plus index map; swap with the last	380
least frequently used; value at a time	frequency lists; binary search on timestamps	460, 981
find a pattern fast; repeated substring	KMP failure function or rolling hash	28, 459
maximum XOR of a pair	trie over bits	421
primes up to *n*; huge power; gcd	sieve; fast exponentiation; Euclid	204, 50
tree path sum through a node	post-order returns one-sided gain, a global holds the best	124, 543
serialise or rebuild a tree	preorder with null markers	297, 105
jump, gas station, scheduling	greedy; justify with an exchange argument	55, 134
k-th smallest in a sorted matrix	heap of rows, or binary search on the value	378, 23
stack or queue with an extra query	pair each element with the auxiliary value	155, 232

Four of these ideas are not taught as templates in Pillar 1, so they are printed here once. Each is a few lines and is tested.

★ Four small templates.

```cpp
// 1. Cyclic sort: values 0..n with one missing -> send each value home
int missing(vector<int> a) {
    int n = a.size(), i = 0;
    while (i < n) {
        int j = a[i];
        if (j < n && a[j] != j) swap(a[i], a[j]);   // while, not if
        else i++;
    }
    for (i = 0; i < n; i++) if (a[i] != i) return i;
    return n;                                        // {3,0,1} -> 2
}
// 2. Difference array: add v to a[l..r] in O(1), rebuild once
//    d[l] += v; d[r+1] -= v;  then  d[i] += d[i-1]  for i = 1..n-1
// 3. Reservoir sampling of one item from a stream of unknown length
//    seen = 0; for x in stream: if rand(0..seen) == 0: chosen = x; seen++
//    (item i survives with probability 1/(i+1) * (i+1)/(i+2) * ... = 1/n)
// 4. 0-1 BFS: relax as in Dijkstra, but push_front when w == 0, push_back when
w == 1
```

The pattern index

The twenty-two patterns below are the reusable shapes of the pillar. The template of each is printed once, in the section named; the bug is the one interviewers most often see, and checking for it is the cheapest review one can run.

#	Pattern	Trigger	Classic bug	Template
1	Two pointers, opposite ends	sorted; pair or triple	3Sum without skipping duplicates	§1.2 (p. 12)
2	Two pointers, read and write	in place; remove or compact	writing over an element not yet read	§1.2 (p. 12)
3	Fixed window	"of size *k*"	evicting `a[i-k]` off by one	§1.2 (p. 12)
4	Variable window	longest or shortest with a condition	shrinking with `if` instead of `while`	§1.2 (p. 12)
5	Prefix sums and a map	subarray sum equals *k*	forgetting the seed `{0: 1}`	§1.2 (p. 12)
6	Fast and slow pointers	cycle, middle, *k*-th from end	testing `fast->next` before `fast`	§1.6 (p. 27)
7	List reversal	reverse a part; reorder	losing `next` before re-pointing	§1.6 (p. 27)
8	Binary search	sorted; first or last position	`(lo+hi)/2` overflow; `lo = mid` loops forever	§1.13 (p. 55)
9	Binary search on the answer	minimise the maximum	bounds: low is the largest item, high is the sum	§1.13 (p. 55)
10	Monotonic stack	next greater; histogram	storing values instead of indices	§1.7 (p. 33)
11	Monotonic deque	window maximum	not popping the expired index at the front	§1.7 (p. 33)
12	Heap	top *k*; stream median; merge *k*	C++ heap is a max-heap, Python's a min-heap	§1.10 (p. 45)
13	Intervals	overlap; merge	sorting by end when start is needed	§1.13 (p. 55)

15	Tree DFS	depth, path, subtree	mixing the returned value with the global answer	§1.8 (p. 36)
16	BFS	fewest steps; level order	marking visited on pop, not on push	§1.11 (p. 47)
17	Graph DFS and cycles	components; cycle in a directed graph	two colours where three are needed	§1.11 (p. 47)
18	Topological sort	prerequisites; ordering	not comparing the output length with *V*	§1.11 (p. 47)
19	Union–find	dynamic connectivity; loop edge	uniting nodes instead of their roots	§1.11 (p. 47)
20	Shortest paths	cheapest route	Dijkstra with a negative edge; no stale check	§1.11 (p. 47)
21	Backtracking	all combinations	storing the path itself instead of a copy	§1.12 (p. 54)
22	Dynamic programming	ways, minimum cost, reachability	wrong loop order; missing base case	§1.14 (p. 58)

A twenty-third card, bit tricks, is used heavily at Qualcomm and Broadcom: its bug is `1 << 31` on a signed `int`, and its home is §1.5 (p. 24). The shortest-path row has a networking tie, which interviewers like as a bridge question: Dijkstra is the SPF of OSPF and IS-IS (§4.5 (p. 246)), and Bellman–Ford is the distance-vector idea of RIP.

Problems that smell like networking

Network-company rounds dress ordinary algorithms in network clothes, and the disguise is the decoder. Each row names the task, the algorithm underneath and the question that solves it.

The task as stated	The algorithm beneath	Solved in
route lookup; most specific firewall rule; delete a prefix	binary trie, deepest marked node	Q1.19 (p. 71), Q1.75 (p. 71), Q1.74 (p. 71), Q1.76 (p. 71)
validate, convert or restore an IP string	single-scan validator; backtracking	Q1.5 (p. 71), Q1.39 (p. 71), Q1.22 (p. 71)
same subnet; network and broadcast of an address	mask and compare	Q1.6 (p. 71), Q1.7 (p. 71)
merge address ranges; ranges to the fewest CIDR blocks	sort and merge; align and split	Q1.94 (p. 71), Q1.51 (p. 71)
rate limiting, with bursts	token bucket; sliding window	Q1.27 (p. 71), Q1.101 (p. 71), Q1.102 (p. 71)
an ARP or route cache with expiry	hash map and doubly linked list	Q1.26 (p. 71), Q1.99 (p. 71)
SPF, and “cheapest route”	Dijkstra	Q1.20 (p. 71), Q1.85 (p. 71)
a routing loop; a link whose failure splits the network	cycle detection; bridges	Q1.12 (p. 71), Q1.87 (p. 71), Q1.89 (p. 71)
reorder packets by sequence number	min-heap	Q1.81 (p. 71)
the Internet checksum	16-bit one’s-complement sum	Q1.52 (p. 71)
allocate VLAN identifiers or ports	bitmap	Q1.53 (p. 71)

peak simultaneous flows; top sources by bytes in a huge log	sweep line; heap of size *k*	Q1.95 (p. 71), Q1.80 (p. 71), Q1.37 (p. 71)
validate a sequence of TCP events	table-driven state machine	Q1.28 (p. 71)

Which practice list

Four public lists are in common use: Blind 75 (the shortest), NeetCode 150 (the same families, more depth), the Striver SDE and A2Z sheets (broad, many topics), and Sean Prashad's list grouped by pattern. They overlap heavily, so one should be chosen and finished. For this book's readers the order is: the pattern families of the NeetCode list for breadth, then the networking problems of the table above, which no public list collects, and then the C output traps of §2.15 (p. 128), because Qualcomm, Broadcom and Arista examine C directly. Re-solve a problem from a blank page on the third and fourteenth day after the first attempt (the calendar), and retire it once it is solved without hints in under 25 minutes.

The Big-O and edge-case cards

The data-structure, sorting and graph tables, the recurrences and the landmark numbers ($\log_2 10^6 \approx 20$, `INT_MAX` $\approx 2.1 \times 10^9$) are in the Revision Spread of Pillar 1 (Big-O (p. 101)), as is the edge-case checklist with its networking inputs. They are not reprinted. Before saying "done", run that checklist aloud.

The 45-minute round and the talk-track

UMPIRE, from CodePath, names six steps: **U**nderstand, **M**atch, **P**lan, **I**mplement, **R**eview, **E**valuate. They are the six-step routine of §1.1 (p. 9) under other names, and the minute-by-minute table of §1.17 (p. 67) already allots the time. What the book does not print elsewhere is the spoken script, which is the part that earns the communication marks.

★ UMPIRE, minutes and phrases.

Step	Minutes	Say this	Not this
Understand	0–5	"Let me restate it: given … return …. Can the input be empty? Are values bounded? Duplicates, negatives?"	starting to type
Match	5–8	"Contiguous and longest suggests a sliding window; let me check that it fits."	"I think I have seen this one."
Plan	8–15	"A naive way is O(n^2) over all pairs. The inner loop repeats a sum, so a prefix sum removes it. Is that direction fine with you?"	coding before agreement
Implement	15–32	"I will write `isValid` first and come back to it." / "This loop keeps the window valid."	silence for more than two minutes
Review	32–38	"Trace `[2, 7, 11, 15]`, target 9: at *i* = 1 I need 2, it is in the map at 0, so I return `[0, 1]`."	"It should work."
Evaluate	38–45	"O(*n*) time, O(*n*) space for the map. If memory is tight, sort and use two pointers: O(*n* log *n*), O(1)."	claiming a complexity without a reason

Three further phrases cover the awkward moments. When stuck: "I am looking for repeated work; this recomputation is the candidate." When given a hint: "That helps; if the array is sorted, two pointers replace the hash set." When one finds a bug: "This fails for a single element; I will add a guard and re-run the trace." Candidate reports for Google add that interviewers expect two or three approaches and their trade-offs before any code, and one report describes an interviewer

steering a candidate from union–find to BFS because the first was lengthy to write. Hence, always offer the simpler alternative and say why it suffices.

The standard-library quick reference

The containers are taught in §2.21 (p. 141) (C++) and §2.16 (p. 134) to §2.18 (p. 137) (Python); C has none, and its hash-table and list templates are in §1.3 (p. 16) and §1.6 (p. 27). This table answers the question asked at the keyboard: “what do I type?”

★ I need …, in C++ and in Python.

I need	C++	Python
sort by a key, ties by another	`sort(v.begin(), v.end(), [](auto& a, auto& b){ return a.x < b.x; })` (not stable)	`v.sort(key=lambda t: (t[0], -t[1]))` (stable)
a min-heap	`priority_queue<int, vector<int>, greater<int>> pq;`	`heapq.heappush(h, x)`; negate for a max-heap
first element not less than *x*	`lower_bound(v.begin(), v.end(), x)`	`bisect.bisect_left(a, x)`
frequency count	`unordered_map<int,int> c; c[x]++;`	`Counter(a)`
adjacency list	`vector<vector<int>> g(n); g[u].push_back(v);`	`defaultdict(list)`
a double-ended queue	`deque<int> d; d.push_front(x);`	`deque(); d.appendleft(x)`
remove duplicates after sorting	`v.erase(unique(v.begin(), v.end()), v.end());`	`sorted(set(a))`
prefix sums	`partial_sum(a.begin(), a.end(), p.begin());`	`list(accumulate(a))`
a 64-bit sum	`accumulate(a.begin(), a.end(), 0LL)`	`sum(a)` (no overflow)
memoisation	`vector` or `unordered_map` table	`@functools.cache`
“infinity”	`INT_MAX`, or `1e9` for sums; `LLONG_MAX`	`math.inf`
copy the current path in backtracking	`res.push_back(path);`	`res.append(path[:])`
fast input	`ios::sync_with_stdio(false); cin.tie(nullptr);`	`input = sys.stdin.readline`
delete one copy from a multiset	`ms.erase(ms.find(x));` (`erase(x)` deletes all)	`Counter` and decrement

Four costs are worth knowing by heart. In Python, `list.pop(0)` and `insert(0, x)` are O(*n*) (use `deque`), `x in list` is O(*n*) whereas `x in set` is O(1) on average, and `s += c` in a loop can be quadratic (use `"".join`). In C++, `m[key]` on a `map` inserts a default value when the key is absent, so use `count` or `find` for a pure lookup. In C there is no library: the comparator for `qsort` must return `(x > y) - (x < y)` and never `x - y`, which overflows (§2.5 (p. 112)); `strncpy` does not terminate a truncated string (§1.4 (p. 19)); and `memset` is safe only for the byte patterns 0 and −1.

A2 Networking cheat codes MUST · ■□□

BIG IDEA

> Networking answers are lists with an order, such as states, tie-breakers and numbers, and a list is retrieved far more reliably from a hook than from a paragraph.

Most reference tables already sit in the Revision Spreads of the pillars: the /8 to /32 mask table, the ports and protocol numbers, the TCP states and the IPv6 prefixes in Pillar 3 (the spread (p. 225)), and administrative distance, the OSPF states and LSAs, the BGP best-path order and the STP timers in Pillar 4 (the spread (p. 298)). They are not reprinted. This section holds what those spreads lack.

The mnemonic page

Mnemonics that live elsewhere: the OSI order (*All People Seem To Need Data Processing*, §3.2 (p. 165)), the BGP order (*N WLLA OMNI*, §4.7 (p. 256)), the syslog levels (*Every Awesome Cisco Engineer Will Need Ice cream Daily*, §4.13 (p. 279)) and DORA (§3.6 (p. 183)). The rest are printed here. They are our own inventions unless stated, and a silly sentence is easier to keep than a sensible one.

★ Mnemonics.

Topic	Hook	It decodes to
TCP/IP layers, top to bottom	*Aunt Tina Is Nice*	Application, Transport, Internet, Network access
PDU names, top to bottom	*Do Some People Fear Birthdays*	Data, Segment, Packet, Frame, Bits
TCP flags in header order	*Cats Eat Under All People's Red Sofas Freely*	CWR, ECE, URG, ACK, PSH, RST, SYN, FIN
IPv4 flag bits	*Zero, Don't, More*	reserved 0, DF, MF
802.1Q tag	*3-1-12 after 8100*	TPID 0x8100, then PCP 3 bits, DEI 1 bit, VID 12 bits
OSPF neighbour states	*Don't Ignore Two Excited Elephants Leaving Fridges*	Down, Init, 2-Way, ExStart, Exchange, Loading, Full. Stuck in ExStart or Exchange: MTU. Stuck in Init: one-way hellos. Stuck in 2-Way between DROthers: normal.
OSPF packet types 1 to 5	*Hello Dear Little Sweet Angel*	Hello, Database Description, Link-State Request, Link-State Update, Link-State Acknowledgement
OSPF LSA types	*Really Nice Summaries Are Easy Now*	1 Router, 2 Network, 3 Summary, 4 ASBR summary, 5 External, 7 NSSA external
BGP session states	*Idle Cats Always Open Old Envelopes*	Idle, Connect, Active, OpenSent, OpenConfirm, Established. Active means "still trying to connect", not "healthy".
STP 802.1D port states	*Big Lions Like Food*	Blocking, Listening, Learning, Forwarding
Process memory, low to high	*Terrible Dogs Bark, Hounds Squeal*	Text, Data, BSS, Heap, Stack
Deadlock conditions	*My Hungry Nephew Cries*	Mutual exclusion, Hold and wait, No pre-emption, Circular wait
Early ports	*the 21-22-23-25 walk*	FTP 21, SSH 22, Telnet 23, then SMTP 25 (24 is skipped)

Subnetting speed tricks

The method (block size, then the multiple not above the address) is taught in §3.4 (p. 174) and the master table is there. Speed comes from not computing what can be recalled, so four strips are worth learning cold.

★ **Strip 1: powers of two.**

2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8	2^9	2^{10}	2^{11}	2^{12}	2^{16}	2^{20}	2^{24}	2^{32}
1	2	4	8	16	32	64	128	256	512	1,024	2,048	4,096	65,536	1,048,576	16,777,216	4,294,967,296

★ **Strip 2: the mask octet ladder.** Each extra bit adds half of the previous step: 128, then +64, then +32 and so on. The block size is 256 minus the octet.

Bits set in the octet	1	2	3	4	5	6	7	8
Octet value	128	192	224	240	248	252	254	255
Block size	128	64	32	16	8	4	2	1

★ **Strip 3: hosts needed to prefix.** Find the first row whose usable hosts are at least the requirement. The same table reads backwards for “how many hosts fit”.

Prefix	/30	/29	/28	/27	/26	/25	/24	/23	/22	/21	/20	/19	/18	/17	/16
Usable hosts	2	6	14	30	62	126	254	510	1,022	2,046	4,094	8,190	16,382	32,766	65,534

For subnets rather than hosts, borrow the smallest s with 2^s at least the number required: 2 or fewer subnets need 1 bit, 3 or 4 need 2, 5 to 8 need 3, 9 to 16 need 4. Thus, six subnets from a /24 borrow 3 bits and give /27.

★ **Strip 4: summarising.** A run of 2^k equal-sized blocks can become one route only if the run is contiguous and its first block starts at a multiple of the whole run. The prefix then shortens by k.

Blocks in the run	2	4	8	16	32	64
Prefix shortens by	1	2	3	4	5	6
The first block must be a multiple of	2	4	8	16	32	64

For example, eight /24 blocks from 192.168.24.0 form 192.168.24.0/21, because 24 is a multiple of 8. Four blocks from 10.1.3.0 do not combine, because 3 is not a multiple of 4; the run 10.1.3.0 to 10.1.6.0 needs three routes (10.1.3.0/24, 10.1.4.0/23, 10.1.6.0/24).

★ **Sixty-second drills.** Cover the answer column, give each result in under a minute, and aim for thirty seconds by the end of Pillar 3.

Address and prefix	Network	Broadcast	Usable hosts
10.9.77.18/22	10.9.76.0	10.9.79.255	1,022
172.17.130.9/19	172.17.128.0	172.17.159.255	8,190
192.168.40.201/27	192.168.40.192	192.168.40.223	30
203.0.113.90/26	203.0.113.64	203.0.113.127	62

10.200.5.99/20	10.200.0.0	10.200.15.255	4,094
172.30.9.250/23	172.30.8.0	172.30.9.255	510
192.0.2.37/29	192.0.2.32	192.0.2.39	6
10.77.130.1/17	10.77.128.0	10.77.255.255	32,766
198.51.100.130/25	198.51.100.128	198.51.100.255	126
172.16.7.7/21	172.16.0.0	172.16.7.255	2,046
10.3.3.3/28	10.3.3.0	10.3.3.15	14
100.100.100.100/18	100.100.64.0	100.100.127.255	16,382

Three more kinds of drill complete the set. *Same subnet?* 172.16.20.30 and 172.16.31.1 under 255.255.240.0: yes, both lie in 172.16.16.0 to 172.16.31.255. 10.0.5.250 and 10.0.6.2 under 255.255.254.0: no, the blocks are 10.0.4.0 and 10.0.6.0. 192.168.1.62 and 192.168.1.65 under 255.255.255.192: no, because 64 divides them. *Wildcard:* the wildcard of /26 is 0.0.0.63 and of /21 is 0.0.7.255. *Summary:* 192.168.9.0/26 to 192.168.9.192/26 (four blocks, start 0) is 192.168.9.0/24.

Ports and protocol numbers, second page

The ports every candidate knows (FTP, SSH, Telnet, SMTP, DNS, DHCP, HTTP, BGP, HTTPS and so on), the IP protocol numbers up to OSPF and the common EtherTypes are in the Revision Spread of Pillar 3 (the table (p. 225)). The table below adds the ports and numbers that networking roles are asked about.

★ Good to know, for networking roles.

Port or number	Service	Note
TCP 49	TACACS+	device administration; RADIUS is UDP 1812 and 1813
UDP 500, UDP 4500	IKE, IPsec NAT traversal	ESP is IP protocol 50 and AH is 51
TCP 646	LDP	also UDP 646 for hellos
TCP 830	NETCONF over SSH	gNMI has no old default; many devices use 9339 or 57400
UDP 3784, 3785	BFD single-hop control, echo	multi-hop control is UDP 4784
UDP 4789, UDP 6081	VXLAN, Geneve	VXLAN adds 50 bytes of overhead
UDP 1985	HSRP	224.0.0.2 (v1) and 224.0.0.102 (v2); VRRP is IP protocol 112
UDP 521	RIPng	RIP is UDP 520
TCP/UDP 389, 636	LDAP, LDAPS	
TCP/UDP 2049	NFS	
TCP/UDP 5060, 5061	SIP, SIP over TLS	
UDP 5353	mDNS	
TCP 6653	OpenFlow	6633 in older controllers
IP protocol 4, 41	IP-in-IP, IPv6-in-IPv4	6in4 tunnels

IP protocol 88, 103, 132	EIGRP, PIM, SCTP	
EtherType 0x8809	slow protocols	LACP is the best-known user
EtherType 0x88CC	LLDP	
EtherType 0x8847	MPLS unicast	multicast is 0x8848
EtherType 0x88A8	802.1ad (QinQ)	outer tag of a double-tagged frame
EtherType 0x8863, 0x8864	PPPoE discovery, session	MTU 1492

Ephemeral ports: IANA recommends 49152 to 65535, whereas Linux defaults to 32768 to 60999 (`/proc/sys/net/ipv4/ip_local_port_range`). Control-plane multicast addresses are also asked: OSPF 224.0.0.5 and .6, RIPv2 .9, EIGRP .10, PIM .13, VRRP .18; Layer 2: STP 01:80:C2:00:00:00, LACP 01:80:C2:00:00:02, LLDP 01:80:C2:00:00:0E, and CDP, VTP and UDLD 01:00:0C:CC:CC:CC.

★ **Protocols that ride on others.** BGP rides TCP 179. OSPF and EIGRP are IP protocols 89 and 88, with no TCP or UDP. RIP rides UDP 520. IS-IS sits directly on Layer 2 and has no IP header at all. DHCP and DNS are applications over UDP. ICMP is a Layer 3 protocol that travels inside IP. BGP uses TCP because it needs reliable, ordered delivery of incremental updates, and so need not build its own retransmission (Q4.32 (p. 286)).

Route preference across vendors

The Cisco administrative-distance table is in the Revision Spread of Pillar 4 (the table (p. 298)); the decision order (longest prefix, then distance, then metric) is in §4.4 (p. 243). Junos calls the same idea *route preference*, and Arista and Nokia use numbers of their own, so the table puts them side by side.

★ **Default distance or preference; lower wins everywhere.**

Source	Cisco IOS	Junos	Arista EOS	Nokia SR OS
Connected, local	0	0	0	0
Static	1	5	1	5
OSPF internal	110	10	110	10
OSPF external	110	150	110	150
IS-IS	115	15 (L1), 18 (L2) internal	115	15 (L1), 18 (L2) internal
RIP	120	100	120	100
eBGP	20	170	200	170
iBGP	200	170	200	170

> **TRAP**
>
> *Claim:* "eBGP beats OSPF for the same prefix." *Why it is wrong:* on Cisco IOS it does (20 against 110), but on Junos and SR OS the OSPF route wins (10 against 170), and on EOS it also wins (110 against 200). *Correct statement:* name the platform first; distance is a vendor default, and a floating static route must be set above the number of the protocol it backs up, not above 1. Treat the Arista and Nokia columns as reported defaults and confirm them with `show ip route` or the platform documentation.

Pointers

Need	Where	Question
Header bit layouts: Ethernet, 802.1Q, ARP, IPv4, IPv6, TCP, UDP	§3.3 (p. 168), §3.5 (p. 179), §3.6 (p. 183), §3.7 (p. 187), §3.9 (p. 192), §4.2 (p. 233)	Q3.3 (p. 212)
"What happens when you type google.com": the nine-stage ladder and its two-minute version	§3.11 (p. 206)	Q3.45 (p. 212)
Troubleshooting skeleton, the SCALPEL card and the layer command matrix	§11.2 (p. 636), §11.5 (p. 642)	Q11.1 (p. 678)
MTU, MSS and overheads (PPPoE 1492, GRE 24 bytes)	§3.5 (p. 179), §4.14 (p. 282)	Q3.16 (p. 212)
Wi-Fi and cellular standards	Pillar 6 (p. 358)	

A3 Online-assessment cheat codes MUST · ■□□

> BIG IDEA
>
> An online assessment rewards the candidate who knows the format before it starts, because the format fixes the time per question and hence which questions to skip.

Reported formats

The platforms (HackerRank, AMCAT, Mercer Mettl, CoCubes) and their habits, such as hidden test cases, adaptive sections and locked sections, are tabulated in §12.1 (p. 688). What differs by company is the paper itself. The table records what candidates report, with the year where a report gave one, and nothing in it is a rule.

★ Reported OA formats, nine companies (candidate reports, 2022 to 2026).

Company	Platform reported	Reported format	Topics
Cisco	HackerRank	2023: 90 min, 42 questions (40 MCQ and 2 coding); 2024: 41 MCQ and 1 coding; 2026 guides: about 90 min, about 40 MCQ and 2 DSA. Coding in C, Python or Java; AI proctoring; no negative marking reported. The 2025 network-engineer drive (Bengaluru) added a presentation after the test.	networking (TCP/IP, OSI, routing, security), OS, DBMS, aptitude, C output
Juniper	HackerRank	90 min, 33 questions (32 MCQ and 1 coding). October 2024 internship: 38 questions (35 MCQ and 3 coding), tight on time. Older reports: about 50 networking and C MCQs. Four to five rounds in all.	aptitude, CN, DBMS, OS, HTML/CSS, output prediction, time complexity
Qualcomm	varies by drive	sectioned MCQs; no fixed duration reported	aptitude, C (pointers, `const`, structs and unions, padding, `volatile`, `static`), OS, architecture, digital logic, networking
Arista	HackerRank, then CoderPad	1.5 h test in C or C++: four to five coding problems, or a mix of coding and MCQs; another report gives two medium-to-hard problems	C debugging, linked lists, bits, arrays, OS, networking

		ic programs, in C or C++ in some drives	aptitude, GCD, sorting, balanced brackets
Microsoft	Codility, HackerRank or AMCAT	2 coding problems, medium to hard; on-campus timelines are short	DP, graphs, greedy, binary search
Amazon (NDE)	Amazon's own tests, varies	CV screen, a phone screen heavy on troubleshooting, then a loop of four to five rounds	BGP, OSPF, TCP/IP, MTU and MSS, Python, Leadership Principles
Google	online coding or phone screen	one to two LeetCode-style problems; the network-engineer track adds networking and a basic Python question in a shared document	DSA, networking, scripting
Broadcom	varies	C-heavy technical rounds; several interviews	bits, linked lists, pointers, stacks and queues, OS

Note that the Cisco and Juniper rows are the best documented, because both recur in reports every year; the others are thin. The coding half of each company is described in §1.0 (p. 8).

Know your battlefield — before each drive, read three things on the instruction screen: negative marking, section locking, and whether a language is forced. Everything in A3.2 depends on them.

The time-management plan

The method is the two-pass method of §12.1 (p. 688): read for three minutes, answer under 45 seconds in pass one, flag, and return. The worked 90-minute plan for a Cisco-style paper is in §1.0 (p. 8). The table applies the same rules to the other reported formats; each row sums to the stated total.

★ Budgets by format (minutes).

Format	Read	MCQ	Coding	Buffer	Total	Rule of thumb
Juniper A: 32 MCQ and 1 coding, 90 min	3	27	45	15	90	about 50 s per MCQ; the buffer funds a second pass
Juniper B: 35 MCQ and 3 coding, 90 min	3	26	20 + 20 + 15	6	90	45 s per MCQ; bank every easy coding case first
Qualcomm: 50 MCQ, 60 min	2	17 (aptitude, 15) + 20 (C, 20) + 8 (OS and architecture, 10) + 4 (CN, 5)	none	9	60	answer CN and OS first if sections are not locked: they are the quickest marks
Arista: 4 coding problems, 90 min	5	none	18 × 4 = 72	13	90	order by ease; submit a brute force at minute 10 of each
Nokia: 48 MCQ then 2 programs, about 100 min	0	15 (English) + 22 (quantitative) + 18 (logical)	20 + 20 (after the MCQs)	5	100	AMCAT times its own sections and adapts to accuracy, so early answers matter most
Microsoft or Amazon: 2 coding problems, 90 min	5	none	35 + 40	10	90	give the harder problem the extra five minutes only if the first is complete

The checks in each row are arithmetic: 3 + 27 + 45 + 15 = 90; 3 + 26 + 55 + 6 = 90; 2 + 49 + 9 = 60; 5 + 72 + 13 = 90; 55 + 40 + 5 = 100; 5 + 75 + 10 = 90. For the Qualcomm paper, 17 + 20 + 8 + 4 = 49. In every case the buffer is for re-running edge cases, not for starting a new problem.

★ Coding triage: when the verdict is not “accepted”.

Verdict or symptom	Usual cause	Fix
time limit exceeded, C++	`cin`/`cout` unsynchronised, or `endl` flushing every line	`ios::sync_with_stdio(false); cin.tie(nullptr);` and `'\n'`
time limit exceeded, Python	`input()` on large input; a nested loop	`sys.stdin.readline`; a built-in or a hash map
wrong answer on a few hidden cases	overflow in a sum or product	`long long`; reduce modulo early
runtime error on one case	empty input, $n = 1$, an index of -1	guard before the loop
recursion error or crash on a deep case	recursion depth about 10^5 in Python or a skewed tree	iterative version, or raise the limit
wrong format	trailing space or missing newline; reading n lines when input ends at end of file	print exactly as the sample; read to end of file when n is not given
memory limit exceeded	an $n \times n$ table	a rolling row, or a map of used cells
passes samples only	the brute-force logic is wrong, not the speed	write a 10-line brute force and compare on random small input

MCQ elimination tactics

Eight habits remove options before any arithmetic starts. A seconds budget applies to all of them: if no option has fallen after 90 seconds, flag the question and leave.

★ Elimination, with one worked case each.

#	Tactic	Worked case
1	Strike absolute words (“always”, “never”, “guarantees”) unless the statement is a definition	*Which statement about UDP is correct? (a) It always delivers in order (b) It never uses a checksum (c) It has no connection set-up (d) It guarantees delivery.* Three options carry absolutes and are false; the answer is (c).
2	Map each option to its layer; wrong-layer options are distractors	*Which device separates broadcast domains? Hub, switch, router, repeater.* Only a router (or a Layer 3 switch with VLANs) works.
3	Estimate the order of magnitude first	*Transmission time of a 1,500-byte frame on 1 Gb/s: (a) 12 ms (b) 1.2 ms (c) 12 µs (d) 1.2 µs.* 1,500 bytes is about 10^4 bits and $10^4 / 10^9$ is 10^{-5} s, so (c); exact: 12,000 / 10^9 = 12 µs.
4	In output questions, look first for a compile error or undefined behaviour	`const int c = 5; c = 6; printf("%d", c);` *prints (a) 5 (b) 6 (c) compile-time error (d) undefined.* The assignment to a `const` object is rejected: (c).
5	If two options are opposites, one of them is probably right	*TCP is (a) connectionless (b) connection-oriented (c) without flow control (d) unordered.* (a) and (b) contradict each other; (c) and (d) are false, so (b).
6	Options that are identical in effect cannot be a unique answer; in a “which is NOT” question the odd one out is	*Which does not add one to* `x`*? (a)* `x++` *(b)* `++x` *(c)* `x += 1` *(d)* `x == x + 1`. Three are equivalent, so the odd one is (d), a comparison.
7	Read the unit of the question: “hosts” subtracts two, “addresses” does not	*192.168.5.0/26: maximum hosts?* $2^6 - 2 = 62$, not 64.

8	Check the units digit and the digit sum before multiplying	*143 × 87 is (a) 12,341 (b) 12,441 (c) 12,451 (d) 12,641.* The units digit is 3 × 7 = 21, so 1: (b) or (c). Digit sums: 143 gives 8, 87 gives 6, 48 gives 3 (mod 9); 12,441 sums to 12, then 3; 12,451 sums to 13, then 4. So (b); exact: 143 × 87 = 12,441.

Aptitude and C-trap indexes

Aptitude formula cards are not repeated here. Each topic has one card with two solved examples, and the forty further MCQs of the Pillar 12 set (p. 733) follow the same order.

Aptitude topic	Card
Divisibility, divisors, cycles of unit digits, remainders, HCF and LCM, bases	§12.2 (p. 690)
Percentages, profit and loss, ratio, mixtures, averages	§12.3 (p. 692)
Work and pipes, speed, trains, boats	§12.4 (p. 695)
Interest, permutations, probability, ages, clocks and calendars	§12.5 (p. 697)
Series, coding, blood relations, seating, syllogisms	§12.6 (p. 701)
Data interpretation, sufficiency, verbal ability	§12.7 (p. 704)

The C output traps are numbered T1 to T36 in §2.15 (p. 128). Use the index below to jump from what a snippet contains to the trap that explains it.

The snippet contains	Look at
two writes to one variable, `i++` with `++i`	T1, T12, T17
`sizeof`, array parameters, `strlen`	T2, T6, T25
`printf` return value, format specifiers	T3, T31
signed with unsigned, plain `char`, overflow, a countdown loop	T4, T5, T21, T28
macros, `static` locals	T7, T11
`switch` without `break`; comma and `?:` operators	T8, T9, T30
`&&` and `||` with side effects	T10
pointer arithmetic, `const`, string literals, dead locals	T12, T13, T14, T15, T16
structures, padding and unions	T18, T19
`volatile`	T20
floating point, shifts, division of negatives	T22, T24, T32
character constants, octal literals	T23, T29
uninitialised data, double free, where a variable lives	T26, T27, T36
`=` in a condition, `==` against `&`, `i[a]`	T33, T34, T35

A4 HR cheat codes MUST · ■□□

BIG IDEA

> An HR round is short, and the offer that follows it is a document, so both reward the candidate who has prepared a frame in advance, not a speech.

Pointers

The behavioural material is taught once, in Pillar 12.

Need	Home	Model answers
STAR and STAR-L, with the Action share of the time	§12.12 (p. 721)	Q12.43 to Q12.50 (p. 733)
The sixteen Amazon Leadership Principles and the story bank	§12.12 (p. 721)	Q12.43 to Q12.50 (p. 733)
“Tell me about yourself”, with the product named for each company	§12.12 (p. 721)	Q12.41 (p. 733)
The eleven standard HR questions and their frames	§12.12 (p. 721)	Q12.42 (p. 733)
The résumé, the bullet formula and project defence	§12.13 (p. 726)	Q12.51 to Q12.54 (p. 733)

The HR question bank, continued

Every frame has the same three parts: a plain claim, one piece of evidence, and a link to the role. Keep the answer under 60 seconds and finish with the link. The eleven questions already framed in §12.12 (p. 721) (why this company, why networking, strengths, weakness, five years, relocation and on-call, a conflict, why hire you, higher studies, fresher salary, questions for us) are not repeated.

★ Twenty further questions.

Question	Frame
What do you know about us?	One product, one recent public announcement, one team; check the careers page the day before. Then: “this is why the role interests me.”
Your biggest achievement?	One project or result with a number, in two sentences; then what it taught you.
How do you handle pressure?	Name a method (list tasks, ask early for help, cut scope), then a deadline from a project or exam that shows it working.
Night shifts or on-call?	Answer honestly. Operations teams run around the clock, so show that you know it; ask about the rota after the offer, not before.
Do you have other offers?	Tell the truth. “I am in process elsewhere, and this role is a strong fit because …”. Name no company unless asked directly.
Why is your CGPA low, or why a backlog?	State the fact, the cause in one clause, and the recovery: a rising trend, a cleared backlog, a project built meanwhile. No excuses.
Why did you choose your branch (non-CS)?	Link it to networking: signals, communication systems, embedded hardware. Add a lab that bridged the two.
How do you keep up with technology?	Name sources and a habit: vendor documentation, RFCs, a home lab, one recent topic you learnt and what you did with it.
How do you take criticism?	One instance, what you changed, and how the next result differed.
Describe your ideal manager or team.	Clear goals, regular feedback, technical depth. Avoid “no supervision” and any complaint about a former guide.
What will you do in your first 90 days?	Learn the systems, shadow on-call, fix one small defect, ask questions. Never promise to rebuild anything.

Will you do support or legacy work?	Yes; operations work is where networks are learnt. Add the skill you expect to gain from it.
Gap in studies or a delayed degree?	One factual sentence, then what you did in the gap (course, certification, project).
How would friends describe you?	Two traits, each with a small example. Choose traits the job needs, such as patience when debugging.
Alone or in a team?	Both: the example should show individual ownership inside a team result.
Would you take a different team?	Yes if the scope is technical; ask what the team owns and how success is measured.
Salary expectation (lateral)?	Give a range anchored in market data and the scope, after learning the scope: "for this scope I would expect …".
Why are you leaving your current role?	Forward-looking: what you want to learn and cannot learn there. No blame.
Notice period and joining date?	State it plainly; mention a buyout only if the company has offered one.
You disagree with a senior's technical choice?	Raise it privately with data, accept the decision if it stands, and support it (Q12.46 (p. 733)).

Offer negotiation, continued

The basic lesson (compare *guaranteed annual cash*, and see which items can move off campus) and the first worked comparison are in §12.12 (p. 721). The tables below add what an Indian offer letter hides, a second comparison that includes shares, and the words to use.

★ What each component is and how to value it.

Component	What to ask	How to value it
Fixed base	monthly gross, and the split into basic, house-rent allowance and special allowance	fully; the basic sets the provident-fund and gratuity amounts
Employer provident fund	percentage of basic, and whether it is inside or outside the base	counted in CTC but not in monthly pay; value it, but it is not cash you can spend
Gratuity	confirm it is accrued, not paid yearly	last drawn basic × 15 / 26 × years of service, payable after five years under the Payment of Gratuity Act; value at zero for a short stay
Variable pay	the payout percentage of the last two years	target × the recent payout, never the target
Joining bonus	the clawback period and whether it is taxed at once	one-off; spread over the clawback period, and drop it if you doubt the stay
Relocation	amount, whether it is reimbursed or lump-sum, any repayment	one-off, near face value
RSUs	vesting schedule (25% a year, or front-loaded), grant value, the currency of the share price	grant value ÷ years, then discount for share-price risk; unlisted shares at a large haircut
Insurance, meal cards	cover amount; whether it extends to family	small; compare only if the rest is equal
Notice period, bond	length, and any recovery amount	a cost if you leave early; ask for it to be waived or reduced

★ **A second comparison, with shares.** Offer C is advertised at ₹21 LPA and Offer D at ₹18 LPA. All figures are in lakhs of rupees.

Item	Offer C	Offer D
Fixed base	14	17
Variable: target, payout last year	2, paid at 90%	1, paid at 100%
Joining bonus (year one only)	2	none
RSU grant, vesting 25% a year	12 in total, so 3 a year	none
Headline CTC (base + variable target + RSU per year + bonus)	14 + 2 + 3 + 2 = 21	17 + 1 = 18
Cash in year one	14 + 1.8 + 2 = 17.8	17 + 1 = 18
Cash in years two to four, each	14 + 1.8 = 15.8	18
Four-year cash	17.8 + 3 × 15.8 = 65.2	4 × 18 = 72
Four-year total if the shares hold their grant value	65.2 + 12 = 77.2	72
Four-year total if the shares halve	65.2 + 6 = 71.2	72

Hence the headline ranks C above D by 3 per year, whereas the cash ranks D above C by 6.8 over four years, and C wins only if its shares keep at least 6.8 of their 12 (about 57%). Further, RSUs are taxed as salary on the date they vest, at the market value then, so tax in cash is payable even if the shares are unsold. The tax regime (new or old) is chosen in the employer's payroll declaration and its slabs change often, so use the company's calculator and not a rule from this page.

★ **Words for the conversation.**

Step	Say	Because
Thank	"Thank you; I am glad to receive this offer and I would like to join."	enthusiasm comes first and costs nothing
Ask for the details	"Could you share the break-up in writing: base, variable payout history, bonus terms and vesting?"	you cannot negotiate what you cannot see
Ask for time	"May I have a few working days to decide?"	a considered answer is normal; pressure is a signal
Make one request	"With the competing offer at ₹X guaranteed cash, could the base be ₹Y?"	one figure with one data point is easy to approve
Pause	say nothing after the request	the next speaker concedes or explains
Close	"If that is possible, I will accept today."	it gives the manager a reason to act

Courtesy rules apply after the call. Do not decline other offers or resign before the signed letter arrives; verify the joining conditions and background check in the letter; and read the placement cell's policy before declining a campus offer, since some forbid reneging. Do not invent a competing offer, because a recruiter may ask for the letter.